Morten Hjorth-Jensen

Machine Learning and the Physical Sciences

From Discriminative methods to generative methods and reinforcement learning

October 25, 2026

Springer

Contents

Introduction xxi
Machine learning, a small and probably biased introduction xxii
A frequentist approach to data analysis xxiii
What is a good model? xxiv
Aims and learning outcomes xxv
The structure of this book xxv
Part I: the mathematical and statistical basis xxvi
Part II: classical methods xxvii
Part III: deep learning xxvii
Part IV: generative methods xxviii
Part V and the concluding chapter xxix
How to read this book xxix
Choice of programming language xxx
The programs, the figures and the notebooks xxx
Data handling, machine learning and ethical aspects xxxi

Part I Mathematical and statistical basis with essential computational elements

1 Linear Algebra for Machine Learning 3
1.1 Notation and conventions 3
1.2 Vectors 4
1.3 Matrices: definitions and algebraic operations 5
1.4 Special matrix types 6
1.5 Arrays in practice: numpy, BLAS and LAPACK 7
1.6 Orthonormal bases and projections 9
1.7 Orthogonal transformations 11
1.8 Derivatives with respect to vectors and matrices 12
1.8.1 The Jacobian 12
1.8.2 A warning about layout conventions 12
1.8.3 Four worked examples 13
1.8.4 The mean squared error and its derivative 15
1.8.5 The Hessian matrix 17
1.8.6 Derivatives involving the trace 18
1.8.7 The chain rule 18
1.9 The spectral decomposition of symmetric matrices 19
1.10 The covariance matrix 20
1.11 From algebra to computation 22
1.12 Vector and matrix norms 23

1.12.1 The Frobenius norm as a trace . . . 24
1.13 Linear systems and Gaussian elimination . . . 27
1.14 LU and Cholesky decompositions . . . 29
1.15 Iterative methods for linear systems . . . 31
1.16 The conjugate gradient method . . . 32
1.17 The algebraic eigenvalue problem . . . 34
1.17.1 Similarity transformations . . . 34
1.17.2 The power method . . . 35
1.17.3 What library routines actually do . . . 35
1.18 The singular value decomposition . . . 36
1.19 Rank, the pseudoinverse and ill-conditioned design matrices . . . 38
1.20 Low-rank approximation . . . 41
1.21 Principal component analysis . . . 42
1.22 Ridge regression through the singular value decomposition . . . 44
1.23 Summary and the programs . . . 44
1.24 Exercises . . . 45
Warm-up exercises . . . 45
Exercise session, week 34 . . . 46

2 Elements of Probability Theory and Statistics . . . 49
2.1 Domains, stochastic variables and probability distributions . . . 49
2.2 Distributions we shall need . . . 51
2.3 Expectation values and moments . . . 53
2.4 Covariance, correlation and independence . . . 54
2.5 The central limit theorem . . . 55
2.6 Samples, estimators and their properties . . . 57
2.7 Correlated data and the autocorrelation function . . . 58
2.8 The statistics of the least-squares estimator . . . 60
2.9 Training error, test error and generalisation . . . 63
2.10 The bias-variance tradeoff . . . 64
2.11 Resampling: the jackknife and the bootstrap . . . 67
2.12 Cross-validation . . . 68
2.13 Random numbers . . . 70
2.14 Markov chains and the Metropolis algorithm . . . 72
2.15 Summary and the programs . . . 74
2.16 Exercises . . . 75
Warm-up exercises . . . 75
Exercise session on statistics and resampling . . . 76

Part II Classical methods

3 Linear Regression . . . 81
3.1 The regression problem . . . 81
3.2 The linear model and the design matrix . . . 82
3.3 Ordinary least squares . . . 83
3.4 Weighted least squares and the χ^2 function . . . 85
3.5 Measures of quality . . . 87
3.6 Deriving least squares from a probability distribution . . . 88
3.7 Statistical properties of the least-squares estimator . . . 90
3.8 Ridge regression . . . 95
3.9 The Lasso . . . 99
3.10 Comparing the three estimators . . . 104

3.11 A Bayesian reading . . . 106
3.12 Kernel methods: regression without coordinates . . . 107
3.12.1 An identity that moves the problem . . . 107
3.12.2 Kernels . . . 108
3.12.3 Why the same trick works for any loss: the representer theorem . . . 109
3.12.4 There is no kernel Lasso . . . 110
3.13 Scaling, centring and the intercept . . . 113
3.14 Choosing the penalty: cross-validation for Ridge and Lasso . . . 117
3.15 A complete example: the Franke function . . . 129
3.16 A second example: the one-dimensional Ising model . . . 133
3.16.1 The model and the design matrix . . . 133
3.16.2 Fitting with p four times n . . . 134
3.16.3 Why the symmetry, and why only for two of the three . . . 136
3.16.4 The penalty, over eight decades . . . 137
3.16.5 What the example is for . . . 138
3.17 Summary and the programs . . . 139
3.18 Exercises . . . 140
Warm-up exercises . . . 140
Project-style exercise: the Franke function . . . 142

4 Optimization . . . 145
4.1 Convexity . . . 145
4.2 Newton's method . . . 147
4.3 Gradient descent . . . 148
4.4 Gradient descent for linear regression . . . 149
4.4.1 Gradient descent for Ridge regression . . . 150
4.5 The learning rate and the condition number . . . 151
4.6 Momentum . . . 155
4.7 Stochastic gradient descent . . . 160
4.7.1 The finite-sum problem and the minibatch gradient . . . 160
4.7.2 What an update costs . . . 161
4.7.3 A worked example: the running mean . . . 162
4.7.4 The minibatch gradient as an estimator . . . 163
4.7.5 Expected progress in one update, and the rates it implies . . . 164
4.7.6 Which accuracy is worth paying for . . . 165
4.7.7 An example where SGD beats gradient descent . . . 166
4.7.8 Learning rate schedules and stopping . . . 167
4.7.9 The batch size, and how to compare fairly . . . 171
4.8 Why adapt the step size at all . . . 174
4.9 AdaGrad . . . 176
4.10 RMSProp . . . 178
4.11 Adam . . . 179
4.12 Implementations . . . 183
4.13 None of these can compete with Newton's method . . . 186
4.14 Automatic differentiation . . . 187
4.14.1 Three ways to differentiate a program . . . 187
4.14.2 Evaluation traces and the computational graph . . . 188
4.14.3 Forward mode: tangents and dual numbers . . . 189
4.14.4 Reverse mode: adjoints . . . 191
4.14.5 Two examples worked by hand . . . 192
4.14.6 Forward or reverse: the chain rule as a matrix product . . . 195

4.14.7 Accuracy, and points where the derivative does not exist . . . 196
4.14.8 Automatic differentiation in Python: autograd and JAX . . . 196
4.14.9 The optimisers of this chapter on a non-convex function . . . 199
4.14.10 Finite differences against automatic differentiation . . . 201
4.15 Practical tips . . . 202
4.16 Summary and the programs . . . 203
4.17 Exercises . . . 205
Warm-up exercises . . . 205
Project-style exercise: comparing optimisers . . . 207

5 Logistic Regression . . . 209
5.1 Classification problems . . . 209
5.2 The logistic function . . . 210
5.3 Maximum likelihood and the cross-entropy . . . 212
5.4 Gradients, the Hessian and convexity . . . 215
5.5 More predictors, and regularisation . . . 216
5.6 More than two classes: the softmax . . . 217
5.7 Optimising the cross entropy . . . 218
5.8 An implementation . . . 220
5.8.1 Logistic regression as a single neuron . . . 222
5.8.2 The same algorithm by automatic differentiation . . . 224
5.9 Stochastic gradient descent for logistic regression . . . 228
5.9.1 Minibatches and the per-observation gradient . . . 229
5.9.2 Gradient descent against SGD, measured . . . 230
5.9.3 AdaGrad, RMSProp and Adam on the same problem . . . 231
5.10 Other loss functions, and automatic differentiation . . . 236
5.10.1 Margin-based losses . . . 236
5.10.2 What each loss estimates . . . 238
5.10.3 One program for every loss . . . 239
5.11 Kernel logistic regression . . . 241
5.11.1 The problem, and what the representer theorem gives . . . 242
5.11.2 Gradient, Hessian and kernel IRLS . . . 242
5.11.3 The method, verified . . . 243
5.11.4 What the coefficients are . . . 244
5.11.5 The penalty, measured . . . 245
5.12 Measuring the quality of a classifier . . . 245
5.13 The Wisconsin breast cancer data . . . 247
5.14 Summary and the programs . . . 250
5.15 Exercises . . . 252
Warm-up exercises . . . 252
Project-style exercise: classification . . . 254

6 Support Vector Machines . . . 255
6.1 Hyperplanes and the margin . . . 255
6.2 A first attempt, and why it fails . . . 256
6.3 The maximum-margin problem . . . 257
6.4 A reminder on Lagrange multipliers . . . 258
6.5 The dual problem . . . 258
6.6 The soft margin . . . 260
6.7 Kernels and non-linearity . . . 262
6.7.1 Common kernels and Mercer’s theorem . . . 263
6.8 The quadratic programme . . . 265

6.9 Sequential minimal optimisation . . . 265
6.10 Implementation . . . 267
6.11 Verifying the solver . . . 270
6.12 The primal view: the hinge loss . . . 271
6.13 One penalty, three losses . . . 273
6.13.1 The common problem . . . 274
6.13.2 The three, measured together . . . 275
6.14 Summary and the programs . . . 276
6.15 Exercises . . . 277
Warm-up exercises . . . 277
Project-style exercise: building an SVM . . . 278

7 Ensemble methods; from simple decision trees to Gradient Boosting . . . 281
7.1 Basics of a tree . . . 281
7.2 Regression trees . . . 281
7.2.1 Cost-complexity pruning . . . 282
7.3 Classification trees . . . 283
7.4 The CART algorithm . . . 285
7.5 A worked example: computing the Gini index . . . 285
7.6 Implementing a decision tree . . . 287
7.7 Strengths and weaknesses of trees . . . 290
7.8 Why averaging helps . . . 290
7.9 Bagging . . . 292
7.10 Random forests . . . 292
7.11 Boosting: a bird’s eye view . . . 294
7.12 AdaBoost . . . 295
7.13 Gradient boosting . . . 298
7.14 XGBoost: extreme gradient boosting . . . 300
7.15 Boosting any loss: automatic differentiation . . . 302
7.16 Summary and the programs . . . 308
7.17 Exercises . . . 309
Warm-up exercises . . . 309
Project-style exercise: trees and ensembles . . . 310

Part III Deep Learning

8 Neural Networks . . . 315
8.1 Artificial neurons . . . 315
8.2 Why one layer is not enough: the XOR problem . . . 316
8.3 Multilayer perceptrons and other architectures . . . 318
8.4 The universal approximation theorem . . . 318
8.4.1 How wide? A constructive bound . . . 321
8.4.2 The curse of dimensionality . . . 323
8.4.3 Why depth? An exact separation . . . 324
8.4.4 Representability is not learnability . . . 325
8.5 Notation and the feed-forward pass . . . 326
8.6 Activation functions . . . 329
8.6.1 Saturating activations . . . 329
8.6.2 The rectified family . . . 329
8.6.3 Smooth gated activations: GELU and its relatives . . . 330
8.6.4 Which activation should we use? . . . 331
8.7 The cost function and the optimisation problem . . . 333

8.8 Backpropagation . . . 334
8.8.1 Two scalar examples: no hidden layer and one hidden layer . . . 335
8.8.2 Two inputs, two hidden nodes, one output . . . 342
8.8.3 The general case: dense layers . . . 346
8.8.4 The four equations of backpropagation . . . 347
8.8.5 The algorithm . . . 349
8.9 Vanishing and exploding gradients . . . 350
8.10 Weight initialisation . . . 351
8.11 Classification with neural networks . . . 352
8.12 Regularisation and hyperparameters . . . 352
8.13 A neural network from scratch . . . 353
8.14 Examples . . . 357
8.15 Limitations, and a top-down view . . . 358
8.16 An extended example: learning the Ising energy . . . 359
8.16.1 The linear model on raw spins is exactly powerless . . . 359
8.16.2 A network that gets it exactly right . . . 360
8.16.3 Representable is not learnable . . . 361
8.16.4 Against Chapter 3 . . . 362
8.17 Summary and the programs . . . 364
8.18 Exercises . . . 365
Warm-up exercises . . . 365
Project-style exercise: neural networks . . . 366

9 Neural Networks for Differential Equations . . . 369
9.1 Ordinary differential equations . . . 369
9.2 Differentiating the network with respect to its input . . . 370
9.3 Exponential decay . . . 373
9.4 Logistic population growth . . . 374
9.5 The one-dimensional Poisson equation . . . 375
9.6 Partial differential equations . . . 377
9.7 The diffusion equation . . . 378
9.8 The wave equation . . . 379
9.9 When is this worth doing? . . . 380
9.10 Physics-informed neural networks . . . 381
9.11 A stochastic equation: Black-Scholes option pricing . . . 388
9.11.1 From a stochastic process to a deterministic equation . . . 388
9.11.2 Conditions, and the link to the diffusion equation . . . 389
9.11.3 Why a trial solution will not do . . . 390
9.11.4 Scaling matters more than anything else . . . 390
9.11.5 Implementation . . . 391
9.11.6 Results . . . 392
9.12 Summary and the programs . . . 394
9.13 Exercises . . . 395
Warm-up exercises . . . 395
Project-style exercise: solving equations . . . 396

10 Convolutional Neural Networks . . . 399
10.1 Why not simply use a dense layer? . . . 399
10.2 What a convolutional network is: volumes and layers . . . 401
10.3 Convolution . . . 402
10.3.1 Convolution is polynomial multiplication . . . 402
10.3.2 Toeplitz structure . . . 403

10.3.3 Diagonalisation: the convolution theorem . . . 404
10.3.4 Padding, and why indices go out of range . . . 405
10.3.5 Cross-correlation, which is what libraries actually compute . . . 405
10.4 Two dimensions, channels and the convolutional layer . . . 406
10.4.1 Output size and parameter count . . . 407
10.4.2 The unrolled matrix . . . 407
10.5 Equivariance, and what it does and does not give . . . 408
10.6 Pooling . . . 410
10.6.1 Dilation: receptive field without cost . . . 411
10.7 Filters at work: from hand-made to learned . . . 413
10.7.1 Four edge filters and a blur . . . 413
10.7.2 A filter that detects a roof: template matching . . . 414
10.7.3 From hand-made filters to learned ones . . . 415
10.8 Backpropagation through a convolutional layer . . . 416
10.9 Implementation from scratch . . . 418
10.9.1 The im2col transformation . . . 418
10.9.2 Verifying the implementation . . . 420
10.9.3 A complete network . . . 421
10.10 Does it actually help? An experiment . . . 422
10.11 Dropout . . . 423
10.12 The same network in PyTorch and TensorFlow . . . 424
10.12.1 PyTorch . . . 424
10.12.2 TensorFlow and Keras . . . 425
10.12.3 Do the three implementations agree? . . . 426
10.12.4 Parameter counts, checked . . . 427
10.12.5 Training on MNIST, and what the dense layer is worth . . . 427
10.12.6 Convolutional against dense networks on MNIST . . . 428
10.13 Convolution in the frequency domain: Fourier filters . . . 429
10.13.1 The Fourier transform and the convolution theorem . . . 430
10.13.2 Three filters, read in frequency . . . 432
10.13.3 Sampled data: the discrete Fourier transform . . . 433
10.13.4 Two dimensions: the photograph in Fourier space . . . 436
10.13.5 A convolutional layer in Fourier language . . . 437
10.13.6 How Fourier filters are used . . . 440
10.14 Summary and the programs . . . 441
10.15 Exercises . . . 443
Warm-up exercises . . . 443
Project-style exercise: a CNN from scratch . . . 446

11 Recurrent Neural Networks . . . 449
11.1 Sequences, and what we want from them . . . 449
11.2 The architecture . . . 450
11.2.1 The RNN as a discrete-time dynamical system . . . 451
11.3 Where the recurrence comes from: a differential equation . . . 452
11.4 Backpropagation through time . . . 453
11.5 Why long sequences are hard . . . 454
11.5.1 Measuring it . . . 455
11.5.2 Gradient clipping . . . 457
11.6 Implementation and verification . . . 457
11.6.1 The damped oscillator . . . 459
11.7 Long short-term memory . . . 459

11.7.1 Why it works, and by how much . . . 460
11.7.2 The gated recurrent unit . . . 463
11.8 The same networks in PyTorch and TensorFlow . . . 463
11.8.1 Forecasting a sine wave . . . 463
11.8.2 An LSTM on MNIST, read row by row . . . 465
11.8.3 Do the three implementations agree? . . . 466
11.8.4 Row by row: the two libraries on MNIST . . . 467
11.8.5 The experiment the theory demands . . . 468
11.9 Summary and the programs . . . 468
11.10 Exercises . . . 470
Warm-up exercises . . . 470
Project-style exercise: RNNs from scratch . . . 471

12 Autoencoders . . . 473
12.1 Structure . . . 473
12.2 Reconstruction, projection and representation . . . 475
12.3 The linear autoencoder . . . 476
12.3.1 Reduction to an orthogonal projector . . . 477
12.3.2 Reduction to a trace maximisation . . . 477
12.3.3 The variational characterisation . . . 478
12.3.4 The theorem . . . 478
12.4 Verifying the theorem . . . 480
12.5 Beyond PCA . . . 481
12.6 Regularised autoencoders . . . 483
12.7 The probabilistic view: PPCA . . . 485
12.8 Implementations . . . 486
12.8.1 Our own . . . 486
12.8.2 PyTorch and TensorFlow . . . 486
12.8.3 Does Adam find the principal subspace? . . . 489
12.8.4 What the bottleneck costs, and what the nonlinearity buys . . . 490
12.8.5 The regularised variants, measured . . . 491
12.9 Summary and the programs . . . 491
12.10 Exercises . . . 493
Warm-up exercises . . . 493
Project-style exercise: autoencoders and PCA . . . 494

13 Transformers . . . 497
13.1 Tokens . . . 497
13.2 Scaled dot-product attention . . . 498
13.2.1 What attention is . . . 498
13.2.2 Why the $1/\sqrt{d_k}$ . . . 499
13.3 Masking and position . . . 500
13.4 Multi-head attention and the transformer block . . . 502
13.5 Implementation and verification . . . 503
13.6 What one attention layer cannot do . . . 504
13.7 Comparisons and interpretations . . . 505
13.7.1 Against the other architectures . . . 505
13.7.2 A statistical-mechanics reading . . . 506
13.7.3 Kernels and graphs . . . 506
13.7.4 Why this matters for differential equations . . . 506
13.8 PyTorch and TensorFlow . . . 507
13.8.1 Do the three implementations agree? . . . 509

13.8.2 The scaling, in the library . . . 510
13.8.3 Permutation equivariance, tested . . . 510
13.8.4 Associative recall, in the frameworks . . . 510
13.9 Strengths, weaknesses and what to remember . . . 511
13.10 Exercises . . . 513
Warm-up exercises . . . 513
Project-style exercise: a transformer . . . 514

Part IV Generative methods

14 Boltzmann Machines . . . 519
14.1 Energy models . . . 519
14.1.1 The counting problem . . . 520
14.2 Maximum likelihood and the two phases . . . 520
14.2.1 The same statement as a divergence . . . 521
14.3 Markov chain Monte Carlo . . . 521
14.3.1 Metropolis-Hastings . . . 522
14.3.2 Gibbs sampling . . . 523
14.4 Boltzmann machines . . . 523
14.4.1 The restriction . . . 524
14.4.2 Gaussian-binary machines . . . 525
14.5 Contrastive divergence . . . 525
14.6 Implementation and verification . . . 526
14.6.1 How biased is CD-k? . . . 528
14.7 Implementations in the libraries . . . 528
14.7.1 Do the implementations agree, and is the theorem right? . . . 530
14.7.2 On real data: CD-1, CD-10 and persistent CD . . . 531
14.8 Summary and the programs . . . 532
14.9 Exercises . . . 534
Warm-up exercises . . . 534
Project-style exercise: a Boltzmann machine . . . 535

15 Variational Autoencoders . . . 537
15.1 The latent-variable model . . . 537
15.2 The evidence lower bound . . . 538
15.2.1 A tighter bound, and how to see the gap . . . 539
15.2.2 The two terms . . . 540
15.2.3 The Gaussian case in closed form . . . 540
15.3 The reparameterisation trick . . . 541
15.4 Implementation and verification . . . 542
15.5 Two characteristic failures . . . 543
15.6 Implementations in the libraries . . . 544
15.6.1 The variance that decides the matter . . . 547
15.6.2 How far is the ELBO from the truth? . . . 547
15.6.3 Collapse, and the price of preventing it . . . 548
15.7 Towards diffusion models . . . 548
15.8 Summary and the programs . . . 549
15.9 Exercises . . . 550
Warm-up exercises . . . 550
Project-style exercise: a VAE from scratch . . . 551

16 Diffusion Models and Normalizing Flows . . . 553
16.1 From hierarchical VAEs to diffusion . . . 553
16.1.1 The forward process . . . 554
16.1.2 The forward posterior . . . 554
16.2 The objective . . . 555
16.2.1 Predicting the noise . . . 556
16.2.2 The same thing as score matching . . . 556
16.3 Sampling . . . 557
16.4 Normalizing flows . . . 558
16.4.1 Verifying exactness . . . 559
16.5 Comparison . . . 560
16.6 Summary and the programs . . . 561
16.7 Exercises . . . 562
Warm-up exercises . . . 562
Project-style exercise: diffusion and flows . . . 562

17 Generative Adversarial Networks . . . 565
17.1 Two networks and a game . . . 565
17.1.1 What GANs are used for . . . 566
17.2 The minimax objective . . . 566
17.3 The optimal discriminator . . . 567
17.4 The objective is a Jensen-Shannon divergence . . . 569
17.5 Training: alternating gradients . . . 571
17.5.1 The non-saturating generator loss . . . 571
17.5.2 One-sided label smoothing . . . 572
17.5.3 What convergence looks like . . . 573
17.6 What actually goes wrong . . . 574
17.6.1 Non-convergence: the game itself . . . 574
17.6.2 Vanishing gradients: the discriminator wins . . . 575
17.6.3 Mode collapse, and a topological obstruction . . . 575
17.7 The Wasserstein distance . . . 576
17.8 Evaluating a model with no likelihood . . . 578
17.8.1 Maximum mean discrepancy . . . 579
17.8.2 Fréchet inception distance . . . 579
17.9 Architectures . . . 580
17.9.1 The fully connected baseline . . . 580
17.9.2 Convolutions: DCGAN . . . 581
17.9.3 Conditioning . . . 581
17.9.4 Reading the latent space . . . 581
17.10 GANs in PyTorch and TensorFlow . . . 582
17.11 Summary and the programs . . . 584
17.12 Exercises . . . 586
Warm-up exercises . . . 586
Project-style exercise: an adversarial generator . . . 586

Part V Reinforcement Learning

Part VI Concluding remarks and perspectives

18 Concluding remarks and perspectives 593
18.1 The shape of the argument 593
18.2 The threads 595
18.2.1 One number governs accuracy, variance and training time 595
18.2.2 The loss is derived, not chosen 595
18.2.3 Bias, variance, and the two ways to trade them 596
18.2.4 The chain rule is enough 597
18.2.5 Structure is a prior, and it is the cheapest one available 597
18.2.6 One decomposition, read several ways 598
18.2.7 An intractable normalisation, and four ways round it 598
18.2.8 Representability is not learnability, and neither is generalisation 599
18.3 A protocol for a new problem 599
18.4 What this book does not contain 601
18.5 Perspectives 603
18.5.1 Physics 603
18.5.2 Chemistry and materials science 604
18.5.3 Mathematics: operators, inverse problems and guarantees 604
18.5.4 Earth, ocean and climate science 605
18.5.5 Bioscience and medicine 605
18.5.6 Reinforcement learning and agentic systems 606
18.5.7 Foundation models and transfer 606
18.6 What to be sceptical about 607
18.7 A last word 608
References 609

Index 613

Acronyms

The acronyms used in this book are collected here, chapter by chapter. An acronym is listed under the chapter in which it is first introduced and defined; subsequent chapters use it freely. Names of software libraries and data sets that are written as acronyms are included as well, since they appear throughout the code examples.

Chapter 1: Linear Algebra for Machine Learning

BLAS	Basic Linear Algebra Subprograms, the standard low-level library of vector and matrix operations that NumPy and every numerical library call
LAPACK	Linear Algebra PACKage, the standard library of dense matrix factorisations and solvers built on BLAS
LU	Lower–upper (triangular) factorisation of a square matrix
QR	Factorisation of a matrix into an orthogonal factor $\boldsymbol{Q}$ and an upper triangular factor $\boldsymbol{R}$
SVD	Singular value decomposition
PCA	Principal component analysis
IEEE	Institute of Electrical and Electronics Engineers, here for the IEEE 754 standard for floating-point arithmetic

Chapter 2: Elements of Probability Theory and Statistics

PDF	Probability density function (also probability distribution function)
CDF	Cumulative distribution function
MSE	Mean squared error

Chapter 3: Linear Regression

OLS	Ordinary least squares
WLS	Weighted least squares
GLS	Generalised least squares
TSS	Total sum of squares
ESS	Explained sum of squares
RSS	Residual sum of squares
SE	Standard error
CV	Cross-validation; k-fold CV splits the data into k folds
LOO	Leave-one-out (cross-validation)
PRESS	Predicted residual error sum of squares, the leave-one-out residual sum
GCV	Generalised cross-validation
MAP	Maximum a posteriori (estimation)
KKT	Karush–Kuhn–Tucker (optimality conditions)

Chapter 4: Optimization

GD	Gradient descent
SGD	Stochastic gradient descent
AdaGrad	Adaptive gradient algorithm
RMSProp	Root mean square propagation
Adam	Adaptive moment estimation
AD	Automatic differentiation
JAX	Google's library for composable transformations of NumPy programs (automatic differentiation, just-in-time compilation, vectorisation)

Chapter 5: Logistic Regression

IRLS	Iteratively reweighted least squares
CHD	Coronary heart disease (the data set of the worked example)
TP, TN	True positives, true negatives
FP, FN	False positives, false negatives
ROC	Receiver operating characteristic (curve)
AUC	Area under the ROC curve
AdaBoost	Adaptive boosting (the exponential loss; the algorithm itself is in Chapter 7)

Chapter 6: Support Vector Machines

SVM	Support vector machine
SVC	Support vector classifier (also scikit-learn's class of that name)
RBF	Radial basis function (kernel)
SMO	Sequential minimal optimisation

Chapter 7: Ensemble Methods

CART	Classification and regression trees
ID3	Iterative Dichotomiser 3, the entropy-based tree-growing algorithm
XGBoost	Extreme gradient boosting

Chapter 8: Neural Networks

MLP	Multilayer perceptron
XOR	Exclusive or (the classic problem a single-layer perceptron cannot solve)
ReLU	Rectified linear unit
ELU	Exponential linear unit
GELU	Gaussian error linear unit
MNIST	Modified National Institute of Standards and Technology data set of handwritten digits
GPU	Graphics processing unit

Chapter 9: Neural Networks for Differential Equations

ODE	Ordinary differential equation
PDE	Partial differential equation
IC	Initial condition
BC	Boundary condition
FD	Finite differences
RMSE	Root mean squared error
PINN	Physics-informed neural network

Chapter 10: Convolutional Neural Networks

CNN	Convolutional neural network
FFT	Fast Fourier transform

Chapter 11: Recurrent Neural Networks

RNN	Recurrent neural network
BPTT	Backpropagation through time
LSTM	Long short-term memory
GRU	Gated recurrent unit

Chapter 12: Autoencoders

AE	Autoencoder
PPCA	Probabilistic principal component analysis
BCE	Binary cross-entropy

Chapter 13: Transformers

PE	Positional encoding
MHA	Multi-head attention
LN	Layer normalisation

Chapter 14: Boltzmann Machines

RBM	Restricted Boltzmann machine
CD	Contrastive divergence; CD-k runs k Gibbs sweeps
PCD	Persistent contrastive divergence
PL	Pseudo-likelihood
TV	Total variation (distance)
KL	Kullback–Leibler (divergence)

Chapter 15: Variational Autoencoders

VAE	Variational autoencoder
ELBO	Evidence lower bound
REINFORCE	The score-function (likelihood-ratio) gradient estimator of Williams

Chapter 16: Diffusion Models and Normalizing Flows

DDPM	Denoising diffusion probabilistic model
DDIM	Denoising diffusion implicit model
SNR	Signal-to-noise ratio

Chapter 17: Generative Adversarial Networks

GAN	Generative adversarial network
JS	Jensen–Shannon (divergence)
WGAN	Wasserstein GAN
GP	Gradient penalty, as in WGAN-GP
DCGAN	Deep convolutional GAN
MMD	Maximum mean discrepancy
FID	Fréchet inception distance

Introduction

During the last two decades there has been a swift and remarkable development of machine learning techniques and algorithms, and their impact is felt not only in science and technology but also in the humanities, the social sciences, medicine and law – indeed in almost every discipline one can name. The applications are innumerable, from self-driving cars to the solution of high-dimensional differential equations or complicated quantum mechanical many-body problems. Machine learning is perceived by many as one of the main disruptive technologies of our time.

Statistics, data science and machine learning form central fields of research in modern science. They describe how to learn and make predictions from data, and they allow us to extract correlations about physical processes and the underlying laws of motion from large data sets. Such large data sets appear frequently in essentially all disciplines, from the traditional science, technology, mathematics and engineering fields to life science, law, educational research, the humanities and the social sciences. It has become more and more common to see research projects on big data in, for example, the social sciences, where extracting patterns from complicated survey data is one of many research directions. A solid grasp of data analysis and machine learning is thus becoming central to scientific computing in many fields, and competences within machine learning and scientific computing are strongly requested by many potential employers. The latter can hardly be overstated: familiarity with machine learning has almost become a prerequisite for many of the most interesting employment opportunities, whether in bioinformatics, life science, physics or finance, in the private or the public sector. This author has had several students who were hired on the basis of their skills in scientific computing and data science, often with only a marginal knowledge of machine learning itself.

Machine learning is a subfield of computer science, and is closely related to computational statistics. It evolved from the study of pattern recognition in artificial intelligence research, and has made contributions to tasks like computer vision, natural language processing and speech recognition. Many of the methods we study here are, however, just as strongly rooted in basic mathematics and physics. A restricted Boltzmann machine is a spin system in thermal equilibrium; a diffusion model is a discretised stochastic differential equation; a convolutional layer is a discrete convolution, with all the Fourier theory that goes with it; and the algorithm that trains all of them is the chain rule. This is not a coincidence, and the reader who comes to the subject from the physical sciences already possesses a good deal of what is needed.

This introductory chapter says what machine learning is taken to mean here, what this book aims to do, how it is put together, and which ethical obligations come with the material. The last of these is not an ornament. We end the chapter with it deliberately.

Machine learning, a small and probably biased introduction

Ideally, machine learning represents the science of giving computers the ability to learn without being explicitly programmed. The idea is that there exist generic algorithms which can find patterns in a broad class of data sets without our having to write code specifically for each problem; the algorithm builds its own logic from the data. One should however always keep in mind that machines and algorithms are to a large extent developed by humans, and that the insight and knowledge we have about a specific system play a central role when we develop a specific machine learning algorithm. This book is written from that conviction. Almost every chapter contains a place where knowing something about the problem – that an image is translation invariant, that a probability must be positive, that a differential equation must satisfy a boundary condition – buys more than a larger model would.

Machine learning is an extremely rich field, in spite of its young age. The increase in computational capability over the last decades has been followed by the development of methods for analysing and handling large data sets, relying heavily on statistics, computer science and mathematics. Popular software libraries written in Python, such as Scikit-learn,[1] TensorFlow,[2] PyTorch[3] and Keras,[4] all freely available at their respective GitHub sites, encompass communities of developers numbering in the thousands or more, and the number of contributors keeps increasing. Not all the algorithms and methods can be given a rigorous mathematical justification – decision trees and random forests are good examples – which opens up large rooms for experimentation, trial and error, and thereby for exciting new developments. A solid command of linear algebra, multivariate calculus, probability theory, statistical data analysis, error estimation and Monte Carlo methods is nevertheless central to a proper understanding of most of the algorithms we shall discuss, and Chapters 1 and 2 are there to supply it.

The approaches to machine learning are many, but are often split into two main categories. In *supervised learning* we know the answer to a problem and let the computer deduce the logic behind it. In *unsupervised learning* we look for patterns and relationships in data sets without any prior knowledge of the system. Some authors operate with a third category, *reinforcement learning*, a paradigm inspired by behavioural psychology in which learning is achieved by trial and error, solely from rewards and punishments. The first two categories organise this book; the third is discussed, but not developed, in Chapter 18.

Another way to categorise machine learning tasks is by the desired output of the system. Three of the most common tasks are the following.

- **Classification.** Outputs are divided into two or more classes, and the goal is to produce a model that assigns inputs to one of these classes. Identifying digits from pictures of handwritten ones is the standard example. Classification is usually supervised learning, and it is the subject of Chapters 5, 6 and 7, and again of Chapter 8 onwards.
- **Regression.** Finding a functional relationship between an input data set and a reference data set: the goal is to construct a map from inputs to continuous output values. This is where we begin, in Chapter 3.
- **Clustering.** Data are divided into groups sharing certain traits, without knowing the groups beforehand. It is thus a form of unsupervised learning, and it is closely related to the dimensionality reduction we meet as principal component analysis in Section 1.21 and again, in a nonlinear guise, in Chapter 12.

The methods we cover have three ingredients in common, irrespective of whether we deal with supervised or unsupervised learning.

[1] `http://scikit-learn.org/stable/`

[2] `https://www.tensorflow.org/`

[3] `http://pytorch.org/`

[4] `https://keras.io/`

- The first ingredient is our **data set**, which is normally subdivided into training, validation and test data. Many find the most difficult part of applying machine learning to be the setting up of the data in a meaningful way. Why the subdivision is necessary at all, and what goes wrong when it is neglected, is the subject of Sections 2.9 and 2.12.
- The second is a **model**, which is normally a function of a set of parameters. The model reflects our knowledge of the system, or our lack of it. If we know that our data behave roughly like a polynomial, fitting a polynomial of some degree determines our model.
- The third is a **cost function**, also called a loss function or an error function, which measures how well the model reproduces the data it is trained on.

The third ingredient deserves a warning already here, because it is one of the threads that runs through the entire book. The cost function is not a free design decision. Specify what the target is assumed to be, and the cost function follows: Gaussian noise gives the squared error of Section 3.3, Bernoulli observations give the cross entropy of Section 5.3, a categorical target gives the multiclass cross entropy that pairs with the softmax of Section 5.6, and even the penalties are not free – Section 3.11 shows that a flat prior gives ordinary least squares, a Gaussian prior gives Ridge regression and a Laplace prior gives the Lasso. A reader who acquires the habit of asking which assumption generated a given loss will find that the strange-looking objectives of the generative chapters are not strange at all.

At the heart of essentially all machine learning algorithms we find a minimisation or optimisation problem, and a large family of methods for solving it goes under the name of *gradient methods*. These are developed in Chapter 4 and then used, without further apology, everywhere else.

A remark on what is new and what is not. Of the ingredients above, only the third is peculiar to machine learning, and even that only mildly. Fitting a model by minimising a cost function is what a physicist does when fitting a resonance curve, and least squares is two centuries old. What has changed is the size of the model class, the fact that the features are learned rather than chosen, and the availability of a cheap and exact gradient for arbitrarily complicated compositions of functions. That last point – reverse-mode automatic differentiation, Section 4.14 – is the technical development that made the rest possible.

A frequentist approach to data analysis

When you hear phrases like *predictions and estimations* or *correlations and causations*, what do you think of? Perhaps of the difference between classifying new data points and generating new ones. Or perhaps you consider that correlations are symmetric statements – if A is correlated with B, then B is correlated with A – while causation is directional: if A causes B, B need not cause A.

These concepts mark, in some sense, the difference between machine learning and statistics. In machine learning and prediction-based tasks we are often interested in algorithms that learn patterns from given data in an automated fashion, and then use these learned patterns to make predictions about new data. In many cases our primary concern is the quality of the predictions, and we are less concerned about the underlying patterns that were learned in order to make them.

In machine learning we normally use a so-called frequentist approach,[5] where the aim is to make predictions and find correlations. We focus less on extracting a probability distribution

[5] `https://en.wikipedia.org/wiki/Frequentist_inference`

function, which could in turn be used to make estimations and find causations, such as: given A, what is the likelihood of finding B?

The distinction is one of emphasis rather than of principle, and this book crosses it in both directions. Chapter 2 is written from the frequentist side: an estimator is a random variable, and what we want to know about it is its bias and its variance. Section 3.11 then reads exactly the same regression estimators as maximum *a posteriori* estimates under different priors, and the whole of Part IV is Bayesian in spirit, since a generative model is a statement about $p(\boldsymbol{x})$ and nothing else. The reader who keeps both readings available will find several results easier, not harder: the Ridge estimator is a shrunken least-squares estimator and a Gaussian posterior mode, and it is useful to be able to see it as both.

What is a good model?

In science and engineering we often end up in situations where we want to infer, or learn, a quantitative model M for a given set of sample points $\boldsymbol{X} \in [x_1, x_2, \ldots, x_N]$. As we shall see repeatedly, we could try to fit these data points to a straight line, or, if we wish to be more sophisticated, to a more complex function.

The reason for inferring such a model is that it serves many useful purposes. On the one hand, the model can reveal information encoded in the data, or underlying mechanisms from which the data were generated; we could discover important correlations that admit an interesting physical interpretation. In addition, a model simplifies the representation of a given data set and helps us make predictions about future samples.

A first important consideration to keep in mind is that inferring the *correct* model for a given data set is an elusive, if not impossible, task. The fundamental difficulty is that if we are not specific about what we mean by a correct model, there could easily be many different models that fit the given data set *equally well*.

The central question is then: what leads us to say that a model is correct, or optimal, for a given data set? To make the model inference problem well posed, that is, to guarantee that there is a unique optimal model for the given data, we need to impose additional assumptions or restrictions on the class of models considered. We should not be looking for just any model that can describe the data; we should look for a model M that is the best among a restricted class of models. In addition, to make the problem computationally tractable, we need to specify how restricted that class needs to be. A common strategy is to start with the simplest class of models that is just necessary to describe the data, or to solve the problem at hand. More precisely, the model class should be rich enough to contain at least one model that fits the data to the desired accuracy, and yet be restricted enough that it is relatively simple to find the best model within it.

The most popular strategy is therefore to start from the simplest class of models and to increase the complexity only when the simpler models become inadequate. If we work with a regression problem, one may first try linear models, followed by more complex ones.

How to evaluate which model fits the data best is something we come back to over and over again. The single most important tool is the bias-variance decomposition of Section 2.10, which states that the expected test error splits into a bias term, a variance term and an irreducible noise floor, and that the first two move in opposite directions as model complexity grows. It is used in almost every chapter of this book, and it is the reason why the answer to “which model is best?” is never “the one with the smallest training error”. The two structurally different ways of exploiting the decomposition – constraining a single model, as in Section 3.8, and averaging many, as in Chapter 7 – are among the few ideas in this book that will still be useful when every architecture in it has been superseded.

Aims and learning outcomes

This book aims at giving an overview of central aspects of statistical data analysis as well as of the central algorithms used in machine learning. Its origin is a one-semester course, and the pedagogical order of that course has been kept: nothing is used before it has been derived.

Hands-on work with data and algorithms plays a central role, and the hope is, through the exercises and the accompanying programs, to expose the reader to fundamental research problems in these fields, with the aim of reproducing state-of-the-art scientific results. The reader will learn to develop and structure codes for studying such systems, become acquainted with computing facilities, and learn to handle large scientific projects. A good scientific and ethical conduct is emphasised throughout. More specifically, the reader will

1. learn about basic data analysis, statistical inference, Monte Carlo methods, data optimisation and machine learning;
2. be capable of extending the acquired knowledge to other systems and cases;
3. gain an understanding of the central algorithms used in data analysis and machine learning;
4. acquire knowledge of central aspects of Monte Carlo methods, Markov chains and Gibbs samplers, and of their possible applications, from numerical integration to the simulation of physical systems;
5. understand methods for regression and classification;
6. learn about neural networks of several architectures, Boltzmann machines and modern generative models;
7. work on numerical projects that illustrate the theory. The projects play a central role, and the reader is expected to know a modern programming language such as Python or C++, in addition to having a basic knowledge of linear algebra, typically taught during the first one or two years of undergraduate studies.

The topics covered span statistical data analysis and its basic concepts – expectation values, variance, covariance, correlation functions and errors – via well-known probability distributions such as the uniform, binomial and Poisson distributions and the simple and multivariate normal distributions, to central elements of Bayesian statistics and modelling. We also remind the reader of central elements from linear algebra and of the standard methods built upon it: the family of gradient descent methods, the singular value decomposition, and least-squares methods for parameterising data. We cover Monte Carlo methods, Markov chains, and well-known algorithms for sampling stochastic events such as the Metropolis-Hastings and Gibbs sampling methods. An important aspect of all our calculations is a proper estimation of errors, and we therefore discuss resampling techniques such as blocking, the bootstrap and the jackknife, together with the infamous bias-variance trade-off. The remainder of the material covers the algorithms of machine learning proper.

The structure of this book

The book is organised in five parts and a closing chapter. It is not a catalogue of methods. A catalogue would have been easier to write and would have dated faster. What the book tries to do instead is to derive a small number of ideas carefully and then follow them until they stop working, because the place where an idea stops working is the place where the next one is invented. Read in that order, the seventeen chapters are not seventeen subjects but one

subject seventeen times, and Chapter 18 closes the book by making that claim explicitly and drawing the whole as a graph.

The table below is the short version; the subsections that follow are the long one.

Ch.	Title	What it adds
Part I – Mathematical and statistical basis		
1	Linear algebra	The objects: design matrices, projections, matrix calculus, the singular value decomposition and the condition number.
2	Probability and statistics	The discipline: estimators as random variables, the bias-variance decomposition, resampling, Markov chains.
Part II – Classical methods		
3	Linear regression	Least squares derived three times, Ridge, the Lasso, kernels.
4	Optimisation	What to do when the closed form is withdrawn: gradient descent and its adaptive descendants.
5	Logistic regression	One function changes, and the loss changes with it.
6	Support vector machines	A third loss, reached from geometry, belonging to the same family.
7	Trees and ensembles	Improving a model by averaging copies of it rather than by changing it.
Part III – Deep learning		
8	Neural networks	The hinge of the book: a hidden layer, and backpropagation.
9	Differential equations	The same network, asked a different question.
10	Convolutional networks	Weights that are local and shared; equivariance under translation.
11	Recurrent networks	Weights shared across time, and why long memories are hard.
12	Autoencoders	A bottleneck, and its exact identity with principal components.
13	Transformers	Weights computed from the data; equivariance under all permutations.
Part IV – Generative methods		
14	Boltzmann machines	Refuse to compute the normalisation, and sample instead.
15	Variational autoencoders	Bound it.
16	Diffusion and flows	Learn a score, or restrict the map.
17	Adversarial networks	Abandon the likelihood and train a classifier in its place.
Part V and closing		
18	Concluding remarks	The threads, a working protocol, the omissions, and where the methods are being taken in the sciences.

Part I: the mathematical and statistical basis

The two opening chapters are not preliminaries in the usual sense of material one may skip.

Chapter 1 supplies the objects. A data set is a design matrix, a model is very often a matrix acting on a parameter vector, and fitting is projection onto a column space. The chapter develops the algebraic language – vectors and matrices, orthogonality and projection, derivatives with respect to vectors and matrices in Section 1.8, the spectral decomposition, the covariance matrix – and then asks how these objects are actually computed, from Gaussian elimination and the LU and Cholesky factorisations through the iterative and conjugate gradient methods of Sections 1.15 and 1.16 to the singular value decomposition of Section 1.18. It also supplies the one number, the condition number, that will reappear in three quite different roles: as the loss of precision in a linear solve, as the variance of a fitted parameter, and as the number of iterations gradient descent will need.

Chapter 2 supplies the discipline. Every quantity computed from a finite sample is itself a random variable, with a bias and a variance. The chapter builds the probability distributions we shall need, expectation values and moments, covariance and correlation, the central limit theorem, and then the statistics of estimators: the properties of the least-squares estimator in

Section 3.7, the distinction between training and test error in Section 2.9, the bias-variance decomposition of Section 2.10, the resampling methods of Section 2.11 and cross-validation in Section 2.12. It closes with random numbers, Markov chains and the Metropolis algorithm of Section 2.14, which is not needed again until Chapter 14 but is best learned here.

Without the first of these chapters the algebra of the later ones is unreadable; without the second, their numbers are unbelievable.

Part II: classical methods

The five chapters that follow are, deliberately, largely the same model seen from different angles.

Chapter 3 derives the least-squares estimator three times – as an optimisation problem, as a projection, and as a maximum-likelihood estimate – because each derivation generalises differently and all three are needed later. It then adds a penalty and obtains Ridge regression and the Lasso, reads both as priors in Section 3.11, and lifts the whole construction into a feature space with the kernel methods of Section 3.12. Two extended examples run through the chapter: the Franke function of Section 3.15 and the one-dimensional Ising model of Section 3.16, the latter chosen because its exact answer is known and can be compared against.

Chapter 4 is what happens when the closed form is withdrawn. It begins with convexity and Newton's method, works through plain and stochastic gradient descent, and then derives momentum, AdaGrad, RMSProp and Adam as answers to a single question – how should the step length adapt? – before admitting, in Section 4.13, that none of them can compete with Newton's method except in the one respect that matters at scale. Section 4.14 explains why the gradient is cheap at all.

Chapter 5 changes one function, the identity to the sigmoid, and finds that the loss changes with it, without being chosen. It is the first classification chapter, it introduces the cross entropy and the softmax, and it introduces the measures – accuracy, precision, recall, the confusion matrix, the ROC curve – by which every classifier in the rest of the book is judged.

Chapter 6 arrives at a third loss from a purely geometric principle, the maximum margin, and finds that it belongs to the same family: Ridge regression, logistic regression and the support vector machine turn out to be one model with one penalty and three losses, which is the observation Section 6.13 closes on. On the way it supplies the Lagrange duality and the Karush-Kuhn-Tucker conditions [1] that are useful well beyond this chapter.

Chapter 7 is the odd one out, and is the more valuable for it. It is the only place in the book where a model is improved not by changing it but by averaging many copies of it, and Section 7.8 gives the specification that every ensemble in the sciences obeys. Bagging, random forests, AdaBoost, gradient boosting and XGBoost then follow as variations on that one equation, with boosting reversing the strategy: instead of averaging strong learners to remove variance, it sums weak ones to remove bias.

Part III: deep learning

Chapter 8 is the hinge of the book. It opens by admitting that inserting a hidden layer costs every guarantee the previous chapters relied on: no convexity, no closed form, no normal equations, no optimality conditions to check the answer against. What is left is gradient descent and the chain rule, and the rest of the book is a demonstration that this is enough. The

chapter derives the four equations of backpropagation in Section 8.8.4, and they are then used, unmodified, by every architecture that follows. It is also careful, in Section 8.4, to keep apart three questions that are routinely run together: whether the model class *contains* the function, whether gradient descent will *find* it, and whether the result will *generalise*. The universal approximation theorem [2, 3] answers only the first.

The five chapters that follow are four answers to a single question – what structure should be imposed on the weight matrix? – and one answer to a different question altogether.

Chapter 10 ties the weights and makes them local, and proves in Section 10.5 that the result commutes with translation, so that a feature learned in one place is available everywhere. Chapter 11 ties them across time instead, and proves that the gradients then decay exponentially with lag, which is why long-memory tasks fail even when the architecture could express the answer, and why the gated cells of Section 11.7 were invented [4]. Chapter 12 removes the labels and asks the network to reproduce its own input through a narrow layer, and proves in Section 12.3.4 that in the linear case this recovers exactly the principal subspace of Chapter 1 – which gives the exact floor against which any nonlinear autoencoder must be measured. Chapter 13 computes the weights from the data rather than storing them, and is equivariant under *all* permutations, a far larger group, which is why position has to be handed back to it explicitly in Section 13.3.

Chapter 9 is the different question. It keeps the network and changes what is asked of it: instead of fitting data, the network is differentiated with respect to its own input and inserted into a differential equation, so that the residual becomes the cost function. Exponential decay, logistic growth, the Poisson equation, the diffusion and wave equations and a Black-Scholes problem are solved this way, and Section 9.9 is honest about when this is worth doing and when a finite-difference scheme is simply better. Physics-informed neural networks [5, 6] are developed in Section 9.10.

Part IV: generative methods

The last four chapters change the question. Everything before them predicts a target given an input; these model the distribution of the data itself.

That change costs one thing, and it is worth naming precisely because the whole part is organised around it: a probability model needs a normalising constant, and that constant is a sum or an integral over a space too large to touch. It cannot be computed and it will not go away. What can be done is to find a formulation in which it is not needed, and the four chapters are four such formulations.

Chapter 14 refuses to compute it and samples instead. The gradient of the log-likelihood is a difference of two expectations, neither of which requires the value of the partition function, and Metropolis and Gibbs supply the necessary samples using ratios alone. The price is measured in Section 14.6.1: contrastive divergence with a single step gives a gradient that is biased, and by how much is stated there rather than glossed over. Chapter 15 bounds it, replacing the log-likelihood by the evidence lower bound of Section 15.2, and then measures the size of the gap it has introduced. Chapter 16 sidesteps it twice over, in one direction by learning a score, in which the normalisation has been differentiated away, and in the other by restricting to invertible maps whose Jacobian determinant is cheap [7, 8]. Chapter 17 abandons likelihood altogether and trains a classifier in its place [9]; the price is that the training problem becomes a game rather than a minimisation, and games need not converge.

Four evasions, four different prices. A reader who has understood the obstruction will be able to place the next generative model to appear by asking which of the four routes it takes, and what it therefore has to give up.

Part V and the concluding chapter

Part V is reserved for reinforcement learning, which learns from interaction rather than from a fixed data set, and which needs a treatment of its own – Markov decision processes, the Bellman equations, temporal-difference learning and policy gradients. Sutton and Barto [10] remains the standard reference, and Chapter 18 says what connects it to the material developed here.

Chapter 18 is not a summary. It sets out the shape of the whole argument and the dependencies between its parts as a graph in Section 18.1; it follows the handful of ideas that recur – conditioning, the derived loss, bias and variance, the chain rule, structure as a prior, the singular value decomposition, the intractable normalisation – and argues in Section 18.2 that these are the part of the material with the longest shelf life; it turns them round in Section 18.3 into a working order for attacking a problem that has not been seen before; it says plainly, in Section 18.4, what the book does not contain; and it closes with a survey of where these methods are being taken in the physical sciences and with a list, in Section 18.6, of things to be sceptical about, drawn wherever possible from results in this book that did not come out as expected.

Readers in a hurry may reasonably read that chapter first.

How to read this book

Every chapter is built the same way. The theory is developed with the intermediate algebra shown; each result that can be checked numerically is checked, by a program that is printed in the text and shipped with the book; each chapter closes with a section called *Summary and the programs*, which lists what was established and which file establishes it, and with a set of exercises graded from warm-up to project scale.

The dependencies are real but not oppressive. Chapters 1 and 2 are used by everything. Chapters 3-4 are used by everything after them. Chapter 8 is required for all of Parts III and IV. Within Part III, Chapters 9, 10, 11, 12 and 13 are independent of one another and may be read in any order or omitted. Within Part IV, Chapter 12 is a prerequisite for Chapter 15, and Chapter 15 for Chapter 16; Chapters 14 and 17 can be read on their own.

Three routes through the material have been used in practice.

- **A first course in data analysis and machine learning** (one semester): Chapters 1-5, then Chapter 7 and Chapter 8, ending with Chapter 10. The support vector machines of Chapter 6 are then optional.
- **A course on deep learning** for readers who already have the statistics: a rapid pass through Chapters 3 and 4, then Chapters 8-13 in full, followed by as much of Part IV as time allows.
- **Generative modelling and the physical sciences**: Chapters 1, 2, 4 and 8 as prerequisites, then Chapter 12 and the whole of Part IV, with Chapter 9 added for readers interested in differential equations.

Notation is fixed once, in Section 1.1, and adhered to throughout: vectors and matrices are boldface, vectors lower case and matrices upper case, and the design matrix is always $\boldsymbol{X}$ with rows indexing samples and columns indexing features. Where a chapter needs a symbol that clashes with an earlier one, the clash is pointed out rather than silently tolerated.

Choice of programming language

Python plays a central role in the development of machine learning techniques and tools for data analysis. The wealth of libraries written in Python, the ease with which they visualise results, the impressive galleries of existing examples, and the popularity of the Jupyter notebook framework – with the possibility of running compiled programs written in C++ or Fortran from within a notebook – made the choice of programming language for this book an easy one.

However, since the focus here is not only on *using* existing libraries such as Scikit-Learn, TensorFlow and PyTorch, but also on developing our own algorithms and codes, we present most algorithms first as plain Python built on NumPy, and only afterwards in the library form. Chapters 10, 11, 13, 14, 15 and 17 each carry a section in which the same model is written a second and a third time, in PyTorch and in TensorFlow, and the outputs are compared numerically against the implementation from scratch. The point of that exercise is not to teach an API. It is to make visible which parts of a library call correspond to which lines of the derivation, so that when a library result looks wrong the reader knows where to look.

Where computational speed genuinely matters we also give C++ or Fortran versions, written in an object-oriented style. A high-level language such as Python is excellent at structuring the flow of a calculation and at visualising it, while repetitive and CPU-intensive work is best left to a compiled language; a typical program in practice combines the two. Being multilingual is an advantage that applies to computing environments as much as it does to our globalised society.

The programs, the figures and the notebooks

A rule has been followed throughout, and it is worth stating so that the reader can hold the book to it: *every number quoted in the text is produced by a program printed in the text*, and every figure is generated by running that program. No figure has been drawn by hand and no result has been quoted from a paper without being re-derived here.

This has three practical consequences. The programs are collected in the `BookPrograms` directory of the book's source tree, one subdirectory per chapter, extracted verbatim from the chapters themselves, so that the LaTeX source and the runnable code cannot drift apart. The figures are generated by the scripts in `BookFigures`, with the same seeds as the programs, so that a figure and the number in the sentence beside it agree. And everything runs on a laptop in minutes – deliberately, because a result that the reader cannot reproduce is a result the reader has to take on trust.

The same material is available as an executable Jupyter-book, in which each chapter appears as a notebook and the parameters of every experiment can be varied. The book and the notebooks are maintained as parallel tracks covering the same subject matter; the printed book is self-contained, and a reader who never opens a notebook loses nothing but the ability to experiment conveniently.

On the results that did not come out well. A number of the experiments in this book fail, and the failures have been kept. A mode-collapsing generative model that covers fifteen of twenty-five modes, a recurrent network that cannot learn a dependency at lag one hundred, a constructed network that gradient descent cannot match, a bound that turns out to be several nats loose – these are reported as measured. They are the most useful numbers in the book, because they mark where the theory stops being a good

description of practice, and a reader who has seen only successful runs has no calibration at all.

Data handling, machine learning and ethical aspects

In most of the cases we study we either generate the data ourselves or use data sets distributed with Scikit-Learn or TensorFlow. Many of the examples we deal with are, from a privacy and data-protection point of view, rather innocuous and even boring results of numerical calculations. This does not absolve us from developing a sound ethical attitude to the data we use, to how we analyse them and to how we handle them.

The most immediate and simplest ethical aspects concern our approach to the scientific process itself. With version control software such as Git[6] and online repositories such as GitHub[7] and GitLab,[8] we can easily make the codes and data sets we have used freely and openly accessible to a wider community. This helps us almost automatically in making our science reproducible. The large open-source communities behind Scikit-Learn, TensorFlow, PyTorch and Keras are excellent examples: the codes can be tested and improved upon continuously, which helps the scientific community at large. It is much easier today to gain traction and acceptance for making one's science reproducible than it was a decade ago. From a societal standpoint this matters, since many developers are employees of public institutions such as universities and research laboratories. Our fellow taxpayers deserve to get something back for their money.

This more mechanical aspect of the ethics of science – the reproducibility of results – ought to be obvious, and it is part of the dialectics of science. The fact that many scientists are still unwilling to share their codes or their data is detrimental to the scientific discourse.

Before we proceed, a disclaimer is in order. Even though we may dream of computers developing some kind of higher learning capability, in the end it is we, yes you reading these lines, who construct and instruct, via various algorithms, the machine learning approaches. Self-driving cars, for example, rely on sophisticated programs which take into account the situations a car can encounter; extensive training data from GPS information and maps are fed into the software, and sensors and cameras feed further information to the programs. A great many ethical issues arise from this.

For self-driving cars, in which many of the standard machine learning algorithms discussed here enter the codes, at a certain stage we have to make choices – we, the people who wrote the program for a specific brand of car. All carmakers have as their utmost priority the security of the driver and the accompanying passengers. Suppose there are two obstacles ahead and a collision with one of them cannot be avoided. One obstacle is a heavy truck, the other a group of children crossing the road. An algorithm that ranks the survival probability of the occupants above everything else would opt for the children, since the likelihood of surviving a collision with the truck is much lower.

This leads to serious ethical questions. Why should we opt for such a rule? Who decides, and who is entitled to make such choices? Keep in mind that many of the algorithms you will encounter in this book, or hear about later, are based on simple programming instructions, and that you are very likely to be one of the people who end up writing such a code. Developing a sound ethical attitude to what we do, an approach well beyond the simple mechanistic one of making our science available and reproducible, is much needed. The self-driving car is just one of very many cases where we have to make choices. When you analyse data on

[6] `https://git-scm.com/`

[7] `https://github.com/`

[8] `https://about.gitlab.com/`

economic inequality, who guarantees that you are not weighting some data in a particular way, perhaps because you would dearly like a specific conclusion which supports your political views? Or what about the recurring claims that the credit-scoring algorithms of large technology companies carry a gender bias?

There is a technical counterpart to all of this, and it is not separate from the ethical one. A model reproduces the distribution it was trained on. If that distribution encodes a historical inequity, so will the model, and the bias-variance decomposition of Section 2.10 says nothing whatever about it, because it measures the error with respect to the data generating process and not with respect to the world we should like to have. Nothing in this book will detect such a problem for you. What the book can do, and tries to do, is to insist that you know what your model assumes, what data it saw, and what it would do on data it did not see; a model whose assumptions can be named is a model whose failures can be explained. Section 18.6 returns to this.

We do not have the answers here, nor shall we venture into a deeper discussion of these aspects, but we do want the reader to think about these topics in a more overarching way. A statistical data analysis, with its dry numbers and its graphs meant to guide the eye, does not necessarily reflect the truth, whatever that is. As a scientist, and after a university education, you are supposedly a better citizen, with an improved critical view and understanding of the scientific method, and perhaps some deeper understanding of the ethics of science at large. Use these insights. Be a critical citizen. You owe it to our society.

Part I
Mathematical and statistical basis with essential computational elements

Chapter 1
Linear Algebra for Machine Learning

Machine learning is, to a remarkable extent, applied linear algebra. A data set is a matrix, a model is very often a matrix acting on a vector, training a model means solving a linear system or descending along a gradient built from matrix products, and the two questions one asks most frequently of a data set – how many independent directions does it really contain, and which of them matter – are questions about the spectrum of a matrix. This chapter assembles the linear algebra we shall need, in the order in which we shall need it, and it does so with an eye on the computer throughout.

The chapter has two halves. The first develops the algebraic language: vectors and matrices, orthogonality and projection, the calculus of derivatives with respect to vectors and matrices, the spectral decomposition of symmetric matrices, and the covariance matrix in which so much of statistics is encoded. The second half asks how these objects are actually computed: how errors are measured, how linear systems are solved directly and iteratively, how eigenvalue problems are attacked, and finally how the singular value decomposition answers, in one construction, the rank question, the least-squares question, the compression question and the principal component question.

1.1 Notation and conventions

Throughout this book vectors, matrices and higher-order tensors are set in boldface. Vectors are denoted by lower-case letters ($\boldsymbol{x}$, $\boldsymbol{y}$, $\boldsymbol{\theta}$, ...) and matrices by upper-case letters ($\boldsymbol{X}$, $\boldsymbol{A}$, $\boldsymbol{U}$, ...). A vector is always a *column* vector unless we say otherwise.

Unless stated otherwise the elements v_i of a vector $\boldsymbol{v}$ are real. A real vector of length n satisfies $\boldsymbol{x} \in \mathbb{R}^n$; for complex vectors we write $\boldsymbol{x} \in \mathbb{C}^n$. A real $n \times p$ matrix satisfies $\boldsymbol{X} \in \mathbb{R}^{n \times p}$. Index counting starts at zero, so that the matrix elements of an $n \times p$ matrix run from x_{00} to $x_{n-1,p-1}$. This is not the convention of most linear algebra textbooks, but it is the convention of Python, C++ and most other languages we shall write code in, and keeping the mathematics and the code in step is worth more than agreement with tradition.

We use the standard shorthand symbols $\forall$ (for all), $\Rightarrow$ (implies) and $\equiv$ (defined as), with $\mathbb{R}$, $\mathbb{I}$ and $\mathbb{C}$ denoting the real, integer and complex number fields.

The data conventions of this book.

A supervised learning problem presents us with n observations, each described by p features (also called predictors, inputs or independent variables), together with n target values (outputs, responses or dependent variables). We collect the features in the *design matrix*

$$\boldsymbol{X} = \begin{bmatrix} x_{00} & x_{01} & \cdots & x_{0,p-1} \\ x_{10} & x_{11} & \cdots & x_{1,p-1} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n-1,0} & x_{n-1,1} & \cdots & x_{n-1,p-1} \end{bmatrix} \in \mathbb{R}^{n\times p}, \tag{1.1}$$

in which *row i is one observation* and *column j is one feature*, and the targets in a vector $\boldsymbol{y} \in \mathbb{R}^n$. Model parameters are collected in $\boldsymbol{\theta} \in \mathbb{R}^p$, so that the linear model reads $\tilde{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\theta}$. Almost every matrix in this book is either a design matrix, something built from one, or an operator acting on the parameter vector, and it pays to fix these three symbols now.

The two dimensions play very different roles. The number of observations n is a property of the experiment; the number of features p is a property of our description of it. Regimes with $n \gg p$ and $n \ll p$ behave in entirely different ways, and much of what follows – rank, conditioning, regularisation – is a way of speaking precisely about that difference.

1.2 Vectors

A column vector of length n and its transpose are

$$\boldsymbol{x} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix}, \qquad \boldsymbol{x}^T = \begin{bmatrix} x_0 & x_1 & \cdots & x_{n-1} \end{bmatrix}.$$

For complex vectors the *Hermitian conjugate* (adjoint) is

$$\boldsymbol{x}^\dagger = \begin{bmatrix} x_0^* & x_1^* & \cdots & x_{n-1}^* \end{bmatrix}.$$

Complex vectors will appear only rarely in this book – in the Fourier analysis of signals and in a few spectral arguments – but the definitions cost nothing and we state them once and for all.

The *inner (dot) product* is the scalar

$$\boldsymbol{x}^T\boldsymbol{y} = \sum_{i=0}^{n-1} x_i y_i, \tag{1.2}$$

and for complex vectors $\boldsymbol{x}^\dagger\boldsymbol{x} = \sum_i x_i^* x_i = \|\boldsymbol{x}\|^2$. The inner product is the single most heavily used operation in this book. It computes a prediction, $\tilde{y}_i = \boldsymbol{x}_i^T\boldsymbol{\theta}$; it measures similarity between two observations; and, once both vectors are centred and scaled, it *is* the correlation coefficient between two features.

The *outer product* of $\boldsymbol{y} \in \mathbb{R}^n$ and $\boldsymbol{z} \in \mathbb{R}^m$ is the $n \times m$ matrix

$$\boldsymbol{y}\boldsymbol{z}^T \implies (\boldsymbol{y}\boldsymbol{z}^T)_{ij} = y_i z_j, \tag{1.3}$$

with $\boldsymbol{y}\boldsymbol{z}^\dagger$ and $(\boldsymbol{y}\boldsymbol{z}^\dagger)_{ij} = y_i z_j^*$ in the complex case. The outer product always has rank one, a fact we shall lean on heavily when we come to low-rank approximation in Section 1.20: every rank-one matrix is an outer product and every outer product has rank one.

The elementary vector operations are

$$\begin{aligned}
\text{Addition / subtraction:} &\quad \boldsymbol{x} = \boldsymbol{y} \pm \boldsymbol{z} \;\Rightarrow\; x_i = y_i \pm z_i, \\
\text{Scalar multiplication:} &\quad \boldsymbol{x} = \gamma \boldsymbol{y} \;\Rightarrow\; x_i = \gamma y_i, \\
\text{Hadamard (element-wise) product:} &\quad \boldsymbol{x} = \boldsymbol{y} \circ \boldsymbol{z} \;\Rightarrow\; x_i = y_i z_i.
\end{aligned}$$

The Hadamard product deserves a remark. It looks like the least interesting operation on the list, and in classical linear algebra it very nearly is; but it is exactly what a neural network does when it applies an activation function element-wise, and it is what appears in the back-propagation equations of the chapter on neural networks when the chain rule is written in matrix form.

1.3 Matrices: definitions and algebraic operations

A general $n \times p$ matrix can be read either entry by entry or as a list of columns,

$$\boldsymbol{X} = \begin{bmatrix} x_{00} & x_{01} & \cdots & x_{0,p-1} \\ x_{10} & x_{11} & \cdots & x_{1,p-1} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n-1,0} & \cdots & \cdots & x_{n-1,p-1} \end{bmatrix} = \begin{bmatrix} \boldsymbol{x}_0 & \boldsymbol{x}_1 & \cdots & \boldsymbol{x}_{p-1} \end{bmatrix},$$

where $\boldsymbol{x}_j \in \mathbb{R}^n$ is the jth column, that is the jth feature measured across all n observations. Both readings are useful and we shall switch between them without warning.

The fundamental algebraic operations are:

$$\begin{aligned}
\text{Scalar multiplication:} &\quad \boldsymbol{A} = \gamma \boldsymbol{B} \;\Rightarrow\; a_{ij} = \gamma b_{ij}, \\
\text{Addition:} &\quad \boldsymbol{A} = \boldsymbol{B} \pm \boldsymbol{C} \;\Rightarrow\; a_{ij} = b_{ij} \pm c_{ij}, \\
\text{Matrix-vector product:} &\quad \boldsymbol{y} = \boldsymbol{A}\boldsymbol{x} \;\Rightarrow\; y_i = \sum_j a_{ij} x_j, \\
\text{Matrix-matrix product:} &\quad \boldsymbol{A} = \boldsymbol{B}\boldsymbol{C} \;\Rightarrow\; a_{ij} = \sum_k b_{ik} c_{kj}, \\
\text{Transpose:} &\quad \boldsymbol{A} = \boldsymbol{B}^T \;\Rightarrow\; a_{ij} = b_{ji}.
\end{aligned}$$

Matrix multiplication is associative and distributive but *not* commutative; $\boldsymbol{A}\boldsymbol{B} \neq \boldsymbol{B}\boldsymbol{A}$ in general, and a surprising number of errors in derivations come from forgetting it. Useful consequences of the definitions are $(\boldsymbol{A}\boldsymbol{B})^T = \boldsymbol{B}^T\boldsymbol{A}^T$ and $(\boldsymbol{A}\boldsymbol{B})^{-1} = \boldsymbol{B}^{-1}\boldsymbol{A}^{-1}$ when the inverses exist.

Inverse.

If $\boldsymbol{A}$ is square and nonsingular, its inverse satisfies $\boldsymbol{A}^{-1}\boldsymbol{A} = \boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{I}$, with $\boldsymbol{I}$ the identity matrix. Rectangular matrices have no inverse; what replaces it is the pseudoinverse of Section 1.19, and that replacement is the mathematical content of ordinary least squares.

Hermitian conjugate.

The Hermitian conjugate $\boldsymbol{A}^\dagger$ is obtained by transposing $\boldsymbol{A}$ and complex-conjugating every element, $(A^\dagger)_{ij} = a^*_{ji}$. For real matrices it coincides with the transpose.

Trace.

The trace of a square matrix is the sum of its diagonal elements, $\mathrm{Tr}(\boldsymbol{A}) = \sum_i a_{ii}$. It is linear, and it is invariant under cyclic permutation,

$$\mathrm{Tr}(\boldsymbol{ABC}) = \mathrm{Tr}(\boldsymbol{BCA}) = \mathrm{Tr}(\boldsymbol{CAB}), \tag{1.4}$$

provided the products are defined. The cyclic property will do a disproportionate amount of work in Section 1.8, where it turns awkward index manipulations into one-line derivations.

Nonsingularity: equivalent characterisations.

For an $n \times n$ matrix $\boldsymbol{A}$ the following statements are equivalent: (i) $\boldsymbol{A}^{-1}$ exists; (ii) $\boldsymbol{Ax} = \boldsymbol{0}$ implies $\boldsymbol{x} = \boldsymbol{0}$; (iii) the rows of $\boldsymbol{A}$ form a basis of $\mathbb{R}^n$; (iv) the columns of $\boldsymbol{A}$ form a basis of $\mathbb{R}^n$; (v) $\boldsymbol{A}$ is a product of elementary matrices; (vi) zero is not an eigenvalue of $\boldsymbol{A}$; (vii) $|\boldsymbol{A}| \neq 0$.

This list is worth reading twice, because in machine learning we spend most of our time near its boundary. Condition (iv) fails exactly when one feature is a linear combination of the others – perfect multicollinearity – and condition (vi) then fails with it. What makes the situation delicate in practice is that features are rarely *exactly* dependent; they are nearly dependent, so that the list technically holds while every quantity it guarantees is numerically worthless. Section 1.12 makes that statement quantitative.

1.4 Special matrix types

Table 1.1 lists the matrix types that recur throughout this book.

Name	**Defining relation**	**Element condition**
Symmetric	$\boldsymbol{A} = \boldsymbol{A}^T$	$a_{ij} = a_{ji}$
Real orthogonal	$\boldsymbol{A} = (\boldsymbol{A}^T)^{-1}$	$\sum_k a_{ik} a_{jk} = \delta_{ij}$
Hermitian	$\boldsymbol{A} = \boldsymbol{A}^\dagger$	$a_{ij} = a_{ji}^*$
Unitary	$\boldsymbol{A} = (\boldsymbol{A}^\dagger)^{-1}$	$\sum_k a_{ik} a_{jk}^* = \delta_{ij}$
Real matrix	$\boldsymbol{A} = \boldsymbol{A}^*$	$a_{ij} = a_{ij}^*$

Table 1.1 Important special matrix types and their defining properties. For real matrices, Hermitian reduces to symmetric and unitary to orthogonal; these are the two cases we shall meet almost exclusively.

Structured (sparse) matrices worth naming, because recognising one can change the cost of an algorithm by orders of magnitude:

- **Diagonal**: $a_{ij} = 0$ for $i \neq j$.
- **Upper/lower triangular**: $a_{ij} = 0$ for $i > j$ / $i < j$.
- **Upper/lower Hessenberg**: $a_{ij} = 0$ for $i > j+1$ / $i < j+1$.
- **Tridiagonal**: $a_{ij} = 0$ for $|i-j| > 1$.
- **Banded with bandwidth** p: $a_{ij} = 0$ for $|i-j| > p$.
- Block upper/lower triangular matrices, and more.

Two classes deserve special mention because of the frequency with which they appear in what follows. A *symmetric* matrix has real eigenvalues and an orthonormal eigenbasis (Section 1.9); every covariance matrix, every kernel matrix and every matrix of the form $\boldsymbol{X}^T\boldsymbol{X}$ is symmetric. A symmetric matrix is *positive semi-definite* if $\boldsymbol{z}^T\boldsymbol{A}\boldsymbol{z} \geq 0$ for all $\boldsymbol{z}$, and *positive definite* if the inequality is strict for $\boldsymbol{z} \neq \boldsymbol{0}$; equivalently, if all its eigenvalues are non-negative, respectively positive. Positive definiteness is what makes the Cholesky factorisation of Section 1.14 and the conjugate gradient method of Section 1.16 applicable, and it is what guarantees that a quadratic loss has a unique minimum rather than a saddle.

Machine learning connection. Sparsity is not merely a numerical convenience in machine learning; it is often the structure of the data itself. A bag-of-words document-term matrix, a one-hot encoding of a categorical variable with many levels, and the user-item matrix of a recommender system are all overwhelmingly zero. Storing such a matrix densely can be the difference between a calculation that fits in memory and one that does not, and `scipy.sparse` exists for precisely this reason. We return to the point in Section 1.5.

1.5 Arrays in practice: numpy, BLAS and LAPACK

Before going further it is worth spending a few pages on how the objects of the preceding sections are actually represented in a program, because a certain number of otherwise mysterious performance results and error messages become obvious once the representation is understood.

The libraries underneath.

Almost every numerical linear algebra computation performed anywhere, regardless of the language it is written in, eventually calls one of two libraries. *BLAS* (Basic Linear Algebra Subprograms) provides the elementary operations in three levels: level 1 is vector-vector work such as the inner product, level 2 is matrix-vector work, and level 3 is matrix-matrix work. *LAPACK* builds on BLAS and provides the higher-level decompositions – LU, Cholesky, QR, the symmetric eigenvalue problem, the singular value decomposition – and supersedes the older LINPACK and EISPACK packages. Both are freely available from `netlib.org`, and both are shipped in heavily optimised vendor implementations (OpenBLAS, MKL, Accelerate) tuned to the cache hierarchy of the machine.

The practical consequence is worth stating plainly. When we write `A @ B` in numpy, we are calling a level-3 BLAS routine that has been tuned by specialists over decades; when we write the same product as a triple loop in Python, we are not. The difference is routinely two to three orders of magnitude. *Vectorise, and let the library do the work.* For Python, *numpy* is the standard array package; for C++, *Armadillo* provides a comparable interface with a syntax close to the mathematics, and both ultimately dispatch to BLAS and LAPACK.

Declaring arrays.

The standard import and a first vector:

```
import numpy as np

n = 10
```

```
x = np.random.normal(size=n)      # n samples from N(0, 1)
print(x)

x = np.array([1, 2, 3])           # explicit entries: x_0=1, x_1=2, x_2=3
print(x)
```

Both Python and C++ number array elements from zero, so a vector with n elements is the sequence $x_0, x_1, \ldots, x_{n-1}$, exactly as in Section 1.1.

A frequent source of confusion is the data type. The array `np.array([4, 7, 8])` holds integers, and applying an integer-valued operation to it silently truncates. Compare

```
import numpy as np
from math import log

x = np.array([4, 7, 8])
for i in range(len(x)):
    x[i] = log(x[i])              # integer array: results are truncated
print(x)                          # prints [1 1 2]

x = np.log(np.array([4.0, 7.0, 8.0]))   # float array, vectorised
print(x)                                # prints [1.386... 1.945... 2.079...]
print(x.itemsize)                       # 8 bytes = 64 bits per element
```

The second version is shorter, an order of magnitude faster, and correct. Numpy's unary functions such as `np.log` are vectorised: the loop happens inside compiled code rather than in the interpreter. We recommend using them in preference to the corresponding functions from Python's `math` module throughout.

Matrices are declared as nested lists, and the usual constructors produce arrays of a given shape:

```
import numpy as np

A = np.array([[4.0, 7.0, 8.0], [3.0, 10.0, 11.0], [4.0, 5.0, 7.0]])
print(A.shape)                    # (3, 3)
print(A[:, 0])                    # first column  -- all rows, column 0
print(A[1, :])                    # second row

n = 10
print(np.zeros((n, n)))           # all elements zero
print(np.ones((n, n)))            # all elements one
print(np.random.rand(n, n))       # uniform on [0, 1)
print(np.eye(n))                  # identity matrix
```

Row-major and column-major storage.

A matrix is a two-dimensional object stored in a one-dimensional memory. C, C++ and numpy store it in *row-major* order, so that consecutive elements of a row are adjacent in memory; Fortran, MATLAB and Armadillo use *column-major* order. The distinction is invisible mathematically and very visible in a profiler: traversing an array along the direction in which it is stored uses each cache line fully, and traversing it across that direction can be several times slower for large matrices. When interfacing Python with Fortran or C++ code, it is also the single most common source of transposed results. The `ravel` function makes the ordering explicit:

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]], dtype=np.float64)
```

```
print(np.ravel(a))                # 'C' (row-major) order, the default
print(np.ravel(a, order='F'))     # 'F' (column-major) order
print(a.reshape(-1))              # same as np.ravel(a)
```

Reductions along an axis.

Preparing data for a machine learning algorithm almost always involves centring and scaling, and both are reductions along an axis of the design matrix. Since rows are observations and columns are features (Eq. 1.1), *a per-feature quantity is an average over axis 0*:

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]], dtype=np.float64)

print(np.mean(a))                         # mean over all elements
print(np.mean(a, axis=0, keepdims=True))  # one mean per feature (column)
print(np.mean(a, axis=1, keepdims=True))  # one mean per observation (row)

# Standardising the design matrix: zero mean and unit variance per feature
X = (a - np.mean(a, axis=0)) / np.std(a, axis=0)
```

The `keepdims` argument controls whether the result keeps its orientation as a row or column, which matters as soon as the result is broadcast against the original array. Getting the axis wrong is perhaps the most common bug in data preprocessing, and it is a silent one: the code runs, the numbers are wrong.

Machine learning connection. The scaling above must be performed using statistics computed on the *training* data only, and the same numbers must then be applied to the test data. Computing the mean and standard deviation over the full data set before splitting leaks information from the test set into the training procedure and produces optimistically biased estimates of the generalisation error. We return to this in Chapter 2; it is mentioned here because the mistake is made at the level of two lines of numpy.

1.6 Orthonormal bases and projections

A set of vectors $\{\boldsymbol{q}_0, \dots, \boldsymbol{q}_{k-1}\}$ in $\mathbb{R}^n$ is *orthonormal* if

$$\boldsymbol{q}_i^T \boldsymbol{q}_j = \delta_{ij}, \tag{1.5}$$

that is if the vectors are mutually orthogonal and each has unit length. If $k = n$ the set is an orthonormal basis, and collecting the vectors as the columns of a matrix $\boldsymbol{Q}$ gives $\boldsymbol{Q}^T\boldsymbol{Q} = \boldsymbol{Q}\boldsymbol{Q}^T = \boldsymbol{I}$ together with the completeness relation

$$\sum_{i=0}^{n-1} \boldsymbol{q}_i \boldsymbol{q}_i^T = \boldsymbol{I}. \tag{1.6}$$

Equation (1.6) says that the identity can be resolved into a sum of rank-one outer products, one per basis direction. Any vector then expands as

$$\boldsymbol{x} = \sum_{i=0}^{n-1} \left(\boldsymbol{q}_i^T \boldsymbol{x}\right) \boldsymbol{q}_i, \tag{1.7}$$

with the coefficient $\boldsymbol{q}_i^T \boldsymbol{x}$ the component of $\boldsymbol{x}$ along $\boldsymbol{q}_i$. Expanding a vector in an orthonormal basis requires no linear system to be solved – each coefficient is one inner product – and that is the entire reason orthonormal bases are worth constructing.

Projection onto a subspace.

Let $\boldsymbol{Q} \in \mathbb{R}^{n \times k}$ have orthonormal columns spanning a subspace $\mathscr{S}$. The matrix

$$\boldsymbol{P} = \boldsymbol{Q}\boldsymbol{Q}^T \tag{1.8}$$

is the *orthogonal projector* onto $\mathscr{S}$. It is symmetric and idempotent,

$$\boldsymbol{P}^T = \boldsymbol{P}, \qquad \boldsymbol{P}^2 = \boldsymbol{Q}\underbrace{\boldsymbol{Q}^T\boldsymbol{Q}}_{=\boldsymbol{I}_k}\boldsymbol{Q}^T = \boldsymbol{Q}\boldsymbol{Q}^T = \boldsymbol{P}, \tag{1.9}$$

and these two properties characterise orthogonal projectors completely. The vector $\boldsymbol{P}\boldsymbol{x}$ is the point of $\mathscr{S}$ closest to $\boldsymbol{x}$, and the residual $(\boldsymbol{I} - \boldsymbol{P})\boldsymbol{x}$ is orthogonal to every vector in $\mathscr{S}$. The complementary matrix $\boldsymbol{I} - \boldsymbol{P}$ is itself a projector, onto the orthogonal complement of $\mathscr{S}$, and $\boldsymbol{P}(\boldsymbol{I} - \boldsymbol{P}) = \boldsymbol{0}$.

Idempotency has an immediate spectral consequence. If $\boldsymbol{P}\boldsymbol{v} = \lambda \boldsymbol{v}$ then $\boldsymbol{P}^2\boldsymbol{v} = \lambda^2 \boldsymbol{v} = \lambda \boldsymbol{v}$, so $\lambda^2 = \lambda$ and every eigenvalue of a projector is either 0 or 1. The multiplicity of the eigenvalue 1 is the dimension of $\mathscr{S}$, whence

$$\operatorname{Tr}(\boldsymbol{P}) = \dim \mathscr{S} = k. \tag{1.10}$$

Machine learning connection. Equation (1.8) is the whole of ordinary least squares in one line. Fitting $\boldsymbol{y}$ with the linear model $\tilde{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\theta}$ means finding the point of the column space of $\boldsymbol{X}$ closest to $\boldsymbol{y}$, that is projecting $\boldsymbol{y}$ onto that column space. The projector is the *hat matrix*

$$\boldsymbol{H} = \boldsymbol{X}\left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T,$$

which one verifies is symmetric and idempotent exactly as in Eq. (1.9), and which reduces to $\boldsymbol{Q}\boldsymbol{Q}^T$ when the columns of $\boldsymbol{X}$ have been orthonormalised. By Eq. (1.10), $\operatorname{Tr}(\boldsymbol{H}) = p$, the number of parameters – which is why the trace of the hat matrix appears as the effective number of degrees of freedom in model selection criteria, and why Ridge regression, whose hat matrix is not idempotent, is said to have a *fractional* number of degrees of freedom. We derive all of this in Chapter 3.

Gram-Schmidt and the QR decomposition.

Given a set of linearly independent columns $\boldsymbol{x}_0, \ldots, \boldsymbol{x}_{p-1}$, the Gram-Schmidt procedure builds an orthonormal basis for their span by subtracting, from each new vector, its projection onto everything already accepted:

$$\tilde{\boldsymbol{q}}_j = \boldsymbol{x}_j - \sum_{i<j} \left(\boldsymbol{q}_i^T \boldsymbol{x}_j\right) \boldsymbol{q}_i, \qquad \boldsymbol{q}_j = \frac{\tilde{\boldsymbol{q}}_j}{\|\tilde{\boldsymbol{q}}_j\|_2}. \tag{1.11}$$

Collecting the coefficients gives the *QR decomposition*

$$\boldsymbol{X} = \boldsymbol{Q}\boldsymbol{R}, \tag{1.12}$$

with $\boldsymbol{Q} \in \mathbb{R}^{n\times p}$ having orthonormal columns and $\boldsymbol{R} \in \mathbb{R}^{p\times p}$ upper triangular. The classical Gram-Schmidt algorithm as written in Eq. (1.11) is numerically unstable – orthogonality is lost progressively as rounding errors accumulate – and practical implementations use either the modified Gram-Schmidt variant or, better, a sequence of Householder reflections. This is what `numpy.linalg.qr` calls through LAPACK, and we shall use the library routine. The QR decomposition provides one of the two standard stable routes to the least-squares solution; the other, through the singular value decomposition, is the subject of Section 1.18.

1.7 Orthogonal transformations

A real $n \times n$ matrix $\boldsymbol{Q}$ is *orthogonal* if

$$\boldsymbol{Q}^T\boldsymbol{Q} = \boldsymbol{Q}\boldsymbol{Q}^T = \boldsymbol{I}, \qquad \text{equivalently} \qquad \boldsymbol{Q}^{-1} = \boldsymbol{Q}^T. \tag{1.13}$$

The complex analogue is a unitary matrix, $\boldsymbol{U}^\dagger\boldsymbol{U} = \boldsymbol{I}$. Orthogonal matrices are the rotations and reflections of $\mathbb{R}^n$, and they are distinguished by the property that they change nothing that we measure.

Inner products, lengths and angles are preserved.

For any two vectors,

$$(\boldsymbol{Q}\boldsymbol{x})^T(\boldsymbol{Q}\boldsymbol{y}) = \boldsymbol{x}^T \underbrace{\boldsymbol{Q}^T\boldsymbol{Q}}_{=\boldsymbol{I}}\boldsymbol{y} = \boldsymbol{x}^T\boldsymbol{y}, \tag{1.14}$$

so inner products are unchanged; taking $\boldsymbol{y} = \boldsymbol{x}$ shows that lengths are unchanged, $\|\boldsymbol{Q}\boldsymbol{x}\|_2 = \|\boldsymbol{x}\|_2$, and the two facts together show that angles are unchanged. An orthonormal set therefore remains orthonormal: if $\boldsymbol{q}_i^T\boldsymbol{q}_j = \delta_{ij}$ then $(\boldsymbol{Q}\boldsymbol{q}_i)^T(\boldsymbol{Q}\boldsymbol{q}_j) = \delta_{ij}$ as well. Since $|\boldsymbol{Q}|^2 = |\boldsymbol{Q}^T\boldsymbol{Q}| = 1$, the determinant of an orthogonal matrix is ± 1; the value $+1$ corresponds to a rotation and -1 to a rotation combined with a reflection.

Why this matters numerically.

Applying an orthogonal matrix cannot amplify an error. If $\boldsymbol{x}$ carries a perturbation $\delta\boldsymbol{x}$, then $\boldsymbol{Q}(\boldsymbol{x}+\delta\boldsymbol{x})$ carries the perturbation $\boldsymbol{Q}\delta\boldsymbol{x}$, of exactly the same length. This is the reason why every stable algorithm in this chapter is built out of orthogonal transformations: the QR decomposition, the reduction to tridiagonal form, and the singular value decomposition all achieve their stability by using nothing but rotations and reflections. An algorithm that instead forms $\boldsymbol{X}^T\boldsymbol{X}$, as we shall see in Section 1.18, throws away half of the available significant digits before it starts.

Machine learning connection. A change of orthonormal basis is a rotation of the feature space, and it leaves distances, and therefore any distance-based method, unchanged: k-nearest neighbours, k-means clustering and radial-basis-function kernels all give identical results before and after an orthogonal transformation of the features. This is precisely why principal component analysis (Section 1.21) is a *safe* preprocessing step: it rotates the data into a new orthonormal basis, but it does not distort the geometry. Scaling the features, by contrast, is not an orthogonal transformation and does change the geometry – which is exactly why standardisation changes the answer that k-means returns.

1.8 Derivatives with respect to vectors and matrices

Training a model means minimising a cost function, and minimising means differentiating. The cost functions of machine learning are scalars that depend on a vector of parameters, or on a whole matrix of them, so we need a calculus of derivatives with respect to vectors and matrices. The subject has a reputation for being fiddly, which it deserves only if one insists on writing everything in indices. A handful of results, derived once, cover essentially every gradient computation in this book, and we derive them here in the order in which they build on one another.

Throughout this section $\boldsymbol{y}$ is a vector of length m with elements $y_0, \ldots, y_{m-1}$, and $\boldsymbol{x}$ is a vector of length n with $\boldsymbol{x}^T = [x_0, x_1, \ldots, x_{n-1}]$, following the zero-based convention of Section 1.1. We assume $\boldsymbol{y}$ to be a function of $\boldsymbol{x}$,

$$\boldsymbol{y} = f(\boldsymbol{x}). \tag{1.15}$$

1.8.1 The Jacobian

The partial derivatives of the components of $\boldsymbol{y}$ with respect to the components of $\boldsymbol{x}$ are collected in the *Jacobian matrix*

$$\boldsymbol{J} = \frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} = \begin{bmatrix} \dfrac{\partial y_0}{\partial x_0} & \dfrac{\partial y_0}{\partial x_1} & \cdots & \dfrac{\partial y_0}{\partial x_{n-1}} \\ \dfrac{\partial y_1}{\partial x_0} & \dfrac{\partial y_1}{\partial x_1} & \cdots & \dfrac{\partial y_1}{\partial x_{n-1}} \\ \vdots & \vdots & \ddots & \vdots \\ \dfrac{\partial y_{m-1}}{\partial x_0} & \dfrac{\partial y_{m-1}}{\partial x_1} & \cdots & \dfrac{\partial y_{m-1}}{\partial x_{n-1}} \end{bmatrix}, \tag{1.16}$$

an $m \times n$ matrix whose entry (i,k) is $\partial y_i / \partial x_k$. If $\boldsymbol{x}$ is a scalar the Jacobian reduces to a single column, an $m \times 1$ matrix; if $\boldsymbol{y}$ is a scalar it reduces to a single row, a $1 \times n$ matrix. When $m = n$ the matrix is square and its determinant is the *Jacobian determinant*; both the matrix and, when it exists, the determinant are commonly called simply the Jacobian. The Jacobian represents the differential of $\boldsymbol{y}$ at every point at which the function is differentiable, and it is the object from which every other result in this section follows.

1.8.2 A warning about layout conventions

Equation (1.16) forces a choice that is the single most common source of confusion in this subject, and it is best to confront it immediately. Take $\boldsymbol{y}$ to be a scalar α. The Jacobian is then a $1 \times n$ matrix, that is a *row* vector,

$$\frac{\partial \alpha}{\partial \boldsymbol{x}} = \begin{bmatrix} \dfrac{\partial \alpha}{\partial x_0} & \cdots & \dfrac{\partial \alpha}{\partial x_{n-1}} \end{bmatrix} \qquad (\textit{numerator} \text{ layout}). \tag{1.17}$$

This is the convention that follows naturally from the Jacobian, it is the one used in the lecture notes accompanying this book, and it is the one we shall use in the derivations that follow.

The gradient, on the other hand, is conventionally a *column* vector, so that it can be subtracted from the parameter vector in a gradient-descent step. That is the *denominator* layout,

$$\nabla_{\boldsymbol{x}}\alpha = \left(\frac{\partial \alpha}{\partial \boldsymbol{x}}\right)^T = \begin{bmatrix} \dfrac{\partial \alpha}{\partial x_0} \\ \vdots \\ \dfrac{\partial \alpha}{\partial x_{n-1}} \end{bmatrix} \qquad (\textit{denominator} \text{ layout}). \tag{1.18}$$

The two differ by a transpose and by nothing else. Neither is more correct than the other; what is fatal is to mix them inside a single calculation, and a large fraction of the sign and shape errors students make in this course come from doing exactly that.

Our practice in this book is the following. Derivations are carried out in the numerator layout, because the Jacobian (1.16) delivers it and because the algebra is then bookkeeping-free. Final results that are to be *used* as gradients – fed to an optimiser, or set to zero to obtain a normal equation – are quoted in both forms, with the column form marked by a derivative with respect to $\boldsymbol{\theta}^T$ rather than $\boldsymbol{\theta}$:

$$\frac{\partial C}{\partial \boldsymbol{\theta}^T} = \left(\frac{\partial C}{\partial \boldsymbol{\theta}}\right)^T = \nabla_{\boldsymbol{\theta}} C. \tag{1.19}$$

Whenever a boxed result below carries both forms, Eq. (1.19) is the only thing relating them. For a scalar function of a matrix we use throughout the shape-preserving convention

$$\left(\frac{\partial f}{\partial \boldsymbol{A}}\right)_{ij} = \frac{\partial f}{\partial a_{ij}}, \tag{1.20}$$

so that the derivative has the same shape as the matrix, which is what an optimiser updating a weight matrix requires.

1.8.3 Four worked examples

The following four cases, built up in order, generate essentially every derivative used in this book.

Example 1: a matrix times a vector.

Let $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}$ with $\boldsymbol{A}$ an $m \times n$ matrix that does not depend on $\boldsymbol{x}$. Componentwise

$$y_i = \sum_{j=0}^{n-1} a_{ij} x_j, \qquad \forall\, i = 0, 1, \ldots, m-1,$$

so that differentiating picks out a single term,

$$\frac{\partial y_i}{\partial x_k} = a_{ik}.$$

By the definition (1.16) of the Jacobian this says

$$\boxed{\frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} = \frac{\partial\,(\boldsymbol{A}\boldsymbol{x})}{\partial \boldsymbol{x}} = \boldsymbol{A}.} \tag{1.21}$$

The derivative of a linear map is the map itself, exactly as in one dimension.

Example 2: a bilinear form.

Define the scalar

$$\alpha = \boldsymbol{y}^T \boldsymbol{A} \boldsymbol{x}, \tag{1.22}$$

with $\boldsymbol{y}$ of length m, $\boldsymbol{A}$ an $m \times n$ matrix independent of both vectors, and $\boldsymbol{x}$ of length n. Cost functions are scalars – the mean squared error is one – so this is the shape of object we shall usually be differentiating. Introduce the intermediate vector $\boldsymbol{z}^T = \boldsymbol{y}^T \boldsymbol{A}$, of length n, so that $\alpha = \boldsymbol{z}^T \boldsymbol{x}$. Then, by Example 1 applied to the inner product,

$$\frac{\partial \alpha}{\partial \boldsymbol{x}} = \frac{\partial \left(\boldsymbol{z}^T \boldsymbol{x} \right)}{\partial \boldsymbol{x}} = \boldsymbol{z}^T = \boldsymbol{y}^T \boldsymbol{A}. \tag{1.23}$$

The elements of $\boldsymbol{z}^T$ and $\boldsymbol{z}$ are of course the same numbers; the transpose appears because the inner product of two vectors is a scalar, whose Jacobian is a row. Since α is a scalar it equals its own transpose, $\alpha = \alpha^T = \boldsymbol{x}^T \boldsymbol{A}^T \boldsymbol{y}$, and repeating the argument with $\boldsymbol{z}^T = \boldsymbol{x}^T \boldsymbol{A}^T$ gives

$$\frac{\partial \alpha}{\partial \boldsymbol{y}} = \boldsymbol{x}^T \boldsymbol{A}^T. \tag{1.24}$$

The special case $\boldsymbol{A} = \boldsymbol{I}$ is worth recording on its own: for a constant vector $\boldsymbol{a}$,

$$\frac{\partial}{\partial \boldsymbol{x}} \left(\boldsymbol{a}^T \boldsymbol{x} \right) = \frac{\partial}{\partial \boldsymbol{x}} \left(\boldsymbol{x}^T \boldsymbol{a} \right) = \boldsymbol{a}^T, \qquad \nabla_{\boldsymbol{x}} \left(\boldsymbol{a}^T \boldsymbol{x} \right) = \boldsymbol{a}. \tag{1.25}$$

Example 3: a quadratic form.

Now let $\boldsymbol{A}$ be square, $n \times n$, and replace $\boldsymbol{y}$ by $\boldsymbol{x}$:

$$\alpha = \boldsymbol{x}^T \boldsymbol{A} \boldsymbol{x} = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} x_i a_{ij} x_j. \tag{1.26}$$

Here x_k occurs in two places, once through the index i and once through j, so the product rule produces two sums,

$$\frac{\partial \alpha}{\partial x_k} = \sum_{i=0}^{n-1} a_{ik} x_i + \sum_{j=0}^{n-1} a_{kj} x_j, \qquad \forall k = 0, 1, \ldots, n-1.$$

Recognising the first sum as component k of $\boldsymbol{x}^T \boldsymbol{A}$ and the second as component k of $\boldsymbol{x}^T \boldsymbol{A}^T$,

$$\boxed{\frac{\partial \alpha}{\partial \boldsymbol{x}} = \boldsymbol{x}^T \left(\boldsymbol{A} + \boldsymbol{A}^T \right), \qquad \nabla_{\boldsymbol{x}} \alpha = \left(\boldsymbol{A} + \boldsymbol{A}^T \right) \boldsymbol{x}.} \tag{1.27}$$

If $\boldsymbol{A}$ is symmetric this collapses to

$$\frac{\partial \alpha}{\partial \boldsymbol{x}} = 2 \boldsymbol{x}^T \boldsymbol{A}, \qquad \nabla_{\boldsymbol{x}} \alpha = 2 \boldsymbol{A} \boldsymbol{x}, \tag{1.28}$$

the matrix analogue of $\mathrm{d}(ax^2)/\mathrm{d}x = 2ax$. The symmetric case is the one that occurs in practice, since the matrix sandwiched in a quadratic form can always be replaced by its symmetric part $(\boldsymbol{A} + \boldsymbol{A}^T)/2$ without changing the value of the form. Differentiating once more gives the Hessian,

$$\frac{\partial^2 \alpha}{\partial \boldsymbol{x} \, \partial \boldsymbol{x}^T} = 2 \boldsymbol{A}, \tag{1.29}$$

so that positive definiteness of $\boldsymbol{A}$ and convexity of the quadratic form are the same statement.

Example 4: an inner product of two dependent vectors.

Finally let

$$\alpha = \boldsymbol{y}^T \boldsymbol{x} = \sum_{i=0}^{n-1} y_i x_i, \tag{1.30}$$

where $\boldsymbol{y}$ and $\boldsymbol{x}$ have the same length n and both now *depend* on a third vector $\boldsymbol{z}$. This is the case we shall actually need, because in a cost function the residual depends on the parameters. The product rule gives

$$\frac{\partial \alpha}{\partial z_k} = \sum_{i=0}^{n-1} \left(x_i \frac{\partial y_i}{\partial z_k} + y_i \frac{\partial x_i}{\partial z_k} \right), \qquad \forall k = 0, 1, \ldots, n-1,$$

which in terms of the Jacobians of $\boldsymbol{y}$ and $\boldsymbol{x}$ reads

$$\frac{\partial \alpha}{\partial \boldsymbol{z}} = \boldsymbol{x}^T \frac{\partial \boldsymbol{y}}{\partial \boldsymbol{z}} + \boldsymbol{y}^T \frac{\partial \boldsymbol{x}}{\partial \boldsymbol{z}}. \tag{1.31}$$

When the two vectors coincide, $\boldsymbol{y} = \boldsymbol{x}$, the two terms are equal and

$$\boxed{\frac{\partial}{\partial \boldsymbol{z}} \left(\boldsymbol{x}^T \boldsymbol{x} \right) = 2 \boldsymbol{x}^T \frac{\partial \boldsymbol{x}}{\partial \boldsymbol{z}}.} \tag{1.32}$$

Equation (1.32) is the chain rule for a squared length, and it is all that the next subsection requires.

1.8.4 The mean squared error and its derivative

We can now differentiate the cost function of ordinary least squares without expanding anything. The mean squared error is

$$C(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2 = \frac{1}{n} (\boldsymbol{y} - \tilde{\boldsymbol{y}})^T (\boldsymbol{y} - \tilde{\boldsymbol{y}}), \tag{1.33}$$

or, using the design matrix $\boldsymbol{X}$ of Eq. (1.1) and the linear model $\tilde{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\theta}$,

$$C(\boldsymbol{\theta}) = \frac{1}{n} (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta})^T (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}) = \frac{1}{n} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2. \tag{1.34}$$

The design matrix does not depend on the unknown parameters $\boldsymbol{\theta}$, and we wish to minimise C with respect to them.

Introduce the residual vector

$$\boldsymbol{w} = \boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}, \tag{1.35}$$

which does depend on $\boldsymbol{\theta}$, so that $C(\boldsymbol{\theta}) = \boldsymbol{w}^T \boldsymbol{w}/n$. Equation (1.32) with $\boldsymbol{z} = \boldsymbol{\theta}$ gives immediately

$$\frac{\partial C(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \frac{2}{n} \boldsymbol{w}^T \frac{\partial \boldsymbol{w}}{\partial \boldsymbol{\theta}}, \tag{1.36}$$

and the remaining Jacobian is supplied by Example 1, Eq. (1.21), since $\boldsymbol{y}$ is constant:

$$\frac{\partial \boldsymbol{w}}{\partial \boldsymbol{\theta}} = -\boldsymbol{X}. \tag{1.37}$$

Inserting this we obtain the gradient of the mean squared error,

$$\boxed{\frac{\partial C(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = -\frac{2}{n}\left(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\right)^T \boldsymbol{X}, \qquad \frac{\partial C(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}^T} = -\frac{2}{n}\boldsymbol{X}^T\left(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\right).} \tag{1.38}$$

The two expressions are the same object in the two layouts of Section 1.8.2; the second is the column vector one hands to a gradient-descent routine. Note also that the factor $1/n$ is a matter of taste – some authors write $1/(2n)$ so that the 2 cancels here – and that it disappears entirely when the gradient is set to zero.

Setting Eq. (1.38) to zero gives the *normal equations*

$$\boxed{\boldsymbol{X}^T\boldsymbol{X}\,\boldsymbol{\theta} = \boldsymbol{X}^T\boldsymbol{y}} \tag{1.39}$$

with solution $\hat{\boldsymbol{\theta}} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}$ whenever the inverse exists. Equation (1.39) is the linear system that motivates all of Sections 1.13-1.16, and the qualification "whenever the inverse exists" is what motivates Sections 1.18-1.22. It is worth pausing over how little was needed to obtain it: the Jacobian of a linear map, and the derivative of a squared length.

An alternative route.

The same result follows by expanding the product first,

$$C(\boldsymbol{\theta}) = \frac{1}{n}\left(\boldsymbol{y}^T\boldsymbol{y} - 2\boldsymbol{\theta}^T\boldsymbol{X}^T\boldsymbol{y} + \boldsymbol{\theta}^T\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\theta}\right), \tag{1.40}$$

where the cross terms were combined using $\boldsymbol{y}^T\boldsymbol{X}\boldsymbol{\theta} = \boldsymbol{\theta}^T\boldsymbol{X}^T\boldsymbol{y}$, legitimate because both are scalars and a scalar equals its own transpose. The first term is constant, the second is linear and yields $-2\boldsymbol{y}^T\boldsymbol{X}/n$ by Eq. (1.25), and the third is a quadratic form with the symmetric matrix $\boldsymbol{X}^T\boldsymbol{X}$ and yields $2\boldsymbol{\theta}^T\boldsymbol{X}^T\boldsymbol{X}/n$ by Eq. (1.28). Adding them reproduces Eq. (1.38). The two derivations are worth comparing: the first never expands anything and generalises unchanged to non-linear models, where $\partial \boldsymbol{w}/\partial \boldsymbol{\theta}$ is simply a different Jacobian; the second is more elementary but tied to the linear case.

Adding a penalty.

Ridge regression augments Eq. (1.34) with a penalty on the size of the parameters. Dropping the factor $1/n$ for clarity,

$$C_{\text{ridge}}(\boldsymbol{\theta}) = \left(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\right)^T\left(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\right) + \lambda\,\boldsymbol{\theta}^T\boldsymbol{\theta}, \qquad \lambda \geq 0. \tag{1.41}$$

The penalty is a quadratic form with $\boldsymbol{A} = \boldsymbol{I}$, so Eq. (1.28) contributes $2\lambda\boldsymbol{\theta}^T$ and the stationarity condition becomes

$$\left(\boldsymbol{X}^T\boldsymbol{X} + \lambda\boldsymbol{I}\right)\boldsymbol{\theta} = \boldsymbol{X}^T\boldsymbol{y}, \qquad \hat{\boldsymbol{\theta}}_{\text{ridge}} = \left(\boldsymbol{X}^T\boldsymbol{X} + \lambda\boldsymbol{I}\right)^{-1}\boldsymbol{X}^T\boldsymbol{y}, \tag{1.42}$$

with $\boldsymbol{I}$ the $p \times p$ identity. Had we retained the factor $1/n$ in the first term, λ would appear as $n\lambda$ here; the two conventions differ only in how the penalty parameter is scaled, and one must simply know which is in force. The penalty has added λ to every eigenvalue of $\boldsymbol{X}^T\boldsymbol{X}$, which

makes the matrix positive definite for any $\lambda > 0$ however dependent the columns of $\boldsymbol{X}$ may be. This one line explains most of what Ridge regression does, and Section 1.22 reads it off the singular values.

1.8.5 The Hessian matrix

Differentiating the gradient once more produces the second most important matrix in this book. Applying Eq. (1.21) to the column form of Eq. (1.38),

$$\frac{\partial}{\partial \boldsymbol{\theta}} \frac{\partial C(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}^T} = \frac{\partial}{\partial \boldsymbol{\theta}} \left[-\frac{2}{n} \boldsymbol{X}^T (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}) \right] = \frac{2}{n} \boldsymbol{X}^T \boldsymbol{X}, \tag{1.43}$$

so that, up to the factor $2/n$, the Hessian of the mean squared error is

$$\boxed{\boldsymbol{H} = \boldsymbol{X}^T \boldsymbol{X}.} \tag{1.44}$$

Three observations follow, and each will be taken up later in the book.

First, the Hessian decides the nature of the optimisation problem. A twice differentiable function is convex precisely when its Hessian is positive semi-definite everywhere, and strictly convex when the Hessian is positive definite. By the argument of Section 1.4, $\boldsymbol{z}^T \boldsymbol{X}^T \boldsymbol{X} \boldsymbol{z} = \|\boldsymbol{X}\boldsymbol{z}\|_2^2 \geq 0$ for every $\boldsymbol{z}$, so the least-squares problem is always convex: it has no local minima to be trapped in and no saddle points, and any stationary point is a global minimum. It is *strictly* convex, and the minimum therefore unique, exactly when $\boldsymbol{X}\boldsymbol{z} = \boldsymbol{0}$ implies $\boldsymbol{z} = \boldsymbol{0}$, that is when the columns of $\boldsymbol{X}$ are linearly independent. This is the sense in which linear regression is an easy problem and the neural networks of later chapters, whose Hessians are indefinite, are not.

Second, the Hessian is, aside from the factor $1/n$ and the centring, the covariance matrix of Section 1.10. The matrix that describes the spread of the data and the matrix that describes the curvature of the cost function are the same object, which is why the principal component analysis of Section 1.21 and the conditioning of a regression fit are so closely related.

Third, the inverse Hessian governs the uncertainty of the fitted parameters. We shall show in Chapter 2 that $\mathrm{Var}(\hat{\boldsymbol{\theta}}) = \sigma^2 (\boldsymbol{X}^T \boldsymbol{X})^{-1}$ for independent noise of variance σ^2: a flat direction of the cost function – a small eigenvalue of $\boldsymbol{H}$ – is precisely a direction in which the data leave the parameters poorly determined. The same inverse Hessian appears as the Newton step in optimisation, and the quasi-Newton methods of the chapter on optimisation exist because forming and inverting it is usually too expensive.

Machine learning connection. It is worth noticing what Eq. (1.38) says as an algorithm rather than as an equation. Written as $\nabla C = -(2/n)\boldsymbol{X}^T \boldsymbol{w}$ with $\boldsymbol{w}$ the residual, it states that the gradient is obtained by one matrix-vector product to form the residual and a second, with the transpose, to propagate it back to the parameters. Neither product requires the Hessian $\boldsymbol{X}^T \boldsymbol{X}$ to be formed or stored. This is why gradient descent scales to design matrices for which the normal equations are entirely out of reach, and it is the same observation that makes the iterative methods of Section 1.15 preferable to the direct ones for large problems. The Hessian remains indispensable as an object of *analysis* – it tells us that the problem is convex, how fast gradient descent will converge, and how uncertain the answer is – while being avoided as an object of *computation*.

1.8.6 Derivatives involving the trace

Cost functions built from matrices rather than vectors – in matrix factorisation, in the Gaussian log-likelihood, in the backpropagation equations – are almost always expressed through the trace, because Tr turns a matrix into a scalar without choosing a component. With the shape convention (1.20) the results we need are

$$\frac{\partial}{\partial \boldsymbol{X}} \operatorname{Tr}(\boldsymbol{X}\boldsymbol{A}) = \boldsymbol{A}^T, \tag{1.45}$$

$$\frac{\partial}{\partial \boldsymbol{X}} \operatorname{Tr}\left(\boldsymbol{X}^T\boldsymbol{A}\right) = \boldsymbol{A}, \tag{1.46}$$

$$\frac{\partial}{\partial \boldsymbol{X}} \operatorname{Tr}\left(\boldsymbol{X}^T\boldsymbol{X}\right) = 2\boldsymbol{X}, \tag{1.47}$$

$$\frac{\partial}{\partial \boldsymbol{X}} \operatorname{Tr}\left(\boldsymbol{X}^T\boldsymbol{A}\boldsymbol{X}\right) = \left(\boldsymbol{A}+\boldsymbol{A}^T\right)\boldsymbol{X}. \tag{1.48}$$

Each follows from writing the trace as a double sum and differentiating one entry. For Eq. (1.45), for instance, $\operatorname{Tr}(\boldsymbol{X}\boldsymbol{A}) = \sum_{k,l} x_{kl}a_{lk}$, so $\partial/\partial x_{ij}$ picks out a_{ji}, which is entry (i,j) of $\boldsymbol{A}^T$. The reader will recognise Eq. (1.48) as Example 3, Eq. (1.27), with the vector promoted to a matrix, and Eq. (1.47) as the matrix analogue of $\mathrm{d}(x^2)/\mathrm{d}x = 2x$. Combined with the identity $\|\boldsymbol{X}\|_F^2 = \operatorname{Tr}(\boldsymbol{X}^T\boldsymbol{X})$ of Section 1.12.1, the latter is the single derivative behind every Frobenius-norm minimisation we shall meet. Two further results are needed occasionally, and we record them without proof:

$$\frac{\partial}{\partial \boldsymbol{A}} \ln|\boldsymbol{A}| = \left(\boldsymbol{A}^{-1}\right)^T, \qquad \frac{\partial}{\partial \boldsymbol{A}} \operatorname{Tr}\left(\boldsymbol{A}^{-1}\boldsymbol{B}\right) = -\left(\boldsymbol{A}^{-1}\boldsymbol{B}\boldsymbol{A}^{-1}\right)^T. \tag{1.49}$$

The first is what one differentiates when fitting the covariance matrix of a multivariate Gaussian by maximum likelihood.

1.8.7 The chain rule

Composite models require the chain rule, and with the Jacobian already in hand it is immediate. If $\boldsymbol{u} = \boldsymbol{g}(\boldsymbol{x})$ and $z = f(\boldsymbol{u})$, then in the numerator layout the Jacobians simply compose,

$$\frac{\partial z}{\partial \boldsymbol{x}} = \frac{\partial z}{\partial \boldsymbol{u}}\,\frac{\partial \boldsymbol{u}}{\partial \boldsymbol{x}}, \tag{1.50}$$

a $(1\times k)$ row times a $(k\times n)$ Jacobian giving a $1\times n$ row. Transposing into the denominator layout,

$$\nabla_{\boldsymbol{x}} z = \left(\frac{\partial \boldsymbol{u}}{\partial \boldsymbol{x}}\right)^T \nabla_{\boldsymbol{u}} z. \tag{1.51}$$

Equation (1.50) is the cleaner statement – derivatives multiply, in order – and Eq. (1.51) is the one that is implemented, because it says that a gradient is propagated *backwards* through a composition by multiplying with transposed Jacobians. That is exactly the backpropagation algorithm of the chapter on neural networks, and the reason that algorithm is nothing more than the chain rule with the products arranged in the cheapest order: for a scalar output it is far cheaper to accumulate from the left, row times matrix, than from the right, matrix times matrix.

The derivation of Section 1.8.4 was already an instance. There $\boldsymbol{u} = \boldsymbol{w} = \boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}$ and $z = \boldsymbol{w}^T\boldsymbol{w}/n$, with $\partial z/\partial \boldsymbol{w} = 2\boldsymbol{w}^T/n$ and $\partial \boldsymbol{w}/\partial \boldsymbol{\theta} = -\boldsymbol{X}$; multiplying the two in the order of Eq. (1.50) gives Eq. (1.38) in one step. Replacing the linear model by a network changes $\partial \boldsymbol{w}/\partial \boldsymbol{\theta}$ and nothing else.

1.9 The spectral decomposition of symmetric matrices

The eigenvalue equation

$$\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}, \qquad \boldsymbol{v} \neq \boldsymbol{0}, \tag{1.52}$$

has non-trivial solutions only when $\boldsymbol{A} - \lambda \boldsymbol{I}$ is singular, that is when

$$\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = 0, \tag{1.53}$$

the characteristic polynomial. For a general matrix the eigenvalues may be complex and the eigenvectors need not be independent. For a real *symmetric* matrix – and every matrix whose spectrum we shall actually want in this book is symmetric – the situation is as favourable as it could possibly be:

1. all eigenvalues λ_i are *real*;
2. eigenvectors belonging to distinct eigenvalues are *orthogonal*;
3. there exists a complete *orthonormal* set of n eigenvectors, whether or not the eigenvalues are degenerate.

The proofs of the first two are short and worth seeing. Let $\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}$ with $\boldsymbol{v}$ normalised. Then $\boldsymbol{v}^\dagger \boldsymbol{A}\boldsymbol{v} = \lambda$, while taking the conjugate transpose of the same scalar and using $\boldsymbol{A}^\dagger = \boldsymbol{A}$ gives $\boldsymbol{v}^\dagger \boldsymbol{A}\boldsymbol{v} = \lambda^*$; hence $\lambda = \lambda^*$ is real. For the second, let $\boldsymbol{A}\boldsymbol{v}_1 = \lambda_1 \boldsymbol{v}_1$ and $\boldsymbol{A}\boldsymbol{v}_2 = \lambda_2 \boldsymbol{v}_2$ with $\lambda_1 \neq \lambda_2$. Then

$$\lambda_1 \boldsymbol{v}_2^T \boldsymbol{v}_1 = \boldsymbol{v}_2^T \boldsymbol{A}\boldsymbol{v}_1 = (\boldsymbol{A}\boldsymbol{v}_2)^T \boldsymbol{v}_1 = \lambda_2 \boldsymbol{v}_2^T \boldsymbol{v}_1,$$

so $(\lambda_1 - \lambda_2)\boldsymbol{v}_2^T \boldsymbol{v}_1 = 0$ and the inner product must vanish.

The decomposition.

Collect the orthonormal eigenvectors as the columns of an orthogonal matrix $\boldsymbol{Q} = [\boldsymbol{v}_0\, \boldsymbol{v}_1\, \cdots\, \boldsymbol{v}_{n-1}]$ and the eigenvalues in the diagonal matrix $\boldsymbol{\Lambda} = \mathrm{diag}(\lambda_0, \ldots, \lambda_{n-1})$. The n eigenvalue equations written side by side are $\boldsymbol{A}\boldsymbol{Q} = \boldsymbol{Q}\boldsymbol{\Lambda}$, and multiplying from the right by $\boldsymbol{Q}^T$ gives the *spectral decomposition*

$$\boxed{\boldsymbol{A} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^T = \sum_{i=0}^{n-1} \lambda_i \boldsymbol{v}_i \boldsymbol{v}_i^T = \sum_{i=0}^{n-1} \lambda_i \boldsymbol{P}_i} \tag{1.54}$$

with $\boldsymbol{P}_i = \boldsymbol{v}_i \boldsymbol{v}_i^T$ the rank-one projector onto the ith eigendirection. A symmetric matrix is a weighted sum of orthogonal projectors, the weights being the eigenvalues. Setting all $\lambda_i = 1$ recovers the completeness relation (1.6), and the equivalent statement $\boldsymbol{Q}^T \boldsymbol{A}\boldsymbol{Q} = \boldsymbol{\Lambda}$ says that a symmetric matrix becomes diagonal in its own eigenbasis: in the right coordinate system, the matrix does nothing but stretch each axis by a factor.

Two identities follow immediately from Eq. (1.54) and the cyclic property of the trace,

$$\mathrm{Tr}(\boldsymbol{A}) = \sum_i \lambda_i, \qquad |\boldsymbol{A}| = \prod_i \lambda_i. \tag{1.55}$$

Functions of a symmetric matrix are defined through the same decomposition: for any scalar function f,

$$f(\boldsymbol{A}) = \boldsymbol{Q}\, f(\boldsymbol{\Lambda})\, \boldsymbol{Q}^T = \sum_i f(\lambda_i)\, \boldsymbol{v}_i \boldsymbol{v}_i^T, \tag{1.56}$$

so that $\boldsymbol{A}^{-1}$ has eigenvalues $1/\lambda_i$ and $\boldsymbol{A}^{1/2}$ has eigenvalues $\sqrt{\lambda_i}$, the latter being real only when $\boldsymbol{A}$ is positive semi-definite. The whitening transformation of Section 1.10 is Eq. (1.56) with $f(\lambda) = \lambda^{-1/2}$.

Machine learning connection. The spectral decomposition is the mathematical content of principal component analysis. Applied to the covariance matrix of a data set, the eigenvectors $\boldsymbol{v}_i$ are the principal directions and the eigenvalues λ_i are the variances along them, so that Eq. (1.54) decomposes the total variability of the data into orthogonal contributions ordered by importance. Because $\mathrm{Tr}(\boldsymbol{A}) = \sum_i \lambda_i$, the fraction of the variance explained by the first k components is simply $\sum_{i<k} \lambda_i / \sum_i \lambda_i$ – the quantity plotted in every scree plot. We develop this properly in Section 1.21, having first obtained it more stably from the singular value decomposition.

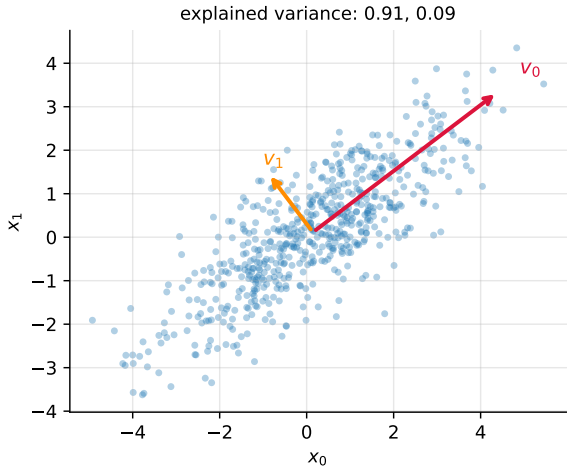


Fig. 1.1 Six hundred samples from a Gaussian with covariance $\boldsymbol{\Sigma} = \left(\begin{smallmatrix} 3 & 2 \\ 2 & 2 \end{smallmatrix}\right)$, with the two eigenvectors of the sample covariance matrix drawn as arrows of length proportional to $\sqrt{\lambda_i}$. The principal directions are orthogonal because $\boldsymbol{\Sigma}$ is symmetric, Section 1.9.

1.10 The covariance matrix

The single most important symmetric matrix in this book is built from the data themselves. Given n observations of p features collected in the design matrix $\boldsymbol{X}$, write $\bar{x}_j$ for the sample mean of column j. The sample covariance between features j and k is

$$\sigma_{jk} = \frac{1}{n-1} \sum_{i=0}^{n-1} (x_{ij} - \bar{x}_j)\,(x_{ik} - \bar{x}_k), \tag{1.57}$$

and these numbers assembled into a $p \times p$ array form the covariance matrix $\boldsymbol{\Sigma}$. For three features it reads

$$\boldsymbol{\Sigma} = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} & \sigma_{xz} \\ \sigma_{yx} & \sigma_{yy} & \sigma_{yz} \\ \sigma_{zx} & \sigma_{zy} & \sigma_{zz} \end{bmatrix},$$

with the variances on the diagonal and the covariances off it. The divisor $n-1$ rather than n makes the estimate unbiased when the means have themselves been estimated from the same data; it is what `numpy.cov` uses, and the distinction matters only for small n. A fuller discussion belongs to Chapter 2; here we are interested in $\boldsymbol{\Sigma}$ as a matrix.

Covariance as a matrix product.

Let $\boldsymbol{X}_c$ denote the *centred* design matrix, obtained by subtracting from each column its mean. Then Eq. (1.57) is one entry of a matrix product and

$$\boldsymbol{\Sigma} = \frac{1}{n-1} \boldsymbol{X}_c^T \boldsymbol{X}_c. \tag{1.58}$$

This compact form tells us everything we need. The matrix is symmetric, since $(\boldsymbol{X}_c^T \boldsymbol{X}_c)^T = \boldsymbol{X}_c^T \boldsymbol{X}_c$; it is positive semi-definite, since $\boldsymbol{z}^T \boldsymbol{X}_c^T \boldsymbol{X}_c \boldsymbol{z} = \|\boldsymbol{X}_c \boldsymbol{z}\|_2^2 \geq 0$ for every $\boldsymbol{z}$; and by Section 1.9 it therefore has real non-negative eigenvalues and an orthonormal eigenbasis. It is singular exactly when some linear combination of the centred features vanishes identically, which is to say when the features are exactly collinear, and it is nearly singular when they are nearly collinear. Note also that $\boldsymbol{\Sigma}$ differs from $\boldsymbol{X}^T \boldsymbol{X}$ of the normal equations (1.39) only by the centring and the factor $1/(n-1)$: the matrix that governs the accuracy of a regression fit and the matrix that describes the spread of the data are, up to bookkeeping, the same object.

The companion matrix $\boldsymbol{X}_c \boldsymbol{X}_c^T$, of size $n \times n$, contains the inner products between *observations* rather than between features. It is the *Gram matrix*, and when the inner product is replaced by a kernel function it becomes the kernel matrix on which support vector machines and Gaussian processes are built. Sections 1.18 and 1.21 will show that these two matrices carry exactly the same spectral information, which is why one may always work with whichever of p and n is smaller.

Correlation and whitening.

Dividing each covariance by the two corresponding standard deviations gives the correlation matrix, $\rho_{jk} = \sigma_{jk} / \sqrt{\sigma_{jj}\sigma_{kk}}$, with unit diagonal and entries in $[-1, 1]$. Correlation is covariance made scale-free, and it is what one should inspect when features are measured in different units. Going further, the transformation

$$\boldsymbol{Z} = \boldsymbol{X}_c \boldsymbol{\Sigma}^{-1/2} \tag{1.59}$$

produces data whose covariance matrix is the identity: uncorrelated features of unit variance. This is *whitening*, and by Eq. (1.56) it is computed from the spectral decomposition of $\boldsymbol{\Sigma}$. It is also where near-collinearity takes its revenge, since $\boldsymbol{\Sigma}^{-1/2}$ divides by $\sqrt{\lambda_i}$ and a tiny eigenvalue amplifies whatever noise happens to lie along that direction. Section 1.19 explains what to do instead.

The following code constructs three correlated variables and extracts the spectrum of their covariance matrix.

```
import numpy as np

n = 100
```

```
x = np.random.normal(size=n)
y = 4.0 + 3.0 * x + np.random.normal(size=n)      # strongly correlated with x
z = x**3 + np.random.normal(size=n)

# np.cov expects variables in rows, so stack the three vectors vertically
W = np.vstack((x, y, z))
Sigma = np.cov(W)                                  # 3 x 3 covariance matrix
print(Sigma)

eigvals, eigvecs = np.linalg.eigh(Sigma)           # eigh: symmetric matrices
print(eigvals)
print(eigvals / np.sum(eigvals))                   # fraction of variance each
```

Note the use of `np.linalg.eigh` rather than `np.linalg.eig`. The former exploits symmetry, returns real eigenvalues in ascending order and guarantees orthonormal eigenvectors; the latter treats the matrix as general, does none of these things, and is both slower and less accurate. Whenever a matrix is known to be symmetric, say so.

Machine learning connection. The eigenvalues printed by the code above are the variances along the principal directions, and their ratios are what a scree plot displays. When one eigenvalue is very much smaller than the others, the data lie close to a lower-dimensional subspace: there are fewer genuinely independent features than columns. That single observation is the common root of dimensionality reduction, of multicollinearity in regression, and of the regularisation methods introduced to control it.

1.11 From algebra to computation

Everything said so far is exact algebra. The moment we hand a matrix to a computer, however, two new questions arise. The first is one of *accuracy*: floating-point numbers carry a finite number of digits, and we need a way of measuring how large the resulting error is. The second is one of *cost*: a design matrix may have 10^6 rows, or a kernel matrix 10^5 rows and as many columns, and an algorithm whose work grows as n^3 is then simply not available to us.

These two questions are not academic in machine learning; they are the subject. A regression problem whose features are nearly collinear will produce parameter estimates that swing wildly under a perturbation of the data too small to see, and no amount of statistical sophistication will repair that – it is a property of the matrix. A method that requires the factorisation of an $n \times n$ matrix is unusable on a data set with a million observations, however elegant its derivation.

The remainder of this chapter is devoted to these two questions. We first introduce the norms with which errors are measured and the condition number that quantifies sensitivity, then develop the direct (elimination) methods for linear systems, their iterative alternatives, and the algorithms for the eigenvalue problem. We close with the singular value decomposition, which answers simultaneously the questions of rank, of least squares, of compression and of principal components. All programs are written in Python; the complete, runnable versions are collected in the `BookPrograms` directory.

Throughout we keep the convention of Section 1.1 that indices run from 0 to $n-1$, so that the mathematical formulae and the Python code carry the same indices.

1.12 Vector and matrix norms

A class of vector norms is given by the so-called p-norms

$$\|\boldsymbol{x}\|_p = \left(|x_0|^p + |x_1|^p + \cdots + |x_{n-1}|^p\right)^{1/p}, \qquad p \geq 1. \tag{1.60}$$

The three cases we shall need are $p = 1$, $p = 2$ and the limit $p \to \infty$,

$$\|\boldsymbol{x}\|_1 = |x_0| + |x_1| + \cdots + |x_{n-1}|, \tag{1.61}$$

$$\|\boldsymbol{x}\|_2 = \left(|x_0|^2 + \cdots + |x_{n-1}|^2\right)^{1/2} = \left(\boldsymbol{x}^T\boldsymbol{x}\right)^{1/2}, \tag{1.62}$$

$$\|\boldsymbol{x}\|_\infty = \max_{0 \leq i \leq n-1} |x_i|. \tag{1.63}$$

The 2-norm is the one we have been using implicitly: it is the square root of the inner product of Section 1.2, and it is the ordinary Euclidean length.

These are not merely three ways of measuring the same thing. Applied to the residual $\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}$, the choice of norm *is* the choice of loss function: minimising the 2-norm gives ordinary least squares, which is the maximum-likelihood estimate under Gaussian noise and which weights large residuals heavily; minimising the 1-norm gives least absolute deviations, which is far less sensitive to outliers; minimising the ∞-norm gives the minimax fit, which cares only about the single worst point. Applied instead to the parameter vector $\boldsymbol{\theta}$ as a penalty, the same three norms give Ridge regression ($\|\boldsymbol{\theta}\|_2^2$), the Lasso ($\|\boldsymbol{\theta}\|_1$) and a rarely used minimax regularisation. The difference between Ridge and the Lasso – that the Lasso sets coefficients exactly to zero and Ridge merely shrinks them – is entirely a statement about the geometry of the unit balls of the two norms, as we show in Chapter 3.

Two inequalities follow from the definitions and will be used repeatedly. The first is the Cauchy-Schwarz inequality, valid for any two vectors in an inner product space,

$$\left|\boldsymbol{x}^T\boldsymbol{y}\right| \leq \|\boldsymbol{x}\|_2 \, \|\boldsymbol{y}\|_2, \tag{1.64}$$

with equality only when $\boldsymbol{x}$ and $\boldsymbol{y}$ are linearly dependent. Divided through by the two norms it states that a correlation coefficient can never exceed unity in magnitude. The second is the triangle inequality

$$\|\boldsymbol{x} + \boldsymbol{y}\|_2 \leq \|\boldsymbol{x}\|_2 + \|\boldsymbol{y}\|_2, \tag{1.65}$$

which follows from Eq. (1.64). Proofs may be found in Golub and Van Loan [11].

For matrices the most frequently used norms are the Frobenius norm

$$\|\boldsymbol{A}\|_F = \sqrt{\sum_i \sum_j |a_{ij}|^2}, \tag{1.66}$$

which is simply the 2-norm of the matrix read as a long vector, and the induced or operator p-norms

$$\|\boldsymbol{A}\|_p = \max_{\boldsymbol{x} \neq 0} \frac{\|\boldsymbol{A}\boldsymbol{x}\|_p}{\|\boldsymbol{x}\|_p}. \tag{1.67}$$

The induced norm measures the largest factor by which $\boldsymbol{A}$ can stretch a vector. For a symmetric matrix $\|\boldsymbol{A}\|_2$ is the largest absolute eigenvalue, a fact which follows immediately from the spectral decomposition of Section 1.9. An orthogonal matrix has $\|\boldsymbol{Q}\|_2 = 1$; it rotates but never stretches, which is Section 1.7 restated in the language of norms.

1.12.1 The Frobenius norm as a trace

Definition (1.66) is a double sum over entries, which is convenient for a computer and inconvenient for a proof. The identity that makes the Frobenius norm usable in analysis is

$$\boxed{\|\boldsymbol{A}\|_F^2 = \operatorname{Tr}\left(\boldsymbol{A}^T\boldsymbol{A}\right), \qquad \|\boldsymbol{A}\|_F = \sqrt{\operatorname{Tr}\left(\boldsymbol{A}^T\boldsymbol{A}\right)}} \tag{1.68}$$

for a real $\boldsymbol{A}$, with $\boldsymbol{A}^T$ replaced by $\boldsymbol{A}^\dagger$ in the complex case. We give two proofs. The first is a line of index bookkeeping; the second explains *why* the identity has to hold, and it is the one worth remembering.

First proof: component by component.

The product $\boldsymbol{A}^T\boldsymbol{A}$ has entries

$$\left(\boldsymbol{A}^T\boldsymbol{A}\right)_{jk} = \sum_i a_{ij}a_{ik}, \tag{1.69}$$

so that on the diagonal, where $k = j$,

$$\left(\boldsymbol{A}^T\boldsymbol{A}\right)_{jj} = \sum_i a_{ij}^2,$$

which is the squared Euclidean length of column j of $\boldsymbol{A}$. The trace is the sum of the diagonal entries, so

$$\operatorname{Tr}\left(\boldsymbol{A}^T\boldsymbol{A}\right) = \sum_j \left(\boldsymbol{A}^T\boldsymbol{A}\right)_{jj} = \sum_j \sum_i a_{ij}^2 = \sum_i \sum_j a_{ij}^2,$$

the last step being nothing more than an interchange of two finite sums. The right-hand side is $\|\boldsymbol{A}\|_F^2$ by definition, which proves Eq. (1.68). Read the calculation once more and it says something slightly stronger: the Frobenius norm squared is the sum of the squared column lengths, and by running the same argument on $\boldsymbol{A}\boldsymbol{A}^T$ it is equally the sum of the squared row lengths. Hence $\operatorname{Tr}(\boldsymbol{A}^T\boldsymbol{A}) = \operatorname{Tr}(\boldsymbol{A}\boldsymbol{A}^T)$ even though the two matrices have different dimensions.

The Frobenius inner product.

The second proof begins by noticing that Eq. (1.68) is the norm statement of an inner product. For two matrices $\boldsymbol{A}, \boldsymbol{B}$ of the same shape define

$$\langle \boldsymbol{A}, \boldsymbol{B} \rangle_F = \operatorname{Tr}\left(\boldsymbol{A}^T\boldsymbol{B}\right) = \sum_i \sum_j a_{ij}b_{ij}, \tag{1.70}$$

where the second equality follows by repeating Eq. (1.69) with b_{ik} in place of a_{ik} and taking the trace. This is bilinear, symmetric, and positive definite – $\langle \boldsymbol{A}, \boldsymbol{A} \rangle_F$ is a sum of squares and vanishes only when every entry vanishes – so it is an inner product on the vector space of $n \times p$ matrices, the *Frobenius* or Hilbert-Schmidt inner product. Every inner product induces a norm through $\|\boldsymbol{A}\| = \sqrt{\langle \boldsymbol{A}, \boldsymbol{A} \rangle}$, and for Eq. (1.70) that induced norm is precisely Eq. (1.66). Equation (1.68) is therefore not a coincidence about traces but the statement that the Frobenius norm is the norm belonging to the trace inner product.

Second proof: vectorisation.

The same point can be made constructively. Let $\mathrm{vec}(\boldsymbol{A})$ stack the columns of a matrix into one long vector,

$$\mathrm{vec}(\boldsymbol{A}) = \left(a_{00}\ a_{10}\ \ldots\ a_{n-1,0}\ a_{01}\ \ldots\ a_{n-1,p-1}\right)^T. \tag{1.71}$$

The particular ordering is irrelevant for what follows; all that matters is that every entry appears exactly once. Then

$$\|\mathrm{vec}(\boldsymbol{A})\|_2^2 = \sum_{i,j} a_{ij}^2 = \|\boldsymbol{A}\|_F^2,$$

by inspection, while for any two matrices of the same shape

$$\mathrm{vec}(\boldsymbol{A})^T \mathrm{vec}(\boldsymbol{B}) = \sum_{i,j} a_{ij} b_{ij} = \mathrm{Tr}\left(\boldsymbol{A}^T \boldsymbol{B}\right), \tag{1.72}$$

using Eq. (1.70). Setting $\boldsymbol{B} = \boldsymbol{A}$ gives Eq. (1.68) again. Geometrically, vectorisation is an isometry between matrix space equipped with the Frobenius norm and $\mathbb{R}^{np}$ equipped with the ordinary Euclidean norm,

$$\|\boldsymbol{A}\|_F = \|\mathrm{vec}(\boldsymbol{A})\|_2, \tag{1.73}$$

so $\mathrm{Tr}(\boldsymbol{A}^T\boldsymbol{A})$ is simply the squared length of the matrix regarded as a point in np dimensions. Matrix space is a Euclidean space, and the Frobenius norm is what makes it one. Vectorisation is not only a proof device: it is what a deep learning framework does when it flattens the weights of a network into a single parameter vector for the optimiser.

Two consequences we shall use.

First, the Frobenius norm is invariant under orthogonal transformations. If $\boldsymbol{U}$ and $\boldsymbol{V}$ are orthogonal of the appropriate sizes, then by the cyclic property of the trace

$$\|\boldsymbol{U}\boldsymbol{A}\boldsymbol{V}^T\|_F^2 = \mathrm{Tr}\left(\boldsymbol{V}\boldsymbol{A}^T\boldsymbol{U}^T\boldsymbol{U}\boldsymbol{A}\boldsymbol{V}^T\right) = \mathrm{Tr}\left(\boldsymbol{A}^T\boldsymbol{A}\right) = \|\boldsymbol{A}\|_F^2.$$

Choosing $\boldsymbol{U}$ and $\boldsymbol{V}$ to be the singular vectors of Section 1.18 turns $\boldsymbol{A}$ into its diagonal matrix of singular values, so

$$\|\boldsymbol{A}\|_F^2 = \sum_k \sigma_k^2, \tag{1.74}$$

the Frobenius norm is the 2-norm of the vector of singular values, and the Eckart-Young theorem of Section 1.20 – that truncating the SVD gives the best low-rank approximation in the Frobenius norm – becomes a statement about discarding the smallest entries of that vector.

Second, because the Frobenius norm comes from an inner product, minimising it subject to linear constraints is an ordinary least-squares problem. This is what turns matrix factorisation into a tractable optimisation: the recommender-system objective $\|\boldsymbol{R} - \boldsymbol{W}\boldsymbol{H}\|_F^2$, the non-negative matrix factorisation objective, and the secant-condition derivation of the BFGS update in the chapter on optimisation are all Frobenius-norm minimisations. The trace identity (1.68) is what makes the derivative $\partial\,\mathrm{Tr}(\boldsymbol{X}^T\boldsymbol{X})/\partial\boldsymbol{X} = 2\boldsymbol{X}$ of Eq. (1.47) available, and that single derivative is the whole of the derivation.

Relative error.

Let $fl(\boldsymbol{x})$ denote the machine representation of a vector $\boldsymbol{x} \neq 0$. The relative error is

$$\varepsilon = \frac{\|fl(\boldsymbol{x}) - \boldsymbol{x}\|}{\|\boldsymbol{x}\|}, \tag{1.75}$$

and if we use the ∞-norm the statement $\|fl(\boldsymbol{x}) - \boldsymbol{x}\|_\infty / \|\boldsymbol{x}\|_\infty \approx 10^{-l}$ says that the largest component of $fl(\boldsymbol{x})$ carries roughly l correct significant digits. In IEEE double precision the machine epsilon is $\varepsilon \approx 2.2 \times 10^{-16}$, so about sixteen significant decimal digits are available before any computation has been performed.

The condition number.

Suppose we solve $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ but, because of rounding, actually solve a slightly perturbed problem with right-hand side $\boldsymbol{b} + \delta\boldsymbol{b}$. The resulting error $\delta\boldsymbol{x}$ satisfies $\boldsymbol{A}\,\delta\boldsymbol{x} = \delta\boldsymbol{b}$, hence $\|\delta\boldsymbol{x}\| \le \|\boldsymbol{A}^{-1}\|\,\|\delta\boldsymbol{b}\|$, while $\|\boldsymbol{b}\| \le \|\boldsymbol{A}\|\,\|\boldsymbol{x}\|$. Combining the two gives

$$\frac{\|\delta\boldsymbol{x}\|}{\|\boldsymbol{x}\|} \le \kappa(\boldsymbol{A})\,\frac{\|\delta\boldsymbol{b}\|}{\|\boldsymbol{b}\|}, \qquad \kappa(\boldsymbol{A}) = \|\boldsymbol{A}\|\,\|\boldsymbol{A}^{-1}\|. \tag{1.76}$$

The quantity $\kappa(\boldsymbol{A})$ is the *condition number*. It tells us how much an input error can be amplified, and it is a property of the problem, not of the algorithm: no amount of cleverness can rescue an ill-conditioned system. In the 2-norm and for a symmetric matrix

$$\kappa_2(\boldsymbol{A}) = \frac{\max_i |\lambda_i|}{\min_i |\lambda_i|}, \tag{1.77}$$

so a matrix is ill-conditioned exactly when it has eigenvalues of very different magnitude, that is, when it is close to being singular. As a rule of thumb, if $\kappa \approx 10^{k}$ one loses about k significant digits.

Figure 1.2 makes the point concrete for the polynomial design matrix we shall fit in Chapter 3. The condition number of $\boldsymbol{X}$ itself grows exponentially with the degree, and that of $\boldsymbol{X}^{T}\boldsymbol{X}$ grows as its square, so that by degree eight the normal equations already have a condition number exceeding $1/\varepsilon_{\text{mach}}$ – at which point double precision retains no correct digits at all. Nothing is wrong with the mathematics; the difficulty is entirely one of representation, and it is the reason the least-squares routines of Chapter 3 avoid forming the product.

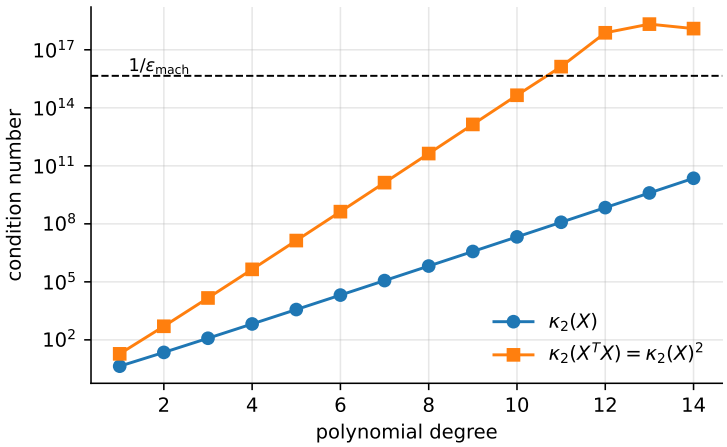


Fig. 1.2 Condition number of the Vandermonde matrix of a polynomial fit to 100 points on $[0,1]$, against the polynomial degree. Squaring the matrix squares the condition number, Eq. (1.118); the dashed line marks the reciprocal of the machine epsilon, beyond which no significant digits survive.

Machine learning connection. Ill-conditioning is the numerical name for multicollinearity. If two features are strongly correlated, the matrix $\boldsymbol{X}^T\boldsymbol{X}$ of the normal equations (1.39) has a small eigenvalue, its condition number is large, and the fitted coefficients become enormous and of opposite sign – the model compensates one feature against the other, and a change in the data too small to notice moves the coefficients by a great deal. Two remarks follow. First, the damage is done before any statistics is involved; it is visible in $\kappa_2(\boldsymbol{X})$ alone. Second, Eq. (1.42) shows what the Ridge penalty does about it: adding λ to every eigenvalue changes the condition number from $\lambda_{\max}/\lambda_{\min}$ to $(\lambda_{\max}+\lambda)/(\lambda_{\min}+\lambda)$, which is smaller for every $\lambda > 0$. Regularisation is, among other things, a numerical conditioning device.

1.13 Linear systems and Gaussian elimination

We now turn to the solution of

$$\boldsymbol{A}\boldsymbol{x} = \boldsymbol{w}, \tag{1.78}$$

with $\boldsymbol{A}$ square and non-singular. Before describing the algorithms it is worth recalling why such systems appear at all.

Where linear systems come from.

The example that matters most in this book has already been derived. Setting the gradient (1.38) of the least-squares cost to zero produced the normal equations

$$\boldsymbol{X}^T\boldsymbol{X}\,\boldsymbol{\theta} = \boldsymbol{X}^T\boldsymbol{y},$$

which is Eq. (1.78) with $\boldsymbol{A} = \boldsymbol{X}^T\boldsymbol{X}$ of size $p \times p$ and $\boldsymbol{w} = \boldsymbol{X}^T\boldsymbol{y}$. Fitting a linear model is solving a linear system, and the properties of that system – its size, its symmetry, its conditioning – decide which of the algorithms below is appropriate. The same structure recurs throughout: the interpolation coefficients of a spline, the dual coefficients of a kernel ridge regression, the Newton step of an optimisation method and the update of a Gaussian process posterior are all solutions of a linear system with a symmetric positive definite matrix.

The elimination.

The idea of Gaussian elimination is to use the first equation to eliminate x_0 from the remaining $n-1$ equations, then the new second equation to eliminate x_1 from the remaining $n-2$, and so on. After $n-1$ such steps we are left with an upper triangular system

$$\boldsymbol{U}\boldsymbol{x} = \boldsymbol{y}, \tag{1.79}$$

which is solved from the bottom upwards by *backward substitution*

$$x_m = \frac{1}{u_{mm}}\left(y_m - \sum_{k=m+1}^{n-1} u_{mk}x_k\right), \qquad m = n-1, n-2, \ldots, 0. \tag{1.80}$$

To eliminate x_0 we multiply the first equation by a_{j0}/a_{00} and subtract it from equation j, for $j = 1, \ldots, n-1$. The elements of the remaining $(n-1)\times(n-1)$ block are updated according to

$$a_{jk}^{(1)} = a_{jk} - \frac{a_{j0}\,a_{0k}}{a_{00}}, \qquad j,k = 1,\dots,n-1, \tag{1.81}$$

and the same formula, applied to successively smaller blocks, defines the whole algorithm. Counting operations, the elimination requires $2n^3/3 + \mathcal{O}(n^2)$ floating point operations, while the backward substitution costs only $\mathcal{O}(n^2)$. The elimination dominates, and this cubic scaling is the reason direct methods become unusable for the large problems of Section 1.15.

Pivoting.

The derivation assumed $a_{00} \neq 0$, and more generally that no pivot element vanishes. Even a small pivot is dangerous, since dividing by it amplifies rounding errors. Suppose the first division produces -10^{-7} where the original entry was of order one: we are then adding $10^7 + 1$, and in single precision the result is 10^7. The information carried by the small entry has been destroyed. The cure is *partial pivoting*: before eliminating column k we search the column for the element of largest absolute value and interchange rows so that this element becomes the pivot. Row interchanges do not alter the solution, and they keep all multipliers bounded by unity. In practice one always pivots.

The following class implements the algorithm.

```
class GaussianElimination(DirectSolver):
    """Gaussian elimination with partial pivoting."""

    def solve(self, b):
        n = self.n
        M = self.A.copy()                # the elimination destroys its input
        y = np.asarray(b, dtype=float).copy()

        for k in range(n - 1):           # forward elimination
            # partial pivoting: use the largest element in the column
            p = k + np.argmax(np.abs(M[k:, k]))
            if abs(M[p, k]) < 1.0e-14:
                raise np.linalg.LinAlgError("matrix is singular")
            if p != k:
                M[[k, p]] = M[[p, k]]
                y[k], y[p] = y[p], y[k]
            for i in range(k + 1, n):
                factor = M[i, k] / M[k, k]
                M[i, k:] -= factor * M[k, k:]
                y[i] -= factor * y[k]

        return self._back_substitute(M, y)

    @staticmethod
    def _back_substitute(U, y):
        """Solve U x = y for an upper triangular U."""
        n = U.shape[0]
        x = np.zeros(n)
        for m in range(n - 1, -1, -1):
            x[m] = (y[m] - U[m, m+1:] @ x[m+1:]) / U[m, m]
        return x
```

1.14 LU and Cholesky decompositions

Gaussian elimination as written above throws away the multipliers once it has used them. If we keep them instead, we obtain a factorisation of $\boldsymbol{A}$ itself, and this is what makes the method reusable. The LU decomposition writes

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}, \tag{1.82}$$

with $\boldsymbol{L}$ unit lower triangular and $\boldsymbol{U}$ upper triangular; for a 4×4 matrix

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ l_{10} & 1 & 0 & 0 \\ l_{20} & l_{21} & 1 & 0 \\ l_{30} & l_{31} & l_{32} & 1 \end{bmatrix} \begin{bmatrix} u_{00} & u_{01} & u_{02} & u_{03} \\ 0 & u_{11} & u_{12} & u_{13} \\ 0 & 0 & u_{22} & u_{23} \\ 0 & 0 & 0 & u_{33} \end{bmatrix}. \tag{1.83}$$

The factorisation exists whenever $\boldsymbol{A}$ is non-singular and, with the normalisation $l_{ii}=1$, it is unique. Multiplying out the first column gives $a_{00}=u_{00}$ and $a_{j0}=l_{j0}u_{00}$, which determines u_{00} and the multipliers l_{j0}. The second column then gives u_{01}, u_{11} and l_{j1}, and so on: at every stage the unknowns are determined by quantities already computed. In practice $\boldsymbol{L}$ and $\boldsymbol{U}$ are stored in the same array as $\boldsymbol{A}$, since the unit diagonal of $\boldsymbol{L}$ need not be kept.

With partial pivoting the factorisation reads $\boldsymbol{P}\boldsymbol{A}=\boldsymbol{L}\boldsymbol{U}$ with $\boldsymbol{P}$ a permutation matrix, which in the program is recorded as a permutation array rather than as a matrix.

The determinant.

Because $\det(\boldsymbol{L})=1$ and the determinant of a triangular matrix is the product of its diagonal elements,

$$|\boldsymbol{A}| = \pm\, u_{00}u_{11}\cdots u_{n-1,n-1}, \tag{1.84}$$

where the sign is $(-1)^{r}$ with r the number of row interchanges. This is the only sensible way of computing a determinant numerically; expanding in minors would cost $\mathcal{O}(n!)$ operations.

Solving the system.

Writing $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{L}\boldsymbol{U}\boldsymbol{x}=\boldsymbol{w}$ and introducing the intermediate vector $\boldsymbol{y}=\boldsymbol{U}\boldsymbol{x}$, the problem splits into two triangular systems,

$$\boldsymbol{L}\boldsymbol{y}=\boldsymbol{P}\boldsymbol{w} \quad \text{(forward substitution)}, \qquad \boldsymbol{U}\boldsymbol{x}=\boldsymbol{y} \quad \text{(backward substitution)}, \tag{1.85}$$

each costing only $\mathcal{O}(n^2)$ operations. This is the decisive advantage of LU over plain elimination: the expensive $\mathcal{O}(n^3)$ factorisation is performed once, after which any number of right-hand sides can be treated cheaply. In a cross-validation loop, or when refitting a model with several different targets on the same features, this is the difference between one factorisation and a dozen.

The inverse, and why not to compute it.

Column j of $\boldsymbol{A}^{-1}$ is the solution of $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{e}_j$, where $\boldsymbol{e}_j$ is the jth unit vector. The inverse therefore costs one factorisation plus n substitutions. It is worth stressing that one should almost never compute an inverse: if the aim is to solve a linear system, solving it directly is both faster and

more accurate than forming $\boldsymbol{A}^{-1}$ and multiplying. The formula $\boldsymbol{\theta} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}$ is a statement about $\boldsymbol{\theta}$, not an instruction to a computer; the instruction is `np.linalg.solve` or, better still, `np.linalg.lstsq`.

```
class LUDecomposition(DirectSolver):
    """Doolittle LU factorisation with partial pivoting, P A = L U."""

    def _factorize(self):
        n = self.n
        LU = self.A.copy()
        perm = np.arange(n)
        sign = 1.0

        for k in range(n):
            p = k + np.argmax(np.abs(LU[k:, k]))
            if abs(LU[p, k]) < 1.0e-14:
                raise np.linalg.LinAlgError("matrix is singular")
            if p != k:
                LU[[k, p]] = LU[[p, k]]
                perm[[k, p]] = perm[[p, k]]
                sign = -sign
            # the multipliers l_ik are stored in place, below the diagonal
            LU[k+1:, k] /= LU[k, k]
            # rank-one update of the trailing submatrix
            LU[k+1:, k+1:] -= np.outer(LU[k+1:, k], LU[k, k+1:])

        self.LU, self.perm, self.sign = LU, perm, sign

    def solve(self, b):
        """Solve A x = b in two steps: L y = P b, then U x = y."""
        y = np.asarray(b, dtype=float)[self.perm].copy()
        n = self.n
        for i in range(1, n):                    # forward, L has unit diagonal
            y[i] -= self.LU[i, :i] @ y[:i]
        x = np.zeros(n)
        for i in range(n - 1, -1, -1):           # backward
            x[i] = (y[i] - self.LU[i, i+1:] @ x[i+1:]) / self.LU[i, i]
        return x

    def determinant(self):
        return self.sign * np.prod(np.diag(self.LU))
```

Cholesky's factorisation.

If $\boldsymbol{A}$ is real, symmetric and positive definite it admits the special factorisation

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^T, \tag{1.86}$$

with $\boldsymbol{L}$ lower triangular. The elements follow from

$$L_{ii} = \left(A_{ii} - \sum_{k=0}^{i-1} L_{ik}^2 \right)^{1/2}, \tag{1.87}$$

$$L_{ji} = \frac{1}{L_{ii}} \left(A_{ji} - \sum_{k=0}^{i-1} L_{ik} L_{jk} \right), \qquad j = i+1, \ldots, n-1. \tag{1.88}$$

The algorithm costs half of a general LU factorisation, and since the pivots L_{ii} are guaranteed positive there is no need for pivoting at all. The square root in Eq. (1.87) also provides the

cheapest available test of positive definiteness: if the argument turns negative, the matrix is not positive definite.

Machine learning connection. Cholesky is the workhorse of every method whose central object is a symmetric positive definite matrix, and there are many. The normal equations (1.39) have $\boldsymbol{X}^T\boldsymbol{X}$ on the left, and their regularised form (1.42) guarantees positive definiteness for any $\lambda > 0$; Gaussian process regression requires the solution of $(\boldsymbol{K}+\sigma^2\boldsymbol{I})\boldsymbol{\alpha} = \boldsymbol{y}$ with $\boldsymbol{K}$ a kernel matrix, and its log-marginal-likelihood needs $\ln|\boldsymbol{K}+\boldsymbol{\sigma^2}\boldsymbol{I}|$, which Eq. (1.84) reads off the Cholesky factor as $2\sum_i \ln L_{ii}$; and sampling from a multivariate Gaussian with covariance $\boldsymbol{\Sigma}$ is done by drawing $\boldsymbol{z} \sim N(\boldsymbol{0},\boldsymbol{I})$ and forming $\boldsymbol{L}\boldsymbol{z}$. When a Cholesky factorisation fails in a Gaussian process code, the message is not that the linear algebra is broken but that the kernel matrix has lost positive definiteness to rounding, and the standard remedy – adding a small multiple of the identity, the "jitter" – is Eq. (1.42) under another name.

1.15 Iterative methods for linear systems

The methods of the previous sections are *direct*: up to rounding they produce the exact answer in a finite, known number of operations. Their cost, however, grows as n^3, and they require the matrix to be stored. Iterative methods take the opposite view. We start from a guess and improve it until the change falls below a tolerance. The matrix enters only through the product $\boldsymbol{A}\boldsymbol{x}$, so a matrix that is sparse, or that is never formed at all but only applied, costs nothing extra. This is the reason iterative methods, and not the direct ones, are what one uses for large-scale problems.

We split the matrix as

$$\boldsymbol{A} = \boldsymbol{D} + \boldsymbol{L} + \boldsymbol{U}, \tag{1.89}$$

with $\boldsymbol{D}$ diagonal, $\boldsymbol{L}$ strictly lower and $\boldsymbol{U}$ strictly upper triangular.

Jacobi's method.

Solving row i for x_i and evaluating the right-hand side with the previous iterate gives

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j\neq i} a_{ij}x_j^{(k)}\right), \tag{1.90}$$

or, in matrix form,

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{D}^{-1}\left(\boldsymbol{b} - (\boldsymbol{L}+\boldsymbol{U})\boldsymbol{x}^{(k)}\right). \tag{1.91}$$

Every component of the new iterate is computed from the old ones only, so all n updates are independent and the method parallelises trivially. It converges whenever $\boldsymbol{A}$ is strictly diagonally dominant or symmetric positive definite.

Gauss-Seidel.

Nothing forces us to wait: the components $x_0^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ are already available when we compute $x_i^{(k+1)}$. Using them immediately gives the Gauss-Seidel iteration

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right). \tag{1.92}$$

This is forward substitution applied to the splitting, and it typically halves the number of iterations. The price is that the sweep is now inherently sequential. Readers who have met coordinate descent for the Lasso will recognise the pattern: update one coordinate at a time, using the most recent values of all the others.

Successive over-relaxation.

Gauss-Seidel usually moves in the right direction but not far enough. Over-relaxation exaggerates the step by a factor ω,

$$x_i^{(k+1)} = (1-\omega) x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right), \tag{1.93}$$

which in matrix form reads

$$\boldsymbol{x}^{(k+1)} = (\boldsymbol{D} + \omega \boldsymbol{L})^{-1} \left(\omega \boldsymbol{b} - [\omega \boldsymbol{U} + (\omega - 1)\boldsymbol{D}] \boldsymbol{x}^{(k)} \right). \tag{1.94}$$

For symmetric positive definite matrices convergence is guaranteed for $0 < \omega < 2$, and $\omega = 1$ returns Gauss-Seidel. A well chosen ω can reduce the iteration count by an order of magnitude, but the optimal value depends on the spectrum of $\boldsymbol{A}$ and is rarely known in advance – an early instance of the hyperparameter problem that pervades this book.

1.16 The conjugate gradient method

The methods above treat the linear system as a fixed-point problem. The conjugate gradient method instead treats it as a minimisation, which puts it in the same family as the optimisation algorithms of the chapter on optimisation. For a symmetric positive definite $\boldsymbol{A}$ the solution of $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ is the unique minimum of the quadratic form

$$P(\boldsymbol{x}) = \tfrac{1}{2} \boldsymbol{x}^T \boldsymbol{A} \boldsymbol{x} - \boldsymbol{x}^T \boldsymbol{b}, \tag{1.95}$$

since by Eqs. (1.25) and (1.27) $\nabla P = \boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}$, which vanishes precisely at the solution. Taking $\boldsymbol{A} = \boldsymbol{X}^T \boldsymbol{X}$ and $\boldsymbol{b} = \boldsymbol{X}^T \boldsymbol{y}$ recovers the least-squares problem, so what follows is a solver for linear regression as much as for a linear system. The residual

$$\boldsymbol{r} = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x} \tag{1.96}$$

is the negative gradient of P, and steepest descent would move along it. Steepest descent is however notoriously slow, because successive steps undo part of the progress of their predecessors; the number of iterations grows with the condition number of $\boldsymbol{A}$, which for the normal equations is the square of that of $\boldsymbol{X}$.

The remedy is to search along directions that are *conjugate*, meaning orthogonal with respect to the inner product defined by $\boldsymbol{A}$,

$$\boldsymbol{p}_i^T \boldsymbol{A} \boldsymbol{p}_j = 0, \qquad i \neq j. \tag{1.97}$$

The eigenvectors of $\boldsymbol{A}$ are an example, since $\boldsymbol{v}_i^T \boldsymbol{A} \boldsymbol{v}_j = \lambda_j \boldsymbol{v}_i^T \boldsymbol{v}_j$ vanishes for $i \neq j$; but the whole point of the method is that we shall construct conjugate directions without knowing any eigenvectors. A set of n mutually conjugate vectors forms a basis, so we may expand

$$\boldsymbol{x} = \sum_{i=0}^{n-1} \alpha_i \boldsymbol{p}_i . \tag{1.98}$$

Multiplying $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ from the left by $\boldsymbol{p}_k^T$ and using Eq. (1.97) collapses the sum to a single term,

$$\alpha_k = \frac{\boldsymbol{p}_k^T \boldsymbol{b}}{\boldsymbol{p}_k^T \boldsymbol{A} \boldsymbol{p}_k}, \tag{1.99}$$

so each direction can be treated independently. In exact arithmetic the method therefore terminates after at most n steps, which makes it a direct method in disguise; what makes it useful is that a good approximation is normally reached in far fewer. The directions are generated on the fly: starting from $\boldsymbol{x}_0 = \boldsymbol{0}$ and $\boldsymbol{p}_0 = \boldsymbol{r}_0 = \boldsymbol{b}$, the new direction is the residual with its component along the previous direction projected out,

$$\boldsymbol{p}_{k+1} = \boldsymbol{r}_{k+1} - \frac{\boldsymbol{p}_k^T \boldsymbol{A} \boldsymbol{r}_{k+1}}{\boldsymbol{p}_k^T \boldsymbol{A} \boldsymbol{p}_k} \boldsymbol{p}_k , \tag{1.100}$$

which is one step of Gram-Schmidt, Eq. (1.11), performed in the inner product defined by $\boldsymbol{A}$. Remarkably, the new direction is automatically conjugate to *all* previous ones, not merely the last, so the recursion needs to store only a handful of vectors.

```python
def conjugate_gradient(A, b, x0=None, tol=1.0e-10, maxiter=None):
    """Solve A x = b for symmetric positive definite A.

    A may be a matrix or any callable implementing the product A @ v,
    which is what makes the method usable when A is never formed.
    """
    matvec = A if callable(A) else (lambda v: A @ v)
    n = b.shape[0]
    x = np.zeros(n) if x0 is None else np.asarray(x0, dtype=float).copy()

    r = b - matvec(x)
    p = r.copy()
    rsold = r @ r

    for _ in range(maxiter or n):
        Ap = matvec(p)
        alpha = rsold / (p @ Ap)
        x += alpha * p
        r -= alpha * Ap
        rsnew = r @ r
        if np.sqrt(rsnew) < tol:
            break
        p = r + (rsnew / rsold) * p        # Fletcher-Reeves update
        rsold = rsnew

    return x
```

Note the signature. The matrix is accepted either as an array or as a function computing $\boldsymbol{A}\boldsymbol{v}$, and for the normal equations that function is `lambda v: X.T @ (X @ v)`, which never forms the $p \times p$ product $\boldsymbol{X}^T\boldsymbol{X}$ at all. For a design matrix with many features this is the difference between a feasible computation and an impossible one, and it is the same matrix-free idea that makes stochastic gradient descent practical.

Machine learning connection. The convergence of conjugate gradient is governed by $\sqrt{\kappa_2(\boldsymbol{A})}$, whereas that of steepest descent is governed by $\kappa_2(\boldsymbol{A})$ itself; the square root is the whole advantage. Since $\kappa_2(\boldsymbol{X}^T\boldsymbol{X}) = \kappa_2(\boldsymbol{X})^2$, applying conjugate gradient to the normal equations restores exactly the conditioning one loses by forming them. Preconditioning – solving $\boldsymbol{M}^{-1}\boldsymbol{A}\boldsymbol{x} = \boldsymbol{M}^{-1}\boldsymbol{b}$ for some cheaply invertible $\boldsymbol{M}$ that approximates $\boldsymbol{A}$ – pushes this further, and the simplest choice, $\boldsymbol{M} = \boldsymbol{D}$, amounts to rescaling each feature by its standard deviation. Feature standardisation, which we introduced in Section 1.5 as a statistical convention, is also a preconditioner.

1.17 The algebraic eigenvalue problem

We return to the eigenvalue problem of Section 1.9, this time asking how it is actually solved. The eigenvalues of a matrix $\boldsymbol{A}$ of dimension n are defined by

$$\boldsymbol{A}\boldsymbol{x}^{(\nu)} = \lambda^{(\nu)}\boldsymbol{x}^{(\nu)}, \tag{1.101}$$

where, unless stated otherwise, eigenvector means right eigenvector. Rewriting Eq. (1.101) as $(\boldsymbol{A} - \lambda^{(\nu)}\boldsymbol{I})\boldsymbol{x}^{(\nu)} = \boldsymbol{0}$, a non-trivial solution exists only if the matrix is singular, that is only if the characteristic polynomial

$$P(\lambda) = \det(\lambda\boldsymbol{I} - \boldsymbol{A}) = \prod_{i=0}^{n-1}(\lambda_i - \lambda) \tag{1.102}$$

vanishes. Its roots form the *spectrum* $\lambda(\boldsymbol{A})$, and the factorisation gives back Eq. (1.55).

It is tempting to conclude that one should find eigenvalues by computing the characteristic polynomial and looking for its roots. This is almost always a bad idea: the roots of a polynomial are extremely sensitive to its coefficients, so the detour through $P(\lambda)$ throws away accuracy that the original matrix still contained. The standard approach is entirely different. We apply a sequence of *similarity transformations* that bring $\boldsymbol{A}$ towards diagonal form.

1.17.1 Similarity transformations

Let $\boldsymbol{A}$ be real and symmetric. Then there exists a real orthogonal matrix $\boldsymbol{S}$ such that

$$\boldsymbol{S}^T\boldsymbol{A}\boldsymbol{S} = \operatorname{diag}(\lambda_0, \lambda_1, \ldots, \lambda_{n-1}) \equiv \boldsymbol{D}, \tag{1.103}$$

and the jth column of $\boldsymbol{S}$ is the eigenvector belonging to λ_j [11]. More generally, $\boldsymbol{B}$ is a similarity transform of $\boldsymbol{A}$ if

$$\boldsymbol{B} = \boldsymbol{S}^T\boldsymbol{A}\boldsymbol{S}, \qquad \boldsymbol{S}^T\boldsymbol{S} = \boldsymbol{S}^{-1}\boldsymbol{S} = \boldsymbol{I}. \tag{1.104}$$

The importance of such a transformation is that it leaves the eigenvalues unchanged while altering the eigenvectors in a controlled way. The proof is one line. Multiply $\boldsymbol{A}\boldsymbol{x} = \lambda\boldsymbol{x}$ from the left by $\boldsymbol{S}^T$ and insert $\boldsymbol{S}\boldsymbol{S}^T = \boldsymbol{I}$ between $\boldsymbol{A}$ and $\boldsymbol{x}$,

$$(\boldsymbol{S}^T\boldsymbol{A}\boldsymbol{S})(\boldsymbol{S}^T\boldsymbol{x}) = \lambda\,(\boldsymbol{S}^T\boldsymbol{x}), \tag{1.105}$$

that is $\boldsymbol{B}(\boldsymbol{S}^T\boldsymbol{x}) = \lambda(\boldsymbol{S}^T\boldsymbol{x})$. So λ is an eigenvalue of $\boldsymbol{B}$ as well, with eigenvector $\boldsymbol{S}^T\boldsymbol{x}$. The strategy is therefore to apply transformations until

$$S_N^T \cdots S_1^T A S_1 \cdots S_N = D. \tag{1.106}$$

1.17.2 The power method

The simplest of all eigenvalue algorithms is worth understanding, because much of what a library routine does can be seen as a refinement of it, and because it reappears in its own right in the analysis of Markov chains and of network centrality. Assume A can be diagonalised, with eigenvalues $\lambda_0, \dots, \lambda_{n-1}$ and eigenvectors $v_0, \dots, v_{n-1}$, and assume there is a dominant eigenvalue, $|\lambda_0| > |\lambda_j|$ for $j > 0$. An arbitrary starting vector expands as

$$b_0 = c_0 v_0 + c_1 v_1 + \cdots + c_{n-1} v_{n-1}, \tag{1.107}$$

and applying A repeatedly gives

$$A^k b_0 = c_0 \lambda_0^k \left(v_0 + \sum_{j>0} \frac{c_j}{c_0} \left(\frac{\lambda_j}{\lambda_0} \right)^k v_j \right). \tag{1.108}$$

Every ratio $|\lambda_j/\lambda_0|$ is smaller than one, so the bracket tends to v_0: repeated multiplication filters out everything except the dominant eigenvector. Normalising at every step, $b_k = A^k b_0 / \|A^k b_0\|$, and estimating the eigenvalue by the Rayleigh quotient

$$\mu_k = \frac{b_k^T A b_k}{b_k^T b_k}, \tag{1.109}$$

we obtain a method that converges geometrically with ratio $|\lambda_1/\lambda_0|$. When the two largest eigenvalues are close, convergence is correspondingly slow.

The power method has two obvious weaknesses: it gives only one eigenpair, and it discards the information contained in the intermediate vectors $b_0, A b_0, \dots, A^{k-1} b_0$. Removing the second weakness by working in the whole space spanned by those vectors, the *Krylov space*, leads to the Lanczos and Arnoldi algorithms, which is how `scipy.sparse.linalg.eigsh` computes a few extreme eigenpairs of a large sparse matrix without ever factorising it. Applying the power method to A^{-1} instead gives inverse iteration, which converges to the eigenvalue of smallest magnitude and, with a shift, to any eigenvalue one can bracket.

Machine learning connection. Equation (1.108) is the PageRank algorithm. The importance vector of a set of web pages is the dominant eigenvector of a stochastic link matrix, and it is computed by precisely the iteration above – multiply, normalise, repeat – on a matrix far too large to factorise. The same iteration, applied to the covariance matrix, gives the first principal component, and applied to $X^T X$ implicitly through two matrix-vector products it gives the first singular vector of X without forming any product at all.

1.17.3 What library routines actually do

The algorithms used in practice reduce the matrix to a simpler form by orthogonal similarity transformations before extracting eigenvalues. For a symmetric matrix, a finite sequence of *Householder reflections*,

$$H = I - 2\frac{u u^T}{u^T u}, \qquad H^T = H = H^{-1}, \tag{1.110}$$

each of which zeroes all but one entry of a column below the diagonal, brings $\boldsymbol{A}$ to tridiagonal form in $\mathcal{O}(n^3)$ operations; an iterative QR or QL sweep with shifts then drives the off-diagonal elements to zero, and converges cubically. This two-stage strategy – a finite orthogonal reduction followed by an iterative sweep on the reduced matrix – is what LAPACK implements and what `numpy.linalg.eigh` calls. Jacobi's method of successive plane rotations, which attacks the full matrix directly, is the historically older alternative and remains attractive on parallel hardware.

We shall use the library routines throughout. It is worth knowing what they do, and worth knowing that they cost $\mathcal{O}(n^3)$: an eigendecomposition of a $10^4 \times 10^4$ covariance matrix is a serious computation, and one of a $10^6 \times 10^6$ kernel matrix is not a computation at all. The remedies are the Krylov methods mentioned above, randomised algorithms, and the observation of Section 1.21 that one rarely needs the whole spectrum.

1.18 The singular value decomposition

Every algorithm so far has assumed a square matrix, and most of them a symmetric one. The design matrix of Eq. (1.1) is neither. It has n rows and p columns, and there is no reason whatever for those two numbers to agree. For rectangular matrices we need a decomposition that does not require squareness, and preferably one that does not require diagonalisability either.

Recall that a square matrix can be diagonalised if and only if it is *normal*, that is if $\boldsymbol{X}\boldsymbol{X}^T = \boldsymbol{X}^T\boldsymbol{X}$. Not every matrix satisfies this. The simplest counterexample is

$$\boldsymbol{X} = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}, \tag{1.111}$$

whose determinant vanishes and for which $\boldsymbol{X}\boldsymbol{X}^T \neq \boldsymbol{X}^T\boldsymbol{X}$. It is *defective*: it cannot be diagonalised, and it has no inverse. A data set with two perfectly anticorrelated features produces exactly this matrix, so the case is not pathological but routine.

The singular value decomposition has no such restriction.

The theorem.

Any $n \times p$ matrix $\boldsymbol{X}$, real or complex, square or rectangular, can be written

$$\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T, \tag{1.112}$$

where $\boldsymbol{U}$ is an $n \times n$ orthogonal matrix, $\boldsymbol{U}\boldsymbol{U}^T = \boldsymbol{U}^T\boldsymbol{U} = \boldsymbol{I}_n$, where $\boldsymbol{V}$ is a $p \times p$ orthogonal matrix, $\boldsymbol{V}\boldsymbol{V}^T = \boldsymbol{V}^T\boldsymbol{V} = \boldsymbol{I}_p$, and where $\boldsymbol{\Sigma}$ is an $n \times p$ matrix which is zero except on the main diagonal, on which sit the *singular values*

$$\sigma_0 \geq \sigma_1 \geq \cdots \geq \sigma_{r-1} \geq 0, \qquad r = \min(n, p). \tag{1.113}$$

The columns of $\boldsymbol{U}$ are the *left* singular vectors and the columns of $\boldsymbol{V}$ the *right* singular vectors. The decomposition always exists. For the defective matrix of Eq. (1.111) it reads

$$\boldsymbol{X} = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}\begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T, \tag{1.114}$$

with $\sigma_0 = 2$ and $\sigma_1 = 0$. A matrix that cannot be diagonalised at all is decomposed without difficulty, and the vanishing singular value states plainly what the determinant only hinted at: the two columns carry one direction of information between them.

Geometry.

Equation (1.112) says that every linear map is a rotation, followed by a scaling along the new axes, followed by another rotation. The singular values are the scale factors, and $\|\boldsymbol{X}\|_2 = \sigma_0$, which is the statement of Section 1.12 that the induced 2-norm measures the largest stretching a matrix can produce. For a design matrix the picture is concrete: the right singular vectors are directions in feature space, the left singular vectors are patterns across observations, and the singular values say how much of the data lies along each such pairing.

Relation to the eigenvalue problem.

The SVD is not an independent construction; it is the symmetric eigenvalue problem of Section 1.17 in disguise. Using Eq. (1.112) and the orthogonality of $\boldsymbol{U}$,

$$\boldsymbol{X}^T\boldsymbol{X} = \boldsymbol{V}\boldsymbol{\Sigma}^T\boldsymbol{U}^T\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T = \boldsymbol{V}\boldsymbol{\Sigma}^T\boldsymbol{\Sigma}\boldsymbol{V}^T \equiv \boldsymbol{V}\tilde{\boldsymbol{\Sigma}}^2\boldsymbol{V}^T, \tag{1.115}$$

where $\tilde{\boldsymbol{\Sigma}}$ is the $p \times p$ diagonal matrix of singular values. Multiplying from the right by $\boldsymbol{V}$ gives

$$\left(\boldsymbol{X}^T\boldsymbol{X}\right)\boldsymbol{v}_i = \sigma_i^2\,\boldsymbol{v}_i, \tag{1.116}$$

so the right singular vectors are the eigenvectors of $\boldsymbol{X}^T\boldsymbol{X}$ and the squared singular values its eigenvalues. In exactly the same way

$$\boldsymbol{X}\boldsymbol{X}^T = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{\Sigma}^T\boldsymbol{U}^T, \qquad \left(\boldsymbol{X}\boldsymbol{X}^T\right)\boldsymbol{u}_i = \sigma_i^2\,\boldsymbol{u}_i, \tag{1.117}$$

with the remaining $n - r$ eigenvalues equal to zero. Every singular value is therefore the non-negative square root of an eigenvalue of $\boldsymbol{X}^T\boldsymbol{X}$, and if $\boldsymbol{X}$ is itself symmetric the singular values are the absolute values of its eigenvalues.

These two relations do a great deal of work in what follows. Read together with Section 1.10 they say that the covariance matrix $\boldsymbol{X}_c^T\boldsymbol{X}_c/(n-1)$ and the Gram matrix $\boldsymbol{X}_c\boldsymbol{X}_c^T$ share their non-zero spectrum entirely: the eigenvalues of the $p \times p$ matrix and of the $n \times n$ one are the same numbers, up to zeros. One may therefore always work with whichever is smaller, which is what makes principal component analysis feasible both when $p \gg n$ and when $n \gg p$.

The economy-size decomposition.

If $n > p$ the last $n - p$ columns of $\boldsymbol{U}$ are multiplied by zeros in $\boldsymbol{\Sigma}$ and can never contribute. Removing them, and the corresponding rows of zeros in $\boldsymbol{\Sigma}$, gives the economy-size decomposition, in which $\boldsymbol{U}$ is $n \times p$ and $\boldsymbol{\Sigma}$ is $p \times p$. If $p > n$ one keeps instead the first n columns of $\boldsymbol{V}$. Nothing is lost and both storage and work are reduced; this is `full_matrices=False` in numpy, and it is what one should almost always ask for.

How not to compute it.

Equation (1.116) suggests an algorithm: form $\boldsymbol{X}^T\boldsymbol{X}$, diagonalise it with the machinery of Section 1.17.3, take square roots. This is mathematically correct and numerically poor. From Eq. (1.77) the condition number of the product is the square of that of $\boldsymbol{X}$,

$$\kappa_2(\boldsymbol{X}^T\boldsymbol{X}) = \kappa_2(\boldsymbol{X})^2, \tag{1.118}$$

so half of the available significant digits are destroyed before the eigenvalue solver is even called. Table 1.2 shows the damage for a 60×12 matrix constructed with singular values spread logarithmically between 1 and 10^{-7}: the direct SVD returns all twelve to machine precision, the detour through $\boldsymbol{X}^T\boldsymbol{X}$ loses seven digits on the smallest.

Exact σ_i	From the SVD	From $\boldsymbol{X}^T\boldsymbol{X}$	Relative error
1.00000000×10^{0}	1.00000000×10^{0}	1.00000000×10^{0}	4.4×10^{-16}
$6.57933225 \times 10^{-4}$	$6.57933225 \times 10^{-4}$	$6.57933225 \times 10^{-4}$	2.0×10^{-11}
$1.87381742 \times 10^{-6}$	$1.87381742 \times 10^{-6}$	$1.87381889 \times 10^{-6}$	7.8×10^{-7}
$4.32876128 \times 10^{-7}$	$4.32876128 \times 10^{-7}$	$4.32893930 \times 10^{-7}$	4.1×10^{-5}
$1.00000000 \times 10^{-7}$	$1.00000000 \times 10^{-7}$	$9.99683051 \times 10^{-8}$	3.2×10^{-4}

Table 1.2 Singular values of a 60×12 matrix with $\kappa_2(\boldsymbol{X}) = 10^7$, computed directly and through the eigenvalues of $\boldsymbol{X}^T\boldsymbol{X}$, for which $\kappa_2 = 10^{14}$. Forming the product costs roughly half the significant digits.

The stable algorithm never forms the product. It reduces $\boldsymbol{X}$ to bidiagonal form by Householder reflectors, Eq. (1.110), applied alternately from the left and the right, and then runs an implicitly shifted QR sweep on the bidiagonal matrix. This is the Golub-Kahan algorithm, and it is what `numpy.linalg.svd` calls through LAPACK.

Figure 1.3 shows the same comparison graphically. The singular values computed directly by the Golub-Kahan algorithm lie on the exact values over the whole range of thirteen orders of magnitude, while those obtained as square roots of the eigenvalues of $\boldsymbol{X}^T\boldsymbol{X}$ peel away from the truth once σ_i falls below about $\sqrt{\varepsilon}\sigma_0$. The failure is not gradual in any useful sense: precisely the small singular values, which are the ones we consult in order to detect near-collinearity, are the ones destroyed.

Machine learning connection. Equation (1.118) is the reason that solving a regression problem through the normal equations (1.39) is a poor idea whenever the features are at all collinear. Forming $\boldsymbol{X}^T\boldsymbol{X}$ squares the condition number, so a design matrix with $\kappa_2(\boldsymbol{X}) = 10^8$ – unremarkable for a polynomial fit of modest degree – yields a system with $\kappa_2 = 10^{16}$, at which point double precision retains nothing. This is why `numpy.linalg.lstsq`, and `LinearRegression` in `scikit-learn`, do not solve the normal equations at all. They use the SVD, or a QR decomposition, and work with $\kappa_2(\boldsymbol{X})$ throughout. We write the normal equations because they are how one *derives* the estimator; we compute with the SVD because it is how one *evaluates* it.

1.19 Rank, the pseudoinverse and ill-conditioned design matrices

The *rank* of a matrix is the number of linearly independent columns, and the SVD reads it off immediately: it is the number of non-zero singular values. In floating-point arithmetic

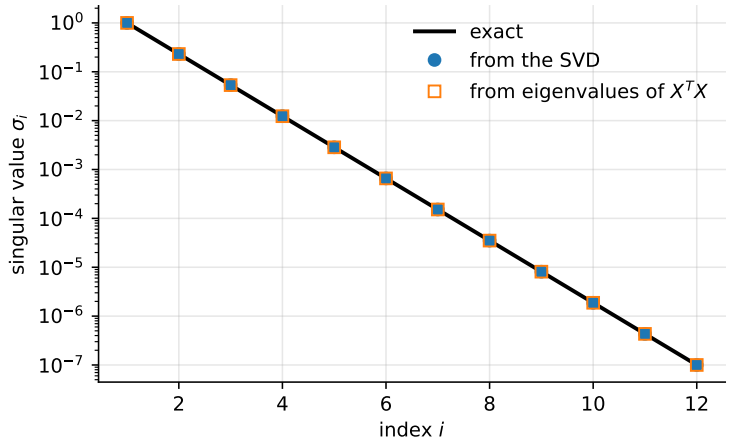


Fig. 1.3 Singular values of a 60×12 matrix constructed with σ_i spread logarithmically between 1 and 10^{-7}. The direct SVD reproduces all twelve; forming $\boldsymbol{X}^T\boldsymbol{X}$ first loses the smallest, as anticipated by Table 1.2.

one asks instead for the *numerical rank*, the number of singular values above a threshold, typically $\max(n,p)\,\varepsilon\,\sigma_0$ with ε the machine precision. A design matrix is rank deficient when some feature is an exact linear combination of others, and near rank deficient – which is the practically dangerous case – when it nearly is. The condition number of Section 1.12 is likewise a statement about singular values,

$$\kappa_2(\boldsymbol{X}) = \frac{\sigma_0}{\sigma_{r-1}}, \tag{1.119}$$

which is infinite exactly when $\boldsymbol{X}$ is singular. The SVD thus not only detects near-singularity, it quantifies it.

The Moore-Penrose pseudoinverse.

When $\boldsymbol{X}^{-1}$ does not exist – and for a rectangular design matrix it never does – or exists but is useless because $\boldsymbol{X}$ is nearly singular, the SVD provides the natural substitute

$$\boldsymbol{X}^{+} = \boldsymbol{V}\boldsymbol{\Sigma}^{+}\boldsymbol{U}^{T}, \tag{1.120}$$

where $\boldsymbol{\Sigma}^{+}$ is obtained from $\boldsymbol{\Sigma}$ by transposing it and replacing every non-zero singular value by its reciprocal, *leaving the zeros as zeros*. In practice one treats as zero every singular value below some $\varepsilon_{\mathrm{cut}}\sigma_0$. Inverting a singular value of 10^{-13} would amplify the rounding noise in the corresponding direction by 10^{13}; discarding it simply declares that the data contain no information about that direction, which is the truth.

Least squares through the pseudoinverse.

The pseudoinverse is not merely a formal device. Substituting the SVD into the normal equations (1.39) and using Eq. (1.115),

$$\boldsymbol{\theta} = \left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T\boldsymbol{y} = \boldsymbol{V}\tilde{\boldsymbol{\Sigma}}^{-2}\boldsymbol{V}^T\boldsymbol{V}\boldsymbol{\Sigma}^T\boldsymbol{U}^T\boldsymbol{y} = \boldsymbol{V}\boldsymbol{\Sigma}^{+}\boldsymbol{U}^T\boldsymbol{y} = \boldsymbol{X}^{+}\boldsymbol{y}, \tag{1.121}$$

so the least-squares estimator *is* the pseudoinverse applied to the targets. Written componentwise this reads

$$\boldsymbol{\theta} = \sum_{i=0}^{r-1} \frac{\boldsymbol{u}_i^T \boldsymbol{y}}{\sigma_i} \boldsymbol{v}_i, \tag{1.122}$$

which repays careful reading. The fit is a sum over directions in feature space. Each contributes the component of the target along the corresponding left singular vector, divided by the singular value. A direction along which the data barely vary has a small σ_i, and dividing by it amplifies whatever noise happens to lie along $\boldsymbol{u}_i$. *This* is multicollinearity, seen from the inside: it is not that the estimator is biased or that the statistics is wrong, it is that Eq. (1.122) divides by a small number. When the smallest singular values are dropped rather than inverted, the result is *truncated SVD regression* or principal component regression, and when they are damped rather than dropped, the result is Ridge regression – which is Section 1.22.

Equation (1.122) also disposes of the case $p > n$, more features than observations, in which $\boldsymbol{X}^T\boldsymbol{X}$ is singular and the normal equations have infinitely many solutions. The pseudoinverse selects among them the solution of smallest 2-norm, which is a perfectly definite and often sensible answer, and it does so without any special-case logic.

```
import numpy as np

def lstsq_svd(X, y, rcond=1.0e-12):
    """Least-squares fit through the SVD, with explicit truncation.

    Returns the coefficients, the singular values, and the numerical rank.
    """
    U, s, Vt = np.linalg.svd(X, full_matrices=False)
    keep = s > rcond * s[0]                 # numerical rank
    theta = (Vt[keep].T * (1.0 / s[keep])) @ (U[:, keep].T @ y)
    return theta, s, int(np.sum(keep))

# A deliberately collinear design matrix: the third column is nearly the
# sum of the first two.
rng = np.random.default_rng(2024)
n = 200
x1, x2 = rng.normal(size=n), rng.normal(size=n)
X = np.column_stack([x1, x2, x1 + x2 + 1.0e-6 * rng.normal(size=n)])
y = 1.0 + 2.0 * x1 - x2 + 0.1 * rng.normal(size=n)

theta, s, rank = lstsq_svd(X, y)
print("singular values:", s)
print("condition number:", s[0] / s[-1])
print("numerical rank:", rank)
```

The three singular values of this design matrix span some seven orders of magnitude, and the coefficients returned by an untruncated fit are enormous and mutually cancelling. Truncating the smallest singular value returns a rank-two fit whose predictions are essentially identical and whose coefficients are of order unity. Nothing has been lost: the third feature never carried independent information.

Machine learning connection. It is worth being explicit about what the truncation threshold buys. Keeping a direction with a tiny singular value adds a term to Eq. (1.122) with an enormous coefficient, which fits the noise in the training data and generalises catastrophically. Discarding it introduces a small bias and a large reduction in variance. The threshold is therefore not a numerical detail but a hyperparameter, chosen by the same cross-validation that chooses λ in Ridge regression, and the bias-variance trade-off of Chapter 2 is visible here in its purest algebraic form.

1.20 Low-rank approximation

We now come to the property that makes the SVD indispensable. Truncating the sum

$$\boldsymbol{X} = \sum_{k=0}^{r-1} \sigma_k \, \boldsymbol{u}_k \boldsymbol{v}_k^{T} \tag{1.123}$$

after χ terms gives a matrix $\boldsymbol{X}_\chi$ of rank χ. Note the form of the summands: each is an outer product, and by Section 1.2 each therefore has rank one. The SVD writes an arbitrary matrix as a weighted sum of rank-one pieces, ordered by importance.

The *Eckart-Young theorem* states that the truncation is the *best* rank-χ approximation to $\boldsymbol{X}$, in both the 2-norm and the Frobenius norm, and that the error it makes is

$$\|\boldsymbol{X} - \boldsymbol{X}_\chi\|_2 = \sigma_\chi, \qquad \|\boldsymbol{X} - \boldsymbol{X}_\chi\|_F^2 = \sum_{k \geq \chi} \sigma_k^2. \tag{1.124}$$

No cleverer choice of χ vectors exists. If the singular values decay quickly, a matrix with np entries is captured by $\chi(n+p+1)$ numbers, and the second of Eqs. (1.124) together with Eq. (1.74) says precisely how much has been given up: the fraction of the squared Frobenius norm retained is $\sum_{k<\chi} \sigma_k^2 / \sum_k \sigma_k^2$.

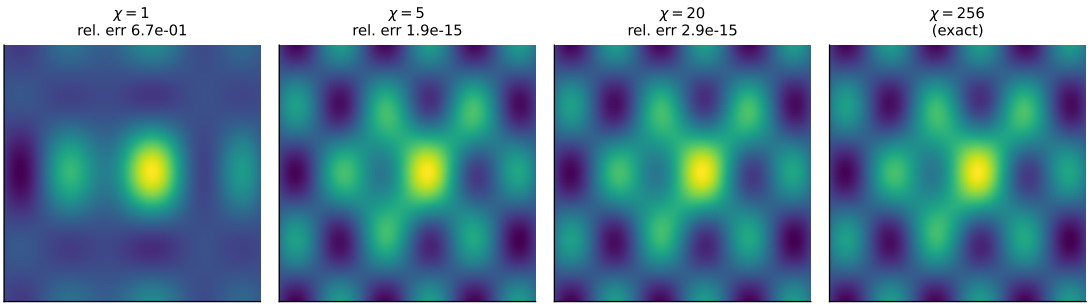


Fig. 1.4 Rank-χ truncations of a 256×256 array, Eq. (1.123). A single outer product already captures the gross structure; twenty terms are visually indistinguishable from the original while storing eight per cent of the numbers.

The two panels of Figures 1.4 and 1.5 should be read together. The reconstructions show what is seen; the spectrum shows why. The singular values fall by three orders of magnitude within the first twenty indices, so the cumulative fraction of $\|\boldsymbol{A}\|_F^2$ reaches 0.999 long before the rank is exhausted, and Eq. (1.124) guarantees that no other rank-χ matrix does better. Whether a given data matrix admits such a truncation is a question about its singular value spectrum and about nothing else.

The theorem is the mathematical justification for a large part of applied machine learning. Compressing an image means truncating the SVD of its pixel matrix. Latent semantic analysis means truncating the SVD of a document-term matrix, so that documents and words are both represented in a χ-dimensional space of "topics". A recommender system means approximating a sparse user-item matrix $\boldsymbol{R}$ by a product $\boldsymbol{W}\boldsymbol{H}$ of low rank, the χ retained directions being interpreted as latent tastes; the objective $\|\boldsymbol{R} - \boldsymbol{W}\boldsymbol{H}\|_F^2$ is the Frobenius-norm minimisation of Section 1.12.1, and when all entries are observed its solution is exactly Eq. (1.123) truncated. And, as the next section shows, principal component analysis is the same construction applied to the centred design matrix.

```
import numpy as np
```

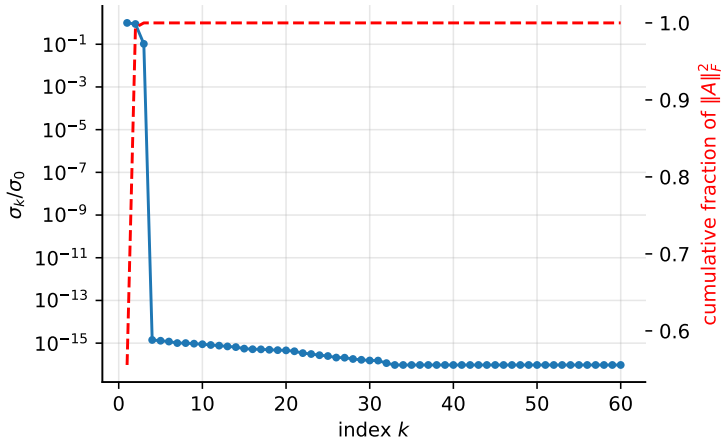


Fig. 1.5 Singular value spectrum of the array of Figure 1.4 (left axis, logarithmic) and the cumulative fraction of $\|\boldsymbol{A}\|_F^2$ retained by a rank-k truncation (right axis).

```python
def truncated_svd(X, chi):
    """Best rank-chi approximation to X, with the retained variance."""
    U, s, Vt = np.linalg.svd(X, full_matrices=False)
    X_chi = (U[:, :chi] * s[:chi]) @ Vt[:chi]
    retained = np.sum(s[:chi]**2) / np.sum(s**2)
    return X_chi, retained

# Storage: n*p entries become chi*(n + p + 1)
n, p, chi = 512, 512, 40
print("compression ratio:", chi * (n + p + 1) / (n * p))
```

1.21 Principal component analysis

Principal component analysis asks a question that sounds statistical and turns out to be algebraic: along which directions does the data vary most, and how many such directions are there?

Let $\boldsymbol{X}_c$ be the centred design matrix of Section 1.10. We look for the unit vector $\boldsymbol{w}$ along which the projected data $\boldsymbol{X}_c\boldsymbol{w}$ have the largest variance,

$$\max_{\|\boldsymbol{w}\|_2=1} \frac{1}{n-1}\|\boldsymbol{X}_c\boldsymbol{w}\|_2^2 = \max_{\|\boldsymbol{w}\|_2=1} \boldsymbol{w}^T\boldsymbol{\Sigma}\boldsymbol{w}, \tag{1.125}$$

with $\boldsymbol{\Sigma}$ the covariance matrix. Introducing a Lagrange multiplier for the constraint, the stationarity condition of $\boldsymbol{w}^T\boldsymbol{\Sigma}\boldsymbol{w} - \lambda(\boldsymbol{w}^T\boldsymbol{w}-1)$ follows from Eq. (1.27) and reads

$$\boldsymbol{\Sigma}\boldsymbol{w} = \lambda\boldsymbol{w}. \tag{1.126}$$

The maximising direction is therefore an eigenvector of the covariance matrix, and since $\boldsymbol{w}^T\boldsymbol{\Sigma}\boldsymbol{w} = \lambda$ at any such point, the *largest* eigenvalue gives the largest variance. Repeating the argument in the subspace orthogonal to $\boldsymbol{w}$ gives the second eigenvector, and so on. The principal directions are the eigenvectors of $\boldsymbol{\Sigma}$ ordered by eigenvalue, and the eigenvalues are the variances along them. That is the entire content of PCA, and it is Eq. (1.54) read as a statement about data.

PCA through the SVD.

In practice one never forms $\boldsymbol{\Sigma}$. By Eq. (1.116) applied to $\boldsymbol{X}_c = \boldsymbol{U}\boldsymbol{\Sigma}_s\boldsymbol{V}^T$ – writing $\boldsymbol{\Sigma}_s$ for the matrix of singular values, to avoid a collision with the covariance matrix – the eigenvectors of $\boldsymbol{\Sigma} = \boldsymbol{X}_c^T\boldsymbol{X}_c/(n-1)$ are the right singular vectors of $\boldsymbol{X}_c$, and its eigenvalues are

$$\lambda_i = \frac{\sigma_i^2}{n-1}. \tag{1.127}$$

The principal components – the coordinates of the observations in the new basis – are the columns of

$$\boldsymbol{T} = \boldsymbol{X}_c\boldsymbol{V} = \boldsymbol{U}\boldsymbol{\Sigma}_s, \tag{1.128}$$

and keeping only the first χ of them is exactly the truncation of Section 1.20. Dimensionality reduction by PCA and best low-rank approximation are the same operation described in two vocabularies: Eq. (1.124) guarantees that no other χ-dimensional linear projection retains more of the variance.

Going through the SVD rather than the covariance matrix is not a stylistic preference. By Eq. (1.118) forming $\boldsymbol{X}_c^T\boldsymbol{X}_c$ squares the condition number, and the small eigenvalues – which are exactly the ones a scree plot is used to inspect – are the first casualties.

```
import numpy as np

def pca(X, n_components=None):
    """Principal component analysis through the SVD of the centred data.

    Returns the scores T, the principal directions V, and the explained
    variance ratio of each component.
    """
    Xc = X - np.mean(X, axis=0)                  # centre each feature
    U, s, Vt = np.linalg.svd(Xc, full_matrices=False)

    explained = s**2 / np.sum(s**2)
    k = n_components if n_components is not None else len(s)
    T = U[:, :k] * s[:k]                         # scores = Xc @ V
    return T, Vt[:k].T, explained[:k]
```

Two practical remarks. Centring is not optional: without it the first principal component points at the mean of the data and says nothing about their variability. Scaling, on the other hand, is a genuine choice. Because variance has units, a feature measured in millimetres will dominate one measured in metres purely through its numerical size; standardising every feature to unit variance before the decomposition performs PCA on the correlation matrix instead of the covariance matrix. Neither is universally right, but the choice must be made deliberately, and it must be made using training-set statistics only.

Machine learning connection. PCA is unsupervised: it never looks at $\boldsymbol{y}$. The directions of largest variance in $\boldsymbol{X}$ need not be the directions most useful for predicting the target, and it is entirely possible for the discriminating signal to lie along the last principal component. Principal component regression – fitting on the first χ components – therefore works well when the informative directions happen to be the high-variance ones and badly when they do not. Partial least squares and linear discriminant analysis, which do consult the target, are the supervised answers to the same question, and we return to them in the chapter on dimensionality reduction.

1.22 Ridge regression through the singular value decomposition

We close the chapter by returning to the regularised normal equations (1.42) with the SVD in hand, because the combination explains in two lines what Ridge regression does.

Substituting $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T$ into $(\boldsymbol{X}^T\boldsymbol{X} + \lambda \boldsymbol{I})\boldsymbol{\theta} = \boldsymbol{X}^T\boldsymbol{y}$ and using $\boldsymbol{I} = \boldsymbol{V}\boldsymbol{V}^T$ gives $\boldsymbol{V}(\tilde{\boldsymbol{\Sigma}}^2 + \lambda \boldsymbol{I})\boldsymbol{V}^T\boldsymbol{\theta} = \boldsymbol{V}\boldsymbol{\Sigma}^T\boldsymbol{U}^T\boldsymbol{y}$, whence

$$\boldsymbol{\theta}_{\text{ridge}} = \sum_{i=0}^{r-1} \frac{\sigma_i}{\sigma_i^2 + \lambda} \left(\boldsymbol{u}_i^T \boldsymbol{y}\right) \boldsymbol{v}_i. \tag{1.129}$$

Compare this with the ordinary least-squares result (1.122), in which the coefficient of $\boldsymbol{u}_i^T\boldsymbol{y}$ was $1/\sigma_i$. Ridge regression replaces

$$\frac{1}{\sigma_i} \qquad \longrightarrow \qquad \frac{\sigma_i}{\sigma_i^2 + \lambda} = \frac{1}{\sigma_i} \cdot \frac{\sigma_i^2}{\sigma_i^2 + \lambda}, \tag{1.130}$$

that is, it multiplies each term by a *shrinkage factor* $\sigma_i^2/(\sigma_i^2 + \lambda)$ lying between zero and one. Directions with $\sigma_i^2 \gg \lambda$ are left essentially untouched; directions with $\sigma_i^2 \ll \lambda$ are suppressed almost completely. The penalty acts selectively, and it acts hardest exactly where Eq. (1.122) was dividing by a dangerously small number.

The fitted values inherit the same structure. From $\tilde{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\theta}_{\text{ridge}}$,

$$\tilde{\boldsymbol{y}} = \sum_i \frac{\sigma_i^2}{\sigma_i^2 + \lambda} \left(\boldsymbol{u}_i^T \boldsymbol{y}\right) \boldsymbol{u}_i, \tag{1.131}$$

so the Ridge hat matrix is not a projector: its eigenvalues are the shrinkage factors rather than zeros and ones, and its trace,

$$\mathrm{df}(\lambda) = \sum_i \frac{\sigma_i^2}{\sigma_i^2 + \lambda}, \tag{1.132}$$

decreases smoothly from p at $\lambda = 0$ towards zero as $\lambda \to \infty$. This is the *effective degrees of freedom* promised in the notebox of Section 1.6, and it makes precise the intuition that a regularised model is "smaller" than an unregularised one without having fewer parameters.

Truncated SVD regression, by contrast, replaces $1/\sigma_i$ by zero below a cutoff and leaves it alone above: a hard threshold where Ridge applies a soft one. The two are the same idea applied with different degrees of politeness, and both are, in the end, statements about the singular values of the design matrix. We take up the statistical consequences – the bias each introduces, the variance each removes, and how to choose λ – in Chapter 3.

1.23 Summary and the programs

The chapter has moved from notation to algorithms. The algebra of Sections 1.1-1.10 supplies the language in which machine learning is written: a data set is a design matrix, a linear model is a matrix acting on a parameter vector, fitting is projection onto a column space, differentiating a cost function is matrix calculus, and the structure of a data set is the spectrum of its covariance matrix. The numerical sections supply the means of extracting numbers from that language.

Three dividing lines are worth keeping in mind. The first separates direct from iterative methods. Direct methods – Gaussian elimination, LU, Cholesky, the Householder-based eigenvalue and SVD routines – deliver the complete answer in a fixed $\mathcal{O}(n^3)$ number of operations

and require the matrix to be stored. Iterative methods – Jacobi, Gauss-Seidel, over-relaxation, conjugate gradient, the Krylov eigensolvers – deliver an approximation that improves with each step and require only the action of the matrix on a vector. Every large-scale method in this book, gradient descent included, lives on the second side of that line.

The second line separates well-conditioned from ill-conditioned problems. The condition number $\kappa_2(\boldsymbol{X}) = \sigma_0/\sigma_{r-1}$ is a property of the data, not of the algorithm, and it decides how much of the input precision survives. Since $\kappa_2(\boldsymbol{X}^T\boldsymbol{X}) = \kappa_2(\boldsymbol{X})^2$, an algorithm that forms the normal equations pays twice; this is why the least-squares routines one should actually call are built on the QR decomposition or the SVD.

The third line is the one drawn by the singular value decomposition, and it is of a different kind. It separates the part of a problem that carries information from the part that does not. The same list of singular values tells us the numerical rank of a design matrix, how far a set of features is from collinearity, how much of the variance a χ-dimensional projection retains, how well a recommender matrix can be factorised, and how strongly Ridge regression will shrink each direction. Sections 1.18-1.22 are five readings of one decomposition, and the thread connecting them – Eq. (1.124), that truncating the singular values is the best approximation of its rank – runs through the rest of this book.

The complete programs are collected in `doc/BookML/BookPrograms`:

- `direct_solvers.py` – Gaussian elimination, LU and Cholesky, with determinants and inverses.
- `iterative_solvers.py` – Jacobi, Gauss-Seidel, over-relaxation and the conjugate gradient method, in both a matrix-based and a matrix-free form.
- `eigenvalues.py` – the power method, inverse iteration and the Rayleigh quotient, together with the timing comparisons against the LAPACK routines quoted in Section 1.17.3.
- `svd_tools.py` – the singular value decomposition applied to rank determination, the pseudoinverse, truncated least squares, low-rank approximation, principal component analysis and Ridge regression, including the collinear design matrix of Section 1.19 and the accuracy comparison of Table 1.2.

Each file runs as a script and reproduces the numbers quoted in this chapter. An executable version of the same material, in which the reader can vary the parameters, is available as a Jupyter notebook in the accompanying Jupyter-book.

1.24 Exercises

Warm-up exercises

1. **Inner and outer products.** Let $\boldsymbol{u} = \begin{bmatrix}1 & 2 & -1\end{bmatrix}^T$ and $\boldsymbol{v} = \begin{bmatrix}3 & 0 & 1\end{bmatrix}^T$. (a) Compute $\boldsymbol{u}^T\boldsymbol{v}$ and $\|\boldsymbol{u}\|_1$, $\|\boldsymbol{u}\|_2$, $\|\boldsymbol{u}\|_\infty$. (b) Compute the outer product $\boldsymbol{u}\boldsymbol{v}^T$ and verify that $\mathrm{Tr}(\boldsymbol{u}\boldsymbol{v}^T) = \boldsymbol{v}^T\boldsymbol{u}$. (c) What is the rank of $\boldsymbol{u}\boldsymbol{v}^T$? Justify your answer without computing a determinant.
2. **Special matrix types.** For each matrix below, identify which type(s) from Table 1.1 it belongs to, and verify your answer using the element conditions in the table: (a) a rotation by an angle θ in the plane, $\boldsymbol{R} = \begin{bmatrix}\cos\theta & -\sin\theta \\ \sin\theta & \cos\theta\end{bmatrix}$; (b) any covariance matrix; (c) $\boldsymbol{H} = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1 \\ 1 & -1\end{bmatrix}$.
3. **Projectors.** Let $\boldsymbol{q} \in \mathbb{R}^n$ with $\|\boldsymbol{q}\|_2 = 1$ and set $\boldsymbol{P} = \boldsymbol{q}\boldsymbol{q}^T$. (a) Verify $\boldsymbol{P}^T = \boldsymbol{P}$ and $\boldsymbol{P}^2 = \boldsymbol{P}$. (b) Show that the eigenvalues of $\boldsymbol{P}$ are 1 (once) and 0 ($n-1$ times), and that $\mathrm{Tr}(\boldsymbol{P}) = 1$. (c) Show that $\boldsymbol{I} - \boldsymbol{P}$ is also a projector and that $\boldsymbol{P}(\boldsymbol{I} - \boldsymbol{P}) = \boldsymbol{0}$.

4. **Matrix calculus.** Using only Eqs. (1.25) and (1.27), compute (a) $\partial\left(\boldsymbol{a}^T\boldsymbol{x}\boldsymbol{b}^T\boldsymbol{x}\right)/\partial\boldsymbol{x}$; (b) $\partial\|\boldsymbol{x}-\boldsymbol{a}\|_2^2/\partial\boldsymbol{x}$; (c) the gradient and the Hessian of $C(\boldsymbol{\theta})=\|\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}\|_2^2+\lambda\|\boldsymbol{\theta}\|_2^2$, and use them to confirm Eq. (1.42).
5. **Positive semi-definiteness.** (a) Show that $\boldsymbol{X}^T\boldsymbol{X}$ is positive semi-definite for any $\boldsymbol{X}$. (b) Show that it is positive definite if and only if the columns of $\boldsymbol{X}$ are linearly independent. (c) Deduce that if $p>n$ the least-squares problem cannot have a unique solution, and explain what Eq. (1.122) returns instead.
6. **Trace identities.** (a) Prove $\mathrm{Tr}(\boldsymbol{A}\boldsymbol{B})=\mathrm{Tr}(\boldsymbol{B}\boldsymbol{A})$ directly from the definition, and note that $\boldsymbol{A}\boldsymbol{B}$ and $\boldsymbol{B}\boldsymbol{A}$ need not even have the same dimensions. (b) Use it to prove $\|\boldsymbol{A}\|_F^2=\mathrm{Tr}(\boldsymbol{A}^T\boldsymbol{A})=\mathrm{Tr}(\boldsymbol{A}\boldsymbol{A}^T)$. (c) Show that $\|\boldsymbol{Q}\boldsymbol{A}\|_F=\|\boldsymbol{A}\|_F$ for orthogonal $\boldsymbol{Q}$.
7. **Conditioning in practice (numerical).** Build a design matrix whose columns are $1,x,x^2,\dots,x^{d-1}$ for 100 points x equally spaced on $[0,1]$ – the Vandermonde matrix of a polynomial fit. (a) Compute $\kappa_2(\boldsymbol{X})$ for $d=2,4,\dots,14$ and plot it on a logarithmic scale. (b) For which d does $\kappa_2(\boldsymbol{X}^T\boldsymbol{X})$ exceed $1/\varepsilon$? (c) Repeat with the columns rescaled to unit norm, and comment.
8. **Least squares three ways (numerical).** For the collinear design matrix of Section 1.19, compute $\boldsymbol{\theta}$ in three ways: (a) by solving the normal equations with `np.linalg.solve`; (b) with `np.linalg.lstsq`; and (c) with the truncated SVD of the same section. Compare the coefficients, the training error and the predictions. Which two agree, and why?
9. **Spectral decomposition.** Let $\boldsymbol{A}=\begin{bmatrix}2&1\\1&2\end{bmatrix}$. (a) Find its eigenvalues and normalised eigenvectors. (b) Write the spectral decomposition $\boldsymbol{A}=\sum_i\lambda_i\boldsymbol{v}_i\boldsymbol{v}_i^T$ explicitly and verify it by multiplying out. (c) Compute $\boldsymbol{A}^{1/2}$ and $\boldsymbol{A}^{-1}$ using Eq. (1.56), and check the latter against a direct inversion.
10. **PCA by hand (numerical).** Generate $n=500$ points in $\mathbb{R}^2$ from a Gaussian with covariance $\begin{bmatrix}3&2\\2&2\end{bmatrix}$. (a) Compute the covariance matrix of the sample and its eigendecomposition. (b) Compute the SVD of the centred data and verify Eq. (1.127). (c) Plot the data with the two principal directions drawn as arrows scaled by $\sqrt{\lambda_i}$. (d) Repeat after multiplying the first coordinate by 100. What happens to the principal directions, and what does this tell you about scaling?

Exercise session, week 34: orthogonal transformations, least squares and the SVD

The exercises of this session are meant as reminders of the specific linear algebra elements that we shall use throughout the course, and as a first encounter with the least-squares problem that occupies Chapter 3.

Exercise 1: orthogonal transformations and invariance.

It is common in data analysis to express the data in a new basis. Principal component analysis, the discrete Fourier transform and the whitening transformation of Eq. (1.59) are all of this kind, and in each case the change of basis is described by an orthogonal (or, for complex data, unitary) matrix.

Assume that $\boldsymbol{Q}$ is orthogonal with dimension $n\times n$ and matrix elements q_{ij}, and that $\{\boldsymbol{\phi}_\lambda\}$ is an orthonormal basis of $\mathbb{R}^n$ which diagonalises a symmetric matrix $\boldsymbol{O}$,

$$\boldsymbol{O}\boldsymbol{\phi}_\lambda=o_\lambda\boldsymbol{\phi}_\lambda. \tag{1.133}$$

Define a new set of vectors by $\boldsymbol{\psi}_p = \boldsymbol{Q}\boldsymbol{\phi}_p$.

a) Write down the defining properties of orthogonal and unitary matrices, and show that $|\boldsymbol{Q}| = \pm 1$.
b) Write down the properties of symmetric (and Hermitian) matrices, and prove that their eigenvalues are real.
c) Show that the new set $\{\boldsymbol{\psi}_p\}$ is orthonormal as well, and that consequently distances and angles between data points are unchanged by the transformation.
d) Show that the eigenvalues of the transformed matrix $\boldsymbol{Q}^T\boldsymbol{O}\boldsymbol{Q}$ are the same as those of $\boldsymbol{O}$.
e) A colleague proposes to reduce the dimensionality of a data set by multiplying each feature by a constant of their choosing, and argues that this cannot change the result of a k-nearest-neighbours classifier because it is "just a change of basis". Explain, using parts (c) and (d), why they are mistaken.

Exercise 2: the normal equations.

Consider the linear model $\tilde{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\theta}$ with $\boldsymbol{X} \in \mathbb{R}^{n\times p}$ and the cost function $C(\boldsymbol{\theta}) = \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2$.

a) Expand $C(\boldsymbol{\theta})$ as in Eq. (1.40), being explicit about why $\boldsymbol{y}^T\boldsymbol{X}\boldsymbol{\theta}$ and $\boldsymbol{\theta}^T\boldsymbol{X}^T\boldsymbol{y}$ may be combined.
b) Differentiate with respect to $\boldsymbol{\theta}$ and derive the normal equations (1.39).
c) Compute the Hessian and state the condition under which the stationary point is a unique minimum.
d) Show that the residual $\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}$ at the optimum is orthogonal to every column of $\boldsymbol{X}$, and interpret this geometrically using Eq. (1.8).
e) Show that the fitted values are $\tilde{\boldsymbol{y}} = \boldsymbol{H}\boldsymbol{y}$ with $\boldsymbol{H} = \boldsymbol{X}(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T$, and verify that $\boldsymbol{H}$ is symmetric and idempotent with $\mathrm{Tr}(\boldsymbol{H}) = p$.

Exercise 3: the SVD and rank.

Let $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T$ be the singular value decomposition of an $n \times p$ matrix.

a) Show that the columns of $\boldsymbol{V}$ are eigenvectors of $\boldsymbol{X}^T\boldsymbol{X}$ with eigenvalues σ_i^2, and that the columns of $\boldsymbol{U}$ are eigenvectors of $\boldsymbol{X}\boldsymbol{X}^T$ with the same non-zero eigenvalues.
b) Deduce that $\|\boldsymbol{X}\|_2 = \sigma_0$ and $\|\boldsymbol{X}\|_F^2 = \sum_i \sigma_i^2$.
c) Show that $\boldsymbol{X}^+\boldsymbol{y}$ with $\boldsymbol{X}^+$ given by Eq. (1.120) reproduces the least-squares solution whenever $\boldsymbol{X}^T\boldsymbol{X}$ is invertible.
d) Using Eq. (1.122), explain what goes wrong when one singular value is very small, and what the Ridge shrinkage factor of Eq. (1.130) does about it.
e) (Numerical.) Take a 256×256 greyscale image, compute its SVD, and plot the reconstruction error $\|\boldsymbol{X} - \boldsymbol{X}_\chi\|_F$ against χ together with the prediction of Eq. (1.124). At what χ does the image become visually indistinguishable from the original, and what compression ratio does that correspond to?

Chapter 2
Elements of Probability Theory and Statistics

Machine learning is inference from data, and data are noisy. Every number we compute from a finite sample – a mean, a fitted parameter, a test error – is itself a random quantity, and a result quoted without an estimate of its uncertainty is not a result. This chapter assembles the probability theory and statistics needed to make such statements precisely, in the order in which the rest of the book will need them.

The chapter has four parts. The first develops the language of probability: stochastic variables, distributions, expectation values, covariance, and the central limit theorem which explains why so much of statistics reduces to the Gaussian distribution. The second turns to *estimators* – the quantities we actually compute from a finite sample – and asks how far they can be trusted, including the awkward and frequently ignored case of correlated data. The third part is about assessing a model rather than a number: the split between training and test error, the bias-variance decomposition that explains why more complexity is not always better, and the resampling methods, bootstrap and cross-validation, that let us estimate generalisation error from the data we already have. The fourth part deals with generating randomness rather than analysing it: pseudo-random number generators, Markov chains and the Metropolis algorithm.

It is useful at the outset to distinguish two kinds of error. *Statistical* errors arise from the finiteness and randomness of the sample; they shrink, usually as $1/\sqrt{n}$, as more data are collected, and they are what the machinery of this chapter is designed to quantify. *Systematic* errors arise from the method itself – a biased measurement, a model that cannot represent the truth, a preprocessing step that leaks information from the test set – and they do not shrink with more data. No amount of statistics will reveal a systematic error; only thinking about the method will. Much of the discipline of machine learning consists of converting what would otherwise be systematic errors into statistical ones.

2.1 Domains, stochastic variables and probability distributions

Consider the tossing of two dice. The possible outcomes are

$$\{2,3,4,5,6,7,8,9,10,11,12\},$$

a set we call the *domain*, and to each element of the domain there corresponds a probability,

$$\{1/36,\, 2/36,\, 3/36,\, 4/36,\, 5/36,\, 6/36,\, 5/36,\, 4/36,\, 3/36,\, 2/36,\, 1/36\}.$$

We cannot say in advance whether the next throw will give 3 or 9; that is what we mean by calling the outcome random. What we can say in advance is that each outcome has a definite probability. Recording a sequence of throws might give

$$\{10, 8, 6, 3, 6, 9, 11, 8, 12, 4, 5\},$$

in which the numbers of the domain appear in no predictable order but, over sufficiently many throws, in the predicted proportions.

A *stochastic variable* (or random variable) is thus characterised by two things: a domain containing all values it may assume, and a probability distribution over that domain. We write $\mathbb{D} = \{x\}$ for the domain, so that $X \in \mathbb{D}$, and reserve upper-case X for the variable and lower-case x for a particular value it takes.

Discrete and continuous distributions.

In the discrete case the *probability distribution function* (PDF) $p(x)$ gives the probability with which each value occurs,

$$p(x) = \mathrm{Prob}(X = x). \tag{2.1}$$

In the continuous case $p(x)$ does not itself give a probability – the probability of any single real number is zero – but a probability *density*: the probability that X lies in an infinitesimal interval around x is $p(x)\,dx$, and the probability of landing in a finite interval is the integral

$$\mathrm{Prob}(a \le X \le b) = \int_a^b p(x)\,dx. \tag{2.2}$$

Qualitatively, a stochastic variable represents numbers drawn as if by chance from a specified PDF, in such a way that a large sample of them reproduces that PDF.

Closely related is the *cumulative distribution function* (CDF), the probability that X takes any value below x,

$$P(x) = \mathrm{Prob}(X \le x) = \int_{-\infty}^{x} p(x')\,dx', \qquad p(x) = \frac{d}{dx}P(x). \tag{2.3}$$

Throughout this book we reserve the lower-case p for a density and the upper-case P for the corresponding cumulative function. The CDF is not merely bookkeeping: Section 2.13 shows that inverting it is the standard way of generating samples from an arbitrary distribution.

Properties every PDF must have.

Two conditions are required. The first is positivity: a probability can be neither negative nor, in the discrete case, greater than one,

$$0 \le p(x_i) \le 1 \quad \text{(discrete)}, \qquad p(x) \ge 0 \quad \text{(continuous)}. \tag{2.4}$$

Note the asymmetry: a continuous *density* may perfectly well exceed unity, since only its integral is a probability. The second is normalisation – the probability that anything at all happens is one,

$$\sum_{x_i \in \mathbb{D}} p(x_i) = 1, \qquad \int_{x \in \mathbb{D}} p(x)\,dx = 1. \tag{2.5}$$

Table 2.1 collects these and the corresponding statements for the cumulative function.

	Discrete PDF	**Continuous PDF**
Domain	$\{x_0, x_1, \ldots, x_{N-1}\}$	$[a,b]$
Probability	$p(x_i)$	$p(x)\,dx$
Cumulative	$P_i = \sum_{l=0}^{i} p(x_l)$	$P(x) = \int_a^x p(t)\,dt$
Positivity	$0 \le p(x_i) \le 1$	$p(x) \ge 0$
Positivity	$0 \le P_i \le 1$	$0 \le P(x) \le 1$
Monotonicity	$P_i \ge P_j$ if $x_i \ge x_j$	$P(x_i) \ge P(x_j)$ if $x_i \ge x_j$
Normalisation	$P_{N-1} = 1$	$P(b) = 1$

Table 2.1 Properties of discrete and continuous probability distribution functions and of the corresponding cumulative distributions.

2.2 Distributions we shall need

A handful of distributions account for most of what happens in this book.

The uniform distribution.

The simplest PDF assigns equal density to every point of an interval,

$$p(x) = \frac{1}{b-a}\,\theta(x-a)\,\theta(b-x), \tag{2.6}$$

with θ the Heaviside step function. For $a=0$ and $b=1$ this is simply $p(x)\,dx = dx$ on $[0,1]$, which is what a random number generator produces and from which, as we shall see in Section 2.13, every other distribution is obtained by transformation.

The Gaussian (normal) distribution.

By far the most important continuous distribution is

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \tag{2.7}$$

with mean μ and standard deviation σ. For $\mu = 0$ and $\sigma = 1$ it is the *standard normal distribution*

$$p(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right), \tag{2.8}$$

and we write $X \sim \mathcal{N}(\mu, \sigma^2)$ for a Gaussian variable. Its pre-eminence is not a matter of convenience: the central limit theorem of Section 2.5 shows that averages of almost anything become Gaussian, which is why Gaussian noise is the default assumption in regression and why the squared-error loss of Chapter 3 is the maximum-likelihood loss for that assumption.

The following code plots Eq. (2.7) for three parameter pairs.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-20.0, 20.0, 400)
```

```
for mu, sigma in [(0.0, 1.0), (1.0, 2.0), (2.0, 4.0)]:
    p = np.exp(-(x - mu)**2 / (2 * sigma**2)) / np.sqrt(2 * np.pi * sigma**2)
    plt.plot(x, p, label=rf"$\mu={mu}$, $\sigma={sigma}$")

plt.xlabel(r"$x$"); plt.ylabel(r"$p(x)$")
plt.legend(); plt.show()
```

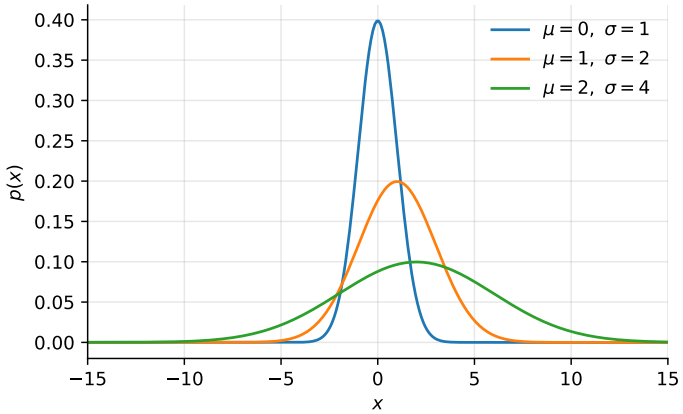


Fig. 2.1 The Gaussian density (2.7) for three parameter pairs. Changing μ translates the curve, changing σ both widens it and lowers its peak, the area remaining unity.

The multivariate Gaussian.

The generalisation to p correlated variables collected in a vector $\boldsymbol{x} \in \mathbb{R}^p$ is

$$p(\boldsymbol{x}) = \frac{1}{(2\pi)^{p/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\tfrac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right), \tag{2.9}$$

with $\boldsymbol{\mu}$ the vector of means and $\boldsymbol{\Sigma}$ the covariance matrix of Section 1.10. Everything in Eq. (2.9) is linear algebra from Chapter 1: the exponent is a quadratic form, the normalisation involves a determinant computed from a Cholesky factorisation through Eq. (1.84), and the surfaces of constant density are ellipsoids whose axes are the eigenvectors of $\boldsymbol{\Sigma}$ with lengths proportional to $\sqrt{\lambda_i}$ – which is to say the principal components of Section 1.21. Fitting Eq. (2.9) by maximum likelihood is what the derivative $\partial \ln|\boldsymbol{A}|/\partial \boldsymbol{A}$ of Eq. (1.49) was recorded for.

Other distributions.

The *exponential* distribution

$$p(x) = \alpha \exp(-\alpha x), \qquad x \geq 0, \tag{2.10}$$

describes waiting times and appears whenever events occur independently at a constant rate. The *binomial* distribution

$$p(k) = \binom{n}{k} q^k (1-q)^{n-k} \tag{2.11}$$

gives the probability of k successes in n independent trials each succeeding with probability q; it is the distribution behind the logistic regression and binary classification of later chapters, and the quantity q is exactly what such a classifier estimates. Its limit for large n and small q at fixed $nq = \lambda$ is the *Poisson* distribution $p(k) = \lambda^k e^{-\lambda}/k!$, appropriate for counts of rare events.

2.3 Expectation values and moments

Let $h(x)$ be a function on the domain of a stochastic variable X with PDF $p(x)$. The *expectation value* of h with respect to p is

$$\langle h \rangle_X \equiv \int h(x)\, p(x)\, dx, \tag{2.12}$$

with the integral replaced by a sum in the discrete case. Where the distribution is clear from the context we drop the subscript and write simply $\langle h \rangle$; we shall also use the equivalent notation $\mathbb{E}[h]$, which is more common in the machine learning literature and which we prefer when several distributions are in play at once.

A particularly useful class of expectation values are the *moments*

$$\langle x^n \rangle \equiv \int x^n p(x)\, dx. \tag{2.13}$$

The zeroth moment $\langle 1 \rangle$ is the normalisation condition (2.5). The first moment is the *mean*,

$$\langle x \rangle = \mu \equiv \int x\, p(x)\, dx, \qquad \langle x \rangle = \mu \equiv \sum_{i=0}^{N-1} x_i\, p(x_i), \tag{2.14}$$

for the continuous and discrete cases respectively; qualitatively it is the centroid of the distribution.

The *central moments* are taken about the mean,

$$\langle (x - \langle x \rangle)^n \rangle \equiv \int (x - \langle x \rangle)^n p(x)\, dx. \tag{2.15}$$

The zeroth and first are trivially 1 and 0. The second is the *variance*, and it is the quantity we shall use most,

$$\begin{aligned} \sigma_X^2 = \mathrm{Var}(X) &= \langle (x - \langle x \rangle)^2 \rangle = \int (x - \langle x \rangle)^2 p(x)\, dx \\ &= \int \left(x^2 - 2x\langle x \rangle + \langle x \rangle^2 \right) p(x)\, dx \\ &= \langle x^2 \rangle - 2\langle x \rangle \langle x \rangle + \langle x \rangle^2 \\ &= \langle x^2 \rangle - \langle x \rangle^2. \end{aligned} \tag{2.16}$$

The last line is worth memorising: the variance is the mean of the square minus the square of the mean. Its square root $\sigma = \sqrt{\langle (x - \langle x \rangle)^2 \rangle}$ is the *standard deviation*, the root-mean-square deviation of the distribution from its mean, interpreted qualitatively as the spread of p about μ. Two properties follow immediately from the linearity of Eq. (2.12) and will be used repeatedly,

$$\mathbb{E}[aX + b] = a\,\mathbb{E}[X] + b, \qquad \mathrm{Var}(aX + b) = a^2 \mathrm{Var}(X). \tag{2.17}$$

The variance is insensitive to a shift and scales with the *square* of a rescaling – which is the reason a feature measured in millimetres has a variance a million times larger than the same feature in metres, and hence the reason standardisation matters so much in Section 1.21.

2.4 Covariance, correlation and independence

Consider now a set $\{X_i\}$ of n stochastic variables, not necessarily independent, with a joint PDF $P(x_0,\dots,x_{n-1})$. The *covariance* of two of them is

$$\begin{aligned}\mathrm{Cov}(X_i,X_j) &= \langle (x_i-\langle x_i\rangle)(x_j-\langle x_j\rangle)\rangle \\ &= \int\cdots\int (x_i-\langle x_i\rangle)(x_j-\langle x_j\rangle)P(x_0,\dots,x_{n-1})\,dx_0\cdots dx_{n-1},\end{aligned} \tag{2.18}$$

with the marginal means

$$\langle x_i\rangle = \int\cdots\int x_i\,P(x_0,\dots,x_{n-1})\,dx_0\cdots dx_{n-1}.$$

Expanding the product exactly as in Eq. (2.16) gives the computationally convenient form

$$\begin{aligned}\mathrm{Cov}(X_i,X_j) &= \langle x_ix_j - x_i\langle x_j\rangle - \langle x_i\rangle x_j + \langle x_i\rangle\langle x_j\rangle\rangle \\ &= \langle x_ix_j\rangle - \langle x_i\rangle\langle x_j\rangle - \langle x_i\rangle\langle x_j\rangle + \langle x_i\rangle\langle x_j\rangle \\ &= \langle x_ix_j\rangle - \langle x_i\rangle\langle x_j\rangle,\end{aligned} \tag{2.19}$$

of which Eq. (2.16) is the special case $i=j$: the variance is the covariance of a variable with itself, $\mathrm{Cov}(X_i,X_i)=\mathrm{Var}(X_i)$.

Collecting these numbers into a matrix $\Sigma_{ij}=\mathrm{Cov}(X_i,X_j)$ gives the *covariance matrix*, whose diagonal holds the variances. This is the population version of the sample covariance matrix already met in Section 1.10, and everything proved there applies: it is symmetric, positive semi-definite, and its eigenvectors are the principal directions of the distribution.

Independence implies zero covariance.

Two variables are *independent* if their joint PDF factorises, $P(x_i,x_j)=p(x_i)p(x_j)$. In that case the expectation of the product factorises too, $\langle x_ix_j\rangle=\langle x_i\rangle\langle x_j\rangle$, and Eq. (2.19) gives

$$\mathrm{Cov}(X_i,X_j)=0, \qquad i\neq j. \tag{2.20}$$

The covariance matrix of independent variables is therefore diagonal.

The converse is false, and it matters. Zero covariance does *not* imply independence. Let X be symmetric about zero and put $Y=X^2$; then $\mathrm{Cov}(X,Y)=\langle x^3\rangle-\langle x\rangle\langle x^2\rangle=0$ although Y is a deterministic function of X. Covariance detects only *linear* dependence. Two consequences run through this book. First, principal component analysis produces uncorrelated components, not independent ones – which is precisely why independent component analysis exists as a separate method. Second, a correlation matrix showing nothing is not evidence that features carry no shared information; a neural network may still exploit a non-linear relationship that the covariance cannot see.

Correlation.

Dividing by the two standard deviations makes the measure scale-free,

$$\rho_{ij} = \frac{\mathrm{Cov}(X_i, X_j)}{\sigma_i \sigma_j} \in [-1, 1], \tag{2.21}$$

the bound being the Cauchy-Schwarz inequality (1.64) applied to the centred variables. The correlation matrix has unit diagonal, and it is what one should inspect when features carry different units.

The following code computes a covariance by hand and checks it against numpy.

```
import numpy as np

def covariance(x, y):
    """Sample covariance of two vectors, using the 1/n convention."""
    return np.mean((x - np.mean(x)) * (y - np.mean(y)))

rng = np.random.default_rng(2024)
n = 1000
x = rng.normal(size=n)
y = 4.0 + 3.0 * x + rng.normal(size=n)      # correlated with x by construction
z = rng.normal(size=n)                      # independent of both

print(covariance(x, y), covariance(x, z))
print(np.cov(np.vstack((x, y, z))))         # note: numpy uses 1/(n-1)
print(np.corrcoef(np.vstack((x, y, z))))    # the correlation matrix
```

Note the difference in convention: `np.cov` divides by $n-1$ rather than n, for reasons explained in Section 2.6.

2.5 The central limit theorem

We can now state and derive the result that makes the Gaussian distribution inescapable. Suppose we draw values from some PDF $p(x)$ and form the average of m of them,

$$z = \frac{x_0 + x_1 + \cdots + x_{m-1}}{m}, \tag{2.22}$$

and ask for the distribution $\tilde{p}(z)$ of the new variable z. The probability of obtaining a particular average is the product of the probabilities of the individual values, integrated over all combinations consistent with that average,

$$\tilde{p}(z) = \int dx_0\, p(x_0) \int dx_1\, p(x_1) \cdots \int dx_{m-1}\, p(x_{m-1})\; \delta\!\left(z - \frac{x_0 + \cdots + x_{m-1}}{m} \right), \tag{2.23}$$

where the δ-function enforces the constraint. The product form presupposes that the draws are independent, and this assumption is essential: everything in this section fails for correlated data, which is the subject of Section 2.7.

Carrying out the integral – most easily by writing the δ-function as a Fourier integral, expanding the resulting characteristic function to second order in its argument, and letting m grow – yields the *central limit theorem*:

Central limit theorem. The PDF $\tilde{p}(z)$ of the average of m independent values drawn from a distribution $p(x)$ with mean μ and finite variance σ^2 tends, as m grows, to a Gaussian with mean μ and variance σ^2/m, *whatever the shape of* $p(x)$.

The practical statement is the familiar one for the standard deviation of a mean,

$$\boxed{\sigma_m = \frac{\sigma}{\sqrt{m}}} \tag{2.24}$$

usually called the *standard error*. Averaging four times as much data halves the uncertainty, and this square-root law is the reason large data sets help and the reason they help so slowly.

Two qualifications deserve emphasis, because both are routinely forgotten. The first is the requirement of a finite variance: distributions with heavy tails, such as the Cauchy distribution, have no finite σ^2 and their sample means do not become Gaussian at all. The second is independence. When the values are correlated, Eq. (2.24) *underestimates* the true uncertainty, often by a large factor, and we take this up next.

Figure 2.2 illustrates both the theorem and its limits. Averages of uniform and of exponential variables become visibly Gaussian by $m = 10$ and are indistinguishable from it by $m = 100$, with the width shrinking as $1/\sqrt{m}$; note that the exponential is strongly skewed and that the skew disappears nonetheless. The Cauchy distribution behaves entirely differently: its average is no better determined for $m = 100$ than for $m = 1$. The reason is that it has no finite variance, so the hypothesis of the theorem fails, and no amount of averaging helps. It is worth remembering that the central limit theorem is a statement with conditions and not a law of nature.

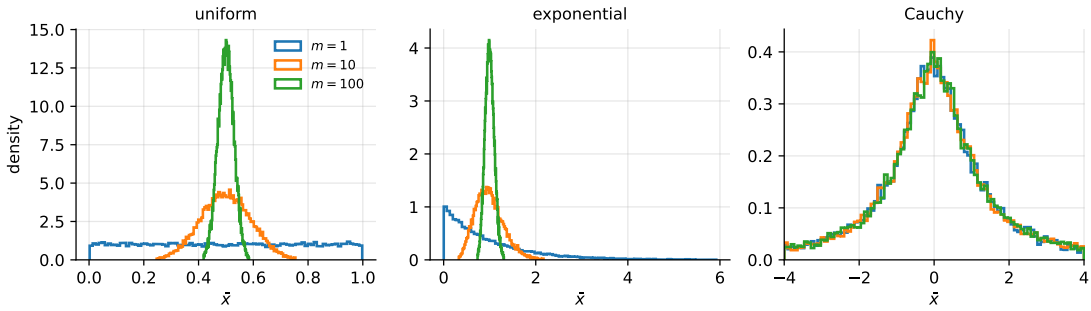


Fig. 2.2 The central limit theorem at work, and failing. Histograms of the average of m independent draws for three distributions. Uniform and exponential averages converge to a Gaussian of width $\sigma/\sqrt{m}$; Cauchy averages do not converge at all, the distribution of $\bar{x}$ being independent of m.

Machine learning connection. Equation (2.24) is why a test set of 1000 points gives a test error known to roughly 3% relative accuracy, and why comparing two models whose test errors differ by less than that is meaningless. It is also the theoretical justification for the bootstrap of Section 2.11, and, applied to mini-batches, it explains why the gradient estimate in stochastic gradient descent has noise scaling as $1/\sqrt{B}$ with batch size B – so that quadrupling the batch size only halves the gradient noise, at four times the cost per step.

2.6 Samples, estimators and their properties

The framework of the preceding sections is idealised: it presumes we know $p(x)$. In practice we have a finite set of measurements and must infer from them. A *stochastic process* produces a chain of values $\{x_0, x_1, \dots\}$; we call these our measurements and the whole set our *sample*, and we assume they are distributed according to some unknown PDF $p_X(x)$. Rather than determine p_X in full, we usually want only its lowest moments.

For a sample of size n the *sample mean* and *sample variance* are

$$\bar{x} = \frac{1}{n}\sum_{k=0}^{n-1} x_k, \qquad s^2 = \frac{1}{n-1}\sum_{k=0}^{n-1} (x_k - \bar{x})^2 . \tag{2.25}$$

These are *estimators*: functions of the data, used to approximate properties of the underlying distribution. Being functions of random variables, estimators are themselves random variables, with distributions of their own. This observation is the foundation of everything that follows, and in particular of the resampling methods of Sections 2.11 and 2.12, whose entire purpose is to explore that distribution numerically.

Bias, variance and mean squared error.

Let $\hat{\theta}$ be an estimator of a quantity θ. Its *bias* is

$$\operatorname{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta, \tag{2.26}$$

the systematic part of the error, and an estimator with zero bias is called *unbiased*. Its *variance* $\operatorname{Var}(\hat{\theta})$ measures how much it moves when the sample is redrawn. The two combine into the mean squared error, by adding and subtracting $\mathbb{E}[\hat{\theta}]$,

$$\boxed{\mathbb{E}\left[(\hat{\theta} - \theta)^2\right] = \underbrace{\left(\mathbb{E}[\hat{\theta}] - \theta\right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}\left[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2\right]}_{\operatorname{Var}(\hat{\theta})}} \tag{2.27}$$

the cross term vanishing because $\mathbb{E}[\hat{\theta} - \mathbb{E}[\hat{\theta}]] = 0$. Equation (2.27) is the bias-variance decomposition in its simplest form, and Section 2.10 is nothing but this identity applied to a prediction rather than a parameter. It already carries the central lesson: an unbiased estimator is not automatically a good one, since a biased estimator with much smaller variance can have the smaller total error. Ridge regression, introduced in Eq. (1.42) and analysed in Chapter 3, is exactly such a trade.

An estimator is *consistent* if it converges in probability to θ as $n \to \infty$, which by Eq. (2.27) follows if both bias and variance vanish in that limit.

Why $n-1$.

The sample mean is unbiased, $\mathbb{E}[\bar{x}] = \mu$, directly from the linearity (2.17). The variance is more subtle. Using $\bar{x}$ in place of the unknown μ makes the sum of squared deviations systematically too small, because $\bar{x}$ is the value that *minimises* that sum for the given sample. One data point has been spent estimating the mean, and a short calculation shows

$$\mathbb{E}\left[\frac{1}{n}\sum_{k}(x_k - \bar{x})^2\right] = \frac{n-1}{n}\,\sigma^2, \tag{2.28}$$

so dividing by $n-1$ rather than n restores unbiasedness. This is *Bessel's correction*, and it is why `np.cov` and `np.std` differ in their default `ddof` settings – a discrepancy that matters for small samples and is irrelevant for large ones.

Repeated experiments.

It is often natural to think of a sample as one of several repetitions of an experiment. Suppose an experiment yielding n measurements is repeated m times, giving values $x_{\alpha,k}$ with $\alpha = 1,\dots,m$ and $k = 1,\dots,n$. The total average is

$$\langle X_m \rangle = \frac{1}{m}\sum_{\alpha=1}^{m} \bar{x}_\alpha = \frac{1}{mn}\sum_{\alpha,k} x_{\alpha,k}, \tag{2.29}$$

and the variance of that average is

$$\sigma_m^2 = \frac{1}{mn^2}\sum_{\alpha=1}^{m}\sum_{k,l=1}^{n} \left(x_{\alpha,k} - \langle X_m\rangle\right)\left(x_{\alpha,l} - \langle X_m\rangle\right), \tag{2.30}$$

while the sample variance over all mn individual measurements is

$$\sigma^2 = \frac{1}{mn}\sum_{\alpha=1}^{m}\sum_{k=1}^{n} \left(x_{\alpha,k} - \langle X_m\rangle\right)^2. \tag{2.31}$$

These are experimental quantities and may differ, sometimes considerably, from the exact μ_X, $\mathrm{Var}(X)$ and $\mathrm{Cov}(X,Y)$ they estimate. The relation between Eqs. (2.30) and (2.31) is the subject of the next section.

2.7 Correlated data and the autocorrelation function

Equation (2.30) contains a double sum over k and l. Separating the diagonal terms $k = l$ from the rest gives

$$\sigma_m^2 = \frac{\sigma^2}{n} + \frac{2}{mn^2}\sum_{\alpha=1}^{m}\sum_{k<l}^{n} \left(x_{\alpha,k} - \langle X_m\rangle\right)\left(x_{\alpha,l} - \langle X_m\rangle\right), \tag{2.32}$$

in which the first term is the sample variance divided by n – exactly the central limit result (2.24) – and the second is a sum of covariances between distinct measurements. If the measurements are uncorrelated the second term vanishes and Eq. (2.24) is recovered. If they are correlated it does not, and the naive standard error is wrong.

The first term is cheap: we need only accumulate running sums of x and x^2. The correlation term is expensive, since it requires every pair and therefore the storage of all measurements until the end. To organise it, group the pairs by their separation $d = l - k$,

$$f_d = \frac{1}{nm}\sum_{\alpha=1}^{m}\sum_{k=1}^{n-d} \left(x_{\alpha,k} - \langle X_m\rangle\right)\left(x_{\alpha,k+d} - \langle X_m\rangle\right), \tag{2.33}$$

so that the correlation term becomes $2\sum_{d=1}^{n-1} f_d/n$ and

$$\sigma_m^2 = \frac{\sigma^2}{n} + \frac{2}{n}\sum_{d=1}^{n-1} f_d. \tag{2.34}$$

The quantity f_d measures the correlation between measurements separated by d steps, and $f_0 = \sigma^2$ by construction. Normalising by the variance defines the *autocorrelation function*

$$\kappa_d = \frac{f_d}{\sigma^2}, \qquad \kappa_0 = 1, \tag{2.35}$$

in terms of which Eq. (2.34) takes the compact form

$$\boxed{\sigma_m^2 = \frac{\tau}{n}\,\sigma^2, \qquad \tau = 1 + 2\sum_{d=1}^{n-1} \kappa_d .} \tag{2.36}$$

The factor τ is the *autocorrelation time*. For independent data $\tau = 1$ and we recover Eq. (2.24); for correlated data $\tau > 1$, and the effective number of independent measurements is not n but n/τ. Ignoring this is one of the most common ways of quoting an error bar that is too small by an order of magnitude.

Figure 2.3 shows the effect for a first-order autoregressive sequence, $x_{t+1} = \phi x_t + \eta_t$, which decays as $\kappa_d = \phi^d$ and for which the correlation time can be computed exactly, $\tau = (1+\phi)/(1-\phi)$. With $\phi = 0.9$ this gives $\tau = 19$: the effective number of independent measurements is one nineteenth of the number recorded, and an error bar computed from Eq. (2.24) is too small by a factor of $\sqrt{19} \approx 4.4$.

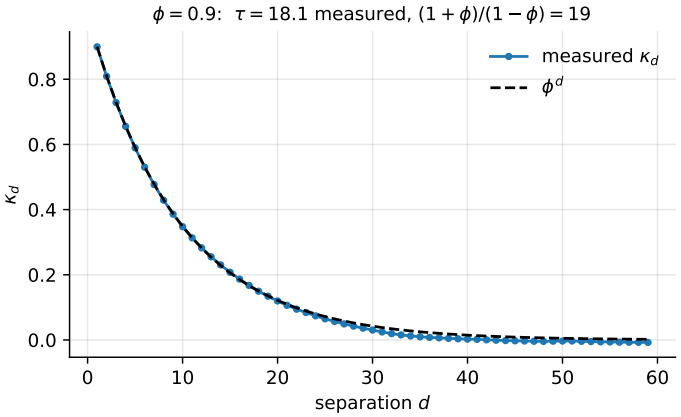


Fig. 2.3 Autocorrelation function (2.35) of an autoregressive sequence with $\phi = 0.9$, compared with the exact ϕ^d. The measured correlation time agrees with the prediction $(1+\phi)/(1-\phi)$.

Machine learning connection. Correlated samples are the rule rather than the exception. Successive states of a Markov chain (Section 2.14) are correlated by construction, so any expectation value estimated from a Metropolis run needs Eq. (2.36) rather than Eq. (2.24). Time series – prices, sensor readings, patient records – are correlated, which is why splitting them randomly into training and test sets is a serious error: a randomly chosen test point sits between two training points that are nearly identical to it, and the test error measures interpolation rather than prediction. The same applies to data with group structure, such as several images of the same patient; the split must then respect the groups. A test error computed on correlated data is optimistic for the same reason that Eq. (2.24) is optimistic.

2.8 The statistics of the least-squares estimator

The general theory above becomes concrete when applied to the estimator we already derived in Chapter 1. Assume the data are generated by a linear model with additive noise,

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\theta} + \boldsymbol{\varepsilon}, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2), \tag{2.37}$$

with the errors independent,

$$\operatorname{Cov}(\varepsilon_i, \varepsilon_j) = \begin{cases} \sigma^2 & i = j, \\ 0 & i \neq j, \end{cases} \tag{2.38}$$

so that the noise covariance matrix is $\sigma^2 \boldsymbol{I}_{nn}$. Here $\boldsymbol{X}$ is the fixed $n \times p$ design matrix of Eq. (1.1) and $\boldsymbol{\theta}$ the true parameter vector. The randomness of $\boldsymbol{\varepsilon}$ makes $\boldsymbol{y}$ random too: since $\boldsymbol{X}_{i,*}\boldsymbol{\theta}$ is a non-random scalar,

$$\mathbb{E}[y_i] = \boldsymbol{X}_{i,*}\boldsymbol{\theta}, \qquad \operatorname{Var}(y_i) = \sigma^2, \tag{2.39}$$

where $\boldsymbol{X}_{i,*}$ denotes row i of the design matrix. This is the first place in the book where a random and a non-random quantity are added, and since every statistical statement about regression rests on the distinction, it is worth being precise about what it means and why Eq. (2.39) follows from it.

What is random here, and what is not.

Equation (2.37) contains four objects, and only one of them is stochastic. The noise $\boldsymbol{\varepsilon}$ is a random vector: it takes a different value every time the experiment is repeated, and those values follow the distribution (2.38). The parameter vector $\boldsymbol{\theta}$ is a set of fixed numbers – unknown to us, which is the whole problem, but not random; nature does not redraw it between measurements. The design matrix $\boldsymbol{X}$ is likewise treated as a fixed array of numbers: the inputs $\boldsymbol{x}_i$ at which the measurements were taken are regarded as chosen, or at least as given, and the analysis is carried out for those inputs. Statisticians call this the *fixed design* viewpoint, and everything in this section is conditional on the observed $\boldsymbol{X}$. And the function f that the model is trying to capture is a *non-stochastic*, or deterministic, function: a fixed rule that returns the same value $f(\boldsymbol{x})$ every time it is evaluated at the same input. Nothing about f changes from one repetition of the experiment to the next; what changes is the noise added to it. If we could measure the response at the same $\boldsymbol{x}_i$ a thousand times, the thousand values of y_i would scatter about the single fixed number $f(\boldsymbol{x}_i)$, and the scatter would be entirely that of ε_i.

A non-random quantity c can be accommodated in the formalism of Section 2.3 by regarding it as a stochastic variable whose distribution is concentrated at a single point, $p(c') = \delta(c' - c)$ with δ the Dirac delta function. Inserting this into the definition (2.12) gives

$$\mathbb{E}[c] = \int c'\,\delta(c'-c)\,dc' = c, \qquad \operatorname{Var}(c) = \int (c'-c)^2\,\delta(c'-c)\,dc' = 0\,: \tag{2.40}$$

the expectation of a constant is the constant, and a constant has no variance. These two lines are all that "non-random" means for the purposes of computing moments.

Deriving Eq. (2.39).

Write $\mu_i = \boldsymbol{X}_{i,*}\boldsymbol{\theta} = \sum_j X_{ij}\theta_j$ for the ith fitted value at the true parameters. It is a sum of products of fixed numbers and is therefore itself a fixed number, and the model says $y_i = \mu_i + \varepsilon_i$. Because μ_i is a constant, the distribution of y_i is that of ε_i shifted by μ_i: if p_ε is the noise density, $p(y_i) =$

$p_\varepsilon(y_i-\mu_i)$. The expectation then follows directly from the definition (2.12) by the substitution $e = y_i - \mu_i$,

$$\mathbb{E}[y_i] = \int y\, p_\varepsilon(y-\mu_i)\, dy = \int (\mu_i + e)\, p_\varepsilon(e)\, de = \mu_i \underbrace{\int p_\varepsilon(e)\, de}_{=1} + \underbrace{\int e\, p_\varepsilon(e)\, de}_{=\mathbb{E}[\varepsilon_i]=0} = \mu_i, \tag{2.41}$$

where the first integral is the normalisation (2.5) and the second vanishes because the noise has zero mean. The variance follows from the central moment (2.15) by the same substitution,

$$\mathrm{Var}(y_i) = \int (y-\mu_i)^2\, p_\varepsilon(y-\mu_i)\, dy = \int e^2\, p_\varepsilon(e)\, de = \mathrm{Var}(\varepsilon_i) = \sigma^2. \tag{2.42}$$

Both results are the rule (2.17) with $a = 1$ and $b = \mu_i$: adding a constant shifts the mean by that constant and leaves the variance untouched, because the variance measures spread about the mean and a constant shift moves the mean and every value together. The same argument applied to two different observations gives $\mathrm{Cov}(y_i, y_j) = \mathbb{E}[(y_i-\mu_i)(y_j-\mu_j)] = \mathbb{E}[\varepsilon_i\varepsilon_j] = \mathrm{Cov}(\varepsilon_i, \varepsilon_j)$, so the whole vector satisfies

$$\mathbb{E}[\boldsymbol{y}] = \boldsymbol{X\theta}, \qquad \mathrm{Cov}(\boldsymbol{y}) = \mathrm{Cov}(\boldsymbol{\varepsilon}) = \sigma^2 \boldsymbol{I}_{nn}\,: \tag{2.43}$$

the observations inherit the covariance structure of the noise exactly, and the deterministic part of the model contributes only to the mean. When the noise is Gaussian, $\boldsymbol{y}$ is the multivariate Gaussian (2.9) with $\boldsymbol{\mu} = \boldsymbol{X\theta}$ and $\boldsymbol{\Sigma} = \sigma^2\boldsymbol{I}$, which is the form the likelihood of Chapter 3 will take.

A short simulation makes the statement concrete. The design and the parameters are fixed once; only the noise is redrawn, a hundred thousand times; and the sample mean and variance of each y_i over the repetitions are compared with Eq. (2.39).

```
import numpy as np

rng = np.random.default_rng(2024)
x = np.linspace(0.0, 1.0, 5)            # the design is fixed once and for all
X = np.column_stack([np.ones_like(x), x, x**2])
theta, sigma = np.array([1.0, -2.0, 3.0]), 0.5
mu = X @ theta                          # non-random: the same in every repetition

Y = mu + sigma * rng.normal(size=(100000, len(x)))  # 100000 draws of the noise only
print("X theta        :", np.round(mu, 3))
print("mean of y      :", np.round(Y.mean(axis=0), 3))
print("variance of y  :", np.round(Y.var(axis=0), 3), "  sigma^2 =", sigma**2)
print("cov(y_0, y_1)  :", np.round(np.cov(Y[:, 0], Y[:, 1])[0, 1], 4))
```

```
X theta        : [1.    0.688 0.75  1.188 2.   ]
mean of y      : [1.    0.686 0.751 1.188 2.   ]
variance of y  : [0.251 0.252 0.252 0.251 0.249]   sigma^2 = 0.25
cov(y_0, y_1)  : 0.0015
```

The means agree with $\boldsymbol{X\theta}$ to the third decimal, the variances with σ^2, and the covariance between two observations is zero to within the sampling error of Eq. (2.24). Note what was *not* redrawn: the array `X` is the same object in all hundred thousand repetitions, which is exactly what the fixed-design assumption says.

The model as an approximation of f.

Equation (2.37) asserts more than that f is non-stochastic: it asserts that f is *linear in the features*, so that $f(\boldsymbol{x}_i) = \boldsymbol{X}_{i,*}\boldsymbol{\theta}$ exactly, for some $\boldsymbol{\theta}$. The two assumptions are independent and it is worth separating them. Suppose only the weaker one holds, so that the data are generated by $y_i = f(\boldsymbol{x}_i) + \varepsilon_i$ with f a fixed but otherwise arbitrary function. Nothing in the derivation of Eqs. (2.41) and (2.42) used the form of μ_i, only that it was a constant, so the conclusion survives unchanged with $\mu_i = f(\boldsymbol{x}_i)$:

$$\mathbb{E}[y_i] = f(\boldsymbol{x}_i), \qquad \mathrm{Var}(y_i) = \sigma^2. \tag{2.44}$$

This is the sense in which the linear model $\boldsymbol{X}\boldsymbol{\theta}$ *approximates* f: it is a candidate for the deterministic part of the data, the mean of $\boldsymbol{y}$, and the noise variance is untouched by how good or bad the candidate is. If f really is linear in the features, Eq. (2.44) reduces to Eq. (2.39) and the estimator below is unbiased. If it is not, then no choice of $\boldsymbol{\theta}$ makes $\boldsymbol{X}_{i,*}\boldsymbol{\theta}$ equal to $f(\boldsymbol{x}_i)$ at every i, and the discrepancy $f(\boldsymbol{x}_i) - \boldsymbol{X}_{i,*}\boldsymbol{\theta}$ is a fixed number, not a random one: it does not average away with more data, it is invisible in the residuals, whose scatter still has variance σ^2, and it reappears as the squared-bias term of Section 2.10. A deterministic f that the model cannot represent is thus the source of bias, exactly as the random $\boldsymbol{\varepsilon}$ is the source of variance; keeping the two apart is what the decomposition of that section is for.

Machine learning connection. The fixed-design viewpoint is the natural one for a designed experiment, where the inputs are chosen by the experimenter. In machine learning the inputs are usually themselves a random sample – the pixels of whichever images happened to be collected – and one speaks of a *random design*. The two viewpoints are reconciled by conditioning: every expectation in this section is $\mathbb{E}[\cdot \mid \boldsymbol{X}]$, an average over the noise for the inputs actually observed, and the results hold as stated for every $\boldsymbol{X}$ that might have been drawn. The bias-variance decomposition of Section 2.10 averages over training sets, inputs included, and the bootstrap that estimates it resamples whole rows $(\boldsymbol{x}_i, y_i)$ rather than the noise alone; the reader should keep in mind which expectation is meant when a result from one setting is quoted in the other.

The estimator is unbiased.

The least-squares estimator of Eq. (1.39) is $\hat{\boldsymbol{\theta}} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}$. Taking its expectation and using $\mathbb{E}[\boldsymbol{y}] = \boldsymbol{X}\boldsymbol{\theta}$,

$$\mathbb{E}[\hat{\boldsymbol{\theta}}] = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\,\mathbb{E}[\boldsymbol{y}] = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\theta} = \boldsymbol{\theta}, \tag{2.45}$$

so ordinary least squares is unbiased, provided the model (2.37) is correct. That proviso carries all the weight: if the true $f(\boldsymbol{x})$ is not linear in the features, the estimator is biased no matter how much data we collect, and that bias is the first term of the decomposition in Section 2.10.

The variance.

Writing $\boldsymbol{A} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T$ so that $\hat{\boldsymbol{\theta}} = \boldsymbol{A}\boldsymbol{y}$, and using $\mathbb{E}[\boldsymbol{y}\boldsymbol{y}^T] = \boldsymbol{X}\boldsymbol{\theta}\boldsymbol{\theta}^T\boldsymbol{X}^T + \sigma^2\boldsymbol{I}_{nn}$,

$$\begin{aligned}\mathrm{Var}(\hat{\boldsymbol{\theta}}) &= \mathbb{E}\left[\hat{\boldsymbol{\theta}}\hat{\boldsymbol{\theta}}^T\right] - \boldsymbol{\theta}\boldsymbol{\theta}^T \\ &= (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\left\{\boldsymbol{X}\boldsymbol{\theta}\boldsymbol{\theta}^T\boldsymbol{X}^T + \sigma^2\boldsymbol{I}\right\}\boldsymbol{X}(\boldsymbol{X}^T\boldsymbol{X})^{-1} - \boldsymbol{\theta}\boldsymbol{\theta}^T \\ &= \boldsymbol{\theta}\boldsymbol{\theta}^T + \sigma^2(\boldsymbol{X}^T\boldsymbol{X})^{-1} - \boldsymbol{\theta}\boldsymbol{\theta}^T, \end{aligned} \tag{2.46}$$

that is

$$\boxed{\mathrm{Var}(\hat{\boldsymbol{\theta}}) = \sigma^2\left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}.} \tag{2.47}$$

This is the result promised in Section 1.8.5: the covariance matrix of the fitted parameters is the inverse Hessian (1.44) scaled by the noise variance. The standard error of the jth coefficient is the square root of the corresponding diagonal element,

$$\sigma(\hat{\theta}_j) = \sigma\sqrt{\left[(\boldsymbol{X}^T\boldsymbol{X})^{-1}\right]_{jj}}, \tag{2.48}$$

from which a confidence interval follows in the usual way, and in practice σ^2 is itself estimated from the residuals as $\hat{\sigma}^2 = \|\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{\theta}}\|_2^2/(n-p)$, the divisor $n-p$ being Bessel's correction (2.28) generalised to p fitted parameters.

Machine learning connection. Read Eq. (2.47) through the singular value decomposition. By Eq. (1.115), $(\boldsymbol{X}^T\boldsymbol{X})^{-1} = \boldsymbol{V}\tilde{\boldsymbol{\Sigma}}^{-2}\boldsymbol{V}^T$, so the variance along the ith right singular direction is σ^2/σ_i^2. A small singular value – near-collinear features – produces an enormous variance in that direction, which is the statistical statement of the numerical warning given in Section 1.19. The ill-conditioning we diagnosed there as a loss of significant digits is here diagnosed as an inability of the data to determine the parameter. Both readings describe the same fact, and both are repaired by the same shrinkage: Ridge regression trades the unbiasedness (2.45) for a variance reduced by the factors of Eq. (1.130), a bargain which Eq. (2.27) tells us is worth taking whenever the squared bias introduced is smaller than the variance removed.

2.9 Training error, test error and generalisation

Up to this point we have asked how accurately a number is determined. We now ask a different question: how well will a fitted model perform on data it has never seen? This is the question of *generalisation*, and it cannot be answered by looking at the data used for fitting.

The standard measure for continuous predictions is the mean squared error of Eq. (1.33); the mean absolute error is an alternative, less sensitive to outliers, corresponding to the 1-norm rather than the 2-norm in the sense of Section 1.12. Whichever loss is chosen, we must distinguish two quantities:

- the *training error* $\mathrm{Err}_{\mathrm{Train}}$, the average loss over the data used to fit the model;
- the *test error* or prediction error $\mathrm{Err}_{\mathrm{Test}}$, the average loss on data held out from fitting entirely.

Only the second estimates generalisation. The first can always be driven to zero by a sufficiently flexible model – a polynomial of degree $n-1$ passes exactly through n points – and a model that has done so has memorised the sample rather than learned the process behind it.

As model complexity grows, the training error decreases monotonically and saturates near zero. The test error behaves differently: it falls at first, as the model becomes able to represent genuine structure, reaches a minimum, and then rises again as the model begins to fit

the noise. The location of that minimum is what model selection seeks, and the shape of the curve is explained by the decomposition of the next section.

Machine learning connection. A test set is only a test set as long as it has been used once. Every time a hyperparameter is tuned by looking at the test error, information leaks from the test set into the model, and the test error becomes an optimistic estimate of generalisation – a systematic error in the sense of the introduction to this chapter, invisible to any of the statistics of this chapter. The standard discipline is a three-way split: train on one part, select hyperparameters on a *validation* part, and touch the test part exactly once, at the end. When data are scarce, the cross-validation of Section 2.12 replaces the validation set. The same warning applies to preprocessing: as noted in Section 1.5, means and standard deviations used for scaling must be computed on the training part alone.

2.10 The bias-variance tradeoff

We now derive the decomposition that governs the shape of the test-error curve. The argument is a direct application of Eq. (2.27), with the estimator being a *prediction* rather than a parameter. It is stated here for continuous predictions, but the intuition carries over to classification.

Let the data be generated by a noisy process,

$$\boldsymbol{y} = f(\boldsymbol{x}) + \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(0, \sigma^2), \tag{2.49}$$

with f the unknown true function and the noise of zero mean and variance σ^2. We fit a model on a training set drawn at random, obtaining predictions $\tilde{\boldsymbol{y}}$, and we ask for the expected squared error on a test point. The expectation is taken over the randomness of the training set *and* of the noise; this is the essential point, and the source of most confusion about the result. The model $\tilde{\boldsymbol{y}}$ is a random variable because the data it was fitted to were random.

The cost we wish to decompose is

$$\mathbb{E}\left[(\boldsymbol{y} - \tilde{\boldsymbol{y}})^2\right] = \frac{1}{n}\sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2 \,. \tag{2.50}$$

Insert Eq. (2.49) and add and subtract $\mathbb{E}[\tilde{\boldsymbol{y}}]$, the average prediction over training sets,

$$\mathbb{E}\left[(\boldsymbol{y} - \tilde{\boldsymbol{y}})^2\right] = \mathbb{E}\left[(\boldsymbol{f} + \boldsymbol{\varepsilon} - \tilde{\boldsymbol{y}} + \mathbb{E}[\tilde{\boldsymbol{y}}] - \mathbb{E}[\tilde{\boldsymbol{y}}])^2\right]. \tag{2.51}$$

Expanding the square produces three squared terms and three cross terms. The cross terms all vanish: those involving $\boldsymbol{\varepsilon}$ because the noise has zero mean and is independent of the training set, and the remaining one because $\mathbb{E}[\tilde{\boldsymbol{y}} - \mathbb{E}[\tilde{\boldsymbol{y}}]] = 0$ while $\boldsymbol{f}$ is not stochastic. What survives is

$$\boxed{\mathbb{E}\left[(\boldsymbol{y} - \tilde{\boldsymbol{y}})^2\right] = \underbrace{\frac{1}{n}\sum_i (f_i - \mathbb{E}[\tilde{\boldsymbol{y}}])^2}_{\text{Bias}^2} + \underbrace{\frac{1}{n}\sum_i (\tilde{y}_i - \mathbb{E}[\tilde{\boldsymbol{y}}])^2}_{\text{Var}[\tilde{\boldsymbol{y}}]} + \sigma^2.} \tag{2.52}$$

The three terms have distinct meanings.

The **bias** measures how far the *average* prediction is from the truth. It is the error caused by the simplifying assumptions built into the model: a straight line fitted to curved data has a large bias, and no quantity of data will remove it.

The **variance** measures how much the prediction moves when the training set is redrawn. A flexible model chases the noise in whatever sample it is given, so its predictions differ substantially from one training set to the next, even though it may be unbiased on average.

The **irreducible error** σ^2 is the variance of the noise itself. No model, however good, can predict a random term, so σ^2 is a floor beneath the test error. A model whose test error is at or below σ^2 is not excellent; it has leaked test data or the noise estimate is wrong.

Increasing model complexity decreases the bias and increases the variance. The test error, being their sum plus a constant, has a minimum somewhere in between, and finding it is the entire business of model selection.

Estimating the three terms.

Equation (2.52) involves an expectation over training sets, which in practice we do not have – we have one data set. The bootstrap of Section 2.11 manufactures the missing ensemble by resampling, and the following code uses it to compute all three terms for a polynomial fit of fixed degree.

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.utils import resample

np.random.seed(2018)

n, n_bootstraps, degree = 500, 100, 18      # a deliberately high degree
x = np.linspace(-1, 3, n).reshape(-1, 1)
y = np.exp(-x**2) + 1.5 * np.exp(-(x - 2)**2) + np.random.normal(0, 0.1, x.shape)

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
model = make_pipeline(PolynomialFeatures(degree=degree),
                      LinearRegression(fit_intercept=False))

# Column i holds the predictions of the model fitted to bootstrap sample i
y_pred = np.empty((y_test.shape[0], n_bootstraps))
for i in range(n_bootstraps):
    x_, y_ = resample(x_train, y_train)
    y_pred[:, i] = model.fit(x_, y_).predict(x_test).ravel()

# Expectations over training sets are averages along axis 1; keepdims=True
# preserves the column shape and is essential for the bias to come out right.
error    = np.mean(np.mean((y_test - y_pred)**2, axis=1, keepdims=True))
bias     = np.mean((y_test - np.mean(y_pred, axis=1, keepdims=True))**2)
variance = np.mean(np.var(y_pred, axis=1, keepdims=True))

print(f"Error = {error:.5f}, Bias^2 = {bias:.5f}, Var = {variance:.5f}")
print(f"{error:.5f} >= {bias:.5f} + {variance:.5f} = {bias + variance:.5f}")
```

Note that the bias computed this way absorbs the irreducible error, since y_{test} rather than the unknown f appears in it; the printed identity is therefore an inequality. Sweeping the polynomial degree traces out the whole trade-off:

```
import matplotlib.pyplot as plt

n, n_bootstraps, maxdegree = 40, 100, 14
x = np.linspace(-3, 3, n).reshape(-1, 1)
y = np.exp(-x**2) + 1.5 * np.exp(-(x - 2)**2) + np.random.normal(0, 0.1, x.shape)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
```

```
error    = np.zeros(maxdegree)
bias     = np.zeros(maxdegree)
variance = np.zeros(maxdegree)

for degree in range(maxdegree):
    model = make_pipeline(PolynomialFeatures(degree=degree),
                          LinearRegression(fit_intercept=False))
    y_pred = np.empty((y_test.shape[0], n_bootstraps))
    for i in range(n_bootstraps):
        x_, y_ = resample(x_train, y_train)
        y_pred[:, i] = model.fit(x_, y_).predict(x_test).ravel()

    error[degree]    = np.mean(np.mean((y_test - y_pred)**2, axis=1, keepdims=True))
    bias[degree]     = np.mean((y_test - np.mean(y_pred, axis=1, keepdims=True))**2)
    variance[degree] = np.mean(np.var(y_pred, axis=1, keepdims=True))

plt.plot(range(maxdegree), error,    label="Error")
plt.plot(range(maxdegree), bias,     label="Bias$^2$")
plt.plot(range(maxdegree), variance, label="Variance")
plt.xlabel("Polynomial degree"); plt.legend(); plt.show()
```

The resulting figure is the canonical one: the bias falls steeply and flattens, the variance is negligible at low degree and grows without bound, and their sum has a clear minimum.

Figure 2.4 is that figure. Three features are worth naming. The squared bias falls by two orders of magnitude between degrees zero and four and then flattens: beyond degree four the polynomial family is rich enough to represent the underlying function, and additional flexibility buys no reduction in bias. The variance is negligible at low degree and then grows steadily, since a more flexible model chases the noise in whichever bootstrap sample it is given. Their sum, the test error, therefore falls, reaches a minimum around degree four or five, and rises thereafter. Model selection is the business of locating that minimum, and Section 2.12 is how it is done without a test set.

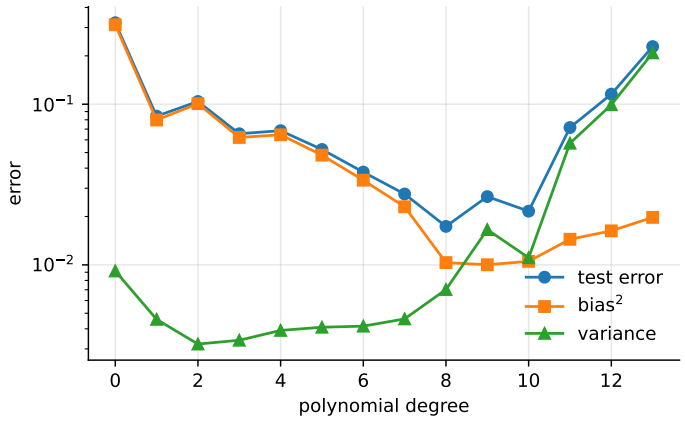


Fig. 2.4 The bias-variance decomposition (2.52) for a polynomial fit, with the three terms estimated by the bootstrap. Bias falls and saturates, variance grows without limit, and the test error has a minimum in between.

Machine learning connection. Every regularisation method in this book is a deliberate move along this curve. Ridge and Lasso add bias in order to remove variance; early stopping halts training before the variance has grown; dropout and data augmentation reduce variance in neural networks; bagging and random forests average many high-

variance models to cancel their variance while leaving the bias untouched, whereas boosting reduces bias by combining many high-bias models. Knowing which of the two terms a method attacks is often enough to predict whether it will help on a given problem. It should be added that the clean picture above is a statement about models of moderate capacity; the double-descent behaviour observed in heavily overparameterised networks shows that the test error can fall again beyond the interpolation threshold, and we return to this in the chapter on neural networks.

2.11 Resampling: the jackknife and the bootstrap

Since $\hat{\theta} = \hat{\theta}(\boldsymbol{X})$ is a function of random variables, it is itself a random variable with some PDF $p(t)$. If we knew $p(x)$, the distribution generating the data, we could learn everything about $p(t)$ by brute force: draw a fresh sample $(x_0^*, \dots, x_{n-1}^*)$ from $p(x)$, compute a replica $\hat{\theta}^*$, repeat many times, and histogram the results. The difficulty is that $p(x)$ is exactly what we do not know.

Efron’s idea.

In 1979 Efron asked what happens if we replace $p(x)$ by the *empirical* distribution – the relative frequency of the observations we actually have. If we draw new samples in accordance with the observed frequencies, do we recover the right answer in an asymptotic sense? The answer is yes, and drawing from the empirical distribution is nothing more than drawing from the observed values *with replacement*.

The independent bootstrap therefore works as follows:

1. Draw with replacement n values from the observations $\boldsymbol{x} = (x_0, \dots, x_{n-1})$, forming a resample $\boldsymbol{x}^*$.
2. Evaluate the estimator on the resample to obtain $\hat{\theta}^*$.
3. Repeat k times.

The relative frequency of the k values $\hat{\theta}^*$ estimates $p(t)$. In practice one never draws the histogram; one simply applies the estimator of interest to the collection, using for instance the sample variance (2.25) of the $\hat{\theta}^*$ as an estimate of $\mathrm{Var}(\hat{\theta})$.

The central limit theorem of Section 2.5 is what makes this work: the bootstrap distribution of a mean-like statistic approaches the same Gaussian as the true sampling distribution. It also marks the limits of the method. The bootstrap assumes independent, identically distributed observations; for correlated data it fails, for the reason discussed in Section 2.7, and variants such as the block bootstrap exist precisely to repair this.

The bootstrap is attractive because it is general: it requires no distributional assumption such as Gaussian errors, it applies to statistics whose sampling distributions are difficult or impossible to derive analytically, it copes with complicated sampling designs, and it is often more accurate than asymptotic formulae when the sample is small. It should be said that when the data are independent and identically distributed and all we want is the variance of a mean, Eq. (2.24) already answers the question and no bootstrap is needed.

The jackknife.

The older jackknife is a special case. Instead of resampling, it systematically leaves out one observation at a time. Writing

$$\boldsymbol{x}_i = (x_0, \dots, x_{i-1}, x_{i+1}, \dots, x_{n-1}) \tag{2.53}$$

for the sample with observation i removed, and $\hat{\theta}_i$ for the estimator computed on it, the spread of the n values $\hat{\theta}_i$ estimates the variance of $\hat{\theta}$ and their mean gives a bias estimate. Both jackknife and bootstrap require independent, identically distributed variables and both fail without that assumption.

```
import numpy as np

def bootstrap(data, statistic, k=1000, rng=None):
    """Bootstrap estimate of the distribution of a statistic."""
    rng = np.random.default_rng() if rng is None else rng
    n = len(data)
    replicas = np.empty(k)
    for i in range(k):
        sample = data[rng.integers(0, n, n)]      # draw n values with replacement
        replicas[i] = statistic(sample)
    return replicas


def jackknife(data, statistic):
    """Leave-one-out replicas of a statistic."""
    n = len(data)
    return np.array([statistic(np.delete(data, i)) for i in range(n)])


rng = np.random.default_rng(2024)
mu, sigma, n = 100.0, 15.0, 10000
x = rng.normal(mu, sigma, n)

t = bootstrap(x, np.mean, k=1000, rng=rng)
print(f"original {np.mean(x):.5f}  bootstrap mean {np.mean(t):.5f}"
      f"  std. error {np.std(t):.5f}")
print(f"central limit theorem prediction: {np.std(x) / np.sqrt(n):.5f}")

tj = jackknife(x, np.mean)
print(f"jackknife bias {(n - 1) * (np.mean(tj) - np.mean(x)):.3e}"
      f"  std. error {np.sqrt((n - 1) * np.var(tj)):.5f}")
```

For the sample mean of independent Gaussian data, all three estimates of the standard error agree, as they must. The value of the bootstrap is that the first two lines continue to work when `np.mean` is replaced by a statistic for which no formula exists – a median, a ratio, a cross-validated test error, or the coefficient of a fitted model.

2.12 Cross-validation

A single split into training and test sets wastes data and gives an estimate that depends on which points happened to fall where. Repeating the split at random improves matters but does not eliminate the problem: some samples may land in the test set far more often than others, giving them undue influence. *k-fold cross-validation* removes the imbalance by structuring the splitting.

The data are divided into k roughly equal, mutually exclusive and exhaustive subsets, the *folds*. Each fold in turn plays the role of the test set while the union of the remaining $k-1$ folds serves as the training set. Each observation is therefore used for testing exactly once and for training exactly $k-1$ times. The procedure is:

1. Shuffle the data set at random.

2. Split it into k folds.
3. For each fold: hold it out, fit the model on the remaining folds, evaluate on the held-out fold, record the score and discard the model.
4. Summarise the model by the mean, and the spread, of the k scores.

Choosing $k = n$ leaves out one observation at a time and removes the residual randomness of the fold assignment entirely; this is *leave-one-out cross-validation* (LOOCV), and the reader will recognise it as the jackknife of Eq. (2.53) applied to prediction error. It is unbiased but expensive, requiring n fits, and its estimates have high variance because the n training sets are almost identical. Values of k between five and ten are the usual compromise.

The most common use of cross-validation is to select a hyperparameter. For Ridge regression, whose estimator was given in Eq. (1.42), one proceeds as follows:

- Define a grid of values for the penalty parameter λ, usually logarithmically spaced.
- For each fold, and for each λ in the grid, fit

$$\boldsymbol{\theta}_{-i}(\lambda) = \left(\boldsymbol{X}_{-i,*}^T \boldsymbol{X}_{-i,*} + \lambda \boldsymbol{I}_{pp}\right)^{-1} \boldsymbol{X}_{-i,*}^T \boldsymbol{y}_{-i} \tag{2.54}$$

  on the data with fold i removed.
- Evaluate the prediction performance on the held-out fold, using the squared error, the absolute error or the R^2 score.
- Average over folds to obtain, for each λ, an estimate of the prediction error on unseen data.
- Choose the λ minimising that estimate, and refit on all the data.

This is the outline; the details that make it correct – the standardisation inside each fold, the refit, the one-standard-error rule, the handling of the Lasso, and the comparison with the bootstrap – are the subject of Section 3.14.

```
import numpy as np
from sklearn.linear_model import Ridge
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import KFold, cross_val_score

np.random.seed(3155)

x = np.random.randn(100)[:, np.newaxis]
y = 3 * x.ravel()**2 + np.random.randn(100)          # noise variance is 1

lambdas = np.logspace(-3, 5, 100)
kfold = KFold(n_splits=5, shuffle=True, random_state=3155)

mse = np.empty(len(lambdas))
for i, lmb in enumerate(lambdas):
    # The scaler is refitted on each training fold, never on the test fold
    pipe = make_pipeline(PolynomialFeatures(degree=6),
                         StandardScaler(),
                         Ridge(alpha=lmb))
    scores = -cross_val_score(pipe, x, y, cv=kfold,
                              scoring="neg_mean_squared_error")
    mse[i] = scores.mean()

best = np.argmin(mse)
print(f"best lambda {lambdas[best]:.4g}, cross-validated MSE {mse[best]:.4f}")
```

The run above returns $\lambda \approx 0.028$ with a cross-validated mean squared error of 1.01, which is reassuring: the noise added to the data had variance one, so the procedure has recovered the irreducible error σ^2 of Eq. (2.52) and nothing more. At the other end of the grid, $\lambda = 10^5$

shrinks every coefficient to almost zero and the error rises to 19, the variance of $\boldsymbol{y}$ itself – the model has been regularised into predicting the mean.

The `StandardScaler` in the pipeline is not decoration. A degree-six polynomial basis built from $x \sim \mathcal{N}(0,1)$ has columns whose scales differ by orders of magnitude, so the design matrix is severely ill-conditioned in the sense of Eq. (1.77); without scaling, the cross-validation curve is dominated by numerical noise at the tails and its minimum lands at whichever end of the grid one happened to choose. This is the Vandermonde conditioning problem of Chapter 1 appearing in statistical clothing, and it is a good illustration of why the two halves of this book cannot be kept apart.

Machine learning connection. Any preprocessing that learns from the data – centring, scaling, feature selection, imputation of missing values, dimensionality reduction by PCA – must be performed *inside* the cross-validation loop, refitted on each training fold. Scaling the whole data set once before the loop lets each training fold see the mean and variance of its own test fold, which is a small leak with a surprisingly large effect when the number of features is large. In `scikit-learn` the remedy is to wrap the preprocessing and the estimator in a `Pipeline` and to cross-validate the pipeline, which is what the `make_pipeline` calls in Section 2.10 were doing. The same discipline applies to the bootstrap.

2.13 Random numbers

Every method in the second half of this chapter – the bootstrap, the shuffling of cross-validation folds, the train-test split – consumes random numbers, as do the weight initialisation of a neural network, the mini-batch sampling of stochastic gradient descent, and the Markov chains of the next section. It is worth knowing where they come from.

Uniform deviates are random numbers lying within a specified range, typically $[0,1)$, with every number in the range equally likely. They are the basic building block: numbers from other distributions are almost always produced by transforming uniform deviates, as we show below.

A disclaimer is necessary at once. Something as deterministic as a computer cannot generate genuinely random numbers. What the standard algorithms produce are *pseudo-random* numbers, which one hopes satisfy the following criteria:

1. they are uniformly distributed on $[0,1)$;
2. correlations between successive numbers are negligible;
3. the period, after which the sequence repeats, is as long as possible;
4. the algorithm is fast.

The second criterion matters because every argument in this chapter has assumed independent draws, and a generator with detectable correlations silently violates the hypotheses of the central limit theorem and of the bootstrap alike.

That deterministic iteration can look random is easy to demonstrate. The logistic map

$$x_{i+1} = c\,x_i\,(1 - x_i) \tag{2.55}$$

produces, for suitable c, sequences that pass many naive tests of randomness while being entirely reproducible.

Linear congruential generators.

The classical construction is

$$N_{i+1} = (aN_i + c) \bmod M, \tag{2.56}$$

with a, c and M integers chosen with care, and $x_i = N_i/M$ the resulting deviate on $[0,1)$. The sequence is completely determined by the seed N_0, which is what makes runs reproducible, and it necessarily repeats after at most M steps. Poor choices of a, c and M give catastrophically bad generators: the notorious RANDU produced numbers which, plotted as consecutive triples, lie on fifteen planes in three dimensions. Modern generators such as the Mersenne Twister and the PCG family used by `numpy.random.default_rng` have astronomically long periods and pass extensive test suites, and there is no reason to write one's own.

Reproducibility.

Because the sequence is determined by the seed, setting it makes an experiment repeatable, and reporting results from a stochastic method without fixing and stating the seed makes them impossible to check. The modern numpy interface is explicit about this:

```
import numpy as np

rng = np.random.default_rng(seed=2024)     # a generator object, not global state
print(rng.random(5))                       # uniform on [0, 1)
print(rng.normal(loc=0.0, scale=1.0, size=5))
print(rng.integers(0, 10, size=5))

# Passing the generator explicitly keeps functions reproducible and
# independent of any other code that also draws random numbers.
def experiment(rng):
    return np.mean(rng.normal(size=1000))
```

Passing an explicit generator, rather than calling `np.random.seed` and relying on global state, is strongly preferable: it makes reproducibility a property of the function rather than of the whole program, and it avoids the situation in which one library's reseeding silently changes another's results.

From uniform to arbitrary distributions.

Suppose we want samples from a distribution $p(x)$ with cumulative distribution $P(x)$ from Eq. (2.3). If U is uniform on $[0,1)$ then the variable

$$X = P^{-1}(U) \tag{2.57}$$

has exactly the distribution p. The proof is one line: $\mathrm{Prob}(X \le x) = \mathrm{Prob}(P^{-1}(U) \le x) = \mathrm{Prob}(U \le P(x)) = P(x)$, using the monotonicity of P and the uniformity of U. This is *inverse transform sampling*. For the exponential distribution (2.10) we have $P(x) = 1 - e^{-\alpha x}$ and hence $x = -\ln(1-u)/\alpha$, an explicit recipe. The method requires P to be invertible in closed form, which for the Gaussian it is not; there one uses instead the Box-Muller transformation, which draws two independent standard normal variables from two uniform ones,

$$x_0 = \sqrt{-2\ln u_0}\,\cos(2\pi u_1), \qquad x_1 = \sqrt{-2\ln u_0}\,\sin(2\pi u_1), \tag{2.58}$$

by exploiting the fact that the two-dimensional Gaussian is separable in polar coordinates. When neither route is available, one falls back on acceptance-rejection methods or on the Markov chain machinery of the next section.

2.14 Markov chains and the Metropolis algorithm

Inverse transform sampling requires an invertible cumulative distribution, and in high dimensions we usually have neither that nor even a normalised distribution: models of interest frequently specify $p(\boldsymbol{x})$ only up to an unknown constant. Markov chain Monte Carlo solves this problem, and it underlies the Boltzmann machines, variational autoencoders and diffusion models of the later chapters of this book.

Markov chains.

A *Markov chain* is a sequence of states in which the probability of the next state depends only on the current one and not on the history,

$$\mathrm{Prob}(\boldsymbol{x}_{t+1} \mid \boldsymbol{x}_t, \boldsymbol{x}_{t-1}, \dots) = \mathrm{Prob}(\boldsymbol{x}_{t+1} \mid \boldsymbol{x}_t) \equiv W(\boldsymbol{x}_t \to \boldsymbol{x}_{t+1}). \tag{2.59}$$

For a finite state space the transition probabilities form a matrix $\boldsymbol{W}$ whose columns sum to unity, and the distribution after t steps is

$$\boldsymbol{w}(t) = \boldsymbol{W}^t \boldsymbol{w}(0), \tag{2.60}$$

which the reader will recognise as the power method of Section 1.17.2. Under mild conditions – the chain must be *irreducible*, able to reach any state from any other, and *aperiodic* – the dominant eigenvalue of $\boldsymbol{W}$ is unity and the chain converges to the corresponding eigenvector, the unique *stationary distribution* $\boldsymbol{w}^*$ with

$$\boldsymbol{W}\boldsymbol{w}^* = \boldsymbol{w}^*. \tag{2.61}$$

The rate of convergence is governed by the second eigenvalue, exactly as in Eq. (1.108), and the initial transient during which the chain forgets its starting point is the *burn-in* period, which must be discarded.

Detailed balance.

We want to run this backwards: given a target distribution $p(\boldsymbol{x})$, construct a chain whose stationary distribution is p. A sufficient condition is *detailed balance*,

$$p(\boldsymbol{x})\, W(\boldsymbol{x} \to \boldsymbol{x}') = p(\boldsymbol{x}')\, W(\boldsymbol{x}' \to \boldsymbol{x}), \tag{2.62}$$

which states that in equilibrium the flow of probability from $\boldsymbol{x}$ to $\boldsymbol{x}'$ balances the reverse flow. Summing Eq. (2.62) over $\boldsymbol{x}$ and using the normalisation of W gives $\sum_{\boldsymbol{x}} p(\boldsymbol{x}) W(\boldsymbol{x} \to \boldsymbol{x}') = p(\boldsymbol{x}')$, which is Eq. (2.61): any chain satisfying detailed balance has p as a stationary distribution.

The Metropolis algorithm.

It remains to construct such a W. Split each move into a *proposal* $T(\boldsymbol{x} \to \boldsymbol{x}')$, which we are free to choose, and an *acceptance probability* $A(\boldsymbol{x} \to \boldsymbol{x}')$, so that $W = TA$. With a symmetric proposal, $T(\boldsymbol{x} \to \boldsymbol{x}') = T(\boldsymbol{x}' \to \boldsymbol{x})$, detailed balance reduces to

$$\frac{A(\boldsymbol{x} \to \boldsymbol{x}')}{A(\boldsymbol{x}' \to \boldsymbol{x})} = \frac{p(\boldsymbol{x}')}{p(\boldsymbol{x})}, \tag{2.63}$$

and the Metropolis choice

$$\boxed{A(\boldsymbol{x} \to \boldsymbol{x}') = \min\left(1, \frac{p(\boldsymbol{x}')}{p(\boldsymbol{x})}\right)} \tag{2.64}$$

satisfies it. The algorithm is then simply: propose a move; if it increases the probability, accept it; if it decreases the probability, accept it anyway with probability $p(\boldsymbol{x}')/p(\boldsymbol{x})$; otherwise stay where you are and count the current state again.

The decisive feature of Eq. (2.64) is that only the *ratio* of probabilities appears. Any normalisation constant cancels, so we may sample from a distribution we can evaluate only up to a factor – which is exactly the situation for the Boltzmann distribution $p(\boldsymbol{x}) \propto e^{-E(\boldsymbol{x})/kT}$ of statistical physics, whose partition function is intractable, and for the posterior distributions of Bayesian inference, whose evidence integral is equally so. Dropping the symmetry requirement on T gives the more general Metropolis-Hastings algorithm, in which Eq. (2.64) acquires a ratio of proposal probabilities.

```python
import numpy as np

def metropolis(log_p, x0, n_steps, step_size=1.0, rng=None):
    """Sample from a distribution known up to a constant.

    log_p returns the logarithm of the unnormalised density, which is the
    numerically sensible way to handle the ratio in Eq. (2.metropolis).
    """
    rng = np.random.default_rng() if rng is None else rng
    x = np.atleast_1d(np.asarray(x0, dtype=float))
    chain = np.empty((n_steps, x.size))
    logp_x = log_p(x)
    accepted = 0

    for t in range(n_steps):
        proposal = x + step_size * rng.normal(size=x.size)   # symmetric proposal
        logp_new = log_p(proposal)
        # accept if log of the ratio exceeds the log of a uniform deviate
        if np.log(rng.random()) < logp_new - logp_x:
            x, logp_x = proposal, logp_new
            accepted += 1
        chain[t] = x

    return chain, accepted / n_steps


rng = np.random.default_rng(2024)
# Unnormalised standard Gaussian: the constant 1/sqrt(2 pi) is never needed
chain, rate = metropolis(lambda x: -0.5 * np.sum(x**2), x0=[0.0],
                         n_steps=100000, step_size=2.0, rng=rng)

burn_in = 1000
samples = chain[burn_in:, 0]
print(f"acceptance rate {rate:.3f}")
print(f"mean {samples.mean():.4f}  variance {samples.var():.4f}")
```

Two practical points are worth stressing, and both are statistics from earlier in this chapter. The step size controls the acceptance rate: too large and almost every move is rejected, too small and the chain crawls; rates in the range 0.2 to 0.5 are usually sought. And the samples produced are *correlated by construction*, since each state is a small perturbation of the last. The error on any expectation value estimated from a Metropolis run must therefore be computed with the autocorrelation time (2.36) and not with the naive $\sigma/\sqrt{n}$ of Eq. (2.24); with the step size above, the chain of the code fragment has an autocorrelation time of several steps, so the naive error would be too small by a factor of order $\sqrt{\tau}$.

Figure 2.5 shows both the chain and its output. The trace on the left makes the correlation visible directly: successive states are small perturbations of one another, so the walk wanders rather than jumping independently, and this is exactly what the correlation time quantifies. The histogram on the right shows that the stationary distribution is nevertheless the intended one, reproducing $\mathcal{N}(0,1)$ accurately although the normalisation of the target was never supplied to the algorithm.

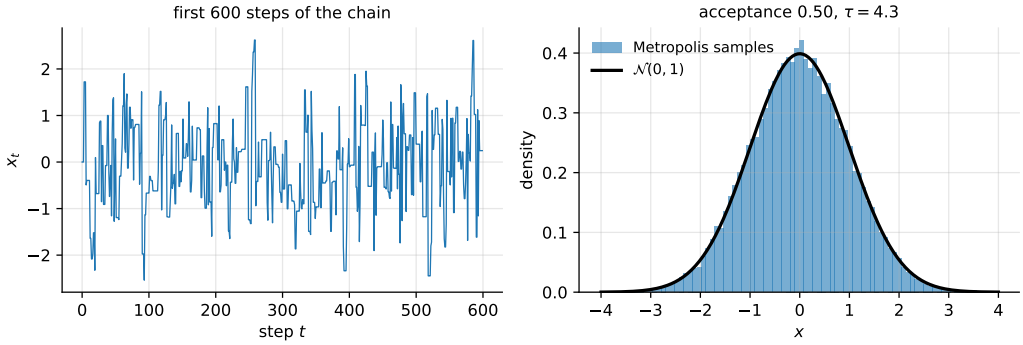


Fig. 2.5 Metropolis sampling of an unnormalised standard Gaussian, Eq. (2.64). Left: the first 600 states of the chain, showing the correlation between successive samples. Right: the sampled histogram against the exact density.

Machine learning connection. The Metropolis algorithm is the sampling engine behind several of the generative models later in this book. Training a restricted Boltzmann machine requires expectation values over the model distribution, which are estimated by Gibbs sampling – a variant in which the proposal is a conditional distribution and the acceptance probability is therefore always unity. Bayesian neural networks sample the posterior over weights by Hamiltonian Monte Carlo, a Metropolis method whose proposal follows the gradient of the log-density. And the denoising process of a diffusion model is a learned approximation to the reverse of a stochastic chain of exactly this type. In all three cases the acceptance criterion (2.64) is doing the same work: it lets us sample from a distribution we can score but cannot normalise.

2.15 Summary and the programs

The chapter has moved from probability to practice. The first part supplied the language: a stochastic variable is a domain together with a distribution, expectation values summarise that distribution through its moments, covariance measures linear dependence between variables, and the central limit theorem explains why averages become Gaussian and why their uncertainty falls as $1/\sqrt{n}$.

The second part turned to what we actually compute. Every quantity extracted from a finite sample is an estimator, hence a random variable, and it is characterised by a bias and a variance which combine into Eq. (2.27). Two warnings were issued there and both are easy to forget: the $1/\sqrt{n}$ law assumes independence, and when the data are correlated the effective sample size is n/τ with τ the autocorrelation time of Eq. (2.36); and an unbiased estimator is not automatically preferable to a biased one, since Eq. (2.27) counts variance equally. Applied to the least-squares estimator, the same machinery gave $\mathrm{Var}(\hat{\boldsymbol{\theta}}) = \sigma^2(\boldsymbol{X}^T\boldsymbol{X})^{-1}$, tying the statistical uncertainty of a fit directly to the conditioning of the design matrix discussed in Chapter 1.

The third part concerned models rather than numbers. The bias-variance decomposition (2.52) explains the characteristic shape of the test error as a function of model complexity, and identifies the irreducible noise floor beneath it. The bootstrap manufactures the ensemble of training sets that the decomposition presupposes but that we never possess, and cross-validation organises the train-test split so that every observation is used for both purposes without ever being used for both at once. A single methodological thread runs through all of it: anything learned from the data must be learned from the training data only.

The fourth part dealt with producing randomness rather than analysing it. Pseudo-random generators supply uniform deviates, from which other distributions follow by inverse transform sampling, and when neither the normalisation nor the inverse cumulative distribution is available, the Metropolis algorithm samples using ratios alone.

The complete programs are collected in `doc/BookML/BookPrograms`:

- `distributions.py` – the distributions of Section 2.2, with the plots reproduced here, and a numerical demonstration of the central limit theorem for several non-Gaussian starting distributions.
- `estimators.py` – sample moments, Bessel's correction, the autocorrelation function and the correlation time of Eq. (2.36), applied both to independent data and to a correlated Markov chain.
- `resampling.py` – the jackknife and the bootstrap, the bias-variance decomposition as a function of polynomial degree, and k-fold cross-validation for the selection of a Ridge penalty.
- `sampling.py` – linear congruential and modern generators, inverse transform and Box-Muller sampling, and the Metropolis algorithm with an autocorrelation analysis of its output.

Each file runs as a script and reproduces the numbers quoted in this chapter. An executable version of the same material is available as a Jupyter notebook in the accompanying Jupyterbook.

2.16 Exercises

Warm-up exercises

1. **Moments of familiar distributions.** (a) Show that the uniform distribution (2.6) on $[a,b]$ has mean $(a+b)/2$ and variance $(b-a)^2/12$. (b) Show that the exponential distribution (2.10) has mean $1/\alpha$ and variance $1/\alpha^2$. (c) Verify Eq. (2.16), $\mathrm{Var}(X) = \langle x^2\rangle - \langle x\rangle^2$, for both.
2. **Linearity and its limits.** Prove Eq. (2.17), and then show that for two variables

$$\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\mathrm{Cov}(X,Y).$$

Under what condition do variances simply add? Explain why this is the identity behind Eq. (2.24).

3. **Covariance without dependence, dependence without covariance.** (a) Let X be uniform on $[-1,1]$ and $Y = X^2$. Compute $\mathrm{Cov}(X,Y)$ and comment. (b) Construct two variables that are uncorrelated but not independent, other than this example. (c) Explain what this implies for a feature-selection procedure based on the correlation matrix alone.
4. **The central limit theorem numerically.** Draw m values from (a) a uniform distribution, (b) an exponential distribution, and (c) a Cauchy distribution $p(x) = 1/[\pi(1+x^2)]$, and histogram the average over many repetitions for $m = 1, 2, 10, 100$. Two of the three converge to a Gaussian with width $\sigma/\sqrt{m}$. Which does not, and why does the theorem not apply?
5. **Bessel's correction.** Prove Eq. (2.28). Then verify it numerically by drawing many samples of size $n = 5$ from a known distribution and comparing the average of the two estimators with the true variance.
6. **Bias and variance of an estimator.** Consider estimating the variance of a Gaussian by $\hat{\sigma}^2_c = c\sum_k (x_k - \bar{x})^2$ for a constant c. (a) Find the c that makes the estimator unbiased. (b) Find the c that minimises the mean squared error (2.27). (c) Are they the same? Comment on what this says about the value of unbiasedness.
7. **Autocorrelation (numerical).** Generate a correlated sequence by the autoregressive rule $x_{t+1} = \phi x_t + \eta_t$ with η_t standard normal and $\phi = 0.9$. (a) Estimate the mean and its naive standard error (2.24). (b) Compute the autocorrelation function (2.35) and the correlation time τ. (c) By how much was the naive error underestimated? Compare with the prediction $\tau = (1+\phi)/(1-\phi)$.
8. **Variance of the least-squares estimator (numerical).** Generate data from Eq. (2.37) with known $\boldsymbol{\theta}$ and σ. (a) Fit $\hat{\boldsymbol{\theta}}$ many times with fresh noise and compute the empirical covariance of the estimates. (b) Compare with the prediction $\sigma^2(\boldsymbol{X}^T\boldsymbol{X})^{-1}$ of Eq. (2.47). (c) Repeat with two nearly collinear columns in $\boldsymbol{X}$ and relate the result to the singular values, using Eq. (1.122).
9. **Bootstrap of a statistic without a formula.** For a sample from a skewed distribution of your choice, use the bootstrap to estimate the standard error of (a) the mean, (b) the median, and (c) the ratio of the mean to the standard deviation. For which of the three could you have obtained the answer analytically?
10. **Cross-validation and leakage (numerical).** Take a data set with $n = 100$ observations and $p = 5000$ pure-noise features, entirely unrelated to the target. (a) Select the 100 features most correlated with the target using all the data, then cross-validate a model built on them. Report the estimated error. (b) Now perform the same selection *inside* each cross-validation fold. Report the estimated error again. (c) Explain the discrepancy in terms of the notebox of Section 2.12.

Exercise session: from probability to resampling

Exercise 1: expectation values and the covariance matrix.

Let $\boldsymbol{X} = (X_0, \ldots, X_{p-1})^T$ be a vector of stochastic variables with mean $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$, and let $\boldsymbol{A}$ be a constant $q \times p$ matrix.

a) Show that $\mathbb{E}[\boldsymbol{A}\boldsymbol{X}] = \boldsymbol{A}\boldsymbol{\mu}$.
b) Show that the covariance matrix of $\boldsymbol{A}\boldsymbol{X}$ is $\boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^T$. This is the matrix generalisation of $\mathrm{Var}(aX) = a^2\mathrm{Var}(X)$ in Eq. (2.17).
c) Use part (b) with $\boldsymbol{A} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T$ and noise covariance $\sigma^2\boldsymbol{I}$ to rederive Eq. (2.47) in two lines.

d) Show that $\boldsymbol{\Sigma}$ is positive semi-definite by considering $\mathrm{Var}(\boldsymbol{z}^T\boldsymbol{X})$ for an arbitrary constant vector $\boldsymbol{z}$.
e) Using the spectral decomposition (1.54) of $\boldsymbol{\Sigma}$, show that the whitening transformation (1.59) produces variables with covariance matrix equal to the identity.

Exercise 2: the bias-variance decomposition.

a) Derive Eq. (2.27) for a scalar estimator, being explicit about why the cross term vanishes.
b) Repeat the derivation for the prediction case, Eq. (2.52), stating clearly which quantities the expectation is taken over and which of $\boldsymbol{f}$, $\boldsymbol{\varepsilon}$ and $\tilde{\boldsymbol{y}}$ are stochastic.
c) Explain why the term σ^2 cannot be reduced by any model, and what it would mean to observe a test error below it.
d) A colleague reports that their model has zero bias and concludes that it is therefore optimal. Using Eq. (2.27), explain what they have overlooked, and relate your answer to the Ridge shrinkage factors (1.130).
e) (Numerical.) Reproduce the polynomial-degree sweep of Section 2.10 and identify the degree minimising the test error. Then repeat with ten times as many data points, and explain the shift in the minimum in terms of the two competing terms.

Exercise 3: resampling and sampling.

a) Implement the independent bootstrap from scratch, without using `sklearn.utils.resample`, and verify on Gaussian data that the bootstrap standard error of the mean agrees with Eq. (2.24).
b) Show that the probability of a given observation *not* appearing in a bootstrap resample of size n tends to $1/e \approx 0.368$ as n grows. These left-out observations are the "out-of-bag" sample used by random forests.
c) Implement k-fold cross-validation from scratch and compare its estimate of the test error with that of a single train-test split, repeated many times. Which has the smaller variance, and why?
d) Implement inverse transform sampling (2.57) for the exponential distribution and verify the resulting histogram against Eq. (2.10).
e) Use the Metropolis algorithm (2.64) to sample from the unnormalised density $p(x) \propto \exp(-x^4/4)$. Estimate $\langle x^2 \rangle$, and give an honest error bar using the correlation time of Eq. (2.36) rather than Eq. (2.24). Study how the acceptance rate and the correlation time vary with the step size.

Part II
Classical methods

Chapter 3
Linear Regression

Linear regression is the oldest method in this book and, for the purposes of learning the subject, by far the most valuable. It is the only family of models for which every question we shall want to ask of a machine learning method can be answered in closed form: we can write down the optimal parameters, their bias, their variance, their sensitivity to the data, and the exact effect of regularisation, all with the linear algebra of Chapter 1 and the statistics of Chapter 2. Everything that follows in this book – logistic regression, neural networks, tree ensembles – introduces complications that make one or more of these questions intractable. It is worth understanding thoroughly the one case where nothing is hidden.

The chapter proceeds from the general to the particular. We set up the linear model and derive ordinary least squares from three independent starting points – as an optimisation problem, as an orthogonal projection, and as maximum likelihood under Gaussian noise – and then examine the statistical properties of the resulting estimator. We then add regularisation, first the 2-norm penalty that gives Ridge regression and then the 1-norm penalty that gives the Lasso, and show that the two behave very differently for reasons that are entirely geometric. A Bayesian reading unifies all three as maximum-a-posteriori estimates under different priors. We close with the practical questions of scaling and the intercept, with the procedure by which the penalty is actually chosen – cross-validation, done properly, and compared with the bootstrap – and with a complete worked analysis of the Franke function.

3.1 The regression problem

We are given n observations of p features, collected in the design matrix $\boldsymbol{X} \in \mathbb{R}^{n \times p}$ of Eq. (1.1), together with n targets $\boldsymbol{y} \in \mathbb{R}^n$. We assume the targets are generated by

$$\boldsymbol{y} = f(\boldsymbol{x}) + \boldsymbol{\varepsilon}, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2), \tag{3.1}$$

with f an unknown function and $\boldsymbol{\varepsilon}$ independent noise, exactly as in Eq. (2.49). Our task is to construct an approximation $\tilde{\boldsymbol{y}}$ to f from the data.

What makes a model good?

It is worth being explicit, because the answer is not obvious and the obvious answer is wrong. A good model is *not* one that reproduces the training data accurately; by Section 2.9 that is trivial to achieve and tells us nothing. A good model is one that predicts well on data it has not seen, and by the decomposition (2.52) its expected error on such data is the sum of a

squared bias, a variance and an irreducible noise term. The entire content of this chapter is the management of the first two.

Our approach throughout is *frequentist*: we treat the parameters $\boldsymbol{\theta}$ as fixed but unknown quantities and the data as random, so that statements about uncertainty are statements about what would happen if the data were collected again. Section 3.11 shows what the same methods look like from a Bayesian viewpoint, in which the parameters are random and the data are fixed, and the two readings turn out to produce identical formulae from different premises.

3.2 The linear model and the design matrix

The model we shall study throughout is linear in the parameters,

$$\tilde{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\theta}, \qquad \tilde{y}_i = \sum_{j=0}^{p-1} x_{ij}\theta_j = \boldsymbol{X}_{i,*}\boldsymbol{\theta}, \tag{3.2}$$

with $\boldsymbol{X}_{i,*}$ the ith row. The word *linear* refers to the parameters and not to the features, and this distinction is the source of most of the method's usefulness. Nothing prevents the columns of $\boldsymbol{X}$ from being non-linear functions of some underlying variable. If we have a single input x and choose

$$\boldsymbol{X} = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{p-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{p-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{p-1} \end{bmatrix}, \tag{3.3}$$

then Eq. (3.2) is a polynomial fit of degree $p-1$, and it remains a *linear* regression problem. More generally, replacing x by any set of basis functions $\phi_j(x)$ – polynomials, splines, Fourier modes, radial basis functions – gives

$$\tilde{y}_i = \sum_{j=0}^{p-1} \theta_j \, \phi_j(x_i), \tag{3.4}$$

which is a *basis expansion*, and every method of this chapter applies verbatim. The matrix (3.3) is the Vandermonde matrix whose appalling conditioning we met in Section 1.12; the choice of basis is therefore not only a modelling decision but a numerical one, and an orthogonal polynomial basis is very much better behaved than the monomials.

The first column of ones deserves a name. Its coefficient θ_0 is the *intercept*, the predicted value when every other feature vanishes, and it will require separate treatment when we come to regularisation in Section 3.13.

Complexity.

The number of columns p measures the flexibility of the model, and it is the knob we turn when tracing out the bias-variance curve of Section 2.10. For a polynomial in one variable of degree d we have $p = d+1$; for a polynomial of degree d in two variables x and y, which is the case we shall need for the Franke function, every monomial $x^a y^b$ with $a+b \le d$ appears and

$$p = \frac{(d+1)(d+2)}{2}. \tag{3.5}$$

A fifth-order fit in two variables therefore has 21 parameters, and a tenth-order fit has 66: complexity grows quickly, and with it the danger that p approaches or exceeds n.

```python
import numpy as np

def design_matrix_2d(x, y, degree):
    """Design matrix for a two-dimensional polynomial of the given degree.

    Columns are the monomials x^a y^b with a + b <= degree, ordered by
    total degree.  The first column is the intercept.
    """
    x, y = np.ravel(x), np.ravel(y)
    columns = []
    for total in range(degree + 1):
        for b in range(total + 1):
            columns.append(x**(total - b) * y**b)
    return np.column_stack(columns)


# A degree-5 fit in two variables has (5+1)(5+2)/2 = 21 parameters
X = design_matrix_2d(np.random.rand(100), np.random.rand(100), degree=5)
print(X.shape)
```

3.3 Ordinary least squares

The cost function of ordinary least squares (OLS) is the mean squared error of Eq. (1.34),

$$C(\boldsymbol{\theta}) = \frac{1}{n}\,\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2 = \frac{1}{n}\sum_{i=0}^{n-1}\left(y_i - \tilde{y}_i\right)^2, \tag{3.6}$$

and the optimisation problem is

$$\min_{\boldsymbol{\theta}\in\mathbb{R}^p} \frac{1}{n}\,\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2. \tag{3.7}$$

We derived the solution in Section 1.8.4: setting the gradient (1.38) to zero gives the normal equations

$$\boldsymbol{X}^T\boldsymbol{X}\,\boldsymbol{\theta} = \boldsymbol{X}^T\boldsymbol{y}, \qquad \hat{\boldsymbol{\theta}}_{\mathrm{OLS}} = \left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T\boldsymbol{y}, \tag{3.8}$$

with Hessian $2\boldsymbol{X}^T\boldsymbol{X}/n$, positive semi-definite by Eq. (1.44), so that the problem is convex and the stationary point is a global minimum – unique precisely when the columns of $\boldsymbol{X}$ are linearly independent.

The geometric reading.

Equation (3.8) has an interpretation which is worth as much as the algebra. The vector $\boldsymbol{X}\boldsymbol{\theta}$ ranges over the column space of $\boldsymbol{X}$ as $\boldsymbol{\theta}$ ranges over $\mathbb{R}^p$, so Eq. (3.7) asks for the point of that subspace closest to $\boldsymbol{y}$. By Section 1.6 the answer is the orthogonal projection of $\boldsymbol{y}$ onto the column space, and the projector is the *hat matrix*

$$\boldsymbol{H} = \boldsymbol{X}\left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T, \qquad \tilde{\boldsymbol{y}} = \boldsymbol{H}\boldsymbol{y}, \tag{3.9}$$

which is symmetric and idempotent as required by Eq. (1.9), with $\mathrm{Tr}(\boldsymbol{H}) = p$ by Eq. (1.10). Equivalently, the normal equations (3.8) may be read as $\boldsymbol{X}^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}) = \boldsymbol{0}$: at the optimum the

residual is orthogonal to every column of the design matrix. The fit has extracted from $\boldsymbol{y}$ everything that lies in the span of the features and left the rest untouched.

Optimality without calculus.

Setting a gradient to zero shows that $\hat{\boldsymbol{\theta}}$ is a stationary point, and convexity then makes it a minimum. There is a shorter route which gives more, and which will be reused for Ridge regression, for the Lasso and for the kernel methods of Section 3.12.

Proposition 3.1 (Least-squares decomposition). *Let $\hat{\boldsymbol{\theta}}$ be any solution of Eq. (3.8), the normal equations. Then for every $\boldsymbol{\theta}$ in $\mathbb{R}^p$,*

$$\|\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}\|_2^2 = \underbrace{\left\|\boldsymbol{y}-\boldsymbol{X}\hat{\boldsymbol{\theta}}\right\|_2^2}_{\text{residual sum of squares}} + \underbrace{\left(\boldsymbol{\theta}-\hat{\boldsymbol{\theta}}\right)^T \boldsymbol{X}^T\boldsymbol{X}\left(\boldsymbol{\theta}-\hat{\boldsymbol{\theta}}\right)}_{\geq 0}. \tag{3.10}$$

Proof. Write $\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta} = (\boldsymbol{y}-\boldsymbol{X}\hat{\boldsymbol{\theta}}) - \boldsymbol{X}(\boldsymbol{\theta}-\hat{\boldsymbol{\theta}})$ and expand the square:

$$\|\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}\|_2^2 = \left\|\boldsymbol{y}-\boldsymbol{X}\hat{\boldsymbol{\theta}}\right\|_2^2 - 2\left(\boldsymbol{\theta}-\hat{\boldsymbol{\theta}}\right)^T\boldsymbol{X}^T\left(\boldsymbol{y}-\boldsymbol{X}\hat{\boldsymbol{\theta}}\right) + \left(\boldsymbol{\theta}-\hat{\boldsymbol{\theta}}\right)^T\boldsymbol{X}^T\boldsymbol{X}\left(\boldsymbol{\theta}-\hat{\boldsymbol{\theta}}\right).$$

The middle term vanishes identically, because $\boldsymbol{X}^T(\boldsymbol{y}-\boldsymbol{X}\hat{\boldsymbol{\theta}}) = \boldsymbol{0}$ is precisely the normal equations. The last term is a quadratic form in the positive semi-definite matrix $\boldsymbol{X}^T\boldsymbol{X}$ and is therefore non-negative. □

Three consequences follow at once, and it is worth being explicit about them because each is used later.

First, $\hat{\boldsymbol{\theta}}$ is a global minimiser, with no appeal to convexity or to second derivatives: the second term of Eq. (3.10) is non-negative and vanishes at $\boldsymbol{\theta} = \hat{\boldsymbol{\theta}}$. *Second*, the minimiser is unique if and only if $\boldsymbol{X}^T\boldsymbol{X}$ is positive definite, that is if and only if the columns of $\boldsymbol{X}$ are linearly independent; otherwise the second term vanishes on the whole null space of $\boldsymbol{X}$ and every point of an affine subspace attains the minimum. Note carefully what remains unique in that case: the *fit* $\boldsymbol{X}\hat{\boldsymbol{\theta}}$, since two minimisers differ by a vector in the null space of $\boldsymbol{X}$. This distinction between a non-unique parameter and a unique prediction will return for the Lasso in Proposition 3.9. *Third*, the cost rises away from the optimum at a rate set by the eigenvalues of $\boldsymbol{X}^T\boldsymbol{X}$: sharply in the directions of large singular value and almost not at all in the directions of small ones. That is the same statement as the variance formula of Section 3.7 and as the convergence rate of Chapter 4; a flat valley is a direction that the data do not determine, and all three chapters are describing it in their own vocabulary.

How to compute it.

Equation (3.8) is a statement about $\hat{\boldsymbol{\theta}}$ and not an instruction to a computer. Forming $\boldsymbol{X}^T\boldsymbol{X}$ squares the condition number, Eq. (1.118), and for a polynomial design matrix of even modest degree this destroys most of the available precision. The stable routes are the QR decomposition (1.12) or the singular value decomposition, and through the latter the solution is the pseudoinverse of Eq. (1.121),

$$\hat{\boldsymbol{\theta}}_{\mathrm{OLS}} = \boldsymbol{X}^{+}\boldsymbol{y} = \sum_{i=0}^{r-1} \frac{\boldsymbol{u}_i^T\boldsymbol{y}}{\sigma_i}\,\boldsymbol{v}_i, \tag{3.11}$$

which additionally handles the rank-deficient case, returning the minimum-norm solution when $p > n$ or when features are exactly collinear. Equation (3.11) will be our main ana-

lytical tool in this chapter: almost everything that follows is a statement about what happens to the factors $1/\sigma_i$.

```python
import numpy as np

def ols(X, y):
    """Ordinary least squares through the SVD -- never the normal equations."""
    U, s, Vt = np.linalg.svd(X, full_matrices=False)
    return Vt.T @ (U.T @ y / s)


def ols_normal_equations(X, y):
    """The textbook formula.  Shown for comparison; do not use it."""
    return np.linalg.pinv(X.T @ X) @ X.T @ y
```

Machine learning connection. The two functions above agree to machine precision on a well-conditioned design matrix and can disagree in the first significant digit on a badly conditioned one. A simple experiment makes the point: fit a polynomial of degree twelve to a hundred points on $[0,1]$ using both. The condition number of the Vandermonde matrix (3.3) is then of order 10^{8}, that of $\boldsymbol{X}^{T}\boldsymbol{X}$ of order 10^{16}, and the second function returns coefficients with no correct digits at all while the first is still accurate. This is the practical content of Section 1.18, and it is why `numpy.linalg.lstsq` exists.

3.4 Weighted least squares and the χ^2 function

The cost function (3.6) treats every observation as equally reliable. In the physical sciences this is rarely true: a measurement is normally accompanied by an error estimate, and points known to ten per cent should not constrain the fit as strongly as points known to one per cent. Introducing the standard deviation σ_i of measurement i, we define the χ^2 function

$$\chi^2(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=0}^{n-1}\frac{(y_i - \tilde{y}_i)^2}{\sigma_i^2} = \frac{1}{n}\,(\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta})^{T}\,\boldsymbol{\Sigma}^{-2}\,(\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}), \tag{3.12}$$

where $\boldsymbol{\Sigma}$ is the diagonal matrix with entries σ_i. Each residual is measured in units of its own uncertainty, so that the terms of the sum are comparable.

Minimising Eq. (3.12) is the same calculation as before with a weight inserted. Differentiating with the machinery of Section 1.8.3 gives

$$\frac{\partial \chi^2}{\partial \boldsymbol{\theta}^{T}} = -\frac{2}{n}\boldsymbol{X}^{T}\boldsymbol{\Sigma}^{-2}\,(\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}) = \boldsymbol{0}, \tag{3.13}$$

and hence the *weighted* normal equations

$$\hat{\boldsymbol{\theta}} = \left(\boldsymbol{X}^{T}\boldsymbol{\Sigma}^{-2}\boldsymbol{X}\right)^{-1}\boldsymbol{X}^{T}\boldsymbol{\Sigma}^{-2}\boldsymbol{y}. \tag{3.14}$$

Defining $\boldsymbol{A} = \boldsymbol{\Sigma}^{-1}\boldsymbol{X}$ and $\boldsymbol{b} = \boldsymbol{\Sigma}^{-1}\boldsymbol{y}$ turns Eq. (3.14) into the ordinary solution (3.8) for $\boldsymbol{A}$ and $\boldsymbol{b}$: weighted least squares is unweighted least squares on rescaled data. Setting all σ_i equal recovers OLS, which shows what the unweighted method silently assumes – that the noise is *homoscedastic*, of the same variance everywhere. When it is not, OLS remains unbiased but is no longer the estimator of smallest variance, and Eq. (3.14) is.

Correlated noise: generalised least squares.

The same rescaling handles the general case in which the noise has an arbitrary covariance matrix, $\mathrm{Var}(\boldsymbol{\varepsilon}) = \boldsymbol{\Omega}$, symmetric and positive definite. Factor it as $\boldsymbol{\Omega} = \boldsymbol{L}\boldsymbol{L}^T$ by the Cholesky decomposition of Section 1.14 and multiply the model $\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\theta} + \boldsymbol{\varepsilon}$ by $\boldsymbol{L}^{-1}$: the transformed noise $\boldsymbol{L}^{-1}\boldsymbol{\varepsilon}$ has covariance $\boldsymbol{L}^{-1}\boldsymbol{\Omega}\boldsymbol{L}^{-T} = \boldsymbol{I}$, so it is homoscedastic and uncorrelated, and OLS on the *whitened* data $\boldsymbol{A} = \boldsymbol{L}^{-1}\boldsymbol{X}$, $\boldsymbol{b} = \boldsymbol{L}^{-1}\boldsymbol{y}$ gives

$$\hat{\boldsymbol{\theta}}_{\mathrm{GLS}} = \left(\boldsymbol{X}^T\boldsymbol{\Omega}^{-1}\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T\boldsymbol{\Omega}^{-1}\boldsymbol{y}, \qquad \mathrm{Var}\left(\hat{\boldsymbol{\theta}}_{\mathrm{GLS}}\right) = \left(\boldsymbol{X}^T\boldsymbol{\Omega}^{-1}\boldsymbol{X}\right)^{-1}, \tag{3.15}$$

the second statement following from $\mathrm{Var}(\boldsymbol{A}^{+}\boldsymbol{b}) = \boldsymbol{A}^{+}\boldsymbol{I}(\boldsymbol{A}^{+})^T = (\boldsymbol{A}^T\boldsymbol{A})^{-1}$. Weighted least squares is the diagonal case $\boldsymbol{\Omega} = \boldsymbol{\Sigma}^2$, and its variance is $(\boldsymbol{X}^T\boldsymbol{\Sigma}^{-2}\boldsymbol{X})^{-1}$. That this is the smallest variance attainable by any linear unbiased estimator is the Gauss-Markov theorem of Section 3.7 applied to the whitened problem, for which its hypotheses hold exactly.

The value of χ^2 at the minimum.

A physicist reports not only the fitted parameters but the minimum value of χ^2, and reads it as a test of the model. The reason is a small calculation. Write $\boldsymbol{b} = \boldsymbol{A}\boldsymbol{\theta} + \boldsymbol{\eta}$ for the whitened data, with $\mathrm{Var}(\boldsymbol{\eta}) = \boldsymbol{I}$, and let $\boldsymbol{H}_A = \boldsymbol{A}(\boldsymbol{A}^T\boldsymbol{A})^{-1}\boldsymbol{A}^T$ be its hat matrix. The whitened residual is $\boldsymbol{b} - \boldsymbol{A}\hat{\boldsymbol{\theta}} = (\boldsymbol{I} - \boldsymbol{H}_A)\boldsymbol{b} = (\boldsymbol{I} - \boldsymbol{H}_A)\boldsymbol{\eta}$, because $(\boldsymbol{I} - \boldsymbol{H}_A)\boldsymbol{A} = \boldsymbol{0}$ removes the model exactly. Then, using $\mathbb{E}[\boldsymbol{\eta}^T\boldsymbol{M}\boldsymbol{\eta}] = \mathrm{Tr}(\boldsymbol{M}\,\mathrm{Var}(\boldsymbol{\eta})) = \mathrm{Tr}(\boldsymbol{M})$ for any fixed matrix $\boldsymbol{M}$ and the projector properties (1.9) and (1.10),

$$\mathbb{E}\left[n\chi^2_{\min}\right] = \mathbb{E}\left[\boldsymbol{\eta}^T(\boldsymbol{I} - \boldsymbol{H}_A)^2\boldsymbol{\eta}\right] = \mathrm{Tr}\left(\boldsymbol{I} - \boldsymbol{H}_A\right) = n - p. \tag{3.16}$$

If the model is right and the σ_i are right, the sum of squared standardised residuals is on average the number of data points minus the number of fitted parameters, so the *reduced* χ^2, $n\chi^2_{\min}/(n-p)$, should be close to one. A value well above one says that the model does not describe the data at the stated precision; a value well below one says that the error bars were overestimated. Under Gaussian noise $n\chi^2_{\min}$ has exactly the χ^2 distribution with $n-p$ degrees of freedom, which Proposition 3.3 will establish, and the statement can then be made quantitative. A short simulation confirms Eq. (3.16) and the variance in Eq. (3.15):

```
import numpy as np

rng = np.random.default_rng(2024)
n, p = 50, 3
x = np.linspace(0, 1, n)
X = np.c_[np.ones(n), x, x**2]
theta_true = np.array([1.0, -2.0, 3.0])
sigmas = 0.05 + 0.3 * x                    # heteroscedastic: error grows with x

chi2_min, theta_wls, theta_ols = [], [], []
for trial in range(20000):
    y = X @ theta_true + sigmas * rng.normal(size=n)
    A, b = X / sigmas[:, None], y / sigmas   # whitened problem, Eq. (3.chisquared)
    th = np.linalg.lstsq(A, b, rcond=None)[0]
    chi2_min.append(np.sum((b - A @ th)**2) / n)
    theta_wls.append(th)
    theta_ols.append(np.linalg.lstsq(X, y, rcond=None)[0])

print("mean of n*chi2_min:", n * np.mean(chi2_min), "   n - p =", n - p)
print("variance of WLS  :", np.var(theta_wls, axis=0))
print("predicted, (X^T S^-2 X)^-1:", np.diag(np.linalg.inv(X.T @ (X / sigmas[:, None]**2))))
print("variance of OLS  :", np.var(theta_ols, axis=0))
```

```
mean of n∗chi2_min: 47.07    n - p = 47
variance of WLS  : [0.00090 0.05706 0.09014]
predicted, (X^T S^-2 X)^-1: [0.00088 0.05677 0.08989]
variance of OLS  : [0.00244 0.13275 0.17519]
```

The unweighted estimator is unbiased here too, but its variance is between two and three times larger: it lets the noisy points at large x pull the fit as hard as the precise ones at small x.

3.5 Measures of quality

Before comparing models we must agree on how to score them. The mean squared error

$$\mathrm{MSE}(\boldsymbol{y},\tilde{\boldsymbol{y}}) = \frac{1}{n}\sum_{i=0}^{n-1}(y_i-\tilde{y}_i)^2 \tag{3.17}$$

is the quantity we minimise and the natural quantity to report, but it carries the squared units of the target and its numerical value therefore says nothing on its own. The *coefficient of determination*

$$R^2(\boldsymbol{y},\tilde{\boldsymbol{y}}) = 1-\frac{\sum_{i=0}^{n-1}(y_i-\tilde{y}_i)^2}{\sum_{i=0}^{n-1}(y_i-\bar{y})^2}, \qquad \bar{y}=\frac{1}{n}\sum_{i=0}^{n-1}y_i, \tag{3.18}$$

is dimensionless and compares the model against the best constant prediction: $R^2=1$ is a perfect fit, $R^2=0$ means the model does no better than predicting the mean, and negative values – entirely possible on test data – mean it does worse. Note that the denominator is n times the sample variance of $\boldsymbol{y}$, so that R^2 is one minus the fraction of variance left unexplained.

Where R^2 comes from.

The definition (3.18) looks like a convention, but for a least-squares fit with an intercept it is forced by geometry, and the derivation explains both its range and its most abused property. Write $\boldsymbol{e}=\boldsymbol{y}-\tilde{\boldsymbol{y}}=(\boldsymbol{I}-\boldsymbol{H})\boldsymbol{y}$ for the residual. Since $\boldsymbol{H}$ is an orthogonal projector, $\tilde{\boldsymbol{y}}=\boldsymbol{H}\boldsymbol{y}$ and $\boldsymbol{e}$ are orthogonal and Pythagoras gives

$$\|\boldsymbol{y}\|_2^2 = \|\boldsymbol{H}\boldsymbol{y}\|_2^2 + \|(\boldsymbol{I}-\boldsymbol{H})\boldsymbol{y}\|_2^2 = \|\tilde{\boldsymbol{y}}\|_2^2 + \|\boldsymbol{e}\|_2^2. \tag{3.19}$$

Now suppose the design matrix contains the constant column $\mathbf{1}$, as it does whenever an intercept is fitted. The normal equations $\boldsymbol{X}^T\boldsymbol{e}=\boldsymbol{0}$ then include the row $\mathbf{1}^T\boldsymbol{e}=0$: the residuals sum to zero, so $\bar{\tilde{y}}=\bar{y}$, and the vector $\boldsymbol{y}-\bar{y}\mathbf{1}$ decomposes as $(\tilde{\boldsymbol{y}}-\bar{y}\mathbf{1})+\boldsymbol{e}$ with the two parts again orthogonal, because $\boldsymbol{e}\perp\tilde{\boldsymbol{y}}$ and $\boldsymbol{e}\perp\mathbf{1}$. Applying Pythagoras once more,

$$\underbrace{\sum_i (y_i-\bar{y})^2}_{\text{TSS}} = \underbrace{\sum_i (\tilde{y}_i-\bar{y})^2}_{\text{ESS}} + \underbrace{\sum_i (y_i-\tilde{y}_i)^2}_{\text{RSS}}, \tag{3.20}$$

the *analysis-of-variance identity*: the total sum of squares splits exactly into a part explained by the fit and a residual part. Dividing by the left-hand side,

$$R^2 = 1-\frac{\mathrm{RSS}}{\mathrm{TSS}} = \frac{\mathrm{ESS}}{\mathrm{TSS}} = \cos^2\!\angle(\boldsymbol{y}-\bar{y}\mathbf{1},\ \tilde{\boldsymbol{y}}-\bar{y}\mathbf{1}), \tag{3.21}$$

so on the training data R^2 is the squared cosine of the angle between the centred targets and the centred fit, and lies in $[0,1]$. Both halves of that statement fail off the training set: for test data the residuals do not sum to zero and are not orthogonal to the fit, the identity (3.20) breaks, and R^2 can be negative.

The identity also proves the warning below. Let $\boldsymbol{X}' = [\boldsymbol{X}\ \boldsymbol{z}]$ be the design matrix with one further column. Every $\boldsymbol{\theta}$ available to $\boldsymbol{X}$ is available to $\boldsymbol{X}'$ with a zero in the last place, so $\min_{\boldsymbol{\theta}'} \|\boldsymbol{y} - \boldsymbol{X}'\boldsymbol{\theta}'\|^2 \le \min_{\boldsymbol{\theta}} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|^2$: the training RSS can only fall when a column is added, and by Eq. (3.21) the training R^2 can only rise. When $p = n$ the column space is all of $\mathbb{R}^n$, $\boldsymbol{H} = \boldsymbol{I}$, the RSS is zero and $R^2 = 1$ whatever the columns contain. The *adjusted* coefficient

$$\bar{R}^2 = 1 - \frac{\mathrm{RSS}/(n-p)}{\mathrm{TSS}/(n-1)} = 1 - \left(1 - R^2\right)\frac{n-1}{n-p} \tag{3.22}$$

replaces the two sums of squares by unbiased variance estimates – the divisor $n - p$ will be justified in Proposition 3.2 – and can decrease when a useless column is added, but it is a palliative and not a cure: only data the model has not seen measure what we actually care about.

Two further measures appear in the sources of this book. The *mean absolute error*

$$\mathrm{MAE}(\boldsymbol{y}, \tilde{\boldsymbol{y}}) = \frac{1}{n}\sum_{i=0}^{n-1} |y_i - \tilde{y}_i| \tag{3.23}$$

is the 1-norm counterpart of Eq. (3.17) and is far less sensitive to outliers, for the reason discussed in Section 1.12: squaring gives a point at ten standard deviations a hundred times the weight of one at a single standard deviation, whereas the absolute value gives it ten times. The *relative error* $|y_i - \tilde{y}_i|/|y_i|$ is natural when the targets span orders of magnitude, and dangerous when any of them is near zero.

```
import numpy as np

def mse(y, y_tilde):
    return np.mean((y - y_tilde)**2)

def r2(y, y_tilde):
    return 1.0 - np.sum((y - y_tilde)**2) / np.sum((y - np.mean(y))**2)

def mae(y, y_tilde):
    return np.mean(np.abs(y - y_tilde))
```

A warning is in order. All three are meaningful only on data held out from fitting. Computed on the training set, R^2 increases monotonically as columns are added to $\boldsymbol{X}$ and reaches unity when $p = n$, whatever those columns contain – including pure noise. Every number reported in this chapter is a test quantity unless we say otherwise.

3.6 Deriving least squares from a probability distribution

So far the squared-error cost (3.6) has been an assumption. We now show that it follows from a statement about the noise, which both explains where it comes from and makes clear when it is the wrong choice.

Under the model (3.1), and given that the entries of the design matrix are not stochastic, the target y_i is Gaussian with mean $\boldsymbol{X}_{i,*}\boldsymbol{\theta}$ and variance σ^2,

$$y_i \sim \mathcal{N}\left(\boldsymbol{X}_{i,*}\boldsymbol{\theta},\, \sigma^2\right) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(y_i - \boldsymbol{X}_{i,*}\boldsymbol{\theta})^2}{2\sigma^2}\right], \tag{3.24}$$

which is Eq. (2.39) written out in full. Reading Eq. (3.24) as a function of the parameters rather than of the data gives the *likelihood* of observing y_i given $\boldsymbol{\theta}$, and since the observations are independent and identically distributed the likelihood of the whole data set $\boldsymbol{D}$ is the product

$$p(\boldsymbol{D} \mid \boldsymbol{\theta}) = \prod_{i=0}^{n-1} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(y_i - \boldsymbol{X}_{i,*}\boldsymbol{\theta})^2}{2\sigma^2}\right], \tag{3.25}$$

where $\boldsymbol{D}$ denotes the domain of events, inputs and targets together; in the one-dimensional case $\boldsymbol{D} = [(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})]$.

Maximum likelihood estimation chooses the parameters making the observed data most probable, that is maximising Eq. (3.25). Differentiating a product of n terms is awkward and numerically hazardous, since the product of many small numbers underflows. Because the logarithm is monotonically increasing, maximising the likelihood is equivalent to maximising its logarithm, and we prefer to minimise the negative of that:

$$C(\boldsymbol{\theta}) = -\log \prod_{i=0}^{n-1} p(y_i, \boldsymbol{X} \mid \boldsymbol{\theta}) = -\sum_{i=0}^{n-1} \log p(y_i, \boldsymbol{X} \mid \boldsymbol{\theta}). \tag{3.26}$$

Inserting Eq. (3.24) the sum evaluates to

$$C(\boldsymbol{\theta}) = \frac{n}{2}\log\left(2\pi\sigma^2\right) + \frac{\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2}{2\sigma^2}. \tag{3.27}$$

The first term does not involve $\boldsymbol{\theta}$, and the second is the least-squares cost (3.6) up to a positive constant. Differentiating therefore returns $\boldsymbol{X}^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}) = \boldsymbol{0}$ and

$$\hat{\boldsymbol{\theta}}_{\mathrm{OLS}} = \left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T\boldsymbol{y} \tag{3.28}$$

once more.

The noise variance by maximum likelihood.

The likelihood (3.27) also depends on σ^2, and maximising over it as well is instructive. Differentiating Eq. (3.27) with respect to σ^2,

$$\frac{\partial C}{\partial \sigma^2} = \frac{n}{2\sigma^2} - \frac{\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2}{2\sigma^4} = 0 \quad \Longrightarrow \quad \hat{\sigma}^2_{\mathrm{ML}} = \frac{\left\|\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{\theta}}\right\|_2^2}{n}, \tag{3.29}$$

the training mean squared error. This estimator is *biased*: it divides by n where Proposition 3.2 will show that $n - p$ is required, because the residuals of a fitted model are systematically smaller than the noise that produced them. Maximum likelihood is a principle for choosing estimators, not a guarantee that they are unbiased, and this is the simplest example of the difference.

Fisher information and the Cramér-Rao bound.

The Gaussian likelihood delivers one more result, and it puts the variance formula that Section 3.7 will use into a wider frame. The *Fisher information* is the expected negative Hessian

of the log-likelihood with respect to the parameters,

$$\mathscr{I}(\boldsymbol{\theta}) = -\mathbb{E}\left[\frac{\partial^2 \log p(\boldsymbol{D} \mid \boldsymbol{\theta})}{\partial \boldsymbol{\theta}\, \partial \boldsymbol{\theta}^T}\right] = \frac{\partial^2 C}{\partial \boldsymbol{\theta}\, \partial \boldsymbol{\theta}^T} = \frac{\boldsymbol{X}^T \boldsymbol{X}}{\sigma^2}, \tag{3.30}$$

where the second equality uses $C = -\log p$ and the third differentiates Eq. (3.27) twice, as in Section 1.8.5; the expectation is trivial because the Hessian does not depend on the data. It measures how sharply the data pin down the parameters – how curved the log-likelihood is at its peak. The Cramér-Rao inequality, proved in any text on mathematical statistics, states that *every* unbiased estimator $\tilde{\boldsymbol{\theta}}$ of $\boldsymbol{\theta}$ satisfies

$$\operatorname{Var}\left(\tilde{\boldsymbol{\theta}}\right) \succeq \mathscr{I}(\boldsymbol{\theta})^{-1} = \sigma^2 \left(\boldsymbol{X}^T \boldsymbol{X}\right)^{-1}, \tag{3.31}$$

in the sense that the difference is positive semi-definite. Comparing with the variance of the least-squares estimator, Eq. (3.32) below, the bound is attained: under Gaussian noise OLS is *efficient*, of minimum variance among all unbiased estimators, linear or not. This is stronger than the Gauss-Markov theorem, which restricts the competition to linear estimators but in return needs no Gaussian assumption; the two results bracket the same estimator from different sides.

Machine learning connection. Equation (3.27) says that *least squares is maximum likelihood for Gaussian noise*, and the converse statement is the useful one: a different noise model gives a different loss function. Laplace-distributed noise gives the mean absolute error (3.23); Bernoulli-distributed targets give the cross-entropy loss of logistic regression; Poisson counts give the Poisson deviance. When we later write down a loss function for a neural network we are, whether or not we say so, making an assumption about the distribution of the residuals. Choosing the squared error for data with heavy-tailed noise is not a matter of taste but an error, and it is why a handful of outliers can dominate a least-squares fit.

3.7 Statistical properties of the least-squares estimator

Because $\hat{\boldsymbol{\theta}}$ is a function of the random targets, it is itself a random variable, and Section 2.8 established its first two moments:

$$\mathbb{E}[\hat{\boldsymbol{\theta}}_{\mathrm{OLS}}] = \boldsymbol{\theta}, \qquad \operatorname{Var}(\hat{\boldsymbol{\theta}}_{\mathrm{OLS}}) = \sigma^2 \left(\boldsymbol{X}^T \boldsymbol{X}\right)^{-1}. \tag{3.32}$$

The estimator is unbiased *provided the linear model is correct*, and its covariance matrix is the inverse Hessian scaled by the noise variance. Since σ^2 is unknown it is estimated from the residuals,

$$\hat{\sigma}^2 = \frac{\left\| \boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{\theta}} \right\|_2^2}{n - p}, \tag{3.33}$$

the divisor $n - p$ accounting for the p degrees of freedom consumed by the fit, in the manner of Bessel's correction (2.28). The standard error of coefficient j is then $\hat{\sigma}\sqrt{[(\boldsymbol{X}^T\boldsymbol{X})^{-1}]_{jj}}$, from which a confidence interval follows,

$$\hat{\theta}_j \pm t_{\alpha/2,\,n-p}\, \hat{\sigma} \sqrt{\left[(\boldsymbol{X}^T \boldsymbol{X})^{-1}\right]_{jj}}, \tag{3.34}$$

with t the appropriate quantile of Student's distribution, which for $n-p$ large is simply the Gaussian quantile, 1.96 for 95%.

Each of these statements is used constantly and each deserves a proof; the proofs are short and they introduce the residual projector $\boldsymbol{I}-\boldsymbol{H}$, which is the key to everything in this section. Throughout, $\boldsymbol{y}=\boldsymbol{X}\boldsymbol{\theta}+\boldsymbol{\varepsilon}$ with $\mathbb{E}[\boldsymbol{\varepsilon}]=\boldsymbol{0}$ and $\mathrm{Var}(\boldsymbol{\varepsilon})=\sigma^2\boldsymbol{I}$, and $\boldsymbol{X}$ has full column rank.

Proposition 3.2 (The residual variance estimator is unbiased). *The residual vector is $\boldsymbol{e}=\boldsymbol{y}-\boldsymbol{X}\hat{\boldsymbol{\theta}}=(\boldsymbol{I}-\boldsymbol{H})\boldsymbol{\varepsilon}$, and $\mathbb{E}\big[\|\boldsymbol{e}\|_2^2\big]=(n-p)\sigma^2$. Hence $\hat{\sigma}^2$ of Eq. (3.33) satisfies $\mathbb{E}[\hat{\sigma}^2]=\sigma^2$.*

Proof. Since $\boldsymbol{H}\boldsymbol{X}=\boldsymbol{X}$, we have $(\boldsymbol{I}-\boldsymbol{H})\boldsymbol{y}=(\boldsymbol{I}-\boldsymbol{H})\boldsymbol{X}\boldsymbol{\theta}+(\boldsymbol{I}-\boldsymbol{H})\boldsymbol{\varepsilon}=(\boldsymbol{I}-\boldsymbol{H})\boldsymbol{\varepsilon}$: the residual projector annihilates the model and sees only the noise. Then, because $\boldsymbol{I}-\boldsymbol{H}$ is symmetric and idempotent,

$$\mathbb{E}\left[\|\boldsymbol{e}\|_2^2\right]=\mathbb{E}\left[\boldsymbol{\varepsilon}^T(\boldsymbol{I}-\boldsymbol{H})\boldsymbol{\varepsilon}\right]=\mathrm{Tr}\left[(\boldsymbol{I}-\boldsymbol{H})\,\mathbb{E}[\boldsymbol{\varepsilon}\boldsymbol{\varepsilon}^T]\right]=\sigma^2\,\mathrm{Tr}(\boldsymbol{I}-\boldsymbol{H})=\sigma^2(n-p),$$

using $\mathbb{E}[\boldsymbol{z}^T\boldsymbol{M}\boldsymbol{z}]=\mathrm{Tr}(\boldsymbol{M}\mathrm{Var}(\boldsymbol{z}))$ for a zero-mean vector $\boldsymbol{z}$ and $\mathrm{Tr}(\boldsymbol{H})=p$ from Eq. (1.10). □

The projector $\boldsymbol{I}-\boldsymbol{H}$ has $n-p$ eigenvalues equal to one and p equal to zero, so the residual lives in an $(n-p)$-dimensional subspace: that is the precise sense in which the fit "consumes p degrees of freedom". Under Gaussian noise the same projector gives the full distribution theory behind the confidence interval (3.34).

Proposition 3.3 (Distribution of the least-squares estimator). *Assume in addition that the noise is Gaussian, $\boldsymbol{\varepsilon}\sim\mathcal{N}(\boldsymbol{0},\sigma^2\boldsymbol{I})$. Then*

1. *$\hat{\boldsymbol{\theta}}\sim\mathcal{N}\big(\boldsymbol{\theta},\sigma^2(\boldsymbol{X}^T\boldsymbol{X})^{-1}\big)$;*
2. *$\hat{\boldsymbol{\theta}}$ and the residual $\boldsymbol{e}$ are independent;*
3. *$(n-p)\hat{\sigma}^2/\sigma^2=\|\boldsymbol{e}\|_2^2/\sigma^2$ has the χ^2 distribution with $n-p$ degrees of freedom;*
4. *for each j, the ratio $(\hat{\theta}_j-\theta_j)\big/\big(\hat{\sigma}\sqrt{[(\boldsymbol{X}^T\boldsymbol{X})^{-1}]_{jj}}\big)$ has Student's t distribution with $n-p$ degrees of freedom.*

Proof. (i) $\hat{\boldsymbol{\theta}}=\boldsymbol{\theta}+(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{\varepsilon}$ is an affine map of a Gaussian vector and hence Gaussian, with the moments of Eq. (3.32). (ii) Both are linear in $\boldsymbol{\varepsilon}$, so joint Gaussianity holds and independence is equivalent to zero cross-covariance:

$$\mathrm{Cov}\left(\hat{\boldsymbol{\theta}},\boldsymbol{e}\right)=(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\,\sigma^2\boldsymbol{I}\,(\boldsymbol{I}-\boldsymbol{H})=\sigma^2(\boldsymbol{X}^T\boldsymbol{X})^{-1}\left(\boldsymbol{X}^T-\boldsymbol{X}^T\boldsymbol{H}\right)=\boldsymbol{0},$$

because $\boldsymbol{X}^T\boldsymbol{H}=(\boldsymbol{H}\boldsymbol{X})^T=\boldsymbol{X}^T$. (iii) By the spectral decomposition of Section 1.9, $\boldsymbol{I}-\boldsymbol{H}=\boldsymbol{Q}\boldsymbol{D}\boldsymbol{Q}^T$ with $\boldsymbol{Q}$ orthogonal and $\boldsymbol{D}$ diagonal with $n-p$ ones and p zeros. Put $\boldsymbol{w}=\boldsymbol{Q}^T\boldsymbol{\varepsilon}/\sigma$; an orthogonal map of $\mathcal{N}(\boldsymbol{0},\boldsymbol{I})$ is again $\mathcal{N}(\boldsymbol{0},\boldsymbol{I})$, so the w_k are independent standard normals, and $\|\boldsymbol{e}\|_2^2/\sigma^2=\boldsymbol{w}^T\boldsymbol{D}\boldsymbol{w}$ is the sum of $n-p$ of their squares, which is the definition of χ^2_{n-p}. (iv) By (i), $z_j=(\hat{\theta}_j-\theta_j)/(\sigma\sqrt{[(\boldsymbol{X}^T\boldsymbol{X})^{-1}]_{jj}})$ is standard normal; by (iii), $u=(n-p)\hat{\sigma}^2/\sigma^2$ is χ^2_{n-p}; by (ii) they are independent; and $z_j/\sqrt{u/(n-p)}$, which is exactly the ratio in (iv), is by definition Student's t with $n-p$ degrees of freedom. □

Statement (iv) is the confidence interval (3.34): the interval $\hat{\theta}_j\pm t_{\alpha/2,n-p}\,\hat{\sigma}\sqrt{[(\boldsymbol{X}^T\boldsymbol{X})^{-1}]_{jj}}$ covers the true θ_j with probability exactly $1-\alpha$, not approximately, and the Student quantile rather than the Gaussian one is what pays for having estimated σ from the same data. Statement (iii) is also the promised justification of the reduced-χ^2 test of Section 3.4. All four statements can be checked by simulation, and the check is worth running once because it also exposes what the confidence interval does *not* promise – see the last line of the output.

```python
import numpy as np
from scipy import stats

rng = np.random.default_rng(2024)
n, p, sigma = 30, 4, 0.5
x = np.linspace(-1, 1, n)
X = np.vander(x, p, increasing=True)                 # cubic polynomial
theta_true = np.array([1.0, 0.5, -2.0, 1.5])
XtX_inv = np.linalg.inv(X.T @ X)
t_q = stats.t.ppf(0.975, n - p)                      # 2.056 for n-p = 26

s2, covered, joint, train, test = [], [], [], [], []
for trial in range(20000):
    y = X @ theta_true + sigma * rng.normal(size=n)
    theta = XtX_inv @ X.T @ y
    e = y - X @ theta
    s2.append(e @ e / (n - p))                        # Eq. (3.sigmahat)
    se = np.sqrt(s2[-1] * np.diag(XtX_inv))
    inside = np.abs(theta - theta_true) <= t_q * se  # Eq. (3.confint)
    covered.append(inside); joint.append(inside.all())
    train.append(e @ e / n)                           # training MSE
    y_new = X @ theta_true + sigma * rng.normal(size=n)
    test.append(np.mean((y_new - X @ theta)**2))     # fresh targets, same X

print("E[sigma_hat^2] =", np.mean(s2), "  sigma^2 =", sigma**2)
print("coverage per coefficient:", np.mean(covered, axis=0))
print("all four covered at once:", np.mean(joint))
print("E[train MSE] =", np.mean(train), "  sigma^2 (1 - p/n) =", sigma**2 * (1 - p/n))
print("E[test  MSE] =", np.mean(test),  "  sigma^2 (1 + p/n) =", sigma**2 * (1 + p/n))
```

```
E[sigma_hat^2] = 0.24999   sigma^2 = 0.25
coverage per coefficient: [0.9499 0.9498 0.9510 0.9492]
all four covered at once: 0.861
E[train MSE] = 0.21666   sigma^2 (1 - p/n) = 0.21667
E[test  MSE] = 0.28420   sigma^2 (1 + p/n) = 0.28333
```

The first three lines are Propositions 3.2 and 3.3: the variance estimate is unbiased and each interval covers its coefficient 95% of the time. The probability that all four intervals cover simultaneously is only 86% – individual confidence statements do not combine into a joint one, and a table of p separate intervals says less than it appears to. The last two lines are a result we have not yet derived, and it is the quantitative form of the warning in Section 3.5.

Why the training error is optimistic.

Suppose the linear model is correct. By Proposition 3.2 the expected training error is

$$\mathbb{E}\left[\frac{1}{n}\|\boldsymbol{y}-\boldsymbol{X}\hat{\boldsymbol{\theta}}\|_2^2\right]=\sigma^2\left(1-\frac{p}{n}\right): \tag{3.35}$$

smaller than the noise level, by exactly the fraction of the noise that the fit has absorbed. Now draw fresh targets $\boldsymbol{y}'=\boldsymbol{X}\boldsymbol{\theta}+\boldsymbol{\varepsilon}'$ at the same inputs, with $\boldsymbol{\varepsilon}'$ independent of $\boldsymbol{\varepsilon}$, and score the fitted model on them. Since $\boldsymbol{y}'-\boldsymbol{X}\hat{\boldsymbol{\theta}}=\boldsymbol{\varepsilon}'-\boldsymbol{H}\boldsymbol{\varepsilon}$ and the two terms are independent with zero mean,

$$\mathbb{E}\left[\frac{1}{n}\|\boldsymbol{y}'-\boldsymbol{X}\hat{\boldsymbol{\theta}}\|_2^2\right]=\frac{1}{n}\left(\mathbb{E}\|\boldsymbol{\varepsilon}'\|_2^2+\mathbb{E}\|\boldsymbol{H}\boldsymbol{\varepsilon}\|_2^2\right)=\frac{1}{n}\left(n\sigma^2+\sigma^2\mathrm{Tr}(\boldsymbol{H})\right)=\sigma^2\left(1+\frac{p}{n}\right). \tag{3.36}$$

The difference,

$$\text{optimism} = \frac{2p\sigma^2}{n}, \tag{3.37}$$

is the amount by which the training error flatters every least-squares fit, and it grows linearly with the number of parameters. A model that adds columns will always look better in training and, once the added columns stop carrying signal, will look worse in test by exactly this margin: that is the bias-variance curve of Section 2.10 for the case in which the bias is already zero, and the variance term is $p\sigma^2/n$. Adding $2p\hat{\sigma}^2/n$ to the training error to correct for it is Mallows' C_p statistic, and the same correction with $\hat{\sigma}^2$ replaced by the likelihood is Akaike's information criterion; both are attempts to estimate the test error from training quantities. Cross-validation, to which we now turn, estimates it directly.

Leave-one-out cross-validation in closed form.

The most thorough version of the cross-validation of Section 2.12 is leave-one-out: fit n times, each time omitting one observation, and predict the omitted point. For linear regression it need not be done n times, and the derivation is a small classic. Let $\boldsymbol{x}_i^T$ be row i of $\boldsymbol{X}$, let $\hat{\boldsymbol{\theta}}_{(-i)}$ be the estimator without observation i, and let $h_{ii} = \boldsymbol{x}_i^T(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{x}_i$ be the ith diagonal element of the hat matrix, the *leverage* of point i. Removing a row changes $\boldsymbol{X}^T\boldsymbol{X}$ by a rank-one term, $\boldsymbol{X}^T\boldsymbol{X} - \boldsymbol{x}_i\boldsymbol{x}_i^T$, and the Sherman-Morrison formula

$$\left(\boldsymbol{M} - \boldsymbol{u}\boldsymbol{v}^T\right)^{-1} = \boldsymbol{M}^{-1} + \frac{\boldsymbol{M}^{-1}\boldsymbol{u}\boldsymbol{v}^T\boldsymbol{M}^{-1}}{1 - \boldsymbol{v}^T\boldsymbol{M}^{-1}\boldsymbol{u}} \tag{3.38}$$

– verified by multiplying both sides by $\boldsymbol{M} - \boldsymbol{u}\boldsymbol{v}^T$ – gives the inverse without a new factorisation. Applying it with $\boldsymbol{M} = \boldsymbol{X}^T\boldsymbol{X}$ and $\boldsymbol{u} = \boldsymbol{v} = \boldsymbol{x}_i$ to $\hat{\boldsymbol{\theta}}_{(-i)} = (\boldsymbol{X}^T\boldsymbol{X} - \boldsymbol{x}_i\boldsymbol{x}_i^T)^{-1}(\boldsymbol{X}^T\boldsymbol{y} - \boldsymbol{x}_i y_i)$ and simplifying – the exercises at the end of the chapter ask for the algebra – yields

$$\hat{\boldsymbol{\theta}}_{(-i)} = \hat{\boldsymbol{\theta}} - \frac{(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{x}_i\, e_i}{1 - h_{ii}}, \qquad y_i - \boldsymbol{x}_i^T\hat{\boldsymbol{\theta}}_{(-i)} = \frac{e_i}{1 - h_{ii}}, \tag{3.39}$$

where $e_i = y_i - \boldsymbol{x}_i^T\hat{\boldsymbol{\theta}}$ is the ordinary residual of the full fit. The leave-one-out prediction error is therefore the ordinary residual inflated by $1/(1-h_{ii})$: a point with high leverage pulls the fit towards itself, its residual is deceptively small, and Eq. (3.39) undoes exactly that. Summing gives the leave-one-out estimate of the test error from a single fit,

$$\text{CV}_{\text{LOO}} = \frac{1}{n}\sum_{i=0}^{n-1}\left(\frac{e_i}{1 - h_{ii}}\right)^2, \tag{3.40}$$

Allen's PRESS statistic. Nothing in the derivation used the particular form of $\boldsymbol{H}$ beyond $\tilde{\boldsymbol{y}} = \boldsymbol{H}\boldsymbol{y}$ being linear in $\boldsymbol{y}$, and the same identity holds for any *linear smoother* $\tilde{\boldsymbol{y}} = \boldsymbol{S}\boldsymbol{y}$ with h_{ii} replaced by S_{ii}; in particular for Ridge regression with $\boldsymbol{S}_\lambda = \boldsymbol{X}(\boldsymbol{X}^T\boldsymbol{X} + \lambda\boldsymbol{I})^{-1}\boldsymbol{X}^T$, so that the penalty can be chosen by leave-one-out at the price of one fit per candidate λ. Replacing each S_{ii} by their average $\text{Tr}(\boldsymbol{S})/n$ gives the *generalised cross-validation* criterion

$$\text{GCV}(\lambda) = \frac{\frac{1}{n}\|\boldsymbol{y} - \boldsymbol{S}_\lambda\boldsymbol{y}\|_2^2}{\left(1 - \text{df}(\lambda)/n\right)^2}, \qquad \text{df}(\lambda) = \text{Tr}(\boldsymbol{S}_\lambda) = \sum_i \frac{\sigma_i^2}{\sigma_i^2 + \lambda}, \tag{3.41}$$

which is invariant under rotations of the data and depends on the design only through the effective degrees of freedom of Section 3.8. The following code checks Eq. (3.40) against n explicit refits, for OLS and for Ridge, and uses it to choose λ.

```
import numpy as np
```

```
rng = np.random.default_rng(7)
n = 40
x = np.sort(rng.random(n))
y = np.sin(2 * np.pi * x) + 0.3 * rng.normal(size=n)
X = np.vander(x, 11, increasing=True)                # degree-10 polynomial

def loo_explicit(X, y, lmbda=0.0):
    # leave-one-out by n separate fits
    n, p = X.shape
    errs = np.empty(n)
    for i in range(n):
        keep = np.arange(n) != i
        theta = np.linalg.solve(X[keep].T @ X[keep] + lmbda * np.eye(p),
                                X[keep].T @ y[keep])
        errs[i] = y[i] - X[i] @ theta
    return np.mean(errs**2)

def loo_closed_form(X, y, lmbda=0.0):
    # the same number from one fit, Eq. (3.press)
    n, p = X.shape
    S = X @ np.linalg.solve(X.T @ X + lmbda * np.eye(p), X.T)   # smoother matrix
    e = y - S @ y
    return np.mean((e / (1.0 - np.diag(S)))**2)

def gcv(X, y, lmbda=0.0):
    # generalised cross-validation, Eq. (3.gcv)
    n, p = X.shape
    S = X @ np.linalg.solve(X.T @ X + lmbda * np.eye(p), X.T)
    e = y - S @ y
    return np.mean(e**2) / (1.0 - np.trace(S) / n)**2

for lmbda in [0.0, 1e-4, 1e-2]:
    print(f"lambda = {lmbda:<6}  explicit {loo_explicit(X, y, lmbda):.6f}"
          f"  closed form {loo_closed_form(X, y, lmbda):.6f}"
          f"  GCV {gcv(X, y, lmbda):.6f}")

lambdas = np.logspace(-8, 0, 81)
print("lambda chosen by LOO:", lambdas[np.argmin([loo_closed_form(X, y, l) for l in
    lambdas])])
print("lambda chosen by GCV:", lambdas[np.argmin([gcv(X, y, l) for l in lambdas])])
```

```
lambda = 0.0     explicit 0.107484  closed form 0.107485  GCV 0.108301
lambda = 0.0001  explicit 0.080349  closed form 0.080349  GCV 0.084748
lambda = 0.01    explicit 0.125840  closed form 0.125840  GCV 0.127780
lambda chosen by LOO: 0.0006309573444801943
lambda chosen by GCV: 0.0006309573444801943
```

The closed form reproduces the forty explicit refits to every printed digit – the difference of one unit in the last place at $\lambda = 0$ is the rounding of the unpenalised solve on a badly conditioned Vandermonde matrix, Section 3.3 – at a fortieth of the cost. Both criteria have a clear interior minimum, at a penalty that lowers the leave-one-out error from 0.107 for the unpenalised degree-ten fit to 0.080, and here they pick the same λ; in general GCV differs a little because averaging the leverages discounts the high-leverage points at the ends of the interval that dominate the leave-one-out sum for a polynomial fit.

Reading the variance through the SVD.

Substituting Eq. (1.115) into Eq. (3.32) gives

$$\mathrm{Var}(\hat{\boldsymbol{\theta}}_{\mathrm{OLS}}) = \sigma^2 \boldsymbol{V}\tilde{\boldsymbol{\Sigma}}^{-2}\boldsymbol{V}^T = \sigma^2 \sum_{i=0}^{p-1} \frac{\boldsymbol{v}_i \boldsymbol{v}_i^T}{\sigma_i^2}, \tag{3.42}$$

so the variance along the ith right singular direction is σ^2/σ_i^2. Together with the expansion (3.11) this gives the complete picture of what collinearity does. A direction in feature space along which the data barely vary has a small singular value; the coefficient along it is obtained by dividing by that small number, and its variance is obtained by dividing by its square. The fit becomes an enormous positive coefficient on one feature cancelling an enormous negative one on another, and the pair swings wildly when the data are perturbed. Nothing is wrong with the algebra; the data simply do not determine that combination.

The Gauss-Markov theorem.

Among all estimators that are both linear in $\boldsymbol{y}$ and unbiased, OLS has the smallest variance. This is the Gauss-Markov theorem, and it requires only that the noise have zero mean, constant variance and be uncorrelated – not that it be Gaussian. Its proof is a few lines and shows exactly where the competing estimators lose.

Theorem 3.4 (Gauss-Markov). *Let $\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\theta} + \boldsymbol{\varepsilon}$ with $\mathbb{E}[\boldsymbol{\varepsilon}] = \boldsymbol{0}$, $\mathrm{Var}(\boldsymbol{\varepsilon}) = \sigma^2 \boldsymbol{I}$ and $\boldsymbol{X}$ of full column rank. If $\tilde{\boldsymbol{\theta}} = \boldsymbol{C}\boldsymbol{y}$ is any estimator linear in $\boldsymbol{y}$ with $\mathbb{E}[\tilde{\boldsymbol{\theta}}] = \boldsymbol{\theta}$ for every $\boldsymbol{\theta}$, then $\mathrm{Var}(\tilde{\boldsymbol{\theta}}) - \mathrm{Var}(\hat{\boldsymbol{\theta}}_{\mathrm{OLS}})$ is positive semi-definite.*

Proof. Unbiasedness for every $\boldsymbol{\theta}$ means $\mathbb{E}[\boldsymbol{C}\boldsymbol{y}] = \boldsymbol{C}\boldsymbol{X}\boldsymbol{\theta} = \boldsymbol{\theta}$ for all $\boldsymbol{\theta}$, hence $\boldsymbol{C}\boldsymbol{X} = \boldsymbol{I}$. Write $\boldsymbol{C} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T + \boldsymbol{D}$, which defines $\boldsymbol{D}$; then $\boldsymbol{C}\boldsymbol{X} = \boldsymbol{I} + \boldsymbol{D}\boldsymbol{X}$, so $\boldsymbol{D}\boldsymbol{X} = \boldsymbol{0}$. Now

$$\mathrm{Var}\left(\tilde{\boldsymbol{\theta}}\right) = \sigma^2 \boldsymbol{C}\boldsymbol{C}^T = \sigma^2\left[(\boldsymbol{X}^T\boldsymbol{X})^{-1} + (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{D}^T + \boldsymbol{D}\boldsymbol{X}(\boldsymbol{X}^T\boldsymbol{X})^{-1} + \boldsymbol{D}\boldsymbol{D}^T\right] = \sigma^2(\boldsymbol{X}^T\boldsymbol{X})^{-1} + \sigma^2\boldsymbol{D}\boldsymbol{D}^T,$$

the two cross terms vanishing because $\boldsymbol{D}\boldsymbol{X} = \boldsymbol{0}$ and $\boldsymbol{X}^T\boldsymbol{D}^T = (\boldsymbol{D}\boldsymbol{X})^T = \boldsymbol{0}$. The first term is $\mathrm{Var}(\hat{\boldsymbol{\theta}}_{\mathrm{OLS}})$ and the second is a Gram matrix, positive semi-definite, with equality if and only if $\boldsymbol{D} = \boldsymbol{0}$. □

Every linear unbiased competitor is OLS plus a piece $\boldsymbol{D}\boldsymbol{y}$ that averages to zero and only adds variance. The theorem is often quoted as though it settled the matter. It does not, and the reason is Eq. (2.27): the mean squared error counts bias and variance equally, and Gauss-Markov restricts attention to the unbiased estimators only. The moment we admit biased estimators, better ones exist. The rest of this chapter constructs them.

3.8 Ridge regression

Ridge regression adds a penalty on the squared length of the parameter vector,

$$\min_{\boldsymbol{\theta}\in\mathbb{R}^p} \left\{ \frac{1}{n}\,\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2 + \lambda\,\|\boldsymbol{\theta}\|_2^2 \right\}, \qquad \lambda \geq 0, \tag{3.43}$$

with $\|\boldsymbol{\theta}\|_2^2 = \sum_j \theta_j^2$. We derived the solution in Eq. (1.42); dropping the factor $1/n$ so that λ has its conventional scaling,

$$\boxed{\hat{\boldsymbol{\theta}}_{\mathrm{Ridge}} = \left(\boldsymbol{X}^T\boldsymbol{X} + \lambda \boldsymbol{I}\right)^{-1}\boldsymbol{X}^T\boldsymbol{y}} \tag{3.44}$$

with $\boldsymbol{I}$ the $p \times p$ identity. Ridge regression is therefore ordinary least squares with a modified diagonal – a fact which Section 3.12 will turn into a method that never forms the design

matrix at all – and the modification cures the central defect of Eq. (3.8): whatever the rank of $\boldsymbol{X}$, the matrix $\boldsymbol{X}^T\boldsymbol{X}+\lambda\boldsymbol{I}$ has all eigenvalues at least λ and is invertible for every $\lambda>0$. The estimator exists even when $p>n$.

The constrained form.

Equation (3.43) is the Lagrangian form of a constrained problem: minimising the squared error subject to

$$\sum_{j=0}^{p-1}\theta_j^2 \le t \tag{3.45}$$

for a finite $t>0$ gives the same solutions, with a one-to-one decreasing correspondence between t and λ. The constraint region is a ball in parameter space, and this geometric picture is what will distinguish Ridge from the Lasso in Section 3.9.

Shrinkage, through the SVD.

The analytical content of Ridge regression is best seen in the singular basis. Section 1.22 showed that Eq. (3.44) is

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}} = \sum_{i=0}^{p-1}\frac{\sigma_i}{\sigma_i^2+\lambda}\left(\boldsymbol{u}_i^T\boldsymbol{y}\right)\boldsymbol{v}_i, \tag{3.46}$$

to be compared with the OLS expansion (3.11) in which the coefficient was $1/\sigma_i$. The fitted values follow from Eq. (1.131),

$$\tilde{\boldsymbol{y}}_{\text{Ridge}} = \sum_{i=0}^{p-1}\boldsymbol{u}_i\boldsymbol{u}_i^T\frac{\sigma_i^2}{\sigma_i^2+\lambda}\,\boldsymbol{y}, \tag{3.47}$$

whereas for OLS the same expression holds with every factor replaced by one, $\tilde{\boldsymbol{y}}_{\text{OLS}}=\boldsymbol{U}\boldsymbol{U}^T\boldsymbol{y}$, which is the hat matrix (3.9) in the singular basis. Since $\lambda\ge 0$,

$$\frac{\sigma_i^2}{\sigma_i^2+\lambda}\le 1, \tag{3.48}$$

so Ridge regression expresses $\boldsymbol{y}$ in the orthonormal basis $\boldsymbol{U}$ and then *shrinks* each coordinate. Because the singular values are ordered descendingly, the shrinkage is mild for the leading directions and severe for the trailing ones: precisely the directions whose coefficients Eq. (3.42) showed to have the largest variance are the ones suppressed. The effective number of parameters is $\mathrm{df}(\lambda)=\sum_i \sigma_i^2/(\sigma_i^2+\lambda)$ from Eq. (1.132), falling smoothly from p to zero.

Figure 3.1 plots the shrinkage factor as a function of the singular value, for several penalties. Each curve is a smooth switch: directions with $\sigma_i^2 \gg \lambda$ pass through essentially untouched, directions with $\sigma_i^2 \ll \lambda$ are suppressed almost entirely, and the transition is centred on $\sigma_i=\sqrt{\lambda}$. Increasing λ slides the switch to the right, discarding progressively more of the spectrum. Truncated SVD regression replaces this smooth curve by a step, which is the only difference between the two methods.

An instructive special case.

Suppose the design matrix is orthonormal, $\boldsymbol{X}^T\boldsymbol{X}=\boldsymbol{I}$. Then $\hat{\boldsymbol{\theta}}_{\text{OLS}}=\boldsymbol{X}^T\boldsymbol{y}$ and

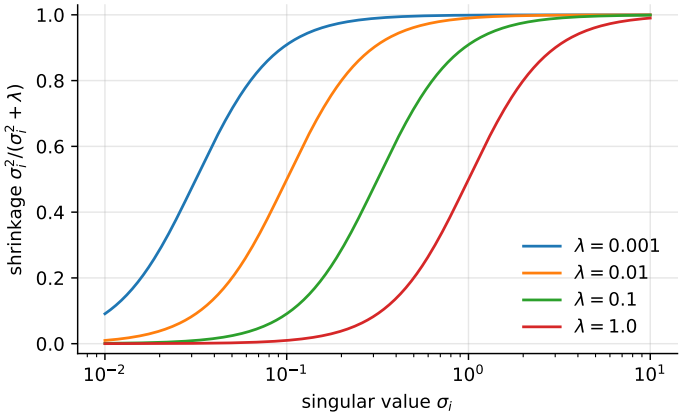


Fig. 3.1 The Ridge shrinkage factor $\sigma_i^2/(\sigma_i^2+\lambda)$ of Eq. (3.48) against the singular value, for four penalties. The transition is centred on $\sigma_i = \sqrt{\lambda}$.

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}} = (\boldsymbol{I}+\lambda\boldsymbol{I})^{-1}\boldsymbol{X}^T\boldsymbol{y} = \frac{1}{1+\lambda}\,\hat{\boldsymbol{\theta}}_{\text{OLS}}, \tag{3.49}$$

so every coefficient is scaled by the same factor $1/(1+\lambda)$ and the estimator tends to zero as $\lambda \to \infty$. Ridge shrinks all coefficients *proportionally*; it never sets any of them exactly to zero. Remember this when we reach the Lasso.

Bias and variance.

Ridge regression is biased. Taking the expectation of Eq. (3.44) and using $\mathbb{E}[\boldsymbol{y}] = \boldsymbol{X}\boldsymbol{\theta}$,

$$\mathbb{E}\left[\hat{\boldsymbol{\theta}}_{\text{Ridge}}\right] = \left(\boldsymbol{X}^T\boldsymbol{X}+\lambda\boldsymbol{I}\right)^{-1}\left(\boldsymbol{X}^T\boldsymbol{X}\right)\boldsymbol{\theta} \neq \boldsymbol{\theta} \tag{3.50}$$

for any $\lambda > 0$, the bias growing towards $-\boldsymbol{\theta}$ as $\lambda \to \infty$. Applying the transformation rule $\text{Var}(\boldsymbol{A}\boldsymbol{y}) = \boldsymbol{A}\,\text{Var}(\boldsymbol{y})\boldsymbol{A}^T$ to Eq. (3.44) gives the variance,

$$\text{Var}\left(\hat{\boldsymbol{\theta}}_{\text{Ridge}}\right) = \sigma^2\left[\boldsymbol{X}^T\boldsymbol{X}+\lambda\boldsymbol{I}\right]^{-1}\boldsymbol{X}^T\boldsymbol{X}\left\{\left[\boldsymbol{X}^T\boldsymbol{X}+\lambda\boldsymbol{I}\right]^{-1}\right\}^T, \tag{3.51}$$

which vanishes as $\lambda \to \infty$. Subtracting the two variances,

$$\text{Var}(\hat{\boldsymbol{\theta}}_{\text{OLS}}) - \text{Var}(\hat{\boldsymbol{\theta}}_{\text{Ridge}}) = \sigma^2\left[\boldsymbol{X}^T\boldsymbol{X}+\lambda\boldsymbol{I}\right]^{-1}\left[2\lambda\boldsymbol{I}+\lambda^2\left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\right]\left\{\left[\boldsymbol{X}^T\boldsymbol{X}+\lambda\boldsymbol{I}\right]^{-1}\right\}^T, \tag{3.52}$$

and the middle bracket is non-negative definite for $\lambda > 0$, hence so is the whole product. The variance of the Ridge estimator is therefore *always* smaller than that of OLS.

This is the bias-variance trade-off of Section 2.10 in closed form. Increasing λ increases the squared bias and decreases the variance, monotonically in both cases; by Eq. (2.27) the total error is their sum, and since the variance falls immediately while the bias starts at zero with zero derivative, there always exists a $\lambda > 0$ giving a smaller mean squared error than OLS. Gauss-Markov is not contradicted – the Ridge estimator is not unbiased, so it was never in the competition. Both of the assertions just made can be proved, and the rest of this section does so.

The constrained form, exactly.

The correspondence between Eq. (3.43) and the constraint (3.45) is worth stating precisely rather than by assertion, because it is the first appearance of the Karush-Kuhn-Tucker conditions that Chapter 6 will lean on throughout.

Proposition 3.5 (Penalty and constraint). *Let $t > 0$ and let $\hat{\boldsymbol{\theta}}_t$ solve $\min \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2$ subject to $\|\boldsymbol{\theta}\|_2^2 \le t$. If the constraint is active, there is a unique $\lambda > 0$ for which $\hat{\boldsymbol{\theta}}_t$ also solves (3.43), namely the multiplier of the constraint; and conversely, the solution of (3.43) at penalty λ solves the constrained problem with $t = \|\hat{\boldsymbol{\theta}}_{\mathrm{Ridge}}(\lambda)\|_2^2$. The map $\lambda \mapsto \|\hat{\boldsymbol{\theta}}_{\mathrm{Ridge}}(\lambda)\|_2^2$ is strictly decreasing.*

Proof. The Lagrangian of the constrained problem is $\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2 + \lambda(\|\boldsymbol{\theta}\|_2^2 - t)$, whose stationarity condition in $\boldsymbol{\theta}$ is exactly that of (3.43); the term $-\lambda t$ does not involve $\boldsymbol{\theta}$ and cannot change the minimiser. Complementary slackness gives $\lambda > 0$ when the constraint is active and $\lambda = 0$ when it is not, and the problem is convex with a strictly feasible point, so the conditions are sufficient as well as necessary. For the monotonicity, Eq. (3.46) gives

$$\left\| \hat{\boldsymbol{\theta}}_{\mathrm{Ridge}}(\lambda) \right\|_2^2 = \sum_{i=0}^{p-1} \frac{\sigma_i^2 \left(\boldsymbol{u}_i^T \boldsymbol{y} \right)^2}{\left(\sigma_i^2 + \lambda \right)^2}, \tag{3.53}$$

in which every term is strictly decreasing in λ. $\square$

That a useful penalty always exists.

Section 3.8 closed with the assertion that some $\lambda > 0$ gives a smaller mean squared error than ordinary least squares. That statement is due to Hoerl and Kennard and it can be proved in half a page, which is worth doing because the proof shows *why* the effect is unavoidable rather than fortunate.

Work in the singular basis. Put $\boldsymbol{b} = \boldsymbol{V}^T \boldsymbol{\theta}$ for the true parameter and recall from Eq. (3.46) that the ith coefficient of the estimator in that basis is $\sigma_i (\boldsymbol{u}_i^T \boldsymbol{y}) / (\sigma_i^2 + \lambda)$. With $\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\theta} + \boldsymbol{\varepsilon}$ and $\mathrm{Var}(\boldsymbol{\varepsilon}) = \sigma^2 \boldsymbol{I}$ we have $\mathbb{E}[\boldsymbol{u}_i^T \boldsymbol{y}] = \sigma_i b_i$ and $\mathrm{Var}(\boldsymbol{u}_i^T \boldsymbol{y}) = \sigma^2$, so

$$\mathbb{E}\left[\hat{b}_i\right] = \frac{\sigma_i^2}{\sigma_i^2 + \lambda}\, b_i, \qquad \mathrm{Var}\left(\hat{b}_i\right) = \frac{\sigma^2 \sigma_i^2}{\left(\sigma_i^2 + \lambda\right)^2}. \tag{3.54}$$

Because $\boldsymbol{V}$ is orthogonal the total mean squared error of the estimator is the sum over i of squared bias plus variance,

$$M(\lambda) = \mathbb{E}\left\| \hat{\boldsymbol{\theta}}_{\mathrm{Ridge}} - \boldsymbol{\theta} \right\|_2^2 = \sum_{i=0}^{p-1} \frac{\lambda^2 b_i^2 + \sigma^2 \sigma_i^2}{\left(\sigma_i^2 + \lambda\right)^2}, \tag{3.55}$$

the first term in the numerator being the squared bias $[\lambda/(\sigma_i^2 + \lambda)]^2 b_i^2$ and the second the variance.

Theorem 3.6 (Hoerl and Kennard). *Suppose $\boldsymbol{X}$ has full column rank and $\sigma^2 > 0$. Then $M'(0) < 0$, and consequently there exists $\lambda > 0$ with $M(\lambda) < M(0)$: some Ridge estimator has a strictly smaller mean squared error than ordinary least squares, whatever the true $\boldsymbol{\theta}$.*

Proof. Differentiate Eq. (3.55) term by term. Writing $N_i(\lambda) = \lambda^2 b_i^2 + \sigma^2 \sigma_i^2$ and $D_i(\lambda) = (\sigma_i^2 + \lambda)^2$,

$$\frac{d}{d\lambda}\frac{N_i}{D_i} = \frac{2\lambda b_i^2\left(\sigma_i^2+\lambda\right)^2 - 2\left(\sigma_i^2+\lambda\right)\left(\lambda^2 b_i^2+\sigma^2\sigma_i^2\right)}{\left(\sigma_i^2+\lambda\right)^4} = \frac{2\left[\lambda b_i^2\sigma_i^2 - \sigma^2\sigma_i^2\right]}{\left(\sigma_i^2+\lambda\right)^3},$$

where the second step cancels the $\lambda^2 b_i^2$ terms. Setting $\lambda = 0$ leaves $-2\sigma^2\sigma_i^2/\sigma_i^6 = -2\sigma^2/\sigma_i^4$, so

$$M'(0) = -2\sigma^2 \sum_{i=0}^{p-1} \frac{1}{\sigma_i^4} \; < \; 0. \tag{3.56}$$

Since M is differentiable at 0 with a strictly negative derivative, it is strictly smaller than $M(0)$ for all sufficiently small $\lambda > 0$. □

Two things about this proof deserve attention. The bias term contributes *nothing* to $M'(0)$: it enters as λ^2 and so has zero derivative at the origin, while the variance falls linearly. That asymmetry is the whole content of the theorem, and it is Eq. (2.27) speaking again – a little bias is free to second order, and the variance it buys is not. And Eq. (3.56) says the effect is largest exactly when the design is badly conditioned, since the sum is dominated by the smallest singular value. The worse the collinearity, the more Ridge has to offer, which is the practical rule the theorem underwrites. Running the arithmetic on four designs:

```
 n     p    sigma^2   M(0)=OLS    min_lambda M   best lambda   M'(0) predicted   M'(0)
measured
40     6       1.00      0.1823          0.1636         3.256            -0.0129
    -0.0129
40     6       0.10      0.0214          0.0213        0.1001          -0.001986
  -0.001985
80    20       1.00      0.3461          0.3353         1.363           -0.01642
   -0.01642
25    15       0.50      0.7361          0.6067        0.9295            -0.3386
    -0.3386
```

The minimising penalty is strictly positive in every case, and the numerically measured slope at the origin agrees with Eq. (3.56). The theorem is an existence statement and not a recipe: the optimal λ depends on the unknown $\boldsymbol{b}$ and σ^2, which is why in practice it is chosen by the cross-validation of Section 2.12.

3.9 The Lasso

Replacing the 2-norm penalty of Eq. (3.43) by a 1-norm gives

$$\min_{\boldsymbol{\theta}\in\mathbb{R}^p}\left\{\frac{1}{n}\left\|\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}\right\|_2^2 + \lambda\left\|\boldsymbol{\theta}\right\|_1\right\}, \qquad \left\|\boldsymbol{\theta}\right\|_1 = \sum_{j=0}^{p-1}\left|\theta_j\right|, \tag{3.57}$$

which is the Lasso – least absolute shrinkage and selection operator. The change looks slight and its consequences are not: the Lasso sets coefficients exactly to zero, and thereby performs variable selection as part of the fitting, which Ridge regression by Eq. (3.49) never does.

No closed form.

Differentiating Eq. (3.57) requires the derivative of the absolute value,

$$\frac{d\left|\theta\right|}{d\theta} = \operatorname{sgn}(\theta) = \begin{cases} +1 & \theta > 0, \\ -1 & \theta < 0, \end{cases} \tag{3.58}$$

which is undefined at the origin. Ignoring that difficulty for a moment and dropping the $1/n$, the stationarity condition reads

$$-2\boldsymbol{X}^T(\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta})+\lambda\,\mathrm{sgn}(\boldsymbol{\theta})=\boldsymbol{0}, \qquad \text{that is} \qquad 2\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\theta}+\lambda\,\mathrm{sgn}(\boldsymbol{\theta})=2\boldsymbol{X}^T\boldsymbol{y}. \tag{3.59}$$

Because $\mathrm{sgn}(\boldsymbol{\theta})$ depends on the unknown in a non-differentiable way, Eq. (3.59) cannot be solved by a matrix inversion, and no analogue of Eqs. (3.8) or (3.44) exists. The problem is nonetheless *convex* – a sum of a convex quadratic and a convex norm – so it has a global minimum and can be solved reliably; it is the closed form that is lost, not the solution.

The orthonormal case and soft thresholding.

The behaviour becomes transparent in the simplest possible design. Take $\boldsymbol{X}=\boldsymbol{I}$ with $n=p$, so that $\tilde{\boldsymbol{y}}=\boldsymbol{\theta}$ and the problem decouples completely into p scalar problems. For OLS,

$$C(\boldsymbol{\theta})=\sum_{i=0}^{p-1}(y_i-\theta_i)^2, \qquad \hat{\theta}_i^{\mathrm{OLS}}=y_i. \tag{3.60}$$

For Ridge,

$$C(\boldsymbol{\theta})=\sum_i(y_i-\theta_i)^2+\lambda\sum_i\theta_i^2, \qquad \hat{\theta}_i^{\mathrm{Ridge}}=\frac{y_i}{1+\lambda}, \tag{3.61}$$

in agreement with Eq. (3.49). For the Lasso,

$$C(\boldsymbol{\theta})=\sum_i(y_i-\theta_i)^2+\lambda\sum_i|\theta_i|, \tag{3.62}$$

and treating each i separately, the derivative for $\theta_i\neq 0$ is $-2(y_i-\theta_i)+\lambda\,\mathrm{sgn}(\theta_i)=0$. If $\theta_i>0$ this gives $\theta_i=y_i-\lambda/2$, which is consistent with $\theta_i>0$ only when $y_i>\lambda/2$; if $\theta_i<0$ it gives $\theta_i=y_i+\lambda/2$, consistent only when $y_i<-\lambda/2$. For $|y_i|\le\lambda/2$ neither branch is consistent and the minimum lies at the kink $\theta_i=0$. Collecting,

$$\boxed{\hat{\theta}_i^{\mathrm{Lasso}}=\begin{cases}y_i-\lambda/2 & y_i>\lambda/2,\\ y_i+\lambda/2 & y_i<-\lambda/2,\\ 0 & |y_i|\le\lambda/2.\end{cases}} \tag{3.63}$$

This is the *soft thresholding* operator, written compactly as

$$S_{\lambda/2}(y)=\mathrm{sgn}(y)\,\max\left(|y|-\lambda/2,\,0\right). \tag{3.64}$$

Compare the three results. OLS leaves the coefficient alone. Ridge multiplies it by $1/(1+\lambda)$, shrinking it towards zero but reaching zero only in the limit $\lambda\to\infty$. The Lasso subtracts a constant $\lambda/2$ from its magnitude and *truncates at zero*, so every coefficient smaller than the threshold is eliminated outright at finite λ. The difference between shrinkage and selection is contained in this one comparison.

Why the geometry does it.

The constrained forms explain the same fact without any calculus. Ridge minimises the squared error subject to $\|\boldsymbol{\theta}\|_2^2\le t$, a ball; the Lasso subject to $\|\boldsymbol{\theta}\|_1\le t$, a cross-polytope – a diamond in two dimensions. The solution lies where the elliptical contours of the squared error

first touch the constraint region. A ball has no distinguished points, so the contact occurs at a generic location with all coordinates non-zero. A diamond has corners, and the corners lie *on the axes*, where some coordinates vanish. Contours are far more likely to meet a protruding corner than a flat face, and every corner is a solution with a zero coefficient. In p dimensions the 1-norm ball has corners, edges and faces of every dimension, each corresponding to a different set of variables being eliminated.

Figure 3.2 makes the argument concrete for two parameters, where the ball becomes a circle and the diamond a rhombus. Both panels show the *same* squared-error contours – ellipses centred on the unconstrained minimum $\hat{\boldsymbol{\theta}}_{\mathrm{OLS}}$, with shape set by the Hessian $\boldsymbol{X}^T\boldsymbol{X}$ – and the *same* budget t; the constrained solutions are computed exactly, so the points of contact in the figure are not sketched but solved for. Three things are worth reading off. First, on the circle the touching point is a genuine tangency at a generic location: the Ridge solution is pulled towards the origin, from $(2.0, 0.5)$ to $(0.92, 0.39)$ here, but *both* coordinates survive – shrinkage without selection, exactly as Eq. (3.49) predicted. Second, on the rhombus the first contour to arrive strikes the protruding corner $(t, 0)$, and the corner lies on an axis: $\hat{\theta}_2 = 0$ exactly, at finite t, with nothing gradual about it. Third – and this is why the Lasso selects so readily – a corner does not need tangency at all. The corner is the solution whenever the negative cost gradient there lies anywhere inside the *cone* spanned by the normals of its two adjacent edges, so a whole open set of positions of $\hat{\boldsymbol{\theta}}_{\mathrm{OLS}}$ is claimed by each corner, while each point on a smooth boundary claims only the single direction that is exactly normal to it. Selection is the rule, not a lucky accident of where the data happen to lie. The figure also displays the two limits of the constrained problem: shrinking t towards zero collapses both regions, and both estimators, onto the origin, while any $t \geq \|\hat{\boldsymbol{\theta}}_{\mathrm{OLS}}\|$ (in the corresponding norm) makes the constraint inactive and returns ordinary least squares – the one-to-one correspondence between t and λ of Proposition 3.5. The normal-cone picture at the corner is precisely the subdifferential condition that the next paragraph derives: Proposition 3.7's statement that a coefficient stays at zero as long as $|\frac{2}{n}\boldsymbol{x}_j^T\boldsymbol{r}| \leq \lambda$ is the algebraic form of "the gradient points into the cone".

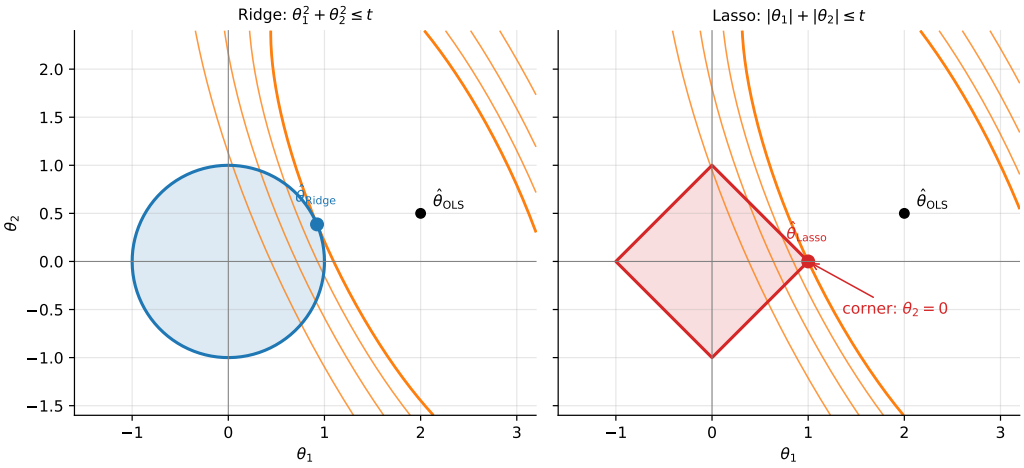


Fig. 3.2 The constrained forms of Ridge and Lasso for two parameters. Both panels share the same squared-error contours – ellipses centred on $\hat{\boldsymbol{\theta}}_{\mathrm{OLS}} = (2.0, 0.5)$ with shape given by the Hessian $\boldsymbol{X}^T\boldsymbol{X}$ – and the same budget $t = 1$; the heavier contour is the first to touch each constraint region, and the touching point is the exactly computed constrained solution. On the Ridge circle (left) the contact is a tangency at a generic point and both coordinates remain non-zero. On the Lasso rhombus (right) the contour reaches the corner $(1, 0)$ first, so that $\hat{\theta}_2 = 0$ exactly: the corner solves the problem for a whole cone of gradient directions, which is the geometric origin of the selection property.

Optimality conditions, done properly.

Equation (3.59) was obtained by ignoring the point at which sgn is undefined. The difficulty is removed rather than ignored by the *subdifferential*: for a convex function g, the subdifferential at a point is the set of slopes of all lines lying below the graph and touching it there, and for the absolute value

$$\partial\,|\theta| = \begin{cases} \{+1\} & \theta > 0, \\ \{-1\} & \theta < 0, \\ [-1,1] & \theta = 0. \end{cases} \tag{3.65}$$

A convex function is minimised exactly where 0 belongs to its subdifferential, which is the generalisation of "the derivative vanishes" and reduces to it wherever the function is differentiable.

Proposition 3.7 (Lasso optimality). *A vector $\hat{\boldsymbol{\theta}}$ minimises the Lasso objective (3.57) if and only if, writing $\boldsymbol{r} = \boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{\theta}}$ for the residual and $\boldsymbol{x}_j$ for the jth column,*

$$\frac{2}{n}\boldsymbol{x}_j^{T}\boldsymbol{r} = \lambda\,\operatorname{sgn}\!\big(\hat{\theta}_j\big) \quad \text{if } \hat{\theta}_j \neq 0, \qquad \left|\frac{2}{n}\boldsymbol{x}_j^{T}\boldsymbol{r}\right| \le \lambda \quad \text{if } \hat{\theta}_j = 0. \tag{3.66}$$

Proof. The objective is the sum of the differentiable term $\|\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}\|_2^2/n$, whose gradient is $-\frac{2}{n}\boldsymbol{X}^{T}\boldsymbol{r}$, and of $\lambda\|\boldsymbol{\theta}\|_1$, whose subdifferential is the product of the sets (3.65) taken coordinatewise. For a sum of a differentiable convex function and a convex one the subdifferential is the sum of the two, so $\boldsymbol{0}$ lies in it exactly when $\frac{2}{n}\boldsymbol{x}_j^{T}\boldsymbol{r} \in \lambda\,\partial|\hat{\theta}_j|$ for every j, which is Eq. (3.66). □

Read the second half of Eq. (3.66) again, because it is the selection property in its exact form. A coefficient is zero not by accident but because its column's correlation with the residual is *too small to pay the price* λ; the variable is excluded as long as it fails to earn its penalty. Two corollaries follow immediately.

Proposition 3.8 (The penalty that kills everything). *The solution is $\hat{\boldsymbol{\theta}} = \boldsymbol{0}$ if and only if*

$$\lambda \;\ge\; \lambda_{\max} = \frac{2}{n}\left\|\boldsymbol{X}^{T}\boldsymbol{y}\right\|_{\infty} = \frac{2}{n}\max_{j}\left|\boldsymbol{x}_j^{T}\boldsymbol{y}\right|. \tag{3.67}$$

Proof. Put $\hat{\boldsymbol{\theta}} = \boldsymbol{0}$ in Eq. (3.66). The residual is then $\boldsymbol{y}$, every coefficient is zero, and the second condition reads $|\frac{2}{n}\boldsymbol{x}_j^{T}\boldsymbol{y}| \le \lambda$ for all j, which is Eq. (3.67). Since the conditions are necessary and sufficient, the equivalence is exact. □

Equation (3.67) is why every Lasso implementation computes a path of solutions on a geometric grid running down from $\lambda_{\max}$: above it there is nothing to compute, and below it the solution changes continuously.

Proposition 3.9 (The fit is unique even when the coefficients are not). *If $\hat{\boldsymbol{\theta}}^{(1)}$ and $\hat{\boldsymbol{\theta}}^{(2)}$ both minimise Eq. (3.57), then $\boldsymbol{X}\hat{\boldsymbol{\theta}}^{(1)} = \boldsymbol{X}\hat{\boldsymbol{\theta}}^{(2)}$ and $\|\hat{\boldsymbol{\theta}}^{(1)}\|_1 = \|\hat{\boldsymbol{\theta}}^{(2)}\|_1$.*

Proof. Let $c^{\star}$ be the common minimum value and consider the midpoint $\bar{\boldsymbol{\theta}} = \frac{1}{2}(\hat{\boldsymbol{\theta}}^{(1)} + \hat{\boldsymbol{\theta}}^{(2)})$. The 1-norm is convex, so $\|\bar{\boldsymbol{\theta}}\|_1 \le \frac{1}{2}\|\hat{\boldsymbol{\theta}}^{(1)}\|_1 + \frac{1}{2}\|\hat{\boldsymbol{\theta}}^{(2)}\|_1$. The squared error is *strictly* convex along any direction in which $\boldsymbol{X}\boldsymbol{\theta}$ changes, so if $\boldsymbol{X}\hat{\boldsymbol{\theta}}^{(1)} \neq \boldsymbol{X}\hat{\boldsymbol{\theta}}^{(2)}$ then the squared error at the midpoint is strictly less than the average of the two. Adding the two inequalities gives an objective value at $\bar{\boldsymbol{\theta}}$ strictly below $c^{\star}$, a contradiction. Hence the fits coincide, the squared-error terms are equal, and the 1-norms must therefore be equal too. □

This is the precise sense in which the instability noted in the notebox below is a property of the parameterisation and not of the prediction. With two identical columns the Lasso may place all the weight on either, or split it, and every such choice has the same 1-norm and gives the same fitted values; what is arbitrary is the attribution, not the answer.

Coordinate descent.

Equation (3.63) is exact only for an orthonormal design, but it suggests the algorithm that solves the general problem. Suppose we fix all coefficients but one and minimise over θ_j alone. Writing the partial residual with the jth contribution removed,

$$\boldsymbol{r}^{(j)} = \boldsymbol{y} - \sum_{k \neq j} \boldsymbol{x}_k \theta_k, \tag{3.68}$$

where $\boldsymbol{x}_k$ is column k, the one-dimensional problem is exactly of the form solved above, and its answer is

$$\theta_j \leftarrow \frac{S_{\lambda/2}\left(\boldsymbol{x}_j^T \boldsymbol{r}^{(j)}\right)}{\boldsymbol{x}_j^T \boldsymbol{x}_j}, \tag{3.69}$$

which for standardised columns has $\boldsymbol{x}_j^T \boldsymbol{x}_j = n$. Cycling through the coordinates until the parameters stop changing is *coordinate descent*, and because the cost is convex and the non-differentiable part is separable – it is a sum of terms each involving one coordinate – the procedure is guaranteed to converge to the global minimum. This is what `sklearn.linear_model.Lasso` runs. The reader will recognise the pattern from the Gauss-Seidel iteration of Eq. (1.92): update one coordinate at a time, using the most recent values of all the others.

```python
import numpy as np

def soft_threshold(z, gamma):
    """Soft thresholding operator S_gamma(z), Eq. (3.softthresholdop)."""
    return np.sign(z) * np.maximum(np.abs(z) - gamma, 0.0)


def lasso_coordinate_descent(X, y, lmbda, n_iter=1000, tol=1e-8):
    """Lasso by cyclic coordinate descent.

    Minimises ||y - X theta||^2 / n + lmbda * ||theta||_1.
    The columns of X are assumed centred and standardised, and no
    intercept is penalised -- see Section on scaling and the intercept.
    """
    n, p = X.shape
    theta = np.zeros(p)
    col_norms = np.sum(X**2, axis=0)
    r = y - X @ theta                              # full residual

    for _ in range(n_iter):
        theta_old = theta.copy()
        for j in range(p):
            # partial residual: add back the current contribution of column j
            r += X[:, j] * theta[j]
            rho = X[:, j] @ r
            theta[j] = soft_threshold(rho, lmbda * n / 2.0) / col_norms[j]
            r -= X[:, j] * theta[j]                # remove the updated one
        if np.max(np.abs(theta - theta_old)) < tol:
            break

    return theta
```

Machine learning connection. The selection property makes the Lasso the natural first tool when p is large and most features are believed irrelevant – gene expression studies, text features, high-order polynomial bases. It has real limitations. When several features are strongly correlated the Lasso tends to pick one arbitrarily and discard the rest, which is unstable under resampling and misleading if the discarded features are scientifically meaningful; when $p > n$ it can select at most n variables. The *elastic net*, which penalises $\alpha\|\boldsymbol{\theta}\|_1 + (1-\alpha)\|\boldsymbol{\theta}\|_2^2$, was designed to repair both defects by combining selection with the grouping behaviour of Ridge. Note finally that the sparsity is a property of the 1-norm, not of regression: the same penalty produces sparse solutions in compressed sensing, in dictionary learning and in the pruning of neural networks.

3.10 Comparing the three estimators

It is worth seeing the three methods act on the same small problem. Take the targets and design matrix

$$\boldsymbol{y} = \begin{bmatrix} 4 \\ 2 \\ 3 \end{bmatrix}, \qquad \boldsymbol{X} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}, \tag{3.70}$$

so that there are three observations, two features and two parameters. The third row contributes nothing to the fit but does contribute to the error, which is what makes the example non-trivial. Since $\boldsymbol{X}\boldsymbol{\theta} = (2\theta_0, \theta_1, 0)^T$, the unpenalised cost is

$$C(\boldsymbol{\theta}) = (4 - 2\theta_0)^2 + (2 - \theta_1)^2 + 3^2, \tag{3.71}$$

minimised at

$$\hat{\boldsymbol{\theta}}_{\mathrm{OLS}} = \left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T\boldsymbol{y} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}. \tag{3.72}$$

Adding the Ridge penalty gives $C(\boldsymbol{\theta}) = (4-2\theta_0)^2 + (2-\theta_1)^2 + \lambda(\theta_0^2 + \theta_1^2)$ up to the constant, and differentiating with respect to each parameter separately,

$$\hat{\boldsymbol{\theta}}_{\mathrm{Ridge}} = \begin{bmatrix} \dfrac{8}{4+\lambda} \\[2ex] \dfrac{2}{1+\lambda} \end{bmatrix}, \tag{3.73}$$

which reproduces Eq. (3.72) at $\lambda = 0$ and decays to zero as λ grows. Notice that the two coefficients are shrunk by *different* factors, $4/(4+\lambda)$ and $1/(1+\lambda)$: the penalty is applied uniformly in parameter space, but the curvature of the cost differs between directions, so the coefficient belonging to the better-determined feature resists shrinkage more strongly. This is Eq. (3.48) with $\sigma_0^2 = 4$ and $\sigma_1^2 = 1$. Neither coefficient ever becomes exactly zero.

The Lasso behaves differently. Because the problem is diagonal, each coefficient can be treated separately as in Eq. (3.63): differentiating $(2-\theta_1)^2 + \lambda|\theta_1|$ gives $\theta_1 = \max(2 - \lambda/2, 0)$, while $(4-2\theta_0)^2 + \lambda|\theta_0|$ gives $\theta_0 = \max\left((16-\lambda)/8, 0\right)$. The first coefficient is eliminated at $\lambda = 4$ and the second at $\lambda = 16$, both at finite λ and both exactly. Plotting all three coefficient paths against λ shows the distinction at a glance: the Ridge curves approach the axis asymptotically, the Lasso curves reach it and stop.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Ridge, Lasso
```

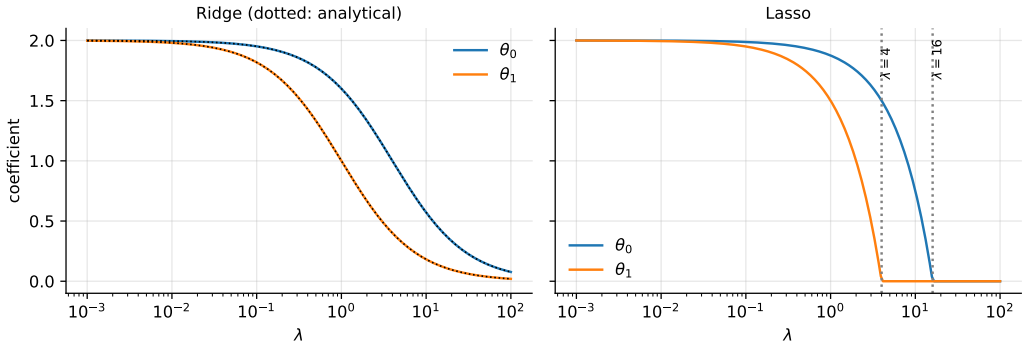


Fig. 3.3 Coefficient paths for the toy problem (3.70). The Ridge coefficients (left) follow the analytical result (3.73), shown dotted, and approach zero only asymptotically. The Lasso coefficients (right) reach zero at the finite thresholds $\lambda = 4$ and $\lambda = 16$ predicted by Eq. (3.63).

```
X = np.array([[2.0, 0.0], [0.0, 1.0], [0.0, 0.0]])
y = np.array([4.0, 2.0, 3.0])

n = X.shape[0]
lambdas = np.logspace(-3, 2, 200)

# scikit-learn's Ridge minimises ||y - X t||^2 + alpha ||t||^2, so alpha = lambda,
# but its Lasso minimises ||y - X t||^2 / (2n) + alpha ||t||_1, so alpha = lambda / (2n).
ridge_path = np.array([Ridge(alpha=l, fit_intercept=False).fit(X, y).coef_
                       for l in lambdas])
lasso_path = np.array([Lasso(alpha=l / (2 * n), fit_intercept=False,
                             max_iter=100000).fit(X, y).coef_
                       for l in lambdas])

# Analytical results: Eq. (3.toyridge) for Ridge, Eq. (3.softthreshold) for Lasso
ridge_exact = np.column_stack([8.0 / (4.0 + lambdas), 2.0 / (1.0 + lambdas)])
lasso_exact = np.column_stack([np.maximum((16.0 - lambdas) / 8.0, 0.0),
                               np.maximum(2.0 - lambdas / 2.0, 0.0)])
assert np.allclose(ridge_path, ridge_exact) and np.allclose(lasso_path, lasso_exact)

fig, ax = plt.subplots(1, 2, figsize=(10, 4), sharey=True)
for k in range(2):
    ax[0].semilogx(lambdas, ridge_path[:, k], label=rf"$\theta_{k}$")
    ax[1].semilogx(lambdas, lasso_path[:, k], label=rf"$\theta_{k}$")
ax[0].set_title("Ridge"); ax[1].set_title("Lasso")
for a in ax:
    a.set_xlabel(r"$\lambda$"); a.legend()
plt.show()
```

The two rescalings in that code deserve attention, because they are exactly the trap warned against in Section 3.13. `scikit-learn` minimises $\|\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}\|_2^2+\alpha\|\boldsymbol{\theta}\|_2^2$ for Ridge but $\|\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}\|_2^2/(2n)+\alpha\|\boldsymbol{\theta}\|_1$ for the Lasso – the same library uses *different* conventions for the two methods, so $\alpha=\lambda$ in one case and $\alpha=\lambda/(2n)$ in the other. With the conversions in place the computed paths agree with the analytical results (3.73) and (3.63) to machine precision, which is what the assertion checks. Such factors differ between every implementation and every textbook, and reconciling one's own code with a library means checking them explicitly rather than assuming.

3.11 A Bayesian reading

The three estimators can be derived a second time, from a viewpoint in which the parameters rather than the data are random. The unification is elegant, and it explains where the penalties come from rather than merely postulating them.

Bayes' theorem.

From the product rule for joint probabilities, $p(X,Y) = p(X \mid Y)p(Y) = p(Y \mid X)p(X)$, we obtain immediately

$$p(\boldsymbol{\theta} \mid \boldsymbol{D}) = \frac{p(\boldsymbol{D} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta})}{p(\boldsymbol{D})} \;\propto\; p(\boldsymbol{D} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta}), \tag{3.74}$$

dropping the normalisation $p(\boldsymbol{D})$, which does not depend on $\boldsymbol{\theta}$. The left-hand side is the *posterior*, the probability of the parameters given the data; on the right are the *likelihood* $p(\boldsymbol{D} \mid \boldsymbol{\theta})$, which we already modelled in Eq. (3.25), and the *prior* $p(\boldsymbol{\theta})$, which encodes what we believed about the parameters before seeing any data. Choosing the $\boldsymbol{\theta}$ that maximises the posterior is *maximum-a-posteriori* (MAP) estimation.

A Gaussian prior gives Ridge.

Suppose we believe, before seeing the data, that the parameters are small: independent, zero-mean Gaussians of variance τ^2,

$$p(\boldsymbol{\theta}) = \prod_{j=0}^{p-1} \exp\left(-\frac{\theta_j^2}{2\tau^2}\right). \tag{3.75}$$

The posterior is then the product of Eq. (3.25) and Eq. (3.75). Taking the negative logarithm and discarding terms independent of $\boldsymbol{\theta}$,

$$C(\boldsymbol{\theta}) = \frac{\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2}{2\sigma^2} + \frac{1}{2\tau^2}\,\|\boldsymbol{\theta}\|_2^2, \tag{3.76}$$

which, on identifying $\lambda = \sigma^2/\tau^2$ after multiplying through by $2\sigma^2$, is exactly the Ridge cost function (3.43).

A Laplace prior gives the Lasso.

Replace the Gaussian prior by a Laplace (double exponential) distribution with zero mean,

$$p(\boldsymbol{\theta}) = \prod_{j=0}^{p-1} \exp\left(-\frac{|\theta_j|}{\tau}\right). \tag{3.77}$$

The same steps give

$$C(\boldsymbol{\theta}) = \frac{\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2}{2\sigma^2} + \frac{1}{\tau}\,\|\boldsymbol{\theta}\|_1, \tag{3.78}$$

the Lasso cost function (3.57). A flat prior, expressing no preference at all, leaves only the likelihood and returns ordinary least squares.

What the unification tells us.

The strength of the prior and the strength of the penalty are the same quantity. A large λ corresponds to a small prior variance – a firm belief that the coefficients are near zero – and a small λ to a diffuse prior that lets the data speak. The difference between Ridge and the Lasso is now visibly a difference between two prior *shapes*: the Laplace density has a sharp peak at the origin and heavier tails than the Gaussian, so it simultaneously expresses a stronger belief that coefficients are exactly zero and a greater tolerance for the few that are large. That is precisely the behaviour of Eq. (3.63).

It should be said that MAP estimation is not the whole of Bayesian inference. It returns a single point, the mode of the posterior, and discards the distribution around it – which is the part a Bayesian would consider the answer. A full treatment would report the posterior itself, giving credible intervals directly rather than through the sampling argument of Eq. (3.34), and would integrate over $\boldsymbol{\theta}$ when making predictions instead of fixing it at the mode. We return to this when we discuss Bayesian neural networks and the Gaussian processes for which the Cholesky factorisation of Section 1.14 was introduced.

3.12 Kernel methods: regression without coordinates

Every model in this chapter is linear in the parameters and can be made non-linear in the inputs by the basis expansion of Section 3.2: replace $\boldsymbol{x}$ by $\boldsymbol{\phi}(\boldsymbol{x})$ and fit a linear model in the enlarged space. The difficulty is arithmetic. A quadratic expansion of d inputs has $\mathcal{O}(d^2)$ features, a cubic one $\mathcal{O}(d^3)$, and the map one would often *like* to use has infinitely many. This section shows that for Ridge regression the expansion need never be carried out, that the resulting method is an exact rewriting and not an approximation, and that the same device is the one Chapter 6 uses for support vector machines and Section 5.11 for logistic regression. It also shows, because it is true and usually left unsaid, that the device does *not* work for the Lasso.

3.12.1 An identity that moves the problem

Everything rests on one line of algebra.

Lemma 3.10 (The push-through identity). *For any $\boldsymbol{X} \in \mathbb{R}^{n\times p}$ and any $\lambda > 0$,*

$$\left(\boldsymbol{X}^T\boldsymbol{X} + \lambda \boldsymbol{I}_p\right)^{-1}\boldsymbol{X}^T = \boldsymbol{X}^T\left(\boldsymbol{X}\boldsymbol{X}^T + \lambda \boldsymbol{I}_n\right)^{-1}. \tag{3.79}$$

Both inverses exist, the left of a $p\times p$ matrix and the right of an $n\times n$ one.

Proof. Both matrices being inverted are symmetric with eigenvalues at least λ, hence invertible. Start from the trivial identity

$$\boldsymbol{X}^T\left(\boldsymbol{X}\boldsymbol{X}^T + \lambda \boldsymbol{I}_n\right) = \boldsymbol{X}^T\boldsymbol{X}\boldsymbol{X}^T + \lambda \boldsymbol{X}^T = \left(\boldsymbol{X}^T\boldsymbol{X} + \lambda \boldsymbol{I}_p\right)\boldsymbol{X}^T, \tag{3.80}$$

in which the middle expression is read in two ways. Multiplying on the left by $(\boldsymbol{X}^T\boldsymbol{X} + \lambda \boldsymbol{I}_p)^{-1}$ and on the right by $(\boldsymbol{X}\boldsymbol{X}^T + \lambda \boldsymbol{I}_n)^{-1}$ gives Eq. (3.79). $\square$

The name is descriptive: $\boldsymbol{X}^T$ is pushed through the inverse, and in doing so the matrix that must be inverted changes size from p to n. The identity fails for $\lambda = 0$, which is the first hint that a penalty is not optional here.

Theorem 3.11 (The dual form of Ridge regression). *The Ridge estimator (3.44) satisfies*

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}} = \boldsymbol{X}^T \boldsymbol{\alpha}, \qquad \boldsymbol{\alpha} = (\boldsymbol{K} + \lambda \boldsymbol{I}_n)^{-1} \boldsymbol{y}, \qquad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{X}^T, \tag{3.81}$$

so that the fitted function at a new point is

$$\tilde{y}(\boldsymbol{x}) = \boldsymbol{x}^T \hat{\boldsymbol{\theta}}_{\text{Ridge}} = \sum_{i=0}^{n-1} \alpha_i \boldsymbol{x}_i^T \boldsymbol{x}. \tag{3.82}$$

Proof. Apply Lemma 3.10 to Eq. (3.44): $\hat{\boldsymbol{\theta}} = (\boldsymbol{X}^T\boldsymbol{X} + \lambda \boldsymbol{I})^{-1}\boldsymbol{X}^T\boldsymbol{y} = \boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{X}^T + \lambda \boldsymbol{I})^{-1}\boldsymbol{y} = \boldsymbol{X}^T\boldsymbol{\alpha}$. Equation (3.82) is then $\boldsymbol{x}^T\boldsymbol{X}^T\boldsymbol{\alpha}$ written out. □

Look at what Eqs. (3.81) and (3.82) contain. The matrix $\boldsymbol{K}$ has entries $K_{ij} = \boldsymbol{x}_i^T\boldsymbol{x}_j$; the prediction needs $\boldsymbol{x}_i^T\boldsymbol{x}$. *The data enter only through inner products.* The individual vectors $\boldsymbol{x}_i$ are never needed once the $n \times n$ matrix of their inner products is known, and the parameter vector $\hat{\boldsymbol{\theta}}$ – which lives in the feature space and may be enormous – is never formed. The estimator has been rewritten in terms of n numbers α_i, one per observation, instead of p numbers θ_j, one per feature.

```
=== 1. the push-through identity, Lemma 3.pushthrough ===
     n     p   lambda   max |LHS - RHS|   ||theta_p - theta_n||
    20     5     0.01         4.012e-14               8.120e-14
    20    50     0.01         1.386e-13               4.110e-13
   200     3     0.01         1.817e-13               3.997e-13
    50    50     0.01         1.989e-13               2.683e-12
```

3.12.2 Kernels

Now perform the basis expansion. Fitting Ridge regression to the expanded design matrix $\boldsymbol{\Phi}$ with rows $\boldsymbol{\phi}(\boldsymbol{x}_i)^T$, Theorem 3.11 gives the same formulas with $K_{ij} = \boldsymbol{\phi}(\boldsymbol{x}_i)^T\boldsymbol{\phi}(\boldsymbol{x}_j)$. Define the *kernel*

$$k(\boldsymbol{x}, \boldsymbol{x}') = \boldsymbol{\phi}(\boldsymbol{x})^T \boldsymbol{\phi}(\boldsymbol{x}'), \tag{3.83}$$

which is the same object as Eq. (6.30) in the chapter on support vector machines. Then

$$\boxed{\boldsymbol{\alpha} = (\boldsymbol{K} + \lambda \boldsymbol{I})^{-1}\boldsymbol{y}, \qquad K_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_j), \qquad \tilde{y}(\boldsymbol{x}) = \sum_{i=0}^{n-1} \alpha_i k(\boldsymbol{x}_i, \boldsymbol{x})} \tag{3.84}$$

is *kernel ridge regression*. Its cost is the $\mathcal{O}(n^3)$ of one linear solve, whatever the dimension of $\boldsymbol{\phi}$ – including infinite.

The reason this is not a swindle is that for many useful maps the inner product has a closed form which is cheaper than the map. For the quadratic map of Eq. (6.28) the kernel is $(\boldsymbol{x}^T\boldsymbol{x}')^2$; the polynomial and Gaussian kernels of Section 6.7.1 are the standard choices, and Mercer's theorem stated there says exactly which functions k arise from some $\boldsymbol{\phi}$ in this way: the symmetric positive semi-definite ones. Everything in Section 6.7.1 applies verbatim here, and nothing in this section is specific to regression.

```
=== 3. kernel ridge regression, Theorem 3.krr ===
  (a) the quadratic kernel against its explicit feature map
      max |k(x,x') - phi(x).phi(x')| : 1.421e-14
      max |f_kernel(x) - f_primal(x)|: 1.915e-14
```

The second line is the claim of this section, measured: the function obtained from the 6×6 primal system with an explicit feature map and the function obtained from the 60×60 kernel system are the same function. Against `scikit-learn`, whose `KernelRidge` solves the identical system with its `alpha` playing the part of our λ:

```
  (b) the Gaussian kernel, ours against scikit-learn
      gamma    lambda    our test MSE   sklearn test MSE   max |f_ours - f_sk|
       0.25     0.001        0.008634           0.008634            1.847e-13
       1.00     0.001        0.056491           0.056491            8.349e-14
       4.00     0.100        0.037478           0.037478            1.277e-15
```

```python
import numpy as np

def gaussian_kernel(A, B, gamma):
    """k(x,x') = exp(-gamma ||x - x'||^2), the Gaussian kernel of Section 6.mercer."""
    d2 = (A**2).sum(1)[:, None] + (B**2).sum(1)[None, :] - 2.0 * A @ B.T
    return np.exp(-gamma * np.maximum(d2, 0.0))


def kernel_ridge_fit(K, y, lmbda):
    """alpha = (K + lambda I)^{-1} y, Eq. (3.krr).  One n x n solve."""
    return np.linalg.solve(K + lmbda * np.eye(len(y)), y)


def kernel_ridge_predict(alpha, Xtrain, Xnew, gamma):
    """f(x) = sum_i alpha_i k(x_i, x), Eq. (3.krr)."""
    return gaussian_kernel(Xnew, Xtrain, gamma) @ alpha
```

Which side to solve. Theorem 3.11 gives two routes to the same estimator, and the choice between them is arithmetic. The primal system is $p \times p$ and the dual $n \times n$, so the primal wins whenever $p < n$ and the dual whenever $p > n$; with a thousand observations and ten features one should never form a kernel matrix, and with a hundred observations and a million features one should never form $\boldsymbol{X}^T\boldsymbol{X}$. The kernel route additionally costs $\mathcal{O}(n^2)$ *memory*, which is what limits it to sample sizes in the tens of thousands and is the reason for the approximations of Section 3.12.4. A method that scales with the number of observations rather than the number of features is not automatically an improvement; it is a different trade.

3.12.3 Why the same trick works for any loss: the representer theorem

Theorem 3.11 was proved by an algebraic identity special to the squared error. The result is far more general, and stating it in the general form is what allows Chapters 5 and 6 to use it without repeating the derivation.

Let $\mathscr{H}$ be the space of functions spanned by $\{k(\boldsymbol{x}, \cdot)\}$ with the inner product determined by $\langle k(\boldsymbol{x}, \cdot), k(\boldsymbol{x}', \cdot)\rangle = k(\boldsymbol{x}, \boldsymbol{x}')$ – the *reproducing kernel Hilbert space* of k. The name records its one essential property, that $\langle f, k(\boldsymbol{x}, \cdot)\rangle = f(\boldsymbol{x})$ for every $f \in \mathscr{H}$: evaluation at a point is an inner product.

Theorem 3.12 (Representer theorem). *Let L be any function of its arguments and let Ω be strictly increasing on $[0, \infty)$. Then every minimiser of*

$$\min_{f \in \mathscr{H}} \sum_{i=0}^{n-1} L(y_i, f(\boldsymbol{x}_i)) + \Omega(\|f\|_{\mathscr{H}}) \tag{3.85}$$

admits the finite expansion

$$f(\cdot) = \sum_{i=0}^{n-1} \alpha_i k(\boldsymbol{x}_i, \cdot) \tag{3.86}$$

for some $\boldsymbol{\alpha} \in \mathbb{R}^n$.

Proof. Let $\mathscr{S}$ be the span of $k(\boldsymbol{x}_0,\cdot),\dots,k(\boldsymbol{x}_{n-1},\cdot)$ and decompose any $f \in \mathscr{H}$ orthogonally as $f = f_\parallel + f_\perp$ with $f_\parallel \in \mathscr{S}$ and $f_\perp \perp \mathscr{S}$. By the reproducing property,

$$f(\boldsymbol{x}_i) = \langle f, k(\boldsymbol{x}_i,\cdot)\rangle = \big\langle f_\parallel, k(\boldsymbol{x}_i,\cdot)\big\rangle + \underbrace{\langle f_\perp, k(\boldsymbol{x}_i,\cdot)\rangle}_{=0} = f_\parallel(\boldsymbol{x}_i),$$

so the component $f_\perp$ has no effect whatever on the data-fitting term: the loss sees only $f_\parallel$. It does affect the penalty, since by orthogonality $\|f\|_{\mathscr{H}}^2 = \|f_\parallel\|_{\mathscr{H}}^2 + \|f_\perp\|_{\mathscr{H}}^2 \geq \|f_\parallel\|_{\mathscr{H}}^2$, with equality only when $f_\perp = 0$. Since Ω is strictly increasing, deleting $f_\perp$ leaves the loss unchanged and strictly decreases the penalty unless it was already zero. A minimiser therefore has $f_\perp = 0$, that is $f \in \mathscr{S}$, which is Eq. (3.86). □

This is a remarkable statement and it is worth pausing on. The minimisation is over an infinite-dimensional space of functions; the answer is guaranteed to lie in the n-dimensional subspace spanned by the kernel evaluated at the training points. *The data determine the dimension of the problem, not the model.* With the squared loss and $\Omega(t) = \frac{\lambda}{2}t^2$ the theorem reproduces Eq. (3.84); with the logistic loss it gives the kernel logistic regression of Section 5.11; with the hinge loss it gives the support vector machine of Chapter 6, whose expansion (6.33) over support vectors is Eq. (3.86) with most coefficients equal to zero. Why the hinge loss and only the hinge loss produces those zeros is Theorem 6.1.

3.12.4 There is no kernel Lasso

It is natural to ask for the same treatment of the Lasso, and the request appears in the literature under the name *kernel Lasso*. The honest answer is that the construction that works for Ridge regression does not exist here, and it is worth being precise about why, because the reason is instructive rather than technical.

Theorem 3.12 requires the penalty to be a strictly increasing function of $\|f\|_{\mathscr{H}}$. The Lasso penalty is $\|\boldsymbol{\theta}\|_1$, a function not of f but of the *coordinates* of f in the feature space – and those coordinates are not determined by the kernel.

Proposition 3.13 (The 1-norm is not a property of the function). *Let $\boldsymbol{\phi}$ be a feature map for k and let $\boldsymbol{R}$ be any orthogonal matrix on the feature space. Then $\boldsymbol{R}\boldsymbol{\phi}$ is also a feature map for k, the two parameterisations $\boldsymbol{\theta}$ and $\boldsymbol{R}\boldsymbol{\theta}$ describe the same function, and*

$$\|\boldsymbol{R}\boldsymbol{\theta}\|_2 = \|\boldsymbol{\theta}\|_2 \quad \textit{always}, \qquad \|\boldsymbol{R}\boldsymbol{\theta}\|_1 \neq \|\boldsymbol{\theta}\|_1 \quad \textit{in general}. \tag{3.87}$$

Consequently $\|\boldsymbol{\theta}\|_1$ is not a functional of f, and no penalty built from it can be expressed through the kernel alone.

Proof. Two lines of algebra:

$$(\boldsymbol{R}\boldsymbol{\phi}(\boldsymbol{x}))^T\big(\boldsymbol{R}\boldsymbol{\phi}(\boldsymbol{x}')\big) = \boldsymbol{\phi}(\boldsymbol{x})^T\boldsymbol{R}^T\boldsymbol{R}\boldsymbol{\phi}(\boldsymbol{x}') = k(\boldsymbol{x},\boldsymbol{x}'), \qquad (\boldsymbol{R}\boldsymbol{\theta})^T(\boldsymbol{R}\boldsymbol{\phi}(\boldsymbol{x})) = \boldsymbol{\theta}^T\boldsymbol{\phi}(\boldsymbol{x}),$$

so $\boldsymbol{R}\boldsymbol{\phi}$ has the same kernel and describes the same function. The 2-norm is invariant because $\boldsymbol{R}$ is orthogonal. The 1-norm is not invariant under rotations – a rotation by $\pi/4$ turns $(1,0)$, of 1-norm one, into $(\frac{1}{\sqrt{2}},\frac{1}{\sqrt{2}})$, of 1-norm $\sqrt{2}$. □

Measured on the quadratic feature map of this chapter's example, with a random orthogonal $\boldsymbol{R}$:

```
quantity                                value (phi)    value (R phi)
||theta||_2                              0.302326        0.302326
||theta||_1                              0.414053        0.637376
max |f(x) - f_rotated(x)| on test       5.551e-16
```

The two weight vectors describe the same function to machine precision and disagree in 1-norm by more than fifty per cent. A Lasso in feature space is therefore not a well-posed problem given only a kernel: it depends on which square root of $\boldsymbol{K}$ one happens to have chosen, and Mercer's theorem supplies no canonical choice. Two repairs are available, and they do different things.

Repair one: penalise the dual coefficients.

Take the representation (3.86) as given and put the 1-norm on $\boldsymbol{\alpha}$ instead of on $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\alpha}\in\mathbb{R}^n} \left\{ \frac{1}{n} \|\boldsymbol{y} - \boldsymbol{K}\boldsymbol{\alpha}\|_2^2 + \mu \|\boldsymbol{\alpha}\|_1 \right\}. \tag{3.88}$$

This is well posed – $\boldsymbol{\alpha}$ is determined by the kernel – and it is nothing more mysterious than an ordinary Lasso whose design matrix happens to be the Gram matrix. Everything proved in Section 3.9 therefore applies unchanged: Proposition 3.7 gives the optimality conditions with $\boldsymbol{x}_j$ replaced by the jth column $\boldsymbol{k}_j$ of $\boldsymbol{K}$,

$$\frac{2}{n}\boldsymbol{k}_j^T(\boldsymbol{K}\boldsymbol{\alpha} - \boldsymbol{y}) = -\mu\,\mathrm{sgn}(\alpha_j) \;\text{ if } \alpha_j \neq 0, \qquad \left|\frac{2}{n}\boldsymbol{k}_j^T(\boldsymbol{K}\boldsymbol{\alpha} - \boldsymbol{y})\right| \leq \mu \;\text{ if } \alpha_j = 0, \tag{3.89}$$

Proposition 3.8 gives $\mu_{\max} = \frac{2}{n}\|\boldsymbol{K}^T\boldsymbol{y}\|_\infty$, and the coordinate descent of Eq. (3.69) solves it.

What must be understood is *what* is being selected. The columns of $\boldsymbol{K}$ are indexed by training points, so Eq. (3.88) is sparse in the *examples* and not in the features: it discards data points, not variables. That is a useful thing – it produces a prediction rule that evaluates the kernel at a handful of points rather than all n, which is exactly the economy support vectors provide in Chapter 6 – but it is not what the Lasso does in Section 3.9, and calling both by the same name has caused a good deal of confusion.

```
      mu      non-zero alpha of 60   train MSE   max KKT violation
  0.0001                        28    0.002055            1.266e-06
  0.0010                        20    0.003362            1.416e-07
  0.0100                         7    0.008227            2.227e-13
  0.1000                         3    0.059848            1.688e-13
```

```
def kernel_lasso(K, y, mu, n_iter=20000, tol=1e-12):
    """Cyclic coordinate descent on ||y - K a||^2 / n + mu ||a||_1.

    Identical to lasso_coordinate_descent above with the Gram matrix as the
    design; the solution is sparse in the training points, Eq. (3.klasso).
    """
    n = len(y)
    a = np.zeros(n)
    cn = (K**2).sum(0)
    r = y - K @ a
    for _ in range(n_iter):
        a_old = a.copy()
        for j in range(n):
```

```
            r += K[:, j] * a[j]
            a[j] = soft_threshold(K[:, j] @ r, mu * n / 2.0) / cn[j]
            r -= K[:, j] * a[j]
        if np.abs(a - a_old).max() < tol:
            break
    return a
```

The last column above is the largest violation of Eq. (3.89) at the returned solution, and it is the check that should accompany any implementation of a non-differentiable problem: an optimality condition that can be evaluated is worth more than a convergence message. The measured $\mu_{\max}$ for this problem is 0.290436, and the solver returns exactly zero non-zero coefficients just above it, as Proposition 3.8 requires.

Repair two: build the feature map explicitly, then use the Lasso.

If sparsity in *features* is what is wanted, the honest route is to construct an explicit finite map $\boldsymbol{z}(\boldsymbol{x}) \in \mathbb{R}^m$ whose inner products approximate the kernel, $\boldsymbol{z}(\boldsymbol{x})^T \boldsymbol{z}(\boldsymbol{x}') \approx k(\boldsymbol{x}, \boldsymbol{x}')$, and then run the ordinary Lasso of Section 3.9 in that basis. Two constructions are standard. The *Nyström* map picks m landmark points, forms the $m \times m$ kernel matrix $\boldsymbol{K}_{mm}$ among them, and sets $\boldsymbol{Z} = \boldsymbol{K}_{nm} \boldsymbol{K}_{mm}^{-1/2}$; it is exact when the landmarks are all n points. *Random Fourier features* exploit Bochner's theorem, which represents a shift-invariant kernel as the Fourier transform of a probability measure, and estimate that integral by sampling m frequencies.

Both make an approximation the exact kernel method does not make, and both give back what the exact method cannot: a finite, named set of features among which the 1-norm can genuinely select. A Nyström feature is a landmark point and a Fourier feature is a frequency, so the selected set is interpretable in a way that a sparse $\boldsymbol{\alpha}$ is not.

```
method            m    max |Z Z^T - K|   Lasso non-zeros   test MSE
Nystrom          10         9.937e-01                9    0.026111
Nystrom          30         2.947e-01               18    0.013798
Nystrom          60         6.439e-15               22    0.012815
Fourier          64         4.371e-01               18    0.013377
Fourier         256         1.668e-01               14    0.001599
Fourier        1024         1.105e-01               11    0.000907
```

The Nyström map reproduces the Gram matrix exactly once $m = n$, as it must; the random map converges only in expectation and slowly, its error falling like $m^{-1/2}$, which is the price of not looking at the data when choosing the features.

Machine learning connection. The pattern of this section recurs throughout the book and is worth naming. A method is characterised by a *penalty*, and a penalty is meaningful only relative to a *parameterisation*. The 2-norm survives a change of orthonormal basis and can therefore be expressed through the kernel, which is why Ridge regression kernelises and why the RKHS norm is the natural penalty on functions. The 1-norm does not survive, which is why the Lasso is a statement about a chosen set of features and cannot be lifted to a statement about a function. When a regulariser is proposed, the first question to ask is what it is invariant under, because that determines what it can be a penalty *on*.

3.13 Scaling, centring and the intercept

We now address a set of practical questions which cause more confusion than any of the mathematics above, and which are the usual reason a hand-written implementation disagrees with a library.

Why scaling matters for penalised regression.

The penalties $\|\boldsymbol{\theta}\|_2^2$ and $\|\boldsymbol{\theta}\|_1$ treat all coefficients alike, but a coefficient's magnitude depends on the units of its feature. Suppose one predictor is a person's height. Measured in millimetres the fitted coefficient is a thousand times smaller than the same coefficient measured in metres, and it is therefore penalised a thousand times less. The consequence is stark: for Ridge and the Lasso, a change of units is not a harmless relabelling but a change of model. Unless the features are already on comparable scales, they must be standardised – each column centred and divided by its standard deviation, as in Section 1.5 – before a penalty is applied. Ordinary least squares is exempt, since Eq. (3.8) is equivariant under rescaling of the columns; the fitted values do not change, only the coefficients, in a compensating way.

Why OLS is equivariant – and why the penalised methods are not.

Both claims of the previous paragraph deserve proof, and the proofs are three lines each. A change of units multiplies each column of the design matrix by a positive constant: measuring feature j in units a factor d_j smaller turns the column $\boldsymbol{x}_j$ into $d_j\boldsymbol{x}_j$ (a length recorded in millimetres rather than metres has $d_j = 1000$). Collecting the factors in $\boldsymbol{D} = \operatorname{diag}(d_0, \ldots, d_{p-1})$, the design matrix becomes $\boldsymbol{XD}$, and a new input rescales the same way. For ordinary least squares, assuming full column rank,

$$\hat{\boldsymbol{\theta}}(\boldsymbol{XD}) = \left(\boldsymbol{DX}^T\boldsymbol{XD}\right)^{-1}\boldsymbol{DX}^T\boldsymbol{y} = \boldsymbol{D}^{-1}\left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{D}^{-1}\boldsymbol{DX}^T\boldsymbol{y} = \boldsymbol{D}^{-1}\,\hat{\boldsymbol{\theta}}(\boldsymbol{X}),$$

using that diagonal matrices commute with themselves and invert entrywise. The coefficients rescale by exactly the inverse factors – a slope per millimetre is a thousandth of a slope per metre – and every prediction is untouched: the fitted values are $\boldsymbol{XD}\boldsymbol{D}^{-1}\hat{\boldsymbol{\theta}} = \boldsymbol{X}\hat{\boldsymbol{\theta}}$, and a new point $\boldsymbol{x}_*$, which in the new units reads $\boldsymbol{Dx}_*$, predicts $(\boldsymbol{Dx}_*)^T\boldsymbol{D}^{-1}\hat{\boldsymbol{\theta}} = \boldsymbol{x}_*^T\hat{\boldsymbol{\theta}}$. This is *equivariance*: the parametrisation moves, the model does not.

For Ridge regression the same computation tells a different story. Factor the regularised matrix as $\boldsymbol{DX}^T\boldsymbol{XD} + \lambda\boldsymbol{I} = \boldsymbol{D}\left(\boldsymbol{X}^T\boldsymbol{X} + \lambda\boldsymbol{D}^{-2}\right)\boldsymbol{D}$; inverting and multiplying by $\boldsymbol{DX}^T\boldsymbol{y}$ gives

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}}(\boldsymbol{XD};\lambda) = \boldsymbol{D}^{-1}\left(\boldsymbol{X}^T\boldsymbol{X} + \lambda\boldsymbol{D}^{-2}\right)^{-1}\boldsymbol{X}^T\boldsymbol{y}.$$

The rescaled problem is therefore *not* the original Ridge problem in disguise: it is ordinary-units Ridge with the spherical penalty $\lambda\|\boldsymbol{\theta}\|_2^2$ replaced by the axis-weighted penalty $\lambda\,\boldsymbol{\theta}^T\boldsymbol{D}^{-2}\boldsymbol{\theta}$ – an ellipsoid whose axes are set by the units. The fitted values $\boldsymbol{X}(\boldsymbol{X}^T\boldsymbol{X} + \lambda\boldsymbol{D}^{-2})^{-1}\boldsymbol{X}^T\boldsymbol{y}$ change unless $\boldsymbol{D}$ is a multiple of the identity (a *uniform* rescaling only relabels λ as λ/d^2 and stays within the Ridge family; it is the *relative* scales of the features that change the model). For the Lasso the substitution $\boldsymbol{\theta} = \boldsymbol{D}\boldsymbol{\theta}'$ in the rescaled objective gives

$$\left\|\boldsymbol{y} - \boldsymbol{XD}\boldsymbol{\theta}'\right\|_2^2 + \lambda\left\|\boldsymbol{\theta}'\right\|_1 = \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2 + \lambda\sum_{j=0}^{p-1}\frac{|\theta_j|}{d_j}\,:$$

each feature now pays its own price λ/d_j, and the feature recorded in millimetres pays a thousandth of what it paid in metres. A change of units is a change of the penalty, hence of the model – which coefficients are shrunk, and which are selected, becomes an artefact of the bookkeeping. The following program shows all three statements at work on one small data set, with the intercept handled by centring as derived below.

```
import numpy as np
from sklearn.linear_model import Lasso

rng = np.random.default_rng(2026)
n = 50
x1 = rng.random(n)                      # a length, measured in metres
x2 = rng.random(n)                      # a time, measured in seconds
y = 2.0 * x1 - 3.0 * x2 + 0.1 * rng.standard_normal(n)

X = np.column_stack([x1, x2])
D = np.diag([1000.0, 1.0])              # metres -> millimetres in column 1
XD = X @ D                              # the same data, in the new units

Xc, yc = X - X.mean(axis=0), y - y.mean()      # centred: intercept handled
XDc = XD - XD.mean(axis=0)                     # separately and unpenalised

lmbda = 0.1
def ols(X, y):
    return np.linalg.lstsq(X, y, rcond=None)[0]
def ridge(X, y, P):                     # generalised penalty lambda theta^T P theta
    return np.linalg.solve(X.T @ X + lmbda * P, X.T @ y)

I = np.eye(2)
t, tD = ols(Xc, yc), ols(XDc, yc)
print("OLS   coefficients (m,s):", t, "  (mm,s):", tD)
print("OLS   D theta_mm = theta_m:", np.allclose(np.diag(D) * tD, t),
      "  max |prediction change|: %.2e" % np.max(np.abs(XDc @ tD - Xc @ t)))

r, rD = ridge(Xc, yc, I), ridge(XDc, yc, I)
print("Ridge coefficients (m,s):", r, "  (mm,s):", rD)
print("Ridge max |prediction change|: %.3f" % np.max(np.abs(XDc @ rD - Xc @ r)))
r_mod = ridge(Xc, yc, np.linalg.inv(D @ D))    # penalty lambda theta^T D^-2 theta
print("Ridge on XD = modified-penalty ridge on X:",
      np.allclose(np.diag(D) * rD, r_mod))

la = Lasso(alpha=lmbda / 2, fit_intercept=False, max_iter=100000)
lm, lmm = la.fit(Xc, yc).coef_.copy(), la.fit(XDc, yc).coef_.copy()
print("Lasso coefficients (m,s):", lm, "  (mm,s):", lmm)
print("Lasso max |prediction change|: %.3f" % np.max(np.abs(XDc @ lmm - Xc @ lm)))
```

```
OLS   coefficients (m,s): [ 1.98213925 -2.97802972]   (mm,s): [ 1.98213925e-03
    -2.97802972e+00]
OLS   D theta_mm = theta_m: True   max |prediction change|: 8.88e-16
Ridge coefficients (m,s): [ 1.91639278 -2.90255695]   (mm,s): [ 1.98248632e-03
    -2.90233356e+00]
Ridge max |prediction change|: 0.033
Ridge on XD = modified-penalty ridge on X: True
Lasso coefficients (m,s): [ 1.12291528 -2.32899154]   (mm,s): [ 1.98426741e-03
    -2.32600432e+00]
Lasso max |prediction change|: 0.426
```

The output is the two propositions in numerical form. For OLS the millimetre coefficient is exactly a thousandth of the metre coefficient and the predictions agree to machine precision. For Ridge the predictions move – and the third printed line confirms *why*: the rescaled fit coincides, after undoing $\boldsymbol{D}$, with the original-units fit under the modified penalty $\lambda\boldsymbol{\theta}^T\boldsymbol{D}^{-2}\boldsymbol{\theta}$, to machine precision. For the Lasso the effect is largest: in metres the length coefficient is

shrunk from its OLS value 1.98 down to 1.12, while in millimetres it escapes the penalty almost entirely ($1000 \times 1.98 \cdot 10^{-3} \approx 1.98$) because its price is $\lambda/1000$ – same data, same λ, different model. Standardising the columns fixes $\boldsymbol{D}$ once and for all and makes the penalty unit-free, which is the real content of the advice that opened this section.

The intercept should not be penalised.

The intercept θ_0 is the expected target when all predictors are zero. Penalising it shrinks the fitted values towards zero rather than towards the mean of the data, which is almost never what is wanted and which makes the result depend on where the origin of the target happens to be. If we shift every y_i by a constant, we would like the fit simply to shift with it, and that fails if θ_0 is penalised.

The standard remedy is to centre both $\boldsymbol{X}$ and $\boldsymbol{y}$. If every column of $\boldsymbol{X}$ and the target $\boldsymbol{y}$ have had their means subtracted, the intercept of the centred problem is exactly zero, so it can be dropped from the design matrix altogether and the penalty then applies only to the genuine slopes. Where the recovery formula for the intercept comes from is worth the few lines it takes. Single the intercept out in the cost function, the design matrix now containing no column of ones,

$$C(\theta_0, \theta_1, \ldots, \theta_{p-1}) = \frac{1}{n} \sum_{i=0}^{n-1} \left(y_i - \theta_0 - \sum_{j=1}^{p-1} X_{ij} \theta_j \right)^2 .$$

At the optimum every partial derivative vanishes; for the intercept,

$$\frac{\partial C}{\partial \theta_0} = -\frac{2}{n} \sum_{i=0}^{n-1} \left(y_i - \theta_0 - \sum_{j=1}^{p-1} X_{ij} \theta_j \right) = 0,$$

and since the sum over i of the constant θ_0 is $n\theta_0$,

$$n\,\theta_0 = \sum_{i=0}^{n-1} y_i - \sum_{i=0}^{n-1} \sum_{j=1}^{p-1} X_{ij} \theta_j .$$

Dividing by n and writing $\bar{y} = \frac{1}{n}\sum_i y_i$ and $\bar{x}_j = \frac{1}{n}\sum_i X_{ij}$ for the column means, the simplest case of one slope reads $\theta_0 = \bar{y} - \theta_1 \bar{x}_1$ – the fitted line passes through the point of means $(\bar{x}_1, \bar{y})$ – and in general the intercept is recovered from

$$\hat{\theta}_0 = \bar{y} - \sum_{j=1}^{p-1} \bar{x}_j \hat{\theta}_j , \tag{3.90}$$

with $\bar{y}$ and $\bar{x}_j$ the training means. The same computation explains why centring removes the intercept altogether: substituting the optimal θ_0 back into the cost turns it into the cost of a centred problem,

$$C(\boldsymbol{\theta}) = \left(\tilde{\boldsymbol{y}} - \tilde{\boldsymbol{X}} \boldsymbol{\theta} \right)^T \left(\tilde{\boldsymbol{y}} - \tilde{\boldsymbol{X}} \boldsymbol{\theta} \right), \qquad \tilde{\boldsymbol{y}} = \boldsymbol{y} - \bar{y}\mathbf{1}, \quad \tilde{X}_{ij} = X_{ij} - \bar{x}_j ,$$

up to the factor $1/n$, whose minimiser is

$$\hat{\boldsymbol{\theta}} = \left(\tilde{\boldsymbol{X}}^T \tilde{\boldsymbol{X}} \right)^{-1} \tilde{\boldsymbol{X}}^T \tilde{\boldsymbol{y}} \qquad \text{for OLS,} \qquad \hat{\boldsymbol{\theta}} = \left(\tilde{\boldsymbol{X}}^T \tilde{\boldsymbol{X}} + \lambda \boldsymbol{I} \right)^{-1} \tilde{\boldsymbol{X}}^T \tilde{\boldsymbol{y}} \qquad \text{for Ridge :}$$

the intercept has disappeared from the optimisation, and the penalty touches only the genuine slopes. Fitting with the penalised intercept instead drags the whole fit towards zero and typically *raises* the mean squared error; the centred fit, whose regularisation term runs over

$j = 1, \ldots, p-1$ only, is free to place the level of the data where the data put it. This is what `scikit-learn` does internally when the intercept is fitted, and it is why its default solutions are derived under the assumption that both $\boldsymbol{y}$ and $\boldsymbol{X}$ are zero centred.

```python
import numpy as np

def fit_with_intercept(X, y, fit_centred):
    """Fit a penalised model without penalising the intercept.

    fit_centred(Xc, yc) must return coefficients for centred data with no
    intercept column.  Returns (theta_0, theta) plus the training
    statistics needed to transform future data identically.
    """
    x_mean = np.mean(X, axis=0)
    y_mean = np.mean(y)
    x_std = np.std(X, axis=0)
    x_std[x_std == 0.0] = 1.0                 # leave constant columns alone

    Xc = (X - x_mean) / x_std
    theta = fit_centred(Xc, y - y_mean)

    # Undo the scaling so the coefficients apply to the raw features
    theta = theta / x_std
    theta_0 = y_mean - x_mean @ theta
    return theta_0, theta, (x_mean, x_std, y_mean)


def predict(X_new, theta_0, theta):
    return theta_0 + X_new @ theta
```

The same transformation must be applied to new data.

The means and standard deviations are computed on the *training* data and then applied unchanged to validation, test and future data. Recomputing them on the test set, or computing them once on the full data set before splitting, leaks information and produces an optimistic error estimate, as warned in Sections 2.9 and 2.12. Inside a cross-validation loop this means the scaler must be refitted on every training fold, which is what a `Pipeline` does automatically.

Machine learning connection. A checklist for reconciling one's own code with a library, in decreasing order of how often each is the culprit: whether the intercept is included in the design matrix or handled separately; whether the intercept is penalised; whether the data have been centred and scaled, and with which convention for the standard deviation (n or $n-1$); whether the cost function carries a factor $1/n$, $1/(2n)$ or nothing, which rescales λ correspondingly; and whether the library's regularisation parameter is our λ or some multiple of it. Every one of these changes the numbers while leaving the method unchanged, and discovering which is responsible is a routine part of the work.

Normalising and scaling data in practice.

For data sets gathered from real applications it is the rule rather than the exception that different features carry very different units and numerical scales. A data set on health habits may hold an *age* in the range 0–80 next to a *caloric intake* of order 2000; many methods are sensitive to such differences and perform poorly when they are left in place. The workhorse

transformation is the *standardisation* used throughout this section: for each feature j, subtract the mean and divide by the standard deviation over the (training) data,

$$x_j^{(i)} \;\to\; \frac{x_j^{(i)} - \bar{x}_j}{\sigma(x_j)},$$

so that every column has zero mean and unit standard deviation (`StandardScaler` in `scikit-learn`; when the standard deviation is unavailable or one prefers not to estimate it, it is common to simply set it to one, which reduces the transformation to centring). Standardisation does not, however, confine the values to any particular interval. When a fixed range matters – inputs to functions defined on $[0,1]$, pixel intensities, activations with saturating nonlinearities – *min–max scaling* (`MinMaxScaler`) maps each feature exactly onto $[0,1]$ by subtracting the minimum and dividing by the range. Two further scalers are worth knowing. The `Normalizer` acts on rows rather than columns: it rescales each *data point* to unit Euclidean length, projecting it onto the unit sphere, so that every sample is divided by its own norm; this is the natural choice when only the direction of the feature vector matters and not its magnitude. And the `RobustScaler` plays the role of the `StandardScaler` with the mean and standard deviation replaced by the median and the quartiles, which makes it insensitive to points far from the bulk of the data – outliers, often simple measurement errors, that can otherwise dominate the estimated mean and variance and thereby corrupt the scaling of every remaining point. Whichever scaler is chosen, the rule derived earlier in this section stands unchanged: its statistics are computed on the training data and applied, frozen, to everything that comes later.

3.14 Choosing the penalty: cross-validation for Ridge and Lasso

Everything said about Ridge regression and the Lasso so far has taken the penalty λ as given. Theorem 3.6 guarantees that a good value exists and Eq. (3.55) tells us what it depends on – the true parameters and the noise variance, which we do not know – so in practice λ must be estimated from the data like everything else. Section 2.12 gave the recipe in outline: define a grid, cross-validate, take the minimum, refit. The present section carries it out properly, because the details are where the mistakes live, and closes by asking why cross-validation rather than the bootstrap of Section 2.11 is the standard tool for the job.

Parameters and hyperparameters.

A penalised regression has two kinds of unknowns. The *model parameters* – the intercept θ_0 and the slopes $\boldsymbol{\theta}$ – are fixed, for a given λ, by minimising the cost, Eq. (3.43) or (3.57). The penalty λ is a *hyperparameter*: it cannot be fixed by the same minimisation, and the reason is worth stating exactly. Write $\mathrm{Err}_{\mathrm{Train}}(\lambda)$ for the training error $\frac{1}{n}\|\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{\theta}}(\lambda)\|_2^2$ of the fitted model. The unpenalised estimator $\hat{\boldsymbol{\theta}}(0)$ minimises that quantity over *all* $\boldsymbol{\theta}$, so for any $\lambda > 0$,

$$\mathrm{Err}_{\mathrm{Train}}(\lambda) \;\geq\; \mathrm{Err}_{\mathrm{Train}}(0), \tag{3.91}$$

and a stronger statement holds: the training error is non-decreasing in λ along the whole path (this is proved as an exercise). Minimising the training error over λ therefore always returns $\lambda = 0$, which is ordinary least squares, and the whole discussion of Section 3.8 would be pointless. The hyperparameter must be chosen by an estimate of the *test* error, and cross-validation is that estimate. There are accordingly three errors in play, and it is essential to

keep them apart: the training error, which is minimised to obtain the parameters at fixed λ; the cross-validation error, which is minimised over λ to obtain the hyperparameter; and the test error, which is computed once, at the very end, and minimised over nothing.

The procedure, precisely.

Let the training data have n observations and let $\kappa : \{0,\dots,n-1\} \to \{1,\dots,K\}$ assign each observation to one of K folds of equal size, after a random shuffle. For every fold k and every λ on the grid, let $\hat{\theta}_0^{(-k)}(\lambda)$ and $\hat{\boldsymbol{\theta}}^{(-k)}(\lambda)$ be the intercept and slopes fitted to the observations *not* in fold k, with the standardisation of Section 3.13 also computed from those observations alone. The cross-validation error is the average squared error of every observation predicted by the model that did not see it,

$$\mathrm{CV}_K(\lambda) = \frac{1}{n}\sum_{i=0}^{n-1}\left(y_i - \hat{\theta}_0^{(-\kappa(i))}(\lambda) - \boldsymbol{x}_i^T\hat{\boldsymbol{\theta}}^{(-\kappa(i))}(\lambda)\right)^2 = \frac{1}{K}\sum_{k=1}^{K}\mathrm{MSE}_k(\lambda), \tag{3.92}$$

where $\mathrm{MSE}_k(\lambda)$ is the mean squared error on fold k and the second equality holds for folds of equal size. The spread of the K fold errors gives the standard error of the estimate,

$$\mathrm{SE}_K(\lambda) = \frac{1}{\sqrt{K}}\sqrt{\frac{1}{K-1}\sum_{k=1}^{K}\left(\mathrm{MSE}_k(\lambda) - \mathrm{CV}_K(\lambda)\right)^2}, \tag{3.93}$$

which is Eq. (2.24) applied to the fold errors. It should be read as a rough guide rather than as a confidence interval: the K training sets overlap, each pair sharing a fraction $(K-2)/(K-1)$ of its points, so the fold errors are positively correlated and Eq. (3.93) understates the true uncertainty. The selected penalty is

$$\lambda_{\min} = \arg\min_{\lambda}\mathrm{CV}_K(\lambda), \tag{3.94}$$

and the final model is a *new* fit at $\lambda_{\min}$ to all n training observations, standardised with the statistics of all n. The K sets of fold coefficients were temporary; they served to evaluate candidate penalties and are discarded. Only after this refit is the test set touched, once, to report the generalisation error of the completed procedure. The whole recipe is:

1. Set aside a test set and do not look at it again until step 6.
2. Choose a logarithmic grid of penalties.
3. Shuffle the training data and split it into K folds, K between five and ten.
4. For every fold and every λ: standardise with the training folds' statistics, fit on the training folds, predict the held-out fold, record the squared error. Average to obtain Eqs. (3.92) and (3.93).
5. Select $\lambda_{\min}$, or the one-standard-error choice described below, and refit on all training data at that penalty.
6. Evaluate the refitted model on the test set. Report the selected λ, the final coefficients, the cross-validation error with its standard error, and the test error.

Which λ is being compared: the factor n.

There is a detail in step 5 that is easy to miss. The models of step 4 are fitted to $n(K-1)/K$ observations, the final model to n, and the question is whether the same numerical λ means the same thing for both. It depends on the convention. With the cost written as in Eq. (3.43),

squared error *divided by* n plus $\lambda\|\boldsymbol{\theta}\|_2^2$, the normal equations become

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}} = \left(\boldsymbol{X}^T\boldsymbol{X} + n\lambda\boldsymbol{I}\right)^{-1}\boldsymbol{X}^T\boldsymbol{y}, \tag{3.95}$$

which differs from Eq. (3.44) by the factor n on the penalty. Every entry of $\boldsymbol{X}^T\boldsymbol{X}$ is a sum of n products and grows in proportion to n; in Eq. (3.95) the penalty $n\lambda$ grows with it, so the balance between fit and penalty is independent of the sample size and λ is a penalty *per observation*. A value found on folds of 80 points is then the right value for the refit on 100. In the convention of Eq. (3.44), where nothing divides the squared error, the penalty is a fixed number set against a matrix that grows with n: the same λ is weaker, by the factor $(K-1)/K$, on the full training set than it was inside the folds, and a faithful transfer would multiply it by $K/(K-1)$. The effect is a single step on a fine grid and is usually invisible, but it is the reason we keep the $1/n$ of Eqs. (3.43) and (3.57) throughout this section, and it is one more entry for the checklist of Section 3.13: `scikit-learn`'s `Ridge` uses the unnormalised convention, so its `alpha` is our $n\lambda$, while its `Lasso` divides by $2n$ and its `alpha` is our $\lambda/2$. The Lasso penalty of Eq. (3.57) is likewise per observation, and the threshold $\lambda_{\max}$ of Proposition 3.8, being $2/n$ times a sum of n products, is independent of n in the same sense.

The grid.

For the Lasso the grid has a natural top: above $\lambda_{\max}$ of Eq. (3.67) every coefficient is zero and there is nothing to compute. The standard choice is a geometric sequence of fifty to a hundred values from $\lambda_{\max}$ down to $10^{-3}\lambda_{\max}$ or $10^{-4}\lambda_{\max}$, traversed from the top so that each coordinate descent can start from the previous solution – the *warm start* that makes the whole path cheaper than a handful of cold fits. For Ridge there is no finite $\lambda_{\max}$, but the shrinkage factors of Eq. (3.48), which in the convention of Eq. (3.95) read $\sigma_i^2/(\sigma_i^2 + n\lambda)$, tell us where the action is: between $n\lambda \approx \sigma_{\min}^2$, below which the fit is indistinguishable from OLS, and $n\lambda \approx \sigma_{\max}^2$, above which every direction is suppressed and the model predicts the mean. For standardised columns $\boldsymbol{X}^T\boldsymbol{X}/n$ is the correlation matrix of the features, whose eigenvalues are of order one, so σ_i^2/n is of order one and the natural unit of λ is 1: a penalty $\lambda = 1$ shrinks a typical direction by about half. A grid from 10^{-4} to 10 covers every regime for moderately correlated features, and this is what the code below uses. Either way the grid should be logarithmic, since the cross-validation curve varies on a logarithmic scale in λ, and it should be inspected: a minimum at the end of the grid means the grid was wrong, not that the end is the answer.

Standardisation inside the loop.

Section 3.13 established that the penalised methods are not equivariant under rescaling of the features, so the columns must be standardised, and that the intercept must be left unpenalised, which is done by centring and then recovering θ_0 from Eq. (3.90). Cross-validation adds the requirement that the means and standard deviations be computed *inside* each fold, from the $K-1$ training folds alone, and applied frozen to the held-out fold. Standardising the full training set once, before the loop, lets every training fold see the mean and variance of its own held-out fold. For scaling alone the leak is small. It becomes large for any preprocessing that looks at the targets: screening the features by their correlation with $\boldsymbol{y}$ on the full data set and then cross-validating a model on the survivors is the classic example, and it can produce a cross-validation error far below the noise floor for features that are pure noise. The rule is that the held-out fold must be treated exactly as future data will be treated, which means that

nothing computed from it may enter the fit. In `scikit-learn` the mechanism that enforces this is the `Pipeline`; in the code below it is done by hand so that there is nothing hidden.

The one-standard-error rule.

Near its minimum the cross-validation curve is flat, and the location of the minimum is determined only to within the noise of the estimate. Among penalties whose cross-validation errors cannot be told apart, the sensible choice is the most strongly regularised, and the *one-standard-error rule* makes that precise:

$$\lambda_{1\mathrm{SE}} = \max\left\{\lambda : \mathrm{CV}_K(\lambda) \le \mathrm{CV}_K(\lambda_{\min}) + \mathrm{SE}_K(\lambda_{\min})\right\}, \tag{3.96}$$

the largest penalty whose error lies within one standard error of the best. For Ridge this buys a smaller variance at the cost of a little bias that the data cannot detect; for the Lasso it buys a sparser model, which is often the point of using the Lasso at all. The rule is a convention, not a theorem, and it errs deliberately on the side of simplicity: the improvement offered by the more complex model is not larger than the uncertainty with which the improvement is measured, so the improvement is not believed.

An example with a sparse truth.

The Franke study of Section 3.15 will apply all this to a polynomial fit; here we use a problem designed to show the two estimators behaving differently. There are $p = 30$ features, Gaussian with the correlation $\rho^{|i-j|}$, $\rho = 0.7$, between features i and j, so that neighbouring columns are strongly collinear; only the first eight coefficients are non-zero; the noise has $\sigma = 1$, so the floor of Eq. (2.52) is at 1; and there are $n = 100$ training observations and 50 test observations. The code implements every step of the recipe from scratch, with Ridge in the closed form (3.95) and the Lasso from `scikit-learn` with the factor of two accounted for.

```
import numpy as np
from sklearn.linear_model import Lasso

def make_data(n, p, rho, sigma, rng):
    """Correlated Gaussian features, cov[i, j] = rho^|i-j|, and a sparse truth:
    only the first eight of the p coefficients are non-zero."""
    cov = rho ** np.abs(np.subtract.outer(np.arange(p), np.arange(p)))
    X = rng.multivariate_normal(np.zeros(p), cov, size=n)
    theta = np.zeros(p)
    theta[:8] = [3.0, -2.0, 1.5, -1.0, 1.0, -0.8, 0.6, -0.5]
    y = 2.0 + X @ theta + sigma * rng.normal(size=n)
    return X, y, theta

def ridge_fit(X, y, lmbda):
    """Minimiser of ||y - X theta||^2 / n + lmbda ||theta||_2^2, Eq. (3.ridgeproblem),
    for centred X and y: theta = (X^T X + n lmbda I)^{-1} X^T y, Eq. (3.ridgepern)."""
    n, p = X.shape
    return np.linalg.solve(X.T @ X + n * lmbda * np.eye(p), X.T @ y)

def lasso_fit(X, y, lmbda):
    """Minimiser of ||y - X theta||^2 / n + lmbda ||theta||_1, Eq. (3.lassoproblem).
    scikit-learn minimises ||y - X theta||^2 / (2n) + alpha ||theta||_1: alpha = lmbda/2."""
    return Lasso(alpha=lmbda / 2.0, fit_intercept=False,
                 max_iter=100000, tol=1e-10).fit(X, y).coef_

def cv_curve(fit, X, y, lambdas, K=5, rng=None):
    """K-fold cross-validation error for every lambda: mean and standard error."""
```

```
    rng = np.random.default_rng() if rng is None else rng
    n = X.shape[0]
    folds = np.array_split(rng.permutation(n), K)          # shuffle, then split
    errors = np.empty((K, len(lambdas)))
    for k, held_out in enumerate(folds):
        train = np.setdiff1d(np.arange(n), held_out)
        # standardise with the statistics of the K-1 training folds only
        mu, sd, y_mean = X[train].mean(0), X[train].std(0), y[train].mean()
        X_tr, X_va = (X[train] - mu) / sd, (X[held_out] - mu) / sd
        for j, lmbda in enumerate(lambdas):
            theta = fit(X_tr, y[train] - y_mean, lmbda)
            errors[k, j] = np.mean((y[held_out] - y_mean - X_va @ theta) ** 2)
    return errors.mean(axis=0), errors.std(axis=0, ddof=1) / np.sqrt(K)

def select(lambdas, cv_mean, cv_se):
    """The minimum-CV lambda, and the largest lambda within one standard error of it."""
    i_min = np.argmin(cv_mean)
    within = cv_mean <= cv_mean[i_min] + cv_se[i_min]
    return lambdas[i_min], lambdas[within].max()

def refit(fit, X, y, lmbda):
    """The final model: one fit on all training data, mapped back to the raw scale."""
    mu, sd, y_mean = X.mean(0), X.std(0), y.mean()
    theta = fit((X - mu) / sd, y - y_mean, lmbda)
    theta_0 = y_mean - (mu / sd) @ theta               # intercept, Eq. (3.intercept)
    return theta_0, theta / sd

rng = np.random.default_rng(3155)
X, y, theta_true = make_data(n=150, p=30, rho=0.7, sigma=1.0, rng=rng)
X_train, y_train, X_test, y_test = X[:100], y[:100], X[100:], y[100:]
lambdas = np.logspace(-4, 1, 60)

for name, fit in (("Ridge", ridge_fit), ("Lasso", lasso_fit)):
    cv_mean, cv_se = cv_curve(fit, X_train, y_train, lambdas, K=5,
                              rng=np.random.default_rng(2024))
    for rule, lmbda in zip(("min-CV", "one-SE"), select(lambdas, cv_mean, cv_se)):
        theta_0, theta = refit(fit, X_train, y_train, lmbda)
        test_mse = np.mean((y_test - theta_0 - X_test @ theta) ** 2)   # used once
        j = np.searchsorted(lambdas, lmbda)
        print(f"{name} {rule}: lambda {lmbda:.4f}  CV {cv_mean[j]:.3f} +- {cv_se[j]:.3f}"
              f"  test {test_mse:.3f}  non-zero {np.sum(theta != 0):2d}"
              f"  true ones kept {np.sum(theta[:8] != 0)}")
```

```
Ridge min-CV: lambda 0.0131  CV 1.205 +- 0.141  test 1.374  non-zero 30  true ones kept 8
Ridge one-SE: lambda 0.0424  CV 1.324 +- 0.192  test 1.139  non-zero 30  true ones kept 8
Lasso min-CV: lambda 0.0626  CV 1.098 +- 0.126  test 1.059  non-zero 21  true ones kept 8
Lasso one-SE: lambda 0.1366  CV 1.216 +- 0.130  test 1.140  non-zero 13  true ones kept 8
```

Figure 3.4 shows the two cross-validation curves behind these four lines, together with two curves that are never available in practice: the training error of the refitted model, and its *true* expected test error, computed on twenty thousand fresh observations from the same process. Several things are visible at once.

The training error, dotted, increases monotonically with λ, as Eq. (3.91) says it must, and it lies far below the test error everywhere: at the OLS end it is 0.49 against a true error of 1.46. This is the optimism of Section 3.7 at a ratio $p/n = 0.3$ that makes it severe, and it is why the training error can select nothing. Both cross-validation curves have the shape of the test-error curve – flat for small λ, then a shallow minimum, then a steep rise as the model is shrunk towards predicting the mean – and they track it within their own standard error. For Ridge, cross-validation selects $\lambda_{\min} = 0.013$ where the true curve has its minimum at 0.024; the true error at the selected value is 1.316 against a best possible 1.291, so the choice costs

nothing measurable. For the Lasso the selected value coincides with the minimum of the true curve.

The Lasso does better than Ridge on this problem, 1.15 against 1.29 at the respective optima, and the reason is the one Section 3.11 gave: the truth is sparse, twenty-two of thirty coefficients are exactly zero, and the Laplace prior is the right prior. In the Franke study, where the truth is a smooth surface with no exactly-zero monomials, the two estimators will finish level. The Lasso's sparsity is, however, less clean than one might hope. At $\lambda_{\min}$ it keeps 21 of the 30 coefficients: all eight true ones and thirteen spurious ones. This is typical and should be expected. Cross-validation minimises the *prediction* error, and a spurious coefficient with a small non-zero value costs almost nothing in prediction while removing it costs a little; the prediction-optimal penalty therefore includes a tail of small, false coefficients. The one-standard-error choice, $\lambda_{1\mathrm{SE}} = 0.137$, removes eight of them, keeping 13 coefficients with all eight true ones still present, at a true error of 1.28 instead of 1.15. That is the trade the rule offers: a measurably simpler model for a barely measurable loss of accuracy. For Ridge, whose thirty coefficients are all non-zero at every λ, the same rule moves the penalty from 0.013 to 0.042 and the true error from 1.316 to 1.338.

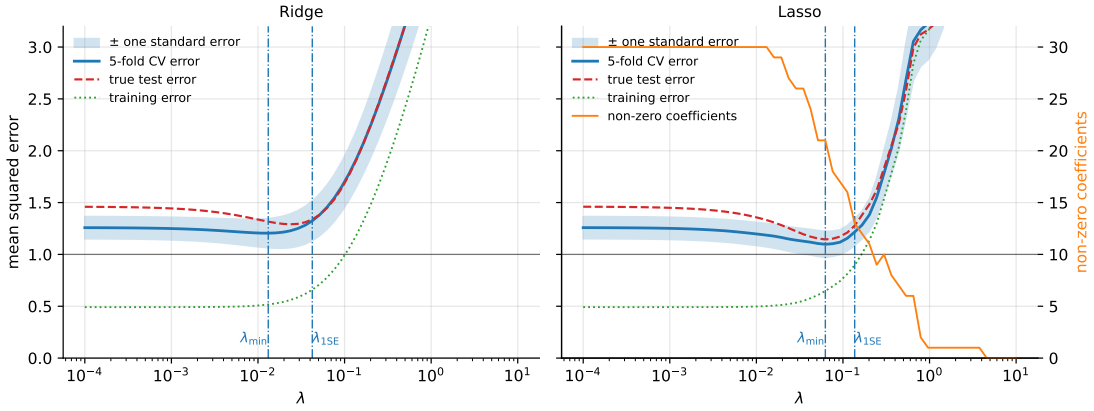


Fig. 3.4 Five-fold cross-validation of Ridge (left) and the Lasso (right) on the sparse problem of the text: $n = 100$, $p = 30$, correlation $0.7^{|i-j|}$, eight non-zero coefficients, $\sigma = 1$. The cross-validation error (3.92) is shown with its one-standard-error band (3.93); the dashed curve is the true expected test error of the model refitted at each λ, measured on 20000 fresh observations, and the dotted curve the training error. The horizontal line is the noise floor $\sigma^2 = 1$. The vertical lines mark $\lambda_{\min}$ of Eq. (3.94) and $\lambda_{1\mathrm{SE}}$ of Eq. (3.96); for the Lasso the number of non-zero coefficients is drawn against the right-hand axis.

The four test errors in the output deserve a comment of their own, because they illustrate a point that is easy to forget. They range from 1.06 to 1.37 and would seem to say that the one-standard-error Ridge beats the minimum-CV Ridge by a wide margin. They say no such thing. A mean squared error computed from 50 residuals is itself an estimate, and for Gaussian residuals its standard error is about $\mathrm{MSE}\sqrt{2/50} \approx 0.2$ (the squared residuals have variance $2\mathrm{MSE}^2$, and Eq. (2.24) does the rest). The honest summary of the last column is that all four models generalise with an error of about 1.2 ± 0.2, in agreement with their cross-validation estimates and above the floor of 1, and that a test set of fifty points cannot resolve differences of a tenth. The true errors quoted above, which can resolve them, were available only because we wrote the generating process ourselves.

Two further remarks on the cross-validation curves. First, the curve in the left panel lies *below* the true error for small λ, by about 0.2. Cross-validation is if anything pessimistic – its models are fitted to 80 points rather than 100 – so this is not a systematic effect but the fluctuation of a single training set, of the size the standard error suggests; the repeated

experiment later in this section will put a number on it. Second, the flatness of the curves near the minimum has a consequence for the selected penalty. Repeating the five-fold procedure with twenty different shuffles of the same training data,

```
for name, fit in (("Ridge", ridge_fit), ("Lasso", lasso_fit)):
    selected = [select(lambdas, *cv_curve(fit, X_train, y_train, lambdas, K=5,
                                          rng=np.random.default_rng(seed)))[0]
                for seed in range(20)]
    print(f"{name}: lambda_min over 20 shuffles between"
          f" {min(selected):.4f} and {max(selected):.4f}")
```

```
Ridge: lambda_min over 20 shuffles between 0.0028 and 0.0131
Lasso: lambda_min over 20 shuffles between 0.0050 and 0.0626
```

moves $\lambda_{\min}$ by a factor of five for Ridge and by more than an order of magnitude for the Lasso. Read against the dashed curves of Figure 3.4, the Ridge choices all lie on the flat part and cost between nothing and a tenth in true error; the Lasso choices span true errors from 1.15 to 1.41 and retain anywhere between 21 and all 30 coefficients. The penalty selected from a single five-fold split is, in other words, a noisy quantity, and the Lasso's support inherits the noise. *Repeated* cross-validation, which averages the curves over several shuffles before taking the minimum, reduces the wandering at a proportional cost; the one-standard-error rule is the cheaper remedy, since it moves the choice off the flat part and onto the rising flank, where the curve determines its own position; and the bootstrap experiment at the end of the section is the way to see how much of the selected support to trust.

The same procedure with scikit-learn.

Once the mechanics are understood, the library does the bookkeeping. A `Pipeline` refits the scaler inside every fold, `GridSearchCV` runs the loop over folds and penalties, and with `refit=True` it ends by fitting the whole pipeline on all training data at the selected value. The folds are passed explicitly so that they coincide with those of the code above, and `alpha` is set to $n_{\text{fold}}\lambda = 80\lambda$ so that, inside the folds, `Ridge` solves exactly Eq. (3.95).

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

folds = np.array_split(np.random.default_rng(2024).permutation(100), 5)
cv = [(np.setdiff1d(np.arange(100), f), f) for f in folds]  # the same folds as above

model = Pipeline([("scale", StandardScaler()), ("ridge", Ridge())])
# scikit-learn's Ridge minimises ||y - X theta||^2 + alpha ||theta||^2, so
# alpha = n lambda, with n = 80 the size of the training folds
search = GridSearchCV(model, {"ridge__alpha": 80 * lambdas},
                      scoring="neg_mean_squared_error", cv=cv, refit=True)
search.fit(X_train, y_train)

lambda_star = search.best_params_["ridge__alpha"] / 80
cv_curve_sklearn = -search.cv_results_["mean_test_score"]
final_model = search.best_estimator_    # a new fit on all of X_train, not a fold fit
print(f"lambda* {lambda_star:.4f}, CV {-search.best_score_:.3f}, "
      f"test {np.mean((y_test - final_model.predict(X_test)) ** 2):.3f}")
cv_mean, _ = cv_curve(ridge_fit, X_train, y_train, lambdas, K=5,
                      rng=np.random.default_rng(2024))
print("largest difference from the hand-written curve:",
      np.max(np.abs(cv_curve_sklearn - cv_mean)))
```

```
lambda* 0.0131, CV 1.205, test 1.422
largest difference from the hand-written curve: 1.7763568394002505e-15
```

The two cross-validation curves agree to rounding error, which is the check one should always make before trusting a library with a convention. The test error does not agree – 1.422 against 1.374 from the hand-written refit – and the reason is the factor n once more, now acting on the refit: `GridSearchCV` refits with the same `alpha`$= 80\lambda^*$ on all 100 observations, which in the per-observation convention is a penalty of $0.8\lambda^*$, and the final model is therefore slightly less regularised than the one selected. Setting `alpha` to 100λ instead makes the refit exact but applies the full-sample conversion inside 80-point folds, which moves the selected value one grid point, to 0.0108, with the cross-validation error unchanged in the third decimal. Neither discrepancy matters here, both being a fraction of a grid step on a flat curve, but both are the factor-n effect described above, and knowing where it enters is what allows one to say so. Letting `KFold(5, shuffle=True, random_state=2024)` choose its own folds instead of ours moves it much further, to $\lambda^* = 0.0034$ with a cross-validation error of 1.00: the wandering of the minimum along a flat valley again, and a reminder that the second decimal of $\lambda_{\min}$ is not a property of the data.

Leave-one-out and generalised cross-validation, revisited.

For Ridge regression Section 3.7 derived the leave-one-out error in closed form, Eq. (3.40), and its rotation-invariant relative GCV, Eq. (3.41), both at the price of a single fit per λ. When they apply they are excellent, and for small n they are the natural choice. Two things limit them. Their cost is that of forming the $n \times n$ smoother matrix, which for large n exceeds that of K fits; and, more fundamentally, they require the fitted values to be a *linear* function of $\boldsymbol{y}$. The Lasso fails this test: which coefficients are non-zero depends on $\boldsymbol{y}$, so $\tilde{\boldsymbol{y}}$ is piecewise linear in $\boldsymbol{y}$ and no smoother matrix exists. (There is a remarkable partial substitute – the number of non-zero coefficients is an unbiased estimate of the Lasso's effective degrees of freedom, so a GCV-like criterion can be written down – but it is not standard practice.) For the Lasso, and for every method in the later chapters of this book, K-fold cross-validation is the tool, and it is as well to learn it on a problem where its answer can be checked.

The bootstrap as an alternative.

Section 2.10 used the bootstrap of Section 2.11 to manufacture the ensemble of training sets that the bias-variance decomposition requires, and it is natural to ask whether the same device could estimate the test error and so replace cross-validation in the selection of λ. It can be made to, and the care this takes is instructive, so we go through it.

Start with a fact about resampling with replacement. Each of the n draws picks observation i with probability $1/n$, so the probability that i appears in none of them is

$$\text{Prob}(i \notin \boldsymbol{x}^*) = \left(1 - \frac{1}{n}\right)^n \xrightarrow{n\to\infty} e^{-1} \approx 0.368, \tag{3.97}$$

already 0.366 at $n = 100$. A bootstrap sample therefore contains, on average, only 63.2% of the distinct observations, the remainder appearing as duplicates; the 36.8% left out are the *out-of-bag* observations of the replica. This number governs everything that follows.

Three estimates of the prediction error can be built from bootstrap replicas. The *apparent* error is the training error $\overline{\text{err}}$ of the model fitted to the full sample; it is optimistic for the reasons of Section 3.7 and needs no bootstrap at all. The naive bootstrap estimate – fit to each

replica, evaluate on the original n observations, average – is nearly as optimistic, because 63% of the observations it is evaluated on were in the sample it was fitted to. The estimate that respects the training-test separation is the *leave-one-out bootstrap*,

$$\mathrm{Err}^{(1)} = \frac{1}{n}\sum_{i=0}^{n-1} \frac{1}{|C_{-i}|} \sum_{b \in C_{-i}} \left(y_i - \hat{f}^{*b}(\boldsymbol{x}_i)\right)^2, \tag{3.98}$$

where C_{-i} is the set of replicas in which observation i does not appear and $\hat{f}^{*b}$ is the model fitted to replica b: every observation is predicted only by models that never saw it, exactly as in cross-validation. But each of those models was fitted to about $0.632n$ distinct points, so Eq. (3.98) estimates the error of a model trained on a substantially smaller sample and is *pessimistic*. In terms of Eq. (3.92) it behaves like K-fold cross-validation with $(K-1)/K = 0.632$, that is $K \approx 2.7$, whereas five folds train on 80% of the data and ten on 90%. Efron's remedy is to average the two biases away against each other,

$$\mathrm{Err}^{(.632)} = 0.368\,\overline{\mathrm{err}} + 0.632\,\mathrm{Err}^{(1)}, \tag{3.99}$$

the *.632 estimator*, whose weights are those of Eq. (3.97). It is a heuristic rather than a theorem, and it has a known failure: for a model that interpolates its training data, $\overline{\mathrm{err}} = 0$ and Eq. (3.99) returns 0.632 of a pessimistic number, which can be badly optimistic. The *.632+* estimator repairs this by making the weight depend on an estimate of the degree of overfitting, at the price of further heuristics.

The following code computes the three bootstrap quantities for the problem of Figure 3.4, at the OLS end of the grid and at the two selected penalties, and sets them beside the five-fold estimate and the true error.

```
def bootstrap_errors(fit, X, y, lmbda, B=200, rng=None):
    """Training (apparent) error, leave-one-out bootstrap error, .632 estimate."""
    rng = np.random.default_rng() if rng is None else rng
    n = X.shape[0]
    theta_0, theta = refit(fit, X, y, lmbda)
    apparent = np.mean((y - theta_0 - X @ theta) ** 2)
    err_sum, err_count = np.zeros(n), np.zeros(n)
    for _ in range(B):
        idx = rng.integers(0, n, n)                          # draw n with replacement
        out = np.setdiff1d(np.arange(n), idx)                # the points left out
        theta_0, theta = refit(fit, X[idx], y[idx], lmbda)
        err_sum[out] += (y[out] - theta_0 - X[out] @ theta) ** 2
        err_count[out] += 1
    seen = err_count > 0
    loo_boot = np.mean(err_sum[seen] / err_count[seen])    # Eq. (3.loobootstrap)
    return apparent, loo_boot, 0.368 * apparent + 0.632 * loo_boot

rng = np.random.default_rng(3155)
X, y, theta_true = make_data(n=150, p=30, rho=0.7, sigma=1.0, rng=rng)
X_train, y_train = X[:100], y[:100]
# The generating process is known, so the true error of any fit can be measured
# on a large fresh sample -- a luxury never available in practice.
X_big, y_big, _ = make_data(n=20000, p=30, rho=0.7, sigma=1.0,
                            rng=np.random.default_rng(1))
lambdas = np.logspace(-4, 1, 60)

for name, fit in (("Ridge", ridge_fit), ("Lasso", lasso_fit)):
    cv_mean, cv_se = cv_curve(fit, X_train, y_train, lambdas, K=5,
                              rng=np.random.default_rng(2024))
    lambda_min, lambda_1se = select(lambdas, cv_mean, cv_se)
    for lmbda in (1e-4, lambda_min, lambda_1se):
        j = np.searchsorted(lambdas, lmbda)
```

```
        apparent, loo_boot, e632 = bootstrap_errors(fit, X_train, y_train, lmbda,
                                                    B=200, rng=np.random.default_rng(2024))
        theta_0, theta = refit(fit, X_train, y_train, lmbda)
        true = np.mean((y_big - theta_0 - X_big @ theta) ** 2)
        print(f"{name} lambda {lmbda:.4f}: train {apparent:.3f}  CV {cv_mean[j]:.3f}"
              f"  LOO-boot {loo_boot:.3f}  .632 {e632:.3f}  true {true:.3f}")

# When p approaches n the bootstrap training sets, with only 0.632 n distinct
# points, reach the singular regime of the Franke study long before the K-fold ones
rng = np.random.default_rng(5)
X, y, _ = make_data(n=100, p=60, rho=0.7, sigma=1.0, rng=rng)
X_big, y_big, _ = make_data(n=20000, p=60, rho=0.7, sigma=1.0,
                            rng=np.random.default_rng(6))
apparent, loo_boot, e632 = bootstrap_errors(ridge_fit, X, y, 1e-4, B=200, rng=rng)
cv_mean, _ = cv_curve(ridge_fit, X, y, np.array([1e-4]), K=5, rng=rng)
theta_0, theta = refit(ridge_fit, X, y, 1e-4)
print(f"p = 60, n = 100, lambda 0.0001: train {apparent:.3f}  CV {cv_mean[0]:.3f}"
      f"  LOO-boot {loo_boot:.3f}  .632 {e632:.3f}"
      f"  true {np.mean((y_big - theta_0 - X_big @ theta) ** 2):.3f}")
```

```
Ridge lambda 0.0001: train 0.491  CV 1.258  LOO-boot 1.476  .632 1.114  true 1.459
Ridge lambda 0.0131: train 0.517  CV 1.205  LOO-boot 1.409  .632 1.080  true 1.316
Ridge lambda 0.0424: train 0.659  CV 1.324  LOO-boot 1.647  .632 1.284  true 1.338
Lasso lambda 0.0001: train 0.491  CV 1.258  LOO-boot 1.476  .632 1.114  true 1.460
Lasso lambda 0.0626: train 0.648  CV 1.098  LOO-boot 1.286  .632 1.051  true 1.145
Lasso lambda 0.1366: train 0.896  CV 1.216  LOO-boot 1.526  .632 1.294  true 1.277
p = 60, n = 100, lambda 0.0001: train 0.480  CV 5.088  LOO-boot 24.969  .632 15.957  true
    3.255
```

The pattern predicted above is the pattern observed. The training error is optimistic by a factor of up to three; the leave-one-out bootstrap is pessimistic at every penalty, by an amount that grows with λ and reaches 0.3; the .632 estimator lands between them, on this training set below the truth at the smaller penalties by up to 0.35; and the five-fold estimate is within one to one and a half of its own standard errors of the truth at every penalty. Figure 3.5 shows the same four estimates along the whole grid, with the true curve, and marks the minimum of each. The last line of the output is the case where the difference stops being a matter of tenths. With $p = 60$ features and $n = 100$ observations the five-fold models, fitted to 80 points, are still comfortably overdetermined, and cross-validation overestimates the true error of 3.3 by a factor of 1.5 – pessimistic, as models trained on fewer points must be near the $p \approx n$ cliff of Section 3.15. The bootstrap models are fitted to about 63 distinct points, right at the edge of that cliff, and their out-of-bag error is 25; the .632 correction brings it down to 16, still five times the truth. Duplicated rows make the matter worse than the count of distinct points suggests: a replica is a weighted least-squares problem with integer weights, so its design matrix has the column space of the 63 distinct rows but a distorted geometry, and for the Lasso the duplicates change which variables enter.

A single training set cannot separate bias from luck, so we repeat the comparison over a hundred fresh training sets at a fixed penalty and record, for each estimator, the mean and the spread of its deviation from the true error of the model it is estimating.

```
X_big, y_big, _ = make_data(n=20000, p=30, rho=0.7, sigma=1.0,
                            rng=np.random.default_rng(1))
# bias and spread of the four estimators over 100 fresh training sets, at fixed lambda
for name, fit, lmbda in (("Ridge", ridge_fit, 0.0131), ("Lasso", lasso_fit, 0.0626)):
    rows = []
    for seed in range(100):
        rng = np.random.default_rng(100 + seed)
        X, y, _ = make_data(n=100, p=30, rho=0.7, sigma=1.0, rng=rng)
        cv_mean, _ = cv_curve(fit, X, y, np.array([lmbda]), K=5, rng=rng)
        apparent, loo_boot, e632 = bootstrap_errors(fit, X, y, lmbda, B=100, rng=rng)
```

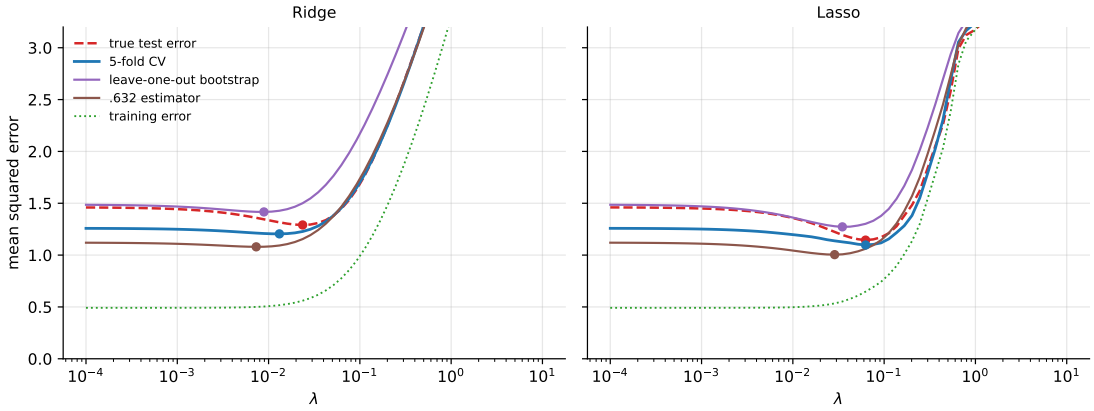


Fig. 3.5 Four estimates of the test error of Ridge (left) and the Lasso (right) along the penalty grid, for the training set of Figure 3.4: the training error, five-fold cross-validation (3.92), the leave-one-out bootstrap (3.98) and the .632 estimator (3.99), with $B = 100$ replicas per grid point. The dashed curve is the true expected test error. Dots mark the minimum of each curve: the .632 minimum sits to the left of the true one because the estimator inherits 0.368 of the training error, which decreases towards $\lambda = 0$.

```
        theta_0, theta = refit(fit, X, y, lmbda)
        true = np.mean((y_big - theta_0 - X_big @ theta) ** 2)
        rows.append((apparent, cv_mean[0], loo_boot, e632, true))
    rows = np.array(rows)
    deviation = rows[:, :4] - rows[:, 4:5]                # estimate minus true error
    print(f"{name} at lambda = {lmbda}: mean true error {rows[:, 4].mean():.3f}")
    print("  bias (training, CV, LOO-bootstrap, .632):", np.round(deviation.mean(0), 3))
    print("  spread                                  :", np.round(deviation.std(0), 3))
```

```
Ridge at lambda = 0.0131: mean true error 1.444
  bias (training, CV, LOO-bootstrap, .632): [-0.737  0.137  0.477  0.03 ]
  spread                                  : [0.165 0.346 0.323 0.259]
Lasso at lambda = 0.0626: mean true error 1.327
  bias (training, CV, LOO-bootstrap, .632): [-0.532  0.055  0.287 -0.014]
  spread                                  : [0.164 0.274 0.267 0.225]
```

The biases are as advertised: the training error is optimistic by half a unit or more, the leave-one-out bootstrap pessimistic by 0.3 to 0.5, and five-fold cross-validation pessimistic by 0.05 to 0.14, which is the price of fitting to 80 points instead of 100 and shrinks as K grows. The gap of 0.2 noticed in Figure 3.4 was, as claimed, a fluctuation, of about one spread. What the table also shows, and what an honest account must say, is that at a *fixed* penalty the .632 estimator is the best of the four: nearly unbiased, and with a spread smaller than that of cross-validation by a fifth to a quarter. This is Efron and Tibshirani's finding and it is not a coincidence – averaging over B replicas is a smoother operation than averaging over five folds. If the question were "what is the test error of this model at this λ", the bootstrap would have a strong claim.

That is not the question. The question is which λ to choose, and for that purpose cross-validation is preferred for four reasons which the experiments above have made concrete. First, selection depends on the *location* of the minimum, not on the level of the curve. A bias that is nearly constant in λ, which is what cross-validation has, leaves the minimum where it is; the .632 estimator adds 0.368 of the training error, a function that decreases monotonically towards $\lambda = 0$, and its minimum is thereby dragged towards under-regularisation. In Figure 3.5 it selects 0.007 for Ridge and 0.029 for the Lasso, against true optima of 0.024 and 0.063 and cross-validation choices of 0.013 and 0.063; for the Lasso the .632 choice keeps 26 coefficients where cross-validation keeps 21. Second, cost: five or ten fits per grid point

against one or two hundred, which for the sixty-point grid above is three hundred fits against twelve thousand, and the ratio is the same for a neural network taking an hour per fit. Third, sample size: the bootstrap models see 63% of the distinct data and fall off the $p \approx n$ cliff first, as the last line of the table shows, while K-fold models see $(K-1)/K$ of it and can be pushed as close to the full sample as one is willing to pay for. Fourth, simplicity: every observation is predicted exactly once, by a model that did not see it, from a training set of the same character as the final one, and no correction constants are needed – whereas the bootstrap estimate requires the .632 heuristic, and the .632 heuristic requires the .632+ repair as soon as the apparent error approaches zero, which for $p \geq n$ it does.

What the bootstrap is for: the stability of the selection.

None of this retires the bootstrap. It remains the tool for estimating the *distribution* of a statistic when no formula exists, which is how Chapter 2 used it, and cross-validation produces a statistic to which no formula applies: the set of coefficients the Lasso selects. The output above found 21 non-zero coefficients at $\lambda_{\min}$, eight true and thirteen spurious, and nothing in the cross-validation curve says which are which. The bootstrap can. Refit the Lasso, at the selected penalty, to B replicas of the training set and record how often each coefficient is non-zero:

```
cv_mean, cv_se = cv_curve(lasso_fit, X_train, y_train, lambdas, K=5,
                          rng=np.random.default_rng(2024))
lambda_min, _ = select(lambdas, cv_mean, cv_se)

# selection frequency of every feature over B bootstrap replicas at lambda_min
rng, B, n = np.random.default_rng(2024), 200, 100
count = np.zeros(X_train.shape[1])
for _ in range(B):
    idx = rng.integers(0, n, n)
    theta_0, theta = refit(lasso_fit, X_train[idx], y_train[idx], lambda_min)
    count += theta != 0
frequency = count / B
theta_0, theta = refit(lasso_fit, X_train, y_train, lambda_min)
print("selected on the full training set:", np.flatnonzero(theta))
print("frequency, the eight true features:", np.round(frequency[:8], 2))
print("frequency, the spurious ones selected:")
print(np.round(frequency[8:][theta[8:] != 0], 2))
print("selected in more than 80% of the replicas:", np.flatnonzero(frequency > 0.8))
```

```
selected on the full training set: [ 0  1  2  3  4  5  6  7 10 12 13 14 16 17 20 23 24 26
    27 28 29]
frequency, the eight true features: [1.   1.   1.   1.   1.   1.   0.97 1.  ]
frequency, the spurious ones selected:
[0.64 0.74 0.78 0.72 0.53 0.68 0.56 0.64 0.86 0.75 0.61 0.51 0.82]
selected in more than 80% of the replicas: [ 0  1  2  3  4  5  6  7 24 29]
```

The eight true features are selected in 97 to 100 per cent of the replicas; the thirteen spurious ones in 51 to 86 per cent, most of them in around two thirds. A coefficient that appears in two replicas out of three is one whose presence depends on which observations happened to be drawn, and that is the operational meaning of “spurious” when the truth is unknown. Keeping only the features selected in more than 80% of the replicas leaves the eight true ones and two impostors, which is a far better support than the 21 the cross-validation minimum returned – and it was obtained without the one-standard-error rule, without knowing the truth, and without any estimate of the test error at all. This procedure, refined and supplied with an error bound on the number of false selections, is *stability selection*, and it is the natural

companion to a cross-validated Lasso: cross-validation chooses the penalty for prediction, and the bootstrap tells us which parts of the resulting model to believe. The same replicas can be used to attach a standard error to every non-zero coefficient, to the cross-validation curve itself, and to $\lambda_{\min}$, which is how the wandering reported earlier would be quantified in practice.

Nested cross-validation.

One question remains, and it is the one the test set answered above: what is the generalisation error of the *whole procedure*, selection included? The temptation is to report $\mathrm{CV}_K(\lambda_{\min})$, the smallest value on the curve, and it is a mistake, for the same reason that the training error is a mistake one level down. The minimum of sixty noisy estimates is biased downwards – among sixty candidates, the one with the luckiest noise wins – and the more penalties one tries, the larger the effect. With a test set the question does not arise: the test error of the refitted model is an unbiased estimate of exactly the right thing. Without one, the remedy is to nest the selection inside a second cross-validation loop: an outer split into K_{out} folds, and within each outer training set the full inner procedure – grid, K folds, selection, refit – followed by evaluation on the outer held-out fold. Every outer fold then tests a model whose λ was chosen without it, the outer average estimates the error of the procedure rather than of a particular λ, and the cost is K_{out} times that of the inner loop. For the linear models of this chapter the cost is negligible and the selection bias is small – the cross-validation minimum of 1.10 for the Lasso above sits below the true 1.15 by an amount well inside the noise – but for flexible models with several hyperparameters tuned on a fine grid it is neither, and nested cross-validation is the honest estimate.

Machine learning connection. The procedure of this section is not specific to λ. Every method in the remainder of this book has hyperparameters – the learning rate and batch size of Chapter 4, the depth of a tree, the kernel width of Chapter 6, the width and depth of a network – and none of them can be chosen by minimising the training error, for the reason of Eq. (3.91): a more flexible model always fits the training data better. They are all chosen the way λ was chosen here, by cross-validation on the training data, with the preprocessing inside the loop, with a refit at the selected values, and with the test set used exactly once afterwards. Two things change as the models grow. The cost of a fit grows, which is why one sees $K = 3$ and single validation splits in deep learning where this chapter used $K = 5$ and could have afforded leave-one-out; and the number of hyperparameters grows, so that the grid becomes a random or adaptive search over a space rather than a line, and the selection bias that nested cross-validation guards against becomes real. The discipline is the same. Cross-validation chooses the hyperparameters; refitting produces the final parameters; the test set is a test set only as long as it has been used once.

3.15 A complete example: the Franke function

We now assemble everything into a single study. The Franke function is a weighted sum of four exponentials on the unit square,

$$f(x,y) = \frac{3}{4}\exp\left(-\frac{(9x-2)^2}{4} - \frac{(9y-2)^2}{4}\right) + \frac{3}{4}\exp\left(-\frac{(9x+1)^2}{49} - \frac{9y+1}{10}\right) + \frac{1}{2}\exp\left(-\frac{(9x-7)^2}{4} - \frac{(9y-3)^2}{4}\right) - \frac{1}{5}\exp\left(-(9x-4)^2 - (9y-7)^2\right), \tag{3.100}$$

for $x, y \in [0,1]$. It is a standard test surface for interpolation and regression: smooth, but with two peaks, a saddle and a depression, so that it cannot be captured by a low-order polynomial and is well captured by a moderate one. Adding Gaussian noise gives us a problem with a known truth, a known noise level and a tunable complexity – everything needed to see the bias-variance trade-off of Section 2.10 in action.

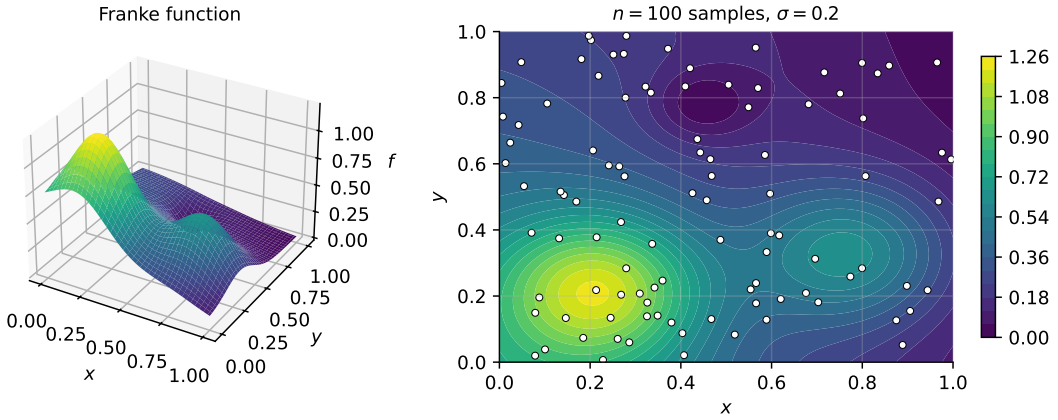


Fig. 3.6 The Franke function (3.100) on the unit square (left) and a contour view with the $n = 100$ noisy samples used in this section marked (right). Two peaks, a saddle and a depression make it too rich for a low-order polynomial and well within reach of a moderate one.

```python
import numpy as np

def franke_function(x, y):
    """The Franke function, Eq. (3.franke)."""
    t1 = 0.75 * np.exp(-(0.25 * (9 * x - 2)**2) - 0.25 * ((9 * y - 2)**2))
    t2 = 0.75 * np.exp(-((9 * x + 1)**2) / 49.0 - 0.1 * (9 * y + 1))
    t3 = 0.50 * np.exp(-(9 * x - 7)**2 / 4.0 - 0.25 * ((9 * y - 3)**2))
    t4 = -0.20 * np.exp(-(9 * x - 4)**2 - (9 * y - 7)**2)
    return t1 + t2 + t3 + t4


def make_franke_data(n=400, noise=0.1, rng=None):
    """Sample the Franke function on random points with Gaussian noise."""
    rng = np.random.default_rng() if rng is None else rng
    x, y = rng.random(n), rng.random(n)
    z = franke_function(x, y) + noise * rng.normal(size=n)
    return x, y, z
```

The study.

The analysis proceeds in the order of this book. We build the design matrix of Eq. (3.5) for polynomial degrees from one to some maximum; we split the data into training and test sets; we standardise using training statistics only; we fit by OLS, Ridge and Lasso; and we report

test errors, choosing λ by the k-fold cross-validation of Section 3.14, with the scaler refitted inside every fold and the model refitted on the full training set at the selected value.

```
import numpy as np
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split, KFold, cross_val_score

rng = np.random.default_rng(2024)
# A small, noisy sample: with n = 100 the number of parameters (3.numfeatures)
# overtakes the 80 training points around degree eleven, which is where the
# difference between the three methods becomes visible.
x, y, z = make_franke_data(n=100, noise=0.2, rng=rng)

max_degree = 14
kfold = KFold(n_splits=5, shuffle=True, random_state=2024)
lambdas = np.logspace(-5, 1, 25)

results = {"OLS": [], "Ridge": [], "Lasso": []}
for degree in range(1, max_degree + 1):
    X = design_matrix_2d(x, y, degree)[:, 1:]      # drop the intercept column
    X_train, X_test, z_train, z_test = train_test_split(X, z, test_size=0.2,
                                                        random_state=2024)

    # OLS: the scaler is fitted on the training fold only, inside the pipeline
    ols_pipe = make_pipeline(StandardScaler(), LinearRegression())
    ols_pipe.fit(X_train, z_train)
    results["OLS"].append(mse(z_test, ols_pipe.predict(X_test)))

    # Ridge and Lasso: choose lambda by cross-validation on the training set
    for name, Model in (("Ridge", Ridge), ("Lasso", Lasso)):
        cv_error = [
            -cross_val_score(make_pipeline(StandardScaler(),
                                           Model(alpha=lmb, max_iter=5000)),
                             X_train, z_train, cv=kfold,
                             scoring="neg_mean_squared_error").mean()
            for lmb in lambdas
        ]
        best = make_pipeline(StandardScaler(),
                             Model(alpha=lambdas[int(np.argmin(cv_error))],
                                   max_iter=5000))
        best.fit(X_train, z_train)
        results[name].append(mse(z_test, best.predict(X_test)))
        if name == "Lasso":
            n_kept = int(np.sum(best[-1].coef_ != 0))

for name, errors in results.items():
    print(f"{name:>6}: best degree {int(np.argmin(errors)) + 1:2d}, "
          f"test MSE {min(errors):.4f}, "
          f"MSE at degree {max_degree} {errors[-1]:.4f}")
```

What one observes.

The output of the run above is

```
   OLS: best degree  4, test MSE 0.0570, MSE at degree 14 2428714.9322
 Ridge: best degree  2, test MSE 0.0599, MSE at degree 14       0.0667
 Lasso: best degree  3, test MSE 0.0607, MSE at degree 14       0.0644
```

and three features of it are worth dwelling on, because they are the lessons of the preceding chapters made visible.

First, all three methods achieve very nearly the same best test error, about 0.057 to 0.061, against a noise variance of $\sigma^2 = 0.04$. By Eq. (2.52) that floor cannot be crossed, and the three methods are all within about fifty per cent of it. *Regularisation does not make a good model better.*

Second, the behaviour away from the optimum is entirely different. The OLS error is well behaved up to degree eight, rises to 14.5 at degree ten and then to over two million at degree fourteen – seven orders of magnitude worse than its own best value. The reason is Eq. (3.5): at degree fourteen there are 119 parameters and only 80 training points, so $\boldsymbol{X}^T\boldsymbol{X}$ is singular, the smallest singular values are pure noise, and Eq. (3.11) divides by them. The penalised errors, by contrast, rise from 0.060 to 0.067 and from 0.061 to 0.064: they barely move. Cross-validation increases λ as the degree grows, and the shrinkage factors (3.48) suppress precisely the directions in which Eq. (3.42) shows the variance to be unbounded. What regularisation buys is not a better optimum but insensitivity to choosing the complexity badly – which matters, because in a real problem the right complexity is not known.

Third, the Lasso fits are genuinely sparse. At degree ten it keeps 6 of 65 coefficients, and at degree fourteen 7 of 119; Ridge at comparable test error keeps every one of them, each small. The Lasso has performed model selection, discovering from the data that a handful of low-order monomials suffices – consistent with its optimum at degree three.

Two further experiments make the point sharper. Increasing n from 100 to 400 moves the OLS optimum from degree four to degree eleven and reduces the degradation at degree fourteen from a factor of 4×10^7 to a factor of 3, because p no longer approaches n. Reducing the noise to $\sigma = 0$ while keeping $n = 100$ moves the optimum from degree four to degree eight, since with no noise there is less to overfit. Note that even then the error eventually grows, by a factor of about 180 at degree fourteen: once p exceeds the number of training points the design matrix is rank deficient and Eq. (3.11) is dividing by singular values that are pure rounding error. That failure is one of conditioning rather than of statistics, and it is the only one of the three that no amount of clean data will cure. The best model is a property of the data set – its size and its noise level – and not of the function being approximated.

Figure 3.7 presents the whole experiment at a glance. The left panel is the test error, on a logarithmic scale so that the seven orders of magnitude are visible at all; the right panel counts the coefficients the Lasso retains. Read together they make the argument of this chapter: all three methods reach the same floor, set by the irreducible σ^2, but only the penalised ones stay near it when the complexity is chosen badly, and only the Lasso tells us how many terms were needed.

Machine learning connection. The whole analysis rests on one number being honest: the test error. It is worth listing the ways of corrupting it, all of which appear in student work and published papers alike. Standardising before the split leaks the test distribution into training. Choosing the polynomial degree by looking at the test error, then reporting that same error, turns the test set into a validation set – the reported figure is then optimistic, and the remedy is the three-way split of Section 2.9. Selecting features on the full data set before cross-validating leaks in the same way, and dramatically so when p is large. And splitting spatially or temporally correlated data at random makes the test points near-duplicates of training points, which by Section 2.7 measures interpolation rather than prediction. For data sampled on a grid, as the Franke function often is, this last is a real concern.

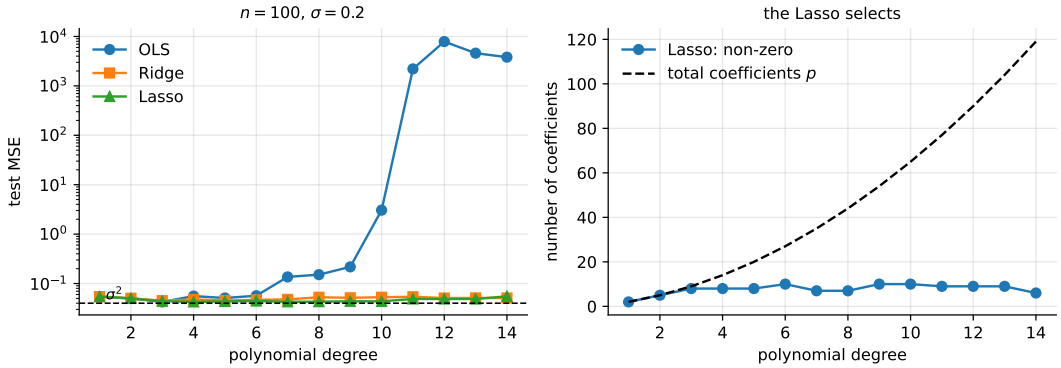


Fig. 3.7 Test error against polynomial degree for the three estimators on the Franke data (left, logarithmic), with the noise floor $\sigma^2 = 0.04$ marked; and the number of coefficients the Lasso retains against the total number available (right).

3.16 A second example: the one-dimensional Ising model

The Franke function was a regression problem dressed as a surface. This section is a regression problem dressed as a piece of physics, and it is worth the detour for three reasons. The design matrix is singular *exactly*, by an amount we can count in advance, so it exhibits everything Section 3.8 said about ill-conditioning without any appeal to rounding error. The three estimators return visibly different answers to the same question, and the differences are the ones the theory predicts rather than accidents of the solver. And the correct answer is known, so we can ask not only which estimator predicts well but which one is *right*.

3.16.1 The model and the design matrix

Consider L classical spins $s_j \in \{-1, +1\}$ arranged on a ring, with the nearest-neighbour Hamiltonian

$$H(\boldsymbol{s}) \;=\; -J \sum_{j=1}^{L} s_j s_{j+1}, \qquad s_{L+1} \equiv s_1. \tag{3.101}$$

This is the one-dimensional Ising model, and $J > 0$ favours aligned neighbours. Now forget that we know Eq. (3.101) and suppose instead that we are handed a pile of configurations $\boldsymbol{s}^{(i)}$ together with their energies $y_i = H(\boldsymbol{s}^{(i)})$, and are asked to discover the physics. A reasonable thing to assume is that the interaction is pairwise but otherwise unrestricted,

$$H(\boldsymbol{s}) \;=\; -\sum_{j=1}^{L}\sum_{k=1}^{L} J_{jk}\, s_j s_k, \tag{3.102}$$

with an unknown coupling matrix $\boldsymbol{J}$. The point of writing it this way is that Eq. (3.102) is *linear in the unknowns*: the L^2 numbers J_{jk} enter multiplied by the observable products $s_j s_k$. Setting $\theta_{jL+k} = -J_{jk}$ and

$$X_{i,\,jL+k} \;=\; s_j^{(i)} s_k^{(i)} \tag{3.103}$$

turns the problem into $\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\theta}$, an ordinary linear model with $p = L^2$ features, and every tool of this chapter applies unchanged. The physics has been hidden inside the design matrix, which is exactly what the phrase *feature engineering* means.

```
import numpy as np

rng = np.random.default_rng(2718)
L, n, Jtrue = 40, 10000, 1.0
spins = rng.choice([-1, 1], size=(n, L))
energies = -Jtrue * np.einsum("ij,ij->i", spins, np.roll(spins, 1, axis=1))

# design matrix, Eq. (3.isingdesign): X[i, j*L+k] = s_j s_k
X = np.einsum("ij,ik->ijk", spins, spins).reshape(n, L * L)
```

There is a price for the convenience, and it is paid immediately. The design matrix (3.103) is not of full column rank, and not by a little.

Proposition 3.14 (The Ising design matrix is exactly rank deficient). *Let $\boldsymbol{X}$ be given by Eq. (3.103) for spins $s_j \in \{-1,+1\}$. Then*

$$\operatorname{rank} \boldsymbol{X} \;\le\; \frac{L(L-1)}{2} + 1 \tag{3.104}$$

for every sample size n, whereas the number of columns is L^2.

Proof. Two families of columns coincide identically, whatever the configurations. First, the column indexed (j,j) has entries $\left(s_j^{(i)}\right)^2 = 1$ for every i, since $s_j \in \{-1,+1\}$; all L diagonal columns are therefore the constant column $\mathbf{1}$ and equal to one another. Second, the columns indexed (j,k) and (k,j) have entries $s_j s_k$ and $s_k s_j$, which are the same number. So the L^2 columns take at most $L(L-1)/2+1$ distinct values: one for the diagonal, and one for each unordered pair $j<k$. The rank cannot exceed the number of distinct columns. □

For $L = 40$ the bound is $40 \cdot 39/2 + 1 = 781$ against $p = 1600$ columns, so more than half the columns are redundant *by construction*. This is worth pausing over. Section 3.8 motivated the penalty by saying that $\boldsymbol{X}^T\boldsymbol{X}$ becomes ill-conditioned when features are nearly collinear; here they are exactly collinear, and no amount of extra data can help, because the coincidences of the proof hold configuration by configuration. The program confirms both the identities and the bound:

```
max |X[:, diagonal] - 1|                    : 0.000e+00
max |X[:, (j,k)] - X[:, (k,j)]| over all j<k: 0.000e+00

distinct columns therefore number L(L-1)/2 + 1 = 781,
so rank(X) <= 781 however many configurations we draw.
measured rank(X) with n = 10000 rows          : 781
number of columns p = L^2                     : 1600
singular values: sigma_0 = 633.8002, sigma_780 = 1.0218e+02, sigma_781 = 4.7516e-13
ratio sigma_780 / sigma_781 : 2.150e+14
```

The bound is attained, and the singular value spectrum falls off a cliff exactly at index 781, dropping by fourteen orders of magnitude between one singular value and the next. Figure 3.8 shows it. In the expansion (3.11) the least-squares coefficient along each right singular vector is $\boldsymbol{u}_i^T\boldsymbol{y}/\sigma_i$; here 819 of those denominators are zero, and the pseudoinverse of Section 3.3 handles them by declaring the corresponding coefficients to be zero – the minimum-norm choice. That convention is about to have a visible physical consequence.

3.16.2 Fitting with p four times n

We use $n_{\text{train}} = 400$ configurations to fit $p = 1600$ coefficients and keep the remaining 9600 for testing. The problem is therefore underdetermined twice over: once because $p > n$, and

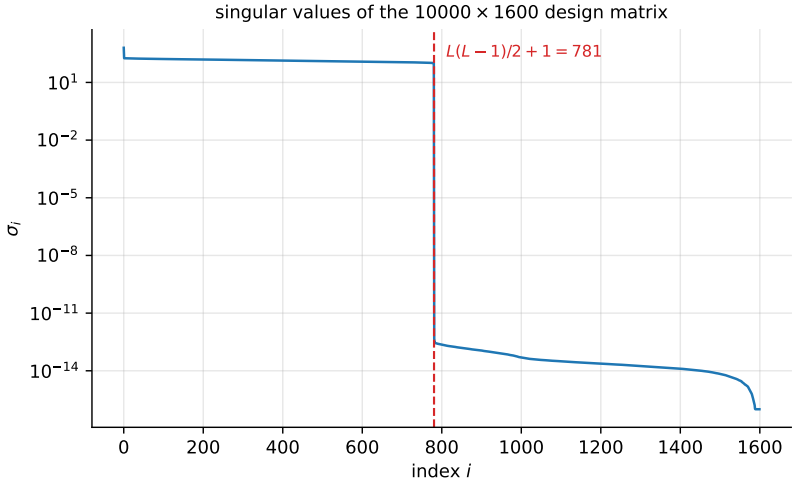


Fig. 3.8 Singular values of the 10000×1600 Ising design matrix (3.103) for $L = 40$. The spectrum collapses precisely at the index $L(L-1)/2+1 = 781$ predicted by Proposition 3.14: the rank deficiency is a property of the features, not of the sample.

once more because of Proposition 3.14. Ridge regression is solved in the dual form of Theorem 3.11, which here means a 400×400 system instead of a 1600×1600 one, and the Lasso by the coordinate descent of Eq. (3.69).

```
estimator            R^2 train  R^2 test  ||theta||_2  ||theta||_1  non-zero
OLS (pseudoinverse)   1.000000  0.522863       3.2610      88.9391      1600
Ridge, lambda = 0.01  1.000000  0.522863       3.2609      88.9379      1600
Lasso, lambda = 0.01  0.999973  0.999963       6.1351      39.7766        78
```

Three things in one table. OLS reproduces the training energies exactly – with 1600 free parameters and 400 equations it could hardly fail – and then explains barely half the variance of the test set. Ridge at $\lambda = 10^{-2}$ is indistinguishable from OLS to four decimal places, which is not a bug and is discussed below. And the Lasso is essentially perfect on data it has never seen, using 78 of the 1600 coefficients.

The last number is the interesting one, because $78 = 2 \times 39$ is very nearly $2L = 80$: the Lasso has found the nearest-neighbour couplings and almost nothing else. Figure 3.9 shows the three recovered coupling matrices, and the difference is not subtle.

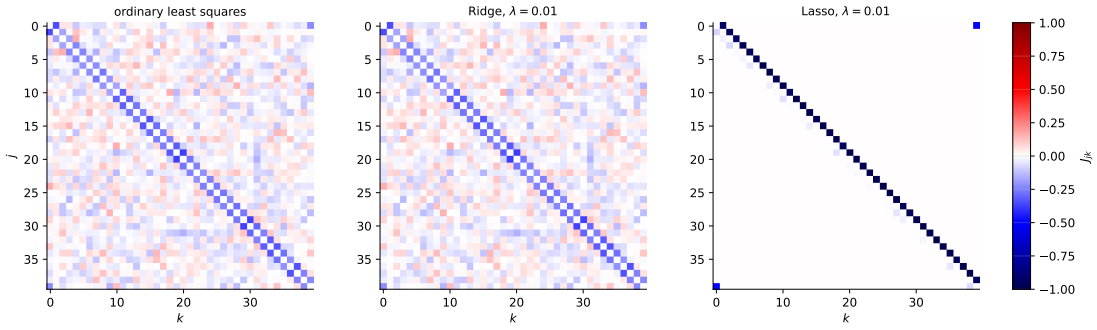


Fig. 3.9 The coupling matrices J_{jk} recovered from 400 configurations of the $L = 40$ Ising chain. The truth is a single off-diagonal band. OLS and Ridge spread the interaction over the whole matrix and split each coupling symmetrically between (j,k) and (k,j); the Lasso finds the band and puts each coupling entirely on one side of the pair. Both behaviours are forced, by Proposition 3.15.

3.16.3 Why the symmetry, and why only for two of the three

The pattern in Figure 3.9 is not a quirk of this data set. It is a theorem about duplicated columns, and it explains the behaviour of all three estimators at once.

Proposition 3.15 (Duplicated columns). *Suppose two columns of $\boldsymbol{X}$ are identical, say columns a and b. Then*

1. *the Ridge estimator (3.44) satisfies $\hat{\theta}_a = \hat{\theta}_b$ for every $\lambda > 0$, and so does the minimum-norm least-squares solution (3.11);*
2. *the Lasso fit $\boldsymbol{X}\hat{\boldsymbol{\theta}}$ is unique, but any redistribution of $\hat{\theta}_a + \hat{\theta}_b$ between the two coefficients that keeps both of the same sign is also a minimiser.*

Proof. Let P be the permutation that swaps coordinates a and b. Because the two columns are equal, $\boldsymbol{X}P = \boldsymbol{X}$, so the residual sum of squares obeys $\mathrm{RSS}(P\boldsymbol{\theta}) = \mathrm{RSS}(\boldsymbol{\theta})$; the 2-norm satisfies $\|P\boldsymbol{\theta}\|_2 = \|\boldsymbol{\theta}\|_2$ as well, so the whole Ridge objective is invariant under P. It is also strictly convex, hence has a unique minimiser $\hat{\boldsymbol{\theta}}$, and $P\hat{\boldsymbol{\theta}}$ is a minimiser too; uniqueness forces $P\hat{\boldsymbol{\theta}} = \hat{\boldsymbol{\theta}}$, that is $\hat{\theta}_a = \hat{\theta}_b$. For OLS the objective is invariant but not strictly convex; the minimum-norm solution is nevertheless unique, and the same argument applies to it because P preserves the norm being minimised.

For the Lasso, uniqueness of the fit is Proposition 3.9. Write $t = \hat{\theta}_a + \hat{\theta}_b$ and replace $(\hat{\theta}_a, \hat{\theta}_b)$ by $(u, t-u)$. The residual depends only on t, since the columns are equal, and if u and $t-u$ share the sign of t then $|u| + |t-u| = |t|$, so the penalty is unchanged as well. The objective is therefore constant along that whole segment. □

Applied here: the columns (j,k) and (k,j) are equal, so OLS and Ridge must report $J_{jk} = J_{kj}$, and the physics – which only ever constrains the sum $J_{jk} + J_{kj}$ – is split down the middle. The Lasso is under no such obligation, and the solver, sweeping the coordinates in order, arrives at a vertex of the optimal segment. Both statements are visible in the output:

```
estimator              J[0,1]    J[1,0]       sum    J[0,0]    J[0,5]
OLS (pseudoinverse)   -0.34863  -0.34863  -0.69726  -0.00490  -0.03488
Ridge, lambda = 0.01  -0.34863  -0.34863  -0.69725  -0.00490  -0.03488
Lasso, lambda = 0.01  -0.91691  -0.08149  -0.99840  -0.00000  -0.00000
```

```
estimator            max |J[j,k] - J[k,j]| over all j<k
OLS (pseudoinverse)                           9.437e-16
Ridge, lambda = 0.01                          0.000e+00
Lasso, lambda = 0.01                          9.990e-01
```

The 2-norm estimators are symmetric to machine precision, as the proposition requires, and the Lasso is as asymmetric as it is possible to be, the two entries differing by very nearly the whole coupling. The sum, though, is -0.9984 against a true value of -1: what the data determine, the Lasso gets right. That the split is genuinely arbitrary can be checked directly, by symmetrising the Lasso solution and re-evaluating:

```
Lasso objective at the solver's answer : 0.3988830154
Lasso objective at the symmetrised one : 0.3988830154
max |fit difference| on the test set   : 7.105e-15
```

Ten identical digits, and identical predictions. This is Proposition 3.9 on real data: the fit is unique, the coefficients are not, and any statement about *which* coefficient the Lasso selected among an exchangeable group is a statement about the solver.

Averaged over all forty nearest-neighbour pairs the same picture holds, and the off-band entries make the case for the penalty:

```
estimator            mean J[j,j+1]  mean J[j+1,j]  mean sum  mean |J| off-band
OLS (pseudoinverse)       -0.26585       -0.26585  -0.53169            0.04559
Ridge, lambda = 0.01      -0.26584       -0.26584  -0.53168            0.04559
Lasso, lambda = 0.01      -0.96324       -0.03115  -0.99439            0.00000
```

The Lasso recovers 99.4% of the true coupling and puts exactly zero everywhere else. OLS recovers barely half of it and smears the missing half over the 1520 entries that should vanish – each individually small, at 0.046 on average, but there are a great many of them and together they are what destroys the test score.

3.16.4 The penalty, over eight decades

The one puzzle left in the first table is why Ridge and OLS agreed to four decimals at $\lambda = 10^{-2}$. Equation (3.46) answers it: the Ridge shrinkage factor is $\sigma_i^2/(\sigma_i^2 + \lambda)$, so the penalty does nothing at all until λ becomes comparable with the squared singular values. On the 400 training rows those are

```
sigma_max^2 = 1.747e+04  down to  sigma_min^2 = 128.8
```

so $\lambda = 10^{-2}$ is smaller than the smallest of them by four orders of magnitude and is, quite literally, negligible. A useful habit follows: the Ridge parameter has the units of a squared singular value, and a penalty grid should be chosen with the spectrum of $\boldsymbol{X}$ in view rather than by reflex around 1. Scanning eight decades:

```
   lambda     Ridge train   Ridge test
     0.01        1.000000     0.522863
        1        0.999999     0.522876
      100        0.994434     0.514548
     1000        0.861276     0.391779
    1e+04        0.324439     0.120117
    1e+06        0.002912     0.001536
    1e+08       -0.001731    -0.000051
```

Ridge never improves on OLS here. Its best test score over the whole scan is 0.522893 at $\lambda = 3.162$, an improvement on OLS in the fifth decimal place, after which the penalty simply destroys the fit. The Lasso, on the same problem, does something else entirely:

```
   lambda     Lasso train   Lasso test   Lasso non-zeros
   0.0001        1.000000     0.455963               964
0.0003162        1.000000     1.000000                76
    0.001        1.000000     1.000000                77
     0.01        0.999973     0.999963                78
      0.1        0.997276     0.996339                78
        1        0.727612     0.633896                55
    3.162        0.034063     0.018081                 3
       10       -0.001778    -0.000067                 0
```

There is a plateau three decades wide over which the test score is essentially one and the model has about $2L$ non-zero coefficients, and it is bounded on both sides for the reasons the chapter gave. Below it the penalty is too weak to resolve the exchangeable columns and the solver keeps 964 of them, at which point the test score collapses to 0.456 – *worse* than OLS, even though the training score is still 1. Above it we pass Proposition 3.8, the coefficients are driven to zero one after another, and at $\lambda = 10$ none survive. Figure 3.10 plots both curves.

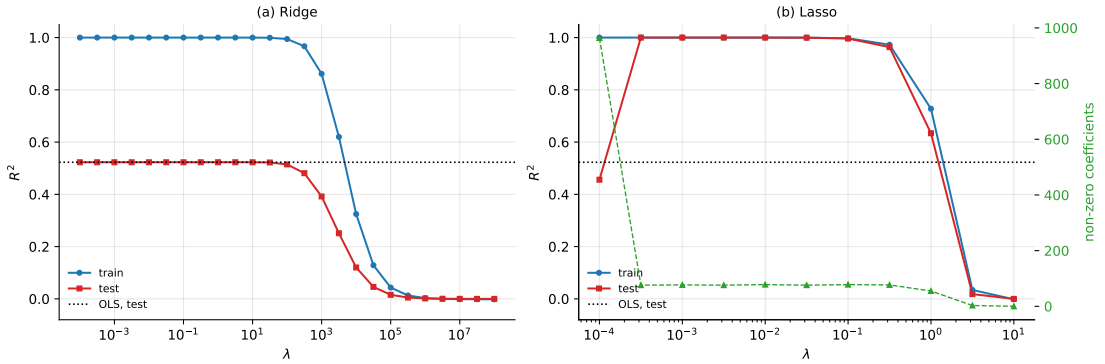


Fig. 3.10 Test and training R^2 against the penalty for the Ising problem, with the OLS test score marked. Ridge (a) needs $\lambda \sim 10^3$ before anything happens, and never beats OLS. The Lasso (b) has a broad plateau on which it is exact, and the count of non-zero coefficients (right axis, dashed) sits at about $2L = 80$ across it.

3.16.5 What the example is for

The headline is that on a problem with a sparse truth the Lasso reached $R^2 = 0.99996$ on unseen data from 400 samples and 1600 parameters, while OLS and Ridge reached 0.52. But the headline is the least transferable part, because it is a consequence of the truth being sparse in the basis we chose, and we chose the basis knowing the answer. Three things generalise.

First, the failure of OLS and Ridge is not a failure of computation. Both returned the correct minimiser of the objective they were given; the objective was simply the wrong one, because 819 directions in parameter space leave the training energies unchanged and something other than the data must decide what happens along them. Ridge decides by the 2-norm and the Lasso by the 1-norm, and Proposition 3.15 says exactly what each choice does. This is the content of Section 3.11 made concrete: in the underdetermined regime the penalty *is* the model.

Second, sparsity and symmetry are in conflict, and one has to choose. The Lasso got the physics right in the only sense the data allow – the sum $J_{jk} + J_{kj}$ – and got the presentation wrong, reporting a coupling matrix that is not symmetric although the underlying interaction certainly is. If a symmetric answer is wanted, the honest fix is not to post-process the Lasso but to remove the redundancy from the design matrix beforehand, keeping one column per unordered pair $j < k$ and none of the diagonal. That reduces p from 1600 to 781, makes the design matrix full rank by Proposition 3.14, and is what one should do in practice. We kept the redundant parametrisation here precisely because the pathology it produces is instructive.

Third, and most important for what follows: everything above depended on someone knowing to build the features $s_j s_k$. The linear model never saw a spin, only a product of two spins, and the products were supplied by a physicist who already suspected a pairwise interaction. Section 8.16 returns to this same chain with a neural network, hands it the raw spins with no products at all, and asks whether the features can be discovered rather than assumed. The answer involves a theorem, and a caution.

Machine learning connection. Discovering an interaction from configurations and energies is a real and active use of regression in statistical physics and in materials science, where it is called inverse statistical mechanics or, in the lattice-model community, cluster expansion: one fits an effective Hamiltonian to energies computed by an expensive first-principles method, and then uses the fitted model in place of the expensive one. Every-

thing this section illustrates matters there. The candidate interactions vastly outnumber the configurations one can afford to compute, the physically correct answer is sparse – few clusters contribute appreciably – and symmetry makes whole groups of features exactly exchangeable. Practitioners consequently reach for the Lasso and for structured variants of it, and they take care to remove symmetry-equivalent duplicates from the design matrix before fitting rather than after, for the reason Proposition 3.15 gives.

3.17 Summary and the programs

Linear regression has given us, in closed form, every quantity the rest of this book will only be able to estimate.

The estimator itself arrived three times over. As an optimisation problem it is the minimiser of the squared error, Eq. (3.8); as geometry it is the orthogonal projection of the targets onto the column space of the design matrix, Eq. (3.9); and as statistics it is the maximum-likelihood estimate under Gaussian noise, Eq. (3.28). The three derivations are worth holding together, because each generalises differently: the first survives into every method in this book, the second is what makes the linear case special, and the third is what tells us that a different noise model demands a different loss.

The statistics of the estimator all flow from one object, the residual projector $\boldsymbol{I} - \boldsymbol{H}$, which annihilates the model and leaves the noise: it gives the divisor $n-p$ in $\hat{\sigma}^2$, the χ^2_{n-p} distribution behind the reduced χ^2 and the t intervals, the optimism $2p\sigma^2/n$ of the training error, and through its diagonal the leave-one-out formula (3.39).

The three penalties then formed a sequence. Ordinary least squares is unbiased and, by Gauss-Markov (Theorem 3.4), of minimum variance among unbiased linear estimators – and that guarantee is worth less than it sounds, because Eq. (2.27) counts variance alongside bias. Ridge regression buys a large reduction in variance for a small bias, shrinking each singular direction by $\sigma_i^2/(\sigma_i^2+\lambda)$ and hence suppressing hardest exactly those directions in which Eq. (3.42) shows the data to be least informative. The Lasso replaces proportional shrinkage by soft thresholding (3.63), and thereby sets coefficients exactly to zero – a difference which is visible in the algebra, in the geometry of the 1-norm ball, and in the shape of the corresponding Laplace prior.

The Bayesian reading of Section 3.11 unified the three: a flat prior gives OLS, a Gaussian prior gives Ridge, a Laplace prior gives the Lasso. Regularisation is the expression of a prior belief, and the regularisation parameter is its strength.

Finally, none of this survives careless practice. Penalties are meaningless unless the features are on comparable scales; the intercept must be excluded from the penalty; every transformation must be fitted on training data alone; and the number one reports must come from data the model has never seen. The penalty itself is chosen by the cross-validation of Section 3.14: a logarithmic grid, K folds with the standardisation inside each, the minimum of Eq. (3.92) or the one-standard-error choice (3.96), a refit on all training data, and one look at the test set. Cross-validation is preferred to the bootstrap for this purpose because its bias is nearly constant in λ and so leaves the minimum in place, because it costs K fits rather than hundreds, and because its models see $(K-1)/K$ of the data rather than the 63% of Eq. (3.97); the bootstrap keeps its role of estimating the distribution of whatever cross-validation selects, above all the Lasso's support.

A last thread runs underneath all three estimators and is the subject of Section 3.12. The 2-norm penalty is invariant under a change of orthonormal basis in the feature space, so it can be expressed through inner products alone; the push-through identity of Lemma 3.10 then rewrites Ridge regression in terms of the $n \times n$ matrix of those inner products, the basis

expansion becomes free, and Theorem 3.12 shows that the same is true for any loss. The 1-norm is *not* invariant, Proposition 3.13, and the Lasso therefore does not kernelise – which is worth knowing, because the literature contains two different repairs travelling under the same name.

The complete programs are collected in `doc/BookML/BookPrograms`:

- `kernel_regression.py` – the push-through identity, the Hoerl-Kennard theorem, kernel ridge regression against `scikit-learn` and against an explicit feature map, the rotation counter-example of Proposition 3.13, and both repairs of Section 3.12.4.
- `linear_regression.py` – OLS by the SVD, by QR and by the normal equations, with the conditioning comparison of Section 3.3; weighted least squares and the reduced-χ^2 simulation of Section 3.4; the quality measures of Section 3.5; and the checks of Section 3.7 – unbiasedness of $\hat{\sigma}^2$, coverage of the t intervals, the optimism formula (3.37) and leave-one-out in closed form.
- `ridge_lasso.py` – Ridge in closed form and through the SVD, the Lasso by coordinate descent with soft thresholding, and the coefficient paths of Section 3.10 compared against the analytical results.
- `cross_validation.py` – the procedure of Section 3.14 from scratch: K-fold cross-validation with standardisation inside the folds for Ridge and the Lasso, the minimum-CV and one-standard-error choices, the refit and the single test evaluation, the `GridSearchCV` equivalent checked to rounding error, the leave-one-out bootstrap and .632 estimators against cross-validation and the true error, the repeated experiment behind the bias and spread table, and the bootstrap selection frequencies of the Lasso support.
- `franke.py` – the complete study of Section 3.15: design matrices for two-dimensional polynomials, the train-test split, cross-validated selection of λ, test error against polynomial degree for all three methods, and the count of non-zero Lasso coefficients.
- `scaling.py` – the intercept and standardisation conventions of Section 3.13, with a reconciliation of a hand-written implementation against `scikit-learn`.
- `ising_regression.py` – the one-dimensional Ising study of Section 3.16: the exact rank deficiency of Proposition 3.14, the three estimators at $p = 4n$, the duplicated-column symmetry of Proposition 3.15 and the indifference of the Lasso objective to the split, and the penalty scan over eight decades.

Each file runs as a script and reproduces the numbers quoted in this chapter. An executable version is available as a Jupyter notebook in the accompanying Jupyter-book.

3.18 Exercises

Warm-up exercises

1. **The normal equations by hand.** For the toy problem of Eq. (3.70): (a) compute $\boldsymbol{X}^T\boldsymbol{X}$ and $\boldsymbol{X}^T\boldsymbol{y}$ and verify Eq. (3.72); (b) compute the hat matrix (3.9) and verify that it is symmetric, idempotent, and has trace 2; (c) verify that the residual is orthogonal to both columns of $\boldsymbol{X}$.
2. **Ridge by hand.** For the same problem, derive Eq. (3.73) by differentiating the penalised cost, and explain why the two coefficients are shrunk by different factors. Relate the factors to the singular values of $\boldsymbol{X}$.
3. **The Lasso threshold.** For the same problem, show that the Lasso sets $\theta_1 = 0$ for $\lambda \geq 4$ and $\theta_0 = 0$ for $\lambda \geq 16$ (with the convention of Eq. (3.57) and the $1/n$ dropped). Sketch all three coefficient paths on one plot.

4. **Equivariance under rescaling.** (a) Show that if a column of $\boldsymbol{X}$ is multiplied by c, the OLS fitted values $\tilde{\boldsymbol{y}}$ are unchanged while the corresponding coefficient is divided by c. (b) Show that this fails for Ridge, and explain in one sentence why standardisation is therefore mandatory for penalised regression and optional for OLS.
5. **Maximum likelihood with a different noise model.** Repeat the derivation of Section 3.6 assuming Laplace noise, $p(\varepsilon) \propto \exp(-|\varepsilon|/b)$. Which cost function results? What does this say about the robustness of the two fits to outliers?
6. **Degrees of freedom.** (a) Show that $\mathrm{Tr}(\boldsymbol{H}) = p$ for the OLS hat matrix. (b) Show that the Ridge analogue $\boldsymbol{H}_\lambda = \boldsymbol{X}(\boldsymbol{X}^T\boldsymbol{X} + \lambda \boldsymbol{I})^{-1}\boldsymbol{X}^T$ has trace $\sum_i \sigma_i^2/(\sigma_i^2 + \lambda)$, which is Eq. (1.132). (c) Verify that $\boldsymbol{H}_\lambda$ is symmetric but *not* idempotent for $\lambda > 0$, and interpret this using the eigenvalue argument of Section 1.6.
7. **The analysis-of-variance identity.** (a) Show that if the design matrix has no constant column, the residuals of a least-squares fit need not sum to zero, and give a two-point example in which Eq. (3.20) fails. (b) Show that for a fit with intercept the sample correlation between $\boldsymbol{y}$ and $\tilde{\boldsymbol{y}}$ squared equals R^2. (c) Compute R^2 and $\bar{R}^2$ of Eq. (3.22) for a polynomial fit of degree $1, \dots, 15$ to 20 noisy points and plot both against the degree.
8. **Generalised least squares.** Noise with an autoregressive structure has covariance $\Omega_{ij} = \sigma^2 \rho^{|i-j|}$. (a) Simulate such noise for $\rho = 0.8$ and fit a straight line by OLS and by GLS, Eq. (3.15), many times; compare the empirical variances of the slope with the two predicted covariance matrices. (b) Show that the OLS variance formula $\sigma^2(\boldsymbol{X}^T\boldsymbol{X})^{-1}$ is wrong in this case, and derive the correct $\mathrm{Var}(\hat{\boldsymbol{\theta}}_{\mathrm{OLS}}) = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{\Omega}\boldsymbol{X}(\boldsymbol{X}^T\boldsymbol{X})^{-1}$.
9. **The Cramér-Rao bound for the noise variance.** Compute the Fisher information for σ^2 from Eq. (3.27), $\mathscr{I}(\sigma^2) = n/(2\sigma^4)$, and hence the bound $\mathrm{Var}(\tilde{\sigma}^2) \geq 2\sigma^4/n$. Using Proposition 3.3(iii), show that $\mathrm{Var}(\hat{\sigma}^2) = 2\sigma^4/(n-p)$: the unbiased estimator does not attain the bound, and the gap closes as $p/n \to 0$.
10. **Leave-one-out by Sherman-Morrison.** (a) Verify Eq. (3.38) by multiplication. (b) Derive both parts of Eq. (3.39); the key step is $\boldsymbol{x}_i^T(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{x}_i = h_{ii}$. (c) Show that $0 \leq h_{ii} \leq 1$ and $\sum_i h_{ii} = p$, so that the average leverage is p/n; explain what happens to Eq. (3.39) at a point with $h_{ii} = 1$. (d) Repeat the numerical comparison of Section 3.7 for the Ridge smoother, and confirm that Eq. (3.40) still holds with S_{ii} in place of h_{ii}.
11. **Optimism (numerical).** For polynomial fits of degree $d = 1, \dots, 12$ to $n = 30$ points generated by a cubic plus Gaussian noise, estimate by simulation the expected training and test errors and plot both against d together with the predictions (3.35) and (3.36). Where do the predictions fail, and why? Add Mallows' C_p and check that its minimum sits at the true degree.
12. **Conditioning in practice (numerical).** Fit a polynomial of degree d to 100 points on $[0,1]$ using both functions of Section 3.3. (a) For $d = 2, 4, \dots, 14$, compare the coefficients returned by the two. (b) Plot $\kappa_2(\boldsymbol{X})$ and $\kappa_2(\boldsymbol{X}^T\boldsymbol{X})$ against d. (c) At which d do the two methods first disagree in the first significant digit, and how does that relate to Eq. (1.118)?
13. **Coordinate descent (numerical).** Implement the Lasso by coordinate descent as in Eq. (3.69). (a) Verify on an orthonormal design that it reproduces the soft thresholding result (3.63) exactly. (b) Compare against `sklearn.linear_model.Lasso` on a general design, taking care over the factor conventions noted in Section 3.10. (c) Plot the number of non-zero coefficients against λ.
14. **Confidence intervals (numerical).** For data generated from a known linear model: (a) compute the confidence intervals (3.34) for each coefficient; (b) repeat the whole experiment 1000 times with fresh noise and count how often the true value lies inside the interval; (c) does the coverage match the nominal 95%? Repeat with two nearly collinear columns and comment.
15. **Ridge and the bias-variance trade-off (numerical).** For a fixed polynomial degree, plot the squared bias, the variance and the test error of a Ridge fit against λ, using the boot-

strap procedure of Section 2.10. Verify that the bias increases monotonically, the variance decreases monotonically, and the minimum of the sum occurs at $\lambda > 0$.

16. **The training error is monotone in the penalty.** Let $\hat{\boldsymbol{\theta}}(\lambda)$ minimise $L(\boldsymbol{\theta}) + \lambda P(\boldsymbol{\theta})$ for a loss L and a penalty P, both arbitrary. For $\lambda_1 < \lambda_2$ write down the two inequalities expressing that each minimiser beats the other at its own penalty, add them, and conclude that $P(\hat{\boldsymbol{\theta}}(\lambda_2)) \le P(\hat{\boldsymbol{\theta}}(\lambda_1))$ and $L(\hat{\boldsymbol{\theta}}(\lambda_1)) \le L(\hat{\boldsymbol{\theta}}(\lambda_2))$. This proves Eq. (3.91) along the whole path, for Ridge and the Lasso alike, and shows that the training error can never select a penalty. Verify it numerically on the dotted curves of Figure 3.4.
17. **The factor n.** (a) Show that the minimiser of $\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2/n + \lambda\|\boldsymbol{\theta}\|_2^2$ is Eq. (3.95), and that the same estimator minimises $\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2 + \lambda'\|\boldsymbol{\theta}\|_2^2$ with $\lambda' = n\lambda$. (b) A penalty λ' is selected by five-fold cross-validation in the second convention and then used to refit on all n training points. By what factor is the effective per-observation penalty changed between selection and refit, and in which direction? (c) Repeat the `GridSearchCV` run of Section 3.14 with `alpha` set to 100λ instead of 80λ and confirm the shift of one grid point.
18. **Out of the bag.** (a) Prove Eq. (3.97) and evaluate $(1 - 1/n)^n$ for $n = 10$, 100 and 1000. (b) Show that the expected number of *distinct* observations in a bootstrap sample is $n[1 - (1 - 1/n)^n]$, and that the expected number of replicas out of B in which a given observation is absent is $B(1 - 1/n)^n$; hence explain why the `seen` mask in the code of Section 3.14 is almost never needed for $B = 200$. (c) For which K does K-fold cross-validation train on the same fraction of the data as the leave-one-out bootstrap?
19. **The one-standard-error rule (numerical).** For the sparse problem of Section 3.14, generate 200 training sets and, for each, select the Lasso penalty by the minimum-CV rule and by the one-standard-error rule. (a) Record for each rule the number of retained coefficients, the number of true coefficients retained, and the true test error measured on a large fresh sample. Report the averages. (b) Which rule retains fewer spurious coefficients, and what does it cost in test error on average? (c) Repeat with $\rho = 0$ and with $\rho = 0.9$ and explain what the correlation between features does to both rules.
20. **Nested cross-validation (numerical).** Without using a test set, estimate the generalisation error of the complete Ridge procedure of Section 3.14 – grid, five-fold selection, refit – by nesting it in an outer five-fold loop. Compare the outer estimate with $\mathrm{CV}_5(\lambda_{\min})$ from the inner loop and with the true error from a large fresh sample, then repeat with a grid of 600 penalties instead of 60 and with $n = 40$ instead of 100. When does the difference between the inner minimum and the outer estimate become important?

Project-style exercise: regression analysis of the Franke function

This extended exercise assembles the whole chapter, and follows the structure of the first project of the course.

Part a: ordinary least squares.

Generate a data set from the Franke function (3.100) with $x, y \in [0,1]$ and added noise $\mathcal{N}(0, \sigma^2)$. Write your own code – using either a matrix inversion or, preferably, the singular value decomposition – and perform a standard least-squares analysis with polynomials in x and y up to fifth order.

a) Compute the mean squared error (3.17) and the R^2 score (3.18) on both training and test data.

b) Compute the confidence intervals (3.34) of the parameters by evaluating their variances from Eq. (3.32).
c) Split the data into training and test sets, using between $2/3$ and $4/5$ for training. Present a critical discussion of why and how you have scaled the data, or of why you have not.
d) Plot the training and test MSE against polynomial degree, and identify the regions of high bias and high variance.

Part b: the bias-variance trade-off with the bootstrap.

Using the bootstrap of Section 2.11, decompose the test error into bias and variance as in Eq. (2.52) and plot all three against polynomial degree. Explain the shape of each curve. Then repeat with a substantially larger data set and account for the shift in the optimum.

Part c: cross-validation.

Implement k-fold cross-validation from scratch, with k between five and ten, and compare its estimate of the test error with the bootstrap estimate of part b. Which is cheaper, and which has the smaller variance? Relate your findings to the comparison of Section 3.14, in particular to the fraction of distinct observations in a bootstrap sample, Eq. (3.97).

Part d: Ridge regression.

Repeat parts a to c for Ridge regression, as a function of both the polynomial degree and the penalty λ. Present your results as a heat map of the cross-validated test error over the two hyperparameters, and identify the optimum. Discuss the results in the light of the shrinkage (3.48) and of the variance difference (3.52).

Part e: the Lasso.

Repeat part d for the Lasso, using either your own coordinate-descent implementation or `scikit-learn`. In addition, count the non-zero coefficients at the optimal λ and compare with the number retained by Ridge. Which monomials survive, and does the selection agree with your expectation from the shape of the Franke surface?

Part f: real data.

Finally, replace the Franke function by a real data set – digital terrain data are a natural choice, being a genuine two-dimensional surface – and repeat the analysis. Discuss what changes when the true function is unknown, the noise level is unknown, and the observations may be spatially correlated. In particular, reconsider whether a random train-test split is defensible, in the light of Section 2.7.

Chapter 4
Optimization

Almost every problem in machine learning begins with a data set $\boldsymbol{X}$, a model $g(\boldsymbol{\theta})$ depending on parameters $\boldsymbol{\theta}$, and a cost function $C(\boldsymbol{X}, g(\boldsymbol{\theta}))$ measuring how badly the model explains the data. The model is fitted by finding the $\boldsymbol{\theta}$ that minimises the cost. In Chapter 3 we were fortunate: the cost was quadratic, its gradient vanished at a point we could write down, and the minimisation reduced to a linear system. That good fortune does not survive contact with logistic regression, and it certainly does not survive neural networks. For everything that follows in this book we must find minima numerically.

This chapter builds the tools. We begin with the question of when a minimum is worth looking for at all – convexity – and with Newton's method, which is the ideal we shall spend the rest of the chapter approximating cheaply. We then develop gradient descent, using linear regression as a test bed precisely because we already know the answer, and we identify exactly what makes plain gradient descent slow: the conditioning of the Hessian, a quantity we have already met twice, in Section 1.12 as a numerical diagnosis and in Section 3.7 as a statistical one. Everything after that – momentum, stochastic gradients, AdaGrad, RMSProp and Adam – is an attempt to defeat that one number without paying the cost of computing the Hessian.

The reader who wants a single organising idea should hold on to this: gradient descent takes the same step in every direction, but the cost function does not curve equally in every direction, and the whole art lies in fixing that mismatch.

4.1 Convexity

Ideally we want our cost function to be convex, and it is worth stating precisely what that buys us.

We first need convex sets. A set $C \subset \mathbb{R}^n$ is *convex* if, for all $\boldsymbol{x}$ and $\boldsymbol{y}$ in C and all $t \in (0,1)$, the point $(1-t)\boldsymbol{x}+t\boldsymbol{y}$ also belongs to C; geometrically, every point on the line segment joining two points of C lies in C. The convex subsets of $\mathbb{R}$ are the intervals; examples in $\mathbb{R}^2$ include the regular polygons and the discs.

Convex functions.

Let $X \subset \mathbb{R}^n$ be a convex set and $f : X \to \mathbb{R}$ continuous. Then f is *convex* if

$$f(t\boldsymbol{x}_1 + (1-t)\boldsymbol{x}_2) \leq t f(\boldsymbol{x}_1) + (1-t) f(\boldsymbol{x}_2) \tag{4.1}$$

for all $\boldsymbol{x}_1, \boldsymbol{x}_2 \in X$ and all $t \in [0,1]$. Replacing $\leq$ by a strict inequality, with $\boldsymbol{x}_1 \neq \boldsymbol{x}_2$ and $t \in (0,1)$, defines a *strictly convex* function. For a function of one variable the condition says that the chord joining $f(x_1)$ and $f(x_2)$ lies above the graph on the whole interval $[x_1, x_2]$.

First-order condition.

Suppose f is differentiable. Then f is convex if and only if its domain D_f is convex and

$$f(\boldsymbol{y}) \geq f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})^T (\boldsymbol{y} - \boldsymbol{x}) \tag{4.2}$$

for all $\boldsymbol{x}, \boldsymbol{y} \in D_f$. In words: the first-order Taylor expansion at any point is a global underestimator of the function. Drawing the tangent to $f(x) = x^2 + 1$ at any point and observing that it lies everywhere below the parabola is enough to make the statement believable.

Second-order condition.

If f is twice differentiable, so that the Hessian exists everywhere, then f is convex if and only if D_f is convex and the Hessian is positive semi-definite at every point of D_f. For one variable this reduces to $f''(x) \geq 0$: non-negative curvature everywhere. This condition is the useful one in practice, because it gives a procedure rather than a definition. Proofs of both conditions may be found in Boyd and Vandenberghe [1].

Why we care.

The result that matters is the following.

Any stationary point of a convex function is a global minimum. Let f be convex and differentiable. Then any $\boldsymbol{x}^*$ satisfying $\nabla f(\boldsymbol{x}^*) = \boldsymbol{0}$ minimises f globally.

The proof is one line from Eq. (4.2): setting $\boldsymbol{x} = \boldsymbol{x}^*$ makes the gradient term vanish, leaving $f(\boldsymbol{y}) \geq f(\boldsymbol{x}^*)$ for every $\boldsymbol{y}$. For a convex cost function, therefore, we need only find a point where the gradient vanishes and we are done – there are no local minima to be trapped in and no saddle points to be delayed by.

We have already used this twice. In Section 1.8.5 we showed that the least-squares Hessian is $\boldsymbol{H} = \boldsymbol{X}^T\boldsymbol{X}$, positive semi-definite because $\boldsymbol{z}^T\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{z} = \|\boldsymbol{X}\boldsymbol{z}\|_2^2 \geq 0$, so the OLS problem is convex and the normal equations (1.39) deliver the global minimum. Adding the Ridge penalty makes the Hessian $\boldsymbol{X}^T\boldsymbol{X} + \lambda\boldsymbol{I}$, positive definite for $\lambda > 0$, so the Ridge problem is *strictly* convex and its minimum is unique. The Lasso cost of Eq. (3.57) is convex but not differentiable, which is why Section 3.9 needed coordinate descent rather than a gradient.

The bad news is that the cost functions of the later chapters are not convex. A neural network with even one hidden layer has a cost surface with many local minima, and the guarantee above does not apply. We shall nonetheless use methods designed for the convex case, because they work in practice; but it is important to know that when we do so we are relying on empirical good behaviour rather than on a theorem.

4.2 Newton’s method

Before descending gradients it is worth recalling the method that uses curvature properly, both because it is the standard against which everything else is measured and because its cost is what motivates the alternatives.

Root finding in one dimension.

Newton’s method, also called Newton-Raphson, finds a root of f by extending the tangent line at the current point until it crosses zero. Taylor expanding about a point x close to the solution s,

$$f(s) = 0 = f(x) + (s-x)f'(x) + \frac{(s-x)^2}{2} f''(x) + \dots, \tag{4.3}$$

and discarding terms beyond the linear one gives $f(x)+(s-x)f'(x) \approx 0$, that is $s \approx x - f(x)/f'(x)$. As an iteration,

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}. \tag{4.4}$$

Geometrically x_{n+1} is where the tangent at $(x_n, f(x_n))$ meets the horizontal axis. Close to a root the convergence is quadratic and extremely fast. Far from one, where the discarded terms matter, the formula can be grossly inaccurate: if an iterate lands near a local extremum, so that f' nearly vanishes, the method can fail completely. If the derivative is available only numerically, or the function is not smooth, Newton’s method is best avoided.

Several variables.

For a system $f_1(x_1,x_2)=0$, $f_2(x_1,x_2)=0$, Taylor expansion gives

$$0 = f_k(x_1+h_1, x_2+h_2) = f_k(x_1,x_2) + h_1 \frac{\partial f_k}{\partial x_1} + h_2 \frac{\partial f_k}{\partial x_2} + \dots, \qquad k = 1,2, \tag{4.5}$$

and collecting the partial derivatives into the Jacobian matrix (1.16),

$$\boldsymbol{J} = \begin{pmatrix} \partial f_1/\partial x_1 & \partial f_1/\partial x_2 \\ \partial f_2/\partial x_1 & \partial f_2/\partial x_2 \end{pmatrix}, \tag{4.6}$$

the update becomes $\boldsymbol{x}^{n+1} = \boldsymbol{x}^n + \boldsymbol{h}^n$ with

$$\boldsymbol{h}^n = -\boldsymbol{J}^{-1} \boldsymbol{f}(\boldsymbol{x}^n). \tag{4.7}$$

We must invert the Jacobian, and difficulties arise when it is nearly singular – the conditioning problem of Section 1.12 again.

Newton’s method for minimisation.

Minimising $C(\boldsymbol{\theta})$ means finding a root of ∇C, so we apply the above with $\boldsymbol{f} = \nabla C$. The Jacobian of the gradient is the Hessian, and the iteration reads

$$\boxed{\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \boldsymbol{H}^{-1}(\boldsymbol{\theta}_k)\, \nabla C(\boldsymbol{\theta}_k).} \tag{4.8}$$

Equation (4.8) is the ideal against which the rest of this chapter should be read. It is what one gets by minimising the second-order Taylor expansion of the cost exactly, and it has two properties no gradient-only method can match: it converges quadratically near the minimum, and it is *invariant* under linear rescaling of the parameters, so that the conditioning problems which dominate everything below simply do not arise.

Applied to ordinary least squares it is not merely fast but exact. With $\nabla C = -2\boldsymbol{X}^T(\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta})/n$ from Eq. (1.38) and $\boldsymbol{H} = 2\boldsymbol{X}^T\boldsymbol{X}/n$ from Eq. (1.44), a single step from any starting point gives

$$\boldsymbol{\theta}_1 = \boldsymbol{\theta}_0 + \left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T\left(\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}_0\right) = \left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T\boldsymbol{y}, \tag{4.9}$$

the exact solution (3.8). This is not a coincidence: for a quadratic cost the second-order Taylor expansion is the function itself, so one Newton step lands on the minimum.

Why we do not simply use it.

For p parameters the Hessian has p^2 entries and inverting it costs $\mathcal{O}(p^3)$ operations by Section 1.14. For a linear regression with twenty features this is nothing. For a neural network with 10^7 parameters the Hessian would need 10^{14} numbers, which cannot be stored, let alone factorised. The entire subject of this chapter is the construction of methods that capture some of the benefit of Eq. (4.8) at a cost linear rather than cubic in p.

4.3 Gradient descent

The basic idea is that a function $F(\boldsymbol{x})$ decreases fastest, locally, if one moves from $\boldsymbol{x}$ in the direction of the negative gradient $-\nabla F(\boldsymbol{x})$. One can show that for

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - \gamma_k \nabla F(\boldsymbol{x}_k), \qquad \gamma_k > 0, \tag{4.10}$$

and for γ_k small enough, $F(\boldsymbol{x}_{k+1}) \leq F(\boldsymbol{x}_k)$: we always move towards smaller function values. Iterating Eq. (4.10) from an initial guess $\boldsymbol{x}_0$ is the method of *steepest descent,* or simply *gradient descent* (GD). The parameter γ_k is the step length, or in machine learning the *learning rate*. We use the symbol γ for it throughout this book, with a subscript when it changes from one iteration to the next.

Ideally the sequence converges to a global minimum. In general we do not know whether a minimum we reach is global or local; but by Section 4.1, if F is convex the question does not arise. The scheme is conceptually simple and easy to implement. It also has severe limitations.

Limitations.

In machine learning we are often faced with non-convex, high-dimensional cost functions with many local minima. Gradient descent is deterministic, so unless the initial guess is good it will descend into whichever local minimum it happens to face, and the result is sensitive to that initial condition. The gradient is a function of all p parameters and can be expensive to evaluate. And the method is delicate in its dependence on γ: we are guaranteed $F(\boldsymbol{x}_{k+1}) \leq F(\boldsymbol{x}_k)$ only for sufficiently small steps, so too large a value produces erratic behaviour or divergence while too small a value converges intolerably slowly. Determining a good γ is the central practical difficulty, and Sections 4.9 to 4.11 are devoted to removing the need to do so by hand.

Many of these shortcomings can be alleviated by introducing randomness, which is the subject of Section 4.7.

4.4 Gradient descent for linear regression

Linear regression is the ideal test case for gradient descent, for three reasons: it has an analytical solution to check against, its gradient can be computed exactly, and its cost function is convex so that convergence is guaranteed for small enough learning rates.

We take the data set used throughout Chapter 3,

$$y_i = 4 + 3x_i + \varepsilon_i, \qquad x_i \sim \mathcal{U}(0,2), \qquad \varepsilon_i \sim \mathcal{N}(0,1),$$

with $n = 100$ points, and the design matrix including an intercept column,

$$\boldsymbol{X} = \begin{bmatrix} 1 & x_0 \\ \vdots & \vdots \\ 1 & x_{n-1} \end{bmatrix} \in \mathbb{R}^{n\times 2}, \qquad \boldsymbol{\theta} = \begin{pmatrix} \theta_0 \\ \theta_1 \end{pmatrix}. \tag{4.11}$$

The cost function is

$$C(\boldsymbol{\theta}) = \frac{1}{n}\,\|\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}\|_2^2 = \frac{1}{n}\sum_{i=0}^{n-1}\left[(\theta_0 + \theta_1 x_i)^2 - 2y_i(\theta_0 + \theta_1 x_i) + y_i^2\right]. \tag{4.12}$$

Computing $\partial C/\partial\theta_0$ and $\partial C/\partial\theta_1$ separately and reassembling,

$$\nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}) = \frac{2}{n}\begin{bmatrix} \sum_i (\theta_0 + \theta_1 x_i - y_i) \\ \sum_i (x_i(\theta_0 + \theta_1 x_i) - y_i x_i) \end{bmatrix} = \frac{2}{n}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}), \tag{4.13}$$

which is Eq. (1.38) written out coordinate by coordinate. The Hessian is

$$\boldsymbol{H} = \begin{bmatrix} \dfrac{\partial^2 C}{\partial\theta_0^2} & \dfrac{\partial^2 C}{\partial\theta_0\partial\theta_1} \\ \dfrac{\partial^2 C}{\partial\theta_0\partial\theta_1} & \dfrac{\partial^2 C}{\partial\theta_1^2} \end{bmatrix} = \frac{2}{n}\boldsymbol{X}^T\boldsymbol{X}, \tag{4.14}$$

in agreement with Eq. (1.44), and positive semi-definite, so C is convex and any stationary point is the global minimum.

The gradient descent iteration is

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \gamma\nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}_k), \qquad k = 0, 1, \ldots \tag{4.15}$$

started from a random $\boldsymbol{\theta}_0$ and stopped when $\|\nabla_{\boldsymbol{\theta}} C\| \le \varepsilon$ for some tolerance, say 10^{-8}.

```
import numpy as np

n = 100
rng = np.random.default_rng(2024)
x = 2.0 * rng.random((n, 1))
y = 4.0 + 3.0 * x + rng.normal(size=(n, 1))
X = np.c_[np.ones((n, 1)), x]

# The analytical solution of Chapter 3, for comparison
theta_exact = np.linalg.pinv(X.T @ X) @ X.T @ y
```

```
print("analytical:", theta_exact.ravel())

# Gradient descent, Eq. (4.gditeration)
theta = rng.normal(size=(2, 1))
gamma = 0.1
for k in range(1000):
    gradient = (2.0 / n) * X.T @ (X @ theta - y)
    theta -= gamma * gradient
    if np.linalg.norm(gradient) < 1.0e-8:
        break
print(f"gradient descent after {k+1} iterations:", theta.ravel())
```

With $\gamma = 0.1$ the iteration reproduces the analytical answer to five decimal places within a few hundred steps. With $\gamma = 0.001$, the value used in the lecture notes, a thousand iterations are not nearly enough; with γ too large it diverges. The next section explains exactly where the boundary lies.

4.4.1 Gradient descent for Ridge regression

The same treatment applies to the Ridge cost of Eq. (3.43). Writing it as

$$C(\boldsymbol{\theta}) = \frac{1}{n}\,\|\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}\|_2^2 + \lambda\,\boldsymbol{\theta}^T\boldsymbol{\theta}, \tag{4.16}$$

the gradient and Hessian follow from Eqs. (1.25) and (1.28),

$$\nabla_{\boldsymbol{\theta}} C = \frac{2}{n}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}) + 2\lambda\,\boldsymbol{\theta}, \qquad \boldsymbol{H} = \frac{2}{n}\boldsymbol{X}^T\boldsymbol{X} + 2\lambda\,\boldsymbol{I}. \tag{4.17}$$

Setting the gradient to zero returns the closed form (3.44), as it must. The Hessian is now positive definite for every $\lambda > 0$, so the problem is strictly convex.

```
import numpy as np

lmbda = 0.001
theta = rng.normal(size=(2, 1))
gamma = 0.1
for k in range(1000):
    gradient = 2.0 * (X.T @ (X @ theta - y) / n + lmbda * theta)
    theta -= gamma * gradient

# Compare with the closed form of Chapter 3
I = np.eye(X.shape[1])
theta_exact = np.linalg.inv(X.T @ X + n * lmbda * I) @ X.T @ y
print("gradient descent:", theta.ravel())
print("closed form:     ", theta_exact.ravel())
```

Note the factor n in the closed form. Our cost (4.16) carries $1/n$ on the data term but not on the penalty, whereas Eq. (3.44) was derived with neither; the two agree only if λ is scaled accordingly. This is precisely the class of bookkeeping error warned against in Section 3.13, and it is worth tracking carefully whenever a penalised gradient is implemented.

4.5 The learning rate and the condition number

We can now answer the question of how large γ may be, and in doing so identify the quantity that governs everything in the rest of this chapter.

Consider a quadratic cost with symmetric positive definite Hessian $\boldsymbol{H}$, which by Eq. (4.14) is the case for least squares. Write the error at step k as $\boldsymbol{e}_k = \boldsymbol{\theta}_k - \boldsymbol{\theta}^*$, with $\boldsymbol{\theta}^*$ the minimum. Since $\nabla C(\boldsymbol{\theta}) = \boldsymbol{H}\boldsymbol{e}$ for a quadratic, the iteration (4.15) gives

$$\boldsymbol{e}_{k+1} = \boldsymbol{e}_k - \gamma \boldsymbol{H}\boldsymbol{e}_k = (\boldsymbol{I} - \gamma \boldsymbol{H})\,\boldsymbol{e}_k. \tag{4.18}$$

Now diagonalise. By the spectral decomposition (1.54), $\boldsymbol{H} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^T$ with orthonormal eigenvectors and eigenvalues $\lambda_i > 0$. Expressing the error in the eigenbasis, $\tilde{\boldsymbol{e}} = \boldsymbol{Q}^T\boldsymbol{e}$, and multiplying Eq. (4.18) from the left by $\boldsymbol{Q}^T$,

$$\tilde{\boldsymbol{e}}_{k+1} = \boldsymbol{Q}^T(\boldsymbol{I} - \gamma \boldsymbol{H})\,\boldsymbol{Q}\,\tilde{\boldsymbol{e}}_k = (\boldsymbol{I} - \gamma \boldsymbol{\Lambda})\,\tilde{\boldsymbol{e}}_k,$$

because $\boldsymbol{Q}^T\boldsymbol{Q} = \boldsymbol{I}$ and $\boldsymbol{Q}^T\boldsymbol{H}\boldsymbol{Q} = \boldsymbol{\Lambda}$. The matrix on the right is diagonal, so the recursion *decouples completely* into p independent scalar recursions:

$$\tilde{e}_{k+1,i} = (1 - \gamma\lambda_i)\,\tilde{e}_{k,i}, \qquad \text{so} \qquad \tilde{e}_{k,i} = (1 - \gamma\lambda_i)^k\,\tilde{e}_{0,i}. \tag{4.19}$$

Each eigendirection contracts by its own factor $|1 - \gamma\lambda_i|$ at every step, and this single equation contains everything.

Stability.

We can now state and prove the condition on the learning rate.

Theorem 4.1 (Stability of gradient descent on a quadratic). *Let $\boldsymbol{H}$ be symmetric positive definite with eigenvalues $0 < \lambda_{\min} \le \lambda_i \le \lambda_{\max}$. The gradient descent iteration (4.15) with fixed learning rate γ converges to $\boldsymbol{\theta}^*$ from every starting point $\boldsymbol{\theta}_0$ if and only if*

$$\boxed{0 < \gamma < \frac{2}{\lambda_{\max}}} \tag{4.20}$$

Equivalently, the spectral radius of the iteration matrix satisfies

$$\rho(\boldsymbol{I} - \gamma \boldsymbol{H}) = \max_i |1 - \gamma\lambda_i| < 1. \tag{4.21}$$

Proof. By Eq. (4.19) the i-th component of the error in the eigenbasis is $\tilde{e}_{k,i} = (1 - \gamma\lambda_i)^k \tilde{e}_{0,i}$, a geometric sequence with ratio $1 - \gamma\lambda_i$. It tends to zero for every $\tilde{e}_{0,i}$ precisely when $|1 - \gamma\lambda_i| < 1$. Since $\lambda_i > 0$ we may unpack the absolute value:

$$-1 < 1 - \gamma\lambda_i < 1 \quad\Longleftrightarrow\quad -2 < -\gamma\lambda_i < 0 \quad\Longleftrightarrow\quad 0 < \gamma < \frac{2}{\lambda_i}.$$

Because $\boldsymbol{Q}$ is orthogonal, $\|\boldsymbol{e}_k\| = \|\tilde{\boldsymbol{e}}_k\|$, so $\boldsymbol{e}_k \to \boldsymbol{0}$ for every $\boldsymbol{e}_0$ if and only if every component converges, that is

$$0 < \gamma < \min_i \frac{2}{\lambda_i} = \frac{2}{\lambda_{\max}},$$

since $2/\lambda_i$ is smallest for the largest eigenvalue. This is Eq. (4.20). For the converse, suppose $\gamma \ge 2/\lambda_{\max}$ and take $\boldsymbol{e}_0$ with a non-zero component along the eigenvector of $\lambda_{\max}$; that com-

ponent has $|1-\gamma\lambda_{\max}| \geq 1$ and does not decay, so the iteration fails to converge from this $\boldsymbol{\theta}_0$. Finally, the eigenvalues of $\boldsymbol{I}-\gamma\boldsymbol{H}$ are $1-\gamma\lambda_i$ (the eigenvectors are those of $\boldsymbol{H}$), so its spectral radius is $\max_i |1-\gamma\lambda_i|$, and the condition just derived is exactly Eq. (4.21).

The bound is set by the *steepest* direction, whatever the others are doing: the direction of greatest curvature is the one along which a gradient step changes the parameters most, and it is therefore the first to become unstable. Exceed the bound and the component along that direction grows geometrically; this is the divergence one observes when the learning rate is set too high.

At and beyond the bound.

It is worth reading off from Eq. (4.19) exactly what the stiffest mode does as γ increases. Its multiplier is $1-\gamma\lambda_{\max}$, so for $0<\gamma<1/\lambda_{\max}$ the mode decays monotonically without changing sign; for $1/\lambda_{\max}<\gamma<2/\lambda_{\max}$ the multiplier is negative and the mode alternates in sign at every step while its magnitude still decreases – this is the zig-zag across the valley in the centre panel of Figure 4.2; at $\gamma=2/\lambda_{\max}$ the multiplier is exactly -1 and the mode oscillates forever without decaying; and for $\gamma>2/\lambda_{\max}$ the multiplier has magnitude greater than one and the mode diverges. The strict inequality in Eq. (4.20) is therefore essential: at the boundary itself nothing blows up, but nothing converges either.

Decrease of the cost.

The same calculation shows that, inside the stable range, the cost itself decreases at every step and not only the error. For a quadratic with Hessian $\boldsymbol{H}$ the excess cost is $C(\boldsymbol{\theta})-C(\boldsymbol{\theta}^*)=\frac{1}{2}\boldsymbol{e}^T\boldsymbol{H}\boldsymbol{e}$, which in the eigenbasis reads

$$C(\boldsymbol{\theta}_k)-C(\boldsymbol{\theta}^*)=\frac{1}{2}\sum_{i=1}^{p}\lambda_i\,\tilde{e}_{k,i}^2,$$
$$C(\boldsymbol{\theta}_{k+1})-C(\boldsymbol{\theta}^*)=\frac{1}{2}\sum_{i=1}^{p}\lambda_i\,(1-\gamma\lambda_i)^2\,\tilde{e}_{k,i}^2. \tag{4.22}$$

Every non-zero term shrinks when $(1-\gamma\lambda_i)^2<1$, which is the condition of the theorem once more. Hence Eq. (4.20) guarantees a strict reduction of the cost at every iterate that is not already the minimum. (For least squares $\boldsymbol{H}=2\boldsymbol{X}^T\boldsymbol{X}/n$, and one checks directly that $C(\boldsymbol{\theta})-C(\boldsymbol{\theta}^*)=\|\boldsymbol{X}\boldsymbol{e}\|^2/n=\frac{1}{2}\boldsymbol{e}^T\boldsymbol{H}\boldsymbol{e}$.)

Beyond quadratics.

Nothing in the argument used that the cost was a quadratic *globally*. Near a minimum $\boldsymbol{\theta}^*$ of any twice differentiable cost, $\nabla C(\boldsymbol{\theta}^*)=\boldsymbol{0}$ and Taylor expansion gives $\nabla C(\boldsymbol{\theta})=\boldsymbol{H}^*\boldsymbol{e}+\mathcal{O}(\|\boldsymbol{e}\|^2)$ with $\boldsymbol{H}^*=\nabla^2 C(\boldsymbol{\theta}^*)$, so the recursion (4.18) describes the local dynamics and the theorem gives the local stability condition, with $\lambda_{\max}$ the largest eigenvalue of the Hessian *at the minimum*. A global statement needs a global bound on the curvature. Suppose the gradient is Lipschitz continuous with constant L,

$$\big\|\nabla C(\boldsymbol{\theta})-\nabla C(\boldsymbol{\theta}')\big\|\leq L\big\|\boldsymbol{\theta}-\boldsymbol{\theta}'\big\| \qquad \text{for all } \boldsymbol{\theta},\boldsymbol{\theta}', \tag{4.23}$$

which for a twice differentiable cost means that all Hessian eigenvalues lie in $[-L, L]$ everywhere. Then the *descent lemma* holds: for any $\boldsymbol{\theta}$ and any step $\boldsymbol{d}$,

$$C(\boldsymbol{\theta}+\boldsymbol{d}) \le C(\boldsymbol{\theta}) + \nabla C(\boldsymbol{\theta})^T \boldsymbol{d} + \frac{L}{2}\|\boldsymbol{d}\|^2. \tag{4.24}$$

To see this, write the change in the cost as an integral along the segment from $\boldsymbol{\theta}$ to $\boldsymbol{\theta}+\boldsymbol{d}$ and subtract and add $\nabla C(\boldsymbol{\theta})^T \boldsymbol{d}$ under the integral sign,

$$\begin{aligned} C(\boldsymbol{\theta}+\boldsymbol{d}) - C(\boldsymbol{\theta}) &= \int_0^1 \nabla C(\boldsymbol{\theta}+t\boldsymbol{d})^T \boldsymbol{d}\, dt \\ &= \nabla C(\boldsymbol{\theta})^T \boldsymbol{d} + \int_0^1 [\nabla C(\boldsymbol{\theta}+t\boldsymbol{d}) - \nabla C(\boldsymbol{\theta})]^T \boldsymbol{d}\, dt; \end{aligned}$$

the Cauchy–Schwarz inequality and Eq. (4.23) bound the integrand by $Lt\|\boldsymbol{d}\|^2$, and $\int_0^1 Lt\, dt = L/2$. Taking the gradient step $\boldsymbol{d} = -\gamma \boldsymbol{g}_k$ with $\boldsymbol{g}_k = \nabla C(\boldsymbol{\theta}_k)$,

$$\begin{aligned} C(\boldsymbol{\theta}_{k+1}) &\le C(\boldsymbol{\theta}_k) - \gamma\|\boldsymbol{g}_k\|^2 + \frac{L\gamma^2}{2}\|\boldsymbol{g}_k\|^2 \\ &= C(\boldsymbol{\theta}_k) - \gamma\left(1 - \frac{L\gamma}{2}\right)\|\boldsymbol{g}_k\|^2, \end{aligned} \tag{4.25}$$

so the cost decreases strictly, wherever the gradient is non-zero, whenever

$$0 < \gamma < \frac{2}{L}.$$

This is the "sufficiently small step" promised in Section 4.3, now made quantitative. For a quadratic the Lipschitz constant of the gradient is $L = \lambda_{\max}(\boldsymbol{H})$, and the descent lemma reproduces the stability bound (4.20) exactly.

Qualifications.

Four remarks delimit what has been proved. Positive definiteness matters: if $\boldsymbol{H}$ has a negative eigenvalue, the corresponding multiplier $1 - \gamma\lambda_i$ exceeds one for every $\gamma > 0$ and no fixed positive learning rate contracts that direction – gradient descent moves away from a saddle point or a maximum, which is what one wants, but the analysis above no longer describes convergence. If $\boldsymbol{H}$ has zero eigenvalues, the multiplier along the corresponding flat directions is exactly one and the quadratic model predicts no contraction at all; for least squares this is the rank-deficient case of Section 1.22, and the remedy is the Ridge penalty discussed below. For a non-quadratic cost the eigenvalues vary with $\boldsymbol{\theta}$: the local bound uses the Hessian at the minimum, whereas a global guarantee needs a Lipschitz constant L valid over the whole region visited by the iterates, which is typically much larger and makes the permitted step correspondingly smaller. And stochastic gradients, the subject of Section 4.7, add noise to every step, so the deterministic bound is a guide to stability rather than a complete convergence theory.

Speed.

Inside the stable range the question becomes how fast the error contracts, and the answer is set by the two extreme eigenvalues. The worst-case contraction factor per step is

$$r(\gamma) = \max_i |1 - \gamma\lambda_i| = \max\{|1 - \gamma\lambda_{\min}|,\ |1 - \gamma\lambda_{\max}|\},$$

because $|1-\gamma\lambda|$ is largest at one end or the other of the spectrum. For $\gamma \le 1/\lambda_{\max}$ the first term dominates and r decreases with γ; for larger γ the second term $\gamma\lambda_{\max} - 1$ grows. The minimum lies where the two are equal, $1 - \gamma\lambda_{\min} = \gamma\lambda_{\max} - 1$, which gives the optimal learning rate $\gamma^* = 2/(\lambda_{\max} + \lambda_{\min})$ and the contraction rate

$$\left|\frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}}\right| = \frac{\kappa - 1}{\kappa + 1}, \qquad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}} = \kappa_2(\boldsymbol{H}), \tag{4.26}$$

where κ is the condition number of the Hessian in the sense of Eq. (1.77). Since $(\kappa-1)/(\kappa+1) \approx 1 - 2/\kappa$ for large κ, the number of iterations needed to reduce the error by a fixed factor is proportional to κ: a large condition number produces slow convergence even when the learning rate is perfectly stable and optimally chosen.

Figure 4.1 confirms both statements numerically. The iteration count falls as γ increases, reaches its minimum at the predicted $\gamma^* = 2/(\lambda_{\max} + \lambda_{\min})$, rises again, and diverges exactly at $2/\lambda_{\max}$. There is no margin of safety at the right-hand edge: the transition from the fastest convergence available to outright divergence occupies a factor of two in γ.

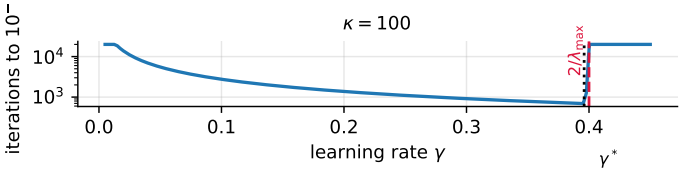


Fig. 4.1 Iterations required to reach an error of 10^{-6} against the learning rate, for a quadratic with eigenvalues $\{0.05, 1, 5\}$. The optimum lies at γ^* and the method diverges beyond $2/\lambda_{\max}$, as predicted by Eqs. (4.20) and (4.26).

This is the central result of the chapter, and it deserves to be read slowly. Gradient descent is not slow because gradients are a bad idea. It is slow because it applies *the same* γ to every eigendirection, while stability forces that single γ to be dictated by the steepest direction. In a problem where $\lambda_{\max}/\lambda_{\min} = 1000$, the step that is barely stable along the steep direction is a thousand times too small along the flat one, and the flat direction is where the remaining error lives. The iterates oscillate across the narrow valley while creeping along its floor.

What this means for regression.

For least squares $\boldsymbol{H} = 2\boldsymbol{X}^T\boldsymbol{X}/n$, so by Eq. (1.118)

$$\kappa(\boldsymbol{H}) = \kappa_2\left(\boldsymbol{X}^T\boldsymbol{X}\right) = \kappa_2(\boldsymbol{X})^2, \tag{4.27}$$

and the iteration count scales with the *square* of the condition number of the design matrix. Three consequences follow, each connecting to earlier chapters.

First, standardising the features is not only the statistical convention of Section 3.13 but a direct accelerator: it makes the columns of $\boldsymbol{X}$ comparable in scale, reduces κ, and thereby reduces the number of iterations. This is the precise sense in which "transform your inputs" is good advice.

Second, Ridge regression improves conditioning as well as variance. By Eq. (4.17) the Hessian becomes $2(\boldsymbol{X}^T\boldsymbol{X}/n + \lambda\boldsymbol{I})$, whose condition number is $(\lambda_{\max} + \lambda)/(\lambda_{\min} + \lambda)$ – smaller

than κ for every $\lambda > 0$. Regularisation makes the optimisation easier as well as the estimator better behaved, exactly as anticipated in the notebox of Section 1.12.

Third, the conjugate gradient method of Section 1.16 achieves a rate governed by $\sqrt{\kappa}$ rather than κ, and for a quadratic cost it is strictly superior to gradient descent. It is the right tool for linear regression at scale. Its disadvantage is that it relies on the cost being quadratic and on exact gradients, and it copes badly with the stochastic, non-convex problems of the following chapters – which is why the machine learning literature developed a different set of remedies, to which we now turn.

Figure 4.2 shows what the algebra describes. On an elongated quadratic with $\kappa = 12$, a small learning rate creeps along the valley floor without ever crossing it; a learning rate near the stability bound (4.20) oscillates across the valley while making slow progress along it; and momentum, at the same learning rate, damps the oscillation and accelerates the drift, arriving in a fraction of the steps. The three panels are the whole of this section and the next in pictures.

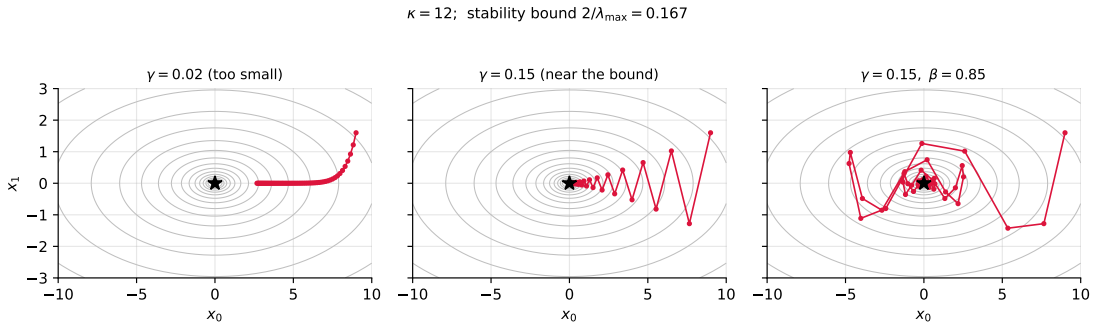


Fig. 4.2 Gradient descent on a quadratic with $\kappa = 12$. Left: too small a step. Centre: a step near the stability bound $2/\lambda_{\max}$, oscillating across the narrow direction. Right: the same step with momentum $\beta = 0.85$, which cancels the oscillation and accumulates along the flat direction.

4.6 Momentum

The first remedy is to give the iteration a memory of where it has been. Momentum-based gradient descent replaces Eq. (4.10) by

$$\boldsymbol{v}_{k+1} = \beta \boldsymbol{v}_k + \gamma \nabla C(\boldsymbol{\theta}_k), \qquad \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \boldsymbol{v}_{k+1}, \tag{4.28}$$

with $\beta \in [0, 1)$ the *momentum parameter*, typically 0.9 (it plays the same role as, and is deliberately given the same letter as, the β_1 of the Adam method in Section 4.11). The update is no longer the current gradient but an exponentially weighted sum of all gradients seen so far,

$$\boldsymbol{v}_{k+1} = \gamma \sum_{j=0}^{k} \beta^{k-j} \nabla C(\boldsymbol{\theta}_j), \tag{4.29}$$

with older gradients discounted geometrically. Setting $\beta = 0$ recovers plain gradient descent.

Where the name comes from.

The word *momentum* is not a metaphor. Consider a particle of mass m moving in the potential $C(\boldsymbol{\theta})$ – the cost function read as a landscape – and subject to a friction force proportional to its velocity, with friction coefficient μ. Newton's second law reads

$$m\,\frac{d^2\boldsymbol{\theta}}{dt^2} + \mu\,\frac{d\boldsymbol{\theta}}{dt} = -\nabla C(\boldsymbol{\theta}). \tag{4.30}$$

Without friction the particle would roll down the slope, overshoot the bottom and oscillate forever; with friction it loses energy on every swing and settles at a minimum. Now discretise Eq. (4.30) with a time step Δt, writing $\boldsymbol{\theta}_k = \boldsymbol{\theta}(k\Delta t)$ and replacing the derivatives by finite differences,

$$\frac{d^2\boldsymbol{\theta}}{dt^2} \approx \frac{\boldsymbol{\theta}_{k+1} - 2\boldsymbol{\theta}_k + \boldsymbol{\theta}_{k-1}}{(\Delta t)^2}, \qquad \frac{d\boldsymbol{\theta}}{dt} \approx \frac{\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k}{\Delta t}.$$

Multiplying through by $(\Delta t)^2$ and collecting the terms in $\boldsymbol{\theta}_{k+1}$ on the left,

$$(m + \mu\Delta t)\,(\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k) = m\,(\boldsymbol{\theta}_k - \boldsymbol{\theta}_{k-1}) - (\Delta t)^2\,\nabla C(\boldsymbol{\theta}_k),$$

so that the position update is

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \underbrace{\frac{m}{m+\mu\Delta t}}_{\beta}\,(\boldsymbol{\theta}_k - \boldsymbol{\theta}_{k-1}) - \underbrace{\frac{(\Delta t)^2}{m+\mu\Delta t}}_{\gamma}\,\nabla C(\boldsymbol{\theta}_k). \tag{4.31}$$

With $\boldsymbol{v}_{k+1} = \boldsymbol{\theta}_k - \boldsymbol{\theta}_{k+1}$ this is exactly Eq. (4.28): the new displacement is β times the old one plus γ times the gradient. The momentum parameter is the fraction of its velocity the particle keeps from one step to the next, and the learning rate is the squared time step divided by the effective mass. (Many texts and the slides of this course write the same iteration as $\boldsymbol{v}_k = \beta(\boldsymbol{\theta}_k - \boldsymbol{\theta}_{k-1}) - \gamma\nabla C(\boldsymbol{\theta}_k)$, $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \boldsymbol{v}_k$; that is Eq. (4.28) with the sign of $\boldsymbol{v}$ reversed, and nothing else changes.)

The two limits of Eq. (4.31) are instructive. When friction dominates, $\mu\Delta t \gg m$, we have $\beta \to 0$ and $\gamma \to \Delta t/\mu$: the particle has no inertia, its velocity is proportional to the force, and the iteration is plain gradient descent – a massless bead sliding through treacle. When friction is negligible, $\mu\Delta t \ll m$, we have $\beta \to 1$ and $\gamma \to (\Delta t)^2/m$: energy is conserved, the particle never settles, and the iteration rings across the valley indefinitely. This is the frictionless "ball" of the example below, where $\beta = 0.9$ overshoots the global minimum. Between the two, $1-\beta = \mu\Delta t/(m+\mu\Delta t)$ measures how much energy is dissipated per step. The gain $1/(1-\beta)$ that we shall meet in Eq. (4.32) below is the terminal velocity of the particle: under a constant force $\boldsymbol{g}$ the steady displacement per step is $\gamma\boldsymbol{g}/(1-\beta) = (\Delta t/\mu)\,\boldsymbol{g}$, that is, the velocity $\boldsymbol{g}/\mu$ at which friction exactly balances the force, independent of the mass. What a heavier ball buys is not a higher terminal velocity but a longer memory: it takes about $1/(1-\beta)$ steps to forget a gradient, which is what averages out the sign-alternating gradients of a steep direction.

The method was introduced under the name *heavy-ball method* by Polyak [12], who also derived the rate quoted below; the reading as a discretised equation of motion, and the limits above, follow Qian [13].

Why it works.

Return to the decoupled recursion (4.19). Along an eigendirection with eigenvalue λ_i, the gradient is $\lambda_i \tilde{e}_i$ and the momentum iteration becomes a two-term recurrence in $\tilde{e}$ rather than a one-term one. Its behaviour splits into two regimes, and the split is exactly what we want.

In a *flat* direction, $\gamma\lambda_i \ll 1$, successive gradients point the same way and Eq. (4.29) adds them up. A constant gradient g accumulates to

$$v_\infty = \gamma g \sum_{j=0}^{\infty} \beta^j = \frac{\gamma g}{1-\beta}, \tag{4.32}$$

so the effective step is multiplied by $1/(1-\beta)$ – a factor of ten for $\beta = 0.9$. The method accelerates precisely where plain gradient descent crawls.

In a *steep* direction, the iterate overshoots and the gradient reverses sign at every step. Successive terms in Eq. (4.29) then alternate in sign and largely cancel, so the accumulated step is *smaller* than the bare gradient. The oscillation across the narrow valley is damped.

Momentum thus amplifies the consistent and cancels the oscillatory, which is the right treatment for the ill-conditioned landscape diagnosed in Section 4.5. Polyak's analysis of the two-term recurrence [12] shows that with optimally chosen γ and β the contraction rate improves from Eq. (4.26) to

$$\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}, \tag{4.33}$$

so the iteration count scales with $\sqrt{\kappa}$ instead of κ. For $\kappa = 10^4$ that is a hundredfold reduction. The reader will notice that $\sqrt{\kappa}$ is also the rate of the conjugate gradient method quoted in Section 1.16; this is not a coincidence, since both build their step out of the current gradient and the previous direction. Momentum is, in effect, conjugate gradients with the optimal coefficients replaced by a fixed constant that need not be recomputed and does not require the cost to be quadratic.

A minimal example.

The behaviour is easiest to see on $f(x) = x^2$, where the exact answer is known.

Figure 4.3 shows what the improvement from κ to $\sqrt{\kappa}$ is worth. At $\kappa = 10^2$ momentum saves an order of magnitude in iterations; at $\kappa = 10^4$, two. Since Eq. (4.27) tells us that the least-squares Hessian has the condition number of the design matrix *squared*, condition numbers in this range are entirely ordinary, and the saving is not academic.

```
import numpy as np

def objective(x):
    return x**2.0

def derivative(x):
    return 2.0 * x

def gradient_descent(derivative, bounds, n_iter, step_size, momentum=0.0,
                     rng=None):
    """Gradient descent with optional momentum, Eq. (4.momentum)."""
    rng = np.random.default_rng() if rng is None else rng
    solution = bounds[:, 0] + rng.random(len(bounds)) * (bounds[:, 1] - bounds[:, 0])
    change = 0.0
    solutions, scores = [], []
    for i in range(n_iter):
        gradient = derivative(solution)
```

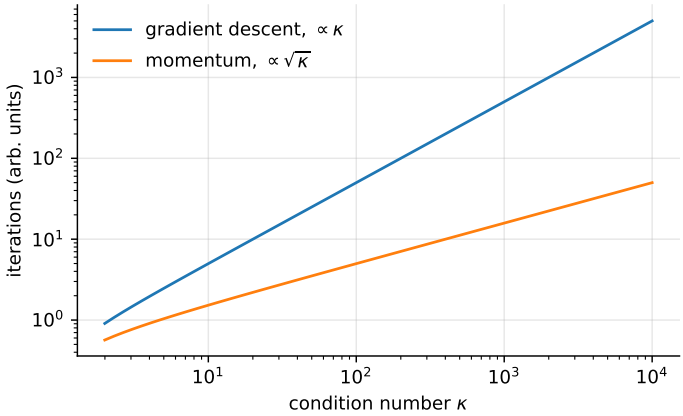


Fig. 4.3 Iterations to fixed accuracy against the condition number, for gradient descent and for momentum with optimal parameters, from the rates (4.26) and (4.33).

```
        new_change = step_size * gradient + momentum * change
        solution = solution - new_change
        change = new_change
        solutions.append(solution.copy())
        scores.append(objective(solution))
    return solutions, scores


bounds = np.asarray([[-1.0, 1.0]])
rng = np.random.default_rng(4)
plain = gradient_descent(derivative, bounds, 30, 0.1, momentum=0.0,
                         rng=np.random.default_rng(4))
withmom = gradient_descent(derivative, bounds, 30, 0.1, momentum=0.3,
                           rng=np.random.default_rng(4))
print(f"after 30 steps: plain f = {plain[1][-1][0]:.3e}, "
      f"momentum f = {withmom[1][-1][0]:.3e}")
```

For this one-dimensional problem $\kappa = 1$ and there is nothing to fix, yet momentum still helps by increasing the effective step through Eq. (4.32). The dramatic gains appear when the eigenvalues differ, which is the case for every realistic problem.

Following the steps.

It is worth watching the two iterations step by step on a function simple enough to be drawn. The function `descend` below is the whole of Eq. (4.28), with the gradient passed in as an argument – the same structure we keep throughout the chapter, so that the analytical gradient of Section 4.4 and the automatic one of Section 4.14 are interchangeable.

```
import numpy as np

def quartic(x):
    """A low-order polynomial with two minima: f(x) = x^4 - 3x^2 + x."""
    return x**4 - 3.0 * x**2 + x

def quartic_grad(x):
    return 4.0 * x**3 - 6.0 * x + 1.0

def descend(grad, x0, gamma, beta=0.0, num_iters=40):
    """Gradient descent with momentum, Eq. (4.momentum); beta = 0 is plain gradient descent.
```

```
    Works for a scalar x as well as for a vector theta.  Returns every iterate."""
    x = np.array(x0, dtype=float)
    v = np.zeros_like(x)
    history = [x.copy()]
    for _ in range(num_iters):
        v = beta * v + gamma * grad(x)
        x = x - v
        history.append(x.copy())
    return np.array(history)

for beta in (0.0, 0.7):
    path = descend(quartic_grad, 2.0, gamma=0.05, beta=beta)
    print(f"beta = {beta}: x = {np.round(path[:6], 3)} ... ends at {path[-1]:.4f}, f =
     {quartic(path[-1]):.4f}")
```

The quartic $f(x) = x^4 - 3x^2 + x$ has a global minimum at $x = -1.30$ ($f = -3.51$), a local one at $x = 1.13$ ($f = -1.07$), and a hump at $x = 0.17$ between them. Started at $x_0 = 2$ with $\gamma = 0.05$, plain gradient descent walks down into the *nearest* minimum, $2 \to 0.95 \to 1.01 \to 1.06 \to \cdots \to 1.131$, and stays there, exactly as Section 4.3 warned. With $\beta = 0.7$ the velocity built up on the steep slope carries the iterate over the hump, $2 \to 0.95 \to 0.28 \to -0.16 \to -0.57 \to \cdots \to -1.302$, into the global minimum: the same code, one extra line, a different answer. This should not be over-read. Momentum knows nothing of the landscape beyond the gradients it has seen; it crossed the hump because the accumulated velocity happened to exceed the barrier, and with $\beta = 0.9$ the "ball" is so frictionless that it overshoots the global minimum, climbs back over the hump and ends in the local basin. Once the cost is not convex, none of the methods of this chapter comes with a guarantee, and Section 4.1 is the last time one was available.

The right panel of Fig. 4.4 repeats the exercise on the bowl $C(\boldsymbol{\theta}) = \frac{1}{2}(\theta_1^2 + 10\,\theta_2^2)$, whose Hessian is $\mathrm{diag}(1, 10)$, so that $\kappa = 10$ and $2/\lambda_{\max} = 0.2$. From $(2, 1.4)$, forty steps at $\gamma = 0.04$ creep along the valley floor (356 iterations to reach 10^{-6}); forty steps at $\gamma = 0.18$, just below the bound, zigzag across the steep direction while advancing slowly along the flat one (74 iterations); momentum with $\beta = 0.3$ at the same γ cancels the zigzag and arrives in 39, and the tuned pair $\gamma = 4/(\sqrt{\lambda_{\max}} + \sqrt{\lambda_{\min}})^2 = 0.231$, $\beta = \big((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)\big)^2 = 0.27$ that attains the rate (4.33) needs 28. Note that the best β is tied to κ: the default 0.9 is right for $\kappa \approx 400$ and too much for $\kappa = 10$, where it makes the iterate ring for longer than plain descent takes to converge.

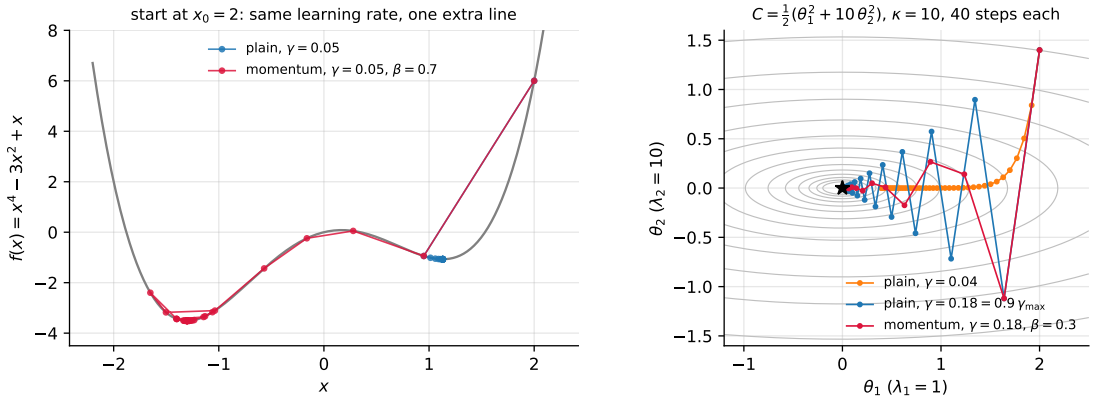


Fig. 4.4 Following the steps of gradient descent with and without momentum. Left: the quartic $f(x) = x^4 - 3x^2 + x$ from $x_0 = 2$ with $\gamma = 0.05$; plain descent stops in the nearer, shallower minimum, momentum with $\beta = 0.7$ carries the iterate over the hump into the global one. Right: forty steps on the bowl $\frac{1}{2}(\theta_1^2 + 10\theta_2^2)$ with $\kappa = 10$, at a small learning rate, at 0.9 of the stability bound (4.20), and with momentum $\beta = 0.3$ at the same learning rate.

Machine learning connection. A widely used variant is *Nesterov* momentum [14, 15], which evaluates the gradient not at $\boldsymbol{\theta}_k$ but at the point the momentum term is about to carry it to, $\boldsymbol{\theta}_k - \beta \boldsymbol{v}_k$. The intuition is that of looking ahead before stepping: if the accumulated velocity is about to overshoot, the gradient at the extrapolated point already points backwards and corrects the step early. For convex problems this improves the constant in Eq. (4.33), and in deep learning frameworks it is usually available as a flag on the standard momentum optimiser.

4.7 Stochastic gradient descent

The second remedy attacks a different problem: cost. Momentum makes each step of gradient descent more effective; stochastic gradient descent makes each step *cheaper*, by a factor that for the data sets of machine learning runs into the thousands, and accepts a noisy direction as the price. This section makes both halves of that bargain quantitative. We count what an update costs, show that the noisy direction is right on average and by how much it is wrong, work out how far one noisy step can be expected to take us, and then ask the question on which everything turns: how accurate a minimum is worth paying for. The claim that "SGD is faster" is meaningless until an accuracy target and a measure of cost have been named, and most of the confusion about the method comes from leaving one of them out.

4.7.1 The finite-sum problem and the minibatch gradient

Almost every cost function in machine learning is a sum over data points,

$$C(\boldsymbol{\theta}) = \sum_{i=1}^{n} c_i(\boldsymbol{x}_i, \boldsymbol{\theta}), \qquad \nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}) = \sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}} c_i(\boldsymbol{x}_i, \boldsymbol{\theta}). \tag{4.34}$$

Computing the full gradient therefore costs one pass over the entire data set for a single parameter update. In large-scale applications – the ImageNet challenge has millions of training images – this is extravagant: we spend an enormous computation to obtain one direction, most of whose precision is wasted, since we are going to take another step immediately afterwards.

It is convenient, and from here on we do so, to divide the sum by n and work with the *mean*,

$$C(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} c_i(\boldsymbol{\theta}), \qquad \nabla C(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \nabla c_i(\boldsymbol{\theta}), \tag{4.35}$$

as we did for the mean squared error in Eq. (4.12). The minimiser is the same, but the curvature, and with it the stable learning rate of Eq. (4.20), no longer grows with the size of the data set, and the gradient becomes an *average* – which is what makes it possible to estimate it from a sample.

Stochasticity is introduced by evaluating the gradient on a random subset of the data, called a *minibatch*. With n data points and minibatches of size M there are n/M minibatches, denoted B_k with $k = 1, \ldots, n/M$. If $n = 10$ and we choose five minibatches, each contains two points: $B_1 = (\boldsymbol{x}_1, \boldsymbol{x}_2)$ through $B_5 = (\boldsymbol{x}_9, \boldsymbol{x}_{10})$. Taking $M = n$ gives a single batch containing everything, which is ordinary gradient descent; taking $M = 1$ gives one point per batch. We approximate

$$\nabla C(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \nabla c_i(\boldsymbol{\theta}) \;\longrightarrow\; \boldsymbol{g}_B(\boldsymbol{\theta}) = \frac{1}{M}\sum_{i\in B} \nabla c_i(\boldsymbol{\theta}), \tag{4.36}$$

and the update becomes

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma_t\, \boldsymbol{g}_{B_t}(\boldsymbol{\theta}_t) = \boldsymbol{\theta}_t - \frac{\gamma_t}{M}\sum_{i\in B_t} \nabla c_i(\boldsymbol{\theta}_t), \tag{4.37}$$

with a fresh minibatch B_t for every update. One pass through all the minibatches is called an *epoch*, and one typically chooses a number of epochs and iterates over the minibatches within each. Dividing by M rather than summing matters in practice: with the mean, the size of the step does not change when the batch size does, and γ keeps the meaning it had for gradient descent. If a library hands us summed gradients, the learning rate must be divided by M instead.

Strictly, *stochastic gradient descent* means using a single example at a time, and the minibatch version should be called minibatch gradient descent; but in practice everyone says SGD and means minibatches, and we shall do the same. The minibatch size is a hyperparameter, but it is seldom tuned by cross-validation: it is usually set by memory constraints, or to a power of two such as 32, 64 or 128, because vectorised operations are fastest at those sizes; Section 4.7.9 returns to this choice with measurements.

Two benefits, and their price.

Taking the gradient on a subset has two distinct advantages. It is *cheaper*: if $M \ll n$ the gradient costs a fraction of the full one, so we take many more steps for the same computation. And it introduces *noise*, which decreases the chance of the scheme becoming stuck in a poor local minimum – the deterministic objection raised in Section 4.3. A stochastic iterate can be jostled out of a shallow basin in a way a deterministic one cannot.

The corresponding disadvantages are that convergence is erratic rather than monotone, that the iterate does not settle at the minimum but rattles around in a neighbourhood whose size is set by the gradient noise and the learning rate, and that the learning rate must be chosen with more care. The memory picture, on the other hand, is entirely in favour of minibatches: full-batch methods must hold the whole data set, or at least stream it, for every update, whereas SGD needs only the current minibatch. This is what makes training on data sets larger than memory possible at all, and SGD can even begin before the data set is complete, which is the natural setting for streaming data.

4.7.2 What an update costs

The claim that SGD is ahead per unit of computation can be made exact for the least-squares cost, whose gradient (4.13) is two matrix–vector products: $\boldsymbol{X}\boldsymbol{\theta}$ costs $2np$ floating-point operations (one multiplication and one addition per matrix element), subtracting $\boldsymbol{y}$ costs n, and $\boldsymbol{X}^T\boldsymbol{r}$ another $2np$. The minibatch gradient $\boldsymbol{g}_B = \frac{2}{M}\boldsymbol{X}_B^T(\boldsymbol{X}_B\boldsymbol{\theta} - \boldsymbol{y}_B)$ is the same formula on the M rows $\boldsymbol{X}_B \in \mathbb{R}^{M\times p}$ of the batch, so

$$\text{one full gradient} \approx 4np \text{ flops}, \qquad \text{one minibatch gradient} \approx 4Mp \text{ flops}, \tag{4.38}$$

and an epoch of SGD – n/M updates – costs $\frac{n}{M}\cdot 4Mp = 4np$, the same as a *single* step of gradient descent. This is the comparison to keep in mind: one epoch of minibatch SGD and one update of gradient descent cost the same arithmetic, but during that epoch SGD has changed the

parameters n/M times. Those extra updates help most early in the optimisation, when any reasonable direction makes progress; near the minimum the gradient noise limits what they are worth, as we shall see.

Table 4.1 What one update and one epoch cost for $n = 10^6$ observations, $p = 100$ parameters and minibatches of $M = 64$, by the count of Eq. (4.38). If gradient descent needs 30 passes over the data to reach a required validation error and SGD needs 4, the totals are 1.2×10^{10} against 1.6×10^9 operations, a factor of 7.5; the numbers of passes must be measured for the problem at hand, and Section 4.7.7 does so.

	full gradient	**minibatch SGD**
examples read per update	10^6	64
operations per update	4×10^8	2.56×10^4
updates per epoch	1	15 625
operations per epoch	4×10^8	4×10^8

Table 4.1 puts numbers on this for a data set of a million points. The arithmetic understates the advantage in wall-clock time, because a minibatch of 64 rows stays in the fast cache memory of the processor, while every full gradient drags the entire design matrix through it; and it overstates it for very small batches, for which the overhead of launching an operation exceeds the operation itself. Both effects are measured in Section 4.7.9.

For comparison, solving the normal equations directly costs about $np^2 + p^3/3$ flops (forming $\boldsymbol{X}^T\boldsymbol{X}$, then a Cholesky factorisation, Section 1.14), which is $p/4$ gradient evaluations: for a linear model with modest p the closed form is the cheapest option of all, and the comparison in this section is between the two *iterative* methods, which are what remains once the model is a network and no closed form exists.

4.7.3 A worked example: the running mean

The simplest finite-sum problem shows the whole idea in one line. Let

$$c_i(\theta) = \frac{1}{2}(\theta - a_i)^2, \qquad C(\theta) = \frac{1}{n}\sum_{i=1}^{n} c_i(\theta), \qquad C'(\theta) = \theta - \bar{a}, \tag{4.39}$$

whose minimiser is the mean $\hat{\theta} = \bar{a}$ of the data. Gradient descent with $\gamma = 1$ finds it in a single update, $\theta_1 = \theta_0 - (\theta_0 - \bar{a}) = \bar{a}$ – but computing that one gradient reads all n values. SGD with $M = 1$ draws an index i_t and takes the step

$$\theta_{t+1} = \theta_t - \gamma_t(\theta_t - a_{i_t}) = (1 - \gamma_t)\,\theta_t + \gamma_t\, a_{i_t}, \tag{4.40}$$

and with the learning rate $\gamma_t = 1/(t+1)$ this is the familiar recursion for a running average: θ_t is exactly the mean of the t values seen so far. By Section 2.5 it therefore satisfies

$$\mathbb{E}[\theta_t] = \bar{a}, \qquad \operatorname{Var}[\theta_t] \approx \frac{s_a^2}{t} \qquad (t \ll n),$$

with s_a the standard deviation of the data. SGD is here nothing but the observation that the mean of a million numbers is known to three digits after looking at a small fraction of them:

```
import numpy as np
```

```
rng = np.random.default_rng(2026)
a = 2.0 + 3.0 * rng.normal(size=1_000_000)        # the data: n = 10^6 numbers
theta = 0.0
for t, i in enumerate(rng.integers(0, len(a), 10_000)):
    gamma = 1.0 / (t + 1)
    theta -= gamma * (theta - a[i])               # SGD step on c_i = (theta - a_i)^2 / 2
    if t + 1 in (100, 1000, 10_000):
        print(f"after {t + 1:6d} samples: theta = {theta:.4f}, error {theta -
    a.mean():+.4f}, "
              f"s_a/sqrt(t) = {a.std() / np.sqrt(t + 1):.4f}")
print(f"full gradient descent, one step with gamma = 1: theta = {a.mean():.4f}, "
      f"after reading all {len(a)} values")
```

```
after    100 samples: theta = 1.6404, error -0.3591, s_a/sqrt(t) = 0.3000
after   1000 samples: theta = 2.0473, error +0.0478, s_a/sqrt(t) = 0.0949
after  10000 samples: theta = 1.9868, error -0.0127, s_a/sqrt(t) = 0.0300
full gradient descent, one step with gamma = 1: theta = 1.9995, after reading all 1000000
    values
```

After one percent of the data the error is 0.013, well inside the $s_a/\sqrt{t} = 0.030$ that the central limit theorem allows; over 2000 repetitions with $t = 1000$ the errors have mean 0.005 and standard deviation 0.094, against the predicted 0.095. Three features of this toy problem carry over to everything that follows. The stochastic method delivers a *useful* answer long before the deterministic one has delivered anything. Its error falls as $1/\sqrt{t}$ in the number of samples and no faster, so that it is a poor way to obtain ten digits. And it needed a *decaying* learning rate: with a constant γ the recursion (4.40) forgets old samples at the rate it absorbs new ones, and θ_t fluctuates forever with variance $\gamma s_a^2/(2-\gamma)$.

4.7.4 The minibatch gradient as an estimator

Let us make the statement about noise precise. Write $\boldsymbol{g}_i = \nabla c_i(\boldsymbol{\theta})$ for the per-example gradients and $\boldsymbol{g} = \nabla C = \frac{1}{n}\sum_i \boldsymbol{g}_i$ for their mean, and draw the indices $I_1, \dots, I_M$ of a minibatch independently and uniformly from $\{1, \dots, n\}$. Conditioned on the current iterate $\boldsymbol{\theta}$ – the only thing the batch is allowed to depend on –

$$\mathbb{E}\big[\boldsymbol{g}_B \mid \boldsymbol{\theta}\big] = \frac{1}{M}\sum_{m=1}^{M} \mathbb{E}\big[\boldsymbol{g}_{I_m} \mid \boldsymbol{\theta}\big] = \frac{1}{M}\sum_{m=1}^{M} \frac{1}{n}\sum_{i=1}^{n} \boldsymbol{g}_i = \nabla C(\boldsymbol{\theta}), \tag{4.41}$$

since each index takes each value with probability $1/n$. The minibatch gradient is an *unbiased* estimate of the full gradient, whatever M. The conditioning is not a technicality: the argument requires the batch to be chosen without reference to the gradients it contains. Selecting the examples with the largest loss, or reusing the batch on which the previous step did badly, gives a biased direction – sometimes a useful one, but no longer covered by what follows.

For the variance, let $\boldsymbol{\xi}_i = \boldsymbol{g}_i - \boldsymbol{g}$ be the deviation of example i from the mean, so that $\mathbb{E}[\boldsymbol{\xi}_I \mid \boldsymbol{\theta}] = 0$. The M draws are independent, the cross terms vanish, and

$$\mathbb{E}\Big[\big\|\boldsymbol{g}_B - \nabla C\big\|^2 \,\Big|\, \boldsymbol{\theta}\Big] = \frac{1}{M}\,\mathbb{E}\big[\|\boldsymbol{\xi}_I\|^2 \mid \boldsymbol{\theta}\big] = \frac{\sigma_g^2(\boldsymbol{\theta})}{M}, \qquad \sigma_g^2 = \frac{1}{n}\sum_{i=1}^{n} \|\boldsymbol{g}_i - \boldsymbol{g}\|^2. \tag{4.42}$$

If the batch is instead drawn *without* replacement, as it is when we shuffle the data and cut the permutation into batches, the elementary theory of sampling from a finite population gives, for component j,

$$\mathbb{E}[\boldsymbol{g}_B] = \boldsymbol{g}, \qquad \mathrm{Var}[g_{B,j}] = \frac{s_j^2}{M}\,\frac{n-M}{n-1}, \qquad s_j^2 = \frac{1}{n-1}\sum_{i=1}^{n}\big(g_{i,j}-g_j\big)^2. \tag{4.43}$$

The standard deviation of the estimate falls as $1/\sqrt{M}$ – not as $1/M$ – times the finite-population factor $\sqrt{(n-M)/(n-1)}$, which is one for $M \ll n$ and vanishes at $M = n$, where the estimate is the full gradient. Quadrupling the batch size halves the gradient noise at four times the cost per step; that unfavourable exchange rate is why very large batches are not automatically better.

Two things follow. The direction of an SGD step is right on average and wrong in detail, so that an individual update need not decrease C at all. The amount by which it is wrong depends on $\boldsymbol{\theta}$ through the spread of the per-example gradients – large far from the minimum, and *not zero at the minimum*, since there the per-example gradients cancel in the sum without vanishing individually. And the noise does not disappear with more iterations, only with a larger M or a smaller step; the consequences are worked out in Section 4.7.8.

4.7.5 Expected progress in one update, and the rates it implies

How far does one noisy step take us? Assume, as in Section 4.5, that the gradient is Lipschitz continuous with constant L, so that the descent lemma (4.24) holds, and insert the step $\boldsymbol{d} = -\gamma \boldsymbol{g}_B$:

$$C(\boldsymbol{\theta}_{t+1}) \le C(\boldsymbol{\theta}_t) - \gamma \nabla C(\boldsymbol{\theta}_t)^T \boldsymbol{g}_B + \frac{L\gamma^2}{2}\,\|\boldsymbol{g}_B\|^2.$$

Now take the expectation over the batch, given $\boldsymbol{\theta}_t$. In the linear term unbiasedness, Eq. (4.41), replaces $\boldsymbol{g}_B$ by ∇C. In the quadratic term we split $\boldsymbol{g}_B = \nabla C + (\boldsymbol{g}_B - \nabla C)$; the cross term has expectation zero and the rest is Eq. (4.42), $\mathbb{E}\|\boldsymbol{g}_B\|^2 = \|\nabla C\|^2 + \sigma_g^2/M$. Collecting terms,

$$\mathbb{E}\big[C(\boldsymbol{\theta}_{t+1}) \mid \boldsymbol{\theta}_t\big] \le C(\boldsymbol{\theta}_t) - \underbrace{\gamma\Big(1 - \frac{L\gamma}{2}\Big)\|\nabla C(\boldsymbol{\theta}_t)\|^2}_{\text{progress}} + \underbrace{\frac{L\gamma^2\sigma_g^2}{2M}}_{\text{noise}}. \tag{4.44}$$

The first two terms are the deterministic result (4.25): on average, a stochastic step makes the progress of a full gradient step. The last term is the price of the noise, and the way the two scale is the whole story of the method. The progress is proportional to γ and to the squared gradient, which is large far from the minimum and vanishes at it; the price is proportional to γ^2/M and does not vanish. Far from the minimum the first dominates and SGD advances as if it had the exact gradient, at M/n of the cost. Close to it the second takes over, and the cost stops decreasing where the two balance – at a level proportional to γ/M, which can be lowered only by a smaller learning rate or a larger batch. At the starting point of the example of Section 4.7.7, with $M = 32$ and $\gamma = 0.02$, the progress term is 25.6 and the noise term 0.74; none of 4000 random minibatches increased the cost. With $M = 1$ and $\gamma = 0.002$, one update in eight did, and the average decrease was still within one percent of the bound.

Rates.

For a strongly convex cost, with all Hessian eigenvalues between $\mu > 0$ and L and condition number $\kappa = L/\mu$, the two regimes turn into two convergence rates. Gradient descent at its best learning rate contracts the error by the factor (4.26) per step, and the cost, which is quadratic in the error, by its square,

$$C(\boldsymbol{\theta}_t)-C^{*} \le \Big(\frac{\kappa-1}{\kappa+1}\Big)^{2t}\big(C(\boldsymbol{\theta}_0)-C^{*}\big) \approx e^{-4t/\kappa}\big(C(\boldsymbol{\theta}_0)-C^{*}\big), \qquad t_{\mathrm{GD}}(\varepsilon) \approx \frac{\kappa}{4}\ln\frac{C(\boldsymbol{\theta}_0)-C^{*}}{\varepsilon}, \quad (4.45)$$

so reaching an excess cost ε takes a number of *full passes* over the data proportional to κ and to the number of digits asked for. For SGD the recursion (4.44) with a learning rate that decays as $\gamma_t = \beta/(t+t_0)$, $\beta > 1/\mu$, gives instead [16]

$$\mathbb{E}\big[C(\boldsymbol{\theta}_t)\big]-C^{*} = \mathcal{O}\Big(\frac{L\sigma_g^2}{\mu^2 M t}\Big), \quad (4.46)$$

once the memory of the starting point has decayed; without strong convexity the rate degrades to $\mathcal{O}(1/\sqrt{t})$. Per *update*, then, SGD is far slower than gradient descent: a rate $1/t$ against a geometric one, the difference between gaining a fixed number of digits per step and needing ten times as many steps for each further digit. But an update of SGD costs M/n of a full pass, and the product Mt in Eq. (4.46) is simply the number of examples processed: the accuracy of SGD depends on how much data it has seen, not on how the data were divided into batches, and *not on n at all*. The folk statement that "SGD converges faster" conflates these two things. Whether the slow rate at the low price beats the fast rate at the high price depends on how small an ε we need, and that is a question for statistics.

4.7.6 Which accuracy is worth paying for

The optimiser works on a finite sample. The minimiser $\hat{\boldsymbol{\theta}}$ of the empirical cost is itself only an estimate of the parameters that minimise the *expected* cost, with an excess of order

$$\varepsilon_{\mathrm{stat}} \sim \frac{\sigma^2 p}{n} \quad (4.47)$$

by the variance formula (2.47), and the same scaling p/n holds for regular parametric models in general. Optimising the empirical cost beyond that accuracy improves the fit to *this* sample without improving the prediction on the next one, because the next data set would move the minimum by more than that anyway. The target is therefore not $\varepsilon \to 0$ but $\varepsilon \sim \varepsilon_{\mathrm{stat}}$, and inserting it in the two rates, with $4np$ flops per step of gradient descent and $4Mp$ per update of SGD, gives

$$\text{gradient descent: } \mathcal{O}\big(np\,\kappa \ln n\big), \qquad \text{SGD: } \mathcal{O}\Big(Mp\cdot\frac{n}{\sigma^2 p}\Big) = \mathcal{O}\big(nM/\sigma^2\big), \quad (4.48)$$

so that SGD wins by a factor of order $p\kappa\ln n/M$, which for $\kappa = 10^2$, $p = 10^3$ and $M = 32$ is several thousand, and grows with the size of the model and the conditioning of the problem. This argument, due to Bottou and Bousquet [17], is the reason the deep learning frameworks are built around minibatches: for large n the statistical accuracy that is worth having is reached after a small number of passes over the data, and in that regime SGD is not merely cheaper per step but cheaper to the goal.

The same reasoning says when the conclusion reverses. If what is wanted is a moderate validation error on a large data set, small or medium minibatches usually reach it in a few passes. If what is wanted is the empirical minimiser to many digits – to compare with an analytical result, say, or because a later calculation differentiates through it – the geometric rate of full gradients wins, and so do the *variance-reduced* methods that combine occasional full gradients with cheap stochastic ones. If the data arrive as a stream, or change while we train, only SGD applies. And if the data set is small, gradient descent, or better the Newton

and conjugate gradient methods of Sections 4.2 and 1.16, is simpler and faster. The statement "SGD is faster" always needs an accuracy target and a measure of cost to mean anything.

4.7.7 An example where SGD beats gradient descent

Figure 4.5 shows the argument at work on a least-squares problem with $n = 10^5$ observations and $p = 20$ standardised but strongly correlated features (an autoregressive correlation with coefficient 0.9 between neighbouring columns, so that $\kappa = 213$), unit noise, and $\boldsymbol{\theta}_0 = \mathbf{0}$. The horizontal axis is the number of floating-point operations by Eq. (4.38); the vertical axis is the excess cost $C(\boldsymbol{\theta}) - C(\hat{\boldsymbol{\theta}})$, and the dotted line is the estimation error $\sigma^2 p/n = 2 \times 10^{-4}$ of Eq. (4.47), below which further optimisation is statistically meaningless (the true parameters sit at 2.6×10^{-4}). Gradient descent at γ^* needs 640 iterations, 5×10^9 flops, to reach that line – the estimate $t_{\mathrm{GD}} = \frac{\kappa}{4} \ln(C_0/\varepsilon)$ of Eq. (4.45) gives 655 – and momentum with the tuned γ and β of Eq. (4.33) needs 81. SGD with $M = 32$ and the schedule $\gamma_t = 20/(t + 1000)$ of Eq. (4.52) first touches it after three to five epochs, depending on the random seed, some 3×10^7 flops – two orders of magnitude fewer than gradient descent and one fewer than momentum – and is within a factor of twenty-five of it after a single epoch, before gradient descent has completed its *first* step. The constant-learning-rate runs make the other point of this section: they are as fast at the start and then level off, at 10^{-2} for $\gamma = 0.02$ and 3×10^{-3} for $\gamma = 0.005$, the signature of the noise that does not decay.

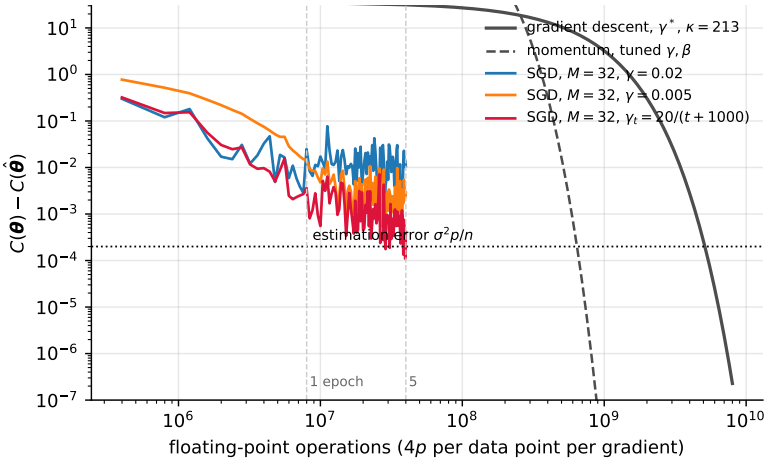


Fig. 4.5 Excess cost against floating-point operations on a least-squares problem with $n = 10^5$, $p = 20$ and $\kappa = 213$, from $\boldsymbol{\theta}_0 = \mathbf{0}$. Full-batch gradient descent at γ^* and momentum with tuned parameters against minibatch SGD ($M = 32$) at two constant learning rates and with the schedule (4.52). The dotted line is the estimation error $\sigma^2 p/n$ of Eq. (4.48); SGD with the schedule is there within five epochs, momentum in 81 full gradients and gradient descent in 640.

A held-out set of 20 000 points from the same model confirms that the dotted line is the right place to stop. The validation error is 1.0005 at the exact minimiser $\hat{\boldsymbol{\theta}}$, 1.0008 where gradient descent crosses the line, and between 1.0006 and 1.003 at the end of the SGD runs of this section: differences in the fourth digit, far below the one percent to which 20 000 points can determine a mean squared error at all. The several hundred further iterations that would take gradient descent from 10^{-4} to 10^{-10} are invisible to any measurement we could make of the quality of the model.

```
import numpy as np

n, p, rho, sigma = 100_000, 20, 0.9, 1.0
rng = np.random.default_rng(2026)
z = rng.normal(size=(n, p))
X = np.empty_like(z)
X[:, 0] = z[:, 0]
for j in range(1, p):                    # AR(1)-correlated columns: kappa = 213 after
    standardising
    X[:, j] = rho * X[:, j - 1] + np.sqrt(1 - rho**2) * z[:, j]
X = (X - X.mean(axis=0)) / X.std(axis=0)
theta_true = rng.normal(size=p)
y = X @ theta_true + sigma * rng.normal(size=n)

theta_hat = np.linalg.solve(X.T @ X, X.T @ y)          # np^2 + p^3/3 flops
c_hat = np.mean((X @ theta_hat - y)**2)
excess = lambda th: np.mean((X @ th - y)**2) - c_hat
eigs = np.linalg.eigvalsh((2.0 / n) * X.T @ X)
gamma_star = 2.0 / (eigs.max() + eigs.min())
flops_per_point = 4 * p                                # Eq. (4.flops)

theta = np.zeros(p)                                    # gradient descent: 4np flops per
    step
for k in range(1, 1001):
    theta -= gamma_star * (2.0 / n) * X.T @ (X @ theta - y)
    if excess(theta) < sigma**2 * p / n:
        print(f"gradient descent reaches sigma^2 p/n after {k} steps = {k * n *
    flops_per_point:.1e} flops")
        break

M, t, theta = 32, 0, np.zeros(p)                       # SGD: 4Mp flops per update, 4np per
    epoch
for epoch in range(1, 6):
    for b in np.array_split(rng.permutation(n), n // M):
        t += 1
        gamma = 20.0 / (t + 1000.0)                        # Eq. (4.timedecay)
        theta -= gamma * (2.0 / len(b)) * X[b].T @ (X[b] @ theta - y[b])
    print(f"SGD epoch {epoch}: excess cost {excess(theta):.2e} after {t * M *
    flops_per_point:.1e} flops")
```

4.7.8 Learning rate schedules and stopping

Because the gradient noise does not decay, a constant learning rate leaves the iterate bouncing around the minimum forever. This can be quantified. Take a single eigendirection of a quadratic cost, with curvature λ, and write the minibatch gradient as the exact gradient λe_t plus a noise term ξ_t of mean zero and variance σ_g^2/M by Eq. (4.42), independent from step to step. The error then obeys

$$e_{t+1} = (1-\gamma\lambda)\, e_t - \gamma \xi_t, \tag{4.49}$$

the recursion (4.19) driven by noise. Its mean contracts as before, but its second moment obeys $\mathbb{E}[e_{t+1}^2] = (1-\gamma\lambda)^2\,\mathbb{E}[e_t^2] + \gamma^2\sigma_g^2/M$, the cross term vanishing because the noise is independent of the current error, and for $0 < \gamma\lambda < 2$ it settles at the fixed point

$$\operatorname{Var}[e_\infty] = \frac{\gamma^2\sigma_g^2/M}{1-(1-\gamma\lambda)^2} = \frac{\gamma\sigma_g^2/M}{\lambda\,(2-\gamma\lambda)} \approx \frac{\gamma\sigma_g^2}{2\lambda M} \qquad (\gamma\lambda \ll 1). \tag{4.50}$$

The iterate does not converge; it *equilibrates*, in a cloud around the minimum whose radius scales as $\sqrt{\gamma/M}$ – largest along the flat directions, where λ is small – and which no number of further iterations will shrink.

The plateau in the cost.

What we observe is not the error along one direction but the excess cost, and for it the result is simpler still. Near the minimum $C - C^* = \frac{1}{2}\sum_i \lambda_i e_i^2$, and in the sum over eigendirections the curvature cancels against the $1/\lambda_i$ of Eq. (4.50):

$$\mathbb{E}\big[C(\boldsymbol{\theta}_\infty)\big] - C^* \approx \frac{1}{2}\sum_i \lambda_i \,\frac{\gamma \sigma_{g,i}^2}{2\lambda_i M} = \frac{\gamma}{4M}\,\mathrm{tr}\,\boldsymbol{S}, \qquad \boldsymbol{S} = \frac{1}{n}\sum_{i=1}^{n} (\boldsymbol{g}_i - \boldsymbol{g})(\boldsymbol{g}_i - \boldsymbol{g})^T, \tag{4.51}$$

with $\boldsymbol{S}$ the covariance matrix of the per-example gradients at the minimum. The plateau is proportional to γ/M and *does not depend on the condition number*: the flat directions have large clouds, but large clouds in flat directions cost little. For least squares $\boldsymbol{g}_i = 2\boldsymbol{x}_i r_i$ with r_i the residual, so that for standardised features $\mathrm{tr}\,\boldsymbol{S} \approx 4\sigma^2 p$ and the plateau is at

$$\frac{\gamma}{M}\,\sigma^2 p \qquad \text{against the estimation error} \qquad \frac{1}{n}\,\sigma^2 p.$$

The comparison is striking: a constant learning rate reaches the statistical accuracy only if $\gamma/M \le 1/n$. In the example above $\mathrm{tr}\,\boldsymbol{S} = 80.2$, and Eq. (4.51) predicts 1.25×10^{-2} for $\gamma = 0.02$ and 3.1×10^{-3} for $\gamma = 0.005$ at $M = 32$ – the two levels seen in Fig. 4.5. The right panel of Fig. 4.6 repeats the measurement for seven combinations of γ and M; all but one fall on the line within thirty percent. The exception, $\gamma = 0.05$ with $M = 512$, lies a factor 1.7 above it, because there $\gamma\lambda$ is of order one for the steepest directions and the factor $2 - \gamma\lambda$ of Eq. (4.50) can no longer be replaced by two.

Equation (4.51) says exactly what the remedies are: a smaller γ (at the price of slower progress early on), a larger M (at the price of M per step), or a γ that starts large and ends small.

Decaying learning rates.

The standard remedy is the last: let γ decay with time. Let $e = 0, 1, 2, \dots$ index the epoch, let $m = n/M$ be the number of minibatches, and count updates by $t = e\cdot m + i$ with $i = 0, \dots, m-1$. A common choice is

$$\gamma_t = \frac{t_0}{t + t_1} \qquad \Big(= \gamma_0\,\frac{t_1}{t+t_1}, \quad \gamma_0 = t_0/t_1\Big), \tag{4.52}$$

with $t_0, t_1 > 0$ fixed. The learning rate starts at $\gamma_0 = t_0/t_1$, stays near it for the first t_1 updates – a warm phase in which the iterate moves rapidly towards the right region – and then decays as $1/t$, shrinking the cloud as it goes. The classical conditions for convergence, due to Robbins and Monro [18], are

$$\sum_{t=0}^{\infty} \gamma_t = \infty, \qquad \sum_{t=0}^{\infty} \gamma_t^2 < \infty. \tag{4.53}$$

The first ensures that the iterate can still reach the minimum from any starting point; the second that the accumulated noise, which by Eq. (4.49) enters with weight γ_t^2, is finite. Equation (4.52) satisfies both; a constant satisfies neither. So does the more general polynomial decay

$$\gamma_t = \frac{\gamma_0}{(1+t/t_1)^{\alpha}}, \qquad \frac{1}{2} < \alpha \le 1, \tag{4.54}$$

whose smaller exponents decay more gently and are more forgiving of a badly chosen t_1. In a finite run the first condition bites in practice as well as in principle: with t_1 small the learning rate has collapsed after a few hundred updates and the accumulated step $\sum_t \gamma_t$ over the run is too short to cross an ill-conditioned valley, so the iterate freezes far from the minimum. The cure is to keep the initial rate t_0/t_1 and enlarge both constants, which delays the decay; the schedule in Fig. 4.5 uses $t_0 = 20$, $t_1 = 1000$. Note also that the schedule counts *updates*, so that changing the batch size changes how far the learning rate has decayed after one epoch; schedules do not transfer from one M to another without adjustment.

If the decay is slow compared with the time the iterate needs to equilibrate, the excess cost simply follows the plateau of the current learning rate, $\gamma_t \operatorname{tr}\boldsymbol{S}/(4M)$. For the schedule (4.52) and $t \gg t_1$ this is $t_0\sigma^2 p/(Mt)$, the $1/(Mt)$ of Eq. (4.46), and measured in epochs $e = tM/n$ it becomes

$$\mathbb{E}\big[C(\boldsymbol{\theta})\big] - C^{*} \approx \frac{t_0}{e} \cdot \frac{\sigma^2 p}{n} : \tag{4.55}$$

after e epochs the *mean* excess cost is t_0/e times the estimation error, whatever the batch size. For the run of Fig. 4.5, averaged over five seeds, the measured values are 1.3×10^{-3}, 8.0×10^{-4} and 2.4×10^{-4} after 3, 5 and 12 epochs, against 1.3×10^{-3}, 7.9×10^{-4} and 3.3×10^{-4} from the formula. The curve *touches* the dotted line after three epochs because it fluctuates by a factor of a few around its mean; by Eq. (4.55) the mean arrives there after about $t_0 = 20$ epochs – still thirty times sooner than gradient descent.

Averaging the iterates.

There is a second way to remove the noise, which does not slow the iterate down at all: keep the learning rate constant and *average* the iterates, $\bar{\boldsymbol{\theta}}_T = \frac{1}{T}\sum_t \boldsymbol{\theta}_t$, over the part of the run spent on the plateau. The iterates scatter around the minimum with mean zero, and their average converges to it at the statistically optimal rate, a result due to Polyak and Juditsky [19]. In the example, five epochs at the constant $\gamma = 0.02$ leave the last iterate at 1.1×10^{-2}, on the plateau of Eq. (4.51); the average of the iterates of epochs three to five has an excess cost of 2.3×10^{-6}, four orders of magnitude lower and a hundred times below the estimation error. The averaged iterate costs one extra vector of storage and one addition per update. For non-convex problems the average can straddle several basins and must be used with care, but exponentially weighted versions of it are common practice in the training of large networks.

A practical SGD loop.

The listing below assembles the pieces into the loop one actually writes. Four details deserve attention. The data are shuffled once per epoch and cut into batches, so that every observation is used exactly once per epoch, and the last, smaller batch is kept rather than discarded. The gradient is the *mean* over the batch, so that the learning rate does not have to change with M. The features are standardised – with the statistics of the training set only – so that no single coordinate dictates the stable learning rate, as discussed in Section 4.5. And the loop stops on the cost of a validation set, not on the gradient.

```
import numpy as np

def shuffled_batches(n, M, rng):
    """One epoch: every index once, in random order; the last batch may be smaller."""
```

```
    idx = rng.permutation(n)
    for k in range(0, n, M):
        yield idx[k:k + M]

def cost(theta, X, y):
    return np.mean((X @ theta - y)**2)

def gradient(theta, X, y):                    # mean over the rows handed in, Eq.
    (4.meancost)
    return (2.0 / len(y)) * X.T @ (X @ theta - y)

rng = np.random.default_rng(2026)
n, p, rho = 60_000, 20, 0.9
z = rng.normal(size=(n, p)); X = np.empty_like(z); X[:, 0] = z[:, 0]
for j in range(1, p):
    X[:, j] = rho * X[:, j - 1] + np.sqrt(1 - rho**2) * z[:, j]
y = X @ rng.normal(size=p) + rng.normal(size=n)
X_train, y_train, X_val, y_val = X[:50_000], y[:50_000], X[50_000:], y[50_000:]
mean, std = X_train.mean(axis=0), X_train.std(axis=0)      # training set only
X_train, X_val = (X_train - mean) / std, (X_val - mean) / std

M, gamma0, t0, max_epochs, patience = 32, 0.02, 1000.0, 100, 5
theta, t = np.zeros(p), 0
best_val, best_theta, waited = np.inf, theta.copy(), 0
for epoch in range(1, max_epochs + 1):
    for batch in shuffled_batches(len(y_train), M, rng):
        t += 1
        gamma = gamma0 * t0 / (t + t0)                     # Eq. (4.timedecay)
        theta -= gamma * gradient(theta, X_train[batch], y_train[batch])
    val = cost(theta, X_val, y_val)
    if val < best_val - 1e-5:                              # early stopping on the
    validation cost
        best_val, best_theta, waited = val, theta.copy(), 0
    else:
        waited += 1
    print(f"epoch {epoch:2d}: gamma = {gamma:.1e}, "
          f"training cost {cost(theta, X_train, y_train):.4f}, validation cost {val:.4f}")
    if waited == patience:
        break
theta_ols = np.linalg.lstsq(X_train, y_train, rcond=None)[0]
print(f"stopped after {epoch} epochs; best validation cost {best_val:.4f} "
      f"(least-squares solution: {cost(theta_ols, X_val, y_val):.4f})")
```

```
epoch  1: gamma = 7.8e-03, training cost 1.0048, validation cost 1.0239
epoch  2: gamma = 4.8e-03, training cost 1.0024, validation cost 1.0205
epoch  3: gamma = 3.5e-03, training cost 1.0019, validation cost 1.0201
...
epoch  9: gamma = 1.3e-03, training cost 1.0010, validation cost 1.0182
...
epoch 14: gamma = 8.7e-04, training cost 1.0017, validation cost 1.0200
stopped after 14 epochs; best validation cost 1.0182 (least-squares solution: 1.0186)
```

After three epochs the validation cost is within two parts in a thousand of what the exact least-squares solution achieves, and the best of the fourteen epochs is, by the luck of the noise, marginally below it; the patience of five epochs is what stops the loop from reacting to fluctuations of that size.

When do we stop?

One possibility is to compute the full gradient every few epochs and stop when its norm falls below a threshold. But a vanishing gradient identifies a stationary point, not necessarily a good one, so it is wiser to evaluate the cost at that point, record it, and continue: if the criterion triggers again later, keep whichever $\boldsymbol{\theta}$ gave the lower value. Because the scheme is random by design, repeating the whole computation gives a different answer each time, and taking the best of several runs is a legitimate and common strategy.

In machine learning the more useful criterion is not about the gradient at all. As advised in Section 2.9, one monitors the cost on a held-out validation set and stops when it begins to *rise* while the training cost is still falling, as the loop above does. This is *early stopping*, and it is a regularisation method as much as a termination rule: halting before the variance term of Eq. (2.52) has grown is another way of trading a little bias for a lot of variance, in the same spirit as the Ridge penalty of Section 3.8.

4.7.9 The batch size, and how to compare fairly

The batch size changes the statistics and the use of the hardware at the same time, and the two pull in opposite directions. Table 4.2 lists what is traded.

Table 4.2 What the minibatch size trades. The statistical rows follow from Eqs. (4.42) and (4.51); the hardware rows are measured for our example in Table 4.3.

	small batch	**large batch**
gradient variance	large	small, $\propto 1/M$
updates per epoch	many	few
hardware efficiency	poor: overhead dominates	good, until the processor saturates
memory	small	larger
late-stage accuracy	needs decay or averaging	more stable at a fixed γ

A simple model of the wall time of one update is

$$T_{\text{update}}(M) \approx T_{\text{overhead}} + \frac{M}{R(M)}, \tag{4.56}$$

with T_{overhead} the fixed cost of launching the operations and $R(M)$ the rate, in examples per second, at which the hardware processes a batch once it is running. The left panel of Fig. 4.6 shows both for our example, on one core of a laptop; the absolute numbers will differ on another machine, the shape will not. An update with $M = 1$ takes $5\,\mu$s and one with $M = 32$ takes $7\,\mu$s: up to a few tens of examples the batch is essentially free, because the time goes into the overhead of the interpreter and of the library calls. The throughput rises from 2×10^5 examples per second at $M = 1$ to 1.5×10^7 at $M \approx 2000$, where it saturates, and *falls* again for the full batch of 10^5 rows, which no longer fits in the cache.

Table 4.3 and Fig. 4.7 put the two halves together. Counted in *epochs*, the batch size hardly matters: from $M = 1$ to $M = 128$ every run is at the line after three passes over the data, as Eq. (4.55) says it should be, and only at $M = 512$, where a pass contains too few updates for the warm phase, does the count rise. Counted in *updates* the runs differ by a factor of three

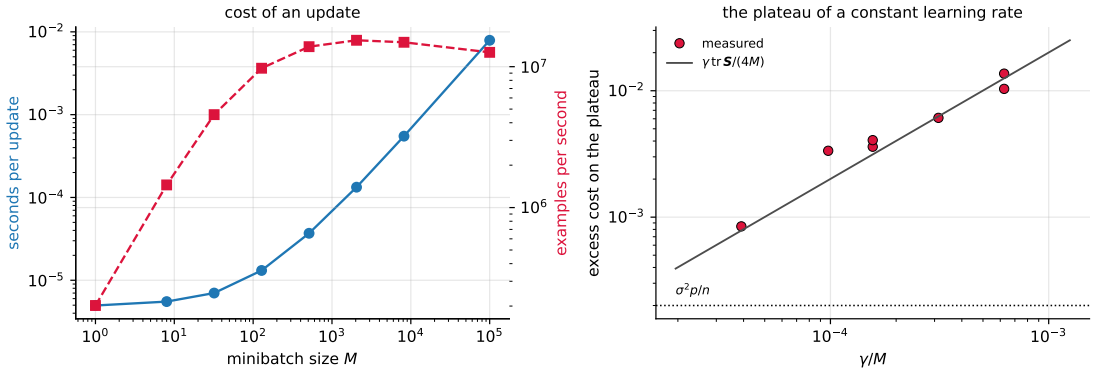


Fig. 4.6 Left: measured time per SGD update and throughput in examples per second against the minibatch size, for the least-squares problem of Fig. 4.5; the update is nearly free up to $M \approx 30$, and the throughput saturates near $M = 2000$, the two regimes of Eq. (4.56). Right: the excess cost at which SGD with a constant learning rate levels off, for seven combinations of γ and M, against the prediction $\gamma \operatorname{tr} \boldsymbol{S}/(4M)$ of Eq. (4.51); the dotted line is the estimation error $\sigma^2 p/n$.

Table 4.3 Reaching the estimation error $\sigma^2 p/n = 2 \times 10^{-4}$ on the problem of Fig. 4.5: gradient descent at γ^* against SGD at five batch sizes, each with the best schedule $\gamma_t = \gamma_0 t_1/(t + t_1)$ of a small grid. Updates, epochs and floating-point operations are medians over five seeds of the first time the excess cost, recorded twenty times per epoch, is below the line; the wall time is the number of updates times the measured time per update and is machine dependent.

Method	γ_0	t_1	**updates**	**epochs**	**flops**	**time (s)**
gradient descent	0.0885	–	640	640	5.1×10^9	5
SGD, $M = 1$	0.002	10^4	335 000	3.4	2.7×10^7	1.7
SGD, $M = 8$	0.01	10^3	40 625	3.2	2.6×10^7	0.2
SGD, $M = 32$	0.02	10^3	9 370	3.0	2.4×10^7	0.07
SGD, $M = 128$	0.08	300	2 344	3.0	2.4×10^7	0.03
SGD, $M = 512$	0.08	10^3	1 124	5.8	4.6×10^7	0.04

hundred, and counted in *seconds* by a factor of sixty, in the opposite order: $M = 1$ wastes the hardware, and the best batch size is the largest one that does not yet cost extra epochs – here $M \approx 128$, near the point where the throughput saturates, and not $M = 1$. Gradient descent, finally, is the fastest of all per update and the slowest by a factor of a hundred and fifty in time.

Three axes, three stories.

Figure 4.7 is a warning as much as a result. A plot of the cost against the number of *updates* favours whoever has the most expensive update, since gradient descent makes one per epoch; a plot against *epochs* compares the amount of data processed; a plot against *wall time* includes memory traffic, parallelism, compilation and communication, and is the one the user of the method cares about. An efficiency claim without a common and stated horizontal axis is incomplete. For a comparison to be fair one should, in addition: use the same training and validation split for all methods; tune the learning rate or the schedule *separately* for each method, since the best γ of one is rarely the best of another; compare at equal validation performance rather than at an equal number of iterations; and repeat the stochastic runs over several seeds and report the spread, which behind Table 4.3 is up to a factor of 1.8 between

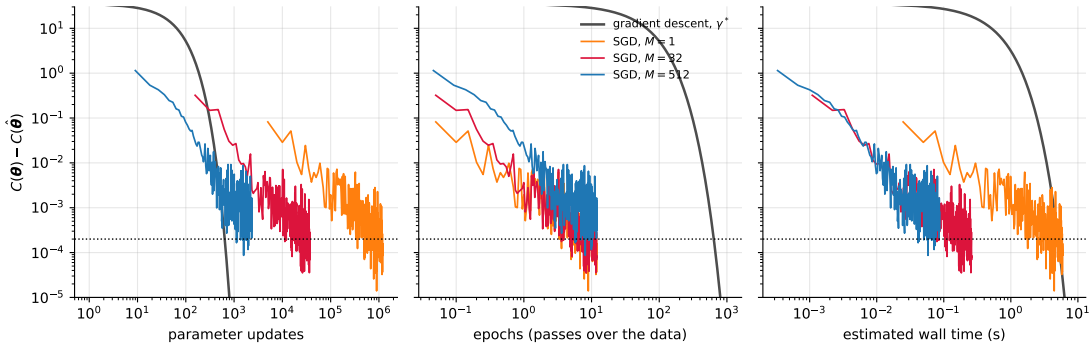


Fig. 4.7 The same four runs – gradient descent at γ^* and SGD with $M = 1$, 32 and 512 and the schedules of Table 4.3 – against three horizontal axes. Left: parameter updates, which favours gradient descent. Centre: epochs, that is, the amount of data processed, on which the three SGD runs coincide. Right: estimated wall time, on which the small batch falls behind. The dotted line is the estimation error $\sigma^2 p/n$.

the luckiest and the unluckiest of five. The exercises ask for this protocol on problems with several condition numbers: ill conditioning slows gradient descent in proportion to κ, and it slows SGD as well, through the requirement $\beta > 1/\mu$ behind Eq. (4.46), which is where momentum and the preconditioning of the following sections come in.

When SGD is not the more efficient method.

Full gradients can win, and it is as well to know when: when n is small enough for the whole data set to sit comfortably in fast memory; when the required solution is very accurate; when the per-example gradients have an extremely large variance, so that a minibatch says little about the direction; when large matrix operations use the hardware so much better than small ones that the throughput of Fig. 4.6 outweighs the number of updates; and when the objective permits a cheap sufficient statistic. Least squares is the standard example of the last: forming $\boldsymbol{X}^T\boldsymbol{X}$ and $\boldsymbol{X}^T\boldsymbol{y}$ once costs a single pass over the data, after which every gradient costs p^2 operations independently of n – attractive when $p \ll n$ and the $p \times p$ matrix fits in memory, though it squares the condition number, Eq. (4.27), and must be weighed against the numerical stability of doing so.

A precise conclusion. SGD is computationally efficient when a useful solution requires only a few passes over the data and each minibatch carries enough information about the direction to make progress. Both conditions hold for the large, noisy, redundant data sets of machine learning, where the accuracy worth having is $\sigma^2 p/n$ and not machine precision; neither holds for a small, clean problem that must be solved to many digits. The minibatch gradient is unbiased with variance $\propto 1/M$; a constant learning rate leaves a plateau $\gamma \mathrm{tr}\boldsymbol{S}/(4M)$ in the cost; decaying rates, iterate averaging and validation-based stopping are what convert cheap noisy updates into useful accuracy; and the efficiency must be reported against examples processed, operations or wall time, at the same validation performance.

4.8 Why adapt the step size at all

In stochastic gradient descent, with or without momentum, we still have to specify a schedule for the learning rate γ_t. Section 4.5 showed exactly why this is awkward, and it is worth restating the diagnosis in the form that motivates what follows.

A fixed γ is hard to get right. If it is too large the updates overshoot and the iteration oscillates or diverges, by Eq. (4.20); if it is too small, convergence takes forever, by Eq. (4.26). But the deeper problem is not the choice of a number: it is that *no single number is right for all directions*. The stability bound is set by $\lambda_{\max}$ and the convergence rate by $\lambda_{\min}$, and when these differ by orders of magnitude the step which is barely safe along the steep direction is hopelessly small along the flat one. Steep coordinates need small steps; flat coordinates could take large ones. In high-dimensional problems with features of varying scale, or with sparse features that are active only occasionally, the mismatch is severe.

Ideally our algorithm would keep track of curvature and take large steps in shallow directions and small steps in steep ones. That is precisely what Newton's method (4.8) does, by normalising the gradient with $\boldsymbol{H}^{-1}$; and the invariance this buys is why Section 4.2 called it the ideal. The trouble, again, is the $\mathcal{O}(p^3)$ cost of forming and inverting the Hessian, which is out of the question for large models.

The methods of this section are a compromise. They approximate the curvature information in $\boldsymbol{H}$ using only quantities already available – the gradients themselves – and they restrict the approximation to a *diagonal* matrix, so that inverting it costs $\mathcal{O}(p)$ rather than $\mathcal{O}(p^3)$. Instead of tracking the gradient alone, they track the *second moment* of the gradient. The family includes AdaGrad, AdaDelta, RMSProp and Adam, and we develop the three named in the chapter title.

The idea in one paragraph.

Suppose we replace the scalar γ by a diagonal matrix, so that coordinate j has its own step size $\gamma/\sqrt{r_j}$. What should r_j be? Near a minimum the gradient along direction j behaves as $g_j \approx \lambda_j e_j$, so a large curvature λ_j produces large gradients and a small curvature produces small ones. *The typical magnitude of the gradient in a direction is a proxy for the curvature in that direction.* Averaging g_j^2 over recent steps therefore estimates something like λ_j^2 without ever forming a second derivative, and dividing by its square root approximates the $\boldsymbol{H}^{-1/2}$ scaling. This is the entire principle behind AdaGrad, RMSProp and Adam; the three differ only in how the average is taken.

The idea made precise: diagonal preconditioning.

All three methods are instances of the *preconditioned* iteration

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma \boldsymbol{D}_t^{-1} \nabla C(\boldsymbol{\theta}_t), \qquad \boldsymbol{D}_t = \operatorname{diag}\big(\sqrt{v_{t,1}}, \ldots, \sqrt{v_{t,p}}\big), \tag{4.57}$$

with Newton's method the case $\boldsymbol{D} = \boldsymbol{H}$ and gradient descent the case $\boldsymbol{D} = \boldsymbol{I}$. For a quadratic cost the error recursion becomes $\boldsymbol{e}_{t+1} = (\boldsymbol{I} - \gamma \boldsymbol{D}^{-1}\boldsymbol{H})\boldsymbol{e}_t$, so what governs stability and speed is no longer the spectrum of $\boldsymbol{H}$ but that of $\boldsymbol{D}^{-1}\boldsymbol{H}$. When $\boldsymbol{H}$ is diagonal and $\boldsymbol{D}$ tracks its diagonal, $\boldsymbol{D}^{-1}\boldsymbol{H} = \boldsymbol{I}$: every direction contracts at the same rate, the condition number has been reduced to one, and the iteration count no longer depends on κ at all. When $\boldsymbol{H}$ has off-diagonal entries, no diagonal $\boldsymbol{D}$ can achieve this; the best a diagonal preconditioner can do is bounded by how far $\boldsymbol{H}$ is from diagonal, and a valley that runs at 45° to the axes, with the two coordinates

equally scaled, is not improved at all. Figure 4.8 shows both cases on the bowl with eigenvalues 1 and 100: on the axis-aligned bowl the adaptive methods set off diagonally, both coordinates moving at about γ per step, and reach $\|\boldsymbol{\theta}\| < 10^{-4}$ in 297 (AdaGrad), 52 (RMSProp) and 151 (Adam) iterations against 546 for gradient descent at $0.9 \cdot 2/\lambda_{\max}$ and 162 for momentum; on the same bowl rotated by 45° gradient descent and momentum are *unchanged*, because Eq. (4.19) depends only on the eigenvalues, while AdaGrad and RMSProp no longer reach 10^{-4} in 5000 iterations and Adam needs 635. The adaptive methods see the diagonal of the Hessian and nothing else.

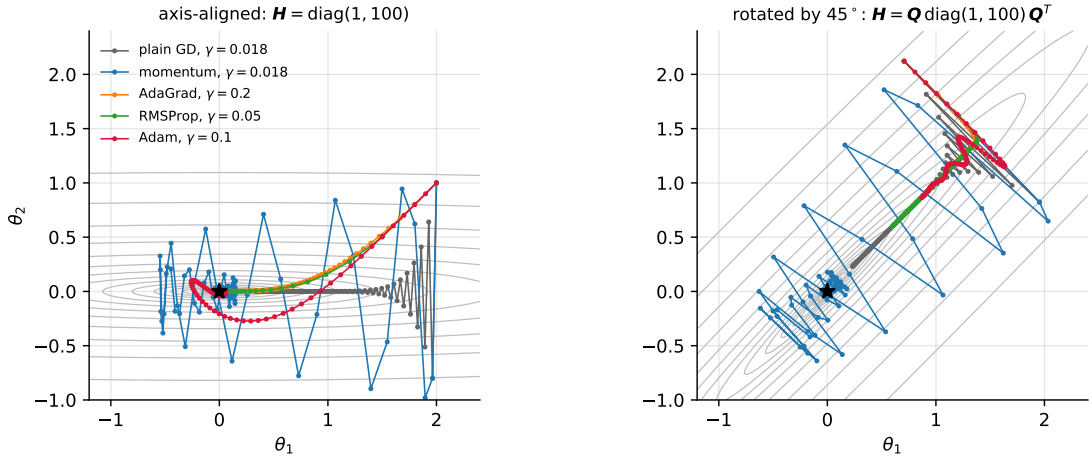


Fig. 4.8 The first hundred steps of five optimisers on the quadratic bowl with Hessian eigenvalues 1 and 100, from the same starting point. Left: the Hessian is diagonal, and the adaptive methods, which rescale each coordinate by its own gradient history, move diagonally towards the minimum while plain gradient descent zigzags down the steep direction and crawls along the flat one. Right: the same bowl rotated by 45°, with the same eigenvalues and condition number. Gradient descent and momentum are unaffected; the adaptive methods, whose diagonal rescaling cannot see the off-diagonal curvature, lose most of their advantage.

Scale invariance.

A second consequence of Eq. (4.57) explains why the adaptive methods are so much less sensitive to preprocessing than gradient descent. Multiply feature j, one column of the design matrix, by a constant c: $\boldsymbol{X}' = \boldsymbol{X}\boldsymbol{S}$ with $\boldsymbol{S} = \mathrm{diag}(1, \dots, c, \dots, 1)$. The least-squares cost (4.12) in the new parameters is

$$C'(\boldsymbol{\theta}') = \frac{1}{n}\|\boldsymbol{X}\boldsymbol{S}\boldsymbol{\theta}' - \boldsymbol{y}\|_2^2 = C(\boldsymbol{S}\boldsymbol{\theta}'),$$

so the same predictions are obtained with $\theta'_j = \theta_j / c$ and the optimum moves to $\theta'^*_j = \theta^*_j / c$. By the chain rule, or directly from Eq. (4.13),

$$\nabla C'(\boldsymbol{\theta}') = \boldsymbol{S}\nabla C(\boldsymbol{S}\boldsymbol{\theta}') = \frac{2}{n}\boldsymbol{S}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{S}\boldsymbol{\theta}' - \boldsymbol{y}),$$

so that $g'_j = c\, g_j$ while $g'_k = g_k$ for $k \neq j$, comparing the two problems at corresponding points $\boldsymbol{\theta} = \boldsymbol{S}\boldsymbol{\theta}'$, and the Hessian (4.14) becomes $\boldsymbol{H}' = \boldsymbol{S}\boldsymbol{H}\boldsymbol{S}$, with $H'_{jj} = c^2 H_{jj}$ and $H'_{jk} = c\, H_{jk}$ for $k \neq j$. For plain gradient descent the step in coordinate j is $\Delta\theta'_j = -\gamma c\, g_j$: it grows by c while the distance to the optimum shrinks by c, a mismatch of c^2, and this is what shifts the stability bound (4.20) through $\lambda_{\max}(\boldsymbol{H}')$, which for large c grows like $c^2 H_{jj}$. A learning rate that was

optimal before the rescaling diverges after it. For the adaptive methods every accumulator is built from the gradient and its square, so $r'_j = c^2 r_j$, $v'_j = c^2 v_j$ and $m'_j = c\,m_j$, and the ratios in the updates are unchanged,

$$\frac{g'_j}{\sqrt{v'_j}} = \frac{c\,g_j}{c\sqrt{v_j}} = \frac{g_j}{\sqrt{v_j}}, \qquad \frac{\hat{m}'_j}{\sqrt{\hat{v}'_j}} = \frac{\hat{m}_j}{\sqrt{\hat{v}_j}}.$$

The step $\Delta\theta'_j = \Delta\theta_j$ does not depend on c at all, as long as $g_j^2 \gg \varepsilon$, and neither does the stability of the method: the iterate takes the same steps in the rescaled units whatever the units of the data.

Why one standardises nonetheless.

The invariance just derived does not make the standardisation of Chapter 3 superfluous for the adaptive methods, for three reasons. First, it is an invariance to the scaling of individual features, not to their correlation: the off-diagonal entries $H'_{jk} = cH_{jk}$ are untouched by a diagonal $\boldsymbol{D}_t$, which is exactly the structure the rotated bowl of Figure 4.8 showed to be out of reach. Centring the columns removes the largest single source of that correlation, the one with the intercept, and leaves $\boldsymbol{H}$ as diagonal as the features allow. Second, the learning rate of an adaptive method is a length in the units of each parameter (Section 4.11 makes this precise), and a single γ suits every coordinate only if the parameters are of comparable size. Scaling a column by c leaves the step unchanged but makes the distance θ_j^*/c it has to cover c times shorter; with $c = 100$ a coordinate whose optimum is 0.06 is walked at steps of $\gamma = 0.1$, straight past it. Standardised columns give parameters of the same order and let one γ serve all of them. Third, neither ε nor a penalty is invariant. A feature scaled down until $g_j^2 \lesssim \varepsilon$ has its denominator dominated by ε, and the update degenerates to $-\gamma g_j/\sqrt{\varepsilon}$, plain gradient descent with an enormous learning rate; and the Ridge cost (4.16) penalises $\theta_j'^2 = \theta_j^2/c^2$ instead of θ_j^2, so the penalty, and with it the minimiser, changes with the units – the reason Chapter 3 standardises before regularising.

4.9 AdaGrad

AdaGrad maintains a running sum of squared gradients for each coordinate. Let $\boldsymbol{g}_t = \nabla C_{i_t}(\boldsymbol{\theta}_t)$ be the gradient at step t, possibly from a minibatch, and initialise $\boldsymbol{r}_0 = \boldsymbol{0}$. At each iteration accumulate

$$\boldsymbol{r}_t = \boldsymbol{r}_{t-1} + \boldsymbol{g}_t \circ \boldsymbol{g}_t, \tag{4.58}$$

where $\circ$ is the Hadamard product of Section 1.2, so that $r_{t,j} = r_{t-1,j} + g_{t,j}^2$ for every coordinate j. We may regard $\boldsymbol{H}_t = \operatorname{diag}(\boldsymbol{r}_t)$ as a diagonal matrix of accumulated squared gradients, with $\boldsymbol{H}_0 = \boldsymbol{0}$.

The update scales the gradient by the inverse square root of this matrix,

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma \boldsymbol{H}_t^{-1/2} \boldsymbol{g}_t, \tag{4.59}$$

where $\boldsymbol{H}_t^{-1/2}$ is diagonal with entries $r_{t,j}^{-1/2}$. In coordinates each parameter has its own step size,

$$\theta_{t+1,j} = \theta_{t,j} - \frac{\gamma}{\sqrt{r_{t,j}}}\, g_{t,j}, \tag{4.60}$$

and in practice a small constant ε is added to the denominator for numerical stability,

$$\boxed{\theta_{t+1,j} = \theta_{t,j} - \frac{\gamma}{\sqrt{\varepsilon + r_{t,j}}}\, g_{t,j}.} \tag{4.61}$$

The effective learning rate for parameter j at time t is $\alpha_{t,j} = \gamma/\sqrt{\varepsilon + r_{t,j}}$, which decreases as $r_{t,j}$ grows.

Note the resemblance between Eq. (4.59) and Newton's step (4.8): both premultiply the gradient by an inverse matrix built from curvature information. AdaGrad differs in using $\boldsymbol{H}^{-1/2}$ rather than $\boldsymbol{H}^{-1}$, in restricting $\boldsymbol{H}$ to be diagonal, and in estimating it from gradient magnitudes rather than second derivatives.

Properties.

AdaGrad tunes the step size for each parameter automatically. Parameters with large or volatile gradients receive smaller steps; those with small or infrequent gradients receive relatively larger ones. No manual schedule is needed: because $\boldsymbol{r}_t$ never decreases, the step sizes $\gamma/\sqrt{r_{t,j}}$ are non-increasing, which has an effect similar to a decaying learning rate but individualised per coordinate.

The benefit for sparse data is worth spelling out, since it is the setting for which AdaGrad was designed. Consider a rare feature – a word appearing in one document in a thousand. Its gradient is zero on almost every minibatch, so $r_{t,j}$ grows very slowly, so its learning rate stays high. When the feature finally does appear, the parameter takes a large, useful step instead of the negligible one a global schedule would have permitted by then. Frequently active features, conversely, accumulate large $r_{t,j}$ and their learning rates fall automatically.

In convex optimisation AdaGrad achieves a convergence rate comparable to the best fixed learning rate tuned in hindsight for the problem, which is a strong guarantee and effectively removes the need to tune γ by hand.

The built-in schedule.

The form of the decay is easy to read off. Along a direction where the gradient keeps a roughly constant magnitude $|g|$ – a flat direction far from the minimum – the accumulator is $r_t \approx t\, g^2$ and the effective learning rate is

$$\alpha_t = \frac{\gamma}{\sqrt{\varepsilon + r_t}} \approx \frac{\gamma}{|g|\sqrt{t}}, \tag{4.62}$$

a $1/\sqrt{t}$ schedule of exactly the kind that Section 4.7.8 introduces by hand, but scaled by the size of the gradient in that coordinate, so that every direction moves by about $\gamma/\sqrt{t}$ regardless of how steep it is. At the very first step $r_1 = g_1^2$ and the update is $\gamma \operatorname{sign}(g_1)$ in every coordinate: the method starts by moving each parameter by the same distance γ, whatever the gradient. This is the sense in which γ is a *length* for the adaptive methods rather than a multiplier of the gradient, a point that Section 4.12 returns to. The cumulative displacement along the constant-gradient direction is $\sum_t \gamma/\sqrt{t} \sim 2\gamma\sqrt{t}$, which grows without bound but ever more slowly: AdaGrad can reach any minimum in principle, and takes increasingly long to do so in practice.

The limitation.

Because $\boldsymbol{r}_t$ accumulates without bound, the learning rates decay monotonically and can become vanishingly small long before the minimum is reached. On a convex problem this is tolerable, since the accumulated gradients genuinely reflect the geometry. In deep learning, where training runs for many epochs and the landscape changes character as the iterate moves, it is fatal: AdaGrad simply stops making progress. The sum has an infinite memory, and remembers gradients from a region of parameter space the iterate left long ago. RMSProp and Adam repair exactly this.

4.10 RMSProp

RMSProp replaces the cumulative sum (4.58) by an exponentially decaying average,

$$\boldsymbol{v}_t = \rho\, \boldsymbol{v}_{t-1} + (1-\rho)\,(\nabla C(\boldsymbol{\theta}_t))^2, \tag{4.63}$$

with the square taken element-wise and ρ typically 0.9 or 0.99. The update is

$$\boxed{\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\gamma}{\sqrt{\boldsymbol{v}_t + \varepsilon}}\,\nabla C(\boldsymbol{\theta}_t),} \tag{4.64}$$

with the division element-wise. The method was proposed by Geoffrey Hinton in lecture notes in 2012 and never formally published, which has not prevented it from becoming standard.

Why the change matters.

Expanding Eq. (4.63) shows what has happened. Write $\boldsymbol{g}_t = \nabla C(\boldsymbol{\theta}_t)$ for the gradient at step $t = 1, 2, \ldots$, start from $\boldsymbol{v}_0 = \boldsymbol{0}$ and substitute the recursion into itself,

$$\begin{aligned}
\boldsymbol{v}_1 &= (1-\rho)\,\boldsymbol{g}_1^2,\\
\boldsymbol{v}_2 &= \rho\,\boldsymbol{v}_1 + (1-\rho)\,\boldsymbol{g}_2^2 = (1-\rho)\left[\rho\,\boldsymbol{g}_1^2 + \boldsymbol{g}_2^2\right],\\
\boldsymbol{v}_3 &= \rho\,\boldsymbol{v}_2 + (1-\rho)\,\boldsymbol{g}_3^2 = (1-\rho)\left[\rho^2\boldsymbol{g}_1^2 + \rho\,\boldsymbol{g}_2^2 + \boldsymbol{g}_3^2\right].
\end{aligned}$$

The pattern is proved by induction: if $\boldsymbol{v}_{t-1} = (1-\rho)\sum_{j=1}^{t-1}\rho^{\,t-1-j}\boldsymbol{g}_j^2$, then $\boldsymbol{v}_t = \rho\,\boldsymbol{v}_{t-1} + (1-\rho)\boldsymbol{g}_t^2 = (1-\rho)\sum_{j=1}^{t-1}\rho^{\,t-j}\boldsymbol{g}_j^2 + (1-\rho)\rho^0\boldsymbol{g}_t^2$, that is,

$$\boldsymbol{v}_t = (1-\rho)\sum_{j=1}^{t}\rho^{\,t-j}\boldsymbol{g}_j^2. \tag{4.65}$$

Gradient $\boldsymbol{g}_j$ therefore enters $\boldsymbol{v}_t$ with the weight $w_{t,j} = (1-\rho)\rho^{\,t-j}$, and the weights form a finite geometric series,

$$\sum_{j=1}^{t} w_{t,j} = (1-\rho)\sum_{k=0}^{t-1}\rho^k = (1-\rho)\,\frac{1-\rho^t}{1-\rho} = 1-\rho^t \;\longrightarrow\; 1.$$

So $\boldsymbol{v}_t$ is a *weighted mean* of the past squared gradients rather than an unbounded sum; the missing mass ρ^t is the weight still carried by the zero initial value $\boldsymbol{v}_0$, a point Section 4.11 returns to. The mean is also a short one. A gradient that entered at step j with weight $1-\rho$

has, $k = t - j$ steps later, the weight $(1-\rho)\rho^k$, a fraction ρ^k of its initial weight, which has fallen below $1/e$ once

$$\rho^k < e^{-1} \iff k > \frac{-1}{\ln\rho} \approx \frac{1}{1-\rho},$$

using $-\ln\rho = (1-\rho) + \frac{1}{2}(1-\rho)^2 + \dots$ for ρ close to one. For $\rho = 0.9$ this gives $-1/\ln\rho = 9.5$, so the tenth step is the first at which the weight has dropped below $1/e$ ($0.9^9 = 0.387$, $0.9^{10} = 0.349$); for $\rho = 0.99$ it gives 99.5 and the hundredth step ($0.99^{100} = 0.366$). Gradients older than roughly $1/(1-\rho)$ steps – ten steps for $\rho = 0.9$, a hundred for $\rho = 0.99$ – are effectively forgotten, whereas in AdaGrad's sum (4.58) every weight is one and the weights add up to t. Two consequences follow. The quantity $\boldsymbol{v}_t$ is now an estimate of the *recent* mean square gradient, so it tracks the local curvature as the iterate moves through a changing landscape. And because it does not grow without bound, the effective learning rate does not decay to zero: AdaGrad's infinite memory problem is gone.

RMSProp is thus the same idea as AdaGrad with a finite memory, and the resemblance to the momentum update (4.28) is not accidental. Both are exponential moving averages; momentum averages the gradient, RMSProp averages its square. It is natural to ask what happens if one does both.

The price of forgetting.

Finite memory removes AdaGrad's decay, and with it AdaGrad's ability to settle. Consider the one-dimensional quadratic $C = \frac{1}{2}\lambda\theta^2$ with a constant learning rate. Once the iterate is close to the minimum the gradient $g_t = \lambda\theta_t$ is small and v_t, an average of recent g^2, is small with it: for a slowly varying iterate $v_t \approx g_t^2$, and the update (4.64) becomes

$$\theta_{t+1} \approx \theta_t - \gamma\,\mathrm{sign}(g_t) \qquad \text{whenever} \qquad g_t^2 \gg \varepsilon, \tag{4.66}$$

a step of fixed length γ that cannot shrink. Convergence would require $|\theta_{t+1}| < |\theta_t|$ for all large t, that is $\gamma\lambda/\sqrt{v_t+\varepsilon} < 2$; but as $\theta_t \to 0$ the denominator tends to $\sqrt{\varepsilon}$, and the condition becomes $\gamma\lambda < 2\sqrt{\varepsilon}$, which with $\varepsilon = 10^{-8}$ means $\gamma\lambda < 2\times 10^{-4}$ – a learning rate too small to be useful. For any practical γ the iterate therefore ends in a limit cycle of amplitude of order γ around the minimum, regardless of λ and of ρ, and the excess cost floors at a value of order $\lambda\gamma^2$. On the convex polynomial fits of Section 4.4 this is exactly what one observes: RMSProp at a constant γ reaches an excess cost of 10^{-4} at best and never 10^{-6}, at every γ between 10^{-3} and 1, while AdaGrad, whose built-in $1/\sqrt{t}$ decay of Eq. (4.62) is precisely what has been discarded, converges to rounding. RMSProp needs a decaying γ of the kind of Section 4.7.8 to converge at all; in the non-convex, noisy setting of deep learning, where nobody expects to sit at a minimum, this is a price cheerfully paid for not stalling.

4.11 Adam

Adam – adaptive moment estimation – was introduced by Kingma and Ba in 2014 [20] and does exactly that. It keeps running averages of both the first and the second moment of the gradient and uses them to adapt the learning rate for each parameter. It is efficient for large problems involving many data and many parameters, and it is the default optimiser in most deep learning frameworks.

Why combine momentum and RMSProp?

The two mechanisms address different defects and do not overlap. Momentum gives fast convergence by smoothing the gradient, accelerating along the consistent long-term direction as in Eq. (4.32) and damping oscillations across narrow valleys. RMSProp gives per-dimension scaling for stability, handling features of different scales and sparse gradients. Using both means the direction of the step is chosen by an averaged gradient while its length in each coordinate is chosen by the recent curvature estimate.

The two moments.

Adam maintains, at each step t,

$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1)\nabla C(\boldsymbol{\theta}_t) \qquad \text{(first moment, the momentum term)}, \tag{4.67}$$

$$\boldsymbol{v}_t = \beta_2 \boldsymbol{v}_{t-1} + (1-\beta_2)\left(\nabla C(\boldsymbol{\theta}_t)\right)^2 \qquad \text{(second moment, the RMS term)}, \tag{4.68}$$

with typical values $\beta_1 = 0.9$ and $\beta_2 = 0.999$, and $\boldsymbol{m}_0 = \boldsymbol{v}_0 = \mathbf{0}$.

Bias correction.

Those initialisations cause a problem which is worth deriving, because it explains a step that otherwise looks arbitrary. Equation (4.68) has the form of Eq. (4.63) with $\rho \to \beta_2$, so the unrolled form (4.65) applies at once,

$$\boldsymbol{v}_t = (1-\beta_2)\sum_{j=1}^{t} \beta_2^{t-j}\boldsymbol{g}_j^2, \qquad \boldsymbol{m}_t = (1-\beta_1)\sum_{j=1}^{t} \beta_1^{t-j}\boldsymbol{g}_j,$$

the second line being the same unrolling of Eq. (4.67). Suppose now that the gradient is stationary, with a true second moment $\mathbb{E}[g_j^2] = \mathbb{E}[g^2]$ and a true mean $\mathbb{E}[g_j] = \mathbb{E}[g]$ that do not depend on j (the coordinate index is suppressed). The expectation is linear, so it passes through the sum and leaves a finite geometric series,

$$\mathbb{E}[v_t] = (1-\beta_2)\sum_{j=1}^{t} \beta_2^{t-j}\,\mathbb{E}[g^2] = \mathbb{E}[g^2]\,(1-\beta_2)\sum_{k=0}^{t-1}\beta_2^{k} = \mathbb{E}[g^2]\left(1-\beta_2^t\right), \tag{4.69}$$

and identically $\mathbb{E}[m_t] = \mathbb{E}[g]\,(1-\beta_1^t)$. Each raw moment is therefore too small by exactly the factor $1-\beta_i^t$, which is the weight the exponential average still assigns to the zero initial value. The bias is severe at the start: with $\beta_2 = 0.999$ the factor is 10^{-3} at $t = 1$, so the raw v_1 underestimates the true second moment by three orders of magnitude, and the step $\gamma/\sqrt{v}$ would be enormous.

The factors $1-\beta_1^t$ and $1-\beta_2^t$ are *deterministic*: they depend on t and on the two constants, not on the data. Dividing by a constant commutes with the expectation, so

$$\hat{\boldsymbol{m}}_t = \frac{\boldsymbol{m}_t}{1-\beta_1^t}, \qquad \hat{\boldsymbol{v}}_t = \frac{\boldsymbol{v}_t}{1-\beta_2^t} \tag{4.70}$$

satisfy $\mathbb{E}[\hat{m}_t] = \mathbb{E}[g]$ and $\mathbb{E}[\hat{v}_t] = \mathbb{E}[g^2]$ for *every* $t \geq 1$, not only asymptotically: the correction removes the bias exactly, and it does so by rescaling rather than by averaging, so it adds no noise. For small t the correction is large, compensating for the initial zero; as t grows,

$1-\beta_i^t \to 1$ and the corrected moments converge to the raw ones. The following values for the default constants show how different the two time scales are:

correction factor $1/(1-\beta_i^t)$	$t=1$	$t=10$	$t=1000$
$\beta_1 = 0.9$	10	1.54	1.00
$\beta_2 = 0.999$	1000	100	1.58

The first moment is essentially unbiased after a few tens of steps, the second needs of the order of $1/(1-\beta_2) = 1000$ steps – the memory of Section 4.10 with $\rho \to \beta_2$. Stationarity is the assumption that makes the correction exact; in a real run the gradient changes as the iterate moves, and a residual bias remains, small because the weights on old gradients are small. Bias correction is what makes Adam stable in its first iterations, and it is the one ingredient AdaGrad and RMSProp lack.

The update.

Finally,

$$\boxed{\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\alpha}{\sqrt{\hat{\boldsymbol{v}}_t} + \varepsilon}\,\hat{\boldsymbol{m}}_t,} \tag{4.71}$$

with ε a small constant, typically 10^{-8}, preventing division by zero. Step by step: compute the gradient; update the two moving averages (4.67) and (4.68); bias-correct with Eq. (4.70); form the step $\Delta\boldsymbol{\theta}_t = \hat{\boldsymbol{m}}_t/(\sqrt{\hat{\boldsymbol{v}}_t} + \varepsilon)$; and update the parameters.

The size of an Adam step.

Two properties of the update (4.71) follow from the algebra and explain much of the method's behaviour in practice. First, at $t = 1$ the raw moments are $\boldsymbol{m}_1 = (1-\beta_1)\boldsymbol{g}_1$ and $\boldsymbol{v}_1 = (1-\beta_2)\boldsymbol{g}_1^2$, and the correction (4.70) divides by exactly these factors, so $\hat{\boldsymbol{m}}_1 = \boldsymbol{g}_1$ and $\hat{\boldsymbol{v}}_1 = \boldsymbol{g}_1^2$. The first step is then

$$\Delta\theta_{1,j} = -\alpha\,\frac{g_{1,j}}{\sqrt{g_{1,j}^2} + \varepsilon} = -\alpha\,\frac{g_{1,j}}{|g_{1,j}| + \varepsilon} = -\alpha\,\mathrm{sign}(g_{1,j}) \qquad (|g_{1,j}| \gg \varepsilon),$$

a move of length α in every coordinate, whatever the gradient – a gradient of 10^{-3} and one of 10^3 produce the same first step, as for AdaGrad in Eq. (4.62). Without the correction the raw moments would enter the update directly,

$$\begin{aligned}\Delta\theta_{1,j} &= -\alpha\,\frac{(1-\beta_1)\,g_{1,j}}{\sqrt{1-\beta_2}\,|g_{1,j}|} = -\frac{1-\beta_1}{\sqrt{1-\beta_2}}\,\alpha\,\mathrm{sign}(g_{1,j}) \\ &= -\frac{0.1}{0.0316}\,\alpha\,\mathrm{sign}(g_{1,j}) \approx -3.2\,\alpha\,\mathrm{sign}(g_{1,j})\end{aligned}$$

with the default constants: the raw first moment is too small by a factor 10, the raw second moment by 10^3, and the ratio inherits $\sqrt{10^3}/10 = 3.16$. RMSProp, which has no correction, starts with $v_1 = (1-\rho)g_1^2$ and Eq. (4.64) gives

$$\Delta\theta_{1,j} = -\gamma\,\frac{g_{1,j}}{\sqrt{(1-\rho)g_{1,j}^2 + \varepsilon}} \approx -\frac{\gamma}{\sqrt{1-\rho}}\,\mathrm{sign}(g_{1,j}),$$

a step of length 10γ for $\rho = 0.99$ and 3.2γ for $\rho = 0.9$, which only settles to $\gamma\,\mathrm{sign}(g)$ after the $1/(1-\rho)$ steps it takes the weights of Eq. (4.65) to sum to about one: one more reason for

the correction. Second, at every step the magnitude of the update is bounded: since $\hat{\boldsymbol{m}}_t$ is a weighted average of the gradients $\boldsymbol{g}_1, \ldots, \boldsymbol{g}_t$ and $\hat{\boldsymbol{v}}_t$ a weighted average of their squares, the ratio $|\hat{m}_{t,j}|/\sqrt{\hat{v}_{t,j}}$ is at most $(1-\beta_1)/\sqrt{1-\beta_2}$ in the worst case and at most one in the common case where the two sets of weights are comparable, so

$$|\Delta\theta_{t,j}| \lesssim \alpha. \tag{4.72}$$

The learning rate of Adam is a *trust region*: the largest distance any parameter moves in one iteration, in the units of that parameter. This is why the defaults transfer between problems as gradient descent's γ never does, why Adam cannot diverge in the sense of Eq. (4.20) however α is chosen (on the polynomial fit of Section 4.4 it converges for every α between 10^{-3} and 1, with the best value in the middle), and why its default $\alpha = 10^{-3}$ is a hundred times smaller than a typical gradient descent learning rate.

The first moment is also what lets Adam settle where RMSProp cannot. Near a minimum the gradient alternates in sign from step to step, so its average $\hat{\boldsymbol{m}}_t$ is much smaller than its typical magnitude $\sqrt{\hat{\boldsymbol{v}}_t}$, and the ratio in Eq. (4.71) goes to zero even at a constant α; the fixed-length step of Eq. (4.66) has been averaged away. On the convex problems of this chapter Adam therefore converges to rounding while RMSProp rattles. It should be added that the convergence proof in the original paper contains an error, and that Reddi, Kale and Kumar [21] constructed simple convex problems on which Adam does not converge, because the effective learning rate $\alpha/\sqrt{\hat{\boldsymbol{v}}_t}$ can *increase* when a large gradient is forgotten; their repair, AMSGrad, keeps the running maximum of $\hat{\boldsymbol{v}}_t$ instead. In practice the failure is rare and Adam as stated remains the default.

Adam against its predecessors.

AdaGrad uses per-coordinate scaling like Adam but has no momentum, and slows down excessively because its accumulation never forgets. RMSProp uses a moving average of squared gradients, so it does not slow down, but includes neither momentum nor bias correction, and by Eq. (4.66) cannot settle at a constant learning rate. Adam is, in effect, RMSProp plus momentum plus bias correction: the first moment provides acceleration, smoother convergence and the ability to settle, the second moderates the step size per dimension, and the correction ensures the estimates are sound from the first iteration.

Why these methods work: the summary argument. It is worth collecting the thread that runs through Sections 4.5 to 4.11, because each method is a response to the same diagnosis.

The trouble with gradient descent is the mismatch between one global step size and a cost function whose curvature varies by direction; the iteration count scales with $\kappa = \lambda_{\max}/\lambda_{\min}$ by Eq. (4.26). Newton's method removes the mismatch entirely by premultiplying with $\boldsymbol{H}^{-1}$, but costs $\mathcal{O}(p^3)$.

Momentum attacks the symptom: it cancels the oscillation along steep directions and accumulates progress along flat ones, improving the rate from κ to $\sqrt{\kappa}$ at no extra cost. It does not, however, use different step sizes in different coordinates.

The adaptive methods attack the cause, by constructing a cheap diagonal approximation to the curvature from the second moment of the gradient. Where gradients are persistently large the curvature is presumed large and the step is shortened; where they are small the step is lengthened. Dividing by $\sqrt{v_j}$ is an $\mathcal{O}(p)$ stand-in for the $\mathcal{O}(p^3)$ operation of applying $\boldsymbol{H}^{-1/2}$.

Two observations follow. The first is that these methods are approximately invariant to the scaling of individual features, since multiplying a feature by c multiplies its gradient by c and its accumulated second moment by c^2, leaving the ratio unchanged. That is why they are so much less sensitive to preprocessing than plain gradient descent – although, as Section 4.15 notes, standardising the inputs remains good practice. The second is that a diagonal approximation can only capture curvature aligned with the coordinate axes. A cost function whose narrow valley runs diagonally in parameter space has a Hessian with large off-diagonal entries, and no diagonal rescaling will fix it; this is the residual gap between Adam and true second-order methods, and it is why decorrelating the inputs helps even when the scales already match.

4.12 Implementations

The three algorithms are described in detail in Goodfellow, Bengio and Courville [22], chapter 8. We give compact implementations here, all applied to the same least-squares problem of Section 4.4 so that they may be compared directly.

```
import numpy as np

def make_batches(n, batch_size, rng):
    """Shuffle the indices and split them into minibatches."""
    idx = rng.permutation(n)
    return [idx[i:i + batch_size] for i in range(0, n, batch_size)]


def sgd_adaptive(X, y, method="adam", n_epochs=100, batch_size=10,
                 gamma=0.01, beta=0.9, rho=0.99,
                 beta1=0.9, beta2=0.999, eps=1e-8, rng=None):
    """Stochastic gradient descent with the optimisers of this chapter.

    method is one of "plain", "momentum", "adagrad", "rmsprop", "adam".
    """
    rng = np.random.default_rng() if rng is None else rng
    n, p = X.shape
    theta = rng.normal(size=(p, 1))

    change = np.zeros((p, 1))          # momentum velocity, Eq. (4.momentum)
    r = np.zeros((p, 1))               # accumulated second moment
    m = np.zeros((p, 1))               # first moment, Eq. (4.adamfirst)
    t = 0

    for epoch in range(n_epochs):
        for batch in make_batches(n, batch_size, rng):
            t += 1
            Xb, yb = X[batch], y[batch]
            g = (2.0 / len(batch)) * Xb.T @ (Xb @ theta - yb)

            if method == "plain":
                update = gamma * g
            elif method == "momentum":
                change = gamma * g + beta * change
                update = change
            elif method == "adagrad":
                r += g * g                                  # Eq. (4.adagradaccum)
                update = gamma * g / (np.sqrt(r) + eps)       # Eq. (4.adagrad)
            elif method == "rmsprop":
                r = rho * r + (1 - rho) * g * g             # Eq. (4.rmspropaccum)
                update = gamma * g / (np.sqrt(r) + eps)       # Eq. (4.rmsprop)
```

```
            elif method == "adam":
                m = beta1 * m + (1 - beta1) * g              # Eq. (4.adamfirst)
                r = beta2 * r + (1 - beta2) * g * g          # Eq. (4.adamsecond)
                m_hat = m / (1 - beta1**t)                   # Eq. (4.adambias)
                r_hat = r / (1 - beta2**t)
                update = gamma * m_hat / (np.sqrt(r_hat) + eps)
            else:
                raise ValueError(f"unknown method {method}")

            theta -= update

    return theta
```

The same five updates can be written as a single *step function* that knows nothing about data, epochs or minibatches: it receives a gradient and returns the new parameters, keeping its running quantities in a dictionary. The data loop of the previous listing, or the full-gradient loop of Section 4.4, then calls it once per gradient. This is the structure we recommend for the projects, because it separates the three things that can go wrong – the cost, its gradient, and the optimiser – and lets an analytical gradient and `jax.grad` of Section 4.14 be exchanged without touching the optimiser.

```
import numpy as np

def optimiser_step(method, theta, g, state, t, gamma, beta=0.9, rho=0.99,
                   beta1=0.9, beta2=0.999, eps=1e-8):
    """One update of theta from the gradient g at step t = 1, 2, ...; state carries the
    running quantities."""
    if method == "plain":                                                    # Eq. (4.gd)
        return theta - gamma * g, state
    if method == "momentum":                                                 # Eq.
    (4.momentum)
        state["v"] = v = beta * state.get("v", 0.0) + gamma * g
        return theta - v, state
    if method == "adagrad":                                                  # Eqs.
    (4.adagradaccum), (4.adagrad)
        state["r"] = r = state.get("r", 0.0) + g * g
        return theta - gamma * g / (np.sqrt(r) + eps), state
    if method == "rmsprop":                                                  # Eqs.
    (4.rmspropaccum), (4.rmsprop)
        state["r"] = r = rho * state.get("r", 0.0) + (1.0 - rho) * g * g
        return theta - gamma * g / (np.sqrt(r) + eps), state
    if method == "adam":                                                     # Eqs.
    (4.adamfirst)-(4.adam)
        state["m"] = m = beta1 * state.get("m", 0.0) + (1.0 - beta1) * g
        state["r"] = r = beta2 * state.get("r", 0.0) + (1.0 - beta2) * g * g
        m_hat, r_hat = m / (1.0 - beta1**t), r / (1.0 - beta2**t)
        return theta - gamma * m_hat / (np.sqrt(r_hat) + eps), state
    raise ValueError(f"unknown method {method}")

def optimise(grad, theta0, method, gamma, num_iters=1000, tol=1e-8, **kw):
    """Full-gradient loop around optimiser_step; returns every iterate."""
    theta, state = np.array(theta0, dtype=float), {}
    history = [theta.copy()]
    for t in range(1, num_iters + 1):
        g = grad(theta)
        theta, state = optimiser_step(method, theta, g, state, t, gamma, **kw)
        history.append(theta.copy())
        if np.linalg.norm(g) < tol:
            break
    return np.array(history)
```

Running all five on the data of Section 4.4 for a hundred epochs with batches of ten, for three values of the learning rate, gives the final cost values of Table 4.4. The analytical solution of Eq. (3.8) is $\hat{\boldsymbol{\theta}} = (4.030, 2.806)$ with cost 1.14947, which is the floor.

Method	$\gamma = 0.01$	$\gamma = 0.1$	$\gamma = 0.5$
Plain SGD	1.1499	1.1532	1.2274
Momentum	1.1553	1.5674	1.3490
AdaGrad	28.049	1.2776	**1.1497**
RMSProp	1.1516	1.1512	1.1904
Adam	1.2185	1.1514	1.1565

Table 4.4 Final value of the cost function (4.12) after 100 epochs of stochastic gradient descent on the data of Section 4.4, for each optimiser and three learning rates. The analytical minimum is 1.14947. Note that no row is best at the same γ as any other.

The table repays study, and not because it flatters the adaptive methods.

The first observation is that on this problem the sophisticated methods are not better. With $\gamma = 0.01$ plain SGD reaches 1.1499, within 0.04% of the optimum, while Adam manages only 1.2185 and AdaGrad is catastrophic at 28.05 – it has not arrived anywhere near the minimum. This is exactly what Section 4.9 predicted: the accumulated $\boldsymbol{r}_t$ drives the effective step $\gamma/\sqrt{r_t}$ towards zero, and with γ already small the iterate stalls long before reaching the solution. Nothing is wrong with the implementation; the method is behaving as designed, and the design is wrong for this problem.

The second observation explains the first. Look along the rows rather than down the columns. AdaGrad is worst at $\gamma = 0.01$ and *best of all five* at $\gamma = 0.5$, where it essentially attains the analytical minimum. Plain SGD and momentum move the other way, degrading as γ grows. The reason is that these methods do not use γ to mean the same thing. Plain gradient descent multiplies γ by the raw gradient, so the stability bound (4.20) applies directly. The adaptive methods divide by $\sqrt{v_j}$ first, which renormalises the gradient to something of order unity, so their γ sets a step length in parameter space rather than a multiple of the gradient. *A learning rate is not comparable across optimisers*, and a comparison at a single fixed γ – which is what one sees most often – says more about which method happens to suit that number than about the methods.

The third observation is the one to carry forward. This problem has $p = 2$, is convex, and has a Hessian with eigenvalues 0.311 and 3.728, hence $\kappa = 11.97$ by Eq. (4.26). That is a benign landscape, and there is simply nothing for the adaptive machinery to repair; the overhead of estimating curvature buys nothing because the curvature is already uniform. The adaptive methods earn their place when κ is large, when p is large, when the gradients are sparse, and when the cost is non-convex – which is to say in the neural networks of the following chapters, and not here. Demonstrating a method on a problem it was not designed for is a good way to understand what it actually does.

Figure 4.9 plots the same comparison as a function of the epoch rather than as a final number, and it makes the second lesson of the table unmistakable. At $\gamma = 0.01$ the adaptive methods are slower than plain stochastic gradient descent and AdaGrad has effectively stopped; at $\gamma = 0.5$ the ordering is reversed and AdaGrad is the best of the five. Nothing about the methods has changed between the panels. Only the number γ has, and it means something different to each of them.

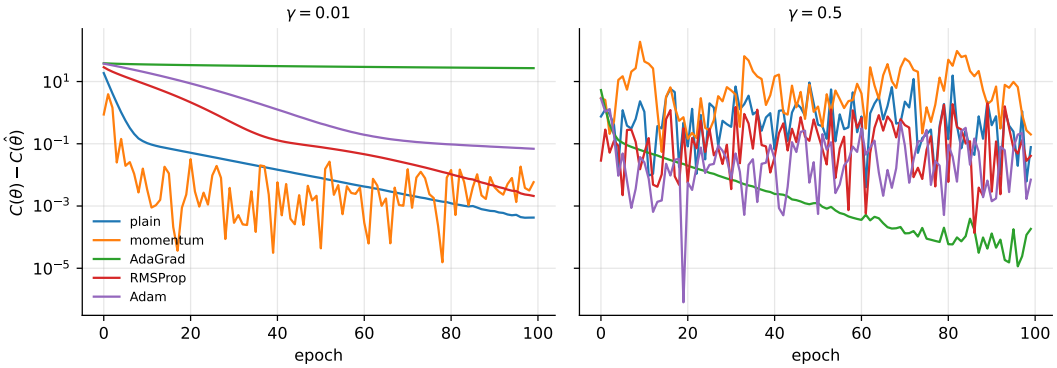


Fig. 4.9 Excess cost against epoch for the five optimisers of Section 4.12 at two learning rates. The vertical axis is $C(\boldsymbol{\theta})$ minus its value at the analytical minimum. The ranking of the methods reverses between the panels.

Machine learning connection. The default hyperparameters $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$ and $\alpha = 10^{-3}$ from the original paper are used almost universally and are usually a reasonable starting point, which is a large part of Adam's popularity. It should be said, however, that adaptive methods do not always generalise as well as plain SGD. Several studies have found that models trained with Adam, RMSProp or AdaGrad reach a worse test error than the same models trained with well-tuned SGD with momentum, particularly in the overparameterised regime where the number of parameters exceeds the number of data points. Why this should be so is not settled. The practical advice is that Adam is an excellent default for getting a model training at all, and that a carefully tuned SGD with momentum is worth trying before the final result is reported.

4.13 None of these can compete with Newton's method

It is a useful corrective, having built up this machinery, to see how it fares against Eq. (4.8) on a problem where the Hessian is affordable. For the least-squares cost of Section 4.4 a single Newton step lands on the exact minimum, by Eq. (4.9), while the methods above need hundreds or thousands of iterations to get five decimal places.

```
import numpy as np

# One Newton step solves the least-squares problem exactly
H = (2.0 / n) * X.T @ X
theta = rng.normal(size=(2, 1))
gradient = (2.0 / n) * X.T @ (X @ theta - y)
theta -= np.linalg.solve(H, gradient)
print("after one Newton step:", theta.ravel())
print("analytical solution:  ", (np.linalg.pinv(X.T @ X) @ X.T @ y).ravel())
```

The two agree to machine precision. The moral is not that we should use Newton's method – for a quadratic cost we would use the closed form of Chapter 3, and for a large model we could not afford the Hessian at all – but that the gradient methods of this chapter are *approximations* to something better, adopted because of cost. Every improvement from momentum to Adam is a step back towards Eq. (4.8), recovering a little more curvature information for a little more bookkeeping, and the quality of a method can fairly be judged by how much of the Hessian it manages to imitate for $\mathcal{O}(p)$ work per step.

4.14 Automatic differentiation

Every method in this chapter needs a gradient. For linear regression we computed it by hand in Eq. (4.13); for a neural network with a dozen layers, doing so by hand is possible but error-prone, and for a model under active development it is impractical. *Automatic differentiation* (AD), also called algorithmic differentiation, solves the problem completely, and this section explains how. We first say precisely what AD is not, then derive its two modes from the chain rule, prove the one result about cost that makes deep learning feasible, and finally show how the same ideas are used from Python through the `JAX` library.

AD exploits the fact that every computer program, however complicated, executes a sequence of elementary arithmetic operations and elementary functions – addition, multiplication, $\exp$, $\log$, $\sin$ – each of whose derivatives is known. Applying the chain rule of Eq. (1.51) repeatedly to that sequence yields derivatives of arbitrary order, *accurate to working precision*, at a cost only a small constant factor above that of the original program. Both halves of that sentence – the accuracy and the cost – are theorems, and we shall prove them.

4.14.1 Three ways to differentiate a program

It is important to distinguish AD from the two things it is often confused with.

Symbolic differentiation.

A symbolic differentiator manipulates expressions: it takes the formula for f and returns the formula for f'. This is what a computer-algebra system does and it is exact, but it requires the program to be a single closed-form expression in the first place – no loops, no branches, no intermediate variables – and it suffers from *expression swell*: the derivative of a product of m factors has m terms, the derivative of that has m^2, and the size of the formula grows without any of the sharing of intermediate results that the original program enjoyed.

Numerical differentiation.

A numerical differentiator replaces the derivative by a difference quotient. For a function of one variable the central difference is

$$f'(x) \approx D_h f(x) = \frac{f(x+h) - f(x-h)}{2h}, \tag{4.73}$$

and its error can be analysed exactly. Expanding both terms in Taylor series about x and subtracting, all even powers of h cancel and

$$D_h f(x) = f'(x) + \frac{h^2}{6} f'''(x) + \mathcal{O}(h^4), \tag{4.74}$$

the *truncation error*, which shrinks as h^2. This suggests taking h as small as possible. But the two function values are computed in floating point, each with a relative error of order the unit roundoff $\varepsilon_M \approx 2.2 \times 10^{-16}$ of Section 1.12, so the numerator carries an absolute error of order $\varepsilon_M |f(x)|$ and the quotient an error of order $\varepsilon_M |f(x)|/h$: the *rounding error*, which *grows* as h shrinks. This is catastrophic cancellation, the subtraction of two nearly equal numbers. The total error is therefore of the form

$$E(h) \approx \frac{h^2}{6}\left|f'''(x)\right| + \frac{\varepsilon_M |f(x)|}{h}, \tag{4.75}$$

and setting $dE/dh = 0$ gives the optimal step and the best attainable accuracy,

$$h^* = \left(\frac{3\varepsilon_M |f|}{|f'''|}\right)^{1/3} \sim \varepsilon_M^{1/3} \approx 6\times 10^{-6}, \qquad E(h^*) \sim \varepsilon_M^{2/3} \approx 4\times 10^{-11}. \tag{4.76}$$

A central difference can never deliver more than about two thirds of the available digits, and only if h is chosen well; the one-sided difference $(f(x+h)-f(x))/h$ has truncation error $\mathcal{O}(h)$ and does correspondingly worse, $E \sim \varepsilon_M^{1/2} \approx 10^{-8}$. Figure 4.12 shows the curve (4.75) measured for a concrete function. And the cost is the second objection: for a function of p variables the gradient needs $2p$ evaluations, or $p+1$ for one-sided differences, and for $p = 10^7$ that is a week of computing to obtain one, mediocre, gradient.

Automatic differentiation.

AD has neither defect. It computes the numerical value of the derivative at a point, not a formula, so there is no expression swell; it does so by propagating exact derivative rules through the program, so there is no truncation error and no step h to choose; and, in the mode used for training, it obtains all p components of the gradient for the price of a small constant number of evaluations of f. Reverse-mode AD is what backpropagation is. For $p = 10^7$ the difference against finite differences is between a fraction of a second and a week, and it is the reason the models of the later chapters can be trained at all.

4.14.2 Evaluation traces and the computational graph

The object AD works on is not the formula for f but the sequence of operations a program executes to evaluate it. Consider the function

$$f(x_1, x_2) = \ln x_1 + x_1 x_2 - \sin x_2, \tag{4.77}$$

which a program evaluates as the following list of assignments, each applying one elementary operation φ_i to variables already computed:

$$\begin{aligned} v_{-1} &= x_1, & v_0 &= x_2, & & \\ v_1 &= \ln v_{-1}, & v_2 &= v_{-1}\, v_0, & v_3 &= \sin v_0, \\ v_4 &= v_1 + v_2, & v_5 &= v_4 - v_3, & y &= v_5. \end{aligned} \tag{4.78}$$

This is the *evaluation trace*, or Wengert list, of f [23,24]: the inputs are numbered $v_{1-n},\dots,v_0$, the intermediate results $v_1,\dots,v_\ell$ in the order in which they are computed, and the output is the last of them. Drawn as a directed graph with an edge $j \to i$ whenever v_i depends directly on v_j, it is the *computational graph* of Fig. 4.10. Any program made of elementary operations has such a trace – loops unroll into repeated assignments, and a branch simply selects which assignments are executed for the given input – and that is what makes AD applicable to programs rather than to formulae. A neural network is a particularly regular trace: the matrix products, additions and activation functions of Chapter 8, one layer after another.

For each node the derivative of the elementary operation with respect to each of its arguments – its *local* or *partial* derivative – is known in closed form:

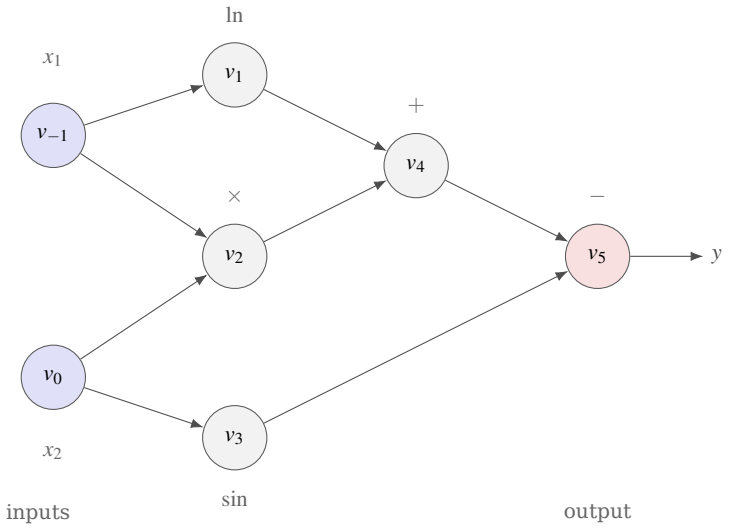


Fig. 4.10 The computational graph of the evaluation trace (4.78) for $f(x_1,x_2) = \ln x_1 + x_1 x_2 - \sin x_2$. Each node is one elementary operation applied to its predecessors. Forward-mode AD sweeps this graph from left to right, carrying a derivative $\dot{v}_i$ with every value; reverse-mode AD first evaluates the values from left to right and then sweeps back from right to left, carrying an adjoint $\bar{v}_i = \partial y/\partial v_i$ with every node.

$$\frac{\partial v_1}{\partial v_{-1}} = \frac{1}{v_{-1}}, \quad \frac{\partial v_2}{\partial v_{-1}} = v_0, \quad \frac{\partial v_2}{\partial v_0} = v_{-1}, \quad \frac{\partial v_3}{\partial v_0} = \cos v_0, \quad \frac{\partial v_4}{\partial v_1} = \frac{\partial v_4}{\partial v_2} = 1, \quad \frac{\partial v_5}{\partial v_4} = 1, \quad \frac{\partial v_5}{\partial v_3} = -1. \tag{4.79}$$

Everything AD does is to combine these local derivatives by the chain rule. The only question is in which order, and there are two natural answers.

4.14.3 Forward mode: tangents and dual numbers

Fix a direction $\dot{\boldsymbol{x}} \in \mathbb{R}^n$ in the input space and define for every node its *tangent*, the directional derivative

$$\dot{v}_i = \sum_{k=1}^{n} \frac{\partial v_i}{\partial x_k}\,\dot{x}_k = \frac{d}{d\varepsilon}\, v_i(\boldsymbol{x} + \varepsilon \dot{\boldsymbol{x}})\Big|_{\varepsilon=0}. \tag{4.80}$$

Since $v_i = \varphi_i(v_j : j \prec i)$ depends only on its predecessors, the chain rule gives the *forward recursion*

$$\dot{v}_i = \sum_{j \prec i} \frac{\partial \varphi_i}{\partial v_j}\,\dot{v}_j, \qquad i = 1,\dots,\ell, \tag{4.81}$$

started from $\dot{v}_{1-n},\dots,\dot{v}_0$ set equal to the chosen direction $\dot{x}_1,\dots,\dot{x}_n$. The recursion runs in the same order as the evaluation itself, so each tangent can be computed alongside its value, and when the sweep reaches the output it holds $\dot{y} = \nabla f(\boldsymbol{x})^T \dot{\boldsymbol{x}}$. For our example with $\dot{\boldsymbol{x}} = (1,0)$, that is $\partial f/\partial x_1$, the sweep at $(x_1,x_2) = (2,5)$ reads

$$\begin{aligned} &\dot{v}_{-1} = 1, && \dot{v}_0 = 0, \\ &\dot{v}_1 = \dot{v}_{-1}/v_{-1} = 0.5, && \dot{v}_2 = \dot{v}_{-1} v_0 + v_{-1}\dot{v}_0 = 5, \quad \dot{v}_3 = \cos(v_0)\,\dot{v}_0 = 0, \\ &\dot{v}_4 = \dot{v}_1 + \dot{v}_2 = 5.5, && \dot{v}_5 = \dot{v}_4 - \dot{v}_3 = 5.5, \end{aligned} \tag{4.82}$$

and indeed $\partial f/\partial x_1 = 1/x_1 + x_2 = 5.5$. To obtain $\partial f/\partial x_2$ the whole sweep is repeated with $\dot{\boldsymbol{x}} = (0,1)$.

Dual numbers.

There is an elegant algebraic way to say the same thing. Introduce a symbol ε with $\varepsilon^2 = 0$ (but $\varepsilon \neq 0$) and consider *dual numbers* $a + b\varepsilon$ with $a, b \in \mathbb{R}$. They add componentwise and multiply as

$$(a + b\varepsilon)(c + d\varepsilon) = ac + (ad + bc)\varepsilon, \tag{4.83}$$

because the ε^2 term vanishes. Now let g be any function with a Taylor series and evaluate it at a dual argument. Every power of ε beyond the first is zero, so the series terminates:

$$g(a + b\varepsilon) = g(a) + g'(a)\, b\varepsilon + \tfrac{1}{2} g''(a)\, b^2\varepsilon^2 + \cdots = g(a) + g'(a)\, b\varepsilon. \tag{4.84}$$

The dual part is exactly the tangent (4.81) of g: dual numbers carry a value and a derivative together, and the multiplication rule (4.83) *is* the product rule. Applying g to $x + 1 \cdot \varepsilon$ and reading off the coefficient of ε therefore returns $g'(x)$ exactly – not approximately, since nothing was truncated: the series terminates by the algebra of ε, not by neglecting small terms. This is forward-mode AD in one line, and it can be implemented in a few more. The following class overloads the arithmetic operators of Python so that ordinary code, written with no thought of derivatives, computes them when handed a dual argument.

```
import math

class Dual:
    """A dual number a + b*eps with eps**2 = 0: a value and its derivative."""
    def __init__(self, val, dot=0.0):
        self.val, self.dot = val, dot
    def _lift(o):                          # promote plain floats to duals
        return o if isinstance(o, Dual) else Dual(o)
    def __add__(self, o):
        o = Dual._lift(o); return Dual(self.val + o.val, self.dot + o.dot)
    __radd__ = __add__
    def __sub__(self, o):
        o = Dual._lift(o); return Dual(self.val - o.val, self.dot - o.dot)
    def __mul__(self, o):                  # the product rule, Eq. (4.dualmult)
        o = Dual._lift(o)
        return Dual(self.val * o.val, self.val * o.dot + self.dot * o.val)
    __rmul__ = __mul__
    def __truediv__(self, o):              # the quotient rule
        o = Dual._lift(o)
        return Dual(self.val / o.val,
                    (self.dot * o.val - self.val * o.dot) / o.val**2)
    def __pow__(self, k):
        return Dual(self.val**k, k * self.val**(k - 1) * self.dot)

# elementary functions: value and local derivative, Eq. (4.dualtaylor)
def sin(d): return Dual(math.sin(d.val), math.cos(d.val) * d.dot)
def exp(d): return Dual(math.exp(d.val), math.exp(d.val) * d.dot)
def log(d): return Dual(math.log(d.val), d.dot / d.val)

def f(x1, x2):                             # Eq. (4.adexample), ordinary code
    return log(x1) + x1 * x2 - sin(x2)

# seed the tangent of the variable we differentiate with respect to
print("df/dx1 =", f(Dual(2.0, 1.0), Dual(5.0, 0.0)).dot)   # 1/x1 + x2 = 5.5
print("df/dx2 =", f(Dual(2.0, 0.0), Dual(5.0, 1.0)).dot)   # x1 - cos(x2)
print("exact  =", 0.5 + 5.0, 2.0 - math.cos(5.0))
```

Jacobian–vector products and the cost of forward mode.

For a vector-valued $\boldsymbol{f}:\mathbb{R}^n\to\mathbb{R}^m$ with Jacobian $\boldsymbol{J}\in\mathbb{R}^{m\times n}$ as in Eq. (1.16), one forward sweep with seed $\dot{\boldsymbol{x}}$ returns

$$\dot{\boldsymbol{y}} = \boldsymbol{J}\,\dot{\boldsymbol{x}}, \tag{4.85}$$

a *Jacobian–vector product* (JVP), without ever forming $\boldsymbol{J}$. Seeding with the unit vector $\boldsymbol{e}_k$ returns the k-th column of the Jacobian, so the full Jacobian costs n sweeps and the gradient of a scalar function of p parameters costs p sweeps – the same count as finite differences, only exact. Each sweep costs a small multiple of one function evaluation, since Eq. (4.81) adds to each elementary operation a bounded number of further ones. Forward mode is therefore the right tool when the number of *inputs* is small: the sensitivity of a whole network output to one physical parameter, or the derivatives with respect to one or two spatial coordinates that Chapter 9 needs when a network is made to satisfy a differential equation. It is the wrong tool for training, where p is enormous and the output is a single scalar.

4.14.4 Reverse mode: adjoints

Reverse mode fixes the *output* instead of the input. For a scalar output y define the *adjoint* of every node,

$$\bar{v}_i = \frac{\partial y}{\partial v_i}, \tag{4.86}$$

the sensitivity of the final result to that intermediate value. Since y depends on v_j only through the nodes v_i that consume it, the chain rule gives the *reverse recursion*

$$\bar{v}_j = \sum_{i\succ j} \bar{v}_i\,\frac{\partial \varphi_i}{\partial v_j}, \qquad j=\ell-1,\dots,1-n, \tag{4.87}$$

started from $\bar{v}_\ell = \partial y/\partial y = 1$. Compare this with the forward recursion (4.81): the same local derivatives appear, but the sum now runs over the *successors* of a node and the recursion runs *backwards* through the trace, from output to inputs. When it reaches the inputs it holds $\bar{v}_{1-n},\dots,\bar{v}_0$, which is the whole gradient $\nabla f(\boldsymbol{x})$ – all n components from a single sweep. For our example, with the values from the forward evaluation at $(2,5)$,

$$\begin{aligned}
&\bar{v}_5 = 1,\\
&\bar{v}_4 = \bar{v}_5\cdot 1 = 1, && \bar{v}_3 = \bar{v}_5\cdot(-1) = -1,\\
&\bar{v}_1 = \bar{v}_4\cdot 1 = 1, && \bar{v}_2 = \bar{v}_4\cdot 1 = 1,\\
&\bar{v}_0 = \bar{v}_3\cos v_0 + \bar{v}_2 v_{-1} = 2-\cos 5 = 1.716, && \bar{v}_{-1} = \bar{v}_2 v_0 + \bar{v}_1/v_{-1} = 5+0.5 = 5.5,
\end{aligned} \tag{4.88}$$

which are $\partial f/\partial x_2$ and $\partial f/\partial x_1$, both obtained at once. The two contributions to $\bar{v}_0$ are the two paths from v_0 to y in Fig. 4.10: the chain rule sums over paths, and the reverse sweep organises that sum so that each edge is visited once.

Reverse mode has a price that forward mode does not: the local derivatives in Eq. (4.87) depend on the values v_j – $\cos v_0$, v_{-1}, $1/v_{-1}$ above – and those must be available when the backward sweep needs them, in the reverse of the order in which they were produced. The forward evaluation must therefore be recorded, on what is called the *tape*, and the memory cost of reverse mode is proportional to the length of the trace rather than to the number of variables. This is why training a deep network is memory-bound: every activation of every layer is stored until the backward pass has consumed it.

Vector–Jacobian products.

For a vector-valued function seeded with a covector $\bar{\boldsymbol{y}} \in \mathbb{R}^m$ the reverse sweep returns

$$\bar{\boldsymbol{x}}^T = \bar{\boldsymbol{y}}^T \boldsymbol{J}, \qquad \text{equivalently} \qquad \bar{\boldsymbol{x}} = \boldsymbol{J}^T \bar{\boldsymbol{y}}, \tag{4.89}$$

a *vector–Jacobian product* (VJP): one row of $\boldsymbol{J}$ per sweep, and the full Jacobian in m sweeps. Equation (4.89) is precisely Eq. (1.51), a gradient propagated backwards by multiplication with a transposed Jacobian, and it is why backpropagation was described in Section 1.8.7 as the chain rule with the products taken in the cheapest order.

The cheap gradient principle.

The result that makes all of deep learning possible is the following.

Proposition 4.2 (Cheap gradient principle). *Let $f : \mathbb{R}^p \to \mathbb{R}$ be evaluated by a trace of ℓ elementary operations, each of which has at most two arguments and whose local derivatives cost at most a fixed multiple of the operation itself. Then reverse-mode AD computes $f(\boldsymbol{x})$ and the complete gradient $\nabla f(\boldsymbol{x})$ with at most $c\,\ell$ operations, where $c \le 4$ is a constant* independent of p.

Proof. The forward evaluation costs ℓ operations and records the trace. In the reverse recursion (4.87) each edge $j \to i$ of the graph contributes exactly one term $\bar{v}_i \partial \varphi_i / \partial v_j$: one multiplication, one addition into $\bar{v}_j$, and the evaluation of the local derivative, which by hypothesis costs a bounded amount and often nothing at all (for $+$ it is 1, for $\times$ it is the other argument, already on the tape). Since each node has at most two incoming edges, the number of edges is at most 2ℓ and the reverse sweep costs at most a fixed multiple of 2ℓ operations. Adding the forward evaluation gives the bound, and nowhere did p enter: it determines how many adjoints are *read off* at the end, not how many operations are performed.

The result is due to Baur and Strassen [25] in the setting of arithmetic circuits and to Linnainmaa [26] and, independently, the backpropagation literature in the setting of programs; the survey by Baydin et al. [27] traces the history. In practice the constant is nearer 2–3 than 4: a gradient costs about as much as two or three function evaluations. Set against the $p+1$ evaluations of finite differences or the p sweeps of forward mode, this is the entire difference between what can and cannot be trained.

Machine learning connection. Backpropagation, the algorithm of Chapter 8 that trains every neural network, is Eq. (4.87) specialised to the layered trace of a network. The forward pass computes and stores the activations; the backward pass propagates the adjoint of the cost – the "error" of the older literature – from the output layer to the input, one transposed weight matrix at a time, exactly as in Eq. (4.89). Nothing in backpropagation is specific to networks: it is reverse-mode AD, and modern frameworks make no distinction.

4.14.5 Two examples worked by hand

The example of Eq. (4.77) had two inputs, which is what makes the difference between the two modes visible. It is nevertheless worth running both sweeps on two functions of a single variable, small enough that every number can be checked against the derivative one would write down by hand, because they show the two things a first encounter with AD tends to

obscure: that the sweeps only ever multiply and add numbers already on the tape, and that the reverse sweep has to *sum* whenever a value is used more than once.

A chain.

Take first

$$f(x) = \exp\bigl(x^2\bigr), \tag{4.90}$$

whose trace has a single input and two intermediate values,

$$v_0 = x, \qquad v_1 = v_0^2, \qquad v_2 = \exp v_1, \qquad y = v_2, \tag{4.91}$$

with local derivatives $\partial v_1/\partial v_0 = 2v_0$ and $\partial v_2/\partial v_1 = \exp v_1 = v_2$. The computational graph, upper panel of Fig. 4.11, is a chain: every node has one predecessor and one successor. The forward recursion (4.81) seeded with $\dot{v}_0 = 1$ reads

$$\dot{v}_1 = 2v_0\,\dot{v}_0 = 2x, \qquad \dot{v}_2 = v_2\,\dot{v}_1 = 2x\exp\bigl(x^2\bigr), \tag{4.92}$$

and the reverse recursion (4.87) started from $\bar{v}_2 = 1$ reads

$$\bar{v}_1 = \bar{v}_2\,v_2 = \exp\bigl(x^2\bigr), \qquad \bar{v}_0 = \bar{v}_1\,2v_0 = 2x\exp\bigl(x^2\bigr). \tag{4.93}$$

Both are the chain rule $df/dx = (df/dv_2)(dv_2/dv_1)(dv_1/dv_0)$; forward mode multiplies the three factors from the right, reverse mode from the left, and on a chain the two are indistinguishable in cost. Evaluating f takes two elementary operations, a square and an exponential. Either sweep adds two multiplications, and notice that $\exp(x^2)$ is not recomputed: it is v_2, read off the tape. At $x = 1$ all three agree on $f'(1) = 2e = 5.437$.

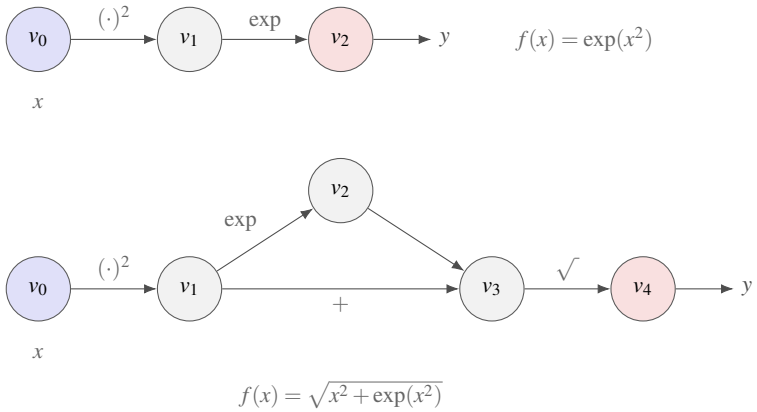


Fig. 4.11 The computational graphs of the two examples of Section 4.14.5. Top: $f(x) = \exp(x^2)$, trace (4.91), a chain in which every value is used once. Bottom: $f(x) = \sqrt{x^2 + \exp(x^2)}$, trace (4.95), where $v_1 = x^2$ feeds both the exponential and the sum. The two paths from v_1 to y are the two terms of the adjoint $\bar{v}_1$ in Eq. (4.97).

A fan-out.

The second function reuses an intermediate value,

$$f(x) = \sqrt{x^2 + \exp\bigl(x^2\bigr)}, \tag{4.94}$$

with the trace

$$v_0 = x, \qquad v_1 = v_0^2, \qquad v_2 = \exp v_1, \qquad v_3 = v_1 + v_2, \qquad v_4 = \sqrt{v_3}, \qquad y = v_4, \tag{4.95}$$

shown in the lower panel of Fig. 4.11. The node v_1 now has two successors, v_2 and v_3, and this is where the two modes part company. The forward sweep is unaffected – each node still collects tangents from its predecessors, and v_3 simply has two of them:

$$\dot v_1 = 2v_0, \qquad \dot v_2 = v_2\,\dot v_1, \qquad \dot v_3 = \dot v_1 + \dot v_2 = (1+v_2)\,\dot v_1, \qquad \dot v_4 = \frac{\dot v_3}{2v_4}. \tag{4.96}$$

The reverse sweep starts from $\bar v_4 = 1$ and walks back through the trace,

$$\begin{aligned} \bar v_3 &= \frac{\bar v_4}{2v_4} = \frac{1}{2\sqrt{v_3}}, & \bar v_2 &= \bar v_3 \cdot 1, \\ \bar v_1 &= \bar v_2\, v_2 + \bar v_3 \cdot 1 = \frac{1+v_2}{2\sqrt{v_3}}, & \bar v_0 &= \bar v_1\, 2v_0 = \frac{x\,(1+v_2)}{\sqrt{v_3}}, \end{aligned} \tag{4.97}$$

and the adjoint of v_1 is a *sum* of two terms, one through each successor, exactly as Eq. (4.87) prescribes: the sensitivity of y to v_1 is the sensitivity through the exponential plus the sensitivity through the direct addition. Substituting $v_2 = \exp(x^2)$ and $v_3 = x^2 + \exp(x^2)$ gives

$$f'(x) = \frac{x\bigl(1+\exp(x^2)\bigr)}{\sqrt{x^2+\exp(x^2)}}, \tag{4.98}$$

which is what the chain rule applied by hand produces. At $x = 1$ the tape holds $v_1 = 1$, $v_2 = 2.718$, $v_3 = 3.718$ and $v_4 = 1.928$, and both sweeps return $f'(1) = 3.718/1.928 = 1.928$. (That $f'(1)$ equals $f(1)$ is a coincidence of the point $x = 1$, where the numerator $x(1+v_2)$ happens to equal v_3.)

The bookkeeping is worth comparing. Evaluating f costs four elementary operations. The reverse sweep adds two for $\bar v_3$, two for $\bar v_1$ and two for $\bar v_0$, six in all, and every quantity it needs – v_2, v_4, v_0 – is already on the tape. Evaluating the closed form (4.98) on its own, without sharing anything with the evaluation of f, costs about eight operations and computes x^2 and $\exp(x^2)$ a second time; on a function with thousands of intermediate values the hand-derived expression grows far faster than the trace, which is the *expression swell* that AD avoids by never forming it. Both sweeps here are within the constant of Proposition 4.2, and the reader can check that the same holds for the example of Fig. 4.10.

The following listing runs the two examples through the `Dual` class of Section 4.14.3 for the forward sweep and through JAX for the reverse sweep (Section 4.14.8 says more about JAX), and compares both with Eq. (4.98). The functions are written exactly as one would write them without derivatives in mind.

```
import math
import jax
import jax.numpy as jnp
jax.config.update("jax_enable_x64", True)

def sqrt(d):                              # one more elementary function for Dual
    r = math.sqrt(d.val)
    return Dual(r, d.dot / (2.0 * r))

def f_chain(x):                           # Eq. (4.adchainfun)
    return exp(x**2)
```

```
def f_fanout(x):                          # Eq. (4.adfanoutfun)
    a = x**2
    return sqrt(a + exp(a))

x = 1.0
# forward mode: seed the tangent of x with 1 and read the dual part
print("chain,   forward:", f_chain(Dual(x, 1.0)).dot, " exact:", 2 * x * math.exp(x**2))
print("fan-out, forward:", f_fanout(Dual(x, 1.0)).dot)

# reverse mode: jax.grad is one reverse sweep, Eq. (4.adfanoutreverse)
def f_fanout_jax(x):
    a = x**2
    return jnp.sqrt(a + jnp.exp(a))

print("fan-out, reverse:", float(jax.grad(f_fanout_jax)(x)))
print("fan-out, closed: ", x * (1 + math.exp(x**2)) / math.sqrt(x**2 + math.exp(x**2)))
```

The three numbers agree to rounding, 1.92828468553247, differing at most in the last binary digit: nothing was approximated in either sweep.

4.14.6 Forward or reverse: the chain rule as a matrix product

The cleanest way to see why the two modes differ in cost is to write a program as a composition of L stages, $\boldsymbol{f} = \boldsymbol{f}_L \circ \boldsymbol{f}_{L-1} \circ \cdots \circ \boldsymbol{f}_1$, with $\boldsymbol{f}_k : \mathbb{R}^{n_{k-1}} \to \mathbb{R}^{n_k}$, $n_0 = n$ and $n_L = m$. The chain rule of Eq. (1.50) says that the Jacobian is a product of the stage Jacobians,

$$\boldsymbol{J} = \boldsymbol{J}_L \boldsymbol{J}_{L-1} \cdots \boldsymbol{J}_1, \qquad \boldsymbol{J}_k = \frac{\partial \boldsymbol{f}_k}{\partial \boldsymbol{x}_{k-1}} \in \mathbb{R}^{n_k \times n_{k-1}}. \tag{4.99}$$

Matrix multiplication is associative, so this product may be evaluated in any order, and the orders differ enormously in cost. Forward mode multiplies from the right, $\boldsymbol{J}\dot{\boldsymbol{x}} = \boldsymbol{J}_L(\cdots(\boldsymbol{J}_2(\boldsymbol{J}_1\dot{\boldsymbol{x}})))$, so every intermediate is a vector of length n_k and never a matrix. Reverse mode multiplies from the left, $\bar{\boldsymbol{y}}^T\boldsymbol{J} = (((\bar{\boldsymbol{y}}^T\boldsymbol{J}_L)\boldsymbol{J}_{L-1})\cdots)\boldsymbol{J}_1$, and again every intermediate is a vector. If the stage Jacobians are dense the cost of the right-to-left product is $\sum_k n_k n_{k-1}$ per column, hence $n\sum_k n_k n_{k-1}$ for the whole Jacobian, while the left-to-right product costs $m\sum_k n_k n_{k-1}$. The rule is therefore simple: use forward mode when $n \ll m$ and reverse mode when $m \ll n$. Training a model is the extreme case $m = 1$, $n = p$, and reverse mode wins by a factor p. When n and m are comparable, neither mode is optimal and finding the cheapest bracketing of Eq. (4.99) – the optimal Jacobian accumulation problem – is NP-hard, though good heuristics exist [24].

Second derivatives.

The two modes compose, and their composition gives cheap access to curvature. The Hessian of a scalar f is the Jacobian of its gradient, and the Hessian–vector product is a directional derivative of the gradient,

$$\boldsymbol{H}\boldsymbol{v} = \frac{d}{d\varepsilon}\nabla f(\boldsymbol{x} + \varepsilon \boldsymbol{v})\Big|_{\varepsilon=0}. \tag{4.100}$$

The right-hand side is a forward-mode derivative, in the direction $\boldsymbol{v}$, of a function – the gradient – that is itself computed by reverse mode. This *forward-over-reverse* construction costs a small constant times one gradient, hence a small constant times one function evaluation, and it never forms the $p \times p$ Hessian. That is exactly the price at which Section 4.13 said curvature information could *not* be had; the truncated-Newton and conjugate-gradient methods

of the optimisation literature [28] use Eq. (4.100) to run Newton's method (4.8) with only Hessian–vector products, and the natural-gradient methods of Chapter 8 rely on the same device. The full Hessian, when it is wanted, is p such products, $\boldsymbol{H} = \boldsymbol{H}[\boldsymbol{e}_1, \dots, \boldsymbol{e}_p]$, and this is how `jax.hessian` computes it.

4.14.7 Accuracy, and points where the derivative does not exist

We claimed that AD is accurate to working precision, and the claim can now be made precise. The recursions (4.81) and (4.87) are exact identities; the only errors are the rounding errors of the floating-point operations that implement them. Each tangent or adjoint is obtained from its predecessors by a short chain of multiplications and additions of quantities that were themselves computed to relative accuracy $\mathcal{O}(\varepsilon_M)$, so the derivative inherits the numerical stability of the function evaluation itself: if the program computes f to d correct digits it computes ∇f to about d correct digits as well. There is no step h, no cancellation of nearly equal numbers, and no term of the form ε_M/h. In Fig. 4.12 the AD derivative sits at the floor of the plot, at 10^{-16}, for the same reason that the function value does.

The one genuine limitation is that AD differentiates the program actually executed. At a point where the function is not differentiable – $|x|$ at zero, or the ReLU activation $\max(0,x)$ of Chapter 8 at $x = 0$ – the program takes one of the two branches and AD returns the derivative of that branch, typically 0 or 1 by convention. This is a valid *subgradient* and is harmless in practice, since the set of such points has measure zero and an iterate never lands on it exactly. More insidious is a program that computes a smooth function by a non-smooth route, for instance $\sqrt{x^2}$ for $|x|$ or a table lookup with interpolation, whose AD derivative is that of the route rather than of the function. Writing functions with AD in mind – using the library's own $\log(1+e^x)$, $\log\sum\exp$ and similar numerically careful primitives, whose derivative rules are hand-written – avoids both this and overflow in intermediate values.

4.14.8 Automatic differentiation in Python: autograd and JAX

Two libraries deliver the machinery above to ordinary numpy code. The older `autograd` package overloads the numpy functions with dual-number-like objects that record a tape, exactly as the `Dual` class above did for the standard library, and it remains a good way to see the mechanism at work:

```
import autograd.numpy as np
from autograd import grad

def f(x):
    return np.sin(2 * np.pi * x + x**2)

df = grad(f)                 # df is a Python function, the derivative of f
print(df(1.0))               # (2 pi + 2) cos(2 pi + 1) = 4.475424121402227

# It composes: the second derivative is just grad applied twice
d2f = grad(grad(f))
print(d2f(1.0))
```

For current work we recommend *JAX* [29]. JAX combines the same differentiation machinery with a compiler, XLA, so that the resulting code can additionally be just-in-time compiled, vectorised over a batch and run on GPUs and TPUs, and it exposes forward mode, reverse

mode and their compositions explicitly. Its four fundamental transformations are `grad`, `jit`, `vmap` and the pair `jvp` and `vjp`, and they compose freely.

Precision. JAX uses single precision by default. Every numerical comparison in this book is made in double precision, and the first line of every JAX program should therefore be `jax.config.update("jax_enable_x64", True)`. Without it the gradient checks below would agree only to about seven digits and one would wrongly conclude that AD is inexact.

```
import jax
jax.config.update("jax_enable_x64", True)   # double precision throughout
import jax.numpy as jnp
from jax import grad, jit, vmap

def f(x):
    return jnp.sin(2 * jnp.pi * x + x**2)

df = grad(f)                        # reverse-mode gradient of a scalar function
print(df(1.0))                      # 4.475424121402227, exact to all digits
print(grad(grad(f))(1.0))           # second derivative, by composing grad

xs = jnp.linspace(0.0, 1.0, 5)
print(vmap(df)(xs))                 # the derivative at five points at once
fast_df = jit(df)                   # compiled on first call, then very fast
print(fast_df(1.0))
```

Forward and reverse mode explicitly.

The two modes of Sections 4.14.3 and 4.14.4 are available directly. `jvp` takes a function, a point and a tangent direction and returns the value and the Jacobian–vector product of Eq. (4.85); `vjp` takes a function and a point and returns the value and a function that maps a covector to the vector–Jacobian product of Eq. (4.89). `jacfwd` and `jacrev` assemble the full Jacobian by the two routes of Section 4.14.6, one column or one row per sweep, and agree to rounding.

```
from jax import jvp, vjp, jacfwd, jacrev

def F(x):                           # a map from R^2 to R^3
    return jnp.array([x[0] * x[1], jnp.sin(x[0]), jnp.exp(x[1])])

x0 = jnp.array([1.0, 2.0])
J = jacfwd(F)(x0)                   # 3 x 2 Jacobian, built from two forward sweeps
print(J)
print(jnp.allclose(J, jacrev(F)(x0)))       # ...and from three reverse sweeps

v = jnp.array([1.0, 0.0])           # a direction in the input space
_, Jv = jvp(F, (x0,), (v,))         # forward mode: J v, first column of J
print(Jv)

u = jnp.array([1.0, 1.0, 1.0])      # a covector in the output space
_, vjp_fn = vjp(F, x0)
print(vjp_fn(u)[0])                 # reverse mode: u^T J, the column sums of J
```

The least-squares gradient without deriving it.

Applied to our least-squares problem, the gradient we derived by hand in Eq. (4.13) need never be written down. We write the cost (4.12) as a function of $\boldsymbol{\theta}$ and the data, differentiate with respect to its first argument, compile the result, and run the gradient descent iteration (4.15) with it.

```
import jax
jax.config.update("jax_enable_x64", True)
import jax.numpy as jnp
import numpy as np
from jax import grad, jit

# The data of Section 4.gdlinreg
n = 100
rng = np.random.default_rng(2024)
x = 2.0 * rng.random((n, 1))
y = 4.0 + 3.0 * x + rng.normal(size=(n, 1))
X = np.c_[np.ones((n, 1)), x]
Xj, yj = jnp.asarray(X), jnp.asarray(y)

def cost(theta, X, y):                      # Eq. (4.gdcost)
    return jnp.sum((X @ theta - y)**2) / len(y)

cost_grad = jit(grad(cost, argnums=0))      # d cost / d theta, compiled

# Check against the hand-derived gradient, Eq. (4.gdgradient)
theta = jnp.asarray(rng.normal(size=(2, 1)))
g_ad = cost_grad(theta, Xj, yj)
g_hand = (2.0 / n) * X.T @ (X @ np.asarray(theta) - y)
print("max |AD - hand| =", np.max(np.abs(np.asarray(g_ad) - g_hand)))

# Gradient descent, Eq. (4.gditeration), with the automatic gradient
gamma = 0.1
@jit
def gd_step(theta):
    return theta - gamma * grad(cost)(theta, Xj, yj)

for k in range(1000):
    theta = gd_step(theta)
print("gradient descent with AD:", np.asarray(theta).ravel())
theta_exact = np.linalg.pinv(X.T @ X) @ X.T @ y
print("analytical:              ", theta_exact.ravel())
```

The two gradients agree to 5×10^{-15}, which is rounding, and the iteration returns the analytical solution $\boldsymbol{\theta} = (4.03012, 2.80580)$. Comparing an analytical gradient against an automatically differentiated one in this way is worth doing once as a matter of habit: it is the standard way of catching an error in a derivation.

The same code differentiates the Ridge cost (4.16) – and, because `grad` can differentiate with respect to any argument, it also returns the derivative of the cost with respect to the penalty λ, which by Eq. (4.16) is $\boldsymbol{\theta}^T\boldsymbol{\theta}$, a quantity one might want when tuning hyperparameters by gradient methods.

```
def ridge_cost(theta, X, y, lmbda):         # Eq. (4.ridgecost)
    return jnp.sum((X @ theta - y)**2) / len(y) + lmbda * jnp.sum(theta**2)

lmbda = 0.001
theta = jnp.asarray(rng.normal(size=(2, 1)))
step = jit(lambda th: th - gamma * grad(ridge_cost)(th, Xj, yj, lmbda))
for k in range(1000):
    theta = step(theta)
```

```
I = np.eye(2)
theta_closed = np.linalg.inv(X.T @ X + n * lmbda * I) @ X.T @ y
print("gradient descent:", np.asarray(theta).ravel())
print("closed form:     ", theta_closed.ravel())

# The derivative with respect to lambda, argument number 3
dC_dlambda = grad(ridge_cost, argnums=3)(theta, Xj, yj, lmbda)
print(dC_dlambda, "=", jnp.sum(theta**2))    # theta^T theta
```

Hessians, Newton's method and Hessian–vector products.

`jax.hessian` is `jacfwd(jacrev(f))`, forward-over-reverse exactly as in Section 4.14.6. For the least-squares cost it returns $\frac{2}{n}\boldsymbol{X}^T\boldsymbol{X}$ of Eq. (4.14) to rounding, and one Newton step (4.8) with the automatic gradient and Hessian lands on the exact minimum, as Section 4.13 said it would. The Hessian–vector product of Eq. (4.100) is one line, and it never forms the matrix.

```
from jax import hessian, jvp

H = hessian(cost)(theta, Xj, yj).reshape(2, 2)     # forward-over-reverse
print(H)
print((2.0 / n) * X.T @ X)                         # Eq. (4.gdhessian)

# One Newton step from a random start, Eq. (4.newtonopt)
theta0 = jnp.asarray(rng.normal(size=(2, 1)))
theta_newton = theta0 - jnp.linalg.solve(H, grad(cost)(theta0, Xj, yj))
print("one Newton step:", np.asarray(theta_newton).ravel())

# Hessian-vector product without the Hessian, Eq. (4.hvp)
def hvp(f, theta, v):
    return jvp(grad(f), (theta,), (v,))[1]

v = jnp.array([[1.0], [0.0]])
print(hvp(lambda th: cost(th, Xj, yj), theta0, v).ravel(), (H @ v).ravel())
```

4.14.9 The optimisers of this chapter on a non-convex function

Everything so far has been demonstrated on the least-squares problem, where the gradient is known. The point of AD is to be able to optimise a function whose gradient one has *not* derived, and a classical test case is the Rosenbrock function [30]

$$f(x,y) = (1-x)^2 + 100\,(y-x^2)^2, \tag{4.101}$$

which has a unique minimum at $(1,1)$ with $f=0$ at the bottom of a long, curved, parabolic valley. It is not convex, and its Hessian at the minimum has eigenvalues 0.40 and 1001.6, hence $\kappa \approx 2500$: by Eq. (4.26) plain gradient descent will be slow, momentum should help by about $\sqrt{\kappa} \approx 50$, and Newton's method, which is invariant to the conditioning, should be fast. With the gradient and Hessian supplied by JAX, all four methods are a few lines each.

```
import jax
jax.config.update("jax_enable_x64", True)
import jax.numpy as jnp
import numpy as np
from jax import grad, jit, hessian
```

```
def rosenbrock(p):                                  # Eq. (4.rosenbrock)
    x, y = p
    return (1 - x)**2 + 100 * (y - x**2)**2

rgrad = jit(grad(rosenbrock))                       # nobody derived these
rhess = jit(hessian(rosenbrock))
p0 = jnp.array([-1.5, 2.0])

def run(stepper, tol=1e-10, max_iter=100000):
    """Iterate p, state = stepper(p, state, t) until f(p) < tol."""
    p, state = p0, None
    for t in range(1, max_iter + 1):
        p, state = stepper(p, state, t)
        if float(rosenbrock(p)) < tol:
            return t, np.asarray(p)
    return None, np.asarray(p)

def gd(p, state, t, gamma=1e-3):                      # Eq. (4.gd)
    return p - gamma * rgrad(p), state

def momentum(p, state, t, gamma=1e-3, beta=0.9):     # Eq. (4.momentum)
    v = jnp.zeros(2) if state is None else state
    v = beta * v + gamma * rgrad(p)
    return p - v, v

def adam(p, state, t, gamma=0.02, b1=0.9, b2=0.999, eps=1e-8):   # Eq. (4.adam)
    m, r = (jnp.zeros(2), jnp.zeros(2)) if state is None else state
    g = rgrad(p)
    m = b1 * m + (1 - b1) * g
    r = b2 * r + (1 - b2) * g * g
    m_hat, r_hat = m / (1 - b1**t), r / (1 - b2**t)
    return p - gamma * m_hat / (jnp.sqrt(r_hat) + eps), (m, r)

def newton(p, state, t):                              # Eq. (4.newtonopt)
    return p - jnp.linalg.solve(rhess(p), rgrad(p)), state

for name, stepper in [("gradient descent", gd), ("momentum", momentum),
                      ("Adam", adam), ("Newton", newton)]:
    steps, p = run(stepper)
    print(f"{name:17s} {steps:6d} iterations, p = {p}")
```

Started from $(-1.5, 2)$ and stopped when $f < 10^{-10}$, the four methods need

Method	**Iterations**
Gradient descent, $\gamma = 10^{-3}$	27 206
Momentum, $\gamma = 10^{-3}$, $\beta = 0.9$	2 591
Adam, $\gamma = 0.02$	2 934
Newton	6

iterations, and every argument of the chapter is visible in the four numbers. Gradient descent is limited by the largest curvature – the Hessian at the starting point has $\lambda_{\max} \approx 2100$, so by Eq. (4.20) γ must be below about 10^{-3} – and then crawls along the valley floor at the rate set by $\lambda_{\min}$. Momentum improves the count by a factor 10.5, in line with the $\sqrt{\kappa}$ of Eq. (4.33); Adam, with no tuning beyond γ, does about as well; and Newton's method, using the curvature that JAX supplies at the cost of a few gradient evaluations, converges quadratically in six steps. None of the derivative code was written by hand, and replacing Eq. (4.101) by a neural network with a million parameters would change only the definition of the function.

4.14.10 Finite differences against automatic differentiation

We close by measuring the error curve (4.75). For $f(x)=\sin(2\pi x+x^2)$ at $x=1$ the exact derivative is $(2\pi+2)\cos(2\pi+1)=4.475424121402227$, and the central difference (4.73) gives, for decreasing h:

```
import numpy as np

def f(x):
    return np.sin(2 * np.pi * x + x**2)

exact = (2 * np.pi + 2) * np.cos(2 * np.pi + 1)
for h in [1e-2, 1e-4, 1e-6, 1e-8, 1e-10, 1e-12]:
    fd = (f(1.0 + h) - f(1.0 - h)) / (2 * h)       # Eq. (4.centraldiff)
    print(f"h = {h:.0e}   error = {abs(fd - exact):.1e}")
```

```
h = 1e-02   error = 5.8e-03
h = 1e-04   error = 5.8e-07
h = 1e-06   error = 3.8e-10
h = 1e-08   error = 6.8e-09
h = 1e-10   error = 1.5e-06
h = 1e-12   error = 2.2e-04
```

The error falls as h^2 – two orders of magnitude for each factor ten in h, with the coefficient $|f'''|/6\approx 51$ of Eq. (4.74) – until $h\approx 10^{-6}$, the $\varepsilon_M^{1/3}$ of Eq. (4.76), and then *rises* as $1/h$ as rounding takes over. The best the finite difference achieves is 4×10^{-10}; the JAX derivative of the same function, `grad(f)(1.0)`, is 4.475424121402227, correct in every digit. Figure 4.12 plots the whole curve, and it is worth memorising its shape: whenever a finite-difference gradient check is used, as in Eq. (4.102), h must sit near the bottom of this valley, and agreement to better than about 10^{-8} relative should not be expected.

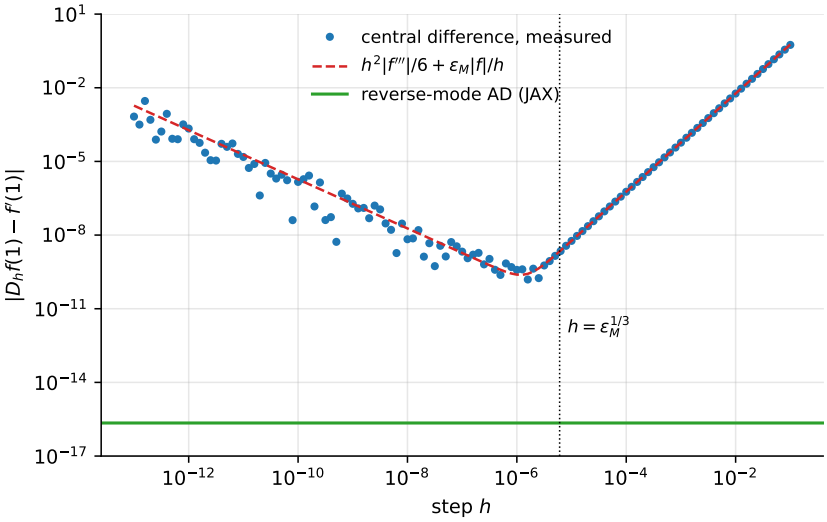


Fig. 4.12 Absolute error of the central finite difference (4.73) for $f(x)=\sin(2\pi x+x^2)$ at $x=1$, as a function of the step h (points), together with the model (4.75) (dashed): truncation error $\propto h^2$ on the right, rounding error $\propto 1/h$ on the left, and a minimum of about 10^{-10} near $h\approx\varepsilon_M^{1/3}$. The reverse-mode derivative computed by JAX has an error at the level of the unit roundoff, shown as the horizontal line, for any h because it involves no h.

The transformations of JAX compose: `jit(grad(f))` compiles the gradient, `vmap(grad(f))` vectorises it over a batch, `grad(grad(f))` differentiates it again and `jvp(grad(f),...)` gives Hessian–vector products. We shall use this machinery freely from Chapter 5 onwards, and in Chapter 8 we shall meet reverse mode again under its older name.

4.15 Practical tips

The following advice is collected from the sources cited in this chapter and from experience; each item connects to something already derived.

Randomise the data when forming minibatches.

Always shuffle before splitting into minibatches. Otherwise the ordering of the data becomes a systematic feature of the gradient sequence, and the method can fit spurious correlations arising purely from the order of presentation. If the data arrive sorted by class, an unshuffled minibatch may contain a single class and its gradient will point somewhere unhelpful.

Transform your inputs.

Learning is difficult when the landscape mixes steep and flat directions, and Section 4.5 made that statement quantitative: the iteration count scales with κ. Standardising the inputs – subtracting the mean and dividing by the standard deviation, as in Section 1.5 – equalises the scales and reduces κ. Whenever possible, decorrelate the inputs as well. The reason is exactly Eq. (1.44): for a squared error cost the Hessian *is* the correlation matrix of the inputs, up to normalisation, so standardising and decorrelating make the landscape as close to spherical as it can be made, which by Eq. (4.26) is the best case. Since most deep networks are linear transformations followed by non-linearities, the intuition carries beyond the linear case; this is also the motivation for batch normalisation.

Monitor out-of-sample performance.

Always track the cost on a validation set held out of training, as a proxy for the test set of Section 2.9. When the validation error begins to rise while the training error still falls, the model has started to overfit and the run should be stopped. This early stopping improves performance substantially in many settings and costs nothing.

Adaptive methods do not always generalise well.

As noted in Section 4.12, several studies have found that Adam, RMSProp and AdaGrad reach a worse test error than well-tuned SGD with momentum, particularly when the number of parameters exceeds the number of data points. It is not settled why adaptive methods train deep networks so effectively yet generalise less well; the practical conclusion is that a properly tuned SGD may match or beat them, and is worth trying.

Check your gradients.

Before trusting an optimiser, verify the gradient it is given. Compare the analytical expression against a central finite difference,

$$\frac{\partial C}{\partial \theta_j} \approx \frac{C(\boldsymbol{\theta} + h\boldsymbol{e}_j) - C(\boldsymbol{\theta} - h\boldsymbol{e}_j)}{2h}, \tag{4.102}$$

with h around 10^{-5}, or against automatic differentiation. An optimiser given a wrong gradient does not usually crash; it quietly converges to the wrong answer, which is far harder to detect.

Do not compare learning rates across optimisers.

Table 4.4 made the point: the same γ means different things to plain SGD and to Adam, because the latter has already normalised the gradient. Each method must be tuned on its own scale before any comparison is meaningful.

4.16 Summary and the programs

The chapter has one argument, developed in stages.

Minimising a cost function is easy when the cost is convex, because by Section 4.1 any stationary point is then a global minimum, and the least-squares and Ridge problems of Chapter 3 are convex. Newton's method (4.8) solves such problems ideally, in a single step for a quadratic, and is invariant to how the parameters are scaled. It costs $\mathcal{O}(p^3)$ per step and is therefore unavailable for large models.

Gradient descent costs $\mathcal{O}(p)$ and pays for it in iterations. The decoupled error recursion (4.19) explained exactly why: the step size is bounded above by $2/\lambda_{\max}$ for stability while progress is governed by $\lambda_{\min}$, so the iteration count scales with the condition number κ of the Hessian – and by Eq. (4.27) with the *square* of the condition number of the design matrix. This single number connects the chapter to the two before it: it is the quantity of Section 1.12 that governs numerical accuracy, the quantity of Section 3.7 that governs the variance of the fitted parameters, and now the quantity that governs how long training takes. Standardising features and adding a Ridge penalty improve all three at once.

The remedies then form a sequence, each recovering a little more of what Newton's method has and gradient descent lacks. Momentum accumulates a discounted history of gradients, amplifying consistent directions by $1/(1-\beta)$ and cancelling oscillatory ones, which improves the rate from κ to $\sqrt{\kappa}$. Stochastic gradients replace the full sum by a minibatch – an unbiased estimate with noise $\propto 1/\sqrt{M}$, Eq. (4.43) – trading a worse rate per iteration for a far better rate per floating-point operation: by the count of Eq. (4.48) the statistical accuracy worth having is reached in a few passes over the data, where gradient descent needs $\kappa \ln n$ of them, and Fig. 4.5 showed the factor to be two orders of magnitude on an ordinary problem. The price is that at a constant learning rate the iterate equilibrates in a cloud of radius $\propto \sqrt{\gamma/M}$, Eq. (4.50), instead of converging, leaving the cost on a plateau $\gamma \operatorname{tr}\boldsymbol{S}/(4M)$ that does not depend on the condition number, Eq. (4.51); the Robbins–Monro schedules, or an average over the iterates, repair this. Measured in epochs the batch size hardly matters, measured in seconds it matters a great deal (Table 4.3), and no comparison between optimisers means anything until the accuracy target and the horizontal axis have been stated. The adaptive methods are diagonal preconditioners, Eq. (4.57), that estimate the curvature from the second moment of the gradient: AdaGrad by an unbounded sum, which gives a built-in

$1/\sqrt{t}$ schedule and eventually stalls; RMSProp by an exponential moving average, which does not stall but cannot settle at a constant learning rate, Eq. (4.66); and Adam by combining RMSProp with momentum, whose averaging restores the ability to settle, and correcting the initialisation bias with the factors $1-\beta_i^t$ derived in Eq. (4.69). Their learning rate is a trust region, Eq. (4.72), not a multiplier of the gradient; they are invariant to the scaling of individual features and blind to their correlation, Fig. 4.8.

The demonstration of Table 4.4 was deliberately unflattering, and its two lessons should be kept. A learning rate is not comparable across optimisers, because the adaptive methods have already normalised the gradient before γ multiplies it. And on a small, convex, well-conditioned problem the adaptive machinery buys nothing and can cost a great deal: these methods exist for large, ill-conditioned, non-convex landscapes, which is where we shall meet them from Chapter 5 onwards.

Finally, none of this is usable without gradients, and Section 4.14 showed that they need not be derived by hand. Automatic differentiation applies the chain rule to the evaluation trace of a program: forward mode carries a tangent with every value and returns one Jacobian–vector product per sweep, reverse mode carries an adjoint with every node and returns one vector–Jacobian product per sweep, and by the cheap gradient principle of Proposition 4.2 the latter delivers the gradient with respect to all p parameters for the price of a few function evaluations, independent of p. It is exact to rounding, where finite differences are limited to $\varepsilon_M^{2/3}$ by Eq. (4.76), and it is the reason the models of the later chapters can be written down at all.

The complete programs are collected in `doc/BookML/BookPrograms`:

- `gradient_descent.py` – plain and momentum gradient descent on $f(x)=x^2$ and on the least-squares and Ridge problems of Section 4.4, with the analytical solutions for comparison, and the empirical verification of the stability bound (4.20) and the rate (4.26).
- `stochastic_gradient.py` – minibatch SGD with a varying number of batches, the time-decay schedule (4.52), the large-n comparison of Fig. 4.5 counted in floating-point operations, and the empirical check of the minibatch variance (4.43) and of the equilibration radius (4.50); the running mean of Section 4.7.3 and the practical loop with shuffled batches and early stopping of Section 4.7.8 are separate listings, and the figure script `BookFigures/ch04_sgd_figures.py` produces Figs. 4.6 and 4.7, Table 4.3 and every number quoted in Section 4.7.
- `adaptive_optimizers.py` – AdaGrad, RMSProp and Adam as in Section 4.12, both as the epoch loop and as the step function `optimiser_step`, the learning-rate sweep of Table 4.4, the axis-aligned and rotated bowls of Fig. 4.8, and the same comparison repeated on a deliberately ill-conditioned design matrix.
- `autodiff_examples.py` – the `Dual` class of Section 4.14.3 and the two hand-worked examples of Section 4.14.5, the `autograd` and `JAX` versions of the least-squares and Ridge gradients, the explicit `jvp`/`vjp` and Hessian–vector products of Section 4.14.8, the four optimisers on the Rosenbrock function of Section 4.14.9, and the finite-difference error curve of Fig. 4.12.

Each file runs as a script and reproduces the numbers quoted in this chapter. An executable version is available as a Jupyter notebook in the accompanying Jupyter-book.

4.17 Exercises

Warm-up exercises

1. **Convexity from the definition.** Show that $f(x) = x^2$ is convex on $\mathbb{R}$ using Eq. (4.1). Hint: show that $\lambda f(x) + (1-\lambda) f(y) - f(\lambda x + (1-\lambda) y) \geq 0$ for all x, y and $\lambda \in [0,1]$.
2. **Convexity from the second-order condition.** Using the second-order condition of Section 4.1, show that (a) $f(x) = e^x$ is convex on $\mathbb{R}$; (b) $g(x) = -\ln(x)$ is convex on $(0, \infty)$.
3. **Compositions.** Let $f(x) = x^2$ and $g(x) = e^x$. Show that $f(g(x))$ and $g(f(x))$ are convex on $\mathbb{R}$. Show more generally that if f is any convex function then $h(x) = e^{f(x)}$ is convex.
4. **Norms are convex.** A norm satisfies $f(\alpha \boldsymbol{x}) = |\alpha| f(\boldsymbol{x})$ and $f(\boldsymbol{x} + \boldsymbol{y}) \leq f(\boldsymbol{x}) + f(\boldsymbol{y})$. Using only these two properties and Eq. (4.1), show that a norm is convex. Deduce that both the Ridge and the Lasso cost functions of Chapter 3 are convex.
5. **The stability bound (numerical).** For the least-squares problem of Section 4.4: (a) compute the eigenvalues of the Hessian (4.14) and hence $\lambda_{\max}$, $\lambda_{\min}$ and κ; (b) run gradient descent for γ just below and just above $2/\lambda_{\max}$ and confirm Eq. (4.20); (c) measure the number of iterations needed to reach a fixed accuracy for several γ, and check that the minimum occurs near $\gamma^* = 2/(\lambda_{\max} + \lambda_{\min})$.
6. **Conditioning and convergence (numerical).** Construct design matrices with condition numbers $\kappa_2(\boldsymbol{X}) = 10, 10^2, 10^3$, for instance by rescaling the columns. (a) Measure the iteration count of gradient descent to fixed accuracy for each, and check the linear scaling with κ predicted by Eq. (4.26). (b) Repeat with momentum and verify the $\sqrt{\kappa}$ scaling of Eq. (4.33). (c) Repeat after standardising the columns, and comment.
7. **Momentum as a damped oscillator.** Consider the one-dimensional quadratic $C(\theta) = \frac{1}{2}\lambda\theta^2$. (a) Write the momentum update (4.28) as a two-term recurrence in θ. (b) Find the values of γ and β for which the recurrence has complex roots, and interpret them as oscillation. (c) Verify Eq. (4.32) numerically by applying momentum to a constant gradient.
8. **Minibatch noise (numerical).** For a fixed $\boldsymbol{\theta}$, compute the minibatch gradient many times for batch sizes $M = 1, 4, 16, 64$ and measure the standard deviation of its components. Verify the $1/\sqrt{M}$ scaling predicted by Section 2.5, and discuss what it implies about the cost of reducing gradient noise.
9. **AdaGrad stalls (numerical).** Reproduce the $\gamma = 0.01$ column of Table 4.4. (a) Plot the effective learning rate $\gamma/\sqrt{r_{t,j}}$ against t for each coordinate and show that it decays towards zero. (b) Estimate the total distance the iterate can still travel after t steps, and compare with the distance remaining to the minimum. (c) Repeat with RMSProp and explain, using Eq. (4.65), why the decay does not occur.
10. **Adam's bias correction (numerical).** (a) Implement Adam with and without the bias correction (4.70) and compare the first twenty steps. (b) For a constant gradient g, verify Eq. (4.69) numerically. (c) Show that with $\beta_2 = 0.999$ the uncorrected $\sqrt{v_1}$ is smaller than the true root mean square gradient by a factor of about 32, and explain what that does to the first step.
11. **Dual numbers.** (a) Using only $\varepsilon^2 = 0$, derive the rules for the quotient $(a + b\varepsilon)/(c + d\varepsilon)$ and for $\sqrt{a + b\varepsilon}$, and check that they reproduce the quotient rule and $d\sqrt{x}/dx$. (b) Show from Eq. (4.84) that dual numbers cannot deliver second derivatives, and that the "hyper-dual" algebra with two symbols $\varepsilon_1, \varepsilon_2$, $\varepsilon_1^2 = \varepsilon_2^2 = 0$ but $\varepsilon_1\varepsilon_2 \neq 0$, can. (c) Extend the `Dual` class of Section 4.14.3 with `sqrt` and `__truediv__` for a dual denominator, and verify your rules numerically.
12. **Forward and reverse sweeps by hand.** For $f(x_1, x_2) = x_1 x_2 + \exp(x_1 x_2) - \sin x_1$ at $(x_1, x_2) = (1, 2)$: (a) write down the evaluation trace and draw the computational graph, as in Eq. (4.78) and Fig. 4.10; (b) carry out the forward sweep (4.81) for both coordinate di-

rections; (c) carry out the reverse sweep (4.87) once, and confirm that it delivers both partial derivatives; (d) count the multiplications in (b) and in (c), and check the operation count against Proposition 4.2.

13. **Reverse mode for least squares.** Write the least-squares cost (4.12) as the trace $\boldsymbol{v}_1 = \boldsymbol{X}\boldsymbol{\theta}$, $\boldsymbol{v}_2 = \boldsymbol{v}_1 - \boldsymbol{y}$, $v_3 = \boldsymbol{v}_2^T \boldsymbol{v}_2 / n$, with vector-valued nodes. Apply the reverse recursion (4.87) with the vector–Jacobian products (4.89) of each stage, and show that it produces the gradient (4.13) $\frac{2}{n}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y})$ without ever forming a Jacobian larger than $\boldsymbol{X}$ itself. Explain why the adjoint $\bar{\boldsymbol{v}}_1$ is what Chapter 8 will call the output error.
14. **Hessian–vector products (numerical).** (a) Implement Eq. (4.100) in JAX as forward-over-reverse, `jvp(grad(f))`, and alternatively as reverse-over-reverse, `grad(lambda x: grad(f)(x) @ v)`. Check both against `jax.hessian` on the Ridge cost (4.16). (b) Use only Hessian–vector products and the conjugate-gradient method to solve the Newton system (4.8) for the Rosenbrock function (4.101), and reproduce the six-step convergence of Section 4.14.9 without ever forming the Hessian.
15. **The minibatch gradient as an estimator.** (a) Derive Eq. (4.43) for sampling without replacement, starting from the fact that each example enters the batch with probability M/n and each pair with probability $M(M-1)/(n(n-1))$. (b) On the least-squares problem of Section 4.4, fix a point $\boldsymbol{\theta}$, draw a few hundred minibatches for $M = 1, 5, 25$ and compare the mean and the standard deviation of the minibatch gradients with the full gradient and with Eq. (4.43). (c) Run SGD at a constant γ for $M = 5$ and $M = 20$, measure the size of the cloud in which the iterate settles along the flattest eigendirection, and compare with Eq. (4.50). (d) Derive the plateau (4.51) from Eq. (4.50), show that $\operatorname{tr}\boldsymbol{S} \approx 4\sigma^2 p$ for least squares with standardised features, and verify both numerically. Why does the condition number drop out of the cost but not out of the distance $\|\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}\|$?
16. **The running mean.** (a) Show by induction that the recursion (4.40) with $\gamma_t = 1/(t+1)$ makes θ_t the mean of the first t samples. (b) Show that with a constant γ the iterate is an exponentially weighted average of the samples with stationary variance $\gamma s_a^2/(2-\gamma)$, and check it numerically for $\gamma = 0.1$ and 0.01. (c) Replace the last iterate of (b) by the average of the iterates over the second half of the run and compare its error with that of (a).
17. **Expected progress of one stochastic step.** (a) Fill in the steps between the descent lemma and Eq. (4.44), stating where unbiasedness and where Eq. (4.42) are used. (b) For a quadratic cost the inequality is an equality if L is replaced by the appropriate Rayleigh quotients. On the least-squares problem of Section 4.4, draw 5000 minibatches at a fixed $\boldsymbol{\theta}$, measure the mean change of the cost and the fraction of steps that *increase* it, for $M = 1$, 8 and 64, far from the minimum and close to it. (c) Setting the progress and noise terms equal and using $\|\nabla C\|^2 \geq 2\mu(C - C^*)$ gives a bound on the plateau. Compare it with Eq. (4.51) and explain the factor κ by which the bound is pessimistic.
18. **Counting flops.** (a) Verify the count (4.38) and extend it to the Ridge gradient (4.17) and to one Newton step with the Hessian (4.14) formed and factorised. (b) For $n = 10^6$, $p = 10^3$, $\kappa = 10^3$ and $M = 64$, estimate from Eq. (4.48) the number of floating-point operations gradient descent, momentum and SGD need to reach the estimation error $\sigma^2 p/n$, and the number of epochs each corresponds to. (c) Repeat the experiment of Fig. 4.5 with $n = 10^3$ and with $n = 10^6$. Where does the crossover at which SGD overtakes gradient descent move, and why?
19. **A fair comparison (numerical).** Generate a least-squares problem with $n = 100\,000$, $p = 50$, correlated standardised features and controlled noise, and set aside a validation set. Compare gradient descent (tuned constant γ), SGD with $M = 1$ (decaying γ_t) and minibatch SGD with $M = 32$, 128 and 512 (tuned, constant and decaying), recording the training cost, the validation cost and the time. (a) Plot the validation cost against updates, examples processed, estimated floating-point operations and wall time, as in Fig. 4.7, with bands from at least five seeds. (b) Report, as in Table 4.3, what each method needs to reach

the validation cost of the exact solution to within the estimation error (4.47). (c) Repeat for autoregressive correlations 0, 0.5, 0.9 and 0.99. How do the epoch counts of gradient descent and of SGD scale with κ, and what does adding momentum change? (d) Replace the decaying schedule by a constant γ with iterate averaging over the second half of the run and place it in the table.

20. **RMSProp cannot settle; Adam can.** (a) For $C=\tfrac{1}{2}\lambda\theta^2$ with $\lambda=1$, run RMSProp at $\gamma=10^{-2}$ from $\theta_0=1$ for 10^4 steps and plot $|\theta_t|$. Confirm the limit cycle predicted by Eq. (4.66) and measure its amplitude for $\gamma=10^{-1},10^{-2},10^{-3}$. (b) Repeat with Adam at the same γ and explain, using the first moment, why it converges. (c) Show from Eqs. (4.67)–(4.71) that the first Adam step is $-\alpha\,\mathrm{sign}(\boldsymbol{g}_1)$, and that without bias correction it would be longer by $(1-\beta_1)/\sqrt{1-\beta_2}$. (d) Multiply one column of the design matrix of Section 4.4 by $c=100$ and rerun gradient descent, Adam and AdaGrad from the same starting point. Which iteration counts change, and why?
21. **The finite-difference error curve (numerical).** Reproduce Fig. 4.12 for $f(x)=e^x$ at $x=1$ and for $f(x)=\tan x$ at $x=1.5$. From the measured minimum, estimate ε_M and $|f'''|$ using Eq. (4.76), and compare with the true values. Repeat in single precision (`np.float32`) and explain the change.

Project-style exercise: optimisers on an ill-conditioned problem

The comparison in Table 4.4 was made on a problem too easy to distinguish the methods. The purpose of this exercise is to build one that does, and thereby to see the arguments of this chapter operate.

Part a: build the landscape.

Generate a polynomial regression problem from the Franke function of Section 3.15, or from a one-dimensional polynomial of degree ten with the Vandermonde design matrix (3.3). Compute $\kappa_2(\boldsymbol{X})$ and $\kappa_2(\boldsymbol{X}^T\boldsymbol{X})$ and confirm Eq. (4.27). Then produce a second version with the columns standardised and compare the two condition numbers.

Part b: plain gradient descent.

Implement gradient descent with the analytical gradient (4.13). Determine empirically the largest stable γ and compare with Eq. (4.20). Plot the cost against iteration for several γ on both the raw and the standardised problem.

Part c: momentum and stochasticity.

Add momentum and confirm the improvement in iteration count. Then implement minibatch SGD with the time-decay schedule (4.52), and compare convergence per iteration and per gradient evaluation. Which comparison is the fair one, and why?

Part d: the adaptive methods.

Implement AdaGrad, RMSProp and Adam. For each, sweep γ over several orders of magnitude and record the best result; present the outcome as a table like Table 4.4. Discuss which methods are sensitive to γ and which are not, and relate your finding to whether the method normalises the gradient before applying γ.

Part e: the diagonal approximation and its limits.

Construct two problems with the same condition number, one whose Hessian is diagonal and one whose narrow valley runs at 45 degrees to the coordinate axes – the second is obtained from the first by an orthogonal rotation of the features, which by Section 1.7 leaves κ unchanged. Run all the methods on both. Explain, using the final notebox of Section 4.11, why the adaptive methods help on the first and not on the second, and what this says about the difference between a diagonal approximation to $\boldsymbol{H}$ and the real thing.

Part f: automatic differentiation.

Repeat part b using `autograd` or `JAX` instead of the hand-derived gradient, and verify that the two agree to machine precision. Then apply the gradient check (4.102) and compare the accuracy of the finite difference with that of automatic differentiation as h is varied over many orders of magnitude. Explain the shape of the resulting curve using the discussion of round-off error in Section 1.12.

Chapter 5
Logistic Regression

Everything so far has predicted a continuous number. We now turn to *classification*, where the target is a label drawn from a finite set: whether a tumour is malignant or benign, whether a credit card holder will default, which of ten digits a handwritten image shows. The change of target forces a change of model and, more interestingly, a change of loss function – and the new loss will turn out to follow from the same maximum-likelihood argument that produced least squares in Section 3.6.

Logistic regression is the natural first classifier, and it occupies the same position in this book that linear regression did. It is simple enough that everything can be computed, its cost function is convex so that the optimisation theory of Chapter 4 applies without qualification, and – most importantly for what follows – it is exactly a neural network with no hidden layers. The sigmoid we introduce here is an activation function, the cross-entropy we derive here is the loss used to train classifiers throughout deep learning, and the gradient we compute here reappears as the last step of backpropagation. A reader who understands this chapter thoroughly has already met most of the ingredients of the next one.

5.1 Classification problems

We consider the case where the responses y_i are discrete and take values from $k = 0, \dots, K-1$, that is K classes. The goal is to predict the class from the design matrix $\boldsymbol{X} \in \mathbb{R}^{n \times p}$ of Eq. (1.1), made of n samples each carrying p features, and in particular to identify the classes of new, unseen samples.

We specialise for most of the chapter to two classes, with outputs $y_i = 0$ and $y_i = 1$. The outcome might represent whether a credit card holder defaults on their debt,

$$y_i = \begin{cases} 0 & \text{no,} \\ 1 & \text{yes.} \end{cases} \tag{5.1}$$

Why not simply use linear regression?

Before introducing anything new, it is worth seeing why the machinery of Chapter 3 is not adequate. We could fit the linear model

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\theta} + \boldsymbol{\varepsilon} \tag{5.2}$$

and classify by thresholding, predicting class 1 when $\tilde{y}_i > 0.5$ and class 0 otherwise. Something like this can be made to work, but it is unsatisfactory for a fundamental reason: the right-hand side of Eq. (5.2) takes values on the entire real axis, while y_i takes only the values 0 and 1. A fitted value of -3.7 or $+2.4$ has no interpretation. Worse, a single distant point drags the fitted line and moves the decision threshold, so that adding a strongly-classified observation can cause a previously correct classification to flip.

One way to force a discrete output is to compose the linear model with a sign function, $f(s_i) = 1$ if $s_i \geq 0$ and 0 otherwise. This is the *perceptron*, historically the first machine learning model and the subject of the opening of the next chapter. It is a *hard* classifier: each data point is assigned deterministically to a category. In many situations we would rather have a *soft* classifier, one that outputs the *probability* of a given category. Probabilities can be thresholded when a decision is needed, but they also express confidence, they can be combined with costs when different errors matter differently, and – as we shall see – they lead to a differentiable cost function, which the step function does not.

An illustration.

Data on coronary heart disease (CHD) as a function of age make the point. Plotting whether a person has had CHD ($y = 1$) or not ($y = 0$) against age gives two horizontal bands of points, and a straight-line fit through them is visibly meaningless.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

chd = pd.read_csv("DataFiles/chddata.csv", names=("ID", "Age", "Agegroup", "CHD"))
plt.scatter(chd["Age"], chd["CHD"], marker="o")
plt.axis([18, 70.0, -0.1, 1.2])
plt.xlabel("Age"); plt.ylabel("CHD")
plt.show()
```

What is meaningful is the *mean* value within each age group.

```
agegroupmean = np.array([0.1, 0.133, 0.250, 0.333, 0.462, 0.625, 0.765, 0.800])
group = np.array([1, 2, 3, 4, 5, 6, 7, 8])
plt.plot(group, agegroupmean, "r-")
plt.axis([0, 9, 0, 1.0])
plt.xlabel("Age group"); plt.ylabel("CHD mean value")
plt.show()
```

Two features of the resulting curve are decisive. It is confined to the interval $[0, 1]$, as any mean of zeros and ones must be. And it is *S-shaped*: nearly flat at both ends, steepest in the middle. We are looking for a function $f(y_i \mid x_i)$ giving the expected value of the output for a given input; in linear regression this was $f(y_i \mid x_i) = \theta_0 + \theta_1 x_i$, which ranges over the whole real line, whereas here we need $0 \leq f(y_i \mid x_i) \leq 1$ together with the observed S-shape. A function with exactly these properties is the subject of the next section, and we shall interpret it as the probability of observing $y_i = 1$ for a given x_i.

5.2 The logistic function

The *logistic* or *sigmoid* function is

$$\sigma(t) = \frac{1}{1+\exp(-t)} = \frac{\exp(t)}{1+\exp(t)}. \tag{5.3}$$

It maps the whole real line onto the open interval $(0,1)$, tends to 0 as $t \to -\infty$ and to 1 as $t \to +\infty$, and takes the value $\frac{1}{2}$ at the origin. It has the S-shape observed in the CHD group means.

Three properties will be used repeatedly. The first is the reflection identity

$$1 - \sigma(t) = \sigma(-t), \tag{5.4}$$

immediate from Eq. (5.3), which says that the probability of class 0 is obtained from the probability of class 1 by flipping the sign of the argument. The second is the derivative,

$$\boxed{\frac{d\sigma}{dt} = \sigma(t)\,[1-\sigma(t)]\,.} \tag{5.5}$$

This is worth deriving once. Writing $\sigma = (1+e^{-t})^{-1}$,

$$\frac{d\sigma}{dt} = \frac{e^{-t}}{(1+e^{-t})^2} = \frac{1}{1+e^{-t}} \cdot \frac{e^{-t}}{1+e^{-t}} = \sigma(t)\,[1-\sigma(t)]\,,$$

using $e^{-t}/(1+e^{-t}) = 1-\sigma(t)$. Equation (5.5) is remarkable in that the derivative is expressed in terms of the function value alone, with no further exponentials to evaluate. Every gradient in this chapter, and every sigmoid backpropagation step in the next, rests on it. Note also that the derivative is largest at $t=0$, where it equals $1/4$, and falls off rapidly in both directions – a fact we shall return to when discussing why deep networks built from sigmoids are hard to train.

The third property is the *logit*, the inverse function,

$$t = \ln \frac{\sigma}{1-\sigma}, \tag{5.6}$$

the logarithm of the odds. Equation (5.6) is what makes logistic regression *linear*: we shall model the log-odds as a linear function of the features, even though the probability itself is a non-linear function of them.

Related functions.

Two relatives appear later. The *step* function, $f(t) = 1$ for $t \geq 0$ and 0 otherwise, is the hard-classifier limit of the sigmoid and is the perceptron's activation; it is not differentiable, which is precisely why it cannot be trained by the gradient methods of Chapter 4. The hyperbolic tangent

$$\tanh(t) = \frac{e^{t}-e^{-t}}{e^{t}+e^{-t}} = 2\sigma(2t) - 1 \tag{5.7}$$

is a rescaled sigmoid mapping onto $(-1,1)$ rather than $(0,1)$, and is a common activation function in neural networks because its output is centred on zero.

```
import numpy as np
import matplotlib.pyplot as plt

z = np.arange(-5, 5, 0.1)

fig, ax = plt.subplots(1, 3, figsize=(12, 3.5))
ax[0].plot(z, 1.0 / (1.0 + np.exp(-z)));      ax[0].set_title("sigmoid")
```

```
ax[1].plot(z, np.where(z >= 0.0, 1.0, 0.0));  ax[1].set_title("step")
ax[2].plot(z, np.tanh(z));                    ax[2].set_title("tanh")
for a in ax:
    a.grid(True); a.set_xlabel("z")
plt.show()
```

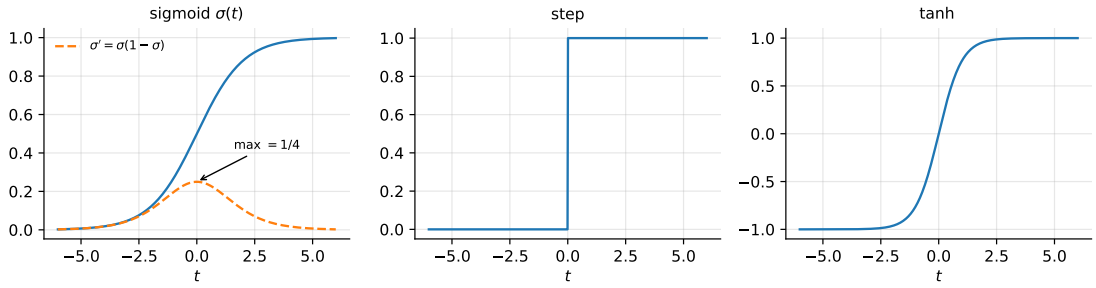


Fig. 5.1 The logistic function (5.3) with its derivative $\sigma' = \sigma(1-\sigma)$, the step function of the perceptron, and tanh. The derivative attains its maximum of $1/4$ at the origin and decays rapidly, a fact that returns when deep networks are trained.

5.3 Maximum likelihood and the cross-entropy

We now derive the cost function. The argument is the one used in Section 3.6 to obtain least squares, with the Gaussian noise model replaced by a Bernoulli one – and the notebox there promised exactly this outcome.

The model.

Assume two classes and, to begin with, a single predictor and two parameters. We model the probability of class 1 by a sigmoid applied to a linear function of the input,

$$p(y_i = 1 \mid x_i, \boldsymbol{\theta}) = \frac{\exp(\theta_0 + \theta_1 x_i)}{1 + \exp(\theta_0 + \theta_1 x_i)} = \sigma(\theta_0 + \theta_1 x_i), \tag{5.8}$$

$$p(y_i = 0 \mid x_i, \boldsymbol{\theta}) = 1 - p(y_i = 1 \mid x_i, \boldsymbol{\theta}), \tag{5.9}$$

where $\boldsymbol{\theta} = (\theta_0, \theta_1)$ are the weights we wish to determine. Equation (5.9) is forced: the two probabilities must sum to one. Taking the logit of Eq. (5.8) using Eq. (5.6) gives the equivalent statement

$$\ln \frac{p(y_i = 1 \mid x_i, \boldsymbol{\theta})}{1 - p(y_i = 1 \mid x_i, \boldsymbol{\theta})} = \theta_0 + \theta_1 x_i, \tag{5.10}$$

which is the cleanest way to say what logistic regression assumes: the log-odds are linear in the features.

Figure 5.2 shows the consequence in the plane. The decision boundary $p = \frac{1}{2}$ is a straight line, because it is the locus $\boldsymbol{x}^T\boldsymbol{\theta} = 0$; what is not linear is the probability, which varies smoothly across the boundary at a rate set by $\|\boldsymbol{\theta}\|$. The dashed contours at $p = 0.1$ and $p = 0.9$ mark the region in which the model declines to commit itself, and it is exactly this graded output – rather than the hard assignment of the perceptron – that a soft classifier provides.

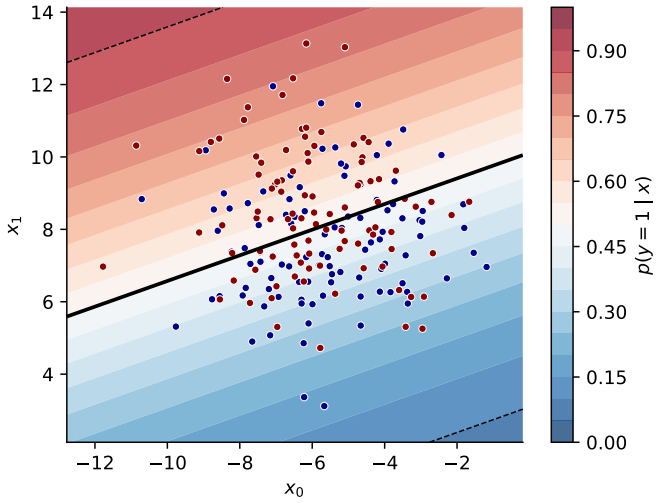


Fig. 5.2 Fitted probability $p(y=1 \mid \boldsymbol{x})$ for two Gaussian classes. The solid line is the decision boundary $p=\frac{1}{2}$, a straight line; the dashed contours at $p=0.1$ and 0.9 bound the region of genuine uncertainty.

The likelihood.

For a data set $\mathscr{D}=\{(y_i,x_i)\}$ with binary labels and independent observations, the maximum likelihood principle instructs us to maximise the probability of the data actually seen. A single observation contributes $p(y_i=1 \mid x_i,\boldsymbol{\theta})$ if $y_i=1$ and $1-p(y_i=1 \mid x_i,\boldsymbol{\theta})$ if $y_i=0$; both cases are captured by the single expression $p^{y_i}(1-p)^{1-y_i}$, in which the exponents switch the factors on and off. Hence

$$P(\mathscr{D} \mid \boldsymbol{\theta}) = \prod_{i=1}^{n} [p(y_i=1 \mid x_i,\boldsymbol{\theta})]^{y_i} \, [1-p(y_i=1 \mid x_i,\boldsymbol{\theta})]^{1-y_i}. \tag{5.11}$$

This is a product of Bernoulli probabilities, exactly as Eq. (3.25) was a product of Gaussians.

As in Section 3.6 we take logarithms, both because it turns the product into a sum and because a product of n numbers smaller than one underflows. The log-likelihood is

$$\log P(\mathscr{D} \mid \boldsymbol{\theta}) = \sum_{i=1}^{n} \Big(y_i \log p(y_i=1 \mid x_i,\boldsymbol{\theta}) + (1-y_i) \log [1-p(y_i=1 \mid x_i,\boldsymbol{\theta})] \Big). \tag{5.12}$$

The cost function.

Since we minimise rather than maximise, the cost is the negative log-likelihood,

$$\boxed{C(\boldsymbol{\theta}) = -\sum_{i=1}^{n} \Big(y_i \log p_i + (1-y_i) \log [1-p_i] \Big), \qquad p_i = \sigma(\theta_0+\theta_1 x_i).} \tag{5.13}$$

This is known in statistics and information theory as the *cross entropy*, and it is the loss function used to train essentially every classifier in this book.

Substituting Eq. (5.8) and reordering the logarithms gives a form which is both more compact and numerically better behaved. Using $\log p_i = (\theta_0+\theta_1 x_i) - \log[1+\exp(\theta_0+\theta_1 x_i)]$ and $\log(1-p_i) = -\log[1+\exp(\theta_0+\theta_1 x_i)]$,

$$C(\boldsymbol{\theta}) = -\sum_{i=1}^{n} \Big(y_i(\theta_0+\theta_1 x_i) - \log [1+\exp(\theta_0+\theta_1 x_i)] \Big). \tag{5.14}$$

Convexity.

The cross entropy is a convex function of $\boldsymbol{\theta}$, so by Section 4.1 any local minimiser is a global minimiser. We verify this in Section 5.4 by exhibiting the Hessian and showing it is positive semi-definite. This is a substantial guarantee, and it is why logistic regression is a well-behaved problem in a way that neural networks are not: we may apply any of the methods of Chapter 4 without worrying about which minimum we reach.

Finally, note that just as in linear regression we often supplement the cross entropy with regularisation terms, usually the ℓ_1 and ℓ_2 penalties of Sections 3.8 and 3.9; we return to this in Section 5.5.

Why not squared error? It is natural to ask why we do not simply minimise $\sum_i (y_i - p_i)^2$, which is also well defined. There are two answers. The statistical one is that the squared error is the maximum-likelihood loss for *Gaussian* noise, which is the wrong model for a binary outcome; the cross entropy is the correct one for a Bernoulli outcome, exactly as promised in the notebox of Section 3.6. The practical one is about gradients. Differentiating the squared error brings down a factor $\sigma'(t) = \sigma(1-\sigma)$ by Eq. (5.5), which is *small* whenever the prediction is confident, whether or not it is correct. A badly wrong, confidently made prediction therefore produces almost no gradient and the model learns nothing from its worst mistakes. The cross-entropy gradient, as the next section shows, contains no such factor: it is simply the error $y_i - p_i$. This cancellation is one of the reasons the cross entropy is universal in classification. It is not the only possible loss, however, and Section 5.10 surveys the alternatives – and fits all of them with a single program.

Figure 5.3 makes the argument visually. The left panel shows the two losses for a point whose true label is $y = 1$: both fall as the prediction improves, and the cross entropy diverges as $p \to 0$ whereas the squared error saturates at one. The right panel shows the gradients, and it is the decisive one. In the shaded region the model is confidently wrong, and there the squared-error gradient has collapsed almost to zero – the model receives no signal from its worst mistakes – while the cross-entropy gradient is at its largest.

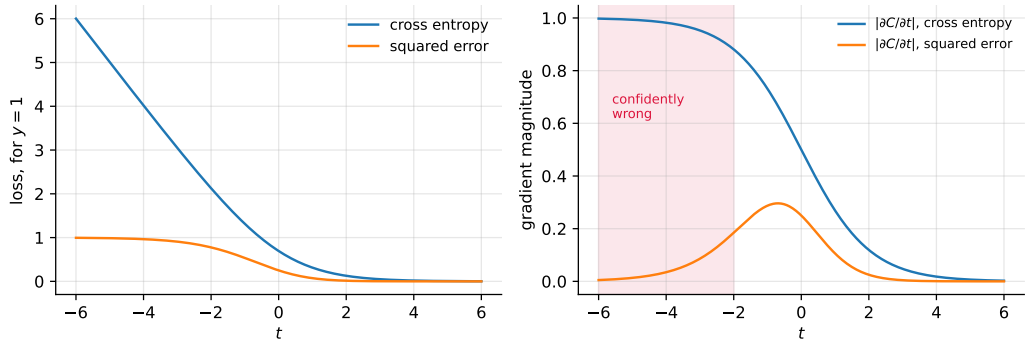


Fig. 5.3 Cross entropy and squared error for a single observation with $y = 1$ (left) and the magnitudes of their gradients with respect to $t = \boldsymbol{x}^T\boldsymbol{\theta}$ (right). In the shaded region the prediction is confidently wrong, and the squared-error gradient vanishes there.

5.4 Gradients, the Hessian and convexity

Minimising Eq. (5.14) with respect to the two parameters, and using $\exp(t)/[1+\exp(t)] = \sigma(t) = p_i$, gives

$$\frac{\partial C(\boldsymbol{\theta})}{\partial \theta_0} = -\sum_{i=1}^{n} (y_i - p_i), \tag{5.15}$$

$$\frac{\partial C(\boldsymbol{\theta})}{\partial \theta_1} = -\sum_{i=1}^{n} (y_i x_i - x_i p_i) = -\sum_{i=1}^{n} x_i (y_i - p_i). \tag{5.16}$$

Both have the same structure: the residual $y_i - p_i$ weighted by the corresponding feature, with the intercept carrying the implicit feature 1.

Compact form.

Define the vector $\boldsymbol{y}$ of the n labels, the $n \times p$ design matrix $\boldsymbol{X}$ containing the inputs including a column of ones, and the vector $\boldsymbol{p}$ of fitted probabilities with entries $p_i = p(y_i = 1 \mid \boldsymbol{x}_i, \boldsymbol{\theta})$. Then Eqs. (5.15) and (5.16) collapse to

$$\boxed{\frac{\partial C(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = -\boldsymbol{X}^T (\boldsymbol{y} - \boldsymbol{p}).} \tag{5.17}$$

This should look extremely familiar. Compare it with the least-squares gradient of Eq. (1.38), $-\frac{2}{n}\boldsymbol{X}^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta})$. The two are identical in form, differing only in that the linear prediction $\boldsymbol{X}\boldsymbol{\theta}$ has been replaced by the probability $\boldsymbol{p} = \sigma(\boldsymbol{X}\boldsymbol{\theta})$ and in an overall constant. In both cases the gradient is the design matrix transposed against the residual, and the algorithmic remark of the notebox in Section 1.8.5 applies unchanged: the gradient costs two matrix-vector products and never requires a matrix to be formed.

The similarity is not superficial. Both models are *generalised linear models*: a linear predictor $\eta_i = \boldsymbol{X}_{i,*}\boldsymbol{\theta}$ is passed through a link function, the identity in one case and the sigmoid in the other, and for the canonical link the maximum-likelihood gradient always takes the form "design matrix transposed times residual".

The Hessian.

Differentiating Eq. (5.17) once more requires the derivative of $\boldsymbol{p}$ with respect to $\boldsymbol{\theta}$, which by Eq. (5.5) and the chain rule (1.50) is $\partial p_i / \partial \boldsymbol{\theta} = p_i(1 - p_i)\boldsymbol{X}_{i,*}$. Defining the diagonal matrix $\boldsymbol{W}$ with entries

$$W_{ii} = p_i (1 - p_i), \tag{5.18}$$

we obtain the compact expression

$$\boxed{\frac{\partial^2 C(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}\, \partial \boldsymbol{\theta}^T} = \boldsymbol{X}^T \boldsymbol{W} \boldsymbol{X}.} \tag{5.19}$$

Compare again with least squares, where the Hessian was $\boldsymbol{X}^T\boldsymbol{X}$ by Eq. (1.44). Logistic regression inserts a diagonal weight matrix between the two factors, and the weights are largest, equal to $\frac{1}{4}$, where $p_i = \frac{1}{2}$ – that is for the observations the model is most uncertain about, those nearest the decision boundary. Points far from the boundary have p_i near 0 or 1, hence W_{ii} near zero, and contribute almost nothing to the curvature. *The fit is determined by the*

points near the boundary, which is a first hint of the idea carried to its conclusion by support vector machines.

Convexity, proved.

For any vector $\boldsymbol{z}$,

$$\boldsymbol{z}^T \boldsymbol{X}^T \boldsymbol{W} \boldsymbol{X} \boldsymbol{z} = (\boldsymbol{X}\boldsymbol{z})^T \boldsymbol{W} (\boldsymbol{X}\boldsymbol{z}) = \sum_{i=1}^{n} W_{ii} (\boldsymbol{X}\boldsymbol{z})_i^2 \;\geq\; 0, \tag{5.20}$$

because every $W_{ii} = p_i(1-p_i)$ is positive for $0 < p_i < 1$. The Hessian is therefore positive semi-definite everywhere, so by the second-order condition of Section 4.1 the cross entropy is convex and by the result quoted there any stationary point is a global minimum. This is the promised proof.

No closed form.

Unlike the normal equations (1.39), setting Eq. (5.17) to zero does not yield a formula for $\hat{\boldsymbol{\theta}}$: the unknown appears inside the non-linear function $\boldsymbol{p} = \sigma(\boldsymbol{X}\boldsymbol{\theta})$, and no rearrangement extracts it. We must solve numerically, which is why Chapter 4 preceded this one. The situation resembles the Lasso of Section 3.9 – a convex problem without a closed form – although here the obstruction is non-linearity rather than non-differentiability, so ordinary gradient methods apply directly.

5.5 More predictors, and regularisation

Extending to p predictors requires no new ideas. The log-odds relation (5.10) becomes

$$\ln \frac{p(\boldsymbol{\theta}, \boldsymbol{x})}{1 - p(\boldsymbol{\theta}, \boldsymbol{x})} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p, \tag{5.21}$$

and with $\boldsymbol{x} = [1, x_1, \ldots, x_p]$ and $\boldsymbol{\theta} = [\theta_0, \theta_1, \ldots, \theta_p]$ the probability is

$$p(\boldsymbol{\theta}, \boldsymbol{x}) = \frac{\exp(\theta_0 + \theta_1 x_1 + \cdots + \theta_p x_p)}{1 + \exp(\theta_0 + \theta_1 x_1 + \cdots + \theta_p x_p)} = \sigma\left(\boldsymbol{x}^T \boldsymbol{\theta}\right). \tag{5.22}$$

The gradient (5.17) and Hessian (5.19) are already written in a form valid for any p.

Regularisation.

In practice the cross entropy is supplemented with a penalty, exactly as in Chapter 3,

$$C_\lambda(\boldsymbol{\theta}) = C(\boldsymbol{\theta}) + \lambda \left\|\boldsymbol{\theta}\right\|_2^2 \qquad \text{or} \qquad C_\lambda(\boldsymbol{\theta}) = C(\boldsymbol{\theta}) + \lambda \left\|\boldsymbol{\theta}\right\|_1, \tag{5.23}$$

giving the ℓ_2 and ℓ_1 regularised versions. The gradient of the ℓ_2 penalty is $2\lambda\boldsymbol{\theta}$ by Eq. (1.28), so Eq. (5.17) becomes $-\boldsymbol{X}^T(\boldsymbol{y} - \boldsymbol{p}) + 2\lambda\boldsymbol{\theta}$, and the Hessian becomes $\boldsymbol{X}^T\boldsymbol{W}\boldsymbol{X} + 2\lambda\boldsymbol{I}$, which is positive *definite* for $\lambda > 0$ and hence strictly convex.

Everything said in Chapter 3 about the two penalties carries over: ℓ_2 shrinks coefficients smoothly, ℓ_1 sets some exactly to zero and performs variable selection, the intercept should

not be penalised, and the features must be standardised first for the penalty to be meaningful. The remarks of Section 3.13 apply verbatim.

There is one addition specific to classification. If the two classes are *linearly separable* – if some hyperplane divides them perfectly – then the unregularised maximum likelihood problem has *no finite solution*. The likelihood can always be increased by scaling $\boldsymbol{\theta}$ up, which makes every predicted probability more extreme and drives the cost towards zero without ever attaining it, so $\|\boldsymbol{\theta}\| \to \infty$. Any penalty $\lambda > 0$ removes the pathology by making the cost eventually increase. This is one reason `scikit-learn` regularises by default, with `C` playing the role of $1/\lambda$ – a convention worth remembering when comparing against one's own implementation, in the spirit of the warnings in Section 3.10.

Machine learning connection. The conditioning discussion of Section 4.5 applies here with a twist. The Hessian is now $\boldsymbol{X}^T\boldsymbol{W}\boldsymbol{X}$, which *changes as the fit proceeds* because $\boldsymbol{W}$ depends on the current probabilities. Early in training, when all $p_i \approx \frac{1}{2}$ and $\boldsymbol{W} \approx \frac{1}{4}\boldsymbol{I}$, the curvature is essentially $\frac{1}{4}\boldsymbol{X}^T\boldsymbol{X}$ and the analysis of Chapter 4 carries over directly. As the fit sharpens, the well-classified points drop out of $\boldsymbol{W}$ and the effective Hessian is built from a shrinking subset of the data, typically becoming worse conditioned. A learning rate chosen at the start may therefore be badly wrong later, which is one concrete reason the adaptive methods of Section 4.8 – whose $\boldsymbol{v}_t$ tracks a *recent* average and so follows the changing landscape – are useful even on this convex problem.

5.6 More than two classes: the softmax

Suppose we wish to extend to K classes. Taking one class as a reference – conventionally the last – we model each of the remaining $K-1$ log-odds ratios as linear. With a single predictor for simplicity,

$$\begin{aligned}
\ln \frac{p(C=1 \mid x)}{p(C=K \mid x)} &= \theta_{10} + \theta_{11} x_1, \\
\ln \frac{p(C=2 \mid x)}{p(C=K \mid x)} &= \theta_{20} + \theta_{21} x_1, \\
&\vdots \\
\ln \frac{p(C=K-1 \mid x)}{p(C=K \mid x)} &= \theta_{(K-1)0} + \theta_{(K-1)1} x_1.
\end{aligned} \tag{5.24}$$

The model is thus specified by $K-1$ logit transformations. Solving for the probabilities, and using the fact that they must sum to one,

$$p(C=k \mid \boldsymbol{x}) = \frac{\exp(\theta_{k0} + \theta_{k1} x_1)}{1 + \sum_{l=1}^{K-1} \exp(\theta_{l0} + \theta_{l1} x_1)}, \qquad k = 1, \dots, K-1, \tag{5.25}$$

with the reference class taking the remainder,

$$p(C=K \mid \boldsymbol{x}) = \frac{1}{1 + \sum_{l=1}^{K-1} \exp(\theta_{l0} + \theta_{l1} x_1)}. \tag{5.26}$$

These sum to one by construction, and setting $K=2$ recovers Eqs. (5.8) and (5.9), so the binary case is the special case it should be. Extending to more predictors requires only replacing $\theta_{k0} + \theta_{k1} x_1$ by $\boldsymbol{x}^T \boldsymbol{\theta}_k$.

The symmetric form.

Singling out a reference class is inelegant, and in practice one uses the symmetric parametrisation

$$\boxed{p(C=k\mid \boldsymbol{x}) = \frac{\exp\left(\boldsymbol{x}^T\boldsymbol{\theta}_k\right)}{\sum_{l=0}^{K-1}\exp\left(\boldsymbol{x}^T\boldsymbol{\theta}_l\right)}, \qquad k=0,\dots,K-1,} \tag{5.27}$$

which is the *softmax* function. It is over-parametrised: adding the same vector to every $\boldsymbol{\theta}_k$ leaves all the probabilities unchanged, since the added factor cancels between numerator and denominator, so the parameters are determined only up to a common shift. Fixing $\boldsymbol{\theta}_{K-1}=\mathbf{0}$ recovers Eq. (5.25), and adding an ℓ_2 penalty also removes the redundancy by selecting the smallest representative. The redundancy is harmless in practice and the symmetric form is easier to implement.

The corresponding cost function is the multiclass cross entropy,

$$C(\boldsymbol{\Theta}) = -\sum_{i=1}^{n}\sum_{k=0}^{K-1} y_{ik}\log p(C=k\mid \boldsymbol{x}_i), \tag{5.28}$$

where y_{ik} is the *one-hot* encoding, equal to one if sample i belongs to class k and zero otherwise. For $K=2$ this reduces to Eq. (5.13). Its gradient with respect to $\boldsymbol{\theta}_k$ retains the familiar structure,

$$\frac{\partial C}{\partial \boldsymbol{\theta}_k} = -\boldsymbol{X}^T\left(\boldsymbol{y}_k - \boldsymbol{p}_k\right), \tag{5.29}$$

with $\boldsymbol{y}_k$ and $\boldsymbol{p}_k$ the columns of the one-hot targets and of the predicted probabilities for class k – again design matrix transposed against residual.

A numerical warning.

Equation (5.27) must never be evaluated as written. If any $\boldsymbol{x}^T\boldsymbol{\theta}_k$ is large, exp of it overflows. Since the softmax is invariant under subtracting a constant from every score, one subtracts the largest before exponentiating,

$$p_k = \frac{\exp\left(s_k - s_{\max}\right)}{\sum_l \exp\left(s_l - s_{\max}\right)}, \qquad s_k = \boldsymbol{x}^T\boldsymbol{\theta}_k, \tag{5.30}$$

which is mathematically identical and numerically safe: every exponent is now non-positive, so no term exceeds one. This trick appears in the implementation of Section 5.8 and in every deep learning library.

The softmax will reappear in the next chapter as the output layer of a classification network, where Eq. (5.27) applied to the final linear layer is exactly multinomial logistic regression performed on learned rather than raw features.

5.7 Optimising the cross entropy

Having no closed form, we must minimise Eq. (5.13) numerically. This is where Chapter 4 is spent, and the good news is that every method there applies without modification: the problem is convex, the gradient is Eq. (5.17), and the Hessian is Eq. (5.19).

Gradient descent.

The simplest scheme is Eq. (4.15) with the logistic gradient,

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \gamma \boldsymbol{X}^T (\boldsymbol{y} - \boldsymbol{p}(\boldsymbol{\theta}_k)), \tag{5.31}$$

noting the plus sign, which comes from the minus in Eq. (5.17). The stability bound (4.20) applies with $\lambda_{\max}$ the largest eigenvalue of $\boldsymbol{X}^T \boldsymbol{W} \boldsymbol{X}$; since $W_{ii} \le \frac{1}{4}$, a safe choice is $\gamma < 8/\lambda_{\max}(\boldsymbol{X}^T \boldsymbol{X})$. For large data sets one uses the minibatch stochastic version of Section 4.7, and in practice one of the adaptive methods of Sections 4.9 to 4.11; Section 5.9 runs all of them on a logistic regression with 50000 observations and measures what each buys.

Newton-Raphson, or iteratively reweighted least squares.

Because the Hessian is available in the closed form (5.19), and because p is often modest, Newton's method of Section 4.2 is frequently the method of choice here – one of the few places in this book where it is affordable. Substituting Eqs. (5.17) and (5.19) into the Newton step (4.8),

$$\boldsymbol{\theta}^{\text{new}} = \boldsymbol{\theta}^{\text{old}} - \left(\frac{\partial^2 C}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}^T} \right)^{-1}_{\boldsymbol{\theta}^{\text{old}}} \left(\frac{\partial C}{\partial \boldsymbol{\theta}} \right)_{\boldsymbol{\theta}^{\text{old}}}, \tag{5.32}$$

that is, in matrix form,

$$\boxed{\boldsymbol{\theta}^{\text{new}} = \boldsymbol{\theta}^{\text{old}} + \left(\boldsymbol{X}^T \boldsymbol{W} \boldsymbol{X}\right)^{-1} \boldsymbol{X}^T (\boldsymbol{y} - \boldsymbol{p}),} \tag{5.33}$$

with the right-hand side evaluated at the old parameters. Unlike the quadratic case of Eq. (4.9), one step does not suffice, since $\boldsymbol{W}$ and $\boldsymbol{p}$ must be recomputed; but convergence is quadratic and a handful of iterations is typical.

Equation (5.33) has an illuminating rewriting. Defining the *adjusted response* $\boldsymbol{z} = \boldsymbol{X}\boldsymbol{\theta}^{\text{old}} + \boldsymbol{W}^{-1}(\boldsymbol{y} - \boldsymbol{p})$, a line of algebra turns it into

$$\boldsymbol{\theta}^{\text{new}} = \left(\boldsymbol{X}^T \boldsymbol{W} \boldsymbol{X}\right)^{-1} \boldsymbol{X}^T \boldsymbol{W} \boldsymbol{z}, \tag{5.34}$$

which is precisely the *weighted least squares* solution (3.14) with weights $\boldsymbol{W}$ and targets $\boldsymbol{z}$. Logistic regression is therefore solved by repeatedly performing a weighted linear regression, recomputing the weights at each pass – whence the name *iteratively reweighted least squares* (IRLS). It is a pleasing connection: the classifier reduces to a sequence of the linear fits of Chapter 3.

In practice one does not form the inverse in Eq. (5.33). Since $\boldsymbol{X}^T \boldsymbol{W} \boldsymbol{X}$ is symmetric positive semi-definite, the step is obtained from a Cholesky factorisation as in Section 1.14, or more safely still from the QR or SVD route of Section 3.3 applied to $\boldsymbol{W}^{1/2}\boldsymbol{X}$, which avoids squaring the condition number.

```
import numpy as np

def sigmoid(z):
    """Numerically stable logistic function, Eq. (5.sigmoid)."""
    out = np.empty_like(z, dtype=float)
    pos, neg = z >= 0, z < 0
    out[pos] = 1.0 / (1.0 + np.exp(-z[pos]))
    ez = np.exp(z[neg])                       # avoids overflow for z << 0
    out[neg] = ez / (1.0 + ez)
    return out
```

```
def logreg_newton(X, y, n_iter=25, tol=1e-10, ridge=1e-8):
    """Logistic regression by Newton-Raphson, Eq. (5.newton).

    A tiny ridge term keeps X^T W X invertible when the classes are
    separable or W becomes numerically singular.
    """
    n, p = X.shape
    theta = np.zeros(p)
    for it in range(n_iter):
        prob = sigmoid(X @ theta)
        gradient = X.T @ (y - prob)                      # Eq. (5.gradient)
        W = prob * (1.0 - prob)                          # Eq. (5.Wmatrix)
        H = X.T @ (W[:, None] * X) + ridge * np.eye(p)   # Eq. (5.hessian)
        step = np.linalg.solve(H, gradient)              # never invert H
        theta += step
        if np.linalg.norm(step) < tol:
            break
    return theta, it + 1
```

Note the stable sigmoid. Evaluating $1/(1+e^{-z})$ directly overflows for large negative z, and the two-branch form above is the standard remedy – the same defensive reasoning as the softmax shift of Eq. (5.30).

Machine learning connection. Which optimiser to use is decided by the shape of the problem, and logistic regression makes the trade-off unusually clear. Newton's method costs $\mathcal{O}(np^2)$ to form the Hessian and $\mathcal{O}(p^3)$ to factorise it, but converges in a handful of iterations; for p in the tens or hundreds it is unbeatable, and it is what `scikit-learn` uses through its `lbfgs` and `newton-cg` solvers. For p in the millions – text classification with a bag-of-words representation, say – the Hessian is unavailable and one falls back on SGD with an adaptive method. The dividing line is exactly the one drawn in Section 4.13, and logistic regression sits close enough to it that both sides are commonly seen.

5.8 An implementation

We now assemble a class handling both the binary and the multiclass case, trained by gradient descent. It is deliberately written to mirror the equations: the reader should be able to point at each line and name the formula it implements.

```
import numpy as np

class LogisticRegression:
    """Logistic regression for binary and multiclass classification.

    Binary problems use the sigmoid (5.sigmoid) with the cross entropy
    (5.crossentropy); multiclass problems use the softmax (5.softmax)
    with the multiclass cross entropy (5.multicrossentropy).
    """

    def __init__(self, lr=0.01, epochs=1000, fit_intercept=True, lmbda=0.0):
        self.lr = lr                        # learning rate for gradient descent
        self.epochs = epochs
        self.fit_intercept = fit_intercept
        self.lmbda = lmbda                  # l2 penalty, Eq. (5.penalised)
        self.weights = None
        self.multi_class = False

    @staticmethod
    def _add_intercept(X):
```

```
        return np.concatenate((np.ones((X.shape[0], 1)), X), axis=1)

    @staticmethod
    def _sigmoid(z):
        out = np.empty_like(z, dtype=float)
        pos, neg = z >= 0, z < 0
        out[pos] = 1.0 / (1.0 + np.exp(-z[pos]))
        ez = np.exp(z[neg])
        out[neg] = ez / (1.0 + ez)
        return out

    @staticmethod
    def _softmax(Z):
        """Softmax with the shift of Eq. (5.softmaxstable)."""
        expZ = np.exp(Z - np.max(Z, axis=1, keepdims=True))
        return expZ / np.sum(expZ, axis=1, keepdims=True)

    def fit(self, X, y):
        X, y = np.asarray(X, dtype=float), np.asarray(y)
        if self.fit_intercept:
            X = self._add_intercept(X)
        n_samples, n_features = X.shape

        self.classes_ = np.unique(y)
        self.multi_class = len(self.classes_) > 2

        if self.multi_class:
            n_classes = len(self.classes_)
            index = {c: k for k, c in enumerate(self.classes_)}
            Y = np.zeros((n_samples, n_classes))            # one-hot targets
            Y[np.arange(n_samples), [index[c] for c in y]] = 1
            self.weights = np.zeros((n_features, n_classes))

            for _ in range(self.epochs):
                probs = self._softmax(X @ self.weights)
                grad = X.T @ (probs - Y) / n_samples        # Eq. (5.softmaxgradient)
                grad += 2.0 * self.lmbda * self.weights
                self.weights -= self.lr * grad
        else:
            yb = (y == self.classes_[1]).astype(float)      # map labels to {0,1}
            self.weights = np.zeros(n_features)

            for _ in range(self.epochs):
                probs = self._sigmoid(X @ self.weights)
                grad = X.T @ (probs - yb) / n_samples       # Eq. (5.gradient)
                grad += 2.0 * self.lmbda * self.weights
                self.weights -= self.lr * grad
        return self

    def predict_proba(self, X):
        X = np.asarray(X, dtype=float)
        if self.fit_intercept:
            X = self._add_intercept(X)
        if self.multi_class:
            return self._softmax(X @ self.weights)
        p1 = self._sigmoid(X @ self.weights)
        return np.column_stack([1.0 - p1, p1])

    def predict(self, X):
        return self.classes_[np.argmax(self.predict_proba(X), axis=1)]
```

Note that the gradients carry a factor $1/n$ absent from Eq. (5.17). This merely rescales the learning rate and is the usual convention, since it makes a sensible γ independent of the size

of the data set; but it is one more of the bookkeeping factors that must be matched when comparing implementations.

Synthetic data.

To exercise the class we generate Gaussian clusters, one per class.

```
import numpy as np

def generate_binary_data(n_samples=100, n_features=2, random_state=None):
    """Two Gaussian clusters, class 0 around -2 and class 1 around +2."""
    rng = np.random.default_rng(random_state)
    n0 = n_samples // 2
    n1 = n_samples - n0
    X0 = rng.normal(size=(n0, n_features)) - 2.0
    X1 = rng.normal(size=(n1, n_features)) + 2.0
    return np.vstack((X0, X1)), np.array([0] * n0 + [1] * n1)


def generate_multiclass_data(n_samples=150, n_features=2, n_classes=3,
                             random_state=None):
    """One Gaussian cluster per class, centred on a circle."""
    rng = np.random.default_rng(random_state)
    per = n_samples // n_classes
    Xs, ys = [], []
    for k in range(n_classes):
        angle = 2.0 * np.pi * k / n_classes
        centre = 4.0 * np.array([np.cos(angle), np.sin(angle)])
        centre = np.resize(centre, n_features)
        Xs.append(rng.normal(size=(per, n_features)) + centre)
        ys.append(np.full(per, k))
    return np.vstack(Xs), np.concatenate(ys)


X, y = generate_binary_data(200, random_state=2024)
model = LogisticRegression(lr=0.1, epochs=2000).fit(X, y)
print("binary training accuracy:", np.mean(model.predict(X) == y))

Xm, ym = generate_multiclass_data(300, n_classes=3, random_state=2024)
multi = LogisticRegression(lr=0.1, epochs=2000).fit(Xm, ym)
print("multiclass training accuracy:", np.mean(multi.predict(Xm) == ym))
```

Both problems are separable by construction, so both accuracies are close to unity. This is the moment to recall the warning of Section 5.5: on perfectly separable data the unpenalised likelihood has no finite maximiser, and the weights grow without bound as training continues. With a fixed number of epochs the growth is simply truncated; setting `lmbda` to a small positive value is the principled fix.

5.8.1 Logistic regression as a single neuron

Before we take the class apart with automatic differentiation, it is worth drawing it. Written out for one observation with p features, the model of Eq. (5.10) is

$$z = \theta_0 + \sum_{j=1}^{p} \theta_j x_j, \qquad \hat{p} = \sigma(z) = P(y = 1 \mid \boldsymbol{x}), \tag{5.35}$$

a weighted sum of the inputs, shifted by the intercept and passed through one non-linear function. Figure 5.4 shows Eq. (5.35) as a diagram: the features enter on the left, each edge carries a parameter, the node Σ forms the linear predictor z, the node σ squashes it into a probability, and, if we also attach the label, the cross entropy of Eq. (5.14) is one more node at the far right. This diagram is the *simple perceptron* of Rosenblatt, mentioned in Section 5.1, with one change: the step function that made the perceptron a hard classifier has been replaced by the sigmoid. In the language of Chapter 8, which begins with this very picture, the x_j are the inputs of an *artificial neuron*, the θ_j are its *weights*, the intercept θ_0 is its *bias*, σ is its *activation function* and z its *pre-activation*. Logistic regression is a neural network with a single neuron and no hidden layer; nothing in this chapter needs to be renamed when we get there.

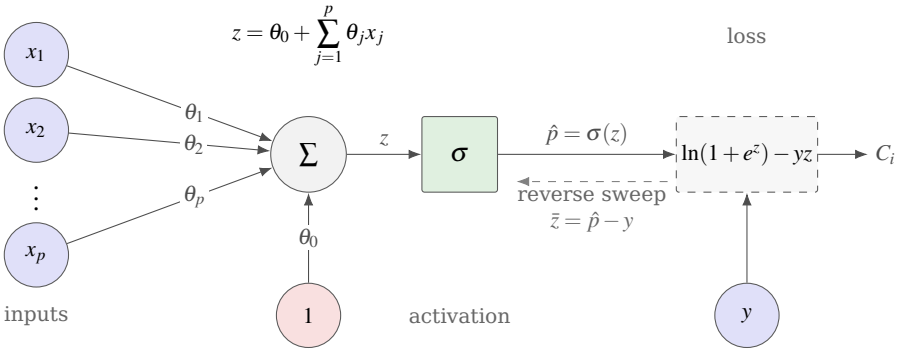


Fig. 5.4 Logistic regression drawn as a single artificial neuron, Eq. (5.35): the features x_j enter on the left, the edges carry the parameters θ_j, the intercept θ_0 is the weight of an input permanently fixed at one, the node Σ forms the linear predictor z and the activation σ turns it into the probability $\hat{p}$. Attaching the label y and the cross entropy of Eq. (5.14) on the right gives the cost of one observation. With the step function in place of σ this is Rosenblatt's perceptron. Read from right to left, the same diagram is the reverse sweep of Section 5.8.2: the sensitivity of the cost to the pre-activation is the residual $\hat{p} - y$, and the gradient with respect to θ_j is that residual times the input x_j on the corresponding edge.

The perceptron learning rule.

The picture makes the gradient obvious. The cost of one observation depends on the parameters only through z, so by the chain rule $\partial C_i/\partial\theta_j = (\partial C_i/\partial z)(\partial z/\partial\theta_j)$. The first factor is the derivative of $\ln(1+e^z) - y_i z$, namely $\sigma(z) - y_i = \hat{p}_i - y_i$, the summand of Eq. (5.15); the second is the input x_{ij} sitting on the edge of θ_j (with $x_{i0} = 1$ for the bias). Hence

$$\frac{\partial C_i}{\partial \theta_j} = (\hat{p}_i - y_i) x_{ij}, \qquad \nabla_{\boldsymbol{\theta}} C_i = (\hat{p}_i - y_i)\boldsymbol{x}_i, \tag{5.36}$$

and a gradient step on this one observation reads

$$\theta_j \;\leftarrow\; \theta_j - \gamma(\hat{p}_i - y_i) x_{ij}. \tag{5.37}$$

Equation (5.37) is, up to the choice of activation, Rosenblatt's original perceptron learning rule of 1958, and with a differentiable activation it is the *delta rule* of Widrow and Hoff: increase the weight on every input that was active when the output was too small, decrease it when the output was too large, in proportion to the error. Summing Eq. (5.36) over the observations gives back $\boldsymbol{X}^T(\boldsymbol{p} - \boldsymbol{y})$, the gradient (5.17) we derived by algebra in Section 5.4.

Applying Eq. (5.37) one observation, or a few observations, at a time rather than summing first is stochastic gradient descent, to which Section 5.9 is devoted.

From the neuron to the computational graph.

Figure 5.4 can be read in two ways, and the difference between them is the subject of the next subsection. Read as a *model*, information flows from left to right: the data $\boldsymbol{x}$ enter, the parameters are fixed numbers on the edges, and a probability comes out. Read as a *computational graph* in the sense of Section 4.14.2, it is the same nodes and the same edges, but the roles are exchanged: the parameters are the inputs with respect to which we differentiate, the data are constants of the trace, and the output is the scalar cost C_i. Automatic differentiation in reverse mode is then nothing but the neuron read backwards. The adjoint of the cost is one; it passes through the loss node to give the sensitivity of C_i to $\hat{p}$, through the activation to give the sensitivity to the pre-activation, $\bar{z} = \hat{p} - y$, and from there along each edge to the parameter it carries, picking up the factor x_j on the way – which is Eq. (5.36) once more, now produced by the fan-out rule rather than by the chain rule written out by hand. The next subsection performs exactly this sweep on the trace of the graph, node by node, and compares it with the forward sweep. The reason for insisting on the picture is what comes after: in Chapter 8 the inputs x_j of this neuron will themselves be the outputs of other neurons, the reverse sweep will not stop at $\bar{z}$ but continue along the edges into the layer before, and Eq. (5.36) will be applied layer by layer. That is backpropagation, and Figure 5.4 is its one-neuron case.

Why the sigmoid, and not the step. The perceptron and logistic regression share Figure 5.4, and the reason the latter replaced the former is visible in it. With the step function as activation the node σ has derivative zero almost everywhere, the reverse sweep dies at that node, and no gradient reaches the parameters: Rosenblatt's rule had to be justified by a separate convergence argument that works only for separable data. With the sigmoid every node of the graph is differentiable, the sweep passes through, and the rule becomes an instance of gradient descent on a convex cost, with all of Chapter 4 behind it. The same requirement – that the activation be differentiable so that the sweep can pass through it – is what shapes the choice of activation functions in Chapter 8.

5.8.2 The same algorithm by automatic differentiation

The class of Section 5.8 hard-wires the gradient (5.17), which we derived by hand in Section 5.4 and read off the neuron of Figure 5.4 in Section 5.8.1. Logistic regression is simple enough that this was no hardship, but it is also simple enough to show, node by node, what the automatic differentiation of Section 4.14 does with a machine learning cost – and why every framework runs it in reverse mode. We take the compact form (5.14) of the cross entropy and follow one observation (x_i, y_i) through it; the subscript i is dropped until we reassemble the sum.

The evaluation trace.

The inputs of the graph are the *parameters*, since it is with respect to them that we differentiate; the data x and y are constants of the trace. Numbering as in Section 4.14.2, with $v_{-1} = \theta_0$ and $v_0 = \theta_1$, a program evaluates the cost of one observation as

$$\begin{aligned} v_1 &= v_0 x, & v_2 &= v_1 + v_{-1}, & v_3 &= \exp v_2, & v_4 &= 1 + v_3, \\ v_5 &= \ln v_4, & v_6 &= y v_2, & v_7 &= v_5 - v_6, & C_i &= v_7, \end{aligned} \tag{5.38}$$

so that $v_2 = \theta_0 + \theta_1 x$ is the linear predictor, $v_5 = \ln(1 + e^{v_2})$ the softplus, and $v_7 = \ln(1 + e^z) - yz$ the term in brackets of Eq. (5.14). The computational graph is drawn in Fig. 5.5; it is the neuron of Fig. 5.4 with the parameters as inputs and the loss attached, the summation node split into its elementary operations. Its one structural feature is that the linear predictor v_2 is used twice, by the exponential and by the product with the label, exactly like the fan-out of Section 4.14.5; every generalised linear model has this shape, a linear predictor feeding both the link function and the term that couples it to the label. The local derivatives are

$$\begin{aligned} \frac{\partial v_1}{\partial v_0} &= x, & \frac{\partial v_2}{\partial v_1} &= \frac{\partial v_2}{\partial v_{-1}} = 1, & \frac{\partial v_3}{\partial v_2} &= v_3, & \frac{\partial v_4}{\partial v_3} &= 1, \\ \frac{\partial v_5}{\partial v_4} &= \frac{1}{v_4}, & \frac{\partial v_6}{\partial v_2} &= y, & \frac{\partial v_7}{\partial v_5} &= 1, & \frac{\partial v_7}{\partial v_6} &= -1, \end{aligned} \tag{5.39}$$

and everything that follows combines these eight numbers.

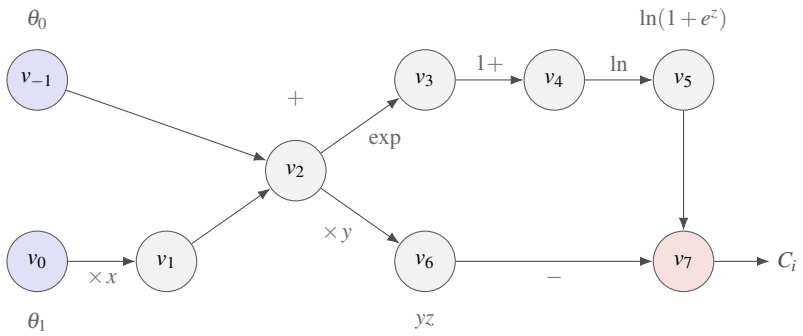


Fig. 5.5 The computational graph of the cross entropy of one observation, trace (5.38). The inputs are the two parameters; the data x and y enter as constants on the edges. The linear predictor $z = v_2$ fans out into the softplus branch along the top and the label branch along the bottom, and the reverse sweep of Eq. (5.41) sums the two when it reaches v_2, producing the residual $p - y$.

Forward mode.

A forward sweep, Eq. (4.81), needs a seed. Seeding the first parameter, $(\dot{v}_{-1}, \dot{v}_0) = (1, 0)$, and pushing the tangent through the trace gives

$$\begin{aligned} &\dot{v}_1 = x\dot{v}_0 = 0, && \dot{v}_2 = \dot{v}_1 + \dot{v}_{-1} = 1, && \dot{v}_3 = v_3\dot{v}_2 = v_3, \\ &\dot{v}_4 = \dot{v}_3 = v_3, && \dot{v}_5 = \dot{v}_4/v_4 = v_3/v_4 = p, && \dot{v}_6 = y\dot{v}_2 = y, \\ &\dot{v}_7 = \dot{v}_5 - \dot{v}_6 = p - y, \end{aligned} \tag{5.40}$$

where we recognised $v_3/v_4 = e^z/(1+e^z) = \sigma(z) = p$. The sweep has returned $\partial C_i/\partial\theta_0 = p - y$, which is the summand of Eq. (5.15) – but only that. For $\partial C_i/\partial\theta_1$ the whole sweep is run again with the seed $(0, 1)$: now $\dot{v}_1 = x$, $\dot{v}_2 = x$, and every subsequent tangent is the one above multiplied by x, ending in $\dot{v}_7 = x(p - y)$, the summand of Eq. (5.16). Two parameters, two sweeps. At $\boldsymbol{\theta} = (0.5, 0.25)$ for the observation $x = 2$, $y = 1$, the tape holds $z = 1$, $v_3 = 2.718$, $v_4 = 3.718$, $v_5 = 1.313$, $C_i = 0.313$, $p = 0.731$, and the two sweeps return -0.269 and -0.538.

Reverse mode.

The reverse sweep, Eq. (4.87), starts from the output with $\bar{v}_7 = 1$ and works back through the trace, using the values on the tape:

$$\begin{aligned} \bar{v}_5 &= \bar{v}_7 \cdot 1 = 1, & \bar{v}_6 &= \bar{v}_7 \cdot (-1) = -1, & \bar{v}_4 &= \bar{v}_5 / v_4 = 1/v_4, \\ \bar{v}_3 &= \bar{v}_4 \cdot 1 = 1/v_4, & \bar{v}_2 &= \bar{v}_3 v_3 + \bar{v}_6 y = \frac{v_3}{v_4} - y = p - y, & \bar{v}_1 &= \bar{v}_2 = p - y, \\ \bar{v}_{-1} &= \bar{v}_2 = p - y, & \bar{v}_0 &= \bar{v}_1 x = x(p - y). \end{aligned} \tag{5.41}$$

One sweep, and *both* partial derivatives are read off at the input nodes. Two things in Eq. (5.41) deserve a second look. The adjoint of v_2 collects one contribution through each of its successors – the softplus branch contributes p, the label branch contributes $-y$ – and the sum is the residual $p - y$: the quantity that Section 5.4 obtained by differentiating the log-odds by hand appears here as the sensitivity of the cost to the linear predictor, produced mechanically by the fan-out rule. And from v_2 down to the parameters the sweep does nothing but multiply the residual by the feature attached to each parameter, 1 for the intercept and x for the slope. Both facts are the derivation of Eqs. (5.15) and (5.16), done by the machine.

The whole data set, and why reverse mode wins.

The cost is the sum over observations, $C = \sum_i C_i$ (or the mean, in the convention of the class above), and its graph is n copies of Fig. 5.5 hanging from the *same* parameter nodes and feeding one final summation node. With p parameters rather than two, v_2 becomes $z_i = \boldsymbol{X}_{i,*}\boldsymbol{\theta}$ and each observation's graph has p input edges instead of two. Consider what the two modes do with it.

In reverse mode the summation node passes $\bar{C}_i = 1$ to every copy, each copy produces its residual $r_i = p_i - y_i$ at z_i by Eq. (5.41), and each parameter node θ_j accumulates $\sum_i X_{ij} r_i$ from its n incoming edges. The sweep has computed

$$\bar{\boldsymbol{\theta}} = \boldsymbol{X}^T(\boldsymbol{p} - \boldsymbol{y}), \tag{5.42}$$

which is Eq. (5.17), in a single pass whose cost is that of the forward evaluation – one matrix–vector product $\boldsymbol{X}\boldsymbol{\theta}$ and n scalar operations – plus one more matrix–vector product $\boldsymbol{X}^T\boldsymbol{r}$ on the way back: $\mathcal{O}(np)$ in all, a small constant times the cost of evaluating C itself, precisely as the cheap gradient principle, Proposition 4.2, promises. The tape it needs is the n values z_i (from which p_i follows), a vector.

In forward mode a sweep is seeded with one direction in parameter space. Seeding $\boldsymbol{e}_j$ gives $\dot{z}_i = X_{ij}$, then $\dot{C}_i = X_{ij} r_i$ by the argument that led to Eq. (5.40), and finally $\dot{C} = \sum_i X_{ij} r_i$ – one component of the gradient. Each sweep costs $\mathcal{O}(np)$, and there are p of them: $\mathcal{O}(np^2)$ for the gradient, a factor p worse than reverse mode. The disparity is not an artefact of the example. Forward mode propagates one input direction to all outputs; reverse mode propagates one output direction to all inputs. A cost function has one output and p inputs, and training means asking for all p sensitivities of that one output. That is the reverse-mode question, and it is why backpropagation – reverse-mode AD on the layered graph of a network – is the algorithm of Chapter 8, where p runs to millions. The price is the tape: the values of every intermediate node must be kept until the backward sweep has consumed them, which for logistic regression is n numbers and for a deep network is every activation of every layer. Forward mode keeps nothing but the current tangent, and remains the right tool when the inputs are few and the outputs many – the Jacobian of a network's outputs with respect to

one or two physical parameters, or the derivatives with respect to the spatial coordinates in Chapter 9.

The listing below runs both sweeps, first on the single observation of Fig. 5.5 – the reverse sweep written out by hand as in Eq. (5.41), and both modes obtained from JAX with `jax.jvp` and `jax.grad` – and then on a data set, where the reverse-mode gradient is checked against Eq. (5.17) and the two modes are timed against the cost of evaluating C alone. The functions are written as ordinary NumPy-style code; JAX derives the trace from them.

```
import time
import numpy as np
import jax
jax.config.update("jax_enable_x64", True)
import jax.numpy as jnp

def cross_entropy_one(theta0, theta1, x, y):
    """The trace of Eq. (5.adtrace) for one observation, one line per node."""
    v1 = theta1 * x
    v2 = v1 + theta0
    v3 = jnp.exp(v2)
    v4 = 1.0 + v3
    v5 = jnp.log(v4)
    v6 = y * v2
    return v5 - v6

def reverse_sweep(theta0, theta1, x, y):
    """Eq. (5.adreverse) by hand: forward evaluation onto the tape, then back."""
    v1 = theta1 * x; v2 = v1 + theta0; v3 = np.exp(v2); v4 = 1.0 + v3   # tape
    b7 = 1.0
    b5, b6 = b7, -b7
    b4 = b5 / v4
    b3 = b4
    b2 = b3 * v3 + b6 * y                  # fan-out: two successors, one sum
    b1 = b2
    return float(b2), float(b1 * x)        # dC/dtheta0, dC/dtheta1

x, y = 2.0, 1.0
theta = jnp.array([0.5, 0.25])
cost_one = lambda th: cross_entropy_one(th[0], th[1], x, y)
print("C_i =", float(cost_one(theta)))
for k in range(2):                         # forward mode: one sweep per parameter
    _, tangent = jax.jvp(cost_one, (theta,), (jnp.eye(2)[k],))
    print(f"forward sweep, seed e_{k}: dC/dtheta_{k} = {float(tangent):.6f}")
print("reverse sweep, jax.grad:", np.asarray(jax.grad(cost_one)(theta)))
print("reverse sweep, by hand: ", reverse_sweep(0.5, 0.25, x, y))

# the whole data set: mean cross entropy, Eq. (5.crossentropycompact) / n
def cost(theta, X, y):
    z = X @ theta
    return jnp.mean(jnp.logaddexp(0.0, z) - y * z)

rng = np.random.default_rng(2024)
n = 5000
for p in (20, 200, 1000):
    X = jnp.asarray(np.column_stack([np.ones(n), rng.normal(size=(n, p - 1))]))
    yb = jnp.asarray(rng.integers(0, 2, n).astype(float))
    th = jnp.zeros(p)
    value = jax.jit(cost)
    reverse = jax.jit(jax.grad(cost))          # one reverse sweep
    forward = jax.jit(jax.jacfwd(cost))        # p forward sweeps
    for fn in (value, reverse, forward):       # compile before timing
        fn(th, X, yb).block_until_ready()
    p_hat = jax.nn.sigmoid(X @ th)
    g_hand = X.T @ (p_hat - yb) / n            # Eq. (5.gradient) / n
```

```
    print(f"p = {p:4d}: max|reverse - hand| = "
          f"{float(jnp.max(jnp.abs(reverse(th, X, yb) - g_hand))):.1e}", end="")
    for name, fn in (("value", value), ("reverse", reverse), ("forward", forward)):
        t0 = time.perf_counter()
        for _ in range(10):
            fn(th, X, yb).block_until_ready()
        print(f"   {name} {1e3 * (time.perf_counter() - t0) / 10:7.2f} ms", end="")
    print()
```

```
C_i = 0.3132616875182228
forward sweep, seed e_0: dC/dtheta_0 = -0.268941
forward sweep, seed e_1: dC/dtheta_1 = -0.537883
reverse sweep, jax.grad: [-0.26894142 -0.53788284]
reverse sweep, by hand:  (-0.2689414213699952, -0.5378828427399904)
p =   20: max|reverse - hand| = 3.5e-17   value    0.13 ms   reverse    0.19 ms   forward
     0.41 ms
p =  200: max|reverse - hand| = 6.8e-17   value    0.32 ms   reverse    0.47 ms   forward
     4.53 ms
p = 1000: max|reverse - hand| = 9.7e-17   value    1.59 ms   reverse    3.31 ms   forward
    85.92 ms
```

The single-observation numbers are those quoted above, and the hand-written sweep, the forward sweeps and `jax.grad` agree to rounding. On the data set the reverse-mode gradient matches Eq. (5.17) to 10^{-16}, and the timings (from a laptop, and only their ratios matter) say the rest: the reverse-mode gradient costs between 1.4 and 2 times the evaluation of C whatever the number of parameters, as Proposition 4.2 requires, while the forward-mode gradient costs 3, 14 and 54 evaluations for $p = 20$, 200 and 1000, growing with p as predicted – less steeply than the factor p itself because JAX vectorises the p sweeps into one matrix product, but without bound. Section 5.10.3 takes the consequence: with the gradient supplied by one reverse sweep the loss can be changed at will, and no derivative need ever be written down again.

Machine learning connection. The residual $p_i - y_i$ that the reverse sweep produces at the linear predictor is the object that every training loop in this book propagates. In a neural network the linear predictor of the last layer is followed by more layers of the same kind, and the reverse sweep continues through them: the adjoint of each layer's output is multiplied by the transposed weight matrix and by the derivative of the activation, layer by layer, until it reaches the first weights. That is backpropagation, Chapter 8, and Eq. (5.41) is its one-layer special case.

5.9 Stochastic gradient descent for logistic regression

The two optimisers of Section 5.7 – gradient descent and Newton's method – both touch every observation before they move the parameters once. For the few hundred points of the examples so far this is of no consequence. It becomes the whole story when n is large, and Section 4.7 argued in general terms that the remedy is to update on *minibatches*: for a cost that is a sum over observations the minibatch gradient is an unbiased estimate of the full one at a fraction of the price, and the statistical accuracy worth having is reached after a few passes over the data. The cross entropy is such a sum, and logistic regression is the simplest model on which the argument can be checked rather than merely asserted. We do so here, and then run the adaptive methods of Chapter 4 on the same problem, so that when the same loop reappears in Chapter 8 with a network in place of the neuron, nothing about the optimiser will be new.

5.9.1 Minibatches and the per-observation gradient

In the $1/n$ convention of the class of Section 5.8 the cost is the mean of the per-observation costs of Eq. (5.14),

$$C(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} c_i(\boldsymbol{\theta}), \qquad c_i(\boldsymbol{\theta}) = \ln\bigl(1+e^{z_i}\bigr) - y_i z_i, \qquad z_i = \boldsymbol{x}_i^T\boldsymbol{\theta}, \tag{5.43}$$

and each term has the gradient (5.36), $\nabla c_i = (p_i - y_i)\boldsymbol{x}_i$, which we read off the neuron of Figure 5.4. A minibatch B of M observations drawn without replacement gives the estimate

$$\boldsymbol{g}_B(\boldsymbol{\theta}) = \frac{1}{M}\sum_{i\in B}(p_i - y_i)\boldsymbol{x}_i = \frac{1}{M}\boldsymbol{X}_B^T\bigl(\boldsymbol{p}_B - \boldsymbol{y}_B\bigr), \tag{5.44}$$

where $\boldsymbol{X}_B$ holds the M selected rows of the design matrix. This is Eq. (5.17) restricted to the rows in B and divided by M, so the code that computes it is the code that computes the full gradient, handed fewer rows. By Eq. (4.43) it is unbiased, $\mathbb{E}[\boldsymbol{g}_B] = \nabla C$, with a variance that scales as $1/M$; the update

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma_t\,\boldsymbol{g}_{B_t}(\boldsymbol{\theta}_t) \tag{5.45}$$

is Eq. (4.37) for the cross entropy, and with $M = 1$ it is the delta rule (5.37) applied to one observation at a time. One pass over the data, an *epoch*, consists of n/M such updates, against a single update of gradient descent.

What an epoch costs, and what it buys.

The count of Eq. (4.38) transfers directly: the minibatch gradient (5.44) is one matrix–vector product $\boldsymbol{X}_B\boldsymbol{\theta}$, M evaluations of the sigmoid and one product $\boldsymbol{X}_B^T\boldsymbol{r}_B$, about $4Mp$ floating-point operations, so that an epoch of SGD costs the same $4np$ as one step of gradient descent. The question is therefore how much each of the two buys per pass over the data, and the two extremes are easy to state. Gradient descent descends at the rate set by the condition number of the Hessian, Eq. (4.26), and needs of order $\kappa\ln(1/\varepsilon)$ passes to reach an excess cost ε. Stochastic gradient descent makes n/M moves per pass; early on, when the parameters are far from the optimum, almost every one of them points downhill and the method covers in one epoch what gradient descent covers in hundreds. Late in the run the minibatch noise takes over: at a constant learning rate the iterate does not settle at the minimum but equilibrates in the cloud of Eq. (4.50), of excess cost proportional to γ/M, and only a decaying learning rate removes it. The relevant accuracy is set by statistics, as in Section 4.7. The minimiser $\hat{\boldsymbol{\theta}}$ of the empirical cross entropy is the maximum-likelihood estimate of Section 3.6, and the classical result on the likelihood ratio says that $2n\,[C(\boldsymbol{\theta}_{\text{true}}) - C(\hat{\boldsymbol{\theta}})]$ is asymptotically χ^2-distributed with p degrees of freedom, so the parameters that generated the data sit at an excess cost of about $p/(2n)$ above the empirical minimum. Optimising below that level chases the noise of this particular sample; it is the analogue for the cross entropy of the floor $\sigma^2 p/n$ of Eq. (4.48).

The learning rate of gradient descent.

For the comparison to be fair, gradient descent must be given its best constant learning rate, and logistic regression has a feature here that least squares does not. The Hessian $\boldsymbol{X}^T\boldsymbol{W}\boldsymbol{X}/n$ of Eq. (5.19) changes along the path, because the weights $W_{ii} = p_i(1-p_i)$ do. At the starting point $\boldsymbol{\theta} = \boldsymbol{0}$ every p_i equals $\frac{1}{2}$, every weight equals its maximum $\frac{1}{4}$, and the stability bound (4.20)

reads

$$\gamma < \frac{2}{\lambda_{\max}\left(\boldsymbol{X}^T \boldsymbol{W} \boldsymbol{X}/n\right)} = \frac{8}{\lambda_{\max}\left(\boldsymbol{X}^T \boldsymbol{X}/n\right)} \qquad \text{at } \boldsymbol{\theta} = \mathbf{0}, \tag{5.46}$$

the safe choice already quoted in Section 5.7. As the fit sharpens the weights shrink, the Hessian becomes smaller, and a larger step would be admissible near the optimum – but a step chosen for the optimum overshoots at the start, where the curvature is largest. A constant learning rate must respect the largest curvature met along the way, and Eq. (5.46) is it.

5.9.2 Gradient descent against SGD, measured

We construct a problem large enough for the difference to show and small enough to run in seconds: $n = 50\,000$ observations and $p = 20$ parameters, an intercept and nineteen standardised features with the same autoregressive correlation of 0.9 between neighbouring columns as in Section 4.7, so that the design is strongly correlated. The labels are drawn as $y_i \sim \text{Bernoulli}(\sigma(\boldsymbol{x}_i^T \boldsymbol{\theta}_{\text{true}}))$ from a known parameter vector; 40% of them are ones, and even the true parameters classify only 76.7% of the points correctly, because the classes overlap. Newton's method of Section 5.7 converges in seven iterations to the reference value $C(\hat{\boldsymbol{\theta}}) = 0.47624$, against which every excess cost below is measured; the Hessian there has condition number $\kappa = 173$, and $\lambda_{\max}(\boldsymbol{X}^T \boldsymbol{X}/n) = 10.9$ gives the learning rate $\gamma = 0.73$ of Eq. (5.46) for gradient descent. The excess cost of $\boldsymbol{\theta}_{\text{true}}$ is 2.7×10^{-4}, close to the $p/(2n) = 2 \times 10^{-4}$ of the χ^2 argument; this is the dotted line in Figure 5.6, and the level below which further optimisation is pointless.

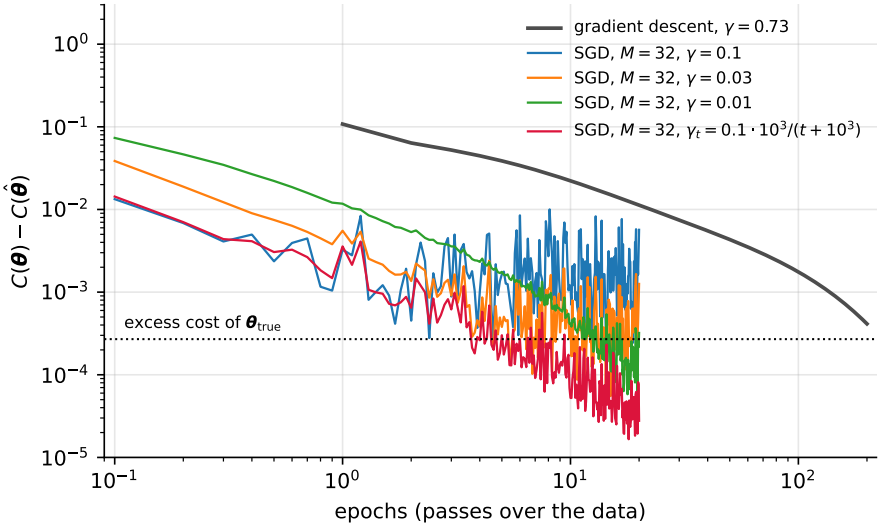


Fig. 5.6 Excess cross entropy $C(\boldsymbol{\theta}) - C(\hat{\boldsymbol{\theta}})$ against passes over the data for the logistic regression problem of Section 5.9.2 ($n = 50\,000$, $p = 20$, $\kappa = 173$). Gradient descent uses the learning rate (5.46); the SGD runs use minibatches of $M = 32$, the cost being recorded ten times per epoch, at three constant learning rates and with the decaying schedule $\gamma_t = 0.1 \cdot 10^3/(t + 10^3)$. The dotted line is the excess cost of the parameters that generated the data, about $p/(2n)$. After a single epoch every SGD run is where gradient descent is after twenty. The constant-rate runs then equilibrate in a cloud whose size grows with γ, as Eq. (4.50) predicts, the smallest rate reaching within twenty epochs the level gradient descent needs two hundred for; only the decaying schedule continues downwards.

Figure 5.6 and the first block of the output below tell the same story. Gradient descent reduces the excess cost to 1.1×10^{-1} after one pass, 3.9×10^{-2} after five, 1.1×10^{-2} after twenty and reaches 4.1×10^{-4}, about the statistical floor, only after 200 passes; the slow, steady slope is the rate $(1-2/\kappa)^k$ of a problem with $\kappa=173$. Stochastic gradient descent with $M=32$ and $\gamma=0.01$ is at 1.8×10^{-3} after five passes and 3.2×10^{-4} after twenty, so that five epochs of SGD are worth about a hundred of gradient descent and twenty are worth two hundred. Since an epoch of either costs the same $4np$ operations, this is a factor of ten to twenty in computation, obtained without tuning anything. With the decaying schedule the excess is 3.0×10^{-4} after five epochs and 2.8×10^{-5} after twenty – an order of magnitude below the floor, and still falling. The larger constant rates show the other side of the coin: $\gamma=0.1$ is fastest in the first epoch but then oscillates in a band an order of magnitude wide, and the band for $\gamma=0.03$ is narrower by about the ratio of the learning rates, as Eq. (4.50) says it should be. One more column of the output deserves attention: the accuracy. Gradient descent classifies 68.8% of the training points correctly after one pass, 74.3% after five and 76.1% after twenty, while both SGD runs of the listing are at 76.7% after twenty epochs, the accuracy of the true parameters. The accuracy saturates long before the cost does – an excess cost of 10^{-3} already gives an accuracy within half a percent of the best possible – which is why, for classification, a few epochs of SGD are usually all that is ever run.

5.9.3 AdaGrad, RMSProp and Adam on the same problem

The three adaptive methods of Sections 4.9 to 4.11 replace the single learning rate of Eq. (5.45) by one per parameter, obtained from a running average of the squared minibatch gradients: AdaGrad accumulates the squares, Eq. (4.61), RMSProp averages them with exponential forgetting, Eq. (4.64), and Adam adds a momentum-like average of the gradient itself with bias corrections, Eq. (4.71). All three are the diagonally preconditioned iteration of Eq. (4.57), and Chapter 4 drew two conclusions from that form: the preconditioner equalises the curvature *along the coordinate axes* and therefore undoes any difference in the scales of the features, but it can do nothing about curvature caused by correlation between them, which is not diagonal. Logistic regression lets us test both conclusions with one switch, because the data of the previous subsection come standardised, and we may unstandardise them.

Figure 5.7 shows the four stochastic methods – plain SGD and the three adaptive ones, minibatches of 32 throughout – and gradient descent, first on the standardised features of Figure 5.6 and then on the same data with each feature column multiplied by a fixed random factor between 0.1 and 10, the kind of disparity in units that raw data carry. For each method the constant learning rate was chosen from a grid of five values a factor of $\sqrt{10}$ apart as the one with the smallest mean excess cost over epochs 15 to 20; the listing below runs the winners, and Table 5.1 collects the numbers.

Three things stand out. First, on standardised features the adaptive methods bring nothing that plain SGD does not already have: after ten epochs all four sit at the statistical floor, within a factor of two of one another, and the ordering among them changes from epoch to epoch with the noise. The condition number $\kappa=173$ of this problem comes from the correlation between the columns, not from their scales, and a diagonal preconditioner cannot see it. Second, on the raw features the picture changes completely. Gradient descent, forced by Eq. (5.46) to a learning rate five times smaller, is at an excess cost of 3×10^{-2} after 200 epochs, worse than SGD on the standardised data was after one; plain SGD is at 10^{-2} after twenty epochs, with a training accuracy nearly a full percent below the optimum, and no constant learning rate does better, since the step that suits the columns scaled by 10 crawls

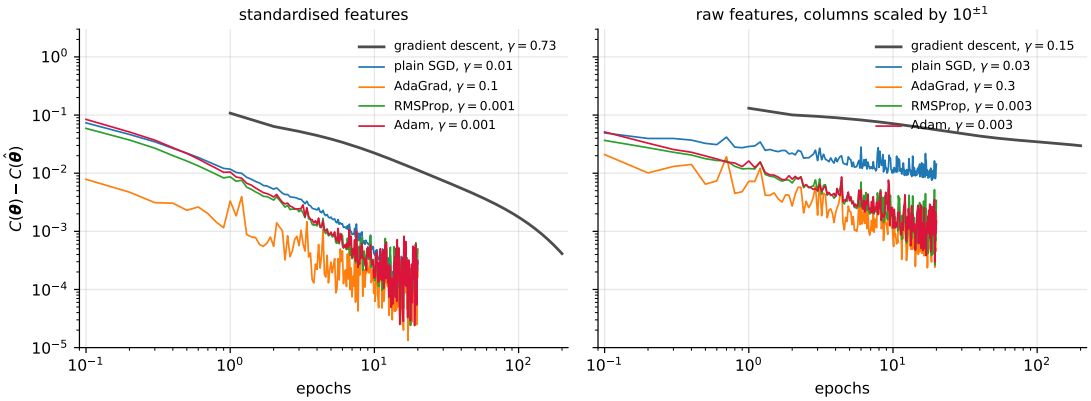


Fig. 5.7 Plain SGD, AdaGrad, RMSProp and Adam, all with $M = 32$ and each at the best of five constant learning rates, against gradient descent at the learning rate (5.46), on the logistic regression problem of Section 5.9.2. Left: standardised features, $\kappa = 173$. Right: the same data with each feature column multiplied by a random factor between 0.1 and 10, which raises the condition number of the Hessian at the optimum to 4.4×10^4 and the excess cost of the two plain-gradient methods by nearly two orders of magnitude, while the three adaptive methods lose less than one. On standardised features the four stochastic methods are indistinguishable after ten epochs.

Table 5.1 Excess cross entropy $C(\boldsymbol{\theta}) - C(\hat{\boldsymbol{\theta}})$ and training accuracy for the methods of Figure 5.7, from the output of the listing below. Gradient descent is listed after 200 epochs, the stochastic methods after 20; the column "mean" averages the excess cost at the ends of epochs 15 to 20, which smooths the fluctuations of the constant-rate runs. The excess cost of the parameters that generated the data is 2.7×10^{-4} in both cases.

Method	γ	**excess, 5 epochs**	**mean, 15–20**	**accuracy**
standardised features, $\kappa = 173$				
gradient descent (200 epochs)	0.73	3.9×10^{-2}	4.1×10^{-4}	0.7671
SGD	0.01	1.8×10^{-3}	1.9×10^{-4}	0.7666
SGD, $\gamma_t = 0.1 \cdot 10^3/(t + 10^3)$	0.1	3.0×10^{-4}	5.1×10^{-5}	0.7671
AdaGrad	0.1	3.0×10^{-4}	1.3×10^{-4}	0.7670
RMSProp	0.001	9.4×10^{-4}	2.3×10^{-4}	0.7663
Adam	0.001	9.7×10^{-4}	2.2×10^{-4}	0.7666
raw features, $\kappa = 4.4 \times 10^4$				
gradient descent (200 epochs)	0.15	8.6×10^{-2}	3.0×10^{-2}	0.7471
SGD	0.03	1.8×10^{-2}	1.2×10^{-2}	0.7585
AdaGrad	0.3	2.2×10^{-3}	9.4×10^{-4}	0.7657
RMSProp	0.003	3.6×10^{-3}	1.7×10^{-3}	0.7647
Adam	0.003	3.3×10^{-3}	1.3×10^{-3}	0.7656

on those scaled by 0.1 and the reverse diverges. The three adaptive methods recover an order of magnitude of that loss, ending at 10^{-3} with accuracies within a quarter of a percent of the optimum, because their per-parameter learning rates absorb the column scales, as the scale invariance of Section 4.8 says they must. They do not recover it all: a factor of five to ten remains against the standardised case, and we return to where it comes from below. Third, the learning rates that work differ by two orders of magnitude between the methods, and this is not a matter of tuning but of units: for plain SGD γ multiplies a gradient, while for RMSProp and Adam the update $\gamma \boldsymbol{g}/\sqrt{\boldsymbol{v}}$ has the size of γ itself in every coordinate (Section 4.11), so γ is a step length in parameter space and 10^{-3} is a sensible value; AdaGrad's ever-growing

accumulator shrinks its steps as $1/\sqrt{t}$, Eq. (4.62), which is why it tolerates, and needs, a rate a hundred times larger.

What the adaptive methods remove, and what they do not.

It is tempting to blame the correlation for the factor of five to ten that the adaptive methods still lose on the raw features, but the correlation is just as present in the standardised data, where all of them reach the floor. The explanation lies elsewhere, and it is worth making precise, because it shows what a per-parameter learning rate can and cannot do. The raw features are the standardised ones in other units, $x'_j = s_j x_j$, and since the model is the same,

$$\theta'_j = \frac{\theta_j}{s_j}, \qquad \boldsymbol{g}' = \boldsymbol{S}\boldsymbol{g}, \qquad \boldsymbol{H}' = \boldsymbol{S}\boldsymbol{H}\boldsymbol{S}, \qquad \boldsymbol{S} = \operatorname{diag}(s_1, \ldots, s_p). \tag{5.47}$$

The curvature is where the condition number 4.4×10^4 comes from, and a diagonal preconditioner removes the scales from it *exactly*: with $\boldsymbol{D} = \operatorname{diag}(\boldsymbol{H}')$ every s_j cancels in $\boldsymbol{D}^{-1/2}\boldsymbol{H}'\boldsymbol{D}^{-1/2}$, and the condition number of that matrix is 189 for the raw and for the standardised data alike. This is what rescues the adaptive methods where plain SGD, with a single γ dictated by the largest column, fails. It is also why they gain nothing on standardised features: 189 is no better than the 173 we started from, because what is left is the correlation, which sits off the diagonal.

What Eq. (5.47) does not leave invariant is the *distance* each parameter has to travel. The update of Adam and RMSProp has a length of about γ in every coordinate, whatever its unit, Eq. (4.72); but the parameters now live on very different scales, from $\hat{\theta}'_j = 7.2$ for the column with the smallest factor, $s_j = 0.10$, to $\hat{\theta}'_j = 0.08$ for the one with the largest, $s_j = 6.2$. Measured in the standardised units, in which all parameters are of order one, a step is γs_j: sixty times longer for the largest column than for the smallest. Figure 5.8 follows Adam for sixty epochs on both versions of the data and shows the two consequences. The parameters of the small columns *arrive late*: the largest of them is 87%, 54%, 32%, 13% and 2% short of its optimum after 1, 5, 10, 20 and 40 epochs, so that at the twenty epochs of Table 5.1 the run has simply not finished. The parameters of the large columns *rattle*: for them $\gamma = 0.003$ is a step eighteen times that of the standardised run, and over epochs 50 to 60 their root-mean-square error is 0.05, against 0.002 for the small columns and 0.007 for every parameter of the standardised run. No single γ is right for both groups. A smaller one, $\gamma = 0.001$, quiets the rattle but makes the late parameters three times later: the excess cost is 2.1×10^{-3} after twenty epochs and reaches 2.5×10^{-4} only after sixty. A larger one, $\gamma = 0.01$, brings every parameter in within five epochs and then rattles at 4×10^{-3} for as long as we care to run. A decaying schedule would reconcile the two, at the price of two more constants to tune; on the standardised features none of this is needed. The adaptive methods equalise the step, not the distance to go; standardising the features equalises both, and that is why it helps every method in Table 5.1, including the ones that are invariant to the scale of the gradient.

The listing runs every method of Table 5.1. The optimiser is the step function of Section 4.12; the data loop knows nothing about which method it is driving, and the gradient function is the same for a minibatch as for the full data, which is all that Eq. (5.44) requires.

```
import numpy as np

def sigmoid(t):                          # the stable form of Section 5.7
    a = np.exp(-np.abs(t))
    return np.where(t >= 0, 1.0 / (1.0 + a), a / (1.0 + a))

def make_data(n=50_000, p=20, rho=0.9, seed=2026):
```

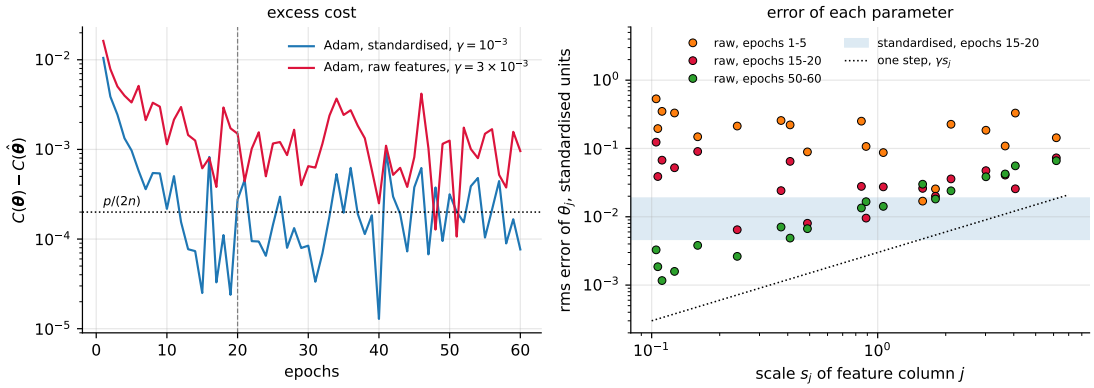


Fig. 5.8 Adam with $M = 32$ followed for sixty epochs on the standardised features ($\gamma = 10^{-3}$) and on the raw ones ($\gamma = 3 \times 10^{-3}$), the learning rates of Table 5.1. Left: excess cost at the end of each epoch; the dashed line marks the twenty epochs of the table, the dotted line the floor $p/(2n)$. Right: root-mean-square error of each parameter of the raw run, converted to standardised units, against the scale s_j of its column, over three windows of epochs; the band is the range of the standardised run and the dotted line the length γs_j of one step in these units, Eq. (5.47). The parameters of the small columns arrive late, those of the large columns never stop rattling.

```
    """AR(1)-correlated standardised features, Bernoulli labels."""
    rng = np.random.default_rng(seed)
    z = rng.normal(size=(n, p - 1))
    xc = np.empty_like(z); xc[:, 0] = z[:, 0]
    for j in range(1, p - 1):
        xc[:, j] = rho * xc[:, j - 1] + np.sqrt(1 - rho**2) * z[:, j]
    xc = (xc - xc.mean(0)) / xc.std(0)
    X = np.column_stack([np.ones(n), xc])
    theta_true = 0.5 * rng.normal(size=p)
    y = (rng.random(n) < sigmoid(X @ theta_true)).astype(float)
    scales = np.concatenate([[1.0], 10.0**rng.uniform(-1, 1, p - 1)])
    return X, y, theta_true, scales     # scales: the "raw" units

def cost(X, y, theta):                  # mean cross entropy, Eq. (5.costmean)
    z = X @ theta
    return np.mean(np.logaddexp(0.0, z) - y * z)

def gradient(X, y, theta):              # Eq. (5.minibatchgrad), any rows
    return X.T @ (sigmoid(X @ theta) - y) / X.shape[0]

def optimiser_step(method, theta, g, state, t, gamma, rho=0.99,
                   beta1=0.9, beta2=0.999, eps=1e-8):
    """The step function of Section 4.12, without momentum."""
    if method == "plain":
        return theta - gamma * g
    if method == "adagrad":                                  # Eq. (4.adagrad)
        state["r"] = r = state.get("r", 0.0) + g * g
        return theta - gamma * g / (np.sqrt(r) + eps)
    if method == "rmsprop":                                  # Eq. (4.rmsprop)
        state["r"] = r = rho * state.get("r", 0.0) + (1 - rho) * g * g
        return theta - gamma * g / (np.sqrt(r) + eps)
    if method == "adam":                                     # Eq. (4.adam)
        state["m"] = m = beta1 * state.get("m", 0.0) + (1 - beta1) * g
        state["r"] = r = beta2 * state.get("r", 0.0) + (1 - beta2) * g * g
        m_hat, r_hat = m / (1 - beta1**t), r / (1 - beta2**t)
        return theta - gamma * m_hat / (np.sqrt(r_hat) + eps)

def train(X, y, method, gamma, M=None, epochs=20, t0=None, seed=1):
    """Minibatch loop, Eq. (5.sgdupdate); M=None is full-batch gradient
```

```
    descent.  Returns theta at the end of every epoch."""
    n, p = X.shape
    M = n if M is None else M
    rng = np.random.default_rng(seed)
    theta, state, t, history = np.zeros(p), {}, 0, []
    for epoch in range(epochs):
        idx = rng.permutation(n)
        for k in range(0, n, M):                    # one epoch = n/M updates
            t += 1
            gam = gamma if t0 is None else gamma * t0 / (t + t0)
            b = idx[k:k + M]
            g = gradient(X[b], y[b], theta)
            theta = optimiser_step(method, theta, g, state, t, gam)
        history.append(theta.copy())
    return history

def newton(X, y):                           # the reference, to machine precision
    theta = np.zeros(X.shape[1])
    for _ in range(50):
        pr = sigmoid(X @ theta)
        H = X.T @ ((pr * (1 - pr))[:, None] * X) / len(y)
        step = np.linalg.solve(H, X.T @ (pr - y) / len(y))
        theta -= step
        if np.linalg.norm(step) < 1e-12:
            break
    return theta

X, y, theta_true, scales = make_data()
settings = {                                # the winners of the learning-rate grid
    "standardised": [("SGD", "plain", 0.01), ("AdaGrad", "adagrad", 0.1),
                     ("RMSProp", "rmsprop", 0.001), ("Adam", "adam", 0.001)],
    "raw":          [("SGD", "plain", 0.03), ("AdaGrad", "adagrad", 0.3),
                     ("RMSProp", "rmsprop", 0.003), ("Adam", "adam", 0.003)]}
for label, Xd, th_true in (("standardised", X, theta_true),
                           ("raw", X * scales, theta_true / scales)):
    c_star = cost(Xd, y, newton(Xd, y))
    lam_max = np.linalg.eigvalsh(Xd.T @ Xd / len(y)).max()
    gamma_gd = 8.0 / lam_max                        # Eq. (5.gdstability)
    runs = [("GD, 200 epochs",
             dict(method="plain", gamma=gamma_gd, epochs=200))]
    runs += [(name, dict(method=m, gamma=gam, M=32))
             for name, m, gam in settings[label]]
    if label == "standardised":
        runs.insert(2, ("SGD, decaying",
                        dict(method="plain", gamma=0.1, M=32, t0=1e3)))
    print(f"{label} features: C* = {c_star:.5f}, "
          f"excess of theta_true {cost(Xd, y, th_true) - c_star:.1e}")
    print("  method           gamma    excess after 1 / 5 / 20 epochs"
          "   mean 15-20   accuracy")
    for name, kw in runs:
        hist = train(Xd, y, **kw)
        ex = np.array([cost(Xd, y, th) - c_star for th in hist])
        if len(ex) > 20:                        # gradient descent, 200 epochs
            e_last, e_mean = ex[199], ex[199]
        else:                                   # SGD: epoch 20, mean of 15-20
            e_last, e_mean = ex[19], ex[14:].mean()
        acc = np.mean((Xd @ hist[-1] > 0) == y)
        print(f"  {name:16s} {kw['gamma']:<7.3g}  {ex[0]:.1e} / {ex[4]:.1e} /"
              f" {e_last:.1e}      {e_mean:.1e}     {acc:.4f}")
```

```
standardised features: C* = 0.47624, excess of theta_true 2.7e-04
  method           gamma    excess after 1 / 5 / 20 epochs   mean 15-20   accuracy
  GD, 200 epochs   0.734    1.1e-01 / 3.9e-02 / 4.1e-04      4.1e-04     0.7671
  SGD              0.01     1.2e-02 / 1.8e-03 / 3.2e-04      1.9e-04     0.7666
```

```
  SGD, decaying    0.1      3.6e-03 / 3.0e-04 / 2.8e-05      5.1e-05     0.7671
  AdaGrad          0.1      3.3e-03 / 3.0e-04 / 2.2e-04      1.3e-04     0.7670
  RMSProp          0.001    8.7e-03 / 9.4e-04 / 4.9e-04      2.3e-04     0.7663
  Adam             0.001    1.0e-02 / 9.7e-04 / 2.8e-04      2.2e-04     0.7666
raw features: C* = 0.47624, excess of theta_true 2.7e-04
  method           gamma    excess after 1 / 5 / 20 epochs   mean 15-20   accuracy
  GD, 200 epochs   0.148    1.3e-01 / 8.6e-02 / 3.0e-02      3.0e-02     0.7471
  SGD              0.03     2.9e-02 / 1.8e-02 / 1.4e-02      1.2e-02     0.7585
  AdaGrad          0.3      7.3e-03 / 2.2e-03 / 1.8e-03      9.4e-04     0.7657
  RMSProp          0.003    1.2e-02 / 3.6e-03 / 3.4e-03      1.7e-03     0.7647
  Adam             0.003    1.6e-02 / 3.3e-03 / 1.5e-03      1.3e-03     0.7656
```

For the gradient-descent rows the column “20 epochs” holds the value after 200; the raw-feature reference C^* is the standardised one, as it must be, since rescaling the columns rescales the parameters and nothing else, Section 4.8. The whole run takes a few seconds, and changing the loss, the minibatch size or the optimiser is one line each – the loop is the one that will train every network in this book.

Machine learning connection. The recommendation that falls out of Table 5.1 is the one every practitioner follows: standardise the features, as Section 3.13 advised on other grounds, use minibatches, and reach for Adam or RMSProp when the scales cannot be controlled – in a neural network the inputs to the inner layers are activations whose scales are set by the training itself, and that is where the adaptive methods earn their keep. Adam is not a substitute for standardising where standardising is possible: on the raw features it is still seven times worse than plain SGD on the standardised ones, Figure 5.8. For logistic regression itself, `scikit-learn`’s `SGDClassifier` with `loss="log_loss"` is Eq. (5.45) with a decaying learning rate and an ℓ_2 penalty, and its `partial_fit` method is the minibatch loop of the listing exposed to the user; its default `LogisticRegression` class, by contrast, is the Newton-type `lbfgs` of Section 5.7, the right choice whenever the data fit in memory and p is modest. The dividing line between the two is the one drawn in Section 4.13, now with a measured example on each side of it.

5.10 Other loss functions, and automatic differentiation

Everything so far has used the cross entropy, and Section 5.3 explained why it is the natural choice: it is the negative log-likelihood of the Bernoulli model. It is not, however, the only choice, and two of the most important classifiers of the following chapters – the support vector machine of Chapter 6 and the boosting methods of Chapter 7 – are obtained from logistic regression by changing nothing but the loss. This section sets out the family, derives what each member actually estimates, and then does something that would have been tedious a chapter ago: fits all of them with one program, in which the loss is an argument and no gradient is ever written down. That is the promise of Section 4.14 kept.

5.10.1 Margin-based losses

It is convenient to relabel the classes as $\tilde{y}_i = 2y_i - 1 \in \{-1, +1\}$ and to define the *margin* of observation i,

$$m_i = \tilde{y}_i t_i, \qquad t_i = \boldsymbol{x}_i^T \boldsymbol{\theta}, \tag{5.48}$$

which is positive when the point is on the correct side of the decision boundary $t = 0$ and large when it is far from it. The quantity we would like to minimise is the number of misclassifications, the *0–1 loss* $\ell_{01}(m) = \mathbf{1}[m < 0]$, but it is piecewise constant, so its gradient is zero wherever it exists and no gradient method can move. Every practical classifier therefore minimises a *surrogate*: a function $\ell(m)$ that upper-bounds the 0–1 loss, decreases in m, and is smooth enough or convex enough to optimise. The cross entropy is one such surrogate. Using $\sigma(-t) = 1 - \sigma(t)$, the per-observation cross entropy of Eq. (5.14) can be written as a function of the margin alone,

$$-y \log \sigma(t) - (1-y) \log[1 - \sigma(t)] = -\log \sigma(\tilde{y}t) = \log\left(1 + e^{-m}\right), \tag{5.49}$$

the *logistic loss*. The other members of the family in common use are collected in Table 5.2 and drawn in the left panel of Fig. 5.9.

Loss	$\ell(m)$ or $\ell(t, y)$	convex in t	$\sigma(t^*)$	**Where it is used**
0–1	$\mathbf{1}[m < 0]$	no	–	the target, never optimised
logistic (cross entropy)	$\log(1 + e^{-m})$	yes	η	logistic regression, networks
squared error (Brier)	$(y - \sigma(t))^2$	no	η	older networks, Brier score
hinge	$\max(0, 1 - m)$	yes	sign only	support vector machines
squared hinge	$\max(0, 1 - m)^2$	yes	$\sigma(2\eta - 1)$	ℓ_2-SVM
exponential	e^{-m}	yes	$\sigma(\frac{1}{2} \log \frac{\eta}{1-\eta})$	AdaBoost
focal	$-(1-p)^\gamma \log p$	no	compressed	imbalanced data

Table 5.2 Loss functions for binary classification. $m = \tilde{y}t$ is the margin, $\eta = p(y = 1 \mid \boldsymbol{x})$ the true class probability, and t^* the value of the linear predictor that minimises the expected loss at a point, Proposition 5.1; for the focal loss p denotes the probability assigned to the correct class. Only the losses whose $\sigma(t^*)$ column reads η estimate the class probability directly.

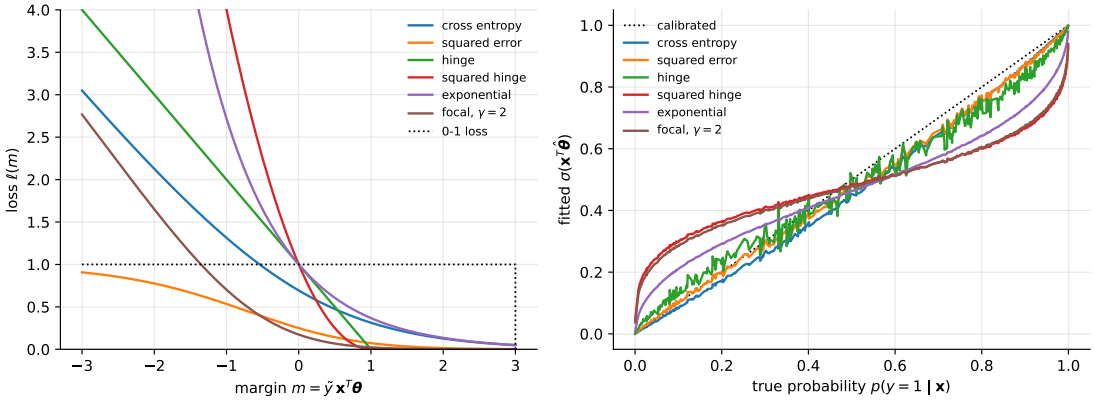


Fig. 5.9 Left: the loss functions of Table 5.2 against the margin, for a point of class $\tilde{y} = +1$; the 0–1 loss is dotted. Right: the probability $\sigma(\boldsymbol{x}^T\hat{\boldsymbol{\theta}})$ obtained by minimising each loss with a linear model on 400 points drawn from a true logistic model, plotted against the true probability. The cross entropy and the squared error follow the diagonal; the exponential loss recovers half the log-odds, so its curve is the diagonal compressed towards $\frac{1}{2}$; the squared hinge and the focal loss are compressed further; the hinge loss does not estimate a probability at all.

A few features of the picture are worth pointing out before the algebra. All the surrogates agree on the sign of what they want – push the margin positive – and differ in how hard they

push and where they stop. The hinge loss is exactly zero once $m \geq 1$, so points comfortably on the right side exert no force whatever; that is what will make support vectors in Chapter 6. The logistic and exponential losses never reach zero and keep rewarding larger margins, exponentially weakly in the first case and linearly in the second; the exponential loss also punishes a large negative margin *exponentially* hard, which is the origin of AdaBoost's known sensitivity to mislabelled points. The squared error and the focal loss flatten out for large negative margins, which is the vanishing-gradient problem of the notebox in Section 5.3 seen again: a confidently wrong point produces almost no gradient.

5.10.2 What each loss estimates

A loss function does two jobs. It has to be minimised, which is a matter of convexity and smoothness, and its minimiser has to mean something. The second question has a clean answer, because in the limit of infinite data the fit at each $\boldsymbol{x}$ minimises the *expected* loss there. Fix a point with true class probability $\eta = p(y = 1 \mid \boldsymbol{x})$ and ask which value of the linear predictor t minimises

$$L(t) = \mathbb{E}_y\left[\ell(t,y)\right] = \eta\,\ell(t,1) + (1-\eta)\,\ell(t,0). \tag{5.50}$$

For a margin loss $\ell(t,1) = \ell(t)$ and $\ell(t,0) = \ell(-t)$. If the minimiser t^* has the sign of $2\eta - 1$, the surrogate is *classification-calibrated* [31]: minimising it recovers the Bayes decision rule of the 0–1 loss. If, further, $\sigma(t^*) = \eta$ – or some fixed, invertible function of t^* equals η – the surrogate is a *proper scoring rule* [32,33] and the model's output can be read as a probability.

Proposition 5.1 (Pointwise minimisers of the classification losses). *For $0 < \eta < 1$ the minimiser of Eq. (5.50) is*

$$\textit{logistic:}\quad t^* = \log\frac{\eta}{1-\eta}, \qquad \sigma(t^*) = \eta; \tag{5.51}$$

$$\textit{squared error:}\quad \sigma(t^*) = \eta; \tag{5.52}$$

$$\textit{exponential:}\quad t^* = \tfrac{1}{2}\log\frac{\eta}{1-\eta}, \qquad \sigma(2t^*) = \eta; \tag{5.53}$$

$$\textit{squared hinge:}\quad t^* = 2\eta - 1; \tag{5.54}$$

$$\textit{hinge:}\quad t^* = \operatorname{sign}(2\eta - 1). \tag{5.55}$$

All five are classification-calibrated. The logistic and squared-error losses are proper; the exponential and squared-hinge losses determine η through $\sigma(2t^*)$ and $(t^*+1)/2$ respectively; the hinge loss carries no information about η beyond its sign.

Proof. Each is a one-variable minimisation. For the logistic loss, $L(t) = \eta\log(1+e^{-t}) + (1-\eta)\log(1+e^{t})$ and $L'(t) = -\eta(1-\sigma(t)) + (1-\eta)\sigma(t) = \sigma(t) - \eta$, which vanishes exactly at $\sigma(t^*) = \eta$; $L'' = \sigma(1-\sigma) > 0$, so it is the minimum. For the squared error, $L(t) = \eta(1-\sigma)^2 + (1-\eta)\sigma^2$ with $\sigma = \sigma(t)$, and $L'(t) = 2(\sigma - \eta)\,\sigma'(t)$; since $\sigma' > 0$ the only stationary point is again $\sigma(t^*) = \eta$, and $L \to \eta$ or $1-\eta$ at $t \to \mp\infty$, both larger than the value $\eta(1-\eta)$ at the stationary point, so it is the global minimum although L is not convex. For the exponential loss $L(t) = \eta e^{-t} + (1-\eta)e^{t}$, $L'(t) = -\eta e^{-t} + (1-\eta)e^{t} = 0$ gives $e^{2t^*} = \eta/(1-\eta)$, and $\sigma(2t^*) = e^{2t^*}/(1+e^{2t^*}) = \eta$. For the squared hinge the minimiser lies in $[-1,1]$, where both hinges are active, $L(t) = \eta(1-t)^2 + (1-\eta)(1+t)^2$ and $L'(t) = -2\eta(1-t) + 2(1-\eta)(1+t) = 2(t - (2\eta-1))$. For the hinge, $L(t) = \eta\max(0,1-t) + (1-\eta)\max(0,1+t)$ is piecewise linear with slope $-\eta + (1-\eta) = 1 - 2\eta$ on $[-1,1]$, slope $1-\eta > 0$ for $t > 1$ and slope $-\eta < 0$ for $t < -1$; it is therefore minimised at $t = 1$ if $\eta > \frac{1}{2}$ and at $t = -1$ if $\eta < \frac{1}{2}$, and the value at the minimum, $2\min(\eta, 1-\eta)$, depends on η but its location does not. In every case $\operatorname{sign}(t^*) = \operatorname{sign}(2\eta - 1)$. □

The proposition is what the right panel of Fig. 5.9 shows empirically, and it explains a common misuse. A support vector machine does not produce probabilities, and passing its decision function through a sigmoid produces numbers that are systematically too close to $\frac{1}{2}$ – which is why `scikit-learn` fits a separate logistic regression on top of the SVM output when probabilities are requested (Platt scaling). A model trained with the exponential loss must be read through $\sigma(2t)$, not $\sigma(t)$; that is why the AdaBoost of Chapter 7 outputs half the log-odds. And the focal loss [34],

$$\ell_{\text{focal}}(t,y) = -y\,(1-p)^{\gamma}\log p - (1-y)\,p^{\gamma}\log(1-p), \qquad p=\sigma(t),\ \gamma \geq 0, \tag{5.56}$$

which multiplies each cross-entropy term by a factor that vanishes for well-classified points and thereby concentrates the gradient on the hard and the rare ones, is deliberately *not* proper: its minimiser is pushed towards $\frac{1}{2}$ (Exercise at the end of the chapter), so a detector trained with it must be recalibrated before its scores are read as probabilities. The cross entropy is the only member of the family that is convex, proper, and free of the vanishing-gradient defect, and that is the whole case for it; but each of the others buys something in exchange, and the following code makes it possible to see what without deriving a single derivative.

5.10.3 One program for every loss

The class of Section 5.8 hard-wired the cross-entropy gradient (5.17). With automatic differentiation the loss becomes a plain Python function of the linear predictor and the label, the cost is its mean over the data plus the penalty of Eq. (5.23), and JAX supplies the gradient – and, if wanted, the Hessian – of whatever we wrote. Trying a new loss means writing one line.

```
import jax
jax.config.update("jax_enable_x64", True)
import jax.numpy as jnp
import numpy as np
from jax import grad, jit, hessian
from jax.nn import sigmoid, softplus, log_sigmoid

# Every loss is a function of the linear predictor t = x^T theta and the label
# y in {0, 1}, evaluated pointwise; softplus(t) = log(1 + e^t) is the stable
# form of Eq. (5.crossentropycompact), log_sigmoid(t) = log sigma(t).
def cross_entropy(t, y):
    return softplus(t) - y * t                          # Eq. (5.logisticloss)

def squared_error(t, y):
    return (y - sigmoid(t))**2                          # the Brier score

def hinge(t, y):
    return jnp.maximum(0.0, 1.0 - (2*y - 1) * t)        # margin m = (2y-1) t

def squared_hinge(t, y):
    return jnp.maximum(0.0, 1.0 - (2*y - 1) * t)**2

def exponential(t, y):
    return jnp.exp(-(2*y - 1) * t)

def focal(t, y, gamma=2.0):                             # Eq. (5.focal)
    p = sigmoid(t)
    return -(y * (1 - p)**gamma * log_sigmoid(t)
             + (1 - y) * p**gamma * log_sigmoid(-t))
```

```
LOSSES = {"cross entropy": cross_entropy, "squared error": squared_error,
          "hinge": hinge, "squared hinge": squared_hinge,
          "exponential": exponential, "focal (gamma=2)": focal}

def cost(theta, X, y, loss, lmbda=0.0):
    """Mean loss over the data plus an l2 penalty that spares the intercept."""
    return jnp.mean(loss(X @ theta, y)) + lmbda * jnp.sum(theta[1:]**2)

def fit(loss, X, y, gamma=0.5, epochs=3000, lmbda=0.0):
    """Gradient descent, Eq. (5.gd), with the gradient supplied by JAX."""
    c = lambda th: cost(th, X, y, loss, lmbda)
    step = jit(lambda th: th - gamma * grad(c)(th))
    theta = jnp.zeros(X.shape[1])
    for _ in range(epochs):
        theta = step(theta)
    return theta

def newton(loss, X, y, iters=10, lmbda=1e-8):
    """Newton-Raphson, Eq. (5.newtonabstract), with the Hessian from JAX."""
    c = lambda th: cost(th, X, y, loss, lmbda)
    g, H = jit(grad(c)), jit(hessian(c))
    theta = jnp.zeros(X.shape[1])
    for _ in range(iters):
        theta = theta - jnp.linalg.solve(H(theta), g(theta))
    return theta
```

Before using it on anything, we check the machinery against the hand-derived results of Section 5.4: the automatic gradient of the cross entropy must be $-\boldsymbol{X}^T(\boldsymbol{y}-\boldsymbol{p})/n$ and its automatic Hessian must be $\boldsymbol{X}^T\boldsymbol{W}\boldsymbol{X}/n$. The data are drawn from a *true* logistic model, so that we can afterwards ask not only whether each loss classifies well but whether it estimates the right probabilities.

```
rng = np.random.default_rng(2024)
n = 400
X = jnp.asarray(np.c_[np.ones(n), rng.normal(size=(n, 2))])
theta_true = jnp.array([0.5, 2.0, -1.0])
p_true = sigmoid(X @ theta_true)                          # the truth we try to recover
y = jnp.asarray((rng.random(n) < np.asarray(p_true)).astype(float))

theta0 = jnp.array([0.1, -0.3, 0.2])
g_ad = grad(cost)(theta0, X, y, cross_entropy)
g_hand = -X.T @ (y - sigmoid(X @ theta0)) / n                 # Eq. (5.gradient)
print("gradient: max |AD - hand| =", float(jnp.max(jnp.abs(g_ad - g_hand))))

p0 = sigmoid(X @ theta0)
H_ad = hessian(cost)(theta0, X, y, cross_entropy)
H_hand = X.T @ ((p0 * (1 - p0))[:, None] * X) / n              # Eq. (5.hessian)
print("Hessian:  max |AD - hand| =", float(jnp.max(jnp.abs(H_ad - H_hand))))

print("Newton :", newton(cross_entropy, X, y))
print("GD     :", fit(cross_entropy, X, y))
```

```
gradient: max |AD - hand| = 2.220446049250313e-16
Hessian:  max |AD - hand| = 3.0531133177191805e-16
Newton : [ 0.26067864  1.93734364 -1.00125907]
GD     : [ 0.26067869  1.93734439 -1.00125944]
```

Both derivatives agree with Eqs. (5.17) and (5.19) to rounding, and Newton's method built from `jax.hessian` lands in ten iterations where three thousand steps of gradient descent land, the two differing in the seventh digit only through the tiny ridge term that keeps the Hessian invertible. Now the loop that motivated the section:

```
def evaluate(theta):
    p = sigmoid(X @ theta)
    accuracy = float(jnp.mean((p > 0.5) == (y > 0.5)))
    calibration = float(jnp.mean(jnp.abs(p - p_true)))    # mean |p_hat - p_true|
    return accuracy, calibration

print(f"{'loss':16s} {'theta':26s} accuracy   mean |p_hat - p_true|")
for name, loss in LOSSES.items():
    theta = fit(loss, X, y)
    acc, cal = evaluate(theta)
    print(f"{name:16s} {np.array2string(np.asarray(theta), precision=2):26s}"
          f" {acc:.3f}      {cal:.3f}")
print("true            ", np.asarray(theta_true))
```

```
loss             theta                      accuracy   mean |p_hat - p_true|
cross entropy    [ 0.26  1.94 -1.  ]        0.790      0.032
squared error    [ 0.32  1.82 -0.97]        0.790      0.024
hinge            [ 0.28  1.53 -0.89]        0.792      0.043
squared hinge    [ 0.1   0.69 -0.36]        0.795      0.156
exponential      [ 0.11  1.06 -0.53]        0.780      0.103
focal (gamma=2)  [ 0.09  0.74 -0.38]        0.782      0.149
true             [ 0.5  2.  -1. ]
```

The accuracies are all within a percent of one another, and of the Bayes rate of 0.79 for this problem: every loss in the table is classification-calibrated, as Proposition 5.1 says, and all of them find essentially the same decision boundary. The probabilities are another matter, and the last column is Proposition 5.1 in numbers. The two proper losses recover the true parameters up to sampling error. The exponential loss returns $(0.11, 1.06, -0.53)$, which is the true vector halved – Eq. (5.53), $t^* = \frac{1}{2}\log[\eta/(1-\eta)]$ – and its probabilities are correct once read through $\sigma(2t)$. The squared hinge and the focal loss compress the parameters by a factor of about three and their $\sigma(t)$ is badly miscalibrated, while their accuracy is unharmed. Nothing about these conclusions required a derivative, and the reader can now add a loss to the dictionary – an asymmetric one that penalises false negatives more than false positives, a class-weighted cross entropy, a Huberised hinge – and have the answer in seconds.

Machine learning connection. The pattern of this section is how loss functions are actually developed. A practitioner writes the loss as a few lines of framework code, lets the framework differentiate it, and judges the result empirically; the theoretical questions – is it convex, is it proper, is it classification-calibrated – are asked afterwards, and often answered by exactly the pointwise argument of Proposition 5.1. Every framework in the later chapters (`PyTorch`, `TensorFlow`, `JAX`) offers the losses of Table 5.2 as library functions, but nothing in them is special: `torch.nn.BCEWithLogitsLoss` is `softplus(t) - y*t`, and a loss the library does not offer is one line away. The one practical rule is the one used above: write losses in terms of the logit t and the stable primitives `softplus` and `log_sigmoid`, never in terms of $\log\sigma(t)$ computed literally, for the reasons of Section 5.7.

5.11 Kernel logistic regression

Logistic regression produces a linear decision boundary, and Section 3.12 showed how to remove that restriction from Ridge regression without ever constructing the enlarged feature space. The same construction applies here, and it applies for the same reason: by Theorem 3.12 the reason had nothing to do with the squared error.

5.11.1 The problem, and what the representer theorem gives

Replace the linear predictor $\boldsymbol{x}^T\boldsymbol{\theta}$ by a function $f \in \mathscr{H}$, the reproducing kernel Hilbert space of a kernel k, and penalise its norm:

$$\min_{f\in\mathscr{H}} \left\{ -\sum_{i=1}^{n} \Big[y_i \ln p_i + (1-y_i)\ln(1-p_i) \Big] + \frac{\lambda}{2}\,\|f\|^2_{\mathscr{H}} \right\}, \qquad p_i = \sigma(f(\boldsymbol{x}_i))\,, \tag{5.57}$$

with σ the logistic function (5.3) and $y_i \in \{0,1\}$. The data-fitting term is the cross entropy (5.13) with $\boldsymbol{x}_i^T\boldsymbol{\theta}$ replaced by $f(\boldsymbol{x}_i)$, and the penalty is the ℓ_2 penalty of Section 5.5 measured in $\mathscr{H}$ rather than in $\mathbb{R}^p$.

Theorem 3.12 applies immediately: the loss depends on f only through its values at the training points, and $\Omega(t) = \frac{\lambda}{2}t^2$ is strictly increasing. The minimiser is therefore

$$f(\cdot) = \sum_{j=1}^{n} \alpha_j k(\boldsymbol{x}_j, \cdot)\,, \qquad \text{so that} \qquad \boldsymbol{f} = \boldsymbol{K}\boldsymbol{\alpha} \quad \text{and} \quad \|f\|^2_{\mathscr{H}} = \boldsymbol{\alpha}^T\boldsymbol{K}\boldsymbol{\alpha}, \tag{5.58}$$

the second identity because $\|f\|^2 = \sum_{ij}\alpha_i\alpha_j\langle k(\boldsymbol{x}_i,\cdot), k(\boldsymbol{x}_j,\cdot)\rangle$, which is $\sum_{ij}\alpha_i\alpha_j K_{ij}$. The infinite-dimensional problem has become the n-dimensional one

$$\boxed{ C(\boldsymbol{\alpha}) = -\sum_{i=1}^{n} \Big[y_i \ln p_i + (1-y_i)\ln(1-p_i) \Big] + \frac{\lambda}{2}\,\boldsymbol{\alpha}^T\boldsymbol{K}\boldsymbol{\alpha}, \qquad \boldsymbol{p} = \sigma(\boldsymbol{K}\boldsymbol{\alpha})\,. } \tag{5.59}$$

5.11.2 Gradient, Hessian and kernel IRLS

The derivatives are obtained from Section 5.4 by the substitution $\boldsymbol{X} \to \boldsymbol{K}$, which is the whole of the work: the chain rule sees $\boldsymbol{K}\boldsymbol{\alpha}$ exactly as it saw $\boldsymbol{X}\boldsymbol{\theta}$. Differentiating Eq. (5.59) and using Eq. (1.28) for the penalty,

$$\frac{\partial C}{\partial \boldsymbol{\alpha}} = -\boldsymbol{K}(\boldsymbol{y}-\boldsymbol{p}) + \lambda\boldsymbol{K}\boldsymbol{\alpha} = \boldsymbol{K}\Big[(\boldsymbol{p}-\boldsymbol{y}) + \lambda\boldsymbol{\alpha} \Big], \tag{5.60}$$

to be compared with Eq. (5.17), and

$$\frac{\partial^2 C}{\partial \boldsymbol{\alpha}\,\partial \boldsymbol{\alpha}^T} = \boldsymbol{K}\boldsymbol{W}\boldsymbol{K} + \lambda\boldsymbol{K}, \tag{5.61}$$

with $\boldsymbol{W}$ the same diagonal matrix (5.18) of weights $p_i(1-p_i)$. Since $\boldsymbol{K}$ is positive semi-definite, so is Eq. (5.61) by the argument of Eq. (5.20), and the problem is convex; for $\lambda > 0$ and $\boldsymbol{K}$ positive definite it is strictly convex, so the minimiser is unique.

Newton's method now takes a form worth deriving, because it turns out to be the exact analogue of the iteratively reweighted least squares of Eq. (5.34). Assume $\boldsymbol{K}$ invertible and factor it out of Eq. (5.61) on the left, $\boldsymbol{K}\boldsymbol{W}\boldsymbol{K} + \lambda\boldsymbol{K} = \boldsymbol{K}(\boldsymbol{W}\boldsymbol{K} + \lambda\boldsymbol{I})$. The Newton step $\boldsymbol{\Delta} = -\boldsymbol{H}^{-1}\boldsymbol{g}$ is then

$$\boldsymbol{\Delta} = -(\boldsymbol{W}\boldsymbol{K} + \lambda\boldsymbol{I})^{-1}\boldsymbol{K}^{-1}\boldsymbol{K}\Big[(\boldsymbol{p}-\boldsymbol{y}) + \lambda\boldsymbol{\alpha} \Big] = (\boldsymbol{W}\boldsymbol{K} + \lambda\boldsymbol{I})^{-1}\Big[(\boldsymbol{y}-\boldsymbol{p}) - \lambda\boldsymbol{\alpha} \Big], \tag{5.62}$$

the inverse Gram matrices cancelling. Adding $\boldsymbol{\alpha}$ and collecting,

$$\boldsymbol{\alpha} + \boldsymbol{\Delta} = (\boldsymbol{W}\boldsymbol{K} + \lambda\boldsymbol{I})^{-1}\Big[\boldsymbol{W}\boldsymbol{K}\boldsymbol{\alpha} + (\boldsymbol{y}-\boldsymbol{p}) \Big] = (\boldsymbol{W}\boldsymbol{K} + \lambda\boldsymbol{I})^{-1}\boldsymbol{W}\boldsymbol{z}, \tag{5.63}$$

where the *working response*

$$z = K\alpha + W^{-1}(y - p) \tag{5.64}$$

is the adjusted response defined just above Eq. (5.34), with $X\theta$ replaced by $K\alpha$. Multiplying Eq. (5.63) through by W^{-1} inside the inverse gives the form to remember,

$$\boxed{\alpha \leftarrow \left(K + \lambda W^{-1}\right)^{-1} z.} \tag{5.65}$$

Compare Eq. (5.65) with the kernel ridge regression (3.84), $\alpha = (K + \lambda I)^{-1} y$. They differ in two places only: the targets y have become the working response z, and the uniform penalty λI has become the per-observation penalty λW^{-1}, which is small where the model is uncertain and large where it is confident. *Kernel logistic regression is iteratively reweighted kernel ridge regression*, exactly as ordinary logistic regression is iteratively reweighted least squares. The parallel of Section 5.7 is complete, and it is not a coincidence but a consequence of Theorem 3.12: the same substitution that turned least squares into kernel ridge regression turns each reweighted step into a reweighted kernel step.

```
import numpy as np

def kernel_irls(K, y, lmbda, n_iter=100, tol=1e-13, jitter=1e-12):
    """Kernel logistic regression by Newton's method, Eq. (5.kirls).

    Each iteration is a kernel ridge regression, Eq. (3.krr), on the working
    response z with the per-observation penalty lambda / W_ii.
    """
    n = len(y)
    alpha = np.zeros(n)
    for _ in range(n_iter):
        f = K @ alpha
        p = sigmoid(f)
        w = np.maximum(p * (1.0 - p), 1e-10)          # W_ii, Eq. (5.Wmatrix)
        z = f + (y - p) / w                           # Eq. (5.kworking)
        new = np.linalg.solve(K + lmbda * np.diag(1.0 / w)
                              + jitter * np.eye(n), z)
        if np.linalg.norm(new - alpha) < tol:
            return new
        alpha = new
    return alpha


def kernel_logistic_predict(alpha, Xtrain, Xnew, gamma):
    """p(y=1|x) = sigma(sum_j alpha_j k(x_j, x))."""
    return sigmoid(gaussian_kernel(Xnew, Xtrain, gamma) @ alpha)
```

The `jitter` is there because K for a Gaussian kernel is positive definite in exact arithmetic and can be numerically singular when two training points nearly coincide; adding 10^{-12} to the diagonal is the standard remedy and changes nothing that matters.

5.11.3 The method, verified

The test problem is one that no linear boundary can solve at all: 120 points in the plane, labelled by whether they fall inside a disc, with five per cent of the labels flipped. Newton's method on Eq. (5.65) with a Gaussian kernel converges in six iterations, and the step lengths show the quadratic rate of Section 5.7 unmistakably:

```
    iteration        cost          ||delta alpha||
            1    56.8201866189          3.602e+00
```

```
 2    56.4403059843          2.888e-01
 3    56.4394190411          1.438e-02
 4    56.4394190342          3.825e-05
 5    56.4394190342          3.059e-10
 6    56.4394190342          2.018e-15
```

Each step length is roughly the square of the one before once the iterate is close, which is what distinguishes Newton's method from the linearly convergent gradient descent of Chapter 4. Plain gradient descent on the same objective reaches the same minimum, more slowly: it agrees with the Newton solution to 9.1×10^{-5} in the fitted function, which is its own convergence tolerance and not a disagreement about the answer.

The more searching test is against the primal problem. With the quadratic kernel $(\boldsymbol{x}^T\boldsymbol{x}'+1)^2$ the feature map is only six-dimensional, so the same fit can be obtained by ordinary penalised logistic regression in that map and the two compared:

```
max |k(x,x') - phi(x).phi(x')|: 4.263e-14
max |f_kernel(x) - f_primal(x)| on 400 test points: 6.004e-13
||theta_primal||^2 = 13.2542532295,   alpha^T K alpha = 13.2542532295

scikit-learn in the same map, C = 1/lambda:
  max |theta_ours - theta_sklearn| : 9.498e-09
  max |f_ours - f_sklearn| on test : 7.232e-08
```

The third line is the one to notice. The primal penalty $\|\boldsymbol{\theta}\|_2^2$ and the dual penalty $\boldsymbol{\alpha}^T\boldsymbol{K}\boldsymbol{\alpha}$ agree to ten digits because they are the same number written twice: both are $\|f\|_{\mathscr{H}}^2$, the squared norm of one function in one space. Note also the conversion to `scikit-learn`'s convention, $C = 1/\lambda$, warned about in Section 5.5; with it, three independent implementations agree to the tolerance of the least accurate.

5.11.4 What the coefficients are

Setting Eq. (5.60) to zero gives a fact about kernel logistic regression that is easy to miss and explains a good deal.

Proposition 5.2 (The coefficients are the residuals). *If $\boldsymbol{K}$ is invertible, the minimiser of Eq. (5.59) satisfies*

$$\alpha_i = \frac{y_i - p_i}{\lambda}, \qquad i = 1, \dots, n. \tag{5.66}$$

Proof. Stationarity is $\boldsymbol{K}[(\boldsymbol{p}-\boldsymbol{y}) + \lambda\boldsymbol{\alpha}] = \boldsymbol{0}$ by Eq. (5.60). Multiplying by $\boldsymbol{K}^{-1}$ gives $\lambda\boldsymbol{\alpha} = \boldsymbol{y} - \boldsymbol{p}$. □

The weight a training point carries in the fitted function (5.58) is its own residual divided by the penalty. Points the model already predicts correctly and confidently have small residuals and contribute little; points it gets wrong contribute much. But p_i is a logistic probability and lies strictly between zero and one, so the residual $y_i - p_i$ is *never exactly zero*: every training point contributes something, and $\boldsymbol{\alpha}$ is dense.

```
 lambda   max |alpha - (y-p)/lambda|   non-zero alpha of 120   |alpha|_min
   0.10                 1.471e-11                 120   2.065e-02
   1.00                 1.477e-13                 120   6.360e-02
  10.00                 1.422e-15                 120   3.472e-02
```

This is the practical drawback of the method and the reason it is less used than its two neighbours. Prediction costs n kernel evaluations no matter how easy the problem is, where

a support vector machine on the same data would retain a fraction of the points. Section 6.13 shows that this contrast is not an accident of the two algorithms but a theorem about their losses, and that Eq. (5.66) is one instance of a formula covering all three of the kernel methods in this book.

Machine learning connection. What kernel logistic regression buys over a support vector machine is a calibrated probability. The output of Eq. (5.58) passed through the sigmoid is an estimate of $p(y = 1 \mid \boldsymbol{x})$ obtained by maximising a likelihood, so it can be thresholded at a cost-sensitive value, combined with other probabilities, or fed to a decision rule – all of which the signed distance produced by Eq. (6.33) cannot support without an extra calibration step fitted on held-out data. Against that stands the dense $\boldsymbol{\alpha}$ of Proposition 5.2 and the $\mathcal{O}(n^3)$ cost of each Newton step. The choice between them is the usual one in this book: a probability, or a sparse representation, but not both from the same fit.

5.11.5 The penalty, measured

Everything Section 5.5 said about the penalty holds here, with one addition: the kernel supplies enough flexibility to interpolate the training data exactly, so the penalty is no longer optional. Running the same fit across six decades:

lambda	train accuracy	test accuracy	alpha^T K alpha
0.001	0.9500	0.9300	5027.0188
0.010	0.9417	0.9275	696.1591
0.100	0.9500	0.9375	144.9844
1.000	0.9333	0.9400	22.4656
10.000	0.8750	0.9150	1.0326
100.000	0.8667	0.8700	0.0146

The best test accuracy is at $\lambda = 1$, in the interior, with the norm of the fitted function falling by five orders of magnitude across the range. At $\lambda = 10^{-3}$ the model is reproducing the five per cent of deliberately flipped labels and pays for it on the test set; at $\lambda = 100$ it has been flattened towards a constant. This is Eq. (2.52) once more, and it is why the kernel and the penalty must be chosen together by the cross-validation of Section 2.12 and never by the training error.

5.12 Measuring the quality of a classifier

The mean squared error and R^2 of Section 3.5 are meaningless for a classifier. We need measures appropriate to discrete outcomes, and it turns out that no single number will do.

The confusion matrix.

Everything starts here. For two classes, cross-tabulating the true label against the predicted one gives four counts:

	predicted 1	predicted 0
true 1	TP	FN
true 0	FP	TN

(5.67)

the true positives, false negatives, false positives and true negatives. Every scalar measure below is a function of these four numbers, and reporting the matrix itself is almost always more informative than reporting any of them.

Accuracy, and why it misleads.

The obvious measure is the fraction correctly classified,

$$\text{accuracy} = \frac{\text{TP}+\text{TN}}{\text{TP}+\text{TN}+\text{FP}+\text{FN}} = \frac{1}{n}\sum_{i=1}^{n} I(y_i = \tilde{y}_i)\,, \tag{5.68}$$

with I the indicator function. Accuracy is the right measure when the classes are balanced and the two kinds of error are equally costly. It is badly misleading otherwise. If one in a thousand transactions is fraudulent, the classifier that declares everything legitimate achieves 99.9% accuracy while being entirely useless. *Always compare an accuracy against the proportion of the majority class*, which is the score obtained for free.

Precision and recall.

Separating the two kinds of error gives

$$\text{precision} = \frac{\text{TP}}{\text{TP}+\text{FP}}, \qquad \text{recall} = \frac{\text{TP}}{\text{TP}+\text{FN}}. \tag{5.69}$$

Precision asks: of the samples flagged positive, what fraction really were? Recall asks: of the samples that really were positive, what fraction did we find? The two trade off against each other through the decision threshold. Lowering the threshold flags more samples, catching more of the true positives – recall rises – at the cost of more false alarms – precision falls. Which matters more is a question about the application and not about the model: in screening for a serious disease a false negative is far worse than a false positive, so recall dominates; in flagging emails as spam the reverse holds. The harmonic mean of the two,

$$F_1 = 2\,\frac{\text{precision}\cdot\text{recall}}{\text{precision}+\text{recall}}, \tag{5.70}$$

is the usual compromise; the harmonic mean rather than the arithmetic one because it is small unless *both* are large.

ROC and AUC.

Because logistic regression outputs a probability rather than a decision, the threshold is ours to choose, and a fair assessment should not depend on one arbitrary choice. The *receiver operating characteristic* curve plots the true positive rate $\text{TP}/(\text{TP}+\text{FN})$, which is the recall, against the false positive rate $\text{FP}/(\text{FP}+\text{TN})$, as the threshold sweeps from one to zero. A perfect classifier passes through the top-left corner; a classifier that ranks at random gives the diagonal. The area under the curve, the *AUC*, summarises the whole sweep in a single number, and it has a clean interpretation: it is the probability that a randomly chosen positive sample receives a higher score than a randomly chosen negative one. An AUC of 0.5 is chance and 1.0 is perfect. Since AUC depends only on the ranking, it is insensitive to class imbalance in a way accuracy is not.

```
import numpy as np

def confusion_matrix(y_true, y_pred):
    """Return TP, FN, FP, TN for binary labels in {0, 1}."""
    tp = int(np.sum((y_true == 1) & (y_pred == 1)))
    fn = int(np.sum((y_true == 1) & (y_pred == 0)))
    fp = int(np.sum((y_true == 0) & (y_pred == 1)))
    tn = int(np.sum((y_true == 0) & (y_pred == 0)))
    return tp, fn, fp, tn


def classification_report(y_true, y_pred):
    tp, fn, fp, tn = confusion_matrix(y_true, y_pred)
    accuracy = (tp + tn) / (tp + tn + fp + fn)
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = (2 * precision * recall / (precision + recall)
          if precision + recall else 0.0)
    return dict(accuracy=accuracy, precision=precision, recall=recall, f1=f1)


def roc_auc(y_true, scores):
    """AUC as the probability that a positive outranks a negative."""
    order = np.argsort(scores)
    ranks = np.empty(len(scores), dtype=float)
    ranks[order] = np.arange(1, len(scores) + 1)
    n_pos = int(np.sum(y_true == 1))
    n_neg = len(y_true) - n_pos
    return (np.sum(ranks[y_true == 1]) - n_pos * (n_pos + 1) / 2) / (n_pos * n_neg)
```

Machine learning connection. The cross entropy is the loss we *optimise*; accuracy, F_1 and AUC are what we *report*. They are not the same and cannot be, because accuracy is a step function of the parameters – flat almost everywhere, with jumps where a prediction crosses the threshold – so its gradient is zero wherever it is defined and no method of Chapter 4 could minimise it. The cross entropy is a smooth surrogate that is minimised in roughly the same place. This split between a differentiable training objective and a non-differentiable evaluation metric runs through all of machine learning, and it is worth being explicit that a model with the lower cross entropy does not automatically have the higher accuracy. It also explains why, when the classes are severely imbalanced, one often weights the cross entropy by class frequency: to move the minimum of the surrogate closer to the optimum of the metric one actually cares about.

5.13 The Wisconsin breast cancer data

We close with a real data set. The Wisconsin breast cancer data contain 569 samples, each described by 30 features computed from a digitised image of a fine-needle aspirate of a breast mass – radius, texture, perimeter, area, smoothness and so on, each in three variants – and each labelled malignant or benign. It is a natural test for a binary classifier and is included with `scikit-learn`.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_validate
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
```

```
from sklearn.linear_model import LogisticRegression

cancer = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, random_state=0)
print(X_train.shape, X_test.shape)

# Scaling inside the pipeline, so it is refitted on each training fold
logreg = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
logreg.fit(X_train, y_train)

print(f"test accuracy: {logreg.score(X_test, y_test):.3f}")
scores = cross_validate(logreg, X_train, y_train, cv=10)["test_score"]
print(f"cross-validated accuracy: {scores.mean():.3f} +/- {scores.std():.3f}")
```

The scaler is not decoration. The thirty features differ enormously in scale – the mean area is around 655 while the mean smoothness is around 0.096, a ratio of some four orders of magnitude, and the ratio of the largest to the smallest standard deviation across the thirty columns exceeds 10^5 – so by Section 5.5 the default ℓ_2 penalty would fall almost entirely on the small-scale features, and by Section 4.5 the optimisation would be badly conditioned. Placing the scaler inside a `Pipeline` ensures it is refitted on each training fold, honouring the discipline of Section 2.12.

Inspecting the features.

Before fitting anything it is worth looking at the data. Plotting, for each feature, the histograms of the two classes superimposed shows immediately which features separate them.

```
import matplotlib.pyplot as plt
import pandas as pd

cancerpd = pd.DataFrame(cancer.data, columns=cancer.feature_names)
malignant = cancer.data[cancer.target == 0]
benign = cancer.data[cancer.target == 1]

fig, axes = plt.subplots(15, 2, figsize=(10, 20))
ax = axes.ravel()
for i in range(30):
    _, bins = np.histogram(cancer.data[:, i], bins=50)
    ax[i].hist(malignant[:, i], bins=bins, alpha=0.5)
    ax[i].hist(benign[:, i], bins=bins, alpha=0.5)
    ax[i].set_title(cancer.feature_names[i])
    ax[i].set_yticks(())
ax[0].set_xlabel("Feature magnitude")
ax[0].set_ylabel("Frequency")
ax[0].legend(["Malignant", "Benign"], loc="best")
fig.tight_layout()
plt.show()

# The correlation matrix of Section 1.covariance, computed with pandas
correlations = cancerpd.corr()
plt.figure(figsize=(10, 9))
plt.imshow(correlations, vmin=-1, vmax=1, cmap="RdBu_r")
plt.colorbar(); plt.title("Correlation matrix of the features")
plt.show()
```

The correlation matrix repays study, and it connects directly to Chapter 1. Several groups of features are almost perfectly correlated – radius, perimeter and area are essentially the same measurement three times over, since a roughly circular region has perimeter and area determined by its radius. By Section 1.19 this makes the design matrix nearly rank deficient,

so the individual coefficients are poorly determined even though the predictions are not. Three remedies are available and all appear in this book: penalise with ℓ_2, as the default settings do; select with ℓ_1, which will keep one member of each correlated group; or reduce the dimension first with the principal component analysis of Section 1.21. The point to carry away is that a high accuracy on this data set does *not* license a statement about which feature matters, because the data cannot distinguish the members of a correlated group.

```
from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_auc_score, RocCurveDisplay)

y_pred = logreg.predict(X_test)
y_proba = logreg.predict_proba(X_test)[:, 1]

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred,
                            target_names=cancer.target_names))
print(f"AUC: {roc_auc_score(y_test, y_proba):.4f}")

RocCurveDisplay.from_predictions(y_test, y_proba)
plt.show()
```

The run above gives a test accuracy of 0.958, a cross-validated accuracy on the training set of 0.981 ± 0.025, and an AUC of 0.9914. The confusion matrix on the 143 test samples is

$$\begin{bmatrix} 50 & 3 \\ 3 & 87 \end{bmatrix},$$

with rows the true class (malignant, benign) and columns the predicted one.

These numbers should be read with the cautions of Section 5.12 in mind. An accuracy of 0.958 sounds impressive until one notices that 62.7% of the samples are benign, so the classifier that predicts "benign" unconditionally already scores 0.629; the model has removed about nine tenths of the remaining error, which is the honest way to state the achievement. The AUC of 0.9914 is the more informative summary, being independent of both the threshold and the class balance.

The confusion matrix is more informative still, because it separates the two kinds of error: three malignant cases were predicted benign and three benign cases malignant. In a screening context those are not equivalent, and one would deliberately move the threshold below 0.5 to reduce the first at the cost of increasing the second – trading precision for recall, exactly as discussed in Section 5.12. Note finally that the cross-validated accuracy (0.981) exceeds the test accuracy (0.958); with 143 test samples the standard error of the latter is about 0.017 by Eq. (2.24), so the difference is within about one and a half standard errors and should not be over-interpreted.

Figure 5.10 shows the two summaries that matter. The ROC curve hugs the top-left corner, giving an AUC of 0.9914; because the AUC depends only on the ranking of the predicted probabilities, it is unaffected by the class imbalance that makes the raw accuracy flattering. The confusion matrix is the more actionable object: it separates the three malignant cases predicted benign from the three benign cases predicted malignant, and in a screening context those two numbers carry very different costs.

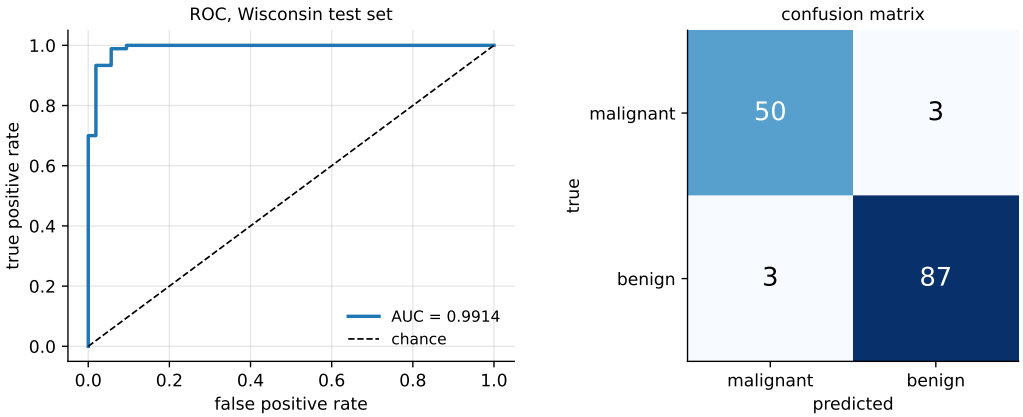


Fig. 5.10 Logistic regression on the Wisconsin test set. Left: the ROC curve, with AUC $= 0.9914$. Right: the confusion matrix, whose off-diagonal entries are the two kinds of error and are not equally costly.

5.14 Summary and the programs

Logistic regression is the bridge between the linear models of Chapter 3 and the neural networks of the next chapter, and it is worth setting out how little had to change and how much followed.

The model changed by one function. We kept the linear predictor $\boldsymbol{x}^T\boldsymbol{\theta}$ and passed it through the sigmoid (5.3), which confines the output to $(0,1)$ so that it can be read as a probability. Equivalently, by Eq. (5.10), we modelled the log-odds as linear while the probability itself is not.

The loss followed from the model rather than being chosen. Repeating the maximum-likelihood argument of Section 3.6 with Bernoulli rather than Gaussian observations produced the cross entropy (5.13), just as the notebox there predicted. This is worth remembering as a general principle: specify the distribution of the target and the loss function is determined. It is not, however, the only usable loss: Section 5.10 placed it in the family of margin-based surrogates for the 0–1 loss, alongside the hinge loss of the coming support vector machine and the exponential loss of boosting, and Proposition 5.1 showed that all of them find the same decision boundary while only the proper ones – cross entropy and squared error – estimate the class probability itself. With automatic differentiation, switching between them cost one line of code.

The mathematics then ran parallel to Chapter 3 throughout. The gradient is $-\boldsymbol{X}^T(\boldsymbol{y}-\boldsymbol{p})$, design matrix against residual, exactly as in least squares with the linear prediction replaced by the probability. The Hessian is $\boldsymbol{X}^T\boldsymbol{W}\boldsymbol{X}$ rather than $\boldsymbol{X}^T\boldsymbol{X}$, with weights $p_i(1-p_i)$ that vanish for confidently classified points – so the fit is controlled by the points near the decision boundary. Positivity of those weights proves convexity, so Chapter 4 applies with its guarantees intact. What was lost is the closed form: the parameter appears inside a non-linear function and cannot be extracted, so we optimise numerically, by gradient descent for large problems and by Newton-Raphson – which is iteratively reweighted least squares, Eq. (5.34) – for small ones. For large problems, Section 5.9 measured what Chapter 4 promised: on 50000 observations five epochs of minibatch SGD are worth a hundred of gradient descent, the adaptive methods add nothing on standardised features and an order of magnitude on unstandardised ones – they remove the column scales from the curvature but not from the distance each parameter has to travel, so that standardising still pays – and the accuracy saturates long before the cost.

Drawn as a diagram, Section 5.8.1, the model is a single artificial neuron – the perceptron with a sigmoid in place of the step – and the gradient is the delta rule (5.37), the residual times the input on each edge. The same diagram read backwards is the reverse sweep of automatic differentiation, Section 5.8.2, which produces $\boldsymbol{X}^T(\boldsymbol{p}-\boldsymbol{y})$ in one pass at the cost of evaluating the cost itself; with more than one layer it is backpropagation.

A third extension removed the linearity altogether. By the representer theorem of Section 3.12.3 the penalised problem in a reproducing kernel Hilbert space has a finite solution $f=\sum_j \alpha_j k(\boldsymbol{x}_j,\cdot)$, and Newton's method on it, Eq. (5.65), is iteratively reweighted *kernel* ridge regression – the same sentence as before with the design matrix replaced by the Gram matrix throughout. Proposition 5.2 then says that the coefficients are the residuals divided by the penalty, so they are never zero: kernel logistic regression buys a calibrated probability and pays for it with a predictor that involves every training point, which is the comparison Section 6.13 completes.

Two extensions completed the picture. The softmax (5.27) generalises the sigmoid to K classes and preserves the gradient structure, and the penalties of Chapter 3 carry over unchanged, with the additional observation that some penalty is *necessary* when the classes are linearly separable. Finally, assessing a classifier needs a different vocabulary from assessing a regression: the confusion matrix and the measures built from it, with the standing warning that accuracy alone is uninformative when the classes are imbalanced.

The complete programs are collected in `doc/BookML/BookPrograms`:

- `kernel_logistic.py` – kernel IRLS, Eq. (5.65), against gradient descent on the same objective, against the primal problem in an explicit feature map and against `scikit-learn`; the residual identity of Proposition 5.2; and the penalty study of Section 5.11.5.
- `logistic_regression.py` – the class of Section 5.8 for the binary and multiclass cases, the synthetic data generators, and the numerically stable sigmoid and softmax of Eqs. (5.3) and (5.30), and the automatic-differentiation version of Section 5.8.2: the one-observation trace, the forward and reverse sweeps by hand and from JAX, and the forward-against-reverse timing.
- `logistic_sgd.py` – the listing of Section 5.9: the minibatch loop of Eq. (5.45) around the step function of Section 4.12, run as gradient descent, SGD with constant and decaying learning rates, AdaGrad, RMSProp and Adam on standardised and on raw features, reproducing Table 5.1; the figure script `BookFigures/ch05_sgd_figures.py` adds the learning-rate grid behind Figures 5.6 and 5.7.
- `logistic_losses_jax.py` – the loss-agnostic trainer of Section 5.10.3: the six losses of Table 5.2 as one-line functions, gradient descent and Newton's method with derivatives from JAX, the checks against Eqs. (5.17) and (5.19), and the accuracy-and-calibration comparison of Proposition 5.1.
- `logreg_newton.py` – Newton-Raphson and IRLS as in Eqs. (5.33) and (5.34), with a comparison of iteration counts against gradient descent and against the adaptive methods of Chapter 4.
- `classification_metrics.py` – the confusion matrix, accuracy, precision, recall, F_1 and AUC of Section 5.12, checked against `scikit-learn`.
- `wisconsin.py` – the complete analysis of Section 5.13, including the feature histograms, the correlation matrix, the cross-validated accuracy and the ROC curve.

Each file runs as a script and reproduces the numbers quoted in this chapter. An executable version is available as a Jupyter notebook in the accompanying Jupyter-book.

Looking ahead.

Logistic regression is a neural network with no hidden layer, Figure 5.4. Its linear predictor is a single artificial neuron, its sigmoid is that neuron's activation function, its cross entropy is the loss used to train classification networks, and the minibatch loop of Section 5.9 is the loop that will train them. The next chapter inserts layers between the input and the output, so that the features fed to the final sigmoid are themselves learned rather than given; the gradient of the last layer will be exactly Eq. (5.17), and backpropagation is the chain rule (1.51) carrying that gradient backwards through the preceding layers. Almost everything in this chapter will be reused.

5.15 Exercises

Warm-up exercises

1. **Properties of the sigmoid.** (a) Prove Eq. (5.4), $1-\sigma(t)=\sigma(-t)$. (b) Prove Eq. (5.5), $\sigma'=\sigma(1-\sigma)$, and show that the derivative attains its maximum value $1/4$ at $t=0$. (c) Verify Eq. (5.7), $\tanh(t)=2\sigma(2t)-1$, and deduce the derivative of tanh in terms of tanh itself. (d) Show that the logit (5.6) is the inverse of the sigmoid.
2. **The cross entropy from the likelihood.** (a) Explain why the single expression $p^{y_i}(1-p)^{1-y_i}$ correctly handles both $y_i=0$ and $y_i=1$. (b) Derive Eq. (5.13) from Eq. (5.11). (c) Derive the compact form (5.14) and explain why it is numerically preferable.
3. **The gradient.** (a) Derive Eqs. (5.15) and (5.16) by differentiating Eq. (5.14). (b) Assemble them into the matrix form (5.17). (c) Compare with the least-squares gradient (1.38) and state precisely what is the same and what differs.
4. **Why not squared error (numerical).** Consider one observation with $y=1$ and a confidently wrong prediction, $\sigma(t)=0.01$. (a) Compute the gradient with respect to t of the squared error $(y-\sigma(t))^2$ and of the cross entropy. (b) Repeat for $\sigma(t)=0.5$ and $\sigma(t)=0.99$. (c) Plot both gradients against t and explain, using the notebox of Section 5.3, why the cross entropy is preferred.
5. **The Hessian and convexity.** (a) Derive Eq. (5.19) from Eq. (5.17). (b) Prove positive semi-definiteness as in Eq. (5.20). (c) Show that adding an ℓ_2 penalty makes the Hessian positive definite, and explain why the problem is then strictly convex. (d) Show that $W_{ii}\le 1/4$ and use this to justify the learning-rate bound quoted in Section 5.7.
6. **Separability (numerical).** Generate two classes that are perfectly separable in one dimension. (a) Fit an unpenalised logistic regression by gradient descent and plot $\|\boldsymbol{\theta}\|$ against the iteration count. What happens? (b) Plot the cost against the iteration count. Does it converge? (c) Repeat with an ℓ_2 penalty and explain the difference.
7. **Softmax.** (a) Show that Eq. (5.27) reduces to Eqs. (5.8) and (5.9) when $K=2$ and $\boldsymbol{\theta}_0=\mathbf{0}$. (b) Show that adding a constant vector to every $\boldsymbol{\theta}_k$ leaves the probabilities unchanged. (c) Show that Eq. (5.30) is mathematically identical to Eq. (5.27), and construct a numerical example in which the naive form overflows.
8. **Newton-Raphson and IRLS (numerical).** (a) Implement Eq. (5.33) and verify that it converges in a handful of iterations on the synthetic data of Section 5.8. (b) Verify algebraically that Eq. (5.33) can be rewritten as Eq. (5.34) with the adjusted response $\boldsymbol{z}=\boldsymbol{X}\boldsymbol{\theta}+\boldsymbol{W}^{-1}(\boldsymbol{y}-\boldsymbol{p})$. (c) Compare the iteration count against plain gradient descent and against Adam from Section 4.11, and discuss when each is preferable.
9. **Metrics (numerical).** For a deliberately imbalanced problem with 5% positives: (a) compute the accuracy of the classifier that always predicts the majority class; (b) fit a logistic

regression and compute accuracy, precision, recall, F_1 and AUC; (c) sweep the decision threshold from 0 to 1, plot precision and recall against it, and identify the threshold maximising F_1; (d) verify your `roc_auc` implementation against `sklearn.metrics.roc_auc_score`.

10. **The neuron and the delta rule.** (a) Derive Eq. (5.36) from Figure 5.4 by the chain rule through the two nodes σ and Σ, and check that summing it over the observations gives Eq. (5.17). (b) Replace the sigmoid by the step function and explain, in terms of the reverse sweep, why no gradient reaches the parameters. (c) Replace it instead by $\tanh$, with labels $y \in \{-1, 1\}$ and the squared error $\frac{1}{2}(y - \tanh z)^2$; derive the corresponding delta rule and compare the factor multiplying the residual with the logistic case.
11. **Gradient descent against SGD (numerical).** Using the listing of Section 5.9: (a) run plain SGD with $M = 1$, 8, 32, 128 and 512 at a constant γ and record the excess cost at the end of each epoch; verify that the size of the cloud in which the iterate settles scales as γ/M, Eq. (4.50), and the time to reach it as the number of updates; (b) repeat the gradient-descent run with γ equal to twice the bound (5.46) and describe what happens in the first few iterations; (c) standardise the raw features inside the listing and confirm that plain SGD then matches the adaptive methods, as Table 5.1 suggests; (d) add momentum, Eq. (4.28), to the step function and place it in Table 5.1; (e) compute the condition number of $\boldsymbol{D}^{-1/2}\boldsymbol{H}\boldsymbol{D}^{-1/2}$, $\boldsymbol{D} = \operatorname{diag}(\boldsymbol{H})$, at the optimum for the standardised and for the raw features and explain why the two agree, Eq. (5.47); then run Adam on the raw features for 60 epochs with $\gamma = 0.001$, 0.003 and 0.01, record the error of each parameter, and reproduce the two regimes of Figure 5.8.
12. **The focal loss is not proper.** For the focal loss (5.56) write the expected loss (5.50) as a function of $p = \sigma(t)$, $L(p) = -\eta(1-p)^{\gamma}\log p - (1-\eta)p^{\gamma}\log(1-p)$. (a) Show that $L'(p) = 0$ reduces to $\eta\,(1-p)^{\gamma-1}\,[\gamma p \log p + (1-p)] = (1-\eta)\,p^{\gamma-1}\,[\gamma(1-p)\log(1-p) + p]$, and that $p = \eta$ solves it only for $\gamma = 0$. (b) Solve it numerically and show that the minimiser $\hat{p}(\eta)$ lies strictly between η and $\frac{1}{2}$ for every $\eta \neq \frac{1}{2}$; plot $\hat{p}$ against η for $\gamma = 0, 1, 2, 5$ and compare with the focal curve in the right panel of Fig. 5.9. Then fit a logistic model with the focal loss to the data of Section 5.10.3, and recalibrate its output by fitting a one-dimensional logistic regression of y on $\hat{t}$ (Platt scaling); confirm that the recalibrated probabilities agree with the truth.
13. **Losses by automatic differentiation (numerical).** Using the code of Section 5.10.3: (a) add an *asymmetric* cross entropy in which a false negative costs c times a false positive, derive its pointwise minimiser as in Proposition 5.1, and verify numerically that the fitted decision boundary moves to $\eta = 1/(1+c)$; (b) add the Huberised hinge, quadratic for $m \in [-1, 1]$ and linear outside, and check that Newton's method with `jax.hessian` converges for it while it fails for the plain hinge (why?); (c) verify the entries of Table 5.2 by fitting each loss on 10^4 points and comparing $\hat{\boldsymbol{\theta}}$ against $\boldsymbol{\theta}_{\text{true}}$, $\frac{1}{2}\boldsymbol{\theta}_{\text{true}}$ or neither; (d) flip the labels of a random 10% of the points with $|t| > 2$ and compare how far each fitted $\hat{\boldsymbol{\theta}}$ moves; relate the ordering to the right-hand tails of Fig. 5.9.
14. **Correlated features.** In the Wisconsin data of Section 5.13: (a) find the pairs of features with correlation above 0.95; (b) fit logistic regression with ℓ_2 and with ℓ_1 penalties and compare which members of each correlated group survive; (c) relate what you find to the discussion of the Lasso and correlated predictors in the notebox of Section 3.9.

Project-style exercise: classification from end to end

Part a: your own implementation.

Write logistic regression from scratch, with the stable sigmoid, the cross-entropy cost (5.13), the analytical gradient (5.17) and an ℓ_2 penalty. Verify the gradient against a finite difference using Eq. (4.102) and against automatic differentiation as in Section 4.14.

Part b: optimisers.

Train it on the Wisconsin data using plain gradient descent, momentum, minibatch SGD, RMSProp and Adam from Chapter 4. For each, sweep the learning rate and record the best cross-entropy achieved and the number of epochs needed. Present the result as a table like Table 4.4, and discuss it in the light of the warning there that learning rates are not comparable across optimisers.

Part c: Newton-Raphson.

Add the Newton solver of Eq. (5.33) and compare its iteration count with the results of part b. Measure the wall-clock time as well, and determine the number of features at which the $\mathcal{O}(p^3)$ factorisation ceases to be worthwhile.

Part d: model selection.

Choose the penalty λ by k-fold cross-validation as in Section 2.12, using the cross entropy and then the AUC as the selection criterion. Do the two agree? Explain any difference using the notebox of Section 5.12.

Part e: assessment.

Report the confusion matrix, accuracy, precision, recall, F_1 and AUC on a test set touched only once. Compare against the majority-class baseline. Then choose a threshold appropriate to a screening application, justify it, and report how the confusion matrix changes.

Part f: multiclass.

Extend to the softmax (5.27) and apply it to a multiclass data set – the handwritten digits included with `scikit-learn` are a natural choice. Report the full $K \times K$ confusion matrix and identify which classes are confused with which. Keep this result: the next chapter applies a neural network to the same data, and the comparison is the point.

Chapter 6
Support Vector Machines

Logistic regression, in Chapter 5, placed a decision boundary by maximising a likelihood. Support vector machines place one by a different principle: among all boundaries that separate the classes, choose the one that stands as far as possible from the data. This is the *maximum margin* idea, and it leads to a strikingly different mathematical apparatus – constrained optimisation, Lagrange duality, and the kernel trick that lets a linear method draw non-linear boundaries at almost no extra cost.

The chapter is also where a promise made in Chapter 4 comes due. There we developed unconstrained methods, gradient descent and its descendants. The SVM problem is constrained, and no amount of gradient descent will respect an inequality. We shall therefore derive the dual problem, see that it is a quadratic program, and – rather than reach for a library – construct a solver for it from scratch. The algorithm we build, sequential minimal optimisation, is the one that made support vector machines practical, and it is simple enough to derive in full.

6.1 Hyperplanes and the margin

Consider two classes of points in a plane. A linear classifier splits the feature space into two half-spaces by placing a hyperplane between them; all points on one side are assigned to one class and all points on the other side to the other. The difficulty, immediately apparent from any picture of separable data, is that there are infinitely many such hyperplanes. Which should we choose?

The support vector machine answers: the one with the *maximum margin*, the largest distance to the nearest data points of either class. Maximising the margin provides a reinforcement, so that future data points can be classified with more confidence: a boundary that passes close to a training point will misclassify a new point drawn slightly to the other side of it, while a boundary in the middle of a wide corridor has room to spare.

Figure 6.1 shows the solution for two well-separated classes. The solid line is the decision boundary, the dashed lines are the margin hyperplanes $\boldsymbol{w}^T\boldsymbol{x} + b = \pm 1$, and the circled points are those with $\lambda_i > 0$. Two features are worth noticing in advance of the derivation. The circled points lie exactly on the dashed lines, which is the content of the complementary slackness condition (6.19); and there are only three of them out of sixty, so fifty-seven of the observations could be deleted without moving the boundary at all.

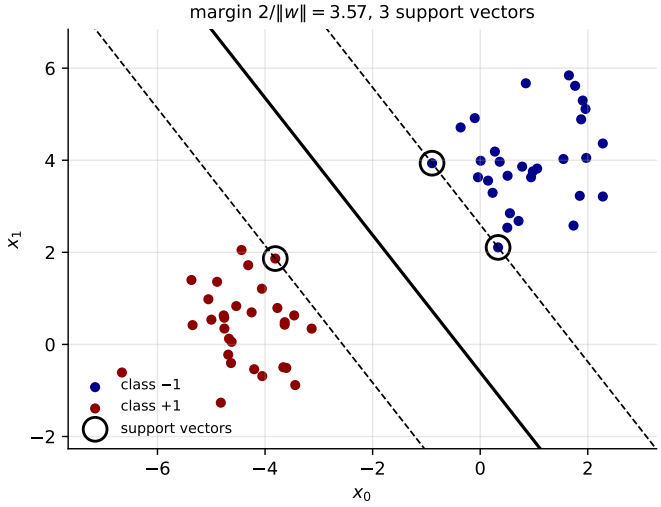


Fig. 6.1 The maximum-margin hyperplane for two separable classes. Dashed lines are the margin hyperplanes $\boldsymbol{w}^T\boldsymbol{x}+b=\pm 1$, separated by $2/\|\boldsymbol{w}\|$; circled points are the support vectors, the only observations entering Eq. (6.16).

The geometry.

Following the conventions of Chapter 1, define the function

$$f(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x}+b = 0, \tag{6.1}$$

whose zero set is the hyperplane L separating the classes. Any two points $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ lying on L satisfy $\boldsymbol{w}^T(\boldsymbol{x}_1-\boldsymbol{x}_2)=0$, so $\boldsymbol{w}$ is orthogonal to every direction within the hyperplane: *$\boldsymbol{w}$ is the normal vector*.

The signed distance from an arbitrary point $\boldsymbol{x}$ to L follows. Writing $\boldsymbol{x}=\boldsymbol{x}_0+\delta\,\boldsymbol{w}/\|\boldsymbol{w}\|$ with $\boldsymbol{x}_0$ on L, so that δ is the distance measured along the normal, and applying Eq. (6.1),

$$\delta = \frac{1}{\|\boldsymbol{w}\|}\left(\boldsymbol{w}^T\boldsymbol{x}+b\right). \tag{6.2}$$

The sign tells us which side of the boundary the point lies on and the magnitude how far away it is. Throughout this chapter we label the classes $y_i \in \{-1,+1\}$ rather than $\{0,1\}$, which is the convention that makes the algebra of margins clean: the product $y_i f(\boldsymbol{x}_i)$ is then positive exactly when the point is correctly classified.

6.2 A first attempt, and why it fails

Before constructing the margin, it is instructive to try the obvious thing. Define a cost function running over the set M of currently misclassified points,

$$C(\boldsymbol{w},b) = -\sum_{i\in M} y_i\left(\boldsymbol{w}^T\boldsymbol{x}_i+b\right), \tag{6.3}$$

where a point is counted as misclassified when $y_i=+1$ but $\boldsymbol{w}^T\boldsymbol{x}_i+b<0$, or the reverse. Each term in the sum is positive for a misclassified point, so the cost measures the total extent of

the errors. Differentiating,

$$\frac{\partial C}{\partial b} = -\sum_{i \in M} y_i, \qquad \frac{\partial C}{\partial \boldsymbol{w}} = -\sum_{i \in M} y_i \boldsymbol{x}_i, \tag{6.4}$$

and we may descend with any method of Chapter 4,

$$b \leftarrow b + \gamma \frac{\partial C}{\partial b}, \qquad \boldsymbol{w} \leftarrow \boldsymbol{w} + \gamma \frac{\partial C}{\partial \boldsymbol{w}}, \tag{6.5}$$

with γ the familiar learning rate. This is the *perceptron* algorithm, and the framework is that of logistic regression in Chapter 5.

It works, after a fashion, and its defects are instructive. If the data are separable it converges to *some* separating hyperplane, but which one depends on the initialisation and the order of the updates – the algorithm stops as soon as M is empty, wherever that happens to be, so it will happily return a boundary grazing the data. When the gap between the classes is small it may need very many iterations. And if the data are *not* separable the set M never empties and the iteration does not converge at all.

The maximum-margin formulation fixes all three defects at once: the solution is unique, it is characterised by a clean optimisation problem, and the soft version of Section 6.6 handles non-separable data gracefully.

6.3 The maximum-margin problem

We seek a margin M, with $\boldsymbol{w}$ normalised so that $\|\boldsymbol{w}\| = 1$, subject to

$$y_i \left(\boldsymbol{w}^T \boldsymbol{x}_i + b \right) \geq M \qquad \forall i = 1, 2, \ldots, n, \tag{6.6}$$

which says that every point lies at signed distance at least M from the boundary, on the correct side. Dropping the normalisation and using Eq. (6.2), the condition reads

$$\frac{1}{\|\boldsymbol{w}\|} y_i \left(\boldsymbol{w}^T \boldsymbol{x}_i + b \right) \geq M \qquad \text{or} \qquad y_i \left(\boldsymbol{w}^T \boldsymbol{x}_i + b \right) \geq M \|\boldsymbol{w}\|. \tag{6.7}$$

Now comes the step that makes the problem tractable. Equation (6.1) is unchanged if $\boldsymbol{w}$ and b are both multiplied by a positive constant, so the pair $(\boldsymbol{w}, b)$ is determined only up to scale and we are free to fix that scale however we like. Choosing $\|\boldsymbol{w}\| = 1/M$ turns the right-hand side of Eq. (6.7) into unity, and the problem becomes

$$\boxed{\min_{\boldsymbol{w}, b} \ \tfrac{1}{2} \boldsymbol{w}^T \boldsymbol{w} \qquad \text{subject to} \qquad y_i \left(\boldsymbol{w}^T \boldsymbol{x}_i + b \right) \geq 1 \quad \forall i.} \tag{6.8}$$

The margin is now the inverse of the norm: since $M = 1/\|\boldsymbol{w}\|$ and the corridor extends by M on each side, the total width between the two classes is $2/\|\boldsymbol{w}\|$. Maximising the margin is therefore minimising the norm, and the factor $\frac{1}{2}$ is inserted purely so that the derivative comes out clean.

Equation (6.8) is a *convex* problem – a quadratic objective with linear inequality constraints – so by Section 4.1 it has a unique global minimum. But the methods of Chapter 4 cannot be applied directly, because they know nothing of constraints. We need Lagrange multipliers.

6.4 A reminder on Lagrange multipliers

Consider a function $f(x,y,z)$ of three independent variables. For f to be extremal we require $df = 0$, and since

$$df = \frac{\partial f}{\partial x}dx + \frac{\partial f}{\partial y}dy + \frac{\partial f}{\partial z}dz, \tag{6.9}$$

a necessary and sufficient condition is that all three partial derivatives vanish.

In many problems, however, the variables are subject to constraints and are no longer independent. In principle one could use each constraint to eliminate a variable and proceed with a smaller independent set. Lagrange's method is the alternative when elimination is inconvenient or destroys the symmetry of the problem.

Suppose the variables satisfy a constraint

$$\phi(x,y,z) = 0, \qquad \text{whence} \qquad d\phi = \frac{\partial \phi}{\partial x}dx + \frac{\partial \phi}{\partial y}dy + \frac{\partial \phi}{\partial z}dz = 0. \tag{6.10}$$

We may no longer set the three partial derivatives of f separately to zero, because only two of the variables are independent: if x and y are chosen freely then dz is determined. But since $d\phi = 0$, we may add $\lambda\, d\phi$ to df for any multiplier λ without changing anything,

$$df + \lambda\, d\phi = \left(\frac{\partial f}{\partial x} + \lambda \frac{\partial \phi}{\partial x}\right)dx + \left(\frac{\partial f}{\partial y} + \lambda \frac{\partial \phi}{\partial y}\right)dy + \left(\frac{\partial f}{\partial z} + \lambda \frac{\partial \phi}{\partial z}\right)dz = 0. \tag{6.11}$$

We now *choose* λ so that the coefficient of the dependent differential dz vanishes. The remaining two differentials are independent and arbitrary, so their coefficients must vanish too, and we obtain the three symmetric conditions

$$\frac{\partial}{\partial x}(f + \lambda\phi) = \frac{\partial}{\partial y}(f + \lambda\phi) = \frac{\partial}{\partial z}(f + \lambda\phi) = 0, \tag{6.12}$$

together with the constraint $\phi = 0$ itself – four equations for the four unknowns x, y, z, λ. Equivalently, we form the *Lagrangian* $\mathscr{L} = f + \lambda\phi$ and treat all variables, the multiplier included, as independent.

For *inequality* constraints of the form $\phi \geq 0$ the situation is richer, and the conditions that replace Eq. (6.12) are the Karush-Kuhn-Tucker conditions, which we shall meet in Eq. (6.19). The essential addition is that a multiplier belonging to an inequality must be non-negative, and that it vanishes whenever its constraint is satisfied strictly – a constraint that is not binding exerts no force.

6.5 The dual problem

We attach a multiplier $\lambda_i \geq 0$ to each of the n inequality constraints of Eq. (6.8) and form the Lagrangian

$$\mathscr{L}(\boldsymbol{\lambda}, b, \boldsymbol{w}) = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} - \sum_{i=1}^{n} \lambda_i \left[y_i \left(\boldsymbol{w}^T \boldsymbol{x}_i + b \right) - 1 \right]. \tag{6.13}$$

The sign is chosen so that violating a constraint – making the bracket negative – increases $\mathscr{L}$, which is what a penalty should do.

Differentiating with respect to the primal variables, using Eqs. (1.25) and (1.28) from Section 1.8.3,

$$\frac{\partial \mathscr{L}}{\partial b} = -\sum_i \lambda_i y_i = 0, \tag{6.14}$$

$$\frac{\partial \mathscr{L}}{\partial \boldsymbol{w}} = \boldsymbol{w} - \sum_i \lambda_i y_i \boldsymbol{x}_i = 0. \tag{6.15}$$

The second is worth pausing over. It says

$$\boxed{\boldsymbol{w} = \sum_i \lambda_i y_i \boldsymbol{x}_i,} \tag{6.16}$$

that the optimal normal vector is a linear combination of the training points, with the labels supplying the signs and the multipliers the weights. The solution lives in the span of the data, a fact which will make the kernel trick of Section 6.7 possible.

Eliminating the primal variables.

Substituting Eqs. (6.14) and (6.16) back into Eq. (6.13) removes $\boldsymbol{w}$ and b entirely. The quadratic term becomes

$$\frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} = \frac{1}{2}\sum_{i,j} \lambda_i \lambda_j y_i y_j \boldsymbol{x}_i^T \boldsymbol{x}_j,$$

the term $\sum_i \lambda_i y_i \boldsymbol{w}^T \boldsymbol{x}_i$ equals $\boldsymbol{w}^T\boldsymbol{w}$ by the same substitution, the term in b vanishes by Eq. (6.14), and the remaining $\sum_i \lambda_i$ survives. Collecting,

$$\boxed{\mathscr{L}(\boldsymbol{\lambda}) = \sum_i \lambda_i - \frac{1}{2}\sum_{i,j}^{n} \lambda_i \lambda_j y_i y_j \boldsymbol{x}_i^T \boldsymbol{x}_j,} \tag{6.17}$$

to be *maximised* subject to

$$\lambda_i \geq 0 \qquad \text{and} \qquad \sum_i \lambda_i y_i = 0. \tag{6.18}$$

This is the *dual problem*. Two features are decisive. The data enter only through the inner products $\boldsymbol{x}_i^T \boldsymbol{x}_j$, never individually – which is what Section 6.7 will exploit. And the number of variables is n, the number of *samples*, rather than p, the number of features; for a problem with few samples and very many features, or with infinitely many implicit features, this is an enormous saving.

The KKT conditions and support vectors.

The solution must in addition satisfy the Karush-Kuhn-Tucker complementary slackness condition

$$\lambda_i \left[y_i \left(\boldsymbol{w}^T \boldsymbol{x}_i + b \right) - 1 \right] = 0 \qquad \forall i, \tag{6.19}$$

which says that for each i at least one of the two factors vanishes. The consequences are the heart of the method:

- If $\lambda_i > 0$ then $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) = 1$: the point lies *exactly on* the margin boundary.
- If $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) > 1$, the point lies strictly beyond the margin and we must have $\lambda_i = 0$: it contributes nothing to Eq. (6.16).

The points with $\lambda_i > 0$ are the *support vectors*. They are the points closest to the boundary, they alone define the margin, and they alone appear in the solution. Every other point could

be deleted from the training set without changing the answer – a property no other method in this book possesses, and the source of the name.

The reader may recognise a foreshadowing. In Section 5.4 we observed that the logistic Hessian $\boldsymbol{X}^T\boldsymbol{W}\boldsymbol{X}$ has weights $p_i(1-p_i)$ which vanish for confidently classified points, so that the fit is controlled by points near the boundary. The support vector machine takes that tendency to its limit: distant points are not merely down-weighted but discarded exactly.

Matrix form.

Collecting $\boldsymbol{\lambda} = [\lambda_1, \dots, \lambda_n]$ and $\boldsymbol{y} = [y_1, \dots, y_n]$, minimising the negative of Eq. (6.17) reads

$$\frac{1}{2}\boldsymbol{\lambda}^T \begin{bmatrix} y_1 y_1 \boldsymbol{x}_1^T \boldsymbol{x}_1 & y_1 y_2 \boldsymbol{x}_1^T \boldsymbol{x}_2 & \cdots & y_1 y_n \boldsymbol{x}_1^T \boldsymbol{x}_n \\ y_2 y_1 \boldsymbol{x}_2^T \boldsymbol{x}_1 & y_2 y_2 \boldsymbol{x}_2^T \boldsymbol{x}_2 & \cdots & y_2 y_n \boldsymbol{x}_2^T \boldsymbol{x}_n \\ \vdots & \vdots & \ddots & \vdots \\ y_n y_1 \boldsymbol{x}_n^T \boldsymbol{x}_1 & y_n y_2 \boldsymbol{x}_n^T \boldsymbol{x}_2 & \cdots & y_n y_n \boldsymbol{x}_n^T \boldsymbol{x}_n \end{bmatrix} \boldsymbol{\lambda} - \mathbb{1}^T \boldsymbol{\lambda}, \tag{6.20}$$

subject to $\boldsymbol{y}^T\boldsymbol{\lambda} = 0$ and $\boldsymbol{\lambda} \succeq \boldsymbol{0}$. The matrix has entries $P_{ij} = y_i y_j \boldsymbol{x}_i^T \boldsymbol{x}_j$ and is positive semi-definite, being of the form $\boldsymbol{Z}^T\boldsymbol{Z}$ with $\boldsymbol{Z} = [y_1\boldsymbol{x}_1, \dots, y_n\boldsymbol{x}_n]$, so by Section 1.4 the problem is convex.

Recovering the classifier.

Solving Eq. (6.20) yields the λ_i. The normal vector follows from Eq. (6.16), and the intercept from the condition that any support vector sits on the margin, $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) = 1$, whence $b = 1/y_i - \boldsymbol{w}^T\boldsymbol{x}_i = y_i - \boldsymbol{w}^T\boldsymbol{x}_i$ using $1/y_i = y_i$. In floating point one averages over all N_s support vectors rather than trusting a single one,

$$b = \frac{1}{N_s} \sum_{j \in N_s} \left(y_j - \sum_{i=1}^{n} \lambda_i y_i \boldsymbol{x}_i^T \boldsymbol{x}_j \right). \tag{6.21}$$

A new observation is then classified by

$$\hat{y} = \operatorname{sign}\left(\boldsymbol{w}^T\boldsymbol{x} + b\right) = \operatorname{sign}\left(\sum_i \lambda_i y_i \boldsymbol{x}_i^T \boldsymbol{x} + b \right), \tag{6.22}$$

where the second form – involving only inner products between the new point and the support vectors – is the one we shall generalise.

6.6 The soft margin

Everything so far assumed the classes to be separable. When they overlap, no $\boldsymbol{w}$ satisfies all the constraints of Eq. (6.8) and the problem is infeasible. The remedy is to allow some points to violate the margin, at a price.

Introduce *slack* variables $\boldsymbol{\xi} = [\xi_1, \dots, \xi_n]$ with $\xi_i \geq 0$ and relax the constraint to

$$y_i\left(\boldsymbol{w}^T\boldsymbol{x}_i + b\right) \geq 1 - \xi_i. \tag{6.23}$$

The value ξ_i measures the amount by which point i falls on the wrong side of *its* margin. A point with $\xi_i = 0$ is correctly placed; one with $0 < \xi_i < 1$ is inside the margin but still correctly

classified; and $\xi_i > 1$ means an outright misclassification. Bounding $\sum_i \xi_i$ therefore bounds the total margin violation, and in particular bounds the number of misclassifications.

The optimisation problem becomes: minimise

$$\mathscr{L} = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} - \sum_{i=1}^{n} \lambda_i \left[y_i \left(\boldsymbol{w}^T \boldsymbol{x}_i + b \right) - (1 - \xi_i) \right] + C \sum_{i=1}^{n} \xi_i - \sum_{i=1}^{n} \gamma_i \xi_i, \tag{6.24}$$

where $C\sum_i \xi_i$ is the penalty for violation and the multipliers $\gamma_i \geq 0$ enforce $\xi_i \geq 0$. The constant C is a hyperparameter controlling the trade-off between a wide margin and few violations.

Differentiating with respect to the three primal variables,

$$\frac{\partial \mathscr{L}}{\partial b} = -\sum_i \lambda_i y_i = 0, \qquad \frac{\partial \mathscr{L}}{\partial \boldsymbol{w}} = \boldsymbol{w} - \sum_i \lambda_i y_i \boldsymbol{x}_i = 0, \qquad \frac{\partial \mathscr{L}}{\partial \xi_i} = C - \lambda_i - \gamma_i = 0. \tag{6.25}$$

The first two are unchanged from Eqs. (6.14) and (6.15). The third gives $\lambda_i = C - \gamma_i$, and since $\gamma_i \geq 0$ this bounds $\lambda_i \leq C$.

Substituting back produces *exactly* the same dual objective (6.17) as before – the slack variables cancel completely – but with the constraints

$$\boxed{0 \leq \lambda_i \leq C \qquad \text{and} \qquad \sum_i \lambda_i y_i = 0.} \tag{6.26}$$

The only change is that the multipliers now have a ceiling. This is an unusually clean outcome: the entire effect of allowing misclassification is to replace the constraint $\lambda_i \geq 0$ by the *box* constraint $0 \leq \lambda_i \leq C$.

The KKT conditions become

$$\lambda_i \left[y_i \left(\boldsymbol{w}^T \boldsymbol{x}_i + b \right) - (1 - \xi_i) \right] = 0, \qquad \gamma_i \xi_i = 0, \qquad y_i \left(\boldsymbol{w}^T \boldsymbol{x}_i + b \right) - (1 - \xi_i) \geq 0, \tag{6.27}$$

and combining them classifies every training point into exactly three cases, which is the most useful summary of the whole construction:

- $\lambda_i = 0$: the point is strictly outside the margin, $y_i f(\boldsymbol{x}_i) > 1$, and is irrelevant to the solution.
- $0 < \lambda_i < C$: then $\gamma_i > 0$, so $\xi_i = 0$, and the point lies exactly on the margin, $y_i f(\boldsymbol{x}_i) = 1$. These are the *free* support vectors, and Eq. (6.21) should use them.
- $\lambda_i = C$: then $\gamma_i = 0$ and ξ_i may be positive – the point is inside the margin or misclassified. These are the *bound* support vectors.

Machine learning connection. The parameter C plays the role that $1/\lambda$ played in Ridge regression, and the correspondence is exact rather than analogical, as Section 6.12 will show. A small C tolerates many violations and buys a wide margin – high bias, low variance in the language of Section 2.10 – while a large C insists on classifying the training data and produces a narrow margin that tracks individual points. As always, C is chosen by the cross-validation of Section 2.12 and never by looking at the test set. One diagnostic is worth knowing: the fraction of training points that are support vectors is a rough upper bound on the leave-one-out error rate, since deleting a non-support vector provably changes nothing. A model in which almost every point is a support vector is a model that has learned very little.

Figure 6.2 shows the effect of C on genuinely overlapping classes. At $C = 0.01$ violations are cheap, the margin is wide, and a large fraction of the data lies inside it as bound support vectors; at $C = 10$ the margin is narrow and supported by fewer points. The decision boundary

itself barely moves, which is the usual finding: C controls the confidence with which the boundary is drawn far more than its location.

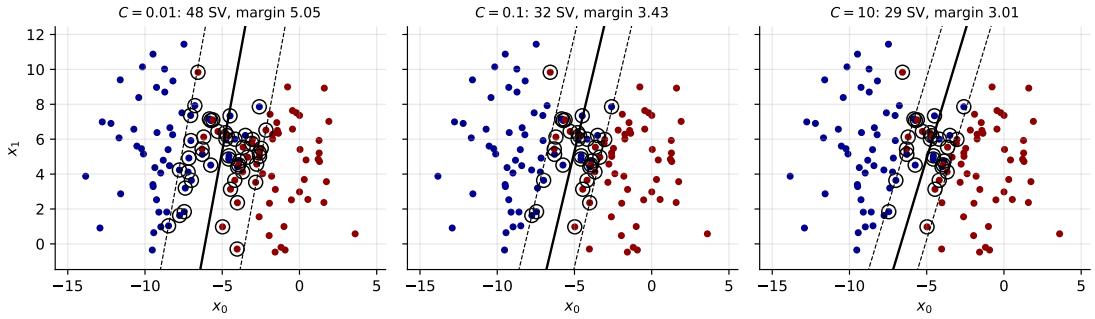


Fig. 6.2 Soft-margin solutions for three values of C on overlapping classes. Circled points are support vectors. Decreasing C widens the margin and admits more violations, in accordance with the three KKT cases of Section 6.6.

6.7 Kernels and non-linearity

Everything so far produces a linear boundary. As in Section 3.2, we can gain flexibility by expanding the features – replacing $\boldsymbol{x}$ by $\boldsymbol{z} = \phi(\boldsymbol{x})$ for some map ϕ into a higher-dimensional space, and separating linearly there. A boundary that is linear in $\boldsymbol{z}$ is curved in $\boldsymbol{x}$.

The simplest illustration is one-dimensional. Take nine points on a line with the two outer groups in one class and the middle group in the other; no single threshold separates them. Now map $x \mapsto (x, x^2)$ into the plane. The points lie on a parabola, and a horizontal line at the right height separates them perfectly.

```
import numpy as np
import matplotlib.pyplot as plt

X1D = np.linspace(-4, 4, 9).reshape(-1, 1)
X2D = np.c_[X1D, X1D**2]                      # the map phi(x) = (x, x^2)
y = np.array([0, 0, 1, 1, 1, 1, 1, 0, 0])

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].plot(X1D[y == 0], np.zeros(4), "bs")
ax[0].plot(X1D[y == 1], np.zeros(5), "g^")
ax[0].set_xlabel(r"$x_1$"); ax[0].set_yticks([]); ax[0].axhline(0, color="k")

ax[1].plot(X2D[y == 0, 0], X2D[y == 0, 1], "bs")
ax[1].plot(X2D[y == 1, 0], X2D[y == 1, 1], "g^")
ax[1].plot([-4.5, 4.5], [6.5, 6.5], "r--", linewidth=3)   # the separating line
ax[1].set_xlabel(r"$x_1$"); ax[1].set_ylabel(r"$x_2$")
plt.show()
```

The problem with doing this literally.

Suppose we work in two dimensions and choose the quadratic map

$$\boldsymbol{z} = \phi(\boldsymbol{x}) = \left(x^2, y^2, \sqrt{2}xy\right), \tag{6.28}$$

writing $\boldsymbol{x} = (x, y)$ for a single sample. The dual objective (6.17) becomes

$$\mathscr{L} = \sum_i \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j \boldsymbol{z}_i^T \boldsymbol{z}_j, \tag{6.29}$$

identical in form, with the constraints and the recovery of b unchanged. The only new work is computing $\boldsymbol{z}_i^T \boldsymbol{z}_j$. Doing that literally means constructing every $\phi(\boldsymbol{x}_i)$ and taking inner products in the larger space – feasible here, prohibitive for a map into a thousand dimensions, and impossible for one into infinitely many.

The kernel trick.

Define the *kernel*

$$K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \boldsymbol{z}_i^T \boldsymbol{z}_j = \phi(\boldsymbol{x}_i)^T \phi(\boldsymbol{x}_j). \tag{6.30}$$

For the map (6.28) we can evaluate it explicitly,

$$K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \begin{bmatrix} x_i^2 \\ y_i^2 \\ \sqrt{2} x_i y_i \end{bmatrix}^T \begin{bmatrix} x_j^2 \\ y_j^2 \\ \sqrt{2} x_j y_j \end{bmatrix} = x_i^2 x_j^2 + 2 x_i x_j y_i y_j + y_i^2 y_j^2 = \left(\boldsymbol{x}_i^T \boldsymbol{x}_j \right)^2, \tag{6.31}$$

the last step by recognising the expansion of a square. This is the whole idea. *The inner product in the three-dimensional feature space is a function of the inner product in the original two-dimensional space.* We never construct ϕ; we compute $(\boldsymbol{x}_i^T \boldsymbol{x}_j)^2$ directly, at the cost of a two-dimensional dot product and one multiplication.

Replacing every inner product by a kernel, the dual becomes

$$\mathscr{L} = \sum_i \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j K(\boldsymbol{x}_i, \boldsymbol{x}_j), \tag{6.32}$$

with the same constraints (6.26), and the classifier Eq. (6.22) becomes

$$\hat{y} = \operatorname{sign} \left(\sum_i \lambda_i y_i K(\boldsymbol{x}_i, \boldsymbol{x}) + b \right). \tag{6.33}$$

Note what has *not* survived: $\boldsymbol{w}$ itself. It lives in the feature space and may be infinite-dimensional, so we never form it; the sum over support vectors in Eq. (6.33) is the only representation we have or need. This is why the dual formulation was worth the effort. The matrix in Eq. (6.20) simply acquires entries $P_{ij} = y_i y_j K(\boldsymbol{x}_i, \boldsymbol{x}_j)$.

6.7.1 Common kernels and Mercer's theorem

Several kernels are in common use:

- Linear: $K(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{x}^T \boldsymbol{y}$.
- Polynomial: $K(\boldsymbol{x}, \boldsymbol{y}) = (\gamma \boldsymbol{x}^T \boldsymbol{y} + r)^d$.
- Gaussian radial basis function: $K(\boldsymbol{x}, \boldsymbol{y}) = \exp\left(-\gamma \|\boldsymbol{x} - \boldsymbol{y}\|^2\right)$.
- Sigmoid: $K(\boldsymbol{x}, \boldsymbol{y}) = \tanh(\gamma \boldsymbol{x}^T \boldsymbol{y} + r)$.

A natural worry is whether an arbitrary function of two arguments corresponds to any feature map at all. *Mercer's theorem* answers it: if K is symmetric, continuous, and such

that the matrix with entries $K(\boldsymbol{x}_i, \boldsymbol{x}_j)$ is positive semi-definite for every finite set of points, then there exists a map ϕ, possibly into a space of very high or infinite dimension, with $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)^T \phi(\boldsymbol{x}_j)$. We may therefore use K knowing that ϕ exists, without ever knowing what it is. Some frequently used kernels – the sigmoid among them – fail Mercer's conditions for some parameter values and nevertheless work acceptably in practice.

The Gaussian kernel deserves a remark, because it is the default choice and its feature map is infinite-dimensional. Expanding the exponential in a power series produces terms of every polynomial degree, so the RBF kernel corresponds to a map into a space containing all monomials, with the higher-order ones progressively down-weighted. The parameter γ sets the length scale over which two points are considered similar: a large γ makes K decay rapidly, so each support vector influences only its immediate neighbourhood and the boundary becomes highly flexible; a small γ makes the kernel nearly constant and the boundary nearly linear. Together with C this gives two hyperparameters, and they interact, so the cross-validated search of Section 2.12 should be a two-dimensional grid.

Machine learning connection. The kernel matrix $K_{ij} = K(\boldsymbol{x}_i, \boldsymbol{x}_j)$ is the Gram matrix met in Section 1.10, and its positive semi-definiteness is the same condition that made covariance matrices well behaved. The same object underlies Gaussian process regression, where the Cholesky factorisation of Section 1.14 is applied to $\boldsymbol{K} + \sigma^2 \boldsymbol{I}$, and kernel ridge regression, which is Ridge regression with inner products replaced by kernels in exactly the manner of this section. A word of caution about cost: the kernel matrix has n^2 entries and most solvers need $\mathcal{O}(n^2)$ to $\mathcal{O}(n^3)$ operations, so kernel methods scale badly in the number of *samples* even while scaling beautifully in the number of features. For n beyond a few tens of thousands one turns to linear SVMs solved in the primal, as in Section 6.12, or to approximations of the kernel map.

Figure 6.3 shows the same data separated by three kernels. The linear kernel cannot represent the crescent boundary and misclassifies a large fraction; the cubic polynomial and the Gaussian both succeed, the latter producing a boundary that follows the data closely. Note the support vector counts printed above each panel: the polynomial kernel achieves the higher accuracy with twenty support vectors while the Gaussian needs more than twice as many, which by the argument of Section 6.6 suggests the polynomial model is the better matched of the two.

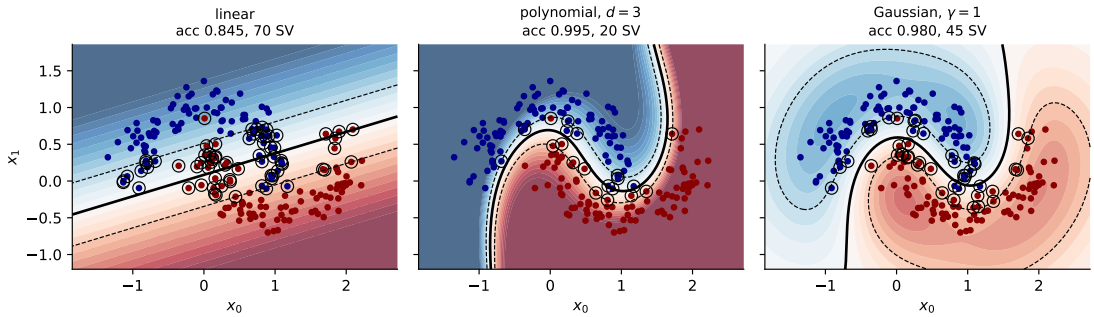


Fig. 6.3 Decision functions for three kernels on the moons data, all fitted with the solver of Section 6.10. Solid contour: the boundary; dashed: the ± 1 margins; circles: support vectors. Colour shows the value of Eq. (6.33) before the sign is taken.

6.8 The quadratic programme

Collecting the results, the problem we must solve is

$$\begin{aligned} \min_{\boldsymbol{\lambda}} \quad & \frac{1}{2}\boldsymbol{\lambda}^{T}\boldsymbol{P}\boldsymbol{\lambda} + \boldsymbol{q}^{T}\boldsymbol{\lambda}, \\ \text{subject to} \quad & \boldsymbol{G}\boldsymbol{\lambda} \preceq \boldsymbol{h} \quad \wedge \quad \boldsymbol{A}\boldsymbol{\lambda} = f, \end{aligned} \tag{6.34}$$

with $P_{ij} = y_i y_j K(\boldsymbol{x}_i, \boldsymbol{x}_j)$, $\boldsymbol{q} = -\mathbb{1}$, $\boldsymbol{A} = \boldsymbol{y}^{T}$ and $f = 0$. The box constraints (6.26) split into $-\lambda_i \le 0$ and $\lambda_i \le C$, which stacked together give

$$\boldsymbol{G} = \begin{bmatrix} -\boldsymbol{I} \\ \boldsymbol{I} \end{bmatrix}, \qquad \boldsymbol{h} = \begin{bmatrix} \boldsymbol{0} \\ C\mathbb{1} \end{bmatrix}. \tag{6.35}$$

This is a *quadratic programme*: a convex quadratic objective with linear constraints, the class of problems treated at length by Boyd and Vandenberghe [1].

One could hand Eq. (6.34) to a general-purpose quadratic programming package – `CVXOPT` is the usual choice in Python – and many treatments stop there. We shall not, for two reasons. A general solver treats $\boldsymbol{P}$ as a dense $n \times n$ matrix and costs $\mathcal{O}(n^3)$, which is wasteful given the very special structure here. And, more importantly, the algorithm that exploits that structure is simple enough to derive completely, which is worth more than a library call.

Why gradient descent will not do.

It is worth being explicit about why Chapter 4 does not simply apply. A gradient step moves $\boldsymbol{\lambda}$ in an arbitrary direction and will in general violate both $0 \le \lambda_i \le C$ and $\sum_i \lambda_i y_i = 0$. One could project back onto the feasible set after each step – projected gradient descent – but the projection onto the intersection of a box and a hyperplane is itself a small optimisation problem. The algorithm below takes a cleverer route: it moves in directions that *preserve* feasibility by construction.

6.9 Sequential minimal optimisation

The key observation is due to Platt. The equality constraint $\sum_i \lambda_i y_i = 0$ couples all the multipliers, so we cannot change one alone without violating it. But we can change *two* at once, adjusting the second to compensate the first. Two is therefore the smallest possible working set, and the resulting subproblem – a quadratic in one effective variable – can be solved *analytically*. No inner numerical optimisation is needed anywhere. Hence the name: sequential minimal optimisation.

The subproblem.

Fix all multipliers except λ_1 and λ_2. The equality constraint requires

$$\lambda_1 y_1 + \lambda_2 y_2 = -\sum_{i \ge 3} \lambda_i y_i \equiv \zeta, \tag{6.36}$$

a constant. Multiplying by y_1 and using $y_1^2 = 1$,

$$\lambda_1 = \gamma - s\lambda_2, \qquad s \equiv y_1 y_2, \qquad \gamma \equiv y_1 \zeta, \tag{6.37}$$

so the pair is confined to a line, and the problem has one free variable.

The bounds.

Feasibility requires both multipliers to lie in $[0,C]$, which restricts λ_2 to an interval $[L,H]$ determined by the geometry of the line and the box. Two cases arise. If $y_1 \neq y_2$ then $s = -1$ and Eq. (6.37) gives $\lambda_1 - \lambda_2 = \gamma$, a line of slope $+1$; requiring both coordinates to lie in $[0,C]$ gives

$$L = \max\left(0,\ \lambda_2^{\text{old}} - \lambda_1^{\text{old}}\right), \qquad H = \min\left(C,\ C + \lambda_2^{\text{old}} - \lambda_1^{\text{old}}\right). \tag{6.38}$$

If $y_1 = y_2$ then $s = +1$ and $\lambda_1 + \lambda_2 = \gamma$, a line of slope -1, giving

$$L = \max\left(0,\ \lambda_1^{\text{old}} + \lambda_2^{\text{old}} - C\right), \qquad H = \min\left(C,\ \lambda_1^{\text{old}} + \lambda_2^{\text{old}}\right). \tag{6.39}$$

If $L = H$ the line meets the box in a single point and there is nothing to do.

The analytic solution.

Substituting Eq. (6.37) into the dual objective (6.32) gives a quadratic in λ_2 alone. Writing $K_{ij} = K(\boldsymbol{x}_i, \boldsymbol{x}_j)$, its second derivative is

$$\eta \equiv \frac{d^2 \mathscr{L}}{d\lambda_2^2} = 2K_{12} - K_{11} - K_{22}, \tag{6.40}$$

which is ≤ 0 because $K_{11} - 2K_{12} + K_{22} = \|\phi(\boldsymbol{x}_1) - \phi(\boldsymbol{x}_2)\|^2 \geq 0$ by Eq. (6.30). The objective is therefore concave along the line and has a unique maximum. Setting the first derivative to zero and expressing the result in terms of the prediction errors

$$E_i = f(\boldsymbol{x}_i) - y_i, \qquad f(\boldsymbol{x}_i) = \sum_j \lambda_j y_j K_{ij} + b, \tag{6.41}$$

the unconstrained maximiser is

$$\boxed{\lambda_2^{\text{new}} = \lambda_2^{\text{old}} - \frac{y_2\,(E_1 - E_2)}{\eta}.} \tag{6.42}$$

Since $\eta < 0$, the step moves λ_2 opposite to $y_2(E_1 - E_2)$, which is the direction of increasing $\mathscr{L}$. Clipping to the feasible interval,

$$\lambda_2^{\text{clip}} = \begin{cases} H & \lambda_2^{\text{new}} > H, \\ \lambda_2^{\text{new}} & L \leq \lambda_2^{\text{new}} \leq H, \\ L & \lambda_2^{\text{new}} < L, \end{cases} \tag{6.43}$$

and recovering the partner from Eq. (6.37),

$$\lambda_1^{\text{new}} = \lambda_1^{\text{old}} + s\left(\lambda_2^{\text{old}} - \lambda_2^{\text{clip}}\right). \tag{6.44}$$

The equality constraint is satisfied exactly, by construction, at every step.

Updating the intercept.

After each step b must be recomputed so that the KKT conditions (6.27) hold for the two points just changed. If λ_1^{new} lies strictly inside $(0,C)$ then point 1 is a free support vector and must satisfy $y_1 f(\boldsymbol{x}_1) = 1$, which gives

$$b_1 = b - E_1 - y_1 \left(\lambda_1^{\text{new}} - \lambda_1^{\text{old}}\right) K_{11} - y_2 \left(\lambda_2^{\text{clip}} - \lambda_2^{\text{old}}\right) K_{12}, \tag{6.45}$$

and symmetrically for point 2,

$$b_2 = b - E_2 - y_1 \left(\lambda_1^{\text{new}} - \lambda_1^{\text{old}}\right) K_{12} - y_2 \left(\lambda_2^{\text{clip}} - \lambda_2^{\text{old}}\right) K_{22}. \tag{6.46}$$

If both multipliers are free the two agree; if only one is free we use the corresponding value; if neither is free any value between them satisfies the conditions and we take the average.

Choosing the pair.

What remains is a heuristic for selecting which two multipliers to optimise, and here we are guided by the KKT conditions rather than by the objective. Define $r_i = y_i E_i$. The conditions of Section 6.6 are violated when

$$r_i < -\varepsilon \ \text{ and } \ \lambda_i < C, \qquad \text{or} \qquad r_i > \varepsilon \ \text{ and } \ \lambda_i > 0, \tag{6.47}$$

for a tolerance ε. The outer loop alternates between sweeping all points and sweeping only the free support vectors – the latter being where the action is, since bound multipliers tend to stay bound – and terminates when a full sweep over all points produces no change, at which point the KKT conditions hold everywhere and we have the solution. Given a violating i, the second index j is chosen to maximise $|E_i - E_j|$, since by Eq. (6.42) the step length is proportional to that difference.

6.10 Implementation

We now write the solver. It is short, it uses nothing beyond `numpy`, and every line corresponds to an equation above.

```
import numpy as np


def linear_kernel(X, Z):
    return X @ Z.T


def polynomial_kernel(X, Z, degree=3, gamma=1.0, coef0=1.0):
    return (gamma * (X @ Z.T) + coef0) ** degree


def rbf_kernel(X, Z, gamma=1.0):
    """Gaussian kernel, computed through the identity
    ||x - z||^2 = ||x||^2 + ||z||^2 - 2 x.z  (Section 1.vectors)."""
    d2 = (np.sum(X**2, axis=1)[:, None] + np.sum(Z**2, axis=1)[None, :]
          - 2.0 * (X @ Z.T))
    return np.exp(-gamma * np.maximum(d2, 0.0))
```

The kernel matrix is computed once and cached, as are the errors E_i; both are the standard economies.

```
class SVC:
    """Support vector classifier trained by sequential minimal optimisation.

    Solves the dual (6.dualkernel) subject to the box constraints
    (6.boxconstraints), using the analytic two-variable update (6.smoupdate).
    Labels must be -1 and +1.
    """

    def __init__(self, C=1.0, kernel=linear_kernel, tol=1e-4, max_iter=10000):
        self.C = C
        self.kernel = kernel
        self.tol = tol
        self.max_iter = max_iter

    # ---------- the two-variable subproblem ----------
    def _take_step(self, i, j):
        if i == j:
            return False
        C, lam, y, K, E = self.C, self.lam_, self.y_, self.K_, self.E_
        li, lj = lam[i], lam[j]

        # Bounds L and H, Eqs. (6.boundsdiff) and (6.boundssame)
        if y[i] != y[j]:
            L, H = max(0.0, lj - li), min(C, C + lj - li)
        else:
            L, H = max(0.0, li + lj - C), min(C, li + lj)
        if L >= H:
            return False

        eta = 2.0 * K[i, j] - K[i, i] - K[j, j]        # Eq. (6.eta)
        if eta >= -1e-12:                              # degenerate, skip
            return False

        lj_new = lj - y[j] * (E[i] - E[j]) / eta       # Eq. (6.smoupdate)
        lj_new = min(H, max(L, lj_new))                # Eq. (6.clip)
        if abs(lj_new - lj) < 1e-10 * (lj_new + lj + 1e-10):
            return False
        li_new = li + y[i] * y[j] * (lj - lj_new)      # Eq. (6.smopartner)

        b1 = (self.b_ - E[i] - y[i] * (li_new - li) * K[i, i]
              - y[j] * (lj_new - lj) * K[i, j])        # Eq. (6.b1)
        b2 = (self.b_ - E[j] - y[i] * (li_new - li) * K[i, j]
              - y[j] * (lj_new - lj) * K[j, j])        # Eq. (6.b2)
        if 1e-8 < li_new < C - 1e-8:
            b_new = b1
        elif 1e-8 < lj_new < C - 1e-8:
            b_new = b2
        else:
            b_new = 0.5 * (b1 + b2)

        lam[i], lam[j] = li_new, lj_new
        self.b_ = b_new
        self.E_ = (self.K_ @ (lam * y)) + self.b_ - y   # refresh the cache
        return True

    # ---------- choose the partner for a violating index ----------
    def _examine(self, i):
        y, lam, C = self.y_, self.lam_, self.C
        r = y[i] * self.E_[i]
        violates = ((r < -self.tol and lam[i] < C - 1e-12) or
                    (r > self.tol and lam[i] > 1e-12))   # Eq. (6.kktviolation)
```

```
        if not violates:
            return False

        free = np.where((lam > 1e-12) & (lam < C - 1e-12))[0]
        if len(free) > 1:                       # heuristic: maximise |E_i - E_j|
            j = free[np.argmax(np.abs(self.E_[i] - self.E_[free]))]
            if self._take_step(i, j):
                return True
        for j in np.random.permutation(free):   # then any free multiplier
            if self._take_step(i, j):
                return True
        for j in np.random.permutation(len(y)):  # then anything at all
            if self._take_step(i, j):
                return True
        return False

    # ---------- the outer loop ----------
    def fit(self, X, y):
        X = np.asarray(X, dtype=float)
        y = np.asarray(y, dtype=float)
        n = len(y)

        self.X_, self.y_ = X, y
        self.K_ = self.kernel(X, X)
        self.lam_ = np.zeros(n)
        self.b_ = 0.0
        self.E_ = -y.copy()                     # since lambda = 0 and b = 0

        examine_all, num_changed, it = True, 0, 0
        while (num_changed > 0 or examine_all) and it < self.max_iter:
            num_changed = 0
            if examine_all:
                index_set = range(n)
            else:                               # only the free support vectors
                index_set = np.where((self.lam_ > 1e-12)
                                     & (self.lam_ < self.C - 1e-12))[0]
            for i in index_set:
                num_changed += self._examine(i)
                it += 1
            if examine_all:
                examine_all = False
            elif num_changed == 0:
                examine_all = True

        sv = self.lam_ > 1e-8
        self.sv_ = sv
        self.X_sv, self.y_sv, self.lam_sv = X[sv], y[sv], self.lam_[sv]
        self.n_iter_ = it
        return self

    def decision_function(self, X):
        """Eq. (6.kernelclassify) without the sign."""
        X = np.asarray(X, dtype=float)
        return self.kernel(X, self.X_sv) @ (self.lam_sv * self.y_sv) + self.b_

    def predict(self, X):
        return np.sign(self.decision_function(X))
```

Note that $\boldsymbol{w}$ never appears. For a linear kernel it can be recovered from Eq. (6.16) as `(lam_sv * y_sv) @ X_sv`, but for the RBF kernel it does not exist as a finite vector and the sum over support vectors in `decision_function` is the only representation available.

6.11 Verifying the solver

A solver written from scratch must be checked, and the KKT conditions give us exactly the tests to apply. Rather than merely comparing predictions against a library, we verify the mathematics.

```
import numpy as np
from sklearn.datasets import make_blobs, make_moons
from sklearn.svm import SVC as SklearnSVC

np.random.seed(0)

# --- a separable linear problem ---
X, y01 = make_blobs(n_samples=80, centers=2, cluster_std=0.8, random_state=3)
y = np.where(y01 == 0, -1.0, 1.0)

model = SVC(C=10.0, kernel=linear_kernel).fit(X, y)
w = (model.lam_sv * model.y_sv) @ model.X_sv            # Eq. (6.wfromlambda)

reference = SklearnSVC(C=10.0, kernel="linear").fit(X, y)
print("ours    w =", np.round(w, 5), " b =", round(model.b_, 5))
print("sklearn w =", np.round(reference.coef_.ravel(), 5),
      " b =", round(reference.intercept_[0], 5))
```

The two agree to about 3×10^{-4} in $\boldsymbol{w}$ and 8×10^{-4} in b, the difference being the finite KKT tolerance. More telling is the direct check of the conditions themselves.

```
X, y01 = make_moons(n_samples=200, noise=0.15, random_state=42)
y = np.where(y01 == 0, -1.0, 1.0)

gaussian = lambda A, B: rbf_kernel(A, B, gamma=1.0)
model = SVC(C=1.0, kernel=gaussian).fit(X, y)

margin = y * model.decision_function(X)                 # y_i f(x_i)
lam = model.lam_
free = (lam > 1e-6) & (lam < model.C - 1e-6)

print(f"free support vectors: |y f - 1| max = "
      f"{np.abs(margin[free] - 1).max():.2e}")          # must be 0
print(f"lambda = 0 points:    min y f = {margin[lam <= 1e-8].min():.4f}")  # >= 1
print(f"equality constraint:  sum lam y = {np.sum(lam * y):.2e}")          # = 0

dual = np.sum(lam) - 0.5 * np.sum(np.outer(lam * y, lam * y) * model.K_)
print(f"dual objective: {dual:.6f}")
```

The output is

```
free support vectors: |y f - 1| max = 8.95e-05
lambda = 0 points:    min y f = 1.0002
equality constraint:  sum lam y = -8.88e-16
dual objective: 27.790313
```

Every condition of Section 6.6 holds. The free support vectors sit on the margin to five decimal places; the points with vanishing multipliers all satisfy $y_i f(\boldsymbol{x}_i) \geq 1$, so none has been wrongly excluded; and the equality constraint holds to machine precision, as it must, since Eq. (6.44) enforces it exactly at every step. The dual objective agrees with the value 27.790310 obtained from `scikit-learn`'s multipliers to six significant figures – our value being very slightly the larger, as it should be for a maximisation. On this data set both classifiers reach 98% training accuracy and agree on every single prediction.

The soft margin in action.

Repeating on deliberately overlapping classes shows what C controls.

```
X, y01 = make_blobs(n_samples=120, centers=2, cluster_std=2.6, random_state=7)
y = np.where(y01 == 0, -1.0, 1.0)

for C in [0.01, 0.1, 1.0, 10.0]:
    m = SVC(C=C, kernel=linear_kernel).fit(X, y)
    w = (m.lam_sv * m.y_sv) @ m.X_sv
    at_bound = int(np.sum(m.lam_ > C - 1e-6))
    print(f"C={C:<6} n_sv={m.sv_.sum():3d} (at bound {at_bound:3d})  "
          f"margin={2/np.linalg.norm(w):7.4f}  "
          f"accuracy={np.mean(m.predict(X) == y):.4f}")
```

```
C=0.01   n_sv= 48 (at bound  46)  margin= 5.0478  accuracy=0.8750
C=0.1    n_sv= 32 (at bound  29)  margin= 3.4275  accuracy=0.8833
C=1.0    n_sv= 29 (at bound  26)  margin= 3.0133  accuracy=0.8833
C=10.0   n_sv= 28 (at bound  25)  margin= 2.9851  accuracy=0.8833
```

The support vector counts match `scikit-learn` exactly for every C. Reading the table, a small C makes violations cheap, so many points sit inside a wide margin and most multipliers are at the bound; increasing C raises the price of a violation, so the margin narrows and fewer points support it. The training accuracy barely moves, because these classes genuinely overlap and no linear boundary can do better – which is the situation the soft margin was invented for.

One caveat about reproducibility. The partner-selection heuristic of Section 6.9 falls back on a random permutation when the free multipliers offer no improvement, so the iterate path – and hence the last few digits of the margin, and occasionally the count of bound multipliers by one – depends on the seed. The solution itself does not: the dual objective and the predictions are reproducible, because the problem is convex and the optimum unique. The table above was produced with `np.random.seed(0)` set immediately before the loop.

The kernel trick, verified.

The identity (6.31) can be checked directly.

```
rng = np.random.default_rng(1)
a, b = rng.normal(size=2), rng.normal(size=2)
phi = lambda v: np.array([v[0]**2, v[1]**2, np.sqrt(2) * v[0] * v[1]])

print(phi(a) @ phi(b), (a @ b)**2)      # 0.9148995105  0.9148995105
```

On the moons data a cubic polynomial kernel with $C = 5$ reaches 99.5% training accuracy with only 20 support vectors, identical to `scikit-learn` in both figures – a more compact model than the RBF kernel needed, because the true boundary happens to be well matched by a low-order polynomial.

6.12 The primal view: the hinge loss

The dual is not the only route, and the alternative reveals what a support vector machine has in common with everything else in this book.

Return to the soft-margin problem. For a given $(\boldsymbol{w}, b)$ the optimal slack is as small as Eq. (6.23) permits, namely $\xi_i = \max\big(0, 1 - y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b)\big)$. Substituting this into the primal objective eliminates the slack variables and leaves an *unconstrained* problem,

$$\min_{\boldsymbol{w}, b} \; \frac{\lambda}{2}\|\boldsymbol{w}\|_2^2 + \frac{1}{n}\sum_{i=1}^{n} \max\big(0,\; 1 - y_i\,(\boldsymbol{w}^T \boldsymbol{x}_i + b)\big)\,, \tag{6.48}$$

where λ is proportional to $1/C$. The function $\max(0, 1 - yf)$ is the *hinge loss*: zero once the point is correctly classified with margin at least one, and growing linearly thereafter.

Equation (6.48) should look familiar. It is a loss plus an ℓ_2 penalty – precisely the structure of Ridge regression in Eq. (3.43) and of penalised logistic regression in Eq. (5.23). The three methods differ *only in the loss*:

Method	**Loss on one sample**
Ridge regression	$(y_i - f_i)^2$
Logistic regression	$\log\big(1 + e^{-y_i f_i}\big)$
Support vector machine	$\max\big(0, 1 - y_i f_i\big)$

with $f_i = \boldsymbol{w}^T \boldsymbol{x}_i + b$ throughout, and the labels ± 1 in the last two rows. Comparing the second and third explains the difference in behaviour. The logistic loss is positive everywhere, however well a point is classified, so every point exerts some pull on the boundary – which is why logistic regression has no support vectors. The hinge loss is exactly zero beyond the margin, so points that are comfortably correct exert none at all – which is why the SVM does. This is the same fact we derived from the KKT conditions in Section 6.5, now visible in the loss function itself.

Figure 6.4 draws the three losses of the table together with the misclassification loss they are all trying to approximate. All three are convex upper bounds on the step function, which is what makes them tractable surrogates. They differ in two respects that matter. Only the hinge loss is exactly zero for $yf > 1$, which produces the support vectors; and the squared loss is the only one that *increases* for large positive margin, penalising a point for being classified too well, which is why it is a poor choice for classification.

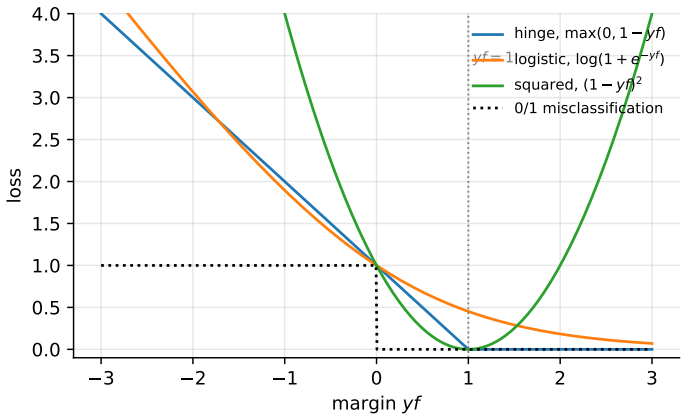


Fig. 6.4 The hinge, logistic and squared losses as functions of the margin yf, with the $0/1$ misclassification loss they bound. Only the hinge vanishes identically beyond $yf = 1$; only the squared loss grows again for large positive margin.

Solving the primal directly.

Because Eq. (6.48) is unconstrained, the methods of Chapter 4 apply after all. The hinge is not differentiable at the kink, but it is convex and possesses a subgradient everywhere, which is enough. The resulting algorithm, Pegasos, is stochastic gradient descent with the learning rate schedule $\gamma_t = 1/(\lambda t)$ of Eq. (4.52).

```
import numpy as np

def pegasos(X, y, lmbda=0.01, epochs=300, rng=None):
    """Primal SVM by stochastic subgradient descent on Eq. (6.hingeobjective)."""
    rng = np.random.default_rng(0) if rng is None else rng
    n, p = X.shape
    w, b, t = np.zeros(p), 0.0, 0
    for _ in range(epochs):
        for i in rng.permutation(n):
            t += 1
            gamma = 1.0 / (lmbda * t)                  # Eq. (4.timedecay)
            if y[i] * (w @ X[i] + b) < 1:              # inside the margin
                w = (1 - gamma * lmbda) * w + gamma * y[i] * X[i]
                b += gamma * y[i]
            else:                                      # outside: only the penalty
                w = (1 - gamma * lmbda) * w
    return w, b
```

On the standardised overlapping blobs above with $\lambda = 0.01$, Pegasos reaches an objective of 0.255097 against 0.255064 for the dual solution computed by `scikit-learn` – agreement to four decimal places – with identical training accuracy. The two formulations solve the same problem.

Which to prefer is decided by the shape of the data, and the criterion is the one that has recurred throughout this book. The dual has n variables and needs the $n \times n$ kernel matrix, so it suits problems with few samples, many features, and a non-linear boundary. The primal has $p+1$ variables and touches one sample at a time, so it suits problems with very many samples and a linear boundary. For text classification with a million documents and a million features, Pegasos is the practical choice and the kernel matrix is unthinkable.

Machine learning connection. The table above is worth memorising, because it organises a large part of supervised learning. Fix the model $f = \boldsymbol{w}^T\boldsymbol{x} + b$ and the penalty $\|\boldsymbol{w}\|^2$, and the choice of loss selects the method: squared error gives Ridge, log loss gives logistic regression, hinge loss gives the SVM. Change the penalty from ℓ_2 to ℓ_1 and each acquires a sparse variant, as in Section 3.9. Replace the linear f by a neural network and the same losses reappear unchanged in the next chapters. The apparent variety of methods is largely a small number of choices made independently, and recognising which choice a new method is making is usually the fastest way to understand it.

6.13 One penalty, three losses

Section 6.12 ended by placing the support vector machine in a family: Ridge regression, logistic regression and the support vector machine are one model with one penalty and three losses. That remark was about linear models. It survives the kernel trick intact, and in the kernel setting it becomes sharper, because it explains the one property that distinguishes this chapter's method from the other two – sparsity – and shows that the property is a consequence of the loss function alone.

6.13.1 The common problem

All three methods solve

$$\min_{f\in\mathscr{H}} \sum_{i=1}^{n} L(y_i, f(\boldsymbol{x}_i)) + \frac{\lambda}{2}\,\|f\|^2_{\mathscr{H}}\,, \tag{6.49}$$

over the reproducing kernel Hilbert space of Section 3.12.3, and differ only in L. With labels $y_i \in \{-1,+1\}$ as in this chapter:

$$L_{\text{sq}}(y,f) = \tfrac{1}{2}\,(y-f)^2 \qquad \text{kernel Ridge regression, Section 3.12,} \tag{6.50}$$

$$L_{\log}(y,f) = \ln\bigl(1+e^{-yf}\bigr) \qquad \text{kernel logistic regression, Section 5.11,} \tag{6.51}$$

$$L_{\text{hinge}}(y,f) = \max(0,\,1-yf) \qquad \text{the support vector machine, Eq. (6.48).} \tag{6.52}$$

By Theorem 3.12 all three minimisers have the form $f = \sum_j \alpha_j k(\boldsymbol{x}_j,\cdot)$, so all three produce a prediction rule of exactly the shape (6.33). The only thing that can distinguish them is the vector $\boldsymbol{\alpha}$, and there is a single formula for it.

Theorem 6.1 (The coefficients are minus the loss derivative). *Let $f^\star$ minimise Eq. (6.49) with L convex and differentiable in its second argument, and write $L_i' = \partial L(y_i,f)/\partial f$ evaluated at $f = f^\star(\boldsymbol{x}_i)$. Then*

$$f^\star(\cdot) = \sum_{i=1}^{n} \alpha_i\, k(\boldsymbol{x}_i,\cdot), \qquad \alpha_i = -\frac{L_i'}{\lambda}. \tag{6.53}$$

If L is convex but not differentiable, the same holds with L_i' any element of the subdifferential $\partial L(y_i, f^\star(\boldsymbol{x}_i))$, in the sense of Eq. (3.65).

Proof. By the reproducing property, $f \mapsto f(\boldsymbol{x}_i)$ has derivative $k(\boldsymbol{x}_i,\cdot)$ as an element of $\mathscr{H}$: perturbing f by δ changes $f(\boldsymbol{x}_i)$ by $\langle \delta, k(\boldsymbol{x}_i,\cdot)\rangle$. The derivative of $\frac{\lambda}{2}\|f\|^2$ is λf. Stationarity of Eq. (6.49) in $\mathscr{H}$ is therefore

$$\sum_{i=1}^{n} L_i' k(\boldsymbol{x}_i,\cdot) + \lambda f^\star = 0, \tag{6.54}$$

which rearranges to Eq. (6.53). For non-differentiable L the same argument applies with the subdifferential, 0 belonging to the sum of the subdifferential of the loss and the gradient of the penalty. □

Equation (6.53) is short and it settles everything. The weight a training point receives in the final predictor is the derivative of the loss at that point, divided by the penalty. A point on which the loss is locally flat receives no weight at all. Applying it in turn:

Squared loss.

$L' = f - y$, so $\alpha_i = (y_i - f_i)/\lambda$: the coefficient is the residual. Substituting into $\boldsymbol{f} = \boldsymbol{K}\boldsymbol{\alpha}$ gives $\boldsymbol{K}\boldsymbol{\alpha} = \boldsymbol{y} - \lambda\boldsymbol{\alpha}$, that is $(\boldsymbol{K} + \lambda \boldsymbol{I})\boldsymbol{\alpha} = \boldsymbol{y}$, which is Eq. (3.84) – the theorem contains kernel ridge regression as a special case.

Logistic loss.

$L' = -y\,\sigma(-yf)$, so $\alpha_i = y_i\sigma(-y_i f_i)/\lambda$. For labels in $\{0,1\}$ this is Proposition 5.2, $\alpha_i = (y_i - p_i)/\lambda$, in the other notation. The logistic function is strictly positive everywhere, so *no* coefficient vanishes.

Hinge loss.

$L' = -y$ where $yf < 1$ and $L' = 0$ where $yf > 1$, so

$$\alpha_i = \frac{y_i}{\lambda} \text{ if } y_i f_i < 1, \qquad \alpha_i = 0 \text{ if } y_i f_i > 1, \qquad y_i \alpha_i \in \left[0, \tfrac{1}{\lambda}\right] \text{ if } y_i f_i = 1. \tag{6.55}$$

The hinge loss is the only one of the three that is exactly zero on an open set, so its derivative is exactly zero there, so its coefficients are exactly zero there. *That is the entire origin of the support vectors.* The points with $\alpha_i \neq 0$ are precisely those with $y_i f_i \leq 1$ – inside the margin or misclassified – which is the characterisation Section 6.5 obtained from the Karush-Kuhn-Tucker conditions, and the bound $y_i \alpha_i \leq 1/\lambda$ is the box constraint $\lambda_i \leq C$ of Eq. (6.26) with $C = 1/\lambda$.

6.13.2 The three, measured together

The program `three_losses.py` fits all three to the same eighty points with the same Gaussian kernel and the same $\lambda = 1$, and checks Eq. (6.53) directly. The hinge problem is solved as the box-constrained quadratic programme obtained by writing $\alpha_i = y_i \beta_i / \lambda$, which is the dual of Section 6.5 without the equality constraint, since Eq. (6.49) carries no unpenalised intercept; the solver certifies itself by the duality gap:

```
primal value  26.4874425949
dual value    26.4874425949
duality gap   1.528e-13
```

The theorem then holds to machine precision for all three:

```
loss          L'(y,f)                    predicted alpha        max |alpha - predicted|
squared       f - y                      (y - f)/lambda        1.332e-15
logistic      -y sigma(-y f)             y sigma(-y f)/lambda  1.468e-13
hinge         -y if yf<1, 0 if yf>1      y/lambda or 0         0.000e+00
```

The hinge row excludes the fifteen points lying exactly on the margin, where L is not differentiable and Eq. (6.55) permits any value between 0 and y_i/λ. Those points are not an inconvenience of the measurement; they are the reason the subdifferential appears in the statement of the theorem.

And here is the consequence the theorem predicts:

```
method                    non-zero alpha   |alpha|_max   sum L(y_i,f_i)
  (lambda/2)||f||^2   objective
kernel ridge                          80        1.9253        10.738297
  4.251415   14.989712
kernel logistic                       80        0.8013        29.757767
  7.832128   37.589895
support vector machine                44        1.0000        17.979270
  8.508173   26.487443
```

Eighty of eighty for the two smooth losses, forty-four of eighty for the hinge. The largest coefficient of the support vector machine is exactly $1.0000 = 1/\lambda$, which is the upper bound in Eq. (6.55) attained at the misclassified points, and no such bound constrains the other two. Varying the penalty:

```
    lambda    SVM non-zeros   ridge non-zeros   logistic non-zeros
      0.10              24                80                   80
      1.00              44                80                   80
```

```
      10.00              79                 80                 80
     100.00              80                 80                 80
```

The sparsity of the support vector machine is not fixed but bought: a large penalty flattens f, more points fall inside the margin $y_i f_i \leq 1$, and more coefficients switch on. In the limit of a very heavy penalty every point is a support vector and the advantage disappears entirely. The other two are dense at every penalty, as Theorem 6.1 says they must be.

Finally, the three fitted functions are not very different:

```
method                    test accuracy   agreement with SVM
kernel ridge                     0.9025               0.9800
kernel logistic                  0.8760               0.9635
support vector machine           0.8825               1.0000
```

Three methods, three losses, one penalty, one representation, decision boundaries agreeing on the large majority of the plane – and one of them built from a little over half the data. The choice among them is a choice of what to pay for: a closed form and a single linear solve (squared error), a calibrated probability (logistic), or a sparse predictor (hinge).

Machine learning connection. Theorem 6.1 is a good example of a result that is worth more than the sum of its applications. It says that in a kernel method the loss function does not merely decide *how well* the fit tolerates an error; it decides *which observations appear in the answer at all*. A loss that is flat somewhere yields a sparse representation, and one that is not, does not. That principle generalises beyond the three cases here – ε-insensitive regression is flat in a band around zero and gives sparse support vectors for a regression problem, for exactly this reason – and it is a useful thing to know when reading a proposal for a new loss function. Ask where it is flat.

6.14 Summary and the programs

The support vector machine begins from a geometric principle rather than a statistical one: among the separating hyperplanes, choose the one standing furthest from the data. Fixing the scale so that the nearest points satisfy $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) = 1$ turns maximising the margin into minimising $\frac{1}{2}\|\boldsymbol{w}\|^2$ under linear inequality constraints, Eq. (6.8).

Lagrange duality then does something remarkable. Eliminating the primal variables produces Eq. (6.17), in which the data appear only through inner products, the number of variables is the number of samples, and the KKT conditions show that most multipliers vanish. The surviving points – the support vectors – are the only ones that matter; every other point could be deleted without changing the answer. Slack variables extend this to overlapping classes and, remarkably, change nothing in the dual but the addition of a ceiling $\lambda_i \leq C$.

Because only inner products appear, they can be replaced by a kernel, and Mercer's theorem guarantees that any symmetric positive semi-definite kernel corresponds to some feature map. We thereby separate data linearly in a space we never construct, possibly of infinite dimension, at the cost of evaluating a function of two arguments.

The dual is a quadratic programme, and rather than call a library we derived and built a solver. Sequential minimal optimisation exploits the fact that the equality constraint forces multipliers to be changed in pairs, so the smallest possible subproblem has two variables, one of them eliminated by the constraint – and a one-dimensional concave quadratic can be maximised in closed form, Eq. (6.42). The whole algorithm is analytic updates plus a selection heuristic, it never needs a matrix factorisation, and the implementation of Section 6.10 re-

produces `scikit-learn` to six significant figures in the dual objective while satisfying every KKT condition to five decimal places.

Finally, eliminating the slack variables in the primal revealed the hinge loss and placed the SVM in a family: Ridge regression, logistic regression and the support vector machine are one model with one penalty and three losses. The hinge loss is the only one of the three that is exactly zero for well-classified points, which is the entire origin of sparsity in the support vectors.

Section 6.13 turned that remark into Theorem 6.1, which holds for the kernel forms of all three methods and gives the coefficients of any of them in one line, $\alpha_i = -L_i'/\lambda$. The weight a training point carries is the derivative of the loss at that point. A loss that is flat on an open set has zero derivative there and therefore zero coefficients; the hinge is the only one of the three that is, and the support vectors follow. Measured on eighty points with the same kernel and the same penalty, the formula holds to between 10^{-13} and machine zero for all three losses, the support vector machine retains forty-four coefficients and the other two retain all eighty, and the three decision boundaries agree on more than ninety-six per cent of the plane.

The complete programs are collected in `doc/BookML/BookPrograms`:

- `three_losses.py` – kernel ridge regression, kernel logistic regression and the support vector machine fitted to the same data with the same kernel, the hinge problem solved as a box-constrained quadratic programme and certified by its duality gap, and Theorem 6.1 checked for all three.
- `svm_smo.py` – the `SVC` class of Section 6.10 with the linear, polynomial and Gaussian kernels, and the KKT verification of Section 6.11.
- `svm_examples.py` – the separable blobs, the effect of C on overlapping classes, the moons data with polynomial and RBF kernels, and the decision-boundary plots.
- `svm_pegasos.py` – the primal subgradient solver of Section 6.12, with the comparison of the primal and dual objectives.
- `losses.py` – the squared, logistic and hinge losses plotted together, reproducing the table of Section 6.12.

Each file runs as a script and reproduces the numbers quoted in this chapter.

6.15 Exercises

Warm-up exercises

1. **Geometry of the hyperplane.** (a) Show that $\boldsymbol{w}$ is orthogonal to the hyperplane $\boldsymbol{w}^T\boldsymbol{x}+b=0$. (b) Derive the signed distance (6.2). (c) Show that the distance between the two margin hyperplanes $\boldsymbol{w}^T\boldsymbol{x}+b=\pm 1$ is $2/\|\boldsymbol{w}\|$.
2. **Scale invariance.** Show that multiplying $(\boldsymbol{w},b)$ by any $\alpha>0$ leaves the decision boundary unchanged, and explain why this licenses the normalisation used to obtain Eq. (6.8).
3. **The dual, by hand.** (a) Derive Eqs. (6.14) and (6.15) from the Lagrangian (6.13). (b) Substitute back and obtain the dual (6.17), being explicit about how the term in b disappears. (c) Show that the matrix $P_{ij}=y_iy_j\boldsymbol{x}_i^T\boldsymbol{x}_j$ is positive semi-definite, and hence that the dual is convex.
4. **A two-point problem.** Take $\boldsymbol{x}_1=(0,0)$ with $y_1=-1$ and $\boldsymbol{x}_2=(2,0)$ with $y_2=+1$. (a) Write out the dual and solve it by hand for λ_1 and λ_2. (b) Recover $\boldsymbol{w}$ and b, and verify that the margin is $2/\|\boldsymbol{w}\|$ and equals the distance between the points. (c) Confirm your answer with the code of Section 6.10.

5. **The three KKT cases.** For the soft-margin problem, derive the classification of points into $\lambda_i = 0$, $0 < \lambda_i < C$ and $\lambda_i = C$ given at the end of Section 6.6, starting from Eqs. (6.27) and $\lambda_i = C - \gamma_i$.
6. **The SMO bounds.** (a) Derive Eqs. (6.38) and (6.39) by drawing the box $[0, C]^2$ and the line $\lambda_1 = \gamma - s\lambda_2$ for both signs of s. (b) Show that $\eta = 2K_{12} - K_{11} - K_{22} \le 0$ for any valid kernel, and identify the case in which it vanishes. (c) What should the algorithm do when $L = H$, and why?
7. **The kernel trick.** (a) Verify Eq. (6.31) algebraically. (b) Find the feature map corresponding to $K(\boldsymbol{x}, \boldsymbol{z}) = (\boldsymbol{x}^T \boldsymbol{z} + 1)^2$ in two dimensions, and count its dimensions. (c) Show that if K_1 and K_2 are valid kernels then so are $K_1 + K_2$ and $K_1 K_2$. (d) Show that the Gaussian kernel can be written as $\exp(-\gamma\|\boldsymbol{x}\|^2)\exp(-\gamma\|\boldsymbol{z}\|^2)\exp(2\gamma \boldsymbol{x}^T \boldsymbol{z})$ and, by expanding the last factor, argue that its feature space is infinite-dimensional.
8. **Implementing SMO (numerical).** Starting from the code of Section 6.10: (a) reproduce the KKT checks of Section 6.11; (b) replace the max-$|E_i - E_j|$ heuristic by a random choice of j and compare the number of iterations to convergence; (c) remove the alternation between sweeping all points and sweeping only the free multipliers, and report what happens to the dual objective.
9. **Hyperparameters (numerical).** On the moons data, perform a two-dimensional cross-validated grid search over C and the RBF γ. (a) Plot the validation accuracy as a heat map. (b) Plot the number of support vectors over the same grid, and comment on the relation between that number and the generalisation error. (c) Show the decision boundary for a badly chosen and a well chosen pair.
10. **The three losses.** Plot the squared, logistic and hinge losses of Section 6.12 against the margin yf on the same axes. (a) Which are zero for large positive margin? (b) Which grow fastest for large negative margin, and what does that imply about sensitivity to outliers and mislabelled points? (c) Relate your answer to the notebox on the squared error in Section 5.3.

Project-style exercise: building a support vector machine

Part a: the linear separable case.

Implement the hard-margin SVM by writing the dual and solving it with your own SMO, taking C very large. Test on two well-separated Gaussian blobs. Plot the decision boundary, both margin hyperplanes, and the support vectors circled. Verify that exactly the circled points have $\lambda_i > 0$, and that deleting any other point leaves the solution unchanged – this last check is the definition of a support vector and is worth performing explicitly.

Part b: the soft margin.

Move the blobs together until they overlap. Sweep C over several orders of magnitude and, for each, record the margin, the number of free and bound support vectors and the training accuracy. Explain the trends using the three KKT cases of Section 6.6.

Part c: kernels.

Add the polynomial and Gaussian kernels and apply them to the moons data and to a set of concentric circles. For the circles, find analytically a feature map making the problem linearly separable, and check that a polynomial kernel of the corresponding degree succeeds.

Part d: model selection.

Cross-validate over C and γ as in Section 2.12, using a proper training, validation and test split. Report the confusion matrix and the metrics of Section 5.12 on a test set used once.

Part e: comparison.

On the Wisconsin breast cancer data of Section 5.13, compare your SVM against the logistic regression of Chapter 5. Compare accuracy, AUC, the number of support vectors, and the training time. Which would you deploy, and why? Note that the SVM produces no probabilities; discuss what is lost thereby and what could be done about it.

Part f: primal against dual.

Implement Pegasos as in Section 6.12 and verify that it reaches the same objective as your dual solver on a linear problem. Then time both as the number of samples grows from 10^2 to 10^5, and as the number of features grows over the same range. Plot the two scalings and identify the regime in which each formulation should be preferred.

Chapter 7
Ensemble methods; from simple decision trees to Gradient Boosting

Every method so far has fitted a single global function to the data: a hyperplane, a sigmoid of a hyperplane, a kernel expansion. Decision trees do something different. They partition the feature space into boxes and fit a constant in each, building the model by a sequence of simple yes-or-no questions. The result is the most interpretable model in this book – a tree can be read aloud – and also, on its own, one of the least accurate.

That combination is what makes the chapter interesting. A single tree has low bias and very high variance, in the sense of Section 2.10: split the training data in two and fit a tree to each half, and the two trees may look nothing alike. The remedy is to build many trees and combine them, and the various ways of doing so – bagging, random forests, AdaBoost, gradient boosting – are the *ensemble methods* of the title. They are, on tabular data, the most accurate methods known, and they routinely outperform neural networks in that setting.

We shall build all of them from scratch, because each is short enough to write in a page and because doing so makes the mathematics concrete. Only for XGBoost, in Section 7.14, do we use a library – and even there we derive the objective it minimises.

7.1 Basics of a tree

A decision tree is a sequence of binary questions. The *root node* contains all the observations; each *internal node* tests one feature against a threshold and sends the observation left or right; each *leaf* or terminal node carries a prediction. Classifying or predicting a new point means walking from the root to a leaf, which costs one comparison per level.

Geometrically, each test $x_j \le t$ cuts the feature space with a hyperplane perpendicular to axis j. A tree therefore partitions the space into axis-aligned boxes, one per leaf, and predicts a constant within each box. This is the central limitation and the central strength: boxes cannot represent a diagonal boundary economically – a tree approximates one by a staircase – but they require no notion of distance, no scaling of the features, and they handle categorical variables and interactions without any special provision.

7.2 Regression trees

Building a regression tree involves two steps. First we split the predictor space – the set of possible values $x_1, x_2, \ldots, x_p$ – into J distinct and non-overlapping regions $R_1, R_2, \ldots, R_J$. Second, for every observation falling into region R_j we make the same prediction, namely the mean of the response values of the training observations in R_j.

In principle the regions could have any shape; we take them to be high-dimensional rectangles, or boxes, for simplicity and for ease of interpretation. The goal is to find boxes minimising

$$\sum_{j=1}^{J} \sum_{i \in R_j} \left(y_i - \bar{y}_{R_j} \right)^2, \tag{7.1}$$

where $\bar{y}_{R_j}$ is the mean response of the training observations within box j. That the optimal constant in each box is the mean is not an assumption but a consequence: minimising $\sum_i (y_i - c)^2$ over c gives $c = \bar{y}$, by the same one-line calculation that produced the sample mean in Section 2.6.

Recursive binary splitting.

It is computationally infeasible to consider every possible partition of the feature space into J boxes – the number of partitions grows combinatorially – so we proceed *top-down* and *greedily*. Top-down because we begin with all observations in a single region and successively split; greedy because at each step we make the best split available *at that step*, rather than looking ahead to a split that might enable a better one later.

We begin by selecting a predictor x_j and a cutpoint s splitting the space into

$$R_1(j,s) = \{X \mid x_j < s\} \qquad \text{and} \qquad R_2(j,s) = \{X \mid x_j \geq s\}, \tag{7.2}$$

so as to minimise

$$\sum_{i : x_i \in R_1} \left(y_i - \bar{y}_{R_1} \right)^2 + \sum_{i : x_i \in R_2} \left(y_i - \bar{y}_{R_2} \right)^2, \tag{7.3}$$

searching over all predictors $x_1, \ldots, x_p$ and, for each, over all possible values of s. In practice only $n-1$ distinct thresholds per feature need be examined – the midpoints between consecutive sorted values – since the criterion is constant between them. The cost of one split is therefore $\mathcal{O}(pn \log n)$ for the sorting, and finding the best (j,s) is quick as long as p is not enormous.

We then repeat, looking for the best predictor and cutpoint with which to split *one of the two regions just created*, giving three regions; then split one of those three; and so on, until a stopping criterion is met – commonly that no region contains more than some minimum number of observations.

7.2.1 Cost-complexity pruning

The procedure above is straightforward but leads to overfitting and to unnecessarily large trees. Grown to completion, a tree places every training point in its own leaf and has zero training error – and, by Section 2.9, has learned nothing. A smaller tree with fewer splits will have larger bias but smaller variance and better interpretability.

The standard remedy is to grow a large tree T_0 and then *prune* it back. Rather than considering every subtree, we consider a sequence indexed by a non-negative parameter α, seeking for each α the subtree $T \subset T_0$ minimising

$$\sum_{m=1}^{\overline{T}} \sum_{i : x_i \in R_m} \left(y_i - \bar{y}_{R_m} \right)^2 + \alpha \overline{T}, \tag{7.4}$$

where $\overline{T}$ is the number of terminal nodes and R_m the region belonging to the m-th of them.

Equation (7.4) should look familiar: it is a training error plus α times a complexity penalty, which is precisely the structure of Ridge regression in Eq. (3.43), with the number of leaves playing the role of $\|\boldsymbol{\theta}\|^2$. When $\alpha = 0$ the subtree is T_0 itself, since the equation then measures only training error; as α grows there is an increasing price for many terminal nodes and the minimising subtree shrinks.

It turns out that as α increases from zero, branches are pruned in a nested and predictable fashion, so the whole sequence of subtrees can be obtained cheaply. We select α by cross-validation, exactly as in Section 2.12, and then return to the full data set and take the corresponding subtree.

The regression tree procedure, in full.

1. Use recursive binary splitting to grow a large tree, stopping only when each terminal node has fewer than some minimum number of observations.
2. Apply cost-complexity pruning to obtain the sequence of best subtrees as a function of α.
3. Use K-fold cross-validation to choose α: for each fold, repeat steps 1 and 2 on all but that fold, evaluate the mean squared prediction error on the held-out fold as a function of α, and average over folds. Pick the α minimising the average.
4. Return the subtree from step 2 corresponding to the chosen α.

7.3 Classification trees

A classification tree differs from a regression tree only in what a leaf predicts and in how splits are scored. For a regression tree the prediction is the mean response in the leaf; for a classification tree it is the most commonly occurring class among the training observations in that leaf. We are often interested not only in the predicted class but in the class *proportions* within the leaf, which give a crude probability estimate.

Growing the tree proceeds by recursive binary splitting as before, but the squared error cannot be used to score a split. A natural alternative is the *misclassification rate*, the fraction of training observations in the region that do not belong to the most common class. In practice the Gini index or the entropy are preferred, because both are more sensitive to node purity, as we shall see.

Impurity measures.

Suppose the targets take K values $k = 1, \dots, K$. Define the proportion of class k among the N_m observations in region R_m,

$$p_{mk} = \frac{1}{N_m} \sum_{x_i \in R_m} I(y_i = k), \tag{7.5}$$

with I the indicator function, and let $k(m)$ denote the majority class in that region. The three common criteria are

$$\text{Misclassification error:} \quad E_m = \frac{1}{N_m} \sum_{x_i \in R_m} I\left(y_i \neq k(m)\right) = 1 - p_{m,k(m)}, \tag{7.6}$$

$$\text{Gini index:} \quad G_m = \sum_{k=1}^{K} p_{mk}\left(1 - p_{mk}\right) = 1 - \sum_{k=1}^{K} p_{mk}^2, \tag{7.7}$$

$$\text{Entropy:} \quad S_m = -\sum_{k=1}^{K} p_{mk} \log_2 p_{mk}. \tag{7.8}$$

All three vanish when a node is pure, and all are maximal when the classes are balanced.

Why Gini and entropy rather than the error rate.

The reason is worth spelling out, since the sources usually assert it. Consider two classes and write p for the proportion of the first. Then $E = \min(p, 1-p)$, which is *piecewise linear*, while $G = 2p(1-p)$ and $S = -p\log_2 p - (1-p)\log_2(1-p)$ are strictly concave. Now take a node with 400 observations of each class, and compare two candidate splits: one producing $(300, 100)$ and $(100, 300)$, the other $(200, 400)$ and $(200, 0)$. Both have misclassification rate 0.25 after the split, so the error rate cannot distinguish them. But the second produces a *pure* node, and both the Gini index and the entropy prefer it. The reason is the concavity: for a strictly concave impurity, the weighted average of the children is strictly less than the parent value unless the split is trivial, so these measures reward purity even when the majority vote does not change. Since a pure node never needs splitting again, this is the behaviour we want.

Figure 7.1 shows the three measures for a two-class node. The misclassification rate is piecewise linear with a corner at $p = \frac{1}{2}$; the Gini index and the entropy are strictly concave. Concavity is precisely the property that makes a weighted average of two children smaller than the parent unless the split is trivial, so the two smooth measures reward a split that purifies one branch even when the majority vote in neither branch changes. The entropy is drawn also at half scale, where it is seen to track the Gini index closely; in practice the two rarely select different splits.

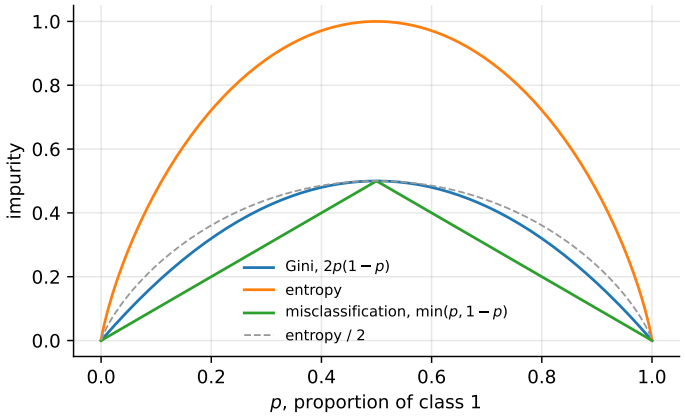


Fig. 7.1 The three impurity measures of Eqs. (7.6)-(7.8) for a node with a fraction p of one class. The misclassification rate is piecewise linear, the Gini index and entropy strictly concave.

7.4 The CART algorithm

Two algorithms dominate: CART (Classification And Regression Tree), used for both tasks and implemented in `scikit-learn`, and ID3, based on information gain and used for classification with categorical attributes.

CART for classification.

CART splits the data set in two using a single feature k and a threshold t_k, searching for the pair producing the purest subsets. The cost function it minimises is

$$C(k,t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}, \tag{7.9}$$

where $G_{\text{left/right}}$ is the impurity of the corresponding subset and $m_{\text{left/right}}$ the number of instances in it. Having split the training set in two it splits each subset by the same logic, recursively, stopping when it reaches a maximum depth or can find no split reducing impurity. Further stopping conditions are controlled by the minimum number of samples required to split a node, the minimum number in a leaf, and the maximum number of leaves.

Equivalently one maximises the *impurity decrease*, or gain,

$$\Delta = G_{\text{parent}} - \frac{m_{\text{left}}}{m} G_{\text{left}} - \frac{m_{\text{right}}}{m} G_{\text{right}}, \tag{7.10}$$

which is the form we shall implement. When the entropy is used in place of the Gini index, Δ is called the *information gain*, and choosing splits by it is the ID3 algorithm: at each node one asks which attribute best separates the training examples, selects it, creates a descendant for each of its values, sorts the examples into the descendants, and repeats. The search is greedy and never backtracks.

CART for regression.

The regression version is identical with the impurity replaced by the mean squared error,

$$C(k,t_k) = \frac{m_{\text{left}}}{m} \mathrm{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \mathrm{MSE}_{\text{right}}, \tag{7.11}$$

where for a node

$$\mathrm{MSE}_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} (y_i - \bar{y}_{\text{node}})^2, \qquad \bar{y}_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} y_i. \tag{7.12}$$

Note that Eq. (7.11) with these definitions is exactly Eq. (7.3) divided by m, so the two formulations agree. Without regularisation – a depth limit or pruning – regression trees overfit just as classification trees do.

7.5 A worked example: computing the Gini index

A classical example makes the arithmetic concrete. Based on meteorological attributes we wish to predict whether we go for a ride. The features are *outlook* (sunny, overcast, rain),

temperature (hot, mild, cool), *humidity* (high, normal) and *wind* (weak, strong); the target is whether we ride (1) or do something else (0).

Day	Outlook	Temperature	Humidity	Wind	Ride
1	Sunny	Hot	High	Weak	0
2	Sunny	Hot	High	Strong	1
3	Overcast	Hot	High	Weak	1
4	Rain	Mild	High	Weak	1
5	Rain	Cool	Normal	Weak	1
6	Rain	Cool	Normal	Strong	0
7	Overcast	Cool	Normal	Strong	1
8	Sunny	Mild	High	Weak	0
9	Sunny	Cool	Normal	Weak	1
10	Rain	Mild	Normal	Weak	1
11	Sunny	Mild	Normal	Strong	1
12	Overcast	Mild	High	Strong	1
13	Overcast	Hot	Normal	Weak	1
14	Rain	Mild	High	Strong	0

Table 7.1 The ride data set: four categorical features and a binary target.

At the root there are 10 rides and 4 non-rides out of 14 days, so $p_{m1} = 10/14$ and $p_{m0} = 4/14$, giving by Eqs. (7.7) and (7.8)

$$G_{\text{root}} = 1 - \left(\tfrac{10}{14}\right)^2 - \left(\tfrac{4}{14}\right)^2 = 0.4082, \qquad S_{\text{root}} = 0.8631. \tag{7.13}$$

Splitting on a categorical feature with several values, and scoring by the weighted average of Eq. (7.9), gives Table 7.2.

Feature	Gini after split	Entropy after split	Information gain
Outlook	**0.3429**	**0.6935**	**0.1696**
Humidity	0.3673	0.7885	0.0747
Temperature	0.4048	0.8571	0.0060
Wind	0.4048	0.8571	0.0060

Table 7.2 Impurity after splitting the root of Table 7.1 on each feature. Outlook is chosen by both criteria. The information gain is S_{root} minus the entropy after the split.

Both criteria select *outlook* as the root test, and for a visible reason: of its three values, overcast is perfectly pure – all four overcast days are rides – so that branch becomes a leaf immediately and needs no further splitting. Humidity is the next best, temperature and wind are nearly useless at the root. This is the ID3 algorithm performing its first step, and the whole tree is built by repeating the calculation within each impure branch.

7.6 Implementing a decision tree

We now write CART from scratch. The implementation handles regression and classification, the three impurity measures, and – looking ahead to Sections 7.10 and 7.12 – both a random subset of features at each split and per-sample weights.

```
import numpy as np


def gini(y, classes):
    """Eq. (7.gini)."""
    p = np.array([np.mean(y == c) for c in classes])
    return 1.0 - np.sum(p**2)


def entropy(y, classes):
    """Eq. (7.entropy)."""
    p = np.array([np.mean(y == c) for c in classes])
    p = p[p > 0]
    return -np.sum(p * np.log2(p))


def mse_impurity(y):
    """Eq. (7.nodemse)."""
    return np.mean((y - y.mean())**2) if len(y) else 0.0


class Node:
    __slots__ = ("feature", "threshold", "left", "right", "value", "n")

    def __init__(self, value=None, n=0):
        self.feature = self.threshold = self.left = self.right = None
        self.value, self.n = value, n
```

The tree itself is a recursive search for the split maximising the impurity decrease (7.10).

```
class DecisionTree:
    """CART for regression and classification.

    task       : "classification" or "regression"
    criterion  : "gini" or "entropy" (classification only)
    max_features: if set, only a random subset of features is tried at each
                  split -- this is what turns bagging into a random forest.
    """

    def __init__(self, task="classification", criterion="gini", max_depth=None,
                 min_samples_split=2, min_samples_leaf=1, max_features=None,
                 rng=None):
        self.task, self.criterion = task, criterion
        self.max_depth = np.inf if max_depth is None else max_depth
        self.min_samples_split = min_samples_split
        self.min_samples_leaf = min_samples_leaf
        self.max_features = max_features
        self.rng = np.random.default_rng() if rng is None else rng

    def _impurity(self, y, w=None):
        if self.task == "regression":
            return mse_impurity(y)
        if w is None:
            return (gini(y, self.classes_) if self.criterion == "gini"
                    else entropy(y, self.classes_))
        p = np.array([w[y == c].sum() / w.sum() for c in self.classes_])
        if self.criterion == "gini":
```

```
            return 1.0 - np.sum(p**2)
        p = p[p > 0]
        return -np.sum(p * np.log2(p))

    def _leaf_value(self, y, w=None):
        if self.task == "regression":
            return y.mean()                          # the optimal constant
        counts = ([np.sum(y == c) for c in self.classes_] if w is None
                  else [w[y == c].sum() for c in self.classes_])
        return self.classes_[int(np.argmax(counts))]      # majority vote

    def _best_split(self, X, y, w):
        n, p = X.shape
        features = np.arange(p)
        if self.max_features is not None and self.max_features < p:
            features = self.rng.choice(p, self.max_features, replace=False)

        parent = self._impurity(y, w)
        W = w.sum() if w is not None else n
        best_gain, best_j, best_t = 0.0, None, None

        for j in features:
            xs = X[:, j]
            uniq = np.unique(xs)
            if len(uniq) < 2:
                continue
            for t in (uniq[:-1] + uniq[1:]) / 2.0:        # candidate midpoints
                mask = xs <= t
                nl, nr = mask.sum(), n - mask.sum()
                if nl < self.min_samples_leaf or nr < self.min_samples_leaf:
                    continue
                if w is None:
                    wl, wr = nl, nr
                else:
                    wl, wr = w[mask].sum(), w[~mask].sum()
                    if wl <= 0 or wr <= 0:
                        continue
                child = ((wl / W) * self._impurity(y[mask],
                                                   None if w is None else w[mask])
                         + (wr / W) * self._impurity(y[~mask],
                                                     None if w is None else w[~mask]))
                gain = parent - child                     # Eq. (7.gain)
                if gain > best_gain + 1e-12:
                    best_gain, best_j, best_t = gain, j, t
        return best_gain, best_j, best_t

    def _build(self, X, y, w, depth):
        node = Node(self._leaf_value(y, w), len(y))
        if (depth >= self.max_depth or len(y) < self.min_samples_split
                or len(np.unique(y)) == 1):
            return node
        gain, j, t = self._best_split(X, y, w)
        if j is None or gain <= 0:
            return node
        mask = X[:, j] <= t
        node.feature, node.threshold = j, t
        node.left = self._build(X[mask], y[mask],
                                None if w is None else w[mask], depth + 1)
        node.right = self._build(X[~mask], y[~mask],
                                 None if w is None else w[~mask], depth + 1)
        return node

    def fit(self, X, y, sample_weight=None):
        X, y = np.asarray(X, dtype=float), np.asarray(y)
```

```
        if self.task == "classification":
            self.classes_ = np.unique(y)
        self.root_ = self._build(X, y, sample_weight, 0)
        return self

    def predict(self, X):
        X = np.asarray(X, dtype=float)
        out = []
        for x in X:
            node = self.root_
            while node.feature is not None:               # walk to a leaf
                node = node.left if x[node.feature] <= node.threshold else node.right
            out.append(node.value)
        return np.array(out)
```

Checking it.

On the moons data of Section 6.7 with 400 samples, our tree and `scikit-learn`'s agree exactly at every depth we tried:

```
depth=1   ours train=0.8300 test=0.7500 | sklearn train=0.8300 test=0.7500
depth=3   ours train=0.9100 test=0.8100 | sklearn train=0.9100 test=0.8100
depth=5   ours train=0.9467 test=0.8900 | sklearn train=0.9467 test=0.9000
depth=inf ours train=1.0000 test=0.9000 | sklearn train=1.0000 test=0.8900
```

and on a regression problem the test mean squared errors agree to the last digit at depths 2 and 4 (6031.03 and 3097.39). The small disagreements at unlimited depth come from ties between equally good splits, broken differently by the two implementations.

The last row is the important one. The fully grown tree fits the training data perfectly and generalises *worse* than the tree of depth five: the classic overfitting signature of Section 2.10, with the variance term growing without limit. Everything in the rest of this chapter is a way of controlling it.

Figure 7.2 shows the boundaries behind those numbers. At depth one the tree is a single threshold; at depth three it has the right gross shape; at depth five it is close to the truth. Grown without limit it develops narrow rectangular islands around individual training points – visibly fitting noise, and the geometric signature of the variance term. Note also that every boundary is a staircase of axis-aligned steps, since a tree can only ask questions of the form $x_j \le t$.

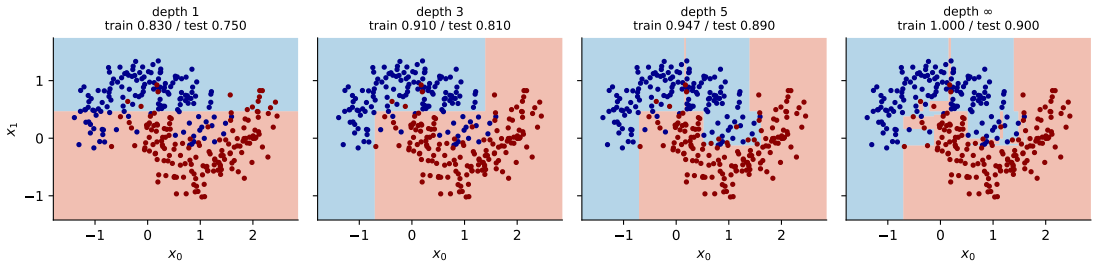


Fig. 7.2 Decision boundaries of a single tree at four depths on the moons data. Growing the tree without limit produces isolated regions around individual points. All boundaries are unions of axis-aligned rectangles.

7.7 Strengths and weaknesses of trees

In favour of trees: they are simple to understand and to interpret, and can be visualised; they require little data preparation, since no scaling or centring is needed – a monotone transformation of any feature leaves the tree unchanged, which no other method in this book can claim; the cost of prediction is logarithmic in the number of training points; they handle numerical and categorical data, and multiple outputs; and the model is a white box, so any prediction can be explained by the sequence of tests that produced it.

Against them: they readily overfit, producing trees that do not generalise, which is why pruning, depth limits and minimum leaf sizes are required; they are *unstable*, in that a small change in the data can produce a completely different tree, which is the high variance we have already seen; the greedy search gives no guarantee of the globally optimal tree, and finding that is NP-complete; they cannot express a diagonal decision boundary economically; and they are biased towards features with many possible split points, and towards the majority class in unbalanced problems.

The instability is the crucial defect and, paradoxically, the opportunity. An unstable, low-bias, high-variance predictor is exactly what averaging helps most, and the whole of the rest of this chapter follows from that observation.

7.8 Why averaging helps

Before constructing any ensemble it is worth deriving how much averaging can gain, because the answer dictates the design of every method that follows.

Averaging independent predictors.

Suppose we have B predictors $\hat{f}_1, \dots, \hat{f}_B$, each unbiased for the target with variance σ^2, and average them, $\bar{f} = \frac{1}{B}\sum_b \hat{f}_b$. If they are *independent* then by the rules of Section 2.3

$$\mathbb{E}[\bar{f}] = \mathbb{E}[\hat{f}_b], \qquad \operatorname{Var}(\bar{f}) = \frac{\sigma^2}{B}. \tag{7.14}$$

The bias is untouched and the variance falls as $1/B$. Referred to the bias-variance decomposition (2.52), averaging attacks the variance term *only* – which is precisely why it is the right treatment for deep trees, whose bias is already small, and the wrong treatment for a model that is too rigid.

The catch: correlation.

Predictors fitted to the same data set are not independent. If each pair has correlation ρ, then expanding the variance of the average,

$$\operatorname{Var}(\bar{f}) = \frac{1}{B^2}\left[\sum_b \operatorname{Var}(\hat{f}_b) + \sum_{b \neq b'} \operatorname{Cov}(\hat{f}_b, \hat{f}_{b'})\right] = \frac{1}{B^2}\left[B\sigma^2 + B(B-1)\rho\sigma^2\right], \tag{7.15}$$

which simplifies to the central formula of this chapter,

$$\boxed{\operatorname{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.} \tag{7.16}$$

Read it carefully. The second term vanishes as $B \to \infty$: adding more predictors always helps, but with diminishing returns. The first term does *not* depend on B at all. However many trees we average, the variance cannot fall below $\rho\sigma^2$.

Equation (7.16) is the design specification for everything that follows. To build a good ensemble we must make σ^2 small (accurate members), make B large (many members) and – the part that is easy to overlook – make ρ small (*diverse* members). Bagging attacks σ^2/B by resampling; random forests exist because bagging alone leaves ρ too large; boosting takes an entirely different route, attacking the bias instead.

Figure 7.3 plots Eq. (7.16) for several correlations. For independent members the variance falls as $1/B$ and can be made as small as we please. For correlated members it falls only until it reaches the floor $\rho\sigma^2$, drawn as a dotted line, and thereafter additional trees are wasted effort. With $\rho = 0.8$ – not unusual for bagged trees sharing one dominant predictor – eighty per cent of the variance survives however many trees are grown. That floor is what the random forest is designed to lower.

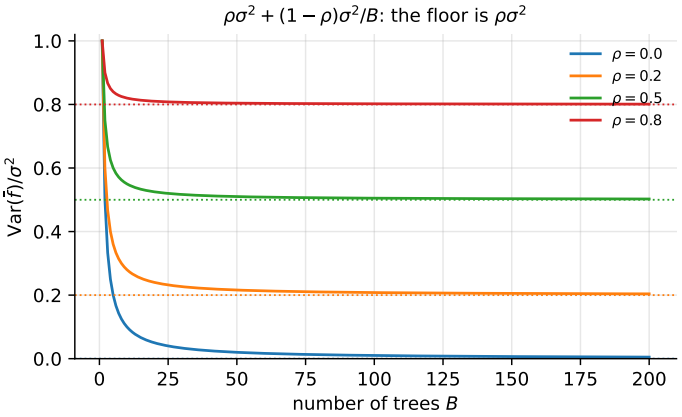


Fig. 7.3 The variance of an average of B predictors, Eq. (7.16), for four values of the pairwise correlation ρ. Dotted lines mark the floor $\rho\sigma^2$, which no number of members can cross.

An aside on voting.

For classification there is a complementary argument. Suppose B independent classifiers are each correct with probability $p > 1/2$ and we take a majority vote. The number correct is binomial, Eq. (2.11), and the probability that the majority is right tends to one as B grows – this is Condorcet's jury theorem. A simple simulation makes it vivid: repeatedly tossing a coin with $p = 0.51$ and plotting the running fraction of heads, the curve wanders at first but settles above 0.5 with near certainty after a few thousand tosses. Weak but independent voters combine into a strong one. The italicised word is again *independent*, and again it is the assumption that reality violates.

7.9 Bagging

Bootstrap aggregation, or bagging, applies Eq. (7.16) directly. We cannot draw B independent training sets – we have only one – so we manufacture them by the bootstrap of Section 2.11: draw n observations with replacement, fit a tree, repeat B times, and average the predictions, or take a majority vote for classification.

Trees are grown deep and *not* pruned. This is deliberate: deep trees have low bias and high variance, averaging removes variance and leaves bias untouched, so we want the individual members to be as unbiased as possible and let the ensemble deal with the rest.

Out-of-bag estimation.

Bagging comes with a free validation set. By the calculation in the exercises of Chapter 2, the probability that a given observation is *not* drawn in a particular bootstrap sample is

$$\left(1 - \frac{1}{n}\right)^n \longrightarrow e^{-1} \approx 0.368 \tag{7.17}$$

as n grows. About a third of the data is therefore omitted from each tree, and for each observation we may average the predictions of exactly those trees that did not see it. The resulting *out-of-bag* error is an almost unbiased estimate of the test error, obtained without any separate split or cross-validation. In our implementation the measured fraction is 0.3683 against the predicted 0.3679.

Interpretability, and what replaces it.

Bagging improves accuracy at the expense of interpretability: a collection of hundreds of trees can no longer be drawn as one. What survives is a summary of *variable importance*. For regression one records the total reduction in mean squared error due to splits on each predictor, averaged over all B trees; for classification, the total reduction in the Gini index. A large value marks an important predictor. This is a genuine loss – the importance ranking is much less informative than a readable tree, and it is known to be biased towards high-cardinality features – but it is what remains.

7.10 Random forests

Random forests improve on bagged trees by a small tweak that *decorrelates* them – that is, that attacks the $\rho\sigma^2$ floor of Eq. (7.16).

As in bagging we build many trees on bootstrapped samples. But each time a split is considered, only a random sample of m of the p predictors is offered as a candidate, and the split must use one of those m. A fresh sample is drawn at every split, and typically

$$m \approx \sqrt{p} \tag{7.18}$$

for classification, with $m \approx p/3$ often used for regression. At each split the algorithm is thus not even allowed to consider the majority of the available predictors.

Why this clever restriction works.

Suppose one predictor is very strong and several others are moderately strong. In a bagged ensemble, most or all of the trees will place the strong predictor at the top split, so the trees will look very similar to one another and their predictions will be highly correlated. By Eq. (7.16), averaging many highly correlated quantities does not reduce variance much – the $\rho\sigma^2$ term dominates. Forcing each split to ignore a random majority of the features means the strong predictor is unavailable in about $1-m/p$ of the splits, so other predictors get their turn, and the trees genuinely differ.

There is a price. Restricting the candidate set means each individual tree is worse than it could be, so σ^2 rises. The gain is a fall in ρ. Random forests work because, for deep trees, the second effect outweighs the first – and m is the hyperparameter trading them off. Taking $m = p$ recovers bagging exactly; taking $m = 1$ gives maximally decorrelated but very weak trees. Equation (7.18) is a rule of thumb, and m deserves cross-validation like any other hyperparameter.

```
class BaggingClassifier:
    """Bootstrap aggregation of unpruned trees, with out-of-bag scoring."""

    def __init__(self, n_estimators=100, max_depth=None, max_features=None,
                 rng=None):
        self.n_estimators, self.max_depth = n_estimators, max_depth
        self.max_features = max_features
        self.rng = np.random.default_rng(0) if rng is None else rng

    def fit(self, X, y):
        X, y = np.asarray(X, dtype=float), np.asarray(y)
        n = len(y)
        self.classes_ = np.unique(y)
        self.trees_, self.oob_ = [], []
        for _ in range(self.n_estimators):
            idx = self.rng.integers(0, n, n)            # bootstrap resample
            self.oob_.append(np.setdiff1d(np.arange(n), idx))
            tree = DecisionTree("classification", "gini", max_depth=self.max_depth,
                                max_features=self.max_features, rng=self.rng)
            self.trees_.append(tree.fit(X[idx], y[idx]))
        return self

    def predict(self, X):
        X = np.asarray(X, dtype=float)
        votes = np.stack([t.predict(X) for t in self.trees_])
        out = np.empty(X.shape[0], dtype=self.classes_.dtype)
        for i in range(X.shape[0]):                     # majority vote
            counts = [np.sum(votes[:, i] == c) for c in self.classes_]
            out[i] = self.classes_[int(np.argmax(counts))]
        return out

    def oob_score(self, X, y):
        """Eq. (7.oob): each point is scored by the trees that never saw it."""
        X, y = np.asarray(X, dtype=float), np.asarray(y)
        tally = np.zeros((len(y), len(self.classes_)))
        for tree, oob in zip(self.trees_, self.oob_):
            if len(oob) == 0:
                continue
            pred = tree.predict(X[oob])
            for k, c in enumerate(self.classes_):
                tally[oob[pred == c], k] += 1
        seen = tally.sum(axis=1) > 0
        vote = self.classes_[np.argmax(tally[seen], axis=1)]
        return np.mean(vote == y[seen])
```

```
class RandomForestClassifier(BaggingClassifier):
    """Bagging plus a random subset of m features at every split."""

    def __init__(self, n_estimators=100, max_depth=None, max_features="sqrt",
                 rng=None):
        super().__init__(n_estimators, max_depth, None, rng)
        self._mf = max_features

    def fit(self, X, y):
        p = np.asarray(X).shape[1]
        self.max_features = int(np.sqrt(p)) if self._mf == "sqrt" else self._mf
        return super().fit(X, y)          # Eq. (7.mtry)
```

On the moons data with 500 samples and noise 0.30, split into training and test sets, the from-scratch implementations give

```
single tree        test acc = 0.8560
bagging  (ours)    test acc = 0.8800   out-of-bag = 0.9013
bagging  (sklearn) test acc = 0.8960   out-of-bag = 0.9040
forest   (ours)    test acc = 0.8880   out-of-bag = 0.9040
forest   (sklearn) test acc = 0.8960   out-of-bag = 0.9067
```

Both ensembles improve substantially on the single tree, the forest edges out plain bagging, and our figures track `scikit-learn`'s within the noise of a 125-point test set – for which, by Eq. (2.24), the standard error of an accuracy near 0.9 is about 0.027, so none of these differences is individually significant. The out-of-bag estimates, computed on the 375 training points, are more reliable than the test figures and tell the same story.

Machine learning connection. Random forests are close to the ideal off-the-shelf method for tabular data. They need no feature scaling, are insensitive to monotone transformations, handle mixed data types, cope with missing values, come with a free error estimate via Eq. (7.17), rarely overfit as B grows – adding trees only reduces the second term in Eq. (7.16), never the bias – and have essentially two hyperparameters, of which one is "as many trees as you can afford". Their weaknesses follow from the same structure: they cannot extrapolate beyond the range of the training targets, since every prediction is an average of observed values; they produce axis-aligned boundaries; and they are large objects to store and slow to evaluate compared with a linear model.

7.11 Boosting: a bird's eye view

Bagging builds many strong learners in parallel and averages away their variance. Boosting does the opposite: it builds many *weak* learners sequentially, each correcting the errors of its predecessors, and reduces *bias*. A weak classifier is one performing only slightly better than random guessing – a tree of depth one, a *stump*, is the canonical example.

Boosting is a way of fitting an additive expansion in a set of elementary basis functions. Assume

$$f_M(x) = \sum_{m=1}^{M} \beta_m \, b(x; \gamma_m), \tag{7.19}$$

where the β_m are expansion coefficients and $b(x;\gamma_m)$ are simple functions characterised by parameters γ_m. The reader has met this structure before. Taking b to be a sigmoid gives the model of Chapter 5, with $t = \gamma_0 + \gamma_1 x$; taking b to be the coordinate functions gives linear re-

gression, where $\boldsymbol{f} = \boldsymbol{X}\boldsymbol{\theta}$ and the coefficients follow in closed form from Eq. (3.8). In boosting, b is a small decision tree and we must determine β_m and γ_m by minimising a cost function.

Minimising over all M terms jointly is intractable. The essential idea is *forward stagewise* fitting: having built f_{m-1}, we add one term,

$$f_m(x) = f_{m-1}(x) + \beta_m b(x;\gamma_m), \tag{7.20}$$

choosing (β_m, γ_m) to minimise the cost *given* f_{m-1}, which is never revisited. Everything in the rest of this chapter is a specialisation of Eq. (7.20) to a particular loss.

7.12 AdaBoost

Take binary classification with $y_i \in \{-1, 1\}$ and a classifier $G(x)$ returning one of the two values. The training error rate is

$$\overline{\mathrm{err}} = \frac{1}{n}\sum_{i=0}^{n-1} I\left(y_i \neq G(x_i)\right). \tag{7.21}$$

We shall produce a sequence of weak classifiers $G_m(x)$ applied to repeatedly modified versions of the data, and combine them as

$$G_M(x) = \operatorname{sign}\left(\sum_{m=1}^{M} \alpha_m G_m(x)\right). \tag{7.22}$$

The exponential loss.

The choice of loss which makes the algebra collapse – and which historically produced AdaBoost – is the *exponential* loss,

$$C(\boldsymbol{y}, \boldsymbol{f}) = \sum_{i=0}^{n-1} \exp\left(-y_i f(x_i)\right). \tag{7.23}$$

Note that the argument $y_i f(x_i)$ is the *margin* met in Section 6.12: positive when correct, and the loss decays exponentially with it. Inserting the stagewise form (7.20),

$$C = \sum_i \exp\left(-y_i\left[f_{m-1}(x_i) + \beta G(x_i)\right]\right) = \sum_i w_i^m \exp\left(-y_i \beta G(x_i)\right), \qquad w_i^m = e^{-y_i f_{m-1}(x_i)}, \tag{7.24}$$

where w_i^m depends on neither β nor G and therefore acts as a *weight* attached to observation i. This is the key structural fact: minimising the exponential loss stagewise is the same as fitting a weighted classifier, with weights that grow for the observations previous rounds got wrong.

Solving for G and β.

Since $y_i G(x_i) = +1$ when correct and -1 when not, split the sum,

$$C = e^{-\beta}\sum_{y_i = G(x_i)} w_i^m + e^{\beta}\sum_{y_i \neq G(x_i)} w_i^m = \left(e^{\beta} - e^{-\beta}\right)\sum_i w_i^m I\left(y_i \neq G(x_i)\right) + e^{-\beta}\sum_i w_i^m. \tag{7.25}$$

For any $\beta > 0$ the first factor is positive, so C is minimised over G by minimising the weighted error rate – the weak learner is simply fitted to the weighted data. Defining

$$\overline{\mathrm{err}}_m = \frac{\sum_i w_i^m I\left(y_i \neq G_m(x_i)\right)}{\sum_i w_i^m}, \tag{7.26}$$

and differentiating Eq. (7.25) with respect to β gives $-e^{-\beta}(1-\overline{\mathrm{err}}_m) + e^{\beta}\overline{\mathrm{err}}_m = 0$, whence

$$\beta_m = \frac{1}{2}\log\frac{1-\overline{\mathrm{err}}_m}{\overline{\mathrm{err}}_m}. \tag{7.27}$$

Updating $f_m = f_{m-1} + \beta_m G_m$ propagates into the weights through Eq. (7.24),

$$w_i^{m+1} = w_i^m \exp\left(-y_i \beta_m G_m(x_i)\right), \tag{7.28}$$

and since $-y_i G_m(x_i) = 2I(y_i \neq G_m(x_i)) - 1$, this equals $w_i^m e^{-\beta_m}\exp\left(\alpha_m I(y_i \neq G_m(x_i))\right)$ with

$$\alpha_m = 2\beta_m = \log\frac{1-\overline{\mathrm{err}}_m}{\overline{\mathrm{err}}_m}. \tag{7.29}$$

The common factor $e^{-\beta_m}$ is the same for every i and disappears on renormalising the weights, which is why the algorithm is usually stated with α_m rather than β_m.

The algorithm.

1. Initialise $w_i = 1/n$ for $i = 0, \dots, n-1$, so that $\sum_i w_i = 1$.
2. For $m = 1, \dots, M$:
   a. fit the weak classifier G_m to the training data using weights w_i;
   b. compute the weighted error (7.26);
   c. set α_m by Eq. (7.29);
   d. update $w_i \leftarrow w_i \exp\left(\alpha_m I(y_i \neq G_m(x_i))\right)$ and renormalise.
3. Output the combined classifier (7.22).

Observations misclassified at round m receive a larger weight at round $m+1$, so each new learner is forced to concentrate on the cases its predecessors found difficult. Note that $\alpha_m > 0$ exactly when $\overline{\mathrm{err}}_m < 1/2$, so a learner better than chance gets a positive vote and one worse than chance gets a negative one – it is used with its predictions inverted, which is correct rather than perverse.

```
class AdaBoost:
    """Discrete AdaBoost with decision stumps; labels must be -1 and +1."""

    def __init__(self, n_estimators=50, max_depth=1, rng=None):
        self.n_estimators, self.max_depth = n_estimators, max_depth
        self.rng = np.random.default_rng(0) if rng is None else rng

    def fit(self, X, y):
        X, y = np.asarray(X, dtype=float), np.asarray(y, dtype=float)
        n = len(y)
        w = np.full(n, 1.0 / n)                    # step 1
        self.trees_, self.alphas_ = [], []

        for _ in range(self.n_estimators):
            tree = DecisionTree("classification", "gini",
                                max_depth=self.max_depth, rng=self.rng)
```

```
            tree.fit(X, y, sample_weight=w)         # weighted weak learner
            miss = (tree.predict(X) != y).astype(float)

            err = np.sum(w * miss) / np.sum(w)      # Eq. (7.weightederr)
            err = min(max(err, 1e-10), 1 - 1e-10)
            alpha = np.log((1 - err) / err)         # Eq. (7.alpha)

            w = w * np.exp(alpha * miss)            # Eq. (7.weightupdate)
            w /= w.sum()

            self.trees_.append(tree)
            self.alphas_.append(alpha)
            if err <= 1e-10:                        # a perfect learner: stop
                break
        return self

    def decision_function(self, X):
        return np.sum([a * t.predict(X)
                       for a, t in zip(self.alphas_, self.trees_)], axis=0)

    def predict(self, X):
        return np.sign(self.decision_function(X))   # Eq. (7.adacombine)
```

On the moons data our implementation reaches a test accuracy of 0.9040 with 200 stumps, *exactly* matching `scikit-learn`'s `AdaBoostClassifier` with the same settings. Tracking the error as stumps accumulate shows the characteristic behaviour:

```
M=  1: train err=0.1813  test err=0.1760
M=  5: train err=0.1147  test err=0.1040
M= 20: train err=0.0720  test err=0.0960
M= 50: train err=0.0720  test err=0.0960
M=100: train err=0.0693  test err=0.0880
M=200: train err=0.0480  test err=0.0960
```

A single stump is a poor classifier, and a few hundred of them together are a good one – the promise of boosting, delivered. Note also that the training error keeps falling after the test error has stopped improving, so M is a hyperparameter to be cross-validated and not increased indefinitely.

Machine learning connection. Why the exponential loss? It is not merely convenient. One can show that the function minimising the *population* exponential loss is

$$f^*(x) = \tfrac{1}{2}\log\frac{p(y=1\mid x)}{p(y=-1\mid x)},$$

one half the log-odds – exactly the quantity logistic regression models in Eq. (5.10). AdaBoost is therefore estimating the same object as Chapter 5, by a completely different route, and $\sigma(2f)$ recovers a probability. The difference lies in the tails: the exponential loss grows as e^{-yf} for badly misclassified points whereas the logistic loss grows only linearly, so AdaBoost is markedly more sensitive to mislabelled data and to outliers. This is the same comparison made for the hinge loss in Section 6.12, and it is why gradient boosting with a logistic loss – the subject of the next section – has largely displaced AdaBoost in practice.

7.13 Gradient boosting

AdaBoost is tied to one loss. Gradient boosting generalises it to any differentiable loss, and the generalisation comes from an idea we have already developed at length: gradient descent, now performed in *function space*.

Functional gradient descent.

Write the cost as a sum over observations, $C(\boldsymbol{y}, \boldsymbol{f}) = \sum_i L(y_i, f(x_i))$, and regard the n numbers $f(x_i)$ as the parameters to be optimised. Gradient descent, in the sense of Eq. (4.10), would compute

$$g_m(x_i) = \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f=f_{m-1}} \tag{7.30}$$

and update $f_m = f_{m-1} - \rho_m g_m$. For the squared error $L = (y_i - f(x_i))^2$ the gradient is $g_m(x_i) = -2\,(y_i - f_{m-1}(x_i))$ – proportional to the residual.

This literal steepest descent is not useful. It optimises f at the n training points only, producing a table of values rather than a function, so it cannot generalise to a new x. The remedy is the whole idea of gradient boosting: *fit a weak learner to approximate the negative gradient*. The learner interpolates the gradient signal to the rest of the space, and adding it moves f downhill everywhere rather than at n points.

The algorithm.

1. Initialise $f_0(x)$, usually the constant minimising the loss.
2. For $m = 1, \ldots, M$:
   a. compute the negative gradient, or *pseudo-residual*, $u_{im} = -\left[\partial L(y_i, f(x_i))/\partial f(x_i)\right]_{f=f_{m-1}}$;
   b. fit a base learner h_m to the pairs (x_i, u_{im}) – a *regression* tree, whatever the original task;
   c. update $f_m(x) = f_{m-1}(x) + \nu\, h_m(x)$.
3. Return $f_M(x) = f_0(x) + \nu \sum_{m=1}^{M} h_m(x)$.

The factor $\nu \in (0, 1]$ is the *learning rate* or shrinkage, playing exactly the role it played in Chapter 4: small steps, many of them. Values around 0.1 are typical, and ν and M trade off directly against each other – halving ν roughly doubles the M required.

Squared error: boosting on residuals.

For $L = \frac{1}{2}(y - f)^2$ the pseudo-residual is

$$u_{im} = y_i - f_{m-1}(x_i), \tag{7.31}$$

the ordinary residual. Gradient boosting with squared error is therefore disarmingly simple: fit a tree to the data, compute what is left over, fit a tree to *that*, and repeat. Each tree corrects its predecessors' mistakes.

Logistic loss: boosting for classification.

For binary classification with $y \in \{0,1\}$ we model the log-odds, as in Eq. (5.10), setting $p = \sigma(f)$ and taking L to be the cross entropy (5.13). Then

$$\frac{\partial L}{\partial f} = -\left(y_i - \sigma(f(x_i))\right), \qquad \text{so} \qquad u_{im} = y_i - p_{i,m-1}, \tag{7.32}$$

using $d\sigma/dt = \sigma(1-\sigma)$ from Eq. (5.5) and the cancellation noted in the notebox of Section 5.3. Once again the pseudo-residual is target minus prediction, now on the probability scale, and once again we fit a *regression* tree to it. The natural initialisation is the constant log-odds of the training set, $f_0 = \log(\bar{y}/(1-\bar{y}))$.

```
class GradientBoostingRegressor:
    """Gradient boosting with the squared-error loss, Eq. (7.gbresidual)."""

    def __init__(self, n_estimators=100, learning_rate=0.1, max_depth=3,
                 rng=None):
        self.n_estimators, self.lr = n_estimators, learning_rate
        self.max_depth = max_depth
        self.rng = np.random.default_rng(0) if rng is None else rng

    def fit(self, X, y):
        X, y = np.asarray(X, dtype=float), np.asarray(y, dtype=float)
        self.f0_ = y.mean()                       # the optimal constant
        f = np.full(len(y), self.f0_)
        self.trees_ = []
        for _ in range(self.n_estimators):
            residual = y - f                      # negative gradient
            tree = DecisionTree("regression", max_depth=self.max_depth,
                                rng=self.rng).fit(X, residual)
            f = f + self.lr * tree.predict(X)     # shrunken update
            self.trees_.append(tree)
        return self

    def predict(self, X):
        X = np.asarray(X, dtype=float)
        f = np.full(X.shape[0], self.f0_)
        for tree in self.trees_:
            f = f + self.lr * tree.predict(X)
        return f


class GradientBoostingClassifier:
    """Binary gradient boosting on the logistic loss, Eq. (7.gblogistic).

    Labels must be 0 and 1.  The model is additive in the log-odds.
    """

    def __init__(self, n_estimators=100, learning_rate=0.1, max_depth=3,
                 rng=None):
        self.n_estimators, self.lr = n_estimators, learning_rate
        self.max_depth = max_depth
        self.rng = np.random.default_rng(0) if rng is None else rng

    def fit(self, X, y):
        X, y = np.asarray(X, dtype=float), np.asarray(y, dtype=float)
        pbar = np.clip(y.mean(), 1e-6, 1 - 1e-6)
        self.f0_ = np.log(pbar / (1 - pbar))      # constant log-odds
        f = np.full(len(y), self.f0_)
        self.trees_ = []
        for _ in range(self.n_estimators):
```

```
            p = 1.0 / (1.0 + np.exp(-f))
            residual = y - p                        # Eq. (7.gblogistic)
            tree = DecisionTree("regression", max_depth=self.max_depth,
                                rng=self.rng).fit(X, residual)
            f = f + self.lr * tree.predict(X)
            self.trees_.append(tree)
        return self

    def decision_function(self, X):
        X = np.asarray(X, dtype=float)
        f = np.full(X.shape[0], self.f0_)
        for tree in self.trees_:
            f = f + self.lr * tree.predict(X)
        return f

    def predict_proba(self, X):
        p = 1.0 / (1.0 + np.exp(-self.decision_function(X)))
        return np.column_stack([1 - p, p])

    def predict(self, X):
        return (self.decision_function(X) > 0).astype(int)
```

Both are checked against `scikit-learn`. On the moons data with 200 trees of depth two and $\nu = 0.1$ our classifier reaches a test accuracy of 0.9120 against 0.8960; on a five-feature regression problem with the same settings our test mean squared error is 932.86 against 934.03. The agreement is as close as the tie-breaking in the underlying trees permits.

Machine learning connection. Gradient boosting is where several threads of this book meet. The optimisation is that of Chapter 4, moved from parameter space to function space, with ν the learning rate and M the number of steps – so that stopping early is exactly the early stopping of Section 4.7.8, and is the main regularisation available. The loss is chosen by the argument of Section 3.6: squared error for Gaussian noise, logistic for Bernoulli targets, and any other differentiable loss one can write down, including the quantile and Huber losses for robust regression – Section 7.15 does exactly that, with the derivatives supplied by automatic differentiation. The base learner is the tree of Section 7.6. And the contrast with bagging is instructive: bagging averages independent deep trees to remove variance, boosting sums dependent shallow trees to remove bias. Because boosting reduces bias by construction, it *can* overfit as M grows, which random forests essentially cannot – so the two families need opposite habits of mind about how many trees to use.

Figure 7.4 shows both families in action. On the left, both bagging and the random forest improve rapidly on the single tree and then flatten, exactly as Eq. (7.16) requires, with the forest holding a small advantage from its lower ρ. On the right, AdaBoost's training error falls steadily towards zero as stumps accumulate while the test error flattens after a few dozen: boosting reduces bias without limit and will eventually overfit, which is precisely the behaviour bagging does not show.

7.14 XGBoost: extreme gradient boosting

XGBoost is the best known of the modern boosting libraries, and it differs from Section 7.13 in two mathematical respects worth deriving, even though here we shall use the library rather than write our own.

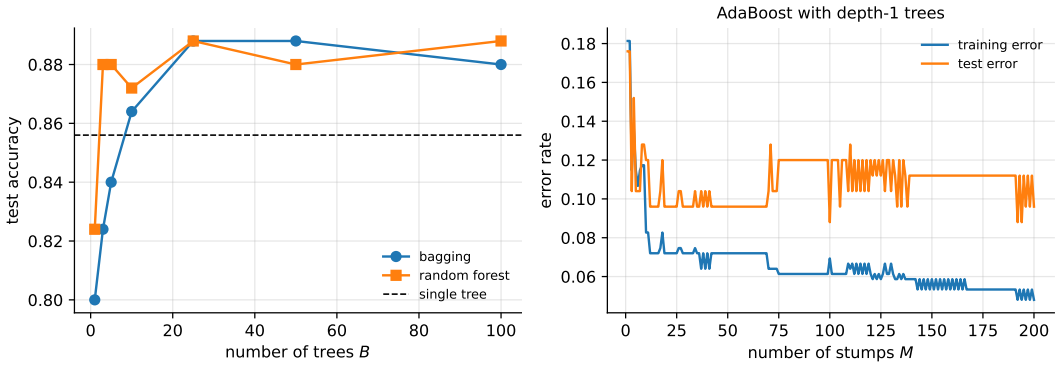


Fig. 7.4 Left: test accuracy of bagging and of a random forest against the number of trees, with the single tree marked. Right: AdaBoost training and test error against the number of stumps. The two panels contrast variance reduction with bias reduction.

A second-order objective.

Ordinary gradient boosting uses only the first derivative of the loss. XGBoost expands the loss to *second* order around the current prediction. Writing $g_i = \partial L(y_i, f)/\partial f$ and $h_i = \partial^2 L(y_i, f)/\partial f^2$ evaluated at $f_{m-1}(x_i)$, and letting the new tree contribute $f_m(x_i)$, a Taylor expansion of the kind used in Section 4.2 gives

$$C^{(m)} \simeq \sum_{i=1}^{n} \left[g_i f_m(x_i) + \tfrac{1}{2} h_i f_m(x_i)^2 \right] + \Omega(f_m), \tag{7.33}$$

where the constant $L(y_i, f_{m-1})$ has been dropped and Ω is an explicit complexity penalty on the tree,

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2, \tag{7.34}$$

with T the number of leaves and w_j the value in leaf j. This is Ridge-style regularisation, Eq. (3.43), applied to the leaf weights, plus a per-leaf cost γ which is the cost-complexity penalty of Eq. (7.4) built into the objective rather than applied afterwards.

The closed-form leaf weight.

A tree assigns a single value w_j to every point in leaf j, so writing I_j for the set of observations landing there, Eq. (7.33) becomes

$$C^{(m)} = \sum_{j=1}^{T} \left[G_j w_j + \tfrac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T, \qquad G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i. \tag{7.35}$$

This is a sum of independent one-dimensional quadratics, so each is minimised exactly, by the argument of Section 1.8.3,

$$\boxed{ w_j^* = -\frac{G_j}{H_j + \lambda}, \qquad C^* = -\frac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T. } \tag{7.36}$$

The second expression is a *score* for a tree structure – the smaller the better – and it gives a principled splitting criterion. Splitting a leaf into left and right parts changes the score by

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma. \tag{7.37}$$

Equation (7.37) replaces the Gini or MSE criterion of Section 7.4: it is derived from the loss rather than chosen, it incorporates the regularisation, and the subtraction of γ means a split whose gain does not exceed γ is simply not made – pruning becomes part of growing.

Beyond the mathematics, XGBoost owes its reputation to engineering: sparsity awareness with a default direction for missing values, an approximate split-finding algorithm using quantile sketches, cache-aware access patterns, and parallel and out-of-core computation. These make it fast rather than different, and they are the reason we use the library here.

```
import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, mean_squared_error

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

# Classification. reg_lambda is the lambda of Eq. (7.xgbpenalty) and
# gamma the per-leaf cost of Eq. (7.xgbgain).
clf = xgb.XGBClassifier(n_estimators=200, learning_rate=0.1, max_depth=3,
                        reg_lambda=1.0, gamma=0.0, subsample=0.8,
                        colsample_bytree=0.8, eval_metric="logloss")
clf.fit(X_train, y_train)
print("accuracy:", accuracy_score(y_test, clf.predict(X_test)))

# Regression, with early stopping on a validation set
reg = xgb.XGBRegressor(n_estimators=1000, learning_rate=0.05, max_depth=4,
                       early_stopping_rounds=20)
reg.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=False)
print("best iteration:", reg.best_iteration)
```

Note `subsample` and `colsample_bytree`: these apply the bootstrap of Section 7.9 and the feature subsampling of Section 7.10 *within* boosting, so that the modern gradient boosting machine borrows from the bagging family as well. The distinction drawn in this chapter between the two approaches is real mathematically and blurred in practice.

7.15 Boosting any loss: automatic differentiation

Sections 7.13 and 7.14 both needed derivatives of the loss – the pseudo-residual $u_{im} = -g_i$ of Eq. (7.30), and the pair g_i, h_i of Eq. (7.33) – and both times we derived them by hand, Eqs. (7.31) and (7.32), for the two losses we happened to want. That is exactly the situation Section 4.14 was written for, and here the payoff is unusually clean. A tree is not differentiable: its split decisions are discrete and automatic differentiation has nothing to say about them. But a boosted tree never looks at the targets directly. It looks only at g_i and h_i, and those are derivatives of a one-line function. Everything that depends on the loss can therefore be handed to JAX, and everything that depends on the tree stays exactly as it was. The result is a boosting machine in which the loss is an argument, as in Section 5.10, and which reproduces `XGBoost` to single precision.

The losses and their derivatives.

Each loss is written pointwise, as a function of one target y and one prediction f; `vmap` maps it over the data, and `grad` applied once and twice supplies g_i and h_i.

```
import numpy as np
import jax
jax.config.update("jax_enable_x64", True)
import jax.numpy as jnp
from jax import grad, vmap, jit
from jax.nn import softplus, sigmoid

# Pointwise losses L(y, f).  f is the additive model: the value itself for
# regression, the log-odds for classification, the log-rate for counts.
def squared(y, f):
    return 0.5 * (y - f)**2                                  # Eq. (7.gbresidual)

def logistic(y, f):
    return softplus(f) - y * f                               # cross entropy, f = log-odds

def huber(y, f, delta=1.0):
    r = jnp.abs(y - f)
    return jnp.where(r <= delta, 0.5 * r**2, delta * (r - 0.5 * delta))

def quantile(y, f, tau=0.9):
    r = y - f
    return jnp.maximum(tau * r, (tau - 1.0) * r)             # pinball loss

def poisson(y, f):
    return jnp.exp(f) - y * f                                # counts, f = log(rate)

def derivatives(loss):
    """g_i = dL/df and h_i = d^2L/df^2 at every data point, from JAX."""
    g = jit(vmap(grad(loss, argnums=1)))
    h = jit(vmap(grad(grad(loss, argnums=1), argnums=1)))
    return g, h
```

Two of these deserve a remark. The Huber loss is quadratic for $|y-f| \le \delta$ and linear beyond, so its gradient is the residual clipped to $\pm\delta$: a gross outlier pulls with bounded force. The quantile or pinball loss has gradient $-\tau$ for $y > f$ and $1-\tau$ for $y < f$, so its minimising constant is the τ-quantile of the targets, and its second derivative is zero everywhere: JAX returns exactly that, and the second-order method below will therefore refuse it, which is correct – Newton's method has nothing to work with on a piecewise-linear loss, and it must be boosted to first order.

The tree that sees only derivatives.

The tree of Section 7.14 is grown on the vectors $\boldsymbol{g}$ and $\boldsymbol{h}$ alone: its split criterion is the gain (7.37) and its leaf values are Eq. (7.36). It is a smaller object than the CART of Section 7.6, because there is no impurity to choose and no leaf value to vote on, and it contains ordinary gradient boosting as a special case. Set every $h_i = 1$ and $\lambda = 0$; then $H_L = n_L$, $H_R = n_R$, and the gain becomes

$$\frac{1}{2}\left[\frac{G_L^2}{n_L} + \frac{G_R^2}{n_R} - \frac{(G_L+G_R)^2}{n_L+n_R}\right] = \frac{1}{2}\left[\sum_i (u_i-\bar{u})^2 - \sum_{i\in L}(u_i-\bar{u}_L)^2 - \sum_{i\in R}(u_i-\bar{u}_R)^2\right], \tag{7.38}$$

with $u_i = -g_i$ the pseudo-residual: one half the reduction in the residual sum of squares obtained by splitting, which is precisely the criterion (7.12) of a least-squares regression tree fitted to u, and the leaf value $-G_j/H_j = \bar{u}_j$ is the leaf mean. (The identity is the analysis-of-variance decomposition of Eq. (3.20) applied within a node.) So a single tree builder serves both algorithms of this chapter: fed $h \equiv 1$ it is Friedman's base learner, fed the true second derivatives it is XGBoost's.

```
class Node:
    __slots__ = ("feature", "threshold", "left", "right", "value")
    def __init__(self, value):
        self.feature = self.threshold = self.left = self.right = None
        self.value = value

class GradientTree:
    """A regression tree grown on the derivatives g_i, h_i of the loss.

    Split criterion: the gain of Eq. (7.xgbgain); leaf value: Eq. (7.xgbweight).
    With h_i = 1 and lmbda = 0 it is a least-squares tree fitted to the
    pseudo-residual -g_i, Eq. (7.gainmse); with the true h_i it is XGBoost's tree.
    """
    def __init__(self, max_depth=3, lmbda=1.0, gamma=0.0, min_child_weight=1.0):
        self.max_depth, self.lmbda, self.gamma = max_depth, lmbda, gamma
        self.min_child_weight = min_child_weight

    def _build(self, X, g, h, depth):
        G, H = g.sum(), h.sum()
        node = Node(-G / (H + self.lmbda))                    # Eq. (7.xgbweight)
        if depth >= self.max_depth or len(g) < 2:
            return node
        best_gain, best_j, best_t = 0.0, None, None
        parent = G**2 / (H + self.lmbda)
        for j in range(X.shape[1]):
            order = np.argsort(X[:, j])
            xs, gs, hs = X[order, j], g[order], h[order]
            GL, HL = np.cumsum(gs)[:-1], np.cumsum(hs)[:-1]   # sums left of each split
            GR, HR = G - GL, H - HL
            gain = 0.5 * (GL**2 / (HL + self.lmbda)
                          + GR**2 / (HR + self.lmbda) - parent) - self.gamma
            valid = ((xs[:-1] < xs[1:]) & (HL >= self.min_child_weight)
                     & (HR >= self.min_child_weight))
            gain = np.where(valid, gain, -np.inf)             # Eq. (7.xgbgain)
            k = int(np.argmax(gain))
            if gain[k] > best_gain:
                best_gain, best_j, best_t = gain[k], j, 0.5 * (xs[k] + xs[k + 1])
        if best_j is None:
            return node
        mask = X[:, best_j] <= best_t
        node.feature, node.threshold = best_j, best_t
        node.left = self._build(X[mask], g[mask], h[mask], depth + 1)
        node.right = self._build(X[~mask], g[~mask], h[~mask], depth + 1)
        return node

    def fit(self, X, g, h):
        self.root_ = self._build(np.asarray(X, float), np.asarray(g), np.asarray(h), 0)
        return self

    def predict(self, X):
        out = []
        for x in np.asarray(X, float):
            node = self.root_
            while node.feature is not None:
                node = node.left if x[node.feature] <= node.threshold else node.right
            out.append(node.value)
```

```
        return np.array(out)
```

Note the cumulative sums: for each feature the candidate splits are scanned in sorted order and G_L, H_L are accumulated once, so the search costs $\mathcal{O}(np\log n)$ per tree rather than the $\mathcal{O}(n^2 p)$ of the naive loop in Section 7.6. This is the "exact" split finder of the libraries; their "histogram" method bins the features first and is faster still.

The boosting loop.

What remains is the loop of Section 7.13, with two additions. The initial constant f_0 is the minimiser of $\sum_i L(y_i, c)$, which for a convex loss is where the monotone function $c \mapsto \sum_i g_i(c)$ crosses zero, so a bisection finds it for every loss at once – the mean for the squared error, the log-odds of the class frequency for the logistic loss, the τ-quantile for the pinball loss – without a case distinction. And a switch selects first- or second-order boosting by deciding what the tree is fed as $\boldsymbol{h}$.

```
class Boosting:
    """Gradient boosting for any pointwise loss, with all derivatives from JAX.

    order=1: Friedman's gradient boosting -- the tree fits the negative
             gradient by least squares (h_i = 1, lmbda = 0), Section 7.gradientboosting;
    order=2: Newton boosting as in XGBoost -- the tree is grown on g_i, h_i
             with the regularised gain and leaf weights of Section 7.xgboost.
    """
    def __init__(self, loss, order=2, n_estimators=100, learning_rate=0.1,
                 max_depth=3, lmbda=1.0, gamma=0.0, min_child_weight=1.0, f0=None):
        self.loss, self.order = loss, order
        self.g, self.h = derivatives(loss)
        self.n_estimators, self.lr, self.f0 = n_estimators, learning_rate, f0
        self.tree_kw = dict(max_depth=max_depth, gamma=gamma,
                            lmbda=lmbda if order == 2 else 0.0,
                            min_child_weight=min_child_weight if order == 2 else 0.0)

    def _constant(self, y):
        """The constant minimising sum_i L(y_i, c): bisection on the monotone sum of g."""
        lo, hi = min(float(y.min()), -50.0) - 1.0, max(float(y.max()), 50.0) + 1.0
        for _ in range(100):
            mid = 0.5 * (lo + hi)
            if float(jnp.sum(self.g(y, jnp.full(len(y), mid)))) > 0:
                hi = mid
            else:
                lo = mid
        return 0.5 * (lo + hi)

    def fit(self, X, y):
        y = jnp.asarray(y, dtype=float)
        self.f0_ = self._constant(y) if self.f0 is None else self.f0
        f = jnp.full(len(y), self.f0_)
        self.trees_ = []
        for _ in range(self.n_estimators):
            g = self.g(y, f)                                  # pseudo-residual is -g
            h = self.h(y, f) if self.order == 2 else jnp.ones_like(g)
            tree = GradientTree(**self.tree_kw).fit(X, g, h)
            f = f + self.lr * jnp.asarray(tree.predict(X))    # shrunken update
            self.trees_.append(tree)
        return self

    def decision_function(self, X):
        f = np.full(np.asarray(X).shape[0], self.f0_)
        for tree in self.trees_:
```

```
            f = f + self.lr * tree.predict(X)
        return f
```

Checking it against the derivations and against the library.

First the derivatives: for the logistic loss JAX must return $g_i = p_i - y_i$ and $h_i = p_i(1-p_i)$, Eq. (7.32) and the weights of Eq. (5.18). Then the whole machine, second order, against `XGBoost` with the same hyperparameters on the moons data of Section 7.6; and finally first order, against the numbers of Section 7.13.

```
from sklearn.datasets import make_moons, make_regression
from sklearn.model_selection import train_test_split
import xgboost as xgb

g, h = derivatives(logistic)
y0, f0 = jnp.array([0., 1., 1., 0.]), jnp.array([-1., 0.3, 2., 0.1])
print("g - (p - y):", float(jnp.max(jnp.abs(g(y0, f0) - (sigmoid(f0) - y0)))))
print("h - p(1-p): ", float(jnp.max(jnp.abs(h(y0, f0) - sigmoid(f0) * (1 - sigmoid(f0))))))

X, y = make_moons(n_samples=500, noise=0.3, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

ours = Boosting(logistic, order=2, n_estimators=100, learning_rate=0.1,
                max_depth=3, lmbda=1.0, f0=0.0).fit(X_train, y_train)
lib = xgb.XGBClassifier(n_estimators=100, learning_rate=0.1, max_depth=3,
                        reg_lambda=1.0, gamma=0.0, min_child_weight=1.0,
                        base_score=0.5, tree_method="exact").fit(X_train, y_train)
p_ours = 1.0 / (1.0 + np.exp(-ours.decision_function(X_test)))
p_lib = lib.predict_proba(X_test)[:, 1]
print("max |p_ours - p_xgboost| =", np.max(np.abs(p_ours - p_lib)))
print("test accuracy: ours", np.mean((p_ours > 0.5) == y_test),
      " xgboost", np.mean((p_lib > 0.5) == y_test))

first = Boosting(logistic, order=1, n_estimators=200, learning_rate=0.1,
                 max_depth=2).fit(X_train, y_train)
print("first-order, 200 trees of depth 2: test accuracy",
      np.mean((first.decision_function(X_test) > 0) == y_test))
```

```
g - (p - y): 1.1102230246251565e-16
h - p(1-p):  2.7755575615628914e-17
max |p_ours - p_xgboost| = 1.1242810449285656e-07
test accuracy: ours 0.896  xgboost 0.896
first-order, 200 trees of depth 2: test accuracy 0.912
```

The derivatives are exact. The second-order machine agrees with `XGBoost` to 10^{-7} in every predicted probability, which is the single precision the library computes in: fifty lines of Python and the equations of Section 7.14 *are* the algorithm, and the library adds speed rather than mathematics. And the first-order machine reproduces the 0.912 of the hand-written `GradientBoostingClassifier` of Section 7.13, as Eq. (7.38) says it must.

Changing the loss.

Now the point of the exercise. A regression problem is given twenty gross outliers, and boosted under the squared error and under the Huber loss; then the same data are boosted under the pinball loss for three quantiles. Not one line of the machinery changes.

```
rng = np.random.default_rng(1)
Xr, yr = make_regression(n_samples=400, n_features=5, noise=10.0, random_state=1)
Xr_train, Xr_test, yr_train, yr_test = train_test_split(Xr, yr, random_state=1)
yr_dirty = yr_train.copy()
bad = rng.choice(len(yr_train), 20, replace=False)
yr_dirty[bad] += rng.choice([-1.0, 1.0], 20) * 500.0           # twenty gross outliers

for name, loss in [("squared error", squared),
                   ("Huber, delta=20", lambda y, f: huber(y, f, 20.0))]:
    for label, target in [("clean   ", yr_train), ("outliers", yr_dirty)]:
        model = Boosting(loss, order=1, n_estimators=300, learning_rate=0.1,
                         max_depth=3).fit(Xr_train, target)
        mse = np.mean((model.decision_function(Xr_test) - yr_test)**2)
        print(f"{name:16s} trained on {label} data: test MSE = {mse:7.1f}")

for tau in [0.1, 0.5, 0.9]:
    model = Boosting(lambda y, f: quantile(y, f, tau), order=1, n_estimators=300,
                     learning_rate=0.1, max_depth=3).fit(Xr_train, yr_train)
    below = np.mean(yr_test < model.decision_function(Xr_test))
    print(f"quantile loss, tau = {tau}: fraction of test targets below = {below:.3f}")
```

```
squared error    trained on clean    data: test MSE =   826.0
squared error    trained on outliers data: test MSE =  4237.1
Huber, delta=20  trained on clean    data: test MSE =  1012.8
Huber, delta=20  trained on outliers data: test MSE =  1053.5
quantile loss, tau = 0.1: fraction of test targets below = 0.120
quantile loss, tau = 0.5: fraction of test targets below = 0.530
quantile loss, tau = 0.9: fraction of test targets below = 0.900
```

The squared error is the better loss on clean data and is wrecked by the outliers, its test error growing fivefold; the Huber loss gives up a fifth on clean data – its clipped gradient descends more slowly for the same M – and is essentially untouched by them, exactly as the notebox of Section 3.6 said a heavy-tailed noise model should demand. The three quantile models bracket the test targets at the requested levels: boosting under the pinball loss returns a conditional quantile rather than a conditional mean, which is how prediction intervals are obtained from tree ensembles. The Poisson loss defined above, and any other differentiable loss the reader cares to write, works the same way.

Custom objectives in the libraries.

This is exactly how the libraries expose the same freedom. `XGBoost` and `LightGBM` accept a *custom objective*: a function returning $\boldsymbol{g}$ and $\boldsymbol{h}$ for the current predictions, which is all their trees need. With JAX that function is a few lines, and it lets the library's fast tree builder be driven by any loss one can write down.

```
def jax_objective(loss):
    """Turn a pointwise JAX loss into the (grad, hess) callback XGBoost accepts."""
    g, h = derivatives(loss)
    def objective(y_true, y_pred):
        y_true, y_pred = jnp.asarray(y_true, float), jnp.asarray(y_pred, float)
        return np.asarray(g(y_true, y_pred)), np.asarray(h(y_true, y_pred))
    return objective

custom = xgb.XGBRegressor(objective=jax_objective(logistic), n_estimators=100,
                          learning_rate=0.1, max_depth=3, reg_lambda=1.0,
                          base_score=0.0, tree_method="exact").fit(X_train, y_train)
p_custom = 1.0 / (1.0 + np.exp(-custom.predict(X_test)))
print("JAX objective vs built-in binary:logistic: max |dp| =",
```

```
      np.max(np.abs(p_custom - lib.predict_proba(X_test)[:, 1])))
```

```
JAX objective vs built-in binary:logistic: max |dp| = 1.1920929e-07
```

Machine learning connection. Two limits of what was done here are worth knowing. Automatic differentiation reaches every part of a boosting machine that touches the loss and none of the parts that touch the tree: there is no gradient through a split, which is why trees are trained by search and networks by descent, and why "differentiable" or *soft* trees, in which the hard threshold $x_j \le t$ is replaced by a sigmoid, are really small neural networks in disguise. And a second-order method needs $h_i > 0$: for the pinball and hinge losses JAX correctly returns zero, and one must fall back on first-order boosting or, as the libraries do, on a smoothed version of the loss. Within those limits the division of labour is exact – the loss to the differentiator, the partition to the search – and it is the same division that lets a neural network of Chapter 8 be trained with any loss from Section 5.10.

7.16 Summary and the programs

A decision tree partitions the feature space into axis-aligned boxes and predicts a constant in each. Building the optimal partition is intractable, so CART proceeds greedily and top-down, choosing at each node the feature and threshold maximising the impurity decrease (7.10) – measured by the mean squared error for regression, and by the Gini index or the entropy for classification, both preferred to the misclassification rate because their concavity rewards pure nodes. Grown without limit a tree fits the training data exactly and generalises poorly, so it must be restrained by a depth limit or by the cost-complexity pruning of Eq. (7.4), which is a training error plus a penalty on the number of leaves and is therefore Ridge regression's idea in another guise.

Trees are interpretable, need no scaling, and are invariant under monotone transformations of the features; they are also unstable, with a variance so high that a small change to the data can rebuild the tree entirely. That instability is what the rest of the chapter exploits.

Equation (7.16), $\mathrm{Var}(\bar{f}) = \rho\sigma^2 + (1-\rho)\sigma^2/B$, is the specification for every ensemble. Bagging drives the second term down by averaging trees fitted to bootstrap resamples, and pays for it with interpretability while gaining the free out-of-bag error estimate of Eq. (7.17). Random forests attack the first term, which bagging cannot touch, by offering each split only $m \approx \sqrt{p}$ of the features, so that the trees cannot all seize on the same strong predictor.

Boosting reverses the strategy. Instead of averaging strong learners to remove variance, it sums weak ones to remove bias, fitting the additive expansion (7.19) one term at a time. With the exponential loss this yields AdaBoost, whose reweighting rule (7.28) and coefficient (7.29) we derived rather than postulated, and which turns out to estimate half the log-odds – the same object as logistic regression. Replacing the exponential loss by an arbitrary differentiable one gives gradient boosting, which is gradient descent performed in function space: compute the negative gradient, fit a regression tree to it, take a shrunken step. XGBoost refines this with a second-order expansion, yielding the closed-form leaf weight and split gain of Eqs. (7.36) and (7.37), in which regularisation and pruning are built into the splitting criterion itself.

Everything in this chapter except XGBoost was implemented from scratch, and each implementation was checked against `scikit-learn`: the tree reproduces it exactly at fixed depth, AdaBoost matches its test accuracy to four decimal places, and the boosting and bagging ensembles agree within the statistical noise of the test sets involved.

The complete programs are collected in `doc/BookML/BookPrograms`:

- `decision_tree.py` – the CART implementation of Section 7.6 with the Gini, entropy and MSE criteria, sample weights, feature subsampling, and the comparison against `scikit-learn` at several depths.
- `impurity.py` – the three impurity measures plotted together, the two-split comparison showing why the error rate is inadequate, and the worked Gini and information-gain calculation of Table 7.2.
- `bagging_forest.py` – bagging and random forests with out-of-bag scoring, the empirical check of the $1/e$ law, and the variance formula (7.16) verified by simulation.
- `boosting.py` – AdaBoost and gradient boosting for regression and classification, with the error-against-M curves.
- `xgboost_examples.py` – the library examples of Section 7.14, with a numerical check of the leaf weight (7.36) against a direct minimisation.
- `boosting_jax.py` – the loss-agnostic machine of Section 7.15: pointwise losses differentiated by JAX, the `GradientTree` grown on g_i, h_i, first- and second-order `Boosting`, the match against `XGBoost`, the Huber and quantile experiments, and the JAX custom objective.

7.17 Exercises

Warm-up exercises

1. **The optimal constant in a leaf.** (a) Show that $\sum_i (y_i - c)^2$ is minimised by $c = \bar{y}$. (b) Show that $\sum_i |y_i - c|$ is minimised by the median, and explain what changes in a regression tree if the absolute error is used. (c) For classification, show that the misclassification rate in a leaf is minimised by predicting the majority class.
2. **Impurity measures.** (a) Show that the Gini index for two classes is $2p(1-p)$ and the misclassification rate $\min(p, 1-p)$; plot both and the entropy on the same axes. (b) Show that all three vanish for a pure node and are maximal at $p = 1/2$. (c) Verify the two-split comparison of Section 7.3 numerically, and explain the role of concavity.
3. **The ride data.** Reproduce Table 7.2 by hand for at least two features. Then continue the tree: having split on outlook, determine the next split within the sunny branch, and draw the resulting tree.
4. **Invariance.** (a) Show that a decision tree is unchanged if any feature is transformed by a strictly increasing function. (b) Which methods of Chapters 3 to 6 have this property? (c) What does this imply about the need for the standardisation of Section 3.13?
5. **Variance of an average.** (a) Derive Eq. (7.16) from Eq. (7.15). (b) For $\rho = 0.6$ and $\sigma^2 = 1$, how much variance remains after averaging $B = 10$, 100 and 10^6 predictors? (c) A colleague proposes to improve a random forest by increasing B from 500 to 50000. Using your answer, comment.
6. **Out-of-bag (numerical).** (a) Prove Eq. (7.17), that the probability of omission tends to $1/e$. (b) Verify it by simulation for $n = 10, 100, 1000$. (c) Implement out-of-bag scoring and compare it with 5-fold cross-validation on the same data, in both accuracy and cost.
7. **Decorrelation (numerical).** Build a data set with one very strong predictor and several moderate ones. (a) Fit a bagged ensemble and a random forest; for each, record how often the strong predictor is used at the root. (b) Estimate the correlation ρ between the predictions of pairs of trees in each ensemble. (c) Use Eq. (7.16) to predict the variance of each ensemble, and check against the measured variance. (d) Sweep m from 1 to p and find the value minimising the test error.

8. **AdaBoost by hand.** (a) Derive Eq. (7.27) by differentiating Eq. (7.25). (b) Show that $\alpha_m = 2\beta_m$ and explain why the factor $e^{-\beta_m}$ in the weight update can be dropped. (c) Show that $\alpha_m > 0$ if and only if the weak learner is better than chance, and interpret a negative α_m. (d) Run three rounds of AdaBoost by hand on a data set of five points with stumps, tabulating the weights at each round.
9. **Gradient boosting (numerical).** (a) Verify that for the squared-error loss the pseudo-residual is the ordinary residual, and that for the logistic loss it is $y_i - p_i$. (b) Implement gradient boosting with the absolute-error loss, for which the pseudo-residual is $\operatorname{sign}(y_i - f_{m-1}(x_i))$, and compare its robustness to outliers against the squared-error version. (c) Plot training and test error against M for $\nu = 0.01, 0.1, 0.5$ and confirm that ν and M trade off against each other.
10. **The XGBoost objective.** (a) Derive Eq. (7.36) by minimising Eq. (7.35) over w_j. (b) Derive the gain (7.37). (c) Show that for the squared-error loss $h_i = 1$, and that Eq. (7.36) then reduces to the mean of the residuals in the leaf shrunk by λ. Relate this to Eq. (1.130). (d) Verify Eq. (7.36) numerically by minimising Eq. (7.35) directly with a one-dimensional search.
11. **The gain reduces to the MSE criterion.** Prove Eq. (7.38) directly: with $u_i = -g_i$ and $h_i = 1$, expand $\sum_i (u_i - \bar{u})^2$ over the two children and show that the cross terms vanish. Then show that with $\lambda > 0$ the leaf value $-G_j/(n_j + \lambda)$ is the leaf mean shrunk towards zero by $n_j/(n_j + \lambda)$, and relate this to the Ridge shrinkage factor (3.48).
12. **Boosting with new losses (numerical).** Using the code of Section 7.15: (a) boost the Poisson loss on counts generated as $y_i \sim \text{Poisson}(\exp(x_{i1} - x_{i2}))$ and compare the fitted log-rate against the truth; explain why the natural initial constant is $\log \bar{y}$ and check that the bisection finds it; (b) add the exponential loss $e^{-(2y-1)f}$ and show that second-order boosting with it recovers the AdaBoost fit of Section 7.12 up to the learning rate – what plays the role of α_m? (c) run second-order boosting with the pinball loss and explain, from Eq. (7.36), what happens to the leaf weights when $h_i \equiv 0$; then replace it by a smoothed pinball loss (quadratic in a band of width ε around $r = 0$) and compare; (d) pass `jax_objective(huber)` to `XGBRegressor` and compare with the library's own `reg:pseudohubererror`.

Project-style exercise: from one tree to a jungle

Part a: your own tree.

Implement CART from scratch for both tasks, with all three impurity measures and a depth limit. Verify against `scikit-learn` at several depths. Then plot training and test error against depth and identify the overfitting point, relating your figure to the bias-variance decomposition of Section 2.10.

Part b: pruning.

Implement cost-complexity pruning, Eq. (7.4). Generate the sequence of subtrees as α increases, choose α by cross-validation, and compare the pruned tree with the best depth-limited tree from part a.

Part c: bagging and forests.

Implement bagging and then random forests. Verify the $1/e$ law and implement out-of-bag scoring. Sweep B and m, and plot the test error against each; interpret both curves using Eq. (7.16).

Part d: boosting.

Implement AdaBoost and gradient boosting. For AdaBoost, plot the weights of a few individual observations against the round number and identify the points the algorithm finds hardest. For gradient boosting, compare the squared-error and logistic losses, and show that early stopping acts as a regulariser.

Part e: comparison.

On the Wisconsin data of Section 5.13 and on the Franke data of Section 3.15, compare a single tree, a random forest, gradient boosting, XGBoost, the support vector machine of Chapter 6 and the logistic or linear regression of the earlier chapters. Report the metrics of Section 5.12, the training time, and the number of hyperparameters each required you to tune. Which method would you deploy on each problem, and why?

Part f: interpretability.

For the best ensemble, compute variable importances by the total impurity decrease and, separately, by permutation – randomly shuffling one feature and measuring the degradation. Compare the two rankings and discuss the known bias of the first towards high-cardinality features. Then compare both with the coefficients of a penalised logistic regression on the same data, and comment on what each method can and cannot tell you about which variables matter.

Part III
Deep Learning

Chapter 8
Neural Networks

Chapter 5 closed with a remark that was left deliberately unexplained: logistic regression is a neural network with no hidden layer. Its linear predictor $\boldsymbol{x}^T\boldsymbol{\theta}$ is a single artificial neuron, its sigmoid is that neuron's activation function, and its cross entropy is the loss used to train classification networks throughout deep learning. This chapter makes good on the remark by inserting layers between the input and the output, so that the features fed to the final sigmoid are themselves *learned* rather than given.

That single change buys a great deal and costs a great deal. What it buys is universality: a network with one hidden layer can approximate any continuous function to arbitrary accuracy, which none of the linear models of the previous chapters can do. What it costs is every guarantee we have relied on. The cost function is no longer convex, so Section 4.1 no longer applies and we can no longer say that the minimum we find is the minimum that exists. There is no closed form, no analogue of the normal equations, and no Karush-Kuhn-Tucker conditions to check the answer against. We are left with gradient descent and the chain rule.

Fortunately the chain rule is enough. The central result of this chapter, backpropagation, is nothing more than Eq. (1.51) applied systematically to a composition of layers, arranged so that the gradient with respect to *all* the parameters is obtained for the price of about two forward passes. It is the reverse-mode automatic differentiation of Section 4.14, specialised to the layered structure of a network and written out by hand.

8.1 Artificial neurons

An artificial neuron takes several inputs, forms a weighted sum, adds a bias and passes the result through a non-linear function. Writing the inputs as x_i, the weights as w_i and the bias as b,

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right) = f(z), \qquad z = \sum_{i=1}^{n} w_i x_i + b, \tag{8.1}$$

where f is the *activation function* and z the *activation* or pre-activation. The bias is needed for the same reason the intercept was needed in Section 3.13: without it the neuron is constrained to pass through the origin. Figure 8.1 is Eq. (8.1) drawn: the weighted sum on the left, the bias entering as an input of its own, and the non-linearity on the right.

Equation (8.1) is not new. Taking f to be the identity gives the linear model of Chapter 3; taking $f = \sigma$, the logistic function of Eq. (5.3), gives logistic regression exactly; taking f to be the step function gives the *perceptron*, historically the first machine learning model, mentioned in Section 5.1 as the hard classifier that the soft one improved upon. A neuron is a familiar object under a new name.

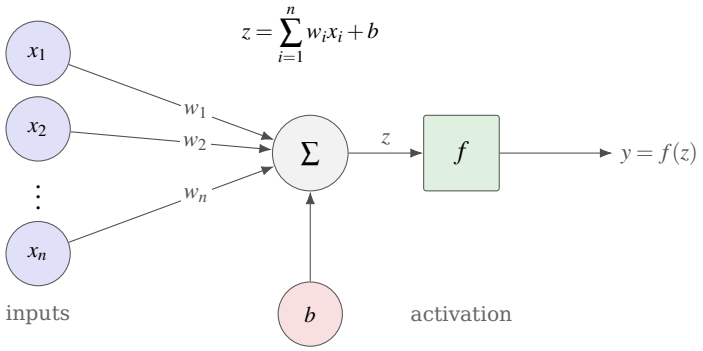


Fig. 8.1 The artificial neuron of Eq. (8.1). The inputs x_i are combined linearly with the weights w_i, the bias b is added – it is drawn as a separate input because that is what it is, an input permanently fixed at one – and the resulting pre-activation z is passed through the non-linear activation function f to give the output y. Taking f to be the identity gives linear regression, the logistic function gives logistic regression, and the step function gives Rosenblatt's perceptron.

In a network of such neurons the inputs x_i to one neuron are the *outputs* of the neurons in the preceding layer. A network is *fully connected* when each neuron receives a weighted sum of the outputs of *all* neurons in the previous layer, and *feed-forward* when the connections carry information in one direction only, from input to output, with no cycles.

8.2 Why one layer is not enough: the XOR problem

The clearest motivation for hidden layers is a problem that a single neuron provably cannot solve. Consider the classical logic gates, with two binary inputs x_1, x_2 and a binary target:

x_1	x_2	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Table 8.1 The AND, OR and XOR gates. The first two are linearly separable in the plane; the third is not.

Let us first try linear regression, using the design matrix of Eq. (1.1) with a column of ones for the intercept.

```
import numpy as np

X = np.array([[1, 0, 0], [1, 0, 1], [1, 1, 0], [1, 1, 1]], dtype=np.float64)
Xinv = np.linalg.pinv(X.T @ X)

for name, y in [("XOR", [0, 1, 1, 0]), ("OR", [0, 1, 1, 1]), ("AND", [0, 0, 0, 1])]:
    y = np.array(y, dtype=np.float64)
    theta = Xinv @ X.T @ y                      # Eq. (3.olssolution)
    print(f"{name}: theta = {np.round(theta, 3)}, "
          f"prediction = {np.round(X @ theta, 3)}")
```

The output is

```
XOR: theta = [ 0.5 -0.  -0. ], prediction = [0.5 0.5 0.5 0.5]
OR:  theta = [0.25 0.5  0.5 ], prediction = [0.25 0.75 0.75 1.25]
AND: theta = [-0.25 0.5  0.5 ], prediction = [-0.25 0.25 0.25 0.75]
```

Look at the first line. For the XOR gate the fitted slopes are *exactly zero* and the model predicts 0.5 for every input: the least-squares fit has found that the best linear function of x_1 and x_2 is the constant one. This is not a numerical accident. For XOR the two classes sit at opposite corners of the unit square, and any straight line separating $\{(0,1),(1,0)\}$ from $\{(0,0),(1,1)\}$ would have to pass on both sides of the square at once. The OR and AND gates, by contrast, *are* linearly separable, and the fitted values do at least order the outputs correctly.

Replacing the linear model by logistic regression does not help, since it changes the link function but not the linearity of the boundary:

```
from sklearn.linear_model import LogisticRegression

for name, y in [("XOR", [0, 1, 1, 0]), ("OR", [0, 1, 1, 1]), ("AND", [0, 0, 0, 1])]:
    y = np.array(y)
    logreg = LogisticRegression().fit(X, y)
    print(f"{name}: accuracy {logreg.score(X, y):.2f}")
```

```
XOR: accuracy 0.50
OR:  accuracy 0.75
AND: accuracy 0.75
```

Fifty per cent on XOR is exactly chance. The verdict is unambiguous and it is the historical one: a single layer of neurons, however trained, computes a linear decision boundary and there are elementary problems that no linear boundary solves. This observation stalled the field for the better part of two decades.

Figure 8.2 shows why, and the picture is more convincing than any amount of algebra. For AND and for OR a single straight line separates the classes, and a single neuron computes exactly such a line. For XOR the two classes occupy opposite corners of the square, and no line can be drawn with both filled squares on one side and both circles on the other. The failure is geometric, and it is complete: it is not that training was insufficient or the learning rate poorly chosen, but that the model class does not contain a solution.

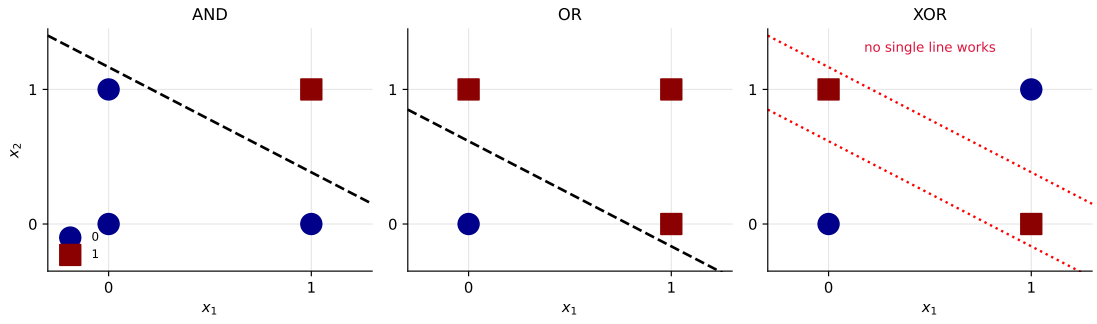


Fig. 8.2 The three gates of Table 8.1 in the plane. AND and OR are linearly separable and a single neuron suffices; XOR is not, and no straight line separates the two classes.

The remedy is a *hidden* layer. With two hidden units the network can form two intermediate features – in effect, something like "x_1 or x_2" and "not both" – and the output neuron combines them linearly. Using the implementation we develop in Section 8.13, a network with two inputs, two hidden nodes and two outputs learns all three gates perfectly:

```
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)

for name, y in [("XOR", [0, 1, 1, 0]), ("OR", [0, 1, 1, 1]), ("AND", [0, 0, 0, 1])]:
    y = np.array(y)
    net = NeuralNetwork([2, 2, 2], "sigmoid", "classification",
                        gamma=1.0, epochs=4000, batch_size=4,
                        rng=np.random.default_rng(7)).fit(X, y)
    print(f"{name}: accuracy {np.mean(net.predict(X) == y):.2f}, "
          f"predictions {net.predict(X)}")
```

```
XOR: accuracy 1.00, predictions [0 1 1 0]
OR:  accuracy 1.00, predictions [0 1 1 1]
AND: accuracy 1.00, predictions [0 0 0 1]
```

Two hidden units and a non-linearity are the whole of the difference between 0.50 and 1.00.

Machine learning connection. It is worth being precise about what the hidden layer supplies, because it is easy to say "non-linearity" and stop there. A network with a hidden layer but *linear* activations computes $\boldsymbol{W}^2(\boldsymbol{W}^1\boldsymbol{x}+\boldsymbol{b}^1)+\boldsymbol{b}^2$, which is again an affine function of $\boldsymbol{x}$ – the product of two matrices is a matrix, and no depth of linear layers escapes linearity. It is the composition of the linear map with a *non-linear* f that creates new features, and it is the reason every activation function in Section 8.6 is non-linear. Seen from Chapter 3, a network performs the basis expansion (3.4) with basis functions that are themselves fitted rather than chosen in advance.

8.3 Multilayer perceptrons and other architectures

A fully connected feed-forward network with an input layer, one or more hidden layers and an output layer, whose neurons carry non-linear activation functions, is called a *multilayer perceptron* (MLP). This is the architecture we shall analyse in detail, and it is the one for which backpropagation takes its simplest form.

Several other architectures appear later in this book, and it is useful to know what distinguishes them. *Convolutional* networks replace full connectivity by a small kernel slid across the input, so that each unit sees only a local patch and the weights are shared across positions; this encodes translation invariance and drastically reduces the parameter count, which is why they dominate image processing. *Recurrent* networks feed the output of a layer back into itself at the next time step, giving the network a memory and making it suitable for sequences; the exploding-gradient problem of Section 8.9 is particularly acute there. *Autoencoders* place a narrow bottleneck between an encoder and a decoder and are trained to reproduce their own input, which forces the bottleneck to learn a compressed representation – the non-linear counterpart of the principal component analysis of Section 1.21.

8.4 The universal approximation theorem

Why should we expect a multilayer perceptron to be able to represent anything useful? The answer is a theorem, or rather a family of theorems refined over some thirty years, and it is worth stating carefully because the loose version in circulation claims both too much and too little.

Cybenko's theorem.

The first result is due to Cybenko in 1989 [2]. Let σ be any continuous *sigmoidal* function, meaning one with the limits

$$\sigma(z) \longrightarrow \begin{cases} 1 & z \to +\infty, \\ 0 & z \to -\infty, \end{cases} \tag{8.2}$$

and let $F(\boldsymbol{x})$ be a continuous function on the unit cube in d dimensions, $\boldsymbol{x} \in [0,1]^d$. Then for every $\varepsilon > 0$ there exists a network with a single hidden layer, $f(\boldsymbol{x};\boldsymbol{\Theta})$ with $\boldsymbol{\Theta} = (\boldsymbol{W}, \boldsymbol{b})$, such that

$$|F(\boldsymbol{x}) - f(\boldsymbol{x};\boldsymbol{\Theta})| < \varepsilon \qquad \text{for all } \boldsymbol{x} \in [0,1]^d. \tag{8.3}$$

In words: *any continuous function on the unit cube in d dimensions can be approximated by a one-layer sigmoidal network to arbitrary accuracy.*

A familiar shape of argument.

Readers who have met the Stone-Weierstrass theorem for polynomial approximation, or the convergence criteria for Fourier series, will recognise the structure: a family of simple functions is shown to be dense in the space of continuous functions on a compact set. Cybenko's proof proceeds by functional analysis, showing that if the closure of the span of the network functions were not everything, a non-zero signed measure would annihilate it, and then deriving a contradiction from the sigmoidal property. The result is an existence statement of exactly the kind Stone-Weierstrass provides for polynomials, and it should be regarded with the same mixture of reassurance and scepticism.

Hornik's extension.

Hornik [3] generalised the result in two directions. First, the activation need only be non-constant and bounded, not specifically sigmoidal. Second, and more useful in a statistical setting, the approximation can be stated in mean square rather than uniformly. If the inputs are distributed with density $p(\boldsymbol{x})$ on a domain D and the target has finite second moment,

$$\mathbb{E}\left[|F(\boldsymbol{x})|^2\right] = \int_{\boldsymbol{x}\in D} |F(\boldsymbol{x})|^2 \, p(\boldsymbol{x})\, d\boldsymbol{x} < \infty, \tag{8.4}$$

then for every $\varepsilon > 0$ there is a network with

$$\mathbb{E}\left[|F(\boldsymbol{x}) - f(\boldsymbol{x};\boldsymbol{\Theta})|^2\right] = \int_{\boldsymbol{x}\in D} |F(\boldsymbol{x}) - f(\boldsymbol{x};\boldsymbol{\Theta})|^2 \, p(\boldsymbol{x})\, d\boldsymbol{x} < \varepsilon. \tag{8.5}$$

Equation (8.5) is the statement one actually wants, because it is the expected squared error of Section 2.10 rather than a worst case over a cube the data may never visit. Hornik's second point was that universality is a property of the two-layer *architecture* and not of any special feature of sigmoids.

What the condition really is.

The conditions usually quoted – non-constant, bounded, monotonically increasing and continuous – are *sufficient but not necessary*. They were convenient for the original proofs, which leaned on the boundedness. The sharp characterisation is due to Leshno, Lin, Pinkus and

Schocken [35]: a feed-forward network with one hidden layer is a universal approximator *if and only if* the activation function is not a polynomial on any interval.

This is a much better statement, and it explains a good deal. It tells us at once that ReLU qualifies, although it is unbounded and so fails Cybenko's hypothesis; that every function in Section 8.6 qualifies; and that the one activation which does *not* work is the linear one, which is a polynomial of degree one – recovering, from the general theory, the elementary observation of the notebox in Section 8.2 that a network of linear layers collapses to a single linear map.

Which functions can be approximated.

The theorems concern *continuous* functions on a compact domain. A discontinuous F cannot in general be approximated uniformly – near a jump, any continuous approximant must be wrong somewhere – although in the mean square sense of Eq. (8.5) a network may still do perfectly well, failing only on a set of small measure. Compactness matters too: nothing is claimed about behaviour outside the region where the approximation was constructed, which is the formal counterpart of the practical warning in Section 8.15 that networks extrapolate unpredictably.

What the theorems do not say.

Three qualifications are essential, and all three are routinely forgotten.

First, the conditions apply to the *hidden* layer only. The output nodes are taken to be linear, so as not to restrict the range of the output values; a sigmoid output could never represent a function taking the value 7.

Second, and most importantly, *none of the proofs gives any relation between the approximation error ε and the number of hidden nodes*, nor any bound on the magnitudes of $\boldsymbol{W}$ and $\boldsymbol{b}$. The required width may grow exponentially with the input dimension. This gap can be closed, and Section 8.4.1 closes it for the rectifier in one dimension with an explicit construction and an explicit width. Worse, the theorems are pure existence results: they assert that suitable weights exist and say nothing whatever about whether gradient descent, on a non-convex cost, will find them. A guarantee of representability is not a guarantee of learnability, and the distance between the two is where the practical difficulty of deep learning lives.

Third, universality is not by itself remarkable. Polynomials are universal on a compact interval by Stone-Weierstrass, and so are Fourier series, splines and radial basis functions; none of these is regarded as a breakthrough. What distinguishes networks is not *that* they approximate but how economically they do so in high dimensions, and in particular that adding *depth* rather than width is empirically far more efficient. There are functions representable by a deep network with a number of units polynomial in the depth which provably require exponentially many units in a shallow one. The classical theorem, which concerns a single hidden layer, says nothing about this – so the result that justifies the field's name is precisely the one the theorem does not cover. Section 8.4.3 gives a case where the separation can be proved in half a page.

Machine learning connection. A useful way to hold the theorem in mind is to compare it with what we know about polynomials from Chapter 3. Polynomials are universal too, yet Figure 3.7 showed a degree-fourteen polynomial fit failing catastrophically – not because the function was unrepresentable but because the coefficients could not be *es-*

timated from a hundred noisy points. Universality is a statement about the model class; generalisation is a statement about the model class, the data and the estimator together, and it is governed by Eq. (2.52) rather than by any approximation theorem. A network large enough to approximate anything is also large enough to overfit anything, which is why Section 8.12 exists.

8.4.1 How wide? A constructive bound

The second qualification above – that no proof relates ε to the number of hidden nodes – is a statement about *those* proofs, not about the problem itself. For a rectified activation in one dimension the question can be settled completely by an explicit construction, and it is worth doing, because the construction is three lines long and answers exactly the question the classical theorems evade.

The key observation is that a rectified network with one hidden layer is not merely dense in the continuous functions: it *is* the set of continuous piecewise linear functions, and the number of pieces can be read off the width.

Lemma 8.1 (Rectified networks are piecewise linear). *Let*

$$g(x) = c + \sum_{i=1}^{N} v_i \,\mathrm{ReLU}(w_i x + b_i), \qquad x \in \mathbb{R}, \tag{8.6}$$

be a network with one hidden layer of N rectified units and a linear output. Then g is continuous and piecewise linear with at most $N+1$ affine pieces, and every breakpoint lies in the set $\{-b_i/w_i : w_i \neq 0\}$.

Proof. A unit with $w_i = 0$ is constant and contributes no breakpoint. A unit with $w_i \neq 0$ is affine on $(-\infty, -b_i/w_i]$ and affine on $[-b_i/w_i, \infty)$, so the only point at which it fails to be affine is $t_i = -b_i/w_i$. The at most N points t_i cut the line into at most $N+1$ open intervals, and on each of them every summand of Eq. (8.6) is affine; a finite sum of affine functions is affine. Continuity is inherited from the continuity of the rectifier. $\square$

The converse is equally elementary – any continuous piecewise linear function with N breakpoints can be written in the form (8.6) – and it is what makes the following bound constructive rather than existential.

Theorem 8.2 (A width for every accuracy). *Let F be L-Lipschitz on $[a,b]$, that is $|F(x) - F(x')| \le L|x - x'|$ for all $x, x' \in [a,b]$. For every $N \ge 1$ there is a network (8.6) with N hidden rectified units such that*

$$\sup_{x \in [a,b]} |F(x) - g(x)| \;\le\; \frac{L(b-a)}{2N}. \tag{8.7}$$

The hidden weights may all be taken equal to one and the hidden biases to $-t_k$ with $t_k = a + k(b-a)/N$; only the $N+1$ output parameters depend on F.

Proof. Write $h = (b-a)/N$, $t_k = a + kh$ for $k = 0, \dots, N$ and $y_k = F(t_k)$, and let $s_k = (y_{k+1} - y_k)/h$ be the slope of the chord over $[t_k, t_{k+1}]$, with the convention $s_{-1} = 0$. Define

$$g(x) = y_0 + \sum_{k=0}^{N-1} (s_k - s_{k-1}) \,\mathrm{ReLU}(x - t_k). \tag{8.8}$$

For $x \in [t_m, t_{m+1}]$ every rectifier with $k \le m$ is active and every rectifier with $k > m$ is not, so on that interval g is affine with slope $\sum_{k=0}^{m} (s_k - s_{k-1}) = s_m$, the sum telescoping. Since $g(t_0) = y_0$

and the slope on $[t_k, t_{k+1}]$ is s_k, induction gives $g(t_k) = y_k$ for every k: the network of Eq. (8.8) is exactly the piecewise linear interpolant of F at the $N+1$ knots.

It remains to bound the interpolation error. Fix $x \in [t_m, t_{m+1}]$ and write $x = t_m + \theta h$ with $\theta \in [0,1]$, so that $g(x) = (1-\theta)y_m + \theta y_{m+1}$ and

$$F(x) - g(x) = (1-\theta)\,(F(x) - F(t_m)) + \theta\,(F(x) - F(t_{m+1}))\,. \tag{8.9}$$

The Lipschitz property gives $|F(x) - F(t_m)| \le L\theta h$ and $|F(x) - F(t_{m+1})| \le L(1-\theta)h$, whence

$$|F(x) - g(x)| \le 2\theta(1-\theta)Lh \le \frac{Lh}{2}, \tag{8.10}$$

the last step because $\theta(1-\theta) \le 1/4$ on $[0,1]$. □

Equation (8.7) is the missing relation. Inverting it, accuracy ε is reached with

$$N = \left\lceil \frac{L(b-a)}{2\varepsilon} \right\rceil \tag{8.11}$$

hidden units – an explicit, finite and entirely unmysterious width. Nothing in the proof is an existence argument: Eq. (8.8) writes the weights down, and Fig. 8.4(a) plots the resulting network for $N = 4$ and $N = 16$.

The constant cannot be improved. Take $F(x) = \operatorname{dist}(x, h\mathbb{Z})$, the triangular wave of period h, which is 1-Lipschitz. It vanishes at every knot $t_k = kh$, so the interpolant of Eq. (8.8) is identically zero, while $\sup|F| = h/2$. The bound (8.7) is therefore attained exactly, and the factor two in the denominator is the best possible for the class of L-Lipschitz functions.

Both statements can be checked numerically. The program `universal.py` builds the network of Eq. (8.8) – it trains nothing – and measures its error on a grid of 200001 points:

```
=== 1. the explicit construction of Theorem 8.relurate ===
  target F(x) = sin(2 pi x) on [0,1], Lipschitz constant L = 2 pi
     N    sup error   bound L/2N   ratio
     2    1.000e+00    1.571e+00   0.6366
     8    7.038e-02    3.927e-01   0.1792
    32    4.792e-03    9.817e-02   0.0488
   128    3.011e-04    2.454e-02   0.0123
   512    1.882e-05    6.136e-03   0.0031
```

```
     N    sup error   bound L/2N   ratio
     2    2.500e-01    2.500e-01   1.0000
     8    6.250e-02    6.250e-02   1.0000
    32    1.562e-02    1.562e-02   1.0000
   128    3.905e-03    3.906e-03   0.9997
   512    9.763e-04    9.766e-04   0.9997
```

The second table is the worst case of the notebox: the ratio is one to every figure the grid can resolve, and the small shortfall at large N is the resolution of the grid, not a failure of the theory. The first table says something else worth noticing. For a *smooth* target the error falls like N^{-2}, not like N^{-1}: that is the classical estimate $\|F - g\|_\infty \le h^2\|F''\|_\infty/8$ for piecewise linear interpolation, and it is the first sign that the Lipschitz class is a pessimistic yardstick. Sharper rates of this kind, for deep rectified networks and for targets with s derivatives, are the subject of [36]; the whole approximation-theoretic picture is surveyed in [37].

8.4.2 The curse of dimensionality

Theorem 8.2 is one-dimensional, and the dimension is where the difficulty lies. Repeating the construction on $[0,1]^d$ means covering the cube with a grid of side h and interpolating on it, which takes of the order of h^{-d} pieces; to reach accuracy ε for an L-Lipschitz target one therefore needs of the order of

$$N \sim \left(\frac{L\sqrt{d}}{2\varepsilon} \right)^{d} \tag{8.12}$$

units. With $L = 1$ and $\varepsilon = 10^{-1}$ this is five units in one dimension, about 10^3 in three, and about 10^{12} in ten. The exponent is the input dimension, and no cleverness in the construction removes it.

Nor is this an artefact of the particular construction. The class of L-Lipschitz functions on $[0,1]^d$ contains, for every grid of side h, a family of $2^{h^{-d}}$ functions obtained by placing a bump of height $Lh/2$ on each cell with an independent sign; any two of them differ by $Lh/2$ somewhere, so distinguishing them requires at least h^{-d} bits and hence at least that many parameters. Made precise – the parameters must be chosen by a map depending continuously on the target – this is the theorem of DeVore, Howard and Micchelli [38]: no approximation scheme with N parameters achieves error better than $c_d L N^{-1/d}$ uniformly over the Lipschitz ball. A network with one hidden layer of width N has $N(d+2)+1$ parameters, so it is covered by the theorem, and the exponent in Eq. (8.12) is sharp.

The way out is not a better architecture but a smaller class of targets. The classical result of this kind is Barron's [39].

Theorem 8.3 (Barron). *Let $F : \mathbb{R}^d \to \mathbb{R}$ have a Fourier representation $F(\boldsymbol{x}) = \int e^{i\boldsymbol{\omega}\cdot\boldsymbol{x}} \hat{F}(\boldsymbol{\omega})\, d\boldsymbol{\omega}$ with finite first moment,*

$$C_F = \int \|\boldsymbol{\omega}\| \, \big|\hat{F}(\boldsymbol{\omega})\big| \, d\boldsymbol{\omega} < \infty. \tag{8.13}$$

Then for every N and every probability measure μ supported in the ball of radius r there is a network with one hidden layer of N sigmoidal units for which

$$\int |F(\boldsymbol{x}) - g(\boldsymbol{x})|^2 \, \mu(d\boldsymbol{x}) \;\le\; \frac{(2rC_F)^2}{N}. \tag{8.14}$$

The rate N^{-1} in the squared error contains no d at all. The mechanism deserves a sentence, because it explains where the dimension went. The Fourier representation exhibits $F - F(\mathbf{0})$ as a member of the closed convex hull of a set $\mathcal{G}$ of individual ridge functions of norm at most $2rC_F$, so F is a limit of *averages* of elements of $\mathcal{G}$; and if N elements are drawn independently from the mixing distribution, the mean squared error of their average is the variance of a single draw divided by N, which is at most $(2rC_F)^2/N$. Some particular draw must be at least as good as the average, so a network of width N with that error exists. This averaging argument is due to Maurey. The dimension enters only through C_F, and for the functions one actually wants to fit – superpositions of a moderate number of ridge directions, which is precisely what a network is – C_F does not grow with d.

We can watch this happen. The target $F(\boldsymbol{x}) = M^{-1}\sum_{m} c_m \cos(\kappa\, \boldsymbol{a}_m \cdot \boldsymbol{x})$ with $M = 12$ unit ridge directions has a Fourier transform consisting of point masses at $\pm\kappa\boldsymbol{a}_m$, so its Barron norm is known exactly, $C_F = \kappa M^{-1}\sum_m |c_m|$, and it does not depend on d; drawing $\boldsymbol{x}$ uniformly from the unit ball fixes $r = 1$ in every dimension as well. Theorem 8.3 therefore predicts the *same* bound in $d = 1$, 5 and 20:

```
kappa = 8:  C_F = 6.9124
    N       d = 1       d = 5       d = 20   bound (2 C_F)^2/N
    4   1.296e-02   2.689e-02   3.186e-02           4.778e+01
```

```
    16   2.330e-04   6.902e-03   2.202e-03           1.195e+01
    64   2.693e-04   1.247e-03   6.183e-04           2.986e+00
   128   1.850e-04   7.520e-04   6.779e-04           1.493e+00
measured slopes of log(mse) against log(N):   -0.77  -1.08  -1.14   (Theorem 8.barron: -1)
ratio of the d = 20 error to the d = 1 error at N = 128:  3.7
```

Three things are worth reading off. The bound holds, and with an enormous margin – it is a worst-case bound over the whole Barron class, and this particular target is far from the worst. The measured decay is close to the predicted N^{-1} in every dimension. And the error at $d = 20$ exceeds the error at $d = 1$ by a factor of under four, a constant, not by anything resembling the exponential of Eq. (8.12). It is the dimension in the *exponent* that the Barron class removes; a moderate constant remains. Set against Eq. (8.12), where reaching a fixed accuracy at $d = 20$ would take more units than there are atoms in the room, this is the whole difference between a usable method and an impossible one.

8.4.3 Why depth? An exact separation

The third qualification – that the classical theorem concerns one hidden layer and so says nothing about depth – can also be repaired, and in one dimension the repair is completely elementary. We exhibit a function that a deep network of $2k$ units represents *exactly*, and that no shallow network with fewer than $2^k - 1$ units can approximate at all.

Definition 8.4 (The tent map and the sawtooth). Let

$$T(x) = 2\,\mathrm{ReLU}(x) - 4\,\mathrm{ReLU}\big(x - \tfrac{1}{2}\big), \tag{8.15}$$

so that $T(x) = 2x$ on $[0, \frac{1}{2}]$ and $T(x) = 2 - 2x$ on $[\frac{1}{2}, 1]$, and let $s_k = T \circ T \circ \cdots \circ T$ be the k-fold composition.

Equation (8.15) is one hidden layer of *two* rectified units followed by a linear combination. Composing it k times is therefore a network with k hidden layers of two units each: $2k$ units and $6k$ parameters in all.

Lemma 8.5 (The sawtooth). *s_k maps $[0,1]$ onto $[0,1]$, is piecewise linear with exactly 2^k affine pieces of slope $\pm 2^k$, and satisfies*

$$s_k\left(\frac{j}{2^k}\right) = \begin{cases} 0 & j \text{ even}, \\ 1 & j \text{ odd}, \end{cases} \qquad j = 0, 1, \ldots, 2^k. \tag{8.16}$$

Proof. Induction on k. For $k = 1$ the statement is Eq. (8.15). Assume it for k and write $s_{k+1} = s_k \circ T$. The map T carries $[0, \frac{1}{2}]$ affinely onto $[0,1]$ and carries $[\frac{1}{2}, 1]$ affinely onto $[0,1]$ with the orientation reversed. Hence s_{k+1} restricted to $[0, \frac{1}{2}]$ is a copy of s_k compressed by a factor two, and restricted to $[\frac{1}{2}, 1]$ is a reflected copy: $2^k + 2^k = 2^{k+1}$ pieces in all, each of slope $\pm 2 \cdot 2^k$. The points $j/2^{k+1}$ are the preimages of the points $j/2^k$, and the alternation of the values is preserved by both branches. □

Theorem 8.6 (Depth against width). *Let g be a network with a single hidden layer of N rectified units, as in Eq. (8.6). If $N \le 2^k - 2$ then*

$$\sup_{x \in [0,1]} |g(x) - s_k(x)| \;\ge\; \frac{1}{2}. \tag{8.17}$$

Proof. Suppose on the contrary that $\|g - s_k\|_\infty < \frac{1}{2}$, and put $x_j = j/2^k$ for $j = 0,\dots,2^k$. By Eq. (8.16) we then have $g(x_j) < \frac{1}{2}$ for even j and $g(x_j) > \frac{1}{2}$ for odd j, so the continuous function $g - \frac{1}{2}$ changes sign on each of the 2^k intervals (x_j, x_{j+1}) and has a zero z_j in each. These zeros are distinct and ordered, $z_0 < z_1 < \cdots < z_{2^k-1}$.

No two consecutive zeros can lie in the same affine piece of g. For if z_j and z_{j+1} both lay in a piece P, then $g - \frac{1}{2}$ would be affine on P with two distinct zeros and hence identically zero there; but $x_{j+1} \in (z_j, z_{j+1}) \subset P$, so $g(x_{j+1}) = \frac{1}{2}$, contradicting the strict inequality above. The 2^k zeros therefore lie in 2^k distinct pieces, so g has at least 2^k pieces, and Lemma 8.1 gives $N + 1 \geq 2^k$. This contradicts $N \leq 2^k - 2$. □

The separation is exponential and it is exact. A network of depth k with two units per layer – $2k$ units, growing *linearly* in k – reproduces s_k with no error whatever, while any network with a single hidden layer needs at least $2^k - 1$ units, growing *exponentially*, merely to come within a half. At $k = 10$ the deep network has twenty units and the shallow one needs more than a thousand.

Both halves of the statement can be measured. Composing the tent map and counting the affine pieces of the result confirms Lemma 8.5:

```
   k   units in the deep net   linear pieces measured   2^k
   1                        2                        2     2
   4                        8                       16    16
   8                       16                      256   256
```

Fitting shallow networks of increasing width to s_3, which has eight pieces and therefore requires $N \geq 7$, produces the wall the theorem predicts:

```
k = 3, s_k has 8 pieces, the theorem needs N >= 7
   N   sup error   pieces of the fit   N+1
   2      0.5002                   2     3
   4      0.5212                   2     5
   8      0.5506                   6     9
  16      0.0096                  14    17
  32      0.0067                  25    33
  64      0.0031                  54    65
```

The number of pieces never exceeds the $N + 1$ of Lemma 8.1, and whenever it falls below eight the error sits at or above the $1/2$ of Theorem 8.6 – including at $N = 8$, where the width is sufficient in principle but training produced a network with only six live pieces and the theorem then applies to *that* network. The whole picture is Fig. 8.3.

Theorem 8.6 is the simplest member of a family. Telgarsky proved a version separating networks of any fixed depth from networks of slightly smaller depth [40], and Montúfar and co-workers counted the linear regions of a general rectified network [41], finding a number that grows exponentially in the depth but only polynomially in the width. The mechanism is always the one in Lemma 8.5: composition *multiplies* the number of pieces, addition merely adds them. This is the result the name of the field refers to, and it is the one Cybenko's statement cannot see.

8.4.4 Representability is not learnability

Every result so far is a statement about which functions a network *contains*. Training is a different question, and the gap between the two is not a technicality. The construction of Theorem 8.2 places the kinks at $t_k = a + kh$ and computes the output weights directly from F; gradient descent has to find both from a random start, on a cost function that Section 4.1

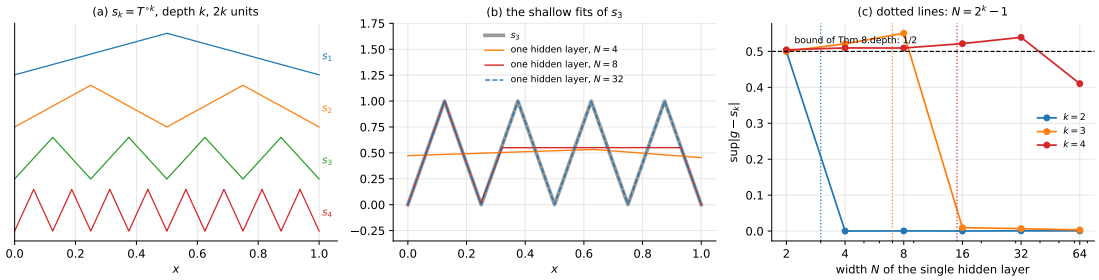


Fig. 8.3 Depth against width. (a) The sawtooth s_k of Definition 8.4, computed exactly by a rectified network of depth k with two units per layer; the number of teeth doubles with each layer while the number of units grows by two. (b) Single-hidden-layer fits of s_3: at $N = 4$ and $N = 8$ the network is flat at about $1/2$, which is the best a function with too few pieces can do, and at $N = 32$ it reproduces s_3 exactly. (c) The measured supremum error against the width, with the threshold $N = 2^k - 1$ of Theorem 8.6 marked by the dotted vertical lines and the guaranteed error $1/2$ by the horizontal one.

tells us is not convex. Running the two side by side at identical widths, on the same target $\sin 2\pi x$:

N	constructed	default init	kinks spread	ratio
2	1.000e+00	1.000e+00	9.170e-01	0.9
4	2.105e-01	1.108e+00	9.170e-01	4.4
8	7.038e-02	1.837e-01	8.465e-02	1.2
16	1.885e-02	7.203e-02	5.557e-02	2.9
32	4.792e-03	4.279e-02	1.660e-02	3.5
64	1.203e-03	4.386e-02	6.359e-03	5.3

Two things are visible. The first is that the default initialisation is a handicap in its own right: PyTorch draws w_i and b_i from $\mathcal{U}(-1,1)$, so the kink $-b_i/w_i$ falls inside the interval of interest for only a minority of the units, and the rest are dead from the start, contributing nothing and receiving no gradient. Spreading the kinks uniformly over $[0,1]$ – which is exactly where Eq. (8.8) puts them – improves matters at every width, and is a concrete instance of the general point of Section 8.10.

The second is that even with the kinks well placed, training does not reach the constructed network, and the shortfall grows with the width: a factor 1.2 at $N = 8$ and 5.3 at $N = 64$. The exception at $N = 4$, where every seed lands in the same poor local minimum, is a reminder that these are non-convex problems and that the failure need not be monotone. The theorem guarantees that a set of weights exists; it says nothing about whether the trajectory of Adam passes anywhere near them. This is the honest reading of the universal approximation theorem. It removes representability as an excuse and leaves optimisation and generalisation as the two real problems, which are the subjects of Chapter 4 and of Section 8.12 respectively. Figure 8.4 collects the three measurements.

8.5 Notation and the feed-forward pass

We now fix notation, and it repays care: almost every difficulty students have with backpropagation is a difficulty with indices rather than with calculus.

Layers are numbered $l = 0, 1, \dots, L$, with $l = 0$ the input layer, $l = L$ the output layer, and M_l the number of nodes in layer l. The *activation* of node j in layer l is

$$z_j^l = \sum_{i=1}^{M_{l-1}} w_{ij}^l a_i^{l-1} + b_j^l, \tag{8.18}$$

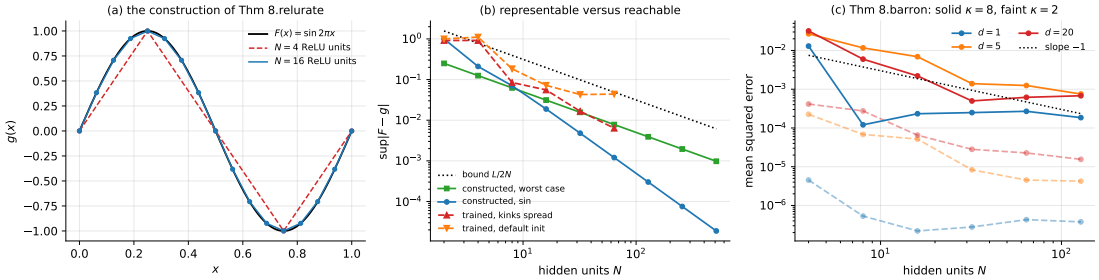


Fig. 8.4 The quantitative content of Section 8.4. (a) The network of Eq. (8.8), whose weights are written down rather than trained, for $N = 4$ and $N = 16$ hidden rectified units; the markers are the knots t_k at which it interpolates. (b) Supremum error against width: the bound (8.7), the constructed network for a smooth and for a worst-case Lipschitz target, and what Adam achieves at the same widths from two initialisations. The gap between the solid and the dashed curves is the distance between representability and learnability. (c) Barron's rate (8.14) measured in $d = 1$, 5 and 20 for two values of the Barron norm; the slope is the predicted -1 and it does not change with the dimension.

where w_{ij}^l is the weight connecting node i of layer $l-1$ to node j of layer l, and b_j^l is the bias of node j in layer l. The *output* of the node is obtained by applying the activation function,

$$a_j^l = f\left(z_j^l\right), \tag{8.19}$$

with $a_i^0 = x_i$ the network inputs. In matrix-vector form, Eqs. (8.18) and (8.19) become

$$\boxed{\boldsymbol{z}^l = \left(\boldsymbol{W}^l\right)^T \boldsymbol{a}^{l-1} + \boldsymbol{b}^l, \qquad \boldsymbol{a}^l = f\left(\boldsymbol{z}^l\right),} \tag{8.20}$$

with f applied element-wise. Evaluating Eq. (8.20) for $l = 1, \dots, L$ is the *feed-forward pass*, and the network output is $\tilde{\boldsymbol{y}} = \boldsymbol{a}^L$.

Figure 8.5 shows the whole object with these labels attached, and Figure 8.6 shows a single node of it, which is where the index convention has to be learned. The point to fix in mind is that the superscript on w_{ij}^l names the layer the weight *arrives* at, that the first subscript names the node it *leaves* and the second the node it arrives at, and that the sum in Eq. (8.18) is therefore over the first index. Every other convention in circulation is a permutation of this one, and the only thing that matters is to choose one and never deviate from it.

The cost of one forward pass is dominated by the matrix products: layer l requires $M_{l-1}M_l$ multiplications, so the whole pass costs $\mathcal{O}(\sum_l M_{l-1}M_l)$, which is also the number of weights. This is why the operations of Chapter 1 matter here: a network is, computationally, a sequence of level-3 BLAS calls interleaved with element-wise non-linearities, and Section 1.5 explains why one must let the library perform them.

Batches.

In practice one propagates a whole minibatch at once. Stacking n samples as rows of $\boldsymbol{A}^{l-1} \in \mathbb{R}^{n \times M_{l-1}}$, the forward pass reads

$$\boldsymbol{Z}^l = \boldsymbol{A}^{l-1}\boldsymbol{W}^l + \boldsymbol{b}^l, \qquad \boldsymbol{A}^l = f\left(\boldsymbol{Z}^l\right), \tag{8.21}$$

with the bias added by broadcasting. This is the form the code of Section 8.13 uses, and note that it transposes the convention of Eq. (8.20): with samples in rows, $\boldsymbol{W}^l$ appears untrans-

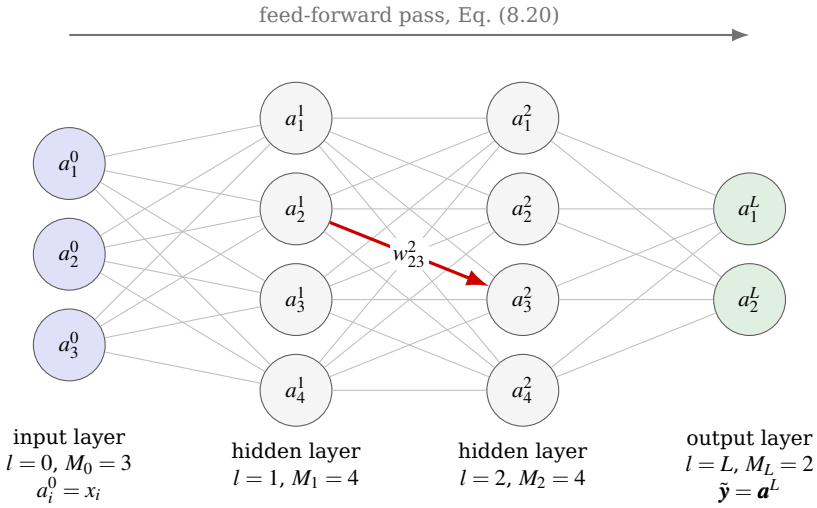
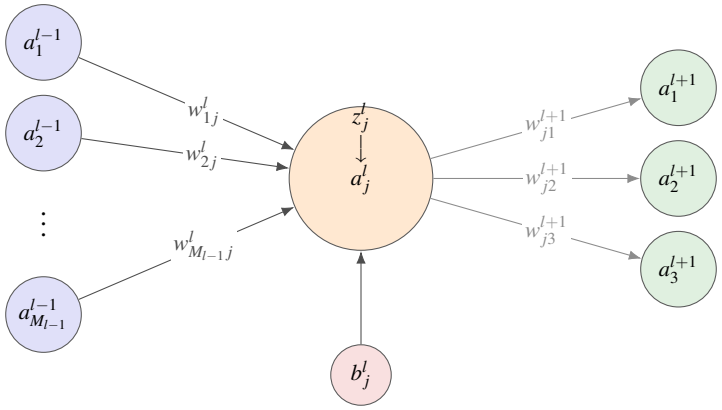


Fig. 8.5 A fully connected feed-forward network in the notation of Section 8.5, here with $L = 3$. Layer l has M_l nodes and node j of that layer carries the output a_j^l of Eq. (8.19); the input layer holds $a_i^0 = x_i$ and the output layer the prediction $\tilde{\boldsymbol{y}} = \boldsymbol{a}^L$. Every node of one layer is connected to every node of the next, and the highlighted edge carries the weight w_{23}^2, from node $i = 2$ of layer $l - 1 = 1$ to node $j = 3$ of layer $l = 2$: the superscript names the *receiving* layer and the first index names the *sending* node.



$$z_j^l = \sum_{i=1}^{M_{l-1}} w_{ij}^l a_i^{l-1} + b_j^l, \qquad a_j^l = f\left(z_j^l\right)$$

Fig. 8.6 The index convention of Eqs. (8.18) and (8.19), seen from a single node. Node j of layer l collects the outputs a_i^{l-1} of all M_{l-1} nodes of the previous layer, weighted by w_{ij}^l, adds its own bias b_j^l to form the pre-activation z_j^l, and applies f to obtain a_j^l, which it then sends on to every node of layer $l+1$ through the weights w_{jk}^{l+1}. The first index of a weight always refers to the layer below and the second to the layer above, so that the sum in Eq. (8.18) runs over the first index and the sum in the backpropagation equation (8.64) runs over the second.

posed. Both conventions are common and mixing them is the single most frequent source of shape errors.

8.6 Activation functions

The activation function must be non-linear, by the argument of the notebox in Section 8.2 and by the sharp characterisation of Section 8.4, and for gradient-based training it must be differentiable almost everywhere. Beyond that the choice is open, and it matters more than one might expect: the history of deep learning is to a surprising extent the history of replacing one activation function by another.

The property to keep in view throughout is the *derivative*, because it is f' and not f that enters the backpropagation recursion (8.66) and therefore decides whether a gradient survives its passage through a layer.

8.6.1 Saturating activations

The *logistic* or sigmoid function of Eq. (5.3),

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)\,[1-\sigma(z)]\,, \tag{8.22}$$

was the standard choice for decades and is the one Cybenko's theorem was proved for. Its derivative is expressible in the function value alone, which is convenient, and it never exceeds $\tfrac{1}{4}$ – which, as Section 8.9 shows, is fatal in deep networks. Its output is also strictly positive with mean near $\tfrac{1}{2}$, so the inputs to the next layer are not centred, which slows learning further.

The *hyperbolic tangent* $\tanh(z) = 2\sigma(2z) - 1$, with $\tanh'(z) = 1 - \tanh^2(z)$, is a rescaled sigmoid mapping onto $(-1,1)$. Its output is centred on zero and its derivative reaches 1 rather than $\tfrac{1}{4}$; it is consistently the better of the two and there is no good reason to prefer the logistic function in a hidden layer. Both nevertheless saturate: for $|z|$ large the derivative is essentially zero and the unit stops learning.

8.6.2 The rectified family

The *rectified linear unit*,

$$\mathrm{ReLU}(z) = \max(0,z), \qquad \mathrm{ReLU}'(z) = \begin{cases} 1 & z > 0, \\ 0 & z < 0, \end{cases} \tag{8.23}$$

broke the deadlock. It is trivial to compute, it does not saturate for positive arguments so the gradient passes through undiminished, and it produces genuinely sparse activations since about half the units output zero. Its derivative is undefined at the origin, which in practice is assigned the value zero and never noticed.

ReLU has one characteristic failure, the *dying ReLU*. If a unit's weights are updated so that its input is negative for every training sample, it outputs zero always; its gradient is then zero as well, so it can never recover, and the unit is dead for the remainder of training. With

a large learning rate a substantial fraction of a network can die this way – in bad cases half of it.

Three repairs give the negative branch a non-zero slope. The *leaky* ReLU uses a fixed small slope,

$$\mathrm{LReLU}(z) = \begin{cases} z & z > 0, \\ \alpha z & z \le 0, \end{cases} \qquad \alpha \approx 0.01, \tag{8.24}$$

so the gradient never vanishes entirely. *PReLU* makes α a parameter learned by backpropagation like any other, one per channel. The *exponential linear unit*

$$\mathrm{ELU}(z) = \begin{cases} \alpha\,(e^{z} - 1) & z < 0, \\ z & z \ge 0, \end{cases} \tag{8.25}$$

does the same more smoothly, saturating at $-\alpha$ for large negative arguments so that the unit output has a mean nearer zero, at the cost of an exponential. *SELU* is ELU with two particular constants chosen so that, under conditions on the initialisation, the activations of successive layers converge to zero mean and unit variance automatically – self-normalising, and so an alternative to the batch normalisation of Section 8.12.

All of these are piecewise linear or piecewise smooth with a kink at the origin. The next family removes the kink.

8.6.3 Smooth gated activations: GELU and its relatives

The *Gaussian error linear unit* takes a different view of what an activation is doing. Rather than thresholding the input, it *weights the input by the probability that it is positive*:

$$\boxed{\mathrm{GELU}(z) = z\,\Phi(z),} \qquad \Phi(z) = \frac{1}{2}\left[1 + \mathrm{erf}\left(\frac{z}{\sqrt{2}}\right)\right], \tag{8.26}$$

where Φ is the cumulative distribution function of the standard normal distribution met in Eq. (2.3). A unit whose input is large and positive is passed through essentially unchanged, since $\Phi \to 1$; one whose input is large and negative is suppressed, since $\Phi \to 0$; and in between the suppression is graded rather than abrupt. ReLU is the limiting case in which Φ is replaced by the step function, that is in which the gating is deterministic rather than probabilistic.

Because the error function is expensive, implementations use the approximation

$$\mathrm{GELU}(z) \approx \frac{z}{2}\left(1 + \tanh\left[\sqrt{\frac{2}{\pi}}\left(z + 0.044715\,z^{3}\right)\right]\right), \tag{8.27}$$

which agrees with Eq. (8.26) to better than 5×10^{-4} everywhere on $[-8, 8]$ and is what most frameworks compute by default.

Two close relatives replace the Gaussian gate by a cheaper one. *Swish*, also called *SiLU*, gates with the logistic function,

$$\mathrm{Swish}(z) = z\,\sigma(z), \tag{8.28}$$

and *Mish* gates with a tanh of the softplus,

$$\mathrm{Mish}(z) = z \tanh\left(\ln\left(1 + e^{z}\right)\right). \tag{8.29}$$

The *softplus* $\ln(1 + e^{z})$ itself is a smooth approximation to ReLU and appears mainly where a strictly positive output is needed, as when a network predicts a variance.

What these functions have in common, and why it matters.

All three are smooth, all three are *non-monotone*, and all three have a derivative exceeding one somewhere. These are not incidental.

The non-monotonicity is easily quantified. Each dips below zero for moderately negative arguments before returning: GELU attains its minimum -0.170 at $z = -0.752$, Swish -0.278 at $z = -1.279$, and Mish -0.309 at $z = -1.192$. A small negative response is therefore preserved rather than discarded, which is the intended contrast with ReLU, and the function is not a squashing function in the sense of any of the classical theorems – it remains a universal approximator because it is not a polynomial, by Section 8.4, which is precisely the situation the sharper characterisation was needed for.

The derivative maxima are 1.129 for GELU, 1.100 for Swish and 1.089 for Mish, against 1 for $\tanh$ and $\frac{1}{4}$ for the sigmoid. A factor slightly greater than one in Eq. (8.71) means these activations can mildly *amplify* a gradient rather than only attenuate it, which is a genuine advantage in depth; it also means they cannot be relied on to damp an exploding gradient, so the clipping of Section 8.12 remains necessary.

The smoothness matters for a reason that has nothing to do with classification. ReLU has a discontinuous first derivative and no second derivative at the origin, so any method requiring curvature – and any application in which the network's *output* is differentiated, as when solving differential equations – is compromised. This is why the GELU family is the default choice for physics-informed networks, a point we return to in the next chapter.

Gated linear units.

A final family generalises the idea by learning the gate. A *gated linear unit* splits a layer's output in two and uses one half to gate the other,

$$\mathrm{GLU}(\boldsymbol{z}) = (\boldsymbol{W}_1\boldsymbol{z} + \boldsymbol{b}_1) \circ \sigma(\boldsymbol{W}_2\boldsymbol{z} + \boldsymbol{b}_2)\,, \tag{8.30}$$

with $\circ$ the Hadamard product. Replacing the logistic gate by GELU gives *GeGLU* and by Swish gives *SwiGLU*; the latter is used in most current large language models. Note that these are not activation *functions* in the sense of the rest of this section – they are layers, with their own parameters, and they double the number of weights for a given output width.

8.6.4 Which activation should we use?

The empirical ordering, on which the literature is reasonably consistent, is that the GELU and ELU families perform better than leaky ReLU and its variants, which perform better than ReLU, which performs better than $\tanh$, which performs better than the logistic function.

That ordering should be read with great caution, and it is worth measuring rather than repeating. Table 8.2 gives the test accuracy of a network with three hidden layers of thirty units on the digits data, averaged over five random seeds with the standard deviation across seeds.

The table is worth more than the ordering it fails to confirm. One result is unambiguous: the sigmoid is far behind, by eight percentage points and many standard deviations, and its training loss is two orders of magnitude higher – with three hidden layers the attenuation of Section 8.9 is already severe. That much of the received wisdom is solidly reproduced.

The rest of the table is a single cluster. The seven non-saturating activations span 0.9661 to 0.9722, a range of 0.006, while the standard deviation across seeds is between 0.004 and 0.008

Activation	Test accuracy	Final training loss
sigmoid	0.8900 ± 0.0096	0.2971
tanh	$\mathbf{0.9722 \pm 0.0039}$	0.0048
ReLU	0.9672 ± 0.0083	0.0009
leaky ReLU	0.9672 ± 0.0071	0.0009
ELU	0.9672 ± 0.0044	0.0011
GELU	0.9694 ± 0.0061	0.0010
Swish	0.9661 ± 0.0054	0.0010
Mish	0.9672 ± 0.0048	0.0010

Table 8.2 Test accuracy of a 64-30-30-30-10 network on the digits data after 60 epochs, as mean and standard deviation over five random seeds. Only the sigmoid is clearly separated from the rest.

– so *the spread between the methods is smaller than the spread between random seeds of the same method*. On this evidence the claimed ordering among them cannot be distinguished from noise, and tanh, which the ordering places second from last, is nominally the best. Note also that the training losses are nearly identical while the test accuracies differ, so what little variation there is comes from generalisation and not from optimisation.

The honest conclusion is that on a problem of this size the choice among the non-saturating activations does not matter much, and that a paper reporting a single run and declaring a winner among them would be reporting seed noise. The differences do become real at greater depth, in transformers, and where the smoothness itself is needed; they are not real here. This is also a reminder of Section 2.5: with 360 test samples the standard error of an accuracy near 0.97 is about 0.009, so a difference of 0.006 was never going to be resolvable. Figure 8.7 shows the same data with error bars, which makes the point at a glance.

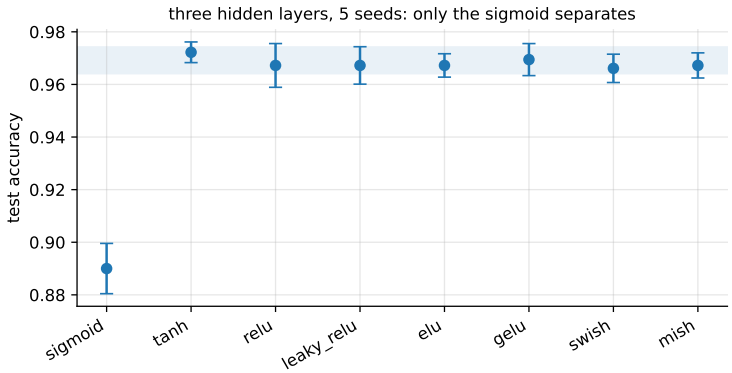


Fig. 8.7 The data of Table 8.2 plotted with error bars of one standard deviation over five seeds. The shaded band spans the seven non-saturating activations; they are mutually indistinguishable, while the sigmoid lies far below.

Three further caveats apply to the literature ordering. It is confounded with architecture, since GELU rose to prominence together with transformers and is best attested there. Runtime matters: ReLU is a comparison and a multiplication, while GELU as computed by Eq. (8.27) requires a tanh and a cubic, which on large models is not negligible. If runtime is a concern, leaky ReLU with the default $\alpha = 0.01$ avoids both the dying-unit problem and the cost, without introducing a hyperparameter worth tuning.

As a working rule: ReLU is a sound default and the right thing to try first; move to leaky ReLU or ELU if units are dying; use GELU or Swish in transformers, in very deep networks, and wherever the network output must be differentiated. Reserve tanh for the cases where a bounded activation is genuinely wanted, and use the logistic function in a hidden layer only when reproducing an old result.

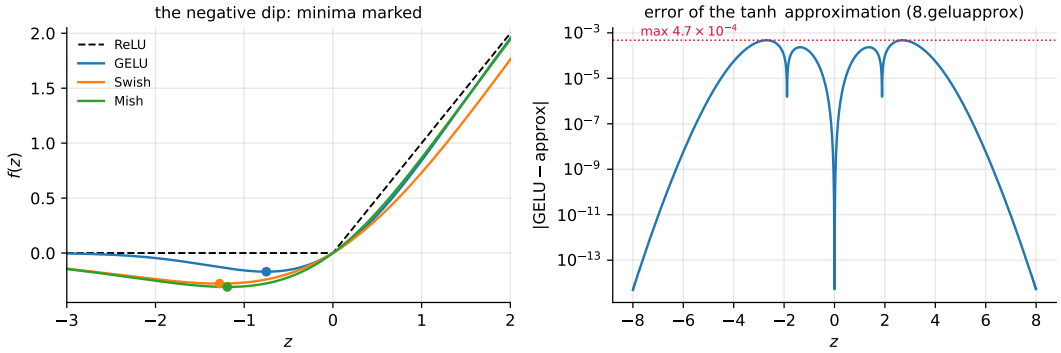


Fig. 8.8 Left: GELU, Swish and Mish against ReLU, with the minimum of each marked – all three dip below zero and are therefore non-monotone. Right: the error of the tanh approximation (8.27) to the exact GELU (8.26), which nowhere exceeds 4.7×10^{-4}.

Figure 8.9 collects the three families with their derivatives. The bottom row is the one to study. The saturating functions have derivatives that vanish in both tails, and the sigmoid's never exceeds $\frac{1}{4}$; the rectified functions have derivative exactly one on the positive axis and zero or nearly zero on the negative; the smooth gated functions have derivatives that rise slightly *above* one near the origin and decay smoothly on both sides. Reading the recursion (8.66) through these three panels explains the empirical ordering better than any list of benchmark numbers.

The output layer is not a free choice.

Everything above concerns hidden layers. The output activation is fixed by the range of the target, exactly as the loss is fixed by its distribution in Section 3.6: the identity for regression, the sigmoid for binary classification, the softmax of Eq. (5.27) for K classes, and softplus or an exponential where the output must be positive. Choosing these independently of the loss is one of the more common ways to produce a network that trains to something meaningless.

8.7 The cost function and the optimisation problem

Collect all weights and biases into a single parameter vector $\boldsymbol{\Theta} = \{\boldsymbol{W}^l, \boldsymbol{b}^l\}_{l=1}^{L}$. Training means solving

$$\hat{\boldsymbol{\Theta}} = \arg\min_{\boldsymbol{\Theta}} C(\boldsymbol{\Theta}), \qquad C(\boldsymbol{\Theta}) = \frac{1}{n}\sum_{i=1}^{n} L(y_i, \tilde{y}_i(\boldsymbol{\Theta})) + \lambda\, \Omega(\boldsymbol{\Theta}), \tag{8.31}$$

with L the per-sample loss and Ω a regularisation term. The loss is chosen by the argument of Section 3.6: the half-squared error for regression with Gaussian noise,

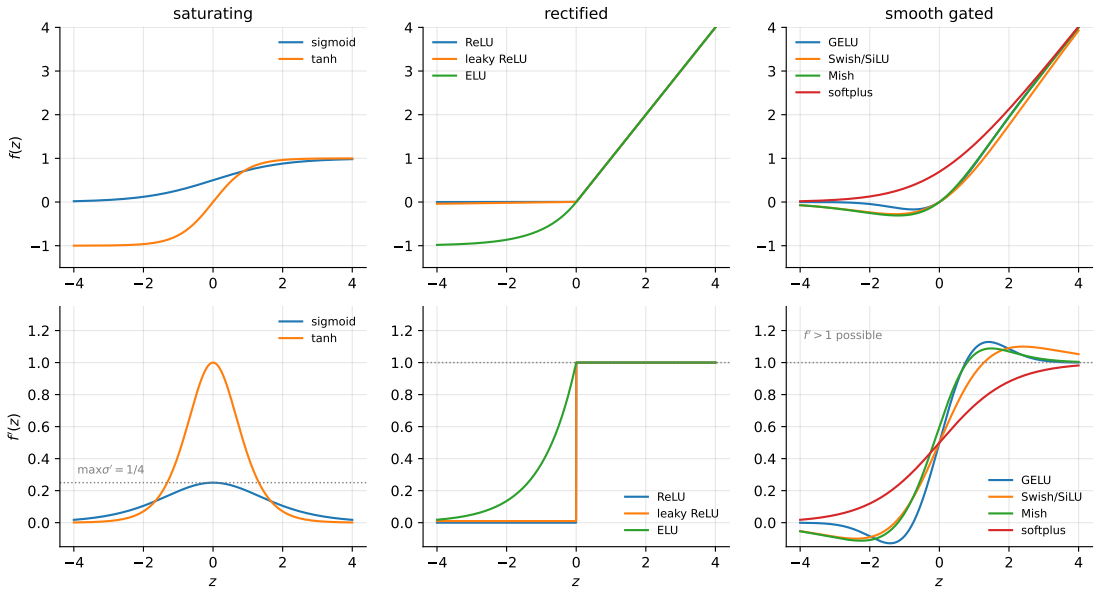


Fig. 8.9 Nine activation functions in three families (top) and their derivatives (bottom). Only the derivative enters the backpropagation recursion (8.66). Dotted lines mark $\max \sigma' = 1/4$ in the left panel and $f' = 1$ in the other two.

$$C(\boldsymbol{\Theta}) = \frac{1}{2n} \sum_{i=1}^{n} \left(a_i^L - y_i \right)^2 , \tag{8.32}$$

and the cross entropy (5.13) for classification, in its multiclass form (5.28) when $K > 2$.

Three features distinguish Eq. (8.31) from every cost function so far in this book. It is *not convex*: the composition of linear maps with non-linearities destroys the property that Chapter 4 leaned on so heavily, so there are many local minima and saddle points, and the minimum we reach depends on the initialisation. It is *highly symmetric*: permuting the units within a hidden layer, together with the corresponding weights, leaves the function computed by the network unchanged, so every minimum comes with a large family of equivalent copies. And it has *very many parameters*, so the Hessian of Section 4.2 is out of the question and we are restricted to the first-order methods of Sections 4.7 to 4.11.

That we nonetheless train such networks successfully is an empirical fact rather than a theorem. Part of the explanation is that in high dimensions strict local minima are rare relative to saddle points, and that the many symmetric copies of a good solution make some good solution easy to reach.

8.8 Backpropagation

We now derive the gradient. The derivation is short once the notation of Section 8.5 is in place, but the indices hide what is going on, so we first work through the two smallest networks there are: a single neuron with a scalar input and a scalar output, which is the logistic regression of Figure 5.4 with a different loss, and the same neuron with one hidden neuron in front of it, first by hand and then with `jax.grad`. A third example, with two inputs and two hidden nodes, then shows where the indices go. Everything that Section 8.8.3 does with matrices and sums, these examples do with a handful of products, and the second of them already contains the backpropagation equation.

8.8.1 Two scalar examples: no hidden layer and one hidden layer

Throughout this subsection every layer has one node, so the node indices i, j of Eq. (8.18) are dropped and only the layer superscript remains: w^l, b^l, z^l and a^l are numbers, not arrays. We take the sigmoid σ of Eq. (8.22) as activation, with $\sigma'(z) = \sigma(z)(1-\sigma(z))$, and the half-squared error of Eq. (8.32) for a single training pair (x, y). Nothing depends on these choices; for another activation replace σ' by f', and for another loss replace $a^L - y$ by $\partial C/\partial a^L$.

No hidden layer.

The network is one neuron,

$$x \longrightarrow z^1 = w^1 x + b^1 \longrightarrow a^1 = \sigma\big(z^1\big) \longrightarrow C = \tfrac{1}{2}\big(a^1 - y\big)^2, \tag{8.33}$$

with the two trainable parameters $\boldsymbol{\Theta} = \{w^1, b^1\}$. It is the neuron of Figure 5.4 with $p = 1$ input and the cross entropy replaced by the squared error, and the top panel of Figure 8.10 draws it. Training means moving w^1 and b^1 so as to reduce the discrepancy between the prediction a^1 and the target y, and for that we need $\partial C/\partial w^1$ and $\partial C/\partial b^1$. The cost depends on w^1 only along the path $w^1 \to z^1 \to a^1 \to C$, so the chain rule (1.51) gives

$$\frac{\partial C}{\partial w^1} = \frac{\partial C}{\partial a^1}\,\frac{\partial a^1}{\partial z^1}\,\frac{\partial z^1}{\partial w^1} = \big(a^1 - y\big)\,\sigma'\big(z^1\big)\,x, \tag{8.34}$$

and the same path with $\partial z^1/\partial b^1 = 1$ as its last factor gives $\partial C/\partial b^1 = (a^1 - y)\sigma'(z^1)$. The two gradients share everything except the last factor, and it pays to give the shared part a name. Define the *error* of the neuron,

$$\delta^1 = \big(a^1 - y\big)\,\sigma'\big(z^1\big) = \frac{\partial C}{\partial z^1}, \tag{8.35}$$

the sensitivity of the cost to the pre-activation. Then

$$\frac{\partial C}{\partial w^1} = \delta^1 x, \qquad \frac{\partial C}{\partial b^1} = \delta^1, \tag{8.36}$$

and one step of gradient descent with learning rate γ reads

$$w^1 \leftarrow w^1 - \gamma\,\delta^1 x, \qquad b^1 \leftarrow b^1 - \gamma\,\delta^1. \tag{8.37}$$

Each update is an error term, δ^1, multiplied by the signal that entered the parameter: x for the weight and the constant 1 for the bias. This is the delta rule of Eq. (5.37) once more, with one difference that is worth pausing on. For logistic regression the derivative of the cross entropy with respect to z was the bare residual $\hat{p} - y$, Eq. (5.36), because the derivative of the softplus cancelled the derivative of the sigmoid. With the squared error nothing cancels and the residual is multiplied by $\sigma'(z^1)$, which is at most $1/4$ and decays to zero as soon as $|z^1|$ is a few units. A saturated neuron therefore learns slowly however wrong it is; this is the factor f' that Section 8.8.3 will find in front of every error, and the seed of the vanishing gradients of Section 8.9.

The same neuron, read backwards.

Section 5.8.2 computed the gradient of logistic regression by writing the neuron as an evaluation trace and sweeping it in reverse. The same can be done here, and the result identifies δ^1. With the parameters as inputs, $v_{-1} = b^1$ and $v_0 = w^1$, and the data as constants, the trace of Eq. (8.33) is

$$v_1 = v_0 x, \qquad v_2 = v_1 + v_{-1} = z^1, \qquad v_3 = \sigma(v_2) = a^1, \qquad v_4 = \tfrac{1}{2}(v_3 - y)^2 = C. \tag{8.38}$$

The reverse sweep of Eq. (4.87) starts from $\bar{v}_4 = 1$ and multiplies by the local derivatives on the way back:

$$\bar{v}_3 = \bar{v}_4 (v_3 - y) = a^1 - y, \qquad \bar{v}_2 = \bar{v}_3 \sigma'(v_2) = \delta^1, \qquad \bar{v}_0 = \bar{v}_2 x = \delta^1 x, \qquad \bar{v}_{-1} = \bar{v}_2 = \delta^1. \tag{8.39}$$

The error δ^1 is the *adjoint of the pre-activation*, exactly as $\bar{z} = \hat{p} - y$ was in Figure 5.4, and Eq. (8.36) is what the sweep delivers when it reaches the two inputs. One sweep gives both gradients; the forward mode of Eq. (4.81) would need one sweep per parameter, the same comparison as in Section 5.8.2. Backpropagation, we shall see, is this sweep continued through as many layers as the network has.

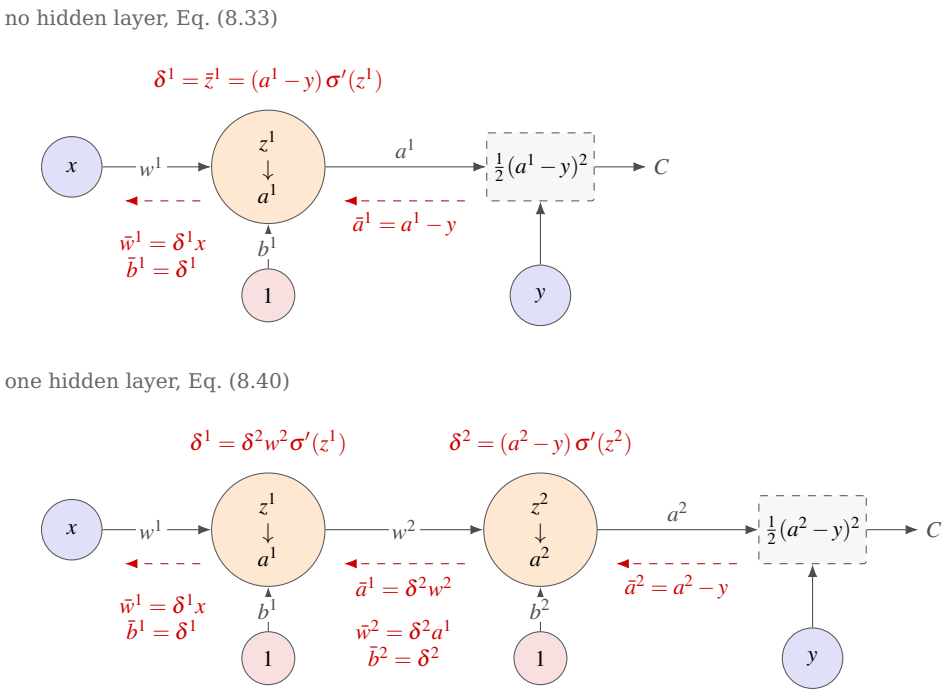


Fig. 8.10 The two scalar networks of Section 8.8.1, drawn in the style of Figure 5.4. Black arrows are the forward pass: each neuron forms $z^l = w^l a^{l-1} + b^l$ from its input and its bias (the weight of an input fixed at one) and emits $a^l = \sigma(z^l)$; the dashed box is the squared-error loss. Red dashed arrows are the reverse sweep: the adjoint of the cost is one, it becomes the residual $a^L - y$ at the loss node, the error $\delta^L = \bar{z}^L$ after the activation, and is then multiplied by whatever sits on the edge it travels along – the input x or a^1 to reach a weight, the constant 1 to reach a bias, and the weight w^2 to reach the hidden neuron, where a further factor $\sigma'(z^1)$ produces δ^1. The bottom panel is Eq. (8.43); with several nodes per layer the single edge w^2 becomes the sum over k in Eq. (8.64).

One hidden layer.

Now place a second neuron in front of the first, so that the network has one hidden layer of one node, the bottom panel of Figure 8.10:

$$\begin{aligned} z^1 &= w^1 x + b^1, \qquad a^1 = \sigma\left(z^1\right), \\ z^2 &= w^2 a^1 + b^2, \qquad a^2 = \sigma\left(z^2\right), \\ C &= \tfrac{1}{2}\left(a^2 - y\right)^2, \end{aligned} \tag{8.40}$$

with $\boldsymbol{\Theta} = \{w^1, b^1, w^2, b^2\}$. The forward pass computes and *stores* (z^1, a^1, z^2, a^2); every one of them is needed again on the way back. The output layer is the previous example with a^1 in place of x: the error is

$$\delta^2 = \left(a^2 - y\right)\sigma'\left(z^2\right), \qquad \frac{\partial C}{\partial w^2} = \delta^2 a^1, \qquad \frac{\partial C}{\partial b^2} = \delta^2. \tag{8.41}$$

For the hidden weight the path to the cost is longer, since w^1 influences C only through z^1, a^1, z^2 and a^2 in turn:

$$\frac{\partial C}{\partial w^1} = \frac{\partial C}{\partial a^2}\frac{\partial a^2}{\partial z^2}\frac{\partial z^2}{\partial a^1}\frac{\partial a^1}{\partial z^1}\frac{\partial z^1}{\partial w^1} = \left(a^2 - y\right)\sigma'\left(z^2\right)\, w^2\, \sigma'\left(z^1\right)\, x. \tag{8.42}$$

The first two factors are δ^2, which we already have. Reading the product from left to right, the error of the output neuron is carried back along the edge w^2 to the hidden neuron and there modulated by the local derivative of its activation. Defining the error of the hidden neuron in the same way as before, $\delta^1 = \partial C/\partial z^1$, we have found

$$\boxed{\delta^1 = \delta^2 w^2 \sigma'\left(z^1\right),} \qquad \frac{\partial C}{\partial w^1} = \delta^1 x, \qquad \frac{\partial C}{\partial b^1} = \delta^1. \tag{8.43}$$

The recurring pattern is now visible. Forwards, every layer computes $z^l = w^l a^{l-1} + b^l$ and $a^l = \sigma(z^l)$. Backwards, every layer computes its error from the error downstream, multiplied by the weight that connects them and by its own $\sigma'(z^l)$, and then the gradient of each weight is that error times the activation that entered the weight, and the gradient of each bias is the error itself. Nothing is recomputed: the long product of Eq. (8.42) is never formed as such, because its first half is δ^2 and had already been evaluated for the output layer. With ten hidden neurons in a chain, each error would be obtained from the next by one multiplication, and each gradient from its error by one more.

In the language of Section 4.14, Eq. (8.43) is the reverse sweep of Eq. (8.39) that did not stop at the pre-activation. In Section 5.8.2 the sweep reached $\bar{z} = \hat{p} - y$ and then stepped onto the edges carrying the parameters, because the inputs x_j of the neuron were constants. Here the input a^1 of the output neuron is itself the output of a node in the graph, so the sweep continues: $\bar{a}^1 = \delta^2 w^2$ by the local derivative $\partial z^2/\partial a^1 = w^2$, then $\bar{z}^1 = \bar{a}^1 \sigma'(z^1) = \delta^1$, and only then the edges w^1 and b^1. The point made in Section 5.8.1, that the inputs of a neuron will themselves be the outputs of other neurons and that the sweep will continue along the edges into the layer before, is exactly this.

Numbers.

At the parameters and the observation of Section 5.8.2, $w^1 = 0.25$, $b^1 = 0.5$, $x = 2$, $y = 1$, the single neuron has $z^1 = 1$, $a^1 = 0.731$ and $C = 0.0362$; then $\sigma'(z^1) = 0.197$, $\delta^1 = -0.0529$, and

the gradient (8.36) is $(\partial C/\partial w^1, \partial C/\partial b^1) = (-0.106, -0.0529)$. The cross entropy at the same point gave $(-0.538, -0.269)$ in Section 5.8.2; the two gradients differ by exactly the factor $\sigma'(z^1) = 0.197$ discussed above. Adding an output neuron with $w^2 = 1$, $b^2 = -0.5$ gives $z^2 = 0.231$, $a^2 = 0.557$, $C = 0.0979$, $\delta^2 = -0.109$, and by Eq. (8.43) $\delta^1 = \delta^2 w^2 \sigma'(z^1) = -0.0215$, so that the four gradients are $\partial C/\partial w^2 = -0.0798$, $\partial C/\partial b^2 = -0.109$, $\partial C/\partial w^1 = -0.0429$ and $\partial C/\partial b^1 = -0.0215$. The following program is Eqs. (8.36)–(8.43) transcribed: a forward pass that stores z^l and a^l, the output error, the backpropagated error, and the products that give the gradients.

```
import numpy as np

def sigma(z):  return 1.0/(1.0+np.exp(-z))
def dsigma(z): s = sigma(z); return s*(1.0-s)

# no hidden layer: x -> z1 = w1 x + b1 -> a1 = sigma(z1) -> C = (a1-y)^2/2
def grad_one(w1, b1, x, y):
    z1 = w1*x + b1; a1 = sigma(z1)                 # forward pass, store z1, a1
    # error delta1 = (a1-y) sigma'(z1)
    delta1 = (a1 - y)*dsigma(z1)
    return 0.5*(a1-y)**2, delta1*x, delta1, delta1 # C, dC/dw1, dC/db1, delta1

# one hidden layer: x -> (w1,b1) -> a1 -> (w2,b2) -> a2 -> C = (a2-y)^2/2
def grad_two(w1, b1, w2, b2, x, y):
    z1 = w1*x + b1; a1 = sigma(z1)                 # forward pass
    z2 = w2*a1 + b2; a2 = sigma(z2)
    delta2 = (a2 - y)*dsigma(z2)                   # output error
    # delta1 = delta2 w2 sigma'(z1)
    delta1 = delta2*w2*dsigma(z1)
    return (0.5*(a2-y)**2, delta1*x, delta1, delta2*a1, delta2, delta1, delta2)

x, y = 2.0, 1.0
C, gw1, gb1, d1 = grad_one(0.25, 0.5, x, y)
print(f"no hidden layer : C = {C:.4f}  delta1 = {d1:.4f}")
print(f"   dC/dw1 = {gw1:.4f}  dC/db1 = {gb1:.4f}")
C, gw1, gb1, gw2, gb2, d1, d2 = grad_two(0.25, 0.5, 1.0, -0.5, x, y)
print(f"one hidden layer: C = {C:.4f}  delta2 = {d2:.4f}  delta1 = {d1:.4f}")
print(f"   dC/dw1 = {gw1:.4f}  dC/db1 = {gb1:.4f}", end="")
print(f"  dC/dw2 = {gw2:.4f}  dC/db2 = {gb2:.4f}")
```

```
no hidden layer : C = 0.0362  delta1 = -0.0529
   dC/dw1 = -0.1058  dC/db1 = -0.0529
one hidden layer: C = 0.0979  delta2 = -0.1092  delta1 = -0.0215
   dC/dw1 = -0.0429  dC/db1 = -0.0215  dC/dw2 = -0.0798  dC/db2 = -0.1092
```

A first training program.

The same equations, written with arrays so that the code survives the transition to several inputs, several hidden nodes and several samples, give the first neural network we train in this chapter. We follow the convention of Eq. (8.21): the samples are the rows of `x`, so that a single scalar observation is an array of shape $(1,1)$, the weight arrays have shape (number of inputs, number of outputs) and the forward pass reads `x @ w_1 + b_1`. Two details differ from the derivation above and are worth noting. The output node is *linear*, $a^2 = z^2$, as befits a regression target that is not confined to $(0,1)$; its error is then simply $\delta^2 = a^2 - y$, the collapse that Section 8.8.4 will discuss for the natural pairings of output activation and loss. And the backward pass is written with matrix products, `delta_2 @ w_2.T` for the pull-back through the weight and `x.T @ delta_1` for the outer product of inputs and errors, which for scalars are just the multiplications of Eq. (8.43) but will be the general Eqs. (8.66) and (8.68) without

any change to the code. The task is to make the network return $y = 2x + 1 = 9$ at $x = 4$, starting from random weights.

```
import numpy as np

def sigmoid(z):
    return 1.0/(1.0+np.exp(-z))

def forwardpropagation(x):
    z_1 = np.matmul(x, w_1) + b_1      # pre-activation of the hidden layer
    a_1 = sigmoid(z_1)                 # hidden activation
    z_2 = np.matmul(a_1, w_2) + b_2    # pre-activation of the output layer
    a_2 = z_2                          # linear output
    return a_1, a_2

def backpropagation(x, y):
    a_1, a_2 = forwardpropagation(x)
    delta_2 = a_2 - y                  # linear output: f' = 1
    # delta_2 w_2 sigma'(z_1)
    delta_1 = np.matmul(delta_2, w_2.T) * a_1 * (1 - a_1)
    dW2 = np.matmul(a_1.T, delta_2);  db2 = np.sum(delta_2, axis=0)
    dW1 = np.matmul(x.T, delta_1);    db1 = np.sum(delta_1, axis=0)
    return dW2, db2, dW1, db1, 0.5*np.sum((a_2 - y)**2)

np.random.seed(0)
x = np.array([[4.0]])                  # one sample, one feature: shape (1, 1)
y = 2*x + 1.0                          # target 9
n_features, n_hidden, n_outputs = 1, 1, 1
w_1 = np.random.randn(n_features, n_hidden); b_1 = np.zeros(n_hidden) + 0.01
w_2 = np.random.randn(n_hidden, n_outputs); b_2 = np.zeros(n_outputs) + 0.01

gamma = 0.1
for i in range(50):
    dW2, db2, dW1, db1, cost = backpropagation(x, y)
    if i % 10 == 0 or i == 49:
        print(f"iteration {i:2d}: cost = {cost:.6f}")
    w_2 -= gamma*dW2;  b_2 -= gamma*db2
    w_1 -= gamma*dW1;  b_1 -= gamma*db1
print("w_1 =", w_1.ravel(), " b_1 =", b_1)
print("w_2 =", w_2.ravel(), " b_2 =", b_2)
```

```
iteration  0: cost = 36.895631
iteration 10: cost = 0.426990
iteration 20: cost = 0.004939
iteration 30: cost = 0.000057
iteration 40: cost = 0.000001
iteration 49: cost = 0.000000
w_1 = [1.79581063]  b_1 = [0.01793957]
w_2 = [4.69500507]  b_2 = [4.30836978]
```

The cost drops by seven orders of magnitude in fifty iterations. This is not yet learning in any interesting sense – there is one observation and four parameters, and a single point can always be hit – but every line of the program will survive unchanged into Section 8.8.2 and, with a loop over layers added, into the class of Section 8.13. The learning rate matters even here, and in an instructive way: with $\gamma = 1$ the first few steps drive the hidden weight to $w^1 \approx 3.9$, so that $z^1 \approx 15$ and $\sigma'(z^1) \approx 10^{-7}$; the error δ^1 is then multiplied by that factor, the hidden neuron stops learning, and the cost stalls at 5.02 for the remaining iterations. This is the saturation mechanism noted after Eq. (8.37), met in a program of twenty lines.

The same by automatic differentiation.

Everything in `backpropagation` above was derived by hand from the chain rule. Section 5.8.2 argued that this derivation is what reverse-mode automatic differentiation performs mechanically, and the scalar examples are the place to see it done by a machine. The first listing hands `jax.grad` of Section 4.14.8 the *forward pass only* of the two networks, Eqs. (8.33) and (8.40), and asks for the gradient at the point used above; it also runs the forward mode, `jax.jvp` with the two unit seeds, to make the two-sweeps-against-one comparison of Eq. (8.39) concrete.

```
import jax, jax.numpy as jnp
jax.config.update("jax_enable_x64", True)

def C_one(p, x, y):                    # forward pass of the single neuron ...
    return 0.5*(jax.nn.sigmoid(p[0]*x + p[1]) - y)**2
def C_two(p, x, y):                    # ... and with the hidden neuron
    a1 = jax.nn.sigmoid(p[0]*x + p[1])
    return 0.5*(jax.nn.sigmoid(p[2]*a1 + p[3]) - y)**2

x, y = 2.0, 1.0
p1, p2 = jnp.array([0.25, 0.5]), jnp.array([0.25, 0.5, 1.0, -0.5])
print("no hidden layer,  jax.grad:", jax.grad(C_one)(p1, x, y))
print("one hidden layer, jax.grad:", jax.grad(C_two)(p2, x, y))
# forward mode, one sweep per parameter, for comparison
seeds = jnp.eye(2)
print("no hidden layer,  jax.jvp: ",
      [float(jax.jvp(C_one, (p1, x, y), (s, 0.0, 0.0))[1]) for s in seeds])
```

```
no hidden layer,  jax.grad: [-0.10575419 -0.05287709]
one hidden layer, jax.grad: [-0.04292404 -0.02146202 -0.07980184 -0.1091593 ]
no hidden layer,  jax.jvp:  [-0.10575418556853343, -0.052877092784266715]
```

The four numbers agree with the hand-derived ones to every printed digit, and the forward sweeps return the same two numbers one at a time. Nothing in the listing mentions δ, σ' or the chain rule: `jax.grad` has traced the forward pass, recorded the tape (z^1, a^1, z^2, a^2) and swept it in reverse, which is Eq. (8.39) continued through the hidden neuron. The training program is now shorter than before, because the whole function `backpropagation` is replaced by one call:

```
import numpy as np
import jax, jax.numpy as jnp
jax.config.update("jax_enable_x64", True)

def cost(params, x, y):
    w_1, b_1, w_2, b_2 = params
    a_1 = jax.nn.sigmoid(x @ w_1 + b_1)        # the forward pass, nothing else
    a_2 = a_1 @ w_2 + b_2
    return 0.5*jnp.sum((a_2 - y)**2)

grad = jax.jit(jax.grad(cost))                 # the backward pass, generated

np.random.seed(0)
x = jnp.array([[4.0]]); y = 2*x + 1.0
params = [jnp.array(np.random.randn(1, 1)), jnp.zeros(1) + 0.01,
          jnp.array(np.random.randn(1, 1)), jnp.zeros(1) + 0.01]

gamma = 0.1
for i in range(50):
    if i % 10 == 0 or i == 49:
        print(f"iteration {i:2d}: cost = {cost(params, x, y):.6f}")
    g = grad(params, x, y)
```

```
    params = [p - gamma*gp for p, gp in zip(params, g)]
w_1, b_1, w_2, b_2 = params
print("w_1 =", w_1.ravel(), " b_1 =", b_1)
print("w_2 =", w_2.ravel(), " b_2 =", b_2)
```

```
iteration  0: cost = 36.895631
iteration 10: cost = 0.426990
iteration 20: cost = 0.004939
iteration 30: cost = 0.000057
iteration 40: cost = 0.000001
iteration 49: cost = 0.000000
w_1 = [1.79581063]  b_1 = [0.01793957]
w_2 = [4.69500507]  b_2 = [4.30836978]
```

The costs and the final parameters coincide with the hand-written version to all printed digits, as they must: the two programs evaluate the same gradient by the same algorithm, one of them written by us and the other generated from the forward pass. This is the pattern every framework of Chapters 10 onwards relies on – the programmer writes the forward pass and the library supplies the backward one – and it is also why the hand-written backpropagation of Section 8.13 is worth having: it is the only way to know what the library does, and the gradient check of Section 4.14.10 is how one verifies that the two agree.

Training the two networks.

Figure 8.11 applies the updates (8.37) to both networks on a small scalar regression problem: $n = 20$ inputs equally spaced in $[-2,2]$ with targets $y_i = \sigma(2x_i - 1) + \varepsilon_i$, $\varepsilon_i \sim N(0, 0.1^2)$, and the cost averaged over the sample as in Eq. (8.32), so that the errors δ^l of each observation are simply summed. Both networks start from the parameters used above and take 1000 steps of plain gradient descent with $\gamma = 2$. The left panel shows the cost surface of the single neuron, which is not convex – the sigmoid sees to that – but has one basin around $(w^1, b^1) = (2, -1)$, and the descent path from $(0.25, 0.5)$ that ends at $(2.13, -0.78)$ with $C = 0.0053$. The middle panel shows that the network with a hidden neuron, although it has four parameters and contains the single neuron as a special case, reaches only $C = 0.0066$ in the same number of steps, and the right panel shows why: the gradient that reaches w^1 is $\delta^1 x$ in both cases, but in the deeper network δ^1 carries the extra factor $w^2\sigma'(z^2)$ of Eq. (8.43), which is initially about 0.25 and shrinks the first-layer gradient by that amount. (The dip in the blue curve near iteration 250 is a change of sign of $\partial C/\partial w^1$, plotted in absolute value.) Every gradient in the figure was again compared with `jax.grad` at every iteration; the largest discrepancy over the whole run was 3×10^{-17}.

What the general case adds. The two examples contain the whole algorithm. A layer with many nodes changes two things only. A weight now carries two node indices, w^l_{ij}, and the gradient $\delta^l_j a^{l-1}_i$ pairs the error of the node it arrives at with the activation of the node it leaves; and a hidden node feeds *several* nodes of the next layer, so the single term $\delta^2 w^2$ of Eq. (8.43) becomes a sum $\sum_k \delta^{l+1}_k w^{l+1}_{jk}$ over everything it feeds – the fan-out rule of Section 4.14.4, exactly as the two branches of Figure 5.5 were summed when the reverse sweep reached v_2. Section 8.8.2 works out the smallest network in which the first of these appears, and Section 8.8.3 adds the second; Section 8.8.4 then collects the result in four equations that the reader may already recognise.

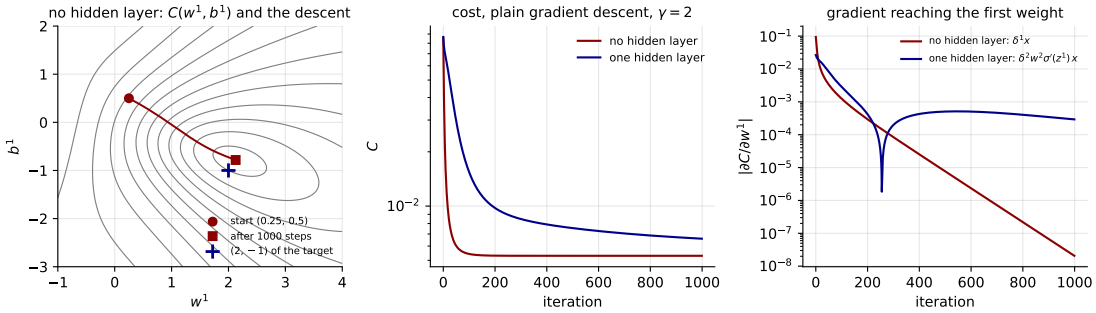


Fig. 8.11 The two scalar networks of Figure 8.10 trained by plain gradient descent, Eq. (8.37), on $n = 20$ scalar observations $y_i = \sigma(2x_i - 1) + \varepsilon_i$ with $\gamma = 2$ and 1000 iterations, starting from $w^1 = 0.25$, $b^1 = 0.5$ (and $w^2 = 1$, $b^2 = -0.5$ for the hidden-layer network). Left: the cost surface $C(w^1, b^1)$ of the single neuron and the descent path from the starting point (circle) to its end (square); the cross marks the parameters used to generate the targets. Middle: the cost against iteration for both networks. Right: the magnitude of the gradient reaching the first weight, $\delta^1 x$ in both cases, but with $\delta^1 = \delta^2 w^2 \sigma'(z^1)$ in the deeper network, Eq. (8.43); the extra factor $w^2 \sigma'(z^2)$ attenuates the signal that trains the hidden neuron, a first glimpse of Section 8.9. Generated by `BookFigures/ch08_scalar_backprop_figures.py`.

8.8.2 Two inputs, two hidden nodes, one output

The scalar examples had one node per layer, so a weight needed one index and an error never had to be summed. The smallest network in which both features of the general case appear has two inputs x_1, x_2, one hidden layer of two nodes and a single output node, Figure 8.12. We now work it out completely, in the index convention of Section 8.5, and this is the last time the derivatives are written one by one.

The network.

The input layer is $a_1^0 = x_1$, $a_2^0 = x_2$. The hidden layer has weights w_{ij}^1 from input i to hidden node j and biases b_j^1, so that by Eq. (8.18)

$$z_j^1 = w_{1j}^1 x_1 + w_{2j}^1 x_2 + b_j^1, \qquad a_j^1 = \sigma\left(z_j^1\right), \qquad j = 1, 2, \tag{8.44}$$

or in the matrix form of Eq. (8.20),

$$\begin{bmatrix} z_1^1 \\ z_2^1 \end{bmatrix} = \begin{bmatrix} w_{11}^1 & w_{21}^1 \\ w_{12}^1 & w_{22}^1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1^1 \\ b_2^1 \end{bmatrix} = \left(\boldsymbol{W}^1\right)^T \boldsymbol{x} + \boldsymbol{b}^1. \tag{8.45}$$

The output layer has one node, so its weights w_{j1}^2 carry a trivial second index that we drop, w_j^2, and

$$z^2 = w_1^2 a_1^1 + w_2^2 a_2^1 + b^2, \qquad a^2 = \sigma\left(z^2\right), \qquad C = \tfrac{1}{2}\left(a^2 - y\right)^2. \tag{8.46}$$

There are nine parameters,

$$\boldsymbol{\Theta} = \left\{w_{11}^1, w_{12}^1, w_{21}^1, w_{22}^1, b_1^1, b_2^1, w_1^2, w_2^2, b^2\right\}, \tag{8.47}$$

and the forward pass stores $z_1^1, z_2^1, a_1^1, a_2^1, z^2, a^2$.

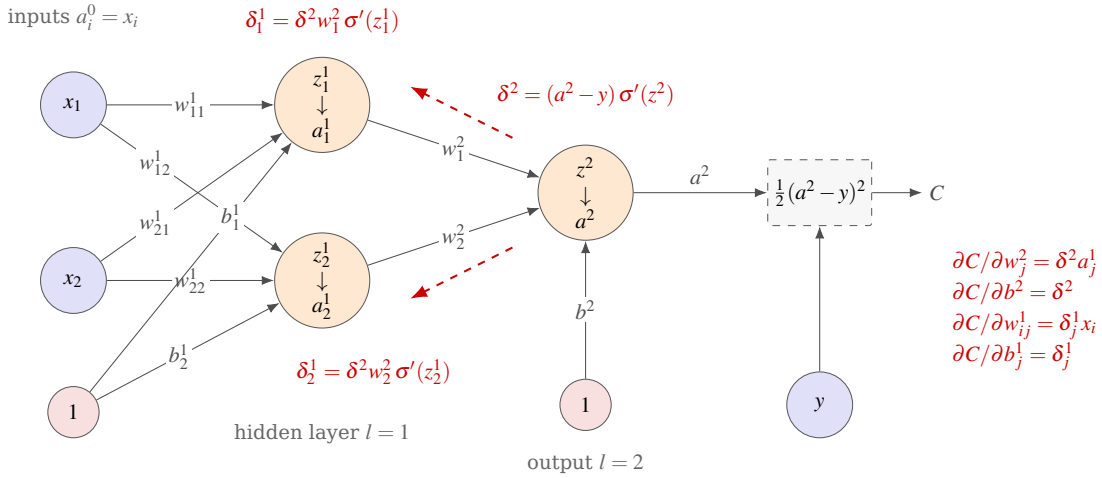


Fig. 8.12 The network of Section 8.8.2: two inputs, one hidden layer of two sigmoid nodes and one output node, nine parameters in all. The weight w_{ij}^1 leaves input i and arrives at hidden node j, as in Fig. 8.6. Red: the backward pass of Eqs. (8.48) and (8.52). The output error δ^2 travels back along each of the two edges w_j^2 separately, and each hidden node multiplies what arrives by its own $\sigma'(z_j^1)$; the gradient of every weight is then the error at the node the weight arrives at times the signal at the node it leaves, Eq. (8.53).

The output layer.

As in the scalar case, the cost depends on the output parameters only through z^2, and

$$\delta^2 = \frac{\partial C}{\partial z^2} = \left(a^2 - y\right)\sigma'\left(z^2\right), \qquad \frac{\partial C}{\partial w_j^2} = \delta^2 a_j^1, \qquad \frac{\partial C}{\partial b^2} = \delta^2, \tag{8.48}$$

for $j = 1, 2$: three of the nine derivatives, and the same three lines as Eq. (8.41) with the input activation a_j^1 now carrying the index of the edge.

The hidden layer.

Take w_{11}^1, the weight from x_1 to the first hidden node. It affects z_1^1 only, z_1^1 affects a_1^1 only, and a_1^1 reaches the cost only through z^2, along the edge w_1^2. The chain is therefore the same five-factor product as Eq. (8.42),

$$\frac{\partial C}{\partial w_{11}^1} = \underbrace{\frac{\partial C}{\partial a^2}\frac{\partial a^2}{\partial z^2}}_{\delta^2}\frac{\partial z^2}{\partial a_1^1}\frac{\partial a_1^1}{\partial z_1^1}\frac{\partial z_1^1}{\partial w_{11}^1} = \delta^2 w_1^2 \sigma'\left(z_1^1\right) x_1, \tag{8.49}$$

using $\partial z^2/\partial a_1^1 = w_1^2$ and $\partial z_1^1/\partial w_{11}^1 = x_1$ from Eq. (8.57). The weight w_{21}^1 from x_2 to the same node has the identical chain with x_2 as its last factor, and the bias b_1^1 has the factor 1. So the first hidden node has an error of its own,

$$\delta_1^1 = \delta^2 w_1^2 \sigma'\left(z_1^1\right), \qquad \frac{\partial C}{\partial w_{11}^1} = \delta_1^1 x_1, \qquad \frac{\partial C}{\partial w_{21}^1} = \delta_1^1 x_2, \qquad \frac{\partial C}{\partial b_1^1} = \delta_1^1, \tag{8.50}$$

and the second hidden node, reached from the output along the other edge w_2^2, has

$$\delta_2^1 = \delta^2 w_2^2 \sigma'\left(z_2^1\right), \qquad \frac{\partial C}{\partial w_{12}^1} = \delta_2^1 x_1, \qquad \frac{\partial C}{\partial w_{22}^1} = \delta_2^1 x_2, \qquad \frac{\partial C}{\partial b_2^1} = \delta_2^1. \tag{8.51}$$

That completes the list of nine. Collected, they read

$$\delta_j^1 = \delta^2 w_j^2 \sigma'(z_j^1), \qquad \frac{\partial C}{\partial w_{ij}^1} = \delta_j^1 a_i^0, \qquad \frac{\partial C}{\partial b_j^1} = \delta_j^1, \qquad i,j \in \{1,2\}, \tag{8.52}$$

and together with Eq. (8.48) the whole gradient is

$$\frac{\partial C}{\partial w_{ij}^l} = \delta_j^l a_i^{l-1}, \qquad \frac{\partial C}{\partial b_j^l} = \delta_j^l, \qquad l = 1,2. \tag{8.53}$$

Equation (8.53) is the rule the scalar examples suggested, now with indices: *the gradient of a weight is the error at the node it arrives at, times the signal at the node it leaves*. In matrix form the two hidden errors are a vector,

$$\boldsymbol{\delta}^1 = \begin{bmatrix} \delta_1^1 \\ \delta_2^1 \end{bmatrix} = \left(\begin{bmatrix} w_1^2 \\ w_2^2 \end{bmatrix} \delta^2 \right) \circ \begin{bmatrix} \sigma'(z_1^1) \\ \sigma'(z_2^1) \end{bmatrix} = \left(\boldsymbol{W}^2 \delta^2 \right) \circ \sigma'\left(\boldsymbol{z}^1\right), \tag{8.54}$$

with $\boldsymbol{W}^2 \in \mathbb{R}^{2\times 1}$, and the four hidden weight gradients are the outer product

$$\frac{\partial C}{\partial \boldsymbol{W}^1} = \boldsymbol{x} \left(\boldsymbol{\delta}^1 \right)^T = \begin{bmatrix} x_1\delta_1^1 & x_1\delta_2^1 \\ x_2\delta_1^1 & x_2\delta_2^1 \end{bmatrix}, \tag{8.55}$$

whose (i,j) entry is $\partial C/\partial w_{ij}^1$, in the same position as w_{ij}^1 in $\boldsymbol{W}^1$. These are, for this network, the backpropagation equations (8.66) and (8.68) of the next sections, and the gradient-descent updates are

$$w_{ij}^l \leftarrow w_{ij}^l - \gamma \delta_j^l a_i^{l-1}, \qquad b_j^l \leftarrow b_j^l - \gamma \delta_j^l, \tag{8.56}$$

for all nine parameters at once.

What the reverse sweep did, and what it did not have to do.

Read as the reverse sweep of Section 4.14.4, the computation has one new feature and one that is still missing. New is that the adjoint δ^2 leaves the output node along *two* edges, w_1^2 and w_2^2, one to each hidden node; that is a fan-in seen from the output, and it costs nothing, since each hidden node simply receives its own share. Still missing is the fan-out: each hidden node feeds only one node downstream, so its adjoint arrives in a single term and no sum is needed in Eq. (8.52). Each *input* x_i does feed both hidden nodes, but the inputs are constants of the trace and we never need their adjoints. Give the network a second output node and z_j^1 would reach the cost along two paths, the adjoints from the two output nodes would have to be added at each hidden node, and Eq. (8.52) would acquire the sum $\sum_k \delta_k^2 w_{jk}^2$ – the fan-out rule of Section 4.14.4, exactly as the two branches of Figure 5.5 were summed when the sweep reached v_2. That sum is the only thing Section 8.8.3 adds.

The program.

The program below is the training program of Section 8.8.1 with the array shapes changed and nothing else: the design matrix has n rows and two columns, $\boldsymbol{W}^1$ is 2×2, $\boldsymbol{W}^2$ is 2×1, and every line of `backward` is one of Eqs. (8.48), (8.54) and (8.55) applied to all samples at once, with `X.T @ delta1` summing the outer products $\boldsymbol{x}_i(\boldsymbol{\delta}_i^1)^T$ over the sample. The output node is linear, as in the scalar program, so that the network can fit the target $y = f(x_1,x_2) = x_1^2 + 3x_1x_2 + x_2^2 + 5$ on $n = 100$ points drawn uniformly from the unit square, a function that ranges

over $[5, 10]$. Before training, the nine hand-derived derivatives are compared with `jax.grad` of the same forward pass.

```
import numpy as np
import jax, jax.numpy as jnp
jax.config.update("jax_enable_x64", True)

def sigmoid(z): return 1.0/(1.0+np.exp(-z))

def forward(X, W1, b1, W2, b2):
    Z1 = X @ W1 + b1; A1 = sigmoid(Z1)         # hidden layer, samples in rows
    Z2 = A1 @ W2 + b2; A2 = Z2                  # linear output node
    return A1, A2

def backward(X, y, W1, b1, W2, b2):
    n = X.shape[0]
    A1, A2 = forward(X, W1, b1, W2, b2)
    delta2 = (A2 - y)/n                         # output error, f' = 1
    # delta_j^1 = delta^2 w_j^2 sigma'(z_j^1)
    delta1 = (delta2 @ W2.T) * A1*(1 - A1)
    dW2 = A1.T @ delta2;  db2 = delta2.sum(axis=0)
    dW1 = X.T @ delta1;   db1 = delta1.sum(axis=0)
    return dW1, db1, dW2, db2, 0.5*np.sum((A2 - y)**2)/n

def cost(params, X, y):                         # the same model for jax.grad
    W1, b1, W2, b2 = params
    A1 = jax.nn.sigmoid(X @ W1 + b1)
    return 0.5*jnp.mean((A1 @ W2 + b2 - y)**2)

rng = np.random.default_rng(41)
n = 100
X = rng.uniform(0, 1, size=(n, 2))
y = (X[:, 0]**2 + 3*X[:, 0]*X[:, 1] + X[:, 1]**2 + 5).reshape(-1, 1)

# w_{ij}^1, i input, j hidden
W1 = rng.standard_normal((2, 2)); b1 = np.zeros(2) + 0.01
W2 = rng.standard_normal((2, 1)); b2 = np.zeros(1) + 0.01   # w_{j}^2

g_hand = backward(X, y, W1, b1, W2, b2)[:4]
params = [jnp.array(p) for p in (W1, b1, W2, b2)]
g_jax = jax.grad(cost)(params, X, y)
print("max |hand - jax.grad| over the nine derivatives:",
      max(float(jnp.max(jnp.abs(h - j))) for h, j in zip(g_hand, g_jax)))

gamma = 0.5
for i in range(5001):
    dW1, db1, dW2, db2, C = backward(X, y, W1, b1, W2, b2)
    if i % 1000 == 0: print(f"iteration {i:4d}: cost = {C:.5f}")
    W1 -= gamma*dW1; b1 -= gamma*db1; W2 -= gamma*dW2; b2 -= gamma*db2
print("W1 =\n", W1.round(3), "\nb1 =", b1.round(3))
print("W2 =", W2.ravel().round(3), " b2 =", b2.round(3))
```

```
max |hand - jax.grad| over the nine derivatives: 1.7763568394002505e-15
iteration    0: cost = 22.76355
iteration 1000: cost = 0.00113
iteration 2000: cost = 0.00082
iteration 3000: cost = 0.00082
iteration 4000: cost = 0.00082
iteration 5000: cost = 0.00081
W1 =
 [[2.524 1.048]
 [2.558 1.052]]
b1 = [-4.123 -1.965]
```

```
W2 = [5.084 2.981]  b2 = [4.454]
```

The nine derivatives agree with reverse-mode automatic differentiation to 2×10^{-15}, and two hidden nodes bring the mean squared error of the fit down to 1.6×10^{-3}, a root-mean-square error of 0.04 on targets between 5 and 10. The reader who compares this listing with the class of Section 8.13 will find the same five lines in its `_backward` method, inside a loop over layers.

Machine learning connection. The three examples of this section are the whole of backpropagation in miniature, and the order in which they were presented is the order in which the general equations are usually misunderstood. The scalar neuron establishes that the error $\delta=\partial C/\partial z$ is the natural intermediate quantity and that a weight's gradient is that error times the weight's input. The scalar chain establishes the recursion $\delta^{l}=\delta^{l+1}w^{l+1}f'(z^{l})$ and that nothing is recomputed. The 2–2–1 network establishes which index goes where: the error carries the index of the receiving node, the input activation the index of the sending node, and the weight gradient has the shape of the weight matrix. What remains is the sum over receiving nodes when a node feeds several, and that is Eq. (8.64).

8.8.3 The general case: dense layers

Everything follows from the chain rule (1.51) and from two elementary derivatives of Eq. (8.18),

$$\frac{\partial z_j^l}{\partial w_{ij}^l}=a_i^{l-1},\qquad \frac{\partial z_j^l}{\partial a_i^{l-1}}=w_{ij}^l, \tag{8.57}$$

together with the derivative of the activation, $\partial a_j^l/\partial z_j^l=f'(z_j^l)$, which for the sigmoid is $a_j^l(1-a_j^l)$ by Eq. (8.22).

The output layer.

Take the squared-error cost (8.32) and specialise to $l=L$. Differentiating with respect to a weight in the last layer,

$$\frac{\partial C}{\partial w_{ij}^L}=\left(a_j^L-y_j\right)\frac{\partial a_j^L}{\partial w_{ij}^L}=\left(a_j^L-y_j\right)\frac{\partial a_j^L}{\partial z_j^L}\frac{\partial z_j^L}{\partial w_{ij}^L}=\left(a_j^L-y_j\right)f'\left(z_j^L\right)a_i^{L-1}. \tag{8.58}$$

The structure of the result invites a definition. Introduce the *error* of node j in the output layer,

$$\delta_j^L=f'\left(z_j^L\right)\frac{\partial C}{\partial a_j^L},\qquad \text{or in vector form}\qquad \boldsymbol{\delta}^L=f'\left(\boldsymbol{z}^L\right)\circ\frac{\partial C}{\partial \boldsymbol{a}^L}, \tag{8.59}$$

where $\circ$ is the Hadamard product of Section 1.2. With it, Eq. (8.58) becomes simply

$$\frac{\partial C}{\partial w_{ij}^L}=\delta_j^L a_i^{L-1}. \tag{8.60}$$

Equation (8.59) is worth reading rather than merely recording. The second factor measures how fast the cost changes with the jth output: if the cost hardly depends on output j, then δ_j^L is small, which is what one would want. The first factor measures how fast the activation function is changing at the value z_j^L actually attained: a saturated unit, with $f'\approx0$, contributes

almost nothing however wrong it is. That second observation is the seed of the vanishing-gradient problem.

The error is the bias gradient.

Note that δ_j^L can be written as

$$\delta_j^L = \frac{\partial C}{\partial z_j^L} = \frac{\partial C}{\partial a_j^L}\frac{\partial a_j^L}{\partial z_j^L}, \tag{8.61}$$

and since $\partial z_j^L / \partial b_j^L = 1$ from Eq. (8.18),

$$\delta_j^L = \frac{\partial C}{\partial b_j^L}. \tag{8.62}$$

The error of a node *is* the derivative of the cost with respect to that node's bias. This is not a coincidence but a consequence of the bias entering z additively, and it is why the same symbol serves both purposes.

Propagating the error backwards.

It remains to obtain $\boldsymbol{\delta}^l$ for a general layer. Define $\delta_j^l = \partial C / \partial z_j^l$ as before, and express it through the layer above. Since C depends on z_j^l only through the activations z_k^{l+1} of the next layer, the chain rule gives

$$\delta_j^l = \sum_k \frac{\partial C}{\partial z_k^{l+1}}\frac{\partial z_k^{l+1}}{\partial z_j^l} = \sum_k \delta_k^{l+1}\frac{\partial z_k^{l+1}}{\partial z_j^l}. \tag{8.63}$$

From $z_k^{l+1} = \sum_i w_{ik}^{l+1} a_i^l + b_k^{l+1}$ and $a_i^l = f(z_i^l)$ we obtain $\partial z_k^{l+1} / \partial z_j^l = w_{jk}^{l+1} f'(z_j^l)$, and therefore

$$\boxed{\delta_j^l = \left(\sum_k \delta_k^{l+1} w_{jk}^{l+1}\right) f'\left(z_j^l\right), \qquad \boldsymbol{\delta}^l = \left(\boldsymbol{W}^{l+1}\boldsymbol{\delta}^{l+1}\right) \circ f'\left(\boldsymbol{z}^l\right).} \tag{8.64}$$

This is the backpropagation equation. The error at a layer is the error at the layer above, pulled back through the transpose of the weights and modulated by the local derivative of the activation. Figure 8.13 draws it next to the forward pass of Figure 8.6: the same graph, the same weights, the arrows reversed.

8.8.4 The four equations of backpropagation

Collecting the results gives the four equations on which everything rests:

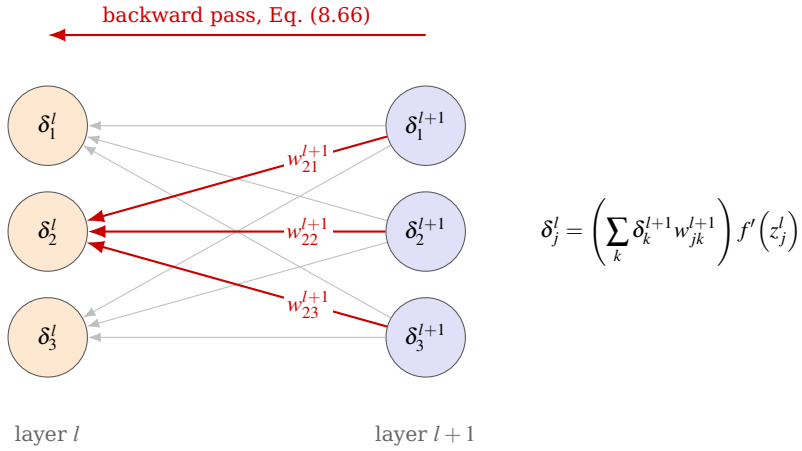


Fig. 8.13 The backward pass of Eq. (8.64), drawn on the same network as Fig. 8.6 but with the arrows reversed. The error δ_j^l of node j in layer l receives a contribution from *every* node k of layer $l+1$, weighted by the same w_{jk}^{l+1} that carried a_j^l forward, and the sum is then modulated by the local derivative $f'(z_j^l)$. Comparing with Fig. 8.6 shows what backpropagation is: the identical graph traversed in the opposite direction, with the transpose of each weight matrix in place of the matrix itself. A saturated node, one with $f'(z_j^l) \approx 0$, blocks the flow entirely, which is the mechanism of Section 8.9.

$$\boldsymbol{\delta}^L = f'\left(\boldsymbol{z}^L\right) \circ \frac{\partial C}{\partial \boldsymbol{a}^L}, \tag{8.65}$$

$$\boldsymbol{\delta}^l = \left(\boldsymbol{W}^{l+1}\boldsymbol{\delta}^{l+1}\right) \circ f'\left(\boldsymbol{z}^l\right), \tag{8.66}$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l, \tag{8.67}$$

$$\frac{\partial C}{\partial w_{ij}^l} = \delta_j^l a_i^{l-1}. \tag{8.68}$$

Equation (8.65) starts the recursion at the output, (8.66) carries it backwards through the network, and (8.67) and (8.68) convert the errors into the gradients we actually want. Every quantity on the right is already available: the $\boldsymbol{z}^l$ and $\boldsymbol{a}^l$ were computed during the forward pass and need only be stored, and f' is a cheap element-wise function.

A simplification worth knowing.

For the two natural pairings of output activation and loss, the first equation collapses. With a linear output layer and the half-squared error (8.32), $f' = 1$ and $\partial C/\partial a_j^L = a_j^L - y_j$, so

$$\boldsymbol{\delta}^L = \boldsymbol{a}^L - \boldsymbol{y}. \tag{8.69}$$

With a softmax output and the multiclass cross entropy (5.28), the derivative of the softmax and the derivative of the logarithm cancel – the same cancellation met in the notebox of Section 5.3 – and Eq. (8.69) holds again. In both cases the output error is simply prediction minus target, with no factor of f' to shrink it. This is the deeper reason for pairing the softmax with the cross entropy, and it is why the implementation of Section 8.13 treats both heads with one line of code.

Cost.

Counting operations, Eq. (8.66) costs one matrix-vector product per layer, exactly as the forward pass does. The whole gradient with respect to *all* parameters therefore costs about twice a forward pass, independently of the number of parameters. This is precisely the claim made for reverse-mode automatic differentiation in Section 4.14, and backpropagation is that algorithm specialised to a layered network. Computing the same gradient by finite differences, Eq. (4.102), would require one forward pass per parameter; for a network with 10^6 weights the difference is between a second and a fortnight.

8.8.5 The algorithm

Putting it together, training a feed-forward network proceeds as follows.

Set-up.

1. Fix the architecture: the number of inputs and outputs, the number of hidden layers and the number of nodes in each.
2. Choose activation functions for the hidden layers and for the output layer, the latter dictated by the range of the target.
3. Choose the cost function, again dictated by the distribution of the target, and any regularisation terms with their hyperparameters.
4. Choose the optimiser from Chapter 4 – plain gradient descent, momentum, RMSProp, Adam – and its learning rate.
5. Initialise the weights and biases, as discussed in Section 8.10.

Each iteration.

1. *Forward pass.* Set $\boldsymbol{a}^0 = \boldsymbol{x}$ and compute $\boldsymbol{z}^l$ and $\boldsymbol{a}^l = f(\boldsymbol{z}^l)$ for $l = 1, 2, \ldots, L$ by Eq. (8.20), storing every $\boldsymbol{z}^l$ and $\boldsymbol{a}^l$.
2. *Output error.* Compute $\boldsymbol{\delta}^L$ from Eq. (8.65), or from Eq. (8.69) for the natural pairings.
3. *Backward pass.* For $l = L-1, L-2, \ldots, 1$ compute $\boldsymbol{\delta}^l$ from Eq. (8.66).
4. *Gradients.* Form $\partial C/\partial w^l_{ij} = \delta^l_j a^{l-1}_i$ and $\partial C/\partial b^l_j = \delta^l_j$.
5. *Update.* With plain gradient descent and learning rate γ,

$$w^l_{ij} \leftarrow w^l_{ij} - \gamma \delta^l_j a^{l-1}_i, \qquad b^l_j \leftarrow b^l_j - \gamma \delta^l_j, \tag{8.70}$$

   or substitute any of the schemes of Chapter 4.

In practice steps 1–5 are applied to a minibatch rather than a single sample, as in Section 4.7, and one pass over all minibatches is an epoch.

Machine learning connection. Always verify a hand-written backpropagation implementation against finite differences before trusting it. Perturb one weight by $h \approx 10^{-6}$, evaluate the cost twice, and compare the central difference (4.102) with the analytical gradient; the relative discrepancy should be around 10^{-6} or smaller. A wrong gradient does not usually announce itself – the network still trains, just to the wrong place, or trains slightly worse than it should – and this check takes minutes to write. The imple-

mentation of Section 8.13 passes it at 10^{-6} to 10^{-8} for every activation function and both output heads, and those numbers are quoted there precisely so that a reader reimplementing it has something to compare against.

8.9 Vanishing and exploding gradients

Backpropagation as derived above is correct, and for deep networks it does not work. Understanding why occupied the field for two decades and the explanation is contained in Eq. (8.66).

Unrolling the recursion from the output down to layer l,

$$\boldsymbol{\delta}^{l} = \left[\prod_{k=l}^{L-1} \boldsymbol{W}^{k+1} \operatorname{diag}\left(f'(\boldsymbol{z}^{k})\right)\right] \boldsymbol{\delta}^{L}, \tag{8.71}$$

so the gradient reaching layer l is a product of $L-l$ factors. A product of many numbers each smaller than one shrinks geometrically; a product of many numbers each larger than one grows geometrically. Neither behaviour is survivable.

In the first case the gradients reaching the early layers are so small that gradient descent leaves those weights essentially unchanged, and training never converges to a good solution. This is the *vanishing gradients* problem. In the second the updates are so large that the iteration diverges: the *exploding gradients* problem, encountered particularly in recurrent networks where the same weight matrix is applied at every time step. More generally, deep networks suffer from *unstable* gradients, with different layers learning at widely different speeds.

The sigmoid is the principal culprit.

Consider the factors in Eq. (8.71) for the logistic activation. By Eq. (8.22) the derivative σ' never exceeds $\tfrac{1}{4}$, and it approaches zero rapidly once $|z|$ is large, so a saturated unit contributes almost nothing. Even in the best case, with every unit at its most sensitive, ten layers multiply ten factors of at most $\tfrac{1}{4}$, giving $4^{-10} \approx 10^{-6}$: the first layer receives a gradient a millionth of the size of the last. Sigmoid networks of any depth are, by construction, almost untrainable at their input end.

Glorot and Bengio identified a second contribution in 2010. With the then standard initialisation – weights drawn from $\mathcal{N}(0,1)$ – the variance of the outputs of each layer is considerably larger than the variance of its inputs. Going forward through the network the variance grows layer by layer until the activations saturate at the top, and a saturated activation has a vanishing derivative. The logistic function makes matters worse still by having mean $\tfrac{1}{2}$ rather than zero, which is why $\tanh$, with mean zero, behaves noticeably better in deep networks.

The two repairs.

The first is the activation function. ReLU does not saturate for positive arguments and its derivative is exactly 1 there, so the factors in Eq. (8.71) do not shrink the gradient at all along the active path. This single change is most of the reason deep networks became trainable, and the ReLU family of Section 8.6 is the standard choice for that reason rather than for any biological one.

The second is initialisation, and it is the subject of the next section.

Figure 8.14 measures the effect directly. For each network the Frobenius norm of the weight gradient is plotted layer by layer after a single backward pass. With sigmoid activations the norm falls by orders of magnitude from the output towards the input, and the deeper the network the more severe the attenuation – at eight hidden layers the first layer receives a gradient several orders of magnitude smaller than the last, so it is effectively frozen. With ReLU the profile is far flatter and the early layers continue to receive a usable signal. This is Eq. (8.71) made visible.

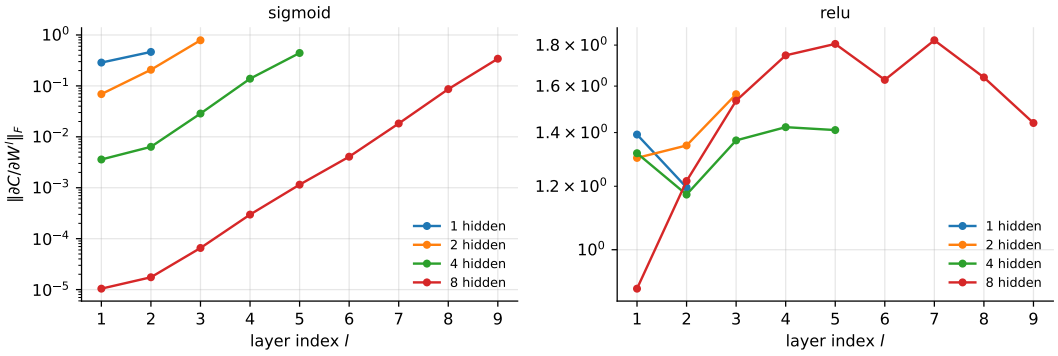


Fig. 8.14 Norm of the weight gradient at each layer after one backward pass, for networks of 1, 2, 4 and 8 hidden layers of 30 units on the digits data. Left: sigmoid activations, showing severe attenuation towards the input. Right: ReLU, which preserves the gradient far better.

8.10 Weight initialisation

The argument of Glorot and Bengio is that for a signal to propagate properly we need the variance of the outputs of each layer to equal the variance of its inputs in the forward direction, and the variance of the gradients to be preserved in the backward direction.

Suppose the inputs to a layer are independent with variance $\mathrm{Var}(a)$, and the M_{l-1} weights are independent with mean zero and variance $\mathrm{Var}(w)$. Then by the rules of Section 2.3, the activation $z_j = \sum_i w_{ij} a_i$ has variance

$$\mathrm{Var}(z) = M_{l-1}\mathrm{Var}(w)\mathrm{Var}(a), \tag{8.72}$$

so $\mathrm{Var}(z) = \mathrm{Var}(a)$ requires $\mathrm{Var}(w) = 1/M_{l-1}$. Repeating the argument for the backward pass, where Eq. (8.66) sums over the M_l nodes of the next layer, gives $\mathrm{Var}(w) = 1/M_l$. The two conditions cannot both hold unless the layers have equal width, and *Xavier* or *Glorot* initialisation compromises between them,

$$\mathrm{Var}(w) = \frac{2}{M_{l-1} + M_l}, \tag{8.73}$$

drawing the weights from a Gaussian or a uniform distribution with this variance.

For ReLU the argument needs one correction. Since ReLU zeroes half of its inputs on average, it halves the variance, and compensating for this gives *He* initialisation,

$$\mathrm{Var}(w) = \frac{2}{M_{l-1}}, \tag{8.74}$$

which is the appropriate choice for the whole ReLU family. Biases are initialised to zero, or to a small positive constant such as 0.01 when using ReLU, so that units start on the active side of the kink.

It is worth appreciating how much rests on Eq. (8.72). Initialising all weights to zero would make every unit in a layer compute the same thing and receive the same gradient, so they would remain identical forever – the symmetry of Section 8.7 must be broken by the initialisation. Initialising them too large saturates the activations; too small and the signal dies out. The permissible window is narrow, and Eqs. (8.73) and (8.74) locate it.

8.11 Classification with neural networks

Classification requires only that the output layer and the loss be chosen correctly, and both were determined in Chapter 5.

For K classes the output layer has K nodes and the softmax activation (5.27),

$$a_k^L = \frac{\exp\left(z_k^L\right)}{\sum_{l=0}^{K-1} \exp\left(z_l^L\right)}, \tag{8.75}$$

so that the outputs are non-negative and sum to one and may be read as class probabilities. As in Section 5.6 the exponentials must be computed after subtracting the largest score, Eq. (5.30), or they overflow. The loss is the multiclass cross entropy (5.28) with one-hot targets, and by the cancellation noted after Eq. (8.69) the output error is simply $\boldsymbol{\delta}^L = \boldsymbol{a}^L - \boldsymbol{y}$.

The whole apparatus therefore reduces to multinomial logistic regression applied to the *learned* features $\boldsymbol{a}^{L-1}$ rather than to the raw inputs. A classification network is Chapter 5 with a representation learner bolted on in front, and this is the most useful way to think about what the hidden layers are for.

8.12 Regularisation and hyperparameters

A network with more parameters than data points can fit the training set exactly, so the bias-variance considerations of Section 2.10 apply with full force and some form of control is essential.

Weight penalties.

Adding $\lambda \|\boldsymbol{\Theta}\|_2^2$ to Eq. (8.31) is the Ridge penalty of Section 3.8 under the name *weight decay*; it contributes $2\lambda \boldsymbol{W}^l$ to each weight gradient and nothing to the bias gradients, since biases are not penalised for the reason given in Section 3.13. An ℓ_1 penalty produces sparse weights exactly as in Section 3.9.

Early stopping.

Monitor the cost on a validation set and stop when it begins to rise while the training cost still falls. This is the criterion of Section 4.7.8, and for networks it is the single most effective and cheapest regulariser available.

Dropout.

During training, delete each unit independently with probability p, typically 0.5 for hidden layers, and rescale the survivors; at test time use the whole network. Each minibatch therefore trains a different thinned network, and the final model behaves like an average over exponentially many of them – which is the variance reduction of Section 7.8 obtained without training an ensemble. Dropout also prevents units from co-adapting, since no unit can rely on any particular other unit being present.

Batch normalisation.

Normalise the activations of a layer across the minibatch to zero mean and unit variance, then rescale by two learned parameters. This keeps the inputs of each layer in the well-behaved region of its activation function and hence attacks the vanishing-gradient problem directly; it also acts as a mild regulariser, since the statistics depend on the composition of the minibatch. The connection to Section 4.5 is exact: standardising the inputs to a layer improves the conditioning of the optimisation problem that layer poses, and Eq. (4.26) says what that is worth.

Gradient clipping.

If the gradient norm exceeds a threshold, rescale it to that threshold. This does not cure exploding gradients but it prevents a single bad minibatch from destroying the parameters, and it is standard practice in recurrent networks.

Hyperparameters.

The number of hidden layers and their widths, the learning rate and its schedule, the batch size, λ, the dropout rate and the activation function are all hyperparameters, and they are chosen by the cross-validation of Section 2.12 on a validation set – never on the test set. Two rules of thumb survive contact with practice: the learning rate is by far the most important, and it is usually better to build a network somewhat too large and regularise it than to search for exactly the right size.

8.13 A neural network from scratch

We now implement the whole of Sections 8.5 to 8.10. The code follows the equations line for line, uses the batched form (8.21) so that samples are rows throughout, and handles both regression and classification.

```
import numpy as np


def sigmoid(z):
    """Numerically stable logistic function, Eq. (8.sigmoid)."""
    out = np.empty_like(z, dtype=float)
    p, n = z >= 0, z < 0
    out[p] = 1.0 / (1.0 + np.exp(-z[p]))
    e = np.exp(z[n]); out[n] = e / (1.0 + e)
```

```
    return out

def sigmoid_prime(z):  s = sigmoid(z); return s * (1 - s)
def relu(z):           return np.maximum(0.0, z)            # Eq. (8.relu)
def relu_prime(z):     return (z > 0).astype(float)
def leaky_relu(z, a=0.01):        return np.where(z > 0, z, a * z)
def leaky_relu_prime(z, a=0.01):  return np.where(z > 0, 1.0, a)
def elu(z, a=1.0):                                          # Eq. (8.elu)
    return np.where(z > 0, z, a * (np.exp(np.minimum(z, 0)) - 1))
def elu_prime(z, a=1.0):
    return np.where(z > 0, 1.0, a * np.exp(np.minimum(z, 0)))
def tanh_(z):          return np.tanh(z)
def tanh_prime(z):     return 1 - np.tanh(z)**2
def identity(z):       return z
def identity_prime(z): return np.ones_like(z)

def softplus(z):       return np.log1p(np.exp(-np.abs(z))) + np.maximum(z, 0)
def softplus_prime(z): return sigmoid(z)

def gelu(z):
    """Exact GELU, Eq. (8.gelu); erf comes from scipy."""
    from scipy.special import erf
    return z * 0.5 * (1.0 + erf(z / np.sqrt(2.0)))

def gelu_prime(z):
    from scipy.special import erf
    Phi = 0.5 * (1.0 + erf(z / np.sqrt(2.0)))
    phi = np.exp(-0.5 * z**2) / np.sqrt(2.0 * np.pi)      # the normal density
    return Phi + z * phi                                   # d/dz [z Phi(z)]

def gelu_tanh(z):
    """The approximation of Eq. (8.geluapprox) used by most frameworks."""
    return 0.5 * z * (1 + np.tanh(np.sqrt(2 / np.pi) * (z + 0.044715 * z**3)))

def swish(z):          return z * sigmoid(z)               # Eq. (8.swish)
def swish_prime(z):
    s = sigmoid(z); return s + z * s * (1 - s)

def mish(z):                                               # Eq. (8.mish)
    sp = np.log1p(np.exp(-np.abs(z))) + np.maximum(z, 0)
    return z * np.tanh(sp)

def mish_prime(z, h=1e-6):
    return (mish(z + h) - mish(z - h)) / (2 * h)           # numerical is adequate

def softmax(z):
    """Eq. (8.softmax), with the shift of Eq. (5.softmaxstable)."""
    e = np.exp(z - np.max(z, axis=1, keepdims=True))
    return e / np.sum(e, axis=1, keepdims=True)

ACT = {"sigmoid": (sigmoid, sigmoid_prime), "relu": (relu, relu_prime),
       "leaky_relu": (leaky_relu, leaky_relu_prime), "elu": (elu, elu_prime),
       "tanh": (tanh_, tanh_prime), "identity": (identity, identity_prime),
       "softplus": (softplus, softplus_prime), "gelu": (gelu, gelu_prime),
       "swish": (swish, swish_prime), "mish": (mish, mish_prime)}
```

The network itself is short. Note in particular `_backward`, which is Eqs. (8.65)–(8.68) transcribed.

```
class NeuralNetwork:
    """Fully connected feed-forward network trained by backpropagation.

    layer_sizes : e.g. [64, 50, 10] for 64 inputs, one hidden layer of 50
                  units and 10 outputs.
```

```
    task        : "classification" (softmax + cross entropy) or
                  "regression" (linear output + half-squared error).
    """

    def __init__(self, layer_sizes, hidden_activation="sigmoid",
                 task="classification", gamma=0.1, lmbd=0.0, epochs=100,
                 batch_size=32, rng=None):
        self.sizes, self.task = layer_sizes, task
        self.f, self.fp = ACT[hidden_activation]
        self.gamma, self.lmbd = gamma, lmbd
        self.epochs, self.batch = epochs, batch_size
        self.rng = np.random.default_rng(0) if rng is None else rng
        self._init_parameters(hidden_activation)

    def _init_parameters(self, act):
        """Xavier (8.xavier) or He (8.he) initialisation, by activation."""
        self.W, self.b = [], []
        for i in range(len(self.sizes) - 1):
            nin, nout = self.sizes[i], self.sizes[i + 1]
            s = np.sqrt(2.0 / nin) if act in ("relu", "leaky_relu", "elu") \
                else np.sqrt(1.0 / nin)
            self.W.append(self.rng.normal(0, s, (nin, nout)))
            self.b.append(np.zeros(nout) + 0.01)

    def _forward(self, X):
        """Eq. (8.forwardbatch); keeps every z and a for the backward pass."""
        a, z = [X], []
        for l in range(len(self.W)):
            zl = a[-1] @ self.W[l] + self.b[l]
            z.append(zl)
            if l == len(self.W) - 1:
                a.append(softmax(zl) if self.task == "classification" else zl)
            else:
                a.append(self.f(zl))
        return a, z

    def _backward(self, X, Y):
        """The four equations (8.bp1)-(8.bp4)."""
        n = X.shape[0]
        a, z = self._forward(X)

        # With softmax + cross entropy, and with a linear output under the
        # half-squared error, the output error is the same expression:
        delta = (a[-1] - Y) / n                       # Eq. (8.deltaLsimple)

        gW, gb = [None] * len(self.W), [None] * len(self.b)
        for l in range(len(self.W) - 1, -1, -1):
            gW[l] = a[l].T @ delta + self.lmbd * self.W[l]   # Eq. (8.bp4)
            gb[l] = np.sum(delta, axis=0)                    # Eq. (8.bp3)
            if l > 0:
                delta = (delta @ self.W[l].T) * self.fp(z[l - 1])  # Eq. (8.bp2)
        return gW, gb

    def fit(self, X, y):
        X = np.asarray(X, dtype=float)
        if self.task == "classification":
            self.classes_ = np.unique(y)
            idx = {c: i for i, c in enumerate(self.classes_)}
            Y = np.zeros((len(y), len(self.classes_)))        # one-hot
            Y[np.arange(len(y)), [idx[c] for c in y]] = 1
        else:
            Y = np.asarray(y, dtype=float).reshape(-1, 1)

        n = X.shape[0]
```

```
        self.loss_ = []
        for _ in range(self.epochs):
            order = self.rng.permutation(n)                    # shuffle, Sec.
    4.practicaltips
            for s in range(0, n, self.batch):
                b = order[s:s + self.batch]
                gW, gb = self._backward(X[b], Y[b])
                for l in range(len(self.W)):                   # Eq. (8.update)
                    self.W[l] -= self.gamma * gW[l]
                    self.b[l] -= self.gamma * gb[l]
            self.loss_.append(self.cost(X, Y))
        return self

    def cost(self, X, Y):
        o = self._forward(X)[0][-1]
        if self.task == "classification":
            return float(-np.mean(np.sum(Y * np.log(np.clip(o, 1e-12, 1)), axis=1)))
        return float(0.5 * np.mean((o - Y)**2))                # Eq. (8.mse)

    def predict_proba(self, X):
        return self._forward(np.asarray(X, dtype=float))[0][-1]

    def predict(self, X):
        o = self.predict_proba(X)
        return (self.classes_[np.argmax(o, axis=1)]
                if self.task == "classification" else o.ravel())
```

Verifying the gradient.

Before using the network for anything we check `_backward` against finite differences, as urged in the notebox of Section 8.8.5.

```
def gradient_check(net, X, Y, h=1e-6, n_samples=15):
    """Compare backpropagation with the central difference (4.gradcheck)."""
    rng = np.random.default_rng(0)
    gW, gb = net._backward(X, Y)
    worst = 0.0
    for l in range(len(net.W)):
        for _ in range(n_samples):
            i = rng.integers(net.W[l].shape[0]); j = rng.integers(net.W[l].shape[1])
            net.W[l][i, j] += h; c1 = net.cost(X, Y)
            net.W[l][i, j] -= 2 * h; c2 = net.cost(X, Y)
            net.W[l][i, j] += h
            num = (c1 - c2) / (2 * h)
            worst = max(worst, abs(num - gW[l][i, j])
                        / (abs(num) + abs(gW[l][i, j]) + 1e-12))
    return worst
```

Run over a three-hidden-layer network for every activation function, and over regression as well as classification heads, the worst relative discrepancy is

```
classification, sigmoid                    1.38e-06
classification, tanh                       1.11e-08
classification, relu                       4.46e-07
classification, leaky_relu                 5.80e-07
classification, elu                        2.08e-08
classification, softplus                   1.48e-06
classification, gelu                       3.25e-07
classification, swish                      3.33e-08
classification, mish                       7.29e-08
regression, tanh                           3.55e-08
```

```
regression, relu, 2 hidden layers          1.19e-08
```

which is the accuracy of the difference scheme itself. The implementation of Eqs. (8.65)–(8.68) is correct.

8.14 Examples

Handwritten digits.

The digits data set bundled with `scikit-learn` contains 1797 images of 8×8 pixels, giving 64 inputs and 10 classes. We standardise the features as urged in Section 1.5, hold out a fifth for testing, and train a network with a single hidden layer of 50 units.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

digits = load_digits()
X = StandardScaler().fit_transform(digits.data)
X_train, X_test, y_train, y_test = train_test_split(X, digits.target,
                                                    test_size=0.2, random_state=42)

for activation in ["sigmoid", "relu"]:
    net = NeuralNetwork([64, 50, 10], activation, "classification",
                        gamma=0.1, lmbd=1e-4, epochs=60, batch_size=32,
                        rng=np.random.default_rng(2024)).fit(X_train, y_train)
    print(f"{activation:8s}: train {np.mean(net.predict(X_train) == y_train):.4f}  "
          f"test {np.mean(net.predict(X_test) == y_test):.4f}  "
          f"final loss {net.loss_[-1]:.4f}")
```

```
sigmoid : train 0.9937  test 0.9722  final loss 0.0585
relu    : train 1.0000  test 0.9722  final loss 0.0059
```

For comparison, `sklearn.neural_network.MLPClassifier` with the same architecture reaches 0.9778 on the same split. Our implementation is within one test sample of a mature library, which is the appropriate standard.

The two activations reach the same test accuracy by different routes. ReLU drives the training loss an order of magnitude lower and fits the training set exactly, while the sigmoid does not; that the test accuracies coincide means the extra fitting bought nothing, which is the bias-variance story of Section 2.10 in a network. With only one hidden layer the vanishing-gradient argument of Section 8.9 has little room to operate; its consequences appear when the depth grows, which is the subject of one of the exercises.

Figure 8.15 shows the training curves and the effect of capacity. On the left, ReLU drives the training cross entropy roughly an order of magnitude below the sigmoid within sixty epochs, and tanh falls between them – the ordering the derivatives of Figure 8.9 predict. The test accuracies quoted in the legend are nevertheless within one sample of one another, which is the reminder that a lower training loss is not the objective. On the right, accuracy against the number of hidden units shows the familiar shape: rapid improvement up to about twenty units, then a plateau on the test set while the training accuracy continues to unity. The gap between the two curves is the variance term of Eq. (2.52).

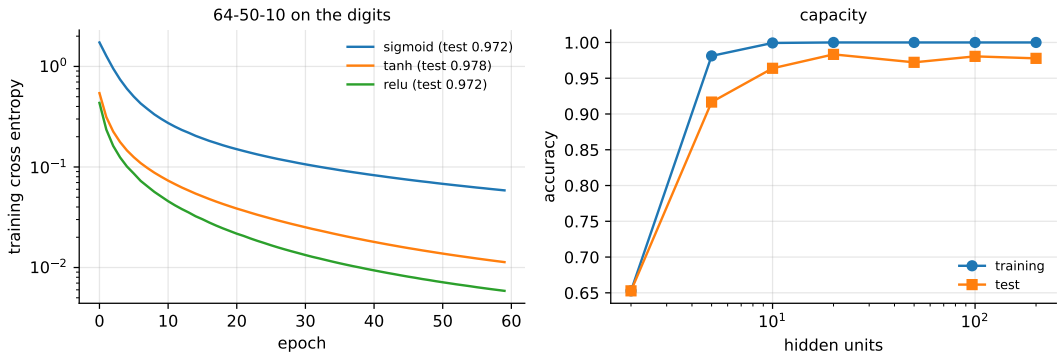


Fig. 8.15 Left: training cross entropy against epoch for three hidden activations on the digits data, with the resulting test accuracies. Right: training and test accuracy against the width of the single hidden layer.

Using established libraries.

For anything beyond a teaching example one uses a framework, which supplies automatic differentiation, GPU execution and a large catalogue of layers. In `scikit-learn`,

```
from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(hidden_layer_sizes=(50,), activation="relu",
                    alpha=1e-4, max_iter=500, random_state=1)
mlp.fit(X_train, y_train)
print(f"test accuracy {mlp.score(X_test, y_test):.4f}")
```

and in `PyTorch`, where the correspondence with our equations is visible:

```
import torch
import torch.nn as nn

model = nn.Sequential(nn.Linear(64, 50),   # Eq. (8.forwardbatch)
                      nn.ReLU(),           # Eq. (8.relu)
                      nn.Linear(50, 10))   # softmax folded into the loss
loss_fn = nn.CrossEntropyLoss()            # Eq. (5.multicrossentropy)
optimiser = torch.optim.Adam(model.parameters(), lr=1e-3)   # Sec. 4.adam

Xt = torch.tensor(X_train, dtype=torch.float32)
yt = torch.tensor(y_train, dtype=torch.long)
for epoch in range(100):
    optimiser.zero_grad()
    loss = loss_fn(model(Xt), yt)
    loss.backward()                        # this is Eqs. (8.bp1)-(8.bp4)
    optimiser.step()                       # this is Eq. (8.update)
```

The single call `loss.backward()` performs exactly the computation we derived by hand, obtained by reverse-mode automatic differentiation over the graph the forward pass recorded. Knowing what it does is the point of having derived it.

8.15 Limitations, and a top-down view

It is worth closing the exposition with some perspective, because the enthusiasm surrounding neural networks makes it easy to lose.

Deep networks are extraordinarily effective on data with strong local structure that can be exploited by an architecture – images, audio, text – and where very large labelled data sets exist. They are frequently *not* the best choice on the tabular data of Chapter 7, where gradient-boosted trees remain at least competitive and often better, while needing less tuning and no scaling. A network with no architectural prior adapted to the problem is a very flexible model with a great many hyperparameters, and flexibility is not free.

The specific limitations worth naming are these. Networks need large amounts of labelled data, and supervised deep learning has no good answer when labels are scarce. They are computationally expensive to train. They are largely uninterpretable: the variable-importance measures of Section 7.9 have no clean analogue, and a network offers no account of *why* it predicted what it did. They extrapolate unpredictably outside the range of the training data. They are sensitive to adversarial perturbations imperceptible to a human. And the optimisation is non-convex, so results depend on initialisation and on the random seed, which makes honest reporting harder than for any method in the preceding chapters.

None of this argues against neural networks; it argues for choosing the method to suit the problem, which has been the theme of this book throughout. Section 8.16 makes the point with numbers: on the Ising chain a network can represent the answer exactly with $2L$ hidden units and cannot find it from four hundred configurations, while the Lasso of Section 3.9, given features a physicist chose, reaches $R^2 = 0.99996$ on the same data.

8.16 An extended example: learning the Ising energy

Section 3.16 fitted the energy of a one-dimensional Ising chain,

$$H(\boldsymbol{s}) \;=\; -J \sum_{j=1}^{L} s_j s_{j+1}, \qquad s_j \in \{-1,+1\}, \quad s_{L+1} \equiv s_1, \tag{8.76}$$

by linear regression, and the Lasso recovered it almost exactly. It did so because someone had first replaced the L spins by the L^2 products $s_j s_k$, at which point Eq. (8.76) became a linear function of the inputs and the whole apparatus of Chapter 3 applied. The products were the physics. The regression only had to find which of them mattered.

This section removes that help. We hand a network the raw spins, with no products at all, and ask whether the interaction can be *discovered*. The example is a good closing test of the chapter because it separates three things that are easy to conflate: what a network can represent, what it can learn, and what it is worth relative to a linear model with well-chosen features. The answers here are, in order: everything, less than everything, and it depends entirely on how much data there is.

8.16.1 The linear model on raw spins is exactly powerless

Before asking what the network gains, we should know what it is being compared against, and the answer is unusually clean.

Proposition 8.7 (The energy is uncorrelated with every spin). *Let the spins $s_1, \dots, s_L$ be independent and uniform on $\{-1,+1\}$ and let H be given by Eq. (8.76). Then $\mathbb{E}[H\, s_j] = 0$ for every j, and the best linear predictor of H from $(s_1, \dots, s_L)$ is the constant $\mathbb{E}[H] = 0$.*

Proof. Fix j and let T be the map that flips the jth spin, $s_j \mapsto -s_j$, leaving the others alone. Since the spins are independent and uniform, T preserves the distribution. The energy con-

tains s_j only through $-J(s_{j-1}s_j + s_j s_{j+1})$, and both terms change sign under T; write $H = A + B$ where $B = -J(s_{j-1}s_j + s_j s_{j+1})$ and A does not involve s_j. Then

$$\mathbb{E}\,[H s_j] = \mathbb{E}\,[A s_j] + \mathbb{E}\,[B s_j]\,. \tag{8.77}$$

In the first term A and s_j are independent and $\mathbb{E}[s_j] = 0$, so it vanishes. In the second, $B s_j = -J(s_{j-1} + s_{j+1})$ after using $s_j^2 = 1$, and $\mathbb{E}[s_{j\pm1}] = 0$, so it vanishes too. Since the covariance matrix of the spins is the identity, the least-squares coefficient vector is $\mathbb{E}[H\boldsymbol{s}] = \mathbf{0}$. □

A linear model on raw spins is therefore not merely a weak model; it is provably equivalent to predicting the mean, and the same is true of Ridge and the Lasso, which can only shrink coefficients that are already zero in the population. Measured on the ten thousand configurations of Section 3.16:

```
max_j |corr(s_j, H)|                    : 0.0276
max_j |mean(H s_j)|                     : 0.1748
intercept                               : +0.0395
max_j |theta_j|                         : 0.1765
R^2                                     : 0.004280
```

Four thousandths of the variance, which is sampling noise around the zero the proposition predicts. Note what this is *not*: it is not a statement that the energy is hard to compute from the spins. It is a deterministic function of them, computable in L multiplications. The information is entirely present and entirely invisible to any method that looks at one feature at a time. That is what a nonlinear model is for.

8.16.2 A network that gets it exactly right

Section 8.4 showed that a one-hidden-layer network can approximate any continuous function on a compact set, and Theorem 8.2 quantified the width needed for a given accuracy. On this problem we can do better than approximate. The construction is short enough to write down.

Proposition 8.8 (An exact ReLU representation of the Ising energy). *For every L and every J there is a one-hidden-layer ReLU network with $2L$ hidden units, all first-layer weights in $\{-1, 0, +1\}$ and all first-layer biases zero, that equals $H(\boldsymbol{s})$ of Eq. (8.76) at every one of the 2^L spin configurations. Explicitly,*

$$H(\boldsymbol{s}) \;=\; JL \;-\; J \sum_{j=1}^{L} \Big[\, \sigma\big(s_j + s_{j+1}\big) + \sigma\big(-s_j - s_{j+1}\big) \,\Big], \qquad \sigma(z) = \max(0, z). \tag{8.78}$$

Proof. For $s, t \in \{-1, +1\}$ the sum $s + t$ is ± 2 when the spins agree and 0 when they differ, so $|s + t| - 1$ equals $+1$ in the first case and -1 in the second, which is exactly st. Hence $s_j s_{j+1} = |s_j + s_{j+1}| - 1$, and

$$H(\boldsymbol{s}) = -J \sum_{j=1}^{L} \big(\,|s_j + s_{j+1}| - 1\,\big) = JL - J \sum_{j=1}^{L} |s_j + s_{j+1}|\,. \tag{8.79}$$

Finally $|z| = \sigma(z) + \sigma(-z)$, which turns each absolute value into two ReLU units, giving Eq. (8.78) with $2L$ of them. The first-layer weight matrix has entries ± 1 in two positions of each column and zero elsewhere; the output weights are all $-J$ and the output bias is JL. □

This is worth comparing with Section 8.4 in three ways. The representation is exact rather than approximate, because the input set is finite – the network need only agree with H on the corners of the hypercube, and Eq. (8.79) happens to be a piecewise-linear function that agrees everywhere in between as well. The width $2L$ grows linearly in the input dimension rather than exponentially, because the target is a sum of terms each involving two coordinates; this is the same reason the Barron class of Theorem 8.3 escapes the curse of dimensionality, and H is a textbook member of it. And the ridge directions $\pm(e_j + e_{j+1})$ are precisely the interactions we are trying to discover: reading the first-layer weights of Eq. (8.78) recovers the topology of the chain. Evaluating the hand-built network on all ten thousand configurations:

```
    hidden units                                : 80
    distinct first-layer weights                : [-1.0, 0.0, 1.0]
    max |network(s) - H(s)|                     : 0.000e+00
    R^2                                         : 1.0000000000
```

Zero error, not small error.

```
import numpy as np

# Proposition 8.isingexact, written down rather than trained
W1 = np.zeros((L, 2 * L))
for j in range(L):
    W1[j, 2 * j], W1[(j + 1) % L, 2 * j] = 1.0, 1.0        # +s_j + s_{j+1}
    W1[j, 2 * j + 1], W1[(j + 1) % L, 2 * j + 1] = -1.0, -1.0
W2, b2 = np.full(2 * L, -Jtrue), Jtrue * L
predict = lambda S: np.maximum(S @ W1, 0.0) @ W2 + b2
```

8.16.3 Representable is not learnable

The network of Proposition 8.8 exists. Gradient descent has to find it, from the same 400 configurations Chapter 3 used, with no knowledge that the interaction is nearest-neighbour or even pairwise. One hidden layer, ReLU, mean squared error, Adam:

```
   epochs   R^2 train    R^2 test
       20    0.919057    0.187639
      100    1.000000    0.277651
      180    1.000000    0.277658
      260    1.000000    0.277658
      340    0.999990    0.282606
```

The training score reaches one after a hundred epochs and stays there; the test score reaches 0.28 and stops. This is overfitting in its purest form, and it is not cured by making the network bigger:

```
   hidden units   R^2 train    R^2 test
             10    0.957966   -0.599135
             25    1.000000   -0.055285
             50    1.000000    0.193549
            100    0.976503    0.185764
            200    0.999999    0.282703
            400    0.996122    0.315478
```

Width helps a little and monotonically, with no sign of the deterioration a naive reading of the bias-variance trade-off of Section 2.10 would predict – but 400 hidden units, five times the 80 that suffice for an exact fit, still reaches only 0.32. What is missing is data:

```
training rows   R^2 test
          100   -0.213211
          200   -0.088174
          400    0.283759
          800    0.610756
         1600    0.875358
         3200    0.952438
```

and with enough of it the network does find something very close to the truth. Given 8000 configurations, 400 hidden units and 1500 epochs:

```
R^2 train                                    : 0.998800
R^2 test                                     : 0.980170
RMS error in units of the energy quantum 2J : 0.4516
```

Figure 8.16 collects the three studies.

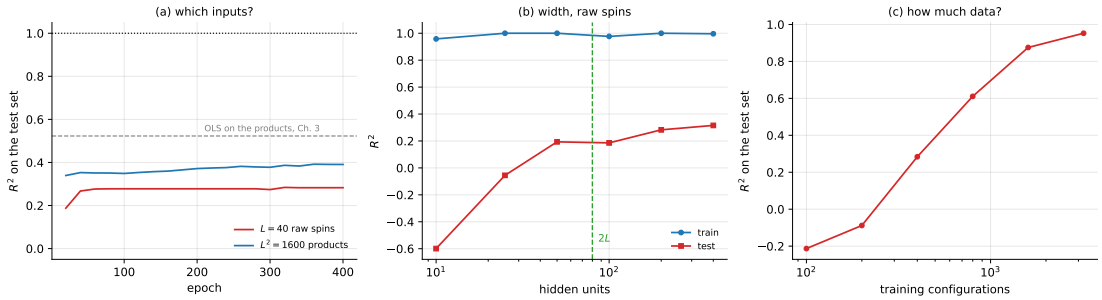


Fig. 8.16 Learning the Ising energy with a one-hidden-layer network. (a) Test R^2 against epoch for the $L = 40$ raw spins and for the $L^2 = 1600$ products of Section 3.16, both from 400 training configurations. (b) Width study on the raw spins; the dashed line marks the $2L$ units that suffice for the exact representation of Proposition 8.8. (c) Test R^2 against the number of training configurations: the limitation is data, not capacity.

The gap between $R^2 = 1$, which a network of 80 units achieves with the right weights, and $R^2 = 0.28$, which a network of 200 units achieves after being trained on 400 examples, is the whole distance between approximation theory and machine learning. Section 8.4 guarantees the former and says nothing at all about the latter. It is worth being explicit about why the guarantee is silent: the theorem quantifies over functions and asserts the existence of weights, whereas training is an optimisation over weights driven by a finite sample, and the sample is what fails here. With 40 inputs there are 780 pairwise interactions the network could believe in, and 400 configurations are not enough to rule out the 740 that are absent.

8.16.4 Against Chapter 3

Now the comparison the section was built for. All models below are fitted on the same 400 configurations and scored on the same 9600 held-out ones.

```
model                          R^2 on the 9600 test configurations
OLS,   L^2 outer products                    0.522863
Ridge, L^2 outer products                    0.522863
Lasso, L^2 outer products                    0.999963
network, L^2 outer products                  0.390631
OLS,   L raw spins                          -0.112870
network, L raw spins                         0.282703
```

```
exact network, L raw spins            1.000000
```

Read from the bottom up. A network can be perfect and a linear model on the same inputs cannot do better than the mean – there is the case for nonlinearity, and it is absolute. Trained rather than constructed, the same network reaches 0.28, which is better than the linear model on the same inputs by an infinite margin and worse than the Lasso on engineered features by a wide one. And a network *given* the engineered features scores 0.39, below the 0.52 of plain OLS on those features and far below the Lasso: told that the answer is linear, the network still has to rediscover that fact from data, and it spends its capacity on a flexibility the problem does not need.

The ordering is uncomfortable if one expects networks to dominate, and it is the honest result. Its lesson is not that networks are weak. It is that in the regime $n = 400$, $p = 1600$, with a truth that is sparse in a known basis, a penalised linear model is the right tool, and the penalty of Section 3.9 encodes knowledge – *most interactions are absent* – that a network trained from scratch has to buy with data. Figure 8.17 shows the same numbers, together with the network's score once it is given 8000 configurations instead of 400, which is where the ordering starts to reverse.

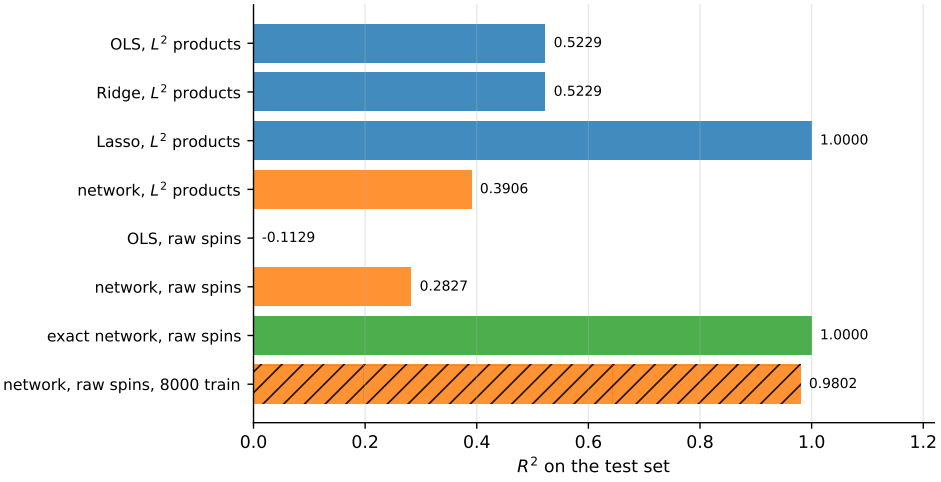


Fig. 8.17 Every model of Section 3.16 and this section on the same problem. Blue: linear models. Orange: trained networks. Green: the exact network of Proposition 8.8. All bars use 400 training configurations and the same 9600 test configurations except the hatched one, which uses 8000 training configurations and a separate held-out set of 2000. Negative scores are drawn as zero and labelled.

The same experiment in PyTorch, with an independent initialisation and an independent shuffle, agrees:

```
framework      R^2 train    R^2 test
numpy           0.999999    0.282703
PyTorch         0.996700    0.267799
```

```
import torch

model = torch.nn.Sequential(
    torch.nn.Linear(L, 200), torch.nn.ReLU(), torch.nn.Linear(200, 1)
).double()
opt = torch.optim.Adam(model.parameters(), lr=3e-3)
lossf = torch.nn.MSELoss()
for ep in range(400):
```

```
    perm = torch.randperm(ntr)
    for s in range(0, ntr, 40):
        b = perm[s:s + 40]
        opt.zero_grad()
        lossf(model(Xt[b]).squeeze(-1), yt[b]).backward()
        opt.step()
```

The two do not agree digit by digit and are not meant to. They agree on the conclusion, which is the only thing a cross-check can establish.

Machine learning connection. The exercise of this section – predicting the energy of a configuration of atoms or spins from the configuration alone – is the basis of machine-learned interatomic potentials, which are now a standard tool in computational chemistry and materials science. The economics are the same as here: a quantum-mechanical energy is expensive, a fitted surrogate is cheap, and the surrogate is only useful if it generalises to configurations never computed. Two features of the successful architectures are visible in miniature in Proposition 8.8. They are built from local terms – each hidden unit in Eq. (8.78) reads two neighbouring sites, not all forty – so the parameter count grows with the range of the interaction rather than the size of the system. And they enforce the symmetries of the problem by construction rather than learning them, which is exactly the data saving that Section 8.16.3 shows is needed: our network had to spend thousands of configurations discovering that couplings beyond nearest neighbours are zero, whereas a locality constraint would have supplied that for free. Chapter 10 makes the same argument for images under translation, and it is the single most useful design principle in the subject.

8.17 Summary and the programs

The chapter began with a promise made in Chapter 5: logistic regression is a neural network with no hidden layer. The XOR gate showed why that is not enough – a single layer computes a linear boundary, and linear regression on XOR returns the constant 0.5 while logistic regression achieves exactly chance – and a hidden layer of two units solved all three gates perfectly.

The mathematics is a composition. A layer computes $\boldsymbol{z}^l = (\boldsymbol{W}^l)^T \boldsymbol{a}^{l-1} + \boldsymbol{b}^l$ followed by an element-wise non-linearity; stacking layers gives the feed-forward pass, and the universal approximation theorem guarantees that one hidden layer suffices to represent any continuous function – while saying nothing about how many units are needed or whether gradient descent will find them.

The gradient follows from the chain rule alone, and Sections 8.8.1 and 8.8.2 worked it out first on two scalar networks, by hand and with `jax.grad`, and on a 2–2–1 network, where the reverse sweep of Section 5.8.2 continues through the hidden nodes. Defining the error $\delta_j^l = \partial C / \partial z_j^l$ gives the four equations (8.65)–(8.68): the output error starts the recursion, Eq. (8.66) pulls it backwards through the transposed weights modulated by f', and the weight and bias gradients follow immediately. The whole gradient costs about two forward passes regardless of the number of parameters, which is reverse-mode automatic differentiation from Section 4.14 specialised to a layered graph. For the natural pairings of output activation and loss – linear with squared error, softmax with cross entropy – the output error collapses to $\boldsymbol{\delta}^L = \boldsymbol{a}^L - \boldsymbol{y}$.

Two practical problems then had to be solved before any of this worked in depth. Equation (8.71) shows the gradient reaching layer l to be a product of $L - l$ factors, which shrinks or grows geometrically; since $\sigma' \leq \frac{1}{4}$, a ten-layer sigmoid network attenuates the gradient

by 10^{-6} before it reaches the first layer. The repairs were the ReLU family, which does not saturate, and the variance-preserving initialisations of Eqs. (8.73) and (8.74).

The implementation of Section 8.13 is those equations transcribed, and it was verified against finite differences to between 10^{-6} and 10^{-8} for every activation function and both output heads before being used on anything.

The complete programs are collected in `doc/BookML/BookPrograms`:

- `gates.py` – linear regression, logistic regression and a neural network on the AND, OR and XOR gates of Table 8.1.
- `neural_network.py` – the activation functions and the `NeuralNetwork` class of Section 8.13, with the gradient check.
- `digits.py` – the handwritten-digit example, the comparison against `scikit-learn`, and the depth study used in the exercises.
- `frameworks.py` – the same network in `scikit-learn` and `PyTorch`.
- `ising_network.py` – the Ising study of Section 8.16: the vanishing spin-energy correlation of Proposition 8.7, the exact $2L$-unit network of Proposition 8.8, width and sample-size studies, the comparison against the three estimators of Section 3.16, and a `PyTorch` cross-check.

8.18 Exercises

Warm-up exercises

1. **Linear layers collapse.** (a) Show that a network with two layers and *linear* activations computes an affine function of its input, and identify the effective weight matrix and bias. (b) Deduce that depth is useless without a non-linearity. (c) Does the same argument apply if only the *output* layer is linear?
2. **The XOR gate by hand.** Construct explicitly a network with two inputs, two hidden units with the step activation and one output unit that computes XOR. Hint: let one hidden unit compute OR and the other NAND. Verify your weights on all four inputs.
3. **Counting parameters.** For an architecture $[M_0, M_1, \ldots, M_L]$, show that the number of weights is $\sum_l M_{l-1} M_l$ and the number of biases $\sum_l M_l$. Evaluate both for $[64, 50, 10]$ and for $[784, 300, 100, 10]$, and compare with the number of training samples in the digits and MNIST data sets respectively.
4. **Derivatives of the activations.** (a) Verify $\sigma' = \sigma(1-\sigma)$ and $\tanh' = 1 - \tanh^2$. (b) Show that $\max_z \sigma'(z) = 1/4$ and $\max_z \tanh'(z) = 1$. (c) Plot σ', $\tanh'$, ReLU' and ELU' together and comment on which of them can attenuate a gradient.
5. **Deriving the four equations.** (a) Derive Eq. (8.59) from the squared-error cost. (b) Show that $\delta^l_j = \partial C / \partial b^l_j$, Eq. (8.67). (c) Derive the recursion (8.66) using the chain rule, being explicit about why the sum over k appears. (d) Show that for a softmax output with the cross entropy the output error reduces to $\boldsymbol{\delta}^L = \boldsymbol{a}^L - \boldsymbol{y}$.
6. **The cost of the gradient.** (a) Count the multiplications in one forward pass through $[M_0, \ldots, M_L]$. (b) Count them in one backward pass, and show that the two are of the same order. (c) How many forward passes would a finite-difference gradient require? Evaluate for $[784, 300, 100, 10]$ and comment.
7. **Gradient check (numerical).** Implement the network of Section 8.13 and reproduce the gradient check. (a) Confirm a relative error around 10^{-6} or smaller. (b) Deliberately introduce a bug – omit the $f'(\boldsymbol{z}^l)$ factor in Eq. (8.66) – and report what the check gives, and

whether the network still trains. (c) Vary h from 10^{-1} to 10^{-12} and plot the discrepancy; explain the shape of the curve using the round-off discussion of Section 1.12.

8. **Vanishing gradients (numerical).** Train networks of depth $1, 2, 4, 8$ hidden layers of 30 units each on the digits data, once with the sigmoid and once with ReLU. (a) For each, record the norm of the gradient at the first and the last hidden layer after one epoch. (b) Plot the ratio against depth and compare with the bound $4^{-(L-1)}$ implied by Eq. (8.71). (c) Which activation permits training at depth eight?
9. **Initialisation (numerical).** (a) Initialise all weights to zero and explain, and then verify, what happens. (b) Compare $\mathcal{N}(0,1)$, Xavier (8.73) and He (8.74) initialisation on a five-layer ReLU network, plotting the variance of the activations against layer index after one forward pass. (c) Relate what you see to Eq. (8.72).
10. **Architecture and regularisation (numerical).** On the digits data, sweep the number of hidden units from 5 to 200 and plot training and test accuracy. (a) Where does overfitting begin? (b) Add weight decay and repeat for several λ. (c) Compare the best network with the random forest and the gradient-boosted trees of Chapter 7 on the same split, and with the logistic regression of Chapter 5. Comment on accuracy, training time and the number of hyperparameters each required.

Project-style exercise: building a neural network

Part a: forward and backward.

Implement a fully connected network from scratch, supporting an arbitrary list of layer sizes, a choice of hidden activation, and both output heads. Verify the gradient against finite differences for every activation and both heads, and report the worst relative error as a table. Do not proceed until this passes.

Part b: the gates.

Reproduce Section 8.2: linear regression, logistic regression and your network on AND, OR and XOR. Then find the smallest hidden layer that solves XOR, and plot the decision boundary your network learns.

Part c: regression.

Apply your network to the Franke function of Section 3.15 with a linear output layer and the squared-error cost. Compare the test error against the OLS, Ridge and Lasso results of Chapter 3, at matched numbers of parameters where possible. Which method wins, and does the answer depend on the noise level?

Part d: classification.

Apply it to the Wisconsin data of Section 5.13 and to the digits data. Report the metrics of Section 5.12 on a test set used once. Compare against logistic regression, the support vector machine of Chapter 6 and the ensembles of Chapter 7.

Part e: optimisers and depth.

Replace plain gradient descent by momentum, RMSProp and Adam from Chapter 4, sweeping the learning rate for each, and present the results as a table like Table 4.4. Then increase the depth and study the vanishing-gradient effect directly, comparing the sigmoid with ReLU.

Part f: regularisation.

Add weight decay, early stopping and dropout, and quantify what each is worth on your hardest problem. Choose all hyperparameters by cross-validation on a validation set, and report a single number on the test set at the very end. Discuss honestly how much of the final performance came from the architecture and how much from the tuning.

Chapter 9
Neural Networks for Differential Equations

Chapter 8 built a neural network and trained it on labelled data. This chapter puts the same network to a different use: solving differential equations, where there are no labels at all. What replaces them is the equation itself. If we can write the residual of a differential equation as a function of the network and its derivatives, then driving that residual to zero *is* training, and the converged network *is* the solution.

The idea is due to Lagaris and collaborators in the 1990s and has been revived, under the name of physics-informed neural networks, as a large research area; a book-length treatment is given by Yadav and Kumar [42] and a recent survey by Cuomo and others [43]. Our aim here is narrower and more concrete. We take the network of Section 8.13, extend it with the ability to differentiate its own output with respect to its *input*, and use it to solve four problems: exponential decay, logistic population growth, the one-dimensional Poisson equation, and then the diffusion and wave equations in one space dimension. At each stage we compare against the exact solution and against a classical numerical method, because a new method that is not compared against the old one has not been evaluated.

One thread from Chapter 8 becomes concrete here. The discussion of activation functions in Section 8.6 ended with the remark that the GELU family is the default choice for physics-informed networks because ReLU has no usable second derivative. In this chapter that remark stops being an aside: we shall see in Section 9.2 that a ReLU network has an *identically zero* second derivative and therefore cannot solve a second-order equation at all.

9.1 Ordinary differential equations

An ordinary differential equation involves a function of one variable. In general it may be written

$$f\left(x, g(x), g'(x), g''(x), \ldots, g^{(n)}(x)\right) = 0, \tag{9.1}$$

where $g(x)$ is the function to be found and $g^{(n)}$ its n-th derivative. The expression f is simply a way of writing that some relation among x, g and its derivatives holds; the highest derivative appearing, n, is the *order* of the equation. Equation (9.1) alone does not determine g, and additional conditions – initial or boundary values – are required for the solution to be unique.

The trial solution.

The central device is to write the candidate solution as

$$\boxed{g_t(x,P) = h_1(x) + h_2\left(x, N(x,P)\right),} \tag{9.2}$$

where $N(x,P)$ is a neural network with parameters P, the function $h_1(x)$ alone makes g_t satisfy the required conditions, and h_2 is constructed so that the network's contribution *vanishes* wherever those conditions are imposed.

This construction is worth dwelling on, because it is what distinguishes the approach from simply adding the boundary conditions to the loss. The conditions are satisfied *by construction*, exactly, for any values whatever of the parameters, including the random values at initialisation. The optimiser is therefore never asked to trade accuracy at the boundary against accuracy in the interior; it has only one job, which is to make the equation hold. We shall see the consequence numerically: the error of our diffusion solution at $t = 0$ is exactly zero, to machine precision, at every stage of training.

The cost function.

Equation (9.1) states that f should vanish. For a single input x we may therefore take the squared residual as the cost, and for N collocation points x_i the mean squared residual,

$$C(\boldsymbol{x},P) = \frac{1}{N}\sum_{i=1}^{N}\left(f\left(x_i, g_t(x_i), g_t'(x_i), \ldots, g_t^{(n)}(x_i)\right)\right)^2. \tag{9.3}$$

The network must find parameters P minimising Eq. (9.3).

Three remarks about this cost. It uses no labelled data: the x_i are points at which we choose to enforce the equation, not measurements, and they may be placed anywhere and changed at will. It is a mean squared error, so everything said in Section 3.6 about squared losses applies, and the optimisation is that of Chapter 4 without modification. And it is emphatically not convex, so the warnings of Section 8.7 apply with full force – a point we shall confirm the hard way in Section 9.3.

9.2 Differentiating the network with respect to its input

Evaluating Eq. (9.3) requires derivatives of the network output with respect to its *input*. This is a different object from the gradients of Chapter 8, which were with respect to the *parameters*, and it is obtained by propagating derivatives *forward* through the layers rather than backwards.

Recall the feed-forward pass of Eq. (8.21), with $\boldsymbol{a}^0 = \boldsymbol{x}$,

$$\boldsymbol{z}^l = \boldsymbol{a}^{l-1}\boldsymbol{W}^l + \boldsymbol{b}^l, \qquad \boldsymbol{a}^l = f\left(\boldsymbol{z}^l\right), \tag{9.4}$$

with a linear output layer. Differentiating with respect to an input coordinate x_k and using the chain rule,

$$\frac{\partial \boldsymbol{z}^l}{\partial x_k} = \frac{\partial \boldsymbol{a}^{l-1}}{\partial x_k}\boldsymbol{W}^l, \qquad \frac{\partial \boldsymbol{a}^l}{\partial x_k} = f'\left(\boldsymbol{z}^l\right) \circ \frac{\partial \boldsymbol{z}^l}{\partial x_k}, \tag{9.5}$$

started from $\partial \boldsymbol{a}^0 / \partial x_k = \boldsymbol{e}_k$. Differentiating once more, and using the product rule on the second of Eqs. (9.5),

$$\boxed{\frac{\partial^2 \boldsymbol{a}^l}{\partial x_k^2} = f''\left(\boldsymbol{z}^l\right) \circ \left(\frac{\partial \boldsymbol{z}^l}{\partial x_k}\right)^2 + f'\left(\boldsymbol{z}^l\right) \circ \frac{\partial^2 \boldsymbol{z}^l}{\partial x_k^2},} \tag{9.6}$$

with $\partial^2 \boldsymbol{z}^l / \partial x_k^2 = (\partial^2 \boldsymbol{a}^{l-1} / \partial x_k^2) \boldsymbol{W}^l$. These recursions run alongside the ordinary forward pass at a cost of one extra matrix product per layer per derivative order, so obtaining N, N' and N'' costs about three forward passes. This is *forward-mode* automatic differentiation, the counterpart of the reverse mode discussed in Section 4.14, and it is the efficient choice here precisely because the input is low-dimensional – one or two variables – whereas the parameters are many, which is why the parameter gradients still go backwards.

Why the activation function now matters more.

Equation (9.6) requires f''. This is where the choice of activation stops being a matter of taste. For the rectified family $f'' \equiv 0$ wherever it is defined, and f' is piecewise constant, so *every* term in Eq. (9.6) vanishes: a ReLU network has an identically zero second derivative almost everywhere. It cannot represent the curvature that a second-order equation is about.

This is easily confirmed. Taking a network with two hidden layers of twenty units, initialised as in Section 8.10, and evaluating $\max |\partial^2 N / \partial x^2|$ over a grid:

```
tanh         max|d2N/dx2| = 1.236e-01
gelu         max|d2N/dx2| = 1.014e+00
swish        max|d2N/dx2| = 6.354e-01
sigmoid      max|d2N/dx2| = 1.585e-02
softplus     max|d2N/dx2| = 2.386e-01
elu          max|d2N/dx2| = 1.385e+00
relu         max|d2N/dx2| = 0.000e+00
leaky_relu   max|d2N/dx2| = 0.000e+00
```

The two piecewise-linear activations give exactly zero, and every smooth activation gives something usable. The remark at the end of Section 8.6.4 – that the GELU family is preferred wherever the network output must be differentiated – is this table. For the examples in this chapter we use tanh, which is smooth, bounded, and has the mild advantage over GELU of a derivative expressible in the function value; the exercises ask the reader to repeat the calculations with the others.

Figure 9.1 shows the same numbers on a logarithmic scale, where the two zeros can only be marked rather than plotted. The gap is not a matter of degree: the smooth activations differ among themselves by two orders of magnitude and all of them work, while the rectified ones are not small but identically zero and cannot work at all.

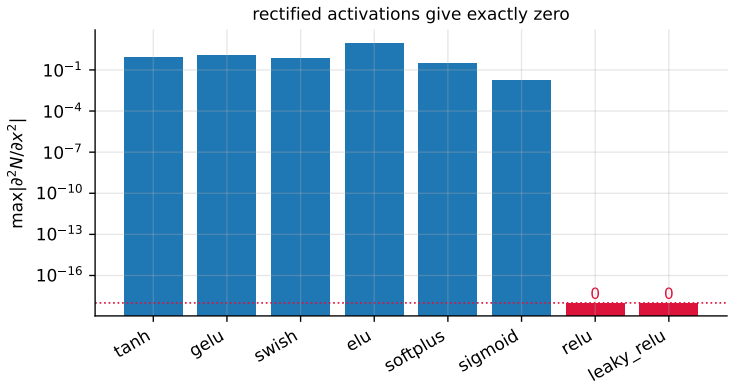


Fig. 9.1 Largest second derivative of a randomly initialised network with two hidden layers of twenty units, by activation function. The rectified activations, in red, give exactly zero because $f'' \equiv 0$ in Eq. (9.6).

Implementation.

The recursions (9.5) and (9.6) are a direct transcription, and reuse the activation table of Section 8.13 unchanged.

```
import autograd.numpy as np
from autograd import grad, elementwise_grad

def network(P, X, activation="tanh"):
    """Feed-forward pass, Eq. (8.forwardbatch), with a linear output layer."""
    f = ACT[activation]
    a = X
    for l, (W, b) in enumerate(P):
        z = a @ W + b
        a = f(z) if l < len(P) - 1 else z
    return a[:, 0]


def network_derivs(P, X, activation="tanh", order=2, k=0):
    """N, dN/dx_k and d2N/dx_k^2 by the forward recursions (9.firstderiv)
    and (9.secondderiv).  The activations and their derivatives are those
    of Section 8.nncode."""
    f = ACT[activation]
    fp = elementwise_grad(f)            # f'
    fpp = elementwise_grad(fp)          # f''

    a = X
    da = np.zeros_like(X) + (np.arange(X.shape[1]) == k)   # da0/dxk = e_k
    d2a = np.zeros_like(X)

    for l, (W, b) in enumerate(P):
        z, dz, d2z = a @ W + b, da @ W, d2a @ W
        if l < len(P) - 1:
            a, da, d2a = f(z), fp(z) * dz, fpp(z) * dz**2 + fp(z) * d2z
        else:                                                # linear output
            a, da, d2a = z, dz, d2z
    if order == 1:
        return a[:, 0], da[:, 0]
    return a[:, 0], da[:, 0], d2a[:, 0]
```

Checked against automatic differentiation of the same network, the analytic recursions agree to 2×10^{-16} in the first derivative and 8×10^{-17} in the second – that is, to machine precision, as two correct computations of the same quantity should.

The parameters are initialised exactly as in Section 8.10, and trained with the Adam optimiser of Eq. (4.71); both are carried over unchanged.

```
def init_parameters(layer_sizes, activation="tanh", rng=None):
    """He (8.he) for the ReLU family, Xavier (8.xavier) otherwise."""
    rng = np.random.default_rng(0) if rng is None else rng
    P = []
    for i in range(len(layer_sizes) - 1):
        nin, nout = layer_sizes[i], layer_sizes[i + 1]
        s = np.sqrt(2.0 / nin) if activation in ("relu", "leaky_relu", "elu") \
            else np.sqrt(1.0 / nin)
        P.append([rng.normal(0, s, (nin, nout)), np.zeros(nout)])
    return P


def solve_de(residual, layer_sizes, X, activation="tanh", n_iter=2000,
             gamma=1e-2, rng=None):
    """Minimise the mean squared residual, Eq. (9.cost), with Adam."""
    P = init_parameters(layer_sizes, activation, rng)
    cost = lambda P: np.mean(residual(P, X)**2)
```

```
    return adam_minimise(cost, P, n_iter=n_iter, gamma=gamma)
```

The parameter gradients of Eq. (9.3) are obtained by reverse-mode automatic differentiation, as introduced in Section 4.14. We could derive them by hand as we did in Section 8.8 – the residual is, after all, an explicit function of the weights – but the expression now involves the input-derivative recursions as well as the forward pass, and the derivation carries no new insight. This is precisely the situation automatic differentiation exists for.

9.3 Exponential decay

The simplest possible test has an exact solution to check against. Exponential decay of a quantity $g(x)$ is described by

$$g'(x) = -\kappa g(x), \qquad g(0) = g_0, \tag{9.7}$$

with analytical solution

$$g(x) = g_0 \exp(-\kappa x). \tag{9.8}$$

We take $\kappa = 2$ and $g_0 = 10$.

The trial solution.

The single condition is $g(0) = g_0$. Choosing $h_1(x) = g_0$ and $h_2(x,N) = xN(x,P)$ in Eq. (9.2) gives

$$g_t(x,P) = g_0 + xN(x,P), \tag{9.9}$$

which satisfies $g_t(0,P) = g_0$ identically, for every P, because the factor x annihilates the network at the origin. Differentiating,

$$g_t'(x,P) = N(x,P) + x\frac{\partial N(x,P)}{\partial x}, \tag{9.10}$$

where $\partial N/\partial x$ comes from Eq. (9.5). Substituting both into Eq. (9.7) gives the residual whose mean square is the cost,

$$r(x,P) = g_t'(x,P) + \kappa g_t(x,P) = N + x\frac{\partial N}{\partial x} + \kappa\left(g_0 + xN\right). \tag{9.11}$$

```
import numpy as np
kappa, g0 = 2.0, 10.0
X = np.linspace(0, 1, 50).reshape(-1, 1)          # collocation points

def residual_decay(P, X):
    """Eq. (9.resdecay)."""
    N, dN = network_derivs(P, X, "tanh", order=1)
    x = X[:, 0]
    g_t  = g0 + x * N                              # Eq. (9.trialode)
    dg_t = N + x * dN                              # Eq. (9.dtrialode)
    return dg_t + kappa * g_t

P, history = solve_de(residual_decay, [1, 40, 40, 1], X, "tanh",
                      n_iter=3000, gamma=2e-2, rng=np.random.default_rng(1))

N, dN = network_derivs(P, X, "tanh", order=1)
g_network = g0 + X[:, 0] * N
g_exact = g0 * np.exp(-kappa * X[:, 0])
```

```
print(f"max relative error {np.abs(g_network - g_exact).max() / g0:.2e}")
```

With this architecture the relative error over three seeds is 4.5×10^{-3}, 4.1×10^{-4} and 2.0×10^{-2} – accurate, but varying by a factor of fifty between runs that differ only in the random initialisation.

A cautionary result. Our first attempt at this problem used a network of two hidden layers of *twenty* units and a learning rate of 10^{-2}, and it failed. The cost fell to 31.9 and stopped there, and it remained at exactly 31.9 whether we ran 2000 iterations or $10\,000$, and whether the learning rate was 10^{-2} or 2×10^{-2}. The solution it returned was wrong by 18%. Raising the learning rate to 5×10^{-2}, or widening the layers to forty units, escaped it immediately.

This is the non-convexity of Section 8.7 in the flesh: a local minimum that no amount of patience escapes, because the gradient there is genuinely zero, and from which a different initialisation or a larger step departs at once. It is worth reporting rather than quietly tuning away, because it is the characteristic failure mode of this whole approach. A classical solver either converges or reports a failure; a network can converge confidently to the wrong answer. *Always plot the residual, and always compare against something.*

9.4 Logistic population growth

A logistic model of population growth assumes the population converges to an equilibrium. With $g(t)$ the population density at time t, $\alpha>0$ the growth rate and $A>0$ the carrying capacity of the environment,

$$g'(t)=\alpha\,g(t)\,(A-g(t)),\qquad g(0)=g_0. \tag{9.12}$$

This equation is non-linear, which the previous one was not, and it has the closed-form solution

$$g(t)=\frac{A g_0}{g_0+(A-g_0)\,e^{-\alpha A t}}. \tag{9.13}$$

We take $\alpha=2$, $A=1$ and $g_0=1.2$, so the population starts above the carrying capacity and decays towards it.

The boundary condition is of the same form as before, so the trial solution (9.9) carries over unchanged with t in place of x, and only the residual changes:

$$r(t,P)=g_t'(t,P)-\alpha\,g_t(t,P)\,(A-g_t(t,P)). \tag{9.14}$$

```
alpha, A, g0 = 2.0, 1.0, 1.2
T = np.linspace(0, 1, 50).reshape(-1, 1)

def residual_logistic(P, X):
    """Eq. (9.reslogistic)."""
    N, dN = network_derivs(P, X, "tanh", order=1)
    t = X[:, 0]
    g_t  = g0 + t * N
    dg_t = N + t * dN
    return dg_t - alpha * g_t * (A - g_t)

P, history = solve_de(residual_logistic, [1, 40, 40, 1], T, "tanh",
                      n_iter=3000, gamma=2e-2, rng=np.random.default_rng(1))
```

Comparison with forward Euler.

The natural classical competitor is the forward Euler scheme $g_{i+1} = g_i + \Delta t\, \alpha g_i(A - g_i)$, whose error is $\mathcal{O}(\Delta t)$. Both methods solved on $[0,1]$:

```
Euler,  10 steps (dt=0.100): max err 9.92e-03
Euler,  20 steps (dt=0.050): max err 4.67e-03
Euler,  50 steps (dt=0.020): max err 1.80e-03
Euler, 100 steps (dt=0.010): max err 8.93e-04
network, 50 collocation points  : max err 1.15e-04
```

The Euler errors halve as the step halves, confirming first-order convergence, and the network is about an order of magnitude more accurate than Euler with the same number of points. That comparison flatters the network, and it is worth saying why it is not the whole story. Euler took a few microseconds; the network took several seconds of Adam iterations. A second-order Runge-Kutta scheme with fifty steps would beat the network at a cost still negligible beside it. The network's advantages lie elsewhere, and we return to them in Section 9.9.

9.5 The one-dimensional Poisson equation

We now move to a second-order equation, which is where Eq. (9.6) and the choice of activation begin to matter. The Poisson equation in one dimension is

$$-g''(x) = f(x), \qquad x \in (0,1), \tag{9.15}$$

with the boundary conditions

$$g(0) = g(1) = 0. \tag{9.16}$$

We take $f(x) = (3x + x^2)e^x$, for which the exact solution is

$$g(x) = x(1-x)e^x. \tag{9.17}$$

The trial solution.

Two conditions must now hold, one at each end. The choice

$$g_t(x,P) = x(1-x)N(x,P) \tag{9.18}$$

satisfies both identically, since the prefactor vanishes at $x = 0$ and at $x = 1$; here $h_1 \equiv 0$. Its second derivative follows from the product rule,

$$g_t''(x,P) = -2N + 2(1-2x)\frac{\partial N}{\partial x} + x(1-x)\frac{\partial^2 N}{\partial x^2}, \tag{9.19}$$

and the residual is $r = -g_t'' - f$. Note that all three of N, N' and N'' appear, so this is the first example that would fail outright with a ReLU network.

```
f = lambda x: (3 * x + x**2) * np.exp(x)
X = np.linspace(0, 1, 60).reshape(-1, 1)

def residual_poisson(P, X):
    """Eqs. (9.trialpoisson) and (9.d2trialpoisson)."""
    N, dN, d2N = network_derivs(P, X, "tanh", order=2)
```

```
    x = X[:, 0]
    d2g_t = -2 * N + 2 * (1 - 2 * x) * dN + x * (1 - x) * d2N
    return -d2g_t - f(x)

P, history = solve_de(residual_poisson, [1, 30, 30, 1], X, "tanh",
                      n_iter=4000, gamma=1e-2, rng=np.random.default_rng(2))
```

Comparison with finite differences.

The classical method here is the three-point formula for the second derivative. On a uniform grid $x_i = i\Delta x$, $i = 0, 1, \dots, n$, with $\Delta x = 1/n$, a Taylor expansion of u about x_i in both directions gives

$$u(x_i \pm \Delta x) = u_i \pm \Delta x\, u_i' + \frac{\Delta x^2}{2} u_i'' \pm \frac{\Delta x^3}{6} u_i''' + \mathcal{O}(\Delta x^4), \tag{9.20}$$

and adding the two expansions cancels every odd-order term, so that

$$u_i'' = \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2} + \mathcal{O}(\Delta x^2). \tag{9.21}$$

Inserting Eq. (9.21) into Eq. (9.15) and using the boundary values $u_0 = u_n = 0$ turns the differential equation into the linear system

$$\frac{1}{\Delta x^2} \begin{bmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n-2} \\ u_{n-1} \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ \vdots \\ f_{n-2} \\ f_{n-1} \end{bmatrix}, \tag{9.22}$$

whose matrix is the discrete one-dimensional Laplacian. It is symmetric, positive definite and tridiagonal, and by the LU discussion of Section 1.14 it can be factorised and solved in $\mathcal{O}(n)$ operations rather than the $\mathcal{O}(n^3)$ of a dense elimination. This is the benchmark the network has to beat.

```
FD,  10 intervals (dx=0.100): max err 2.15e-03
FD,  20 intervals (dx=0.050): max err 5.41e-04
FD,  50 intervals (dx=0.020): max err 8.66e-05
FD, 100 intervals (dx=0.010): max err 2.17e-05
network, 60 collocation points  : max err 5.35e-05
```

The finite-difference errors fall by a factor of four when the spacing is halved, confirming the $\mathcal{O}(\Delta x^2)$ accuracy of the three-point formula, and the network with sixty points sits between the fifty- and hundred-interval finite-difference solutions. The two methods are, on this problem, comparable in accuracy per point – and the finite-difference solution is obtained by one tridiagonal solve costing $\mathcal{O}(n)$ operations by Section 1.14, against four thousand Adam iterations.

Figure 9.2 collects the three ordinary equations. In each case the network output lies on the exact solution to within the width of the plotted markers. The middle panel also shows the forward Euler solution with ten steps, whose visible lag behind the exact curve is the $\mathcal{O}(\Delta t)$ error of a first-order scheme; the network, with fifty collocation points and no notion of a time step, has no such systematic bias.

For a one-dimensional boundary-value problem on a simple domain, the classical method wins decisively, and it would be dishonest to present this example as a demonstration of

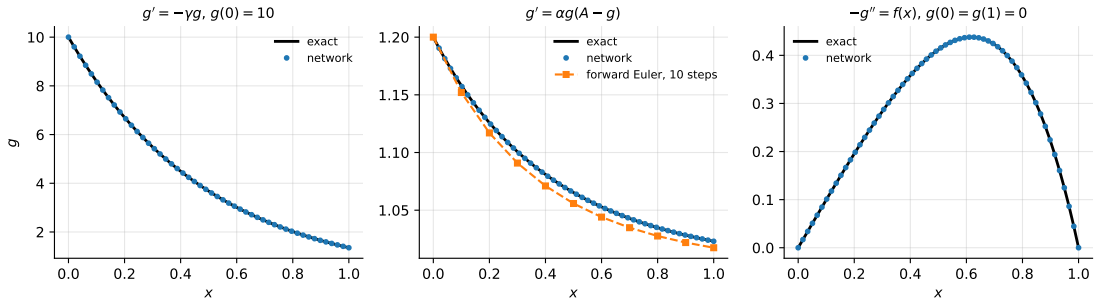


Fig. 9.2 The three ordinary differential equations of Sections 9.3–9.5, with the network solution against the exact one. The middle panel adds the forward Euler solution with ten steps for comparison.

anything else. Its value is as a *verification*: the network reproduces a known answer to five significant figures, which licenses its use where no classical method is convenient.

9.6 Partial differential equations

A partial differential equation involves a function of several variables and may contain any combination of partial derivatives. In general, for a function $g(x_1, \dots, x_N)$,

$$f\left(x_1, \dots, x_N, \frac{\partial g}{\partial x_1}, \dots, \frac{\partial g}{\partial x_N}, \frac{\partial^2 g}{\partial x_1 \partial x_2}, \dots, \frac{\partial^n g}{\partial x_N^n}\right) = 0, \tag{9.23}$$

where f involves mixed derivatives up to order n, and additional conditions are again needed for uniqueness.

The strategy is unchanged. We need a trial solution

$$g_t(x_1, \dots, x_N) = h_1(x_1, \dots, x_N) + h_2\left(x_1, \dots, x_N, N(x_1, \dots, x_N, P)\right), \tag{9.24}$$

in which h_1 alone satisfies the conditions and h_2 is built so that the network contributes nothing where they are imposed. The cost is again the mean squared residual, now over a set of M points $\boldsymbol{x}_i = (x_1^{(i)}, \dots, x_N^{(i)})$ forming the rows of a matrix $\boldsymbol{X}$:

$$C(\boldsymbol{X}, P) = \frac{1}{M} \sum_{i=1}^{M} \left(f\left(\boldsymbol{x}_i, \frac{\partial g_t}{\partial x_1}\Big|_{\boldsymbol{x}_i}, \dots\right)\right)^2. \tag{9.25}$$

The only change to the network is that the input layer now has N nodes rather than one. Everything else – the architecture, the initialisation, the optimiser – is carried over from Chapter 8 unchanged, which is the point of having built it generally.

Partial derivatives in the code.

For the ODEs we differentiated the network directly, using the recursions (9.5) and (9.6). For the PDEs the trial solutions become more elaborate, and hand-differentiating each one is error-prone drudgery of exactly the kind Section 4.14 warned against. We therefore differentiate the trial solution itself, by nesting automatic differentiation with respect to one input coordinate at a time.

```
def d_dxk(fun, k):
    """Partial derivative of fun(P, X) with respect to input column k.

    Nesting this gives higher derivatives: d_dxk(d_dxk(u, 0), 0) is
    the second derivative with respect to x_0.
    """
    def wrapped(P, X):
        def scalarised(xk):
            Xn = np.concatenate([X[:, :k], xk.reshape(-1, 1), X[:, k+1:]], axis=1)
            return fun(P, Xn)
        return elementwise_grad(scalarised)(X[:, k])
    return wrapped
```

9.7 The diffusion equation

In one spatial dimension the diffusion equation reads

$$\frac{\partial g(x,t)}{\partial t} = \frac{\partial^2 g(x,t)}{\partial x^2}, \tag{9.26}$$

and we impose

$$g(0,t)=0, \quad t\geq 0; \qquad g(1,t)=0, \quad t\geq 0; \qquad g(x,0)=u(x), \quad x\in[0,1], \tag{9.27}$$

with $u(x)=\sin(\pi x)$. For this initial condition the exact solution is

$$g(x,t) = e^{-\pi^2 t}\sin(\pi x), \tag{9.28}$$

which decays rapidly – by a factor of $e^{-\pi^2}\approx 5\times 10^{-5}$ over a unit of time.

The trial solution.

Three conditions must hold: two boundaries in x and one initial condition in t. The choice

$$\boxed{g_t(x,t,P) = (1-t)\,u(x) + x(1-x)\,t\,N(x,t,P)} \tag{9.29}$$

satisfies all three identically. At $t=0$ the second term vanishes and the first reduces to $u(x)$; at $x=0$ and $x=1$ the factor $x(1-x)$ kills the network while $u(0)=u(1)=0$ kills the first term. Here $h_1(x,t)=(1-t)u(x)$ and $h_2 = x(1-x)tN$.

```
nx, nt = 20, 20
xs, ts = np.linspace(0, 1, nx), np.linspace(0, 1, nt)
Xg, Tg = np.meshgrid(xs, ts, indexing="ij")
X = np.column_stack([Xg.ravel(), Tg.ravel()])       # M = 400 collocation points

def trial_diffusion(P, X):
    """Eq. (9.trialdiff)."""
    x, t = X[:, 0], X[:, 1]
    return (1 - t) * np.sin(np.pi * x) + x * (1 - x) * t * network(P, X, "tanh")

u_t  = d_dxk(trial_diffusion, 1)
u_x  = d_dxk(trial_diffusion, 0)
u_xx = d_dxk(u_x, 0)

def residual_diffusion(P, X):
```

```
    return u_t(P, X) - u_xx(P, X)                    # Eq. (9.diffusion)

P, history = solve_de(residual_diffusion, [2, 30, 30, 1], X, "tanh",
                      n_iter=800, gamma=1e-2, rng=np.random.default_rng(1))
```

After 800 iterations the maximum absolute error over the whole space-time grid is 1.0×10^{-2}, and slicing by time,

```
t=0.000: max err 0.00e+00
t=0.105: max err 1.68e-03
t=0.316: max err 4.50e-04
```

The first line is the point of the construction. The error at $t = 0$ is *exactly* zero – not small, but zero to machine precision – because Eq. (9.29) satisfies the initial condition identically, whatever the parameters happen to be. No amount of training was required to achieve it and no amount of bad training could destroy it. The same is true along both spatial boundaries. A formulation that instead added the boundary conditions to the loss as extra terms would satisfy them only approximately, and would have to trade that accuracy against the interior residual.

The remaining error is concentrated at small but non-zero times, where the solution is changing fastest; by $t = 0.3$ the solution has decayed by a factor of twenty and the absolute error with it.

9.8 The wave equation

The wave equation is second order in *both* variables,

$$\frac{\partial^2 g(x,t)}{\partial t^2} = c^2 \frac{\partial^2 g(x,t)}{\partial x^2}, \tag{9.30}$$

with c the wave speed. The conditions are now four, since a second-order equation in time requires both the initial displacement and the initial velocity:

$$g(0,t) = 0, \quad g(1,t) = 0, \quad g(x,0) = u(x), \quad \left.\frac{\partial g(x,t)}{\partial t}\right|_{t=0} = v(x). \tag{9.31}$$

We take $c = 1$, $u(x) = \sin(\pi x)$ and $v(x) = 0$, for which the exact solution is the standing wave

$$g(x,t) = \cos(\pi t)\sin(\pi x). \tag{9.32}$$

The trial solution.

The extra condition on $\partial g/\partial t$ at $t = 0$ requires a little more care, and the device is to use t^2 rather than t:

$$g_t(x,t,P) = \left(1 - t^2\right) u(x) + x(1-x)\, t^2\, N(x,t,P). \tag{9.33}$$

At $t = 0$ this gives $u(x)$ as required. Differentiating with respect to t produces a factor t in every term – from $-2t\,u(x)$ and from the product rule on $t^2 N$ – so the time derivative vanishes at $t = 0$, matching $v(x) = 0$. Both conditions hold identically for every P. Had v been non-zero we would add a term $t\,v(x)$, which contributes $v(x)$ to the derivative at $t = 0$ and nothing to the value.

```
c = 1.0
```

```
def trial_wave(P, X):
    """Eq. (9.trialwave); the t^2 makes dg/dt vanish at t = 0."""
    x, t = X[:, 0], X[:, 1]
    return (1 - t**2) * np.sin(np.pi * x) + x * (1 - x) * t**2 * network(P, X, "tanh")

w_t, w_x = d_dxk(trial_wave, 1), d_dxk(trial_wave, 0)
w_tt, w_xx = d_dxk(w_t, 1), d_dxk(w_x, 0)

def residual_wave(P, X):
    return w_tt(P, X) - c**2 * w_xx(P, X)           # Eq. (9.wave)

P, history = solve_de(residual_wave, [2, 30, 30, 1], X, "tanh",
                      n_iter=800, gamma=1e-2, rng=np.random.default_rng(1))
```

The maximum absolute error over the grid is 8.0×10^{-3} after 800 iterations, on a solution of amplitude one. Note that this problem requires $\partial^2/\partial t^2$ as well as $\partial^2/\partial x^2$, so the smoothness argument of Section 9.2 applies in both variables: a piecewise-linear activation would give identically zero on both sides of Eq. (9.30) and would report a perfect residual of zero for a completely wrong solution. That failure mode – a satisfied residual and a meaningless answer – is worth keeping in mind whenever a residual falls suspiciously fast.

Figure 9.3 shows both partial differential equations. The left and right panels give time slices against the exact solutions; the middle panel gives the absolute error of the diffusion solution over the whole space-time domain. The error map is worth studying: it vanishes identically along all three edges where conditions were imposed – $t = 0$, $x = 0$ and $x = 1$ – which is the trial solution (9.29) doing its work, and it is largest at small non-zero times where the solution changes fastest.

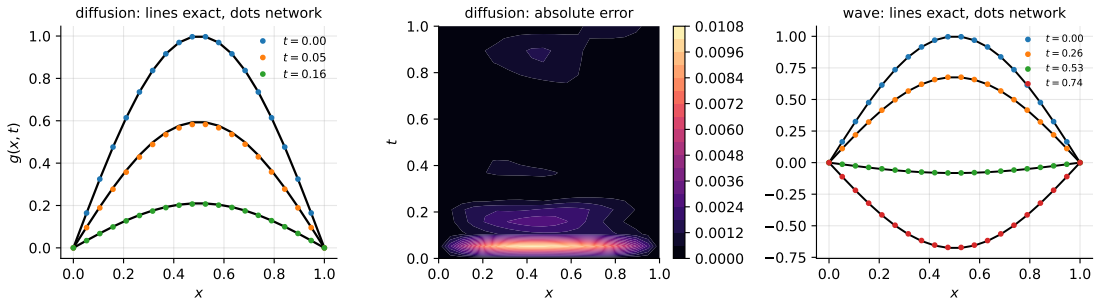


Fig. 9.3 Left: time slices of the diffusion solution against the exact $e^{-\pi^2 t}\sin(\pi x)$. Centre: absolute error over the space-time domain, vanishing identically on the three edges where conditions are imposed. Right: time slices of the wave solution against $\cos(\pi t)\sin(\pi x)$.

9.9 When is this worth doing?

The comparisons above were deliberately unflattering. On the logistic equation a Runge-Kutta scheme is faster and more accurate; on the one-dimensional Poisson equation a tridiagonal solve is faster by orders of magnitude and just as accurate; on the diffusion equation an explicit finite-difference scheme would do very well. For textbook problems in one or two dimensions, classical numerical analysis wins, and it wins comfortably. Any presentation of this material that omits the comparison is selling something.

The case for the network method rests on the situations where classical methods become awkward.

High dimension. A finite-difference or finite-element method needs a mesh, and the number of mesh points grows exponentially with the number of dimensions. A network takes collocation points, which may be sampled randomly, and its cost grows with the number of *parameters* rather than with the volume of the domain. For equations in five or ten dimensions – the Hamilton-Jacobi-Bellman equation, the many-body Schrödinger equation – meshes are impossible and networks are not.

Complicated geometry. Collocation points need no mesh, so an irregular domain costs nothing extra beyond sampling it.

The solution as a function. A finite-difference method returns values on a grid; interpolating between them is an extra step with its own error. The network returns a differentiable closed-form function valid everywhere in the domain, which can be evaluated, differentiated and composed at will.

Inverse problems and data assimilation. This is the strongest case. If some parameter of the equation is unknown but measurements of the solution are available, one simply adds a data-misfit term to Eq. (9.25) and optimises over the equation parameters alongside the network weights. The machinery is unchanged; for a classical solver this would require an adjoint calculation and a separate optimisation loop. This combination of equation and data is what the term *physics-informed neural network* usually refers to, and it is the subject we take up next.

Machine learning connection. The cost function (9.25) should be recognised as a regression problem in disguise, with the residual playing the role of the target and the collocation points the role of the data. Everything in Chapters 4 and 8 therefore applies without modification – and so do the difficulties. The optimisation is non-convex, as Section 9.3 demonstrated painfully; the conditioning arguments of Section 4.5 govern how quickly it converges; the choice of activation function decides whether the required derivatives exist at all; and there is no analogue of the convergence theory that tells us a finite-difference scheme is $\mathcal{O}(\Delta x^2)$. We have traded a method with guarantees for one with flexibility, and the exchange is worth making only when the flexibility is needed.

9.10 Physics-informed neural networks

Everything so far has rested on the trial solution $g_t = h_1 + h_2(\boldsymbol{x}, N)$. It is an elegant device and, as Section 9.7 showed, a powerful one: the conditions hold to machine precision for every value of the parameters, and the optimiser is left with the single job of satisfying the equation.

It is also a straitjacket. Constructing h_1 and h_2 requires a function that satisfies the conditions and a second function that vanishes wherever they are imposed, and both must be written down by hand for each new problem. For the box $[0,1]^2$ with homogeneous Dirichlet conditions this is easy, and $x(1-x)$ does the job. For a domain that is not a box, for conditions imposed on a curved boundary, for a Neumann or Robin condition, or for a system of coupled equations, it ranges from tedious to impossible. The factor $x(1-x)t$ in Eq. (9.29) has no counterpart on an L-shaped domain or an annulus.

The *physics-informed neural network* takes the opposite view. We drop the trial solution entirely and let the network itself be the approximate solution,

$$u(\boldsymbol{x}) \approx N(\boldsymbol{x}, P), \tag{9.34}$$

with no algebraic scaffolding whatsoever. Nothing then guarantees the boundary and initial conditions, so they are demanded of the optimiser instead: each condition contributes its own least-squares penalty to the cost. The conditions become *soft* constraints, satisfied approximately at the optimum, in place of the *hard* constraints of Eqs. (9.29) and (9.33), which held identically.

The material of this section follows a tutorial written for this course by Fredrik Nysæter, Oskar Fausko and Kristian Liodden, transcribed into the framework built up in this chapter.

The composite cost function.

Take the diffusion problem of Section 9.7 again. We need four sets of points rather than one. The *collocation* points $\{(x_i,t_i)\}_{i=1}^{N_{\text{pde}}}$ lie in the interior, where the equation is to hold; the *initial* points $\{(x_i,0)\}$ lie on $t=0$; and two sets of *boundary* points lie on $x=0$ and $x=1$. Each set gets a residual. For the equation itself,

$$L_{\text{PDE}}(P)=\frac{1}{N_{\text{pde}}}\sum_{i=1}^{N_{\text{pde}}}\left(\frac{\partial N}{\partial t}\Big|_{(x_i,t_i)}-\frac{\partial^2 N}{\partial x^2}\Big|_{(x_i,t_i)}\right)^2, \tag{9.35}$$

which is exactly the cost (9.25) we have been minimising all along, only with N in place of g_t. For the initial condition,

$$L_{\text{IC}}(P)=\frac{1}{N_{\text{ic}}}\sum_{i=1}^{N_{\text{ic}}}\left(N(x_i,0,P)-u(x_i)\right)^2, \tag{9.36}$$

and for the two boundaries,

$$L_{\text{BC}}(P)=\frac{1}{N_{\text{bc}}}\sum_{i=1}^{N_{\text{bc}}}N(0,t_i,P)^2+\frac{1}{N_{\text{bc}}}\sum_{i=1}^{N_{\text{bc}}}N(1,t_i,P)^2. \tag{9.37}$$

The quantity actually minimised is a weighted sum,

$$\boxed{L_{\text{total}}(P)=L_{\text{PDE}}(P)+\lambda_{\text{IC}}L_{\text{IC}}(P)+\lambda_{\text{BC}}L_{\text{BC}}(P)} \tag{9.38}$$

with positive weights λ whose choice we return to below.

Equation (9.38) should look familiar. It is a least-squares problem with several groups of targets, and the weights play the role of the penalty parameters of Section 3.8: they say how much we care about one group of residuals relative to another. Nothing in the machinery of Chapters 4 and 8 needs to change.

Implementation.

The code is correspondingly small, because everything it needs already exists. The network is `network` from Section 9.2, the derivatives come from `d_dxk` of Section 9.6, the initialisation and the optimiser are those of Chapter 8. The only new element is that the cost sums several mean-squared residuals, each evaluated on its own set of points.

```
def pinn_solve(terms, layer_sizes, activation="tanh", n_iter=2000, gamma=1e-2,
               rng=None, every=200):
    """terms: list of (name, weight, residual_fn, points), Eq. (9.pinntotal).

    Each residual_fn(P, X) returns the residual on its own point set, so the
    equation, the initial condition and the boundaries are treated alike.
    """
```

```
    P = init_parameters(layer_sizes, activation, rng)

    def cost(P):
        total = 0.0
        for _, w, residual, Xk in terms:
            total = total + w * np.mean(residual(P, Xk) ** 2)
        return total

    return adam_minimise(cost, P, n_iter=n_iter, gamma=gamma)
```

Contrast this with `solve_de`, which took a single residual on a single set of points. That is the whole difference between the two formulations at the level of code: one point set becomes several, and the cost acquires weights.

The diffusion equation revisited.

We solve exactly the problem of Section 9.7 – Eq. (9.26) with the conditions (9.27) and $u(x) = \sin \pi x$ – so that the two formulations can be compared on identical ground. The architecture is the same $[2, 30, 30, 1]$ network with tanh activation, and Adam with $\gamma = 10^{-2}$.

```
nx, nt = 20, 20
xs, ts = np.linspace(0, 1, nx), np.linspace(0, 1, nt)

Xi, Ti = np.meshgrid(xs[1:-1], ts[1:-1], indexing="ij")   # interior only
X_col = np.column_stack([Xi.ravel(), Ti.ravel()])         # 324 points
X_ic  = np.column_stack([xs, np.zeros(nx)])               # t = 0
X_l   = np.column_stack([np.zeros(nt), ts])               # x = 0
X_r   = np.column_stack([np.ones(nt),  ts])               # x = 1

def u_net(P, X): return network(P, X, "tanh")             # Eq. (9.pinnu)

u_t = d_dxk(u_net, 1)
u_x = d_dxk(u_net, 0)
u_xx = d_dxk(u_x, 0)

def r_pde(P, X): return u_t(P, X) - u_xx(P, X)            # Eq. (9.pinnpde)
def r_ic(P, X):  return u_net(P, X) - np.sin(np.pi * X[:, 0])
def r_bc(P, X):  return u_net(P, X)

terms = [("pde", 1.0, r_pde, X_col), ("ic",  10.0, r_ic, X_ic),
         ("bcL", 10.0, r_bc, X_l),   ("bcR", 10.0, r_bc, X_r)]

P, history = pinn_solve(terms, [2, 30, 30, 1], "tanh", n_iter=4000, gamma=1e-2,
                        rng=np.random.default_rng(1))
```

Note that the collocation points deliberately exclude the boundaries. A point that carries both the equation residual and a boundary residual is being asked two things at once, and the two requests compete.

The results, measured on a 100×100 grid against the exact solution (9.28), are collected in Table 9.1, together with the hard-constraint results of Section 9.7 obtained with the same architecture and the same optimiser.

The hard formulation wins, and it wins on every measure. Its root-mean-square error is three to four times smaller, and its error at $t = 0$ is not merely smaller but exactly zero, as it must be. Five times more training does not close the gap; the soft solution's largest error is still 3×10^{-2} after 4000 iterations, and it sits at $x = 0.465$, $t = 0.040$, in the region where the solution is decaying fastest.

This is worth stating plainly, because presentations of physics-informed networks do not always state it: on a problem where a trial solution can be constructed, constructing it is

Table 9.1 The diffusion equation solved two ways with the same network, optimiser and learning rate. The last column is the largest error on the line $t = 0$, where the initial condition is imposed.

Formulation	Iterations	max error	RMSE	error at $t = 0$
Hard, Eq. (9.29)	800	1.04×10^{-2}	1.72×10^{-3}	0 exactly
Hard, Eq. (9.29)	4000	1.09×10^{-2}	1.66×10^{-3}	0 exactly
Soft, $\lambda = 1$	800	2.62×10^{-2}	6.08×10^{-3}	2.62×10^{-2}
Soft, $\lambda = 10$	800	2.98×10^{-2}	5.73×10^{-3}	1.36×10^{-2}
Soft, $\lambda = 100$	800	3.00×10^{-2}	1.07×10^{-2}	3.00×10^{-2}
Soft, $\lambda = 10$	4000	3.35×10^{-2}	6.77×10^{-3}	7.21×10^{-3}

better. Information built into the ansatz is free and exact; information supplied through the cost function must be paid for in optimisation effort and is never exact.

The weights are a real difficulty.

Table 9.1 shows something else. Raising λ from 1 to 10 cuts the error at $t = 0$ roughly in half but leaves the overall error unchanged; raising it to 100 makes everything worse, because the optimiser now spends its capacity on the boundary at the expense of the equation, and L_{PDE} rises by nearly an order of magnitude. There is a best λ, it is problem-dependent, and we found it by trying three values.

The underlying reason is a conditioning problem of the kind discussed in Section 4.5. The four terms in (9.38) have gradients of very different magnitudes – L_{PDE} involves a second derivative of the network and therefore carries factors of $\boldsymbol{W}^2$, while L_{IC} involves the network value alone – so the composite cost is far more anisotropic than any of its parts. A single learning rate must serve all of them. Figure 9.4(a) shows the consequence: the four terms fall at different rates and by different amounts, and they do not fall smoothly. Choosing the weights automatically, by balancing gradient magnitudes or by using the neural tangent kernel, is an active area of research and is not a solved problem.

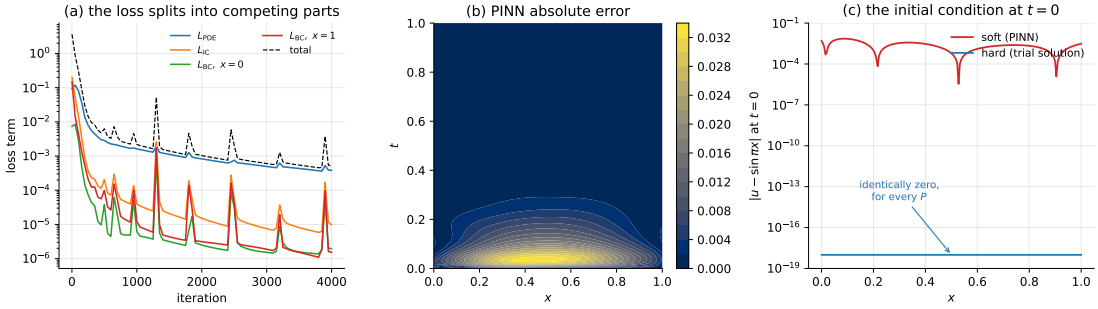


Fig. 9.4 The diffusion equation solved with soft constraints. (a) The four terms of Eq. (9.38) during training, with $\lambda = 10$; they descend at different rates and the intermittent spikes are the terms trading against one another. (b) Absolute error of the converged solution, largest in the fast-decaying region near $t = 0$ rather than on the boundaries. (c) The initial condition at $t = 0$: the soft formulation misses it by up to 7×10^{-3}, while the hard formulation of Section 9.7 satisfies it identically, for every value of the parameters.

Panel (c) of the figure makes the comparison as sharply as it can be made. The hard-constraint curve is not small; it is zero, and it would be zero for an untrained network. The soft-constraint curve is a genuine error that training reduces but never removes.

The wave equation revisited.

Where the soft formulation pays off in convenience is the initial *velocity*. Recall the trouble it caused in Section 9.8: to make $\partial g_t/\partial t$ vanish at $t=0$ we had to replace t by t^2 throughout Eq. (9.33), and had $v(x)$ been non-zero we would have needed a further term $t\,v(x)$ whose interaction with the rest of the ansatz must be checked by hand. A condition on a derivative, imposed exactly, takes thought.

Imposed softly it takes one line, because a derivative residual is no different from any other residual:

```
def u_net(P, X): return network(P, X, "tanh")
u_t, u_x = d_dxk(u_net, 1), d_dxk(u_net, 0)
u_tt, u_xx = d_dxk(u_t, 1), d_dxk(u_x, 0)

def r_pde(P, X): return u_tt(P, X) - c**2 * u_xx(P, X)    # Eq. (9.wave)
def r_ic(P, X):  return u_net(P, X) - np.sin(np.pi * X[:, 0])
def r_iv(P, X):  return u_t(P, X)                         # dg/dt = v(x) = 0 at t=0
def r_bc(P, X):  return u_net(P, X)

terms = [("pde", 1.0, r_pde, X_col), ("ic", 10.0, r_ic, X_ic),
         ("iv",  10.0, r_iv,  X_ic),                     # the initial velocity
         ("bcL", 10.0, r_bc,  X_l), ("bcR", 10.0, r_bc, X_r)]

P, history = pinn_solve(terms, [2, 30, 30, 1], "tanh", n_iter=4000, gamma=1e-2,
                        rng=np.random.default_rng(1))
```

The line `r_iv` is the entire treatment of the initial velocity. Changing $v(x)$ from zero to anything else means editing that one line; no part of the construction has to be rethought. A Neumann boundary condition $\partial u/\partial x = q$ would be added the same way, and so would a Robin condition $\alpha u + \beta\,\partial u/\partial x = q$, neither of which fits comfortably into a trial solution.

The accuracy, however, tells the same story as before:

```
Soft, lambda=10,  800 it : max err 1.20e-02   rmse 4.44e-03
Soft, lambda=10, 4000 it : max err 8.73e-03   rmse 3.34e-03
Soft, lambda=50, 4000 it : max err 1.39e-02   rmse 4.99e-03
Hard, Eq. (9.trialwave),  800 it : max err 8.07e-03   rmse 2.21e-03
Hard, Eq. (9.trialwave), 4000 it : max err 4.81e-04   rmse 1.80e-04
```

The soft solution after 4000 iterations is about as good as the hard solution after 800, and the hard solution given the same 4000 iterations is *eighteen times* better. The gap here is wider than for the diffusion equation, and the reason is instructive: the standing wave $\cos(\pi t)\sin(\pi x)$ has amplitude one throughout the domain, so the boundary terms never become negligible and continue to compete with the equation residual for the whole of training. In the diffusion problem the solution decays by a factor of 2×10^4 and the competition fades.

So why use soft constraints at all?

Because of the last item in Section 9.9. Suppose the diffusion coefficient D in

$$\frac{\partial u}{\partial t} = D\,\frac{\partial^2 u}{\partial x^2} \tag{9.39}$$

is *unknown*, and instead we have a handful of noisy measurements $\{(\boldsymbol{x}_i, y_i)\}$ of the solution at scattered points. This is an inverse problem, and for a classical solver it is a serious undertaking: one wraps the solver in an outer optimisation loop and differentiates through it, usually via an adjoint equation derived by hand for the problem at hand.

For a physics-informed network it is one more term in the cost and one more number in the parameter list. We minimise

$$L(P,D)=\frac{1}{N_{\text{pde}}}\sum_i\left(\frac{\partial N}{\partial t}\Big|_i - D\,\frac{\partial^2 N}{\partial x^2}\Big|_i\right)^2+\lambda_{\text{data}}\frac{1}{N_{\text{data}}}\sum_i\big(N(\boldsymbol{x}_i,P)-y_i\big)^2 \tag{9.40}$$

over the weights and D together. Note what is *not* in Eq. (9.40): no initial condition and no boundary conditions. We are not told them; the data replace them. A trial solution cannot even be written down here, so the soft formulation is not merely more convenient, it is the only one available.

Since D is not a weight matrix, the optimiser must accept a slightly more general parameter object. Flattening is the tidy way to arrange this:

```
from autograd.misc import flatten

def adam_general(cost, params, n_iter=2000, gamma=1e-2, b1=0.9, b2=0.999, eps=1e-8):
    """Adam on any nested list of arrays -- here the weights and the scalar D."""
    flat, unflatten = flatten(params)
    gradient = grad(lambda f: cost(unflatten(f)))
    m = np.zeros_like(flat)
    v = np.zeros_like(flat)
    for it in range(1, n_iter + 1):
        g = gradient(flat)
        m = b1 * m + (1 - b1) * g
        v = b2 * v + (1 - b2) * g ** 2
        flat = flat - gamma * (m / (1 - b1**it)) / (np.sqrt(v / (1 - b2**it)) + eps)
    return unflatten(flat)
```

The problem itself is then four lines. We generate forty observations from the exact solution with $D_{\text{true}}=0.5$, corrupt them with Gaussian noise of standard deviation 0.01, and start the search from $D_0=2.0$, four times too large.

```
D_true = 0.5
rng = np.random.default_rng(3)
X_obs = np.column_stack([rng.uniform(0, 1, 40), rng.uniform(0, 1, 40)])
y_obs = np.exp(-D_true * np.pi**2 * X_obs[:, 1]) * np.sin(np.pi * X_obs[:, 0]) \
        + rng.normal(0, 0.01, 40)

def u_net(P, X): return network(P[0], X, "tanh")     # P = [weights, D]
u_t, u_x = d_dxk(u_net, 1), d_dxk(u_net, 0)
u_xx = d_dxk(u_x, 0)

def cost(P):
    D = P[1][0]
    return np.mean((u_t(P, X_col) - D * u_xx(P, X_col)) ** 2) \
         + 10.0 * np.mean((u_net(P, X_obs) - y_obs) ** 2)   # Eq. (9.pinninv)

P = [init_parameters([2, 30, 30, 1], "tanh", np.random.default_rng(1)),
     np.array([2.0])]                                  # initial guess D_0 = 2
P = adam_general(cost, P, n_iter=3000, gamma=1e-2)
```

```
  it     1  cost 1.8735e+00  D 1.990000
  it   500  cost 2.5141e-02  D 1.673616
  it  1000  cost 7.0864e-03  D 0.637187
  it  1500  cost 2.7637e-03  D 0.496297
  it  2000  cost 2.3463e-03  D 0.512378
  it  2500  cost 1.4782e-03  D 0.507987
  it  3000  cost 1.3524e-03  D 0.515397

D_true = 0.5, recovered D = 0.515397, rel err 3.08%
```

Forty noisy points and no boundary information at all recover the coefficient to three per cent, and Figure 9.5(a) shows that starting values of $D_0 = 2.0$, 1.0 and 0.1 all converge to the same answer. The network and the coefficient are found together, in one optimisation, by the same Adam update, and the only line of code that knows this is an inverse problem is `D = P[1][0]`.

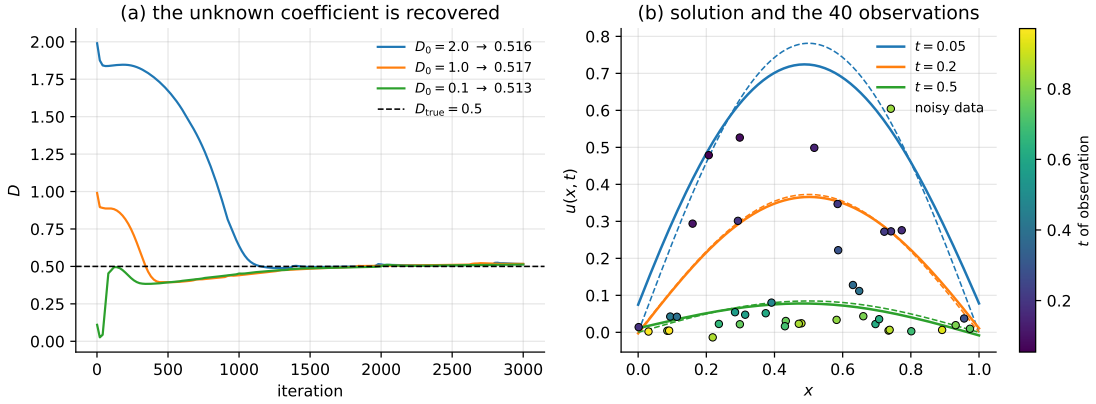


Fig. 9.5 Recovering an unknown diffusion coefficient from forty noisy observations. (a) The estimate of D during training from three different starting values, converging to $D \approx 0.515$ against a true value of 0.5. (b) The recovered solution at three times, solid, against the exact solution, dashed, with the observations shaded by the time at which they were taken. The visible gap at $t = 0.05$ is the three per cent error in D.

The result should not be oversold. Inverse problems are ill-posed, and the recovered coefficient degrades quickly as the data get noisier:

```
noise   D_recovered   rel.err
 0.00   0.4770         4.59%
 0.01   0.5154         3.08%
 0.05   0.6888        37.76%
```

The first two rows differ by less than the scatter between random seeds and should be read as the same result. The third should not: five per cent noise on the observations produces a thirty-eight per cent error in D. The sensitivity is a property of the inverse problem and not of the method – a classical adjoint-based inversion on the same data would face it too – but it is a reminder that recovering parameters from data is a statistical estimation problem, with all that Chapter 2 implies about variance and about the danger of reporting a single number without an error estimate.

Soft or hard?

The two formulations are not rivals so much as two points on a scale, and the question is how much of what we know can be built into the ansatz rather than requested through the cost.

- If a trial solution can be constructed, construct it. The conditions then hold exactly, the cost has one term, there are no weights to tune, and Table 9.1 shows the accuracy that buys.
- If the geometry is awkward, the conditions involve derivatives or mixed terms, or the equations are coupled, use soft constraints. The loss of accuracy is real but bounded, and the alternative may be no method at all.

- If parameters of the equation are unknown and data are available, soft constraints are the only option, and they are extremely convenient.
- Nothing prevents mixing the two. A trial solution can enforce the boundary conditions exactly while an initial condition is handled by a penalty, and this hybrid is often the best of the three.

A last practical remark, and it applies to both formulations. Both networks above were trained on a 20×20 grid of points and evaluated on a 100×100 grid; the errors quoted throughout are the evaluation-grid errors. This is not interpolation in the finite-difference sense but simple evaluation of a closed-form function, and it is one thing the method genuinely does better than a grid-based scheme, which returns numbers at grid points and nothing in between.

9.11 A stochastic equation: Black-Scholes option pricing

Every equation solved so far was deterministic and had homogeneous boundary conditions on a unit box. The Black-Scholes equation of mathematical finance has neither property, and it arrives by a different route: it is the deterministic equation satisfied by the price of a derivative whose underlying asset follows a *stochastic* process. It therefore makes a good final test, and it is the problem for which the physics-informed formulation of Section 9.10 is not a convenience but a necessity.

9.11.1 From a stochastic process to a deterministic equation

Assume the price S_t of an asset follows geometric Brownian motion,

$$\mathrm{d}S_t = \mu S_t\,\mathrm{d}t + \sigma S_t\,\mathrm{d}W_t, \tag{9.41}$$

where μ is the drift, $\sigma>0$ the volatility and W_t a Wiener process, so that $\mathrm{d}W_t$ is normally distributed with mean zero and variance $\mathrm{d}t$. The multiplicative form matters: it keeps $S_t>0$ and makes the *relative* return $\mathrm{d}S_t/S_t$ the quantity with stationary statistics, which is what the empirical distributions of Chapter 2 suggest for asset prices.

Let $C(S,t)$ be the price of a derivative written on this asset. Because S_t is a stochastic process, ordinary calculus does not apply to $C(S_t,t)$; the correct chain rule is Itô's lemma, which for a twice-differentiable C reads

$$\mathrm{d}C = \left(\frac{\partial C}{\partial t} + \mu S\frac{\partial C}{\partial S} + \frac{1}{2}\sigma^2 S^2\frac{\partial^2 C}{\partial S^2}\right)\mathrm{d}t + \sigma S\frac{\partial C}{\partial S}\,\mathrm{d}W_t. \tag{9.42}$$

The extra second-derivative term, absent from the deterministic chain rule, is the whole content of Itô calculus: it appears because $(\mathrm{d}W_t)^2=\mathrm{d}t$ rather than $\mathcal{O}(\mathrm{d}t^2)$, so the second-order term in a Taylor expansion survives the limit.

Now form the portfolio $\Pi = C-\Delta S$ consisting of one derivative and a short position of Δ units of the asset. Using Eqs. (9.41) and (9.42),

$$\mathrm{d}\Pi = \left(\frac{\partial C}{\partial t} + \frac{1}{2}\sigma^2 S^2\frac{\partial^2 C}{\partial S^2}\right)\mathrm{d}t + \left(\frac{\partial C}{\partial S} - \Delta\right)\left(\mu S\,\mathrm{d}t + \sigma S\,\mathrm{d}W_t\right). \tag{9.43}$$

The choice

$$\Delta = \frac{\partial C}{\partial S} \tag{9.44}$$

annihilates the second bracket, and with it the $\mathrm{d}W_t$ term. The portfolio has become *riskless* over the infinitesimal interval, and a riskless portfolio must earn the risk-free rate r on pain of arbitrage, $\mathrm{d}\Pi = r\Pi\,\mathrm{d}t$. Equating the two expressions and dividing by $\mathrm{d}t$ gives the Black-Scholes equation,

$$\boxed{\frac{\partial C}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 C}{\partial S^2} + rS\frac{\partial C}{\partial S} - rC = 0} \tag{9.45}$$

on $S > 0$, $0 \le t \le T$.

Two features of this derivation are worth recording. The drift μ has *disappeared*: the price of the derivative does not depend on how fast the underlying is expected to grow, only on how much it fluctuates. And the equation is deterministic even though its subject is stochastic – the randomness has been hedged away by Eq. (9.44), and what remains is a partial differential equation of exactly the kind this chapter has been solving.

9.11.2 Conditions, and the link to the diffusion equation

The contract is specified by conditions at maturity rather than at the start. For a European call option – the right, but not the obligation, to buy one unit of the asset at the strike price K at time T – the holder exercises only if $S > K$, so

$$C(S,T) = \max(S-K, 0). \tag{9.46}$$

This is a *terminal* condition, and Eq. (9.45) is a backward parabolic equation: it is well posed running backwards from $t = T$, exactly as the diffusion equation is well posed running forwards from $t = 0$. The substitution $\tau = T - t$ turns one into the other and is the first thing to do in any implementation.

The boundary conditions in S follow from arbitrage arguments. If the asset becomes worthless it stays worthless, by Eq. (9.41), so the option expires unexercised:

$$C(0,t) = 0. \tag{9.47}$$

If instead S is very large the option will certainly be exercised, and holding it is equivalent to holding the asset less the discounted strike:

$$C(S,t) \;\longrightarrow\; S - Ke^{-r(T-t)} \qquad \text{as } S \to \infty. \tag{9.48}$$

In practice the domain is truncated at some $S_{\max} \gg K$ and Eq. (9.48) imposed there.

This is the diffusion equation in disguise. Set $x = \ln(S/K)$, $\tau = \frac{1}{2}\sigma^2(T-t)$ and $u(x,\tau) = e^{\alpha x + \beta\tau}C(S,t)$. The chain rule gives $\partial u/\partial x = e^{\alpha x+\beta\tau}(\alpha C + SC_S)$ and $\partial^2 u/\partial x^2 = e^{\alpha x+\beta\tau}(\alpha^2 C + (2\alpha+1)SC_S + S^2 C_{SS})$, and the choices

$$2\alpha + 1 = \frac{2r}{\sigma^2}, \qquad \beta = \frac{r}{\sigma^2}\,(\alpha+1)$$

reduce Eq. (9.45) to $u_\tau = \frac{1}{2}\sigma^2 u_{xx}$, which is Eq. (9.26) up to the constant. The logarithm is what removes the variable coefficients S and S^2; geometric Brownian motion is ordinary Brownian motion in $\log S$. We do *not* exploit this in the solver below, and deliberately

so: the transformation exists for this equation and not for the multi-asset or stochastic-volatility generalisations, whereas the network formulation is indifferent.

Because a closed form is available for this one-dimensional case, we can check the answer. The Black-Scholes formula is

$$C(S,t) = SN(d_1) - Ke^{-r(T-t)}N(d_2), \tag{9.49}$$

with N the standard normal cumulative distribution and

$$d_1 = \frac{\ln(S/K) + \left(r + \frac{1}{2}\sigma^2\right)(T-t)}{\sigma\sqrt{T-t}}, \qquad d_2 = d_1 - \sigma\sqrt{T-t}. \tag{9.50}$$

9.11.3 Why a trial solution will not do

Every construction of Sections 9.7 and 9.8 rested on finding h_1 satisfying the conditions and h_2 vanishing where they are imposed. Here that programme fails at the first step. The condition at the far boundary, Eq. (9.48), is inhomogeneous and time-dependent; the terminal condition (9.46) is not differentiable at $S = K$; and the domain is a truncation of a half-line rather than a box. One can, with effort, manufacture an h_1 that interpolates the three conditions, but it will be elaborate, problem-specific, and worthless the moment a second asset or a non-constant volatility is introduced.

The soft formulation asks for none of it. Each condition becomes a residual, exactly as in Eq. (9.38):

$$L_{\text{total}} = L_{\text{PDE}} + \lambda\left(L_{\text{term}} + L_{\text{low}} + L_{\text{high}}\right), \tag{9.51}$$

with L_{PDE} the mean square of Eq. (9.45) at interior collocation points and the other three the mean squares of the misfits in Eqs. (9.46), (9.47) and (9.48).

9.11.4 Scaling matters more than anything else

Before any code, a remark that turns out to dominate the result. In the variables of Eq. (9.45) the network is asked to take inputs $S \in [0,20]$ and $t \in [0,1]$ and produce outputs in $[0,15]$. A tanh network initialised by Eq. (8.73) produces outputs of order one from inputs of order one; feeding it $S = 20$ saturates the first layer immediately. Worse, the residual (9.45) contains $S^2 C_{SS}$, whose magnitude varies by a factor of 400 across the domain, so the collocation points near $S_{\max}$ dominate L_{PDE} and the region near the strike – the only region anyone cares about – is effectively ignored.

The remedy is to non-dimensionalise. Measure the asset price in units of the strike and the option price likewise, and run time backwards:

$$s = \frac{S}{K}, \qquad \tau = T - t, \qquad c(s,\tau) = \frac{C(S,t)}{K}, \tag{9.52}$$

under which Eq. (9.45) becomes

$$-\frac{\partial c}{\partial \tau} + \frac{1}{2}\sigma^2 s^2 \frac{\partial^2 c}{\partial s^2} + rs\frac{\partial c}{\partial s} - rc = 0, \tag{9.53}$$

with terminal condition $c(s,0) = \max(s-1,0)$ now an *initial* condition. The equation is unchanged in form – it is homogeneous of degree one in the price, which is why the strike is the natural unit – but the inputs and outputs are now of order one.

Table 9.2 shows what this is worth. Both runs use the same network, the same optimiser, the same weights, the same number of iterations and the same seed; they differ only in the choice of variables.

Table 9.2 The Black-Scholes equation with $K=5$, $T=1$, $r=0.05$, $\sigma=0.3$ and $S_{\max}=20$, solved by a $[2,40,40,1]$ network with tanh activation, Adam with $\gamma = 5\times10^{-3}$, $\lambda = 10$ and 4000 iterations. Errors are in units of the option price and measured on a 120×120 grid against Eq. (9.49); the largest option value on the grid is about 15.

Variables	max error	RMSE	final L_{PDE}
(S,t) as written, Eq. (9.45)	0.498	0.272	2.32×10^{-2}
(s,τ) scaled, Eq. (9.53)	**0.318**	**0.042**	$\mathbf{1.59\times10^{-4}}$

Non-dimensionalising improves the root-mean-square error by a factor of 6.5 and the equation residual by a factor of 145, at no computational cost whatsoever. This is the single most useful practical fact in the chapter, and it generalises: before training a physics-informed network on any problem with dimensional quantities, rescale so that inputs, outputs and each term of the residual are of order one.

9.11.5 Implementation

The code is the machinery of Section 9.10 with a different residual.

```
K, T, r, sigma, S_max = 5.0, 1.0, 0.05, 0.3, 20.0
hi = S_max / K                                  # domain in scaled units

def c_net(P, X): return network(P, X, "tanh")   # X = (s, tau)

c_tau = d_dxk(c_net, 1)
c_s = d_dxk(c_net, 0)
c_ss = d_dxk(c_s, 0)

def r_pde(P, X):                                # Eq. (9.bsscaledpde)
    s = X[:, 0]
    return (-c_tau(P, X) + 0.5 * sigma**2 * s**2 * c_ss(P, X)
            + r * s * c_s(P, X) - r * c_net(P, X))

def r_term(P, X):                               # payoff at tau = 0
    return c_net(P, X) - np.maximum(X[:, 0] - 1.0, 0.0)

def r_lo(P, X):                                 # c(0, tau) = 0
    return c_net(P, X)

def r_hi(P, X):                                 # c(hi, tau) = hi - exp(-r tau)
    return c_net(P, X) - (hi - np.exp(-r * X[:, 1]))

terms = [("pde", 1.0, r_pde, X_col), ("term", 10.0, r_term, X_term),
         ("lo", 10.0, r_lo, X_lo),   ("hi", 10.0, r_hi, X_hi)]

P, history = pinn_solve(terms, [2, 40, 40, 1], "tanh", n_iter=4000, gamma=5e-3,
                        rng=np.random.default_rng(1))
```

Nothing here is specific to finance beyond the four residuals. Replacing `r_pde` by a two-asset Black-Scholes equation with a correlation term, or `r_term` by the payoff of a different contract, is a one-line change in each case, and neither requires rethinking the construction.

9.11.6 Results

The prices at $t = 0$, which is what a trader would actually want, come out as

```
   S   network    exact   abs err
 2.0    0.0050   0.0005    0.0044
 4.0    0.2437   0.2277    0.0160
 5.0    0.7301   0.7116    0.0186
 6.0    1.4423   1.4440    0.0017
 8.0    3.2608   3.2748    0.0139
12.0    7.2684   7.2445    0.0239
```

so that an option worth \$0.71 at the money is priced to about two cents, and one worth \$7.24 deep in the money to about two cents as well. Whether that is good enough depends entirely on the application, and it is an order of magnitude worse than the closed form, which is exact.

More interesting is *where* the error lives. The largest discrepancy over the whole domain is 0.318, and it occurs at

```
max err 0.3179 at S=5.04 t=1.000  (K=5.0 T=1.0)
err  t>0.9 (near maturity): 0.3179    t<0.5: 0.1065
err  |S-K|<1 (near strike): 0.3179    |S-K|>3: 0.1458
```

that is, at $S = K$ and $t = T$ to within one grid spacing – precisely the corner where the payoff (9.46) has its kink. This is not an accident of training and it will not go away with more iterations. The exact solution is continuous but not differentiable there, while a tanh network is infinitely differentiable everywhere; the network is being asked to represent a function it structurally cannot represent, and it does the best a smooth function can do, which is to round the corner off. Away from the kink, at $t < 0.5$ or more than three strikes from the money, the error falls by a factor of two to three. Figure 9.6(b) shows the whole error surface and the point where it peaks.

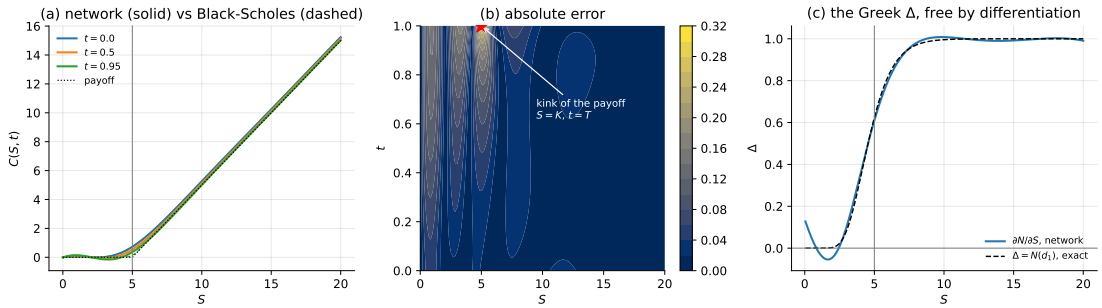


Fig. 9.6 Black-Scholes by a physics-informed network. (a) The option price at three times, network solid and Eq. (9.49) dashed, with the terminal payoff (9.46) dotted; the vertical line is the strike. (b) Absolute error over the whole domain, peaking at the starred point $S = K$, $t = T$ where the payoff has its kink and the exact solution is not differentiable. (c) The hedge ratio $\Delta = \partial C/\partial S$ obtained by differentiating the trained network, against the exact $N(d_1)$. The network reproduces it without having been asked to, but goes slightly negative near $S = 2$, which no true option price can do.

The Greeks come for free.

A practitioner needs not only the price but its sensitivities, of which the most important is the hedge ratio $\Delta = \partial C/\partial S$ that appeared in Eq. (9.44). A finite-difference or lattice solver returns prices on a grid and Δ must be recovered by numerical differentiation, which amplifies the error. The network *is* a differentiable closed-form function, so Δ is one call to `d_dxk`:

```
   S   network    exact   abs err
 2.0   -0.0441   0.0031   0.0472
 4.0    0.3553   0.3346   0.0206
 5.0    0.6103   0.6243   0.0139
 6.0    0.8012   0.8224   0.0212
 8.0    0.9795   0.9702   0.0094
12.0    0.9965   0.9994   0.0029
```

Against $\Delta = N(d_1)$ this is accurate to about two per cent over the range that matters, and the network was never trained on Δ at all. It was trained on Eq. (9.45), and the derivative it produces is a consequence. This is the concrete form of the "solution as a function" argument of Section 9.9.

An honest failure.

The first line of that table should not be passed over. At $S = 2$, deep out of the money, the network reports $\Delta = -0.044$. A negative delta on a call option is impossible: the price of a call is non-decreasing in the price of the underlying, since a call on a more valuable asset cannot be worth less. The network has produced a small violation of a structural property that the exact solution satisfies identically, in a region where the option is nearly worthless and the residual is correspondingly insensitive. Figure 9.6(c) shows the dip.

This is worth generalising. The soft formulation enforces the equation and the conditions only in the mean-square sense, and *nothing else at all*. Monotonicity, convexity in S, the no-arbitrage bounds $\max(S - Ke^{-r(T-t)}, 0) \le C \le S$ – none of these is in the loss, so none is guaranteed, and a small residual is perfectly compatible with violating them where the solution is flat. A finite-difference scheme on this problem can be shown to preserve monotonicity under a step-size condition. We have traded a method with structural guarantees for one with flexibility, and this is what that trade costs. If such properties matter, they must be added to the loss as further penalty terms, or enforced by construction.

Why bother, if there is a closed form? For a European call on a single asset there is no case for a network: Eq. (9.49) is exact and evaluates in microseconds. The case begins where the closed form ends. A basket option on d correlated assets satisfies a d-dimensional analogue of Eq. (9.45) with a full covariance matrix in the second-derivative term, for which no closed form exists and a finite-difference grid needs n^d points; at $d = 5$ this is already hopeless and the collocation approach of Section 9.6 is not. American options, which may be exercised early, turn the equation into a free boundary problem. Stochastic volatility replaces the constant σ by a second stochastic process and adds a dimension. And calibration – inferring the volatility surface from quoted market prices – is precisely the inverse problem of Section 9.10, with σ in place of D. The one-dimensional call is the test case, not the application.

9.12 Summary and the programs

This chapter used the network of Chapter 8 unchanged and asked it a different question. There were no labels; the differential equation supplied the only information, through the residual, and minimising the mean squared residual (9.3) was the training.

Three constructions carry the method. The *trial solution* $g_t = h_1 + h_2(x, N)$ builds the boundary and initial conditions into the ansatz so that they hold identically, for every value of the parameters – which is why the error of the diffusion solution at $t = 0$ was exactly zero rather than merely small. The optimiser is then asked to satisfy the equation and nothing else. And the *forward-mode recursions* (9.5) and (9.6) supply the derivatives of the network with respect to its input, propagating alongside the ordinary forward pass at a cost of one extra matrix product per layer per order. The third is the *composite cost* (9.38) of the physics-informed network, which abandons the trial solution and asks the optimiser for the conditions instead. Where both formulations apply, the trial solution is more accurate by a factor of three to eighteen at equal cost; the composite cost earns its place where no trial solution can be written down.

That second construction made concrete a claim left hanging in Chapter 8. Equation (9.6) requires f'', and for ReLU and leaky ReLU this vanishes identically: the measured $\max|\partial^2 N/\partial x^2|$ was exactly 0 for both, against 10^{-2} to 1 for every smooth activation. A rectified network cannot solve a second-order equation, and worse, it will report a residual of zero while doing so.

Six problems were solved and every one checked against its exact solution: exponential decay, logistic growth, the one-dimensional Poisson equation, the diffusion and wave equations – the last two in both the hard and the soft formulation – and the Black-Scholes equation of option pricing. Two were also compared against classical schemes, and the comparison was not flattering. Forward Euler and the three-point finite-difference formula are faster by orders of magnitude and comparably accurate; the network's case rests instead on high dimension, irregular geometry, a differentiable closed-form solution, and above all on inverse problems, where data and equation can be combined in a single cost. Section 9.10 made that last case concrete by recovering an unknown diffusion coefficient, $D = 0.515$ against a true 0.5, from forty noisy observations and no boundary information at all – a calculation for which the trial-solution formulation has nothing to offer, since there is no condition to build in.

Section 9.11 added the other half of the argument. The Black-Scholes equation (9.45) is the deterministic equation left behind when the randomness of geometric Brownian motion is hedged away, and its conditions – inhomogeneous, time-dependent, imposed at maturity rather than at the start, and non-differentiable at the strike – defeat the trial-solution construction entirely. Three lessons came out of it. Non-dimensionalising the problem, Eq. (9.52), improved the root-mean-square error by a factor of 6.5 and the residual by a factor of 145 for no cost at all, and is the first thing to try on any dimensional problem. The largest error sat exactly at the kink of the payoff, at $S = K$ and $t = T$, because a smooth network cannot represent a corner. And the hedge ratio $\Delta = \partial C/\partial S$ came out to two per cent by differentiating a network that was never trained on it – against which must be set the fact that the same network reported a *negative* Δ deep out of the money, violating a no-arbitrage property that nothing in the loss had asked it to respect.

The failure recorded in Section 9.3 deserves to be remembered. A network of the wrong width settled into a local minimum at cost 31.9 and stayed there for ten thousand iterations, returning a solution wrong by eighteen per cent with no indication that anything was amiss. Non-convexity is not an abstraction, and a method that can converge confidently to the wrong answer must always be checked against something.

The programs are collected in the directory
`doc/BookML/BookPrograms/chapter09_differential_equations`.

Every listing printed above appears there as a numbered file, extracted automatically and in chapter order, and three of them are also provided as self-contained modules that run start to finish and reproduce the numbers quoted in the text:

- `nn_de.py` – the network of Chapter 8 extended with the input-derivative recursions of Section 9.2, the Adam minimiser and `solve_de`. Everything else imports from here.
- `pinn.py` – `pinn_solve`, the composite cost (9.38), and the diffusion and wave equations in the soft formulation. Running it reproduces the soft-constraint rows of Table 9.1 and the wave-equation errors of Section 9.10.
- `pinn_inverse.py` – `adam_general` and the recovery of the unknown diffusion coefficient from noisy data, together with the noise study.
- `blackscholes.py` – the Black-Scholes residuals of Section 9.11.5, the scaled and unscaled runs of Table 9.2, the price and Δ tables, and the comparison against the closed form (9.49).

The figures are generated by the three scripts `ch09_figures.py`, `ch09_pinn_figures_a.py` and `ch09_pinn_figures_b.py` in `doc/BookML/BookFigures`; none is drawn by hand.

9.13 Exercises

Warm-up exercises

1. **Trial solutions.** Construct a trial solution of the form (9.2) for each of the following, and verify that the conditions hold identically: (a) $g(0) = a$ and $g(1) = b$ on $[0,1]$; (b) $g(0) = a$ and $g'(0) = v$; (c) g periodic on $[0,1]$, that is $g(0) = g(1)$ and $g'(0) = g'(1)$.
2. **The derivative recursions.** (a) Derive Eq. (9.5) from the forward pass (9.4). (b) Derive Eq. (9.6), being explicit about where the product rule is used. (c) Show that for a linear output layer no f' appears in the last step. (d) Count the operations and confirm that N, N' and N'' together cost about three forward passes.
3. **Why ReLU fails.** (a) Show that $\mathrm{ReLU}'' \equiv 0$ wherever it is defined and hence that Eq. (9.6) gives zero throughout. (b) Confirm this numerically for a randomly initialised network. (c) Solve the Poisson problem of Section 9.5 with a ReLU network and report both the final residual and the actual error. Explain the discrepancy. (d) Repeat with GELU, Swish and tanh and compare.
4. **Exponential decay by hand.** For the trial solution (9.9), write out the residual for a network with a single hidden unit, $N(x) = w_2 \tanh(w_1 x + b_1) + b_2$, and obtain $\partial C/\partial w_2$ analytically. Verify against automatic differentiation.
5. **Collocation points (numerical).** For the Poisson problem, vary the number of collocation points from 10 to 200. (a) Plot the final error against the number of points. (b) Compare with the $\mathcal{O}(\Delta x^2)$ behaviour of finite differences. (c) Try randomly sampled rather than uniformly spaced points, and comment.
6. **Reproducing the local minimum.** Reproduce the failure described in Section 9.3: two hidden layers of twenty units, $\gamma = 10^{-2}$. (a) Confirm that the cost stalls near 31.9 and does not move. (b) Plot the solution it returns against the exact one. (c) Find the smallest change – in width, learning rate or seed – that escapes it. (d) What diagnostic would have revealed the failure without knowing the exact solution?
7. **Activation functions (numerical).** Solve the diffusion equation with tanh, GELU, Swish, sigmoid and softplus, five seeds each. Report the median error and the spread, and compare your conclusions with Table 8.2 of Chapter 8.

8. **A harder initial condition.** Solve the diffusion equation with $u(x) = \sin(\pi x) + \frac{1}{2}\sin(3\pi x)$, whose exact solution superposes two modes decaying at different rates. (a) Verify against the exact solution. (b) Which mode is captured less well, and why?
9. **Non-zero initial velocity.** Modify the wave-equation trial solution (9.33) to accommodate $v(x) \neq 0$, as suggested in Section 9.8. Verify that both conditions hold identically, then solve with $v(x) = \sin(2\pi x)$ and check against the exact solution obtained by separation of variables.
10. **Comparison with classical methods.** For the diffusion equation, implement an explicit finite-difference scheme and compare against the network in accuracy, runtime and memory as the grid is refined. At what point, if any, does the network become competitive? Then repeat the argument in your head for a five-dimensional domain and explain why the answer changes.

Project-style exercise: a differential-equation solver

Part a: the machinery.

Implement the input-derivative recursions of Section 9.2 on top of your Chapter 8 network, and verify them against automatic differentiation to machine precision for every activation function. Produce the second-derivative table and confirm that the rectified family gives exactly zero.

Part b: ordinary equations.

Solve the three ODEs of Sections 9.3 to 9.5, checking each against its exact solution. For each, compare against a classical scheme of your choice at matched cost, and present the comparison honestly.

Part c: robustness.

For each problem, run ten seeds and report the median and spread of the error. Study the dependence on width, depth, learning rate and number of iterations. Identify at least one configuration that fails, and diagnose it.

Part d: partial equations.

Solve the diffusion and wave equations with trial solutions. Plot the solution as a surface and as time slices against the exact solution, and verify that the conditions built into the trial solution hold to machine precision throughout training – not merely at the end.

Part e: a problem without an exact solution.

Solve a problem for which no closed form is available – a non-linear Poisson equation, or diffusion with a spatially varying coefficient. Since you cannot check against the truth, design your own verification: mesh refinement of a classical solver, the residual on points not used

for training, and consistency across seeds. Discuss what would convince you that the answer is right.

Part f: soft constraints.

Re-solve both partial differential equations of part d with the composite cost (9.38) instead of a trial solution, and reproduce Table 9.1 for your own implementation. Scan $\lambda \in \{1, 10, 100, 1000\}$ and plot the four loss terms separately against the iteration count, as in Figure 9.4(a). Then answer two questions with evidence. Does the best λ for the diffusion equation remain the best for the wave equation? And is the error at $t = 0$ ever driven below the interior error, or does the initial condition remain the worst-satisfied part of the problem however λ is chosen?

Part g: an inverse problem.

Take the diffusion equation with an unknown coefficient D, generate synthetic noisy measurements at scattered points, and recover D as in Section 9.10. Go beyond the worked example in three directions. Study how the recovered D degrades as the measurements become sparser as well as noisier, and map the region of the (noise, N_{data}) plane in which the answer is usable. Attach an uncertainty to D by bootstrapping the observations, using the machinery of Chapter 2, and report an interval rather than a number. Finally, make the coefficient a function $D(x)$ rather than a constant – represent it by a second small network – and investigate at what point the problem becomes too ill-posed to solve.

Part h: option pricing.

Reproduce Section 9.11. (a) Derive Eq. (9.45) from Eqs. (9.41) and (9.42), and say precisely where the drift μ cancels and why that is remarkable. (b) Verify the change of variables in the notebox of Section 9.11.2 by carrying out the chain rule and solving for α and β. (c) Solve the equation with the composite cost and reproduce Table 9.2, confirming for yourself how much the scaling is worth. (d) Locate the largest error and confirm that it sits at the kink of the payoff. Then replace the call payoff by a smooth approximation, for instance $\frac{1}{2}\left(S - K + \sqrt{(S-K)^2 + \varepsilon^2}\right)$, and study how the error at the corner behaves as $\varepsilon \to 0$. (e) Compute Δ, $\Gamma = \partial^2 C/\partial S^2$ and $\mathcal{V} = \partial C/\partial \sigma$ from the trained network. The last one requires thought: σ is not an input, so either retrain over a range of σ with volatility as a third input, or use a finite difference in σ over two trainings. Compare against the closed-form Greeks. (f) Check the no-arbitrage bounds $\max(S - Ke^{-r(T-t)}, 0) \le C \le S$ and the monotonicity and convexity of C in S over the whole grid, and report where your solution violates them. Then add a penalty term that punishes $\min(\partial C/\partial S, 0)$ and report whether it helps and what it costs elsewhere. (g) Extend to two correlated assets, for which

$$\frac{\partial C}{\partial t} + \frac{1}{2}\sum_{i,j=1}^{2} \rho_{ij}\sigma_i\sigma_j S_i S_j \frac{\partial^2 C}{\partial S_i \partial S_j} + r\sum_{i=1}^{2} S_i \frac{\partial C}{\partial S_i} - rC = 0,$$

with the payoff $\max(\max(S_1, S_2) - K, 0)$. There is no closed form, so verify against a Monte Carlo estimate using Eq. (9.41) and the machinery of Chapter 2, and state how many paths you need for the Monte Carlo error to fall below the network error.

Part i: a domain that is not a box.

Solve the Poisson equation $-\nabla^2 u = f$ on an L-shaped domain, or on an annulus, with $u = 0$ on the boundary. Sample collocation points in the interior and boundary points on the edges. Explain, in writing, why the trial-solution approach of Section 9.5 cannot be used here, and what that implies about the trade-off set out at the end of Section 9.10.

Chapter 10
Convolutional Neural Networks

The network of Chapter 8 treats its input as a vector. Whatever the data were before – an image, a sound clip, a spectrum – they are flattened into $\boldsymbol{x} \in \mathbb{R}^d$ and multiplied by a dense weight matrix. Every input coordinate is treated identically and interchangeably: permute the pixels of every image in the training set by one fixed permutation and the network learns exactly as well, because nothing in an affine map knows that pixel (i, j) sits next to pixel $(i, j+1)$.

For data with spatial or temporal structure this is throwing away information before training begins. Images, sound and volumetric scans share three properties that a vector does not record. They are stored as multi-dimensional arrays; they have one or more axes along which *ordering matters*; and they have a *channel* axis giving different views of the same location, such as the red, green and blue values of a pixel.

The convolutional neural network is the architecture obtained by insisting that the linear map respect this structure. This chapter builds it from the definition of convolution, establishes the properties that make it the right map – sparsity, parameter sharing and above all equivariance – derives the backward pass, and implements the whole thing from scratch in NumPy. It then does something the previous chapters did not: it loads the same weights into PyTorch and into TensorFlow and checks, gradient by gradient, that the three implementations compute the same function. That check is the point of writing a convolution by hand at all. Two further sections look at filters as objects in their own right: Section 10.7 applies hand-made filters to a digit, a drawn house and a photograph, builds a filter that detects a roof, and compares the result with what a network learns from random initial filters; Section 10.13 describes filters in the frequency domain, where they have always been designed, and follows that description into the layers of a network and into the places – signal processing, differential equations, the architectures themselves – where Fourier filters are used. Chapter 11 will make the corresponding restriction along a time axis and obtain recurrence, so the two chapters are a pair: one symmetry each, one architecture each.

The material draws on the lecture notes for weeks forty-three and forty-four of FYS-STK3155/4155, on the survey of convolution arithmetic by Dumoulin and Visin [44], and on the course's lecture on Fourier transforms and filtering.

10.1 Why not simply use a dense layer?

The objection is one of counting. Consider a colour image of side L, so the input has $3L^2$ components, and a first hidden layer of the same size. A dense layer connecting them has

$$N_{\text{dense}} = \left(3L^2\right)^2 = 9L^4 \tag{10.1}$$

weights. For the 32×32 images of CIFAR-10 this is already 9.4 million; for a modest 200×200 photograph it is 1.4×10^{10}, ten billion parameters in a single layer. Figure 10.3(c) plots the growth. The quartic scaling is fatal, and it is fatal twice over: such a layer cannot be stored, and even if it could, a model with more parameters than the training set has pixels will overfit catastrophically, as Section 2.10 would predict.

A convolutional layer with K filters of spatial extent F acting on C channels has, as we derive in Section 10.4.1,

$$N_{\text{conv}} = K\left(CF^2 + 1\right) \tag{10.2}$$

weights – *independent of the image size*. For $F = 3$, $C = 3$ and $K = 64$ this is 1792 parameters whether the image is 32×32 or 4096×4096. That is the flat dashed line in the figure.

The counting is not confined to images. A sound clip of $d = 10^6$ samples and one hidden layer of 10^4 units need a weight matrix with $10^4 \times 10^6 = 10^{10}$ entries; with the 10^4 biases and an output layer the total is $10^{10} + 10^4 + 10^4 + 1 \approx 10^{10}$, ten billion parameters, to decide whether the clip is a human voice. A 200×200 colour photograph costs 120 000 weights *per neuron* of the first hidden layer; a few hundred neurons are already tens of millions of parameters, wasteful and an invitation to overfit.

The saving is not free. It is bought by two structural assumptions, and the rest of this chapter is largely about what they are and when they are true:

- *Locality.* A given output depends only on a small neighbourhood of the input. Pixels close together are far more likely to be related than pixels far apart.
- *Stationarity, or parameter sharing.* The same weights are applied at every location. A feature worth detecting in one part of the image is worth detecting everywhere.

Both are properties of the data, not of the method. For a table of unrelated clinical measurements neither holds, and a convolutional layer would be an actively bad choice. For images, sound and time series both hold very well.

Figure 10.1 draws the two assumptions side by side on a one-dimensional input, in the layer notation of Section 8.5. The dense layer of panel (a) is one layer of Figure 8.5; the convolutional layer of panel (b) differs from it in exactly two respects, and those two respects are the whole of this chapter.

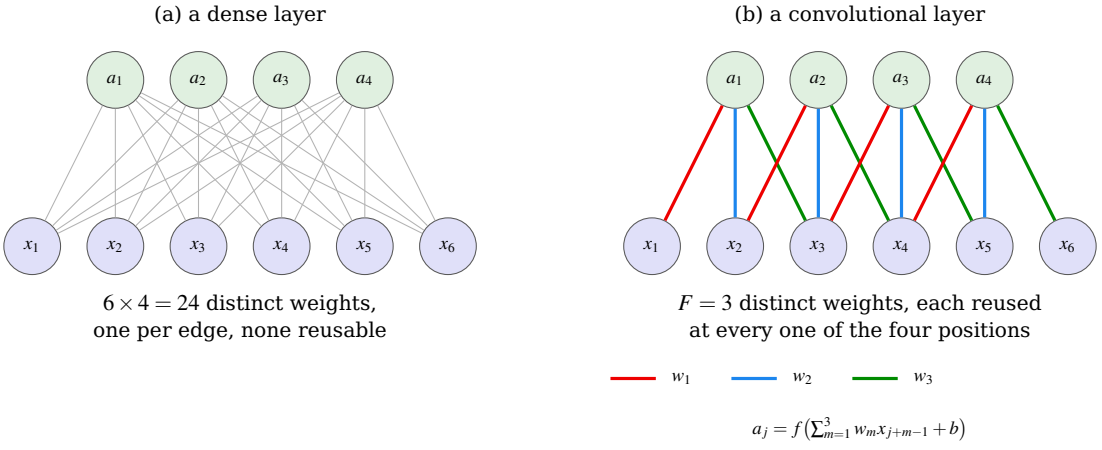


Fig. 10.1 The two structural assumptions of Section 10.1, drawn on a one-dimensional input of six components. (a) A dense layer connects every input to every output and gives each edge its own weight. (b) A convolutional layer with $F = 3$ connects each output to a window of three neighbouring inputs – that is locality – and uses the *same* three weights w_1, w_2, w_3 at every position, drawn here in the same three colours – that is parameter sharing. Twenty-four weights become three, and the three do not grow when the input is lengthened. Both pictures use the layer notation of Section 8.5: panel (a) is one layer of Figure 8.5.

10.2 What a convolutional network is: volumes and layers

Before the mathematics, the vocabulary. Convolutional networks were developed in the 1990s for the recognition of handwritten characters [45], and they are today the central element of the success of deep learning on images. It is worth saying at the outset how little is new. A convolutional network is made of neurons with learnable weights and biases; each neuron receives some inputs, computes a dot product and applies a non-linearity; the whole network is one differentiable function from raw pixels to class scores; there is a cost on the last layer – the softmax cross-entropy of Chapter 5 for classification – and everything we know about training applies unchanged: backpropagation, gradient descent, Adam. The difference is one explicit assumption, *that the inputs are images*, and that assumption is built into the architecture.

Neurons arranged in three dimensions.

The layers of an ordinary network are vectors. The layers of a convolutional network are *volumes*: arrays of neurons arranged along a width, a height and a depth, where "depth" is the third axis of one layer and not the number of layers. The input volume of a 32×32 colour image is $32 \times 32 \times 3$ – width, height and the three colour channels. A neuron of the next layer is connected only to a small region of the layer before it, not to all of it, and every layer transforms one volume into another through a differentiable function. At the end the volume is reduced to a single vector of class scores, $1 \times 1 \times 10$ for ten classes.

The layers.

A simple convolutional network is a sequence of a few kinds of layer:

- **INPUT**, $[32 \times 32 \times 3]$: the raw pixel values.
- **CONV**: each neuron computes a dot product between its weights and the small region of the input it is connected to. With 12 filters the output is $[32 \times 32 \times 12]$: the same width and height (with suitable padding), one map per filter.
- **RELU**: the elementwise $\max(0, x)$; the size is unchanged.
- **POOL**: a downsampling along width and height, for example to $[16 \times 16 \times 12]$; the depth is unchanged.
- **FC**, fully connected: the class scores, $[1 \times 1 \times 10]$; every neuron sees all the numbers of the previous volume, exactly as in the networks of Chapter 8.

CONV and FC layers have weights and biases, trained by gradient descent so that the class scores agree with the labels. RELU and POOL are fixed functions with no parameters. CONV, FC and POOL have *hyperparameters* – the filter size, the number of filters, the stride, the pooling window – and RELU has none. The usual arrangement is convolution and pooling alternating, the volumes shrinking in width and height and growing in depth, and one or two fully connected layers at the end: for the MNIST network of Section 10.12,

$$28 \times 28 \times 1 \to 28 \times 28 \times 32 \to 14 \times 14 \times 32 \to 14 \times 14 \times 64 \to 7 \times 7 \times 64 \to 1024 \to 10,$$

which Figure 10.11 draws with every shape and every parameter count.

Filters: hand-made in signal processing, learned in a network.

In signal and image processing one *designs* filters. A filter is a small array of weights, "applying the filter" is a convolution of the image with it, and there are filters for smoothing, for detecting edges, for reshaping the frequency content of a signal – hand-crafted, from experience and from the mathematics of Section 10.13. The idea of a convolutional network is to *not* design the filters: to learn them from the data, to use many of them at once, and to put non-linear activations between layers of filters so that the simple patterns found by the first layer – edges – can be combined into more complex ones – corners, strokes, windows, roofs, houses – by the next. Section 10.7 shows what hand-made filters do to a digit, a drawn house and a photograph, and then what training does to filters that start out random. First, however, the word "convolution" has to be made precise.

10.3 Convolution

For two functions on the real line the convolution is

$$y(t) = (x * w)(t) = \int_{-\infty}^{\infty} x(a)\, w(t-a)\, \mathrm{d}a, \tag{10.3}$$

where x is called the input and w the *kernel*, *filter* or *weight function*. The discrete version, which is what we shall use throughout, replaces the integral by a sum,

$$y(i) = (x * w)(i) = \sum_{k=-\infty}^{\infty} w(k)\, x(i-k). \tag{10.4}$$

In practice w has only finitely many non-zero entries, say $w(0), \ldots, w(m-1)$, and the sum is finite.

Two elementary properties will be used repeatedly. Convolution is *commutative*, $x * w = w * x$, which follows from the substitution $k \to i-k$ in Eq. (10.4); and it is *linear* in each argument separately. Linearity is what allows a convolution to be written as a matrix, which we do next.

10.3.1 Convolution is polynomial multiplication

The quickest route to intuition is an example that has nothing to do with images. Take two polynomials

$$p(t) = \alpha_0 + \alpha_1 t + \alpha_2 t^2, \qquad s(t) = \beta_0 + \beta_1 t + \beta_2 t^2 + \beta_3 t^3, \tag{10.5}$$

whose product is a polynomial of degree five, $z(t) = \sum_{l=0}^{5} \delta_l t^l$. Collecting terms,

$$\begin{aligned}
\delta_0 &= \alpha_0\beta_0, \\
\delta_1 &= \alpha_1\beta_0 + \alpha_0\beta_1, \\
\delta_2 &= \alpha_0\beta_2 + \alpha_1\beta_1 + \alpha_2\beta_0, \\
\delta_3 &= \alpha_1\beta_2 + \alpha_2\beta_1 + \alpha_0\beta_3, \\
\delta_4 &= \alpha_2\beta_2 + \alpha_1\beta_3, \\
\delta_5 &= \alpha_2\beta_3.
\end{aligned} \tag{10.6}$$

Every line has the same shape: the indices of the two factors sum to the index of the result. That is precisely Eq. (10.4),

$$\delta_l = \sum_k \alpha_k \beta_{l-k} = (\alpha * \beta)_l. \tag{10.7}$$

Polynomial multiplication *is* discrete convolution. This is not an analogy; the coefficient sequences convolve. With numbers: $\boldsymbol{\alpha} = (1,2,3)$ and $\boldsymbol{\beta} = (4,-1,0.5,2)$ give $\boldsymbol{\delta} = (4,7,10.5,0,5.5,6)$ – check the third entry, $1\cdot 0.5 + 2\cdot(-1) + 3\cdot 4 = 10.5$ – and `np.convolve([1,2,3],[4,-1,0.5,2])` returns the same six numbers, as does `np.polymul` on the reversed coefficient lists.

10.3.2 Toeplitz structure

Since Eq. (10.7) is linear in $\boldsymbol{\beta}$ we may write it as a matrix acting on $\boldsymbol{\beta}$. Reading off the coefficients of Eq. (10.6),

$$\boldsymbol{\delta} = \begin{bmatrix} \alpha_0 & 0 & 0 & 0 \\ \alpha_1 & \alpha_0 & 0 & 0 \\ \alpha_2 & \alpha_1 & \alpha_0 & 0 \\ 0 & \alpha_2 & \alpha_1 & \alpha_0 \\ 0 & 0 & \alpha_2 & \alpha_1 \\ 0 & 0 & 0 & \alpha_2 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \beta_3 \end{bmatrix} \equiv \boldsymbol{T}_\alpha \boldsymbol{\beta}. \tag{10.8}$$

Because convolution is commutative the same vector is obtained from $\boldsymbol{T}_\beta \boldsymbol{\alpha}$ with a 6×3 matrix built from $\boldsymbol{\beta}$; which of the two sequences is called the filter is a matter of interpretation, not of mathematics.

The matrix in Eq. (10.8) has a name.

Definition 10.1 (Toeplitz matrix). A matrix $\boldsymbol{A} \in \mathbb{R}^{p\times q}$ is *Toeplitz* if its entries depend only on the difference of the indices,

$$A_{ij} = a_{i-j} \qquad \text{for all } i, j, \tag{10.9}$$

equivalently if $A_{i+1,j+1} = A_{ij}$ whenever both are defined. Each descending diagonal is constant. A Toeplitz matrix need not be square.

Theorem 10.2 (Convolution and Toeplitz matrices). *Let $w \in \mathbb{R}^m$ and $x \in \mathbb{R}^n$. The linear map $x \mapsto w * x$ from $\mathbb{R}^n$ to $\mathbb{R}^{n+m-1}$ is represented by the Toeplitz matrix $\boldsymbol{T}_w \in \mathbb{R}^{(n+m-1)\times n}$ with $(\boldsymbol{T}_w)_{ij} = w_{i-j}$, where $w_k \equiv 0$ outside $0 \le k \le m-1$. Conversely, every Toeplitz matrix arises this way. A matrix therefore represents a convolution if and only if it is Toeplitz.*

Proof. The ith component of $w * x$ is $\sum_k w_k x_{i-k}$. Substituting $j = i-k$ gives $\sum_j w_{i-j} x_j$, which is the ith component of $\boldsymbol{T}_w \boldsymbol{x}$ with $(\boldsymbol{T}_w)_{ij} = w_{i-j}$; this satisfies Definition 10.1 by construction. For the converse, given Toeplitz $\boldsymbol{A}$ with $A_{ij} = a_{i-j}$, set $w_k = a_k$; then $(\boldsymbol{A}\boldsymbol{x})_i = \sum_j a_{i-j} x_j = (w * x)_i$.

Theorem 10.2 is worth pausing on, because it says the two structural assumptions of Section 10.1 are not merely convenient but *exhaustive*. Locality makes the matrix banded; parameter sharing makes each diagonal constant; and a banded matrix with constant diagonals is a Toeplitz matrix, which is a convolution. There is no third option. Figure 10.3(a) shows such a matrix.

Do not build these matrices. The Toeplitz form is for reasoning, not for computing. Storing $\boldsymbol{T}_w$ costs $\mathcal{O}(n^2)$ and multiplying by it costs $\mathcal{O}(n^2)$ operations, whereas evaluating Eq. (10.4) directly costs $\mathcal{O}(nm)$ with $m \ll n$, and $\mathcal{O}(n \log n)$ by the fast Fourier transform,

since convolution in the spatial domain is multiplication in the frequency domain. The code in Section 10.9 never forms a Toeplitz matrix.

10.3.3 Diagonalisation: the convolution theorem

The notebox above claimed that a convolution costs $\mathcal{O}(n\log n)$ by the fast Fourier transform because convolution in one domain is multiplication in the other. That claim is a theorem about the eigenvectors of the Toeplitz structure, and it is worth proving, both because it is the reason the fast algorithms exist and because it is the cleanest illustration in this book of the principle of Chapter 1: a matrix is easy to work with in the basis of its own eigenvectors.

Take the periodic case, in which indices wrap modulo n. The Toeplitz matrix of Definition 10.1 then becomes *circulant*: each row is the previous row shifted one place to the right, and $C_{ij} = c_{(i-j) \bmod n}$.

Theorem 10.3 (Circular convolution is diagonalised by the DFT). *Let $\boldsymbol{C}$ be the circulant matrix generated by $\boldsymbol{c} \in \mathbb{C}^n$ and let $\boldsymbol{F}$ be the discrete Fourier matrix, $F_{kj} = n^{-1/2}\omega^{-kj}$ with $\omega = e^{2\pi i/n}$. Then*

$$\boldsymbol{C} = \boldsymbol{F}^{*}\,\mathrm{diag}(\hat{\boldsymbol{c}})\,\boldsymbol{F}, \qquad \hat{c}_k = \sum_{m=0}^{n-1} c_m\,\omega^{-km}, \tag{10.10}$$

so that every circulant matrix has the same *eigenvectors – the Fourier modes – and eigenvalues given by the discrete Fourier transform of its generating vector. Consequently*

$$\widehat{\boldsymbol{c} * \boldsymbol{x}} = \hat{\boldsymbol{c}} \odot \hat{\boldsymbol{x}}, \tag{10.11}$$

the convolution theorem: convolution becomes elementwise multiplication.

Proof. Let $\boldsymbol{v}_k$ have components $(\boldsymbol{v}_k)_j = \omega^{kj}$, the kth Fourier mode. Then

$$(\boldsymbol{C}\boldsymbol{v}_k)_i = \sum_{j=0}^{n-1} c_{(i-j) \bmod n}\,\omega^{kj} = \sum_{m=0}^{n-1} c_m\,\omega^{k(i-m)} = \omega^{ki}\sum_{m=0}^{n-1} c_m\omega^{-km} = \hat{c}_k\,(\boldsymbol{v}_k)_i,$$

where the substitution $m = (i-j) \bmod n$ is a bijection of $\{0,\dots,n-1\}$ onto itself and the periodicity of ω removes the modulus. So $\boldsymbol{v}_k$ is an eigenvector with eigenvalue $\hat{c}_k$, for every k; the $\boldsymbol{v}_k$ are orthogonal and $n^{-1/2}\boldsymbol{v}_k$ are the columns of $\boldsymbol{F}^{*}$, which is Eq. (10.10). Equation (10.11) follows by applying $\boldsymbol{F}$ to $\boldsymbol{C}\boldsymbol{x}$ and reading off the diagonal.

Three consequences follow immediately, and each is used somewhere later in this book. Computationally, Eq. (10.11) says a convolution costs two transforms and n multiplications, so $\mathcal{O}(n\log n)$ rather than $\mathcal{O}(n^2)$; this is how large kernels are actually applied. Structurally, all circulant matrices commute, since they are simultaneously diagonalised – which is the algebraic content of the statement that convolutions compose into convolutions. And conceptually, Eq. (10.10) says a convolutional layer acts on each Fourier mode independently, multiplying it by a single learned number: a convolutional network is a learned filter bank, and what it learns is a frequency response.

Why the theorem does not simply end the chapter. If convolution is elementwise multiplication in Fourier space, why not build networks there and skip the spatial domain? Because the kernels used in practice are tiny. With $F = 3$ the direct cost is $9n$ operations against $\mathcal{O}(n\log n)$ for two transforms plus the product, and the crossover on typical image sizes is around $F \approx 11$. More fundamentally, the non-linearity between layers is element-

wise *in space*, and an elementwise operation in one domain is a convolution in the other – so the network cannot stay in either domain. The architecture alternates between a map that is diagonal in frequency and a map that is diagonal in space, and that alternation is where its expressive power comes from.

10.3.4 Padding, and why indices go out of range

Rename $\boldsymbol{\alpha}$ as the filter $\boldsymbol{w}$, of length m, and $\boldsymbol{\beta}$ as the input $\boldsymbol{x}$, of length n, with $m \le n$. Then

$$y(i) = \sum_{k=0}^{m-1} w(k)\, x(i-k). \tag{10.12}$$

For $i = 0$ this requires $x(-1)$ and $x(-2)$, which do not exist, and for $i = n+m-2$ it requires indices past the end. The standard remedy is *padding*: extend $\boldsymbol{x}$ with P zeros at each end, so that its length becomes $n+2P$. With $P = m-1$ every index in Eq. (10.12) is defined and the output has full length $n+m-1$; this is called *full* padding. Two other choices are used constantly: $P = 0$, or *valid* padding, which evaluates the sum only where it is defined and shrinks the output to $n-m+1$; and $P = (m-1)/2$ for odd m, or *same* padding, which returns an output of exactly length n.

Padding with zeros is a choice, and it is visible in the result: outputs near the boundary are computed from fewer genuine inputs than outputs in the interior, so a padded convolution treats the border differently from the middle. Reflecting or wrapping the signal instead are common alternatives.

10.3.5 Cross-correlation, which is what libraries actually compute

Equation (10.4) contains $x(i-k)$: as k increases the input is read *backwards*. Implementing this means flipping the kernel before sliding it, which is a nuisance and, when the kernel is learned rather than prescribed, entirely pointless. Every deep learning library therefore computes

$$y(i) = \sum_{k} w(k)\, x(i+k), \tag{10.13}$$

the *cross-correlation*, while calling it convolution. We follow the same convention from here on.

Why the distinction does not matter here, and when it does. Cross-correlation with w equals convolution with the reflected kernel $\tilde{w}(k) = w(-k)$. Since w is learned, the network simply learns the reflected filter, and the set of functions the layer can represent is identical. The distinction matters only when the kernel is prescribed rather than learned – a Gaussian blur, a derivative filter, a matched filter from signal processing – or when comparing against a formula from a textbook that uses the true convolution. Note also that cross-correlation is *not* commutative, so the symmetry between filter and input in Eq. (10.8) is lost.

10.4 Two dimensions, channels and the convolutional layer

For a two-dimensional input $\boldsymbol{X}$ and kernel $\boldsymbol{W}$ the definition extends in the obvious way,

$$Y(i,j) = (X * W)(i,j) = \sum_m \sum_n X(i-m,\, j-n)\, W(m,n), \tag{10.14}$$

and in the cross-correlation form actually used,

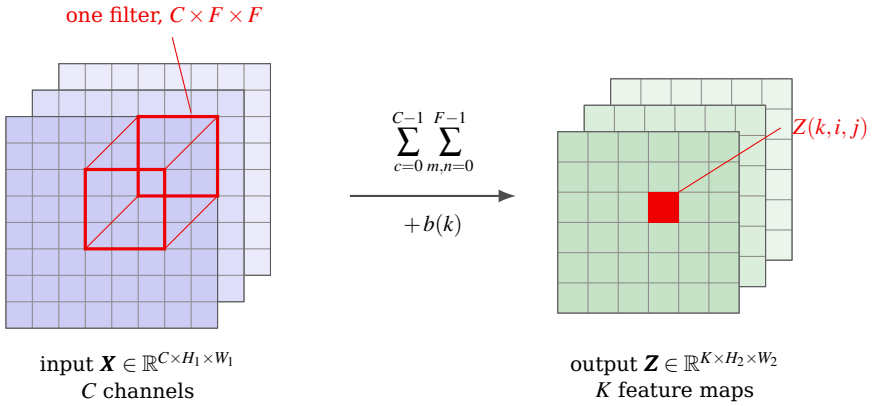
$$Y(i,j) = \sum_m \sum_n X(i+m,\, j+n)\, W(m,n). \tag{10.15}$$

Real inputs have a channel axis, and a layer produces several outputs called *feature maps*. Let the input be $\boldsymbol{X} \in \mathbb{R}^{C\times H\times W}$ with C channels, and let there be K filters, each of shape $C\times F\times F$, collected in $\boldsymbol{W} \in \mathbb{R}^{K\times C\times F\times F}$ with biases $\boldsymbol{b} \in \mathbb{R}^K$. The layer computes, for $k = 0,\dots,K-1$,

$$\boxed{Z(k,i,j) = \sum_{c=0}^{C-1} \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} X(c,\, Si+m-P,\, Sj+n-P)\, W(k,c,m,n) + b(k)} \tag{10.16}$$

followed by an elementwise activation, $A = f(Z)$. Here S is the *stride*, the step by which the filter is moved, and P the padding.

Three points about Eq. (10.16) deserve emphasis. The sum over c runs over *all* input channels: a filter is not applied channel by channel but sees the full depth at each spatial location, which is why its shape is $C\times F\times F$ and not $F\times F$. The indices i,j range over output positions while m,n range over the filter, so the filter is small and the output is large. And W does not depend on i or j – that is parameter sharing, written out. Figure 10.2 shows the three statements as one picture: a filter drawn as a block spanning the full depth, one output value produced from it, and the K maps that K such filters make.



$$H_2 = \left\lfloor \frac{H_1 - F + 2P}{S} \right\rfloor + 1, \qquad N = K\left(CF^2 + 1\right) \text{ parameters}$$

Fig. 10.2 Equation (10.16) drawn. A filter is a $C\times F\times F$ block, not an $F\times F$ patch: it spans the *whole* depth of the input at each spatial location, which is why the sum over c in Eq. (10.16) has no counterpart among the output indices. The block shown produces the single value $Z(k,i,j)$ marked in the output; sliding it over all positions (i,j) with step S fills one feature map, and using K different filters produces the K maps stacked behind it. The spatial sizes follow Proposition 10.4 and the parameter count is independent of H_1 and W_1.

10.4.1 Output size and parameter count

Proposition 10.4 (Convolution arithmetic). *Let the input have height H_1, width W_1 and depth C, and apply K filters of spatial extent F with stride S and padding P. Then the output volume has depth $D_2 = K$ and spatial dimensions*

$$H_2 = \left\lfloor \frac{H_1 - F + 2P}{S} \right\rfloor + 1, \qquad W_2 = \left\lfloor \frac{W_1 - F + 2P}{S} \right\rfloor + 1, \tag{10.17}$$

and the layer has

$$N = K\left(CF^2 + 1\right) \tag{10.18}$$

trainable parameters.

Proof. Along the height, the padded input has $H_1 + 2P$ entries. The filter occupies positions $0, S, 2S, \dots$ and its last element must not run past the end, so the largest admissible starting index t satisfies $t + F - 1 \le H_1 + 2P - 1$, that is $t \le H_1 + 2P - F$. The number of admissible multiples of S in $[0, H_1 + 2P - F]$ is $\lfloor (H_1 + 2P - F)/S \rfloor + 1$, which is Eq. (10.17); the width is identical. Each output channel is produced by one filter of CF^2 weights plus one bias, and there are K of them, giving Eq. (10.18).

The floor in Eq. (10.17) is not decoration. When S does not divide $H_1 + 2P - F$ the filter cannot reach the last few rows, and they are silently discarded. Choosing F, S and P so that the division is exact is the reason for the familiar combinations $F = 3, S = 1, P = 1$ and $F = 5, S = 1, P = 2$, both of which give $H_2 = H_1$.

Example 10.5 (A concrete count). An input volume of $32 \times 32 \times 3$ with $K = 10$ filters of extent $F = 5$, stride $S = 1$ and padding $P = 0$ gives $H_2 = W_2 = (32 - 5)/1 + 1 = 28$, so the output is $28 \times 28 \times 10$. Each filter has $5 \times 5 \times 3 = 75$ weights and one bias, so the layer has $10 \times 76 = 760$ parameters. A dense layer with the same number of outputs would have $3072 \times 7840 \approx 2.4 \times 10^7$.

10.4.2 The unrolled matrix

Equation (10.16) is linear in $\boldsymbol{X}$, so it must be a matrix. Take the small example of a 3×3 input and a 2×2 filter with $S = 1$, $P = 0$. Flattening the input row by row into $\boldsymbol{X}' \in \mathbb{R}^9$ and the output into $\boldsymbol{Y}' \in \mathbb{R}^4$, the convolution is $\boldsymbol{Y}' = \boldsymbol{W}'\boldsymbol{X}'$ with

$$\boldsymbol{W}' = \begin{bmatrix} w_{00} & w_{01} & 0 & w_{10} & w_{11} & 0 & 0 & 0 & 0 \\ 0 & w_{00} & w_{01} & 0 & w_{10} & w_{11} & 0 & 0 & 0 \\ 0 & 0 & 0 & w_{00} & w_{01} & 0 & w_{10} & w_{11} & 0 \\ 0 & 0 & 0 & 0 & w_{00} & w_{01} & 0 & w_{10} & w_{11} \end{bmatrix}. \tag{10.19}$$

With numbers: for $\boldsymbol{X}$ the matrix with entries $1, \dots, 9$ row by row and $\boldsymbol{W} = \left(\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}\right)$, the top-left output is $1 \cdot 1 + 2 \cdot 2 + 4 \cdot 3 + 5 \cdot 4 = 37$, and

$$\boldsymbol{Y} = \begin{pmatrix} 37 & 47 \\ 67 & 77 \end{pmatrix}, \qquad \boldsymbol{W}'\boldsymbol{X}' = (37, 47, 67, 77)^T,$$

the same four numbers from the matrix–vector product. This matrix is *doubly block Toeplitz*: it is a Toeplitz matrix whose entries are themselves Toeplitz blocks, the outer structure coming from the row index and the inner from the column index. Four numbers appear in thirty-six positions. We verify this claim numerically in Section 10.9 by reconstructing $\boldsymbol{W}'$ column by

column from the implementation and checking that it has exactly sixteen non-zero entries taking exactly four distinct values; Figure 10.3(b) shows the same pattern for a 5×5 input and a 3×3 kernel.

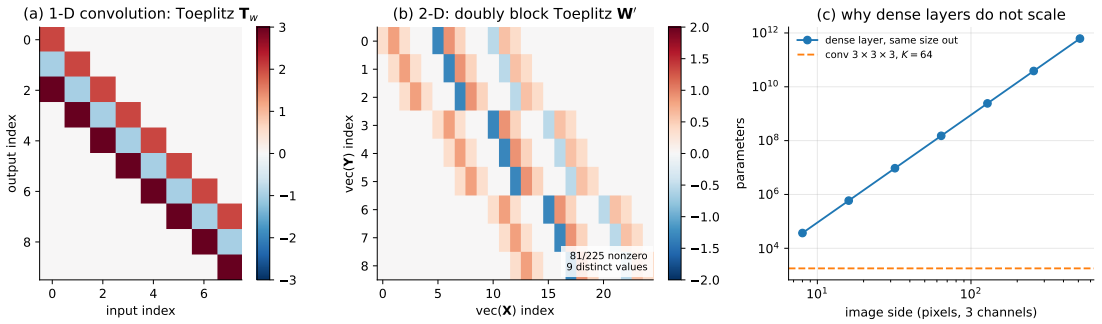


Fig. 10.3 Convolution as a structured sparse matrix. (a) A one-dimensional convolution with a kernel of length three acting on an input of length eight, Eq. (10.8); each descending diagonal is constant, which is Definition 10.1. (b) The doubly block Toeplitz matrix $\boldsymbol{W}'$ of Eq. (10.19) for a 5×5 input and a 3×3 kernel, reconstructed column by column from the code of Section 10.9: nine distinct numbers fill eighty-one of two hundred and twenty-five positions. (c) Parameters in a single layer as a function of image side, Eqs. (10.1) and (10.2); the dense layer grows as L^4 while the convolutional layer does not grow at all.

10.5 Equivariance, and what it does and does not give

Sparsity and parameter sharing explain the parameter count. The property that explains why convolution is the *right* restriction, rather than merely a cheap one, is equivariance.

Definition 10.6 (Equivariance and invariance). Let T_s denote translation by s, $(T_s x)(i) = x(i-s)$. A map Φ is *equivariant* to translation if

$$\Phi(T_s x) = T_s\,\Phi(x) \qquad \text{for all } x, s, \tag{10.20}$$

and *invariant* if $\Phi(T_s x) = \Phi(x)$ for all x, s.

Equivariance says that translating the input translates the output by the same amount without otherwise changing it: if a filter responds to an edge, it responds to that edge wherever the edge moves. Invariance says the output does not move at all. Both are wanted, but at different places in the network: equivariance in the convolutional layers, where we want to know *where* features are, and invariance at the end, where we want to know only *what* the image contains.

Theorem 10.7 (Convolution is exactly the equivariant linear map). *Let $\Phi : \mathbb{R}^{\mathbb{Z}} \to \mathbb{R}^{\mathbb{Z}}$ be linear. Then Φ is equivariant to all translations if and only if Φ is a convolution, that is, there is a w with $\Phi(x) = w * x$.*

Proof. ($\Leftarrow$) With $\Phi(x) = w * x$,

$$\Phi(T_s x)(i) = \sum_k w(k)\,(T_s x)(i-k) = \sum_k w(k)\,x(i-s-k) = \Phi(x)(i-s) = \big(T_s \Phi(x)\big)(i).$$

($\Rightarrow$) Let δ be the sequence with $\delta(0) = 1$ and $\delta(i) = 0$ otherwise, and put $w = \Phi(\delta)$, the *impulse response*. Any x may be written $x = \sum_s x(s)\,T_s\delta$. By linearity and then equivariance,

$$\Phi(x) = \sum_s x(s)\,\Phi(T_s\delta) = \sum_s x(s)\,T_s\Phi(\delta) = \sum_s x(s)\,T_s w,$$

whose ith component is $\sum_s x(s)w(i-s) = (w * x)(i)$.

This is the theorem that justifies the architecture. We did not choose convolution from a menu of plausible local operations; *once we demand a linear map that commutes with translation, convolution is the only thing left.* The restriction from $\mathcal{O}(L^4)$ parameters to $\mathcal{O}(F^2)$ is the exact price of that symmetry.

Two qualifications matter in practice, and both are easy to state and easy to forget.

Proposition 10.8 (Stride breaks equivariance). *A convolution with stride S is equivariant to translations by multiples of S only. For s not divisible by S, in general $\Phi(T_s x) \neq T_{\lfloor s/S \rfloor}\Phi(x)$.*

Proof. Striding retains outputs at positions Si. Shifting the input by s shifts the underlying full-stride output by s, and the retained subset $\{Si\}$ maps to $\{Si+s\}$, which coincides with $\{Si\}$ as a set if and only if $S \mid s$. Otherwise a different subsample of the same signal is retained and no translation of the output reproduces it.

We confirm this numerically in Section 10.9: for $S = 1$ the equivariance error is exactly zero, for $S = 2$ it is exactly zero for even shifts and $\mathcal{O}(1)$ for odd ones. Padding is the second qualification. With zero padding the boundary rows are computed from invented zeros, so equivariance holds in the interior but fails within $F/2$ of the edge – which is why the numerical check in Section 10.9 is stated on the interior.

Figure 10.4 shows the property directly.

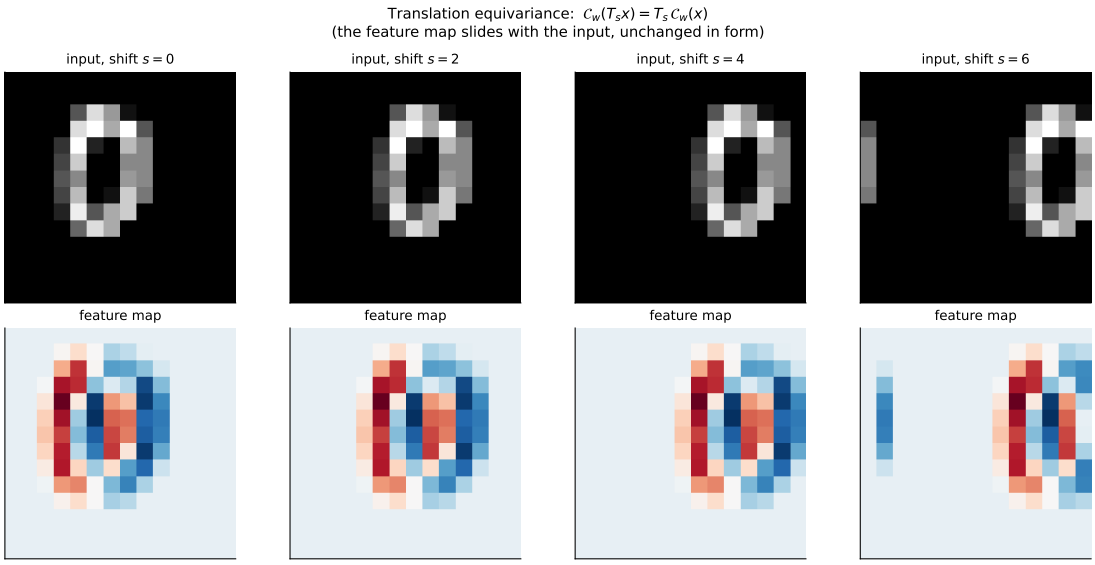


Fig. 10.4 Translation equivariance, Eq. (10.20), for a fixed 3×3 derivative filter applied to a digit placed at four positions on a 14×14 canvas. The feature map in the lower row slides with the input in the upper row and is otherwise unchanged: measured on the interior, the discrepancy $\max|\mathscr{C}_w(T_s x) - T_s\mathscr{C}_w(x)|$ is exactly zero. A dense layer has no such property, and must learn each position separately.

10.6 Pooling

Equivariance propagates information about position through the network. At some point we want to discard it: a classifier should report “digit three” regardless of where the three sat. Pooling is the operation that trades position for robustness.

A pooling layer slides a window of size F with stride S over each channel *independently* and applies a fixed function to the contents. Max pooling,

$$Y(c,i,j) = \max_{0 \le m,n < F} X(c, Si+m, Sj+n), \tag{10.21}$$

is by far the most common; average pooling replaces the maximum by the mean. Pooling has *no trainable parameters*, and it does not mix channels, so the depth is unchanged and Eq. (10.17) governs the spatial sizes with $P = 0$.

The usual choice $F = S = 2$ discards three quarters of the activations in one step, as in Figure 10.5. The gain is a limited invariance: if the largest value in a window moves within that window, the output does not change at all. Pooling therefore converts *equivariance at fine scale* into *invariance at fine scale*, and stacking convolution and pooling alternately builds up invariance to progressively larger displacements while the receptive field grows.

Proposition 10.9 (Local invariance of max pooling). *Let Π be max pooling with window F and stride $S = F$, so that the windows tile the input without overlap, and let x be supported in one window $\mathscr{W}$. If $T_s x$ is still supported in $\mathscr{W}$, then $\Pi(T_s x) = \Pi(x)$. If instead s moves the support into the neighbouring window, the output is unchanged in* value *but appears one position later.*

Proof. The pooling output at window $\mathscr{W}$ is $\max_{p \in \mathscr{W}} x(p)$, which depends on the multiset of values in $\mathscr{W}$ and not on their arrangement; a translation that permutes positions within $\mathscr{W}$ therefore leaves it fixed. A translation carrying the support into the next window leaves the multiset intact but attaches it to the next output index.

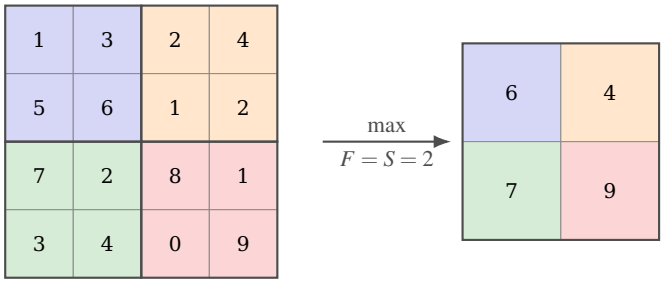


Fig. 10.5 Max pooling with $F = S = 2$, Eq. (10.21). The four non-overlapping windows are shaded in four colours and each contributes one value, its maximum, to the correspondingly shaded output cell. Three quarters of the activations are discarded and nothing is learned: the operation has no parameters at all. Proposition 10.9 is visible here – move the 9 anywhere within its own window and the output is unchanged, move it into a neighbouring window and the output merely shifts.

Figure 10.6 shows pooling at work on a real feature map, the vertical-edge map of the five of Figure 10.8 after a ReLU. Max pooling with $F = S = 2$ keeps the strongest response in each

2×2 window and halves the side, $26 \to 13$; average pooling blurs instead. The last panel is Proposition 10.9 measured: moving the digit one pixel to the right changes the full-resolution map everywhere, but 73% of the max-pooled values are unchanged, because in most windows the largest value simply moved within its window. After two convolution–pooling blocks a 28×28 digit has become a stack of 7×7 maps, which is flattened and handed to the dense layers.

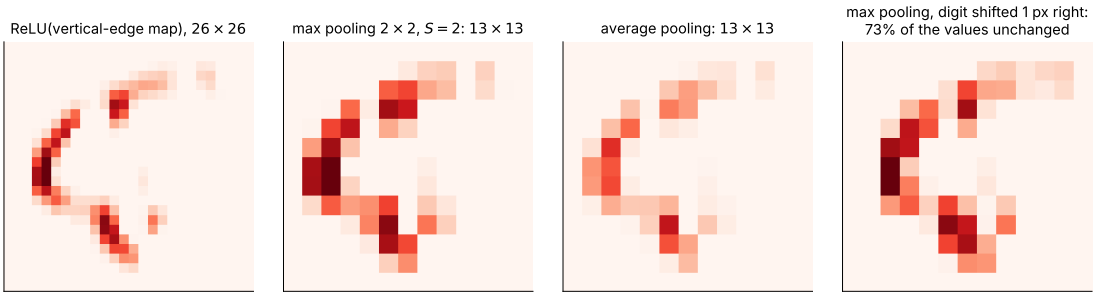


Fig. 10.6 Pooling on a feature map. The vertical-edge map of the five after a ReLU (26×26), max-pooled and average-pooled with 2×2 windows and stride 2 (13×13), and the max-pooled map when the digit is first moved one pixel to the right: 73% of the pooled values are unchanged, which is Proposition 10.9 on real data. Max pooling keeps the strongest response in each window; average pooling blurs.

Proposition 10.9 is the precise version of "a limited invariance". Pooling is exactly invariant to displacements that stay inside a window and merely equivariant to larger ones – so a stack of pooling layers buys invariance to displacements up to the size of the last window measured in input pixels, and no more. That quantity is the receptive field, which we now compute.

Proposition 10.10 (Receptive field). *Consider L layers, layer ℓ having filter size F_ℓ and stride S_ℓ. The receptive field R_L of one output unit – the number of input pixels along one axis that can influence it – satisfies $R_0 = 1$ and*

$$R_\ell = R_{\ell-1} + (F_\ell - 1) \prod_{j=1}^{\ell-1} S_j. \tag{10.22}$$

Proof. A unit at layer ℓ draws on F_ℓ consecutive units of layer $\ell - 1$, whose receptive fields are each of size $R_{\ell-1}$ and are offset from one another by the cumulative stride $\prod_{j<\ell} S_j$ measured in input pixels. The union of F_ℓ intervals of length $R_{\ell-1}$ spaced by that offset has length $R_{\ell-1} + (F_\ell - 1)\prod_{j<\ell} S_j$.

Equation (10.22) explains a design rule. With 3×3 filters and unit stride the receptive field grows only as $2\ell + 1$, painfully slowly; every pooling layer of stride two doubles the multiplier and the growth becomes geometric. Three blocks of two 3×3 convolutions followed by a 2×2 pool reach a receptive field of $R = 44$, enough to see most of a 32×32 image. Stacking small filters rather than using one large one is also cheaper: two 3×3 layers have receptive field 5 and $2 \times 9C^2$ weights, against $25C^2$ for a single 5×5 layer, and they include an extra non-linearity.

10.6.1 Dilation: receptive field without cost

Equation (10.22) shows two ways to enlarge the receptive field, and both are expensive: larger filters cost $\mathcal{O}(F^2)$ parameters, and stride or pooling costs resolution. A third way costs

neither. A *dilated* or *atrous* convolution with rate D spreads the filter taps apart,

$$Z(k,i,j) = \sum_{c,m,n} X(c, i+Dm,\ j+Dn)\, W(k,c,m,n) + b(k), \tag{10.23}$$

which for $D=1$ is Eq. (10.15) and for $D>1$ reads from a grid of the same F^2 positions spaced D apart.

Proposition 10.11 (Effective filter size and dilated receptive field). *A dilated convolution with filter size F and rate D has the same $K(CF^2+1)$ parameters as an undilated one and an effective filter size*

$$F_{\text{eff}} = D(F-1)+1. \tag{10.24}$$

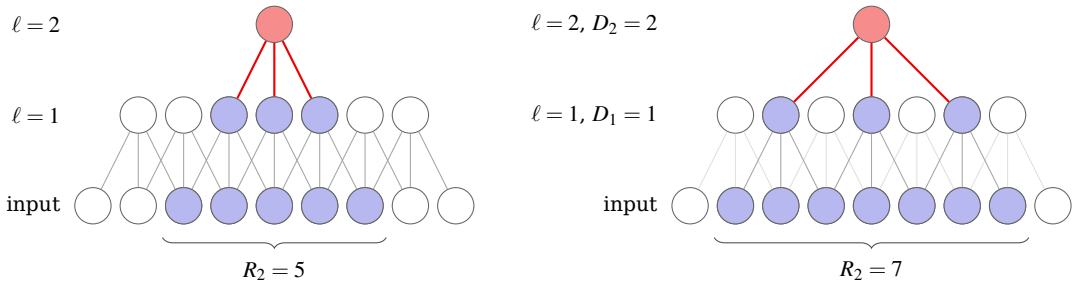
Stacking L such layers with unit stride and rates D_ℓ gives a receptive field

$$R_L = 1 + \sum_{\ell=1}^{L} D_\ell\,(F_\ell - 1). \tag{10.25}$$

In particular, $F=3$ with $D_\ell = 2^{\ell-1}$ gives $R_L = 2^{L+1}-1$: the receptive field grows exponentially *in depth while the parameter count grows linearly and the resolution is never reduced.*

Proof. The taps occupy positions $0, D, 2D, \dots, D(F-1)$, spanning $D(F-1)+1$ pixels, which is Eq. (10.24); the number of taps, and hence of weights, is unchanged. Substituting $F_\ell \to D_\ell(F_\ell-1)+1$ into Eq. (10.22) with all $S_j = 1$ gives $R_\ell = R_{\ell-1} + D_\ell(F_\ell - 1)$, and summing from $R_0 = 1$ gives Eq. (10.25). With $F_\ell = 3$ and $D_\ell = 2^{\ell-1}$ the sum is $1 + 2\sum_{\ell=1}^{L} 2^{\ell-1} = 2^{L+1}-1$.

Compare the three routes to a receptive field of $R = 63$ with 3×3 filters. Unit stride and no dilation needs 31 layers, by $R = 2L+1$. Pooling of stride two every second layer needs about 10 and throws away a factor of 32 in resolution. Exponential dilation needs 5, by Proposition 10.11, at full resolution throughout. This is why dilation is standard in segmentation, where the output must have the same size as the input, and in the audio models where the receptive field must span thousands of samples. Figure 10.7 shows both cases for two layers, with the reach of a single output unit shaded on the input row.



(a) $D_1 = D_2 = 1$: $R_2 = 1+2+2 = 5$ (b) $D_\ell = 2^{\ell-1}$: $R_2 = 1+2+4 = 7$

$$R_\ell = R_{\ell-1} + D_\ell\,(F_\ell - 1)\prod_{j<\ell} S_j, \qquad F_\ell = 3,\ S_\ell = 1$$

Fig. 10.7 The receptive field of Proposition 10.10 and what dilation does to it, for two layers of $F=3$ with unit stride. The shaded input units are those the single top unit can see. (a) Without dilation the two layers reach five inputs, and Eq. (10.22) gives the familiar $R_\ell = 2\ell+1$. (b) With $D_2 = 2$ the second layer reads every other unit of the first – the pale edges are first-layer units that are computed but not read by this particular output – and the reach becomes seven. The filter has the same three taps in both panels and the resolution is reduced in neither; Proposition 10.11 continues the pattern to $R_L = 2^{L+1}-1$.

10.7 Filters at work: from hand-made to learned

Everything so far has been about the *form* of a convolutional layer. This section is about what a filter does, on pictures, and it is worth reading with Figures 10.8–10.10 in front of you. We apply filters designed by hand – the kind signal processing has used for decades – to a digit, to a drawn house and to a photograph of houses, we build a filter that *detects a roof*, and we then look at what a network does to filters that start out random. The point of the section is the last step: the learned filters of Figure 10.12(c) are not mysterious objects but members of the family we are about to meet.

10.7.1 Four edge filters and a blur

Every feature map below is the cross-correlation of Eq. (10.15), $Y(i,j) = \sum_{m,n} X(i+m, j+n) W(m,n)$, of one image with one 3×3 filter. The filters are

$$W_{\text{vert}} = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \qquad W_{\text{hor}} = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}, \tag{10.26}$$

$$W_{\diagup} = \begin{pmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{pmatrix}, \qquad W_{\diagdown} = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 0 & -1 \\ 0 & -1 & -2 \end{pmatrix}, \qquad W_{\text{blur}} = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}. \tag{10.27}$$

W_{vert} is the *Sobel* operator. Read row by row it is the central difference along a row, $X(i, j+1) - X(i, j-1)$, taken in three neighbouring rows and averaged with weights $(1,2,1)$: a smoothed estimate of $\partial X / \partial x$. It is large where the intensity changes as we move sideways, which is at a *vertical* stroke or edge, positive where the ink begins and negative where it ends, and zero inside a stroke and along a horizontal stroke, where every row is constant. $W_{\text{hor}} = W_{\text{vert}}^T$ is the same construction down a column, $\partial X / \partial y$, and responds to horizontal edges. The diagonal pair are the same differences along the two diagonals – the Sobel operator rotated by 45°. All four have weights that sum to zero, so a region of constant intensity gives exactly 0 whatever its brightness: they respond to *changes* only. The blur is the opposite kind of object. Its weights are positive and sum to one, so it replaces every pixel by the mean of its 3×3 neighbourhood, preserves constants, and removes fine detail – it is a *low-pass* filter, in the language of Section 10.13, where the edge filters are high-pass.

Figure 10.8 applies three of them to a digit. The vertical strokes of the five light up in the first map and vanish in the second; the horizontal strokes do the reverse; the blur keeps the digit and softens its boundary. Each output is one *feature map*, and a convolutional layer with $K = 3$ filters produces all three at once, by Proposition 10.4 with $3 \times (1 \cdot 9 + 1) = 30$ parameters.

Figure 10.9 does the same to a drawn house, with $P = 1$ so that the maps keep the size of the picture. The vertical-edge filter finds the walls and the sides of the door and the window, the horizontal one the floor, the lintels and the roof as a whole, and the two diagonal filters each pick out exactly one slope of the roof. Four filters, each a few integers, already decompose the drawing into the oriented edges it is made of. That is the first layer of a convolutional network, written by hand.

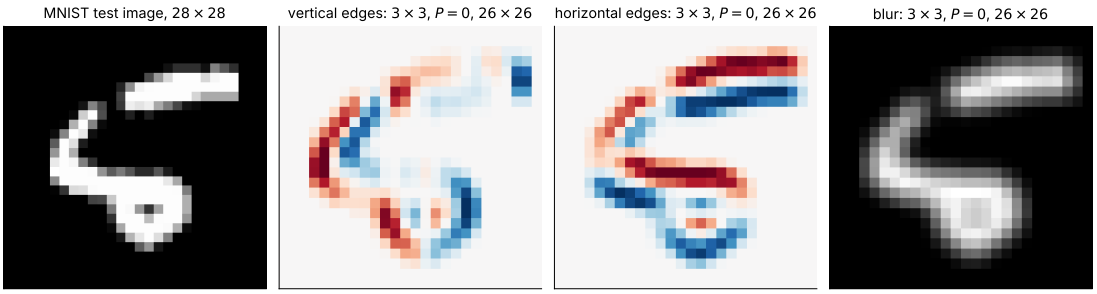


Fig. 10.8 Three hand-made filters on an MNIST test image, a five. The vertical-edge filter W_{vert} of Eq. (10.26) is a derivative along a row: positive (red) where the ink starts, negative (blue) where it ends, zero inside a stroke and on the horizontal strokes. Its transpose finds the horizontal strokes. The blur of Eq. (10.27) is a 3×3 mean. No padding, so the maps are 26×26; each is one feature map, and a layer with $K=3$ filters computes all three at once with $3\times(9+1)=30$ parameters.

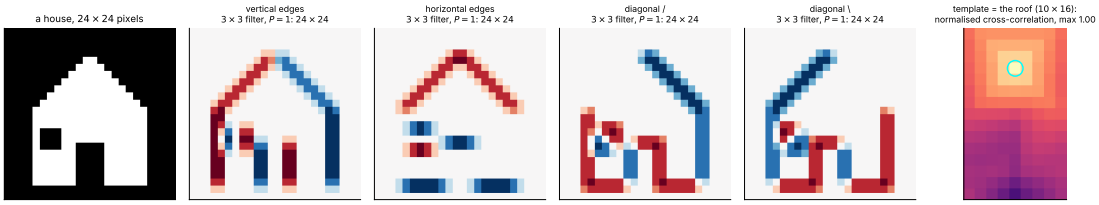


Fig. 10.9 A 24×24 drawing of a house – walls, a gabled roof, a door, a window – and four feature maps, each the cross-correlation with one of the 3×3 filters of Eqs. (10.26)–(10.27) with $P=1$, so the maps keep the size of the picture. The vertical-edge filter finds the walls and the sides of door and window, the horizontal one the floor, the lintels and the roof as a whole, and each diagonal filter picks out *one* slope of the roof. The last panel uses the roof itself, cut out of the picture, as a 10×16 template: the normalised cross-correlation of Eq. (10.28) equals 1.00 exactly where the roof is (circled) and is smaller everywhere else – a filter that detects a roof.

10.7.2 A filter that detects a roof: template matching

Edge filters are small. Nothing in Eq. (10.15) says a filter has to be, and the last panel of Figure 10.9 uses a large one: the roof itself, cut out of the picture. Take the 10×16 patch T containing the roof, subtract its mean and divide by its norm, so that $\sum_{m,n} T(m,n)=0$ and $\sum_{m,n} T(m,n)^2=1$. For every patch X_{ij} of the same size in the image, with mean $\bar{X}_{ij}$, define

$$\mathrm{NCC}(i,j)=\frac{\sum_{m,n}\big(X_{ij}(m,n)-\bar{X}_{ij}\big)\,T(m,n)}{\sqrt{\sum_{m,n}\big(X_{ij}(m,n)-\bar{X}_{ij}\big)^2}}, \tag{10.28}$$

the *normalised cross-correlation*. The numerator is the cross-correlation of the image with T, Eq. (10.15); the denominator is the norm of the centred patch. The quotient is the cosine of the angle between the centred patch and the template, and that geometry is what makes it a detector.

Proposition 10.12 (Normalised cross-correlation is a detector). *For a zero-mean, unit-norm template T and any patch X_{ij} that is not constant, $-1\le \mathrm{NCC}(i,j)\le 1$, with $\mathrm{NCC}(i,j)=1$ if and only if $X_{ij}=aT+b$ for some $a>0$ and b: the patch is the template, up to brightness b and contrast a. The value does not change if the image is rescaled or shifted in intensity.*

Proof. Since $\sum T=0$, the numerator of Eq. (10.28) equals $\langle X_{ij}-\bar{X}_{ij},T\rangle$ and the denominator equals $\|X_{ij}-\bar{X}_{ij}\|\,\|T\|$ with $\|T\|=1$, so the quotient is the cosine of the angle between two vectors, which lies in $[-1,1]$ by the Cauchy–Schwarz inequality, with equality $+1$ exactly when

the centred patch is a positive multiple of T, that is $X_{ij} - \bar{X}_{ij} = aT$ with $a > 0$. Replacing X by $\alpha X + \beta$ with $\alpha > 0$ multiplies numerator and denominator by α and leaves $\bar{X}_{ij}$ to absorb β.

In code the whole map costs three cross-correlations: of X with T, of X with a matrix of ones (the patch sums, hence the means) and of X^2 with a matrix of ones (the patch norms), each a single call to `correlate2d` from `scipy.signal` with `mode="valid"`. On the house the map peaks at exactly 1.00 at the position of the roof and nowhere else above 0.82.

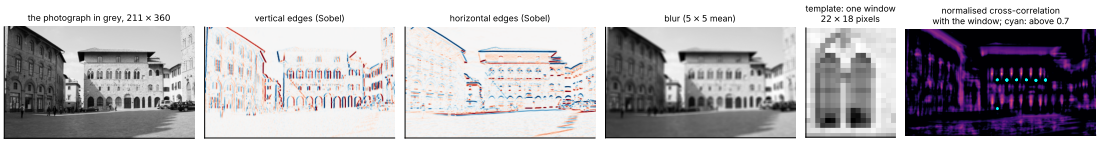


Fig. 10.10 The same filters on a photograph of houses, in grey, 211×360 pixels. The Sobel filters draw the vertical lines (walls, window edges, people) and the horizontal ones (roof lines, floors, arches); a 5×5 blur removes the texture of the stone. Right: one double-arched window, 22×18 pixels, cut from the photograph and used as the template in Eq. (10.28). The normalised cross-correlation lights up at every window of that type – six on the middle building and one on the left – and nowhere else above 0.7. A filter that finds windows is the beginning of a filter that finds houses.

Figure 10.10 moves from the drawing to a photograph. The Sobel filters again separate the vertical from the horizontal structure, and the blur removes the texture of the stone while keeping the buildings. For the template we cut out one double-arched window, 22×18 pixels. The normalised cross-correlation with it reaches 1.00 at the window we cut and exceeds 0.7 at six more windows of the same building and one on the next – and nowhere else. A filter that finds windows is the beginning of a filter that finds houses, and the pattern is general: a template is a filter for one specific thing, and a bank of templates for edges, corners, windows and roofs is a crude object detector.

What the template cannot do is also visible. It matches a window of *that* size and orientation; a window twice as large, or seen at an angle, gives a much weaker response. Convolution handles *position* for free, by the equivariance of Theorem 10.7; size and rotation it does not handle at all. A network learns those from examples at several scales and orientations, or the data are *augmented* with rescaled and rotated copies to supply them.

10.7.3 From hand-made filters to learned ones

Everything above was chosen by hand, and choosing it took knowledge of what a house looks like. A convolutional network starts instead from *random* filters and adjusts them by gradient descent on the classification cost, through the gradients derived in Section 10.8. Nothing in the cost mentions edges. Yet the first-layer filters of the network trained in Section 10.10 – Figure 10.12(c), eight 3×3 filters on 8×8 digits – and of the network trained on MNIST in Section 10.12.5 – Figure 10.13(d), thirty-two of them – are oriented patterns with a positive lobe beside a negative one: edge detectors of the family of Eq. (10.26), at various orientations, with various widths, learned because edges are what separate the digits. Nobody asked for them. They are what minimising a cross-entropy on images produces.

Three remarks connect the hand-made filters to the learned ones.

First, the ReLU. The Sobel map in Figure 10.8 is signed: red where the ink begins, blue where it ends. A ReLU after the layer keeps only the positive half. A trained network therefore typically learns a filter *and* its negative, so that both polarities of an edge survive the activation; the paired filters in Figure 10.13(d) are an instance.

Second, depth. The second layer sees feature maps, not pixels, and its filters span the full depth of the first layer's output by Eq. (10.16). A second-layer filter can therefore respond to "a vertical edge next to a horizontal one" – a corner – or to "two diagonal edges meeting at the top" – a gable – and a third layer to arrangements of those. The template of Figure 10.9 was a single large filter for a roof; a deep network builds the same detector out of small filters composed through non-linearities, which is cheaper in parameters (Proposition 10.10: two 3×3 layers see as far as one 5×5 with fewer weights) and, unlike the template, shared between every object that contains a corner. A network trained on houses would learn the window and the roof of Figure 10.10 without being shown them.

Third, reading a feature map. Why does the vertical-edge filter, whose rows read $(-1,0,1)$, light up on the *vertical* strokes? Because it is a difference between the pixel to the right and the pixel to the left – a derivative along a row – and that derivative is large where the intensity changes as we move sideways, which is where a vertical stroke begins or ends, and zero on a horizontal stroke, where every row is constant. The same reading applies to a learned filter: display it as a small image, look for the opposed lobes, and the orientation of the edge it detects is perpendicular to the line joining them. Section 10.13 gives the same reading in the language of frequencies, where W_{vert} has the response $4|\sin\Omega_x|(1+\cos\Omega_y)$: zero on the axis $\Omega_x = 0$ – every pattern that does not change along a row is invisible to it – and largest for oscillations across the row.

10.8 Backpropagation through a convolutional layer

Chapter 8 derived the four equations of backpropagation for dense layers. Nothing about them changes here – the network is still a composition of differentiable maps and the chain rule is still the chain rule – but the gradients of Eq. (10.16) are worth deriving explicitly, because the result is elegant and because the derivation is where errors hide.

Write $\delta(k,i,j) = \partial\mathscr{C}/\partial Z(k,i,j)$ for the error propagated back to the output of the convolution, as in Chapter 8. We want $\partial\mathscr{C}/\partial W$, $\partial\mathscr{C}/\partial b$ and $\partial\mathscr{C}/\partial X$.

The filter gradient.

By the chain rule, summing over every output position that $W(k,c,m,n)$ touched,

$$\frac{\partial\mathscr{C}}{\partial W(k,c,m,n)} = \sum_{i,j}\delta(k,i,j)\,\frac{\partial Z(k,i,j)}{\partial W(k,c,m,n)} = \sum_{i,j}\delta(k,i,j)\,X(c,Si+m-P,Sj+n-P)\,, \tag{10.29}$$

using Eq. (10.16) for the inner derivative. The right-hand side is itself a cross-correlation, of the input with the error. *Every* output position contributes to *every* filter weight: this is the price and the point of parameter sharing, and it is why a convolutional layer with few parameters can still receive a large, well-averaged gradient.

The bias gradient.

Since $b(k)$ is added at every spatial position of feature map k,

$$\frac{\partial\mathscr{C}}{\partial b(k)} = \sum_{i,j}\delta(k,i,j). \tag{10.30}$$

The input gradient.

Here we must collect every output that a given input pixel influenced,

$$\frac{\partial \mathscr{C}}{\partial X(c,p,q)} = \sum_{k} \sum_{i,j} \delta(k,i,j)\, W(k,c,\, p - Si + P,\; q - Sj + P), \tag{10.31}$$

where terms with filter indices outside $[0, F-1]$ are omitted. For $S = 1$ this is a convolution of δ with the *spatially flipped* kernel – the transposed convolution – which is the cleanest way to remember it: the forward pass correlates, the backward pass convolves.

The backward pass is the adjoint. All three results are instances of one fact. The forward map $\boldsymbol{X} \mapsto \boldsymbol{Z}$ is linear, represented by the matrix $\boldsymbol{W}'$ of Eq. (10.19), so the gradient with respect to the input is $\boldsymbol{W}'^{\top}\boldsymbol{\delta}$. Backpropagation through any linear layer is multiplication by its adjoint, and Eq. (10.31) is simply $\boldsymbol{W}'^{\top}$ written out in index form. The code of Section 10.9 exploits this directly, and we test it by checking the adjoint identity $\langle \mathscr{A}\boldsymbol{x}, \boldsymbol{y}\rangle = \langle \boldsymbol{x}, \mathscr{A}^{*}\boldsymbol{y}\rangle$ numerically.

The transposed convolution.

The remark that the backward pass is a convolution with the flipped kernel is worth promoting to a definition, because the same operation appears in the forward direction in every generative architecture of Part IV: the decoder of Chapter 15, the denoiser of Chapter 16 and the generator of Chapter 17 all have to *enlarge* a small array into a large one.

Proposition 10.13 (Transposed convolution). *Let $\mathscr{A}$ be the convolution of Eq. (10.16) with filter size F, stride S and padding P, mapping an input of side H_1 to an output of side H_2 given by Eq. (10.17). Its adjoint $\mathscr{A}^{*}$ – the map applied in the backward pass, and the layer called* `ConvTranspose2d` *or* `Conv2DTranspose` *– maps an array of side H_2 to one of side*

$$H_1' = S(H_2 - 1) + F - 2P, \tag{10.32}$$

and is itself a convolution: for $S = 1$ it is the correlation with the spatially flipped kernel and padding $F - 1 - P$, and for $S > 1$ it is the same after inserting $S - 1$ zeros between adjacent input elements.

Proof. Since $\mathscr{A}$ is linear with matrix $\boldsymbol{W}'$ as in Eq. (10.19), its adjoint has matrix $\boldsymbol{W}'^{\top}$ and therefore maps $\mathbb{R}^{H_2 \times H_2} \to \mathbb{R}^{H_1 \times H_1}$; solving Eq. (10.17) for H_1 in the case where S divides $H_1 + 2P - F$ gives Eq. (10.32). For the second statement, Eq. (10.31) with $S = 1$ reads $\partial\mathscr{C}/\partial X(c,p,q) = \sum_k \sum_{i,j} \delta(k,i,j) W(k,c,p-i+P, q-j+P)$, which is the correlation of δ with W read in reverse index order, that is with the flipped kernel, and the index range requires $F - 1 - P$ zeros of padding on δ. For $S > 1$, Eq. (10.31) has δ evaluated only at positions Si, which is δ upsampled by inserting $S - 1$ zeros.

Equation (10.32) is the formula to reach for when a generator has to be designed backwards from its output size, and the zero insertion is the reason transposed convolutions produce the checkerboard artefacts that are visible in early GAN samples: neighbouring output pixels are computed from different numbers of genuine inputs when S does not divide F. Choosing F a multiple of S – $F = 4$, $S = 2$ is the usual pair, and is what Chapter 17 uses – removes them.

Through a pooling layer.

Max pooling is not differentiable everywhere, but it is differentiable almost everywhere, and where it is the derivative is a routing:

$$\frac{\partial \mathscr{C}}{\partial X(c,p,q)} = \sum_{i,j} \delta(c,i,j) \mathbb{1}\left[(p,q) = \operatorname*{argmax}_{(m,n)} X(c, Si+m, Sj+n)\right]. \tag{10.33}$$

The gradient flows only to the element that won the maximum, and the argmax must be recorded during the forward pass. Ties have measure zero and are broken arbitrarily. For average pooling the gradient is instead spread equally, each of the F^2 inputs receiving δ/F^2.

10.9 Implementation from scratch

We now write the whole thing in NumPy, in the same style as Chapter 8: parameters are plain arrays, the forward pass is a function, and every gradient is checked against finite differences before anything is trained.

10.9.1 The im2col transformation

The six nested loops implied by Eq. (10.16) are correct and unusably slow in Python. The standard remedy is *im2col*: rearrange every $F \times F$ patch of the input into a column, so that the convolution becomes a single matrix product. If $\boldsymbol{X}$ has shape (N, C, H, W), the result has shape $(N, CF^2, H_2 W_2)$ and

$$\boldsymbol{Z} = \boldsymbol{W}_{\text{flat}} \, \text{im2col}(\boldsymbol{X}) + \boldsymbol{b}, \qquad \boldsymbol{W}_{\text{flat}} \in \mathbb{R}^{K \times CF^2}, \tag{10.34}$$

which BLAS executes at full speed. The cost is memory: each input element is replicated up to F^2 times.

```
def im2col(X, F, S, P):
    """Rearrange every F x F patch of X into a column, Eq. (10.im2col).

    X has shape (N, C, H, W); the result has shape (N, C*F*F, H2*W2) with
    H2 = (H - F + 2P)/S + 1, so that a convolution becomes one matrix product.
    """
    N, C, H, W = X.shape
    H2 = (H - F + 2 * P) // S + 1
    W2 = (W - F + 2 * P) // S + 1
    Xp = pad2d(X, P)
    cols = np.empty((N, C * F * F, H2 * W2))
    for i in range(F):
        for j in range(F):
            patch = Xp[:, :, i:i + S * H2:S, j:j + S * W2:S]      # (N,C,H2,W2)
            cols[:, (i * F + j)::F * F, :] = patch.reshape(N, C, -1)
    return cols


def col2im(cols, X_shape, F, S, P):
    """Adjoint of im2col: scatter columns back, accumulating overlaps."""
    N, C, H, W = X_shape
    H2 = (H - F + 2 * P) // S + 1
    W2 = (W - F + 2 * P) // S + 1
```

```
    Xp = np.zeros((N, C, H + 2 * P, W + 2 * P))
    for i in range(F):
        for j in range(F):
            patch = cols[:, (i * F + j)::F * F, :].reshape(N, C, H2, W2)
            np.add.at(Xp, (slice(None), slice(None),
                           slice(i, i + S * H2, S), slice(j, j + S * W2, S)), patch)
    return Xp if P == 0 else Xp[:, :, P:-P, P:-P]
```

The loops run over the filter, which is small, not over the image. The forward and backward passes follow directly from Eqs. (10.29)–(10.31).

```
def conv_forward(X, W, b, S=1, P=0):
    """Cross-correlation, Eq. (10.crosscorr2d).  W has shape (K, C, F, F)."""
    N, C, H, Wd = X.shape
    K, _, F, _ = W.shape
    H2 = (H - F + 2 * P) // S + 1
    W2 = (Wd - F + 2 * P) // S + 1
    cols = im2col(X, F, S, P)                          # (N, C*F*F, H2*W2)
    out = np.einsum("kd,ndp->nkp", W.reshape(K, -1), cols) + b[None, :, None]
    return out.reshape(N, K, H2, W2), cols


def conv_backward(dY, X, W, cols, S=1, P=0):
    """Gradients of the convolution, Eqs. (10.dconvW), (10.dconvb), (10.dconvX)."""
    N, K, H2, W2 = dY.shape
    _, C, F, _ = W.shape
    dYf = dY.reshape(N, K, -1)                                      # (N,K,H2W2)
    dW = np.einsum("nkp,ndp->kd", dYf, cols).reshape(W.shape)
    db = dYf.sum(axis=(0, 2))
    dcols = np.einsum("kd,nkp->ndp", W.reshape(K, -1), dYf)
    dX = col2im(dcols, X.shape, F, S, P)
    return dX, dW, db
```

Max pooling stores the argmax so that Eq. (10.33) can route the gradient:

```
def maxpool_forward(X, F=2, S=2):
    """Max pooling, Eq. (10.maxpool).  Returns the output and an argmax mask."""
    N, C, H, W = X.shape
    H2, W2 = (H - F) // S + 1, (W - F) // S + 1
    patches = np.empty((N, C, H2, W2, F * F))
    for i in range(F):
        for j in range(F):
            patches[..., i * F + j] = X[:, :, i:i + S * H2:S, j:j + S * W2:S]
    idx = patches.argmax(axis=-1)
    out = np.take_along_axis(patches, idx[..., None], axis=-1)[..., 0]
    return out, idx


def maxpool_backward(dY, X, idx, F=2, S=2):
    """Route each gradient to the argmax that produced it, Eq. (10.dmaxpool)."""
    N, C, H, W = X.shape
    H2, W2 = dY.shape[2], dY.shape[3]
    dX = np.zeros_like(X)
    for i in range(F):
        for j in range(F):
            mask = (idx == i * F + j)
            np.add.at(dX, (slice(None), slice(None),
                           slice(i, i + S * H2, S), slice(j, j + S * W2, S)),
                      dY * mask)
    return dX
```

10.9.2 Verifying the implementation

Four checks, in increasing order of strength. First, that `conv_forward` agrees with the definition (10.16) coded as an explicit quadruple loop, for several strides and paddings:

```
S=1 P=0  shape (2, 4, 5, 5)  max|conv-naive| = 3.553e-15
S=1 P=1  shape (2, 4, 7, 7)  max|conv-naive| = 3.553e-15
S=2 P=1  shape (2, 4, 4, 4)  max|conv-naive| = 3.553e-15
S=2 P=0  shape (2, 4, 3, 3)  max|conv-naive| = 1.776e-15
S=3 P=2  shape (2, 4, 3, 3)  max|conv-naive| = 2.220e-15
```

Second, the adjoint identity of the notebox in Section 10.8, $\langle \mathrm{im2col}(\boldsymbol{x}), \boldsymbol{y} \rangle = \langle \boldsymbol{x}, \mathrm{col2im}(\boldsymbol{y}) \rangle$, which must hold to rounding:

```
F=3 S=1 P=0:  |<Ax,y>-<x,A*y>| = 7.11e-15
F=3 S=2 P=1:  |<Ax,y>-<x,A*y>| = 4.44e-15
F=5 S=1 P=2:  |<Ax,y>-<x,A*y>| = 4.44e-15
```

Third, that the analytic gradients match central differences:

```
 S=1 P=0:  dX 9.73e-09   dW 7.76e-09   db 1.53e-09
 S=1 P=1:  dX 8.52e-09   dW 8.45e-09   db 5.43e-09
 S=2 P=1:  dX 4.94e-09   dW 5.36e-09   db 1.78e-09
 max-pool  dX 1.19e-09
```

Fourth, the structural claims of Sections 10.3.2 and 10.4.2. Reconstructing $\boldsymbol{W}'$ column by column from `conv_forward` for the 3×3 input and 2×2 kernel reproduces Eq. (10.19) exactly: sixteen non-zero entries out of thirty-six, taking four distinct values, and $\boldsymbol{W}'\boldsymbol{X}' = \mathrm{vec}(\boldsymbol{Y})$ to 0.0. The polynomial identity of Section 10.3.1 is confirmed by comparing the coefficients from `polymul`, from `np.convolve` and from $\boldsymbol{T}_\alpha \boldsymbol{\beta}$, which agree exactly.

Equivariance, Theorem 10.7, and its failure under stride, Proposition 10.8, are equally sharp:

```
S=1 shift 1: max|conv(T_s x)-T_s conv(x)| = 0.00e+00
S=1 shift 2: max|conv(T_s x)-T_s conv(x)| = 0.00e+00
S=1 shift 3: max|conv(T_s x)-T_s conv(x)| = 0.00e+00
S=2 shift 1: max|.| = 7.119e+00   <- NOT equivariant
S=2 shift 2: max|.| = 0.000e+00   <- equivariant
```

You cannot naively gradient-check a ReLU and max-pool network. Running the finite-difference test on the full network of Section 10.9.3 gives relative errors of 10^{-8} for five of the six parameter arrays and a relative error of 1.0 – complete disagreement – for the first-layer bias $\boldsymbol{b}_1$. This is not a bug. A bias shifts an entire feature map, so a perturbation of $h = 10^{-6}$ pushes some pre-activation across zero or changes which element wins a max-pool window. The network is piecewise linear, the central difference straddles a kink, and the two-sided quotient converges to nothing in particular. Repeating the same check with $\tanh$ in place of ReLU and average in place of max pooling – smooth everywhere, same backward-pass code paths – gives

```
W1 3.51e-07  b1 3.85e-07  W2 6.70e-08  b2 4.28e-09  W3 1.13e-07  b3 2.02e-09
```

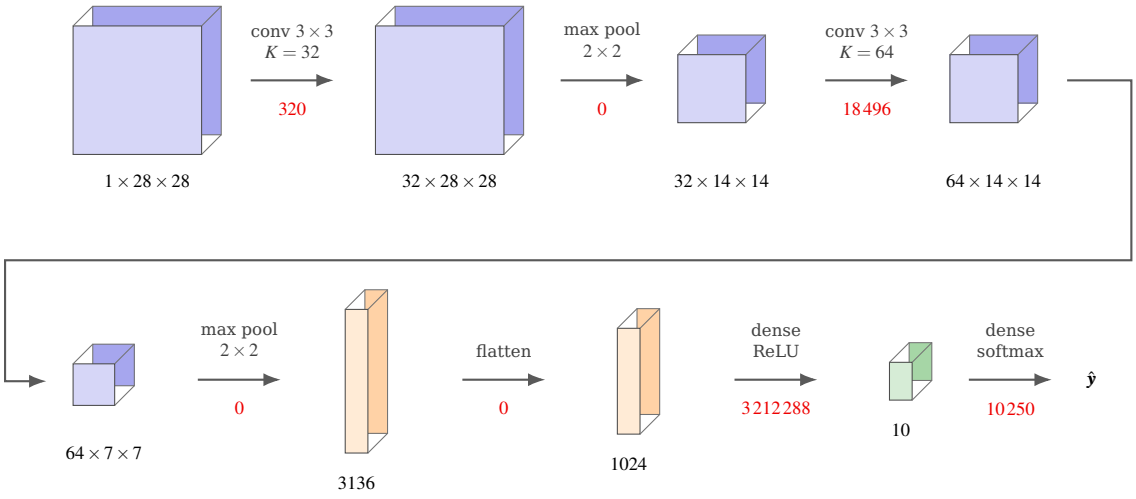
which is the finite-difference floor. The lesson is general: verify gradients on a smooth surrogate, then switch the activations back.

10.9.3 A complete network

The layers assemble into the standard pattern, convolution and pooling alternating and a dense layer at the end:

$$\boldsymbol{X} \xrightarrow{\text{conv}} \text{ReLU} \xrightarrow{\text{pool}} \text{conv} \xrightarrow{\text{ReLU}} \text{pool} \xrightarrow{\text{flatten}} \text{dense} \xrightarrow{\text{softmax}} \hat{\boldsymbol{y}}. \tag{10.35}$$

The softmax and cross-entropy of Chapter 5 are unchanged, and so is the convenient identity $\delta^L = (\boldsymbol{a}^L - \boldsymbol{y})/n$. Figure 10.11 draws the whole chain with the tensor shapes and parameter counts of the network we hand to PyTorch and Keras in Section 10.12, and it is worth looking at before reading the code: the shapes are the only thing that has to be got right, and every one of them follows from Proposition 10.4.



parameter counts in red; the total is 3 241 354, of which the two convolutional layers hold 18 816, or 0.58%

Fig. 10.11 The pattern of Eq. (10.35), instantiated with the widths of the PyTorch and Keras listings of Section 10.12 on MNIST, with $F = 3$, $S = 1$, $P = 1$ and 2×2 pooling throughout; the dropout layer of Section 10.11, which has no parameters, is not drawn. Each slab is one tensor and is labelled with its shape; every spatial size follows Proposition 10.4 and every parameter count is the one checked against Keras in Section 10.12.4. The shape of the picture is the argument of this chapter and its own refutation at once. The two convolutional layers, which carry all the locality, sharing and equivariance, hold 18 816 of the 3 241 354 parameters; the single dense layer that follows the flattening, which has none of those properties, holds 3 212 288.

```
def forward(p, X):
    """Returns the class probabilities and everything the backward pass needs."""
    Z1, c1 = conv_forward(X, p["W1"], p["b1"], S=1, P=1)
    A1 = relu(Z1)
    P1, i1 = maxpool_forward(A1, 2, 2)
    Z2, c2 = conv_forward(P1, p["W2"], p["b2"], S=1, P=1)
    A2 = relu(Z2)
    P2, i2 = maxpool_forward(A2, 2, 2)
    flat = P2.reshape(P2.shape[0], -1)
    Z3 = flat @ p["W3"] + p["b3"]
    return softmax(Z3), (X, Z1, A1, i1, P1, c1, Z2, A2, i2, P2, flat, c2)


def backward(p, cache, probs, Y):
    """Backpropagation, Eqs. (10.dconvW)-(10.dmaxpool); delta^L = (a-y)/n."""
    X, Z1, A1, i1, P1, c1, Z2, A2, i2, P2, flat, c2 = cache
```

```
    n = X.shape[0]
    d3 = (probs - Y) / n                                   # softmax + CE
    g = {"W3": flat.T @ d3, "b3": d3.sum(axis=0)}
    dflat = d3 @ p["W3"].T
    dP2 = dflat.reshape(P2.shape)
    dA2 = maxpool_backward(dP2, A2, i2, 2, 2)
    dZ2 = dA2 * relu_prime(Z2)
    dP1, g["W2"], g["b2"] = conv_backward(dZ2, P1, p["W2"], c2, S=1, P=1)
    dA1 = maxpool_backward(dP1, A1, i1, 2, 2)
    dZ1 = dA1 * relu_prime(Z1)
    _, g["W1"], g["b1"] = conv_backward(dZ1, X, p["W1"], c1, S=1, P=1)
    return g
```

The initialisation is that of Chapter 8, with the observation that the fan-in of a convolutional filter is CF^2 rather than the number of units in the previous layer:

```
def init_cnn(C_in=1, K1=8, K2=16, F=3, n_out=10, flat=None, rng=None):
    """He initialisation, Eq. (8.he), with fan-in C*F*F for a conv kernel."""
    rng = np.random.default_rng(0) if rng is None else rng
    def he(shape, fan_in):
        return rng.normal(0.0, np.sqrt(2.0 / fan_in), shape)
    return {
        "W1": he((K1, C_in, F, F), C_in * F * F), "b1": np.zeros(K1),
        "W2": he((K2, K1, F, F), K1 * F * F),     "b2": np.zeros(K2),
        "W3": he((flat, n_out), flat),            "b3": np.zeros(n_out),
    }
```

10.10 Does it actually help? An experiment

The arguments of Sections 10.1 and 10.5 are arguments; this section measures. We use the 8×8 handwritten digits shipped with scikit-learn, 1797 images, split $70/30$, and compare the network of Eq. (10.35) against a dense network of the same size. Both are trained by Adam with $\gamma = 3\times 10^{-3}$ for twenty epochs with batches of thirty-two, and every figure below is the mean over five random seeds.

Table 10.1 Convolutional and dense networks compared on the scikit-learn digits. The upper block uses the images as supplied, centred in an 8×8 frame; the lower block places the same digits at a uniformly random offset in a 12×12 frame. Ranges are minimum and maximum over five seeds.

Data	Model	Parameters	Test accuracy
centred 8×8	CNN	1898	0.9656 (0.9574–0.9704)
centred 8×8	dense	1885	0.9670 (0.9593–0.9704)
random offset 12×12	CNN	2698	**0.8774** (0.8574–0.9185)
random offset 12×12	dense	6830	0.7070 (0.6815–0.7315)

The first block is a negative result and we report it as such. On centred 8×8 digits the convolutional network is *not* better: 0.9656 against 0.9670, a difference far smaller than the seed-to-seed spread of both. With matched parameter counts the two architectures are indistinguishable. This is not a failure of the implementation but a statement about the data. The digits are 64 pixels, pre-centred and pre-scaled; there is almost no translation for equivariance to exploit, and a dense layer can afford one weight per pixel per hidden unit. A reader who met CNNs only through MNIST-style demonstrations might reasonably conclude they are always better, and on this problem they are not.

The second block is where the argument lands. Placing the same digits at a random offset in a slightly larger frame – a change that a human classifier would not notice – costs the dense network twenty-six points of accuracy, from 0.9670 to 0.7070, while the convolutional network gives up nine, from 0.9656 to 0.8774. The CNN wins by seventeen points *using* 2.5 *times fewer parameters*, 2698 against 6830. The dense network must learn each digit separately at each of the twenty-five possible positions from a training set that contains only 1257 images; the convolutional network learns each feature once, by Theorem 10.7, and pooling supplies the tolerance to where it landed.

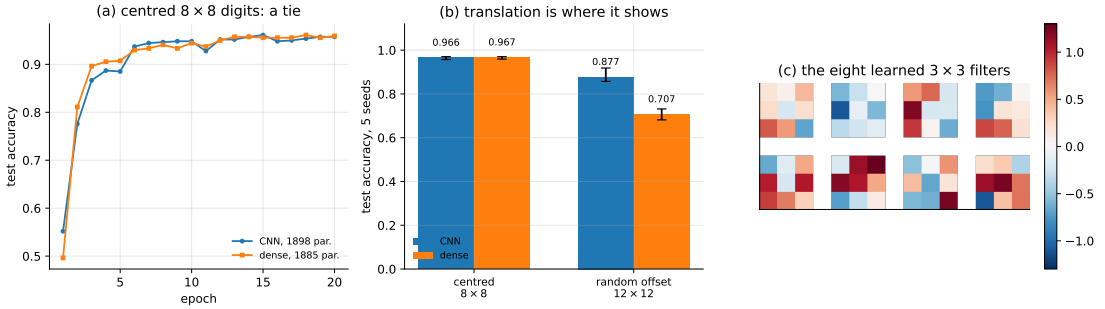


Fig. 10.12 (a) Test accuracy against epoch on the centred digits for the two architectures of Table 10.1 at matched parameter count; the curves are on top of one another. (b) The same comparison on centred and randomly translated digits, bars showing the mean and whiskers the range over five seeds. Translation is what separates the two. (c) The eight 3×3 first-layer filters after training: oriented opposed-sign pairs, which is to say edge detectors, learned rather than prescribed.

Panel (c) of Figure 10.12 is worth a glance. The first-layer filters, initialised at random and trained only against a classification loss, converge to small oriented patterns with a positive lobe beside a negative one. These are edge detectors of the kind that signal processing prescribes by hand – the family of Eq. (10.26) – and nobody asked for them; they are what minimising cross-entropy on images produces. Section 10.7.3 said why, and Section 10.13 reads them as directional high-pass filters.

10.11 Dropout

The network we are about to hand to the libraries uses one ingredient we have not yet met. During training each unit is set to zero independently with probability p,

$$\tilde{a}_j = \frac{1}{1-p}\, m_j a_j, \qquad m_j \sim \text{Bernoulli}(1-p), \tag{10.36}$$

where the factor $1/(1-p)$ keeps $\mathbb{E}[\tilde{a}_j] = a_j$ so that no rescaling is needed at test time, when dropout is switched off entirely. The effect is to prevent any unit from relying on the presence of any other, forcing the representation to be distributed rather than concentrated; it can also be read as training an ensemble of exponentially many subnetworks with shared weights, in the spirit of the bagging of Chapter 7.

Dropout is applied mainly after dense layers, where the parameters are and hence where overfitting is. Convolutional layers are already regularised by parameter sharing – a filter that overfits at one location would have to overfit identically everywhere – so dropout after a convolution is less common and often unhelpful.

10.12 The same network in PyTorch and TensorFlow

Nothing above is a substitute for a library, and no serious work uses a NumPy convolution. The point of writing one was to know what the library is doing – and that is a claim we are now in a position to *test* rather than assert. This section gives the same architecture in both major frameworks, then loads one set of weights into all three implementations and compares them gradient by gradient, and finally trains the network on the sixty thousand MNIST images that the arithmetic of Section 10.4.1 was always about.

10.12.1 PyTorch

PyTorch expects tensors of shape (N, C, H, W), the same layout as our NumPy code, and the layer
`nn.Conv2d(C_in, K, F, padding=P)`
implements exactly Eq. (10.16). The example below is the MNIST network of the lecture notes, with two convolutional blocks, dropout and two dense layers.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,)),      # MNIST mean and std
])
train_set = datasets.MNIST(root="./data", train=True,  download=True,
                           transform=transform)
test_set  = datasets.MNIST(root="./data", train=False, download=True,
                           transform=transform)
train_loader = torch.utils.data.DataLoader(train_set, batch_size=64, shuffle=True)
test_loader  = torch.utils.data.DataLoader(test_set,  batch_size=64)


class CNN(nn.Module):
    """Eq. (10.arch) with two conv blocks; 28 -> 14 -> 7 by Eq. (10.outsize)."""
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, padding=1)   # 'same': 28 x 28
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)  # 'same': 14 x 14
        self.pool = nn.MaxPool2d(2, 2)                # halves each time
        self.fc1 = nn.Linear(64 * 7 * 7, 1024)
        self.fc2 = nn.Linear(1024, 10)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))          # (N, 32, 14, 14)
        x = self.pool(F.relu(self.conv2(x)))          # (N, 64,  7,  7)
        x = x.view(-1, 64 * 7 * 7)
        x = self.dropout(F.relu(self.fc1(x)))
        return self.fc2(x)                            # logits, not probabilities


model = CNN().to(device)
criterion = nn.CrossEntropyLoss()      # applies softmax internally
```

```
optimizer = optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(10):
    model.train()
    running = 0.0
    for data, target in train_loader:
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        loss = criterion(model(data), target)
        loss.backward()               # Eqs. (10.dconvW)-(10.dmaxpool), by autograd
        optimizer.step()
        running += loss.item()
    print(f"epoch {epoch+1}: loss {running/len(train_loader):.4f}")

model.eval()
correct = total = 0
with torch.no_grad():
    for data, target in test_loader:
        pred = model(data.to(device)).argmax(dim=1)
        correct += (pred == target.to(device)).sum().item()
        total += target.size(0)
print(f"test accuracy {100*correct/total:.2f}%")
```

Two details are easy to get wrong. `nn.CrossEntropyLoss` expects *logits* and applies the softmax itself, so the network must not end with a softmax; applying one twice is a common and quiet bug. And `model.train()` versus `model.eval()` matters because of dropout, which is active only during training – at test time all units are used and the activations are rescaled to compensate.

10.12.2 TensorFlow and Keras

Keras uses the opposite channel convention, (N, H, W, C), and infers input shapes after the first layer.

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models

(train_images, train_labels), (test_images, test_labels) = datasets.mnist.load_data()
train_images = train_images.reshape((-1, 28, 28, 1)).astype("float32") / 255.0
test_images  = test_images.reshape((-1, 28, 28, 1)).astype("float32") / 255.0

model = models.Sequential([
    layers.Conv2D(32, (3, 3), padding="same", activation="relu",
                  input_shape=(28, 28, 1)),          # note channels LAST
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), padding="same", activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(1024, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(10),                                # logits
])

model.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=["accuracy"])
model.summary()          # prints Eq. (10.paramcount) layer by layer

model.fit(train_images, train_labels, epochs=10, batch_size=64,
          validation_data=(test_images, test_labels))
```

The flag `from_logits=True` plays the role that `nn.CrossEntropyLoss` plays in PyTorch. It is worth calling `model.summary()` and checking the parameter counts against Eq. (10.18) by hand: the first layer should report $32 \times (1 \times 9 + 1) = 320$ and the second $64 \times (32 \times 9 + 1) = 18496$, while the dense layer that follows carries $64 \cdot 7 \cdot 7 \cdot 1024 + 1024 = 3212288$. That last number is instructive. The two convolutional layers together hold 18816 parameters, less than one per cent of the network, and the dense layer holds almost all of it – which is why modern architectures replace the large dense layer with global average pooling.

10.12.3 Do the three implementations agree?

The listings above and the NumPy code of Section 10.9 are supposed to be the same function. Nothing so far has checked it, and there is more room for error than one might think: the three carry their arrays in different layouts, and a transposition that happens to be self-consistent will train perfectly well while computing something other than what is on the page. The conventions are

	input	kernel
ours	(N,C,H,W)	(K,C,F,F)
PyTorch	(N,C,H,W)	(K,C,F,F)
Keras	(N,H,W,C)	(F,F,C,K)

so ours and PyTorch agree and Keras differs in both. There is a third trap: our `flatten` runs over (K,H,W) in C order while Keras flattens (H,W,K), so the rows of the dense weight matrix have to be permuted before they can be compared.

With all of that reconciled, we generate one set of weights, push it into all three, and evaluate the same eight real MNIST images. In double precision:

```
=== 1. one convolutional layer, identical weights ===
  output shape (N,K,H,W)      : (8, 4, 28, 28)
  max |ours  - torch|         : 1.332e-15
  max |ours  - keras|         : 1.776e-15
  max |torch - keras|         : 1.776e-15
  scale of the outputs        : 7.980

=== 3. the whole network of Eq. (10.arch), forward ===
  max |p_ours - p_torch|      : 3.053e-16
  max |p_ours - p_keras|      : 4.441e-16
  cross-entropy, ours         : 3.130017831683
  cross-entropy, torch        : 3.130017831724
  cross-entropy, keras        : 3.130017831724
```

The forward pass is settled. The interesting comparison is the backward one, because Eqs. (10.29)–(10.31) were derived by hand and the libraries obtain the same quantities by automatic differentiation, which shares no code and no reasoning with our derivation:

```
=== 4. the gradients of Section 10.convbackprop ===
   array      shape           |ours-torch|   |ours-keras|   scale
   W1     (4, 1, 3, 3)         2.082e-16     2.220e-16  4.05e-01
   b1     (4,)                 8.327e-17     2.776e-16  1.98e-01
   W2     (6, 4, 3, 3)         4.441e-16     4.441e-16  1.20e+00
   b2     (6,)                 8.327e-17     1.110e-16  2.69e-01
   W3     (294, 10)            3.331e-16     2.776e-16  6.38e-01
   b3     (10,)                5.551e-17     2.776e-17  2.46e-01
  worst disagreement anywhere : 4.441e-16
```

Every entry is at the double-precision rounding level, $\varepsilon \approx 2.2 \times 10^{-16}$, and the gradients themselves are of order one. Figure 10.13(b) plots the whole comparison against ε. The filter gradient (10.29), the bias gradient (10.30) and the input gradient (10.31) are what the frameworks compute; the derivation in Section 10.8 is not a simplified account of it.

This is a stronger statement than the finite-difference check of Section 10.9.2. That check compared our gradient against our own forward pass, and would have passed even if both had implemented the wrong convolution. This one compares against two independent implementations written by other people.

10.12.4 Parameter counts, checked

Running the arithmetic of Proposition 10.4 against what Keras reports for the network of the listings:

```
=== 5. parameter counts against Eq. (10.paramcount) ===
   layer                keras   K(C F^2 + 1) or n_in n_out + n_out
   conv2d(1->32)            320   32*(1*9+1) = 320
   conv2d(32->64)         18496   64*(32*9+1) = 18496
   dense(3136->1024)    3212288   3136*1024+1024 = 3212288
   dense(1024->10)        10250   1024*10+10 = 10250
   total                3241354
  the two convolutional layers hold 18816 parameters, 0.58% of the network;
  the first dense layer alone holds 3212288, 99.10%
```

The last two lines are the point, and Figure 10.13(c) shows them on a logarithmic axis. Everything this chapter has been about – the locality, the sharing, the equivariance, the fall from $\mathcal{O}(L^4)$ to $K(CF^2+1)$ – applies to 0.58% of this network. The other 99.1% sits in one dense layer that has none of those properties.

10.12.5 Training on MNIST, and what the dense layer is worth

Both programs were then trained on the full sixty thousand MNIST images for five epochs with Adam at 10^{-3}, batches of 64 and the same seed:

```
  torch   epoch 1: loss 0.1318  test accuracy 0.9878
  torch   epoch 3: loss 0.0346  test accuracy 0.9912
  torch   epoch 5: loss 0.0220  test accuracy 0.9873
  keras   epoch 1: loss 0.1237  test accuracy 0.9868
  keras   epoch 3: loss 0.0289  test accuracy 0.9922
  keras   epoch 5: loss 0.0187  test accuracy 0.9916
```

The two frameworks reach 0.9873 and 0.9916. They do not agree exactly and they should not be expected to: the initialisation, the shuffling and the dropout masks are drawn from different generators, and the epoch-to-epoch variation within each run is already 0.004. What the comparison establishes is that the two implement the same model, which the previous subsection settled to 10^{-16}; the difference here is the noise of a stochastic optimiser, not a difference of architecture.

Now the experiment that Section 10.12.4 invites. Replace the dense layer holding 99.1% of the parameters by *global average pooling* – average each of the 64 feature maps over its 7×7 extent, giving one number per map, and classify from those 64 numbers:

A factor of 167 fewer parameters costs about five points of accuracy after five epochs, and halves the time per epoch. That is the trade, stated honestly: global average pooling is not

Table 10.2 The network of Section 10.12 on MNIST, five epochs, with the dense head of the listings and with global average pooling in its place. The convolutional trunk is identical; only the classifier head changes.

Framework	Head	Parameters	Test accuracy	Seconds/epoch
PyTorch	dense 1024	3 241 354	0.9873	49
TensorFlow	dense 1024	3 241 354	0.9916	42
PyTorch	global average pooling	19 466	0.9342	24
TensorFlow	global average pooling	19 466	0.9399	20

free, and the claim sometimes made that the dense layer is pure waste is too strong. What is true is that the 3.2 million parameters buy five points, and that the pooled network is still learning when the run stops – its accuracy climbs from 0.83 to 0.94 across the five epochs while the dense network is flat after the first, which is Figure 10.13(a). On a harder problem, with more epochs and with the overfitting that three million parameters invite, the ordering commonly reverses; on MNIST in five epochs it does not.

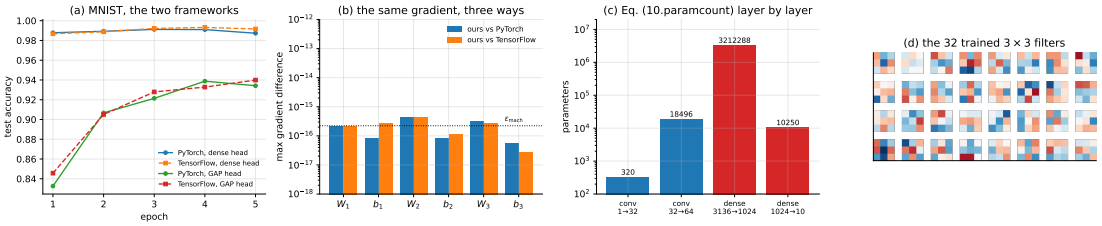


Fig. 10.13 (a) Test accuracy against epoch on MNIST for the network of Section 10.12 in both frameworks, with the dense head of the listings and with global average pooling in its place; the two frameworks lie on top of one another and the two heads do not. (b) The largest disagreement between our hand-derived gradients, Eqs. (10.29)–(10.31), and the same gradients obtained by automatic differentiation in each framework, against the double-precision unit roundoff. (c) Parameters per layer from Proposition 10.4: the two convolutional layers hold 0.58% of the network. (d) The thirty-two 3×3 first-layer filters of the trained PyTorch model, on the same colour scale; compare Figure 10.12(c), which shows the same structure emerging in the NumPy network on 8×8 digits.

10.12.6 Convolutional against dense networks on MNIST

Table 10.2 compared two heads on one convolutional trunk. The comparison that motivated the chapter – a convolutional network against the dense network of Chapter 8 on the same data – is Table 10.3, produced by `mnist_dense_vs_cnn.py`. Eight networks are trained on the full sixty thousand images with the same seed and evaluated on the ten thousand test images: a 784–100–10 ReLU network with softmax output, in NumPy with plain stochastic gradient descent and in PyTorch and Keras with Adam; the convolutional network of the listings in both frameworks; the same trunk with global average pooling; and the chapter's own NumPy convolutional network with $K_1 = 8$ and $K_2 = 16$ filters, 9 098 parameters, for three epochs. Figure 10.14 shows the learning curves, the parameter counts, the learned filters and the confusion matrix.

Four things are worth reading off the table.

The convolutional network against the dense one. Same data, same optimiser, same number of epochs: the PyTorch convolutional network makes 127 errors on the ten thousand test images where the dense network makes 280, and in Keras the counts are 84 against 280. The

Table 10.3 Dense and convolutional networks on MNIST: sixty thousand training images, the ten thousand test images, seed 1, five epochs unless stated, Adam at 10^{-3} with batches of 64 except for the NumPy dense network (plain SGD, $\gamma = 0.1$, batches of 100) and the NumPy convolutional network (Adam, batches of 32, three epochs). Seconds per epoch on two CPU cores.

Code	Model	Parameters	Test accuracy	Errors	s/epoch
NumPy	dense 784–100–10, SGD	79 510	0.9745	255	1
PyTorch	dense 784–100–10, Adam	79 510	0.9720	280	1
Keras	dense 784–100–10, Adam	79 510	0.9720	280	3
PyTorch	CNN of the listings	3 241 354	**0.9873**	127	47
Keras	CNN of the listings	3 241 354	**0.9916**	84	33
PyTorch	CNN, global average pooling	19 466	0.9342	658	18
Keras	CNN, global average pooling	19 466	0.9399	601	18
NumPy	CNN, $K_1 = 8$, $K_2 = 16$, 3 epochs	9 098	0.9858	142	72

standard error of an accuracy p measured on n images, $\sqrt{p(1-p)/n}$, is about 0.0017 for the dense network and 0.0011 for the convolutional one, so the gap of 1.5 points in PyTorch and 2.0 in Keras is many standard errors wide. This is the result that the digits experiment of Section 10.10 did *not* give on 8×8 centred digits: on 28×28 handwritten digits, with their strokes at varying positions, thickness and slant, the inductive bias of the convolutional layers pays.

PyTorch and Keras agree to the noise of the optimiser. The two dense networks reach 0.9720 and 0.9720, the two convolutional networks 0.9873 and 0.9916: the same models, as Section 10.12.3 established to 10^{-16}, with different initialisations, shuffles and dropout masks. The convolutional numbers are those of Table 10.2, reproduced exactly because the seed is the same.

Where the parameters are. Replacing the 3.2-million-parameter dense layer by the average of each 7×7 map – global average pooling, 19 466 parameters – costs about five points in five epochs, as Section 10.12.5 found; the pooled network is still climbing when the run stops, Figure 10.14(a). Ninety-nine per cent of the network does none of the convolutional work, and it is not pure waste either.

The architecture, not the count. The NumPy convolutional network with 9 098 parameters, trained for three epochs, reaches 0.9858 and beats every dense network in the table, each of which has nine times as many parameters. What the digits reward is the structure, not the number of weights. It is also slow: 72 seconds per epoch for a network a hundredth the size of the library one, which trains in 47. The NumPy code is for understanding; the library is the tool.

10.13 Convolution in the frequency domain: Fourier filters

Theorem 10.3 said that every circulant matrix is diagonalised by the Fourier basis, and the notebox that followed it said why that theorem does not end the chapter. This section takes the frequency picture seriously nonetheless, because it is the language in which filters have always been designed and read: a filter is described not by its weights but by what it does to each frequency, and the edge detectors and blurs of Section 10.7, the learned filters of Figure 10.13(d) and the layers of a network all have such a description. We first state the convolution theorem on the real line and for sampled data, with the examples that make it concrete; we then write a convolutional layer in Fourier language and see exactly where that language stops – at the activation and at the pooling; and we close with a survey of how Fourier filters are used, in signal processing, in differential equations and inside convolutional

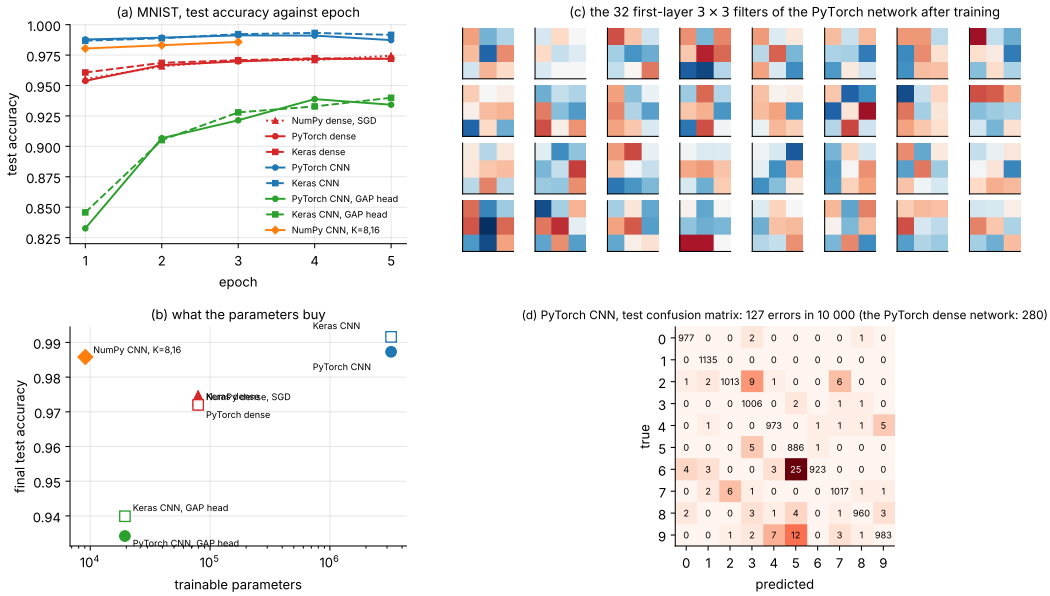


Fig. 10.14 The runs of Table 10.3. (a) Test accuracy against epoch for all eight networks; the two frameworks lie on top of one another model by model, and the three families – dense, convolutional with a dense head, convolutional with global average pooling – do not. (b) Final test accuracy against the number of trainable parameters: the 9098-parameter NumPy convolutional network sits above the 79510-parameter dense networks. (c) The thirty-two first-layer 3×3 filters of the trained PyTorch network: opposed-sign pairs at various orientations, the edge detectors of Section 10.7, learned. (d) The confusion matrix of the PyTorch convolutional network on the test set, diagonal removed from the colour scale; the largest remaining confusions are a six read as a five (25 images), a nine read as a five (12) and a two read as a three (9).

networks. The material follows the lecture on Fourier transforms and filtering of the course, and the conventions of Rowell's MIT notes [46].

10.13.1 The Fourier transform and the convolution theorem

We use angular frequency $\omega = 2\pi\nu$, with ν in cycles per unit time, and the pair

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(t)\, e^{-i\omega t}\, \mathrm{d}t, \qquad f(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \hat{f}(\omega)\, e^{i\omega t}\, \mathrm{d}\omega. \tag{10.37}$$

The coefficients $\hat{f}(\omega)$ are complex: they carry an amplitude *and a phase*, and both are needed to reconstruct f – a power spectrum $|\hat{f}|^2$ alone does not determine the signal. The integral formulas apply, for instance, when f and $\hat{f}$ are integrable; Fourier theory also provides an L^2 interpretation in which the integrals are limits in norm, and we shall need it once, for the ideal low-pass filter below.

Three different operations are easy to confuse, and the distinction is the subject of this section:

Operation	What it does
Fourier transform, $f \mapsto \hat{f}$	expresses the signal through its frequency components
Frequency filter, $f \mapsto \mathscr{F}^{-1}[H\hat{f}]$	multiplies each component by a response $H(\omega)$
Convolution filter, $f \mapsto h * f$	adds shifted, weighted copies of a kernel h

The transform is an invertible change of representation and loses nothing. Keeping all the coefficients allows reconstruction; suppressing or scaling some of them is a *filter*, and it is the filter, not the transform, that decides what is signal and what is noise. The convolution theorem will make the last two rows the same operation.

Convolution as a weighted superposition.

Equation (10.3) says that each input value $f(\tau)$ contributes a copy of the kernel shifted to τ and weighted by $f(\tau)$. For the normalised averaging kernel

$$h_a(s) = \begin{cases} 1/(2a), & |s| \le a, \\ 0, & |s| > a, \end{cases} \qquad (f * h_a)(t) = \frac{1}{2a}\int_{t-a}^{t+a} f(\tau)\,\mathrm{d}\tau, \tag{10.38}$$

the output is the local average of the input over an interval of width $2a$: a narrower kernel averages over a shorter interval. The factor $h(t-\tau)$, read as a function of τ, contains a reversal as well as a translation; a symmetric kernel hides the reversal, and Section 10.3.5 explained why a learned kernel does not care.

Theorem 10.14 (The convolution theorem on the real line). *For $f, h \in L^1(\mathbb{R})$,*

$$\widehat{f * h}(\omega) = \hat{f}(\omega)\,\hat{h}(\omega)\,: \tag{10.39}$$

convolution in the original variable is pointwise multiplication in frequency.

Proof. Interchange the integrals and substitute $s = t - \tau$:

$$\widehat{f * h}(\omega) = \int_{\mathbb{R}}\int_{\mathbb{R}} f(\tau)\,h(t-\tau)\,e^{-i\omega t}\,\mathrm{d}\tau\,\mathrm{d}t = \int_{\mathbb{R}}\int_{\mathbb{R}} f(\tau)\,h(s)\,e^{-i\omega(s+\tau)}\,\mathrm{d}s\,\mathrm{d}\tau = \left[\int_{\mathbb{R}} f(\tau)e^{-i\omega\tau}\,\mathrm{d}\tau\right]\left[\int_{\mathbb{R}} h(s)e^{-i\omega s}\,\mathrm{d}s\right].$$

The L^1 assumption makes the double integral absolutely convergent, which is what Fubini's theorem needs for the interchange. There is no 2π prefactor with the convention of Eq. (10.37); other conventions distribute the normalisation differently.

Proposition 10.15 (Frequency response). *Let $y = h * f$ be a linear, translation-invariant filter with impulse response $h \in L^1$, and let $H(\omega) = \hat{h}(\omega)$. Then*

$$\hat{y}(\omega) = H(\omega)\,\hat{f}(\omega), \tag{10.40}$$

and the Fourier modes are eigenfunctions of the filter,

$$\big(h * e^{i\omega(\cdot)}\big)(t) = e^{i\omega t}\int_{\mathbb{R}} h(s)\,e^{-i\omega s}\,\mathrm{d}s = H(\omega)\,e^{i\omega t}. \tag{10.41}$$

$|H(\omega)|$ is the gain *of the mode $e^{i\omega t}$ and $\arg H(\omega)$ its phase shift.*

Equation (10.40) is Theorem 10.14 renamed; Eq. (10.41) is a one-line computation, and it is the continuous form of the statement in Theorem 10.3 that every circulant matrix has the Fourier modes as eigenvectors. The sinusoid is not in L^1 on the real line, but the integral in Eq. (10.41) is absolutely convergent because h is, and that is all the computation uses. Non-linear filters – a median filter, or a convolution followed by a ReLU – have no such description, which is the point we return to in Section 10.13.5.

Multiplication and convolution exchange roles.

With the same convention,

$$\mathscr{F}[f*h] = \hat{f}\hat{h}, \qquad \mathscr{F}[wf] = \frac{1}{2\pi}\,\hat{w}*\hat{f}. \tag{10.42}$$

Multiplying a record by a window $w(t)$ therefore spreads its spectrum according to $\hat{w}$. A finite observation interval acts as a window on a longer signal, so a measured spectrum is always the true spectrum convolved with the transform of the window – the *leakage* of spectral analysis. Smooth windows reduce the sidelobes of that spreading at the price of broader peaks, and zero-padding a record samples its spectrum more densely but adds no information that was not measured.

10.13.2 Three filters, read in frequency

Gaussian smoothing.

The Gaussian kernel and its response are

$$h_\sigma(t) = \frac{1}{\sqrt{2\pi}\,\sigma}\,e^{-t^2/(2\sigma^2)}, \qquad H_\sigma(\omega) = e^{-\sigma^2\omega^2/2}, \tag{10.43}$$

a kernel that integrates to one, so that $H_\sigma(0) = 1$ and constants pass unchanged, and a gain that falls smoothly with frequency with no sharp cutoff: increasing σ strengthens the attenuation at every non-zero frequency. Send through it the two-tone signal

$$f(t) = \sin(2\pi\,3t) + 0.35\,\sin(2\pi\,25t), \qquad \sigma = 0.02\ \text{s}. \tag{10.44}$$

By Eq. (10.41) each sinusoid is multiplied by its own gain, $H_\sigma(2\pi\cdot 3) = 0.9314$ and $H_\sigma(2\pi\cdot 25) = 0.00719$, so

$$(h_\sigma * f)(t) = 0.9314\,\sin(2\pi\,3t) + 0.002517\,\sin(2\pi\,25t)\,: \tag{10.45}$$

the 25 Hz component is attenuated by a factor of 140, and the 3 Hz component is changed too, by seven per cent. Figure 10.15(a)–(c) shows the kernel, the gain and the two signals.

A three-point average.

For equally spaced samples x_n the centred average

$$y_n = \tfrac{1}{4}x_{n-1} + \tfrac{1}{2}x_n + \tfrac{1}{4}x_{n+1} \tag{10.46}$$

is the convolution with the three-tap kernel $h = (1,2,1)/4$ at offsets $(-1,0,1)$. Its response, with $\Omega = \omega\Delta t$ the dimensionless angular frequency in radians per sample, is

$$H(\Omega) = \sum_m h_m e^{-i\Omega m} = \tfrac{1}{4}e^{i\Omega} + \tfrac{1}{2} + \tfrac{1}{4}e^{-i\Omega} = \tfrac{1}{2}\big(1+\cos\Omega\big) = \cos^2(\Omega/2). \tag{10.47}$$

Constants ($\Omega = 0$) pass unchanged; the alternating sequence $x_n = (-1)^n$, which is the mode $\Omega = \pi$, is removed entirely – $\frac{1}{4} - \frac{1}{2} + \frac{1}{4} = 0$ at every n. The complementary kernel $(-1,2,-1)/4$, which is δ minus the average, has $H(\Omega) = \sin^2(\Omega/2)$: it kills constants and passes the alternating sequence unchanged. The two gains add to one, so the two filters split any signal into a smooth part and a varying part, Figure 10.16(c). The blur of Section 10.7 is the two-dimensional cousin of the first, the edge filters of the second.

An ideal low pass, and why sharp is wide.

The opposite extreme is the ideal low-pass filter, $H(\omega) = 1$ for $|\omega| \le \omega_c$ and 0 otherwise. Its kernel is

$$h(t) = \frac{1}{2\pi}\int_{-\omega_c}^{\omega_c} e^{i\omega t}\,\mathrm{d}\omega = \frac{\sin(\omega_c t)}{\pi t}, \qquad h(0) = \frac{\omega_c}{\pi}, \tag{10.48}$$

which is not in L^1 – the filter is well defined as a Fourier multiplier on L^2, which is where the L^2 interpretation is needed – and has infinite, slowly decaying oscillatory tails. A sharp cutoff in frequency means a wide and oscillatory kernel in time, and those tails are the origin of the *ringing* seen near sharp edges of a signal that has been filtered this way. Figure 10.16(d) shows the sampled version.

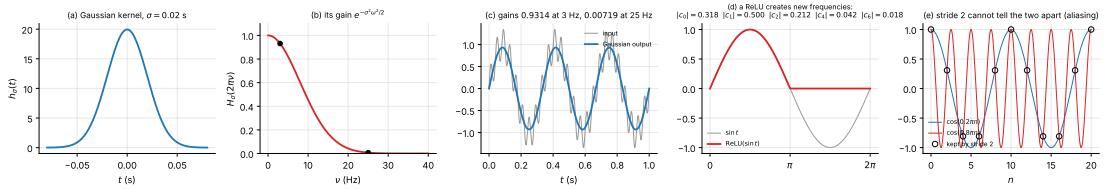


Fig. 10.15 (a) The Gaussian kernel of Eq. (10.43) with $\sigma = 0.02$ s and (b) its gain $e^{-\sigma^2\omega^2/2}$, with the values at 3 and 25 Hz marked. (c) The two-tone signal of Eq. (10.44) and its Gaussian output, Eq. (10.45): each sinusoid is multiplied by its own gain and nothing else happens, which is Proposition 10.15. (d) A ReLU applied to $\sin t$: the output has a constant term and all the even harmonics, Eq. (10.64), with the amplitudes measured by a DFT printed above; a non-linearity creates frequencies that were not there. (e) Aliasing: the two sequences $\cos(0.2\pi n)$ and $\cos(0.8\pi n)$ take identical values on the even samples, so a stride of two cannot tell them apart, Eq. (10.65).

10.13.3 Sampled data: the discrete Fourier transform

For N samples the transform pair is

$$X_k = \sum_{n=0}^{N-1} x_n\, e^{-2\pi ikn/N}, \qquad x_n = \frac{1}{N}\sum_{k=0}^{N-1} X_k\, e^{2\pi ikn/N}, \tag{10.49}$$

the normalisation of NumPy's `fft` and `ifft`: an unscaled forward transform and a $1/N$ on the inverse. With sample spacing Δt the record has length $T = N\Delta t$, the bins are spaced $\Delta\nu = 1/T$

apart, and the highest frequency the samples can represent is the Nyquist frequency

$$\nu_{\text{Nyquist}} = \frac{1}{2\Delta t}. \tag{10.50}$$

The bins hold positive and negative frequencies: in the standard order, for even N, $k = 0,\dots,N/2-1$ represents k/T and $k = N/2,\dots,N-1$ represents $(k-N)/T$, with the Nyquist bin shared by the two signs (`np.fft.fftfreq` returns these signed frequencies). The DFT is invertible, and the fast Fourier transform is an algorithm that computes it in $\mathcal{O}(N\log N)$ operations rather than N^2.

The two-tone signal, sampled.

Take $N = 256$ samples of Eq. (10.44) in one second, $t_n = n/256$, so that $T = 1$ s and both frequencies fall exactly on bins. Figure 10.16(b) shows the one-sided amplitudes $2|X_k|/N$: two lines, 1.00 at 3 Hz and 0.35 at 25 Hz, and nothing else. The transform reveals both components and retains both. Now choose a cutoff of 8 Hz on the signed bin frequencies ν_k,

$$H_k = \begin{cases} 1, & |\nu_k| \le 8 \text{ Hz}, \\ 0, & |\nu_k| > 8 \text{ Hz}, \end{cases} \qquad Y_k = H_k X_k, \qquad y = \text{IDFT}(Y). \tag{10.51}$$

The result is $y_n = \sin(2\pi 3t_n)$ to 2×10^{-15}: the 25 Hz component is gone, panel (a). Keeping both the positive and the negative low-frequency bins is what keeps the output real; the mask has no phase shift on the retained modes; and the separation succeeds because the wanted and the unwanted components occupy disjoint bands – if they overlapped in frequency, no mask could separate them.

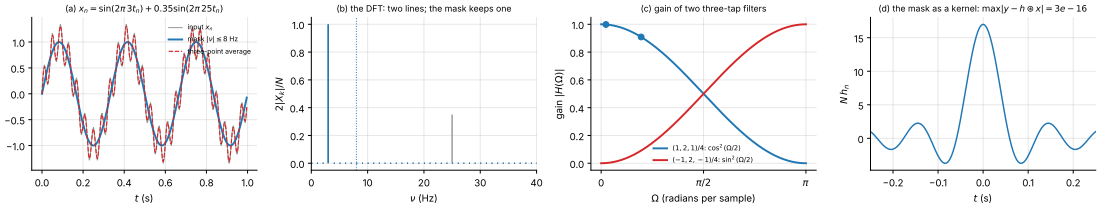


Fig. 10.16 (a) The two-tone signal of Eq. (10.44) sampled 256 times in one second, the output of the 8 Hz mask of Eq. (10.51), which is $\sin(2\pi 3t_n)$ to rounding, and the output of the three-point average of Eq. (10.46). (b) The DFT amplitudes $2|X_k|/N$: two lines; the mask keeps one. (c) The gains $\cos^2(\Omega/2)$ and $\sin^2(\Omega/2)$ of the two three-tap filters, Eq. (10.47), with the values of the average at the two tones marked (0.999 and 0.909: a gentle low pass). (d) The mask is itself a convolution: its kernel $h = \text{IDFT}(H)$ is a sampled $\sin(\omega_c t)/(\pi t)$ with long oscillating tails, Eq. (10.48), and the circular convolution $h \circledast x$ reproduces the masked output to 3×10^{-16}.

The DFT convolution theorem is circular.

For length-N sequences define the circular convolution

$$(h \circledast x)_n = \sum_{m=0}^{N-1} h_m x_{(n-m) \bmod N}, \tag{10.52}$$

in which the indices wrap around the record. With the normalisation of Eq. (10.49),

$$\mathrm{DFT}(h \circledast x)_k = H_k X_k, \qquad h \circledast x = \mathrm{IDFT}\big[\mathrm{DFT}(h)\,\mathrm{DFT}(x)\big], \tag{10.53}$$

which is Theorem 10.3 written for sequences; there is no extra $1/N$ in the sum, because the inverse transform already supplies it. The DFT treats the record as periodic, and that determines what happens at its ends. Two consequences are visible in Figure 10.16. The mask of Eq. (10.51) is *also* a convolution, with the kernel $h = \mathrm{IDFT}(H)$ of panel (d), a sampled and periodised version of Eq. (10.48), and $h \circledast x$ reproduces y to 3×10^{-16}: the second and third rows of the table at the start of this section are one operation. And the three-point average of panel (a), computed with `np.roll`, is exactly the circular convolution $(1,2,1)/4 \circledast x$ with the ends joined.

Proposition 10.16 (Linear convolution by zero padding). *Let x and h be finite sequences of lengths L_x and L_h, extended by zeros outside their supports. If both are padded with zeros to a common length*

$$N \geq L_x + L_h - 1, \tag{10.54}$$

their circular convolution of length N coincides with their linear convolution $x * h$ on its first $L_x + L_h - 1$ entries, and Eq. (10.53) computes it.

Proof. The linear convolution has support $0, \ldots, L_x + L_h - 2$. In the circular sum of Eq. (10.52) the index $(n-m) \bmod N$ differs from $n-m$ only when $n-m < 0$, that is when a term wraps from the end of the record to its start; the wrapped term $x_{n-m+N} h_m$ is non-zero only if $n - m + N \leq L_x - 1$ and $m \leq L_h - 1$, which requires $N \leq L_x + L_h - 2 - n \leq L_x + L_h - 2$. For N satisfying Eq. (10.54) no term wraps.

The smallest example makes the wrap-around visible. For $x = (1,2,3)$ and $h = (1,1)$ the linear convolution is $x * h = (1,3,5,3)$, but through the DFT,

FFT length	result	
$N = 3$	$(4,3,5)$	the last 3 has wrapped into the first bin: $1+3=4$
$N = 4$	$(1,3,5,3)$	the full linear convolution

and the code is three lines:

```
import numpy as np
x = np.array([1., 2., 3.])
h = np.array([1., 1.])
N = len(x) + len(h) - 1                          # Eq. (10.zeropad)
y = np.fft.ifft(np.fft.fft(x, N) * np.fft.fft(h, N))
y = np.real_if_close(y)                          # [1., 3., 5., 3.]
```

Larger N is also allowed; keep the first $L_x + L_h - 1$ outputs. Zero extension is a boundary assumption, exactly the padding P of Section 10.3.4; reflection and periodic extension model different problems. One more factor is easy to forget: when the array stores samples of a *continuous* kernel, the convolution integral is approximated by a quadrature and carries the spacing,

$$(h * f)(t_n) \approx \Delta t \sum_m h(t_n - t_m)\, f(t_m), \tag{10.55}$$

whereas the weights of a discrete filter such as $(1,2,1)/4$ already include it.

Cost, and when the FFT wins.

Direct convolution of N samples with a kernel of K taps costs $\mathcal{O}(NK)$ operations; the FFT route costs three transforms, $\mathcal{O}(N \log N)$, plus N multiplications. For the 3×3 filters of a convolutional network the direct product wins, and that is what the libraries do (the notebox after

Theorem 10.3 put the crossover near $F \approx 11$). For long kernels – the sinc of Eq. (10.48), a wide blur, an audio reverberation, a correlation over a whole record – the FFT wins by orders of magnitude. Padding is the boundary model in either case:

Padding	Consequence for an FFT implementation
circular	a same-size DFT diagonalises the filter directly
zero	pad to Eq. (10.54), reverse the correlation kernel, crop and align
reflection	extend the data by the boundary rule first, then filter

An FFT and a direct kernel agree only when padding, stride, dilation, kernel origin and output cropping all match, and using an FFT does not turn zero boundaries into periodic ones unless one forgets the extension.

10.13.4 Two dimensions: the photograph in Fourier space

Everything extends to images by transforming along both axes, with a pair of frequencies (Ω_x, Ω_y) or, in cycles per image, (k_x, k_y). Figure 10.17(a) is the log-amplitude spectrum of the photograph of Section 10.7: most of the energy sits at low frequencies, and the bright horizontal and vertical lines through the origin are the signatures of the many vertical and horizontal edges in the picture. Panel (b) keeps only the coefficients with $\sqrt{k_x^2 + k_y^2} \le 25$ cycles per image – 2.6% of them – and transforms back: a blur, the low-pass filter of Section 10.7 done by multiplication. Panel (c) keeps the rest: the edges, a high-pass filter.

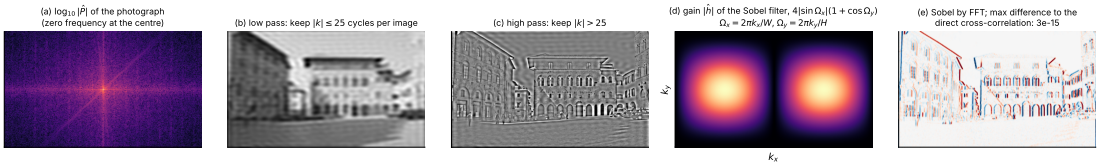


Fig. 10.17 (a) $\log_{10}|\hat{P}(k_x,k_y)|$ of the 211×360 photograph of Figure 10.10, zero frequency at the centre. (b) Keep only $\sqrt{k_x^2+k_y^2} \le 25$ cycles per image, 2.6% of the coefficients, and transform back: a blur. (c) Keep the rest: the edges. (d) The gain of the Sobel filter W_{vert}, Eq. (10.58): zero on the axis $\Omega_x = 0$, where a horizontal edge lives, largest at $|\Omega_x| = \pi/2$, and attenuated at high Ω_y by the $(1,2,1)$ smoothing. (e) The Sobel map computed as $\text{IDFT}(\hat{h}\hat{P})$ agrees with the direct cross-correlation to 3×10^{-15}: the two routes are one operation.

The small filters of Section 10.7 have responses that can be written down. The two-dimensional smoothing kernel and the discrete negative Laplacian,

$$h_{\text{sm}} = \frac{1}{16}\begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}, \qquad h_{\text{L}} = \begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix}, \tag{10.56}$$

have

$$H_{\text{sm}}(\Omega_x,\Omega_y) = \cos^2(\Omega_x/2)\cos^2(\Omega_y/2), \qquad H_{\text{L}}(\Omega_x,\Omega_y) = 4 - 2\cos\Omega_x - 2\cos\Omega_y \approx \Omega_x^2 + \Omega_y^2, \tag{10.57}$$

the first because h_{sm} is the outer product of $(1,2,1)/4$ with itself and the response of a separable kernel is the product of the one-dimensional responses, the second by direct summation; the Laplacian vanishes at zero frequency and grows like the squared frequency, which is what a second derivative should do. The Sobel filter is separable too, $W_{\text{vert}} = (1,2,1)^T(-1,0,1)$,

a central difference along x times the $(1,2,1)$ smoothing along y. In the cross-correlation convention of the layer, Eq. (10.15), the response of $(-1,0,1)$ is $-e^{-i\Omega_x}+e^{i\Omega_x}=2i\sin\Omega_x$ and that of $(1,2,1)$ is $2+2\cos\Omega_y$, so

$$\left|\hat{h}_{\text{vert}}(\Omega_x,\Omega_y)\right| = 4\,|\sin\Omega_x|\,(1+\cos\Omega_y). \tag{10.58}$$

Figure 10.18 draws the three. Equation (10.58) is the frequency reading of the house of Figure 10.9: the gain is zero on the whole axis $\Omega_x=0$, so every pattern that does not change along a row – a horizontal edge – is invisible to W_{vert} and needs W_{hor}; it is largest at $|\Omega_x|=\pi/2$, an oscillation of period four pixels across the row; and the $(1+\cos\Omega_y)$ factor suppresses patterns that oscillate rapidly down a column, which is the smoothing. Finally, panel (e) of Figure 10.17 applies W_{vert} to the photograph twice: directly, and as $\text{IDFT}(\hat{h}\hat{P})$ with the reversed kernel embedded in an $H\times W$ array. The two maps agree to 3×10^{-15}.

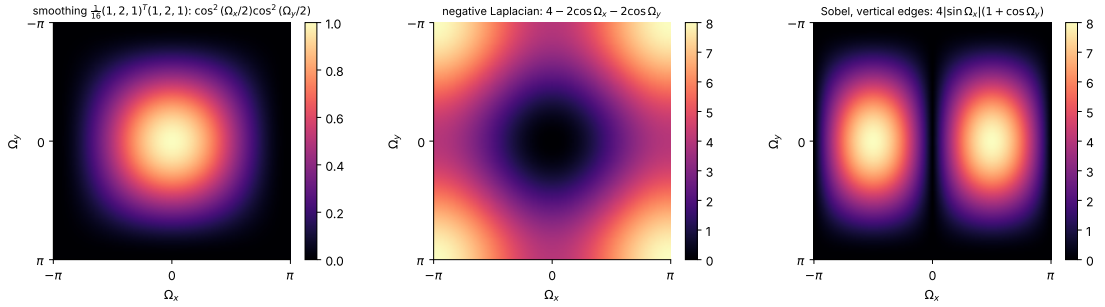


Fig. 10.18 The gains of three 3×3 filters over the square $[-\pi,\pi]^2$ of spatial frequencies, Eqs. (10.57) and (10.58). The smoothing kernel passes long wavelengths; the Laplacian vanishes at the origin and emphasises rapid variation in every direction; the Sobel filter is a directional high pass, zero along $\Omega_x=0$. All three are trigonometric polynomials of degree one in each variable – the smooth surfaces are what a 3×3 kernel can do, and a sharp cutoff such as Eq. (10.51) is not among them.

10.13.5 A convolutional layer in Fourier language

Consider a one-dimensional layer with stride one, real weights and circular boundaries, Eq. (10.16) with its indices centred,

$$z_a[n] = b_a + \sum_{b=1}^{C_{\text{in}}}\sum_{r=-R}^{R} w_{ab}[r]\,x_b[n+r], \qquad y_a[n]=\phi\big(z_a[n]\big), \tag{10.59}$$

where b indexes input channels, a output feature maps, and each w_{ab} is a short kernel shared across positions (a library stores the weights with non-negative indices and realises the centring through padding). For one channel and zero bias, $z_n=\sum_r w_r x_{n+r}$ is the circular convolution with the *reversed* kernel $h_r=w_{-r}$, so with the DFT convention of Eq. (10.49),

$$Z_k = A_k X_k, \qquad A_k=\sum_r w_r\,e^{+2\pi ikr/N} = W_{(-k)\bmod N} = \overline{W_k} \quad \text{for real } w, \tag{10.60}$$

where $W_k=\sum_r w_r e^{-2\pi ikr/N}$ is the response the kernel would have as a convolution. Cross-correlation and convolution have the same gain and opposite phase shifts; symmetric kernels such as $(1,2,1)$ do not notice, and a learned kernel simply learns the reversed weights –

the remark of Section 10.3.5, now in frequency. The reversal does matter when a prescribed kernel, an FFT implementation and a phase response are compared.

Proposition 10.17 (The linear stage mixes channels, frequency by frequency). *With circular boundaries, stride one and zero bias, the DFT of Eq. (10.59) is*

$$Z_a[k] = \sum_{b=1}^{C_{\text{in}}} A_{ab}[k]\, X_b[k], \qquad \textit{that is} \qquad \boldsymbol{Z}[k] = \boldsymbol{A}[k]\,\boldsymbol{X}[k], \quad \boldsymbol{A}[k] \in \mathbb{C}^{C_{\text{out}} \times C_{\text{in}}}, \tag{10.61}$$

one small matrix per frequency, with entries given by Eq. (10.60) for each kernel w_{ab}. A spatially constant bias adds $Nb_a\delta_{k0}$ to channel a. Different frequencies never mix in the linear stage; the same holds in two dimensions with $k \to (k_x, k_y)$.

Proof. Equation (10.59) is linear in x and a sum over b of circular cross-correlations, to each of which Eq. (10.60) applies; the DFT of the constant b_a is Nb_a at $k = 0$ and zero elsewhere.

Spatial filtering has become a frequency-dependent mixing of channels: a convolutional layer is a bank of $C_{\text{out}}C_{\text{in}}$ frequency responses, applied as a matrix at each k. This is Theorem 10.3 with channels added, and three further statements follow from it.

What a short kernel can and cannot do.

For one kernel with $2R+1$ taps the response

$$A(\Omega) = \sum_{r=-R}^{R} w[r]\, e^{i\Omega r} \tag{10.62}$$

is a trigonometric polynomial of degree R; a 3×3 kernel has degree one in each direction, which is why the surfaces of Figure 10.18 are so smooth. Its frequency bins cannot be set independently. A sharp cutoff such as the mask of Eq. (10.51) needs a long kernel, or a layer that acts in frequency space directly (Section 10.13.6).

Training determines the frequency responses.

Training solves $\min_\theta \frac{1}{M}\sum_j \ell(\text{CNN}_\theta(x^{(j)}), t^{(j)}) + \lambda\mathscr{R}(\theta)$ for the weights, and since

$$\frac{\partial A_\theta(\Omega)}{\partial w_\theta[r]} = e^{i\Omega r}, \tag{10.63}$$

every gradient step on one weight moves the response at *all* frequencies at once. A convolutional network computes no FFT during training; the spatial weights and their responses are two descriptions of the same kernel. What the responses converge to depends on the task and the data, not on any built-in preference for smoothing or sharpening: the edge detectors of Figure 10.13(d) are directional high-pass filters of the kind of Eq. (10.58), learned because oriented intensity changes separate the digits. Learning independent spectral weights instead, as some architectures do, changes the model class unless the short spatial support is enforced.

An FFT reproduces the linear stage.

For circular boundaries and centred weights at offsets $(-1, 0, 1)$:

```
import numpy as np
x = np.array([1., 2., 3., 4.])
w = np.array([1., 2., 0.])                          # weights at offsets -1, 0, +1
z_spatial = w[0] * np.roll(x, 1) + w[1] * x + w[2] * np.roll(x, -1)

h = np.zeros_like(x)
h[1], h[0], h[-1] = w                               # the reversed kernel, Eq. (10.corrfourier)
z_fft = np.fft.ifft(np.fft.fft(h) * np.fft.fft(x)).real
# both: [6., 5., 8., 11.]
```

This is PyTorch's one-dimensional convolution layer with weight $(1,2,0)$, unit padding and the circular padding mode; the zero-padded layer differs at the two endpoints. The FFT changes how the linear stage is computed and nothing about what it computes.

Where the frequency picture stops, 1: the activation.

A block also applies $y_n = \phi(z_n)$, and for the ReLU

$$\mathrm{ReLU}(\sin t) = \max(0, \sin t) = \tfrac{1}{2}\sin t + \tfrac{1}{2}|\sin t| = \frac{1}{\pi} + \frac{1}{2}\sin t - \frac{2}{\pi}\sum_{j\geq 1}\frac{\cos 2jt}{4j^2-1}, \tag{10.64}$$

the Fourier series of the rectified sine. A single input frequency comes out with a constant term and every even harmonic – amplitudes $1/\pi = 0.318$, 0.5, $2/(3\pi) = 0.212$, $2/(15\pi) = 0.042$, ..., which Figure 10.15(d) measures with a DFT. Non-linearities couple frequencies. A full convolutional network therefore has no single, input-independent response H; the linear stage of each layer has one, and several linear circular layers without biases or activations would compose by multiplying their matrices $\boldsymbol{A}[k]$ at each k, but the ReLU between them breaks the diagonalisation. Around a fixed activation pattern the network is locally linear, but the pattern depends on the input, so the local filter is not one global filter. This is the content of the notebox after Theorem 10.3: the architecture alternates a map diagonal in frequency with a map diagonal in space.

Where the frequency picture stops, 2: stride and pooling.

A stride s samples the filtered map on a coarser grid, $y_m = z_{sm}$, and a sampled signal can only represent frequencies up to its Nyquist limit, Eq. (10.50), which the stride divides by s. Frequencies above the new limit do not disappear; they *alias* into lower ones. For stride two the two distinct inputs

$$z_n^{(1)} = \cos(0.2\pi n), \qquad z_n^{(2)} = \cos(0.8\pi n), \qquad z_{2m}^{(1)} = \cos(0.4\pi m) = \cos(1.6\pi m) = z_{2m}^{(2)}, \tag{10.65}$$

become identical after sampling, Figure 10.15(e). The classical remedy is a low-pass filter before the decimation, with passband $|\Omega| < \pi/s$; a learned strided kernel satisfies no such condition by itself, and max pooling is a non-linearity *and* a subsampling at once. Average pooling smooths, by Eq. (10.47) in two dimensions, but is far from an ideal anti-aliasing filter. This is the frequency-domain reading of Proposition 10.8: a strided layer is equivariant only to shifts by multiples of s, and aliasing is what happens to the other shifts.

10.13.6 How Fourier filters are used

In signal processing.

Three uses are classical. *Denoising and band selection*: a mask or a smooth response H removes what lies outside the band of interest – the 25 Hz tone of Eq. (10.44), the stone texture of the photograph, the mains hum in a measurement. The choice of H is the model of what is signal and what is noise; the transform itself decides nothing. *Fast convolution*: a long kernel is applied with three FFTs in $\mathcal{O}(N \log N)$ instead of $\mathcal{O}(NK)$, by Proposition 10.16. *Reading a filter*: the response H of a kernel says what it does – low pass, high pass, oriented, and at which scale – more directly than its weights do; $(1,2,1)/4$ is $\cos^2(\Omega/2)$, the Sobel filter is Eq. (10.58), and a learned filter can be read the same way by transforming it.

In differential equations.

Section 9.5 solved the Poisson problem $-u''(x) = f(x)$ on $(0,1)$ with $u(0) = u(1) = 0$ by a neural network. The boundary conditions suggest the sine modes, which vanish at both ends:

$$f(x) = \sum_{n=1}^{\infty} f_n \sin(n\pi x), \quad f_n = 2\int_0^1 f(x)\sin(n\pi x)\,\mathrm{d}x, \quad u(x) = \sum_{n=1}^{\infty} \frac{f_n}{(n\pi)^2}\sin(n\pi x), \tag{10.66}$$

since $-\frac{\mathrm{d}^2}{\mathrm{d}x^2}\sin(n\pi x) = (n\pi)^2 \sin(n\pi x)$. The inverse differential operator is a *filter*: it multiplies mode n by $1/(n\pi)^2$, damping high spatial frequencies quadratically, and no zero mode occurs because n starts at one. The same solution has the Green-function representation

$$u(x) = \int_0^1 G(x,s)\, f(s)\,\mathrm{d}s, \qquad G(x,s) = \min(x,s)\,\big[1 - \max(x,s)\big], \tag{10.67}$$

and G depends on x and s separately, not on $x - s$ alone: it is *not* a convolution kernel. Fixed endpoints break translation invariance, and Theorem 10.7 then says there is no convolution to be had. The sine basis diagonalises the operator while respecting its boundaries; a periodic FFT without a boundary treatment solves a different problem. (One may use an odd periodic extension and a periodic convolution, but that is a construction, not the whole-line convolution.) This is the general lesson behind the padding table of Section 10.13.3: the boundary model decides which basis diagonalises the operator.

Inside convolutional networks.

Four ideas bring the frequency picture back into the architecture.

- *Anti-aliasing.* A blur before every stride-two pooling or convolution, so that a shifted input gives a shifted output rather than an aliased one, restores much of the shift invariance that strides destroy (Zhang [47]). It trades resolution for the suppression of aliases and does not improve accuracy on every task.
- *Spectral pooling.* Reduce the resolution of a feature map by truncating its DFT instead of taking maxima: for a periodic record of odd length N reduced to odd length $M < N$, keep the signed low-frequency bins $|k| \le (M-1)/2$ and set

$$Y_k = \frac{M}{N} X_k, \qquad y_m = \frac{1}{M} \sum_{|k| \le (M-1)/2} Y_k\, e^{2\pi i k m / M}, \tag{10.68}$$

so that the output has M samples over the same physical period and the factor M/N preserves amplitudes under the normalisation of Eq. (10.49). The low frequencies are kept exactly and the conjugate pairs must be retained for real data (Rippel, Snoek and Adams [48]); for images, crop in both directions. What is lost is the fine structure above the cut, which may be what the task needs.

- *Learning the filters in frequency space.* Some architectures learn matrices $\boldsymbol{R}_\theta[k]$ directly,

$$z = \mathrm{IDFT}\big[\boldsymbol{R}_\theta[k]\boldsymbol{X}[k]\big], \qquad y = \phi(z), \qquad \boldsymbol{R}_\theta[(-k) \bmod N] = \overline{\boldsymbol{R}_\theta[k]}, \tag{10.69}$$

one learned channel-mixing matrix per retained frequency, with the conjugate symmetry that keeps the output real and real weights at the DC and, for even N, Nyquist bins. By Eq. (10.62) read backwards, independent spectral weights describe a *global* periodic kernel with few parameters – the opposite of a 3×3 filter. This is the spectral layer of the Fourier neural operator [49], which learns the solution operator of a partial differential equation from examples; for prescribed non-periodic boundaries the sine bases and extensions of the previous paragraph are needed.

- *Fourier-weighted losses.* For a reconstruction task with prediction y_n and target t_n on the same N-point grid, let $e_n = y_n - t_n$ and $E_k = \mathrm{DFT}(e)_k$. Parseval's identity,

$$\frac{1}{N}\sum_{n=0}^{N-1} |e_n|^2 = \frac{1}{N^2}\sum_{k=0}^{N-1} |E_k|^2, \tag{10.70}$$

says the mean squared error is a sum over frequencies, and a preference enters through non-negative weights,

$$\mathscr{L}_q = \frac{1}{N^2}\sum_{k=0}^{N-1} q_k\,|E_k|^2, \qquad q_k \geq 0, \tag{10.71}$$

with $q_k = 1$ the ordinary mean squared error. Changing q_k changes which errors the training emphasises; E_k is complex, so the loss sees phase, whereas a magnitude-only spectral loss would miss a spatial shift. Zero weights make the loss blind to those modes.

The distinction in equations, and what to remember. The three maps of this section are

representation: $f \mapsto \hat{f}$, frequency filtering: $f \mapsto \mathscr{F}^{-1}[H\hat{f}]$, convolution filter: $f \mapsto h * f$, $H = \hat{h}$,

and the convolution theorem makes the last two the same operation under the stated assumptions. A Fourier transform is a change of representation and loses nothing; a filter is a choice of what to keep; zeroing coefficients is information-losing, scaling them need not be. For sampled data the transform convention and the boundary extension decide the normalisation and whether the convolution is linear or circular. A convolutional network learns its frequency responses one short kernel at a time, Eq. (10.62), with activations that create new frequencies and strides that alias them – which is why the frequency picture describes each linear stage exactly and the whole network not at all.

10.14 Summary and the programs

A convolutional layer is what remains of a dense layer after we insist that it respect the structure of the data. Locality makes the matrix banded, parameter sharing makes its diagonals constant, and by Theorem 10.2 a banded matrix with constant diagonals is exactly a con-

volution. The stronger statement is Theorem 10.7: convolution is the *only* linear map that commutes with translation, so the architecture is not a heuristic but the unique consequence of a symmetry assumption. The parameter count falls from $\mathcal{O}(L^4)$ per layer to $K(CF^2+1)$, independent of image size.

Backpropagation required no new ideas. The gradients (10.29)–(10.31) are the chain rule applied to Eq. (10.16), and all three are the adjoint of the forward map, which we verified as an inner-product identity to 10^{-14}. Max pooling routes the gradient to the argmax and is differentiable almost everywhere.

The experiment of Section 10.10 should be remembered together with the theory. On centred 8×8 digits the convolutional network was *not* better than a dense network with the same number of parameters, 0.9656 against 0.9670; on the same digits placed at random offsets it was better by seventeen points with 2.5 times fewer parameters. The inductive bias pays exactly when the assumption behind it is true, and not otherwise.

Three structural results were added along the way and each pays off later. Theorem 10.3 says every circulant matrix is diagonalised by the same Fourier basis, so a convolutional layer multiplies each frequency by a learned number; the alternation between a map diagonal in frequency and a non-linearity diagonal in space is where the expressive power lives. Proposition 10.11 shows that dilation buys an exponentially growing receptive field at linear parameter cost and no loss of resolution – five layers rather than thirty-one for $R=63$. And Proposition 10.13 identifies the backward pass as the transposed convolution, with the output size (10.32) that every generator in Part IV is designed around, and explains the checkerboard artefacts that follow from choosing F not a multiple of S.

The comparison against the libraries in Section 10.12.3 is the strongest verification in the chapter. Pushing one set of weights into our NumPy code, PyTorch and TensorFlow, the forward passes agree to 1.8×10^{-15} and every gradient to 4.4×10^{-16} – unit roundoff. The finite-difference check of Section 10.9.2 compared our gradient against our own forward pass and would have passed had both been wrong in the same way; this one compares against two independent implementations. Trained on the full MNIST set the same architecture reaches 0.9873 in PyTorch and 0.9916 in TensorFlow after five epochs, a difference the size of the epoch-to-epoch variation of either.

Two sections looked at filters as objects in their own right. Section 10.7 applied hand-made filters to pictures: the Sobel operators and their diagonal rotations are smoothed derivatives that respond to intensity changes in one orientation and to nothing else, a blur is their low-pass complement, and a large template with the normalised cross-correlation of Eq. (10.28) – the cosine between a patch and the template, Proposition 10.12 – detects a roof in a drawing and every window of one kind in a photograph. A network starts from random filters and, trained only against a classification cost, arrives at the same family of edge detectors; what the hand cannot supply is the second and third layer, which combine edges into corners and corners into objects, and what convolution cannot supply is invariance to scale and rotation, which comes from data. Section 10.13 described the same filters in the frequency domain. The convolution theorem, Theorem 10.14 on the line and Eq. (10.53) for sampled data, makes a convolution with h and a multiplication by $H=\hat{h}$ the same operation, so a filter is read off its gain: $\cos^2(\Omega/2)$ for the three-point average, $e^{-\sigma^2\omega^2/2}$ for the Gaussian, $4|\sin\Omega_x|(1+\cos\Omega_y)$ for the Sobel operator; a sharp cutoff in frequency is a wide, ringing kernel in space, and an FFT computes a long convolution in $\mathcal{O}(N\log N)$ once both sequences are zero-padded to $N\geq L_x+L_h-1$. A convolutional layer is, by Proposition 10.17, a channel-mixing matrix at every frequency, with responses that are trigonometric polynomials of the kernel size; the description is exact for the linear stage of each layer and stops at the ReLU, which creates harmonics, and at the stride, which aliases. Fourier filters are used for denoising, fast convolution and reading filters, for solving Poisson's equation in the sine basis that its boundaries

select, and inside networks as anti-aliasing, spectral pooling, spectral parameterisations of the kind the Fourier neural operator uses, and frequency-weighted losses.

Two practical warnings are worth carrying forward. A ReLU and max-pool network cannot be gradient-checked naively, because the function is piecewise linear and a central difference straddles the kinks; check on a smooth surrogate instead. And in a classical architecture the convolutional layers hold a tiny fraction of the parameters – 0.58%, measured – while the first dense layer holds 99.1%, so that is where the memory and the overfitting live. Replacing it by global average pooling cuts the parameter count by a factor of 167 and costs about five points of accuracy in five epochs, which is a real trade rather than a free lunch.

The programs are in the directory
`doc/BookML/BookPrograms/chapter10_convolutional_networks`.
Every listing above appears there as a numbered file, and two self-contained modules run start to finish and reproduce the numbers quoted in the text:

- `cnn.py` – im2col, the convolution and pooling layers with their gradients, the network of Eq. (10.35), He initialisation and Adam.
- `verify_cnn.py` – all four checks of Section 10.9.2: the naive-loop comparison, the adjoint identity, the finite-difference gradient checks including the ReLU failure and its smooth-surrogate resolution, the Toeplitz and doubly block Toeplitz reconstructions, and the equivariance tests.
- `run_digits.py` and `run_shift.py` – the two blocks of Table 10.1.
- `cross_check.py` – the three-way comparison of Section 10.12.3: one set of weights pushed into our code, PyTorch and TensorFlow, with the layout conversions written out, and the parameter counts of Section 10.12.4.
- `cnn_torch.py` and `cnn_tf.py` – the MNIST training of Section 10.12.5 in each framework, with the dense and the global-average-pooling heads selectable.
- `mnist_dense_vs_cnn.py` – the eight runs of Table 10.3: dense networks and convolutional networks on the full MNIST set in NumPy, PyTorch and TensorFlow, each framework in its own process, with the histories, predictions and first-layer filters saved for the figure.
- `mnist_data.py` – a loader that reads MNIST from a local `.npz`, so that the programs run on a machine with no outbound network access.

The figures are generated by two scripts in the `BookFigures` directory. The first is `ch10_figures.py`. The second is `ch10_filters_figures.py`, which makes the figures of Sections 10.7, 10.6, 10.12.6 and 10.13 from the chapter's own convolution code, the local MNIST file and the grey photograph `houses_photo.png` kept next to the figures, and prints every number quoted in those sections. None of the figures is drawn by hand.

10.15 Exercises

Warm-up exercises

1. **Convolution arithmetic.** Use Proposition 10.4 throughout. (a) An input of $32 \times 32 \times 3$ with $K = 32$ filters of extent $F = 3$, $S = 1$, $P = 0$: what is the output volume and how many parameters does the layer have? (b) Repeat with $P = 1$ and explain why this padding is the usual choice. (c) Find all (F, P) with $S = 1$ that preserve the spatial size, and show that F must be odd. (d) An input of 28×28 with $F = 5$, $S = 3$, $P = 1$: show that the floor in Eq. (10.17) is doing real work, and identify which input rows are never seen.

2. **Polynomial multiplication.** Multiply $p(t) = 2 - t + 3t^2$ by $s(t) = 1 + 4t - 2t^2 + 5t^3$ by hand, then reproduce the coefficients with `np.convolve` and with the Toeplitz matrix of Eq. (10.8). Verify that the matrix satisfies Definition 10.1, and confirm the commutativity $\boldsymbol{T}_\alpha \boldsymbol{\beta} = \boldsymbol{T}_\beta \boldsymbol{\alpha}$.
3. **Convolution or cross-correlation.** Take a 5×5 input and a non-symmetric 3×3 kernel. Compute both Eq. (10.14) and Eq. (10.15) and show that the results differ but are related by a 180° rotation of the kernel. Then argue why this makes no difference to a trained network, and construct a case where it does.
4. **Receptive field.** (a) Using Eq. (10.22), find the receptive field of a stack of five 3×3 convolutions with $S = 1$. (b) Insert a 2×2 pooling layer after every second convolution and recompute. (c) Show that two 3×3 layers have the same receptive field as one 5×5 layer but fewer parameters, and state the other advantage.
5. **Equivariance by hand.** Prove the forward direction of Theorem 10.7 for the two-dimensional case. Then show by explicit counterexample that a dense layer is not equivariant, and that max pooling with $F = S = 2$ is invariant to shifts of one pixel only when the argmax does not cross a window boundary.
6. **The adjoint.** Show that col2im as implemented is the adjoint of im2col, by verifying $\langle \mathscr{A}\boldsymbol{x}, \boldsymbol{y} \rangle = \langle \boldsymbol{x}, \mathscr{A}^* \boldsymbol{y} \rangle$ for random $\boldsymbol{x}, \boldsymbol{y}$ and several (F, S, P). Explain why `np.add.at` rather than plain assignment is essential when $S < F$.
7. **The convolution theorem.** (a) Prove Theorem 10.3 for $n = 4$ by writing out the 4×4 circulant matrix and the four Fourier modes explicitly. (b) Verify Eq. (10.11) numerically with `np.fft`, for a random kernel and input, and confirm that the error is at rounding level. (c) Time a direct convolution against the FFT route as a function of the kernel length F at fixed $n = 4096$, and find the crossover. (d) A convolutional layer multiplies each Fourier mode by one number. What does that say about the function a *single* convolutional layer with no activation can represent, however many filters it has?
8. **Dilation.** (a) Derive Eq. (10.25) from Eq. (10.22). (b) How many layers of 3×3 filters are needed for a receptive field of 255 with $D_\ell = 2^{\ell - 1}$, and how many without dilation? (c) Dilated convolutions can miss pixels – the "gridding" artefact. For two stacked layers with $F = 3$ and $D = 2$ in both, find an input pixel that no output depends on, and suggest a schedule of rates that avoids it.
9. **Transposed convolution.** (a) Verify Eq. (10.32) for $F = 4$, $S = 2$, $P = 1$ and $H_2 = 7$, and check the answer against `nn.ConvTranspose2d`. (b) Build the matrix $\boldsymbol{W}'$ of Eq. (10.19) for a small example and confirm that $\boldsymbol{W}'^\top$ is what the library computes. (c) Show that with $F = 3$, $S = 2$ the output pixels are produced from either one or two input pixels depending on parity, and that $F = 4$, $S = 2$ makes the count uniform. This is the checkerboard artefact.
10. **Reproducing the three-way check.** Take the small network of Section 10.12.3 and reconstruct the weight conversions yourself: our (K, C, F, F) into Keras's (F, F, C, K), and the flatten permutation between (K, H, W) and (H, W, K) ordering. Then deliberately omit the flatten permutation and report what happens to the forward pass, to the gradients, and to the training curve. Which of the three would have caught the mistake?
11. **Hand-made filters.** Draw a 16×16 picture of your own – a letter, a simple house – and apply the five filters of Eqs. (10.26)–(10.27) with $P = 1$ through `conv_forward`. (a) Explain every map in terms of the derivative each filter estimates. (b) Show that the four edge filters give exactly zero on any constant region and that the blur preserves constants, and relate both facts to the sums of the weights. (c) Apply a ReLU to the vertical-edge map and explain what has been lost; then apply the negated filter and show that the two ReLU maps together carry the original.
12. **Template matching.** Implement Eq. (10.28) with three calls to `correlate2d` from `scipy.signal` and reproduce the last panel of Figure 10.9. (a) Verify Proposition 10.12 numerically: multiply the image by 3, add 0.4, and show the map is unchanged. (b) Draw the house at twice the size and apply the same roof template; report the maximum of the map and explain

why it falls. (c) Rotate the house by 20° and repeat. What does convolution handle for free, and what does it not?

13. **Reading a learned filter.** Take the thirty-two first-layer filters of the trained PyTorch network (saved by `mnist_dense_vs_cnn.py`). For each, compute the two-dimensional response $|\hat{w}(\Omega_x, \Omega_y)|$ on a 64×64 grid by zero-padding and `np.fft.fft2`, locate its maximum, and classify the filter as low pass, horizontal edge, vertical edge or oblique. How many of each are there, and how many come in opposed-sign pairs?
14. **Two sequences through two filters.** Send the alternating sequence $x_n = (-1)^n$ and the constant sequence $x_n = 1$ through the two three-tap filters $(1,2,1)/4$ and $(-1,2,-1)/4$, by hand and with `np.convolve`. Explain the four outputs with Eq. (10.47) and its complement, and show that the two gains add to one at every Ω. Then explain, from Eq. (10.58), why a horizontal edge is invisible to W_{vert}.
15. **Gaussian smoothing.** (a) Derive $H_\sigma(\omega)$ in Eq. (10.43) by completing the square. (b) Sample Eq. (10.44) at $N = 4096$ points in one second, convolve with the sampled Gaussian including the factor Δt of Eq. (10.55), and compare the amplitudes of the two tones in the output with 0.9314 and 0.002517. (c) Repeat with the Gaussian truncated to $|t| \le 2\sigma$ and explain the discrepancy.
16. **Circular against linear convolution.** (a) Prove Proposition 10.16 for $L_x = 3$, $L_h = 2$ by writing out the circular sums for $N = 3$ and $N = 4$ and locating the wrapped term. (b) For random x of length 1000 and h of length K, time `np.convolve` against the FFT route of the listing in Section 10.13.3 for $K = 3, 10, 30, 100, 300$ and find the crossover. (c) Apply a 3×3 filter to an image once with zero padding and once circularly through `np.fft.fft2`; where do the two differ, and by how much?
17. **A convolutional layer by FFT.** Build a PyTorch `Conv1d` layer with one input and one output channel, $F = 3$, unit padding and the circular padding mode, give it the weight $(1,2,0)$ and no bias, apply it to $x = (1,2,3,4)$, and reproduce the output with the FFT listing of Section 10.13.5. Then switch to zero padding and explain which outputs change and why. Finally add a second input channel and a second output channel and verify Proposition 10.17 numerically: the DFT of the output is the 2×2 matrix $\boldsymbol{A}[k]$ applied to the DFT of the input, at every k.
18. **The ReLU creates frequencies.** (a) Derive the Fourier series in Eq. (10.64) from the series of $|\sin t|$. (b) Sample $\sin t$ at 4096 points, apply the ReLU, and compare the DFT amplitudes with $1/\pi$, $1/2$ and $2/(\pi(4j^2-1))$. (c) Replace the ReLU by $\tanh$ and by the identity; which harmonics appear in each case, and why does the identity produce none?
19. **Aliasing.** (a) Verify Eq. (10.65) numerically and find all Ω in $[0, \pi]$ that alias to 0.4π under stride two. (b) Apply a stride-two max pooling to $\cos(0.2\pi n)$ and to $\cos(0.8\pi n)$ and compare. (c) Insert the three-point average of Eq. (10.46) before the stride and quantify, with Eq. (10.47), how much of the 0.8π component survives to alias.
20. **Spectral pooling.** Implement Eq. (10.68) for $N = 63$ and $M = 31$ on a signal with components at $k = 2$ and $k = 20$. Show that the $k = 2$ component comes out with the same amplitude, that the $k = 20$ component is gone, and that the output is real. Compare with max pooling and with average pooling of the same signal at stride two.
21. **Poisson's equation as a filter.** For $f(x) = (3x + x^2)e^x$, the right-hand side of Section 9.5, compute the sine coefficients f_n of Eq. (10.66) numerically for $n \le 64$, form the series for u, and compare with the exact solution $x(1-x)e^x$. (a) How many modes are needed for an error below 10^{-6}, and why so few? (b) Show numerically that $G(x,s)$ of Eq. (10.67) is not a function of $x - s$. (c) Solve the same problem with a periodic FFT on $[0,1)$ without any boundary treatment, and explain what equation you have actually solved.

Project-style exercise: a convolutional network from scratch

Part a: the layers.

Implement `im2col`, `col2im`, the convolution forward and backward passes and max pooling. Verify each against the explicit loop form of Eq. (10.16) for at least four combinations of stride and padding, and verify the adjoint identity.

Part b: gradients.

Gradient-check the complete network against central differences. You should find that the check fails for at least one parameter array; diagnose it, and show that replacing ReLU and max pooling by smooth surrogates repairs it. State what this implies about testing any piecewise-linear model.

Part c: the structural claims.

Reconstruct the matrix $\boldsymbol{W}'$ of Eq. (10.19) column by column from your own `conv_forward`. Confirm the number of non-zero entries and the number of distinct values, and display its doubly block Toeplitz structure. Measure the equivariance error for $S = 1$ and $S = 2$ over a range of shifts and compare with Proposition 10.8.

Part d: the experiment.

Reproduce Table 10.1: the centred and the translated digits, CNN and dense, matched parameter counts, at least five seeds. Report the spread honestly and say whether the difference in the upper block is significant.

Part e: what breaks it.

Equivariance is to *translation*. Rotate or rescale the digits instead and repeat the comparison. Does the convolutional network retain its advantage? Explain your answer with reference to Theorem 10.7, and describe what would have to change in the architecture to obtain equivariance to rotation.

Part f: against a library.

Rebuild the same architecture in PyTorch or TensorFlow and compare accuracy, runtime and memory with your NumPy version on the same data. Then verify that the library agrees with your implementation numerically: feed both the same weights and the same input and compare the forward and backward passes to machine precision. Any discrepancy is a bug in one of them, and finding out which is the point of the exercise.

Part g: scaling.

Using Eqs. (10.1) and (10.18), tabulate the parameter count of a dense and a convolutional first layer for images of side 32, 128, 512 and 2048. Then measure how the runtime of your own `conv_forward` scales with image side and with filter size, and compare against the operation counts $\mathcal{O}(NKCF^2H_2W_2)$. Where does im2col memory become the binding constraint?

Chapter 11
Recurrent Neural Networks

Chapter 10 restricted the dense layer of Chapter 8 by insisting that it respect the spatial structure of an image, and obtained convolution. This chapter makes the corresponding restriction for data with a *temporal* structure, and obtains recurrence.

The assumption being given up is a familiar one. Almost everything so far has supposed that the training examples are independent and identically distributed, and the bias-variance and resampling arguments of Chapter 2 lean on it heavily. For a sequence this is simply false: the value of a detector trace at one sample is strongly informative about the next, that is the entire content of the data, and a model that treats the samples as independent has thrown the signal away before it starts.

Sequences also break a structural assumption. A feed-forward network has a fixed input dimension; a convolutional network has a fixed input size too, up to the pooling that precedes its dense layers. Sequences come in different lengths, and we would like one model to handle all of them. A recurrent network does this by processing one element at a time and carrying a state forward, so that the same parameters serve a sequence of any length.

The material follows the lecture notes for weeks five to seven of FYS-STK3155/4155. We derive the architecture, derive backpropagation through time, prove the theorem that explains why long sequences are hard, measure the effect, and then examine the gated architectures that were invented to defeat it. As in Chapter 10, the from-scratch implementation is checked against PyTorch and TensorFlow with identical weights, so that the recursion of Eqs. (11.16)–(11.18) can be compared with what automatic differentiation actually produces; and the chapter closes with the experiment that the theory demands but the usual account omits – a task that a simple recurrent network provably cannot learn and a gated one can.

11.1 Sequences, and what we want from them

Write $x_{1:T} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T)$ for a sequence of inputs. Three tasks cover most of what is asked of a sequence model:

$$p(x_{1:T}), \qquad p(y_{1:T} \mid x_{1:T}), \qquad p(y \mid x_{1:T}), \tag{11.1}$$

that is, modelling the sequence itself, mapping a sequence to a sequence (filtering, forecasting), and mapping a sequence to a single label (classification, regime detection).

The first of these can always be factorised exactly, by the chain rule of probability,

$$p(x_{1:T}) = \prod_{t=1}^{T} p(\boldsymbol{x}_t \mid x_{1:t-1}) \,. \tag{11.2}$$

Equation (11.2) is exact and useless as it stands, because the conditioning set grows without bound. What makes it tractable is the assumption that everything relevant in $x_{1:t-1}$ can be summarised in a finite-dimensional *state* $\boldsymbol{h}_t$, updated recursively:

$$\boxed{\boldsymbol{h}_t = f_\theta(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t), \qquad \hat{\boldsymbol{y}}_t = g_\theta(\boldsymbol{h}_t).} \tag{11.3}$$

This is a latent state-space model, and it should look familiar from physics: Eq. (11.3) is exactly the structure of a Markovian dynamical system, with f_θ the update map and $\boldsymbol{h}_t$ the state. The Kalman filter is the case in which f_θ is linear and the noise Gaussian. A recurrent neural network is the case in which f_θ and g_θ are neural networks, so that the update map is learned rather than derived.

Ordered, correlated data of this kind is the norm rather than the exception in the physical sciences: detector readout streams and waveform traces, climate and seismic records, turbulence probes, molecular dynamics trajectories, spectroscopic scans, and state estimation in control and feedback. Sequential does not have to mean temporal – text and genomic sequences are ordered without carrying a time stamp – but our examples will be time series.

11.2 The architecture

The simplest recurrent network, usually called the *vanilla* or *simple* RNN, realises Eq. (11.3) with one affine map and one activation. At each step t,

$$\begin{aligned} \boldsymbol{a}^{(t)} &= \boldsymbol{U}\boldsymbol{x}^{(t)} + \boldsymbol{W}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}, \\ \boldsymbol{h}^{(t)} &= \sigma_h\left(\boldsymbol{a}^{(t)}\right), \\ \boldsymbol{o}^{(t)} &= \boldsymbol{V}\boldsymbol{h}^{(t)} + \boldsymbol{c}, \\ \hat{\boldsymbol{y}}^{(t)} &= \sigma_y\left(\boldsymbol{o}^{(t)}\right), \end{aligned} \tag{11.4}$$

with $\boldsymbol{U} \in \mathbb{R}^{n_h \times n_x}$ mapping the input into the state, $\boldsymbol{W} \in \mathbb{R}^{n_h \times n_h}$ mapping the state to itself, $\boldsymbol{V} \in \mathbb{R}^{n_y \times n_h}$ reading the state out, and $\boldsymbol{b}$, $\boldsymbol{c}$ the biases. The hidden activation σ_h is almost always $\tanh$, for reasons that Section 11.5 will make precise, and σ_y is the identity for regression or a softmax for classification. The recursion is started from $\boldsymbol{h}^{(0)} = \boldsymbol{0}$ unless there is reason to do otherwise.

There are two equivalent ways of drawing Eq. (11.4), and Figure 11.1 shows both. On the left is the *folded* form: a single layer with an arrow leaving it and returning to itself, which is the literal content of the recursion and the reason for the name. On the right is the *unrolled* form, in which the same layer is drawn once per time step and the self-arrow becomes an arrow from one copy to the next. The two pictures describe the same computation. The folded one is how the network is *stored* – one copy of $\boldsymbol{U}, \boldsymbol{W}, \boldsymbol{V}$ – and the unrolled one is how it is *executed*, and Section 11.4 will show that the unrolled picture is also how it is differentiated.

Three properties of Eq. (11.4) are worth stating explicitly, because each is a deliberate design decision and each has a consequence.

The parameters do not depend on t. The same $\boldsymbol{U}, \boldsymbol{W}, \boldsymbol{V}$ act at every step. This is parameter sharing along the time axis, exactly analogous to the parameter sharing along the spatial axes that made a convolutional layer in Chapter 10, and it has the same two consequences: the parameter count is independent of the sequence length, and the model is equivariant to translation in time.

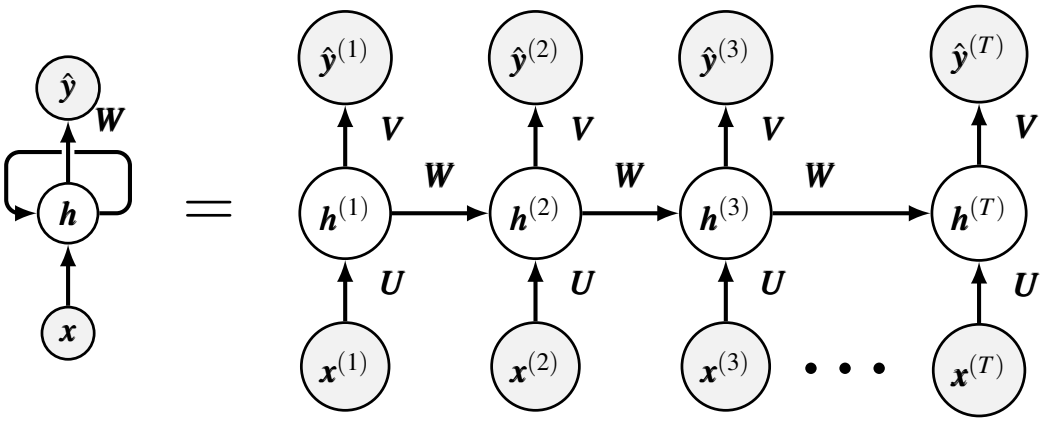


Fig. 11.1 The recurrent network of Eq. (11.4), folded on the left and unrolled on the right. The input matrix $\boldsymbol{U}$, the recurrent matrix $\boldsymbol{W}$ and the output matrix $\boldsymbol{V}$ are the *same* arrays at every step; the unrolled diagram repeats them but does not duplicate them. Unrolling turns the network into a feed-forward graph of depth T with tied weights, which is what makes backpropagation applicable and what makes Theorem 11.1 unavoidable.

The state is a bottleneck. Everything the network will ever know about $x_{1:t}$ when it computes $\hat{\boldsymbol{y}}^{(t+1)}$ must fit in the n_h numbers of $\boldsymbol{h}^{(t)}$. This is what makes the model tractable and also what limits it.

The cost accumulates over the sequence. For a regression task the cost is

$$\mathscr{L} = \frac{1}{T}\sum_{t=1}^{T} L^{(t)}, \qquad L^{(t)} = \frac{1}{2}\left\|\hat{\boldsymbol{y}}^{(t)} - \boldsymbol{y}^{(t)}\right\|^2, \tag{11.5}$$

and for classification $L^{(t)}$ is the cross-entropy of Chapter 5. In the sequence-to-label case only $L^{(T)}$ is non-zero, and Section 11.5 will show that this is the hardest case of all.

Parameter count. A simple RNN has $n_h(n_x + n_h + 1) + n_y(n_h + 1)$ parameters, and the sequence length T appears nowhere. A feed-forward network given the same sequence flattened into a vector of length Tn_x would need $\mathcal{O}(Tn_x n_h)$ weights in its first layer alone, and would have to be retrained for every new T. This is the same argument as Eq. (10.1), transposed from space to time.

11.2.1 The RNN as a discrete-time dynamical system

Set the input to zero, or hold it constant. Equation (11.4) becomes an autonomous map

$$\boldsymbol{h}^{(t)} = F\left(\boldsymbol{h}^{(t-1)}\right) := \sigma_h\left(\boldsymbol{W}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}\right), \tag{11.6}$$

and the entire vocabulary of dynamical systems applies. Fixed points satisfy $\boldsymbol{h}^\star = F(\boldsymbol{h}^\star)$; their stability is decided by the eigenvalues of the Jacobian

$$\boldsymbol{J} = \left.\frac{\partial F}{\partial \boldsymbol{h}}\right|_{\boldsymbol{h}^\star} = \operatorname{diag}\left(\sigma_h'(\boldsymbol{a}^\star)\right)\boldsymbol{W}, \tag{11.7}$$

with the fixed point attracting if the spectral radius $\rho(\boldsymbol{J}) < 1$ and repelling if $\rho(\boldsymbol{J}) > 1$. In more than one dimension the map can also possess limit cycles, and in three or more it can be chaotic.

This is not an idle observation. It tells us what a recurrent network is capable of representing – oscillations, decay to a point attractor, chaotic wandering – and it tells us what the network will do with information. If the dynamics are strongly contracting the network forgets its initial condition quickly and cannot carry information far; if they are expanding it is unstable. Equation (11.7) is also, as Section 11.5 will show, exactly the quantity that controls the gradient. The stability of the forward dynamics and the survival of the backward gradient are the same question.

11.3 Where the recurrence comes from: a differential equation

The recurrence of Eq. (11.4) can look arbitrary. It is not, and the quickest way to see why is to derive it from a differential equation, which also connects this chapter to Chapter 9.

Take Newton's equation for a damped harmonic oscillator, scaled so that time is dimensionless,

$$\frac{\mathrm{d}^2 x}{\mathrm{d}t^2} + \eta \frac{\mathrm{d}x}{\mathrm{d}t} + x(t) = F(t), \tag{11.8}$$

with η the damping and F an external force. As always we reduce it to two first-order equations by introducing the velocity,

$$v = \frac{\mathrm{d}x}{\mathrm{d}t}, \qquad \frac{\mathrm{d}v}{\mathrm{d}t} = F(t) - \eta v(t) - x(t). \tag{11.9}$$

Now discretise the velocity equation by the forward Euler method of Section 9.4:

$$v_{i+1} = v_i + \Delta t \left(F_i - \eta v_i - x_i \right) =: h_i(x_i, v_i, F_i) \,. \tag{11.10}$$

The right-hand side is a function of the current state and the current input. But v_i was itself produced by the same rule one step earlier,

$$v_i = v_{i-1} + \Delta t \left(F_{i-1} - \eta v_{i-1} - x_{i-1} \right) = h_{i-1}, \tag{11.11}$$

so substituting gives

$$v_{i+1} = h_i(x_i, h_{i-1}, F_i) \,. \tag{11.12}$$

Absorbing the forcing into the input and renaming the output, this is

$$\boldsymbol{h}_i = h(\boldsymbol{x}_i, \boldsymbol{h}_{i-1}) \,, \tag{11.13}$$

which is Eq. (11.3) exactly. A recurrent network is a learned, non-linear time-stepper. The hidden state plays the role of the integrator's internal state, $\boldsymbol{W}$ plays the role of $1 - \eta \Delta t$, and the training data supply what the differential equation would otherwise have to tell us.

The comparison with Chapter 9. Two chapters have now solved differential equations with networks, and they do it in opposite ways. In Chapter 9 the equation was known and supplied the cost through its residual; no data were needed and the network learned a function of continuous time. Here the equation is *unknown* and the data supply the cost; the network learns a discrete update rule. The first is appropriate when the physics is

known and the data are absent, the second when the data are present and the physics is not.

11.4 Backpropagation through time

Training uses gradient descent, so we need $\nabla \mathscr{L}$. The standard way to see how is to *unroll* the recursion: a recurrent network run for T steps is a feed-forward network of depth T in which every layer shares the same weights. Backpropagation applies unchanged; the only novelty is that gradients from every step must be summed into the same parameter arrays.

The algorithm has a pleasing description in the time domain. The forward pass builds a stack of the activities at each step; the backward pass peels the activities off the stack, computing error derivatives at each step; and afterwards the derivatives at all steps are added together for each weight.

The output layer.

Differentiating Eq. (11.5) gives, for each t,

$$\left(\nabla_{\boldsymbol{o}^{(t)}} \mathscr{L}\right)_i = \frac{\partial \mathscr{L}}{\partial L^{(t)}} \frac{\partial L^{(t)}}{\partial o_i^{(t)}}, \tag{11.14}$$

which for the quadratic cost and a linear σ_y is simply $(\hat{\boldsymbol{y}}^{(t)} - \boldsymbol{y}^{(t)})/T$.

The last hidden state.

At $t = T$ the state influences the cost only through the output at that step,

$$\nabla_{\boldsymbol{h}^{(T)}} \mathscr{L} = \boldsymbol{V}^{\mathsf{T}} \nabla_{\boldsymbol{o}^{(T)}} \mathscr{L}. \tag{11.15}$$

Earlier hidden states.

For $t < T$ the state has *two* routes to the cost: through the output at step t, and through the state at step $t+1$. The chain rule therefore gives two terms,

$$\boxed{\nabla_{\boldsymbol{h}^{(t)}} \mathscr{L} = \left(\frac{\partial \boldsymbol{h}^{(t+1)}}{\partial \boldsymbol{h}^{(t)}}\right)^{\mathsf{T}} \nabla_{\boldsymbol{h}^{(t+1)}} \mathscr{L} + \left(\frac{\partial \boldsymbol{o}^{(t)}}{\partial \boldsymbol{h}^{(t)}}\right)^{\mathsf{T}} \nabla_{\boldsymbol{o}^{(t)}} \mathscr{L}} \tag{11.16}$$

which, using Eq. (11.4), is

$$\nabla_{\boldsymbol{h}^{(t)}} \mathscr{L} = \boldsymbol{W}^{\mathsf{T}} \operatorname{diag}\left(\sigma_h'\left(\boldsymbol{a}^{(t+1)}\right)\right) \nabla_{\boldsymbol{h}^{(t+1)}} \mathscr{L} + \boldsymbol{V}^{\mathsf{T}} \nabla_{\boldsymbol{o}^{(t)}} \mathscr{L}. \tag{11.17}$$

Equations (11.15) and (11.17) are run backwards from $t = T$ to $t = 1$, and they are the whole of BPTT.

The parameters.

Because the same matrices act at every step, each accumulates a contribution from every step:

$$\begin{aligned}
\nabla_{\boldsymbol{c}}\mathscr{L} &= \sum_t \nabla_{\boldsymbol{o}^{(t)}}\mathscr{L},\\
\nabla_{\boldsymbol{V}}\mathscr{L} &= \sum_t \left(\nabla_{\boldsymbol{o}^{(t)}}\mathscr{L}\right)\boldsymbol{h}^{(t)\mathsf{T}},\\
\nabla_{\boldsymbol{b}}\mathscr{L} &= \sum_t \boldsymbol{\delta}^{(t)},\\
\nabla_{\boldsymbol{W}}\mathscr{L} &= \sum_t \boldsymbol{\delta}^{(t)}\boldsymbol{h}^{(t-1)\mathsf{T}},\\
\nabla_{\boldsymbol{U}}\mathscr{L} &= \sum_t \boldsymbol{\delta}^{(t)}\boldsymbol{x}^{(t)\mathsf{T}},
\end{aligned} \tag{11.18}$$

where $\boldsymbol{\delta}^{(t)} = \operatorname{diag}(\sigma_h'(\boldsymbol{a}^{(t)}))\nabla_{\boldsymbol{h}^{(t)}}\mathscr{L}$ is the error at the pre-activation of step t.

The backward pass is linear. There is an asymmetry between the two passes that is easy to miss and important to notice. The forward pass is non-linear: the squashing function σ_h keeps the activities bounded, and this is what prevents them from exploding. The backward pass, by contrast, is *completely linear* in the incoming gradient – Eq. (11.16) is a matrix acting on $\nabla_{\boldsymbol{h}^{(t+1)}}\mathscr{L}$, with the matrix fixed by the forward pass. Double the error at the final step and every error derivative doubles. Nothing bounds the backward pass, which is precisely why it can explode when the forward pass cannot.

11.5 Why long sequences are hard

Iterating Eq. (11.17) from T back to t produces a *product* of Jacobians, one factor per step. It is this product, and not anything about the architecture as such, that makes recurrent networks hard to train.

Theorem 11.1 (Vanishing and exploding gradients). *Let $\boldsymbol{h}^{(t)} = \sigma_h(\boldsymbol{U}\boldsymbol{x}^{(t)} + \boldsymbol{W}\boldsymbol{h}^{(t-1)} + \boldsymbol{b})$ and suppose $|\sigma_h'(z)| \le \mu$ for all z. Then for any $t < T$*

$$\frac{\partial \boldsymbol{h}^{(T)}}{\partial \boldsymbol{h}^{(t)}} = \prod_{k=t+1}^{T} \operatorname{diag}\left(\sigma_h'\left(\boldsymbol{a}^{(k)}\right)\right)\boldsymbol{W}, \tag{11.19}$$

and its spectral norm obeys

$$\left\|\frac{\partial \boldsymbol{h}^{(T)}}{\partial \boldsymbol{h}^{(t)}}\right\|_2 \le \left(\mu\,\sigma_{\max}(\boldsymbol{W})\right)^{T-t}, \tag{11.20}$$

where $\sigma_{\max}(\boldsymbol{W})$ is the largest singular value. Consequently, if $\mu\,\sigma_{\max}(\boldsymbol{W}) < 1$ the gradient contribution from step t decays exponentially *in the lag $T-t$. If $\mu\,\sigma_{\max}(\boldsymbol{W}) > 1$ it may grow exponentially.*

Proof. Equation (11.19) is the chain rule applied to the recursion, one factor for each step, with $\partial \boldsymbol{h}^{(k)}/\partial \boldsymbol{h}^{(k-1)} = \operatorname{diag}(\sigma_h'(\boldsymbol{a}^{(k)}))\boldsymbol{W}$ read off from Eq. (11.4). For the bound, the spectral norm is submultiplicative, so the norm of the product is at most the product of the norms. Each factor satisfies $\|\operatorname{diag}(\sigma_h'(\boldsymbol{a}^{(k)}))\boldsymbol{W}\|_2 \le \|\operatorname{diag}(\sigma_h'(\boldsymbol{a}^{(k)}))\|_2\|\boldsymbol{W}\|_2 \le \mu\,\sigma_{\max}(\boldsymbol{W})$, since the spectral norm

of a diagonal matrix is the largest absolute entry and that of $\boldsymbol{W}$ is $\sigma_{\max}(\boldsymbol{W})$. Multiplying $T-t$ such bounds gives Eq. (11.20).

For tanh we have $\mu = 1$, attained only at the origin, and for the logistic function $\mu = 1/4$, which is why tanh is preferred: the logistic activation guarantees decay by a factor of at least four per step regardless of the weights. For ReLU, $\mu = 1$ as well, but the derivative is 0 or 1 rather than a number smoothly approaching 1, and a ReLU recurrent network is prone to explosion instead; this is why the rectified family, dominant everywhere else in this book, is largely absent from recurrent architectures.

Corollary 11.2 (Memory horizon). *If $\mu\,\sigma_{\max}(\boldsymbol{W}) < 1$, define*

$$\tau = \frac{-1}{\log(\mu\,\sigma_{\max}(\boldsymbol{W}))} > 0. \tag{11.21}$$

Then Eq. (11.20) reads $\|\partial \boldsymbol{h}^{(T)}/\partial \boldsymbol{h}^{(t)}\|_2 \le e^{-(T-t)/\tau}$: the influence of step t on step T decays with a characteristic scale of τ steps, and falls below a tolerance ε once $T - t > \tau \log(1/\varepsilon)$.

Proof. Write $(\mu\sigma_{\max})^{T-t} = e^{(T-t)\log(\mu\sigma_{\max})}$ and substitute Eq. (11.21).

The number τ deserves a name and a measurement, because it converts Theorem 11.1 from a qualitative warning into a design parameter: it is the length of sequence the network can actually use. We measure it in Section 11.5.1, and the answer is smaller than one expects.

The bound is sufficient, not necessary. Equation (11.20) uses $\sigma_{\max}(\boldsymbol{W})$, not the spectral radius $\rho(\boldsymbol{W})$, and the two can differ substantially for non-normal $\boldsymbol{W}$ – in the experiment below, $\rho = 0.5$ corresponds to $\sigma_{\max} = 0.85$ and $\rho = 1.5$ to $\sigma_{\max} = 2.56$. Since $\rho(\boldsymbol{W}) \le \sigma_{\max}(\boldsymbol{W})$ always, $\mu\sigma_{\max} < 1$ is a sufficient condition for the gradient to vanish, while $\rho > 1/\mu$ is necessary for sustained growth. Between the two lies a regime in which transient growth is possible even though the asymptotic behaviour is decay, and this is where carefully initialised recurrent networks actually operate.

11.5.1 Measuring it

Theorem 11.1 is a bound; it is worth seeing the real thing. We take $n_h = 20$, sequences of length $T = 60$, and inputs small enough that the network stays near the origin where $\tanh' \approx 1$, so that the product is not confounded by saturation. We then form Eq. (11.19) explicitly and measure its spectral norm as a function of the lag, scaling $\boldsymbol{W}$ so that its spectral radius takes prescribed values.

```
 rho(W)  sigma_max  mean max(tanh')   ||dh_T/dh_1||   bound sigma_max^59
  0.50     0.853        1.0000        1.683e-18        8.319e-05
  0.90     1.535        1.0000        3.487e-03        9.575e+10
  1.10     1.876        0.9999        3.751e+02        1.327e+16
  1.50     2.558        0.9882        2.998e+03        1.175e+24
```

At $\rho(\boldsymbol{W}) = 0.5$ the influence of the first step on the last has fallen by *eighteen orders of magnitude* over fifty-nine steps. This is not a gradient that a floating-point optimiser can use; it is zero. At $\rho(\boldsymbol{W}) = 1.1$ the same quantity has grown to 375, and at 1.5 to 3000, so that a single unlucky mini-batch can throw the parameters anywhere. The last column shows the bound (11.20) is satisfied and is very loose in the growing regime, as one would expect from a worst-case argument applied to a product of matrices whose singular vectors are not aligned.

Figure 11.2(a) plots the whole decay. The straight lines on the logarithmic axis are the exponential of Theorem 11.1, and the sign of the slope is decided by whether $\rho(\boldsymbol{W})$ sits below or above one.

Because those lines are straight, Corollary 11.2 can be turned around: instead of bounding τ from $\sigma_{\max}(\boldsymbol{W})$, fit it to the measured decay. Doing so at every lag from 1 to 59 gives

```
=== 6. the memory horizon: fitted decay rate, Corollary 11.horizon ===
   rho(W)  sigma_max   fitted tau (steps)   lag for a factor 1e-6   norm at lag 59
     0.50     0.9129                 1.45                      20        1.967e-17
     0.70     1.2780                 2.82                      39        8.229e-09
     0.90     1.6431                 9.66                     133        2.264e-02
     1.10     2.0083               -10.30                    -142        3.116e+03
```

and this is the number to remember. At $\rho(\boldsymbol{W}) = 0.5$ the memory horizon is *one and a half steps*. Not fifty, not ten: a network initialised this way has forgotten its input after two steps, and no amount of training on long sequences will change that until the weights themselves grow. At $\rho = 0.9$ the horizon reaches ten steps, which is still short against the $T = 60$ of the experiment, and by $\rho = 1.1$ the fitted τ is negative – the product grows rather than decays, and the network is in the exploding regime. The useful range is narrow and it sits just below one.

Truncated backpropagation through time. Corollary 11.2 also licenses the standard economy. Running BPTT over a sequence of length $T = 10^4$ requires storing 10^4 activations; *truncated* BPTT propagates the gradient back only k steps and discards the rest. The error this introduces is exactly the discarded tail of Eq. (11.19), bounded by

$$\left\| \nabla_{\boldsymbol{\theta}} \mathscr{L} - \nabla^{(k)}_{\boldsymbol{\theta}} \mathscr{L} \right\| \le C \sum_{j>k} e^{-j/\tau} = \frac{C\, e^{-k/\tau}}{e^{1/\tau} - 1}, \tag{11.22}$$

so the bias falls exponentially in the truncation length with the same constant τ. Choosing k a few multiples of τ makes the truncation error negligible *against the gradient that survives* – but note what this says: when τ is small enough for truncation to be safe, the network has no long memory to lose, and when it has long memory, truncation destroys it. Truncated BPTT is cheap precisely when the recurrence is not doing the thing it was introduced for.

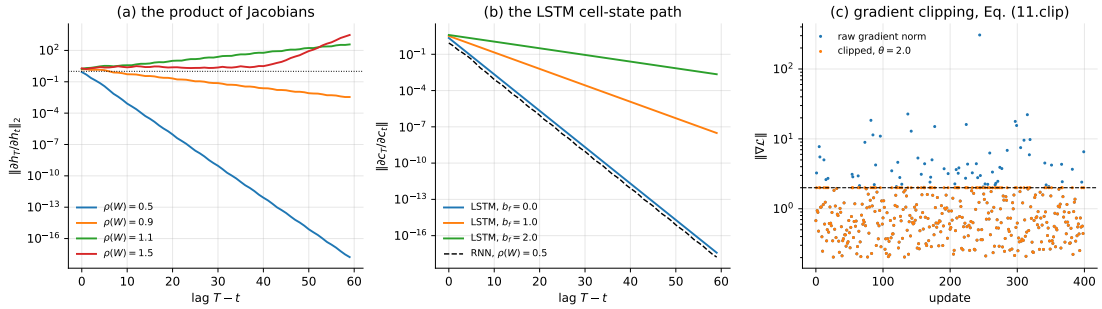


Fig. 11.2 (a) The spectral norm of the Jacobian product (11.19) against the lag $T-t$, for four values of the spectral radius of $\boldsymbol{W}$. The exponential behaviour of Theorem 11.1 is the straight line on a logarithmic axis. (b) The same quantity for the cell state of an LSTM, Eq. (11.29), at three values of the forget-gate bias, with the $\rho = 0.5$ curve of panel (a) repeated as a dashed line. At $b_f = 0$ the LSTM is no better than the vanilla RNN; the advantage comes entirely from biasing the gate open. (c) Gradient clipping, Eq. (11.23): the rare large norms that would otherwise destroy the parameters are rescaled, the direction is kept and everything below the threshold is untouched.

11.5.2 Gradient clipping

Vanishing and exploding gradients are not symmetric problems, and they do not have symmetric remedies. Explosion is the easier of the two, because it is visible: the gradient norm spikes and the parameters jump. The standard fix is to rescale any gradient whose norm exceeds a threshold, keeping the direction and discarding only the magnitude,

$$\boldsymbol{g} \longleftarrow \begin{cases} \boldsymbol{g}, & \|\boldsymbol{g}\| \le \theta, \\ \dfrac{\theta}{\|\boldsymbol{g}\|}\,\boldsymbol{g}, & \|\boldsymbol{g}\| > \theta. \end{cases} \tag{11.23}$$

Two properties make Eq. (11.23) safe to apply unconditionally.

Proposition 11.3 (Clipping is a trust region and preserves descent). *Let $\tilde{\boldsymbol{g}}$ be the clipped gradient of Eq. (11.23). Then (i) $\tilde{\boldsymbol{g}} = \lambda \boldsymbol{g}$ with $\lambda = \min(1, \theta/\|\boldsymbol{g}\|) \in (0,1]$, so $\langle \tilde{\boldsymbol{g}}, \boldsymbol{g} \rangle = \lambda \|\boldsymbol{g}\|^2 > 0$ whenever $\boldsymbol{g} \neq \boldsymbol{0}$ and $-\tilde{\boldsymbol{g}}$ is a descent direction; (ii) $\tilde{\boldsymbol{g}}$ is the solution of*

$$\tilde{\boldsymbol{g}} = \operatorname*{arg\,max}_{\|\boldsymbol{u}\| \le \theta} \langle \boldsymbol{u}, \boldsymbol{g} \rangle \quad \textit{scaled to} \quad \|\boldsymbol{u}\| = \min(\|\boldsymbol{g}\|, \theta), \tag{11.24}$$

that is, the Euclidean projection of $\boldsymbol{g}$ onto the ball of radius θ; so a step of size γ on the clipped gradient never moves the parameters further than $\gamma\theta$.

Proof. (i) is immediate from the definition, since $\lambda > 0$ in both branches. For (ii), the projection of $\boldsymbol{g}$ onto $\{\|\boldsymbol{u}\| \le \theta\}$ minimises $\|\boldsymbol{u} - \boldsymbol{g}\|^2$; writing $\boldsymbol{u} = r\hat{\boldsymbol{u}}$ with $r \le \theta$ gives $\|\boldsymbol{u} - \boldsymbol{g}\|^2 = r^2 - 2r\langle \hat{\boldsymbol{u}}, \boldsymbol{g} \rangle + \|\boldsymbol{g}\|^2$, minimised over directions at $\hat{\boldsymbol{u}} = \boldsymbol{g}/\|\boldsymbol{g}\|$ and then over r at $r = \min(\|\boldsymbol{g}\|, \theta)$.

The two statements are why clipping is not a hack. Property (i) says the optimiser is still doing gradient descent, on a rescaled gradient; property (ii) says the effective step length is bounded by $\gamma\theta$ regardless of what the loss surface does, which is exactly the guarantee a trust-region method provides. What is lost is scale information: near a genuine cliff the clipped step is far shorter than the true gradient suggests, so progress is slow rather than catastrophic. That is the intended trade. This is not a descent direction for any modified cost and it has no convergence theory worth the name, but it is close to universal in practice and it is essentially free. Figure 11.2(c) shows the effect: the bulk of the updates are untouched and only the rare spikes are cut. Note that clipping does nothing whatever for vanishing gradients – rescaling a gradient of 10^{-18} upwards would amplify rounding noise, not signal. Vanishing gradients require a change of architecture, which is the subject of Section 11.7.

Careful initialisation helps with both. Setting $\sigma_{\max}(\boldsymbol{W}) \approx 1$ puts the network at the edge between the two regimes, which is the best one can do with this architecture; the orthogonal or identity initialisations sometimes used for recurrent weights are attempts to sit exactly there.

11.6 Implementation and verification

As in Chapters 8 and 10 we write the network from scratch and check the gradients before trusting anything.

```
def forward(p, X, h0=None):
    """X has shape (T, n_in).  Returns outputs (T, n_out) and the cache.

    a_t = U x_t + W h_{t-1} + b,   h_t = tanh(a_t),   yhat_t = V h_t + c
```

```
    """
    T = X.shape[0]
    n_h = p["W"].shape[0]
    H = np.zeros((T + 1, n_h))              # H[0] is h_{-1}
    if h0 is not None:
        H[0] = h0
    A = np.zeros((T, n_h))
    Y = np.zeros((T, p["V"].shape[0]))
    for t in range(T):
        A[t] = p["U"] @ X[t] + p["W"] @ H[t] + p["b"]
        H[t + 1] = np.tanh(A[t])
        Y[t] = p["V"] @ H[t + 1] + p["c"]
    return Y, (X, A, H)
```

The backward pass is Eqs. (11.15)–(11.18), read off line by line. Note the single loop running backwards and the accumulation with += into arrays that were zeroed once: that accumulation *is* the weight sharing.

```
def bptt(p, cache, Y, target, return_norms=False):
    """Gradients of Eq. (11.cost) by the recursion (11.bptth)."""
    X, A, H = cache
    T = X.shape[0]
    g = {k: np.zeros_like(v) for k, v in p.items()}
    dY = (Y - target) / T                   # dL/dyhat_t for the cost (11.cost)
    dh_next = np.zeros(p["W"].shape[0])
    norms = []
    for t in reversed(range(T)):
        g["V"] += np.outer(dY[t], H[t + 1])
        g["c"] += dY[t]
        dh = p["V"].T @ dY[t] + dh_next     # Eq. (11.bptth)
        da = (1.0 - H[t + 1] ** 2) * dh     # through tanh
        g["U"] += np.outer(da, X[t])
        g["W"] += np.outer(da, H[t])
        g["b"] += da
        dh_next = p["W"].T @ da
        if return_norms:
            norms.append(np.linalg.norm(dh))
    return (g, norms[::-1]) if return_norms else g
```

Clipping is Eq. (11.23) applied to all parameter arrays jointly, since it is the norm of the full gradient vector that matters:

```
def clip(g, theta):
    """Rescale the whole gradient if its norm exceeds theta."""
    n = np.sqrt(sum(np.sum(v ** 2) for v in g.values()))
    if n > theta:
        for k in g:
            g[k] *= theta / n
    return g, n
```

Checking against central differences over *every* entry of every parameter array, for a sequence of length twelve:

```
--- BPTT vs central differences (T=12) ---
  U: max relative error over all 12 entries = 1.38e-09
  W: max relative error over all 36 entries = 3.22e-08
  V: max relative error over all 6 entries = 1.53e-09
  b: max relative error over all 6 entries = 8.25e-10
  c: max relative error over all 1 entries = 1.41e-10
```

Unlike the convolutional network of Chapter 10, this check passes everywhere without a smooth surrogate, because tanh and the quadratic cost are differentiable everywhere and there is no max-pooling argmax to switch.

11.6.1 The damped oscillator

We test the network on the problem of Section 11.3. A fourth-order Runge-Kutta integrator generates the trajectory of Eq. (11.8) with $\eta = 0.2$ over $t \in [0, 40]$ at 800 points, and the network is trained by BPTT with clipping at $\theta = 1$ to predict x_{i+1} from x_i along sequences of length 40. The training data come from a *single* trajectory, with initial condition $(x_0, v_0) = (1, 0)$ and no forcing. We then test on data the network has never seen:

```
train, IC (1,0)            MSE 2.275e-05   Var(x)=0.0655  rel 0.035%
test, unseen IC (0,1)      MSE 2.218e-05   Var(x)=0.0618  rel 0.036%
test, driven+unseen IC     MSE 3.595e-05   Var(x)=0.1488  rel 0.024%
```

The second line is the interesting one. The network was trained on a trajectory starting from rest at unit displacement, and tested on one starting from the origin at unit velocity – a different solution of the same equation, out of phase with everything it saw – and the error is *identical* to the training error, three parts in ten thousand of the variance. The third line goes further: an oscillator driven by $F(t) = 0.3\cos(0.7t)$, which the network never saw in any form, is predicted just as well.

The network has therefore not memorised a trajectory; it has learned the *update rule*, which is what Eq. (11.12) said it would do. That is the payoff of the recurrent structure, and it is the same payoff parameter sharing gave in Chapter 10: a rule learned once applies everywhere it is valid.

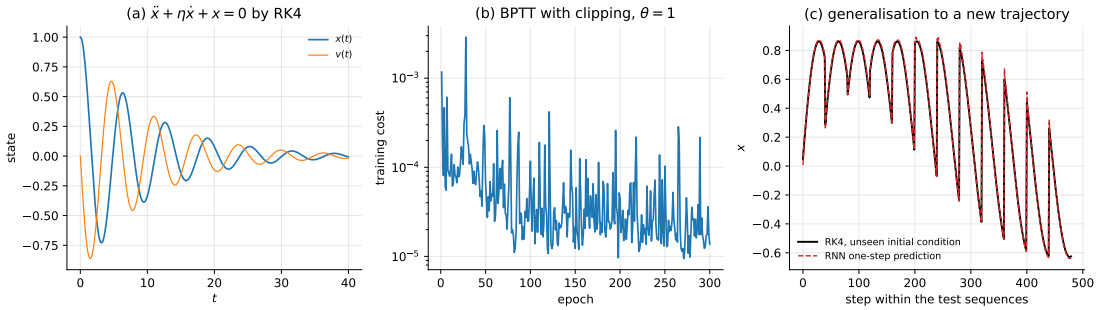


Fig. 11.3 (a) The damped oscillator (11.8) with $\eta = 0.2$, integrated by fourth-order Runge-Kutta; the network sees only $x(t)$. (b) The training cost under BPTT with clipping at $\theta = 1$. (c) One-step predictions on a trajectory with an initial condition the network never saw, against the Runge-Kutta solution.

11.7 Long short-term memory

Theorem 11.1 says that the trouble comes from a product of Jacobians, each carrying a factor of $\boldsymbol{W}$ and a factor of σ_h'. Any architecture that keeps the gradient alive over long lags must break that product. The long short-term memory unit of Hochreiter and Schmidhuber [4] does so by introducing a second state variable whose update is *additive* rather than multiplicative.

The design is usually described as a memory cell with gates. A linear unit with a self-connection of weight one maintains its value indefinitely; information is written into it when a write gate opens, kept while a keep gate is open, and read out when a read gate opens. Because the gates are logistic functions they are differentiable, so the whole circuit can be trained by backpropagation. In modern notation the three gates are called the input, forget and output gates, and the cell carries a long-term state $\boldsymbol{c}$ alongside the short-term state $\boldsymbol{h}$.

The forget gate

decides how much of the existing cell state to retain:

$$\boldsymbol{f}^{(t)} = \sigma\left(\boldsymbol{W}_{fx}\boldsymbol{x}^{(t)} + \boldsymbol{W}_{fh}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_f\right). \tag{11.25}$$

The logistic σ returns values near 0 for very negative arguments and near 1 for very positive ones, so $\boldsymbol{f}^{(t)}$ acts as a soft, learned, per-component switch on the long-term memory.

The input gate

decides what to write, and is two functions rather than one: a logistic deciding *how much* to write and a tanh deciding *what*,

$$\boldsymbol{i}^{(t)} = \sigma\left(\boldsymbol{W}_{ix}\boldsymbol{x}^{(t)} + \boldsymbol{W}_{ih}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_i\right), \qquad \boldsymbol{g}^{(t)} = \tanh\left(\boldsymbol{W}_{gx}\boldsymbol{x}^{(t)} + \boldsymbol{W}_{gh}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_g\right). \tag{11.26}$$

The cell update

combines the two, with $\odot$ the elementwise product:

$$\boxed{\boldsymbol{c}^{(t)} = \boldsymbol{f}^{(t)} \odot \boldsymbol{c}^{(t-1)} + \boldsymbol{i}^{(t)} \odot \boldsymbol{g}^{(t)}.} \tag{11.27}$$

The output gate

decides how much of the cell to expose as the visible state:

$$\boldsymbol{o}^{(t)} = \sigma\left(\boldsymbol{W}_{ox}\boldsymbol{x}^{(t)} + \boldsymbol{W}_{oh}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_o\right), \qquad \boldsymbol{h}^{(t)} = \boldsymbol{o}^{(t)} \odot \tanh\left(\boldsymbol{c}^{(t)}\right). \tag{11.28}$$

The unit has four times the parameters of a simple RNN with the same n_h, since each of $\boldsymbol{f}, \boldsymbol{i}, \boldsymbol{g}, \boldsymbol{o}$ has its own $\boldsymbol{W}_{\cdot x}$, $\boldsymbol{W}_{\cdot h}$ and bias.

Figure 11.4 assembles the four equations into the circuit they describe. It repays a slow reading, because the geometry *is* the argument. Follow the horizontal line at the top, entering as $\boldsymbol{c}^{(t-1)}$ and leaving as $\boldsymbol{c}^{(t)}$: along the way it is multiplied once, by the forget gate, and added to once, by the input gate. That is all. It passes through no weight matrix and no squashing function, which is precisely why Eq. (11.29) contains neither. Every other path in the diagram – the four gates along the bottom, the output multiplication on the right – hangs off that line rather than lying on it.

Contrast the same picture for the simple RNN, which would have the state line running through a matrix multiplication and a tanh at every step. The LSTM did not remove those operations; it moved them off the memory path.

11.7.1 Why it works, and by how much

The point of Eq. (11.27) is what it does to the Jacobian. Differentiating the cell update with respect to the previous cell state, and treating the gates as slowly varying,

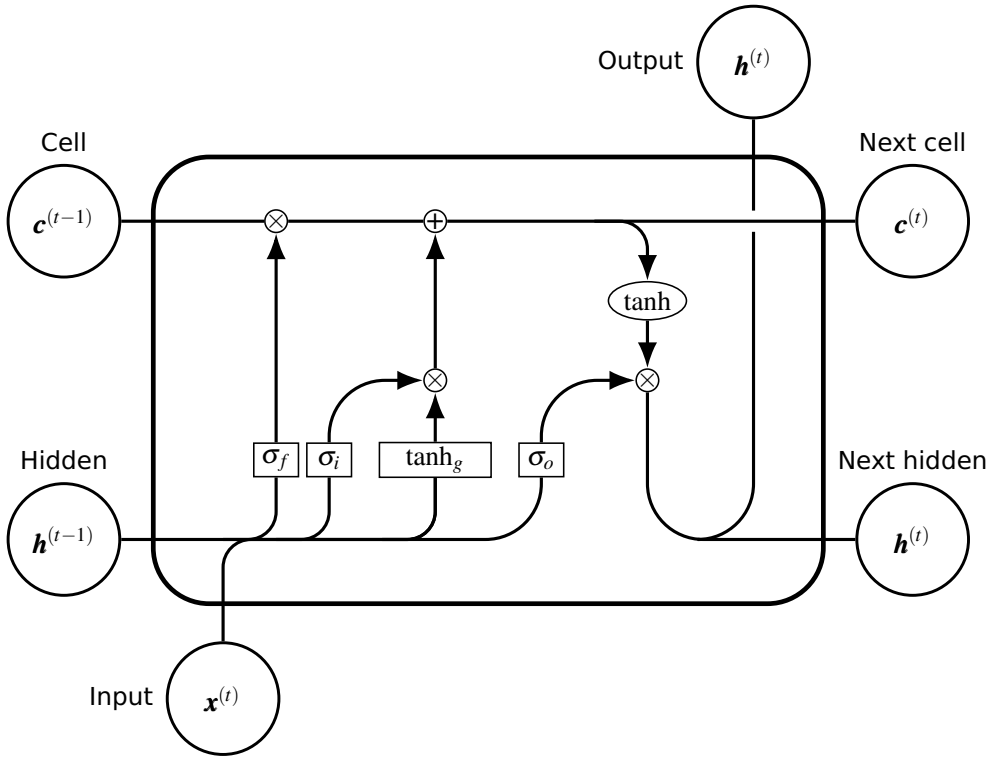


Fig. 11.4 The LSTM cell of Eqs. (11.25)–(11.28). The four boxes along the bottom are the gates: σ_f the forget gate, σ_i and $\tanh_g$ the two halves of the input gate, and σ_o the output gate, each taking $\boldsymbol{x}^{(t)}$ and $\boldsymbol{h}^{(t-1)}$ and each with its own weights. The circled $\times$ and $+$ are elementwise product and sum. The horizontal line carrying $\boldsymbol{c}^{(t-1)}$ to $\boldsymbol{c}^{(t)}$ is the *constant error carousel*: it meets one multiplication and one addition and no weight matrix, which is the content of Eq. (11.29) and the reason gradients survive along it.

$$\frac{\partial \boldsymbol{c}^{(t)}}{\partial \boldsymbol{c}^{(t-1)}} \approx \operatorname{diag}\left(\boldsymbol{f}^{(t)}\right), \qquad \text{so} \qquad \frac{\partial \boldsymbol{c}^{(T)}}{\partial \boldsymbol{c}^{(t)}} \approx \prod_{k=t+1}^{T} \operatorname{diag}\left(\boldsymbol{f}^{(k)}\right). \tag{11.29}$$

Compare with Eq. (11.19). There is no $\boldsymbol{W}$ and no σ_h'; the product is over the forget gates alone. If the network learns to hold $\boldsymbol{f}^{(k)} \approx 1$ on some component, the gradient along that component is transmitted essentially undamped for as long as the gate stays open. This is often called the *constant error carousel*, and it is the whole idea.

That is the argument. It is worth checking, because the argument is often stated as though the LSTM simply solves the problem, and it does not. Measuring Eq. (11.29) on an untrained LSTM with $n_h = 20$ and $T = 60$, for three values of the forget-gate bias:

```
forget bias 0.0: mean f=0.5000  ||dc_T/dc_1||=3.879e-18
forget bias 1.0: mean f=0.7311  ||dc_T/dc_1||=3.074e-08
forget bias 2.0: mean f=0.8810  ||dc_T/dc_1||=2.232e-03
```

against 1.683×10^{-18} for the vanilla RNN at $\rho(\boldsymbol{W}) = 0.5$. With $\boldsymbol{b}_f = 0$ the forget gates sit at $\sigma(0) = 1/2$, the product is $2^{-59} \approx 10^{-18}$, and the LSTM is *no better than the simple RNN it was invented to replace*. The gain is entirely a matter of where the gates start: biasing them open moves the mean gate to 0.73 and buys ten orders of magnitude, and a bias of two buys fifteen. Figure 11.2(b) shows the three curves with the vanilla RNN underneath, and the $b_f = 0$ curve lies almost exactly on top of it.

Initialise the forget-gate bias positive. This is the practical consequence, and it is why implementations commonly default to $\boldsymbol{b}_f = 1$; the empirical survey of Jozefowicz *et al.* [50] found it to be among the most reliable single adjustments to recurrent training. The architecture supplies a *path* along which gradients can travel undamped; the initialisation decides whether the network starts anywhere near it. This is the same lesson as the He and Xavier initialisations of Chapter 8, in a setting where the consequence is measured in orders of magnitude rather than factors.

Equation (11.29) is approximate, and it is worth writing the exact statement, because what the approximation drops is not negligible.

Proposition 11.4 (The exact cell Jacobian). *With the gates of Eqs. (11.25)–(11.28),*

$$\frac{\partial \boldsymbol{c}^{(t)}}{\partial \boldsymbol{c}^{(t-1)}} = \operatorname{diag}\left(\boldsymbol{f}^{(t)}\right) + \boldsymbol{R}^{(t)}, \tag{11.30}$$

where, writing $\boldsymbol{D}_f = \operatorname{diag}(\boldsymbol{f}(1-\boldsymbol{f}))$, $\boldsymbol{D}_i = \operatorname{diag}(\boldsymbol{i}(1-\boldsymbol{i}))$, $\boldsymbol{D}_g = \operatorname{diag}(1-\boldsymbol{g}^2)$ *for the gate derivatives at step* t*,*

$$\boldsymbol{R}^{(t)} = \left[\operatorname{diag}\left(\boldsymbol{c}^{(t-1)}\right)\boldsymbol{D}_f\boldsymbol{W}_{fh} + \operatorname{diag}\left(\boldsymbol{g}^{(t)}\right)\boldsymbol{D}_i\boldsymbol{W}_{ih} + \operatorname{diag}\left(\boldsymbol{i}^{(t)}\right)\boldsymbol{D}_g\boldsymbol{W}_{gh}\right]\operatorname{diag}\left(\boldsymbol{o}^{(t-1)}\left(1-\tanh^2\boldsymbol{c}^{(t-1)}\right)\right). \tag{11.31}$$

Proof. Differentiate Eq. (11.27). The explicit dependence on $\boldsymbol{c}^{(t-1)}$ gives $\operatorname{diag}(\boldsymbol{f}^{(t)})$. The remaining dependence is through $\boldsymbol{h}^{(t-1)} = \boldsymbol{o}^{(t-1)} \odot \tanh \boldsymbol{c}^{(t-1)}$, which enters $\boldsymbol{f}^{(t)}, \boldsymbol{i}^{(t)}, \boldsymbol{g}^{(t)}$ through $\boldsymbol{W}_{fh}, \boldsymbol{W}_{ih}, \boldsymbol{W}_{gh}$; the chain rule with the elementwise derivatives $\boldsymbol{D}_f, \boldsymbol{D}_i, \boldsymbol{D}_g$ gives the bracket, and $\partial \boldsymbol{h}^{(t-1)}/\partial \boldsymbol{c}^{(t-1)} = \operatorname{diag}(\boldsymbol{o}^{(t-1)}(1-\tanh^2\boldsymbol{c}^{(t-1)}))$ is the final factor. Note that $\boldsymbol{o}^{(t-1)}$ depends on $\boldsymbol{h}^{(t-2)}$, not on $\boldsymbol{c}^{(t-1)}$, so the expression closes.

Every term of $\boldsymbol{R}^{(t)}$ carries a logistic derivative, bounded by $1/4$, and the factor $1-\tanh^2\boldsymbol{c}^{(t-1)}$, which vanishes as the cell state saturates. One would therefore expect $\boldsymbol{R}$ to be negligible in the regime the architecture is designed for. Measuring it – assembling $\partial \boldsymbol{c}^{(T)}/\partial \boldsymbol{c}^{(1)}$ exactly by automatic differentiation and comparing against the product of forget gates alone – shows that this expectation is wrong:

```
=== 5. Eq. (11.lstmjac) is approximate: how approximate? ===
  input scale   b_f   mean f   prod diag(f)   exact ||dc_T/dc_1||   ratio
          1.0   0.0   0.5185       1.891e-11             2.255e-08   1192.07
          1.0   1.0   0.7284       2.538e-05             1.514e-03     59.63
          1.0   2.0   0.8693       1.808e-02             5.654e-01     31.27
          1.0   4.0   0.9759       5.970e-01             8.408e+00     14.08
          0.1   0.0   0.5007       2.672e-12             4.828e-07   180716.73
          0.1   1.0   0.7310       6.201e-06             7.472e-02   12048.86
          0.1   2.0   0.8799       1.247e-02             8.597e+00     689.13
          0.1   4.0   0.9789       5.848e-01             1.144e+02     195.66
```

The ratio never approaches one. Biasing the gates open improves it by two orders of magnitude, from 1192 to 14; driving the cell more *gently* makes it far worse, because a cell state near zero has $1-\tanh^2\boldsymbol{c} \approx 1$ and the gate paths of Eq. (11.31) are then at their most conductive. So Eq. (11.29) should be read as a *lower bound* on how much gradient survives, not as an estimate of it.

The sign of the discrepancy is favourable: the true Jacobian is larger than the constant-error-carousel argument promises, so the LSTM transmits more gradient than the standard story claims, by one to five orders of magnitude in these measurements. But the standard story is not quantitative, and it is worth knowing that the gates are not merely a valve on a passive channel – they are themselves a conducting path.

One further point. Nothing here prevents *explosion*: since $\boldsymbol{f} \leq \mathbf{1}$ the diagonal term of Eq. (11.30) cannot amplify, but $\boldsymbol{R}^{(t)}$ contains weight matrices with no such bound, so clipping remains necessary.

11.7.2 The gated recurrent unit

The gated recurrent unit of Cho *et al.* [51] applies the same idea with fewer parts. It drops the separate cell state, keeping only $\boldsymbol{h}$, and uses two gates instead of three:

$$\begin{aligned}
\boldsymbol{z}^{(t)} &= \sigma\left(\boldsymbol{W}_{zx}\boldsymbol{x}^{(t)} + \boldsymbol{W}_{zh}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_z\right), \\
\boldsymbol{r}^{(t)} &= \sigma\left(\boldsymbol{W}_{rx}\boldsymbol{x}^{(t)} + \boldsymbol{W}_{rh}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_r\right), \\
\tilde{\boldsymbol{h}}^{(t)} &= \tanh\left(\boldsymbol{W}_{hx}\boldsymbol{x}^{(t)} + \boldsymbol{W}_{hh}\left(\boldsymbol{r}^{(t)} \odot \boldsymbol{h}^{(t-1)}\right) + \boldsymbol{b}_h\right), \\
\boldsymbol{h}^{(t)} &= \left(1 - \boldsymbol{z}^{(t)}\right) \odot \boldsymbol{h}^{(t-1)} + \boldsymbol{z}^{(t)} \odot \tilde{\boldsymbol{h}}^{(t)}.
\end{aligned} \tag{11.32}$$

The *update* gate $\boldsymbol{z}$ plays the roles of both the forget and input gates at once – what is not written is retained, by construction – and the *reset* gate $\boldsymbol{r}$ controls how much of the past enters the candidate state. The last line has the same additive form as Eq. (11.27), so the same constant-error-carousel argument applies with $1 - \boldsymbol{z}$ in place of $\boldsymbol{f}$.

A GRU has three sets of weight matrices against the LSTM's four, so about three quarters of the parameters and correspondingly less computation. Which performs better is task-dependent and not settled; the two are usually close. More recent work continues in the same direction: Beck *et al.* [52] report that exponential gating and modified memory structures make an extended LSTM competitive with transformers and state-space models in both performance and scaling.

11.8 The same networks in PyTorch and TensorFlow

Everything so far has been derived and then implemented in NumPy. This section hands the same architectures to the two libraries, and does three things with them: runs the two standard examples, checks that the libraries and our code compute the same function to rounding, and then asks the question the theory has been building towards – whether the gating of Section 11.7 lets a network learn something a simple recurrent network cannot.

11.8.1 Forecasting a sine wave

The task is the one-step-ahead forecasting of Section 11.6.1, on a sine wave rather than a damped oscillator. Sequences of 20 consecutive values are used to predict the next.

```
import numpy as np
import tensorflow as tf

# 1. Data: a sine wave cut into overlapping windows
time_steps = np.linspace(0, 100, 500)
data = np.sin(time_steps)
seq_length = 20
```

```
X, y = [], []
for i in range(len(data) - seq_length):
    X.append(data[i:i+seq_length])
    y.append(data[i+seq_length])
X = np.array(X).reshape(-1, seq_length, 1)     # (samples, timesteps, features)
y = np.array(y).reshape(-1, 1)

split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# 2. Model: a SimpleRNN layer is exactly Eq. (11.rnn)
model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(16, input_shape=(seq_length, 1)),
    tf.keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse")
model.summary()      # 16*(1+16+1) = 288 recurrent parameters, then 17

# 3. Training and evaluation
history = model.fit(X_train, y_train, epochs=50, batch_size=32,
                    validation_split=0.2, verbose=1)
print(f"test loss {model.evaluate(X_test, y_test, verbose=0):.4f}")
```

The same in PyTorch, where the loop is explicit:

```
import numpy as np
import torch
import torch.nn as nn

seq_length = 20
data = np.sin(np.linspace(0, 100, 500)).astype("float32")
X = np.stack([data[i:i+seq_length] for i in range(len(data)-seq_length)])
y = data[seq_length:].reshape(-1, 1)
X = torch.tensor(X).unsqueeze(-1)              # (N, T, 1), batch_first
y = torch.tensor(y)
split = int(0.8 * len(X))


class SineRNN(nn.Module):
    """nn.RNN implements Eq. (11.rnn) with tanh by default."""
    def __init__(self, hidden=16):
        super().__init__()
        self.rnn = nn.RNN(input_size=1, hidden_size=hidden, batch_first=True)
        self.fc = nn.Linear(hidden, 1)

    def forward(self, x):
        out, h_T = self.rnn(x)                 # out: (N, T, hidden)
        return self.fc(out[:, -1, :])          # read out the last state only


model = SineRNN()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
lossfn = nn.MSELoss()
for epoch in range(50):
    model.train()
    opt.zero_grad()
    loss = lossfn(model(X[:split]), y[:split])
    loss.backward()                            # BPTT, Eqs. (11.bptth)-(11.bpttparams)
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)   # Eq. (11.clip)
    opt.step()
    if (epoch + 1) % 10 == 0:
        print(f"epoch {epoch+1}: train loss {loss.item():.5f}")
```

```
model.eval()
with torch.no_grad():
    print(f"test loss {lossfn(model(X[split:]), y[split:]).item():.5f}")
```

Note `clip_grad_norm_`, which is Eq. (11.23) exactly and is justified by Proposition 11.3, and the line `out[:, -1, :]`, which selects the sequence-to-label case of Eq. (11.1): only the final state is read.

Run for two hundred epochs with minibatches of thirty-two, both reach the same place:

```
PyTorch    epoch 200: train mse 0.000008   test mse 0.000004   (321 parameters)
TensorFlow epoch 200: train mse 0.000003   test mse 0.000002   (305 parameters)
```

A test mean squared error of 4×10^{-6} is a root-mean-square error of 0.003 on a signal of unit amplitude, and Figure 11.5(a) shows the two predictions lying on the true curve. The parameter counts differ by sixteen, and the reason is the convention noted below: PyTorch's `nn.RNN` carries two bias vectors where Eq. (11.4) and Keras carry one. Their sum is our $\boldsymbol{b}$, so the models are the same; only the parameterisation is redundant.

11.8.2 An LSTM on MNIST, read row by row

A less obvious use is to treat each 28×28 image as a sequence of 28 rows, each of 28 features, and classify it with an LSTM. This is not what one would do in practice – Chapter 10 explains why a convolutional network is the right tool for images – but it is a clean demonstration of a sequence-to-label task, and comparing it against Section 10.10 is instructive.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.utils import to_categorical

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train = x_train.reshape((-1, 28, 28))        # 28 timesteps of 28 features
x_test = x_test.reshape((-1, 28, 28))
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

model = Sequential([
    LSTM(128, input_shape=(28, 28)),           # 4*128*(28+128+1) = 80384 params
    Dense(10, activation="softmax"),
])
model.compile(loss="categorical_crossentropy", optimizer="adam",
              metrics=["accuracy"])
model.summary()
model.fit(x_train, y_train, batch_size=64, epochs=10, validation_split=0.2)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"test accuracy {test_acc:.4f}")
```

and in PyTorch:

```
import torch
import torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
transform = transforms.Compose([transforms.ToTensor(),
```

```
                                transforms.Normalize((0.1307,), (0.3081,))])
train_loader = DataLoader(datasets.MNIST("./data", train=True, download=True,
                                         transform=transform),
                          batch_size=64, shuffle=True)
test_loader = DataLoader(datasets.MNIST("./data", train=False,
                                        transform=transform), batch_size=64)


class LSTMModel(nn.Module):
    """Eqs. (11.lstmf)-(11.lstmh); nn.LSTM already biases b_f, see below."""
    def __init__(self, input_size=28, hidden_size=128, num_classes=10):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        x = x.reshape(-1, 28, 28)              # image as a sequence of rows
        out, _ = self.lstm(x)                  # (batch, seq, hidden)
        return self.fc(out[:, -1, :])          # last state -> class scores


model = LSTMModel().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(10):
    model.train()
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        loss = criterion(model(images), labels)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
    model.eval()
    correct = total = 0
    with torch.no_grad():
        for images, labels in test_loader:
            pred = model(images.to(device)).argmax(dim=1)
            correct += (pred == labels.to(device)).sum().item()
            total += labels.size(0)
    print(f"epoch {epoch+1}: test accuracy {100*correct/total:.2f}%")
```

Two remarks. PyTorch's `nn.LSTM` does *not* initialise the forget-gate bias to one; it draws all biases uniformly from $(-1/\sqrt{n_h}, 1/\sqrt{n_h})$, so the mean forget gate starts at about 0.5 – which by Section 11.7.1 is the worst case. Setting it explicitly is a few lines and is worth doing for long sequences. And the parameter count $4n_h(n_x + n_h + 1)$ printed by `model.summary()` is a useful check of the four gates of Eqs. (11.25)–(11.28); if the number is not four times that of a simple RNN, something is wrong.

11.8.3 Do the three implementations agree?

The listings and the NumPy code of Section 11.6 are supposed to be the same function, and the recurrent case has more room for disagreement than the convolutional one of Section 10.12.3, because three conventions differ at once:

- PyTorch's `nn.RNN` and `nn.LSTM` carry *two* bias vectors, `bias_ih` and `bias_hh`, where Eq. (11.4) has one. Their sum is our $\boldsymbol{b}$.

- Keras stores the input matrix transposed – `kernel` of shape (n_x, n_h) against our $\boldsymbol{U}$ of shape (n_h, n_x) – and likewise the recurrent matrix.
- Both concatenate the four LSTM gates into one array in the order $(\boldsymbol{i}, \boldsymbol{f}, \boldsymbol{g}, \boldsymbol{o})$, whereas Eqs. (11.25)–(11.28) introduce them as $(\boldsymbol{f}, \boldsymbol{i}, \boldsymbol{g}, \boldsymbol{o})$. Getting this wrong produces a network that trains perfectly well and is not the one on the page.

With those reconciled, one set of weights in all three:

```
=== 1. the simple RNN of Eq. (11.rnn), forward ===
  hidden states  max |ours - torch| : 2.220e-16
  hidden states  max |ours - keras| : 4.441e-16
  outputs        max |ours - torch| : 2.220e-16
  outputs        max |ours - keras| : 3.331e-16

=== 2. backpropagation through time, Eqs. (11.bptth)-(11.bpttparams) ===
  our loss, Eq. (11.cost)   : 1.171647902250
  torch loss                : 1.171647902250
   array   shape        |ours - torch|      scale
   U     (8, 3)            5.551e-17   2.105e-01
   W     (8, 8)            5.551e-17   2.329e-01
   V     (2, 8)            1.110e-16   2.509e-01
   b     (8,)              1.110e-16   2.766e-01
   c     (2,)              1.110e-16   6.103e-01

=== 3. the LSTM cell, Eqs. (11.lstmf)-(11.lstmh) ===
  cell output    max |ours - torch| : 8.327e-17
  cell output    max |ours - keras| : 1.388e-16
```

Every entry is at unit roundoff; Figure 11.5(d) plots the backward-pass comparison. The recursion of Eqs. (11.15)–(11.18), derived by hand in Section 11.4, is what reverse-mode automatic differentiation computes.

The same idea settles Theorem 11.1. Rather than forming the product of Eq. (11.19) ourselves, we can ask autograd for $\partial \boldsymbol{h}^{(T)} / \partial \boldsymbol{h}^{(1)}$ column by column, with no reference to the theorem at all:

```
=== 4. Theorem 11.vanishing, measured by autograd in PyTorch ===
   rho(W)   ||dh_T/dh_1|| autograd   ||dh_T/dh_1|| Eq. (11.jacprod)   ratio
    0.50               1.967e-17                  1.967e-17    1.0000
    0.90               2.264e-02                  2.264e-02    1.0000
    1.10               3.116e+03                  3.116e+03    1.0000
    1.50               7.351e+04                  7.351e+04    1.0000
```

The vanishing gradient is not an artefact of our implementation, and it is not something a better library avoids. It is a property of the composition.

11.8.4 Row by row: the two libraries on MNIST

Training the LSTM of the listing above, and a simple RNN of the same width for comparison, for three epochs on the full sixty thousand images:

Three things are worth reading off Table 11.1. The LSTM beats the simple RNN, by three points in PyTorch and one in TensorFlow, on a task with only 28 time steps – short enough that Corollary 11.2 does not forbid the simple RNN from working, and it does work, just less well. The parameter counts differ between frameworks by exactly $4n_h$ and n_h, the extra bias vector again. And the whole exercise reaches 0.97 where the convolutional network of Section 10.12.5 reached 0.99 with a comparable budget: reading an image as a sequence throws away the vertical structure that Chapter 10 was built to exploit, and the cost of doing so is visible.

Table 11.1 MNIST read as 28 rows of 28, after three epochs with Adam at 10^{-3} and batches of 64. Both cells have $n_h = 128$; the LSTM has four gates and therefore about four times the parameters. Compare the convolutional network of Section 10.12.5, which reaches 0.99 on the same data.

Cell	Framework	Parameters	Test accuracy	Seconds/epoch
LSTM	PyTorch	82 186	0.9708	8
LSTM	TensorFlow	81 674	0.9743	21
simple RNN	PyTorch	21 514	0.9333	6
simple RNN	TensorFlow	21 386	0.9623	7

11.8.5 The experiment the theory demands

Section 11.7.1 showed that the LSTM keeps a larger Jacobian than a simple RNN, and Table 11.1 showed it classifying slightly better over 28 steps. Neither is the claim the architecture was invented to support, which is that gating lets a network learn dependencies a simple recurrence *cannot*. Testing that needs a task where nothing but long-range memory is being measured.

The standard one is the *adding problem*. Each sequence has two channels: the first carries values drawn uniformly from $[0, 1]$, the second is zero except for exactly two markers, one in each half of the sequence. The target is half the sum of the two marked values. Nothing can be inferred from the local statistics – the values are independent and identically distributed – so a model must hold one number from the first half of the sequence until the second marker arrives, while ignoring everything in between. Predicting the mean and stopping gives a mean squared error of about 0.043, and that is the number to beat.

```
=== 3. the adding problem at increasing sequence length ===
   T   baseline          rnn         lstm          gru
  10     0.0427       0.0016       0.0000       0.0000
  30     0.0429       0.0411       0.0001       0.0000
  60     0.0411       0.0407       0.0002       0.0001
 120     0.0425       0.0427       0.0006       0.0001
```

This is the cleanest result in the chapter. At $T = 10$ the simple recurrent network solves the task, reaching 0.0016 against a baseline of 0.0427. At $T = 30$ it has already failed completely – 0.0411 against 0.0429 is the error of a model that has learned to output the mean and nothing else – and it never recovers at 60 or 120. The LSTM and the GRU solve every case, including $T = 120$, at errors between 10^{-4} and 10^{-3}: two to three orders of magnitude below the simple RNN, with the same hidden width, the same optimiser and the same number of epochs.

Corollary 11.2 predicted this. A simple recurrent network initialised in the usual way has a memory horizon of a few steps, so a dependency at lag 30 is invisible to it; the gated architectures replace the product of Jacobians by Eq. (11.30), whose leading term is a product of forget gates that training can drive towards one. The gap between $T = 10$ and $T = 30$ in the table is where the horizon runs out.

11.9 Summary and the programs

A recurrent network is the architecture obtained by insisting that a model respect the ordering of its data and reuse its parameters along the time axis. The state-space form (11.3) makes it a learned dynamical system, and Section 11.3 showed that the recurrence is not arbitrary but exactly the structure of an explicit time-stepping scheme for a differential equation.

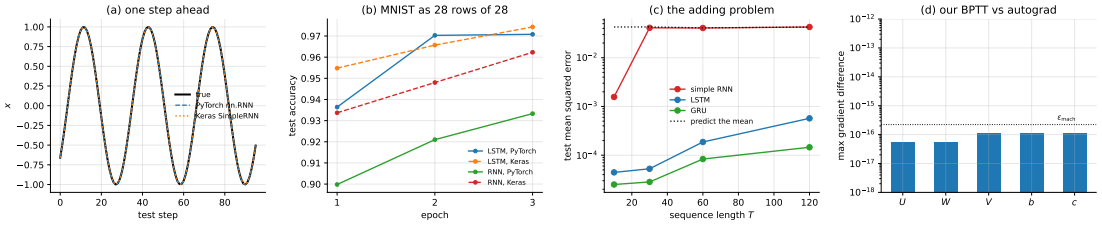


Fig. 11.5 (a) One-step-ahead forecasts of the sine wave of Section 11.8.1 on the held-out segment, from `nn.RNN` and from `SimpleRNN`; both lie on the true curve at a root-mean-square error of 0.003. (b) Test accuracy against epoch for MNIST read row by row, Table 11.1. (c) The adding problem: mean squared error against sequence length for the three cells, with the error of predicting the mean shown dotted. The simple RNN leaves the dotted line only at $T = 10$. (d) The largest disagreement between our BPTT recursion, Eqs. (11.15)–(11.18), and the same gradients from automatic differentiation, against the double-precision unit roundoff.

Backpropagation through time is ordinary backpropagation on the unrolled graph, with the gradients from all steps summed into shared parameters. The forward pass is non-linear and bounded; the backward pass is linear and unbounded, and that asymmetry is the source of every difficulty in this chapter.

Theorem 11.1 makes the difficulty precise: the gradient across a lag is a product of Jacobians, bounded by $(\gamma\sigma_{\max}(\boldsymbol{W}))^{T-t}$, so it vanishes or explodes exponentially unless the network sits at the boundary. Measured on a network with $\rho(\boldsymbol{W}) = 0.5$ over fifty-nine steps, the influence of the first state on the last fell by eighteen orders of magnitude. Clipping, Eq. (11.23), handles explosion cheaply and does nothing at all for vanishing.

The LSTM handles vanishing by giving the gradient an additive path, Eq. (11.27), whose Jacobian is a product of forget gates alone. The measurement in Section 11.7.1 should be kept in mind alongside the argument: at $\boldsymbol{b}_f = 0$ the LSTM transmits 3.9×10^{-18} over fifty-nine steps, no better than the vanilla RNN's 1.7×10^{-18}, and it is biasing the gate open that buys the ten to fifteen orders of magnitude. The architecture supplies the path; the initialisation decides whether training starts near it.

The experiment of Section 11.6.1 is the positive result. Trained on one trajectory of a damped oscillator and tested on a different initial condition, and then on a driven oscillator it had never seen, the network's error was unchanged at three parts in ten thousand of the variance. It learned the update rule rather than the trajectory.

Three quantitative additions sharpen the picture. Corollary 11.2 converts Theorem 11.1 into a *memory horizon* τ, and fitting it to the measured decay gives $\tau = 1.45$ steps at $\rho(\boldsymbol{W}) = 0.5$ and 9.66 at $\rho = 0.9$ – the usable range is narrow and it sits just below one. Proposition 11.3 shows that clipping is the projection onto a ball, hence a trust region that preserves the descent direction, which is why it can be applied unconditionally. And Proposition 11.4 gives the cell Jacobian exactly; measuring it against Eq. (11.29) shows the constant-error carousel to be a lower bound that understates the true gradient by one to five orders of magnitude, since the gate paths conduct as well.

Section 11.8 then checked all of it against the libraries. Our forward pass and our BPTT recursion agree with PyTorch and TensorFlow to 4.4×10^{-16}, and asking autograd directly for $\partial \boldsymbol{h}^{(T)} / \partial \boldsymbol{h}^{(1)}$ reproduces the product of Eq. (11.19) to a ratio of 1.0000: the vanishing gradient is a property of the composition, not of an implementation. The chapter closes with the experiment the theory demands. On the adding problem a simple recurrent network solves $T = 10$ and fails completely from $T = 30$ onwards, sitting exactly on the predict-the-mean baseline, while the LSTM and the GRU solve every length up to $T = 120$ at errors two to three orders of magnitude lower. That gap, at that lag, is Corollary 11.2 made visible.

The programs are in the directory
`doc/BookML/BookPrograms/chapter11_recurrent_networks`.

Every listing above appears there as a numbered file, and three modules run start to finish and reproduce the numbers quoted in the text:

- `rnn.py` – the forward pass, BPTT, clipping, Adam and the LSTM cell of Eqs. (11.25)–(11.28).
- `verify_rnn.py` – the gradient check against central differences, the Jacobian-product measurements of Section 11.5.1, and the forget-bias study of Section 11.7.1.
- `oscillator.py` – the Runge-Kutta integrator, the training data and the generalisation experiment of Section 11.6.1.
- `cross_check_rnn.py` – the three-way comparison of Section 11.8.3, the autograd measurement of Theorem 11.1, the exact cell Jacobian of Proposition 11.4 against Eq. (11.29), and the fitted memory horizon of Corollary 11.2.
- `rnn_torch.py` – the sine forecast, the row-by-row MNIST classification and the adding problem of Section 11.8.5.
- `rnn_tf.py` – the first two of those in TensorFlow.
- `mnist_data.py` – a loader that reads MNIST from a local `.npz`, so that the programs run without network access.

The two measurement figures are generated by the script `doc/BookML/BookFigures/ch11_figures.py` and contain no hand-placed numbers. The two schematics, Figures 11.1 and 11.4, are Ti*k*Z drawings in the chapter source rather than plots; they carry no data and are adapted from the course diagrams in `rnnsetup.tex` and `lstm.tex`. The script `render_tikz.py` in the same directory compiles them standalone to PNG, so that the notebook version of the chapter, which cannot typeset Ti*k*Z, can display them.

11.10 Exercises

Warm-up exercises

1. **Parameter counting.** (a) How many parameters has a simple RNN with $n_x = 10$, $n_h = 64$, $n_y = 1$? (b) How many has the LSTM of Eqs. (11.25)–(11.28) with the same dimensions, and the GRU of Eq. (11.32)? (c) How many would a feed-forward network need if the sequence of length $T = 200$ were flattened into one vector, and what happens when T changes?
2. **The unrolled graph.** Draw the computational graph of Eq. (11.4) for $T = 4$, marking every place $\boldsymbol{W}$ appears. Use it to explain why $\nabla_{\boldsymbol{W}} \mathscr{L}$ is a sum of four terms and not one.
3. **Fixed points.** For the autonomous map (11.6) with $n_h = 1$, $\sigma_h = \tanh$ and $b = 0$, show that $h^\star = 0$ is always a fixed point, that it is stable for $|W| < 1$, and that a pitchfork bifurcation occurs at $W = 1$ producing two stable non-zero fixed points. Relate this to Theorem 11.1.
4. **The bound.** (a) Prove that $\rho(\boldsymbol{A}) \le \|\boldsymbol{A}\|_2$ for any square $\boldsymbol{A}$, with equality when $\boldsymbol{A}$ is normal. (b) Construct a 2×2 matrix with $\rho = 0.5$ and $\sigma_{\max} > 2$, and explain what this implies about transient growth of the gradient. (c) Why does Theorem 11.1 use $\sigma_{\max}$ rather than ρ?
5. **Activations.** Show that the logistic function has $\gamma = \max |\sigma'| = 1/4$ while $\tanh$ has $\gamma = 1$. Deduce that a logistic recurrent network loses at least two bits of gradient per step regardless of its weights, and explain why $\tanh$ is the standard choice.
6. **The constant error carousel.** Derive Eq. (11.29) from Eq. (11.27), stating clearly which term you are dropping and when that is justified. Then show that the *diagonal* part of Eq. (11.30) has norm at most one, so that the cell path alone can never cause explosion, and explain why Proposition 11.4 nevertheless leaves room for it.

7. **The memory horizon.** (a) Derive Corollary 11.2 and compute τ for $\gamma = 1$ and $\sigma_{\max} = 0.9$, 0.99, 0.999. (b) How large must $\sigma_{\max}$ be for a horizon of 100 steps, and what does Theorem 11.1 then say about the exploding regime? (c) Reproduce the fitted horizons quoted in Section 11.5.1 and explain why the fitted τ is smaller than the one Eq. (11.21) predicts from $\sigma_{\max}$.
8. **Truncated backpropagation.** (a) Derive the bound (11.22), stating what C collects. (b) For $\tau = 5$, find the truncation length k that keeps the bias below one per cent of the surviving gradient. (c) Implement truncated BPTT in `rnn.py` and confirm the bound numerically by comparing against the untruncated gradient.
9. **Clipping.** (a) Prove Proposition 11.3. (b) Show that clipping each parameter array separately, rather than the concatenated gradient, does *not* in general preserve the direction, and construct a two-array example where the angle changes. (c) What happens to Adam's second-moment estimate when clipping is active often, and does that argue for clipping before or after the optimiser state is updated?
10. **The exact cell Jacobian.** (a) Derive Proposition 11.4, being careful about which quantities $\boldsymbol{o}^{(t-1)}$ depends on. (b) Bound $\|\boldsymbol{R}^{(t)}\|_2$ in terms of the gate weight norms and $\|\boldsymbol{c}^{(t-1)}\|_\infty$. (c) Explain, from Eq. (11.31), why the measured ratio in Section 11.7.1 gets *worse* rather than better when the input is scaled down.
11. **The adding problem.** Reproduce Section 11.8.5 and extend it. (a) At which T does the simple RNN fail, and how does that compare with the horizon τ of its initialisation? (b) Set the forget-gate bias of the LSTM to zero and repeat. Does the LSTM still solve $T = 120$? (c) Replace the LSTM by a simple RNN with $\boldsymbol{W}$ initialised to the identity and ReLU activations – the identity RNN – and report where it sits between the two.

Project-style exercise: recurrent networks from scratch

Part a: the machinery.

Implement the forward pass (11.4) and backpropagation through time (11.15)–(11.18) in NumPy. Verify every entry of every gradient against central differences for at least three sequence lengths, and report the largest relative error.

Part b: the dynamics.

Study the autonomous map (11.6) numerically for $n_h = 2$ and $n_h = 3$. Find fixed points, classify them by the eigenvalues of Eq. (11.7), and exhibit a limit cycle. Can you find chaotic behaviour, and how would you establish that it is chaotic rather than merely long-period?

Part c: measuring the theorem.

Reproduce Section 11.5.1: form the Jacobian product (11.19) explicitly and measure its norm against the lag for several spectral radii. Confirm the bound (11.20), report how loose it is, and explain the looseness. Then repeat with saturating inputs and describe what changes.

Part d: the oscillator.

Reproduce Section 11.6.1. Then push it: train on the undamped case $\eta = 0$ and test on $\eta = 0.2$, and vice versa. Train on one forcing frequency and test on another. Where does the learned update rule stop generalising, and does that boundary make physical sense?

Part e: clipping.

Train with and without Eq. (11.23) over a range of thresholds θ and sequence lengths. Record the distribution of gradient norms. At what sequence length does training fail without clipping, and does clipping ever hurt?

Part f: gates.

Implement the LSTM of Eqs. (11.25)–(11.28) and the GRU of Eq. (11.32) from scratch, with verified gradients. Reproduce the forget-bias measurement of Section 11.7.1 for both. Then design a task that genuinely requires long memory – for instance, output the first element of the sequence at the final step, with T ranging from 10 to 200 – and plot the accuracy of the RNN, the LSTM and the GRU against T at matched parameter counts. This is the experiment that decides whether the gates are worth their cost.

Part g: against a library.

Rebuild the same architectures in PyTorch or TensorFlow and verify that they agree with yours numerically: feed both the same weights and the same input and compare the forward and backward passes to machine precision. Check the library's forget-gate bias initialisation, and measure what setting it to one does to your long-memory task from part f.

Chapter 12
Autoencoders

Every network so far has been trained against labels. Regression needed targets y_i, classification needed classes, the differential-equation solvers of Chapter 9 needed an equation. This chapter removes the labels and asks the network to reproduce its own input.

That sounds vacuous – the identity map is available for free – and it would be, were it not for a constraint. An *autoencoder* factors the map through a representation of restricted size or restricted form,

$$\boldsymbol{x} \longmapsto \boldsymbol{z} = f_\theta(\boldsymbol{x}) \longmapsto \hat{\boldsymbol{x}} = g_\phi(\boldsymbol{z}), \tag{12.1}$$

and asks for the best approximation to the identity subject to that factorisation. The whole subject is the study of what that constraint forces the network to discover.

The chapter has two halves. The first is exact: when f and g are linear the problem admits a complete analytic solution, and the answer is principal component analysis. We prove this, carefully, because the proof is short, because it is the one place in deep learning where a global optimum is known, and because knowing exactly what the linear case does tells us what the nonlinear case is buying. The second half is what happens when the linearity is given up: deep, denoising, sparse and contractive autoencoders, the convolutional and recurrent variants that reuse Chapters 10 and 11 unchanged, and the probabilistic reading that connects the whole construction back to Chapter 2.

The material follows the lecture notes for weeks eight to ten of FYS-STK3155/4155.

12.1 Structure

An autoencoder is a network in two halves. The *encoder* $f_\theta : \mathbb{R}^d \to \mathbb{R}^p$ maps an input to a *code* or *latent representation* $\boldsymbol{z}$, and the *decoder* $g_\phi : \mathbb{R}^p \to \mathbb{R}^d$ maps the code back. The two are trained together against the reconstruction error, so that for a data set $\{\boldsymbol{x}_n\}_{n=1}^N$ we minimise

$$\boxed{\mathscr{L}(\boldsymbol{\theta}, \phi) = \frac{1}{N} \sum_{n=1}^{N} \left\| \boldsymbol{x}_n - g_\phi(f_\theta(\boldsymbol{x}_n)) \right\|_2^2 .} \tag{12.2}$$

No labels appear. The target of example n is example n, which is why the method is called *self-supervised*: the supervision is manufactured from the data. Figure 12.1 draws the network in the notation of Section 8.5: it is the feed-forward network of Figure 8.5 with two changes, a narrow layer in the middle and a target that is the input.

The layer that produces $\boldsymbol{z}$ is the *bottleneck*, and its width p is the single most important architectural choice. Three regimes are worth naming.

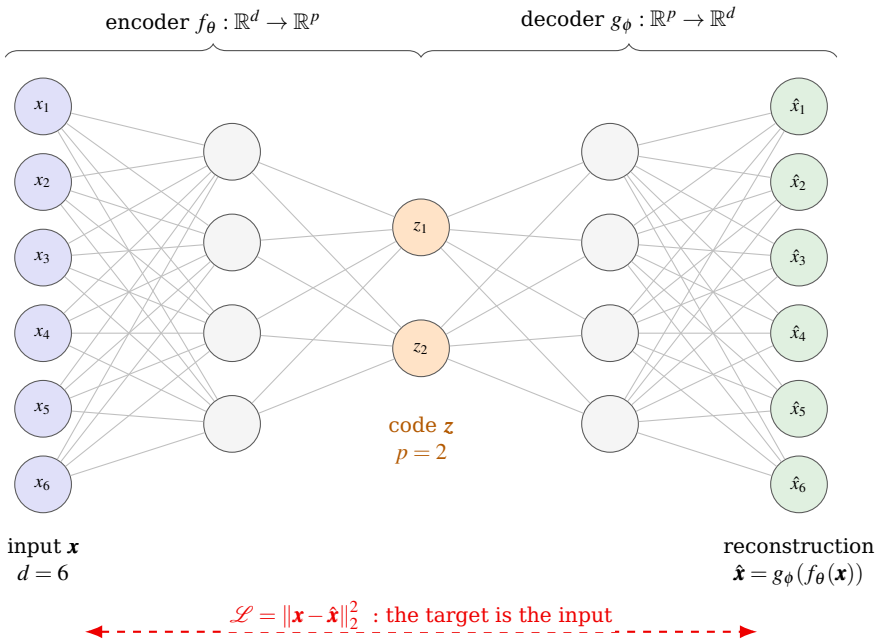


Fig. 12.1 An undercomplete autoencoder in the layer notation of Section 8.5, with $d = 6$ and $p = 2$. Read left to right it is an ordinary feed-forward network of Figure 8.5; what makes it an autoencoder is the dashed arrow at the bottom, which says that the target of example n is example n itself. No labels enter anywhere, which is the content of the word *self-supervised*. Everything the network knows about $\boldsymbol{x}$ after the encoder must pass through the two orange units, and the reconstruction is built from those two numbers alone.

- *Undercomplete*, $p < d$. The code cannot hold the input, so the network must decide what to discard. This is the case in which reconstruction forces compression, and it is the case the first half of this chapter analyses exactly.
- *Complete*, $p = d$. Nothing is forced.
- *Overcomplete*, $p \geq d$. The identity is available: $f = g = \mathrm{id}$ gives zero error and learns nothing. Something other than the bottleneck must supply the constraint, and Section 12.6 is about what.

Figure 12.2 contrasts the three. In practice the two halves are usually *mirrored*: a decoder whose layer widths are those of the encoder in reverse. This is a convention rather than a requirement, but it is a sensible one, and it makes the parameter counts of the two halves match.

Activations and the cost.

The hidden layers use whatever Chapter 8 recommends. The *output* layer is the one that needs thought, and it should be chosen to match the range of the data.

If the inputs are unbounded real numbers, the output activation is the identity and the cost is Eq. (12.2). If they have been scaled to $[0,1]$ – pixel intensities, most commonly – a logistic output is natural, and the binary cross-entropy

$$\mathscr{L}_{\mathrm{BCE}} = -\frac{1}{N}\sum_{n=1}^{N}\sum_{j=1}^{d}\left[x_{nj}\log\hat{x}_{nj} + (1-x_{nj})\log(1-\hat{x}_{nj})\right] \tag{12.3}$$

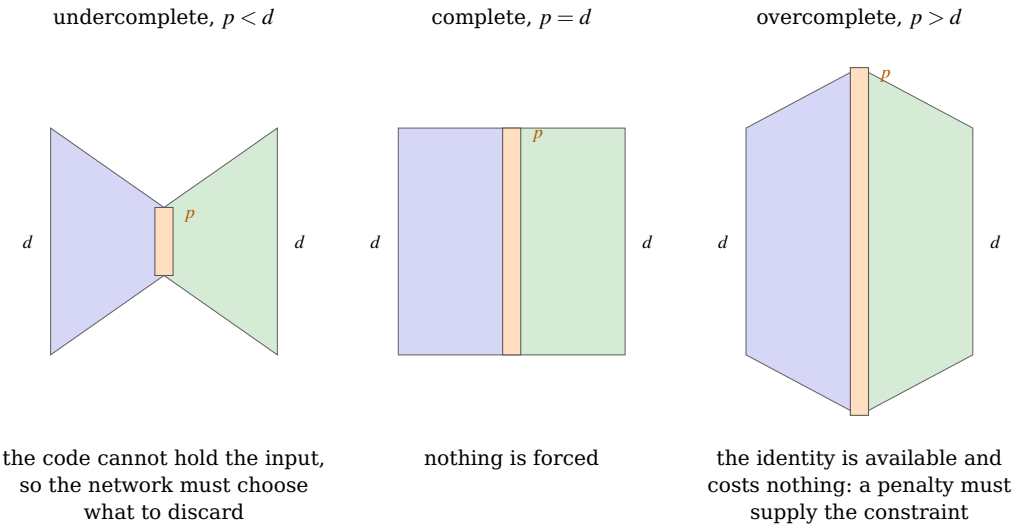


Fig. 12.2 The three regimes of the bottleneck width, drawn as the width of the representation against depth. Only the first compresses. In the second and third the map $g \circ f = \mathrm{id}$ is available and achieves zero reconstruction error while learning nothing whatever about the data, so the constraint that makes the network useful has to come from somewhere other than the architecture – which is what Section 12.6 supplies. The orange bar is the code layer in each case.

is then often preferred to the squared error, because it is the negative log-likelihood of a Bernoulli model and it penalises confident mistakes near the ends of the range much more heavily. A ReLU output is appropriate when the data are non-negative and unbounded, and a linear output is wrong for bounded data because it can predict outside the range.

12.2 Reconstruction, projection and representation

Before the analysis it is worth being precise about what is being asked. The optimisation is over function classes: let $\mathscr{F}$ be a class of encoders and $\mathscr{G}$ a class of decoders, and solve

$$\min_{f \in \mathscr{F},\, g \in \mathscr{G}} \sum_{n=1}^{N} \|\boldsymbol{x}_n - g(f(\boldsymbol{x}_n))\|_2^2 . \tag{12.4}$$

If $\mathscr{F}$ and $\mathscr{G}$ are linear this is a constrained quadratic problem with a closed-form solution. If they are neural networks it is non-convex and highly expressive, and we are back in the territory of Chapter 4.

A theme that will recur is the decomposition

$$\left\|\boldsymbol{X} - \hat{\boldsymbol{X}}\right\|_F^2 = \underbrace{\text{total variance}}_{\text{fixed by the data}} - \underbrace{\text{explained variance}}_{\text{what the code captures}}, \tag{12.5}$$

which says that minimising reconstruction error and maximising captured variance are the same problem seen from two sides. For linear autoencoders this is exact and we prove it below; Figure 12.3 is the proof in one picture, since the decomposition is Pythagoras' theorem applied at each data point and summed.

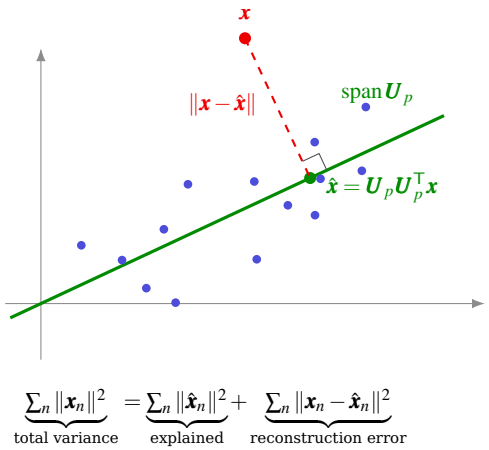


Fig. 12.3 What a linear autoencoder does, in $d = 2$ with $p = 1$, on centred data – which is why the subspace passes through the origin. Lemma 12.1 says that the best reconstruction of a given rank can always be taken to be an orthogonal projection, so $\hat{\boldsymbol{x}}$ is the foot of the perpendicular from $\boldsymbol{x}$ to the subspace, and Theorem 12.5 identifies the subspace that minimises the total squared residual as the principal one. The right angle is the whole of Eq. (12.5): Pythagoras at every data point, summed, splits the total variance into an explained part and a reconstruction error, so making one small is exactly making the other large. A nonlinear autoencoder replaces the straight line by a curve and changes nothing else in the picture.

Reconstruction is not representation. Equation (12.2) scores the network on $\hat{\boldsymbol{x}}$, not on $\boldsymbol{z}$. Nothing in it asks the code to be interpretable, smooth, disentangled or ordered, and Section 12.5 shows an example in which the reconstruction is excellent and the code is nevertheless a poor coordinate for the data. A low reconstruction error is evidence that the information survived the bottleneck. It is not evidence that it survived in a useful form.

12.3 The linear autoencoder

Take the encoder and decoder to be linear and without biases,

$$\boldsymbol{z} = \boldsymbol{W}_e^{\mathsf{T}} \boldsymbol{x}, \qquad \hat{\boldsymbol{x}} = \boldsymbol{W}_d^{\mathsf{T}} \boldsymbol{z}, \qquad \boldsymbol{W}_e \in \mathbb{R}^{d \times p},\ \boldsymbol{W}_d \in \mathbb{R}^{p \times d}, \tag{12.6}$$

so that the reconstruction map is $\hat{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$ with $\boldsymbol{A} = \boldsymbol{W}_d^{\mathsf{T}} \boldsymbol{W}_e^{\mathsf{T}}$. Since $\boldsymbol{A}$ factors through $\mathbb{R}^p$,

$$\operatorname{rank}(\boldsymbol{A}) \le p. \tag{12.7}$$

Dropping the biases costs nothing provided the data are centred, which we assume throughout: replace $\boldsymbol{x}_n$ by $\boldsymbol{x}_n - \bar{\boldsymbol{x}}$. With the data arranged as the rows of $\boldsymbol{X} \in \mathbb{R}^{N \times d}$, the training problem is

$$\min_{\boldsymbol{W}_e, \boldsymbol{W}_d} \|\boldsymbol{X} - \boldsymbol{X}\boldsymbol{A}\|_F^2 \;=\; \min_{\operatorname{rank}(\boldsymbol{A}) \le p} \|\boldsymbol{X} - \boldsymbol{X}\boldsymbol{A}\|_F^2 . \tag{12.8}$$

The equality is the first substantive step: *any* matrix of rank at most p can be written as $\boldsymbol{W}_d^{\mathsf{T}} \boldsymbol{W}_e^{\mathsf{T}}$ by taking a rank factorisation, so optimising over the two factors is the same as optimising over their product subject to the rank bound. Linear autoencoder training is a rank-constrained approximation problem.

Write $\boldsymbol{S} = \boldsymbol{X}^{\mathsf{T}} \boldsymbol{X} / N$ for the empirical covariance, and let its eigendecomposition be

$$\boldsymbol{S} = \boldsymbol{U} \boldsymbol{\Lambda} \boldsymbol{U}^{\mathsf{T}}, \qquad \lambda_1 \ge \lambda_2 \ge \cdots \ge \lambda_d \ge 0, \tag{12.9}$$

with $\boldsymbol{U}$ orthogonal and $\boldsymbol{U}_p = [\boldsymbol{u}_1, \dots, \boldsymbol{u}_p]$ its first p columns. Recall from Chapter 1 that $\boldsymbol{S}$ is symmetric positive semi-definite, so such a decomposition exists with real non-negative eigenvalues.

12.3.1 Reduction to an orthogonal projector

Lemma 12.1 (The optimum may be taken to be an orthogonal projector). *Let $\boldsymbol{A}$ have rank at most p and let $\boldsymbol{P}$ be the orthogonal projector onto the row space of $\boldsymbol{A}$ (equivalently, onto the image of $\boldsymbol{A}^\top$). Then* $\operatorname{rank}(\boldsymbol{P}) \le p$ *and*

$$\|\boldsymbol{X} - \boldsymbol{X}\boldsymbol{P}\|_F^2 \;\le\; \|\boldsymbol{X} - \boldsymbol{X}\boldsymbol{A}\|_F^2 . \tag{12.10}$$

Proof. Let $\mathscr{M}$ be the row space of $\boldsymbol{A}$, of dimension at most p. For each n, the row $\boldsymbol{x}_n^\top \boldsymbol{A}$ lies in $\mathscr{M}$, and $\boldsymbol{x}_n^\top \boldsymbol{P}$ is by definition the point of $\mathscr{M}$ closest to $\boldsymbol{x}_n$ in the Euclidean norm. Hence

$$\|\boldsymbol{x}_n - \boldsymbol{P}\boldsymbol{x}_n\|_2^2 \le \left\|\boldsymbol{x}_n - \boldsymbol{A}^\top \boldsymbol{x}_n\right\|_2^2 \qquad \text{for every } n,$$

and summing over n gives Eq. (12.10), the Frobenius norm being the sum over rows of the squared Euclidean norms.

Lemma 12.1 is what makes the whole analysis possible. It replaces an optimisation over all rank-p matrices, a set with $p(2d-p)$ parameters and no useful structure, by an optimisation over rank-p orthogonal projectors, which are parameterised by a Grassmannian – that is, by a choice of subspace and nothing else. Every orthogonal projector of rank p has the form

$$\boldsymbol{P} = \boldsymbol{B}\boldsymbol{B}^\top, \qquad \boldsymbol{B} \in \mathbb{R}^{d \times p}, \quad \boldsymbol{B}^\top \boldsymbol{B} = \boldsymbol{I}_p, \tag{12.11}$$

with $\boldsymbol{B}$ an orthonormal basis of the subspace.

12.3.2 Reduction to a trace maximisation

Lemma 12.2 (Error equals variance not captured). *With $\boldsymbol{P} = \boldsymbol{B}\boldsymbol{B}^\top$ as in Eq. (12.11),*

$$\|\boldsymbol{X} - \boldsymbol{X}\boldsymbol{P}\|_F^2 = N\left[\operatorname{Tr}(\boldsymbol{S}) - \operatorname{Tr}\left(\boldsymbol{B}^\top \boldsymbol{S}\boldsymbol{B}\right)\right]. \tag{12.12}$$

Minimising the reconstruction error is therefore equivalent to maximising $\operatorname{Tr}(\boldsymbol{B}^\top \boldsymbol{S}\boldsymbol{B})$ *subject to $\boldsymbol{B}^\top \boldsymbol{B} = \boldsymbol{I}_p$.*

Proof. Expand the norm as a trace and use $\boldsymbol{P}^\top = \boldsymbol{P}$ and $\boldsymbol{P}^2 = \boldsymbol{P}$:

$$\|\boldsymbol{X} - \boldsymbol{X}\boldsymbol{P}\|_F^2 = \operatorname{Tr}\Big[(\boldsymbol{I} - \boldsymbol{P})\boldsymbol{X}^\top \boldsymbol{X}(\boldsymbol{I} - \boldsymbol{P})\Big] = N \operatorname{Tr}[(\boldsymbol{I} - \boldsymbol{P})\boldsymbol{S}(\boldsymbol{I} - \boldsymbol{P})],$$

using $\boldsymbol{X}^\top \boldsymbol{X} = N\boldsymbol{S}$. Idempotence and cyclicity of the trace give $\operatorname{Tr}[(\boldsymbol{I} - \boldsymbol{P})\boldsymbol{S}(\boldsymbol{I} - \boldsymbol{P})] = \operatorname{Tr}[\boldsymbol{S}(\boldsymbol{I} - \boldsymbol{P})] = \operatorname{Tr}(\boldsymbol{S}) - \operatorname{Tr}(\boldsymbol{S}\boldsymbol{P})$, and finally $\operatorname{Tr}(\boldsymbol{S}\boldsymbol{B}\boldsymbol{B}^\top) = \operatorname{Tr}(\boldsymbol{B}^\top \boldsymbol{S}\boldsymbol{B})$.

Equation (12.12) is Eq. (12.5) made precise: $\operatorname{Tr}(\boldsymbol{S})$ is the total variance, fixed by the data, and $\operatorname{Tr}(\boldsymbol{B}^\top \boldsymbol{S}\boldsymbol{B})$ is the variance captured by the subspace.

12.3.3 The variational characterisation

It remains to maximise $\operatorname{Tr}(\boldsymbol{B}^{\top}\boldsymbol{S}\boldsymbol{B})$. We do the case $p=1$ by a Lagrange multiplier, which is where the eigenvalue problem appears, and then state the general case.

Proposition 12.3 (The first direction). *The maximiser of $\boldsymbol{w}^{\top}\boldsymbol{S}\boldsymbol{w}$ subject to $\|\boldsymbol{w}\|_2=1$ is the eigenvector $\boldsymbol{u}_1$ of $\boldsymbol{S}$ belonging to the largest eigenvalue λ_1, and the maximum equals λ_1.*

Proof. Without the constraint the objective is unbounded, since scaling $\boldsymbol{w}$ scales it quadratically; the constraint is therefore essential. Impose it with a multiplier and maximise

$$J(\boldsymbol{w})=\boldsymbol{w}^{\top}\boldsymbol{S}\boldsymbol{w}+\lambda\left(1-\boldsymbol{w}^{\top}\boldsymbol{w}\right).$$

By the matrix-calculus identities of Section 1.8,

$$\frac{\partial J}{\partial \boldsymbol{w}}=2\boldsymbol{S}\boldsymbol{w}-2\lambda\boldsymbol{w}=\boldsymbol{0}\quad\Longleftrightarrow\quad \boldsymbol{S}\boldsymbol{w}=\lambda\boldsymbol{w},$$

so every stationary point is an eigenvector of $\boldsymbol{S}$. Left-multiplying by $\boldsymbol{w}^{\top}$ and using $\|\boldsymbol{w}\|=1$ gives $\boldsymbol{w}^{\top}\boldsymbol{S}\boldsymbol{w}=\lambda$: the value of the objective at a stationary point *is* the corresponding eigenvalue. The maximum is therefore attained at the largest one.

The direction that maximises the variance of the projected data is an eigenvector of the covariance matrix, and the variance it captures is the eigenvalue. That single sentence is the content of principal component analysis.

Theorem 12.4 (Ky Fan). *Let $\boldsymbol{S}$ be symmetric with eigenvalues $\lambda_1\geq\cdots\geq\lambda_d$ and eigenvectors $\boldsymbol{u}_1,\dots,\boldsymbol{u}_d$. Then*

$$\max_{\boldsymbol{B}^{\top}\boldsymbol{B}=\boldsymbol{I}_p}\operatorname{Tr}\left(\boldsymbol{B}^{\top}\boldsymbol{S}\boldsymbol{B}\right)=\sum_{i=1}^{p}\lambda_i, \tag{12.13}$$

attained at $\boldsymbol{B}=\boldsymbol{U}_p$, and at $\boldsymbol{B}=\boldsymbol{U}_p\boldsymbol{Q}$ for any orthogonal $\boldsymbol{Q}\in\mathbb{R}^{p\times p}$.

Proof. Write $\boldsymbol{C}=\boldsymbol{U}^{\top}\boldsymbol{B}$, which also satisfies $\boldsymbol{C}^{\top}\boldsymbol{C}=\boldsymbol{I}_p$ since $\boldsymbol{U}$ is orthogonal. Then

$$\operatorname{Tr}\left(\boldsymbol{B}^{\top}\boldsymbol{S}\boldsymbol{B}\right)=\operatorname{Tr}\left(\boldsymbol{C}^{\top}\boldsymbol{\Lambda}\boldsymbol{C}\right)=\sum_{i=1}^{d}\lambda_i m_i,\qquad m_i:=\sum_{j=1}^{p}C_{ij}^2.$$

The coefficients m_i satisfy two constraints. Each row of $\boldsymbol{C}$ has norm at most one, because $\boldsymbol{C}$ has orthonormal columns and may be completed to an orthogonal matrix, so $0\leq m_i\leq 1$; and $\sum_i m_i=\|\boldsymbol{C}\|_F^2=\operatorname{Tr}(\boldsymbol{C}^{\top}\boldsymbol{C})=p$. Maximising the linear functional $\sum_i\lambda_i m_i$ over the polytope $\{0\leq m_i\leq 1,\ \sum_i m_i=p\}$ puts all the available mass on the largest λ_i, giving $m_1=\cdots=m_p=1$ and the rest zero, hence the value $\sum_{i\leq p}\lambda_i$. This is attained by $\boldsymbol{C}=[\boldsymbol{I}_p;\boldsymbol{0}]$, that is $\boldsymbol{B}=\boldsymbol{U}_p$, and the value is unchanged by $\boldsymbol{B}\to\boldsymbol{B}\boldsymbol{Q}$ with $\boldsymbol{Q}$ orthogonal since $\operatorname{Tr}(\boldsymbol{Q}^{\top}\boldsymbol{B}^{\top}\boldsymbol{S}\boldsymbol{B}\boldsymbol{Q})=\operatorname{Tr}(\boldsymbol{B}^{\top}\boldsymbol{S}\boldsymbol{B})$.

12.3.4 The theorem

Assembling Lemmas 12.1 and 12.2 with Theorem 12.4 gives the result.

Theorem 12.5 (Linear autoencoders perform PCA). *Let $\boldsymbol{X}\in\mathbb{R}^{N\times d}$ be centred with empirical covariance $\boldsymbol{S}=\boldsymbol{X}^{\top}\boldsymbol{X}/N$, eigenvalues $\lambda_1\geq\cdots\geq\lambda_d\geq 0$ and eigenvectors $\boldsymbol{U}$. Consider the linear autoencoder (12.6) with bottleneck width $p\leq d$ trained on the reconstruction error (12.2). Then:*

1. *the minimum of the reconstruction error is*

$$\min_{\boldsymbol{W}_e,\boldsymbol{W}_d} \frac{1}{N}\|\boldsymbol{X}-\boldsymbol{X}\boldsymbol{W}_e\boldsymbol{W}_d\|_F^2 = \sum_{i=p+1}^{d} \lambda_i, \tag{12.14}$$

   the sum of the discarded eigenvalues;
2. *at any global minimiser the product satisfies* $\boldsymbol{W}_e\boldsymbol{W}_d = \boldsymbol{U}_p\boldsymbol{U}_p^\top$*, the orthogonal projector onto the leading eigenspace, and the reconstruction is therefore identical to the rank-*p *PCA reconstruction* $\hat{\boldsymbol{x}} = \boldsymbol{U}_p\boldsymbol{U}_p^\top\boldsymbol{x}$*;*
3. *the column space of* $\boldsymbol{W}_e$ *equals* $\operatorname{span}\{\boldsymbol{u}_1,\dots,\boldsymbol{u}_p\}$*, assuming* $\lambda_p > \lambda_{p+1}$*.*

Proof. By Eq. (12.8) the problem is over matrices of rank at most p; by Lemma 12.1 the infimum is attained on orthogonal projectors, which by Eq. (12.11) are parameterised by $\boldsymbol{B}$ with orthonormal columns; by Lemma 12.2 the objective becomes $N[\operatorname{Tr}(\boldsymbol{S}) - \operatorname{Tr}(\boldsymbol{B}^\top\boldsymbol{S}\boldsymbol{B})]$; and by Theorem 12.4 the second trace is maximised at $\sum_{i\le p}\lambda_i$ with $\boldsymbol{B} = \boldsymbol{U}_p\boldsymbol{Q}$. Since $\operatorname{Tr}(\boldsymbol{S}) = \sum_i \lambda_i$, the minimum value is $N\sum_{i>p}\lambda_i$, which after division by N is Eq. (12.14). The optimal projector is $\boldsymbol{B}\boldsymbol{B}^\top = \boldsymbol{U}_p\boldsymbol{Q}\boldsymbol{Q}^\top\boldsymbol{U}_p^\top = \boldsymbol{U}_p\boldsymbol{U}_p^\top$, independent of $\boldsymbol{Q}$, which gives (ii); the gap condition $\lambda_p > \lambda_{p+1}$ makes the maximising subspace unique and gives (iii).

The result deserves a moment's reflection, and one qualification. Theorem 12.5 characterises the *global minimum*; it says nothing about what gradient descent finds, and for a non-convex objective those are different questions. What closes the gap is that this particular objective has no bad places to get stuck.

Proposition 12.6 (The landscape has no spurious minima). *Assume* $\lambda_1 > \cdots > \lambda_d > 0$*. Every critical point of the linear autoencoder objective (12.2) with* $\boldsymbol{W}_e\boldsymbol{W}_d$ *of rank exactly* p *has* $\boldsymbol{W}_e\boldsymbol{W}_d = \boldsymbol{U}_{\mathscr{I}}\boldsymbol{U}_{\mathscr{I}}^\top$ *for some index set* $\mathscr{I} \subset \{1,\dots,d\}$ *with* $|\mathscr{I}| = p$*. Such a point is a global minimum if and only if* $\mathscr{I} = \{1,\dots,p\}$*; every other choice is a saddle. There are no local minima that are not global.*

Proof. Stationarity of Eq. (12.2) in $\boldsymbol{W}_d$ gives $\boldsymbol{W}_e^\top\boldsymbol{S}(\boldsymbol{I}-\boldsymbol{W}_e\boldsymbol{W}_d) = \boldsymbol{0}$ and in $\boldsymbol{W}_e$ gives $\boldsymbol{S}(\boldsymbol{I}-\boldsymbol{W}_e\boldsymbol{W}_d)\boldsymbol{W}_d^\top = \boldsymbol{0}$; together these force $\boldsymbol{\Pi} = \boldsymbol{W}_e\boldsymbol{W}_d$ to satisfy $\boldsymbol{\Pi}\boldsymbol{S} = \boldsymbol{S}\boldsymbol{\Pi} = \boldsymbol{\Pi}\boldsymbol{S}\boldsymbol{\Pi}$, so $\boldsymbol{\Pi}$ is a projector commuting with $\boldsymbol{S}$ and therefore projects onto a span of eigenvectors, indexed by some $\mathscr{I}$. The objective value there is $\sum_{i\notin\mathscr{I}}\lambda_i$ by the argument of Theorem 12.5.

Now suppose $\mathscr{I} \neq \{1,\dots,p\}$, so there are indices $j \notin \mathscr{I}$ and $k \in \mathscr{I}$ with $\lambda_j > \lambda_k$. Consider the curve of projectors obtained by rotating $\boldsymbol{u}_k$ towards $\boldsymbol{u}_j$ by an angle ϑ inside their two-dimensional span; the objective along it is

$$E(\vartheta) = \sum_{i\notin\mathscr{I}} \lambda_i - \sin^2\vartheta\,(\lambda_j - \lambda_k),$$

since the rotation exchanges a fraction $\sin^2\vartheta$ of the weight on λ_k for the same fraction on λ_j. Thus $E'(0) = 0$ – confirming criticality – and $E''(0) = -2(\lambda_j - \lambda_k) < 0$: the point is a strict maximum along this direction while being a minimum along the $GL(p)$ orbit of Proposition 12.7, hence a saddle. Only $\mathscr{I} = \{1,\dots,p\}$ admits no such pair, and by Theorem 12.5 it is the global minimum.

This is what licenses the informal statement that a linear autoencoder "does PCA". A neural network trained by gradient descent on a non-convex objective is guaranteed to land on a classical, spectrally characterised answer – not because the objective is convex, which it is not, but because its only non-global critical points are saddles, which first-order methods with noise leave. It is essentially the only architecture in this book for which we can say that, and Section 12.8.2 measures how well the promise is kept in practice.

Proposition 12.7 (What the autoencoder does *not* recover). *If $(\boldsymbol{W}_e, \boldsymbol{W}_d)$ is a global minimiser then so is $(\boldsymbol{W}_e\boldsymbol{M}, \boldsymbol{M}^{-1}\boldsymbol{W}_d)$ for every invertible $\boldsymbol{M} \in \mathbb{R}^{p\times p}$. Consequently the individual weights are determined only up to $GL(p)$: the autoencoder recovers the principal* subspace*, not the principal* directions*, and its codes are not in general uncorrelated, ordered by variance, or of unit norm.*

Proof. $(\boldsymbol{W}_e\boldsymbol{M})(\boldsymbol{M}^{-1}\boldsymbol{W}_d) = \boldsymbol{W}_e\boldsymbol{W}_d$, so the reconstruction and hence the cost are unchanged.

This is the practical difference between running PCA and training a linear autoencoder, and it is easy to overlook. PCA delivers an orthonormal basis ordered by variance, so that truncating to $q < p$ components is meaningful and the components are uncorrelated by construction. A linear autoencoder delivers some basis of the same subspace, in no particular order, generally not orthogonal. If the directions matter, use PCA; if only the subspace matters, either will do, and PCA is faster.

Relation to Eckart-Young and the SVD. Theorem 12.5 is a close relative of the Eckart-Young theorem of Section 1.18, which states that the best rank-p approximation to $\boldsymbol{X}$ in the Frobenius norm is obtained by truncating its singular value decomposition, with error $\sum_{i>p}\sigma_i^2$. The two are connected by $\sigma_i^2 = N\lambda_i$: the singular values of the centred data matrix are the square roots of N times the eigenvalues of the covariance. The difference is that Eckart-Young approximates $\boldsymbol{X}$ by *any* low-rank matrix, while the autoencoder is constrained to approximate each $\boldsymbol{x}_n$ by a fixed linear map applied to $\boldsymbol{x}_n$ itself. That the two give the same answer is a consequence of Lemma 12.1. In practice one computes PCA by the SVD of $\boldsymbol{X}$ rather than by eigendecomposing $\boldsymbol{S}$, for the conditioning reason given in Section 1.18: forming $\boldsymbol{X}^\top\boldsymbol{X}$ squares the condition number.

12.4 Verifying the theorem

The chapter's code is the network of Chapter 8 with the target set equal to the input. Nothing else changes, which is the point.

```
def ae_forward(P, X, acts):
    """Returns the reconstruction and the cache; X has shape (N, d)."""
    A = [X]; Z = []
    a = X
    for l, (W, b) in enumerate(P):
        z = a @ W + b
        Z.append(z)
        a = ACT[acts[l]][0](z)
        A.append(a)
    return a, (A, Z)


def ae_backward(P, cache, Xhat, X, acts):
    """Backpropagation with the target equal to the input, Eq. (12.cost)."""
    A, Z = cache
    n = X.shape[0]
    g = [[np.zeros_like(W), np.zeros_like(b)] for W, b in P]
    delta = (Xhat - X) / n * ACT[acts[-1]][1](Z[-1])
    for l in reversed(range(len(P))):
        g[l][0] = A[l].T @ delta
        g[l][1] = delta.sum(axis=0)
        if l > 0:
            delta = (delta @ P[l][0].T) * ACT[acts[l - 1]][1](Z[l - 1])
    return g
```

Checking every parameter against central differences, for three architectures:

```
  [6, 3, 6]          ['identity', 'identity']
      max relative error 8.71e-09   max absolute error 6.93e-10
      (9 entries had gradient 0 and were checked absolutely)
  [6, 4, 2, 4, 6]    ['tanh', 'tanh', 'tanh', 'sigmoid']
      max relative error 1.91e-06   max absolute error 6.19e-10
  [6, 3, 6]          ['relu', 'identity']
      max relative error 2.12e-08   max absolute error 7.35e-10
```

A relative check is meaningless on a gradient that is zero. The first line above reports nine parameters excluded from the relative comparison. They are the biases of a linear autoencoder on centred data, whose gradients are *exactly* zero – the analytic value came out as 4.9×10^{-17} and the central difference as -4.4×10^{-10}, pure subtraction noise. Their ratio is $\mathcal{O}(1)$ and means nothing. This is the same lesson as the ReLU kink of Section 10.9.2, in a different disguise: a gradient checker needs an absolute test as well as a relative one, and needs to know which to apply where.

Now the theorem. We generate 600 points in $\mathbb{R}^8$ lying near a three-dimensional subspace, plus isotropic noise, and train a $[8,3,8]$ linear autoencoder by Adam. Against the predictions of Theorem 12.5 and Proposition 12.7:

```
AE  reconstruction error : 0.308313
PCA reconstruction error : 0.306324
eigenvalue tail          : 0.306324
AE/PCA error ratio       : 1.006493
principal angles (deg)   : [0.0684 0.1851 0.318 ]
||W_e W_d - Pi||_F       : 9.895e-03
singular values of W_e W_d: [1.0033 1.0007 0.9989 1.166e-16 6.462e-17
                             5.420e-17 3.645e-17 1.840e-18]
||W_e - U_p||_F          : 1.8737  (NOT small)
dist(W_e, span U_p)      : 2.466e-03  (small: same subspace)
```

Every line is a prediction confirmed. The PCA reconstruction error and the eigenvalue tail agree to all six digits printed, which is Eq. (12.14). The autoencoder gets to within 0.65% of that floor and does not cross it. The singular values of $\boldsymbol{W}_e\boldsymbol{W}_d$ are three ones and five zeros *to machine precision*, so the trained product really is an orthogonal projector of rank three, as Lemma 12.1 said it would be. The principal angles between the learned subspace and the leading eigenspace are a fifth of a degree.

And the last two lines are Proposition 12.7 in numbers. The distance from $\boldsymbol{W}_e$ to $\boldsymbol{U}_p$ is 1.87, which is not small – the weights are nowhere near the eigenvectors – while the distance from $\boldsymbol{W}_e$ to the *span* of $\boldsymbol{U}_p$ is 2.5×10^{-3}. Same subspace, different basis. A reader who trained a linear autoencoder expecting to read principal directions off the encoder weights would get nonsense, and the reconstruction error would give no hint of it.

12.5 Beyond PCA

PCA finds the best linear subspace. Many data sets do not lie near one: images of a rotating object, molecular conformations, solutions of a nonlinear partial differential equation parameterised by a boundary condition, speech. These lie near a curved manifold, and no linear subspace of the manifold's dimension will follow it.

Replacing the linear maps of Eq. (12.6) by the deep networks of Chapter 8,

$$\boldsymbol{z}=f_\theta(\boldsymbol{x})=\sigma_L(\boldsymbol{W}_L\sigma_{L-1}(\cdots\sigma_1(\boldsymbol{W}_1\boldsymbol{x}+\boldsymbol{b}_1)\cdots)+\boldsymbol{b}_L),\qquad \hat{\boldsymbol{x}}=g_\phi(\boldsymbol{z}), \tag{12.15}$$

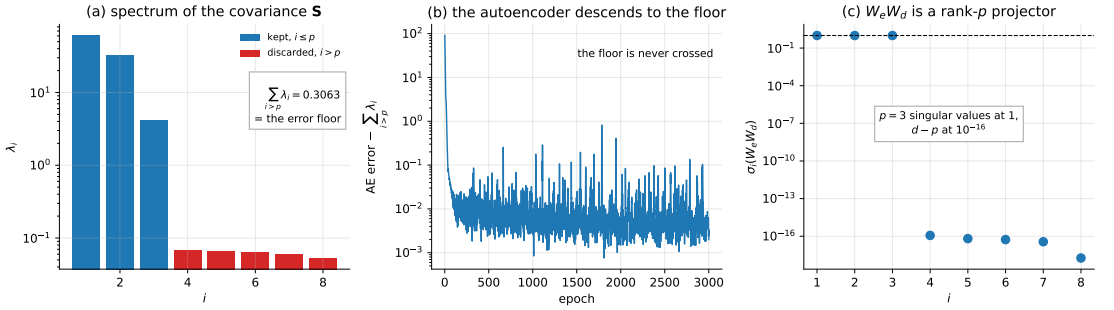


Fig. 12.4 (a) The eigenvalue spectrum of $\boldsymbol{S}$ on a logarithmic scale; the red bars are the discarded eigenvalues whose sum is the error floor of Eq. (12.14). (b) The autoencoder's reconstruction error minus that floor during training: it approaches zero from above and never crosses, as Theorem 12.5 guarantees. (c) Singular values of the trained product $\boldsymbol{W}_e\boldsymbol{W}_d$: exactly $p = 3$ of them equal one and the remaining $d - p = 5$ are at the level of rounding, confirming that the learned map is an orthogonal projector of rank p.

buys the ability to bend. It also costs the entire analysis of Section 12.3: the problem becomes non-convex, no closed form is available, and we are back to gradient descent with no guarantee of a global optimum.

What replaces the projector.

If f and g are differentiable then near a point $\boldsymbol{x}_0$, with $\boldsymbol{z}_0 = f(\boldsymbol{x}_0)$,

$$g(f(\boldsymbol{x})) \approx g(f(\boldsymbol{x}_0)) + \boldsymbol{J}_g(\boldsymbol{z}_0)\boldsymbol{J}_f(\boldsymbol{x}_0)\,(\boldsymbol{x} - \boldsymbol{x}_0)\,, \tag{12.16}$$

so the product of Jacobians $\boldsymbol{J}_g\boldsymbol{J}_f$ is the local reconstruction operator. It plays the role that $\boldsymbol{W}_e\boldsymbol{W}_d$ played in the linear case, with the crucial difference that it now *varies from point to point*. A nonlinear autoencoder is, to first order, a projector onto the tangent space of the data manifold, chosen afresh at every location.

Measuring the difference.

We put 800 training and 400 test points on a one-dimensional curve — a helix — embedded in $\mathbb{R}^3$ with small isotropic noise, and compare PCA against a $[3, 16, p, 16, 3]$ autoencoder with tanh hidden layers.

Table 12.1 A one-dimensional curved manifold in $\mathbb{R}^3$. Errors are $\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|^2$ averaged over points, and the autoencoder figures are means over three seeds. The total variance per point is 1.287.

p	Method	train error	test error
1	PCA	0.7855	0.7914
1	nonlinear autoencoder	0.1511	**0.2012**
2	PCA	0.3208	0.3316
2	nonlinear autoencoder	0.0020	**0.0062**

At $p = 1$ the autoencoder is four times more accurate than PCA; at $p = 2$ it is fifty times more accurate. The sharpest way to put it is to compare across rows: the *one*-dimensional

nonlinear code, at 0.2012, beats the *two*-dimensional linear one, at 0.3316. Curvature is worth more than a dimension here, and Theorem 12.5 tells us exactly why PCA cannot do better – its error is fixed by the eigenvalue tail, and no amount of training changes it.

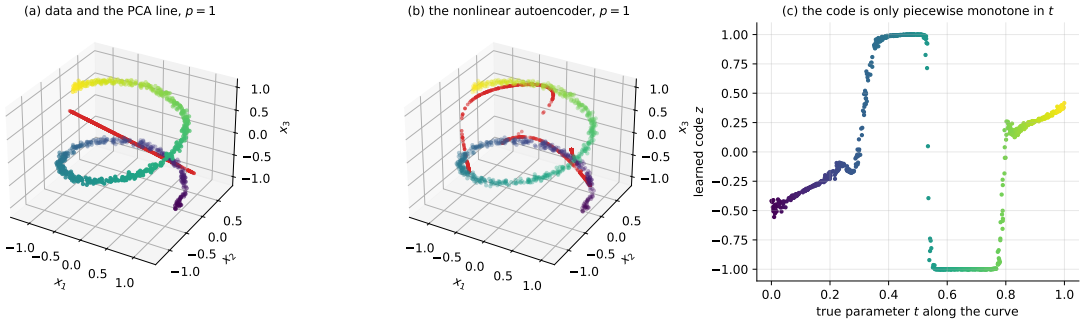


Fig. 12.5 (a) The data, coloured by position along the curve, with the one-dimensional PCA reconstruction in red: a straight line through a helix. (b) The same data with the one-dimensional nonlinear autoencoder reconstruction, which follows the curve. (c) The learned code z against the true parameter t. The relation is monotone in stretches but saturates at ± 1 and jumps: the reconstruction is good and the coordinate is not.

Panel (c) of Figure 12.5 is the honest part, and it is the notebox of Section 12.2 made visible. The autoencoder reconstructs the helix well, but the code it uses to do so is not a faithful coordinate along the curve: it saturates against the limits of the tanh and jumps between branches. Nothing in Eq. (12.2) asked for a good coordinate, so nothing delivered one. If the latent variable is to be interpreted – as a reaction coordinate, an order parameter, a physical parameter of the system – then reconstruction error alone is the wrong objective, and one of the regularisers of the next section, or the probabilistic treatment of Section 12.7, must be brought in.

12.6 Regularised autoencoders

The bottleneck is one way to prevent an autoencoder from learning the identity. It is not the only way, and an *overcomplete* autoencoder with $p \geq d$ needs another, since $g \circ f = \mathrm{id}$ is available and costs nothing. All three constructions below add a penalty to Eq. (12.2) and can replace the bottleneck entirely.

Denoising autoencoders.

Corrupt the input and ask for the clean version back. With $\tilde{\boldsymbol{x}} = \boldsymbol{x} + \boldsymbol{\varepsilon}$,

$$\min_{\theta,\phi} \frac{1}{N} \sum_{n=1}^{N} \left\| \boldsymbol{x}_n - g_\phi\big(f_\theta(\tilde{\boldsymbol{x}}_n)\big) \right\|_2^2 . \tag{12.17}$$

Copying the input is now actively harmful, because the input is wrong. The network must instead learn where the data *are*, so as to move a corrupted point back onto them. There is a striking statistical reading of this: for small Gaussian corruption the optimal reconstruction satisfies

$$r(\boldsymbol{x}) - \boldsymbol{x} \;\approx\; \sigma^2 \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}), \tag{12.18}$$

so the displacement the network applies is proportional to the *score* of the data density. A denoising autoencoder is, without being told so, an estimator of $\nabla \log p$ – which is exactly the object that score-based and diffusion generative models are built on.

Sparse autoencoders.

Penalise the code rather than the reconstruction:

$$\mathscr{L} = \frac{1}{N}\sum_n \|\boldsymbol{x}_n - \hat{\boldsymbol{x}}_n\|_2^2 + \lambda \sum_n \|\boldsymbol{z}_n\|_1 \,. \tag{12.19}$$

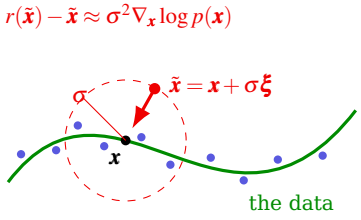
The ℓ_1 penalty is the one that produced exact zeros in Section 3.9, and it does the same here: each input activates only a few latent units, and those units tend to become recognisable feature detectors. The construction is close kin to dictionary learning and sparse coding.

Contractive autoencoders.

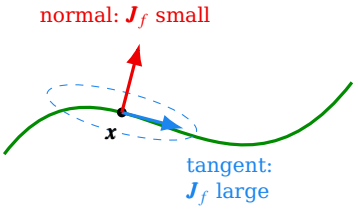
Penalise the sensitivity of the encoder:

$$\mathscr{L} = \frac{1}{N}\sum_n \|\boldsymbol{x}_n - \hat{\boldsymbol{x}}_n\|_2^2 + \lambda \sum_n \left\|\boldsymbol{J}_f(\boldsymbol{x}_n)\right\|_F^2 . \tag{12.20}$$

The two terms pull in opposite directions and the geometry of the compromise is the point. Reconstruction requires the encoder to be sensitive to displacements *along* the data manifold, since those change which point is being encoded. The penalty asks it to be insensitive in *all* directions. The optimum keeps sensitivity where it is needed and suppresses it elsewhere, so that $\boldsymbol{J}_f$ ends up large on the tangent space of the manifold and small on the normal directions. The encoder learns the manifold's tangent structure. Figure 12.6 draws the denoising and the contractive geometry next to one another, which is the quickest way to see why the two are so nearly the same construction.



(a) denoising: the network must learn the map that pushes a corrupted point back onto the data



(b) contraction: the penalty $\lambda\|\boldsymbol{J}_f\|_F^2$ suppresses sensitivity except along the data

Fig. 12.6 The geometry behind the two regularisers of Section 12.6, drawn for data concentrated near a curve in the plane. (a) Denoising, Eq. (12.17): the corrupted input $\tilde{\boldsymbol{x}}$ is displaced off the data by an amount of order σ, and the network is scored on how well it recovers the clean $\boldsymbol{x}$, so it must learn the map that points back towards where the data are. Equation (12.18) identifies that map, to leading order, with the score $\nabla_{\boldsymbol{x}} \log p$. (b) Contraction, Eq. (12.20): reconstruction needs the encoder to respond to displacements *along* the data, while the penalty asks it to respond to nothing at all; the compromise keeps $\boldsymbol{J}_f$ large on the tangent and small on the normal, so the ellipse of responses is flattened onto the manifold. Proposition 12.8 makes the resemblance precise to order σ^2, with the one caveat recorded there: the denoising penalty falls on $\boldsymbol{J}_r$, the sensitivity of the round trip, and not on $\boldsymbol{J}_f$.

The last sentence of the notebox below says these regularisers are “the same idea”. For two of them that is a theorem rather than an impression.

Proposition 12.8 (Denoising is contraction, to second order). *Let $r = g \circ f$ be the reconstruction map and let the denoising objective (12.17) use additive noise $\tilde{\boldsymbol{x}} = \boldsymbol{x} + \sigma\boldsymbol{\xi}$ with $\mathbb{E}[\boldsymbol{\xi}] = \mathbf{0}$ and $\mathbb{E}[\boldsymbol{\xi}\boldsymbol{\xi}^{\mathsf{T}}] = \boldsymbol{I}$. If r is twice continuously differentiable then*

$$\mathbb{E}_{\boldsymbol{\xi}} \left\| \boldsymbol{x} - r(\boldsymbol{x} + \sigma\boldsymbol{\xi}) \right\|^2 = \left\| \boldsymbol{x} - r(\boldsymbol{x}) \right\|^2 + \sigma^2 \left\| \boldsymbol{J}_r(\boldsymbol{x}) \right\|_F^2 + \mathcal{O}(\sigma^3), \tag{12.21}$$

where $\boldsymbol{J}_r = \partial r / \partial \boldsymbol{x}$. Training a denoising autoencoder at noise level σ is therefore, to leading order, training a plain autoencoder with the contractive penalty (12.20) at $\lambda = \sigma^2$ – applied to the whole reconstruction rather than to the encoder alone.

Proof. Taylor-expand $r(\boldsymbol{x} + \sigma\boldsymbol{\xi}) = r(\boldsymbol{x}) + \sigma \boldsymbol{J}_r \boldsymbol{\xi} + \mathcal{O}(\sigma^2)$ and substitute:

$$\left\| \boldsymbol{x} - r(\boldsymbol{x}) - \sigma \boldsymbol{J}_r \boldsymbol{\xi} \right\|^2 = \left\| \boldsymbol{x} - r(\boldsymbol{x}) \right\|^2 - 2\sigma \left(\boldsymbol{x} - r(\boldsymbol{x}) \right)^{\mathsf{T}} \boldsymbol{J}_r \boldsymbol{\xi} + \sigma^2 \boldsymbol{\xi}^{\mathsf{T}} \boldsymbol{J}_r^{\mathsf{T}} \boldsymbol{J}_r \boldsymbol{\xi}.$$

Taking the expectation, the middle term vanishes because $\mathbb{E}[\boldsymbol{\xi}] = \mathbf{0}$, and the last one is $\sigma^2 \operatorname{Tr}(\boldsymbol{J}_r^{\mathsf{T}} \boldsymbol{J}_r) = \sigma^2 \|\boldsymbol{J}_r\|_F^2$, because $\mathbb{E}[\boldsymbol{\xi}\boldsymbol{\xi}^{\mathsf{T}}] = \boldsymbol{I}$. The neglected terms are $\mathcal{O}(\sigma^3)$ since the $\mathcal{O}(\sigma^2)$ term of the expansion enters the square multiplied by the first-order one.

Equation (12.21) explains why the two constructions behave so similarly in practice, and it also explains why they are not identical: the denoising penalty falls on $\boldsymbol{J}_r$, the sensitivity of the round trip, while Eq. (12.20) penalises $\boldsymbol{J}_f$, the sensitivity of the encoder alone. A decoder that amplifies can hide from the second and not from the first. It also predicts what the noise level buys: doubling σ quadruples the effective λ, so the reconstruction error should degrade smoothly rather than sharply, which is what Section 12.8.5 finds.

These are all the same idea. A bottleneck restricts the code by dimension; sparsity restricts it by how many coordinates may be non-zero; contraction restricts it by how fast it may vary; denoising restricts it by requiring robustness. In each case the network is prevented from being the identity and must spend its capacity on structure instead. This is the bias-variance trade-off of Section 2.10 appearing once more, with the regulariser choosing which functions are cheap.

12.7 The probabilistic view: PPCA

Everything so far has been deterministic: a code $\boldsymbol{z} = f(\boldsymbol{x})$ and a reconstruction $\hat{\boldsymbol{x}} = g(\boldsymbol{z})$. A probabilistic autoencoder replaces these by a latent-variable model $p_\theta(\boldsymbol{x}, \boldsymbol{z}) = p_\theta(\boldsymbol{x} \mid \boldsymbol{z}) p(\boldsymbol{z})$, turning representation learning into statistical inference of the kind Chapter 2 set up.

The linear case again has a complete answer. *Probabilistic PCA* assumes

$$\boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}_p), \qquad \boldsymbol{x} = \boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu} + \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \boldsymbol{I}_d), \tag{12.22}$$

so that $p(\boldsymbol{x} \mid \boldsymbol{z}) = \mathcal{N}(\boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu}, \sigma^2 \boldsymbol{I}_d)$. Integrating out the latent variable, which is a Gaussian marginalisation and therefore exact,

$$\boldsymbol{x} \sim \mathcal{N}\left(\boldsymbol{\mu},\, \boldsymbol{W}\boldsymbol{W}^{\mathsf{T}} + \sigma^2 \boldsymbol{I}_d \right). \tag{12.23}$$

The model therefore describes the covariance as a low-rank term plus isotropic noise, $\boldsymbol{\Sigma} = \boldsymbol{W}\boldsymbol{W}^{\mathsf{T}} + \sigma^2 \boldsymbol{I}$, which is the probabilistic statement of “low-dimensional structure plus measurement error”.

Maximum-likelihood estimation in this model, worked out by Tipping and Bishop [53], gives a loading matrix whose column space is exactly the principal subspace, and $\hat{\sigma}^2$ equal to the average of the discarded eigenvalues,

$$\hat{\sigma}^2 = \frac{1}{d-p} \sum_{i=p+1}^{d} \lambda_i. \tag{12.24}$$

Compare Eq. (12.14): the same eigenvalue tail that bounded the autoencoder's reconstruction error reappears as the estimated noise variance. As $\sigma^2 \to 0$ the model degenerates to the deterministic projection, and PPCA becomes ordinary PCA. Once again $\boldsymbol{W}$ is determined only up to an orthogonal factor, for the reason given in Proposition 12.7.

The gain is what a probabilistic model always gives: a likelihood, hence a principled way to choose p; a generative model one can sample from; and a proper treatment of missing data. Replacing the linear map in Eq. (12.22) by a neural network, and the exact marginalisation by a variational bound, gives the variational autoencoder, which belongs to a later discussion of generative models.

12.8 Implementations

12.8.1 Our own

The complete autoencoder is the code of Section 12.4 with a list of layer widths and activations. A deep autoencoder on the helix of Table 12.1 is four lines:

```
acts = ["tanh", "tanh", "tanh", "identity"]
P = init_ae([3, 16, 1, 16, 3], acts, np.random.default_rng(0))
P, hist = train_ae(P, X, acts, n_epoch=600, batch=32, gamma=5e-3, Xval=Xte)
Z = encode(P, X, acts, layer=2)          # the code: run the first two layers
```

Setting `acts = ["identity", "identity"]` and `sizes = [d, p, d]` recovers the linear autoencoder of Theorem 12.5, and `pca(X, p)` in the same module returns the projector, the loadings and the eigenvalues for comparison. The two are worth running side by side once.

Centre the data. Theorem 12.5 assumes it, and PCA without it finds the direction of the mean rather than the direction of greatest variance. Scikit-learn's `PCA` centres internally; our `pca` does so too; a hand-written implementation or a neural network without biases does not. If the features have incomparable units, standardise as well as centre, or the covariance will be dominated by whichever feature happens to be measured in small units.

12.8.2 PyTorch and TensorFlow

Two things are worth doing in a library. The first is to check that it computes what we derived: one set of weights pushed into our code, into PyTorch and into Keras should give the same reconstruction and the same gradients. The second, and the more interesting, is to put Theorem 12.5 and Proposition 12.6 to the test – to train a linear autoencoder by Adam and ask whether it really lands on the principal subspace, and whether it really fails to land on the principal basis.

The conversion is trivial here, which is worth saying because it was not in Chapters 10 and 11: Keras stores a dense weight as $(n_{\text{in}}, n_{\text{out}})$, exactly as our code does, and only PyTorch transposes. With that one transposition:

```
=== 1. one autoencoder, identical weights ===
  reconstruction max |ours - torch| : 3.886e-16
  reconstruction max |ours - keras| : 3.608e-16
  our cost, Eq. (12.cost)           : 5.781582466015
  torch cost                        : 5.781582466015
   layer      shape          |ours - torch| W    |ours - torch| b
   0   (12, 8)                  2.220e-16          6.765e-17
   1   (8, 3)                   2.220e-16          5.551e-17
   2   (3, 8)                   1.110e-16          1.388e-16
   3   (8, 12)                  8.327e-17          5.898e-17
  worst disagreement: 2.220e-16
```

so the network of Section 12.8.1 and the two libraries are the same function, and its gradients are the same gradients.

The listings below are the standard constructions. The first is a linear autoencoder, which should reproduce PCA.

```
import numpy as np
import tensorflow as tf
from tensorflow import keras

# three-dimensional data lying near a plane
np.random.seed(4)
def generate_3d_data(m, w1=0.1, w2=0.3, noise=0.1):
    angles = np.random.rand(m) * 3 * np.pi / 2 - 0.5
    data = np.empty((m, 3))
    data[:, 0] = np.cos(angles) + np.sin(angles)/2 + noise*np.random.randn(m)/2
    data[:, 1] = np.sin(angles) * 0.7 + noise * np.random.randn(m) / 2
    data[:, 2] = data[:, 0]*w1 + data[:, 1]*w2 + noise*np.random.randn(m)
    return data

X_train = generate_3d_data(60)
X_train = X_train - X_train.mean(axis=0, keepdims=True)   # centre: see the notebox

# no activations anywhere: this is Eq. (12.linae)
encoder = keras.models.Sequential([keras.layers.Dense(2, input_shape=[3])])
decoder = keras.models.Sequential([keras.layers.Dense(3, input_shape=[2])])
autoencoder = keras.models.Sequential([encoder, decoder])
autoencoder.compile(loss="mse", optimizer=keras.optimizers.SGD(learning_rate=1.5))
autoencoder.fit(X_train, X_train, epochs=200, verbose=0)   # target == input

codings = encoder.predict(X_train)
# compare with PCA: the subspaces should agree, the bases need not
U, s, Vt = np.linalg.svd(X_train, full_matrices=False)
print("PCA loadings:\n", Vt[:2].T)
print("encoder weights:\n", encoder.layers[0].get_weights()[0])
```

The final two prints are Proposition 12.7 in action: the two matrices will differ, and their column spaces will not.

A stacked, nonlinear autoencoder on MNIST, with a mirrored decoder and a logistic output to match the $[0,1]$ data:

```
(X_train_full, _), (X_test, _) = keras.datasets.mnist.load_data()
X_train_full = X_train_full.astype(np.float32) / 255
X_test = X_test.astype(np.float32) / 255
X_train, X_valid = X_train_full[:-5000], X_train_full[-5000:]

stacked_encoder = keras.models.Sequential([
```

```
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(100, activation="selu"),
    keras.layers.Dense(30, activation="selu"),          # the bottleneck, p = 30
])
stacked_decoder = keras.models.Sequential([             # the exact mirror
    keras.layers.Dense(100, activation="selu", input_shape=[30]),
    keras.layers.Dense(28 * 28, activation="sigmoid"),  # data live in [0,1]
    keras.layers.Reshape([28, 28]),
])
stacked_ae = keras.models.Sequential([stacked_encoder, stacked_decoder])
stacked_ae.compile(loss="binary_crossentropy",          # Eq. (12.bce)
                   optimizer=keras.optimizers.Adam(1e-3))
stacked_ae.fit(X_train, X_train, epochs=10,
               validation_data=(X_valid, X_valid))
```

The convolutional version reuses Chapter 10 without modification. The encoder is a stack of strided convolutions, which by Proposition 10.4 halve the spatial size each time; the decoder mirrors it with transposed convolutions, which are the adjoint operation discussed in the notebox of Section 10.8.

```
# Encoder: (28,28,1) -> (14,14,16) -> (7,7,32) -> (4,4,64), Eq. (10.outsize)
conv_encoder = keras.models.Sequential([
    keras.layers.Reshape([28, 28, 1], input_shape=[28, 28]),
    keras.layers.Conv2D(16, 3, strides=2, padding="SAME", activation="selu"),
    keras.layers.Conv2D(32, 3, strides=2, padding="SAME", activation="selu"),
    keras.layers.Conv2D(64, 3, strides=2, padding="SAME", activation="selu"),
])
# Decoder: the exact mirror, transposed convolutions doubling the size
conv_decoder = keras.models.Sequential([
    keras.layers.Conv2DTranspose(32, 3, strides=2, padding="SAME",
                                 activation="selu", input_shape=[4, 4, 64]),
    keras.layers.Conv2DTranspose(16, 3, strides=2, padding="SAME",
                                 activation="selu"),
    keras.layers.Conv2DTranspose(1, 3, strides=2, padding="SAME",
                                 activation="sigmoid"),
    keras.layers.Lambda(lambda x: x[:, :28, :28, :]),   # crop 32 -> 28
    keras.layers.Reshape([28, 28]),
])
conv_ae = keras.models.Sequential([conv_encoder, conv_decoder])
conv_ae.compile(loss="binary_crossentropy",
                optimizer=keras.optimizers.Adam(1e-3))
```

A recurrent autoencoder reuses Chapter 11 in the same way, reading each image as a sequence of 28 rows. The encoder's final state is the code; `RepeatVector` hands it to the decoder once per output step.

```
recurrent_encoder = keras.models.Sequential([
    keras.layers.LSTM(100, return_sequences=True, input_shape=[28, 28]),
    keras.layers.LSTM(30),                          # final state = the code
])
recurrent_decoder = keras.models.Sequential([
    keras.layers.RepeatVector(28, input_shape=[30]),
    keras.layers.LSTM(100, return_sequences=True),
    keras.layers.TimeDistributed(keras.layers.Dense(28, activation="sigmoid")),
])
recurrent_ae = keras.models.Sequential([recurrent_encoder, recurrent_decoder])
recurrent_ae.compile(loss="binary_crossentropy",
                     optimizer=keras.optimizers.Adam(1e-3))
```

The same dense autoencoder in PyTorch, where the mirror symmetry is explicit:

```
import torch
import torch.nn as nn
```

```
from torchvision import datasets, transforms

train_loader = torch.utils.data.DataLoader(
    datasets.MNIST("./data", train=True, download=True,
                   transform=transforms.ToTensor()),
    batch_size=128, shuffle=True)


class Autoencoder(nn.Module):
    """784 -> 256 -> 128 -> 256 -> 784, an exact mirror; Eq. (12.factor)."""
    def __init__(self, x_dim=784, h_dim=256, z_dim=128):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(x_dim, h_dim), nn.BatchNorm1d(h_dim), nn.ReLU(),
            nn.Linear(h_dim, z_dim), nn.BatchNorm1d(z_dim), nn.ReLU(),
        )
        self.decoder = nn.Sequential(
            nn.Linear(z_dim, h_dim), nn.BatchNorm1d(h_dim), nn.ReLU(),
            nn.Linear(h_dim, x_dim), nn.Sigmoid(),       # data in [0,1]
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))


model = Autoencoder()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.MSELoss()

for epoch in range(1, 101):
    model.train()
    total = 0.0
    for data, _ in train_loader:
        inputs = data.view(-1, 784)
        loss = loss_fn(model(inputs), inputs)            # target == input
        optimizer.zero_grad(set_to_none=True)
        loss.backward()
        optimizer.step()
        total += loss.item()
    if epoch % 10 == 0:
        print(f"epoch {epoch:3d}  train {total/len(train_loader):.6f}")
```

Turning this into a denoising autoencoder, Eq. (12.17), is one line: replace the forward call by `model(inputs + 0.3*torch.randn_like(inputs))` while leaving the target as `inputs`. A sparse autoencoder, Eq. (12.19), adds `lam * model.encoder(inputs).abs().sum()` to the loss. Neither requires a new architecture, which is the point of Section 12.6.

12.8.3 Does Adam find the principal subspace?

Theorem 12.5 is about the global minimum and Proposition 12.6 says there is nowhere else to get stuck. Together they predict that a linear autoencoder trained by any reasonable optimiser will land on the principal subspace. We put eight-dimensional data with a three-dimensional signal through a $8 \to 3 \to 8$ linear autoencoder, train with Adam, and then compare the encoder's column space with the span of the top three eigenvectors. The right measure is the *principal angles* between two subspaces: the arccosines of the singular values of $\boldsymbol{Q}_1^{\top}\boldsymbol{Q}_2$ for orthonormal bases $\boldsymbol{Q}_i$, of which the largest is a complete summary.

```
=== 1. Theorem 12.aepca: does Adam find the principal subspace? ===
  largest principal angle to the PCA subspace : 0.0154 degrees
```

```
max |W_enc - V_pca| entrywise              : 1.3393
reconstruction cost per sample             : 0.155593
Eckart-Young floor, Eq. (12.floor)         : 0.155593
excess over the floor                      : 5.108e-08
||Pi_enc - Pi_pca||_F                      : 4.592e-04
||Pi^2 - Pi||_F, trace Pi                  : 1.30e-07, 3.000000
```

Read the second line against the first. The subspace is recovered to a hundredth of a degree and the reconstruction cost sits on the Eckart-Young floor of Eq. (12.14) to eight decimal places – and the weight matrix is nowhere near the PCA loadings, differing entrywise by 1.34 on entries of order one. That is Proposition 12.7 in a single pair of numbers: the minimiser is fixed only up to an invertible $p \times p$ factor, and gradient descent has no reason at all to prefer the orthonormal representative.

What *is* determined is the projector. The last two lines confirm that $\boldsymbol{\Pi} = \boldsymbol{W}_e \boldsymbol{W}_e^{+}$ is idempotent to 10^{-7}, has trace exactly $p = 3$ as Eq. (12.11) requires, and agrees with $\boldsymbol{U}_p \boldsymbol{U}_p^{\mathsf{T}}$ to 5×10^{-4} in Frobenius norm. So the useful thing to compare between two runs of a linear autoencoder, or between a linear autoencoder and PCA, is never the weights: it is the projector, or equivalently the subspace.

12.8.4 What the bottleneck costs, and what the nonlinearity buys

The theorem gives the exact floor for a *linear* autoencoder at every width, Eq. (12.14). That makes it the natural baseline for the nonlinear case: PCA is not one method among several here, it is the best any linear map of that rank can do, so whatever a nonlinear autoencoder gains over it is precisely what the nonlinearity is worth. On MNIST, with a $784 \to 256 \to p \to 256 \to 784$ network against a $784 \to p \to 784$ linear one, eight epochs of Adam each:

Table 12.2 Mean squared reconstruction error per pixel on the 10000 MNIST test images, against the code dimension p. The PCA column is Eq. (12.14) evaluated on the training set and is a floor for the linear autoencoder, not a competitor to it. The last column is the ratio of the first to the third.

		linear autoencoder		nonlinear autoencoder	
p	PCA (linear optimum)	PyTorch	TensorFlow	PyTorch	TensorFlow
2	0.05595	0.05778	0.05780	0.04522	0.04609
8	0.03787	0.03775	0.03772	0.03177	0.02070
16	0.02729	0.02706	0.02703	0.02043	0.01384
32	0.01724	0.01692	0.01697	0.01041	0.00860
64	0.00928	0.00917	0.00916	0.00656	0.00534

Three readings. The linear autoencoder tracks the PCA floor closely and it does so in *both* frameworks – the two linear columns agree with each other to the fourth decimal at every width, which is Theorem 12.5 again on a real data set. At $p = 2$ both sit *above* the floor, at 0.0578 against 0.05595, because eight epochs of Adam on 784×2 weights have not quite converged. The theorem is about the optimum; reaching it is still an optimisation problem.

The two nonlinear columns, by contrast, differ by up to a third. They are the same architecture and the same optimiser, so the difference is initialisation and the order in which batches arrive: a non-convex objective with no Proposition 12.6 to protect it. That contrast between the two halves of the table is worth more than either half alone – the linear model has a theorem and reproduces across frameworks, the nonlinear one has neither.

The nonlinear autoencoder beats the floor at every width, in both frameworks. It has to be at least as good: no linear map can do better than PCA at that rank, so any improvement is evidence that the data lie near a curved manifold rather than a subspace, and the size of the improvement is a crude measure of how curved.

The ratio of the PCA column to the nonlinear one is not monotonic, and both frameworks agree on the shape. In PyTorch it runs 1.24, 1.19, 1.34, 1.66, 1.42; in TensorFlow 1.21, 1.83, 1.97, 2.01, 1.74. Both rise into the middle of the range and fall off at $p = 64$. At small p both models are starved and the error is dominated by what neither can represent; in the middle the nonlinearity is doing its most useful work; and by $p = 64$ the linear model has enough directions that the remaining error is fine detail which the nonlinear network, at this width and after eight epochs, does not capture much better. The nonlinearity buys most where the bottleneck is tight enough to force a choice and loose enough to permit a good one.

12.8.5 The regularised variants, measured

Section 12.6 introduced three regularisers and argued they were variations on one idea; Proposition 12.8 made part of that precise. Running them on the same $p = 32$ architecture:

```
=== 3. denoising and sparse autoencoders, p = 32 ===
   variant                     test mse   mean |code|   active units
   plain                         0.01041        6.7410           20.9
   denoising, sigma = 0.3        0.01052        6.0570           22.9
   denoising, sigma = 0.6        0.01299        4.4208           23.6
   sparse, lambda = 0.01         0.01757        0.2375           11.3
   sparse, lambda = 0.1          0.06750        0.0000            0.0
```

Denoising at $\sigma = 0.3$ costs almost nothing in reconstruction, 0.01052 against 0.01041, and at $\sigma = 0.6$ costs a quarter of it. That is the smooth degradation Eq. (12.21) predicts: the effective contractive weight is σ^2, so doubling the noise quadruples the penalty and the error rises gently rather than falling off a cliff.

The sparse rows are more interesting, and the last one is a failure we report rather than tune away. At $\lambda = 0.01$ the penalty does what it is supposed to: the mean code magnitude falls from 6.74 to 0.24, the number of active units from 21 to 11, and the reconstruction error rises by two thirds. At $\lambda = 0.1$ the code collapses completely – *zero* active units, mean magnitude 0.0000 – and the reconstruction error of 0.0675 is what a network that ignores its input and predicts the mean image achieves. The penalty has won outright. With a ReLU code this is an absorbing state: once every pre-activation is negative the code is exactly zero, the ℓ_1 gradient is zero, and nothing pushes the units back. A regulariser strong enough to be worth having is close to one strong enough to destroy the model, and the distance between them here is one order of magnitude in λ.

12.9 Summary and the programs

An autoencoder approximates the identity through a constrained factorisation, and everything interesting follows from the constraint rather than from the objective.

The linear case is solved completely. Lemma 12.1 reduces the rank-constrained problem to orthogonal projectors, Lemma 12.2 turns reconstruction error into variance not captured, and Theorem 12.4 maximises that variance at the leading eigenvectors. Theorem 12.5 assembles these: the minimum error is the eigenvalue tail $\sum_{i>p} \lambda_i$ and the learned map is the PCA projector. We measured all of it – the error floor reproduced to six digits, the singular values

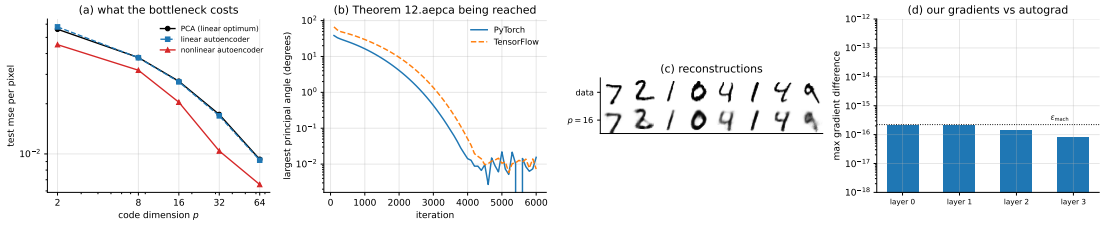


Fig. 12.7 (a) Reconstruction error against code dimension on MNIST, Table 12.2: PCA is the floor for any linear map of that rank, the linear autoencoder tracks it, and the nonlinear one beats it by a margin that grows with p. (b) The largest principal angle between the encoder subspace and the span of the top p eigenvectors, against training iteration, in both frameworks: Theorem 12.5 being reached, not assumed. (c) Test images and their reconstructions at $p = 16$. (d) The largest disagreement between our gradients and each framework's, against the double-precision unit roundoff.

of $\boldsymbol{W}_e\boldsymbol{W}_d$ equal to three ones and five zeros to machine precision, the subspaces agreeing to a fifth of a degree.

Proposition 12.7 is the caveat to carry away. The factorisation is invariant under $\boldsymbol{W}_e \to \boldsymbol{W}_e\boldsymbol{M}$, $\boldsymbol{W}_d \to \boldsymbol{M}^{-1}\boldsymbol{W}_d$, so the autoencoder identifies the principal *subspace* and not the principal *directions*. Measured: $\|\boldsymbol{W}_e - \boldsymbol{U}_p\|_F = 1.87$, while the distance from $\boldsymbol{W}_e$ to the span of $\boldsymbol{U}_p$ is 2.5×10^{-3}. If the directions matter, use PCA.

Proposition 12.6 is what turns the theorem into a statement about training. The objective is not convex, but its only non-global critical points are saddles – projections onto the wrong set of eigenvectors, each with an explicit direction of descent – so a first-order method has nowhere bad to stop. Section 12.8.3 confirms the whole chain in one run: Adam reaches the principal subspace to 0.015 degrees and the Eckart-Young floor to eight decimals, while missing the principal basis by 1.34 per entry.

Nonlinearity buys curvature, at the cost of every guarantee. On a helix in $\mathbb{R}^3$ a one-dimensional nonlinear code beat a two-dimensional linear one, 0.201 against 0.332, and at equal p the margin was a factor of four to fifty. On MNIST, Table 12.2, the nonlinear autoencoder beat the exact linear floor at every code dimension and in both frameworks, with the gain largest in the middle of the range rather than at either end. But Figure 12.5(c) showed the reconstruction to be excellent while the code was a poor coordinate, saturating and jumping. Equation (12.2) scores $\hat{\boldsymbol{x}}$ and never mentions $\boldsymbol{z}$, so a good representation is something one must ask for explicitly – by denoising, sparsity, contraction, or by moving to a probabilistic model.

The regularisers were measured too, and one of them broke. Proposition 12.8 shows that denoising at noise level σ is contraction at strength σ^2 to second order, and the measurements degrade accordingly and gently. The ℓ_1 penalty does not: at $\lambda = 0.01$ it halves the number of active code units for a two-thirds rise in error, and at $\lambda = 0.1$ it drives every unit to exactly zero and the model to predicting the mean image. With a ReLU code that collapse is absorbing, and one order of magnitude in λ separates useful from fatal.

The programs are in the directory
`doc/BookML/BookPrograms/chapter12_autoencoders`.
Every listing above appears there as a numbered file, and three modules run start to finish and reproduce the numbers quoted in the text:

- `ae.py` – the forward and backward passes, Adam, the encode helper, and `pca` for comparison.
- `verify_ae.py` – the gradient checks of Section 12.4 and the linear-autoencoder-versus-PCA comparison, including the principal angles and the singular values of $\boldsymbol{W}_e\boldsymbol{W}_d$.
- `cross_check_ae.py` – the three-way comparison of Section 12.8.2.

- `ae_torch.py` and `ae_tf.py` – the principal-subspace test of Section 12.8.3, the MNIST bottleneck sweep of Table 12.2, and the regularised variants of Section 12.8.5.
- `mnist_data.py` – a loader that reads MNIST from a local `.npz`, so that the programs run without network access.
- `run_nonlinear.py` – the helix experiment of Table 12.1.

The figures are generated by `ch12_figures.py` in `doc/BookML/BookFigures`; neither is drawn by hand.

12.10 Exercises

Warm-up exercises

1. **The rank factorisation.** Show that every $\boldsymbol{A} \in \mathbb{R}^{d\times d}$ with $\operatorname{rank}(\boldsymbol{A}) \le p$ can be written as $\boldsymbol{W}_e\boldsymbol{W}_d$ with $\boldsymbol{W}_e \in \mathbb{R}^{d\times p}$ and $\boldsymbol{W}_d \in \mathbb{R}^{p\times d}$. This is the step that turns Eq. (12.8) into a statement about ranks rather than about networks.
2. **Projection is optimal.** Prove directly that for a subspace $\mathscr{M}$ and a point $\boldsymbol{x}$, the minimiser of $\|\boldsymbol{x}-\boldsymbol{m}\|_2$ over $\boldsymbol{m} \in \mathscr{M}$ is the orthogonal projection, and that the residual is orthogonal to $\mathscr{M}$. Where exactly is this used in Lemma 12.1?
3. **Trace identities.** Verify each step of Lemma 12.2, stating which property of the trace or of $\boldsymbol{P}$ you use at each equality. Then show that $\operatorname{Tr}(\boldsymbol{S}) = \sum_i \lambda_i$ for symmetric $\boldsymbol{S}$.
4. **Ky Fan by hand.** For $d = 3$, $p = 2$ and $\boldsymbol{\Lambda} = \operatorname{diag}(5,3,1)$, verify Theorem 12.4 by direct computation, and exhibit two different $\boldsymbol{B}$ attaining the maximum. What is the set of all maximisers?
5. **The non-uniqueness.** Train a linear autoencoder twice from different seeds and compare (a) the reconstruction errors, (b) the products $\boldsymbol{W}_e\boldsymbol{W}_d$, (c) the individual matrices $\boldsymbol{W}_e$. Which two agree and which does not, and why? Then find the matrix $\boldsymbol{M}$ of Proposition 12.7 relating the two runs and check that $\boldsymbol{W}_e^{(1)}\boldsymbol{M} \approx \boldsymbol{W}_e^{(2)}$.
6. **Overcomplete.** Build an autoencoder with $p > d$ and no regularisation, and confirm that it drives the reconstruction error to zero while learning nothing. Exhibit the weights that achieve this exactly. Then add each of the three regularisers of Section 12.6 and report what changes.
7. **The landscape.** (a) Derive the two stationarity conditions used in the proof of Proposition 12.6 and show they force $\boldsymbol{\Pi}$ to commute with $\boldsymbol{S}$. (b) For $d = 3$, $p = 1$ and $\lambda = (3,2,1)$, list all three critical projectors, give their objective values, and exhibit the descent direction at each of the two that are not optimal. (c) What goes wrong with the proposition if $\lambda_p = \lambda_{p+1}$?
8. **Principal angles.** (a) Show that the largest principal angle between two p-dimensional subspaces is zero if and only if they coincide, and that it equals $\arccos \sigma_{\min}(\boldsymbol{Q}_1^\top \boldsymbol{Q}_2)$. (b) Show $\|\boldsymbol{\Pi}_1 - \boldsymbol{\Pi}_2\|_F = \sqrt{2}\,\|\sin\boldsymbol{\Theta}\|_F$ for the vector of principal angles $\boldsymbol{\Theta}$, and check the identity against the numbers quoted in Section 12.8.3. (c) Why is the principal angle, rather than $\|\boldsymbol{W}_e - \boldsymbol{U}_p\|$, the right way to compare two runs?
9. **Denoising and contraction.** (a) Derive Eq. (12.21), being explicit about where $\mathbb{E}[\boldsymbol{\xi}] = \boldsymbol{0}$ and $\mathbb{E}[\boldsymbol{\xi}\boldsymbol{\xi}^\top] = \boldsymbol{I}$ are used. (b) Repeat for masking noise, where each coordinate is zeroed independently with probability q, and identify the penalty that appears. (c) Verify Eq. (12.21) numerically on a trained autoencoder by computing both sides at several σ and confirming the σ^2 slope.
10. **The sparse collapse.** Reproduce the last row of Section 12.8.5. (a) Show that with a ReLU code the all-zero state is a fixed point of the ℓ_1-penalised gradient, so the collapse cannot undo itself. (b) Find the largest λ that still leaves the code alive, to within a factor of

two. (c) Replace ReLU by tanh or by a soft-thresholding activation and report whether the collapse survives.

11. **Beating PCA.** Reproduce Table 12.2 and extend it. (a) At $p = 2$ the linear autoencoder is above the exact floor. Train it longer and report how many epochs are needed to reach the floor to three digits. (b) The gain column is not monotonic. Test the explanation offered in Section 12.8.4 by widening the hidden layers and repeating. (c) Add a convolutional autoencoder using Chapter 10 and put it in the same table. Does the convolutional prior help at fixed p?

Project-style exercise: autoencoders and PCA

Part a: the machinery.

Implement the autoencoder forward and backward passes and verify every gradient against central differences for at least three architectures, including a purely linear one. You will find parameters whose gradient is exactly zero; explain which and why, and make your checker handle them correctly.

Part b: the theorem.

Reproduce Section 12.4 on data of your own construction. Confirm Eq. (12.14) to as many digits as your optimiser will give; measure the principal angles; and confirm that the singular values of $\boldsymbol{W}_e\boldsymbol{W}_d$ are p ones and $d-p$ zeros. Then break the assumption: make $\lambda_p = \lambda_{p+1}$ exactly and report what happens to part (iii) of the theorem.

Part c: how the floor is approached.

Study the convergence of the linear autoencoder to $\sum_{i>p}\lambda_i$ as a function of the learning rate, the initialisation scale, and the eigenvalue gap $\lambda_p - \lambda_{p+1}$. Saddle points are known to exist in this landscape corresponding to subsets of eigenvectors other than the leading ones; can you find one, and how long does the optimiser linger there?

Part d: curvature.

Reproduce Table 12.1, then vary the curvature of the manifold continuously – for instance by interpolating between a straight line and a helix – and plot the PCA and autoencoder errors against the curvature. At what point does the nonlinear model start to pay for itself?

Part e: representation, not reconstruction.

Reproduce Figure 12.5(c) and quantify how badly the code parameterises the curve, for instance by the Spearman correlation between z and t. Then try to fix it: a wider bottleneck, a different activation, a contractive penalty, more training. Report which of these help and which merely lower the reconstruction error while leaving the code as bad as before. This is the central lesson of the chapter and it deserves evidence.

Part f: the regularisers.

Implement the denoising, sparse and contractive objectives of Eqs. (12.17)–(12.20). For the contractive case you will need $\|\boldsymbol{J}_f\|_F^2$; obtain it by automatic differentiation and verify it against finite differences. Compare the three on an overcomplete architecture where the bottleneck cannot help.

Part g: convolutional and recurrent.

Build a convolutional autoencoder from your Chapter 10 code and a recurrent one from your Chapter 11 code, on the same data. Check the decoder's output sizes against Proposition 10.4 before training. Compare all three architectures at matched parameter counts and say which inductive bias the data actually reward.

Part h: probabilistic.

Implement PPCA, Eq. (12.22), by maximum likelihood, and verify Eq. (12.24) numerically. Use the likelihood to select p on data where you know the true latent dimension, and compare against choosing p by the reconstruction-error elbow. Which is more reliable, and does either recover the truth?

Chapter 13
Transformers

Each architecture in this book has been a restriction of the dense layer, justified by a property of the data. Chapter 10 imposed locality and translation equivariance and obtained convolution; Chapter 11 imposed sequential processing with a carried state and obtained recurrence. In each case a *fixed* interaction pattern was written into the architecture before any data were seen: a convolutional layer couples a pixel to its neighbours whatever the image contains, and a recurrent layer couples each step to the previous one whatever the sequence says.

The transformer takes the opposite decision. It couples every element to every other element, and it lets the *strength* of each coupling be computed from the data at run time. The interaction pattern is not designed; it is inferred, afresh for every input. That single change of principle – from fixed couplings to *adaptive* couplings – is what this chapter is about, and it turns out to have consequences that are mathematically sharp, physically suggestive, and occasionally limiting in ways worth knowing.

This chapter follows the transformer lecture notes for the course. We derive attention, prove the properties that characterise it, implement it from scratch with verified gradients, and measure two things that matter: why the $1/\sqrt{d_k}$ in Eq. (13.5) is not cosmetic, and what a single attention layer provably cannot compute.

13.1 Tokens

A transformer consumes a sequence of vectors,

$$\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n, \qquad \boldsymbol{x}_i \in \mathbb{R}^d, \tag{13.1}$$

called *tokens*, and returns another sequence of the same length, $(\boldsymbol{y}_1, \ldots, \boldsymbol{y}_n)$. What a token *is* depends on the problem: a word in a sentence, a patch of an image, a node of a discretised field, or a sample of a solution to a partial differential equation. Collect them as the rows of $\boldsymbol{X} \in \mathbb{R}^{n \times d}$.

The generality is the point. Anything that can be cut into pieces and given a vector description can be handed to a transformer, and the architecture makes no assumption about how the pieces are arranged. As Theorem 13.2 will prove, it makes *no* assumption about their order either, which is a liberation and a problem in equal measure.

13.2 Scaled dot-product attention

For each position i we want an output that is a weighted combination of information drawn from all positions,

$$\boldsymbol{y}_i = \sum_{j=1}^{n} A_{ij}\, \boldsymbol{v}_j, \tag{13.2}$$

with weights A_{ij} that depend on the data. Three learned linear maps supply the ingredients. From each token we form a *query*, a *key* and a *value*,

$$\boldsymbol{q}_i = \boldsymbol{W}_Q^{\top}\boldsymbol{x}_i, \qquad \boldsymbol{k}_j = \boldsymbol{W}_K^{\top}\boldsymbol{x}_j, \qquad \boldsymbol{v}_j = \boldsymbol{W}_V^{\top}\boldsymbol{x}_j, \tag{13.3}$$

with $\boldsymbol{W}_Q, \boldsymbol{W}_K \in \mathbb{R}^{d\times d_k}$ and $\boldsymbol{W}_V \in \mathbb{R}^{d\times d_v}$. The reading is worth memorising: the query is what position i is looking for, the key is what position j advertises, and the value is what position j contributes if selected.

The weight is the softmax of the query-key inner product,

$$A_{ij} = \frac{\exp(s_{ij})}{\sum_{\ell}\exp(s_{i\ell})}, \qquad s_{ij} = \frac{\boldsymbol{q}_i \cdot \boldsymbol{k}_j}{\sqrt{d_k}}. \tag{13.4}$$

In matrix form, with $\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q$, $\boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K$ and $\boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V$,

$$\boxed{\operatorname{Attention}(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}) = \operatorname{softmax}\!\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\top}}{\sqrt{d_k}}\right)\boldsymbol{V}.} \tag{13.5}$$

The $n\times n$ matrix $\boldsymbol{Q}\boldsymbol{K}^{\top}$ holds the interaction between every pair of positions; the softmax normalises each row; multiplication by $\boldsymbol{V}$ aggregates. When $\boldsymbol{Q}$, $\boldsymbol{K}$ and $\boldsymbol{V}$ all come from the same $\boldsymbol{X}$, as here, this is *self*-attention.

13.2.1 What attention is

Three statements characterise Eq. (13.5), and each has a consequence that is easy to state and easy to forget.

Proposition 13.1 (Attention averages). *The matrix $\boldsymbol{A}$ of Eq. (13.4) is* row-stochastic*: $A_{ij} > 0$ and $\sum_j A_{ij} = 1$ for every i. Consequently each output $\boldsymbol{y}_i$ lies in the convex hull of the value vectors $\{\boldsymbol{v}_1,\dots,\boldsymbol{v}_n\}$.*

Proof. Positivity is immediate from the exponential, and the denominator of Eq. (13.4) is exactly the sum of the numerators over j, so the rows sum to one. A combination with non-negative coefficients summing to one is by definition a convex combination, and Eq. (13.2) exhibits $\boldsymbol{y}_i$ as one.

The consequence is a genuine limitation. A single attention layer cannot produce an output outside the convex hull of its own values: it can select, blend and interpolate, but it cannot extrapolate. This is one reason the feed-forward sublayer of Section 13.4 is not an optional extra – without it a stack of attention layers would be confined to convex hulls forever.

Theorem 13.2 (Permutation equivariance). *Let $\boldsymbol{\Pi}$ be an $n\times n$ permutation matrix. Then self-attention satisfies*

$$\operatorname{Attention}(\boldsymbol{\Pi}\boldsymbol{X}) = \boldsymbol{\Pi}\operatorname{Attention}(\boldsymbol{X}). \tag{13.6}$$

The output at a position depends on the set *of tokens and on which token sits at that position, but not on the order of the others.*

Proof. The three maps of Eq. (13.3) act on each row independently, so $\boldsymbol{\Pi X}$ produces $\boldsymbol{\Pi Q}$, $\boldsymbol{\Pi K}$ and $\boldsymbol{\Pi V}$. The score matrix becomes $(\boldsymbol{\Pi Q})(\boldsymbol{\Pi K})^{\mathsf{T}}/\sqrt{d_k} = \boldsymbol{\Pi S \Pi}^{\mathsf{T}}$. A row-wise softmax commutes with a permutation of the rows, and permuting the columns of a row permutes the entries of that row's softmax identically, so $\operatorname{softmax}(\boldsymbol{\Pi S \Pi}^{\mathsf{T}}) = \boldsymbol{\Pi}\operatorname{softmax}(\boldsymbol{S})\boldsymbol{\Pi}^{\mathsf{T}}$. Finally $\boldsymbol{\Pi A \Pi}^{\mathsf{T}}\boldsymbol{\Pi V} = \boldsymbol{\Pi A V}$, using $\boldsymbol{\Pi}^{\mathsf{T}}\boldsymbol{\Pi} = \boldsymbol{I}$.

Compare Theorem 10.7, which said convolution is the unique linear map equivariant to *translation*. Attention is equivariant to the much larger group of *all* permutations, and a larger symmetry group means a weaker model: attention knows nothing about order at all. For a sentence or a time series that is unacceptable, and Section 13.3 puts the order back in by hand.

Proposition 13.3 (Cost). *Computing Eq. (13.5) requires $\mathcal{O}(n^2 d_k + n^2 d_v)$ arithmetic operations and $\mathcal{O}(n^2)$ memory for the matrix $\boldsymbol{A}$.*

Proof. $\boldsymbol{QK}^{\mathsf{T}}$ is a product of an $n \times d_k$ and a $d_k \times n$ matrix, costing $\mathcal{O}(n^2 d_k)$; the softmax is $\mathcal{O}(n^2)$; and $\boldsymbol{AV}$ costs $\mathcal{O}(n^2 d_v)$.

The quadratic scaling in sequence length is the central practical difficulty of the architecture, and the reason for a large literature on sparse, low-rank and kernelised approximations. Contrast a convolution, which by Eq. (10.18) touches only F^2 neighbours per position, and a recurrent layer, which is linear in n but strictly sequential and therefore cannot be parallelised across time. Attention buys global access and parallelism, and pays for both in n^2.

13.2.2 Why the $1/\sqrt{d_k}$

The scaling in Eq. (13.5) looks like a detail. It is not, and the reason can be made precise.

Proposition 13.4 (Variance of the logits). *Let $\boldsymbol{q}, \boldsymbol{k} \in \mathbb{R}^{d_k}$ have independent entries with mean zero and variance σ^2. Then*

$$\mathbb{E}\left[\boldsymbol{q} \cdot \boldsymbol{k}\right] = 0, \qquad \operatorname{Var}\left[\boldsymbol{q} \cdot \boldsymbol{k}\right] = d_k \sigma^4, \tag{13.7}$$

so the logits have standard deviation $\sigma^2\sqrt{d_k}$, growing without bound with the head dimension. Dividing by $\sqrt{d_k}$ makes the variance independent of d_k.

Proof. $\boldsymbol{q} \cdot \boldsymbol{k} = \sum_r q_r k_r$ is a sum of d_k independent terms, each of mean $\mathbb{E}[q_r]\mathbb{E}[k_r] = 0$ and variance $\mathbb{E}[q_r^2 k_r^2] - 0 = \sigma^4$ by independence. Variances of independent variables add.

Measured with $\sigma^2 = 1$ over 4000 samples:

```
 d_k    Var(q.k) unscaled   Var(q.k) scaled   prediction d_k
   4              4.22            1.0538              4
  16             15.87            0.9916             16
  64             61.48            0.9607             64
 256            251.83            0.9837            256
1024           1011.47            0.9878           1024
```

The middle column tracks the last one over three decades, and the scaled variant sits at one throughout. Figure 13.1(a) plots it.

That is the statistics. The reason it matters is what large logits do to the softmax. As the spread of the scores grows, the softmax approaches a hard maximum: one weight goes to

one, the rest to zero, and the derivative goes to zero with them. Measuring the Frobenius norm of the softmax Jacobian for logits drawn from $\mathcal{N}(0, c^2)$:

```
 scale   max A     entropy    ||d softmax/d s||_F
  0.25  0.06851    3.4214   4.808e-02
  1.00  0.37786    2.4937   1.029e-01
  4.00  0.98926    0.0703   8.223e-03
 16.00  1.00000    0.0000   1.745e-09
 64.00  1.00000    0.0000   3.276e-36
```

Between $c = 1$ and $c = 64$ the gradient falls by *thirty-five orders of magnitude*. A network whose attention weights sit in that regime receives no gradient through them at all and cannot learn what to attend to. Since Proposition 13.4 says c grows as $\sqrt{d_k}$, a model with $d_k = 64$ and unscaled logits starts life at $c \approx 8$, already well into the saturated region. The $1/\sqrt{d_k}$ is what keeps it at $c \approx 1$.

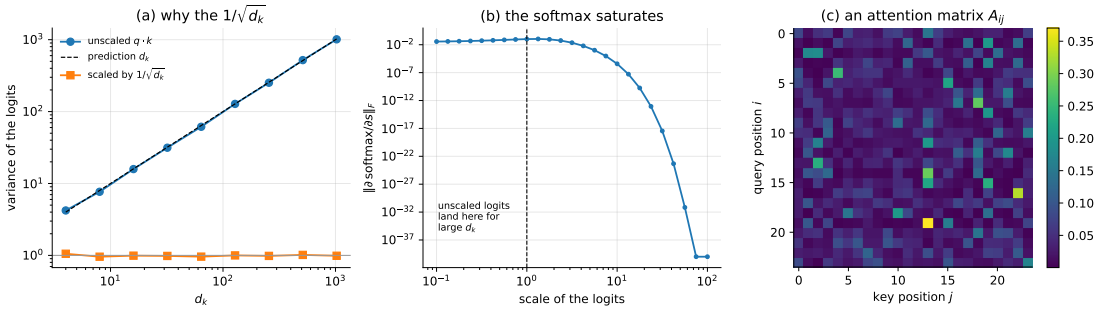


Fig. 13.1 (a) Variance of the attention logits against head dimension. The unscaled dot product follows the prediction d_k of Proposition 13.4 exactly; dividing by $\sqrt{d_k}$ flattens it to one. (b) Frobenius norm of the softmax Jacobian against the scale of the logits: past a scale of about four the softmax saturates and the gradient collapses, reaching 10^{-36} by scale 64. (c) An attention matrix A_{ij} for an untrained network: every row sums to one, by Proposition 13.1.

The same failure as Chapter 11, at a different place. Section 11.5 showed gradients vanishing through a product of Jacobians along a sequence. Here they vanish through a single saturated softmax. Both are cured by keeping a quantity at scale one – the spectral radius there, the logit variance here – and in both cases the cure is a choice of scaling made before training rather than an adjustment made during it. This is the recurring lesson of Chapters 8 to 13: deep networks are trained by controlling scales.

13.3 Masking and position

Masks.

Sometimes a position must not see another. In autoregressive generation, the prediction at step i may depend only on steps $j \le i$, or the model would be allowed to read its own answer. This is imposed by adding a *mask* to the scores before the softmax,

$$s_{ij} \longleftarrow s_{ij} + M_{ij}, \qquad M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i, \end{cases} \tag{13.8}$$

so that $\exp(s_{ij}) = 0$ above the diagonal and $\boldsymbol{A}$ becomes lower triangular. The implementation adds $-\infty$ rather than deleting entries so that the shape of the computation is unchanged.

A mask destroys Theorem 13.2, and deliberately: the whole purpose is to make position i different from position j. We verify both facts in Section 13.5.

Positional encodings.

Theorem 13.2 says a transformer without further information cannot distinguish a sentence from an anagram of it. The standard remedy is to add a position-dependent vector to each token before the first block. The sinusoidal choice of the original architecture is

$$\mathrm{PE}(m,2r) = \sin\left(\frac{m}{10000^{2r/d}}\right), \qquad \mathrm{PE}(m,2r+1) = \cos\left(\frac{m}{10000^{2r/d}}\right), \tag{13.9}$$

for position m and channel r, and $\boldsymbol{x}_m \leftarrow \boldsymbol{x}_m + \mathrm{PE}(m,\cdot)$. The construction is a bank of sinusoids at geometrically spaced frequencies, so that low channels vary slowly across the sequence and high channels vary fast – a positional analogue of the multiscale argument of Section 10.6.

Its useful property is usually stated loosely – that the inner product $\mathrm{PE}(m)\cdot\mathrm{PE}(m')$ depends mainly on $m-m'$. It is in fact exact, and the exactness is the whole reason for choosing sinusoids rather than any other bank of position-dependent features.

Proposition 13.5 (The sinusoidal encoding is a relative code). *Write $\omega_r = 10000^{-2r/d}$ for $r = 0,\dots,d/2-1$. For the encoding Eq. (13.9),*

1. (shift is a fixed rotation) *for every offset k there is an orthogonal matrix $\boldsymbol{R}_k$, independent of m, with*
$$\mathrm{PE}(m+k) = \boldsymbol{R}_k\,\mathrm{PE}(m) \qquad \text{for all } m, \tag{13.10}$$
namely the block-diagonal matrix whose rth block is the plane rotation through $\omega_r k$;
2. (the kernel is exactly relative)
$$\mathrm{PE}(m)\cdot\mathrm{PE}(m') = \sum_{r=0}^{d/2-1} \cos\big(\omega_r\,(m-m')\big), \tag{13.11}$$
a function of $m-m'$ alone, with maximum $d/2$ at $m=m'$.

Proof. For channel pair r the encoding is $(\sin\omega_r m, \cos\omega_r m)$, a point on the unit circle at angle $\omega_r m$ measured from the vertical. The angle-addition formulae give

$$\begin{pmatrix}\sin\omega_r(m+k)\\ \cos\omega_r(m+k)\end{pmatrix} = \begin{pmatrix}\cos\omega_r k & \sin\omega_r k\\ -\sin\omega_r k & \cos\omega_r k\end{pmatrix}\begin{pmatrix}\sin\omega_r m\\ \cos\omega_r m\end{pmatrix},$$

whose matrix depends on k and not on m; stacking the blocks gives Eq. (13.10), and each block is a rotation so $\boldsymbol{R}_k$ is orthogonal. For (ii), the rth pair contributes $\sin\omega_r m\sin\omega_r m' + \cos\omega_r m\cos\omega_r m' = \cos(\omega_r(m-m'))$, and summing over r gives Eq. (13.11); each cosine is at most one, attained together only at $m=m'$.

Both parts matter. Equation (13.11) says the encoding supplies the dot product of Eq. (13.4) with an exact function of relative distance, so a *relative* notion of position is available without ever being written down – Figure 13.2(c) is a plot of it. Equation (13.10) says something stronger and less often noted: because translation acts *linearly* on the encoding, a query matrix $\boldsymbol{W}_Q$ can implement "attend to whatever is k places back" as a single linear map, valid at every position at once. That is what makes the previous-token head of Section 13.6 easy to

learn, and it is exactly the property a learned positional embedding does not have. Learned embeddings and explicit relative encodings are the two common alternatives, and the first of them pays for its freedom in Section 13.8.

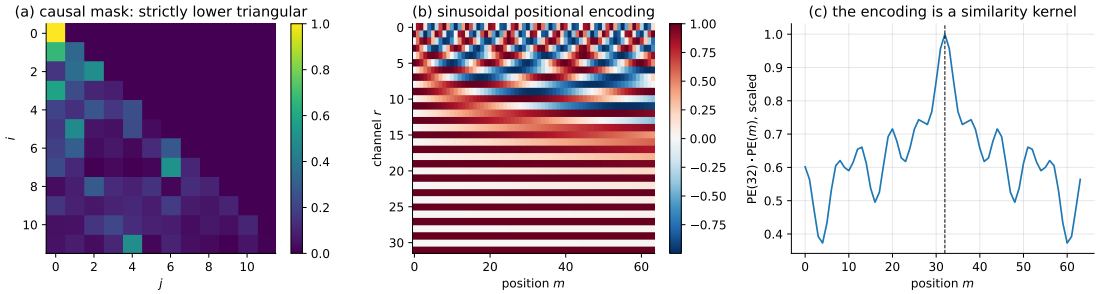


Fig. 13.2 (a) An attention matrix under the causal mask of Eq. (13.8): the strict upper triangle is exactly zero, measured to 0. (b) The sinusoidal positional encoding of Eq. (13.9), channels against position; low channels vary slowly, high channels fast. (c) The inner product of the encoding at position 32 with every other position: a similarity kernel peaked on the diagonal, which is how relative distance reaches the dot product of Eq. (13.4).

13.4 Multi-head attention and the transformer block

One attention matrix imposes a single pattern of couplings. Real problems have several – syntactic and semantic in language, local and long-range in a field. *Multi-head* attention runs H attentions in parallel with separate projections and mixes the results:

$$\mathrm{head}_h = \mathrm{Attention}\Big(\boldsymbol{X}\boldsymbol{W}_Q^{(h)},\, \boldsymbol{X}\boldsymbol{W}_K^{(h)},\, \boldsymbol{X}\boldsymbol{W}_V^{(h)}\Big), \tag{13.12}$$

$$\mathrm{MultiHead}(\boldsymbol{X}) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H)\,\boldsymbol{W}_O. \tag{13.13}$$

Taking $d_k = d_v = d/H$ keeps the cost of H heads equal to that of one head of full width, so multiple heads are free. Different heads reliably learn different patterns, and inspecting them is the beginning of interpretability work.

The block.

Attention is one sublayer of a *transformer block*, which also contains a position-wise feed-forward network, residual connections and normalisation. In the pre-norm arrangement now standard,

$$\begin{aligned} \boldsymbol{X} &\longleftarrow \boldsymbol{X} + \mathrm{MultiHead}(\mathrm{LN}(\boldsymbol{X})), \\ \boldsymbol{X} &\longleftarrow \boldsymbol{X} + \mathrm{MLP}(\mathrm{LN}(\boldsymbol{X})), \end{aligned} \tag{13.14}$$

where the MLP is applied to each token separately, $\mathrm{MLP}(\boldsymbol{z}) = \boldsymbol{W}_2\,\mathrm{GELU}(\boldsymbol{W}_1\boldsymbol{z} + \boldsymbol{b}_1) + \boldsymbol{b}_2$, with the GELU activation of Section 8.6, and layer normalisation acts across the feature axis of each token,

$$\mathrm{LN}(\boldsymbol{z}) = \boldsymbol{\gamma} \odot \frac{\boldsymbol{z} - \mu(\boldsymbol{z})}{\sqrt{\sigma^2(\boldsymbol{z}) + \varepsilon}} + \boldsymbol{\beta}, \qquad \mu(\boldsymbol{z}) = \frac{1}{d}\sum_r z_r. \tag{13.15}$$

Each ingredient earns its place. The MLP supplies the non-linearity that Proposition 13.1 showed attention alone lacks, and it is where most of the parameters live. The residual connections give the gradient an undamped path from the loss to every layer – precisely the additive path that Eq. (11.29) gave the LSTM cell state. Layer normalisation keeps the scale of the activations fixed, which by Section 13.2.2 is what the softmax needs.

A transformer is a stack of these blocks. Nothing in it is unfamiliar: it is attention, plus the multilayer perceptron of Chapter 8, plus residual connections and normalisation.

13.5 Implementation and verification

We write attention with autograd rather than by hand, for the reason Section 4.14 gave: differentiating a softmax nested inside two matrix products is exactly the error-prone drudgery automatic differentiation exists to remove.

```
def softmax_rows(S):
    """Row-wise softmax, shifted for stability."""
    S = S - np.max(S, axis=-1, keepdims=True)
    E = np.exp(S)
    return E / np.sum(E, axis=-1, keepdims=True)


def attention(Q, K, V, mask=None):
    """Scaled dot-product attention, Eq. (13.attention).

    Q is (n, d_k), K is (m, d_k), V is (m, d_v); the output is (n, d_v).
    """
    d_k = Q.shape[-1]
    S = Q @ K.T / np.sqrt(d_k)                # (n, m) scores
    if mask is not None:
        S = S + mask                          # -inf where attention is banned
    A = softmax_rows(S)                       # (n, m), rows sum to one
    return A @ V, A
```

Multi-head attention loops over heads, concatenates and mixes; the block adds the residuals and normalisation of Eq. (13.14).

```
def block(P, X, mask=None):
    """Pre-norm transformer block, Eq. (13.block).

    X -> X + MHA(LN(X)) -> X + MLP(LN(X)).  The residual paths carry the
    identity, which is what keeps deep stacks trainable (cf. Section 11.lstmwhy).
    """
    Y, A = multihead(P, layernorm(X, P["g1"], P["be1"]), mask)
    X = X + Y
    Z = layernorm(X, P["g2"], P["be2"])
    X = X + gelu(Z @ P["W1"] + P["b1"]) @ P["W2"] + P["b2"]
    return X, A
```

The checks follow the propositions one by one.

```
rows of A sum to 1 : 2.22e-16
A >= 0             : True
head output within the bounding box of V : True

=== permutation equivariance:  Att(PX) = P Att(X) ===
  trial 0: max|Att(PX) - P Att(X)| = 1.67e-16
  trial 1: max|Att(PX) - P Att(X)| = 3.33e-16
  trial 2: max|Att(PX) - P Att(X)| = 2.22e-16
```

```
=== a causal mask breaks it (as it must) ===
  max|Att(PX) - P Att(X)| with mask = 1.897e+00
  attention matrix upper triangle (should be 0): 0.00e+00
```

Proposition 13.1 and Theorem 13.2 hold to machine precision, and the mask does exactly what Eq. (13.8) says: it zeroes the upper triangle exactly and destroys the equivariance completely. The batched implementation used for training agrees with the looped one to 8.9×10^{-16}.

13.6 What one attention layer cannot do

Theorem 13.2 and Proposition 13.1 are statements about what attention *is*. This section is about what a given depth of it can *compute*, and the answer is sharper than one might guess.

Consider *associative recall*. A sequence presents L key-value pairs $k_1 v_1 k_2 v_2 \cdots k_L v_L$ and then repeats one of the keys; the model must emit the value that followed it. The task is trivial for a human, has an exact algorithm, and requires reaching a specific earlier position whose distance grows with L. It is the cleanest test of global access there is.

Now look at what one attention layer can do with it. The query at the final position carries the repeated key, and the dot product of Eq. (13.4) can certainly make it match the earlier position holding the same key. But matching that position retrieves *that position's* value vector – the key – and what is wanted is the token *one place to the right*. A single layer has no way to attend to one position and read from another.

It needs two. The first layer copies each token's predecessor into its representation; the second matches the query against keys and now finds the value already attached. This composition is known as an *induction head*, and it is a mechanism rather than a metaphor. The informal argument above can be made into a statement with a proof.

Proposition 13.6 (One content-matching layer cannot do associative recall). *Consider a single attention layer in which the key and value at position j depend only on the symbol s_j there, $\boldsymbol{k}_j = K(s_j)$ and $\boldsymbol{v}_j = V(s_j)$, the query at the final position depends only on s_T, and position enters the scores only through an additive bias b_{Tj} that does not depend on the symbols. Let the recall task be as above, with the L keys distinct, the L values drawn independently and uniformly, and the query $s_T = s_{2q-1}$ for a uniformly chosen q. Then the layer cannot solve the task better than chance in the limit of large vocabulary.*

Proof. The attention weight on position j is $\alpha_j \propto \exp\big(Q(s_T) \cdot K(s_j) + b_{Tj}\big)$, so it is a function of the pair (s_T, s_j) and of j. The answer required is $V(s_{2q})$, the value *following* the matched key, so the layer must place its mass on position $2q$. But s_{2q} is drawn independently of $s_{2q-1} = s_T$ and of every other symbol, so $Q(s_T) \cdot K(s_{2q})$ carries no information about whether $2q$ is the position wanted; and $b_{T,2q}$ is the same for all instances with the same q, which is unknown to the layer. Hence $\mathbb{E}[\alpha_{2q}]$ is the same for every value position, the output is an average of V over value symbols independent of the query, and the readout can do no better than the marginal. A vocabulary of size M makes that $1/M$.

Two layers escape the proposition by breaking its hypothesis. A first layer whose attention pattern is positional rather than content-based – attend one place back, which by Eq. (13.10) is a *single linear map* on the encoding – writes s_{j-1} into position j. After that the key at position $2q$ is a function of $s_{2q-1} = s_T$, the hypothesis $\boldsymbol{k}_j = K(s_j)$ no longer holds, and content matching at the second layer finds exactly the position wanted.

So the prediction is sharp: one block fails and two succeed. We test it, with a vocabulary of eight symbols, so that chance is 0.125.

Table 13.1 Associative recall with L key-value pairs, so sequence length $T = 2L+1$. All models trained by Adam with $\gamma = 3 \times 10^{-3}$ on freshly generated batches; accuracy on 400 unseen sequences, ranges over two seeds. Chance is 0.125.

L	Model	Parameters	Test accuracy
2	transformer, 1 block	8928	1.000
2	RNN (Chapter 11)	8858	1.000
4	transformer, 1 block	8928	0.366 (0.352–0.380)
4	RNN (Chapter 11)	8858	0.351 (0.333–0.370)
4	transformer, 2 blocks	17344	**0.993** (0.985–1.000)

At $L = 2$ the sequence is five tokens long and everything works. At $L = 4$ both the one-block transformer and the recurrent network of Chapter 11 sit at about 0.36, well above chance but nowhere near the task, and they sit there for three thousand updates. Adding a second block takes the same architecture to 0.993.

This is worth dwelling on, because it cuts against the usual way of describing transformers. Global access is *not* sufficient. The one-block model can already see every position – Proposition 13.3 charges it n^2 for the privilege – and it still cannot solve a task that a two-line program solves, because the computation it needs is a composition of two retrievals and it has only one layer in which to do them. Depth is not a way of adding capacity here; it is a way of adding *steps*.

An honest reading of the middle rows. The one-block transformer does not beat the recurrent network on this task; the two are statistically indistinguishable at 0.366 against 0.351. A comparison stopped there would conclude that attention offers nothing. The right conclusion is that the architecture was mismatched to the computation, and the fix was depth rather than width or attention span. When a model plateaus well above chance, it is worth asking what algorithm it would need and how many sequential steps that algorithm takes.

13.7 Comparisons and interpretations

13.7.1 Against the other architectures

The four architectures of Chapters 8 to 13 differ in exactly one respect: which elements interact, and how strongly.

Architecture	Coupling $y_i = \sum_j C_{ij} x_j$	Cost per layer
MLP	$C_{ij} = W_{ij}$, dense and fixed	$\mathcal{O}(n^2 d^2)$
CNN	$C_{ij} = w_{i-j}$, local and fixed	$\mathcal{O}(nFd^2)$
RNN	C lower triangular, applied step by step	$\mathcal{O}(nd^2)$, sequential
Transformer	$C_{ij} = A_{ij}(\boldsymbol{X})$, dense and *adaptive*	$\mathcal{O}(n^2 d)$

An MLP has fixed weights $\boldsymbol{W}$ that do not depend on the input. A convolution is the special case in which those weights are local and shared, by Theorem 10.2. A recurrent layer reaches distant positions only by composing many short steps, which is what Theorem 11.1 punishes. Attention alone lets the coupling matrix be a *function of the data*:

$$\text{fixed couplings} \quad \longrightarrow \quad \text{couplings } A_{ij}(\boldsymbol{X}) \text{ computed at run time.} \tag{13.16}$$

That is the whole of the innovation, and everything else in the architecture is scaffolding to make it trainable.

13.7.2 A statistical-mechanics reading

Equation (13.4) will look familiar to anyone who has met a canonical ensemble. Writing $E_{ij} = -s_{ij}$,

$$A_{ij} = \frac{e^{-E_{ij}}}{\sum_{\ell} e^{-E_{i\ell}}} = \frac{e^{-E_{ij}}}{Z_i}, \tag{13.17}$$

which is a Gibbs distribution over interaction partners at inverse temperature one, with Z_i a per-query partition function. The scores are effective energies; the softmax is a local Boltzmann weighting; and the $1/\sqrt{d_k}$ of Section 13.2.2 is a choice of temperature, with saturation being the zero-temperature limit in which the distribution collapses onto its ground state.

The aggregation step then reads

$$\boldsymbol{y}_i = \sum_j J_{ij}(\boldsymbol{X})\, \boldsymbol{v}_j, \qquad J_{ij} \equiv A_{ij}, \tag{13.18}$$

which is a mean-field update with *state-dependent* couplings. In an Ising or Hopfield model the J_{ij} are fixed by the Hamiltonian; here they are recomputed from the current configuration at every layer. A transformer is, in this reading, a nonlinear adaptive mean-field model, and the analogy is close enough to have been productive: the connection between attention and modern Hopfield networks with exponential capacity is more than superficial.

13.7.3 Kernels and graphs

Equation (13.2) also reads as a discretised integral operator,

$$\boldsymbol{y}_i = \sum_j K(\boldsymbol{x}_i, \boldsymbol{x}_j)\, \boldsymbol{v}_j \qquad \longleftrightarrow \qquad (\mathscr{K} f)(x) = \int K(x,x') f(x')\, \mathrm{d}x', \tag{13.19}$$

with the difference that K is learned rather than prescribed. Compare the kernel methods of Chapter 6, where K was chosen in advance and the representer theorem then fixed everything: attention learns the kernel and gives up the theory.

Viewed as a graph, attention defines a complete weighted graph on the tokens with edge weights A_{ij}, which makes a transformer a graph neural network whose graph is inferred rather than given. A message-passing network on a fixed graph is the special case in which A_{ij} is constrained to the adjacency structure.

13.7.4 Why this matters for differential equations

Chapters 9 and 10 both met the same obstacle from different sides. A convolutional network handles local structure superbly and reaches distant parts of a domain only by stacking layers, at the rate given by Proposition 10.10. But elliptic equations are *globally* coupled – change a boundary value and the solution changes everywhere at once – and so are nonlocal

operators, multiscale dynamics and problems with global constraints. Attention couples the whole domain in one layer.

The natural framing is operator learning. Rather than solving one problem, as in Chapter 9, one learns the solution operator

$$\mathcal{G} : a(x) \longmapsto u(x), \qquad \text{or} \qquad \mathcal{G} : u_0(x) \longmapsto u(x,t), \tag{13.20}$$

mapping a coefficient field or an initial condition to a solution. Discretise the field at points $x_1, \dots, x_n$ and let each token carry what is known at that point,

$$\boldsymbol{x}_i^{\text{token}} = \big(x_i,\, u(x_i),\, a(x_i), \dots\big), \tag{13.21}$$

and Eq. (13.19) becomes a learned Green's function: A_{ij} is how much the solution at x_i depends on the data at x_j. For the Poisson equation of Section 9.5 that dependence is the actual Green's function, and a trained attention matrix can be compared against it directly – an unusually direct check on what such a model has learned, and one worth doing.

The competitor is the Fourier neural operator, which achieves nonlocality by multiplication in the frequency domain, in the spirit of the notebox of Section 10.3.2. Fourier methods are natural for translation-invariant problems on regular grids and cost $\mathcal{O}(n \log n)$; attention handles irregular domains and learns data-dependent kernels, and costs $\mathcal{O}(n^2)$. Which wins depends on whether the problem's structure is known in advance – the same question that decided between hard and soft constraints in Section 9.10.

13.8 PyTorch and TensorFlow

PyTorch supplies attention at three levels. The lowest is `scaled_dot_product_attention`, which is Eq. (13.5) and nothing else; above it `nn.MultiheadAttention` adds the projections of Eq. (13.13); above that `nn.TransformerEncoderLayer` is the whole of Eq. (13.14). Writing the block explicitly is more instructive:

```
import torch
import torch.nn as nn
import torch.nn.functional as F


class Block(nn.Module):
    """Eq. (13.block), pre-norm, written out rather than assembled."""
    def __init__(self, d=64, H=4, d_ff=256, dropout=0.1):
        super().__init__()
        self.ln1 = nn.LayerNorm(d)                  # Eq. (13.layernorm)
        self.attn = nn.MultiheadAttention(d, H, dropout=dropout,
                                          batch_first=True)
        self.ln2 = nn.LayerNorm(d)
        self.mlp = nn.Sequential(
            nn.Linear(d, d_ff), nn.GELU(), nn.Linear(d_ff, d),
            nn.Dropout(dropout))

    def forward(self, x, mask=None):
        h = self.ln1(x)
        a, A = self.attn(h, h, h, attn_mask=mask, need_weights=True)
        x = x + a                                   # residual: see Section 13.block
        x = x + self.mlp(self.ln2(x))
        return x, A


def causal_mask(n, device=None):
```

```
    """Eq. (13.mask): True where attention is forbidden."""
    return torch.triu(torch.ones(n, n, dtype=torch.bool, device=device), 1)


class Transformer(nn.Module):
    def __init__(self, n_vocab, d=64, H=4, d_ff=256, n_blocks=2, n_ctx=64):
        super().__init__()
        self.emb = nn.Embedding(n_vocab, d)
        self.pos = nn.Parameter(torch.zeros(n_ctx, d))   # learned; or Eq. (13.posenc)
        self.blocks = nn.ModuleList(
            [Block(d, H, d_ff) for _ in range(n_blocks)])
        self.ln_f = nn.LayerNorm(d)
        self.head = nn.Linear(d, n_vocab)

    def forward(self, idx, causal=True):
        n = idx.shape[1]
        x = self.emb(idx) + self.pos[:n]
        m = causal_mask(n, idx.device) if causal else None
        for blk in self.blocks:
            x, _ = blk(x, m)
        return self.head(self.ln_f(x))
```

Three details are worth flagging. `batch_first=True` is not the default and the alternative layout catches everyone once. `attn_mask` expects `True` where attention is *forbidden*, the opposite of what the name suggests to many readers. And the parameter count of a block is $4d^2$ for the attention plus $2dd_{\text{ff}}$ for the MLP, so with the usual $d_{\text{ff}} = 4d$ the feed-forward part holds two thirds of the parameters – attention is where the computation is, not where the parameters are.

The same in Keras, where `MultiHeadAttention` plays the same role:

```
import tensorflow as tf
from tensorflow.keras import layers


class Block(layers.Layer):
    """Eq. (13.block) in Keras; note key_dim is d_k, not d."""
    def __init__(self, d=64, H=4, d_ff=256, dropout=0.1):
        super().__init__()
        self.ln1 = layers.LayerNormalization(epsilon=1e-6)
        self.attn = layers.MultiHeadAttention(num_heads=H, key_dim=d // H,
                                              dropout=dropout)
        self.ln2 = layers.LayerNormalization(epsilon=1e-6)
        self.mlp = tf.keras.Sequential([
            layers.Dense(d_ff, activation="gelu"),
            layers.Dense(d), layers.Dropout(dropout)])

    def call(self, x, training=False):
        h = self.ln1(x)
        x = x + self.attn(h, h, h, use_causal_mask=True, training=training)
        return x + self.mlp(self.ln2(x), training=training)


def build(n_vocab, n_ctx=64, d=64, H=4, d_ff=256, n_blocks=2):
    inp = layers.Input(shape=(n_ctx,), dtype="int32")
    tok = layers.Embedding(n_vocab, d)(inp)
    pos = layers.Embedding(n_ctx, d)(tf.range(n_ctx))
    x = tok + pos
    for _ in range(n_blocks):
        x = Block(d, H, d_ff)(x)
    out = layers.Dense(n_vocab)(layers.LayerNormalization(epsilon=1e-6)(x))
    model = tf.keras.Model(inp, out)
    model.compile(
        optimizer=tf.keras.optimizers.Adam(3e-4),
```

```
        loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True))
    return model
```

13.8.1 Do the three implementations agree?

Attention has more places for a convention to differ than a convolution does, and the differences are all in how the heads are packed:

- `nn.MultiheadAttention` stores the three projections in a single `in_proj_weight` of shape $(3d, d)$, stacked in the order Q, K, V, with every head concatenated inside each block and each block transposed relative to our (d, d_k) convention.
- `layers.MultiHeadAttention` keeps the head axis explicit: its query kernel is (d, H, d_k), which is our (H, d, d_k) with the first two axes swapped, and its output kernel is (H, d_k, d) against our (Hd_k, d).
- Both apply the $1/\sqrt{d_k}$ of Eq. (13.5) internally, and both take the mask with the opposite polarity from the other.

With those reconciled, one set of weights in all three, in double precision:

```
=== 1. scaled dot-product attention, Eq. (13.attention) ===
  max |ours - torch|                 : 2.220e-16
  attention rows sum to one, max dev : 2.220e-16
  causal mask, Eq. (13.mask), max diff: 2.220e-16
  strictly-upper attention mass       : 0.000e+00   (must be exactly zero)

=== 2. multi-head attention, Eq. (13.multihead) ===
  output   max |ours - torch|        : 3.886e-16
  weights  max |ours - torch|        : 1.665e-16
  output   max |ours - keras|        : 5.551e-16
  weights  max |ours - keras|        : 2.220e-16

=== 3. layer normalisation, Eq. (13.layernorm) ===
  max |ours - torch| : 1.776e-15
  max |ours - keras| : 1.776e-15
```

and the gradient through a whole block of Eq. (13.14), which is the comparison that matters, since our gradients come from `autograd` on NumPy and PyTorch's from an entirely separate implementation:

```
=== 4. the gradient through a transformer block, Eq. (13.block) ===
  our loss   : 2.675342504804
  torch loss : 2.675342504804
   parameter       shape             |ours - torch|
   WQ        (4, 16, 4)              1.943e-16
   WK        (4, 16, 4)              1.388e-16
   WV        (4, 16, 4)              1.943e-16
   WO        (16, 16)                1.110e-16
   W1        (16, 64)                6.939e-17
   W2        (64, 16)                1.110e-16
   g1        (16,)                   1.388e-16
   g2        (16,)                   5.551e-17
  worst disagreement: 1.943e-16
```

Softmax, layer normalisation, the residual paths and the multi-head repacking all differentiate to the same numbers.

13.8.2 The scaling, in the library

Section 13.2.2 established Proposition 13.4 on our own softmax. Repeating the measurement through PyTorch, over three decades of d_k and with the entropy and largest weight of the attention rows alongside the softmax Jacobian:

```
   d_k   var(logit)   entropy scaled   entropy unscaled   max weight sc/unsc   ||dA/dS||
    sc/unsc
     4         4.22           3.6547             2.6957       0.123/0.314
    0.1847/0.2516
    16        16.92           3.6392             1.1623       0.125/0.644
    0.1902/0.2664
    64        64.91           3.6768             0.5191       0.111/0.808
    0.1849/0.1939
   256       258.73           3.6893             0.2479       0.106/0.901
    0.1827/0.1170
  1024      1050.61           3.6651             0.1024       0.109/0.961
    0.1860/0.0565
```

The second column is Proposition 13.4 over a factor of 256 in d_k. The rest is the consequence. Scaled, the entropy stays at 3.66 – against $\log 64 = 4.159$ for a uniform row – the largest weight stays near 0.11, and the softmax Jacobian stays near 0.185, all essentially independent of d_k. Unscaled, the entropy falls by a factor of 26, the largest weight climbs to 0.96, and the Jacobian falls to 0.057. At $d_k = 1024$ an unscaled attention row is a hard maximum in all but name, and a hard maximum has no derivative. The division by $\sqrt{d_k}$ is not a normalisation convenience; it is what keeps the layer differentiable as it is made wider.

13.8.3 Permutation equivariance, tested

Theorem 13.2 is easy to state and easy to get wrong in an implementation, since any leaked positional information breaks it silently. Permuting the tokens of a random input through `nn.MultiheadAttention`:

```
=== 2. Theorem 13.perm, tested ===
  max |MHA(PX) - P MHA(X)|                      : 8.941e-08
  with positional encoding, Eq. (13.posenc)     : 3.211e-01
```

The first line is zero to single-precision rounding and the second is not, which is the theorem and its remedy in two numbers.

13.8.4 Associative recall, in the frameworks

Table 13.1 was produced by our own NumPy transformer. Repeating it in PyTorch – with `nn.TransformerEncoderLayer` rather than our block, and extending to $L = 8$, the longest the task allows with eight distinct keys – gives Table 13.2.

The first two rows reproduce Table 13.1 in a different implementation: one block reaches 0.367 at $L = 4$ against our 0.366, two blocks 0.999 against 0.993. Proposition 13.6 is not an artefact of our code. Extending to $L = 8$ shows the two-block model degrading too, to 0.708 – the induction head solves the mechanism but the model still has to find the right one of eight keys in a seventeen-token sequence with a 32-dimensional state.

The third row is the finding worth pausing on. Replacing the sinusoidal encoding of Eq. (13.9) by a *learned* positional embedding, initialised small and trained alongside every-

Table 13.2 Associative recall in PyTorch: L key-value pairs, sequence length $T = 2L+1$, eight symbols so chance is 0.125. All models trained by Adam at 3×10^{-3} for 3000 updates on freshly generated batches; accuracy on 400 unseen sequences averaged over two seeds. The last two rows are the recurrent networks of Chapter 11 on the same task.

Model	Parameters	$L=2$	$L=4$	$L=8$
transformer, 1 block	9128	0.988	0.367	0.285
transformer, 2 blocks	17672	1.000	**0.999**	**0.708**
transformer, 2 blocks, learned pos.	19720	0.557	0.382	0.286
LSTM (Chapter 11)	16392	1.000	0.907	0.507
simple RNN (Chapter 11)	4584	0.931	0.349	0.266

thing else, destroys the model: 0.557 at $L=2$ where the sinusoidal version reaches 1.000, and chance-level 0.286 at $L=8$. Nothing else changed. Proposition 13.5 explains it. The previous-token head that the induction mechanism needs is, under Eq. (13.10), a single linear map applied at every position at once, because translation acts linearly on the sinusoidal code. A learned embedding has no such structure at initialisation, so the same head must be discovered separately for every position, and 3000 updates are not enough. Learned positional embeddings work at scale; they are a poor choice when data and compute are short, and this table is what that costs.

Running the same experiment in TensorFlow, with the block assembled by hand from the Keras attention and normalisation layers rather than taken ready-made, gives

```
=== associative recall in TensorFlow, Table 13.recall ===
    L    T   model                  params   test accuracy (2 seeds)
    2    5   transformer, 1 block(s)      9128   1.000   (35s)
    2    5   transformer, 2 block(s)     17672   1.000   (55s)
    4    9   transformer, 1 block(s)      9128   0.539   (38s)
    4    9   transformer, 2 block(s)     17672   1.000   (63s)
    8   17   transformer, 1 block(s)      9128   0.343   (45s)
    8   17   transformer, 2 block(s)     17672   0.359   (74s)
```

The row that matters agrees: at $L=4$, one block reaches 0.539 and two reach 1.000. The parameter counts are identical to PyTorch's, and Section 13.8.1 has already established that the two compute the same function. At $L=8$ they part company – TensorFlow's two-block model gets 0.359 where PyTorch's got 0.708 – and the difference is initialisation and optimiser state, not architecture. We report it rather than choosing the better number: at the edge of what 3000 updates can find, a task can be solved or not depending on where the search starts, and a single run of a single framework is not evidence about an architecture. The $L=4$ column, where both frameworks are decisive and agree, is.

Finally, the recurrent baselines. The LSTM is competitive at $L=2$ and $L=4$ and better than the one-block transformer everywhere, which is the honest reading already given in the notebox of Section 13.6: global access is not by itself an advantage. What separates the architectures here is whether the model can compose two retrievals, and that is a question about depth, not about attention.

13.9 Strengths, weaknesses and what to remember

A transformer replaces the fixed interaction patterns of the earlier architectures by couplings computed from the data, Eq. (13.16). Everything else – the multilayer perceptron, the residual

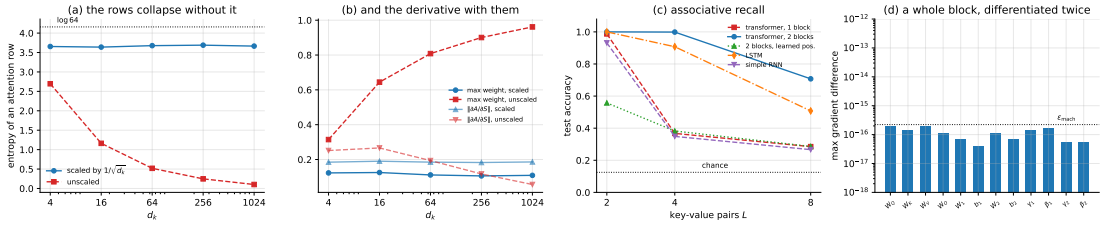


Fig. 13.3 (a) The entropy of an attention row against d_k, with and without the $1/\sqrt{d_k}$ of Eq. (13.5), measured in PyTorch; the dashed line is $\log 64$, the entropy of a uniform row. (b) The largest weight in a row and the norm of the softmax Jacobian: unscaled, the row becomes a hard maximum and the derivative disappears with it. (c) Associative recall, Table 13.2: one block fails where two succeed, and the learned positional encoding fails everywhere. (d) The largest disagreement between our gradients through a whole block and PyTorch's, against the double-precision unit roundoff.

connections, the normalisation – is machinery from earlier chapters, assembled to make that one idea trainable.

The properties are sharp and we proved them. Attention averages, so its output lies in the convex hull of its values (Proposition 13.1) and it cannot extrapolate without the feed-forward sublayer. It is equivariant to *all* permutations (Theorem 13.2), a much larger symmetry group than the translations of Theorem 10.7, which is why position has to be added by hand. It costs $\mathcal{O}(n^2 d)$ (Proposition 13.3), which is the price of global access and the reason for a large literature on approximating it.

The $1/\sqrt{d_k}$ is not cosmetic. Proposition 13.4 predicts $\mathrm{Var}(\boldsymbol{q}\cdot\boldsymbol{k}) = d_k$, measured as 1011 at $d_k = 1024$; and the softmax Jacobian falls by thirty-five orders of magnitude between logit scale 1 and scale 64. Without the scaling a wide head starts training with no usable gradient through its attention weights at all.

The result to carry away is Section 13.6. On associative recall at $L = 4$, a one-block transformer reached 0.366 and a matched recurrent network 0.351 – indistinguishable, and both far from the task. Two blocks reached 0.993. Seeing every position is not the same as being able to compose two retrievals, and depth here supplies *steps* of computation rather than capacity. Global access is necessary and not sufficient. Proposition 13.6 turns that observation into a statement with a proof: if keys and values depend on the symbol at their own position, no content-based score can single out the position *after* the match, because the symbol there is independent of the query. Repeating the whole table in PyTorch, Table 13.2, reproduces it to the third decimal – 0.367 against 0.366 for one block, 0.999 against 0.993 for two.

Two further things came out of running the libraries rather than reading them. Proposition 13.5 shows the sinusoidal encoding to be an exactly relative code, with translation acting on it as a fixed rotation, Eq. (13.10); replacing it by a learned embedding and changing nothing else drops recall from 1.000 to 0.557 at $L = 2$ and to chance at $L = 8$, because the previous-token head that the induction mechanism needs is one linear map on the sinusoidal code and a separate discovery at every position without it. And our block, differentiated by `autograd` on NumPy, agrees with PyTorch's implementation of the same block to 1.9×10^{-16} on every parameter array – softmax, layer normalisation and multi-head repacking included.

Set against the strengths – global context, full parallelism across positions, excellent scaling with data and model size, a learned rather than assumed interaction structure – are real weaknesses: quadratic cost in sequence length, large data requirements, no inductive bias whatever about locality or order, and an interpretability that is better than a dense layer's but far from complete. The architecture is a good default when the structure of the interactions is unknown and the data are plentiful. When the structure *is* known – locality in an image, causality in a time series, translation invariance in a homogeneous medium – a convolution

or a recurrence encodes it for free, and free structure beats learned structure whenever the structure is right.

The programs are in the directory
`doc/BookML/BookPrograms/chapter13_transformers`.
Every listing above appears there as a numbered file, and three modules run start to finish and reproduce the numbers quoted in the text:

- `attention.py` – scaled dot-product attention, multi-head attention, layer normalisation, the block of Eq. (13.14), causal masks, sinusoidal positional encodings, and batched versions of all of them.
- `cross_check_attn.py` – the three-way comparison of Section 13.8.1, with the head-packing conversions for both libraries written out.
- `transformer_torch.py` – the scaling measurement of Section 13.8.2, the permutation test of Section 13.8.3 and the recall table of Section 13.8.4.
- `transformer_tf.py` – the recall experiment in TensorFlow.
- `verify_attention.py` – row-stochasticity, the convex-hull property, permutation equivariance and its destruction by a mask, the logit-variance table, the softmax-saturation table, and the agreement between the looped and batched implementations.
- `recall.py` and `run_recall.py` – the associative-recall task of Table 13.1, with the one-block and two-block transformers and the Chapter 11 recurrent baseline.

The figures are generated by `ch13_figures.py` in `doc/BookML/BookFigures`; neither is drawn by hand.

13.10 Exercises

Warm-up exercises

1. **Shapes and costs.** With $n = 512$, $d = 768$, $H = 12$ and $d_k = d/H$: (a) what are the shapes of $\boldsymbol{Q}$, $\boldsymbol{K}$, $\boldsymbol{V}$, $\boldsymbol{A}$ and the output of one head? (b) How many parameters has one block with $d_{\text{ff}} = 4d$, split between attention and the MLP? (c) How much memory does $\boldsymbol{A}$ need in single precision, for all heads at once, and what happens at $n = 8192$?
2. **Permutation equivariance.** Prove Theorem 13.2 for multi-head attention, and then show that adding the positional encoding of Eq. (13.9) destroys it. Is the composition equivariant to any group at all?
3. **The convex hull.** Verify Proposition 13.1 numerically, then construct a target output that a single attention layer provably cannot produce, and confirm that adding the MLP of Eq. (13.14) makes it reachable.
4. **Temperature.** Replace $1/\sqrt{d_k}$ by β in Eq. (13.4) and study the limits $\beta \to 0$ and $\beta \to \infty$. Show that the first gives uniform averaging and the second a hard $\arg\max$, and relate both to Eq. (13.17). What is the entropy of the attention distribution in each limit?
5. **Positional encodings.** (a) Prove Proposition 13.5: exhibit the rotation $\boldsymbol{R}_k$ of Eq. (13.10) and derive Eq. (13.11). (b) Plot $\text{PE}(m) \cdot \text{PE}(m')$ against $m - m'$ for $d = 64$ and confirm that all curves collapse onto one, as part (ii) requires. (c) The kernel is exactly relative but not monotonic in $|m - m'|$. Find the first local maximum away from the origin and explain where it comes from. (d) Show that a learned positional embedding satisfies neither part of the proposition, and connect that to the third row of Table 13.2.
6. **One layer is not enough.** (a) Work through Proposition 13.6 and state precisely which hypothesis a second layer breaks. (b) Construct, by hand, weights for a two-layer model that solve the recall task exactly: a first layer that shifts by one using Eq. (13.10), and

a second that matches on content. (c) Does the argument still apply if the value map is allowed to depend on position as well as on the symbol? If not, why does the one-block model in Table 13.2 still fail?

7. **The scaling, empirically.** Reproduce Section 13.8.2. (a) Confirm $\mathrm{Var}(\boldsymbol{q}\cdot\boldsymbol{k}) = d_k$ over three decades. (b) Show that the softmax Jacobian $\mathrm{diag}(\boldsymbol{a}) - \boldsymbol{a}\boldsymbol{a}^{\mathsf{T}}$ has Frobenius norm tending to zero as $\boldsymbol{a}$ tends to a one-hot vector, and bound its spectral norm by $1/4$. (c) Replace $1/\sqrt{d_k}$ by $1/d_k$ and by 1 and report the entropy at $d_k = 1024$ in each case. Which of the three is right, and why is it not a matter of taste?
8. **Masks.** Write the mask for (a) causal attention, (b) attention restricted to a window of width w, (c) attention that ignores padding tokens. For (b), show that the layer becomes a convolution with a data-dependent kernel, and relate this to Theorem 10.2.

Project-style exercise: a transformer from scratch

Part a: the machinery.

Implement scaled dot-product attention, multi-head attention, layer normalisation and the block of Eq. (13.14). Verify row-stochasticity, the convex-hull property and permutation equivariance to machine precision, and check every gradient against finite differences. Implement the batched version and verify it against the looped one.

Part b: the scaling.

Reproduce Proposition 13.4 and the saturation table of Section 13.2.2. Then train the same model with and without the $1/\sqrt{d_k}$ at several values of d_k, and report at what d_k the unscaled version stops training at all. Plot the entropy of the attention distribution during training in both cases.

Part c: depth versus width.

Reproduce Table 13.1. Then separate the two explanations for the one-block failure: is it depth, or capacity? Train a one-block model with the width increased until its parameter count matches the two-block model, and report what happens. Then extend the table to $L = 8$ and $L = 16$ and plot accuracy against L for one block, two blocks and the recurrent baseline.

Part d: reading the attention.

For a trained two-block model, plot the attention matrices of both blocks on several test sequences. Can you identify the mechanism described in Section 13.6 – one head copying the predecessor, another matching the query? Quantify it: measure how much of the first block's attention mass sits on the diagonal offset by one.

Part e: against the alternatives.

Compare the transformer, the convolutional network of Chapter 10 and the recurrent network of Chapter 11 on the same task at matched parameter counts, measuring accuracy, wall-clock

time per update and memory. Then repeat on a task with genuinely local structure and report which architecture wins there. The two answers should differ, and the discussion of why is the point of the exercise.

Part f: a learned Green's function.

Solve the one-dimensional Poisson equation of Section 9.5 for many right-hand sides, and train an attention model to map the discretised source $f(x_j)$ to the solution $u(x_i)$, as in Eq. (13.21). Compare the learned attention matrix against the exact Green's function of the operator. How close is it, and does it get closer with more data or with more blocks?

Part g: the cost.

Measure the wall-clock time and peak memory of your attention implementation as n grows, and confirm Proposition 13.3. Then implement one approximation – a sliding window, a random sparse pattern, or a low-rank factorisation of $\boldsymbol{A}$ – and report what it costs in accuracy on the recall task of part c.

Part IV
Generative methods

Chapter 14
Boltzmann Machines

Every model so far has been *discriminative*. Given an input we predicted an output: a number, a class, a solution to a differential equation, a reconstruction. Even the autoencoder of Chapter 12, which had no labels, was scored on how well it reproduced a given input.

A *generative* model asks a different question. Rather than $p(y \mid \boldsymbol{x})$ it models $p(\boldsymbol{x})$ itself: not what to predict from an observation, but what observations are likely at all. With such a model in hand one can sample new data, evaluate how surprising an observation is, fill in missing components, and – most valuable in the physical sciences – inspect the structure the model has inferred.

The Boltzmann machine is the oldest such model in this book and the one closest to physics, because it *is* physics: the distribution it fits is the canonical ensemble, its parameters are couplings and fields, and the difficulty of training it is the difficulty of computing a partition function. That difficulty is genuine and is the organising theme of the chapter. We derive it, prove exactly where it enters, and then spend the rest of the chapter on the Markov chain Monte Carlo machinery invented to get around it.

The material follows the lecture notes for weeks eleven and twelve of FYS-STK3155/4155 and the accompanying notes on Gibbs sampling.

14.1 Energy models

Let $\boldsymbol{x} \in \mathscr{X}$ be a configuration of the variables we observe – the *visible* units – and let $\boldsymbol{h} \in \mathscr{H}$ be a configuration of variables we do not observe, the *hidden* units. An energy-based model assigns every joint configuration an energy $E(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta})$ and declares the probability to be the Boltzmann weight of that energy,

$$\boxed{p(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta}) = \frac{f(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})}, \qquad f = e^{-E(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta})},} \tag{14.1}$$

normalised by the *partition function*

$$Z(\boldsymbol{\Theta}) = \sum_{\boldsymbol{x}} \sum_{\boldsymbol{h}} f(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta}). \tag{14.2}$$

The parameters $\boldsymbol{\Theta}$ are what we fit. Anything non-negative can be written as e^{-E}, so Eq. (14.1) is not a restriction on the family of distributions; it is a change of variables that makes the structure convenient, and it is the same change of variables a physicist makes when writing $p \propto e^{-\beta H}$.

Two remarks on notation. A symbol like $\boldsymbol{x}$ now denotes an entire *configuration*, not a single variable, and the sums in Eq. (14.2) run over all configurations. And the temperature has been absorbed into E; setting $\beta = 1$ costs nothing because $\boldsymbol{\Theta}$ can rescale.

Since the hidden units are not observed, what we can compare against data is the marginal

$$p(\boldsymbol{x};\boldsymbol{\Theta}) = \sum_{\boldsymbol{h}} p(\boldsymbol{x},\boldsymbol{h};\boldsymbol{\Theta}) = \frac{1}{Z(\boldsymbol{\Theta})} \sum_{\boldsymbol{h}} f(\boldsymbol{x},\boldsymbol{h};\boldsymbol{\Theta}) \;=:\; \frac{f(\boldsymbol{x};\boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})}, \tag{14.3}$$

and it is convenient to name the exponent of the marginal.

Definition 14.1 (Free energy). The *free energy* of a visible configuration is

$$F(\boldsymbol{x};\boldsymbol{\Theta}) = -\log \sum_{\boldsymbol{h}} e^{-E(\boldsymbol{x},\boldsymbol{h};\boldsymbol{\Theta})}, \qquad \text{so that} \qquad p(\boldsymbol{x};\boldsymbol{\Theta}) = \frac{e^{-F(\boldsymbol{x};\boldsymbol{\Theta})}}{Z(\boldsymbol{\Theta})}. \tag{14.4}$$

The name is the physicist's: F is the free energy of the hidden subsystem at fixed visible configuration, and the sum over $\boldsymbol{h}$ is a partial trace. For the restricted machines of Section 14.4.1 that sum can be done in closed form, which is the whole reason they are used.

14.1.1 The counting problem

Equation (14.2) is a sum over every configuration. If there are M binary visible units and N binary hidden units it has $2^M \times 2^N$ terms. For a machine with $M = 784$ visible units, one per pixel of an MNIST image, and $N = 500$ hidden units, that is $2^{1284} \approx 10^{386}$ terms – more than the number of atoms in the observable universe raised to a considerable power. The partition function is not merely expensive; it is permanently out of reach.

This is not a defect of the model. It is the same intractability that makes statistical mechanics hard, and the same one that made the Ising model resist solution in three dimensions. Everything technical in this chapter is a response to it.

14.2 Maximum likelihood and the two phases

Given data $\boldsymbol{X} = \{\boldsymbol{x}^{(1)}, \dots, \boldsymbol{x}^{(n)}\}$ assumed independent, the likelihood is $\prod_v p(\boldsymbol{x}^{(v)};\boldsymbol{\Theta})$, and as always we work with its logarithm,

$$\mathscr{L}(\boldsymbol{\Theta}) = \frac{1}{n} \sum_{v=1}^{n} \log p(\boldsymbol{x}^{(v)};\boldsymbol{\Theta}) = -\frac{1}{n} \sum_{v=1}^{n} F(\boldsymbol{x}^{(v)};\boldsymbol{\Theta}) - \log Z(\boldsymbol{\Theta}). \tag{14.5}$$

The two terms behave very differently, and the following theorem says exactly how.

Theorem 14.2 (The gradient is a difference of two expectations). *For the model (14.1),*

$$\boxed{\nabla_{\boldsymbol{\Theta}} \mathscr{L} = \underbrace{-\langle \nabla_{\boldsymbol{\Theta}} E(\boldsymbol{x},\boldsymbol{h}) \rangle_{\text{data}}}_{\textit{positive phase}} + \underbrace{\langle \nabla_{\boldsymbol{\Theta}} E(\boldsymbol{x},\boldsymbol{h}) \rangle_{\text{model}}}_{\textit{negative phase}},} \tag{14.6}$$

where $\langle\cdot\rangle_{\text{data}}$ averages over $\boldsymbol{x} \sim \text{data}$ and $\boldsymbol{h} \sim p(\boldsymbol{h} \mid \boldsymbol{x})$, and $\langle\cdot\rangle_{\text{model}}$ averages over the joint model distribution $p(\boldsymbol{x},\boldsymbol{h};\boldsymbol{\Theta})$.

Proof. Take the second term of Eq. (14.5) first. Since $\nabla \log Z = \nabla Z / Z$ and $Z = \sum_{\boldsymbol{x},\boldsymbol{h}} e^{-E}$,

$$\nabla_{\boldsymbol{\Theta}} \log Z = \frac{1}{Z}\sum_{\boldsymbol{x},\boldsymbol{h}} \nabla_{\boldsymbol{\Theta}} e^{-E(\boldsymbol{x},\boldsymbol{h})} = -\frac{1}{Z}\sum_{\boldsymbol{x},\boldsymbol{h}} e^{-E(\boldsymbol{x},\boldsymbol{h})} \nabla_{\boldsymbol{\Theta}} E(\boldsymbol{x},\boldsymbol{h}) = -\left\langle \nabla_{\boldsymbol{\Theta}} E \right\rangle_{\text{model}},$$

recognising e^{-E}/Z as $p(\boldsymbol{x},\boldsymbol{h})$. For the first term, by Definition 14.1,

$$-\nabla_{\boldsymbol{\Theta}} F(\boldsymbol{x}) = \nabla_{\boldsymbol{\Theta}} \log \sum_{\boldsymbol{h}} e^{-E(\boldsymbol{x},\boldsymbol{h})} = -\frac{\sum_{\boldsymbol{h}} e^{-E} \nabla_{\boldsymbol{\Theta}} E}{\sum_{\boldsymbol{h}} e^{-E}} = -\left\langle \nabla_{\boldsymbol{\Theta}} E \right\rangle_{p(\boldsymbol{h}\mid\boldsymbol{x})},$$

since $e^{-E(\boldsymbol{x},\boldsymbol{h})}/\sum_{\boldsymbol{h}'} e^{-E(\boldsymbol{x},\boldsymbol{h}')} = p(\boldsymbol{h} \mid \boldsymbol{x})$. Averaging over the data and subtracting $\nabla \log Z$ gives Eq. (14.6).

Equation (14.6) is the central formula of the chapter and it has a vivid reading. The positive phase lowers the energy of configurations the data actually contain; the negative phase raises the energy of configurations the *model* currently believes in. Learning is a competition between the two, and it stops exactly when the model's expectations match the data's – which is the classical method-of-moments condition of Chapter 2, arrived at from a different direction.

Where the difficulty lives. The positive phase is an average over the training set and costs nothing. The negative phase is an average over the model distribution, which is e^{-E}/Z, and *drawing samples from it is exactly the problem that Z makes hard*. Every technique in the rest of this chapter – Markov chains, Metropolis, Gibbs, contrastive divergence – exists to estimate that one expectation. Note also what is *not* needed: nowhere does Eq. (14.6) require the value of Z, only samples from the distribution it normalises. That is the loophole the whole subject exploits.

14.2.1 The same statement as a divergence

There is an equivalent formulation worth having. For distributions p and q the Kullback-Leibler divergence is

$$\mathrm{KL}(p\,\|\,q) = \sum_{\boldsymbol{x}} p(\boldsymbol{x}) \log \frac{p(\boldsymbol{x})}{q(\boldsymbol{x})} \;\geq\; 0, \tag{14.7}$$

with equality if and only if $p = q$, by Jensen's inequality applied to the convex function $-\log$. Writing p_{data} for the empirical distribution,

$$\mathrm{KL}(p_{\text{data}}\,\|\,p_{\boldsymbol{\Theta}}) = -H(p_{\text{data}}) - \mathscr{L}(\boldsymbol{\Theta}), \tag{14.8}$$

and since the entropy $H(p_{\text{data}})$ does not depend on $\boldsymbol{\Theta}$, *maximising the likelihood is minimising the KL divergence from the data to the model*. The two views are the same computation; the divergence view makes the target explicit and will reappear when variational methods are introduced.

14.3 Markov chain Monte Carlo

We need samples from $p(\boldsymbol{x}) \propto e^{-F(\boldsymbol{x})}$ without knowing the constant. The device is to construct a Markov chain whose long-run distribution is the one we want, run it, and use its states as samples.

A Markov chain on a finite state space is a sequence $\boldsymbol{s}^{(0)}, \boldsymbol{s}^{(1)}, \dots$ in which the next state depends only on the current one,

$$\operatorname{Prob}\Big(\boldsymbol{s}^{(t+1)} = \boldsymbol{y} \mid \boldsymbol{s}^{(t)} = \boldsymbol{x}, \boldsymbol{s}^{(t-1)}, \dots\Big) = T(\boldsymbol{x} \to \boldsymbol{y}), \tag{14.9}$$

with T the transition matrix, $T(\boldsymbol{x} \to \boldsymbol{y}) \geq 0$ and $\sum_{\boldsymbol{y}} T(\boldsymbol{x} \to \boldsymbol{y}) = 1$. A distribution π is *stationary* for T if it reproduces itself,

$$\pi(\boldsymbol{y}) = \sum_{\boldsymbol{x}} \pi(\boldsymbol{x}) T(\boldsymbol{x} \to \boldsymbol{y}). \tag{14.10}$$

Two properties make a chain useful. It is *irreducible* if every state can be reached from every other, and *aperiodic* if it does not return to states only at multiples of some period; together these are what is usually called *ergodicity*, and the standard theorem then says that the chain has a unique stationary distribution and converges to it from any start. What remains is to arrange for π to be the distribution we want, and the following condition is the standard tool.

Definition 14.3 (Detailed balance). A transition matrix T satisfies *detailed balance* with respect to π if

$$\pi(\boldsymbol{x})\, T(\boldsymbol{x} \to \boldsymbol{y}) = \pi(\boldsymbol{y})\, T(\boldsymbol{y} \to \boldsymbol{x}) \qquad \text{for all } \boldsymbol{x}, \boldsymbol{y}. \tag{14.11}$$

Proposition 14.4 (Detailed balance implies stationarity). *If T satisfies Eq. (14.11) then π is stationary for T. The converse is false.*

Proof. Summing Eq. (14.11) over $\boldsymbol{x}$,

$$\sum_{\boldsymbol{x}} \pi(\boldsymbol{x}) T(\boldsymbol{x} \to \boldsymbol{y}) = \sum_{\boldsymbol{x}} \pi(\boldsymbol{y}) T(\boldsymbol{y} \to \boldsymbol{x}) = \pi(\boldsymbol{y}) \sum_{\boldsymbol{x}} T(\boldsymbol{y} \to \boldsymbol{x}) = \pi(\boldsymbol{y}),$$

using that the rows of T sum to one. This is Eq. (14.10). For the converse, a deterministic cycle on three states has a uniform stationary distribution but no reverse transitions at all, so Eq. (14.11) fails.

Detailed balance is a physical statement: in equilibrium the flow of probability from $\boldsymbol{x}$ to $\boldsymbol{y}$ exactly cancels the flow back. It is stronger than stationarity but far easier to check, and every algorithm below is designed around it.

14.3.1 Metropolis-Hastings

Propose a move from $\boldsymbol{x}$ to $\boldsymbol{y}$ with some proposal density $q(\boldsymbol{x} \to \boldsymbol{y})$, and accept it with probability

$$A(\boldsymbol{x} \to \boldsymbol{y}) = \min\left\{1,\ \frac{\pi(\boldsymbol{y})\, q(\boldsymbol{y} \to \boldsymbol{x})}{\pi(\boldsymbol{x})\, q(\boldsymbol{x} \to \boldsymbol{y})}\right\}. \tag{14.12}$$

If rejected, the chain stays where it is.

Proposition 14.5. *The chain defined by Eq. (14.12) satisfies detailed balance with respect to π, and requires π only through ratios $\pi(\boldsymbol{y})/\pi(\boldsymbol{x})$.*

Proof. The full transition for $\boldsymbol{y} \neq \boldsymbol{x}$ is $T = qA$. Suppose without loss of generality that the ratio in Eq. (14.12) is at most one, so $A(\boldsymbol{x} \to \boldsymbol{y}) = \pi(\boldsymbol{y}) q(\boldsymbol{y} \to \boldsymbol{x}) / [\pi(\boldsymbol{x}) q(\boldsymbol{x} \to \boldsymbol{y})]$ and $A(\boldsymbol{y} \to \boldsymbol{x}) = 1$. Then

$$\pi(\boldsymbol{x}) q(\boldsymbol{x} \to \boldsymbol{y}) A(\boldsymbol{x} \to \boldsymbol{y}) = \pi(\boldsymbol{y}) q(\boldsymbol{y} \to \boldsymbol{x}) = \pi(\boldsymbol{y}) q(\boldsymbol{y} \to \boldsymbol{x}) A(\boldsymbol{y} \to \boldsymbol{x}),$$

which is Eq. (14.11). The case $\boldsymbol{x} = \boldsymbol{y}$ is trivial. The ratio statement is immediate: π enters only as $\pi(\boldsymbol{y})/\pi(\boldsymbol{x}) = e^{-[F(\boldsymbol{y}) - F(\boldsymbol{x})]}$, in which Z cancels.

That cancellation is the point. Proposition 14.5 is what makes sampling from e^{-F}/Z possible without ever computing Z, and it is the loophole promised in the notebox of Section 14.2.

14.3.2 Gibbs sampling

Gibbs sampling replaces the proposal-and-accept step by a sequence of exact conditional draws. With the state split into components $\boldsymbol{s} = (s_1, \dots, s_d)$, one sweep visits each component and replaces it by a draw from its conditional given all the others,

$$s_i \;\sim\; \pi(s_i \mid s_1, \dots, s_{i-1}, s_{i+1}, \dots, s_d)\,. \tag{14.13}$$

Proposition 14.6 (Gibbs is Metropolis with acceptance one). *The update (14.13) satisfies detailed balance with respect to π, and is the special case of Eq. (14.12) in which the proposal is the exact conditional and every proposal is accepted.*

Proof. Write $\boldsymbol{s}_{-i}$ for the components other than i, and take $q(\boldsymbol{x} \to \boldsymbol{y}) = \pi(y_i \mid \boldsymbol{x}_{-i})$ for $\boldsymbol{y}$ differing from $\boldsymbol{x}$ only in component i. Then, using $\pi(\boldsymbol{x}) = \pi(x_i \mid \boldsymbol{x}_{-i})\pi(\boldsymbol{x}_{-i})$ and $\boldsymbol{x}_{-i} = \boldsymbol{y}_{-i}$,

$$\frac{\pi(\boldsymbol{y})q(\boldsymbol{y} \to \boldsymbol{x})}{\pi(\boldsymbol{x})q(\boldsymbol{x} \to \boldsymbol{y})} = \frac{\pi(y_i \mid \boldsymbol{x}_{-i})\pi(\boldsymbol{x}_{-i})\,\pi(x_i \mid \boldsymbol{x}_{-i})}{\pi(x_i \mid \boldsymbol{x}_{-i})\pi(\boldsymbol{x}_{-i})\,\pi(y_i \mid \boldsymbol{x}_{-i})} = 1,$$

so the acceptance probability of Eq. (14.12) is $\min\{1, 1\} = 1$ and detailed balance holds by Proposition 14.5.

So Gibbs sampling never rejects, which is its great practical advantage – no tuning of a step size, no wasted proposals. Its cost is that it needs the conditionals in closed form and it moves one component at a time, so strongly correlated components make it mix slowly. Metropolis, by contrast, works with any proposal but throws work away. Which is better depends entirely on whether the conditionals are available, and for the restricted machines of the next section they are available and are trivially cheap.

14.4 Boltzmann machines

A *Boltzmann machine* is the energy model (14.1) with a quadratic energy on binary units. Writing $\boldsymbol{s} = (\boldsymbol{x}, \boldsymbol{h})$ for all units together,

$$E(\boldsymbol{s}) = -\sum_i c_i s_i - \sum_{i<j} J_{ij} s_i s_j, \tag{14.14}$$

which any physicist will recognise as an Ising model with fields c_i and couplings J_{ij}, and which any statistician will recognise as a pairwise undirected graphical model. The visible units are clamped to data during the positive phase; the hidden units are never observed and exist to let the marginal $p(\boldsymbol{x})$ be richer than the pairwise form would otherwise allow.

The general machine of Eq. (14.14) is essentially untrainable. Every unit couples to every other, so no conditional factorises, Gibbs sampling must be done one unit at a time, and *both* phases of Eq. (14.6) require Monte Carlo – the positive phase too, because with visible-visible and hidden-hidden couplings even $p(\boldsymbol{h} \mid \boldsymbol{x})$ is intractable. Two nested approximations, each noisy, compound into an algorithm that in practice does not converge.

14.4.1 The restriction

The remedy is to forbid connections within a layer. A *restricted* Boltzmann machine keeps only visible-hidden couplings, so its graph is bipartite:

$$E(\boldsymbol{x},\boldsymbol{h};\boldsymbol{\Theta}) = -\boldsymbol{a}^{\top}\boldsymbol{x} - \boldsymbol{b}^{\top}\boldsymbol{h} - \boldsymbol{x}^{\top}\boldsymbol{W}\boldsymbol{h}, \tag{14.15}$$

with visible biases $\boldsymbol{a} \in \mathbb{R}^{M}$, hidden biases $\boldsymbol{b} \in \mathbb{R}^{N}$ and weights $\boldsymbol{W} \in \mathbb{R}^{M\times N}$. This single deletion changes everything, and the following theorem says why.

Theorem 14.7 (Conditional independence). *For the restricted machine (14.15) with binary units in $\{0,1\}$, the conditionals factorise completely,*

$$p(\boldsymbol{h} \mid \boldsymbol{x}) = \prod_{j=1}^{N} p(h_j \mid \boldsymbol{x}), \qquad p(\boldsymbol{x} \mid \boldsymbol{h}) = \prod_{i=1}^{M} p(x_i \mid \boldsymbol{h}), \tag{14.16}$$

with logistic on-probabilities

$$p(h_j = 1 \mid \boldsymbol{x}) = \sigma\Big(b_j + \boldsymbol{x}^{\top}\boldsymbol{w}_{*j}\Big), \qquad p(x_i = 1 \mid \boldsymbol{h}) = \sigma\Big(a_i + \boldsymbol{w}_{i*}^{\top}\boldsymbol{h}\Big), \tag{14.17}$$

where $\sigma(z) = 1/(1+e^{-z})$. Furthermore the free energy of Definition 14.1 is available in closed form,

$$F(\boldsymbol{x}) = -\boldsymbol{a}^{\top}\boldsymbol{x} - \sum_{j=1}^{N} \log\Big(1 + e^{\,b_j + \boldsymbol{x}^{\top}\boldsymbol{w}_{*j}}\Big). \tag{14.18}$$

Proof. Sum the joint weight over the hidden configurations. Because Eq. (14.15) contains no $h_j h_{j'}$ term, the exponent is a sum of terms each involving a single h_j, so the exponential factorises and the sum over $\boldsymbol{h} \in \{0,1\}^{N}$ becomes a product of independent sums:

$$\sum_{\boldsymbol{h}} e^{-E} = e^{\boldsymbol{a}^{\top}\boldsymbol{x}} \sum_{\boldsymbol{h}} \prod_{j} e^{(b_j + \boldsymbol{x}^{\top}\boldsymbol{w}_{*j})h_j} = e^{\boldsymbol{a}^{\top}\boldsymbol{x}} \prod_{j} \sum_{h_j \in \{0,1\}} e^{(b_j + \boldsymbol{x}^{\top}\boldsymbol{w}_{*j})h_j} = e^{\boldsymbol{a}^{\top}\boldsymbol{x}} \prod_{j} \Big(1 + e^{\,b_j + \boldsymbol{x}^{\top}\boldsymbol{w}_{*j}}\Big).$$

Taking $-\log$ gives Eq. (14.18). Dividing the joint by this marginal, the factor $e^{\boldsymbol{a}^{\top}\boldsymbol{x}}$ cancels and

$$p(\boldsymbol{h} \mid \boldsymbol{x}) = \prod_{j} \frac{e^{(b_j + \boldsymbol{x}^{\top}\boldsymbol{w}_{*j})h_j}}{1 + e^{\,b_j + \boldsymbol{x}^{\top}\boldsymbol{w}_{*j}}},$$

which is Eq. (14.16); setting $h_j = 1$ and dividing numerator and denominator by $e^{\,b_j + \boldsymbol{x}^{\top}\boldsymbol{w}_{*j}}$ gives the logistic form. The visible case is identical by the symmetry of Eq. (14.15) under $\boldsymbol{x} \leftrightarrow \boldsymbol{h}$, $\boldsymbol{a} \leftrightarrow \boldsymbol{b}$, $\boldsymbol{W} \leftrightarrow \boldsymbol{W}^{\top}$.

Theorem 14.7 is what makes restricted machines usable, and it buys three things at once. The positive phase of Eq. (14.6) becomes exact, since $p(\boldsymbol{h} \mid \boldsymbol{x})$ is known in closed form and no sampling is needed. Gibbs sampling can update an entire layer at once – *block* Gibbs –

$$\boldsymbol{h}^{(t)} \sim p(\boldsymbol{h} \mid \boldsymbol{x}^{(t)}), \qquad \boldsymbol{x}^{(t+1)} \sim p(\boldsymbol{x} \mid \boldsymbol{h}^{(t)}), \tag{14.19}$$

because within a layer the units are independent given the other layer, so $M+N$ scalar updates collapse into two vector operations. And the free energy (14.18) is computable, so unnormalised probabilities can be compared even though Z cannot.

Note also what Eq. (14.17) says about the arithmetic: it is $\sigma(\boldsymbol{W}^{\top}\boldsymbol{x} + \boldsymbol{b})$, which is precisely the forward pass of a single logistic layer from Chapter 5. An RBM is a neural-network layer read as a probability model, with the same weights used in both directions.

Specialising Eq. (14.6) to Eq. (14.15), where $\partial E/\partial w_{ij} = -x_i h_j$, gives the form actually implemented:

$$\frac{\partial \mathscr{L}}{\partial w_{ij}} = \langle x_i h_j \rangle_{\text{data}} - \langle x_i h_j \rangle_{\text{model}}, \quad \frac{\partial \mathscr{L}}{\partial a_i} = \langle x_i \rangle_{\text{data}} - \langle x_i \rangle_{\text{model}}, \tag{14.20}$$

and similarly for b_j. Every term is a correlation between a visible and a hidden unit: learning matches the model's correlations to the data's.

14.4.2 Gaussian-binary machines

Binary visible units are wrong for continuous data. The Gaussian-binary machine replaces them by real variables with a quadratic term,

$$E_{GB}(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta}) = \sum_{i=1}^{M} \frac{(x_i - a_i)^2}{2\sigma_i^2} - \sum_{j=1}^{N} b_j h_j - \sum_{i,j} \frac{x_i w_{ij} h_j}{\sigma_i^2}, \tag{14.21}$$

which leaves the hidden conditional unchanged in form, $p(h_j = 1 \mid \boldsymbol{x}) = \sigma(b_j + \sum_i x_i w_{ij}/\sigma_i^2)$, and makes the visible conditional a Gaussian,

$$p(x_i \mid \boldsymbol{h}) = \mathscr{N}\left(a_i + \boldsymbol{w}_{i*}^{\mathsf{T}} \boldsymbol{h},\ \sigma_i^2\right). \tag{14.22}$$

Block Gibbs works exactly as before, one layer at a time. The variances σ_i^2 are usually fixed to one after standardising the data, because learning them jointly with $\boldsymbol{W}$ is notoriously unstable – the energy can be driven down by shrinking σ_i without improving the fit.

14.5 Contrastive divergence

Theorem 14.2 left one term requiring samples from the model. The honest procedure is to run the chain (14.19) to equilibrium from any start, which takes an unknown and possibly enormous number of sweeps. *Contrastive divergence* does something cheaper and admits it: start the chain *at the data* and run it for k sweeps only, usually $k = 1$.

$$\nabla_{\boldsymbol{\Theta}} \mathscr{L} \ \approx\ -\langle \nabla E \rangle_{\text{data}} + \langle \nabla E \rangle_{k \text{ sweeps from the data}}. \tag{14.23}$$

The reasoning is that the data are, if the model is any good, already close to the model distribution, so a short chain moves in the direction the model is wrong and that direction is what the gradient needs.

This is an approximation of a particular and awkward kind.

CD-k is biased, not merely noisy. A noisy estimator has the right expectation and averages out over updates; a biased one does not. CD-k is biased for every finite k, and it is not the gradient of any function – there is no "contrastive divergence objective" that it descends. What can be said is that the bias vanishes as $k \to \infty$ and that in practice it points close enough to the truth for gradient ascent to work. Section 14.6.1 measures both halves of that claim.

The notebox says the bias vanishes as $k \to \infty$. That is worth converting into a statement about *how fast*, because it is the statement that decides whether $k = 1$ is defensible.

Proposition 14.8 (The CD-k bias is the mixing of the chain). *Write $\boldsymbol{s}(\boldsymbol{x}) = -\nabla_{\boldsymbol{\theta}} F(\boldsymbol{x})$ for the statistic whose expectation forms each phase of Eq. (14.6), let p_k be the distribution of the chain after k sweeps started at the data and p_∞ the model distribution. Then the bias of CD-k obeys*

$$\left\| \mathbb{E}\left[\nabla^{\mathrm{CD}\text{-}k}\right] - \nabla_{\boldsymbol{\theta}} \mathscr{L} \right\|_\infty = \left\| \mathbb{E}_{p_\infty}[\boldsymbol{s}] - \mathbb{E}_{p_k}[\boldsymbol{s}] \right\|_\infty \le 2 \, \|\boldsymbol{s}\|_\infty \, \mathrm{TV}(p_k, p_\infty), \tag{14.24}$$

where TV *is the total-variation distance. If the chain is geometrically ergodic with rate $\rho < 1$, so that $\mathrm{TV}(p_k, p_\infty) \le C\rho^k$, the bias decays geometrically in k.*

Proof. The positive phase of Eq. (14.23) is the same in both estimators, so the whole difference is in the negative phase, which is $\mathbb{E}_{p_k}[\boldsymbol{s}]$ for CD-k and $\mathbb{E}_{p_\infty}[\boldsymbol{s}]$ for the exact gradient. For any bounded f, $|\mathbb{E}_p f - \mathbb{E}_q f| \le 2\|f\|_\infty \mathrm{TV}(p, q)$ by the variational characterisation of total variation, applied coordinatewise. The last statement is the definition of geometric ergodicity substituted into the bound.

Two things follow. The bias is not a property of contrastive divergence as such but of *how quickly the Gibbs chain forgets where it started*, so anything that speeds up mixing reduces it – which is the argument for Section 14.6.1 measuring the mixing directly, and the reason a machine with large weights, whose chain mixes slowly, is the hard case. And because the bound is geometric rather than $\mathcal{O}(1/\sqrt{k})$, a handful of sweeps should buy most of what is available; Section 14.6.1 finds the error falling from 0.33 at $k = 1$ to under 0.01 by $k = 5$, which is what Eq. (14.24) predicts.

A cheap improvement is *persistent* contrastive divergence, which keeps the Gibbs chain running across parameter updates instead of restarting it at the data each time. Since the parameters move slowly, the chain stays near equilibrium and the bias falls, at no extra cost per update. In terms of Eq. (14.24), persistent CD replaces p_k – the data smeared by k sweeps – by the state of a chain that has been running for the whole of training, so $\mathrm{TV}(p_k, p_\infty)$ is small for a different and usually better reason. Section 14.7.2 measures whether the promise is kept on real data, and the answer is not a simple yes.

14.6 Implementation and verification

Everything in Sections 14.4.1 and 14.5 is a few lines, and small machines let us check them against exact answers – because for M up to about twenty, Z *can* be enumerated. That is the strategy of this section: use brute force where it is available to audit the approximations we are forced to use elsewhere.

```
def free_energy(P, X):
    """F(x) = -a.x - sum_j log(1 + exp(b_j + (x^T W)_j)),  Eq. (14.freeenergy).

    The hidden units have been summed out exactly, which is what the
    restriction to a bipartite graph buys.
    """
    z = X @ P["W"] + P["b"]
    return -(X @ P["a"]) - np.sum(np.logaddexp(0.0, z), axis=-1)


def p_h_given_x(P, X):
    return sigmoid(X @ P["W"] + P["b"])


def p_x_given_h(P, H):
    return sigmoid(H @ P["W"].T + P["a"])
```

```
def gibbs_step(P, X, rng):
    """One sweep of block Gibbs: sample all h, then all x, Eq. (14.blockgibbs)."""
    ph = p_h_given_x(P, X)
    H = (rng.random(ph.shape) < ph).astype(float)
    px = p_x_given_h(P, H)
    Xn = (rng.random(px.shape) < px).astype(float)
    return Xn, H, ph, px
```

The exact gradient enumerates the 2^M visible states and weights them by the true model probability; the CD-k gradient replaces that sum by a short chain.

```
def exact_gradient(P, X):
    """The true gradient of the log-likelihood, Eq. (14.gradient).

    The negative phase is computed by enumerating all 2^M visible states and
    weighting by the exact model distribution.  Only feasible for small M, and
    that is exactly why it is useful: it is the ground truth against which
    contrastive divergence can be judged.
    """
    pos = positive_phase(P, X)
    V = all_states(len(P["a"]))
    logp = -free_energy(P, V)
    logp = logp - np.logaddexp.reduce(logp)
    w = np.exp(logp)                                    # exact p(v)
    ph = p_h_given_x(P, V)
    neg = {"W": (V * w[:, None]).T @ ph, "a": w @ V, "b": w @ ph}
    return {k: pos[k] - neg[k] for k in pos}
```

The gradient formula.

Theorem 14.2 is an identity and can be checked as one, by comparing `exact_gradient` against central differences of the exactly computed log-likelihood:

```
--- exact gradient vs finite differences of the log-likelihood ---
  W: max relative error over all 24 entries = 2.61e-08
  a: max relative error over all 6 entries = 6.98e-09
  b: max relative error over all 4 entries = 1.46e-08
```

The sampler.

Proposition 14.6 says block Gibbs has the model distribution as its stationary distribution. With $M = 6$ we can enumerate all 64 visible states, compute $p(\boldsymbol{x})$ exactly, and compare against the empirical frequencies of a long chain in total-variation distance:

```
=== does block Gibbs sample the model distribution? ===
     200 sweeps: total-variation distance to the exact p(v) = 0.0103
    2000 sweeps: total-variation distance to the exact p(v) = 0.0034
   20000 sweeps: total-variation distance to the exact p(v) = 0.0008
  (a uniform guess would give 0.3601)
```

The distance falls as the square root of the number of samples, which is the Monte Carlo rate of Chapter 2 and is visible as the slope in Figure 14.1(b).

14.6.1 How biased is CD-k?

Now the question the notebox raised. We take a machine small enough for the exact gradient, average the CD-k estimate over sixty independent chains so that sampling noise is negligible, and compare.

```
=== CD-k is a BIASED estimator of the gradient ===
  k     cos(CD-k, exact)   ||CD-k - exact|| / ||exact||
    1     0.986291          0.3283
    2     0.998131          0.1037
    5     0.999982          0.0095
   10     0.999956          0.0100
   25     0.999963          0.0089
  100     0.999988          0.0068
```

The two columns tell different halves of the story and both matter. CD-1 is wrong by *thirty-three per cent* in magnitude – a bias no amount of averaging removes, since every one of the sixty chains carries it. But its cosine with the true gradient is 0.986, an angle of about nine degrees. It points very nearly the right way and is merely the wrong length.

That is exactly why CD-1 works. Gradient ascent does not need the gradient; it needs a direction with positive inner product with the gradient, and a step size it can tune anyway. A systematically short vector at nine degrees to the truth is a perfectly serviceable ascent direction. By $k = 5$ the error is under one per cent and the remaining discrepancy is the noise floor of our sixty-chain average.

What the bias costs.

Bias in the gradient need not mean a worse model, so we measure that too. On *bars and stripes* – a standard small benchmark whose 14 patterns on a 3×3 grid live among $2^9 = 512$ configurations, so the likelihood is exact – we train identical machines with CD-1, CD-10 and the exact gradient:

```
bars and stripes 3x3: 14 distinct patterns of 512 possible
log-likelihood of the perfect model: -2.6391 per image

  training signal    final log-likelihood (3 seeds)
  CD-1               -4.3869   (-4.5322 to -4.3061)
  CD-10              -4.2605   (-4.4092 to -4.1716)
  exact gradient     -4.2457   (-4.4597 to -4.1265)
```

CD-10 and the exact gradient are indistinguishable, -4.261 against -4.246, a gap of 0.015 nats that is well inside the seed-to-seed spread. CD-1 is measurably but not catastrophically worse at -4.387, about 0.14 nats behind. So the thirty-three per cent gradient bias of CD-1 costs roughly a tenth of a nat of likelihood, and ten Gibbs sweeps buy essentially all of it back. None of the three reaches the perfect -2.639, which is a statement about the capacity of eight hidden units and three thousand updates rather than about the training signal.

14.7 Implementations in the libraries

Neither PyTorch nor TensorFlow ships a Boltzmann machine, which is itself informative: the architecture predates automatic differentiation and its gradient, Eq. (14.20), is not obtained by backpropagation through a loss. It is a difference of two expectations, and one of them

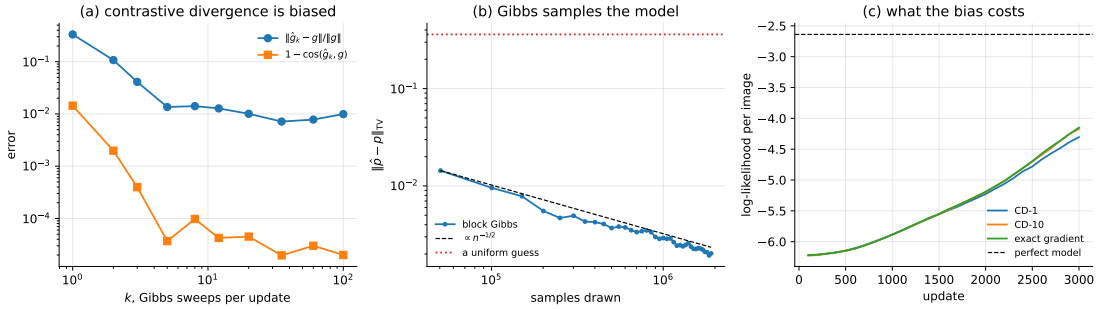


Fig. 14.1 (a) The bias of contrastive divergence against the number of Gibbs sweeps, measured against the exact gradient of Theorem 14.2 with the sampling noise averaged away. The relative error falls from 0.33 at $k = 1$ to under 0.01 by $k = 5$, while the direction is already almost right at $k = 1$. (b) Total-variation distance between the empirical distribution of a block-Gibbs chain and the exact model distribution, against the number of samples drawn; the dashed line is the $n^{-1/2}$ Monte Carlo rate and the dotted line is what a uniform guess would give. (c) Log-likelihood during training on bars and stripes for the three training signals, with the likelihood of a perfect model marked.

comes from a sampler. That makes this section a slightly different exercise from the corresponding sections of Chapters 10 to 13: what is being checked against the libraries is not a layer but a *theorem*. What the libraries provide is the tensor arithmetic and the optimisers, and the RBM is written on top.

```
import torch
import torch.nn as nn


class RBM(nn.Module):
    """Binary-binary RBM, Eq. (14.energyBB).  Note there is no forward()
    returning a loss: the gradient is Eq. (14.rbmgradient), assigned by hand."""

    def __init__(self, M=784, N=256, k=1):
        super().__init__()
        self.W = nn.Parameter(torch.randn(M, N) * 0.01)
        self.a = nn.Parameter(torch.zeros(M))
        self.b = nn.Parameter(torch.zeros(N))
        self.k = k

    def free_energy(self, x):                          # Eq. (14.freeBB)
        return -(x @ self.a) - torch.nn.functional.softplus(
            x @ self.W + self.b).sum(1)

    def gibbs(self, x):                                # Eq. (14.blockgibbs)
        ph = torch.sigmoid(x @ self.W + self.b)
        h = torch.bernoulli(ph)
        px = torch.sigmoid(h @ self.W.t() + self.a)
        return torch.bernoulli(px)

    def cd_loss(self, x):
        """A surrogate whose gradient equals Eq. (14.cdk).

        The negative sample is detached so that no gradient flows through the
        sampler: the chain supplies configurations, not a differentiable path.
        """
        v = x
        for _ in range(self.k):
            v = self.gibbs(v)
        return self.free_energy(x).mean() - self.free_energy(v.detach()).mean()
```

```
model = RBM(784, 256, k=1)
opt = torch.optim.SGD(model.parameters(), lr=0.01)
for epoch in range(20):
    for xb, _ in train_loader:
        xb = torch.bernoulli(xb.view(-1, 784))      # binarise the pixels
        loss = model.cd_loss(xb)
        opt.zero_grad(set_to_none=True)
        loss.backward()
        opt.step()
```

The trick in `cd_loss` is worth understanding rather than copying. The difference of free energies is *not* the log-likelihood – that would need Z – but its gradient is exactly Eq. (14.23), because ∇F at the data gives the positive phase and ∇F at the samples gives the estimated negative phase. The `detach()` is essential: without it, autograd would try to differentiate through the Bernoulli draws, which is both meaningless and wrong.

The same in TensorFlow, where the two phases are written out explicitly rather than routed through a surrogate loss:

```
import tensorflow as tf


class RBM(tf.Module):
    def __init__(self, M=784, N=256, k=1):
        self.W = tf.Variable(tf.random.normal([M, N], stddev=0.01))
        self.a = tf.Variable(tf.zeros([M]))
        self.b = tf.Variable(tf.zeros([N]))
        self.k = k

    def sample(self, p):
        return tf.cast(tf.random.uniform(tf.shape(p)) < p, tf.float32)

    def gibbs(self, x):
        h = self.sample(tf.sigmoid(x @ self.W + self.b))
        return self.sample(tf.sigmoid(h @ tf.transpose(self.W) + self.a))

    @tf.function
    def cd_update(self, x, lr=0.01):
        """Eq. (14.rbmgradient) with the negative phase from k Gibbs sweeps."""
        ph_data = tf.sigmoid(x @ self.W + self.b)
        v = x
        for _ in range(self.k):
            v = self.gibbs(v)
        ph_model = tf.sigmoid(v @ self.W + self.b)
        n = tf.cast(tf.shape(x)[0], tf.float32)
        self.W.assign_add(lr * (tf.transpose(x) @ ph_data
                                - tf.transpose(v) @ ph_model) / n)
        self.a.assign_add(lr * tf.reduce_mean(x - v, axis=0))
        self.b.assign_add(lr * tf.reduce_mean(ph_data - ph_model, axis=0))
```

14.7.1 Do the implementations agree, and is the theorem right?

The three implementations should compute the same free energy, the same conditionals and the same contrastive-divergence step. Pushing one set of weights into all of them, with the Gibbs samples shared so that the comparison is of arithmetic and not of two random number generators:

```
=== 1. free energy and conditionals, identical weights ===
```

```
  F(x), Eq. (14.freeenergy)  max |ours - torch| : 8.882e-16
  F(x), Eq. (14.freeenergy)  max |ours - keras| : 8.882e-16
  p(h|x), Eq. (14.condh)     max |ours - torch| : 1.110e-16
  p(h|x), Eq. (14.condh)     max |ours - keras| : 1.110e-16
  log Z by enumeration (512 states)           : 9.806863470178
  mean log-likelihood, Eq. (14.loglik)          : -6.425359835072

=== 3. one CD-1 gradient with the same Gibbs samples ===
   parameter   |ours - torch|   |ours - keras|
   W             5.551e-17        1.110e-16
   a             0.000e+00        0.000e+00
   b             2.220e-16        2.220e-16
```

There is a third implementation to hand. `sklearn.neural_network.BernoulliRBM` is written by other people for other reasons, and stores its weight matrix transposed, as (N, M) rather than our (M, N). With that one convention undone:

```
=== 4. scikit-learn's BernoulliRBM on the same weights ===
  p(h|x)  max |ours - sklearn|      : 1.110e-16
  F(x)    max |ours - sklearn|      : 0.000e+00
```

Now the check that is worth more than all of those. Theorem 14.2 asserts that the gradient of the log-likelihood splits into a positive and a negative phase, Eq. (14.6). For a machine small enough to enumerate, that assertion is testable directly, because $\log p(\boldsymbol{x}) = -F(\boldsymbol{x}) - \log Z$ is then an ordinary differentiable function of the parameters: F by Eq. (14.4) and $\log Z$ by a `logsumexp` over all 2^M visible states. Handing that expression to PyTorch and calling `.backward()` computes the gradient with no notion of a phase, of a sampler, or of a Boltzmann machine at all.

```
=== 2. Theorem 14.gradient checked against autograd ===
   parameter   shape      |autograd - Eq. (14.gradient)|      scale
   W      (9, 5)                    3.053e-16   2.208e-01
   a      (9,)                      2.220e-16   3.141e-01
   b      (5,)                      2.220e-16   6.851e-02
  our log-likelihood : -6.425359835072
  torch's            : -6.425359835072
  worst disagreement : 3.053e-16
```

The two-phase decomposition is not an approximation, a convention or a heuristic. It *is* the derivative of the log-likelihood, and automatic differentiation rediscovers it without being told. What automatic differentiation cannot do is the thing that makes the architecture hard: the $\log Z$ above required enumerating five hundred and twelve states, and at $M = 784$ there are 2^{784} of them. Everything difficult about training a Boltzmann machine is in that one term, and it is why the chapter needs Markov chains at all.

14.7.2 On real data: CD-1, CD-10 and persistent CD

Section 14.6.1 measured the bias exactly, on nine visible units. The question that leaves open is whether it matters at a scale where the exact answer is unavailable. We train a machine with $M = 784$ and $N = 128$ on binarised MNIST for five epochs with each of the three training signals. The likelihood is out of reach, so we report the *pseudo-likelihood*

$$\log \mathrm{PL}(\boldsymbol{x}) = M \log \sigma\Big(F(\tilde{\boldsymbol{x}}^{(i)}) - F(\boldsymbol{x})\Big), \qquad i \sim \mathrm{Uniform}\{1, \dots, M\}, \tag{14.25}$$

where $\tilde{\boldsymbol{x}}^{(i)}$ is $\boldsymbol{x}$ with pixel i flipped. The partition function cancels between the two free energies, which is the whole point: it is computable exactly where Eq. (14.5) is not.

Table 14.1 A restricted Boltzmann machine on binarised MNIST, $M = 784$, $N = 128$, five epochs of stochastic gradient ascent with learning rate 0.05 and momentum 0.5. Pseudo-likelihood, Eq. (14.25), is per image on 2000 held-out digits; larger is better. The reconstruction error is the mean squared difference between a digit and its one-step mean-field reconstruction.

	PyTorch		TensorFlow	
Signal	log PL	s/epoch	log PL	s/epoch
CD-1	-84.71	1	-93.20	9
CD-10	$-\mathbf{81.00}$	8	-96.15	44
PCD-1	-91.14	1	-101.64	8

Three readings, and the third is the interesting one.

CD-10 beats CD-1 in PyTorch, -81.0 against -84.7, at eight times the cost per epoch. That is Proposition 14.8 on real data: more sweeps, less bias, better model – and the improvement is real but modest, which is the same conclusion the bars-and-stripes experiment of Section 14.6.1 reached with an exact likelihood in hand.

Persistent CD is *worse* than CD-1 here, -91.1 against -84.7, in both frameworks. This is the opposite of what the paragraph introducing it leads one to expect, and we report it rather than dropping the row. The explanation is in Eq. (14.24): persistent CD is a better estimator of the gradient *once the chain has equilibrated*, and five epochs at learning rate 0.05 is not long enough for that. Early in training the parameters move faster than the chain can follow, so the persistent chain samples a model that no longer exists. The standard remedy is a smaller learning rate and many more epochs, which is precisely the regime in which persistent CD is known to win; at five epochs it loses.

The two frameworks disagree by several nats – -84.7 against -93.2 for CD-1 – while Section 14.7.1 established that they compute the same gradient to 10^{-16}. Nothing is wrong. The two draw different Bernoulli samples in the negative phase and shuffle the data differently, and a stochastic gradient with a biased estimator is sensitive to both. The *ordering* of the three signals is identical in the two columns, and that ordering is the result; the individual numbers are one run each of a stochastic procedure.

Finally, what the machine has learned. Starting a Gibbs chain from uniform noise and running it in the trained model:

```
=== samples from the trained machine ===
  after     1 Gibbs sweeps: mean free energy    182.23
  after    10 Gibbs sweeps: mean free energy   -277.05
  after   100 Gibbs sweeps: mean free energy   -360.19
  after  1000 Gibbs sweeps: mean free energy   -377.84
```

The free energy of Eq. (14.4) is $-\log$ of an unnormalised probability, so a chain descending from $+182$ to -378 over a thousand sweeps is a chain walking from a region the model considers absurd into the region it considers likely. That descent is what Section 14.3 promised and it is also, read the other way, a picture of how long the negative phase would take if we insisted on doing it honestly. Figure 14.2 shows the samples themselves at those four times, along with the learned filters.

14.8 Summary and the programs

A Boltzmann machine models the data distribution itself as a canonical ensemble, Eq. (14.1). Everything follows from one obstruction: the partition function (14.2) is a sum over 2^{M+N} configurations and is permanently unavailable.

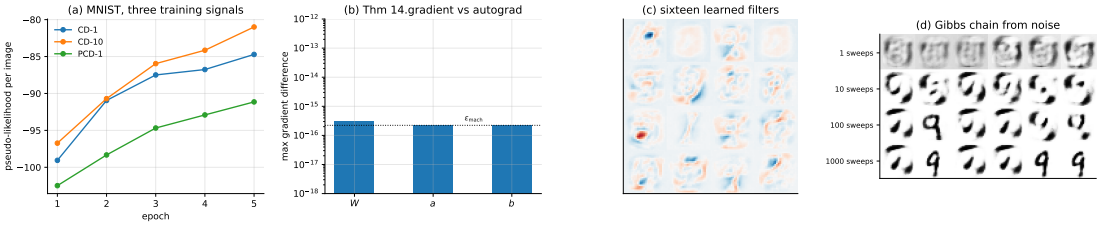


Fig. 14.2 (a) Pseudo-likelihood, Eq. (14.25), against epoch for the three training signals of Table 14.1 in PyTorch. (b) The largest disagreement between the exact gradient of Theorem 14.2 and the same gradient obtained by automatic differentiation of $-F - \log Z$, against the double-precision unit roundoff. (c) Sixteen of the 128 learned filters, columns of $\boldsymbol{W}$ reshaped to 28×28: strokes and pen fragments rather than whole digits. (d) A Gibbs chain of Eq. (14.19) started from uniform noise, shown after 1, 10, 100 and 1000 sweeps; the mean free energy falls from $+182$ to -378 along the way.

Theorem 14.2 locates the damage precisely. The gradient of the log-likelihood is a difference of two expectations – one over the data, which is free, and one over the model, which requires samples. The value of Z is never needed, only samples from the distribution it normalises, and that loophole is what Metropolis (Proposition 14.5) and Gibbs (Proposition 14.6) exploit, both by way of detailed balance.

Theorem 14.7 is what makes the restricted machine practical. Deleting the within-layer couplings makes both conditionals factorise into logistic units, so the positive phase becomes exact, Gibbs sampling updates a whole layer at a time, and the free energy has the closed form (14.18). An RBM is a logistic layer read as a probability model.

We verified what could be verified exactly. The gradient identity matched finite differences to 10^{-8}; block Gibbs converged to the enumerated model distribution at the $n^{-1/2}$ Monte Carlo rate, reaching a total-variation distance of 0.0008 against 0.36 for a uniform guess.

The measurement to remember is Section 14.6.1. CD-1 is wrong by 33% in magnitude, a bias that averaging cannot remove, and yet its cosine with the true gradient is 0.986 – about nine degrees. It is a short vector pointing almost the right way, which is all gradient ascent requires. On bars and stripes that bias cost 0.14 nats of likelihood against the exact gradient, and ten Gibbs sweeps recovered all but 0.015 of it. Approximate gradients are acceptable when their direction is right; it is worth knowing which of the two one is relying on. Proposition 14.8 explains the shape of that result: the bias is $\mathrm{TV}(p_k, p_\infty)$ up to a constant, so it decays geometrically in k and a handful of sweeps buys most of what is available.

Section 14.7.1 closes the loop on Theorem 14.2 itself. For a machine small enough to enumerate, $\log p(\boldsymbol{x}) = -F(\boldsymbol{x}) - \log Z$ is an ordinary differentiable function, and asking PyTorch to differentiate it – with no notion of a positive or a negative phase – returns the two-phase formula to 3×10^{-16}. The decomposition is not a convention; it is what the derivative is. What autograd cannot supply is the $\log Z$ that made it possible: five hundred and twelve states here, 2^{784} on MNIST, and every difficulty in the chapter lives in that gap.

On MNIST, Table 14.1, CD-10 beat CD-1 by four nats of pseudo-likelihood at eight times the cost, in line with the exact measurement. Persistent CD came *last* in both frameworks, which is the opposite of the usual advice and is explained by Eq. (14.24): a persistent chain is the better estimator only once it has caught up with the parameters, and five epochs at learning rate 0.05 do not allow that. The two frameworks differed by several nats while computing identical gradients, which is worth remembering whenever a single run is offered as evidence about an algorithm.

The programs are in the directory
`doc/BookML/BookPrograms/chapter14_boltzmann`.
Every listing above appears there as a numbered file, and three modules run start to finish and reproduce the numbers quoted in the text:

- `rbm.py` – the energy and free energy, the exact partition function by enumeration, the conditionals, block Gibbs, the exact gradient and CD-k with an optional persistent chain.
- `cross_check_rbm.py` – the comparison of Section 14.7.1: free energy, conditionals and one CD-1 step against PyTorch, TensorFlow and scikit-learn's `BernoulliRBM`, and Theorem 14.2 against automatic differentiation of $-F - \log Z$.
- `rbm_torch.py` and `rbm_tf.py` – the MNIST runs of Table 14.1, the pseudo-likelihood of Eq. (14.25), and the Gibbs chain from noise.
- `mnist_data.py` – a loader that reads MNIST from a local `.npz`, so that the programs run without network access.
- `verify_rbm.py` – the gradient check against finite differences, the Gibbs convergence study, and the CD-k bias table.
- `run_bas.py` – the bars-and-stripes experiment.

The figure is generated by `ch14_figures.py` in `doc/BookML/BookFigures`; it is not drawn by hand.

14.9 Exercises

Warm-up exercises

1. **Counting.** (a) How many terms has the partition function of an RBM with $M = 784$ and $N = 500$? Express the answer as a power of ten. (b) At 10^9 terms per second, how long would the sum take? (c) For which M is enumeration feasible on a laptop, and how does that compare with the smallest interesting data set?
2. **The gradient identity.** Derive Eq. (14.20) from Theorem 14.2 by computing $\partial E/\partial w_{ij}$, $\partial E/\partial a_i$ and $\partial E/\partial b_j$ for the energy (14.15). Then explain in one sentence why the positive phase needs no sampling but the negative phase does.
3. **Factorisation.** Prove Theorem 14.7 for the visible conditional by repeating the argument with the roles of $\boldsymbol{x}$ and $\boldsymbol{h}$ exchanged. Then show that adding a single visible-visible coupling $J_{12}x_1x_2$ destroys the factorisation, and say what that costs computationally.
4. **Detailed balance.** (a) Verify Proposition 14.4 on a two-state chain. (b) Construct a three-state chain that is stationary for the uniform distribution but violates detailed balance, confirming that the converse fails. (c) Show that the Metropolis acceptance (14.12) is unchanged if π is multiplied by any constant, and explain why this is the fact the whole chapter depends on.
5. **Gibbs from Metropolis.** Work through Proposition 14.6 for a two-variable distribution of your choice, and confirm numerically that the acceptance probability is one for every proposal. Then find a distribution for which Gibbs mixes very slowly, and explain the geometry that causes it.
6. **Free energy.** Verify Eq. (14.18) numerically by computing $-\log \sum_{\boldsymbol{h}} e^{-E}$ by brute force for a machine with $N = 10$ and comparing. Then show that F is the log-sum-exp of N terms and explain why `logaddexp` rather than `log(1+exp())` is used in the code.
7. **The bias bound.** (a) Prove Proposition 14.8, including the inequality

$$\left|\mathbb{E}_p f - \mathbb{E}_q f\right| \le 2\|f\|_\infty \,\mathrm{TV}(p,q).$$

(b) Measure $\mathrm{TV}(p_k, p_\infty)$ directly for the small machine of Section 14.6.1 at $k = 1, 2, 5, 10$ and check that the measured bias falls at the same rate. (c) Scale the weights of the machine by a factor of three and repeat. Which of the two quantities in Eq. (14.24) grows, and what does that say about training a machine whose weights have become large?

8. **Pseudo-likelihood.** (a) Show that the partition function cancels in Eq. (14.25), so that $\log \mathrm{PL}$ is computable when Eq. (14.5) is not. (b) Show that $\log \mathrm{PL}$ is *not* a bound on $\log p$ in either direction, by constructing a distribution where it is larger and one where it is smaller. (c) For the small machine where both are available, compute the two and plot one against the other over training. Do they agree on which model is better?
9. **Theorem 14.gradient without phases.** Reproduce Section 14.7.1: build $-F(\boldsymbol{x}) - \log Z$ for a machine with $M = 9$, differentiate it with automatic differentiation, and compare with Eq. (14.6). Then raise M one unit at a time and record the time taken, to see where the enumeration becomes impossible.
10. **Persistent contrastive divergence.** Section 14.7.2 found PCD-1 worse than CD-1 after five epochs. (a) Repeat with a learning rate of 0.005 and forty epochs and report whether the ordering reverses. (b) Explain, using Eq. (14.24), why the learning rate and not the number of sweeps is the relevant knob for a persistent chain. (c) Track the free energy of the persistent chain during training. What does it do when the chain falls behind the parameters?

Project-style exercise: a Boltzmann machine from scratch

Part a: the machinery.

Implement the energy, free energy, exact partition function, both conditionals and block Gibbs. Verify Eq. (14.18) against brute-force summation over $\boldsymbol{h}$, and verify Theorem 14.2 by comparing the exact gradient against central differences of the exact log-likelihood.

Part b: the sampler.

Reproduce the Gibbs convergence study of Section 14.6. Measure the integrated autocorrelation time of the chain, using the machinery of Chapter 2, and study how it grows as the weights are scaled up. At what coupling strength does the chain stop mixing usefully, and what is happening physically at that point?

Part c: the bias.

Reproduce the CD-k table of Section 14.6.1, separating bias from variance: average many chains at fixed parameters to isolate the bias, and report the variance separately. Then implement persistent contrastive divergence and place it on the same axes. Does PCD reduce the bias, the variance, or both?

Part d: what the bias costs.

Reproduce the bars-and-stripes comparison, then extend it: plot final log-likelihood against k for $k = 1, \ldots, 20$, and against the number of hidden units. Is there a regime in which CD-1 is not merely worse but qualitatively wrong – for instance, assigning high probability to configurations the data never contain?

Part e: a physical system.

Train an RBM on configurations of the two-dimensional Ising model sampled at several temperatures. Inspect the learned weights $\boldsymbol{W}$: do the hidden units discover local structure, and does anything change across the critical temperature? Compare the model's energy histogram against the true one. This is a case where you know the answer, which makes it the right place to find out whether the method works.

Part f: Gaussian-binary.

Implement the machine of Eq. (14.21) and fit it to continuous data. Verify the conditional (14.22) numerically. Then attempt to learn the σ_i jointly with the weights and document the instability described in Section 14.4.2; propose and test a remedy.

Part g: against an autoencoder.

Train an RBM and the autoencoder of Chapter 12 on the same data with the same number of hidden units, and compare what they learn. The autoencoder optimises reconstruction and the RBM optimises likelihood: exhibit a case where the two disagree, and say which you would trust for which purpose.

Chapter 15
Variational Autoencoders

Two chapters have now approached generative modelling from opposite ends and each hit the same wall. Chapter 12 built an autoencoder that compressed data beautifully but was not a probability model at all: nothing in Eq. (12.2) says what the code $\boldsymbol{z}$ should be distributed like, so sampling a code and decoding it produces nothing in particular. Chapter 14 built a genuine probability model whose likelihood was blocked by an intractable partition function, and spent itself on Markov chains to get around that.

The variational autoencoder takes a third route. It keeps the encoder-decoder architecture of Chapter 12 but makes both halves *probability distributions*, and it defeats the intractable likelihood not by sampling from the model, as in Chapter 14, but by optimising a computable *lower bound* on it. The bound is called the evidence lower bound, or ELBO, and deriving it, understanding when it is tight, and making it differentiable are the three things this chapter does.

The result is a model that is a real probability distribution, can be sampled in a single pass with no Markov chain, and trains by ordinary stochastic gradient descent. It also has two characteristic failures – blurry samples and posterior collapse – both of which follow from the objective rather than from bad luck, and both of which we measure. The chapter ends by showing how stacking the construction leads directly to the diffusion models of Chapter 16.

The material follows the lecture notes for weeks thirteen to fifteen of FYS-STK3155/4155 and the original account by Kingma and Welling [54].

15.1 The latent-variable model

The model is the probabilistic PCA of Section 12.7 with the linear map replaced by a network. A latent variable $\boldsymbol{h} \in \mathbb{R}^{d_h}$ is drawn from a fixed prior, and the observation is drawn from a conditional whose parameters a network computes:

$$\boldsymbol{h} \sim p(\boldsymbol{h}) = \mathcal{N}(\mathbf{0}, \boldsymbol{I}), \qquad \boldsymbol{x} \sim p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid \boldsymbol{h}), \tag{15.1}$$

with $p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid \boldsymbol{h})$ Gaussian for continuous data or factorised Bernoulli for binary data, its mean given by a *decoder* network $\boldsymbol{\theta}$. The prior is standard normal by convention and by convenience; Section 15.5 discusses what that convention costs.

What we want to maximise is the likelihood the model assigns to the data, the *evidence*

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \int p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid \boldsymbol{h})\, p(\boldsymbol{h})\, \mathrm{d}\boldsymbol{h}. \tag{15.2}$$

This integral is the obstruction. It has no closed form once the decoder is a network, and it cannot be estimated well by sampling the prior: for almost every $\boldsymbol{h}$ drawn from $\mathcal{N}(\mathbf{0},\boldsymbol{I})$, the conditional $p_{\boldsymbol{\theta}}(\boldsymbol{x}\mid\boldsymbol{h})$ is effectively zero for a given $\boldsymbol{x}$, so a Monte Carlo estimate of Eq. (15.2) is dominated by the rare draws that happen to be near the right region and has enormous variance.

The same obstruction, three times. Chapter 14 could not compute $Z=\sum_{\boldsymbol{x},\boldsymbol{h}} e^{-E}$; here we cannot compute $\int p(\boldsymbol{x}\mid\boldsymbol{h})p(\boldsymbol{h})\,\mathrm{d}\boldsymbol{h}$. Both are sums over a latent space too large to enumerate, and the responses are different: Chapter 14 sampled the model with a Markov chain and accepted a biased gradient, while this chapter constructs a bound it can differentiate exactly. It is worth keeping the two strategies side by side, because Chapter 16 uses the second.

The key idea is to avoid sampling $\boldsymbol{h}$ from the prior and instead sample it from a distribution that already knows about $\boldsymbol{x}$ – one that concentrates on the latents *likely to have produced* $\boldsymbol{x}$. Introduce therefore a second network, the *encoder* or *inference network*,

$$q_{\boldsymbol{\phi}}(\boldsymbol{h}\mid\boldsymbol{x})=\mathcal{N}\Big(\boldsymbol{h};\,\boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x}),\,\operatorname{diag}\boldsymbol{\sigma}^2_{\boldsymbol{\phi}}(\boldsymbol{x})\Big), \tag{15.3}$$

whose job is to approximate the true posterior $p_{\boldsymbol{\theta}}(\boldsymbol{h}\mid\boldsymbol{x})$. The diagonal covariance is the *mean-field* assumption: the latent coordinates are taken to be independent given $\boldsymbol{x}$. It is what makes everything below computable and it is a genuine restriction, to which we return.

15.2 The evidence lower bound

Theorem 15.1 (Evidence decomposition). *For any distribution $q_{\boldsymbol{\phi}}(\boldsymbol{h}\mid\boldsymbol{x})$ with the same support as the posterior,*

$$\boxed{\log p_{\boldsymbol{\theta}}(\boldsymbol{x})=\underbrace{\mathbb{E}_{q_{\boldsymbol{\phi}}}\left[\log\frac{p_{\boldsymbol{\theta}}(\boldsymbol{x},\boldsymbol{h})}{q_{\boldsymbol{\phi}}(\boldsymbol{h}\mid\boldsymbol{x})}\right]}_{\mathrm{ELBO}(\boldsymbol{\theta},\boldsymbol{\phi})}+\underbrace{\mathrm{KL}\big(q_{\boldsymbol{\phi}}(\boldsymbol{h}\mid\boldsymbol{x})\,\|\,p_{\boldsymbol{\theta}}(\boldsymbol{h}\mid\boldsymbol{x})\big)}_{\geq 0}.} \tag{15.4}$$

Consequently $\mathrm{ELBO}\leq\log p_{\boldsymbol{\theta}}(\boldsymbol{x})$*, with equality if and only if $q_{\boldsymbol{\phi}}(\boldsymbol{h}\mid\boldsymbol{x})=p_{\boldsymbol{\theta}}(\boldsymbol{h}\mid\boldsymbol{x})$ almost everywhere.*

Proof. Multiply by one and split. Since $q_{\boldsymbol{\phi}}$ integrates to one,

$$\begin{aligned}\log p(\boldsymbol{x})&=\int q_{\boldsymbol{\phi}}(\boldsymbol{h}\mid\boldsymbol{x})\log p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{h}=\mathbb{E}_{q_{\boldsymbol{\phi}}}[\log p(\boldsymbol{x})]\\&=\mathbb{E}_{q_{\boldsymbol{\phi}}}\left[\log\frac{p(\boldsymbol{x},\boldsymbol{h})}{p(\boldsymbol{h}\mid\boldsymbol{x})}\right]=\mathbb{E}_{q_{\boldsymbol{\phi}}}\left[\log\frac{p(\boldsymbol{x},\boldsymbol{h})\,q_{\boldsymbol{\phi}}(\boldsymbol{h}\mid\boldsymbol{x})}{p(\boldsymbol{h}\mid\boldsymbol{x})\,q_{\boldsymbol{\phi}}(\boldsymbol{h}\mid\boldsymbol{x})}\right]\\&=\mathbb{E}_{q_{\boldsymbol{\phi}}}\left[\log\frac{p(\boldsymbol{x},\boldsymbol{h})}{q_{\boldsymbol{\phi}}(\boldsymbol{h}\mid\boldsymbol{x})}\right]+\mathbb{E}_{q_{\boldsymbol{\phi}}}\left[\log\frac{q_{\boldsymbol{\phi}}(\boldsymbol{h}\mid\boldsymbol{x})}{p(\boldsymbol{h}\mid\boldsymbol{x})}\right],\end{aligned}$$

where the second line used $p(\boldsymbol{x})=p(\boldsymbol{x},\boldsymbol{h})/p(\boldsymbol{h}\mid\boldsymbol{x})$. The last expectation is the definition of the KL divergence, which is non-negative by Eq. (14.7) and zero only when the two distributions agree.

Equation (15.4) is the whole architecture in one line, and it repays reading twice. The quantity we want, $\log p(\boldsymbol{x})$, is *fixed* – it does not depend on $\boldsymbol{\phi}$ at all. So raising the ELBO by

adjusting the encoder must lower the KL term by exactly as much: improving the bound and improving the posterior approximation are the same act. Meanwhile adjusting the decoder $\boldsymbol{\theta}$ raises the true evidence. One objective does both jobs.

A shorter derivation, and what it hides. Jensen's inequality gives the bound in two lines:

$$\log p(\boldsymbol{x}) = \log \mathbb{E}_{q_{\boldsymbol{\phi}}}\left[\frac{p(\boldsymbol{x},\boldsymbol{h})}{q_{\boldsymbol{\phi}}(\boldsymbol{h}\mid\boldsymbol{x})}\right] \;\geq\; \mathbb{E}_{q_{\boldsymbol{\phi}}}\left[\log \frac{p(\boldsymbol{x},\boldsymbol{h})}{q_{\boldsymbol{\phi}}(\boldsymbol{h}\mid\boldsymbol{x})}\right],$$

by concavity of the logarithm. This is quicker but tells us only that a gap exists. Theorem 15.1 identifies the gap as $\mathrm{KL}(q_{\boldsymbol{\phi}}\|p_{\boldsymbol{\theta}}(\boldsymbol{h}\mid\boldsymbol{x}))$, which is what tells us how to close it and why the encoder is worth training. The longer derivation is the useful one.

15.2.1 A tighter bound, and how to see the gap

Theorem 15.1 has an awkward consequence for anyone who wants to know how good a trained model is. The gap is $\mathrm{KL}(q_{\boldsymbol{\phi}}\|p_{\boldsymbol{\theta}}(\boldsymbol{h}\mid\boldsymbol{x}))$, and the true posterior is exactly the object we could not compute in the first place – so the gap is not directly measurable, and the ELBO could be one nat below $\log p(\boldsymbol{x})$ or twenty. There is a way out, and it is the same Jensen inequality applied more carefully.

Proposition 15.2 (The K-sample bound). *Let $\boldsymbol{h}_1,\dots,\boldsymbol{h}_K$ be independent draws from $q_{\boldsymbol{\phi}}(\boldsymbol{h}\mid\boldsymbol{x})$, write $w_k = p_{\boldsymbol{\theta}}(\boldsymbol{x},\boldsymbol{h}_k)/q_{\boldsymbol{\phi}}(\boldsymbol{h}_k\mid\boldsymbol{x})$, and define*

$$\mathscr{L}_K = \mathbb{E}\left[\log \frac{1}{K}\sum_{k=1}^{K} w_k\right]. \tag{15.5}$$

Then $\mathscr{L}_1 = \mathrm{ELBO}$, and

$$\mathrm{ELBO} = \mathscr{L}_1 \leq \mathscr{L}_2 \leq \cdots \leq \mathscr{L}_K \leq \log p_{\boldsymbol{\theta}}(\boldsymbol{x}), \qquad \mathscr{L}_K \to \log p_{\boldsymbol{\theta}}(\boldsymbol{x}) \text{ as } K\to\infty, \tag{15.6}$$

the last provided w is bounded.

Proof. $\mathscr{L}_1 = \mathbb{E}[\log w]$ is the ELBO by the definition of w. For the upper bound, $\mathbb{E}[\frac{1}{K}\sum_k w_k] = \mathbb{E}_q[w] = p_{\boldsymbol{\theta}}(\boldsymbol{x})$, so Jensen gives $\mathscr{L}_K \leq \log p_{\boldsymbol{\theta}}(\boldsymbol{x})$.

For monotonicity, let $\mathscr{I}$ be a uniformly random subset of $\{1,\dots,K\}$ of size $K-1$. Then

$$\frac{1}{K}\sum_{k=1}^{K} w_k = \mathbb{E}_{\mathscr{I}}\left[\frac{1}{K-1}\sum_{k\in\mathscr{I}} w_k\right],$$

because each w_k appears in a fraction $(K-1)/K$ of the subsets. Applying Jensen to the concave logarithm and then taking the expectation over the draws,

$$\mathscr{L}_K = \mathbb{E}\log \mathbb{E}_{\mathscr{I}}\left[\tfrac{1}{K-1}\textstyle\sum_{k\in\mathscr{I}} w_k\right] \;\geq\; \mathbb{E}\,\mathbb{E}_{\mathscr{I}}\log\left[\tfrac{1}{K-1}\textstyle\sum_{k\in\mathscr{I}} w_k\right] = \mathscr{L}_{K-1},$$

the last equality because every subset has the same distribution. Convergence follows from the strong law: $\frac{1}{K}\sum_k w_k \to p_{\boldsymbol{\theta}}(\boldsymbol{x})$ almost surely, and boundedness allows the limit to pass through the logarithm.

Equation (15.6) is useful twice over. Trained on, it is the importance-weighted autoencoder, which optimises a tighter bound and therefore tolerates a worse encoder. Evaluated on, which is what we shall do in Section 15.6.2, it turns the unmeasurable gap of Theorem 15.1 into

something one can bracket: since $\mathscr{L}_K \le \log p$, the difference $\mathscr{L}_K - \mathscr{L}_1$ is a *lower bound* on the gap, and it stops growing once K is large enough that $\mathscr{L}_K$ has converged.

15.2.2 The two terms

Splitting the joint as $p(\boldsymbol{x},\boldsymbol{h}) = p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid \boldsymbol{h})p(\boldsymbol{h})$ turns the ELBO into a form one can implement:

$$\begin{aligned}\mathrm{ELBO} &= \mathbb{E}_{q_{\boldsymbol{\phi}}}\left[\log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid \boldsymbol{h})\, p(\boldsymbol{h})}{q_{\boldsymbol{\phi}}(\boldsymbol{h} \mid \boldsymbol{x})}\right] \\ &= \underbrace{\mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{h}|\boldsymbol{x})}[\log p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid \boldsymbol{h})]}_{\text{reconstruction}} - \underbrace{\mathrm{KL}\big(q_{\boldsymbol{\phi}}(\boldsymbol{h} \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{h})\big)}_{\text{prior matching}}. \end{aligned} \tag{15.7}$$

The two terms pull against each other and the tension is the whole design. The reconstruction term wants the code to carry as much information about $\boldsymbol{x}$ as possible, so that the decoder can rebuild it; left alone it would drive $\boldsymbol{\sigma}_{\boldsymbol{\phi}} \to 0$ and turn the encoder into the deterministic map of Chapter 12. The prior-matching term wants $q_{\boldsymbol{\phi}}(\boldsymbol{h} \mid \boldsymbol{x})$ to look like $\mathcal{N}(\mathbf{0},\boldsymbol{I})$ whatever $\boldsymbol{x}$ is; left alone it would drive the code to carry no information at all. The optimum keeps enough information to reconstruct and no more.

That second term is what makes a VAE a generative model and an ordinary autoencoder not. It forces the aggregate of the encoded codes towards the prior, so that a fresh draw $\boldsymbol{h} \sim \mathcal{N}(\mathbf{0},\boldsymbol{I})$ lands somewhere the decoder has seen and produces something plausible. Nothing in Chapter 12 did that, which is precisely why sampling an autoencoder's latent space produces noise.

15.2.3 The Gaussian case in closed form

With the encoder (15.3) and the standard normal prior, the prior-matching term needs no sampling at all.

Proposition 15.3 (Closed-form KL). *For $q = \mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}\,\boldsymbol{\sigma}^2)$ and $p = \mathcal{N}(\mathbf{0},\boldsymbol{I})$ in d_h dimensions,*

$$\mathrm{KL}(q \,\|\, p) = \frac{1}{2}\sum_{j=1}^{d_h}\left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1\right). \tag{15.8}$$

Proof. The general formula for two Gaussians is

$$\mathrm{KL}(\mathcal{N}(\boldsymbol{\mu}_0,\boldsymbol{\Sigma}_0) \,\|\, \mathcal{N}(\boldsymbol{\mu}_1,\boldsymbol{\Sigma}_1)) = \frac{1}{2}\left[\mathrm{Tr}(\boldsymbol{\Sigma}_1^{-1}\boldsymbol{\Sigma}_0) + (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^{\top}\boldsymbol{\Sigma}_1^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0) - k + \log\frac{\det \boldsymbol{\Sigma}_1}{\det \boldsymbol{\Sigma}_0}\right],$$

with k the dimension. Setting $\boldsymbol{\mu}_1 = \mathbf{0}$, $\boldsymbol{\Sigma}_1 = \boldsymbol{I}$ and $\boldsymbol{\Sigma}_0 = \mathrm{diag}\,\boldsymbol{\sigma}^2$ gives $\mathrm{Tr}(\boldsymbol{\Sigma}_0) = \sum_j \sigma_j^2$, $\boldsymbol{\mu}_0^{\top}\boldsymbol{\mu}_0 = \sum_j \mu_j^2$, $k = d_h$ and $\log\det \boldsymbol{\Sigma}_0^{-1} = -\sum_j \log \sigma_j^2$, which is Eq. (15.8).

Each latent coordinate contributes independently, which is the payoff of the mean-field assumption, and each contribution is non-negative and vanishes exactly when $\mu_j = 0$ and $\sigma_j = 1$ – that is, when the coordinate carries no information. Section 15.5 uses that observation to measure how many coordinates a trained model actually uses.

15.3 The reparameterisation trick

One obstacle remains. The reconstruction term of Eq. (15.7) is an expectation over $q_{\boldsymbol{\phi}}$, and we must differentiate it with respect to $\boldsymbol{\phi}$ – the parameters of the distribution being sampled from. A sampling operation has no derivative, so backpropagation stops there.

The trick is to move the randomness out of the path. Instead of drawing $\boldsymbol{h}$ from $q_{\boldsymbol{\phi}}$, draw a standard normal variable and transform it deterministically:

$$\boxed{\boldsymbol{h} = \boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x}) + \boldsymbol{\sigma}_{\boldsymbol{\phi}}(\boldsymbol{x}) \odot \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}).} \tag{15.9}$$

The two are the same distribution – an arbitrary diagonal Gaussian is a standard one shifted and stretched – but now $\boldsymbol{\phi}$ appears in a differentiable function and $\boldsymbol{\varepsilon}$ is an *input*, not an operation. The gradient passes straight through:

$$\nabla_{\boldsymbol{\phi}} \, \mathbb{E}_{q_{\boldsymbol{\phi}}}[f(\boldsymbol{h})] = \mathbb{E}_{\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})} \left[\nabla_{\boldsymbol{\phi}} \, f\Big(\boldsymbol{\mu}_{\boldsymbol{\phi}} + \boldsymbol{\sigma}_{\boldsymbol{\phi}} \odot \boldsymbol{\varepsilon} \Big) \right]. \tag{15.10}$$

There is an alternative that needs no such trick. The *score-function* or REINFORCE estimator uses the identity $\nabla q = q \nabla \log q$ to write

$$\nabla_{\boldsymbol{\phi}} \, \mathbb{E}_{q_{\boldsymbol{\phi}}}[f(\boldsymbol{h})] = \mathbb{E}_{q_{\boldsymbol{\phi}}} \left[f(\boldsymbol{h}) \, \nabla_{\boldsymbol{\phi}} \log q_{\boldsymbol{\phi}}(\boldsymbol{h}) \right], \tag{15.11}$$

which is also unbiased and works for discrete latents where Eq. (15.9) does not apply. Both are unbiased; the difference is variance, and it is large enough to decide the matter. On a one-dimensional problem with an exactly known gradient of 1.800000:

```
   n      reparam mean (sd)          score-function mean (sd)     var ratio
  10    1.80487 (0.62312)        1.89474 (1.81999)           8.5
 100    1.81927 (0.20443)        1.77354 (0.54408)           7.1
1000    1.79803 (0.06343)        1.79991 (0.16570)           6.8
10000   1.80102 (0.01987)        1.80119 (0.04843)           5.9
```

Both estimators are centred on the true value, confirming that neither is biased, and the score-function variance is six to eight times larger even in one dimension. The reason the gap grows with dimension can be made precise.

Proposition 15.4 (Why the score-function estimator degrades with dimension). *Let $q_{\boldsymbol{\phi}} = \mathcal{N}(\boldsymbol{\mu}, \operatorname{diag} \boldsymbol{\sigma}^2)$ in d_h dimensions and consider the two estimators of $\nabla_{\boldsymbol{\mu}} \mathbb{E}_q[f(\boldsymbol{h})]$ from a single draw. Then*

$$\nabla^{\mathrm{rep}}_{\boldsymbol{\mu}} = \nabla f(\boldsymbol{h}), \qquad \nabla^{\mathrm{sf}}_{\boldsymbol{\mu}} = f(\boldsymbol{h}) \, \frac{\boldsymbol{h} - \boldsymbol{\mu}}{\boldsymbol{\sigma}^2} = f(\boldsymbol{h}) \, \frac{\boldsymbol{\varepsilon}}{\boldsymbol{\sigma}}, \tag{15.12}$$

and, writing $\sigma_j = \sigma$ for all j,

$$\sum_j \operatorname{Var}\Big[\nabla^{\mathrm{sf}}_{\mu_j}\Big] = \frac{1}{\sigma^2} \Big(d_h \, \mathbb{E}\big[f^2\big] - \sigma^2 \, \|\mathbb{E} \nabla f\|^2 \Big), \qquad \sum_j \operatorname{Var}\Big[\nabla^{\mathrm{rep}}_{\mu_j}\Big] = \mathbb{E}\|\nabla f\|^2 - \|\mathbb{E} \nabla f\|^2 . \tag{15.13}$$

The score-function variance therefore carries an explicit factor d_h from the magnitude of f alone; the reparameterised variance depends on f only through the fluctuation of its gradient.

Proof. The first identity in Eq. (15.12) is the chain rule applied to $\boldsymbol{h} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$, whose derivative in $\boldsymbol{\mu}$ is the identity; the second is $\nabla_{\boldsymbol{\mu}} \log q = (\boldsymbol{h} - \boldsymbol{\mu})/\boldsymbol{\sigma}^2$ substituted into Eq. (15.11). For the variances, use $\operatorname{Var}[X] = \mathbb{E}[X^2] - (\mathbb{E}X)^2$ with both estimators having the same mean. For the score-function one, $\mathbb{E}[f^2 \varepsilon_j^2 / \sigma^2]$ summed over j gives $\mathbb{E}[f^2 \|\boldsymbol{\varepsilon}\|^2]/\sigma^2$, and if f varies slowly compared with $\|\boldsymbol{\varepsilon}\|^2$ – whose mean is d_h – this is $d_h \mathbb{E}[f^2]/\sigma^2$ to leading order. The reparameterised case is immediate.

The contrast in Eq. (15.13) is stark: the score-function estimator's variance is set by the *size* of f multiplied by the dimension, while the reparameterised one is set by the *variation* of ∇f and does not know how many dimensions there are. Since f here is a log-likelihood over 784 pixels and is therefore of order hundreds, the first factor is large before d_h is taken into account. Section 15.6.1 measures the ratio on the actual ELBO and finds it running from 130 at $d_h = 1$ to ten thousand at $d_h = 32$. In a VAE with hundreds of latent dimensions the difference is between training and not training.

Why this is the interesting idea. The ELBO of Theorem 15.1 is decades older than deep learning; variational inference is a classical subject. What Kingma and Welling contributed was Eq. (15.9): the observation that if the approximating family is chosen so that sampling factors through a fixed noise source, then the whole bound becomes an ordinary differentiable function and every tool from Chapter 4 applies unchanged. The architecture is old; the estimator is what made it work.

Putting the pieces together, the quantity actually optimised for one data point, with a single sample of $\boldsymbol{\varepsilon}$, is

$$\widehat{\mathrm{ELBO}} = \log p_{\boldsymbol{\theta}}\Big(\boldsymbol{x} \mid \boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x}) + \boldsymbol{\sigma}_{\boldsymbol{\phi}}(\boldsymbol{x}) \odot \boldsymbol{\varepsilon}\Big) - \frac{1}{2}\sum_{j}\left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1\right). \tag{15.14}$$

One sample suffices because stochastic gradient descent is already averaging over data points; a noisier estimate per point is cheaper than a better one and converges to the same place.

15.4 Implementation and verification

The encoder and decoder are the networks of Chapter 8. Everything new is in the objective, and it is short enough to read in full.

```
def kl_gaussian(mu, logvar):
    """KL(N(mu, sigma^2 I) || N(0, I)) in closed form, Eq. (15.klclosed)."""
    return 0.5 * np.sum(mu ** 2 + np.exp(logvar) - logvar - 1.0, axis=-1)


def elbo(P, X, eps):
    """ELBO with the reparameterisation trick, Eqs. (15.elbo) and (15.reparam).

    eps is supplied from outside so that the randomness is an input rather than
    a side effect: that is exactly what makes the estimator differentiable.
    """
    mu, logvar = encode(P, X)
    H = mu + np.exp(0.5 * logvar) * eps          # Eq. (15.reparam)
    rec = bernoulli_logpdf(decode(P, H), X)
    return np.mean(rec - kl_gaussian(mu, logvar))
```

The encoder emits $\log \sigma^2$ rather than σ, which keeps the variance positive without a constraint and makes Eq. (15.8) numerically well behaved.

The closed-form KL.

Proposition 15.3 is an identity, so it can be checked against a Monte Carlo estimate of $\mathbb{E}_q[\log q - \log p]$:

```
=== closed-form Gaussian KL vs Monte Carlo ===
  d_h    closed form    Monte Carlo (10^6)   |diff|
    1       0.004940          0.004982   4.21e-05
    3       0.488126          0.487928   1.98e-04
    8       2.272325          2.271888   4.38e-04
```

The discrepancies are consistent with the 10^{-6} sampling error of a million-sample average.

The bound.

Theorem 15.1 claims the ELBO never exceeds $\log p(\boldsymbol{x})$. With $d_h = 2$ the evidence integral (15.2) can be evaluated by quadrature on a grid, so the claim is testable directly:

```
=== the ELBO is a lower bound on log p(x) ===
  x    log p(x) (quadrature)   ELBO (10^5 samples)      gap
  0               -4.149346          -4.392573   0.243227
  1               -4.209537          -4.460587   0.251050
  2               -3.921990          -4.570676   0.648686
  3               -3.873145          -4.388270   0.515125
  4               -4.157628          -4.435864   0.278236
```

Every gap is positive, as it must be, and every gap is $\mathrm{KL}(q_{\boldsymbol{\phi}} \| p_{\boldsymbol{\theta}}(\boldsymbol{h} \mid \boldsymbol{x}))$ for that data point – a quantity we could not otherwise compute. The variation across points is informative: the encoder approximates the posterior well for $\boldsymbol{x}_0$ and poorly for $\boldsymbol{x}_2$, and the ELBO is correspondingly looser there. This is the mean-field assumption showing its limits, and it is why a reported ELBO is a *pessimistic* estimate of a model's likelihood: a model may be better than its bound suggests.

15.5 Two characteristic failures

VAEs fail in two recognisable ways. Both follow from the objective, which means both are predictable and neither is a bug.

Blurry samples.

The reconstruction term of Eq. (15.7) is $\mathbb{E}[\log p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid \boldsymbol{h})]$, and for a Gaussian decoder with fixed variance this is a squared error. If several plausible outputs are consistent with a given code – and after a lossy encoding they usually are – then the expected squared error is minimised by their *mean*, not by any one of them. A model uncertain between two sharp images will emit their average, which is a blurred image, and the objective rewards it for doing so. This is the same phenomenon that made the conditional mean the optimal prediction in Chapter 3, appearing here as a defect. It is a property of the likelihood, not of the architecture, and Chapter 16 addresses it by changing what is being predicted.

Posterior collapse.

The second failure is visible in Eq. (15.8). The KL term decomposes into a sum of per-coordinate contributions, each of which is minimised at zero by setting $\mu_j = 0$, $\sigma_j = 1$ – that is, by making coordinate j ignore the input entirely. If the decoder is powerful enough to recon-

struct adequately without coordinate j, the optimiser will happily switch it off, since doing so is free in the reconstruction term and pays in the KL term. The coordinate *collapses to the prior* and carries no information.

This is straightforward to measure: compute the average KL contributed by each latent coordinate and count how many exceed a small threshold. On binarised 8×8 digits, with everything else held fixed:

```
binarised 8x8 digits: 1200 train, 597 test, d=64

  d_h   test ELBO   active units (KL_j > 0.01)   mean KL per unit
    2    -20.6270                          2   1.5142
    5    -18.9499                          5   1.0310
   10    -18.8107                         10   0.6453
   20    -19.1318                         14   0.3279
```

Up to $d_h = 10$ every coordinate is used. At $d_h = 20$ only fourteen survive and six have collapsed – and, tellingly, the test ELBO is *worse* than at $d_h = 10$, not better. Extra latent capacity did not merely go unused; it made the model slightly worse, because the optimiser spent part of training discovering which coordinates to discard. Figure 15.1(b) shows the per-coordinate KL falling off a cliff past the fourteenth unit.

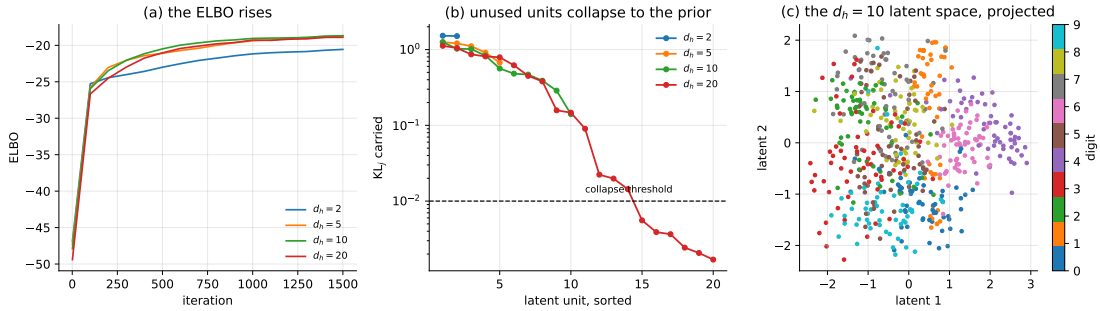


Fig. 15.1 (a) The ELBO during training for four latent dimensions; the curves for $d_h = 5, 10, 20$ are almost on top of one another, which is the first sign that the extra coordinates are not being used. (b) The KL contributed by each latent coordinate, Eq. (15.8), sorted and plotted logarithmically. At $d_h = 20$ the contribution falls below the collapse threshold from the fifteenth coordinate on. (c) The $d_h = 10$ latent means projected onto their two leading principal directions and coloured by digit class: the classes separate, although nothing in Eq. (15.7) asked them to.

Panel (c) is the encouraging counterpart. The model was never shown a label, and the classes nevertheless separate in the latent space. That is the payoff of the prior-matching term: by forcing the codes into a compact, well-covered region it makes the latent space continuous and comparable across inputs, which is exactly what the autoencoder of Chapter 12 failed to do. Recall Figure 12.5(c), where an autoencoder reconstructed a curve beautifully while its code jumped and saturated. The VAE buys a usable latent geometry, and Eq. (15.7) says exactly what it pays: the KL term is the price, measured in nats.

15.6 Implementations in the libraries

The only unusual element is that the loss depends on intermediate quantities – $\boldsymbol{\mu}$ and $\log \boldsymbol{\sigma}^2$ – and not only on the output, so the sampling step must be written explicitly rather than hidden inside a layer. That also makes the comparison against our own implementation unusually

clean: Eq. (15.9) turns the randomness into an *input,* so two implementations given the same weights *and the same* $\boldsymbol{\varepsilon}$ must produce the same number. They do:

```
=== 1. the ELBO of Eq. (15.objective), identical weights and noise ===
  ELBO, ours   : -15.499610160480
  ELBO, torch  : -15.499610160480
  ELBO, keras  : -15.499610160480
  max |mu_ours - mu_keras|                    : 2.220e-16
  max |KL_ours - KL_torch|, Eq. (15.klclosed) : 2.220e-16

   parameter   shape        |ours - torch|      scale
   enc W1    (20, 16)          6.939e-17   2.639e-01
   enc W2    (16, 8)           1.110e-16   3.528e-01
   dec W1    (4, 16)           1.665e-16   6.378e-01
   dec W2    (16, 20)          8.327e-17   3.969e-01
  worst disagreement: 1.665e-16
```

That the three agree to twelve decimal places on the *value* is unremarkable arithmetic. That they agree on the *gradient* is the point of Section 15.3: the estimator is an ordinary differentiable function of the parameters, so three different automatic differentiators applied to three separate implementations return the same thing. A sampling operation would not have had that property, and neither would Eq. (15.11).

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers


class Sampling(layers.Layer):
    """Eq. (15.reparam): h = mu + sigma * eps, with eps an input."""
    def call(self, inputs):
        mu, logvar = inputs
        eps = tf.random.normal(tf.shape(mu))
        return mu + tf.exp(0.5 * logvar) * eps


d, d_h = 784, 16
enc_in = keras.Input(shape=(d,))
e = layers.Dense(256, activation="relu")(enc_in)
mu = layers.Dense(d_h, name="mu")(e)
logvar = layers.Dense(d_h, name="logvar")(e)         # log sigma^2, unconstrained
h = Sampling()([mu, logvar])
encoder = keras.Model(enc_in, [mu, logvar, h], name="encoder")

dec_in = keras.Input(shape=(d_h,))
dd = layers.Dense(256, activation="relu")(dec_in)
logits = layers.Dense(d)(dd)                          # Bernoulli logits
decoder = keras.Model(dec_in, logits, name="decoder")


class VAE(keras.Model):
    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder, self.decoder = encoder, decoder

    def train_step(self, x):
        with tf.GradientTape() as tape:
            mu, logvar, h = self.encoder(x)
            logits = self.decoder(h)
            rec = -tf.reduce_sum(                     # log p(x|h), Bernoulli
                tf.nn.sigmoid_cross_entropy_with_logits(labels=x, logits=logits),
                axis=1)
            kl = 0.5 * tf.reduce_sum(                 # Eq. (15.klclosed)
```

```
                tf.square(mu) + tf.exp(logvar) - logvar - 1.0, axis=1)
            loss = -tf.reduce_mean(rec - kl)          # minimise the negative ELBO
        g = tape.gradient(loss, self.trainable_weights)
        self.optimizer.apply_gradients(zip(g, self.trainable_weights))
        return {"elbo": -loss}


vae = VAE(encoder, decoder)
vae.compile(optimizer=keras.optimizers.Adam(1e-3))
vae.fit(x_train, epochs=30, batch_size=128)
```

The same in PyTorch, where the reparameterisation is a method:

```
import torch
import torch.nn as nn
import torch.nn.functional as F


class VAE(nn.Module):
    def __init__(self, d=784, d_h=16, hidden=256):
        super().__init__()
        self.fc = nn.Linear(d, hidden)
        self.fc_mu = nn.Linear(hidden, d_h)
        self.fc_logvar = nn.Linear(hidden, d_h)
        self.dec = nn.Sequential(nn.Linear(d_h, hidden), nn.ReLU(),
                                 nn.Linear(hidden, d))

    def encode(self, x):                              # Eq. (15.encoder)
        e = F.relu(self.fc(x))
        return self.fc_mu(e), self.fc_logvar(e)

    def reparameterise(self, mu, logvar):             # Eq. (15.reparam)
        return mu + torch.exp(0.5 * logvar) * torch.randn_like(mu)

    def forward(self, x):
        mu, logvar = self.encode(x)
        h = self.reparameterise(mu, logvar)
        return self.dec(h), mu, logvar


def negative_elbo(logits, x, mu, logvar):
    """Eq. (15.objective), summed over pixels and averaged over the batch."""
    rec = F.binary_cross_entropy_with_logits(logits, x, reduction="none").sum(1)
    kl = 0.5 * (mu.pow(2) + logvar.exp() - logvar - 1.0).sum(1)
    return (rec + kl).mean()


model = VAE()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for epoch in range(30):
    for xb, _ in train_loader:
        xb = torch.bernoulli(xb.view(-1, 784))        # binarise
        logits, mu, logvar = model(xb)
        loss = negative_elbo(logits, xb, mu, logvar)
        opt.zero_grad(set_to_none=True)
        loss.backward()
        opt.step()

# generating new data: draw from the prior and decode, no Markov chain needed
with torch.no_grad():
    samples = torch.sigmoid(model.dec(torch.randn(64, 16)))
```

The last two lines are worth pausing on. Generation is a *single forward pass* from a prior sample. Compare Chapter 14, where drawing one sample required running a Markov chain

to equilibrium with no way of knowing when it had arrived. That is what the variational construction bought.

15.6.1 The variance that decides the matter

Section 15.3 compared the two estimators on a one-dimensional toy problem and Proposition 15.4 predicted that the gap grows with the latent dimension. Measuring it on the actual ELBO, drawing two thousand independent single-sample estimates of the encoder gradient at each d_h and summing the variance over all encoder parameters:

```
  d_h   reparam variance   score-function variance   ratio
    1        2.6865e+00               3.5057e+02    130.5
    2        1.3156e+00               1.0648e+03    809.3
    4        1.8912e+00               1.9455e+03   1028.7
    8        2.4841e+00               4.2074e+03   1693.8
   16        1.6908e+00               8.5782e+03   5073.5
   32        1.5594e+00               1.5826e+04  10148.3
```

Read the two middle columns separately, because they behave completely differently. The reparameterised variance is flat: between 1.3 and 2.7 across a factor of thirty-two in d_h, with no trend. The score-function variance rises roughly in proportion to d_h, from 3.5×10^2 to 1.6×10^4. That is exactly the split of Eq. (15.13): the first is governed by the fluctuation of ∇f, which does not care how many latent dimensions there are, and the second carries an explicit factor d_h multiplying $\mathbb{E}[f^2]$.

The ratio at $d_h = 32$ is ten thousand. To match the reparameterised estimator's noise the score-function estimator would need ten thousand times as many samples per step, and a real VAE uses d_h in the hundreds. Figure 15.2(b) plots both. Even in one dimension the ratio is 130 here against the six to eight of Section 15.3, and the difference is instructive: there f was a toy function of order one, here it is a log-likelihood over twenty pixels and therefore of order ten, and Eq. (15.13) makes the score-function variance scale with $\mathbb{E}[f^2]$.

15.6.2 How far is the ELBO from the truth?

Theorem 15.1 says the ELBO falls short of $\log p(\boldsymbol{x})$ by $\mathrm{KL}(q_{\boldsymbol{\phi}} \| p_{\boldsymbol{\theta}}(\boldsymbol{h} \mid \boldsymbol{x}))$, and every account of variational autoencoders says so. Almost none says how big it is, because the true posterior is unavailable – but Proposition 15.2 provides a way in. Training on the full MNIST set for six epochs and then evaluating $\mathscr{L}_K$ at increasing K:

```
=== 2. the ELBO gap, Theorem 15.elbo, measured ===
  d_h      L_1 (ELBO)        L_16        L_128       L_1024    L_1024 - L_1
    2     -155.6954    -153.9240    -153.5464    -153.3117          2.3837
   10     -112.5460    -109.8677    -109.2289    -108.8944          3.6516
   50     -105.7744    -100.9544     -99.6885     -99.0229          6.7515
```

and in TensorFlow, same architecture and same schedule:

```
  d_h      L_1 (ELBO)        L_16        L_128       L_1024    L_1024 - L_1
    2     -158.9083    -157.4499    -157.1135    -156.9733          1.9350
   10     -112.8858    -110.3861    -109.7177    -109.4030          3.4828
   50     -105.1233    -100.7442     -99.4499     -98.7592          6.3641
```

Three things follow, and the third changes how the earlier tables should be read.

The bound behaves as Proposition 15.2 says it must: monotone increasing in K, and flattening – from $K = 128$ to $K = 1024$ the gain is a fraction of a nat, so $\mathscr{L}_{1024}$ is close to $\log p$.

The gap is several nats and it *grows with the latent dimension*: 2.4 nats at $d_h = 2$, 3.7 at $d_h = 10$, 6.8 at $d_h = 50$, with TensorFlow agreeing to within half a nat throughout. A diagonal Gaussian is a worse approximation to the true posterior when there are more coordinates for it to be wrong about, which is Theorem 15.1 made quantitative.

And this reverses a conclusion. On the ELBO, $d_h = 50$ looked no better than $d_h = 20$ and marginally worse than $d_h = 10$ was on the smaller data set of Section 15.5 – the reading offered there was that extra latent capacity is wasted. On $\mathscr{L}_{1024}$, which is much closer to the quantity we actually care about, $d_h = 50$ is nearly ten nats *better* than $d_h = 10$: -99.0 against -108.9. The larger model was not worse; its *bound* was looser, because its posterior is harder to approximate. Comparing models by their ELBOs silently penalises whichever model has the more complicated posterior, and here that penalty was large enough to invert the ranking.

15.6.3 Collapse, and the price of preventing it

Section 15.5 counted active units against d_h. The other knob is the weight on the KL term – the β of the β-VAE – which controls the collapse directly. On the full MNIST set with $d_h = 50$:

```
=== 3. posterior collapse against the KL weight beta ===
   beta   test ELBO   active units of 50   mean KL per active
   0.25   -128.8367                   50                1.2817
   0.50   -114.4514                   50                0.8403
   1.00   -110.4756                   50                0.5177
   2.00   -116.6579                   47                0.3350
   4.00   -134.7284                   34                0.2264
```

The ELBO reported is the true one, at $\beta = 1$, so the table is a comparison of what each β delivers on the objective the model is supposed to optimise. It peaks at $\beta = 1$, which it must – that is the objective being measured – and falls off sharply on both sides. Raising β to 4 switches off sixteen of the fifty coordinates and costs twenty-four nats; lowering it to 0.25 keeps every coordinate alive and costs eighteen. The mean KL per active unit falls monotonically with β throughout, from 1.28 to 0.23 nats, which is the compression β is buying: fewer nats through the channel per coordinate. Whether that is worth having depends on what the code is for, and the ELBO cannot tell you – which is the honest summary of the β-VAE literature.

Figure 15.2(c) shows the per-coordinate KL at $\beta = 1$ sorted from largest to smallest. There is no cliff: the values slide smoothly from 2 nats down to 0.02, and the "number of active units" is an artefact of where the threshold is put. That is worth knowing before quoting one.

15.7 Towards diffusion models

The natural way to strengthen a VAE is to stop at one latent layer no longer. A *hierarchical* VAE introduces a chain $\boldsymbol{h}_1, \dots, \boldsymbol{h}_T$ of latent variables, and in the Markovian case each depends only on its predecessor:

$$q(\boldsymbol{h}_{1:T} \mid \boldsymbol{x}) = \prod_{t=1}^{T} q(\boldsymbol{h}_t \mid \boldsymbol{h}_{t-1}), \qquad p(\boldsymbol{x}, \boldsymbol{h}_{1:T}) = p(\boldsymbol{h}_T) \prod_{t=1}^{T} p_{\boldsymbol{\theta}}(\boldsymbol{h}_{t-1} \mid \boldsymbol{h}_t), \tag{15.15}$$

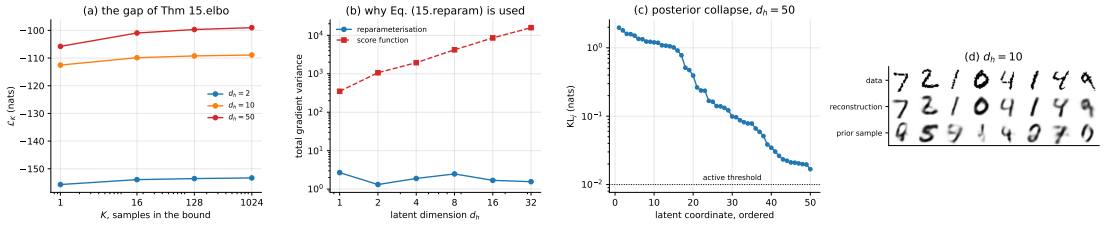


Fig. 15.2 (a) The K-sample bound of Proposition 15.2 against K, for three latent dimensions. $\mathscr{L}_1$ is the ELBO; $\mathscr{L}_{1024}$ is close to $\log p(\boldsymbol{x})$, and the vertical distance between them is the gap of Theorem 15.1. (b) The total variance of the two gradient estimators of Section 15.3 against the latent dimension: the reparameterised one is flat, the score-function one grows linearly, as Eq. (15.13) requires. (c) The average KL contributed by each latent coordinate at $d_h = 50$, sorted; the fall is smooth rather than a cliff. (d) Test digits, their reconstructions and samples decoded from the prior at $d_h = 10$.

with $\boldsymbol{h}_0 \equiv \boldsymbol{x}$. The ELBO of Theorem 15.1 goes through unchanged – the proof used nothing about the dimension or structure of $\boldsymbol{h}$ – and becomes a sum of T terms, one per level.

Now impose three restrictions on Eq. (15.15), each of which looks like a loss of generality and each of which turns out to buy something:

1. every latent has the *same dimension* as the data, so $\boldsymbol{h}_t \in \mathbb{R}^d$ and there is no bottleneck at all;
2. the encoder is not learned but *fixed*, a prescribed Gaussian that adds a little noise at each step;
3. the noise schedule is chosen so that $\boldsymbol{h}_T$ is pure $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$, independent of $\boldsymbol{x}$.

What remains is a model with no encoder to train, no bottleneck to collapse, and a prior that matches by construction rather than by paying a KL penalty. The three characteristic difficulties of this chapter have been designed away, and the entire modelling burden falls on the decoder, which must learn to undo one step of noising at a time. That model is a *diffusion model*, and it is the subject of Chapter 16.

15.8 Summary and the programs

A variational autoencoder is a latent-variable model whose intractable evidence is replaced by a bound that can be differentiated. Theorem 15.1 is the whole construction: $\log p(\boldsymbol{x}) = \mathrm{ELBO} + \mathrm{KL}(q_{\boldsymbol{\phi}} \| p(\boldsymbol{h} \mid \boldsymbol{x}))$, so raising the bound and improving the posterior approximation are one act, and the bound is tight exactly when the encoder recovers the true posterior. We checked the bound directly by quadrature at $d_h = 2$: every gap positive, and varying from 0.24 to 0.65 nats across data points, which is the mean-field assumption showing its limits.

The ELBO splits into a reconstruction term and a prior-matching term, Eq. (15.7), and the second is what an ordinary autoencoder lacks. It forces the codes towards the prior so that a fresh prior sample decodes to something plausible – generation in one forward pass, with no Markov chain.

The reparameterisation trick, Eq. (15.9), is what made any of it trainable. Both it and the score-function estimator are unbiased; measured on a problem with a known answer, the score-function variance was six to eight times larger in one dimension. Proposition 15.4 says why the gap must grow with the latent dimension – the score-function variance carries an explicit factor d_h multiplying $\mathbb{E}[f^2]$, the reparameterised one does not – and Section 15.6.1 measures it on the real ELBO: the reparameterised variance is flat across a factor of thirty-two in d_h while the ratio between the two runs from 130 to $10\,148$.

Proposition 15.2 makes the gap of Theorem 15.1 measurable, by squeezing $\log p$ between $\mathscr{L}_1 = \text{ELBO}$ and the K-sample bound. On MNIST the gap is 2.4 nats at $d_h = 2$ and 6.8 at $d_h = 50$: a diagonal Gaussian approximates a harder posterior less well. That measurement changes a conclusion. Ranked by ELBO, $d_h = 50$ looks no better than $d_h = 20$; ranked by $\mathscr{L}_{1024}$, it is ten nats better than $d_h = 10$. Comparing models by their bounds penalises whichever has the more complicated posterior, and here that was enough to invert the ordering. Both frameworks agree on the whole table to within half a nat.

Two failures are built into the objective rather than accidental. Blurriness follows from a squared-error reconstruction term, which is minimised by the mean of the plausible outputs. Posterior collapse follows from Eq. (15.8), in which each coordinate's KL is minimised by switching that coordinate off: at $d_h = 20$ only fourteen coordinates survived and the test ELBO was worse than at $d_h = 10$. On the full MNIST set the β sweep of Section 15.6.3 shows the same trade continuously – $\beta = 4$ switches off sixteen of fifty coordinates and costs twenty-four nats – and also shows that the per-coordinate KL falls off *smoothly* rather than at a cliff, so any count of "active units" is a statement about where the threshold was put. Against all that, the latent space is genuinely usable: the digit classes separated without ever being shown.

The programs are in the directory
`doc/BookML/BookPrograms/chapter15_vae`.
Every listing above appears there as a numbered file, and three modules run start to finish and reproduce the numbers quoted in the text:

- `vae.py` – the encoder and decoder, the closed-form KL, the reparameterised ELBO and Adam training.
- `verify_vae.py` – the KL check against Monte Carlo, the quadrature check that the ELBO bounds the evidence, and the gradient-variance comparison.
- `run_digits.py` – the latent-dimension study and the posterior-collapse measurement.
- `cross_check_vae.py` – the three-way comparison of Section 15.6 and the estimator-variance measurement of Section 15.6.1.
- `vae_torch.py` and `vae_tf.py` – the full-MNIST runs: the latent-dimension sweep, the K-sample bound of Proposition 15.2 and the β study of Section 15.6.3.
- `mnist_data.py` – a loader that reads MNIST from a local `.npz`, so that the programs run without network access.

The figure is generated by `ch15_figures.py` in `doc/BookML/BookFigures`; it is not drawn by hand.

15.9 Exercises

Warm-up exercises

1. **The bound.** (a) Prove Theorem 15.1 by the Jensen route and by the exact route, and say precisely what the second gives that the first does not. (b) Show that the bound is tight if and only if $q_{\boldsymbol{\phi}}(\boldsymbol{h} \mid \boldsymbol{x}) = p_{\boldsymbol{\theta}}(\boldsymbol{h} \mid \boldsymbol{x})$. (c) If q is restricted to diagonal Gaussians and the true posterior is not one, can the bound ever be tight?
2. **The Gaussian KL.** Derive Eq. (15.8) directly, without quoting the general formula, by integrating $\log q - \log p$ against q. Then show that each term is non-negative and identify the unique minimiser.
3. **Reparameterisation.** (a) Verify that Eqs. (15.10) and (15.11) are both unbiased. (b) Compute both variances analytically for $f(h) = h^2$ with $q = \mathscr{N}(\mu, 1)$ and confirm your answer

numerically. (c) Explain why Eq. (15.9) cannot be used for a discrete latent variable, and name one thing that can.

4. **Collapse by hand.** Consider a VAE whose decoder ignores one latent coordinate entirely. Show from Eq. (15.7) that the ELBO is strictly improved by setting that coordinate's $\mu_j = 0$ and $\sigma_j = 1$, and compute the improvement in nats.
5. **Blurriness.** Let $p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid \boldsymbol{h}) = \mathcal{N}(\boldsymbol{f}(\boldsymbol{h}), \sigma^2 \boldsymbol{I})$ and suppose a code $\boldsymbol{h}$ is consistent with two data points $\boldsymbol{x}_a$ and $\boldsymbol{x}_b$ occurring equally often. Show that the ELBO is maximised by $\boldsymbol{f}(\boldsymbol{h}) = (\boldsymbol{x}_a + \boldsymbol{x}_b)/2$, and comment on what this means for images.
6. **VAE against PCA.** Take the decoder linear and the observation noise Gaussian and isotropic. Show that the model becomes probabilistic PCA, Eq. (12.22), and hence by Section 12.7 that the optimum recovers the principal subspace. What does the KL term correspond to in that limit?
7. **The K-sample bound.** (a) Prove Proposition 15.2, being careful with the random-subset argument that gives monotonicity. (b) Show that $\mathscr{L}_K$ is the ordinary ELBO of an augmented model with K latent draws, and identify that model. (c) Explain why $\mathscr{L}_K - \mathscr{L}_1$ is only a *lower* bound on the gap of Theorem 15.1, and what would have to be true for it to be the gap exactly.
8. **Estimator variance.** (a) Derive Eq. (15.13) and state where the assumption that f varies slowly compared with $\|\boldsymbol{\varepsilon}\|^2$ is used. (b) Predict, from the proposition, what happens to the ratio when the decoder output dimension is doubled at fixed d_h. Then measure it. (c) Add a baseline b to Eq. (15.11), replacing f by $f - b$, and find the b that minimises the variance. How much of the gap does the optimal baseline close?
9. **Measuring the gap.** Reproduce Section 15.6.2. (a) At $d_h = 2$, compare $\mathscr{L}_{1024}$ against $\log p(\boldsymbol{x})$ computed by two-dimensional quadrature, and check that the bound is where you expect. (b) The gap grows with d_h. Does it grow with the *number of active* coordinates or with d_h itself? Design a run that separates the two. (c) Train with the K-sample bound as the objective rather than as a diagnostic and report what happens to the gap and to the number of active units.
10. **The β trade.** (a) Reproduce Section 15.6.3 and add $\beta = 8$ and $\beta = 16$. At what β does the code collapse entirely? (b) Show that at $\beta \neq 1$ the objective is still a valid bound on $\log p$ for some modified model, and identify it. (c) The chapter reports "active units" at a threshold of 0.01 nats. Recompute the counts at 0.001 and 0.1 and comment on how much of the posterior-collapse literature depends on that choice.

Project-style exercise: a variational autoencoder from scratch

Part a: the machinery.

Implement the encoder, decoder, closed-form KL and reparameterised ELBO. Verify Eq. (15.8) against Monte Carlo, and verify every gradient against finite differences.

Part b: the bound.

With $d_h = 1$ or 2, compute $\log p(\boldsymbol{x})$ by quadrature and confirm that the ELBO never exceeds it. Then improve the encoder – more capacity, more training, or a full-covariance Gaussian instead of the mean-field one – and show the gap shrinking. Report how much of the gap the mean-field restriction accounts for.

Part c: the estimators.

Reproduce the variance comparison of Section 15.3, then extend it: plot the variance ratio against latent dimension for $d_h = 1, \ldots, 50$. Add a baseline to the score-function estimator, as REINFORCE implementations do, and report how much of the gap that closes.

Part d: collapse.

Reproduce the latent-dimension study. Then attack the collapse three ways: KL annealing (ramp a weight β on the KL term from 0 to 1), a free-bits constraint (do not penalise KL below a floor per coordinate), and a weaker decoder. Report the active-unit count and the test ELBO for each, and say which intervention actually helps and which merely relabels the problem.

Part e: β-VAE.

Multiply the KL term of Eq. (15.7) by β and scan $\beta \in [0, 10]$. Plot reconstruction quality and latent-space structure against β. At $\beta = 0$ you should recover Chapter 12 exactly – verify that you do – and at large β the model should collapse entirely. Where is the useful regime, and how would you choose β without looking at the answer?

Part f: the latent geometry.

Interpolate linearly between the codes of two data points and decode along the path. Do the intermediate images look like data? Repeat with an autoencoder from Chapter 12 trained on the same data and compare. Then quantify it: measure the log-likelihood the model assigns to points along the interpolation.

Part g: towards diffusion.

Implement the two-level hierarchical VAE of Eq. (15.15) and derive its ELBO explicitly. Then impose restriction (2) of Section 15.7 – freeze the encoder to a fixed Gaussian that adds noise – and describe what the objective becomes. You will have derived the first step of Chapter 16 yourself.

Chapter 16
Diffusion Models and Normalizing Flows

Section 15.7 ended by describing three restrictions that turn a hierarchical variational autoencoder into something else entirely: latent variables of the same dimension as the data, a fixed rather than learned encoder, and a schedule ending in pure noise. This chapter carries that programme out. The result is the *diffusion model*, which produces the best samples of any generative model in this book and does so with an objective of startling simplicity: predict the noise that was added.

The chapter has a second half. Diffusion models buy sample quality by giving up an exact likelihood – like the VAE, they optimise a bound. *Normalizing flows* take the opposite trade: they insist on an exact, computable likelihood and pay for it with an architectural constraint, that every layer be invertible with a tractable Jacobian. Putting the two side by side is the clearest way to see what generative modelling actually costs, and the two turn out to be connected: in the continuous-time limit a diffusion model *is* a flow, one whose velocity field is the score.

The material follows the lecture notes for week fifteen of FYS-STK3155/4155 and the accompanying notes on diffusion models, normalizing flows and their comparison.

16.1 From hierarchical VAEs to diffusion

A Markovian hierarchical VAE, Eq. (15.15), has a chain of latents $\boldsymbol{x}_1, \dots, \boldsymbol{x}_T$ with $\boldsymbol{x}_0 \equiv \boldsymbol{x}$ the data. Impose the three restrictions:

1. *Same dimension.* Every $\boldsymbol{x}_t$ lives in $\mathbb{R}^d$. There is no bottleneck, so nothing can collapse in the sense of Section 15.5.
2. *Fixed encoder.* The forward process q is not learned. It adds a prescribed amount of Gaussian noise at each step. There is no $\boldsymbol{\phi}$ to optimise, so the encoder cannot be blamed for a loose bound.
3. *Terminal noise.* The schedule is chosen so that $q(\boldsymbol{x}_T \mid \boldsymbol{x}_0) \approx \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ for every $\boldsymbol{x}_0$. The prior then matches by construction and costs nothing.

All three difficulties of Chapter 15 have been designed away, and the entire modelling burden falls on the reverse process: a single network that must learn to undo one step of noising.

16.1.1 The forward process

Each step scales the current state down slightly and adds noise:

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}) = \mathcal{N}(\boldsymbol{x}_t;\, \sqrt{\alpha_t}\,\boldsymbol{x}_{t-1},\, (1-\alpha_t)\boldsymbol{I}), \qquad \alpha_t = 1-\beta_t, \tag{16.1}$$

with $\beta_t \in (0,1)$ the *noise schedule*. The scaling by $\sqrt{\alpha_t}$ is what keeps the variance bounded: without it the state would random-walk away to infinity.

The first useful fact is that the whole chain collapses.

Proposition 16.1 (Closed-form forward marginal). *With* $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$,

$$\boxed{q(\boldsymbol{x}_t \mid \boldsymbol{x}_0) = \mathcal{N}\big(\boldsymbol{x}_t;\, \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0,\, (1-\bar{\alpha}_t)\boldsymbol{I}\big), \quad \textit{i.e.} \quad \boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\varepsilon}.} \tag{16.2}$$

Proof. By induction. The case $t = 1$ is Eq. (16.1). Assume the result at $t-1$ and substitute:

$$\boldsymbol{x}_t = \sqrt{\alpha_t}\Big(\sqrt{\bar{\alpha}_{t-1}}\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_{t-1}}\,\boldsymbol{\varepsilon}'\Big) + \sqrt{1-\alpha_t}\,\boldsymbol{\varepsilon}'' = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \boldsymbol{\eta},$$

with $\boldsymbol{\varepsilon}', \boldsymbol{\varepsilon}''$ independent standard normals. Since independent Gaussians add in variance, $\boldsymbol{\eta}$ is Gaussian with variance

$$\alpha_t(1-\bar{\alpha}_{t-1}) + (1-\alpha_t) = \alpha_t - \bar{\alpha}_t + 1 - \alpha_t = 1 - \bar{\alpha}_t,$$

using $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$.

Equation (16.2) is arithmetic, not a network: any noise level can be reached in one step. That is what makes training cheap, since a mini-batch can sample t uniformly and jump straight there. We verify it against simulating the chain step by step:

```
=== 1. the closed-form marginal vs simulating the chain ===
   t    simulated mean/sd of x_t      closed form sqrt(abar), sqrt(1-abar)
   10      1.9946 /  0.0744            1.9945 /  0.0741
   50      1.8787 /  0.3446            1.8763 /  0.3463
  100      1.5585 /  0.6279            1.5524 /  0.6305
  200      0.7230 /  0.9323            0.7271 /  0.9316
```

The two coefficients have names. The *signal-to-noise ratio*

$$\mathrm{SNR}(t) = \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t} \tag{16.3}$$

falls from $\gg 1$ at $t = 0$ to ≈ 0 at $t = T$, and the whole objective can be rewritten in terms of it. Two schedules are standard: the *linear* one of Ho *et al.* [7], β_t from 10^{-4} to 0.02 over $T = 1000$, and the *cosine* one of Nichol and Dhariwal [55], $\bar{\alpha}_t \propto \cos^2\!\left(\frac{t/T+s}{1+s}\cdot\frac{\pi}{2}\right)$ with $s = 0.008$. Figure 16.1(c) shows both. The linear schedule destroys the signal abruptly at the end and wastes early steps on nearly identical images; the cosine schedule spreads the destruction out.

16.1.2 The forward posterior

To reverse the process we would like $q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)$, which is intractable – it requires knowing the data distribution. But conditioned additionally on $\boldsymbol{x}_0$ it is available in closed form, and that is enough.

Theorem 16.2 (Forward posterior).

$$q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) = \mathcal{N}\left(\boldsymbol{x}_{t-1};\, \tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0),\, \tilde{\beta}_t \boldsymbol{I}\right), \tag{16.4}$$

with

$$\tilde{\boldsymbol{\mu}}_t = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\boldsymbol{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\boldsymbol{x}_0, \qquad \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t. \tag{16.5}$$

Proof. By Bayes' rule and the Markov property,

$$q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) = \frac{q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1})\, q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_0)}{q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)},$$

all three factors being Gaussian by Eq. (16.1) and Proposition 16.1. The denominator does not involve $\boldsymbol{x}_{t-1}$, so as a function of $\boldsymbol{x}_{t-1}$ the log-numerator is

$$-\frac{\|\boldsymbol{x}_t - \sqrt{\alpha_t}\boldsymbol{x}_{t-1}\|^2}{2(1-\alpha_t)} - \frac{\|\boldsymbol{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}}\boldsymbol{x}_0\|^2}{2(1-\bar{\alpha}_{t-1})} + \text{const},$$

a quadratic in $\boldsymbol{x}_{t-1}$ and therefore Gaussian. Collecting the quadratic coefficient gives the precision

$$\frac{1}{\tilde{\beta}_t} = \frac{\alpha_t}{1-\alpha_t} + \frac{1}{1-\bar{\alpha}_{t-1}} = \frac{\alpha_t(1-\bar{\alpha}_{t-1}) + (1-\alpha_t)}{(1-\alpha_t)(1-\bar{\alpha}_{t-1})} = \frac{1-\bar{\alpha}_t}{(1-\alpha_t)(1-\bar{\alpha}_{t-1})},$$

using the same identity as in Proposition 16.1, and inverting gives $\tilde{\beta}_t$. Collecting the linear coefficient gives $\tilde{\boldsymbol{\mu}}_t/\tilde{\beta}_t = \sqrt{\alpha_t}\boldsymbol{x}_t/(1-\alpha_t) + \sqrt{\bar{\alpha}_{t-1}}\boldsymbol{x}_0/(1-\bar{\alpha}_{t-1})$, and multiplying through by $\tilde{\beta}_t$ gives Eq. (16.5).

This is a formula worth checking rather than trusting, and with one-dimensional data we can compute the posterior by quadrature and compare:

```
=== 2. the forward posterior vs Bayes by quadrature (1-D) ===
    t     quadrature mean / var        closed form mean / var
    5      1.128590 / 4.287e-04        1.128590 / 4.287e-04
   50      0.930699 / 4.916e-03        0.930699 / 4.916e-03
  150      0.908095 / 1.499e-02        0.908095 / 1.499e-02
  199      0.902574 / 1.994e-02        0.902574 / 1.994e-02
```

Agreement to six decimal places in both mean and variance, at every noise level.

16.2 The objective

The ELBO of Theorem 15.1 applies unchanged, and substituting the Markov structure decomposes it into T terms:

$$\log p(\boldsymbol{x}_0) \;\geq\; \underbrace{\mathbb{E}_q[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}_0 \mid \boldsymbol{x}_1)]}_{\mathcal{L}_0} - \sum_{t=2}^{T} \underbrace{\mathbb{E}_q[\mathrm{KL}(q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) \,\|\, p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t))]}_{\mathcal{L}_{t-1}} - \underbrace{\mathrm{KL}(q(\boldsymbol{x}_T \mid \boldsymbol{x}_0) \,\|\, p(\boldsymbol{x}_T))}_{\mathcal{L}_T}. \tag{16.6}$$

The last term contains no parameters – restriction (3) makes it approximately zero by construction – and each $\mathcal{L}_{t-1}$ is a KL between two Gaussians, the first of which Theorem 16.2 gives exactly. Choosing the learned reverse to be Gaussian with the same variance, $p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t) = \mathcal{N}(\boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t), \tilde{\beta}_t \boldsymbol{I})$, the KL collapses to a squared distance between the means,

$$\mathcal{L}_{t-1} = \frac{1}{2\tilde{\beta}_t}\,\mathbb{E}\left[\|\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\|^2\right]. \tag{16.7}$$

16.2.1 Predicting the noise

Now comes the step that makes diffusion models practical. Inverting Eq. (16.2) for $\boldsymbol{x}_0$ and substituting into Eq. (16.5) re-expresses the target mean in terms of the noise.

Proposition 16.3 (Noise parameterisation). *With* $\boldsymbol{x}_0 = (\boldsymbol{x}_t - \sqrt{1-\bar\alpha_t}\,\boldsymbol{\varepsilon})/\sqrt{\bar\alpha_t}$,

$$\tilde{\boldsymbol{\mu}}_t = \frac{1}{\sqrt{\alpha_t}}\left(\boldsymbol{x}_t - \frac{\beta_t}{\sqrt{1-\bar\alpha_t}}\,\boldsymbol{\varepsilon}\right). \tag{16.8}$$

Proof. Substituting and using $\sqrt{\bar\alpha_{t-1}/\bar\alpha_t} = 1/\sqrt{\alpha_t}$, the coefficient of $\boldsymbol{x}_t$ becomes

$$\frac{\sqrt{\alpha_t}(1-\bar\alpha_{t-1})}{1-\bar\alpha_t} + \frac{\beta_t}{\sqrt{\alpha_t}(1-\bar\alpha_t)} = \frac{\alpha_t(1-\bar\alpha_{t-1})+\beta_t}{\sqrt{\alpha_t}(1-\bar\alpha_t)} = \frac{1-\bar\alpha_t}{\sqrt{\alpha_t}(1-\bar\alpha_t)} = \frac{1}{\sqrt{\alpha_t}},$$

using $\alpha_t + \beta_t = 1$ and $\alpha_t\bar\alpha_{t-1} = \bar\alpha_t$; the coefficient of $\boldsymbol{\varepsilon}$ is $-\beta_t\sqrt{1-\bar\alpha_t}/[\sqrt{\alpha_t}(1-\bar\alpha_t)] = -\beta_t/[\sqrt{\alpha_t}\sqrt{1-\bar\alpha_t}]$.

So if a network $\boldsymbol{\varepsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)$ predicts the noise, setting $\boldsymbol{\mu}_{\boldsymbol{\theta}}$ to Eq. (16.8) with $\boldsymbol{\varepsilon}_{\boldsymbol{\theta}}$ in place of $\boldsymbol{\varepsilon}$ turns Eq. (16.7) into a weighted squared error in the noise. Ho *et al.* observed that *dropping* the weight improves samples, giving the objective actually used:

$$\boxed{\mathscr{L}_{\text{simple}} = \mathbb{E}_{t\sim\mathrm{U}[1,T],\,\boldsymbol{x}_0,\,\boldsymbol{\varepsilon}}\left\|\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_{\boldsymbol{\theta}}\left(\sqrt{\bar\alpha_t}\boldsymbol{x}_0 + \sqrt{1-\bar\alpha_t}\boldsymbol{\varepsilon},\, t\right)\right\|^2.} \tag{16.9}$$

It is worth pausing on how ordinary this is. After all the variational machinery, the training loop is: take a data point, pick a random noise level, add that much noise, and regress on the noise you added. It is a squared-error regression of exactly the kind Chapter 3 began with.

Three equivalent targets. Because $\boldsymbol{x}_t$, $\boldsymbol{x}_0$ and $\boldsymbol{\varepsilon}$ are related by the single linear equation (16.2), predicting any one determines the others:

$$\hat{\boldsymbol{x}}_0 = \frac{\boldsymbol{x}_t - \sqrt{1-\bar\alpha_t}\,\boldsymbol{\varepsilon}_{\boldsymbol{\theta}}}{\sqrt{\bar\alpha_t}}, \qquad \boldsymbol{v} = \sqrt{\bar\alpha_t}\,\boldsymbol{\varepsilon} - \sqrt{1-\bar\alpha_t}\,\boldsymbol{x}_0.$$

Predicting $\boldsymbol{\varepsilon}$ is the DDPM default; predicting $\boldsymbol{x}_0$ allows an explicit prior on images but tends to over-smooth; predicting the "velocity" $\boldsymbol{v}$ is numerically better near $t = 0$ where $\bar\alpha_t \to 1$ and the $\boldsymbol{\varepsilon}$ parameterisation divides by something small. All three give the same KL when substituted back; they differ only in conditioning.

16.2.2 The same thing as score matching

There is a second reading of Eq. (16.9) that connects it to a literature developed independently.

Proposition 16.4 (Noise prediction is score estimation). *For the forward marginal (16.2),*

$$\nabla_{\boldsymbol{x}_t}\log q(\boldsymbol{x}_t \mid \boldsymbol{x}_0) = -\frac{\boldsymbol{x}_t - \sqrt{\bar\alpha_t}\boldsymbol{x}_0}{1-\bar\alpha_t} = -\frac{\boldsymbol{\varepsilon}}{\sqrt{1-\bar\alpha_t}}. \tag{16.10}$$

Hence a network trained by Eq. (16.9) implicitly estimates the score of the noisy data distribution at every noise level, via $\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) = -\boldsymbol{\varepsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)/\sqrt{1-\bar\alpha_t}$.

Proof. The log-density of Eq. (16.2) is $-\|\boldsymbol{x}_t - \sqrt{\bar\alpha_t}\boldsymbol{x}_0\|^2/[2(1-\bar\alpha_t)]$ plus a constant; differentiating gives the first equality, and the second is Eq. (16.2) rearranged. The denoising score matching identity of Vincent, $\nabla \log q(\boldsymbol{x}_t) = \mathbb{E}_{q(\boldsymbol{x}_0\mid\boldsymbol{x}_t)}[\nabla \log q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)]$, then transfers the statement from the conditional to the marginal.

We check Eq. (16.10) by automatic differentiation:

```
=== 3. the score of the forward marginal is the negative noise ===
     t     -eps/sqrt(1-abar_t)      d/dx_t log q(x_t|x_0)   |diff|
    10        -7.76607851             -7.76607851   1.24e-14
    80        -1.18377136             -1.18377136   4.44e-16
   199        -0.67627979             -0.67627979   1.11e-16
```

Recall Eq. (12.18): the denoising autoencoder of Chapter 12 was *already* estimating a score, and we noted then that it anticipated diffusion models. This is where that remark is cashed in. A diffusion model is a denoising autoencoder trained at all noise levels at once, with a sampling procedure attached.

The continuous limit.

Taking $T \to \infty$ with $\beta_t \to \beta(t)\,\mathrm{d}t$, the forward chain becomes a stochastic differential equation,

$$\mathrm{d}\boldsymbol{x} = -\tfrac{1}{2}\beta(t)\,\boldsymbol{x}\,\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\boldsymbol{W}_t, \tag{16.11}$$

an Ornstein-Uhlenbeck process. Anderson's theorem gives the time reversal,

$$\mathrm{d}\boldsymbol{x} = \left[-\tfrac{1}{2}\beta(t)\boldsymbol{x} - \beta(t)\nabla_{\boldsymbol{x}} \log p_t(\boldsymbol{x})\right]\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\bar{\boldsymbol{W}}_t, \tag{16.12}$$

which requires exactly the score that Proposition 16.4 says the network has learned. The physics reading is immediate: Eq. (16.11) is the Langevin equation of Chapter 14 in a harmonic trap, and Eq. (16.12) is that process run backwards with the drift corrected by the force $-\nabla \log p_t$.

16.3 Sampling

Generation runs the reverse chain from noise. Ancestral, or DDPM, sampling takes $\boldsymbol{x}_T \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ and iterates

$$\boldsymbol{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\boldsymbol{x}_t - \frac{\beta_t}{\sqrt{1-\bar\alpha_t}}\,\boldsymbol{\varepsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\right) + \sqrt{\tilde\beta_t}\,\boldsymbol{z}, \qquad \boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}), \tag{16.13}$$

with $\boldsymbol{z} = \mathbf{0}$ at the last step. This costs T network evaluations, which at $T = 1000$ is the reason diffusion models are slow.

Denoising diffusion implicit models reduce that. The key observation is that the training objective (16.9) depends only on the marginals $q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)$, not on the joint, so any process with the same marginals is compatible with the same trained network. Choosing a non-Markovian one gives

$$\boldsymbol{x}_{t-1} = \sqrt{\bar\alpha_{t-1}}\,\hat{\boldsymbol{x}}_0 + \sqrt{1 - \bar\alpha_{t-1} - \sigma_t^2}\,\boldsymbol{\varepsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) + \sigma_t \boldsymbol{z}, \tag{16.14}$$

which at $\sigma_t = 0$ is entirely deterministic and may be run on any *subsequence* of steps. We measure what that is worth on a two-dimensional target, using the energy distance between samples and data:

```
sampler          network calls   energy distance to the data
DDPM, T=200                200   0.00715
DDIM,  50 steps             50   0.01153
DDIM,  20 steps             20   0.01096
DDIM,  10 steps             10   0.01504
```

Twenty steps instead of two hundred – a tenfold saving – costs a factor of about 1.5 in this statistic, and even ten steps remain usable. Figure 16.1(a) shows that the DDIM samples at twenty steps are visually indistinguishable from the DDPM samples at two hundred.

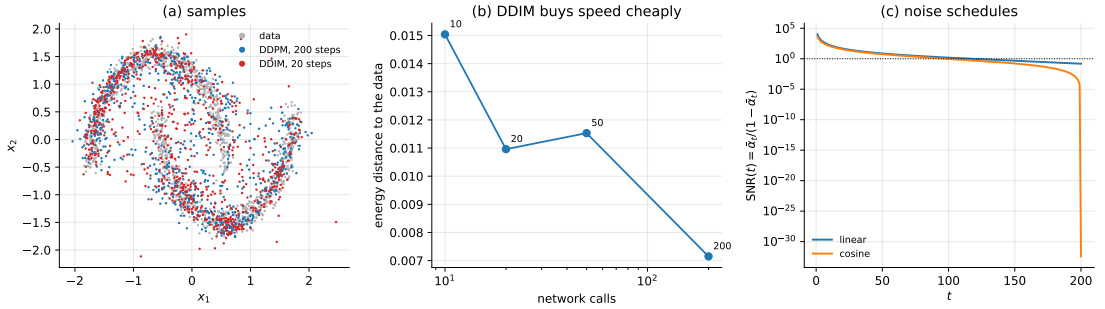


Fig. 16.1 (a) Two-dimensional data with samples from the trained model: DDPM with $T = 200$ ancestral steps and DDIM with 20. (b) Energy distance against the number of network evaluations; the tenfold reduction from 200 to 20 steps costs about 50% in this statistic. (c) The signal-to-noise ratio (16.3) for the linear and cosine schedules. The linear schedule destroys the remaining signal abruptly at the end, which is what the cosine schedule was introduced to avoid.

The implementation is short, because the forward process needs no network:

```
def q_sample(x0, t, abar, eps):
    """x_t = sqrt(abar_t) x_0 + sqrt(1-abar_t) eps,  Eq. (16.marginal)."""
    a = abar[t][:, None]
    return np.sqrt(a) * x0 + np.sqrt(1.0 - a) * eps


def loss(P, x0, t, eps, abar, T):
    """L_simple, Eq. (16.simple): predict the noise from the noisy sample."""
    xt = q_sample(x0, t, abar, eps)
    return np.mean(np.sum((eps - eps_net(P, xt, t, T)) ** 2, axis=1))
```

16.4 Normalizing flows

Diffusion models, like VAEs, optimise a bound. Normalizing flows refuse that compromise: they construct a model whose likelihood is *exact*. The price is paid in architecture.

The idea is one theorem from multivariable calculus. Let $\boldsymbol{z} \sim p_0$ be a simple base distribution – a standard normal – and let $\boldsymbol{x} = f(\boldsymbol{z})$ with f invertible and differentiable. Then the density transforms by the Jacobian:

Theorem 16.5 (Change of variables). *If $f : \mathbb{R}^d \to \mathbb{R}^d$ is a diffeomorphism and $\boldsymbol{x} = f(\boldsymbol{z})$, then*

$$\log p(\boldsymbol{x}) = \log p_0\big(f^{-1}(\boldsymbol{x})\big) + \log \left| \det \frac{\partial f^{-1}}{\partial \boldsymbol{x}} \right| = \log p_0(\boldsymbol{z}) - \log \left| \det \frac{\partial f}{\partial \boldsymbol{z}} \right| . \tag{16.15}$$

Composing K such maps, $f = f_K \circ \cdots \circ f_1$, the log-determinants add:

$$\log p(\boldsymbol{x}) = \log p_0(\boldsymbol{z}) - \sum_{k=1}^{K} \log \left| \det \frac{\partial f_k}{\partial \boldsymbol{h}_{k-1}} \right|. \tag{16.16}$$

Each layer transports probability mass; the total transformation is the composition, and the accounting is exact at every step.

The difficulty is visible in Eq. (16.16). A general $d \times d$ Jacobian determinant costs $\mathcal{O}(d^3)$, which is hopeless for images. Every flow architecture is a way of making that determinant cheap.

Coupling layers.

The construction of RealNVP splits the coordinates in two, passes one half through unchanged, and uses it to compute an affine transformation of the other:

$$\boldsymbol{x}_1 = \boldsymbol{z}_1, \qquad \boldsymbol{x}_2 = \boldsymbol{z}_2 \odot e^{\boldsymbol{s}(\boldsymbol{z}_1)} + \boldsymbol{t}(\boldsymbol{z}_1). \tag{16.17}$$

The Jacobian is block triangular, so its determinant is the product of the diagonal, and

$$\log |\det \boldsymbol{J}| = \sum_j s_j(\boldsymbol{z}_1), \tag{16.18}$$

computable in $\mathcal{O}(d)$ regardless of how complicated the networks $\boldsymbol{s}$ and $\boldsymbol{t}$ are. Alternating which half is transformed lets every coordinate be affected. Autoregressive flows use the same triangular trick with $x_i = f_i(z_i; x_1, \ldots, x_{i-1})$, and *continuous* normalizing flows replace the discrete composition by an ordinary differential equation $\mathrm{d}\boldsymbol{x}/\mathrm{d}t = \boldsymbol{v}(\boldsymbol{x}, t)$, for which the instantaneous change of variables gives

$$\frac{\mathrm{d} \log p(\boldsymbol{x}(t))}{\mathrm{d}t} = -\operatorname{Tr}\left(\frac{\partial \boldsymbol{v}}{\partial \boldsymbol{x}} \right), \tag{16.19}$$

a trace rather than a determinant, and cheaper still.

16.4.1 Verifying exactness

The selling point of flows is that the likelihood is exact, so it is exactly what should be checked. Three tests, each of which a VAE or a diffusion model would fail by construction:

```
=== 1. the analytic log-det vs the numerical Jacobian determinant ===
  sample 0: analytic 0.0269231819   numerical 0.0269231819   diff 5.52e-16
  sample 1: analytic -0.0850673946  numerical -0.0850673946  diff 8.33e-17
  sample 2: analytic -0.0407256759  numerical -0.0407256759  diff 1.18e-16

=== 2. the flow is exactly invertible ===
  max|f^-1(f(z)) - z| = 2.22e-16

=== 3. the density integrates to one (2-D, quadrature) ===
  integral of p(x) dx = 1.00000000
```

The third is the one to dwell on. A flow is a *normalised* density: no partition function, no bound, no Monte Carlo. Chapter 14 could not compute Z; Chapter 15 could only bound

the evidence; a flow evaluates $\log p(\boldsymbol{x})$ exactly for any $\boldsymbol{x}$, and here that claim is confirmed by numerical integration to eight decimal places.

An exact likelihood is not automatically a good model. Our from-scratch RealNVP maximised the likelihood on a two-moons target and reached 5.46 nats per point – and produced useless samples. The diagnosis is in the decomposition (16.15): the latent codes $\boldsymbol{z} = f^{-1}(\boldsymbol{x})$ came out with per-coordinate standard deviations of 0.118 and 0.120 rather than one, contributing $\log p_0(\boldsymbol{z}) = -1.85$, while the Jacobian term contributed $+7.31$. The model was contracting the data into a small ball near the origin, where the Gaussian prior is nearly flat and costs almost nothing, and collecting an unbounded reward from the log-determinant. Sampling $\boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ then lands far outside the region the data occupy.

This is not a coding error – the three checks above pass to machine precision, and the two contributions sum to the reported likelihood. It is the classical observation that the likelihood of a continuous density is unbounded above: the same degeneracy that lets a Gaussian mixture achieve infinite likelihood by collapsing one component onto a single data point. Flows on data concentrated near a lower-dimensional manifold are especially prone to it. The standard remedies – adding noise to the data, batch normalisation between coupling layers, a soft penalty on $\|\boldsymbol{z}\|$, or simply a smaller learning rate with many more steps – are the subject of an exercise. We report the failure rather than tuning until it disappears, because knowing *which* term of Eq. (16.15) is being exploited is worth more than a working demonstration.

16.5 Comparison

Five generative models have now appeared and they differ in one respect above all: what they are willing to give up in order to make the likelihood computable.

Table 16.1 The generative models of Chapters 14 to 16. "Exact likelihood" means $\log p(\boldsymbol{x})$ can be evaluated for a given $\boldsymbol{x}$ without approximation.

Model	Latent space	Likelihood	Sampling cost
RBM (Ch. 14)	fixed d_h, discrete	unnormalised, Z unknown	Markov chain
VAE (Ch. 15)	fixed $d_h < d$	lower bound (ELBO)	one pass
Normalizing flow	same dimension d	**exact**	one pass
Diffusion	same dimension d	lower bound (ELBO)	T passes

The trade is visible along the rows. The RBM has the most flexible energy function and the worst inference: no normalised likelihood and a Markov chain for every sample. The VAE gives up exactness for a bound and buys single-pass sampling and a compressed, interpretable latent space. The flow gives up architectural freedom – invertibility, equal dimension, cheap Jacobians – and buys an exact likelihood. Diffusion gives up sampling speed and buys the best sample quality of the four, because it never has to squeeze the data through a bottleneck and its objective is a sequence of easy regressions rather than one hard one.

The connection.

Flows and diffusion are not as different as the table suggests. A continuous normalizing flow, Eq. (16.19), transports p_0 to p_{data} along a deterministic velocity field. The *probability flow*

ODE of a diffusion model,

$$\frac{\mathrm{d}\boldsymbol{x}}{\mathrm{d}t} = -\tfrac{1}{2}\beta(t)[\boldsymbol{x} + \nabla_{\boldsymbol{x}} \log p_t(\boldsymbol{x})], \tag{16.20}$$

has by construction the same time-marginals as the SDE (16.11) and is exactly such a field. So a trained diffusion model *is* a continuous normalizing flow whose velocity is built from the score, and one may therefore compute exact likelihoods from a diffusion model by integrating Eq. (16.19) along Eq. (16.20). The difference is how the field is obtained: a flow learns it by maximum likelihood through a constrained architecture, a diffusion model learns it by score matching with no architectural constraint at all.

The physics reading. Equation (16.11) is an Ornstein-Uhlenbeck process, the Langevin dynamics of a particle in a harmonic trap; the forward process is relaxation to equilibrium and the reverse process is that relaxation run backwards, which is possible only because we have learned $\nabla \log p_t$, the thermodynamic force. The forward process increases entropy monotonically until the distribution is the maximum-entropy Gaussian; generation is the controlled reversal of that increase, and the network stores exactly the information needed to pay for it. Optimal transport gives a third reading: Eq. (16.20) moves mass along a particular path between p_0 and p_{data}, and asking for the *cheapest* such path is the Benamou-Brenier formulation of the Wasserstein distance, which is what flow-matching methods optimise directly.

Latent diffusion.

The two constructions combine. Run a VAE first to compress images into a lower-dimensional latent space, then run diffusion *in that space* rather than in pixel space. The autoencoder handles the high-frequency detail that makes pixel diffusion expensive; the diffusion model handles the structure the autoencoder would blur. This is how large-scale image generators are built, and it is a good illustration of a theme running through this book: the architectures are components, and the interesting systems are compositions.

16.6 Summary and the programs

A diffusion model is a hierarchical VAE with the encoder frozen, the bottleneck removed and the prior matched by construction. What remains is a single network trained by Eq. (16.9) – add noise, predict the noise – which after all the variational derivation is a squared-error regression.

Two closed forms make it work, and we verified both rather than quoting them. Proposition 16.1 collapses the chain so any noise level is one step away, confirmed against step-by-step simulation. Theorem 16.2 gives the reverse target exactly when conditioned on $\boldsymbol{x}_0$, confirmed against Bayes-by-quadrature to six decimals at every noise level. Proposition 16.4 identifies the trained network as a score estimator, confirmed to 10^{-14}, which connects diffusion to the denoising autoencoder of Chapter 12 and to the Langevin dynamics of Chapter 14.

Sampling is the expensive part, and DDIM makes it cheap: twenty steps rather than two hundred cost about 50% in energy distance and are visually indistinguishable.

Normalizing flows make the opposite trade. Theorem 16.5 gives an exact likelihood, and our implementation confirmed all three consequences to machine precision – analytic log-determinant, exact invertibility, and a density integrating to 1.00000000. But exactness is not sufficiency: our flow reached 5.46 nats while producing useless samples, by contract-

ing the data into a ball where the prior is flat ($\log p_0 = -1.85$) and harvesting the Jacobian term ($+7.31$). The likelihood of a continuous density is unbounded above, and a flow will find that out if allowed to.

The programs are in the directory
`doc/BookML/BookPrograms/chapter16_diffusion`.
Every listing above appears there as a numbered file, and four modules run start to finish and reproduce the numbers quoted in the text:

- `diffusion.py` – the schedules, the closed-form marginal (16.2), the forward posterior (16.4), the noise-prediction network, $\mathscr{L}_{\text{simple}}$, and DDPM and DDIM sampling.
- `flows.py` – coupling layers, the exact log-determinant, the forward and inverse maps and the exact log-likelihood.
- `verify_diffusion.py` – the three diffusion checks and the three flow checks of Sections 16.1.1, 16.1.2, 16.2.2 and 16.4.1.
- `run_compare.py` – the sampler comparison of Section 16.3 and the flow diagnosis of the notebox.

The figure is generated by `ch16_figures.py` in `doc/BookML/BookFigures`; it is not drawn by hand.

16.7 Exercises

Warm-up exercises

1. **The forward marginal.** (a) Prove Proposition 16.1 by induction, stating where independence is used. (b) Show that $\bar{\alpha}_t \to 0$ requires $\sum_t \beta_t \to \infty$, and compute $\bar{\alpha}_T$ for the linear schedule with $T = 1000$. (c) What goes wrong if the $\sqrt{\alpha_t}$ in Eq. (16.1) is omitted?
2. **The posterior.** Complete the square in Theorem 16.2 yourself. Then check the two limits: what are $\tilde{\boldsymbol{\mu}}_t$ and $\tilde{\beta}_t$ when $\beta_t \to 0$, and when $t = 1$?
3. **Noise parameterisation.** Verify Proposition 16.3 algebraically. Then derive the corresponding expression for the $\boldsymbol{x}_0$ and $\boldsymbol{v}$ parameterisations of the notebox, and show all three give the same $\tilde{\boldsymbol{\mu}}_t$.
4. **Schedules.** Plot $\bar{\alpha}_t$, $1 - \bar{\alpha}_t$ and $\mathrm{SNR}(t)$ for the linear and cosine schedules. At what t does each fall to $\mathrm{SNR} = 1$, and what does the cosine schedule fix?
5. **Change of variables.** (a) Prove Theorem 16.5 in one dimension from the definition of a density. (b) For the coupling layer (16.17), write the Jacobian as a block matrix and confirm Eq. (16.18). (c) Why must $\boldsymbol{s}$ and $\boldsymbol{t}$ depend on $\boldsymbol{z}_1$ only?
6. **Counting.** A diffusion model with $T = 1000$ and a flow with $K = 32$ coupling layers both generate one image. How many network evaluations does each need, and how many does each need to evaluate $\log p(\boldsymbol{x})$ for a given $\boldsymbol{x}$?

Project-style exercise: diffusion and flows

Part a: the forward process.

Implement both schedules and verify Proposition 16.1 against step-by-step simulation and Theorem 16.2 against Bayes-by-quadrature, as in Section 16.1.2. Verify Eq. (16.10) by automatic differentiation.

Part b: training.

Train a noise-prediction network on a two-dimensional target. Compare the weighted ELBO objective of Eq. (16.7) against $\mathscr{L}_{\text{simple}}$ and report which gives better samples. Then compare the $\boldsymbol{\varepsilon}$, $\boldsymbol{x}_0$ and $\boldsymbol{v}$ parameterisations.

Part c: sampling.

Reproduce the DDPM-versus-DDIM comparison, extending it to $\eta > 0$ so that DDIM interpolates towards DDPM. Plot sample quality against network evaluations for several η and identify where the frontier lies.

Part d: flows.

Implement coupling layers and verify all three exactness properties of Section 16.4.1. Then reproduce the failure described in the notebox, and fix it: try adding noise to the data, batch normalisation between layers, a penalty on $\|\boldsymbol{z}\|^2$, and a much smaller learning rate. Report which remedy works and, more importantly, explain *why* in terms of the two contributions to Eq. (16.15).

Part e: exact likelihoods from a diffusion model.

Implement the probability flow ODE (16.20) with your trained score network and integrate Eq. (16.19) to obtain exact log-likelihoods. Compare against the ELBO on the same data points. How large is the gap, and does it behave like the VAE gaps of Section 15.4?

Part f: a physics application.

Train a diffusion model on configurations of the two-dimensional Ising model at several temperatures, as in part e of the Chapter 14 project. Compare the energy and magnetisation histograms of the generated configurations against the true ones, and compare the whole thing against the RBM you trained there. Which model reproduces the critical region better, and at what computational cost?

Part g: the comparison.

Take one data set and fit all four models of Table 16.1 at matched parameter counts. Report, for each: training time, sampling time per example, sample quality by a two-sample statistic, and whether a likelihood is available and how it was obtained. Then write a paragraph recommending one of them for a specific scientific task of your choosing, and defend the choice against the other three.

Chapter 17
Generative Adversarial Networks

Every generative model of Chapters 14 to 16 was built around the likelihood. The restricted Boltzmann machine of Chapter 14 wrote down an unnormalised density and struggled with the partition function; the variational autoencoder of Chapter 15 replaced the intractable $\log p(\boldsymbol{x})$ by a bound; the diffusion model of Chapter 16 replaced it by a bound with T terms. In each case the object being optimised was, directly or indirectly, the probability the model assigns to the data.

This chapter is about giving that up. A *generative adversarial network* never evaluates $p_{\boldsymbol{\theta}}(\boldsymbol{x})$, never bounds it, and never approximates a partition function. It trains a generator by asking a second network – a *discriminator* – whether the generated samples look real, and improves the generator until the discriminator can no longer tell. The training signal is not a likelihood but the output of a classifier that is being retrained at the same time.

The construction is due to Goodfellow and coworkers [9] and it is best read as a two-player game. That reading is what gives the chapter its structure: we shall write down the value function of the game (Section 17.2), solve it for one player exactly (Section 17.3), show that the resulting objective for the other player is a Jensen-Shannon divergence (Section 17.4), and then spend the rest of the chapter on the consequences – both the good ones, which are the sharpest samples any single-pass generator produces, and the bad ones, which are that the game may fail to converge at all.

The material follows the lecture notes for the last week of FYS-STK4155 and the accompanying notes on generative adversarial networks; Chapter 20 of Goodfellow, Bengio and Courville [22] is the standard textbook treatment. Every identity stated below is verified numerically by the programs described in Section 17.11: no result in this chapter is quoted without having been recomputed.

17.1 Two networks and a game

A generative adversarial network consists of two networks with opposing objectives.

The *generator* G takes a noise vector $\boldsymbol{z} \in \mathbb{R}^k$ drawn from a fixed prior p_z – usually $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ or a uniform distribution on a cube – and maps it into data space,

$$\boldsymbol{x} = G(\boldsymbol{z}; \boldsymbol{\theta}^{(g)}), \qquad \boldsymbol{z} \sim p_z(\boldsymbol{z}), \qquad G : \mathbb{R}^k \to \mathbb{R}^d. \tag{17.1}$$

The distribution of the output is written p_g. It is the push-forward of p_z through G, and this is the first thing that distinguishes a GAN from everything that came before: we can *sample* from p_g by drawing $\boldsymbol{z}$ and applying one forward pass, but we cannot in general *evaluate* it. If G happened to be a diffeomorphism the change of variables of Chapter 16, Theorem 16.5,

would give

$$p_g(\boldsymbol{x}) = p_z\big(G^{-1}(\boldsymbol{x})\big)\left|\det \frac{\partial G^{-1}}{\partial \boldsymbol{x}}\right|, \tag{17.2}$$

but G is a generic network with $k \neq d$ and no invertibility constraint, so Eq. (17.2) does not apply. We call p_g an *implicit* distribution: defined by a sampling procedure and nothing else.

The *discriminator* D takes a point in data space and returns a probability,

$$D(\boldsymbol{x};\boldsymbol{\theta}^{(d)}) \in (0,1), \qquad D(\boldsymbol{x}) \;\approx\; \text{Prob}(\boldsymbol{x} \text{ came from the data}), \tag{17.3}$$

which we shall always implement as a sigmoid applied to a real-valued logit $u(\boldsymbol{x})$, so that $D = \sigma(u)$ and the losses below can be written with the softplus function and evaluated without overflow.

The two are trained against each other. The generator wants $D(G(\boldsymbol{z}))$ large; the discriminator wants $D(\boldsymbol{x})$ large on data and $D(G(\boldsymbol{z}))$ small on samples. Neither network ever sees the other's parameters: all the information the generator receives about the data arrives through the discriminator's gradient.

Supervised, unsupervised, or neither? A GAN is normally listed as an unsupervised method, and the end product – a model of p_r built from unlabelled samples – justifies that. But internally the discriminator solves a perfectly ordinary *supervised* binary classification problem, with labels that the construction supplies for free: real is 1, generated is 0. The unsupervised task has been converted into a supervised one, at the price that the training set for the classifier changes at every step. This is why GANs extend so naturally to the semi-supervised setting, where the discriminator is given $C+1$ classes – the C real classes plus "generated" – and the unlabelled data still contribute through the adversarial term.

17.1.1 What GANs are used for

The applications that made GANs prominent are almost all in image space: synthesis of photorealistic faces and scenes, super-resolution, inpainting, image-to-image translation between paired or unpaired domains, style transfer, text-to-image generation, video prediction and face ageing. In the physical sciences they have been used for fast surrogate simulation – generating calorimeter showers or lattice configurations far more cheaply than the underlying simulator – for data augmentation when the interesting events are rare, and for anomaly detection through the reconstruction error of an inverted generator.

What all of these share is that the target is a sharp, high-dimensional signal for which no tractable likelihood is available and for which a pixel-wise loss is known to be the wrong criterion. That is the regime where an adversarial loss earns its cost, and Section 17.6 is about what that cost is.

17.2 The minimax objective

The learning problem is posed as a zero-sum game. A single function $V(G,D)$ gives the reward of the discriminator; the generator receives $-V$. The default choice is

$$\boxed{\min_G \max_D \; V(G,D) = \mathbb{E}_{\boldsymbol{x}\sim p_r}\big[\log D(\boldsymbol{x})\big] + \mathbb{E}_{\boldsymbol{z}\sim p_z}\big[\log\big(1 - D(G(\boldsymbol{z}))\big)\big].} \tag{17.4}$$

Read the two terms separately. The first is large when D assigns high probability to real data. The second is large when D assigns low probability to generated data. Maximising their sum over $\boldsymbol{\theta}^{(d)}$ is exactly maximum likelihood for a binary classifier with balanced classes – the right-hand side of Eq. (17.4) is the negative binary cross-entropy of Chapter 5, Eq. (5.13), with the two classes "real" and "generated".

Since G appears only in the second term, minimising V over $\boldsymbol{\theta}^{(g)}$ means minimising $\mathbb{E}_{\boldsymbol{z}}[\log(1-D(G(\boldsymbol{z})))]$; the generator is pushing $D(G(\boldsymbol{z}))$ up. Changing variables from $\boldsymbol{z}$ to $\boldsymbol{x}=G(\boldsymbol{z})$ lets us write the same thing entirely in data space,

$$V(G,D)=\mathbb{E}_{\boldsymbol{x}\sim p_r}\big[\log D(\boldsymbol{x})\big]+\mathbb{E}_{\boldsymbol{x}\sim p_g}\big[\log(1-D(\boldsymbol{x}))\big]=\int\Big[p_r(\boldsymbol{x})\log D(\boldsymbol{x})+p_g(\boldsymbol{x})\log\big(1-D(\boldsymbol{x})\big)\Big]d\boldsymbol{x}, \tag{17.5}$$

which is the form we shall analyse. Note what has happened: the parameters $\boldsymbol{\theta}^{(g)}$ have disappeared into p_g, and the game is now a game between two *distributions*, p_g and D. Everything in the next two sections is a statement about that idealised game; Section 17.5 returns to the parameters.

In the numerical work we write the two terms with the softplus function $\zeta(u)=\log(1+e^u)$, using $\log\sigma(u)=-\zeta(-u)$ and $\log(1-\sigma(u))=-\zeta(u)$:

```
def softplus(u):
    """log(1+exp(u)), evaluated without overflow."""
    return np.maximum(u, 0.0) + np.log1p(np.exp(-np.abs(u)))


def value_function(Q, P, x_real, z):
    """V(G,D) = E[log D(x)] + E[log(1 - D(G(z)))],  Eq. (17.minimax)."""
    u_real = discriminator(Q, x_real)
    u_fake = discriminator(Q, generator(P, z))
    return -np.mean(softplus(-u_real)) - np.mean(softplus(u_fake))
```

17.3 The optimal discriminator

Hold the generator fixed, so that p_g is a fixed distribution, and ask for the discriminator that maximises Eq. (17.5). Because D enters the integral pointwise – the value of D at one $\boldsymbol{x}$ does not constrain its value at another, if we optimise over all measurable functions – the problem separates into one scalar maximisation per point.

Proposition 17.1 (Optimal discriminator). *For fixed p_g, the value function Eq. (17.5) is maximised by*

$$\boxed{D^*(\boldsymbol{x})=\frac{p_r(\boldsymbol{x})}{p_r(\boldsymbol{x})+p_g(\boldsymbol{x})}} \tag{17.6}$$

at every $\boldsymbol{x}$ with $p_r(\boldsymbol{x})+p_g(\boldsymbol{x})>0$.

Proof. Write $A=p_r(\boldsymbol{x})$, $B=p_g(\boldsymbol{x})$ and $t=D(\boldsymbol{x})\in(0,1)$. The integrand is $f(t)=A\log t+B\log(1-t)$, and

$$f'(t)=\frac{A}{t}-\frac{B}{1-t}=\frac{A-(A+B)t}{t(1-t)}, \tag{17.7}$$

which vanishes at $t^*=A/(A+B)$. The second derivative,

$$f''(t)=-\frac{A}{t^2}-\frac{B}{(1-t)^2}<0 \qquad \text{for all } t\in(0,1), \tag{17.8}$$

is negative wherever A and B are not both zero, so f is strictly concave and t^* is its unique maximum. Since the choice of t at one point does not affect any other point, maximising the integrand pointwise maximises the integral.

Equation (17.6) is worth reading twice. It says the optimal discriminator is the *Bayes-optimal classifier* for the two-class problem {real, generated} with equal class priors: the posterior probability that a point came from p_r rather than p_g. A perfectly trained discriminator is therefore not an adversary in any mysterious sense; it is a density ratio estimator. Rearranging,

$$\frac{p_r(\boldsymbol{x})}{p_g(\boldsymbol{x})} = \frac{D^*(\boldsymbol{x})}{1 - D^*(\boldsymbol{x})} = e^{u^*(\boldsymbol{x})}, \tag{17.9}$$

so the *logit* of the optimal discriminator is the log density ratio. The generator is being told, at every point, how much too little or too much mass it has put there – which is precisely the information a likelihood would have given, obtained without ever computing one.

We check Proposition 17.1 in two ways. First, pointwise: for several pairs (A, B) we maximise $f(t)$ numerically and compare against $A/(A+B)$,

```
=== 1. the pointwise maximiser of A log t + B log(1-t) ===
      A          B        numerical t*       A/(A+B)      |diff|
    0.9000     0.1000     0.9000000001  0.9000000000   1.1e-10
    0.5000     0.5000     0.5000000000  0.5000000000   1.1e-16
    0.2000     0.8000     0.1999999970  0.2000000000   3.0e-09
    0.0010     0.4000     0.0024937656  0.0024937656   3.5e-11
    0.3700     0.1100     0.7708333325  0.7708333333   8.7e-10
  worst deviation from Eq. (17.dstar): 2.97e-09

  f''(t*) = -2.7172 < 0, so t* is a maximum
```

Second, and more interestingly, with a network. We take two known one-dimensional densities, $p_r = \mathcal{N}(0,1)$ and $p_g = \mathcal{N}(1.5, 0.6^2)$, draw twenty thousand samples from each, and train a discriminator on the binary cross-entropy of Eq. (17.4) with the generator frozen. Nothing in the training knows the densities; the network sees only samples. Comparing the trained D against Eq. (17.6):

```
=== 2. a trained discriminator against a known pair of densities ===
     x        mixture     D_net(x)   p_r/(p_r+p_g)   |diff|
   -2.00     2.70e-02    0.99998        1.00000    0.0000
    0.00     2.14e-01    0.91331        0.93177    0.0185
    1.00     3.56e-01    0.38890        0.33993    0.0490
    2.00     2.62e-01    0.13343        0.10307    0.0304
    3.00     1.68e-02    0.11105        0.13172    0.0207
    4.00     1.23e-04    0.09246        0.54233    0.4499
    5.00     7.57e-07    0.07672        0.98207    0.9053
  where the mixture density exceeds 1% of its peak (x in [-2.77,3.33]):
      max |D_net - D*| = 0.0911, rms = 0.0240
  over the whole grid, including the empty tails:
      max |D_net - D*| = 0.9053, rms = 0.2647
  V from the trained network : -0.753844
  2 JS - 2 log 2 by quadrature: -0.751959
```

Two things should be noticed. Where there are data the network reproduces Eq. (17.6) to a root-mean-square error of 0.024, which is what one expects from twenty thousand samples and a network of this size. Where there are no data – at $x = 5$ the mixture density is 7.6×10^{-7} – it is wrong by 0.9, and confidently so. Figure 17.1 shows this. It is the first hint of a theme that runs through the whole chapter: statements about D^* are statements about the region where $p_r + p_g$ is appreciable, and the generator receives no reliable guidance anywhere else. The last two lines of the output anticipate the next section.

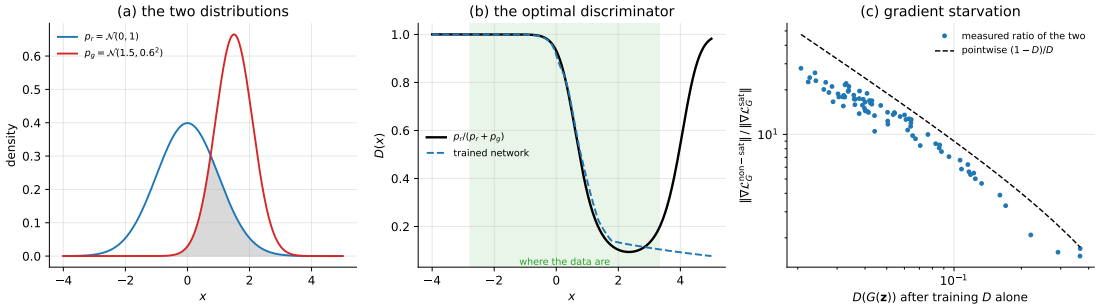


Fig. 17.1 (a) The two one-dimensional distributions used to test Proposition 17.1, with their overlap shaded. (b) The trained discriminator against the closed form $p_r/(p_r+p_g)$ of Eq. (17.6); the shaded band marks where the mixture density exceeds one per cent of its peak. Inside it the agreement is 0.024 rms; outside it the network is arbitrary, because no sample ever landed there. (c) The ratio of the two generator gradients of Section 17.5.1, measured while training the discriminator alone against a frozen generator, against the pointwise prediction $(1-D)/D$ of Eq. (17.19).

17.4 The objective is a Jensen-Shannon divergence

We now substitute Eq. (17.6) back into the value function and ask what the generator is really minimising. Two divergences are needed.

The *Kullback-Leibler divergence* of Chapter 14, Section 14.2.1, is

$$D_{\mathrm{KL}}(p\,\|\,q)=\int p(\boldsymbol{x})\log\frac{p(\boldsymbol{x})}{q(\boldsymbol{x})}\,d\boldsymbol{x}\;\geq\;0, \tag{17.10}$$

with equality if and only if $p=q$ almost everywhere. It is not symmetric and it is infinite whenever p puts mass where q puts none.

The *Jensen-Shannon divergence* repairs both defects by comparing each distribution to their mixture $m=\tfrac{1}{2}(p+q)$:

$$D_{\mathrm{JS}}(p\,\|\,q)=\tfrac{1}{2}D_{\mathrm{KL}}\left(p\,\middle\|\,\frac{p+q}{2}\right)+\tfrac{1}{2}D_{\mathrm{KL}}\left(q\,\middle\|\,\frac{p+q}{2}\right). \tag{17.11}$$

Lemma 17.2 (Range of the Jensen-Shannon divergence). $0\leq D_{\mathrm{JS}}(p\|q)\leq\log 2$ *for any pair of distributions, with* $D_{\mathrm{JS}}=0$ *if and only if* $p=q$ *almost everywhere and* $D_{\mathrm{JS}}=\log 2$ *if and only if* p *and* q *have disjoint supports.*

Proof. Non-negativity and the equality case follow from those of D_{KL}, since $m=p$ almost everywhere forces $p=q$. For the upper bound write out Eq. (17.11),

$$D_{\mathrm{JS}}(p\|q)=\frac{1}{2}\int p\log\frac{2p}{p+q}+\frac{1}{2}\int q\log\frac{2q}{p+q}=\log 2+\frac{1}{2}\int p\log\frac{p}{p+q}+\frac{1}{2}\int q\log\frac{q}{p+q},$$

and note that $p/(p+q)\leq 1$ and $q/(p+q)\leq 1$ pointwise, so both integrals are ≤ 0 and $D_{\mathrm{JS}}\leq\log 2$. Equality requires $p\log\big(p/(p+q)\big)=0$ and $q\log\big(q/(p+q)\big)=0$ almost everywhere, that is, $q=0$ wherever $p>0$ and conversely: disjoint supports.

The bound $\log 2$ is the whole story of Section 17.7, so it is worth having proved rather than quoted.

Theorem 17.3 (The value at the optimum). *With* D^{*} *of Eq. (17.6),*

$$\boxed{V(G,D^{*})=2D_{\mathrm{JS}}(p_r\,\|\,p_g)-2\log 2.} \tag{17.12}$$

Consequently $V(G,D^*) \geq -2\log 2$, with equality if and only if $p_g = p_r$, and at that point $D^* \equiv \frac{1}{2}$.

Proof. Substituting Eq. (17.6) into Eq. (17.5),

$$\begin{aligned} V(G,D^*) &= \int p_r \log \frac{p_r}{p_r+p_g}\,d\boldsymbol{x} + \int p_g \log \frac{p_g}{p_r+p_g}\,d\boldsymbol{x} \\ &= \int p_r \log \frac{p_r}{\frac{1}{2}(p_r+p_g)}\,d\boldsymbol{x} - \log 2 + \int p_g \log \frac{p_g}{\frac{1}{2}(p_r+p_g)}\,d\boldsymbol{x} - \log 2 \\ &= 2\left[\tfrac{1}{2} D_{\mathrm{KL}}\left(p_r \middle\| \tfrac{p_r+p_g}{2}\right) + \tfrac{1}{2} D_{\mathrm{KL}}\left(p_g \middle\| \tfrac{p_r+p_g}{2}\right)\right] - 2\log 2, \end{aligned} \tag{17.13}$$

where in the second line we multiplied and divided each argument of the logarithm by 2, each such step contributing $-\log 2$ because p_r and p_g integrate to one. The bracket is Eq. (17.11). The bound then follows from Lemma 17.2, and $p_g = p_r$ gives $D^* = p_r/(2p_r) = \frac{1}{2}$ directly from Eq. (17.6).

So the game, played to the discriminator's optimum at every step, is

$$G^* = \arg\min_G \left[\max_D V(G,D)\right] = \arg\min_G D_{\mathrm{JS}}(p_r \,\|\, p_g), \tag{17.14}$$

a statement of remarkable economy: fitting a classifier and then fooling it is the same thing as minimising a symmetric divergence between the model and the data. The value $-2\log 2 \approx -1.386$ is the signature of success, and $D \equiv \frac{1}{2}$ is the signature the practitioner actually monitors, since V requires D^* and D is available directly.

Both statements are checked by quadrature on one-dimensional Gaussians, where the JS divergence can be computed independently:

```
=== 3. the value at the optimal discriminator is the JS divergence ===
   m_g   s_g      V(G,D*)       2 JS - 2log2        |diff|      JS
   0.0   1.0   -1.38629436     -1.38629436    2.24e-12   0.00000
   0.5   1.0   -1.32567176     -1.32567176    2.24e-12   0.03031
   1.5   0.6   -0.75195864     -0.75195864    2.24e-12   0.31717
   3.0   1.0   -0.33273975     -0.33273975    1.35e-12   0.52678
   6.0   0.8   -0.00249849     -0.00249869    1.99e-07   0.69190
  the equal case p_g = p_r gives V = -2log2 = -1.38629436 and D* = 1/2
  the JS divergence is bounded by log 2 = 0.69314718
```

The agreement is at the level of the quadrature, 10^{-12}. The last row is the case that Section 17.7 is about: two Gaussians six standard deviations apart have $D_{\mathrm{JS}} = 0.6919$, within 10^{-3} of the ceiling $\log 2 = 0.6931$, and $V(G,D^*)$ has correspondingly flattened to -0.0025. Moving them further apart changes almost nothing.

Which divergence, and why it matters. The VAE of Chapter 15 minimises $D_{\mathrm{KL}}(q\|p)$ in the latent space and maximises a likelihood in data space, which amounts to minimising the *forward* divergence $D_{\mathrm{KL}}(p_r\|p_g)$. That divergence is infinite wherever the model assigns zero density to real data, so it is *mode-covering*: the model is heavily punished for missing anything and will spread mass over regions with no data rather than risk a gap. Blur is the visible symptom.

The reverse divergence $D_{\mathrm{KL}}(p_g\|p_r)$ punishes putting mass where there are no data but not the converse, and is therefore *mode-seeking*: it is content to model one mode well and ignore the rest. The Jensen-Shannon divergence of Eq. (17.11) is a symmetrised average of the two and sits between them – which is why GAN samples are sharper than VAE samples and why GANs, unlike VAEs, can quietly drop modes. Sharpness and mode dropping are two readings of the same choice of divergence, not two independent facts.

17.5 Training: alternating gradients

The theory of the previous two sections assumed that D reaches its optimum for each G. In practice we can afford neither an inner optimisation to convergence nor an optimisation over all measurable functions, and we take one gradient step for each player in turn. One iteration is

1. Freeze $\boldsymbol{\theta}^{(g)}$ and take n_{critic} ascent steps on the discriminator,
$$\boldsymbol{\theta}^{(d)} \leftarrow \boldsymbol{\theta}^{(d)} + \gamma_d \nabla_{\boldsymbol{\theta}^{(d)}} V(G,D), \tag{17.15}$$
where V is estimated on a minibatch of real data and a minibatch of generated data. The generated batch is *detached* from the computational graph, so that no gradient reaches $\boldsymbol{\theta}^{(g)}$ here.
2. Freeze $\boldsymbol{\theta}^{(d)}$ and take one descent step on the generator,
$$\boldsymbol{\theta}^{(g)} \leftarrow \boldsymbol{\theta}^{(g)} - \gamma_g \nabla_{\boldsymbol{\theta}^{(g)}} \mathscr{L}_G. \tag{17.16}$$

The usual choice is $n_{\text{critic}} = 1$: the discriminator is kept *near* its optimum rather than at it, and the theory of Section 17.4 is an idealisation of what the algorithm does. In terms of minibatch estimates the two losses are

$$\mathscr{L}_D = -\frac{1}{N}\sum_{i=1}^{N} \log D(\boldsymbol{x}_i) - \frac{1}{M}\sum_{j=1}^{M} \log\bigl(1 - D(G(\boldsymbol{z}_j))\bigr), \tag{17.17}$$

which is $-V$ estimated on the batch, and the generator loss of the next subsection.

17.5.1 The non-saturating generator loss

Equation (17.4) says the generator should minimise $\mathbb{E}[\log(1-D(G(\boldsymbol{z})))]$. This is a bad idea, and the reason is worth deriving rather than asserting.

Write $u = u(G(\boldsymbol{z}))$ for the discriminator logit at a generated point, so that $D = \sigma(u)$. Then

$$\frac{\partial}{\partial u}\log\bigl(1-\sigma(u)\bigr) = -\sigma(u) = -D, \qquad \frac{\partial}{\partial u}\bigl(-\log\sigma(u)\bigr) = -(1-\sigma(u)) = -(1-D). \tag{17.18}$$

Both losses reach $\boldsymbol{\theta}^{(g)}$ through the same chain $\partial u/\partial \boldsymbol{\theta}^{(g)}$, so the two gradients differ only by the scalar prefactor, and their ratio is

$$\boxed{\frac{\bigl\|\nabla_{\boldsymbol{\theta}^{(g)}} \mathscr{L}_G^{\text{non-sat}}\bigr\|}{\bigl\|\nabla_{\boldsymbol{\theta}^{(g)}} \mathscr{L}_G^{\text{sat}}\bigr\|} = \frac{1-D(G(\boldsymbol{z}))}{D(G(\boldsymbol{z}))}} \tag{17.19}$$

pointwise. Early in training the discriminator wins easily and $D(G(\boldsymbol{z})) = \varepsilon \ll 1$. The saturating gradient is then proportional to ε and vanishes; the non-saturating one is proportional to $1-\varepsilon \approx 1$ and does not. The generator is starved of gradient exactly when it is worst and most needs one. The remedy, which is what everyone uses, is to replace minimising $\log(1-D)$ with maximising $\log D$:

$$\boxed{\mathscr{L}_G = -\mathbb{E}_{\boldsymbol{z}\sim p_z}\bigl[\log D(G(\boldsymbol{z}))\bigr].} \tag{17.20}$$

Both losses are minimised by the same G – both are decreasing functions of $D(G(\boldsymbol{z}))$ – so the fixed point of the game is unchanged. Only the gradient magnitudes differ, and Eq. (17.19)

says by how much. Note that Eq. (17.20) is no longer the negative of anything the discriminator maximises: the game is no longer zero-sum, only adversarial.

The prefactors of Eq. (17.18) are confirmed by automatic differentiation:

```
=== 5. the two generator losses at D(G(z)) = eps ===
      eps     |dL_sat/du|   |dL_nonsat/du|      ratio    (1-eps)/eps
   1.0e-01     1.000e-01        0.900000         9.0           9.0
   1.0e-02     1.000e-02        0.990000        99.0          99.0
   1.0e-03     1.000e-03        0.999000       999.0         999.0
   1.0e-04     1.000e-04        0.999900      9999.0        9999.0
```

Pointwise identities are one thing; what a batch gradient does is another. We therefore construct the situation the argument is about. Take an untrained generator, freeze it, and train the discriminator alone for two thousand steps, measuring both generator gradients along the way without ever applying them:

```
=== 2. the generator gradient under a strong discriminator ===
   D steps    D(G(z))    ||dL_sat/dtheta||   ||dL_nonsat/dtheta||   ratio  1/D(G(z))
         0   3.67e-01          1.533e+00           2.596e+00      1.7        2.7
        50   2.92e-01          1.737e+00           2.778e+00      1.6        3.4
       200   1.17e-01          2.174e+00           1.216e+01      5.6        8.6
       500   8.79e-02          3.111e+00           2.380e+01      7.7       11.4
      1000   3.39e-02          1.484e+00           3.132e+01     21.1       29.5
      2000   2.41e-02          1.489e+00           3.431e+01     23.0       41.5
```

The non-saturating gradient grows by a factor of thirteen as the discriminator improves, while the saturating one stays where it was. The measured ratio tracks $1/D(G(\boldsymbol{z}))$ but stays below it, and the reason is instructive: a batch gradient is a sum over samples, and the saturating gradient is held up by the minority of samples that the discriminator still rates highest. Equation (17.19) is a pointwise statement, and the batch average of a ratio is not the ratio of batch averages. Figure 17.1(c) plots the measured ratio against the prediction over the whole trajectory.

17.5.2 One-sided label smoothing

A second, cheaper defence against an overconfident discriminator [56] is to replace the target label 1 for real data by $s < 1$, leaving the target for generated data at 0. This changes the optimum in a way that is easy to compute.

Proposition 17.4 (Smoothed optimal discriminator). *If the real term of Eq. (17.17) is replaced by the cross-entropy against a soft target $s \in (0,1]$, so that the pointwise objective becomes $f_s(t) = A\big[s\log t + (1-s)\log(1-t)\big] + B\log(1-t)$, then*

$$D_s^*(\boldsymbol{x}) = s\,\frac{p_r(\boldsymbol{x})}{p_r(\boldsymbol{x}) + p_g(\boldsymbol{x})} = s\,D^*(\boldsymbol{x}). \tag{17.21}$$

Proof. With $A = p_r$, $B = p_g$,

$$f_s'(t) = \frac{As}{t} - \frac{A(1-s)+B}{1-t} = \frac{As - \big(As + A(1-s) + B\big)t}{t(1-t)} = \frac{As - (A+B)t}{t(1-t)},$$

which vanishes at $t = sA/(A+B)$. The second derivative is negative as in Eq. (17.8), so this is the maximum.

The optimal discriminator is thus the old one scaled by s, and in particular it can never exceed s. Since the generator's gradient is proportional to $1-D$ by Eq. (17.18), capping D

below 1 keeps that factor bounded away from zero. Note that the smoothing must be *one-sided*: smoothing the generated target to some $\varepsilon > 0$ instead of 0 would put a $+\varepsilon\, p_g$ term in the numerator of Eq. (17.21) and reward the generator for placing mass where p_r is small, which is exactly backwards. Numerically,

```
=== 4. one-sided label smoothing, D*_s = s p_r/(p_r+p_g) ===
     s        A        B     numerical t*    s A/(A+B)      |diff|
  1.00    0.600    0.400    0.600000000  0.600000000    3.9e-10
  0.90    0.600    0.400    0.539999995  0.540000000    5.3e-09
  0.90    0.250    0.750    0.224999999  0.225000000    1.2e-09
  0.70    0.250    0.750    0.174999999  0.175000000    9.7e-10
```

and repeating the strong-discriminator experiment with $s = 0.9$ gives, after the same two thousand steps, $D(\boldsymbol{x}) = 0.8629$ instead of 0.9710: the discriminator has been prevented from becoming certain, which is the entire purpose.

The three losses are three lines of code:

```
def d_loss(Q, P, x_real, z, smooth=1.0):
    """L_D = -V, with one-sided label smoothing s = `smooth`, Eq. (17.dloss)."""
    u_real = discriminator(Q, x_real)
    u_fake = discriminator(Q, generator(P, z))
    real = smooth * softplus(-u_real) + (1.0 - smooth) * softplus(u_real)
    return np.mean(real) + np.mean(softplus(u_fake))


def g_loss_nonsat(P, Q, z):
    """L_G = -E[log D(G(z))],  the non-saturating loss, Eq. (17.nonsat)."""
    return np.mean(softplus(-discriminator(Q, generator(P, z))))


def g_loss_sat(P, Q, z):
    """L_G = E[log(1 - D(G(z)))],  the original minimax loss, Eq. (17.minimax)."""
    return -np.mean(softplus(discriminator(Q, generator(P, z))))
```

17.5.3 What convergence looks like

Theorem 17.3 gives two observable signatures of a converged game: $D(\boldsymbol{x})$ and $D(G(\boldsymbol{z}))$ both at $\frac{1}{2}$, and V at $-2\log 2$. We train a small non-saturating GAN on a mixture of eight Gaussians arranged on a circle – two-dimensional so that everything can be plotted – and watch:

```
=== 1. the game at equilibrium: eight Gaussians, non-saturating ===
  trained in 25.1s, 8/8 modes covered, 86.2% of samples within 0.5 of a mode
  energy distance to the data: 0.05205

     iteration        V        D(x)     D(G(z))
             1    -1.4334   0.5391   0.5449
           900    -1.0864   0.5531   0.3654
          1900    -1.1741   0.5736   0.4186
          3900    -1.3179   0.5873   0.5083
  mean over the last 1000 iterations: V = -1.2752, D(x) = 0.5570, D(G(z)) = 0.4533
  the theoretical equilibrium is V = -2log2 = -1.3863, D = 1/2
```

The discriminator settles at 0.557 on real data and 0.453 on generated data, and the value function at -1.275 against the predicted -1.386; Figure 17.2 shows the trajectories. The game does not converge to the equilibrium so much as circle it, which is the honest picture: the residual gap of 0.11 in V is not numerical error but the game's failure to sit still. Section 17.6.1 explains why it cannot.

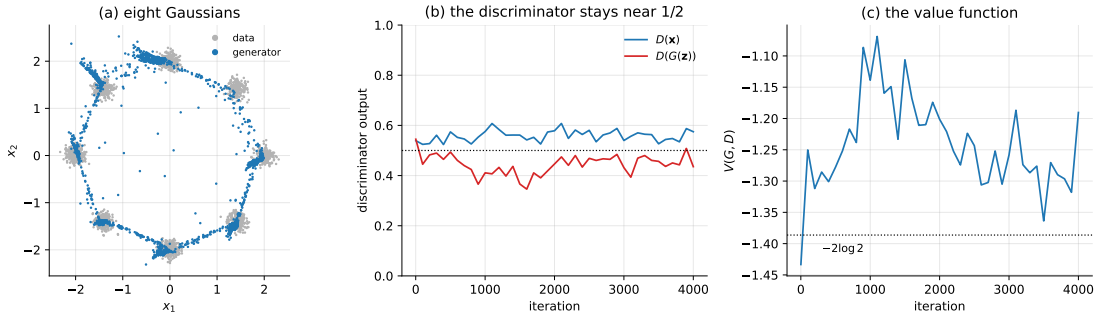


Fig. 17.2 A non-saturating GAN on a mixture of eight Gaussians, trained for 4000 alternating steps. (a) Data and generated samples; all eight modes are covered, with 86% of samples within 0.5 of a mode centre and the remainder strung along the connecting filaments discussed in Proposition 17.5. (b) $D(\boldsymbol{x})$ and $D(G(\boldsymbol{z}))$ against iteration; both hover around the equilibrium value $\frac{1}{2}$ of Theorem 17.3 without settling on it. (c) The value function $V(G,D)$ against the predicted $-2\log 2 = -1.386$.

17.6 What actually goes wrong

GAN training is notoriously unstable, and the instability is not a matter of insufficient tuning. Three distinct failure mechanisms operate, and it is worth separating them because they have different remedies.

17.6.1 Non-convergence: the game itself

The pair (G^*, D^*) of Theorem 17.3 is a Nash equilibrium: neither player can improve unilaterally. Gradient descent-ascent, however, is not an algorithm for finding Nash equilibria, and the difference is not subtle. The cleanest demonstration is the smallest possible zero-sum game,

$$V(x,y) = xy, \tag{17.22}$$

with x minimising and y maximising. The unique Nash equilibrium is the origin. Simultaneous gradient updates give

$$\begin{pmatrix} x_{t+1} \\ y_{t+1} \end{pmatrix} = \begin{pmatrix} 1 & -\gamma \\ \gamma & 1 \end{pmatrix} \begin{pmatrix} x_t \\ y_t \end{pmatrix}, \tag{17.23}$$

whose matrix has eigenvalues $1 \pm i\gamma$ of modulus $\sqrt{1+\gamma^2} > 1$. The iteration therefore *spirals outward*, and does so exactly:

$$x_t^2 + y_t^2 = (1+\gamma^2)^t \left(x_0^2 + y_0^2\right). \tag{17.24}$$

No step size helps; smaller γ only slows the divergence.

Now alternate, using the updated x in the y update as every practical implementation does:

$$\begin{pmatrix} x_{t+1} \\ y_{t+1} \end{pmatrix} = \begin{pmatrix} 1 & -\gamma \\ \gamma & 1-\gamma^2 \end{pmatrix} \begin{pmatrix} x_t \\ y_t \end{pmatrix}. \tag{17.25}$$

The determinant is $1-\gamma^2+\gamma^2 = 1$ and the trace is $2-\gamma^2$, so for $\gamma < 2$ the eigenvalues are complex conjugates on the unit circle: the orbit neither converges nor diverges but circulates forever at constant radius. Both statements are exact and both are checked:

```
=== 7. the bilinear game V(x,y) = xy, unique Nash at the origin ===
  gamma   steps    ||(x,y)|| simultaneous (1+gamma^2)^(T/2)    ||(x,y)|| alternating
```

```
  0.10    1000            2.0474e+02        2.0474e+02          1.399261
  0.05    1000            4.9284e+00        4.9284e+00          1.411927
  0.01    1000            1.4867e+00        1.4867e+00          1.412129
```

The starting point has $\|(x_0, y_0)\| = \sqrt{2} = 1.41421$, and the alternating iteration is still within 1.1% of it after a thousand steps, while the simultaneous one at $\gamma = 0.1$ has grown by a factor of 145. This single 2×2 example explains two facts about GAN practice at once: why the updates are alternated rather than simultaneous, and why even then the loss curves oscillate instead of descending. A GAN is not minimising anything; it is orbiting.

The standard stabilisers all attack this. Setting Adam's $\beta_1 = 0.5$ instead of 0.9 reduces the momentum that turns a circulating orbit into an expanding one. Two-time-scale updates [57] ($\gamma_d > \gamma_g$) keep the discriminator closer to its optimum, which is where the theory of Section 17.4 lives. Spectral normalisation bounds the Lipschitz constant of D [58], damping the feedback. None of them makes the game convex.

17.6.2 Vanishing gradients: the discriminator wins

The second mechanism was derived in Section 17.5.1. If D becomes perfect – $D(\boldsymbol{x}) = 1$ on data, $D(G(\boldsymbol{z})) = 0$ on samples – then V is at its ceiling and, with the saturating loss, the generator receives nothing. The non-saturating loss removes the symptom, but the underlying condition is real: a discriminator that separates the two distributions perfectly has, by Eq. (17.9), an infinite logit, and its gradient carries no information about *how far* the generator is from the data, only that it is on the wrong side. This is the failure the Wasserstein critic of Section 17.7 is designed to remove at the source.

17.6.3 Mode collapse, and a topological obstruction

The third mechanism is the one peculiar to adversarial training. In *mode collapse* the generator maps many different $\boldsymbol{z}$ to the same, or very nearly the same, output. The mechanism is easy to see from the alternating algorithm: for a *fixed* discriminator the generator's optimal response is to place all its mass at $\arg\max_{\boldsymbol{x}} D(\boldsymbol{x})$ – a single point – because Eq. (17.20) contains nothing that rewards diversity. The discriminator then learns to reject that point, the generator moves to the next best one, and the pair cycles. The theory of Section 17.4 does not apply, because it assumed $D = D^*$, and D^* would penalise the collapse immediately.

It is instructive to try to provoke it. We use the standard benchmark, a 5×5 grid of narrow Gaussians, and run three objectives:

```
=== 3. mode collapse: a 5x5 grid of narrow modes ===
  objective              time   modes  in-mode   energy dist  min/max per mode
  non-saturating           46s   15/25    52.4%      0.08026      0 /  354
  strong discriminator     25s   16/25    53.3%      0.12127      2 /  482
  WGAN-GP                 159s   17/25    46.1%      0.02638      3 /  349
  a uniform generator would put 200 of the 5000 samples on each mode
```

None of the three covers the target. The non-saturating GAN reaches fifteen of the twenty-five modes and puts 354 of five thousand samples on its favourite and none at all on its least favourite; strengthening the discriminator makes the imbalance worse, not better, and drives the energy distance up by half. WGAN-GP reaches seventeen modes and cuts the energy distance by a factor of three, which is a real improvement, but it too is far from uniform. Figure 17.4 shows the full distribution of samples over modes for the three runs. We report

the failure rather than tuning until it goes away, because the shape of the failure is more informative than a successful run would be.

Look at Figure 17.3. The generated samples do not sit on twenty-five islands: they lie along connected filaments that thread through the modes and, in the outer runs, trace the boundary of the square. This is not an accident of optimisation but a constraint that no amount of training can remove.

Proposition 17.5 (The support of p_g is connected). *Let $G : \mathbb{R}^k \to \mathbb{R}^d$ be continuous and let p_z have connected support S_z. Then $G(S_z)$ is connected, and* $\operatorname{supp}(p_g) = \overline{G(S_z)}$ *is connected.*

Proof. The continuous image of a connected set is connected, and the closure of a connected set is connected.

A Gaussian or uniform prior has connected support and a neural network is continuous, so p_g is supported on a connected set no matter what the parameters are. If p_r is supported on twenty-five disjoint islands, the two supports cannot coincide: the generator must either miss modes or connect them with filaments carrying mass the data do not have. It does both, and Figure 17.3 is a picture of the compromise. Real image manifolds are not literally disconnected, but they are close enough to it – a picture halfway between a 3 and an 8 is not a digit – that the same tension appears, and it is one reason samples from a GAN occasionally look like a blend of two classes.

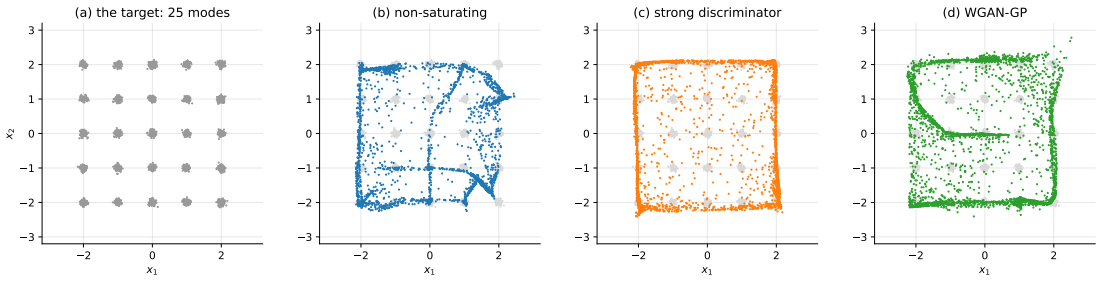


Fig. 17.3 Mode collapse on a 5×5 grid of narrow Gaussians. (a) The target. (b) The non-saturating GAN after 8000 steps: fifteen of the twenty-five modes are visited, and the samples lie along connected curves rather than on islands. (c) The same with a wider discriminator, a larger discriminator learning rate and three discriminator steps per generator step: the imbalance is worse and the energy distance rises from 0.080 to 0.121. (d) WGAN-GP, which reaches seventeen modes and an energy distance of 0.026. The filaments in all three panels are forced by Proposition 17.5.

The standard mitigations follow the diagnosis. *Minibatch discrimination* gives the discriminator access to statistics across a batch rather than single samples, so that a batch of near-identical outputs is detectable and punishable. *Unrolled GANs* [59] differentiate the generator loss through several future discriminator updates, so that G cannot exploit a short-sighted D. *Spectral normalisation* [58] and the gradient penalty of Section 17.7 bound the discriminator's Lipschitz constant. Raising k, the latent dimension, does not help: it cannot change Proposition 17.5, and in our experiments $k = 16$ was measurably worse than $k = 2$.

17.7 The Wasserstein distance

Lemma 17.2 contains a warning that Section 17.6.2 only hinted at. If p_r and p_g have disjoint supports, then $D_{\mathrm{JS}}(p_r \| p_g) = \log 2$ *exactly*, whatever the distance between the supports. By

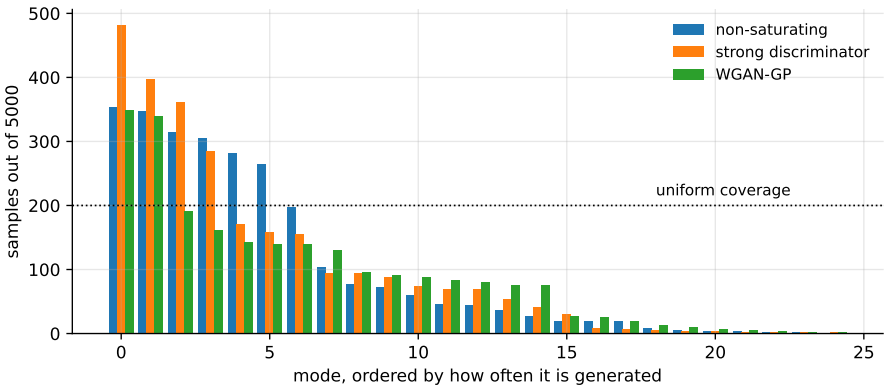


Fig. 17.4 How the five thousand generated samples of Figure 17.3 are distributed over the twenty-five modes, sorted from most to least visited for each objective. A generator matching the target would produce the flat line at 200. All three are far from it; WGAN-GP is flattest in the tail, which is what its lower energy distance is measuring.

Theorem 17.3 the objective is then flat: its gradient with respect to any parameter of G that moves p_g without creating overlap is zero.

This is not a pathological case. A generator maps a k-dimensional latent into $\mathbb{R}^d$ with $k \ll d$, so p_g is supported on a manifold of dimension at most k; the data lie on their own low-dimensional manifold; two such manifolds in general position intersect in a set of measure zero. Early in training the supports are effectively disjoint, and the ideal objective offers no guidance at all.

The canonical illustration is one-dimensional. Let p_r be uniform on $[0,1]$ and p_g uniform on $[\theta,\theta+1]$. The overlap has length $(1-\theta)_+$; on it both densities are 1 and the mixture is also 1, so that region contributes nothing to Eq. (17.11); off it exactly one density is 1 while the mixture is $\frac{1}{2}$, contributing $\log 2$ per unit length. Hence

$$D_{\mathrm{JS}}(p_r\|p_g) = \min(\theta,1)\log 2, \qquad W(p_r,p_g) = \theta. \tag{17.26}$$

For $\theta \geq 1$ the divergence is pinned at $\log 2$ with zero derivative, while the Wasserstein distance defined below keeps a derivative of 1 everywhere. Figure 17.5 plots both, together with the Gaussian case computed by quadrature:

```
=== 6b. JS and W for two unit Gaussians, by quadrature ===
       mu       JS(N(0,1)||N(mu,1))      W = |mu|
     0.50            0.03031130          0.5000
     1.00            0.11142148          1.0000
     2.00            0.33683082          2.0000
     4.00            0.63272019          4.0000
     8.00            0.68516989          8.0000
```

Gaussians have full support, so the divergence never quite reaches $\log 2$; but at $\mu = 8$ it is within 0.008 of the ceiling and for practical purposes the gradient is gone.

The *Wasserstein-1* or earth-mover distance measures how far mass must be moved rather than how much the densities disagree:

$$W(p_r,p_g) = \inf_{\pi\in\Pi(p_r,p_g)} \mathbb{E}_{(\boldsymbol{x},\boldsymbol{y})\sim\pi}\big[\|\boldsymbol{x}-\boldsymbol{y}\|\big], \tag{17.27}$$

where $\Pi(p_r,p_g)$ is the set of joint distributions – transport plans – with the prescribed marginals. The infimum over plans is intractable, but the Kantorovich-Rubinstein duality [60] converts it into a maximisation over functions,

$$W(p_r, p_g) = \sup_{\|f\|_L \le 1} \Big(\mathbb{E}_{\boldsymbol{x}\sim p_r}[f(\boldsymbol{x})] - \mathbb{E}_{\boldsymbol{x}\sim p_g}[f(\boldsymbol{x})] \Big), \tag{17.28}$$

the supremum running over all 1-Lipschitz f. This is a form we can parameterise: replace f by a network f_w, drop the sigmoid – the output is no longer a probability, which is why f_w is called a *critic* rather than a discriminator – and maximise Eq. (17.28) by gradient ascent.

The Lipschitz constraint is the whole difficulty. The original WGAN [61] clipped every weight to $[-c, c]$, which enforces *some* Lipschitz bound but also cripples the critic's capacity. The gradient penalty [62] replaces the hard constraint by a soft one, using the fact that a differentiable function is 1-Lipschitz if and only if $\|\nabla f\| \le 1$ everywhere, and that the optimal critic in Eq. (17.28) has $\|\nabla f\| = 1$ almost everywhere on the transport paths. Sampling those paths by interpolating between real and generated points gives

$$\mathscr{L}_{\text{critic}} = \mathbb{E}_{p_g}[f_w(\boldsymbol{x})] - \mathbb{E}_{p_r}[f_w(\boldsymbol{x})] + \lambda\, \mathbb{E}_{\hat{\boldsymbol{x}}}\Big[\big(\|\nabla_{\hat{\boldsymbol{x}}} f_w(\hat{\boldsymbol{x}})\|_2 - 1 \big)^2 \Big], \tag{17.29}$$

with $\hat{\boldsymbol{x}} = \varepsilon \boldsymbol{x} + (1-\varepsilon) G(\boldsymbol{z})$, $\varepsilon \sim U[0,1]$ and $\lambda = 10$ the standard choice. The generator maximises $\mathbb{E}[f_w(G(\boldsymbol{z}))]$. Because the critic is now expected to be trained close to its optimum, $n_{\text{critic}} = 5$ is used instead of 1.

Two practical differences follow. The critic loss is an estimate of W up to a constant, so unlike V it *decreases as the samples improve* and is usable as a progress indicator; and batch normalisation must be removed from the critic, since the penalty is a statement about the gradient with respect to a single input and batch normalisation makes the output depend on the whole batch. In our two-dimensional experiment WGAN-GP gave the lowest energy distance of the three objectives, 0.026 against 0.080, at four times the cost in wall-clock time. The implementation is short:

```
def gradient_penalty(Q, x_real, x_fake, rng):
    """E[(||grad f_w(xhat)||_2 - 1)^2] on the segments between real and fake."""
    eps = rng.uniform(0.0, 1.0, (len(x_real), 1))
    xhat = eps * x_real + (1.0 - eps) * x_fake
    g = grad(lambda x: np.sum(critic(Q, x)))(xhat)
    return np.mean((np.sqrt(np.sum(g ** 2, axis=1) + 1e-12) - 1.0) ** 2)


def critic_loss(Q, P, x_real, z, rng, lam=10.0):
    """L_critic = E_pg[f] - E_pr[f] + lambda * GP,  Eq. (17.wgangp)."""
    x_fake = generator(P, z)
    w = np.mean(critic(Q, x_fake)) - np.mean(critic(Q, x_real))
    return w + lam * gradient_penalty(Q, x_real, x_fake, rng)
```

17.8 Evaluating a model with no likelihood

Every generative model of the preceding chapters could be scored on held-out data by its likelihood or a bound on it. A GAN offers neither, and the question "is this model better than that one?" has to be answered from samples alone. Three families of answer are in use.

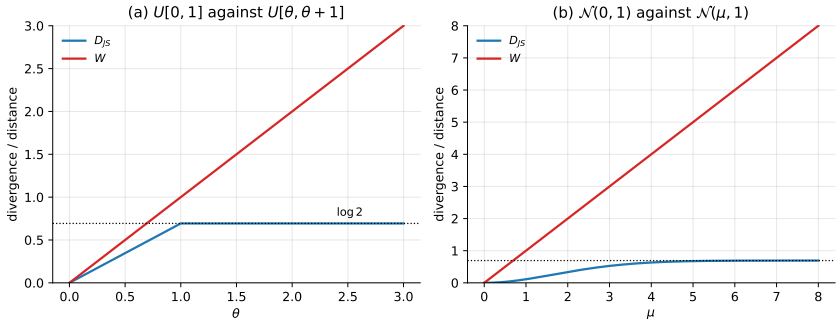


Fig. 17.5 Why the Jensen-Shannon divergence gives up. (a) $p_r = U[0,1]$ against $p_g = U[\theta, \theta+1]$: Eq. (17.26) makes D_{JS} constant at $\log 2$ for $\theta \geq 1$, so the objective has no gradient there, while $W = \theta$ has slope 1 everywhere. (b) The same comparison for $\mathcal{N}(0,1)$ against $\mathcal{N}(\mu,1)$, computed by quadrature: the supports overlap so D_{JS} approaches $\log 2$ asymptotically instead of reaching it, but by $\mu = 4$ it is already within 0.06 of the ceiling.

17.8.1 Maximum mean discrepancy

Given a kernel $k(\cdot,\cdot)$, the *maximum mean discrepancy* [63] between p and q is the distance between their mean embeddings in the corresponding reproducing kernel Hilbert space,

$$\mathrm{MMD}^2(p,q) = \mathbb{E}_{\boldsymbol{x},\boldsymbol{x}'\sim p}[k(\boldsymbol{x},\boldsymbol{x}')] - 2\,\mathbb{E}_{\boldsymbol{x}\sim p,\boldsymbol{y}\sim q}[k(\boldsymbol{x},\boldsymbol{y})] + \mathbb{E}_{\boldsymbol{y},\boldsymbol{y}'\sim q}[k(\boldsymbol{y},\boldsymbol{y}')]. \tag{17.30}$$

For a *characteristic* kernel – the Gaussian $k(\boldsymbol{x},\boldsymbol{y}) = \exp(-\|\boldsymbol{x}-\boldsymbol{y}\|^2/2\sigma^2)$ is one – the mean embedding determines the distribution, so $\mathrm{MMD}(p,q) = 0$ if and only if $p = q$. With n real and m generated samples the unbiased estimator omits the diagonal terms,

$$\widehat{\mathrm{MMD}}^2 = \frac{1}{n(n-1)}\sum_{i\neq j} k(\boldsymbol{x}_i,\boldsymbol{x}_j) - \frac{2}{nm}\sum_{i,j} k(\boldsymbol{x}_i,\boldsymbol{y}_j) + \frac{1}{m(m-1)}\sum_{i\neq j} k(\boldsymbol{y}_i,\boldsymbol{y}_j), \tag{17.31}$$

because $k(\boldsymbol{x}_i,\boldsymbol{x}_i) = 1$ contributes a bias of order $1/n$ that does not cancel between the three terms. The *energy distance* used throughout this chapter is the same construction with the conditionally negative definite kernel $-\|\boldsymbol{x}-\boldsymbol{y}\|$:

$$\mathscr{E}(p,q) = 2\,\mathbb{E}\|\boldsymbol{x}-\boldsymbol{y}\| - \mathbb{E}\|\boldsymbol{x}-\boldsymbol{x}'\| - \mathbb{E}\|\boldsymbol{y}-\boldsymbol{y}'\| \;\geq\; 0, \tag{17.32}$$

which needs no bandwidth to be chosen and is what makes it convenient for the two-dimensional experiments here. In high dimensions the choice of σ in Eq. (17.30) matters a great deal, which is the practical objection to MMD on images.

17.8.2 Fréchet inception distance

The most widely reported metric [57] sidesteps the kernel choice by working in the feature space of a fixed pretrained classifier. Push n real and n generated images through the penultimate layer of an InceptionV3 network, fit a Gaussian to each set of feature vectors, and report the Fréchet distance between the two Gaussians:

$$\mathrm{FID} = \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_g\|^2 + \mathrm{Tr}\Big(\boldsymbol{\Sigma}_r + \boldsymbol{\Sigma}_g - 2\big(\boldsymbol{\Sigma}_r\boldsymbol{\Sigma}_g\big)^{1/2}\Big). \tag{17.33}$$

Equation (17.33) is not an arbitrary formula: it is exactly the squared 2-Wasserstein distance between two Gaussians, so FID is W_2^2 computed in feature space under a Gaussian approximation. The connection to Section 17.7 is therefore closer than it looks – the metric of choice for evaluating GANs is a Wasserstein distance, in a space where the disjoint-support problem has been removed by the feature map.

Lower is better. The caveats are that FID depends on the feature extractor, that it is biased at small n (ten thousand samples is the usual minimum because $\boldsymbol{\Sigma}$ is a 2048×2048 matrix), and that a Gaussian is a crude model of the feature distribution. The older *inception score*, $\exp\big(\mathbb{E}_{\boldsymbol{x}} D_{\mathrm{KL}}(p(y|\boldsymbol{x}) \| p(y))\big)$, rewards confident and diverse classifications but never looks at the real data at all, and is correspondingly easy to fool.

No metric detects mode collapse reliably. A generator that reproduces one mode perfectly can achieve a respectable FID, because the Gaussian fit in Eq. (17.33) is dominated by the mean and the bulk of the covariance. This is why the experiments of Section 17.6.3 count modes explicitly, which is possible only because the target was constructed. For real data the honest procedures are precision-recall curves, which separate fidelity from coverage instead of averaging them, and looking at a large grid of samples. A single number that summarises a distribution will always be able to hide something.

17.9 Architectures

The theory above says nothing about how G and D are built, and in practice the architecture matters as much as the loss.

17.9.1 The fully connected baseline

For MNIST the original recipe is a pair of multilayer perceptrons,

$$\boldsymbol{z} \in \mathbb{R}^{100} \;\to\; 256 \;\to\; 512 \;\to\; 1024 \;\to\; \mathbb{R}^{784}, \tag{17.34}$$

with the generator using batch normalisation and $\mathrm{LeakyReLU}(0.2)$ after each linear layer and $\tanh$ at the output, and the discriminator mirroring it without batch normalisation and with dropout. Four conventions in that sentence deserve their reasons:

- $\tanh$ **at the generator output**, with the data rescaled to $[-1, 1]$. A bounded output prevents the generator from escaping to large values when the discriminator is briefly weak.
- **LeakyReLU rather than ReLU**, in both networks. A dead ReLU unit in D contributes no gradient to G either, and the adversarial game provides plenty of opportunity for units to die.
- **Batch normalisation in G but not in D.** In D it makes the verdict on one sample depend on the rest of the batch, which is both a leak of information and a source of instability; in G it keeps activations scaled while the target distribution moves.
- **Adam with $\beta_1 = 0.5$** and $\gamma = 2 \times 10^{-4}$. The reduced momentum is the practical response to Eq. (17.23).

17.9.2 Convolutions: DCGAN

The deep convolutional GAN of Radford, Metz and Chintala [64] replaced the fully connected layers with strided convolutions: fractionally strided (transposed) convolutions in G to upsample from $\boldsymbol{z}$ to an image, ordinary strided convolutions in D to downsample, no pooling anywhere, and no fully connected layers except the projection of $\boldsymbol{z}$. Weights are initialised from $\mathcal{N}(0, 0.02^2)$. These conventions are still the default starting point, and the architectural reason is the one from Chapter 10: the convolutional prior of locality and weight sharing is right for images, and a generator that must learn translation equivariance from scratch wastes its capacity on it.

17.9.3 Conditioning

A *conditional* GAN [65] gives both networks a side input $\boldsymbol{y}$ – a class label, a segmentation map, a text embedding:

$$\min_G \max_D \; \mathbb{E}_{\boldsymbol{x},\boldsymbol{y}\sim p_r}\big[\log D(\boldsymbol{x}\mid\boldsymbol{y})\big] + \mathbb{E}_{\boldsymbol{y}\sim p_r,\boldsymbol{z}\sim p_z}\big[\log\big(1-D(G(\boldsymbol{z}\mid\boldsymbol{y})\mid\boldsymbol{y})\big)\big]. \tag{17.35}$$

The analysis is unchanged: conditioning on $\boldsymbol{y}$ and applying Proposition 17.1 pointwise in $\boldsymbol{x}$ for each fixed $\boldsymbol{y}$ gives

$$D^*(\boldsymbol{x}\mid\boldsymbol{y}) = \frac{p_r(\boldsymbol{x}\mid\boldsymbol{y})}{p_r(\boldsymbol{x}\mid\boldsymbol{y}) + p_g(\boldsymbol{x}\mid\boldsymbol{y})}, \tag{17.36}$$

and Theorem 17.3 becomes a statement about the expectation over $\boldsymbol{y}$ of the conditional divergences, $\mathbb{E}_{\boldsymbol{y}}\big[D_{\mathrm{JS}}(p_r(\cdot|\boldsymbol{y})\|p_g(\cdot|\boldsymbol{y}))\big]$. For MNIST, $\boldsymbol{y}$ is a one-hot vector in $\mathbb{R}^{10}$ concatenated to $\boldsymbol{z}$ and, as an extra channel or an extra input vector, to $\boldsymbol{x}$. Conditioning is what turns a GAN from a curiosity into a tool: image-to-image translation, super-resolution and text-to-image synthesis are all Eq. (17.35) with different $\boldsymbol{y}$.

17.9.4 Reading the latent space

A trained generator can be probed by interpolating between two noise vectors. Linear interpolation,

$$\boldsymbol{z}(\alpha) = (1-\alpha)\boldsymbol{z}_1 + \alpha\boldsymbol{z}_2, \qquad \alpha\in[0,1], \tag{17.37}$$

is the obvious choice and it is wrong, for a reason worth computing. For $\boldsymbol{z}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I}_k)$ we have $\mathbb{E}\|\boldsymbol{z}\|^2 = k$, and the norm concentrates: its standard deviation tends to $1/\sqrt{2}$ independently of k. The midpoint of two independent draws has

$$\mathbb{E}\,\big\|\tfrac{1}{2}(\boldsymbol{z}_1+\boldsymbol{z}_2)\big\|^2 = \tfrac{1}{4}\big(k+k\big) = \frac{k}{2}, \tag{17.38}$$

so its norm is smaller by $\sqrt{2}$. For $k=100$ that is a shift from 10 to 7.07, some four standard deviations inward – a region from which the prior essentially never draws and on which the generator has therefore never been trained. Measured:

```
=== 8. the norm of a Gaussian latent vector ===
      k     E||z||    sd(||z||)   E||(z1+z2)/2||   sqrt(k/2)   deviation in sd
      2     1.2532      0.6537          0.8861      1.0000             0.6
     16     3.9372      0.7026          2.7835      2.8284             1.6
```

```
   100     9.9754      0.7049          7.0539      7.0711            4.1
```

The remedy is to interpolate along the sphere the prior actually occupies,

$$z(\alpha) = \frac{\sin\big((1-\alpha)\Omega\big)}{\sin\Omega}\, z_1 + \frac{\sin(\alpha\Omega)}{\sin\Omega}\, z_2, \qquad \cos\Omega = \frac{z_1 \cdot z_2}{\|z_1\|\|z_2\|}, \tag{17.39}$$

which preserves the norm when $\|z_1\| = \|z_2\|$ and reduces to Eq. (17.37) as $\Omega \to 0$. A smooth transition under Eq. (17.39) is evidence that the generator has learned a coherent manifold; abrupt jumps indicate that it has memorised a few outputs. It is a qualitative test, but it detects things FID does not.

Table 17.1 The main variants. All but the last two rows differ only in the loss or the conditioning; the architecture rows change what the networks are made of.

Variant	Change	Objective	What it buys
GAN	–	D_{JS}, Eq. (17.12)	the construction
Non-sat.	$-\log D(G(z))$	same fixed point	gradients when losing
cGAN	side input y	conditional D_{JS}	control
WGAN	critic, weight clipping	W, Eq. (17.28)	gradients on disjoint supports
WGAN-GP	gradient penalty	Eq. (17.29)	Lipschitz without clipping
DCGAN	strided convolutions	any of the above	images
StyleGAN [66]	style-modulated generator	any of the above	control of scale-wise detail

17.10 GANs in PyTorch and TensorFlow

The two-dimensional programs above were written with explicit gradients so that the two players could be seen separately. In a library the same structure appears as two optimisers and one detach. Here is the PyTorch version of the architectures of Section 17.9.1; note that the discriminator returns a *logit*, so that `BCEWithLogitsLoss` can apply the sigmoid internally and the losses of Eqs. (17.17) and (17.20) are evaluated stably:

```
import torch
import torch.nn as nn


def make_generator_fc(k_z=100, n_out=784, widths=(256, 512, 1024)):
    """z in R^k -> image in [-1,1]^784, Eq. (17.generator)."""
    layers, n_in = [], k_z
    for w in widths:
        layers += [nn.Linear(n_in, w), nn.BatchNorm1d(w),
                   nn.LeakyReLU(0.2, inplace=True)]
        n_in = w
    layers += [nn.Linear(n_in, n_out), nn.Tanh()]
    return nn.Sequential(*layers)


def make_discriminator_fc(n_in=784, widths=(1024, 512, 256), p_drop=0.3):
    """image -> logit u(x); D(x) = sigmoid(u(x)).  No batch norm in D."""
    layers = []
    for w in widths:
        layers += [nn.Linear(n_in, w), nn.LeakyReLU(0.2, inplace=True),
                   nn.Dropout(p_drop)]
```

```
        n_in = w
    layers += [nn.Linear(n_in, 1)]
    return nn.Sequential(*layers)
```

The training step is the algorithm of Section 17.5 written out. The single most important line is `.detach()`: without it the discriminator update would also send gradients backwards into the generator, which is not what Eq. (17.15) says.

```
G, D = make_generator_fc(), make_discriminator_fc()
opt_g = torch.optim.Adam(G.parameters(), lr=2e-4, betas=(0.5, 0.999))
opt_d = torch.optim.Adam(D.parameters(), lr=2e-4, betas=(0.5, 0.999))
bce = nn.BCEWithLogitsLoss()

for x, _ in loader:                          # x in [-1,1], flattened
    n = x.size(0)

    # --- discriminator: gradient ascent on V, Eq. (17.dstep) --------------
    z = torch.randn(n, 100)
    x_fake = G(z).detach()                   # no gradient into G here
    u_real, u_fake = D(x), D(x_fake)
    loss_d = (bce(u_real, torch.full_like(u_real, smooth))   # smooth = 1.0 or 0.9
              + bce(u_fake, torch.zeros_like(u_fake)))       # Eq. (17.dloss)
    opt_d.zero_grad(set_to_none=True)
    loss_d.backward()
    opt_d.step()

    # --- generator: non-saturating loss, Eq. (17.nonsat) ------------------
    z = torch.randn(n, 100)
    loss_g = bce(D(G(z)), torch.ones(n, 1))    # = -log D(G(z))
    opt_g.zero_grad(set_to_none=True)
    loss_g.backward()
    opt_g.step()
```

The gradient penalty of Eq. (17.29) needs a derivative with respect to an *input*, which is why `create_graph=True` appears: the penalty is itself differentiated when the critic is updated.

```
def gradient_penalty(D, x_real, x_fake):
    eps = torch.rand(x_real.size(0), *([1] * (x_real.dim() - 1)))
    xhat = (eps * x_real + (1 - eps) * x_fake).requires_grad_(True)
    g = torch.autograd.grad(D(xhat).sum(), xhat, create_graph=True)[0]
    return ((g.flatten(1).norm(2, dim=1) - 1.0) ** 2).mean()
```

The TensorFlow version is the same algorithm with different bookkeeping. PyTorch accumulates gradients on the parameters and we zero them; TensorFlow records a tape and we ask it for the gradient of one loss with respect to one list of variables. The two tapes are the two players:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

bce = keras.losses.BinaryCrossentropy(from_logits=True)


def make_generator_fc(k_z=100, n_out=784, widths=(256, 512, 1024)):
    m = keras.Sequential([keras.Input(shape=(k_z,))])
    for w in widths:
        m.add(layers.Dense(w, use_bias=False))
        m.add(layers.BatchNormalization())
        m.add(layers.LeakyReLU(negative_slope=0.2))
    m.add(layers.Dense(n_out, activation="tanh"))
```

```
    return m


def make_discriminator_fc(n_in=784, widths=(1024, 512, 256), p_drop=0.3):
    m = keras.Sequential([keras.Input(shape=(n_in,))])
    for w in widths:
        m.add(layers.Dense(w))
        m.add(layers.LeakyReLU(negative_slope=0.2))
        m.add(layers.Dropout(p_drop))
    m.add(layers.Dense(1))                    # a logit, not a probability
    return m


G, D = make_generator_fc(), make_discriminator_fc()
opt_g = keras.optimizers.Adam(2e-4, beta_1=0.5)
opt_d = keras.optimizers.Adam(2e-4, beta_1=0.5)


@tf.function
def train_step(x, k_z=100, smooth=1.0):
    n = tf.shape(x)[0]

    # --- discriminator -------------------------------------------------
    x_fake = G(tf.random.normal([n, k_z]), training=True)   # outside the tape
    with tf.GradientTape() as tape:                         # = detach()
        u_real, u_fake = D(x, training=True), D(x_fake, training=True)
        loss_d = (bce(tf.fill(tf.shape(u_real), smooth), u_real)
                  + bce(tf.zeros_like(u_fake), u_fake))
    opt_d.apply_gradients(zip(tape.gradient(loss_d, D.trainable_variables),
                              D.trainable_variables))

    # --- generator, Eq. (17.nonsat) ------------------------------------
    with tf.GradientTape() as tape:
        u_fake = D(G(tf.random.normal([n, k_z]), training=True), training=True)
        loss_g = bce(tf.ones_like(u_fake), u_fake)
    opt_g.apply_gradients(zip(tape.gradient(loss_g, G.trainable_variables),
                              G.trainable_variables))
    return loss_d, loss_g, tf.sigmoid(u_real), tf.sigmoid(u_fake)
```

In both frameworks the quantities to watch during training are not the losses – which oscillate, by Section 17.6.1 – but the mean of $D(\boldsymbol{x})$ and of $D(G(\boldsymbol{z}))$. Theorem 17.3 says both should approach $\frac{1}{2}$. If $D(G(\boldsymbol{z}))$ collapses towards zero the discriminator is winning and the generator is starving; if $D(\boldsymbol{x})$ falls towards $\frac{1}{2}$ while $D(G(\boldsymbol{z}))$ rises above it, the discriminator has been overwhelmed and its gradient is no longer informative. Both are reasons to stop, and neither is visible in the loss curves.

When to stop. There is no early-stopping criterion for a GAN, because there is no held-out score that decreases monotonically. The practical procedure is to save samples on a fixed noise batch $\boldsymbol{z}_{\text{fixed}}$ at every epoch – as both programs do – and to select the epoch afterwards, by FID if the domain has a trusted feature extractor and by eye if not. Keeping the noise fixed matters: it separates changes in the generator from changes in the sample.

17.11 Summary and the programs

A generative adversarial network replaces the likelihood by a classifier. The classifier's optimum, Proposition 17.1, is the density ratio $p_r/(p_r+p_g)$ in disguise, verified here to 3×10^{-9} pointwise and to 0.024 rms by a trained network wherever data exist – and badly violated

where they do not, which is Figure 17.1(b) and the first warning of the chapter. Substituting that optimum back, the generator turns out to be minimising a Jensen-Shannon divergence, Theorem 17.3, confirmed against quadrature to 10^{-12}, with the global minimum $-2\log 2$ attained exactly when $p_g = p_r$ and $D \equiv \frac{1}{2}$.

That is the theory, and the rest of the chapter is the distance between it and the algorithm. The saturating loss of Eq. (17.4) starves the generator by a factor $(1-D)/D$, Eq. (17.19), which we measured growing to 23 while the discriminator trained alone; the non-saturating loss of Eq. (17.20) fixes it without moving the fixed point. Label smoothing caps the discriminator at s, Proposition 17.4, and was measured holding $D(\boldsymbol{x})$ down to 0.86 instead of 0.97. Alternating gradient updates do not converge to a Nash equilibrium: on the bilinear game the simultaneous iteration diverges exactly as $(1+\gamma^2)^{t/2}$ and the alternating one circulates forever, both confirmed to five figures, which is why our trained GAN sat at $V = -1.275$ rather than -1.386 and why the loss curves of any GAN oscillate.

Two limitations are structural rather than numerical. The Jensen-Shannon divergence is pinned at $\log 2$ on disjoint supports, Lemma 17.2, so early training has no gradient at all in the idealised objective; the Wasserstein distance of Eq. (17.28) with the gradient penalty of Eq. (17.29) repairs this, and cut the energy distance on our benchmark from 0.080 to 0.026 at four times the cost. And the support of p_g is connected whatever the parameters, Proposition 17.5, so a disconnected target cannot be matched: none of our three objectives covered more than seventeen of twenty-five modes, and all three produced the connecting filaments of Figure 17.3.

What is bought with all this is a generator that produces sharp samples in a single forward pass, with no bound, no partition function and no iterative sampler – and an evaluation problem, since with no likelihood the only available scores are two-sample statistics, Eqs. (17.30) and (17.33), none of which reliably detects the mode collapse the method is prone to.

The programs are in the directory
`doc/BookML/BookPrograms/chapter17_gan`.
Every listing above appears there, and five modules run start to finish and reproduce the numbers quoted in the text:

- `gan.py` – the networks, the three generator losses of Section 17.5.1, the gradient penalty of Eq. (17.29), Adam with $\beta_1 = 0.5$, the alternating training loop, and the two-dimensional targets.
- `verify_gan.py` – the eight checks: the pointwise optimum, the trained discriminator against Eq. (17.6), the Jensen-Shannon identity, label smoothing, the two generator gradients, the failure of D_{JS} on disjoint supports, the bilinear game, and the geometry of the Gaussian prior.
- `run_compare.py` – the equilibrium experiment of Section 17.5.3, the gradient-starvation experiment of Section 17.5.1, and the mode-collapse comparison of Section 17.6.3.
- `gan_torch.py` – the MNIST program in PyTorch, with the fully connected and DCGAN architectures and the non-saturating, saturating and WGAN-GP losses selectable from the command line.
- `gan_tf.py` – the same program in TensorFlow.

The figures are generated by `ch17_figures.py` in `doc/BookML/BookFigures`; none is drawn by hand.

17.12 Exercises

Warm-up exercises

1. **The optimal discriminator.** (a) Derive Eq. (17.6) by differentiating $f(t) = A\log t + B\log(1-t)$ and verify with Eq. (17.8) that it is a maximum and not a minimum. (b) What is D^* at a point where $p_r > 0$ but $p_g = 0$, and what does the generator learn there? (c) Show that the logit of D^* is the log density ratio, Eq. (17.9), and explain why this makes D^* a substitute for a likelihood.
2. **Bounds on the Jensen-Shannon divergence.** Prove Lemma 17.2 and state the conditions for equality at each end. Then compute D_{JS} for two Bernoulli distributions with parameters p and q and plot it; where is it steepest?
3. **Vanishing gradients.** At $D(G(\boldsymbol{z})) = \varepsilon \ll 1$, compute the magnitude of the generator gradient for the saturating and the non-saturating losses and confirm Eq. (17.19). Then explain why the measured ratio in Section 17.5.1 was 23 when $1/D(G(\boldsymbol{z}))$ was 41.5.
4. **Label smoothing.** (a) Derive Proposition 17.4. (b) Repeat the derivation smoothing *both* labels, real to s and generated to $\varepsilon > 0$, and show that the optimum becomes $\big(sp_r + \varepsilon p_g\big)/(p_r + p_g)$. Explain from this expression why two-sided smoothing is a bad idea.
5. **Wasserstein against Jensen-Shannon.** (a) Derive Eq. (17.26) for the two uniform distributions. (b) Show that $W(\mathcal{N}(0,1), \mathcal{N}(\mu,1)) = |\mu|$, using the fact that in one dimension the optimal transport plan matches quantiles. (c) Which of the two has a useful gradient at $\mu = 10$?
6. **The bilinear game.** Verify the eigenvalues quoted for Eqs. (17.23) and (17.25), derive Eq. (17.24), and show that the alternating iteration conserves a quadratic form. What does this conserved quantity correspond to in a real GAN?
7. **Counting.** A GAN, a VAE and a diffusion model with $T = 1000$ each generate one image. How many network evaluations does each need? How many does each need to evaluate, or bound, $\log p(\boldsymbol{x})$ for a given $\boldsymbol{x}$?
8. **Conditioning.** Derive Eq. (17.36) and state the conditional form of Theorem 17.3. What happens if the generator ignores $\boldsymbol{y}$ entirely – can the discriminator detect it, and if so how?

Project-style exercise: an adversarial generator

Part a: the theory, numerically.

Reproduce the checks of Section 17.3 and Section 17.4: maximise the integrand pointwise against Eq. (17.6), train a discriminator on two known one-dimensional densities and compare it with the closed form both where the data are and where they are not, and verify Eq. (17.12) against quadrature for several pairs of Gaussians. Report the accuracy you obtain and explain the size of each discrepancy.

Part b: the two losses.

Implement both generator losses. Freeze an untrained generator, train the discriminator alone, and plot both generator gradient norms against $D(G(\boldsymbol{z}))$ as in Figure 17.1(c). Then train two full GANs, one with each loss, from the same seed, and compare. Under what conditions does the saturating loss actually fail, and why did it not fail on the eight-Gaussian problem of Section 17.5.3?

Part c: the equilibrium.

Train on the eight-Gaussian mixture and monitor $D(\boldsymbol{x})$, $D(G(\boldsymbol{z}))$ and V. Do they approach the values of Theorem 17.3? Vary the learning-rate ratio γ_d/γ_g and n_{critic}, and report how close to $-2\log 2$ you can get. Then add Adam momentum $\beta_1 = 0.9$ and describe what happens in the light of Eq. (17.23).

Part d: mode collapse.

Reproduce the 5×5 experiment. Count modes, plot the histogram of Figure 17.4, and try at least three mitigations: minibatch discrimination, one-sided label smoothing, and WGAN-GP. Which helps most, and does any of them beat Proposition 17.5? Test the proposition directly by plotting G along a straight line in latent space and showing that the output path is continuous.

Part e: Wasserstein.

Implement the critic and the gradient penalty of Eq. (17.29). Verify that the penalty really does keep $\|\nabla f_w\|$ near 1 by measuring it on interpolates during training. Compare $\lambda = 0$, 1, 10 and 100, and compare the gradient penalty against weight clipping at several clip values. Does the critic loss track sample quality, as claimed in Section 17.7?

Part f: MNIST.

Train the fully connected GAN of Section 17.9.1 on MNIST in PyTorch or TensorFlow, saving samples from a fixed noise batch at every epoch. Then train the DCGAN version and compare. Report FID or, if that is too expensive, MMD in the feature space of a small classifier you train yourself. Show the latent interpolations of Eqs. (17.37) and (17.39) side by side and comment on the difference in the light of Eq. (17.38).

Part g: conditioning.

Extend your MNIST GAN to the conditional form of Eq. (17.35) and generate each digit on demand. Then use the conditional generator for data augmentation: train a classifier on 1000 real images, and on 1000 real images plus generated ones, and report the test accuracy of each. Does the augmentation help, and does the answer depend on how far the generator has been trained?

Part h: a physics application.

Train a GAN on configurations of the two-dimensional Ising model at several temperatures, as in part e of the Chapter 14 project, and compare the energy and magnetisation histograms of the generated configurations with the true ones. Then compare against the RBM of Chapter 14 and the diffusion model of Chapter 16 at matched parameter counts. Which reproduces the critical region best, which is fastest to sample, and which would you trust to report an uncertainty? Defend your answer with the divergences of this chapter, not with the pictures.

Part V
Reinforcement Learning

Part VI
Concluding remarks and perspectives

Chapter 18
Concluding remarks and perspectives

Seventeen chapters is a long argument, and it is worth saying plainly what the argument was.

This book has not been a catalogue of methods. A catalogue would have been easier to write and would have dated faster. What it has tried to do instead is to derive a small number of ideas carefully and then follow them until they stop working, because the place where an idea stops working is the place where the next one is invented. Ridge regression stops working when the relationship is not linear, and the sigmoid of Chapter 5 is the smallest possible repair. Logistic regression stops working when the features themselves must be learned, and Chapter 8 inserts a hidden layer. A dense layer stops working when the input is an image, and Chapter 10 replaces full connectivity by a shared local kernel. Maximum likelihood stops working when the normalising constant cannot be computed, and Chapters 14 to 17 are four different ways of proceeding without it. Read in that order the book is not seventeen subjects but one subject seventeen times.

This closing chapter does three things. Section 18.1 sets out the shape of the whole and the dependencies between its parts. Section 18.2 follows the handful of ideas that recur, and argues that they are the part of the material with the longest shelf life. Section 18.3 turns them round and states them as a working order for attacking a problem that has not been seen before. Sections 18.4 and 18.5 then look outward, at what has been left out and at where these methods are being taken in the sciences. Section 18.6 is a list of things to be sceptical about, drawn wherever possible from results in this book that did not come out as expected.

18.1 The shape of the argument

Figure 18.1 is the book drawn as a graph. It is worth a minute of study, because the interesting thing about it is how few of the arrows are merely chronological.

The two foundational chapters are not preliminaries in the usual sense of material one may skip. Chapter 1 supplies the objects – a data set is a design matrix, a model is a matrix acting on a parameter vector, and fitting is projection onto a column space – and it supplies the one number, the condition number, that reappears in three different roles later. Chapter 2 supplies the discipline: every quantity computed from a finite sample is itself a random variable, with a bias and a variance that the mean squared error of Eq. (2.27) weighs equally. Without the first, the algebra of the later chapters is unreadable; without the second, their numbers are unbelievable.

The five chapters of statistical learning that follow are, deliberately, all the same model seen from different angles. Chapter 3 derives the least-squares estimator three times – as an optimisation problem, as a projection and as a maximum-likelihood estimate – because each derivation generalises differently and the reader needs all three later. Chapter 4 is what hap-

pens when the closed form is withdrawn. Chapter 5 changes one function and finds that the loss changes with it, without being chosen. Chapter 6 arrives at a third loss from a geometric principle and finds it belongs to the same family: Ridge regression, logistic regression and the support vector machine are one model with one penalty and three losses, which is the observation Section 6.12 closes on. Chapter 7 is the odd one out and the more valuable for it, because it is the only place in the book where the model is improved not by changing it but by averaging many copies of it, and Eq. (7.16) is the specification that every ensemble in the sciences obeys.

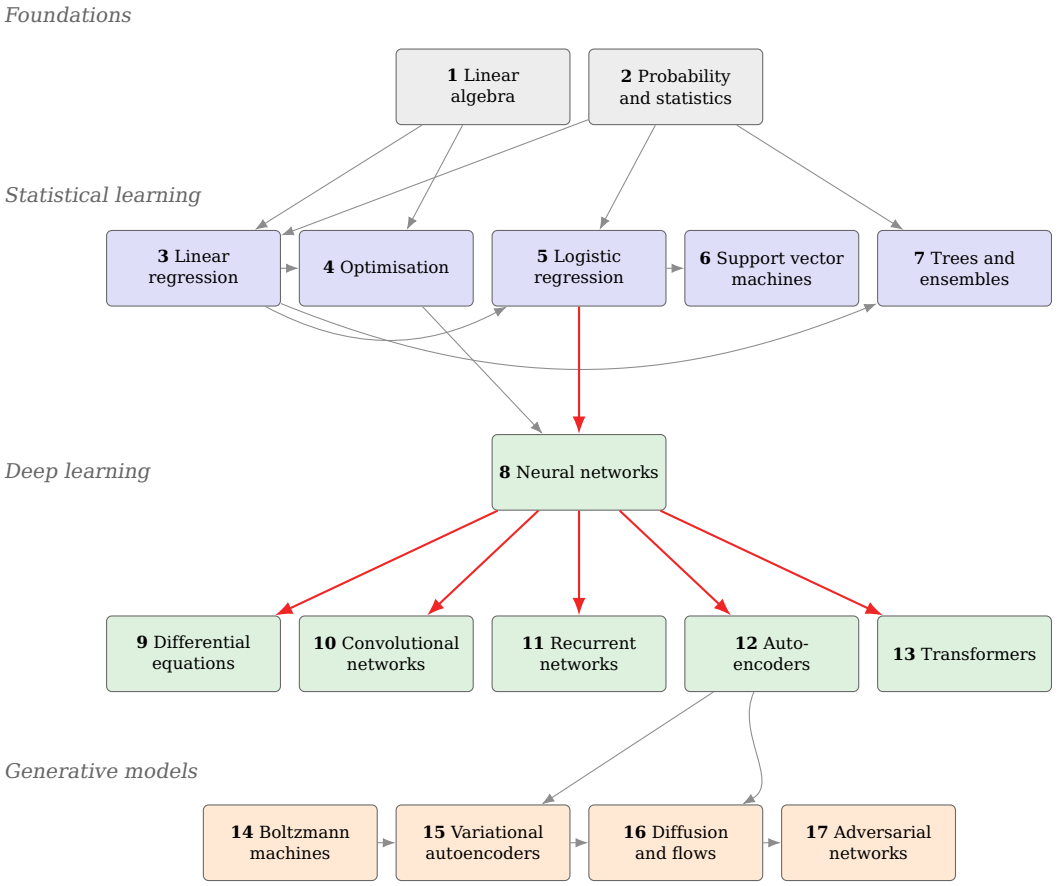


Fig. 18.1 The book as a graph. An arrow means that the later chapter uses a result of the earlier one, not merely that it comes after it. The two red paths are the ones the book is built around: logistic regression *is* a network with no hidden layer, so Chapter 8 begins by inserting one; and Chapters 9 to 13 all use the backpropagation of Section 8.8 unchanged – Chapters 10 to 13 by changing what structure is imposed on the weight matrix, Chapter 9 by keeping the network and changing the question it is asked. Two links are not drawn because they would cross the whole figure. The linear autoencoder of Chapter 12 rediscovers the singular value decomposition of Chapter 1 – that is Theorem 12.5 – and the Boltzmann machine of Chapter 14 is sampled by the Metropolis algorithm of Section 2.14.

Chapter 8 is the hinge of the book, and the fan of arrows leaving it is the point. Convolutional, recurrent, autoencoding and attention-based networks are not four subjects. They are four answers to a single question – what structure should be imposed on the weight matrix? – and they share, without modification, the four equations of backpropagation derived in Section 8.8.4. A convolutional layer is a dense layer whose weights are tied and local; a recurrent layer is a dense layer whose weights are tied across time; an autoencoder is a pair of dense stacks with a narrow layer between them; attention is a dense layer whose weights

are computed from the data rather than stored. Once that is seen, the differences between the four chapters are differences of consequence, not of kind, and the consequences are what those chapters prove: equivariance under translation (Theorem 10.7) for the first, exponential decay of gradients along the time axis (Theorem 11.1) for the second, an exact identity with principal component analysis (Theorem 12.5) for the third, and equivariance under *all* permutations (Theorem 13.2) for the fourth.

The last four chapters change the question. Everything before them predicts a target given an input; these model the distribution of the data itself. That change costs one thing and one thing only, and it is worth naming precisely because the whole part is organised around it: a probability model needs a normalising constant, and the normalising constant is a sum or an integral over a space too large to touch. Chapter 14 refuses to compute it and samples instead. Chapter 15 bounds it. Chapter 16 sidesteps it in one direction by learning a score and in the other by restricting to maps whose Jacobian determinant is cheap. Chapter 17 abandons likelihood altogether and trains a classifier in its place. Four chapters, four evasions of the same obstruction; we return to this in Section 18.2.7.

18.2 The threads

The methods in this book will age at different rates. The ideas that connect them will age more slowly, and this section collects the ones that recurred often enough to be worth carrying away independently of any architecture.

18.2.1 One number governs accuracy, variance and training time

The condition number $\kappa_2(\boldsymbol{X}) = \sigma_0/\sigma_{r-1}$ appeared three times in three different roles, and it is the same number each time.

In Section 1.12 it decides how much of the input precision survives a linear solve, and the observation that $\kappa_2(\boldsymbol{X}^T\boldsymbol{X}) = \kappa_2(\boldsymbol{X})^2$ is why the least-squares routines one should actually call are built on the QR decomposition or the singular value decomposition rather than on the normal equations. In Section 3.7 the same quantity governs the statistical variance of the fitted parameters, through $\mathrm{Var}(\hat{\boldsymbol{\theta}}) = \sigma^2(\boldsymbol{X}^T\boldsymbol{X})^{-1}$: directions in which the data are uninformative are exactly the directions in which the estimate is uncertain. And in Chapter 4, Eq. (4.27) shows that it sets the number of gradient-descent iterations, because the step size is bounded above by the largest eigenvalue of the Hessian while progress is governed by the smallest.

Three apparently unrelated difficulties – numerical, statistical and computational – are therefore one difficulty. This has a practical consequence that is easy to state and easy to forget: standardising the features and adding a Ridge penalty improve all three at once, and they do so for the same reason. A student who leaves this book remembering only that a badly conditioned design matrix is bad news in three separate ways has taken away something that will outlast every architecture in it.

18.2.2 The loss is derived, not chosen

The single most useful habit this book has tried to instil is to stop treating the loss function as a design decision. Specify what the target is assumed to be, and the loss follows.

Gaussian noise gives the squared error, Eq. (3.27). Bernoulli observations give the cross entropy, Eq. (5.13); the notebox in Section 3.6 predicted this before Chapter 5 confirmed it. A categorical target gives the multiclass cross entropy, and the softmax that pairs with it is not a convenience but the inverse link of the same model. Non-negative unbounded data suggest a Poisson likelihood and a different loss again. Even the penalties are not free choices: Section 3.11 showed that a flat prior gives ordinary least squares, a Gaussian prior gives Ridge, and a Laplace prior gives the Lasso, so that regularisation is a prior belief and the regularisation parameter is its strength.

The habit pays off most in the last part of the book, where the models are strange. A restricted Boltzmann machine's loss is the negative log-likelihood of a canonical ensemble, and Theorem 14.2 is nothing but its derivative. A variational autoencoder's loss is a bound on the same log-likelihood, Theorem 15.1. A diffusion model's loss looks like a plain squared error, Eq. (16.9), and it is startling to discover that it is one – but it is one only after a variational derivation that begins from a likelihood. Even the adversarial loss of Chapter 17, which appears to break the pattern entirely, turns out by Theorem 17.3 to be minimising a divergence between distributions. In every case the object being minimised was determined by a modelling assumption, and in every case knowing which assumption tells you when the method will fail.

The reverse reading is the useful one. If a loss function is being used whose generating assumption cannot be named, that is a fact about the model, and usually a fact worth discovering early. Squared error on bounded data assumes Gaussian noise on a quantity that cannot be Gaussian, which is why Section 12.1 recommends a logistic output and the binary cross entropy for pixel intensities. A network trained with a loss nobody can justify is a network whose failures nobody will be able to explain.

18.2.3 Bias, variance, and the two ways to trade them

Equation (2.52) is the most-used result in the book. It says that expected test error decomposes into a bias term, a variance term and an irreducible noise floor, and that the first two move in opposite directions as model complexity grows.

What matters is that the book showed two structurally different ways to exploit it, and they are worth holding apart. The first is *regularisation*: keep one model and constrain it. Ridge shrinks each singular direction by $\sigma_i^2/(\sigma_i^2+\lambda)$, hardest in exactly those directions in which the data are least informative; the Lasso replaces proportional shrinkage by the soft thresholding of Eq. (3.63) and thereby sets coefficients to zero; dropout, weight decay and early stopping do the same job for networks in Section 8.12; and the bottleneck, sparsity, contraction and denoising penalties of Section 12.6 are four more, which the notebox there argues are the same idea four times.

The second is *ensembling*: keep the complexity and average away the variance. Equation (7.16), $\mathrm{Var}(\bar{f}) = \rho\sigma^2 + (1-\rho)\sigma^2/B$, is the entire specification. It says that averaging B models kills the second term and leaves the first untouched, so the only way to improve further is to decorrelate the members – which is precisely what random forests do by restricting each split to a random subset of features. It also explains, in one line, why deep ensembles remain one of the most reliable uncertainty estimates available for neural networks [67], and why they are expensive.

Boosting reverses the strategy: instead of averaging strong learners to remove variance, it sums weak ones to remove bias. That gradient boosting turns out to be gradient descent

performed in function space is the kind of connection this book has looked for throughout, and it is the reason Chapter 7 sits where it does rather than being relegated to an appendix.

18.2.4 The chain rule is enough

Chapter 8 opened by admitting that inserting a hidden layer costs every guarantee the previous chapters relied on: no convexity, no closed form, no normal equations, no Karush-Kuhn-Tucker conditions to check the answer against. What was left was gradient descent and the chain rule, and the rest of the book is a demonstration that this is enough.

The four equations of Section 8.8.4 are used, unmodified, by every architecture that follows. The convolutional gradients of Section 10.8 are those equations with the sum restricted by the kernel's support; backpropagation through time is those equations on an unrolled graph with the gradients from every step summed into shared parameters; attention differentiates like any other composition of matrix products and a softmax. The forward-mode recursions of Chapter 9 are the same calculus run in the other direction, propagating derivatives with respect to the *input* rather than the parameters, at one extra matrix product per layer per order.

The cost accounting is what makes it practical, and it is worth restating because it is the reason deep learning is possible at all. Backpropagation delivers the gradient with respect to *all* parameters for about twice the cost of a forward pass, independently of how many parameters there are. Finite differences would need one forward pass per parameter. For a model with 10^6 weights the difference is between a second and a fortnight, and for a model with 10^{11} weights it is the difference between a research programme and an impossibility. This is not a fact about neural networks; it is reverse-mode automatic differentiation, Section 4.14, and it applies to any differentiable computation, including the classical simulation codes that readers of this book are likely to have written.

18.2.5 Structure is a prior, and it is the cheapest one available

The three architecture chapters make the same argument with different symmetries, and it is the argument that most repays generalisation to a new problem.

A dense layer on a colour image of side L has $9L^4$ weights, which is ten billion for a modest photograph. A convolutional layer has $K(CF^2+1)$, independent of the image size. The saving is bought by two assumptions – locality and stationarity – and Chapter 10 is careful to insist that both are properties of the data and not of the method. For images they hold; for a table of unrelated clinical measurements they do not, and a convolutional layer would be an actively bad choice. Theorem 10.7 turns the informal claim into an exact statement: convolution commutes with translation, so a feature learned in one place is available everywhere. Chapter 13 then makes the complementary choice. Attention is equivariant under *all* permutations, a far larger group, which is why it assumes almost nothing about the data – and why position must be handed back to it explicitly through the encoding of Proposition 13.5.

The lesson generalises well beyond these two cases, and it is the single most transferable idea in the book for a reader working in the physical sciences. A symmetry of the problem, built into the architecture, is a prior that costs no data to enforce and cannot be violated by the fit. Equivariant graph networks enforce rotations and translations in three dimensions exactly, and the consequence is dramatic: interatomic potentials of the NequIP and MACE families [68, 69] reach chemical accuracy from training sets some two to three orders

of magnitude smaller than non-equivariant architectures require. Data efficiency bought by symmetry is the most reliable free lunch in the subject – and Section 18.6 will note that there are few others [70].

18.2.6 One decomposition, read several ways

Chapter 1 closes by observing that the singular value decomposition separates the part of a problem that carries information from the part that does not, and that the same list of singular values answers five different questions. The rest of the book kept finding a sixth.

The numerical rank of a design matrix, the distance of a feature set from collinearity, the variance retained by a projection, the best low-rank approximation by Eq. (1.124), and the shrinkage applied by Ridge in Section 1.22 are the five. The sixth is Theorem 12.5: a linear autoencoder trained by gradient descent on reconstruction error recovers the principal subspace, so that the most fashionable architecture of the deep-learning part and the oldest tool of multivariate statistics are, in the linear case, the same object. That theorem is the reason Chapter 12 is worth its length. It gives the exact floor – the Eckart-Young error – against which a nonlinear autoencoder must be measured, and without such a floor the claim that a nonlinearity helped is untestable.

The generalisation is worth stating: whenever a method compresses, ask what it would do if it were linear, and whether the linear answer is available in closed form. If it is, that answer is the baseline, and a method that fails to beat it has not earned its extra parameters.

18.2.7 An intractable normalisation, and four ways round it

The generative part of this book is organised around a single obstruction. A probability model must integrate to one, and the constant that makes it do so is a sum over a space of size 2^{M+N} for a Boltzmann machine, or an integral over all latent configurations for a latent-variable model. It cannot be computed and it will not go away. What can be done is to find a formulation in which it is not needed, and the four chapters are four such formulations.

Sample instead of computing. Theorem 14.2 shows that the gradient of the log-likelihood is a difference of two expectations, and neither requires the value of Z – only samples from the distribution it normalises. Metropolis and Gibbs supply those samples using ratios alone, in which Z cancels. The price is that the samples are correlated and the chain must be run long enough, and Section 14.6.1 measures exactly what is lost when it is not: contrastive divergence with a single step gives a gradient wrong by 33% in magnitude, a bias that averaging cannot remove, whose cosine with the true gradient is nevertheless 0.986. It is a short vector pointing very nearly the right way, which is why an algorithm that is provably biased trains perfectly well.

Bound it. Theorem 15.1 replaces $\log p(\boldsymbol{x})$ by a quantity that is computable and never larger, with the gap equal to the divergence between the approximate and the true posterior. Raising the bound and improving the posterior approximation are then one act. The price is that the gap is real and unknown; Chapter 15 measures it by importance sampling and finds it is several nats, so the evidence lower bound is not a tight estimate of the log-likelihood and should not be reported as one.

Learn a score, or restrict the map. Chapter 16 makes both moves. A diffusion model never represents a density; it learns $\nabla_{\boldsymbol{x}} \log p$, in which the normalisation has been differentiated away, and Proposition 16.4 identifies the trained noise-prediction network with exactly that

object. A normalizing flow takes the opposite route and keeps the density exactly, by restricting to invertible maps whose Jacobian determinant is cheap, Theorem 16.5.

Replace the likelihood by a classifier. Chapter 17 gives up on densities entirely. Proposition 17.1 shows that an optimal discriminator is a density ratio in disguise, and Theorem 17.3 that the generator is then minimising a Jensen-Shannon divergence. The price is that the training problem is a game rather than a minimisation, and games need not converge: the chapter measures a simultaneous iteration diverging exactly as $(1+\gamma^2)^{t/2}$ on a bilinear problem, and an alternating one circulating forever.

Four evasions, four different prices. A reader who understands the obstruction will be able to place the next generative model to appear – flow matching, consistency models, whatever follows – by asking which of the four routes it takes, and what it therefore has to give up.

18.2.8 Representability is not learnability, and neither is generalisation

Section 8.4 spends its length distinguishing three questions that are routinely run together, and the distinction is worth carrying away as a discipline of thought.

Can the model class contain the function? For networks the answer is almost always yes, and Theorem 8.2 makes it quantitative in one dimension: N rectified units suffice for an error $L(b-a)/2N$, with the weights written down rather than proved to exist. *Will gradient descent find it?* Section 8.4.4 runs the two side by side at matched width and finds that training falls further behind the constructed network as the width grows – a factor of 5.3 at $N=64$ – and that a poor initialisation is a larger handicap than the optimisation itself. *And will the result generalise?* That is Eq. (2.52), and it is a statement about the model class, the data and the estimator together, which no approximation theorem addresses.

The same three-way distinction organises the honest reading of the rest of the book. Theorem 8.6 shows a function a deep network represents exactly and a shallow one provably cannot approximate – a statement about representability, proved in half a page. Theorem 11.1 shows that a recurrent network's gradients decay exponentially with lag – a statement about learnability, which is why the long-memory tasks of Chapter 11 fail even when the architecture could express the answer. And the experiment of Section 10.10 shows a convolutional network with 2.5 times fewer parameters beating a dense one by seventeen points of accuracy on shifted digits while being indistinguishable from it on centred ones – a statement about generalisation, and about the fact that an architectural advantage exists only when the data have the structure the architecture assumes.

18.3 A protocol for a new problem

The threads of Section 18.2 are not only retrospective. They compose into a working order for attacking a problem that has not been seen before, and it is worth writing that order out, because the most common way to waste six months is to begin at step six.

1. Say what the target is.

Not what model you intend to use – what the quantity being predicted actually is. Continuous and unbounded, bounded to an interval, non-negative, a count, a category, an ordered cate-

gory, a direction, a distribution. This determines the output activation and, by Section 18.2.2, the loss; and the question is worth asking before any code is written because it is frequently the question that reveals the problem has been posed wrongly. If no distributional assumption can be named, the loss cannot be justified, and every later disagreement about which model is better will be an argument about an arbitrary choice.

2. Look at the design matrix before the model.

Compute the singular values. The condition number tells you three things at once by Section 18.2.1 – how much precision a solve will lose, how uncertain the fitted parameters will be, and how slowly gradient descent will converge – and the decay of the spectrum tells you the effective dimension of the problem, which is usually far smaller than the number of columns. Standardise, and know why you are standardising. This step costs minutes and routinely changes what is attempted next.

3. Fix the evaluation before fitting anything.

Decide the split, and decide it according to the structure of the data rather than at random: temporally ordered data need a temporal split, grouped data need a grouped split, and spatial data need a spatial one, because the alternative is the leakage of Section 18.6 and it will flatter every result that follows. Decide the metric, and decide it from the decision the model is meant to support – Section 5.12 on why accuracy is close to meaningless under class imbalance is the relevant caution. Decide, in advance, what difference would count as a real difference; and since almost every method in this book has a seed-to-seed spread, plan to run several seeds and report the spread rather than the best.

4. Fit the linear model.

Ordinary least squares, Ridge, logistic regression, whichever the target dictates. It takes an afternoon, it has a closed form or a convex objective, its uncertainty is available analytically, and it is interpretable. It is also, surprisingly often, competitive. Whatever comes later must beat this number, and a great deal of published work would look different if this step were mandatory.

5. Try the ensemble of trees.

Gradient boosting on tabular data remains extremely hard to beat, it needs no feature scaling, it is invariant under monotone transformations of the inputs, and it will tell you which features matter. If the data are a table of heterogeneous measurements, this may well be the end of the exercise; the architectural arguments of Chapters 10 to 13 assume structure that a table does not have, and Section 10.1 is explicit that a convolutional layer applied to unrelated clinical measurements is an actively bad choice.

6. Only now ask what structure the data have.

A network earns its place when there is structure for an architecture to exploit. Locality and stationarity along one or more axes point at a convolution; an ordering that must be respected

and a state that must be carried points at a recurrence; a set with no natural order, or long-range interactions between elements, points at attention; a graph points at message passing; a known symmetry group points at an equivariant architecture, which by Section 18.2.5 is the most data-efficient constraint available. If none of these apply, the honest answer is that a network is unlikely to help, and step five was the answer.

7. Establish the floor before claiming the ceiling.

Whatever the nonlinear model is, find the linear version of it and fit that first. For an autoencoder the linear version is principal component analysis and its error is the Eckart-Young value, known exactly by Theorem 12.5. For a sequence model it is an autoregressive fit. For a physics surrogate it is the existing numerical scheme, and Section 9.9 is a cautionary example of what happens when that comparison is actually run: finite differences beat the neural solver by orders of magnitude in time at comparable accuracy. A nonlinear model that does not clear its own linear floor has not earned its parameters.

8. Verify the gradient, then the model.

Before trusting an implementation, check it against something that cannot be wrong: finite differences for a hand-written gradient, an analytic special case, a small enough instance to enumerate exactly, or an independent implementation in a second framework. This book does all four at various points, and it does so because a wrong gradient does not announce itself – the network still trains, just to the wrong place. The cost is an hour; the alternative is discovering the error after the results are written up, or not at all.

9. Report what happened.

Including the seeds, including the spread, including the configurations that did not work, and including the baseline that was not beaten. The negative result of Section 10.10 took as long to produce as a positive one would have, and it is more informative than most positive results, because it says precisely when the method helps and when it does not.

The order matters more than the list. Every item above is standard advice. What is not standard is doing them in this order, and the reason to insist on it is that steps one to five are cheap and answer the question "is there signal, and how much?", while steps six onwards are expensive and answer only "can I extract a little more of it?". A project that reaches step six without a number from step four has no way of knowing whether anything it does afterwards is an improvement.

18.4 What this book does not contain

A book has to stop somewhere, and it is more useful to say where than to pretend the boundary is a natural one. Five omissions are large enough that a reader should know they are omissions rather than gaps in the subject.

Reinforcement learning.

Every method in this book learns from a fixed data set. Reinforcement learning learns from interaction: an agent takes actions, receives rewards, and must trade exploration against exploitation without ever being told what the correct action was. The mathematics is different enough to need its own treatment – Markov decision processes, the Bellman equations, temporal-difference learning, policy gradients – and the standard reference remains Sutton and Barto [10]. What connects it to this book is that modern reinforcement learning is deep learning with a different loss: the value function or the policy is a network trained by the backpropagation of Chapter 8, and the innovations that made it work at scale, from the replay buffer and target network of deep Q-learning [71] to the clipped surrogate objective of proximal policy optimisation [72], are all devices for making a non-stationary regression problem behave like a stationary one. Section 18.5.6 returns to what it is being used for.

Graph neural networks.

Chapters 10 and 13 cover two extremes of structural assumption: a grid with translation symmetry, and a set with full permutation symmetry. Most scientific data lie between them. A molecule, a crystal, a mesh, an interaction network and a particle-physics event are all graphs, and the architecture that respects a graph's structure is message passing, in which each node updates its state from its neighbours [73]. Everything in Section 18.2.5 applies: a graph network is a dense layer whose weights are shared across edges, it is equivariant to relabelling of the nodes, and adding equivariance under rotations and translations of the embedding space [68,69,74] is what makes it useful for physics. A reader who has understood Chapter 10 has most of what is needed.

Uncertainty quantification.

This book computes point predictions and reports test errors. It does not say what a prediction's uncertainty is, and for scientific use that is often the number that matters. Three practical routes exist. Deep ensembles [67] are the ensembling of Section 18.2.3 applied to networks and remain a strong default. Bayesian deep learning places a posterior over weights and approximates it, usually variationally, which is Chapter 15's machinery pointed at the parameters rather than at a latent code. Conformal prediction [75] is different in kind and deserves to be far better known in the sciences: it wraps any predictor whatever in a procedure that produces intervals with a finite-sample coverage guarantee, assuming only exchangeability, and it is the only one of the three that promises anything provable.

Scale.

Everything in this book runs on a laptop, deliberately, so that the reader can reproduce every number. Nothing here therefore prepares one for the regime in which the current large models operate, where the empirical scaling laws [76] relating loss to parameters, data and compute are the main design tool, and where the engineering – distributed training, mixed precision, data pipelines, checkpointing – is a large part of the work. The mathematics does not change; the constants do, and the constants decide what is possible.

Everything after the model.

Data collection, provenance, versioning, monitoring for drift, and the long maintenance life of a deployed model are absent from this book and are most of the effort in practice. The observation that machine-learning systems accrue technical debt faster than ordinary software is a decade old and has not stopped being true.

18.5 Perspectives

The rest of this chapter looks outward. What follows is not a survey – the field moves too quickly for a survey in a book to be anything but an embarrassment within two years – but an attempt to say, discipline by discipline, which of the methods derived here are being used, for what, and what is still missing. The references are entry points rather than a bibliography.

18.5.1 Physics

The application closest to this book's own machinery is the use of a neural network as a variational ansatz for a quantum many-body wave function. The idea is due to Carleo and Troyer [77]: represent the amplitude $\Psi(\boldsymbol{s})$ of a spin configuration by a restricted Boltzmann machine, and minimise the energy by the stochastic gradient methods of Chapter 4, sampling configurations by exactly the Metropolis algorithm of Section 2.14. Every ingredient is in this book. The extension to continuous space and to fermions requires an antisymmetric ansatz, and FermiNet [78] and PauliNet [79] supply one by embedding determinants in a network; both reach chemical accuracy on small molecules from first principles, with no basis set. The obstruction is the one Chapter 14 identified – the normalisation is unavailable, so everything is done with ratios and samples – and the sign problem in its usual form is not solved but re-expressed.

In high-energy physics the pattern is different: the data are abundant and the task is classification. Deep networks for the identification of exotic particles [80] were among the first demonstrations that a network could outperform hand-engineered high-level features, and jet tagging is now routinely done with graph networks over particle constituents. The generative part of this book is what is being used for the reverse problem, fast simulation: a calorimeter shower takes seconds to simulate exactly and microseconds to generate with a trained model, and both adversarial and diffusion-based generators are in production use. The scientific question that then arises is the one Chapter 17 warns about – a generator that reproduces every summary statistic examined may still not reproduce the one that has not been examined – which is why validation of such surrogates is an active field in its own right rather than a formality.

In condensed matter, the classification of phases directly from configurations by a convolutional network [81] showed that an order parameter need not be known in advance to be detected, and machine-learned interatomic potentials have changed what molecular dynamics can reach [68, 69, 82]. In astrophysics and cosmology the generative methods of Chapters 15 to 16 are used for field-level inference, for denoising gravitational-wave strain, and increasingly as learned priors inside otherwise classical inference pipelines – which is, in the language of Section 18.2.2, an admission that the prior was the part nobody could write down.

18.5.2 Chemistry and materials science

Chemistry is where the graph representation of Section 18.4 is not a modelling choice but the natural encoding of the object, and it is the discipline in which machine learning has so far changed practice most.

Property prediction from molecular graphs by message passing [73] and its equivariant descendants [68, 69, 83] now supplies energies and forces at a small fraction of the cost of the electronic structure calculation they are trained on, which moves molecular dynamics from picoseconds to microseconds at density-functional accuracy. The consequence at scale is a discovery pipeline: the systematic search reported by Merchant and co-workers [82] added of the order of 10^5 predicted stable crystals to a field that had accumulated a few tens of thousands over its history. The caveat is the one of Section 8.15: such models interpolate within the chemistry they were trained on and extrapolate unpredictably outside it, and a stability prediction is not a synthesis.

Generation is the other half. A molecule is a discrete object, so the continuous generative models of Chapters 15 to 16 must be adapted rather than applied, and the practical answer has been to diffuse in a continuous latent space or over atomic coordinates directly. Flow matching [84] has become a favoured alternative because it trains by regression on a velocity field, avoiding both the adversarial game of Chapter 17 and the long sampling chains of Chapter 16. For quantum chemistry proper, the physics-informed formulation of Chapter 9 is being applied to the electronic Schrödinger equation, and the caution recorded there applies with force: a rectified network has $f'' \equiv 0$ and will report a residual of zero while failing to solve a second-order equation at all.

18.5.3 Mathematics: operators, inverse problems and guarantees

Chapter 9 solves a differential equation by fitting one solution. Operator learning changes the object of the fit: instead of approximating u, approximate the map from the coefficient function, or the initial condition, to the solution. DeepONet [85] builds this from a branch network acting on the input function and a trunk network acting on the query point; Fourier neural operators [49] parameterise the kernel in Fourier space and are therefore resolution-independent, so a model trained on a coarse grid evaluates on a fine one. Once trained, a new instance costs a forward pass rather than a solve, which is what makes these methods attractive for many-query problems – optimisation, control, uncertainty propagation – and irrelevant for a single high-accuracy solve, exactly as Section 9.9 argued for physics-informed networks [5, 6].

Inverse problems are the natural home of the generative part of this book. The classical difficulty is that a regularising prior has to be chosen by hand, and the modern answer is to learn it: a diffusion model trained on plausible solutions supplies a score, and the score can be used inside a sampler that also respects the measurement. What is obtained is not a single reconstruction but a sample from an approximate posterior, which is the right object – and it inherits, without dilution, the warning of Section 18.2.7 that "approximate" is doing real work in that sentence.

Two further directions are worth naming. Symbolic regression aims to recover an equation rather than a black box, and the combination of a graph network with a symbolic fit to its learned messages [86] has recovered known force laws from simulation data, which is the most direct attack there is on interpretability. And the guarantee side of the subject is finally maturing: conformal prediction [75] converts any predictor into one with finite-sample

coverage, which is the sort of statement the sciences ought to be demanding as a matter of course.

18.5.4 Earth, ocean and climate science

Numerical weather prediction is the clearest case in any science of a data-driven model displacing a physical one on the physical model's own terms. GraphCast [87] is a graph network over a multi-mesh discretisation of the globe, trained on decades of reanalysis, and it produces a ten-day forecast in about a minute on a single accelerator, outperforming the operational deterministic system on the large majority of verified variables. FourCastNet [88] reaches comparable skill from a Fourier neural operator. Neither model knows any physics. Both are trained by the loss of Section 18.2.2 on the trajectory of a system whose equations we have known for a century.

It is worth being precise about what this does and does not show. It shows that where a very large, homogeneous, quality-controlled record exists, and where the target is a short-horizon forecast, learning the propagator can beat integrating the equations at a fraction of the cost. It does not show that the equations are dispensable: the training data are themselves the output of a physical assimilation system, the models inherit its biases, and their behaviour under a climate they have not seen is exactly the extrapolation regime Section 8.15 warns about. The interesting research is consequently in hybrid schemes, where a learned component sits inside a physically constrained solver and is responsible only for what the solver parameterises.

In seismology the convolutional and recurrent architectures of Chapters 10 and 11 have made detection of small events routine, lowering magnitude-of-completeness thresholds enough to change what catalogues contain. In remote sensing the multi-modal transformer is becoming the default for fusing spectral bands, radar and time series, and in subsurface imaging generative models supply the prior for reservoir characterisation – again, a learned prior standing in for one nobody could write down.

18.5.5 Bioscience and medicine

The prediction of protein structure from sequence is the field's most cited success, and it is instructive because of how many threads of this book it combines. AlphaFold2 [89] is attention (Chapter 13) over a representation of evolutionary couplings, followed by a structure module equivariant under rigid motions (Section 18.2.5), trained end to end by backpropagation (Chapter 8) on a curated database. Language-model approaches [90] dispense with the alignment and predict structure from a single sequence, trading a little accuracy for a large speed-up; later systems extend the same architecture to complexes and ligands [91]. Design is the inverse problem, and it is now done with the generative machinery of Chapter 16: RFdiffusion and its relatives [92] sample backbones conditioned on a functional site, and the designs are synthesised and tested, which is the only validation that counts.

Elsewhere in the life sciences the pattern is more familiar. Variational autoencoders are the standard tool for single-cell transcriptomics, where the latent space of Chapter 15 is used as a coordinate for cell type and developmental trajectory – and where the notebox of Section 12.2 is worth rereading, since a low reconstruction error is evidence that information survived the bottleneck and not that it survived in an interpretable form. Medical imaging uses convolutional and, more recently, diffusion-based models for segmentation and anomaly

detection; the distinguishing feature of the domain is that the cost of a false negative is not symmetric with the cost of a false positive, so the confusion-matrix vocabulary of Section 5.12 is not optional and accuracy is close to meaningless.

18.5.6 Reinforcement learning and agentic systems

Reinforcement learning is the part of machine learning whose scientific applications look least like the rest of this book, because the model is not fitted to a data set but improved against an environment.

The demonstration most likely to convince a physicist is the magnetic control of a tokamak plasma by a learned policy [93]: a network trained entirely in simulation, deployed on a real device, holding a variety of plasma configurations that would each traditionally have required a separately designed controller. The pattern – train in a simulator that is good enough, transfer to the apparatus, keep a classical safety layer underneath – is the template for most of what is currently promising, and it puts the burden squarely on the simulator. Related work is under way on pulse sequences for quantum control and error correction, on adaptive experimental design, where the agent chooses the next measurement rather than the next action, and on the exploration of chemical space under a property reward.

The methodological caution is severe and should be stated. Reinforcement learning has the weakest reproducibility record of anything in this book: results are famously sensitive to random seeds, implementation details and reward shaping, and the difference between two published algorithms is often smaller than the spread across seeds of either. A reader coming from the discipline this book has tried to instil – report the spread, quote the seed, publish the program – will find that discipline is needed there more than anywhere else.

The newest development, and the one hardest to assess soberly, is the use of language-model-based agents to run parts of the scientific process itself: proposing hypotheses, writing and executing analysis code, calling simulators, reading literature, and iterating on the result. Systems of this kind now perform real work in literature synthesis and code generation, and the honest summary is that they are useful where the output can be checked cheaply and unreliable where it cannot. That is not a temporary state of affairs to be waited out; it is a statement about verification, and it means the useful question is not how capable such a system is but how the correctness of its output is established. A generated proof can be checked, a generated program can be run against tests, a generated experimental protocol can be executed. A generated literature summary can be checked only by reading the literature.

18.5.7 Foundation models and transfer

The final trend is the one that most changes how a scientist should approach a new problem. A large model pre-trained on a general corpus – protein sequences, molecular structures, the atmospheric record, sky surveys – can be adapted to a specific task with a small labelled set, because the representation it has already learned does most of the work. The empirical regularity underlying this is the scaling behaviour of [76], and the practical consequence is that the first question about a new problem is no longer "what architecture?" but "what pre-trained representation exists, and how little data do I need to adapt it?".

For a scientific reader this cuts both ways. It puts capable models within reach of groups that could never train one, which is a genuine democratisation. It also introduces a depen-

dency on artefacts whose training data, biases and failure modes are frequently undocumented, and which cannot be interrogated with the tools of this book because their weights or their corpora, or both, are not available. Reproducibility in the sense this book has insisted on – run the program, get the number – is not currently achievable for results that rest on such a model, and pretending otherwise does the field no service.

18.6 What to be sceptical about

This book has reported, deliberately, several results that did not come out as hoped, on the principle that a method's failures are more informative than its successes. Collected here, they make a serviceable checklist.

A baseline you did not try honestly is not a baseline.

Section 10.10 compares a convolutional network with a dense one on centred handwritten digits at matched parameter count and finds 0.9656 against 0.9670 – a difference far smaller than the spread across seeds of either. The convolutional network is not better. It becomes better, by seventeen points, only when the digits are placed at random offsets, because only then do the data have the structure the architecture assumes. A great many published comparisons would not survive being run this way.

Likelihood is not sample quality.

Section 16.4.1 trains a normalizing flow that reports 5.46 nats while producing useless samples, by contracting the data into a region where the prior is nearly flat and harvesting the Jacobian term. The likelihood of a continuous density is unbounded above, and a sufficiently flexible model will find that out. Any single scalar score can be gamed by a model with enough freedom, and the remedy is not a better score but more than one of them, including one that looks at the samples.

An existence proof is not an algorithm.

Section 8.4.4 measures the gap between the network Theorem 8.2 constructs and the network Adam finds at the same width, and the gap grows with width. Universal approximation removes representability as an excuse and leaves optimisation and generalisation, which are the hard parts.

An architecture supplies a path; the initialisation decides whether training starts near it.

Section 11.7.1 measures an LSTM at $\boldsymbol{b}_f = 0$ transmitting 3.9×10^{-18} of the gradient across fifty-nine steps, against 1.7×10^{-18} for the vanilla recurrent network it was designed to improve on. The ten to fifteen orders of magnitude usually attributed to the architecture come from biasing the forget gate open. The general form of this error – crediting a structural change with a benefit that came from an initialisation, a schedule or a normalisation shipped alongside it – is extremely common.

A biased estimator can be perfectly useful, and a correct one useless.

Section 14.6.1 finds contrastive divergence wrong by 33% in magnitude with a cosine of 0.986 against the true gradient. It trains well because direction is what gradient descent consumes. Conversely, the flow above is exactly correct about its likelihood and worthless as a generative model. Ask what property of an estimate the algorithm actually uses.

Games are not minimisations.

Chapter 17 measures a simultaneous gradient iteration diverging as $(1+\gamma^2)^{t/2}$ on a bilinear game, and an alternating one circulating forever. Loss curves that oscillate are not a sign of a bad learning rate but the expected behaviour of the dynamics, and "the loss went down" is not evidence of progress in an adversarial setting.

Networks extrapolate unpredictably, and say nothing when they do.

This is Section 8.15, and it is the failure mode that matters most in the sciences, because the interesting regime is so often outside the training distribution. A network gives no signal that it has left the region it was fitted in. This is the strongest practical argument for the uncertainty methods of Section 18.4 and for the symmetry constraints of Section 18.2.5, which continue to hold where the data run out.

The most common error is not in the model.

It is leakage: a preprocessing step fitted on all the data, a temporal split performed at random, a duplicated record spanning the train-test boundary, a hyperparameter chosen on the test set. A survey across seventeen fields [94] found errors of this family affecting hundreds of papers, and their effect is always in the same direction. The rule that prevents all of them is the one stated in Chapter 2 and repeated throughout: anything learned from the data must be learned from the training data only.

And there is no free lunch.

Averaged over all problems, no algorithm beats any other [70]. Every gain in this book was bought by an assumption – linearity, smoothness, locality, stationarity, a symmetry group, a noise model, a prior – and the method works exactly to the extent that its assumption holds for the data at hand. Choosing a method is choosing an assumption, and it is worth being able to say aloud which one.

18.7 A last word

The most durable thing this book has to offer is not a method but a habit, and it is worth stating explicitly because it is the habit the field most often lacks.

Every number quoted in these seventeen chapters was produced by a program that accompanies the text, and the programs are collected so that the reader can run them and get the same number. Where a claim could be checked against an exact result, it was: the gradient

identity of Chapter 14 against finite differences to 10^{-8}, the forward posterior of Chapter 16 against quadrature to six decimals, the optimal discriminator of Chapter 17 to 3×10^{-9}, the linear autoencoder against the singular value decomposition, three independent implementations of every architecture against one another to the precision of double arithmetic. Where the check failed, or where the result was less flattering than expected, that is what appears in the text. This is not pedantry. A field in which a wrong gradient trains almost as well as a right one, in which a model can score well on the metric it is graded by while being useless, and in which the difference between two methods is routinely smaller than the difference between two random seeds, is a field that has to verify rather than believe.

The methods will change. Some of the architectures treated here at length will be of historical interest within a decade, and something not yet published will be in every chapter of the next edition. What will not change is that the loss follows from the model, that the condition number governs three things at once, that structure is a prior worth more than data, that a normalisation which cannot be computed must be evaded rather than ignored, and that a result nobody can reproduce is not a result. Those are the parts worth carrying forward, and they are, not coincidentally, the parts this book has tried hardest to prove rather than assert.

The rest is your problem now, and it is a good time to have it.

References

1. Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, Cambridge, 2004.
2. George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.
3. Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.
4. Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
5. M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
6. George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021.
7. Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020.
8. Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using Real NVP. In *Proceedings of ICLR*, 2017.
9. Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, 2014.
10. Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, second edition, 2018.
11. Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, 3rd edition, 1996.
12. Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.
13. Ning Qian. On the momentum term in gradient descent learning algorithms. *Neural Networks*, 12(1):145–151, 1999.
14. Yurii Nesterov. A method for solving the convex programming problem with convergence rate $O(1/k^2)$. *Doklady Akademii Nauk SSSR*, 269:543–547, 1983.
15. Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*, pages 1139–1147, 2013.
16. Léon Bottou, Frank E. Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018.

17. Léon Bottou and Olivier Bousquet. The tradeoffs of large scale learning. In *Advances in Neural Information Processing Systems 20*, pages 161–168, 2008.
18. Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22:400–407, 1951.
19. Boris T. Polyak and Anatoli B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992.
20. Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv:1412.6980*, 2014.
21. Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of Adam and beyond. In *International Conference on Learning Representations*, 2018.
22. Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.
23. R. E. Wengert. A simple automatic derivative evaluation program. *Communications of the ACM*, 7(8):463–464, 1964.
24. Andreas Griewank and Andrea Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. SIAM, Philadelphia, 2nd edition, 2008.
25. Walter Baur and Volker Strassen. The complexity of partial derivatives. *Theoretical Computer Science*, 22(3):317–330, 1983.
26. Seppo Linnainmaa. Taylor expansion of the accumulated rounding error. *BIT Numerical Mathematics*, 16(2):146–160, 1976.
27. Atılım Güneş Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.
28. Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, New York, 2nd edition, 2006.
29. James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs. `http://github.com/jax-ml/jax`, 2018.
30. H. H. Rosenbrock. An automatic method for finding the greatest or least value of a function. *The Computer Journal*, 3(3):175–184, 1960.
31. Peter L. Bartlett, Michael I. Jordan, and Jon D. McAuliffe. Convexity, classification, and risk bounds. *Journal of the American Statistical Association*, 101(473):138–156, 2006.
32. Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.
33. Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.
34. Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017.
35. Moshe Leshno, Vladimir Ya. Lin, Allan Pinkus, and Shimon Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6):861–867, 1993.
36. Dmitry Yarotsky. Error bounds for approximations with deep ReLU networks. *Neural Networks*, 94:103–114, 2017.
37. Allan Pinkus. Approximation theory of the MLP model in neural networks. *Acta Numerica*, 8:143–195, 1999.
38. Ronald A. DeVore, Ralph Howard, and Charles Micchelli. Optimal nonlinear approximation. *Manuscripta Mathematica*, 63(4):469–478, 1989.
39. Andrew R. Barron. Universal approximation bounds for superpositions of a sigmoidal function. *IEEE Transactions on Information Theory*, 39(3):930–945, 1993.
40. Matus Telgarsky. Benefits of depth in neural networks. In *Proceedings of the 29th Conference on Learning Theory*, pages 1517–1539, 2016.
41. Guido Montúfar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. On the number of linear regions of deep neural networks. In *Advances in Neural Information Processing Systems*, 2014.
42. Neha Yadav, Anupam Yadav, and Manoj Kumar. *An Introduction to Neural Network Methods for Differential Equations*. Springer, Dordrecht, 2015.
43. Salvatore Cuomo, Vincenzo Schiano di Cola, Fabio Giampaolo, Gianluigi Rozza, Maziar Raissi, and Francesco Piccialli. Scientific machine learning through physics-informed neural networks: Where we are and what's next. *Journal of Scientific Computing*, 92(3):88, 2022.
44. Vincent Dumoulin and Francesco Visin. A guide to convolution arithmetic for deep learning. *arXiv:1603.07285*, 2016.
45. Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
46. Derek Rowell. 2.161 signal processing: Continuous and discrete, lecture 4: Fourier transforms, frequency response and convolution. MIT OpenCourseWare, `https://ocw.mit.edu/courses/2-161-signal-processing-continuous-and-discrete-fall-2008/`, 2008.

47. Richard Zhang. Making convolutional networks shift-invariant again. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *PMLR*, pages 7324–7334, 2019.
48. Oren Rippel, Jasper Snoek, and Ryan P. Adams. Spectral representations for convolutional neural networks. In *Advances in Neural Information Processing Systems 28*, 2015.
49. Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, et al. Fourier neural operator for parametric partial differential equations. In *Proceedings of ICLR*, 2021.
50. Rafal Jozefowicz, Wojciech Zaremba, and Ilya Sutskever. An empirical exploration of recurrent network architectures. In *Proceedings of ICML*, pages 2342–2350, 2015.
51. Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of EMNLP*, pages 1724–1734, 2014.
52. Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xLSTM: Extended long short-term memory. *arXiv:2405.04517*, 2024.
53. Michael E. Tipping and Christopher M. Bishop. Probabilistic principal component analysis. *Journal of the Royal Statistical Society: Series B*, 61(3):611–622, 1999.
54. Diederik P. Kingma and Max Welling. An introduction to variational autoencoders. *Foundations and Trends in Machine Learning*, 12(4):307–392, 2019.
55. Alex Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *Proceedings of ICML*, pages 8162–8171, 2021.
56. Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training GANs. In *Advances in Neural Information Processing Systems*, 2016.
57. Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. In *Advances in Neural Information Processing Systems*, 2017.
58. Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. In *Proceedings of ICLR*, 2018.
59. Luke Metz, Ben Poole, David Pfau, and Jascha Sohl-Dickstein. Unrolled generative adversarial networks. In *Proceedings of ICLR*, 2017.
60. Cédric Villani. *Optimal Transport: Old and New*. Springer, Berlin, 2009.
61. Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein GAN. In *Proceedings of ICML*, pages 214–223, 2017.
62. Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of Wasserstein GANs. In *Advances in Neural Information Processing Systems*, 2017.
63. Arthur Gretton, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13:723–773, 2012.
64. Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. In *Proceedings of ICLR*, 2016.
65. Mehdi Mirza and Simon Osindero. Conditional generative adversarial nets. *arXiv:1411.1784*, 2014.
66. Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks. In *Proceedings of CVPR*, pages 4401–4410, 2019.
67. Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, 2017.
68. Simon Batzner, Albert Musaelian, Lixin Sun, et al. E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nature Communications*, 13(1):2453, 2022.
69. Ilyes Batatia, Dávid Péter Kovács, Gregor N. C. Simm, Christoph Ortner, and Gábor Csányi. MACE: Higher order equivariant message passing neural networks for fast and accurate force fields. In *Advances in Neural Information Processing Systems*, 2022.
70. David H. Wolpert and William G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997.
71. Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
72. John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.
73. Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of ICML*, pages 1263–1272, 2017.
74. Fabian B. Fuchs, Daniel E. Worrall, Volker Fischer, and Max Welling. SE(3)-transformers: 3D roto-translation equivariant attention networks. In *Advances in Neural Information Processing Systems*, 2020.
75. Anastasios N. Angelopoulos and Stephen Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4):494–591, 2023.
76. Jared Kaplan, Sam McCandlish, Tom Henighan, et al. Scaling laws for neural language models. *arXiv:2001.08361*, 2020.

77. Giuseppe Carleo and Matthias Troyer. Solving the quantum many-body problem with artificial neural networks. *Science*, 355(6325):602–606, 2017.
78. David Pfau, James S. Spencer, Alexander G. D. G. Matthews, and W. M. C. Foulkes. Ab initio solution of the many-electron Schrödinger equation with deep neural networks. *Physical Review Research*, 2(3):033429, 2020.
79. Jan Hermann, Zeno Schätzle, and Frank Noé. Deep-neural-network solution of the electronic Schrödinger equation. *Nature Chemistry*, 12(10):891–897, 2020.
80. Pierre Baldi, Peter Sadowski, and Daniel Whiteson. Searching for exotic particles in high-energy physics with deep learning. *Nature Communications*, 5(1):4308, 2014.
81. Juan Carrasquilla and Roger G. Melko. Machine learning phases of matter. *Nature Physics*, 13(5):431–434, 2017.
82. Amil Merchant, Simon Batzner, Samuel S. Schoenholz, et al. Scaling deep learning for materials discovery. *Nature*, 624(7990):80–85, 2023.
83. K. T. Schütt, H. E. Sauceda, P.-J. Kindermans, A. Tkatchenko, and K.-R. Müller. SchNet – a deep learning architecture for molecules and materials. *Journal of Chemical Physics*, 148(24):241722, 2018.
84. Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *Proceedings of ICLR*, 2023.
85. Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.
86. Miles Cranmer, Alvaro Sanchez-Gonzalez, Peter Battaglia, et al. Discovering symbolic models from deep learning with inductive biases. In *Advances in Neural Information Processing Systems*, 2020.
87. Rémi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, et al. Learning skillful medium-range global weather forecasting. *Science*, 382(6677):1416–1421, 2023.
88. Jaideep Pathak, Shashank Subramanian, Peter Harrington, et al. FourCastNet: A global data-driven high-resolution weather model using adaptive Fourier neural operators. *arXiv:2202.11214*, 2022.
89. John Jumper, Richard Evans, Alexander Pritzel, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021.
90. Zeming Lin, Halil Akin, Roshan Rao, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023.
91. Josh Abramson, Jonas Adler, Jack Dunger, et al. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 630(8016):493–500, 2024.
92. Joseph L. Watson, David Juergens, Nathaniel R. Bennett, et al. De novo design of protein structure and function with RFdiffusion. *Nature*, 620(7976):1089–1100, 2023.
93. Jonas Degrave, Federico Felici, Jonas Buchli, et al. Magnetic control of tokamak plasmas through deep reinforcement learning. *Nature*, 602(7897):414–419, 2022.
94. Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023.

Index

.632 estimator, 124

accuracy, 245
acronyms, list of, xvii
activation function, 329
AdaBoost, 295
AdaGrad, 176
  logistic regression, 231
Adam
  logistic regression, 231
Adam optimizer, 179
adaptive learning rate, 174
adding problem, 468
adjoint, 191
agentic AI, 606
agentic systems, 606
aliasing, 439
anti-aliasing, 440
artificial neuron, 222, 315
attention, 498
autocorrelation function, 58
autoencoder, 318, 473
autograd, 196
automatic differentiation, 187
  and backpropagation, 336, 349
  fan-out, 344
  logistic regression, 224
  neural network, 340
average pooling, 410

backpropagation, 191, 334
  as the book's central algorithm, 597
  scalar example, 335
  two hidden nodes example, 342
backpropagation through time, 453
bagging, 292
basis expansion, 82
batch normalisation, 352
Bayes' theorem, 106
bias
  in a neuron, 315
  in data, xxxii
bias-variance trade-off, xxiv
  as a unifying idea, 596
bias-variance tradeoff, 64
Black-Scholes equation, 388
BLAS, 7
block Gibbs sampling, 525
Boltzmann machine, 523
book
  structure of, xxv, 593
boosting, 294
bootstrap, 67
  for prediction error, 124
bootstrap aggregation, 292
bottleneck, 473
BPTT, *see* backpropagation through time

C++, xxx
calibration, 238
CART algorithm, 285
causal mask, 513
central limit theorem, 55
chain rule
  matrix form, 195
channel, 406
χ^2 function, 85
Cholesky decomposition, 30
circulant matrix, 404
circular convolution, 434
classification, 209
classification tree, 283
code, 473
column-major order, 8
computational graph, 188
  logistic regression, 224
condition number, 26
  and gradient descent, 151
  as a unifying quantity, 595
conditional GAN, 581
confidence interval, 90
confusion matrix, 245
conjugate gradient method, 32
contractive autoencoder, 484
contrastive divergence, 525
convergence rate
  stochastic gradient descent, 164
convex function, 145
convex set, 145
convolution, 402

convolution theorem, 404
convolutional layer
  in Fourier space, 437
convolutional neural network, 318
correlation, 54
correlation time, 58
cost function
  derived from a likelihood, xxiii
  neural network, 333
covariance, 54
covariance matrix, 20
Cramér-Rao bound, 89
cross entropy, 212
  gradient, 215
cross-correlation, 405
cross-validation, 68
  for Ridge and Lasso, 117
curse of dimensionality, 323
Cybenko's theorem, 319

DCGAN, 581
DDIM, 557
decision tree, 281
decoder, 537
delta rule, 222, 335, 346
denoising autoencoder, 483
denominator layout, 12
depth separation, 324
descent lemma, 152
  stochastic, 164
design matrix, 3, 82
detailed balance, 72, 522
deterministic function, 60
differential equations
  neural network solution, 369
diffusion equation, 378
diffusion model, 553
dilated convolution, 411
discrete Fourier transform, 404, 433
discriminator, 565
dropout, 352, 423
dual numbers, 189

Earth mover distance, 577
Eckart-Young theorem, 41, 480
eigenvalue, 19
eigenvalue problem, 34
ELBO, *see* evidence lower bound
encoder, 537
energy-based model, 519
ensemble methods, 290
epoch, 160
equivariance, 408
error
  of a node, 346
estimation error, 165
estimator, 57
ethics, xxxi
evaluation trace, 188
evidence lower bound, 538
expectation value, 53
explained variance, 475
exploding gradients, 350, 454
exponential decay, 373

feature engineering, 359
feature map, 416
feed-forward network, 318
feed-forward pass, 326
filter
  as a $C \times F \times F$ block, 406
  hand-made, 413
  hand-made and learned, 402
  learned, 415
finite differences, 187
Fisher information, 89
fixed design, 60
floating-point operations, 161
forget gate, 460
Fortran, xxx
forward-mode differentiation, 189, 370
foundation model, 606
Fourier filter, 429
Fourier neural operator, 440
Fourier transform, 430
  two-dimensional, 436
Fréchet inception distance, 579
Franke function, 129
free energy, 520
frequency response, 429
frequentist inference, xxiii
fully connected layer, 318

gated recurrent unit, 463
Gauss-Markov, 60
Gauss-Markov theorem, 90
Gauss-Seidel method, 31
Gaussian distribution, 51
Gaussian elimination, 27
Gaussian smoothing, 432
GELU, 330
generalisation, 63
generalised cross-validation, 93
generative adversarial network, 565
generator, 565
Gibbs sampling, 523
global average pooling, 426
gradient, 12
gradient boosting, 298
  with automatic differentiation, 302
gradient checking, 349
gradient clipping, 457
gradient descent, 148
  for least squares, 149
  stability bound, 151
gradient penalty, 576
Gram-Schmidt orthogonalisation, 10
graph neural network, 602

He initialisation, 351
Hessian matrix, 17
hidden state, 452

hinge loss, 271
Hornik's theorem, 319
hyperparameter, 117
hyperplane, 255

ideal low-pass filter, 433
im2col, 418
importance-weighted bound, 539
induction head, 504
inductive bias, 597
intercept, 113
inverse transform sampling, 71
Ising model
  as linear regression, 133
  one-dimensional, 133
  with neural networks, 359
iterative methods, 31

jackknife, 67
Jacobi's method, 31
Jacobian, 12
  of a recurrent cell, 451
JAX, 196, 340
  boosting, 302
  logistic regression, 239
Jensen's inequality, 539
Jensen-Shannon divergence, 569

Keras, 425, 486, 508
kernel, 107, 262
  convolutional, 402
kernel Lasso, 110
kernel logistic regression, 241
kernel methods
  unified, 273
kernel trick, 107, 262
Kullback-Leibler divergence, 521, 569

label smoothing, 572
Lagrange multipliers, 258
Lagrangian dual, 258
LAPACK, 7
Lasso, 99
latent-variable model, 537
layer
  hidden, 326
  input, 326
  output, 326
layer normalisation, 503
learning rate, 151
  schedule, 167
leave-one-out cross-validation, 93
Leshno-Lin-Pinkus-Schocken theorem, 319
linear autoencoder, 476
Lipschitz constraint, 576
Lipschitz continuity
  of the gradient, 152
logistic function, 210
logistic growth, 374
logistic regression
  kernel, 241
  multiple predictors, 216
  optimisation, 218
long short-term memory, 459
loss function
  for classification, 236
low-rank approximation, 41
LSTM, *see* long short-term memory
LU decomposition, 29

machine learning
  definition, xxii
manifold hypothesis, 481
margin, 236, 255
Markov chain, 72
Markov chain Monte Carlo, 521
matrix calculus, 12
max pooling, 410
maximum a posteriori, 106
maximum likelihood, 88
  as a source of loss functions, 595
  logistic regression, 212
maximum margin, 257
maximum mean discrepancy, 579
mean squared error, 15, 87
memory horizon, 456
Mercer's theorem, 263
Metropolis algorithm, 72
Metropolis-Hastings algorithm, 522
minibatch, 160, 327
  gradient variance, 163
  logistic regression, 228
  size, 171
minimax objective, 566
Mish, 330
MNIST, 422
  dense against convolutional, 428
mode collapse, 575
moments, 53
momentum, 155
multi-head attention, 502
multilayer perceptron, 318
multinomial logistic regression, 217
multivariate Gaussian, 52

Nash equilibrium, 574
nested cross-validation, 129
neural network
  scalar example, code, 338
neural-network quantum states, 603
Newton-Raphson method, 147
non-saturating loss, 571
norm
  Frobenius, 24
  matrix, 23
  vector, 23
normalised cross-correlation, 414
normalizing flow, 558
numerator layout, 12
numpy, 7

one-standard-error rule, 117, 120
operator learning, 604

optimism of the training error, 92
option pricing, 388
ordinary least squares, 83
orthogonal projector, 475
orthogonal transformation, 11
orthonormal basis, 9
overcomplete autoencoder, 473

padding, 405
partial differential equations, 377
partition function, 519
  as the central obstruction, xxviii, 598
perceptron, 222, 315
permutation equivariance, 498
physics-informed neural network, 381
PINN, *see* physics-informed neural network
pivoting, 28
Poisson equation, 375
  in a sine basis, 440
Polyak–Ruppert averaging, 169
pooling, 410
positional encoding, 501
posterior collapse, 543
power method, 35
precision, 245
PRESS statistic, 93
principal angles, 481
principal component analysis, 42
probabilistic PCA, 485
probability distribution function, 49
projection, 9
proper scoring rule, 238
pruning, 282
pseudo-likelihood, 531
pseudo-random numbers, 70
pseudoinverse, 38
Python, xxx
PyTorch, 399, 449, 486, 507, 528, 546, 582

QR decomposition, 10
quadratic programming, 265
query, key and value, 498

R^2 score, 87
random forest, 292
random numbers, 70
rank, 38
recall, 245
receptive field, 410
reconstruction error, 473
recurrent neural network, 318, 450
regression, 81
regression tree, 281
regularisation
  neural networks, 352
reinforcement learning, xxii, 602
ReLU
  creates new frequencies, 439
reparameterisation trick, 541
representer theorem, 109
  and the three losses, 273
reproducibility, xxx
resampling, 67
residual connection, 502
restricted Boltzmann machine, 524
reverse-mode differentiation, 191
Ridge regression, 44, 95
ringing, 433
RMSProp, 178
  logistic regression, 231
Rosenbrock function, 199
row-major order, 8

sample mean, 57
sample variance, 57
scaled dot-product attention, 514
scaling laws, 602
scikit-learn, 534
score matching, 556
second moment of the gradient, 174
self-attention, *see* attention
sequence modelling, 449
sequential minimal optimisation, 265
Sherman-Morrison formula, 93
sigmoid, 210
sigmoidal function, 319
similarity transformation, 34
singular value decomposition, 36
  recurrence of, 598
slack variables, 260
SMO, 265
Sobel filter, 413
soft margin, 260
soft thresholding, 99
softmax, 217
  in neural networks, 352
sparse autoencoder, 484
spectral decomposition, 19
spectral normalisation, 575
spectral pooling, 440
spectral radius, 151
stability selection, 128
standardisation, 113
  and adaptive methods, 233
steepest descent, 148
stochastic gradient descent, 160
  logistic regression, 228
stochastic variable, 49
stride, 406
successive over-relaxation, 32
supervised learning, xxii
support vector, 258
Swish, 330
symbolic differentiation, 187

template matching, 414
TensorFlow, 442, 467, 511, 530, 547, 583
test error, 63
Toeplitz matrix, 403
token, 497
total variation distance, 526
trace, 6, 24

training algorithm
  feed-forward network, 349
training error, 63
transformer, 497
translation invariance, 408
transposed convolution, 417
truncated backpropagation through time, 456

uncertainty quantification, 602
uniform distribution, 51
universal approximation theorem, 318
  and its limits, 599
  and optimisation, 325
  quantitative form, 321
unrolled GAN, 576
unsupervised learning, xxii

vanishing gradients, 350, 454
variational autoencoder, 537
volume (of activations), 401

Wasserstein distance, 576
wave equation, 379
weight initialisation, 351
weighted least squares, 85
Wisconsin breast cancer data, 247

Xavier initialisation, 351
XGBoost, 300
XOR problem, 316

zero-sum game, 566